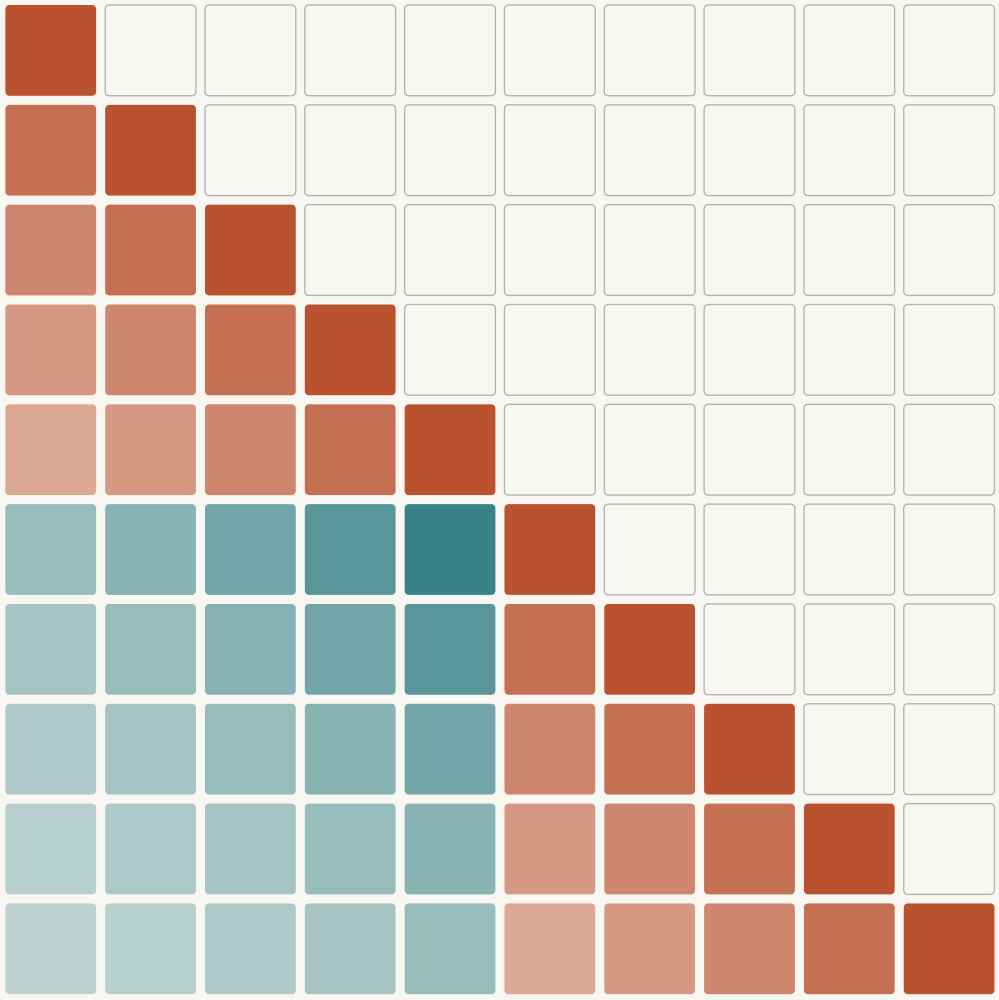
DEP 803 Course Book · First Edition

Mathematical and Statistical Foundations

for Data Science

with Python and Applications to Economic Policy



$$\Sigma = V \Lambda V^{\top}$$

Stephen E. Moore

Department of Mathematics · University of Cape Coast, Ghana

Mathematical and Statistical Foundations for Data Science
with Python and Applications to Economic Policy
Course book for DEP 803 (3 credits), Graduate Programme in Data Science and Economic Policy
First edition, 2026

Stephen E. Moore
Department of Mathematics, University of Cape Coast, Cape Coast, Ghana
Website: https://moorestephen.info/

© 2026 Stephen E. Moore. Some rights reserved.

Licence for the course material. This book (the text, figures and the PDF document) is licensed under the **Creative Commons Attribution 4.0 International Licence (CC BY 4.0)**. To view a copy of the licence, visit https://creativecommons.org/licenses/by/4.0/.

Licence for the code. The Python programs that accompany this book are licensed under the **MIT Licence**. The full text of the licence is distributed with the programs in the file `LICENSE`.

Permissions. The author hereby grants every person permission, free of charge, to copy, redistribute, translate, adapt and build upon this book, in any medium or format and for any purpose, including teaching and commercial purposes, on the condition that appropriate credit is given to the author, a link to the CC BY 4.0 licence is provided, and any changes are indicated. Permission is likewise granted, free of charge, to any person obtaining a copy of the Python programs to use, copy, modify, merge, publish, distribute, sublicense and/or sell copies of them, subject to the conditions of the MIT Licence (the copyright notice and permission notice must be included in all copies or substantial portions of the software). The programs are provided "as is", without warranty of any kind. Requests for permissions beyond the scope of these licences may be addressed to the author through https://moorestephen.info/.

Suggested citation. S. E. Moore (2026). *Mathematical and Statistical Foundations for Data Science, with Python and Applications to Economic Policy*. Course book for DEP 803, University of Cape Coast, Ghana. Licensed under CC BY 4.0.

Companion code. All Python programs referred to in the text are available, organised by chapter, in the companion repository https://github.com/moorekwesi/math-stat-foundations-data-science. The output printed beneath each listing is the actual output of the program, produced with Python 3.9, NumPy, SciPy, pandas, Matplotlib and scikit-learn.

Data. Unless stated otherwise, the data sets used in the examples are *simulated* for teaching purposes. They are designed to resemble the structure of household surveys, labour-force surveys and macroeconomic panels, but they are not official statistics of Ghana or of any other country, and no numbers in this book should be quoted as such.

Typeset by the author in LaTeX.

For my Hilda, Elisa and Yael.

Contents

List of Python Listings xvii

Preface xxi

Course Map xxv

Notation xxvii

I Linear Algebra for Modelling 1

1 Mathematical Notation, Vectors, Matrices and Computational Precision 3
1.1 Notation and the Language of Data Science . . . 4
1.1.1 Sets, functions and indices . . . 4
1.1.2 Summation and product notation . . . 5
1.1.3 The data matrix and the conventions of this book . . . 5
1.1.4 Asymptotic notation . . . 7
1.1.5 Order-of-magnitude reasoning . . . 8
1.2 Vectors . . . 10
1.2.1 The vector space $\mathbb{R}^n$ . . . 10
1.2.2 Inner products . . . 10
1.2.3 Norms, distances and unit balls . . . 10
1.2.4 The Cauchy–Schwarz inequality, angles and similarity . . . 11
1.2.5 Example: a consumer price index is an inner product . . . 13
1.3 Matrices . . . 16
1.3.1 Matrix products and their four interpretations . . . 16
1.3.2 Special matrices . . . 17
1.3.3 Transpose and inverse rules . . . 17
1.3.4 Block matrices and the partitioned inverse . . . 18
1.3.5 Trace and determinant . . . 18
1.3.6 Vectorised computation and broadcasting in NumPy . . . 20
1.3.7 A policy example: the Leontief input–output model . . . 21
1.4 Data as Matrices . . . 24
1.4.1 The design matrix . . . 24
1.4.2 Means, the centring matrix and standardisation . . . 24
1.4.3 Sample covariance and correlation matrices . . . 25
1.4.4 From pandas to NumPy . . . 25
1.4.5 The Kronecker product . . . 27
1.5 Floating-Point Arithmetic and Computational Precision . . . 29
1.5.1 IEEE 754 double precision . . . 29
1.5.2 The standard model of rounding error . . . 31

1.5.3 Catastrophic cancellation . . . 32
1.5.4 Overflow, underflow and the log-sum-exp trick . . . 33
1.5.5 Summation error and compensated (Kahan) summation . . . 35
1.5.6 Computing a variance: one pass, two passes, or Welford . . . 37
1.5.7 Comparing floats; single versus double precision . . . 39
1.6 Computational Cost . . . 42
1.6.1 Flop counts . . . 42
1.6.2 Vectorisation versus Python loops . . . 43
1.6.3 Memory . . . 44
1.6.4 Sparse matrices for large administrative data . . . 45
1.6.5 Reproducibility . . . 47
Further Reading . . . 50

2 Linear Algebra for Modelling: Rank, Projections and Decompositions 51
2.1 Linear Systems and Vector Spaces . . . 52
2.1.1 Three ways to read $A\mathbf{x} = \mathbf{b}$ . . . 52
2.1.2 Vector spaces, subspaces and span . . . 52
2.1.3 The four fundamental subspaces . . . 53
2.1.4 Existence and uniqueness of solutions . . . 54
2.1.5 Computing the four subspaces . . . 54
2.2 Rank and Identification . . . 57
2.2.1 The rank–nullity theorem . . . 57
2.2.2 Full column rank and identification . . . 57
2.2.3 Perfect multicollinearity and the dummy variable trap . . . 58
2.2.4 Numerical rank . . . 59
2.2.5 Identification of a programme effect . . . 60
2.3 Orthogonality and Projections . . . 63
2.3.1 Inner products and orthogonal complements . . . 63
2.3.2 Orthogonal projection onto a subspace . . . 64
2.3.3 Pythagoras and the ANOVA decomposition . . . 65
2.3.4 Orthonormal bases and Gram–Schmidt . . . 66
2.4 Least Squares as Projection . . . 70
2.4.1 The normal equations . . . 70
2.4.2 Algebraic properties of residuals . . . 71
2.4.3 R^2 and its limits . . . 71
2.4.4 The Frisch–Waugh–Lovell theorem . . . 71
2.4.5 Fixed effects as demeaning . . . 72
2.4.6 Leverage . . . 75
2.5 Matrix Decompositions . . . 77
2.5.1 LU decomposition . . . 77
2.5.2 Cholesky decomposition . . . 78
2.5.3 QR decomposition by Householder reflections . . . 80
2.5.4 Least squares via QR . . . 81
2.6 Generalised Inverses . . . 84
2.6.1 The four Penrose conditions . . . 85
2.6.2 Minimum-norm least squares . . . 85
2.6.3 Computing the pseudoinverse . . . 86
2.6.4 Underdetermined problems: $p > n$ . . . 87
2.6.5 Constrained least squares and national accounts balancing . . . 90
Further Reading . . . 92

3 Eigenvalues, the SVD, PCA and Conditioning **95**
3.1 Eigenvalues and Eigenvectors . . . 96
3.1.1 Definitions and the characteristic polynomial . . . 96
3.1.2 Diagonalisation and matrix powers . . . 97
3.1.3 Markov chains and labour-market transitions . . . 99
3.2 Symmetric Matrices, Quadratic Forms and Covariance . . . 103
3.2.1 The spectral theorem . . . 103
3.2.2 Quadratic forms and definiteness . . . 104
3.2.3 Covariance matrices, ellipsoids and the Mahalanobis distance . . . 104
3.3 Computing Eigenvalues . . . 108
3.3.1 Power iteration . . . 108
3.3.2 Shifts, inverse iteration and the Rayleigh quotient . . . 110
3.3.3 The QR algorithm and library routines . . . 111
3.3.4 PageRank on a simulated trade network . . . 112
3.4 The Singular Value Decomposition . . . 114
3.4.1 Existence and geometry . . . 115
3.4.2 Low-rank approximation: the Eckart–Young–Mirsky theorem . . . 117
3.4.3 Image compression and truncated SVD . . . 117
3.5 Principal Component Analysis . . . 120
3.5.1 Variance maximisation . . . 120
3.5.2 PCA via the SVD and explained variance . . . 121
3.5.3 PCA of a simulated macroeconomic panel . . . 122
3.5.4 Factor models and diffusion indexes . . . 123
3.5.5 A wealth index from asset ownership . . . 124
3.6 Conditioning . . . 128
3.6.1 The condition number . . . 128
3.6.2 The Hilbert matrix . . . 129
3.6.3 Conditioning of least squares . . . 131
3.6.4 Multicollinearity and variance inflation factors . . . 132
3.6.5 Ridge regression as a remedy . . . 135
Further Reading . . . 137

II Probability and Statistical Models **139**

4 Probability Foundations, Random Variables and Transformations **141**
4.1 Probability Spaces . . . 142
4.1.1 Sample spaces and events . . . 142
4.1.2 Why we need a σ-algebra . . . 142
4.1.3 Kolmogorov's axioms . . . 143
4.1.4 Consequences of the axioms . . . 143
4.1.5 Interpretations of probability . . . 146
4.2 Conditional Probability and Bayes' Theorem . . . 148
4.2.1 Definition and the multiplication rule . . . 148
4.2.2 The law of total probability . . . 149
4.2.3 Independence and conditional independence . . . 149
4.2.4 Bayes' theorem . . . 149
4.2.5 Diagnostic tests: a malaria rapid test . . . 150
4.2.6 The base-rate fallacy in poverty targeting . . . 151
4.2.7 The Monty Hall problem . . . 153
4.2.8 Preview: naive Bayes classification . . . 154
4.3 Random Variables and Distributions . . . 156

4.3.1 Random variables as measurable functions . . . 156
4.3.2 The cumulative distribution function . . . 157
4.3.3 Discrete and continuous distributions . . . 157
4.3.4 The quantile function . . . 158
4.3.5 The empirical CDF and the Glivenko–Cantelli theorem . . . 158
4.3.6 Mixed distributions: zero-inflated expenditure . . . 160
4.4 Joint, Marginal and Conditional Distributions . . . 163
4.4.1 Discrete joint distributions . . . 163
4.4.2 Continuous joint distributions . . . 164
4.4.3 Independence of random variables . . . 165
4.4.4 Covariance: a preview . . . 165
4.4.5 Simulating income and consumption . . . 165
4.4.6 Copulas and Sklar's theorem . . . 166
4.5 Transformations of Random Variables . . . 169
4.5.1 The CDF method . . . 169
4.5.2 Change of variables in one dimension . . . 170
4.5.3 Log-normal incomes . . . 170
4.5.4 Multivariate change of variables . . . 172
4.5.5 Sums of independent variables and convolution . . . 173
4.5.6 Order statistics: minima and maxima . . . 174
4.5.7 The probability integral transform . . . 176
4.6 Generating Random Variables . . . 178
4.6.1 Pseudo-random number generators . . . 178
4.6.2 Seeds and reproducibility . . . 181
4.6.3 Inverse transform sampling . . . 181
4.6.4 The Box–Muller transform . . . 182
4.6.5 Acceptance–rejection sampling . . . 184
Further Reading . . . 187

5 Common Distributions in Data Science and Econometrics; Likelihood 189
5.1 Discrete Distributions . . . 190
5.1.1 Bernoulli and binomial . . . 190
5.1.2 The Poisson distribution and the law of rare events . . . 190
5.1.3 Geometric and negative binomial . . . 192
5.1.4 The multinomial distribution . . . 193
5.1.5 Computing with `scipy.stats` . . . 194
5.1.6 Count data in policy: overdispersion in clinic visits . . . 195
5.2 Continuous Distributions . . . 198
5.2.1 Uniform and normal . . . 198
5.2.2 Exponential: memorylessness and unemployment durations . . . 198
5.2.3 Gamma and Weibull . . . 199
5.2.4 Beta: a model for proportions . . . 201
5.2.5 Log-normal and Pareto: income and wealth . . . 201
5.2.6 Lorenz curves and the Gini coefficient from a distribution . . . 201
5.2.7 A catalogue in code . . . 206
5.3 Sampling Distributions: Chi-square, Student t and F . . . 208
5.3.1 The chi-square distribution . . . 208
5.3.2 Student's t distribution . . . 209
5.3.3 The F distribution . . . 209
5.3.4 Why these distributions govern regression inference . . . 209
5.3.5 The Cauchy distribution: a warning . . . 211
5.4 The Multivariate Normal Distribution . . . 214

5.4.1 Definition and density . . . 214
5.4.2 Conditional distributions and the Schur complement . . . 214
5.4.3 Relation to linear regression . . . 215
5.4.4 Mahalanobis distance . . . 215
5.4.5 Simulation via the Cholesky factor . . . 215
5.5 The Exponential Family and Generalised Linear Models . . . 218
5.5.1 Canonical form . . . 218
5.5.2 Moments from the log-partition function . . . 219
5.5.3 Generalised linear models . . . 220
5.5.4 Logistic and Poisson regression; the score and IRLS . . . 221
5.6 Likelihood . . . 224
5.6.1 Definition: likelihood versus probability . . . 224
5.6.2 The likelihood principle . . . 225
5.6.3 Normal regression: maximum likelihood is least squares . . . 225
5.6.4 The logistic likelihood and likelihood surfaces . . . 226
5.6.5 Sufficiency and the factorisation theorem . . . 228
5.6.6 Model checking: fitting with `.fit` and QQ plots . . . 229
5.6.7 A Bayesian preview: prior times likelihood . . . 231
Further Reading . . . 234

6 Expectation, Covariance, Limit Theorems and Concentration 235
6.1 Expectation . . . 236
6.1.1 Definition: discrete, continuous and general . . . 236
6.1.2 Linearity and the law of the unconscious statistician . . . 237
6.1.3 Jensen's inequality, risk aversion and certainty equivalents . . . 239
6.1.4 Moments, moment generating and characteristic functions . . . 242
6.2 Variance, covariance and correlation . . . 245
6.2.1 Definitions and basic properties . . . 245
6.2.2 Covariance matrices . . . 245
6.2.3 Correlation versus dependence . . . 247
6.2.4 Correlation versus causation, and Simpson's paradox . . . 247
6.3 Conditional expectation . . . 250
6.3.1 Definition and basic rules . . . 250
6.3.2 The law of iterated expectations and the law of total variance . . . 251
6.3.3 The CEF as the best mean-square predictor . . . 253
6.3.4 Linear projection versus the CEF . . . 253
6.3.5 Regression and average treatment effects . . . 255
6.4 Inequalities and the law of large numbers . . . 257
6.4.1 Markov's and Chebyshev's inequalities . . . 257
6.4.2 Modes of convergence . . . 259
6.4.3 The weak and strong laws of large numbers . . . 259
6.4.4 Consistency . . . 262
6.5 The central limit theorem . . . 263
6.5.1 Lindeberg–Lévy, Lyapunov and Lindeberg . . . 264
6.5.2 How good is the normal approximation? Berry–Esseen . . . 264
6.5.3 Multivariate CLT, Slutsky's theorem and continuous mapping . . . 266
6.5.4 The delta method . . . 267
6.5.5 Heavy tails slow convergence . . . 269
6.6 Concentration inequalities . . . 271
6.6.1 The Chernoff method . . . 271
6.6.2 Hoeffding's inequality and sub-Gaussian variables . . . 272
6.6.3 Concentration bounds versus the CLT . . . 272

6.6.4 Survey sample size determination . . . 274
6.6.5 High-dimensional phenomena . . . 277
Further Reading . . . 280

7 Estimation Principles: Moments, Maximum Likelihood and Information Geometry 281
7.1 Estimators and Their Properties . . . 282
7.1.1 Statistical models, estimators and estimates . . . 282
7.1.2 Consistency, asymptotic normality and efficiency . . . 284
7.1.3 Monte Carlo evaluation of estimators . . . 285
7.2 The Method of Moments and GMM . . . 288
7.2.1 The classical method of moments . . . 288
7.2.2 Generalised method of moments . . . 289
7.2.3 OLS and instrumental variables as moment estimators . . . 290
7.3 Maximum Likelihood Estimation . . . 295
7.3.1 Likelihood, log-likelihood and the score . . . 295
7.3.2 Closed-form examples . . . 296
7.3.3 Numerical MLE . . . 297
7.3.4 Likelihood-based regression: logit, probit and Poisson . . . 299
7.3.5 When maximum likelihood fails: separation . . . 302
7.4 Fisher Information and Asymptotic Theory . . . 304
7.4.1 The score and the information identity . . . 304
7.4.2 The Cramér–Rao lower bound . . . 305
7.4.3 Consistency and asymptotic normality of the MLE . . . 305
7.4.4 Observed versus expected information; standard errors . . . 306
7.4.5 Misspecification: quasi-MLE and the sandwich . . . 309
7.4.6 The trinity: Wald, likelihood-ratio and score tests . . . 311
7.5 Information Geometry Basics . . . 314
7.5.1 The Kullback–Leibler divergence . . . 314
7.5.2 Maximum likelihood as minimum KL . . . 315
7.5.3 Fisher information as a Riemannian metric . . . 317
7.5.4 The natural gradient . . . 318
7.5.5 Cross-entropy loss in machine learning . . . 320
7.6 Bayesian Estimation . . . 321
7.6.1 Prior, likelihood and posterior . . . 321
7.6.2 Conjugate examples . . . 322
7.6.3 MAP estimation as penalised likelihood . . . 324
7.6.4 The Bernstein–von Mises theorem . . . 325
7.6.5 Grid approximation . . . 325
7.6.6 Probabilistic programming . . . 327
Further Reading . . . 329

III Optimisation 331

8 Optimisation: Convexity, Gradients, Hessians, Newton and Stochastic Gradient Methods 333
8.1 Optimisation in data science and econometrics . . . 334
8.1.1 The generic problem . . . 334
8.1.2 Empirical risk minimisation and M-estimators . . . 334
8.1.3 Existence of minimisers . . . 337
8.1.4 Local versus global optima . . . 338
8.2 Multivariate calculus for optimisation . . . 340

8.2.1 Gradients, directional derivatives and Jacobians . . . 340
8.2.2 Hessians and Taylor's theorem . . . 341
8.2.3 Optimality conditions . . . 341
8.2.4 Numerical gradients and gradient checking . . . 343
8.3 Convexity . . . 347
8.3.1 Convex sets and convex functions . . . 347
8.3.2 First- and second-order characterisations . . . 347
8.3.3 Operations that preserve convexity . . . 348
8.3.4 Strong convexity and smoothness . . . 348
8.3.5 Concavity of the logit log-likelihood . . . 349
8.3.6 Quasi-concavity and utility functions . . . 351
8.4 Gradient descent . . . 353
8.4.1 The algorithm . . . 353
8.4.2 Convergence rates . . . 353
8.4.3 The condition number and zig-zagging . . . 354
8.4.4 Choosing the step: backtracking and the Armijo condition . . . 356
8.4.5 Momentum and Nesterov acceleration . . . 357
8.4.6 An econometric example and feature scaling . . . 358
8.5 Newton and quasi-Newton methods . . . 360
8.5.1 Newton's method . . . 361
8.5.2 Damping and globalisation . . . 361
8.5.3 Newton–Raphson and IRLS for logistic regression . . . 362
8.5.4 Gauss–Newton for nonlinear least squares . . . 364
8.5.5 Quasi-Newton methods: BFGS and L-BFGS . . . 366
8.6 Stochastic gradient descent . . . 370
8.6.1 From gradient descent to stochastic approximation . . . 370
8.6.2 Constant step sizes and the noise floor . . . 370
8.6.3 Momentum and adaptive methods: AdaGrad, RMSProp, Adam . . . 373
8.6.4 Practical diagnostics and software . . . 376
Further Reading . . . 379

9 Constrained Optimisation and Lagrange Multipliers in Policy Problems 381
9.1 Equality Constraints and the Lagrangian . . . 382
9.1.1 The problem and its geometry . . . 382
9.1.2 The Lagrange multiplier theorem . . . 383
9.1.3 Consumer utility maximisation . . . 384
9.1.4 Solving numerically in Python . . . 385
9.2 Interpretation of Multipliers . . . 388
9.2.1 Shadow prices and the value function . . . 388
9.2.2 The envelope theorem . . . 389
9.2.3 Comparative statics . . . 391
9.2.4 Second-order conditions and the bordered Hessian . . . 392
9.3 Inequality Constraints and the KKT Conditions . . . 393
9.3.1 The general problem . . . 393
9.3.2 Constraint qualifications and the KKT theorem . . . 394
9.3.3 Corner solutions . . . 395
9.4 Duality and Linear Programming . . . 398
9.4.1 The Lagrange dual function . . . 398
9.4.2 Strong duality and Slater's condition . . . 399
9.4.3 Linear programming and LP duality . . . 401
9.4.4 The least-cost nutritious food basket . . . 402
9.5 Constrained Estimation in Statistics and Machine Learning . . . 405

9.5.1 Restricted least squares . . . 405
9.5.2 Constrained maximum likelihood . . . 407
9.5.3 Ridge and lasso as constrained problems . . . 407
9.5.4 The support vector machine . . . 410
9.5.5 Algorithms: projection, penalties and barriers . . . 411
9.6 Policy Optimisation Applications . . . 414
9.6.1 Allocating a fixed budget across programmes . . . 415
9.6.2 The social planner with equity weights . . . 417
9.6.3 Targeting transfers to minimise the poverty gap . . . 419
9.6.4 A Ramsey-style taxation problem . . . 421
9.6.5 Sensitivity analysis . . . 423
Further Reading . . . 426

IV Computation and Uncertainty 429

10 Numerical Integration and Monte Carlo Simulation 431
10.1 Why Integrals Are Everywhere . . . 432
10.1.1 Expectations and probabilities . . . 432
10.1.2 Bayesian integrals . . . 433
10.1.3 Welfare: consumer surplus and present values . . . 433
10.1.4 A test bench of integrals with closed forms . . . 434
10.2 Deterministic Quadrature . . . 437
10.2.1 The trapezoidal rule . . . 437
10.2.2 Simpson's rule . . . 438
10.2.3 Gaussian quadrature . . . 440
10.2.4 Gauss–Hermite quadrature for normal expectations . . . 441
10.2.5 Adaptive quadrature in SciPy . . . 442
10.2.6 The curse of dimensionality . . . 444
10.3 Monte Carlo Integration . . . 445
10.3.1 The estimator and its error . . . 446
10.3.2 Estimating π . . . 446
10.3.3 High-dimensional integrals: the rate is free, the constant is not . . . 448
10.3.4 Simulated probabilities . . . 449
10.3.5 Risk assessment of a public investment . . . 450
10.4 Variance Reduction and Quasi-Monte Carlo . . . 452
10.4.1 Antithetic variates . . . 453
10.4.2 Control variates . . . 453
10.4.3 Importance sampling . . . 455
10.4.4 Stratified sampling . . . 457
10.4.5 Quasi-Monte Carlo . . . 459
10.5 Sampling from Distributions and Markov Chain Monte Carlo . . . 463
10.5.1 The inverse-CDF method . . . 463
10.5.2 Acceptance–rejection . . . 463
10.5.3 Markov chains with a prescribed stationary distribution . . . 465
10.5.4 The Metropolis–Hastings algorithm . . . 466
10.5.5 Random-walk Metropolis for a logistic regression posterior . . . 467
10.5.6 Gibbs sampling . . . 469
10.5.7 Convergence diagnostics . . . 472
10.5.8 Hamiltonian Monte Carlo and Stan . . . 472
10.6 Designing Simulation Studies for Policy . . . 475
10.6.1 The ADEMP framework . . . 475

10.6.2 Performance measures and their Monte Carlo error . . . 476
10.6.3 Example: confidence intervals under heteroskedasticity . . . 476
10.6.4 Reproducibility and random number generation . . . 478
10.6.5 Microsimulation of a cash-transfer programme . . . 478
Further Reading . . . 482

11 Resampling: the Bootstrap, Permutation Tests and Uncertainty Intervals 483
11.1 Sampling Distributions and the Plug-in Principle . . . 484
11.1.1 Sampling distributions, standard errors and confidence intervals . . . 484
11.1.2 The empirical distribution function . . . 486
11.1.3 Statistical functionals and the plug-in principle . . . 487
11.1.4 When analytic standard errors are hard: the influence function . . . 489
11.2 The Nonparametric Bootstrap . . . 491
11.2.1 The bootstrap principle . . . 491
11.2.2 Bias correction . . . 494
11.2.3 How many bootstrap replications? . . . 494
11.2.4 Why the bootstrap works . . . 495
11.2.5 When the bootstrap fails . . . 495
11.3 Bootstrap Confidence Intervals . . . 497
11.3.1 Five intervals . . . 498
11.3.2 Where the intervals come from . . . 498
11.3.3 A confidence interval for the Gini coefficient . . . 499
11.3.4 Which interval should we use? A coverage study . . . 501
11.4 The Bootstrap for Regression and Dependent Data . . . 503
11.4.1 Pairs and residual bootstrap . . . 503
11.4.2 The wild bootstrap . . . 504
11.4.3 Clustered data: the cluster bootstrap . . . 506
11.4.4 Few clusters: the wild cluster bootstrap . . . 508
11.4.5 Time series: the moving block bootstrap . . . 509
11.5 Permutation and Randomisation Tests . . . 511
11.5.1 Randomisation inference in an experiment . . . 512
11.5.2 Confidence sets by test inversion . . . 514
11.5.3 Permutation tests without an experiment . . . 515
11.5.4 Power . . . 515
11.5.5 Multiple testing . . . 517
11.6 The Jackknife, Cross-Validation and Predictive Uncertainty . . . 519
11.6.1 The jackknife . . . 519
11.6.2 Cross-validation as resampling for prediction error . . . 520
11.6.3 Prediction intervals . . . 523
11.6.4 Split conformal prediction . . . 523
11.6.5 Forecast uncertainty and pseudo-out-of-sample evaluation . . . 525
Further Reading . . . 529

12 Matrix Calculus for Machine Learning, Regularisation and the Bias-Variance Trade-off 531
12.1 Matrix Calculus . . . 532
12.1.1 Layout conventions . . . 532
12.1.2 Linear and quadratic forms . . . 532
12.1.3 The chain rule for vector functions . . . 533
12.1.4 The differential approach of Magnus and Neudecker . . . 533
12.1.5 Jacobians and Hessians of vector functions . . . 535
12.1.6 Numerical verification with finite differences . . . 535
12.2 Applications of Matrix Calculus . . . 538

12.2.1 Ordinary least squares . . . 538
12.2.2 MLE of the multivariate normal mean and covariance . . . 539
12.2.3 Logistic regression: gradient, Hessian and Newton's method . . . 540
12.2.4 Softmax regression (multinomial logit) . . . 542
12.2.5 Weighted and generalised least squares . . . 543
12.3 Automatic Differentiation and Backpropagation . . . 545
12.3.1 Computational graphs . . . 545
12.3.2 Forward and reverse mode . . . 546
12.3.3 Backpropagation for a two-layer network . . . 548
12.4 The Bias–Variance Trade-off . . . 552
12.4.1 Decomposition of expected prediction error . . . 552
12.4.2 A simulation: polynomial regression of increasing degree . . . 552
12.4.3 Under- and overfitting: training versus test error . . . 554
12.4.4 A note on double descent . . . 556
12.5 Regularisation . . . 558
12.5.1 Ridge regression . . . 558
12.5.2 The lasso . . . 561
12.5.3 Sparsity and the geometry of the ℓ_1 ball . . . 563
12.5.4 The elastic net . . . 564
12.5.5 Regularisation paths . . . 565
12.5.6 Standardisation of features . . . 566
12.6 Model Selection and Forecasting with Many Predictors . . . 568
12.6.1 K-fold cross-validation . . . 568
12.6.2 Information criteria: AIC and BIC . . . 569
12.6.3 Pipelines in scikit-learn . . . 570
12.6.4 Forecasting with many predictors: a pseudo-out-of-sample exercise . . . 571
Further Reading . . . 575

V Synthesis **577**

13 Problem Set Clinic and Applied Foundations Review **579**
13.1 How to Solve Problems in This Course . . . 580
13.1.1 A seven-step workflow . . . 580
13.1.2 Proof techniques used throughout the course . . . 580
13.1.3 Checking analytical results by simulation . . . 581
13.1.4 A reproducible research workflow . . . 584
13.2 Case Study 1: Returns to Education from a Simulated Labour Force Survey . . . 587
13.2.1 The simulated survey . . . 587
13.2.2 Least squares via QR . . . 587
13.2.3 Fixed effects and the Frisch–Waugh–Lovell theorem . . . 588
13.2.4 Heteroskedasticity-robust standard errors . . . 588
13.2.5 Bootstrap confidence intervals . . . 590
13.2.6 Interpretation and the endogeneity problem . . . 592
13.3 Case Study 2: Poverty Targeting with Logistic Regression . . . 595
13.3.1 The simulated household survey . . . 596
13.3.2 Two scores: a PCA asset index and a logit PMT . . . 596
13.3.3 Regularisation chosen by cross-validation . . . 599
13.3.4 Targeting under a budget: inclusion and exclusion errors . . . 599
13.4 Case Study 3: Macroeconomic Forecasting with Many Predictors . . . 603
13.4.1 An approximate factor model . . . 603
13.4.2 Three ways to use 100 predictors . . . 603

13.4.3 Pseudo-out-of-sample evaluation . . . 604
13.4.4 Is the improvement significant? Diebold–Mariano and the block bootstrap 604
13.5 Case Study 4: Budget Allocation under Uncertainty . . . 609
13.5.1 The model . . . 609
13.5.2 Monte Carlo: sample average approximation . . . 610
13.5.3 Sensitivity analysis . . . 612
13.6 Comprehensive Review . . . 614
13.6.1 A concept map of the course . . . 614
13.6.2 Key results and where they are used . . . 614
13.6.3 The most common pitfalls . . . 615
13.6.4 Ten exam-style problems with worked solutions . . . 615
Further Reading . . . 621

A A Python Toolkit for the Course **623**
A.1 Setting up . . . 623
A.2 NumPy essentials . . . 623
A.3 SciPy and scikit-learn essentials . . . 625
A.4 From mathematics to code . . . 626
A.5 pandas and Matplotlib in one paragraph each . . . 626

B Answers and Hints to Selected Exercises **627**

References **665**

List of Python Listings

1.1 From index and sum notation to NumPy . . . 6
1.2 Norms, angles and Cauchy–Schwarz in practice . . . 12
1.3 A Laspeyres CPI from a simulated basket . . . 14
1.4 Broadcasting: centring, row scaling, pairwise distances . . . 20
1.5 A three-sector input–output model (illustrative coefficients) . . . 22
1.6 Centring matrix, covariance and correlation of a simulated survey . . . 25
1.7 Inside IEEE 754: epsilon, spacing, overflow, underflow . . . 29
1.8 Catastrophic cancellation and its cures . . . 32
1.9 Stable softmax, log-sum-exp and logit log-likelihood . . . 34
1.10 Naive, pairwise, Kahan and exactly rounded summation . . . 36
1.11 Three variance algorithms under a growing offset . . . 38
1.12 Loops versus vectorised code; cubic cost of dense algebra . . . 43
1.13 A sparse employer–employee matrix (simulated register) . . . 45
1.14 Seeds, independent streams, versions and summation order . . . 47
2.1 Four fundamental subspaces and solvability . . . 54
2.2 Rank, the dummy trap and numerical rank . . . 59
2.3 A programme effect collinear with region effects . . . 61
2.4 Hat matrix, annihilator and the ANOVA decomposition . . . 65
2.5 Loss of orthogonality: CGS, MGS and Householder . . . 67
2.6 Frisch–Waugh–Lovell: LSDV equals the within estimator . . . 72
2.7 Leverage from a thin QR factorisation . . . 75
2.8 LU for an input–output model; Cholesky for simulation . . . 78
2.9 Householder QR versus the normal equations . . . 81
2.10 Penrose conditions and minimum-norm least squares . . . 86
2.11 Minimum-norm least squares across $p = n$ . . . 88
2.12 Balancing illustrative national accounts . . . 90
3.1 Eigenpairs, diagonalisation and matrix powers . . . 98
3.2 A simulated three-state labour-market Markov chain . . . 100
3.3 Spectral decomposition, PD tests and Mahalanobis distance . . . 105
3.4 Power iteration, inverse iteration and Rayleigh-quotient iteration . . . 109
3.5 PageRank of a simulated inter-district trade network; eig versus eigh . . . 112
3.6 The SVD, its relation to eigen-decompositions, and Eckart–Young . . . 115
3.7 Low-rank approximation of a digit mosaic . . . 117
3.8 PCA of a simulated macro panel and a diffusion-index forecast . . . 122
3.9 A Filmer–Pritchett asset index on a simulated household survey . . . 125
3.10 Condition numbers, the Hilbert matrix and least squares . . . 129
3.11 Multicollinearity among simulated policy covariates . . . 132
4.1 Inclusion–exclusion versus simulation for the matching problem . . . 144
4.2 Relative frequencies of a six in five independent sequences of die rolls . . . 146
4.3 Predictive values of a diagnostic test as functions of prevalence . . . 150

4.4 Inclusion and exclusion errors of a simulated proxy means test . . . 151
4.5 Simulating the Monty Hall game with k doors . . . 153
4.6 A naive Bayes poverty classifier built from conditional frequencies . . . 154
4.7 Empirical CDFs of simulated incomes and the sup-distance D_n . . . 159
4.8 A zero-inflated expenditure distribution: CDF, quantiles and moments . . . 161
4.9 Joint, marginal and conditional PMFs of household size and poverty . . . 163
4.10 Simulated bivariate normal log income and log consumption . . . 165
4.11 A Gaussian copula with log-normal and exponential marginals . . . 167
4.12 Log-normal incomes: change-of-variables density versus simulation . . . 170
4.13 Verifying the Jacobian calculation for total and share . . . 172
4.14 Numerical convolution of densities and PMFs . . . 173
4.15 Minima and maxima of iid samples . . . 175
4.16 PIT histograms for calibrated and miscalibrated density forecasts . . . 176
4.17 Middle-square, the RANDU defect, and reproducible streams with PCG64 . . . 179
4.18 Inverse transform sampling for exponential, Pareto and discrete targets . . . 181
4.19 The Box–Muller transform and moment checks . . . 183
4.20 Acceptance–rejection sampling from a Beta density . . . 184
5.1 The Poisson limit and the horse-kick data . . . 191
5.2 Moments and PMFs of discrete distributions . . . 194
5.3 Overdispersed clinic visits . . . 195
5.4 Memorylessness and Weibull hazards . . . 199
5.5 Log-normal incomes, Lorenz curve, Gini and poverty . . . 202
5.6 Pareto tails and the Hill estimator . . . 204
5.7 Moments of continuous distributions . . . 206
5.8 Building χ^2, t and F from normals . . . 210
5.9 Running means under the Cauchy distribution . . . 211
5.10 Multivariate normal: simulation, conditioning, Mahalanobis . . . 216
5.11 Moments from derivatives of the log-partition function . . . 219
5.12 Logistic and Poisson regression by IRLS . . . 221
5.13 Gaussian likelihood and least squares . . . 225
5.14 Likelihood contours for normal and logistic models . . . 227
5.15 Fitting duration models and QQ plots . . . 229
5.16 Beta prior, binomial likelihood, beta posterior . . . 231
6.1 Expectation by summation, quadrature, LOTUS and Monte Carlo . . . 238
6.2 Jensen's inequality, certainty equivalents and insurance . . . 240
6.3 Moments from the MGF; characteristic functions . . . 243
6.4 Covariance matrices, composite indices and a non-PSD trap . . . 246
6.5 Simpson's paradox in a simulated training programme . . . 248
6.6 Iterated expectations and the within/between decomposition . . . 251
6.7 The CEF, binned means and the linear projection . . . 253
6.8 Difference in means, OLS on a dummy, and selection bias . . . 255
6.9 Exact tails versus Markov and Chebyshev bounds . . . 258
6.10 Running means and the failure of the LLN for Cauchy data . . . 260
6.11 Convergence in probability: Monte Carlo versus Chebyshev . . . 261
6.12 Sampling distribution of standardised means . . . 264
6.13 Delta-method standard errors for a ratio and an elasticity . . . 267
6.14 Coverage of CLT intervals for Pareto incomes . . . 269
6.15 Exact tails, CLT, Chebyshev, Hoeffding and Chernoff . . . 272
6.16 Sample sizes for a proportion and for ten regions at once . . . 274
6.17 Family-wise error, Bonferroni and the maximum of m means . . . 276
6.18 Norm concentration and near-orthogonality of random vectors . . . 277

7.1 Monte Carlo bias, variance and MSE . . . 283
7.2 Mean versus median under light and heavy tails (plotting code omitted) . . . 285
7.3 Method of moments for gamma and beta data . . . 288
7.4 OLS, IV, 2SLS and two-step GMM for returns to schooling . . . 291
7.5 Sampling distributions of OLS and IV (plotting code omitted) . . . 293
7.6 Closed-form and numerical MLEs; invariance . . . 296
7.7 Numerical gamma MLE in log-parameters (plotting code omitted) . . . 298
7.8 Logit, probit and Poisson regression by numerical ML . . . 300
7.9 Complete separation in a logit model . . . 302
7.10 CRLB checks; observed vs expected information . . . 306
7.11 Logit standard errors: Hessian, BFGS and simulation . . . 308
7.12 Poisson QMLE: model-based versus sandwich standard errors . . . 310
7.13 Simulated null distributions of the trinity (plotting code omitted) . . . 312
7.14 KL divergence and pseudo-true parameters (plotting code omitted) . . . 315
7.15 Ordinary versus natural gradient for the normal model (plotting code omitted) . . . 318
7.16 Conjugate updating and Bernstein–von Mises . . . 322
7.17 MAP with a Gaussian prior equals ridge regression . . . 324
7.18 Grid approximation of a logit posterior (plotting code omitted) . . . 326
8.1 OLS, LAD and Huber regression as numerical M-estimators . . . 335
8.2 Multi-start local search on a non-convex function . . . 338
8.3 Finding and classifying stationary points . . . 342
8.4 Finite differences and gradient checking for the logit loss . . . 344
8.5 Numerical checks of convexity, smoothness and quasi-concavity . . . 350
8.6 Gradient descent on quadratics with increasing condition number . . . 354
8.7 GD, backtracking, heavy-ball and Nesterov on least squares . . . 358
8.8 Pure versus damped Newton on $f(x) = \sqrt{1 + x^2}$ . . . 361
8.9 Newton–Raphson (IRLS) for a logit of programme take-up . . . 363
8.10 Gauss–Newton for a logistic diffusion curve . . . 365
8.11 Comparing optimisers in `scipy.optimize.minimize` . . . 367
8.12 Minibatch SGD and learning-rate schedules for a large logit . . . 371
8.13 Momentum, AdaGrad, RMSProp and Adam in NumPy . . . 374
9.1 Tangency of level set and constraint (plotting code omitted) . . . 385
9.2 Marshallian demand: closed form and numerics . . . 386
9.3 Envelope theorem, comparative statics and bordered Hessian (plotting code omitted) 390
9.4 KKT examples checked with SLSQP (plotting code omitted) . . . 396
9.5 Dual functions with and without a duality gap (plotting code omitted) . . . 399
9.6 Least-cost nutritious diet and its shadow prices (plotting code omitted) . . . 402
9.7 Restricted least squares and the F test . . . 406
9.8 Penalised versus constrained ridge and lasso (plotting code omitted) . . . 408
9.9 Soft-margin SVM through its dual (plotting code omitted) . . . 410
9.10 Projected gradient, penalty and barrier methods (plotting code omitted) . . . 412
9.11 Budget allocation with diminishing returns (plotting code omitted) . . . 415
9.12 Equity-weighted transfers . . . 417
9.13 Poverty-gap-minimising regional targeting (plotting code omitted) . . . 419
9.14 Ramsey taxes and the marginal cost of public funds . . . 421
9.15 Local (IFT) and Monte Carlo sensitivity analysis (plotting code omitted) . . . 423
10.1 Policy integrals with closed forms, checked by quadrature . . . 434
10.2 Composite trapezoidal and Simpson rules . . . 438
10.3 Gauss–Legendre and Gauss–Hermite quadrature . . . 441
10.4 Adaptive quadrature with `quad` and `dblquad` . . . 442
10.5 Tensor-product quadrature as dimension grows . . . 444

10.6 Monte Carlo estimation of π with standard errors . . . 447
10.7 Monte Carlo in high dimension and simulated choice probabilities . . . 449
10.8 Monte Carlo risk assessment of a public investment . . . 450
10.9 Antithetic and control variates . . . 454
10.10 Importance sampling for a rare event . . . 456
10.11 Stratified sampling: an integral and a simulated survey . . . 458
10.12 Quasi-Monte Carlo with Sobol and Halton sequences . . . 460
10.13 Inverse-CDF and acceptance–rejection sampling . . . 464
10.14 Random-walk Metropolis for a Bayesian logistic regression . . . 467
10.15 Gibbs sampling for a bivariate normal . . . 470
10.16 A minimal Hamiltonian Monte Carlo sampler . . . 473
10.17 An ADEMP simulation study of OLS confidence intervals . . . 476
10.18 Microsimulation of a targeted cash transfer under uncertainty . . . 478
11.1 Coverage of the t-interval for a skewed mean (plotting code omitted) . . . 485
11.2 The ECDF, the DKW band and plug-in estimators (plotting code omitted) . . . 488
11.3 Bootstrap standard errors and bias (plotting code omitted) . . . 492
11.4 Five bootstrap intervals for a Gini coefficient . . . 499
11.5 Pairs, residual and wild bootstrap for regression . . . 504
11.6 Cluster bootstrap for a survey clustered by EA . . . 506
11.7 Randomisation inference for a simulated school-feeding trial (plotting code omitted) 513
11.8 LOO and K-fold cross-validation for ridge regression (plotting code omitted) . . . 521
11.9 Pseudo-out-of-sample evaluation and a bootstrap fan chart (plotting code omitted) 525
12.1 Finite-difference check of matrix-calculus identities . . . 535
12.2 OLS from its gradient; Gaussian MLE and its score . . . 539
12.3 Logistic regression by Newton–Raphson . . . 541
12.4 Softmax gradient check; GLS versus OLS . . . 543
12.5 Forward and reverse mode from scratch . . . 546
12.6 A two-layer neural network with hand-coded backpropagation (plotting code omitted) . . . 548
12.7 Monte Carlo bias–variance decomposition (plotting code omitted) . . . 552
12.8 Training and test error; effective degrees of freedom (plotting code omitted) . . . 554
12.9 Ridge: closed form, SVD, augmented data, bias and variance . . . 559
12.10 The lasso by coordinate descent, with a KKT check . . . 562
12.11 Ridge and lasso paths on the diabetes data (plotting code omitted) . . . 565
12.12 10-fold cross-validation for the lasso, with the one-SE rule (plotting code omitted) 568
12.13 Pipelines: StandardScaler with RidgeCV, LassoCV and AIC/BIC . . . 570
12.14 Expanding-window forecasting: OLS, ridge, lasso, PCR (plotting code omitted) . . . 572
13.1 Simulation checks of a variance formula and a gradient . . . 582
13.2 Seeds, spawned streams, environment and result fingerprints . . . 585
13.3 Mincer regression: QR, FWL, within estimator and HC1 . . . 588
13.4 Pairs and wild bootstrap intervals for the return to schooling . . . 591
13.5 Monte Carlo of ability bias, measurement error and IV . . . 593
13.6 PCA asset index and logit PMT by Newton/IRLS, checked against scikit-learn . . . 596
13.7 Regularised PMT, budget-constrained targeting errors, ROC and a fairness audit 599
13.8 Expanding-window forecasts: AR, diffusion index, ridge and lasso . . . 604
13.9 Budget allocation by SAA and SLSQP: expected and risk-adjusted objectives . . . 610
13.10 Numerical checks of the ten review problems . . . 617
A.1 NumPy essentials . . . 623
A.2 SciPy and scikit-learn essentials . . . 625

Preface

Every data-science method used in economic policy analysis rests on a small number of mathematical ideas. A regression is an orthogonal projection. A principal component is an eigenvector. A standard error is a statement about a sampling distribution, and a confidence interval is justified by a limit theorem. A maximum-likelihood estimate, a lasso fit and a neural network are all solutions of optimisation problems; an optimal allocation of a public budget is the solution of a *constrained* optimisation problem whose Lagrange multiplier is the shadow price of the budget. When the integrals are too hard we simulate, and when the sampling distribution is unknown we resample. These ideas are the subject of this book.

The book accompanies **DEP 803: Mathematical and Statistical Foundations for Data Science**, a three-credit graduate course taken by students of Data Science and Economic Policy at the University of Cape Coast. Its aim, in the words of the course description, is to provide the mathematical toolkit for data science and econometric modelling—linear algebra, probability, optimisation and statistical inference—with an emphasis on *translating formal results into computational practice* for estimation, prediction and uncertainty quantification in policy analytics.

Who this book is for

The readers I have in mind are graduate (MPhil and PhD) students in data science, economics and public policy. Some arrive with a strong background in economics and statistics but little programming; others are confident programmers who have not seen a proof since their first degree. The book therefore does three things at once. It states results precisely and proves the important ones, because a graduate student must be able to read the methodological literature. It implements every idea in Python, because a result that cannot be computed is of little use to an analyst. And it asks, repeatedly, what each result *means* for a policy question: the targeting of a social transfer, the evaluation of a programme, the forecast of inflation, the allocation of a district budget.

Course objectives

On completing the course a student should be able to:

1. derive and manipulate the key results in linear algebra, probability and optimisation that underpin modern data-science methods;
2. implement numerical optimisation and simulation techniques to solve estimation and inference problems;
3. assess uncertainty using sampling distributions, asymptotics and resampling in policy-relevant contexts.

How to use this book

Each chapter corresponds to one lecture topic of the course outline (see the Course Map that follows). Chapters open with *learning objectives*, close with a *chapter summary* and a guide to further reading, and contain exercises at the end of every section. A small number of highlighted boxes are used so that important ideas stand out.

⋆ Key Concept: what it looks like

A *Key Concept* box contains an idea, formula or result that you must understand and be able to use. These are the highlighted concepts that graduate students should master; examination questions are very often built around them.

Historical Note: the people behind the mathematics

A *Historical Note* tells how an idea arose, who discovered it and why. Least squares made its name by recovering a lost dwarf planet, the bootstrap grew out of the jackknife, and the Monte Carlo method was named after a casino by physicists building the first computers. Knowing where an idea came from is one of the best ways of understanding what it is for.

△ Caution: common mistakes

A *Caution* box warns about a common error in reasoning, a trap in computer arithmetic, or a situation in which a method silently fails.

Policy Lens: from theorem to policy

A *Policy Lens* box shows where the mathematics appears in economic-policy analysis: poverty and inequality measurement, programme evaluation, targeting, forecasting and budgeting.

Graduate Perspective: looking ahead

A *Graduate Perspective* box connects the topic to the research literature and to more advanced courses: measure-theoretic probability, asymptotic statistics, convex analysis, high-dimensional statistics and machine learning.

Python. Every Python listing in the book has been executed, and the output printed beneath it is the actual output of the program. The programs are deliberately short and self-contained: each one illustrates a single idea and can be run on its own. They use only the standard scientific Python stack (NumPy, SciPy, pandas, Matplotlib and scikit-learn), and all data are either simulated or bundled with scikit-learn, so no internet connection is needed. Appendix A is a quick reference for the Python tools used in the course. The best way to learn from a listing is to run it, then *change something*—the sample size, the seed, the step size, the distribution—predict what will happen, and check.

Exercises. Exercises are placed at the end of each section and tagged Theory for derivations and proofs, Python for programming, Policy for policy applications and interpretation, and ★ Graduate for harder, research-flavoured problems. Answers and hints to selected exercises are given in Appendix B. Chapter 13 is a problem-set clinic: it works through four integrated case studies and a set of examination-style problems with full solutions.

Reading. The core readings of the course are Murphy (2022), Matsuura (2022), Géron (2022) and Goulet Coulombe et al. (2022). Murphy provides the probabilistic view of machine learning that runs through Chapters 4 to 12; Matsuura the Bayesian modelling perspective; Géron the practical machine-learning workflow in Python; and Goulet Coulombe and co-authors a careful study of when machine learning actually helps macroeconomic forecasting, to which we return in Chapters 3, 11, 12 and 13. Each chapter ends with suggestions for further reading from the wider literature.

Delivery

The course is taught in blended mode: face-to-face lectures, seminars and hands-on computer labs, together with online synchronous sessions and asynchronous material (lecture slides, coding notebooks, discussion forums and recorded demonstrations) on the university's learning management system. The Python programs in the companion repository are the starting point for the labs.

Licence

The book is released under the Creative Commons Attribution 4.0 International Licence (CC BY 4.0) and the programs under the MIT Licence; see the copyright page. You are welcome to use, share and adapt both, provided you give credit. Corrections and suggestions are very welcome.

Stephen E. Moore
Cape Coast, 2026
https://moorestephen.info/

Course Map

Course description

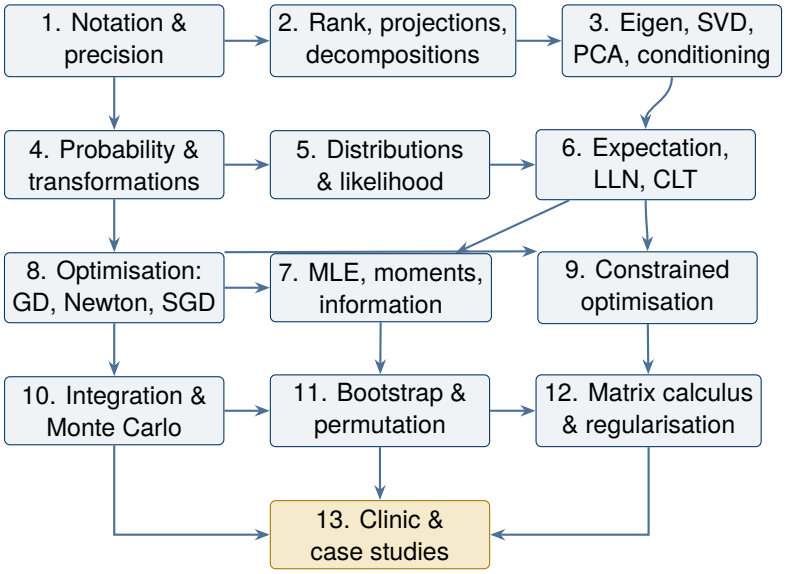
DEP 803: Mathematical and Statistical Foundations for Data Science (3 credits) provides the mathematical toolkit for data science and econometric modelling: linear algebra, probability, optimisation and statistical inference. Emphasis is on translating formal results into computational practice for estimation, prediction and uncertainty quantification in policy analytics.

Lecture plan

The thirteen topics of the course outline correspond one-to-one to the thirteen chapters of the book and to the thirteen lecture slide decks.

Lecture	Topic (course outline)	Chapter	Python lab
1	Mathematical notation, vectors/matrices, and computational precision	1	NumPy arrays, broadcasting, floating point, log-sum-exp
2	Linear algebra for modelling (rank, projections, decompositions)	2	rank, projections, FWL, QR and Cholesky
3	Eigenvalues, SVD/PCA intuition, and conditioning	3	Markov chains, SVD, PCA asset index, condition numbers
4	Probability foundations; random variables and transformations	4	Bayes' rule, targeting errors, sampling and transformations
5	Common distributions used in DS and econometrics; likelihood concepts	5	`scipy.stats`, income distributions, likelihood surfaces
6	Expectations, covariance, LLN/CLT, and concentration intuition	6	simulating the LLN and CLT, delta method, sample sizes
7	Estimation principles (MLE, moments) and information geometry basics	7	numerical MLE, GMM/IV, Fisher information, KL divergence
8	Optimisation (convexity, gradients, Hessians) and Newton/SGD methods	8	gradient descent, Newton/IRLS, BFGS, SGD and Adam
9	Constrained optimisation and Lagrange multipliers in policy problems	9	SLSQP, `linprog`, least-cost diets, budget allocation
10	Numerical integration and Monte-Carlo simulation	10	quadrature, variance reduction, MCMC, microsimulation
11	Resampling (bootstrap, permutation tests) and uncertainty intervals	11	bootstrap CIs, wild/cluster bootstrap, randomisation inference
12	Matrix calculus for ML; regularisation and bias-variance trade-offs	12	backpropagation, ridge/lasso, cross-validation, forecasting
13	Problem set clinic and applied foundations review	13	four integrated case studies

How the chapters fit together



Part I (Chapters 1–3) builds the linear-algebra language of models and data. Part II (Chapters 4–7) develops probability, distributions, limit theorems and the principles of estimation. Part III (Chapters 8–9) treats the optimisation problems that estimation and policy design lead to. Part IV (Chapters 10–12) turns to computation and uncertainty: simulation, resampling and regularisation. Part V (Chapter 13) brings everything together in applied case studies.

Mode of delivery

Blended delivery: (i) face-to-face lectures and seminars, and hands-on computer labs; and (ii) online synchronous sessions, together with asynchronous materials on the learning management system (lecture slides, coding notebooks, discussion forums and recorded demonstrations).

Reading materials

- K. P. Murphy (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- K. Matsuura (2022). *Bayesian Statistical Modeling with Stan, R, and Python*. Springer.
- A. Géron (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media.
- P. Goulet Coulombe, M. Leroux, D. Stevanovic and S. Surprenant (2022). How is machine learning useful for macroeconomic forecasting? *Journal of Applied Econometrics*, 37(5), 920–964.

Further references, chosen chapter by chapter, are collected in the References at the end of the book.

Notation

The following conventions are used throughout the book. Chapter 1 discusses them in more detail.

Symbol	Meaning
$\mathbb{R}$, $\mathbb{R}^n$, $\mathbb{R}^{n\times p}$	real numbers; column vectors with n entries; $n \times p$ real matrices
a, $\mathbf{x}$, $\mathbf{A}$	scalars (italic), vectors (bold lower case), matrices (bold or capital letters)
$\mathbf{x}^\top$, $\mathbf{A}^\top$	transpose; all vectors are column vectors, so $\mathbf{x}^\top$ is a row vector
$\mathbf{X} \in \mathbb{R}^{n\times p}$	data (design) matrix: n observations (rows) on p variables (columns)
$\mathbf{x}_i^\top$	the ith row of $\mathbf{X}$, the record of observation i
$\mathbf{1}$, $\mathbf{0}$, $\mathbf{I}_n$	vector of ones, zero vector, $n \times n$ identity matrix
$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y}$	inner product; $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}$ Euclidean norm
$\operatorname{rank}(\mathbf{A})$, $\operatorname{tr}(\mathbf{A})$, $\det(\mathbf{A})$	rank, trace and determinant
$\operatorname{col}(\mathbf{A})$, $\operatorname{null}(\mathbf{A})$	column space and null space
$\mathbf{A}^{-1}$, $\mathbf{A}^{+}$	inverse; Moore–Penrose pseudoinverse
λ_j, σ_j	eigenvalues; singular values ($\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$)
$\kappa(\mathbf{A})$	condition number $\sigma_{\max}/\sigma_{\min}$
$\varepsilon_{\text{mach}}$	machine epsilon ($2^{-52} \approx 2.2 \times 10^{-16}$ in double precision)
$\mathbb{P}(A)$, $\mathbb{P}(A \mid B)$	probability of an event; conditional probability
X, Y	random variables (capitals); x, y their realised values
F_X, f_X, p_X	cumulative distribution function, density, probability mass function
$\mathbb{E}[X]$, $\operatorname{Var}(X)$, $\operatorname{Cov}(X, Y)$	expectation, variance, covariance
$\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$	mean vector and covariance matrix
$X \sim F$; $X_1, \ldots, X_n \overset{\text{iid}}{\sim} F$	X has distribution F; independent and identically distributed
$\mathcal{N}(\mu, \sigma^2)$, $\mathcal{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	normal and multivariate normal distributions
$\xrightarrow{p}$, $\xrightarrow{\text{a.s.}}$, $\xrightarrow{d}$	convergence in probability, almost surely, in distribution
$O(\cdot)$, $o(\cdot)$, $O_p(\cdot)$, $o_p(\cdot)$	deterministic and stochastic order symbols
θ, $\boldsymbol{\theta}$, Θ	parameter, parameter vector, parameter space
$\hat{\theta}$, $\hat{\theta}_n$	estimator (a hat always denotes an estimate)
$L(\theta)$, $\ell(\theta)$	likelihood and log-likelihood
$\mathcal{I}(\theta)$	Fisher information
$\operatorname{Bias}(\hat{\theta})$, $\operatorname{MSE}(\hat{\theta})$, $\operatorname{se}(\hat{\theta})$	bias, mean squared error, standard error
∇f, $\nabla^2 f$	gradient and Hessian of f
$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$	Lagrangian of a constrained optimisation problem
$\arg\min$, $\arg\max$	a minimiser / maximiser of a function
$D_{\text{KL}}(p \,\|\, q)$	Kullback–Leibler divergence

Python. Code is set in a typewriter font, for example `np.linalg.solve(A, b)`. Throughout we assume the imports `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt` and `from scipy import stats`. Random numbers are generated with NumPy's `Generator` interface, `rng = np.random.default_rng(803)`, so that every result in the book can be reproduced exactly.

Part I

Linear Algebra for Modelling

Chapter 1

Mathematical Notation, Vectors, Matrices and Computational Precision

Learning Objectives

After working through this chapter you should be able to:

1. read and write the index, summation, product and asymptotic (O, o) notation used throughout data science, econometrics and this book, and translate it into NumPy code;
2. compute and interpret inner products, the ℓ_1, ℓ_2 and ℓ_∞ norms, distances and angles, and prove the Cauchy–Schwarz and triangle inequalities;
3. interpret a matrix product in four equivalent ways, manipulate transposes, inverses, traces, determinants and partitioned matrices, and solve a Leontief input–output model;
4. represent a data set as an $n \times p$ matrix, centre and standardise it with the centring matrix $\mathbf{H}$, and compute sample covariance and correlation matrices;
5. explain how IEEE 754 double-precision arithmetic works, quantify rounding error with the model $\mathrm{fl}(x \circ y) = (x \circ y)(1 + \delta)$, and recognise and cure catastrophic cancellation, overflow, underflow and unstable variance formulae;
6. estimate the computational cost and memory of a matrix computation, choose between loops, vectorised code and sparse storage, and make an analysis reproducible.

Every model in this book, from a regression of household consumption on schooling to a gradient-boosted forecast of inflation, is ultimately a statement about arrays of numbers and operations on them. A data set is a matrix; a parameter is a vector; a likelihood is a sum of logarithms; a forecast is an inner product. Before we can study eigenvalues (Chapter 3), probability models (Chapter 4) or optimisation (Chapter 8) we need a precise and shared *language* for these objects, and an honest understanding of how a computer actually stores and manipulates them.

The second half of that sentence matters as much as the first. A computer does not work with real numbers. It works with a finite set of *floating-point* numbers, and every arithmetic operation can introduce a small relative error. Usually these errors are invisible. Sometimes they are not: a stock-market index that silently lost half its value, a rocket destroyed by an unprotected number conversion, a variance that comes out negative, a likelihood that evaluates to $-\infty$. For an analyst producing official statistics, a tax simulation or a central-bank forecast, such errors are not merely technical; they can change a published number, a transfer to a household or a policy decision.

This chapter therefore has two intertwined goals. Sections 1.1–1.4 establish the notation

and the core vector and matrix algebra that the rest of the book takes for granted, always with an eye on how each object is computed. Sections 1.5 and 1.6 then examine the machine itself: what precision we really have, how errors accumulate, and how much time and memory a computation costs. Throughout, the mathematics is paired with Python code (Harris et al., 2020; Virtanen et al., 2020) and with economic-policy examples: a consumer price index, an input–output model, a household survey and a large administrative register. Good companions for this chapter are Strang (2019), Deisenroth et al. (2020) and, for the numerical material, Higham (2002).

Think About It

A national statistics office publishes year-on-year inflation to one decimal place. The headline number is computed from tens of thousands of price quotations, aggregated with expenditure weights, chained across base years and rounded. At which steps of this pipeline could notation be ambiguous, and at which steps could computer arithmetic change the published digit? Keep the question in mind; we return to it in Sections 1.2 and 1.5.

1.1 Notation and the Language of Data Science

Mathematical notation is a compression device: it lets us write in one line what would take a paragraph in words, and it makes structure visible. In data science the same notation also serves as a *specification* for code. A well-written formula such as $\hat{\mu}_j = n^{-1}\sum_{i=1}^{n} x_{ij}$ tells us which array to reduce along which axis.

1.1.1 Sets, functions and indices

We write $\mathbb{N} = \{1, 2, 3, \dots\}$, $\mathbb{Z}$ for the integers and $\mathbb{R}$ for the real numbers. $\mathbb{R}^n$ is the set of ordered n-tuples of real numbers and $\mathbb{R}^{n\times p}$ the set of real matrices with n rows and p columns. Sets are described either by listing elements or by *set-builder notation*, for example

$$\mathbb{R}_+ = \{x \in \mathbb{R} : \; x \geq 0\}, \qquad \Delta^{p-1} = \left\{\mathbf{w} \in \mathbb{R}^p : \; w_j \geq 0, \; \sum_{j=1}^{p} w_j = 1\right\},$$

the second being the *probability simplex*, the natural home of expenditure shares, portfolio weights and probability vectors. We use $A \subseteq B$, $A \cup B$, $A \cap B$, $A \setminus B$ and $A \times B = \{(a, b) : a \in A, b \in B\}$ in the usual way, and $|A|$ for the number of elements of a finite set.

A *function* $f : A \to B$ assigns to every element of the *domain* A exactly one element of the *codomain* B; its *image* is $f(A) = \{f(a) : a \in A\} \subseteq B$. Composition is $(g \circ f)(a) = g(f(a))$. A function is *injective* if $f(a) = f(a')$ implies $a = a'$, *surjective* if $f(A) = B$, and *bijective* if both, in which case it has an inverse f^{-1}.

Example 1.1 (A tax schedule is a function)**.** A progressive income-tax schedule is a function $T : \mathbb{R}_+ \to \mathbb{R}_+$, typically piecewise linear: with thresholds $0 = \tau_0 < \tau_1 < \cdots < \tau_K$ and marginal rates $r_1 < \cdots < r_K$,

$$T(y) = \sum_{k=1}^{K} r_k \left(\min\{y, \tau_k\} - \tau_{k-1}\right)_+, \qquad (z)_+ = \max\{z, 0\},$$

with $\tau_K = \infty$. T is continuous and non-decreasing but not injective when $r_1 = 0$ (everybody below τ_1 pays zero), so "the income that produced a tax payment of zero" is not well defined. The *average* tax rate $T(y)/y$ and the *marginal* rate $T'(y)$ are different functions; confusing them is a classic policy-communication error.

Indices are labels. In this book $i = 1, \dots, n$ indexes observations (households, firms, districts), $j = 1, \dots, p$ indexes variables or features, $t = 1, \dots, T$ indexes time and k is a generic index for iterations, sectors or components. When a sequence depends on the sample size we write x_n or $\hat{\theta}_n$.

1.1.2 Summation and product notation

For numbers $a_1, \dots, a_n$,

$$\sum_{i=1}^{n} a_i = a_1 + a_2 + \cdots + a_n, \qquad \prod_{i=1}^{n} a_i = a_1 a_2 \cdots a_n .$$

By convention an *empty* sum equals 0 and an empty product equals 1 (the identities of addition and multiplication). Summation is linear,

$$\sum_{i=1}^{n} (c\, a_i + b_i) = c \sum_{i=1}^{n} a_i + \sum_{i=1}^{n} b_i, \tag{1.1}$$

and finite double sums can be interchanged, $\sum_{i=1}^{n}\sum_{j=1}^{p} a_{ij} = \sum_{j=1}^{p}\sum_{i=1}^{n} a_{ij}$. Two further rules are used constantly: a sum of products is *not* a product of sums, $\sum_i a_i b_i \neq (\sum_i a_i)(\sum_i b_i)$ in general, but $\big(\sum_i a_i\big)\big(\sum_j b_j\big) = \sum_i \sum_j a_i b_j$. The logarithm turns products into sums, $\log \prod_i a_i = \sum_i \log a_i$ for $a_i > 0$, which is why likelihoods are maximised on the log scale.

Proposition 1.2 (Deviations from the mean sum to zero)**.** *Let* $\bar{x} = n^{-1}\sum_{i=1}^{n} x_i$. *Then* $\sum_{i=1}^{n}(x_i - \bar{x}) = 0$ *and*

$$\sum_{i=1}^{n} (x_i - \bar{x})^2 = \sum_{i=1}^{n} x_i^2 - n\bar{x}^2 .$$

Proof. By linearity, $\sum_i (x_i - \bar{x}) = \sum_i x_i - n\bar{x} = n\bar{x} - n\bar{x} = 0$. Expanding the square, $\sum_i (x_i - \bar{x})^2 = \sum_i x_i^2 - 2\bar{x}\sum_i x_i + n\bar{x}^2 = \sum_i x_i^2 - 2n\bar{x}^2 + n\bar{x}^2$. □

The second identity is algebraically exact but, as we shall see in Section 1.5, it is a numerically *dangerous* way to compute a variance.

Two indicator symbols complete the toolkit. The *indicator function* of an event or set is $\mathbb{1}\{x \in A\}$, equal to 1 if $x \in A$ and 0 otherwise, so that $\sum_i \mathbb{1}\{y_i < z\}$ counts the households below a poverty line z. The *Kronecker delta* is $\delta_{ij} = 1$ if $i = j$ and 0 otherwise; it is the (i, j) entry of the identity matrix $\mathbf{I}$.

Historical Note: The symbols we sum with

The capital sigma for a sum is usually traced to Leonhard Euler, who used it in his *Institutiones calculi differentialis* (1755). The "big-O" symbol was introduced by the number theorist Paul Bachmann in 1894 and popularised by Edmund Landau, who added little-o in his 1909 treatise on the distribution of primes; for this reason they are often called Bachmann–Landau symbols. Donald Knuth's 1976 note in *SIGACT News* standardised Ω and Θ for the analysis of algorithms (Knuth, 1976). Albert Einstein's 1916 convention of summing over repeated indices survives in NumPy as `np.einsum`.

1.1.3 The data matrix and the conventions of this book

A data set with n observations on p variables is stored as the *data matrix*

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^{\top} \\ \mathbf{x}_2^{\top} \\ \vdots \\ \mathbf{x}_n^{\top} \end{pmatrix} = \big(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(p)}\big) \in \mathbb{R}^{n \times p}. \tag{1.2}$$

Row i, written $\mathbf{x}_i^\top$, is the record of observation i; column j, written $\mathbf{x}^{(j)}$, collects variable j across all observations. The entry x_{ij} is "observation i, variable j". Table 1.1 summarises the conventions used in the rest of the book.

Table 1.1: Notational conventions used throughout the book.

Symbol	Meaning	Example
a, α	scalars (lower-case italic or Greek)	n, λ, σ^2
$\mathbf{x}$, $\boldsymbol{\beta}$	column vectors (bold lower-case)	$\mathbf{x} \in \mathbb{R}^p$
$\mathbf{X}$, $\boldsymbol{\Sigma}$	matrices (bold upper-case)	$\mathbf{X} \in \mathbb{R}^{n \times p}$
$\mathbf{x}^\top$, $\mathbf{A}^\top$	transpose; row vectors are transposed columns	$\mathbf{x}_i^\top$
$\mathbf{1}_n$, $\mathbf{0}$, $\mathbf{I}_n$	vector of ones, zero vector, identity	$\mathbf{1}_n^\top \mathbf{x} = \sum_i x_i$
$\hat{\theta}$, $\hat{\boldsymbol{\beta}}$	estimator (a function of the data)	$\hat{\mu} = \bar{x}$
θ_0	true (population) parameter value	$\mathbb{E}\hat{\theta} = \theta_0$
X, Y	random variables (upper-case italic)	$X \sim \mathcal{N}(0,1)$
$\mathbb{E}$, Var, Cov	expectation, variance, covariance	$\mathrm{Var}(X)$
$\langle \mathbf{x}, \mathbf{y} \rangle$	inner product	$\mathbf{x}^\top \mathbf{y}$
$\lVert \cdot \rVert_q$	ℓ_q norm (default $q = 2$)	$\lVert \mathbf{x} \rVert$
$:=$	"is defined as"	$\bar{x} := n^{-1} \sum_i x_i$
$\mathrm{fl}(\cdot)$	floating-point result	$\mathrm{fl}(0.1) \neq 0.1$

There is an unavoidable clash between the statistical convention (capital letters for random variables) and the linear-algebra convention (capitals for matrices); bold type resolves it. Vectors are always *columns*: $\mathbf{x} \in \mathbb{R}^p$ is a $p \times 1$ array, and a row is written $\mathbf{x}^\top$. In NumPy, by contrast, a one-dimensional array of shape `(p,)` is neither a row nor a column, a point we take up in Section 1.3.

Listing 1.1: From index and sum notation to NumPy ch01/notation_basics.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Index, summation and product notation translated into NumPy.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, p = 6, 3
# simulated household survey: income (GHS thousand), household size, schooling
X = np.column_stack([rng.lognormal(2.5, 0.6, n).round(1),
                     rng.integers(1, 9, n),
                     rng.integers(0, 17, n)]).astype(float)
print("Data matrix X has shape (n, p) =", X.shape)
print(X)
print("x_{31} (row 3, column 1) = X[2, 0] =", X[2, 0])   # Python is 0-based
print("row x_3      =", X[2])
print("column x^(2) =", X[:, 1])

# summation notation
print("sum_i x_i1                =", round(X[:, 0].sum(), 2))
print("column means (xbar_j)     =", X.mean(axis=0).round(3))
print("sum_i sum_j x_ij          =", round(X.sum(), 2))
print("sum_i x_i1 x_i3 (= inner) =", round(X[:, 0] @ X[:, 2], 2))

# product notation: cumulative price level from monthly inflation
pi = np.array([0.021, 0.018, 0.025, 0.030, 0.012, 0.015])
gross = 1 + pi
print("prod_t (1 + pi_t)         =", gross.prod().round(6))
```

```python
print("exp(sum_t log(1 + pi_t))   =", np.exp(np.log(gross).sum()).round(6))
print("geometric-mean monthly rate=", (gross.prod() ** (1 / pi.size) - 1).round(6))

# conventions for empty sums and products
print("empty sum =", np.sum([]), "   empty product =", np.prod([]))

# order of magnitude: a likelihood is a product of many small numbers
probs = rng.uniform(0.05, 0.6, size=2000)
print("prod of 2000 probabilities =", np.prod(probs))
print("sum of their logs          =", round(np.log(probs).sum(), 3))
print("log10 of the product       =", round(np.log10(probs).sum(), 1))
```

Output

```
Data matrix X has shape (n, p) = (6, 3)
[[10.2  3.   4. ]
 [11.3  4.  10. ]
 [10.5  7.   7. ]
 [ 7.9  5.  14. ]
 [14.2  2.  13. ]
 [ 8.3  4.   2. ]]
x_{31} (row 3, column 1) = X[2, 0] = 10.5
row x_3      = [10.5  7.   7. ]
column x^(2) = [3. 4. 7. 5. 2. 4.]
sum_i x_i1                 = 62.4
column means (xbar_j)      = [10.4    4.167  8.333]
sum_i sum_j x_ij           = 137.4
sum_i x_i1 x_i3 (= inner)  = 539.1
prod_t (1 + pi_t)          = 1.127149
exp(sum_t log(1 + pi_t))   = 1.127149
geometric-mean monthly rate= 0.020149
empty sum = 0.0    empty product = 1.0
prod of 2000 probabilities = 0.0
sum of their logs          = -2575.451
log10 of the product       = -1118.5
```

The listing shows three things worth remembering. First, mathematical indices start at 1 while Python's start at 0, so x_{31} is `X[2, 0]`. Second, every sum in a formula corresponds to a reduction along an axis: `axis=0` sums over observations i (down the rows), `axis=1` over variables j. Third, the product of 2000 probabilities is *exactly zero* in the computer, although mathematically it is about 10^{-1118}; the sum of logarithms is perfectly representable.

△ Caution: Zero-based indexing and off-by-one errors

A formula $\sum_{t=2}^{T}(y_t - y_{t-1})$ has $T-1$ terms and in Python becomes `np.sum(y[1:] - y[:-1])`. Translating index ranges by hand is the commonest source of silent errors in econometric code (lags shifted by one period, the first survey round dropped). Use slicing and vectorised differences (`np.diff`) rather than explicit index arithmetic, and check array lengths with `assert`.

1.1.4 Asymptotic notation

Data science constantly asks how a quantity behaves as something grows (sample size n, dimension p) or shrinks (a step size h, a bandwidth).

Definition 1.3 (Bachmann–Landau symbols)**.** Let f, g be real functions with $g(n) > 0$ for large n. As $n \to \infty$ we write

- $f(n) = O(g(n))$ if there are constants $C > 0$ and n_0 such that $|f(n)| \le C\, g(n)$ for all $n \ge n_0$ ("f grows no faster than g");
- $f(n) = o(g(n))$ if $f(n)/g(n) \to 0$ ("f is negligible relative to g");
- $f(n) = \Theta(g(n))$ if $f = O(g)$ and $g = O(f)$ (same order);
- $f(n) = \Omega(g(n))$ if $g = O(f)$; and $f \sim g$ if $f(n)/g(n) \to 1$.

The same symbols are used as $h \to 0$, with "for all $n \ge n_0$" replaced by "for all $0 < |h| \le h_0$".

Example 1.4. (i) $3n^2 + 10n \log n + 7 = \Theta(n^2)$ and $= o(n^{2+\epsilon})$ for every $\epsilon > 0$. (ii) $\log n = o(n^{\epsilon})$ for every $\epsilon > 0$: logarithms grow slower than any power. (iii) By Taylor's theorem, $e^h = 1 + h + O(h^2)$ and $\log(1+h) = h - \tfrac{1}{2}h^2 + O(h^3)$ as $h \to 0$; this is why a small growth rate g satisfies $\log(1+g) \approx g$. (iv) The sample mean of n i.i.d. variables with finite variance has standard error $\sigma/\sqrt{n} = \Theta(n^{-1/2})$: halving a survey's margin of error requires four times the sample.

The rules $O(f) + O(g) = O(\max\{f, g\})$, $O(f)\,O(g) = O(fg)$ and $c\,O(f) = O(f)$ follow directly from the definition. Note that "=" here is a one-way statement: $n = O(n^2)$ is true, but $O(n^2) = n$ is meaningless.

Graduate Perspective: Stochastic order symbols

In asymptotic statistics we need random analogues: $X_n = O_p(a_n)$ means that X_n/a_n is *bounded in probability* (for every $\epsilon > 0$ there is M with $\mathbb{P}(|X_n/a_n| > M) < \epsilon$ for all large n), and $X_n = o_p(a_n)$ means $X_n/a_n \xrightarrow{p} 0$. The central limit theorem says $\bar{X}_n - \mu = O_p(n^{-1/2})$, and a consistent estimator satisfies $\hat{\theta}_n - \theta_0 = o_p(1)$. These symbols are developed in Chapter 7; see van der Vaart (1998, Chapter 2).

1.1.5 Order-of-magnitude reasoning

Before running a computation it pays to estimate its size. Write numbers in scientific notation and add exponents: a survey of $n = 10^4$ households with $p = 10^2$ variables has 10^6 entries, 8×10^6 bytes (8 MB) in double precision. A national register with $n = 3 \times 10^7$ persons is 3×10^9 entries, 24 GB, which no longer fits in a laptop's memory. A modern core performs on the order of 10^9–10^{10} floating-point operations per second, so an $O(n^2)$ algorithm on $n = 10^7$ records needs about 10^{14} operations: a day of computing rather than a second. Figure 1.1 and the next listing make the contrast concrete.

Output of `ch01/bigO_growth.py`: run-time estimates

```
Estimated run time at 1e9 operations per second
       n      n log n          n^2          n^3
   1e+03     1.0e-05s     1.0e-03s     1.0e+00s
   1e+05     1.7e-03s     1.0e+01s     1.0e+06s
   1e+07     2.3e-01s     1.0e+05s     1.0e+12s
one year is about 3.16e+07 seconds
```

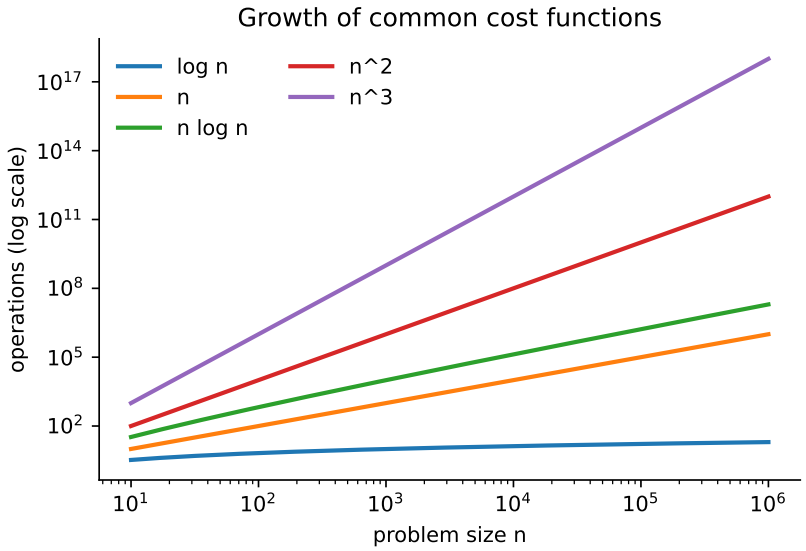


Figure 1.1: Growth of $\log n$, n, $n \log n$, n^2 and n^3 on log–log axes. A power n^k appears as a straight line of slope k, which is how empirical run times are diagnosed in Section 1.6.

⋆ Key Concept: Notation is a specification

Every symbol in a formula should map to a definite object in code: a scalar, a vector of shape $(p,)$, a matrix of shape (n, p), a reduction along a named axis. Indices (i for observations, j for variables, t for time) tell you which axis. Asymptotic symbols O, o, Θ tell you how cost or error scales, and order-of-magnitude arithmetic tells you *before* you run the code whether it will finish.

Exercises

1.1.1 Theory Prove that $\sum_{i=1}^{n}\sum_{j=1}^{n}(x_i - x_j)^2 = 2n\sum_{i=1}^{n}(x_i - \bar{x})^2$. Deduce a formula for the sample variance in terms of all pairwise differences.

1.1.2 Theory Show that $n^k = o(c^n)$ for every fixed k and every $c > 1$, and that $\log(n!) = \Theta(n \log n)$. (Hint: bound $\log n!$ above by $n \log n$ and below by $\sum_{i>n/2} \log i$.)

1.1.3 Theory Let $f(h) = (\sqrt{1+h} - 1)/h$. Show that $f(h) = \tfrac{1}{2} - \tfrac{1}{8}h + O(h^2)$ as $h \to 0$. Is $f(h) - \tfrac{1}{2} = o(1)$? Is it $O(h)$? Is it $o(h)$?

1.1.4 Python Using the tax schedule of Example 1.1 with thresholds $(0, 5, 20, 50)$ thousand and marginal rates $(0, 0.1, 0.2, 0.3)$, write a vectorised function `tax(y)` that accepts a NumPy array of incomes. Plot the average and marginal tax rates for $y \in [0, 100]$ and verify numerically that the average rate never exceeds the marginal rate.

1.1.5 Python Write $\sum_{i=1}^{n}\sum_{j=1}^{p} x_{ij} w_j$ and $\sum_i \sum_k x_{ik} x_{ij}$ with `np.einsum`, and check them against `X @ w` and `X.T @ X` for a random 50×4 matrix.

1.1.6 Policy A poverty headcount is $\hat{P}_0 = n^{-1}\sum_{i=1}^{n} \mathbb{1}\{y_i < z\}$ and the poverty gap index is $\hat{P}_1 = n^{-1}\sum_i (1 - y_i/z)\mathbb{1}\{y_i < z\}$. Write both in vector notation using $\mathbf{1}_n$, and explain why $\hat{P}_1$ but not $\hat{P}_0$ responds to a small transfer to a poor household that leaves it below the line.

1.1.7 ★ Graduate The standard error of a survey mean is $\Theta(n^{-1/2})$ while the cost of fieldwork is $\Theta(n)$. Formulate the choice of n that minimises cost $+ \lambda \times$ (standard error)^2 and show that

the optimal n is $\Theta(\lambda^{1/2})$. Discuss what this implies for doubling the precision of a national labour-force survey.

1.2 Vectors

A vector is simultaneously an *array of numbers* (the record of one household, the coefficients of a model), a *point* in space and an *arrow* that can be added and scaled. Data science uses all three views: arrays for computation, points for distances and clustering, arrows for directions of change such as gradients.

1.2.1 The vector space $\mathbb{R}^n$

For $\mathbf{x} = (x_1, \dots, x_n)^\top$ and $\mathbf{y} = (y_1, \dots, y_n)^\top$ in $\mathbb{R}^n$ and $c \in \mathbb{R}$ we define $\mathbf{x}+\mathbf{y} = (x_1+y_1, \dots, x_n+y_n)^\top$ and $c\mathbf{x} = (cx_1, \dots, cx_n)^\top$. With these operations $\mathbb{R}^n$ is a *vector space*: addition is associative and commutative, there is a zero vector $\mathbf{0}$ and additive inverses $-\mathbf{x}$, and scalar multiplication distributes over both vector and scalar addition with $1\mathbf{x} = \mathbf{x}$.

Definition 1.5 (Span, independence, basis)**.** A *linear combination* of $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$ is $\sum_{l=1}^{k} c_l \mathbf{v}_l$. Their *span* $\operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ is the set of all linear combinations, a *subspace* of $\mathbb{R}^n$ (a subset closed under addition and scaling). The vectors are *linearly independent* if $\sum_l c_l \mathbf{v}_l = \mathbf{0}$ forces $c_1 = \cdots = c_k = 0$. A *basis* of a subspace V is an independent set spanning V; all bases have the same number of elements, the *dimension* $\dim V$.

The standard basis of $\mathbb{R}^n$ is $\mathbf{e}_1, \dots, \mathbf{e}_n$, where $\mathbf{e}_j$ has a 1 in position j and zeros elsewhere, so $\mathbf{x} = \sum_j x_j \mathbf{e}_j$. In a regression, the columns of $\mathbf{X}$ are linearly dependent exactly when there is perfect multicollinearity (the "dummy variable trap"); the fitted values always lie in $\operatorname{span}\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(p)}\}$. Subspaces, rank and projections are developed fully in Chapter 2.

1.2.2 Inner products

Definition 1.6 (Inner product)**.** An *inner product* on $\mathbb{R}^n$ is a map $\langle \cdot, \cdot \rangle : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ that is symmetric ($\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$), linear in each argument ($\langle a\mathbf{x} + b\mathbf{z}, \mathbf{y} \rangle = a \langle \mathbf{x}, \mathbf{y} \rangle + b \langle \mathbf{z}, \mathbf{y} \rangle$) and positive definite ($\langle \mathbf{x}, \mathbf{x} \rangle > 0$ for $\mathbf{x} \neq \mathbf{0}$). The *standard* (Euclidean) inner product is

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{n} x_i y_i .$$

More generally, for a symmetric positive definite matrix $\mathbf{W}$, $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{W}} = \mathbf{x}^\top \mathbf{W} \mathbf{y}$ is an inner product; with $\mathbf{W} = \operatorname{diag}(w_1, \dots, w_n)$ it is the weighted sum $\sum_i w_i x_i y_i$ used with survey weights, and with $\mathbf{W} = \boldsymbol{\Sigma}^{-1}$ it underlies the Mahalanobis distance.

Many familiar statistics are inner products: a weighted mean $\mathbf{w}^\top \mathbf{x}$ with $\mathbf{w} \in \Delta^{n-1}$, total expenditure $\mathbf{p}^\top \mathbf{q}$ (prices times quantities), a linear prediction $\hat{y} = \mathbf{x}^\top \hat{\boldsymbol{\beta}}$, and the sum $\mathbf{1}_n^\top \mathbf{x} = \sum_i x_i$.

1.2.3 Norms, distances and unit balls

Definition 1.7 (Norm)**.** A *norm* is a function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ such that for all $\mathbf{x}, \mathbf{y}$ and c: (i) $\|\mathbf{x}\| \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$; (ii) $\|c\mathbf{x}\| = |c| \, \|\mathbf{x}\|$; (iii) $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality).

For $q \geq 1$ the ℓ_q norms are

$$\|\mathbf{x}\|_q = \Big(\sum_{i=1}^{n} |x_i|^q \Big)^{1/q}, \qquad \|\mathbf{x}\|_1 = \sum_i |x_i|\,, \quad \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|\,.$$

A norm induces a *distance* $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$.

The ℓ_2 norm is Euclidean length; ℓ_1 is the "taxicab" or Manhattan distance and the penalty of the lasso; ℓ_∞ is the largest absolute deviation, natural for tolerances ("no price is off by more than one cedi"). Figure 1.2 shows the unit balls $\{\mathbf{x} : \|\mathbf{x}\|_q \leq 1\}$: the diamond of ℓ_1 has corners on the axes, which is the geometric reason the lasso produces sparse solutions.

The script `ch01/unit_balls.py` draws the figure by rescaling unit directions $(\cos t, \sin t)$ to norm one, and checks the enclosed areas (2, π and 4).

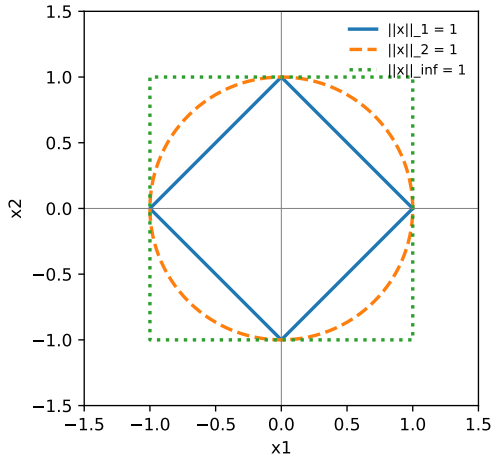


Figure 1.2: Unit balls of the ℓ_1 (diamond), ℓ_2 (disc) and ℓ_∞ (square) norms in $\mathbb{R}^2$. They are nested, $B_1 \subset B_2 \subset B_\infty$, reflecting $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$.

Proposition 1.8 (Equivalence of norms on $\mathbb{R}^n$)**.** *For every* $\mathbf{x} \in \mathbb{R}^n$,

$$\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq \sqrt{n}\, \|\mathbf{x}\|_2 \leq n\, \|\mathbf{x}\|_\infty\,.$$

Proof. Let $m = \max_i |x_i|$. Then $m^2 \leq \sum_i x_i^2$, giving the first inequality. Next, $\|\mathbf{x}\|_1^2 = \sum_i \sum_j |x_i|\,|x_j| \geq \sum_i x_i^2$. The third inequality is Cauchy–Schwarz (Theorem 1.9 below) applied to $(|x_1|, \ldots, |x_n|)$ and $\mathbf{1}_n$: $\sum_i |x_i| \cdot 1 \leq \|\mathbf{x}\|_2\, \|\mathbf{1}_n\|_2 = \sqrt{n}\, \|\mathbf{x}\|_2$. Finally $\|\mathbf{x}\|_2^2 \leq n m^2$. □

The constants depend on n. In high dimensions they matter: a vector with all entries equal to 1 has $\|\mathbf{x}\|_\infty = 1$ but $\|\mathbf{x}\|_1 = n$.

1.2.4 The Cauchy–Schwarz inequality, angles and similarity

Theorem 1.9 (Cauchy–Schwarz inequality)**.** *For any inner product and induced norm* $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$,

$$|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\|\, \|\mathbf{y}\|\,,$$

with equality if and only if $\mathbf{x}$ *and* $\mathbf{y}$ *are linearly dependent.*

Proof. If $\mathbf{y} = \mathbf{0}$ both sides are zero. Otherwise, for every real t,

$$0 \leq \|\mathbf{x} - t\mathbf{y}\|^2 = \|\mathbf{x}\|^2 - 2t\, \langle \mathbf{x}, \mathbf{y} \rangle + t^2\, \|\mathbf{y}\|^2\,.$$

The right-hand side is a quadratic in t that is never negative, so its discriminant is non-positive: $4\langle \mathbf{x}, \mathbf{y}\rangle^2 - 4\|\mathbf{x}\|^2\|\mathbf{y}\|^2 \le 0$, which is the inequality. (Equivalently, put $t = \langle \mathbf{x}, \mathbf{y}\rangle / \|\mathbf{y}\|^2$, the minimiser.) Equality holds iff the quadratic has a real root t^*, i.e. $\|\mathbf{x} - t^*\mathbf{y}\| = 0$, i.e. $\mathbf{x} = t^*\mathbf{y}$. □

Corollary 1.10 (Triangle inequality)**.** $\|\mathbf{x} + \mathbf{y}\| \le \|\mathbf{x}\| + \|\mathbf{y}\|$ *for the norm induced by any inner product.*

Proof. $\|\mathbf{x}+\mathbf{y}\|^2 = \|\mathbf{x}\|^2 + 2\langle \mathbf{x}, \mathbf{y}\rangle + \|\mathbf{y}\|^2 \le \|\mathbf{x}\|^2 + 2\|\mathbf{x}\|\|\mathbf{y}\| + \|\mathbf{y}\|^2 = (\|\mathbf{x}\| + \|\mathbf{y}\|)^2$, using Cauchy–Schwarz in the middle step. □

Cauchy–Schwarz guarantees that the ratio below lies in $[-1, 1]$, so it is legitimate to define the *angle* $\theta \in [0, \pi]$ between non-zero vectors by

$$\cos\theta = \frac{\langle \mathbf{x}, \mathbf{y}\rangle}{\|\mathbf{x}\|\,\|\mathbf{y}\|}. \tag{1.3}$$

Vectors are *orthogonal*, $\mathbf{x} \perp \mathbf{y}$, when $\langle \mathbf{x}, \mathbf{y}\rangle = 0$, and then $\|\mathbf{x}+\mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2$ (Pythagoras). In text mining and recommendation systems (1.3) is called *cosine similarity*; it compares directions and ignores lengths, so it compares the *composition* of two budgets rather than their size. If $\mathbf{x}$ and $\mathbf{y}$ are centred (mean zero), (1.3) is exactly the Pearson correlation coefficient, a fact we use in Section 1.4; Cauchy–Schwarz is then the statement $|r| \le 1$.

Listing 1.2: Norms, angles and Cauchy–Schwarz in practice ch01/vectors_norms.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Inner products, norms, distances, angles and the Cauchy-Schwarz inequality.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
x = np.array([3.0, -4.0, 1.0, 2.0])
y = np.array([1.0, 2.0, -2.0, 5.0])
print("x =", x, "  y =", y)
print("<x, y> =", x @ y)
for name, o in [("1", 1), ("2", 2), ("inf", np.inf)]:
    print(f"||x||_{name:3s} = {np.linalg.norm(x, o):8.4f}"
          f"    d_{name:3s}(x, y) = {np.linalg.norm(x - y, o):8.4f}")
cos = x @ y / (np.linalg.norm(x) * np.linalg.norm(y))
print(f"cosine = {cos:.4f}, angle = {np.degrees(np.arccos(cos)):.2f} degrees")

# Cauchy-Schwarz and triangle inequality on 100,000 random pairs in R^10
U = rng.standard_normal((100_000, 10))
V = rng.standard_normal((100_000, 10))
nu, nv = np.linalg.norm(U, axis=1), np.linalg.norm(V, axis=1)
ratio = np.abs(np.sum(U * V, axis=1)) / (nu * nv)
tri = np.linalg.norm(U + V, axis=1) / (nu + nv)
print(f"max |<u,v>|/(||u|| ||v||) = {ratio.max():.6f}  (must be <= 1)")
print(f"max ||u+v||/(||u||+||v||) = {tri.max():.6f}  (must be <= 1)")
print(f"ratio when v = -3u        = {abs(U[0] @ (-3*U[0])) / (3*nu[0]**2):.6f}")

# norm equivalence: ||x||_inf <= ||x||_2 <= ||x||_1 <= sqrt(n) ||x||_2
n1, n2, ni = (np.linalg.norm(U, o, axis=1) for o in (1, 2, np.inf))
ok = np.all((ni <= n2) & (n2 <= n1) & (n1 <= np.sqrt(10) * n2 * (1 + 1e-12)))
print("norm-equivalence chain holds for all 100,000 vectors:", bool(ok))
print(f"largest ||u||_1/||u||_2 seen = {np.max(n1 / n2):.4f}"
      f"  (bound sqrt(10) = {np.sqrt(10):.4f})")

# cosine similarity of (simulated) household budget-share vectors
```

```python
# columns: food, housing, transport, education, other
shares = np.array([[0.55, 0.15, 0.10, 0.05, 0.15],    # rural household
                   [0.48, 0.20, 0.12, 0.06, 0.14],    # peri-urban household
                   [0.25, 0.30, 0.15, 0.15, 0.15]])   # urban household
Sn = shares / np.linalg.norm(shares, axis=1, keepdims=True)
print("cosine similarity of budget shares (rural, peri-urban, urban):")
print(np.round(Sn @ Sn.T, 4))
```

Output

```
x = [ 3. -4.  1.  2.]   y = [ 1.  2. -2.  5.]
<x, y> = 3.0
||x||_1   =  10.0000    d_1  (x, y) =  14.0000
||x||_2   =   5.4772    d_2  (x, y) =   7.6158
||x||_inf =   4.0000    d_inf(x, y) =   6.0000
cosine = 0.0939, angle = 84.61 degrees
max |<u,v>|/(||u|| ||v||) = 0.951920  (must be <= 1)
max ||u+v||/(||u||+||v||) = 0.988169  (must be <= 1)
ratio when v = -3u        = 1.000000
norm-equivalence chain holds for all 100,000 vectors: True
largest ||u||_1/||u||_2 seen = 3.1229  (bound sqrt(10) = 3.1623)
cosine similarity of budget shares (rural, peri-urban, urban):
[[1.     0.991  0.8084]
 [0.991  1.     0.8759]
 [0.8084 0.8759 1.    ]]
```

Historical Note: Grassmann, Cauchy, Bunyakovsky and Schwarz

The idea of vectors as objects that can be added and scaled in any number of dimensions is due largely to Hermann Grassmann, a Stettin schoolteacher whose *Die lineale Ausdehnungslehre* (1844) was so far ahead of its time that it was almost ignored (Grassmann, 1844). The inequality of Theorem 1.9 appeared for finite sums in Augustin-Louis Cauchy's *Cours d'analyse* (1821); Viktor Bunyakovsky proved the integral version in 1859 and Hermann Schwarz rediscovered it in the 1880s, which is why Russian texts often call it the Cauchy–Bunyakovsky–Schwarz inequality.

1.2.5 Example: a consumer price index is an inner product

Let $\mathbf{p}_0, \mathbf{p}_1 \in \mathbb{R}^m_+$ be the prices of m items in a base period and the current period, and $\mathbf{q}_0$ the base-period quantities. The *Laspeyres* index is the cost of the base basket at current prices relative to its base cost,

$$L = 100\,\frac{\mathbf{p}_1^{\top}\mathbf{q}_0}{\mathbf{p}_0^{\top}\mathbf{q}_0} = 100\sum_{k=1}^{m} w_k\,\frac{p_{1k}}{p_{0k}} = 100\,\langle \mathbf{w}, \mathbf{r}\rangle, \qquad w_k = \frac{p_{0k}q_{0k}}{\mathbf{p}_0^{\top}\mathbf{q}_0}, \tag{1.4}$$

where $\mathbf{r} = (p_{11}/p_{01}, \dots, p_{1m}/p_{0m})^{\top}$ is the vector of *price relatives* and $\mathbf{w} \in \Delta^{m-1}$ the vector of base-period expenditure shares. The second equality is a one-line exercise in summation notation. The *Paasche* index uses current quantities, $P = 100\,\mathbf{p}_1^{\top}\mathbf{q}_1/\mathbf{p}_0^{\top}\mathbf{q}_1$, and the *Fisher ideal* index is the geometric mean $\sqrt{LP}$ (ILO et al., 2004). Because $L - 100 = \sum_k 100\,w_k(r_k - 1)$, the inflation rate decomposes *exactly* into item contributions, the table that statistics offices publish alongside the headline figure.

Listing 1.3: A Laspeyres CPI from a simulated basket ch01/cpi_inner_product.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# A Laspeyres consumer price index as an inner product (simulated basket).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

items = ["food", "alcohol/tobacco", "clothing", "housing/utilities",
         "furnishings", "health", "transport", "communication",
         "recreation", "education"]
# simulated base-period quantities and prices (illustrative, not official)
q0 = np.array([120, 6, 14, 30, 8, 5, 40, 12, 7, 4], dtype=float)
p0 = np.array([5.0, 12.0, 30.0, 25.0, 40.0, 60.0, 8.0, 15.0, 20.0, 150.0])
p1 = p0 * np.array([1.24, 1.10, 1.08, 1.30, 1.12, 1.15, 1.35, 1.05, 1.09, 1.20])
q1 = q0 * np.array([0.95, 1.00, 0.97, 0.96, 0.99, 1.01, 0.90, 1.05, 0.98, 1.00])

w = p0 * q0 / (p0 @ q0)              # base-period expenditure weights
r = p1 / p0                          # price relatives
L = 100 * (w @ r)                    # Laspeyres as a weighted mean of relatives
L2 = 100 * (p1 @ q0) / (p0 @ q0)     # same index as a ratio of inner products
P = 100 * (p1 @ q1) / (p0 @ q1)      # Paasche
F = np.sqrt(L * P)                   # Fisher ideal
print(f"sum of weights = {w.sum():.6f}")
print(f"Laspeyres  100 <w, r>             = {L:.3f}")
print(f"Laspeyres  100 <p1,q0>/<p0,q0>    = {L2:.3f}")
print(f"Paasche    100 <p1,q1>/<p0,q1>    = {P:.3f}")
print(f"Fisher     sqrt(L * P)            = {F:.3f}")
contrib = 100 * w * (r - 1)          # percentage-point contributions
print(f"\n{'item':18s} {'weight':>7s} {'relative':>9s} {'contrib(pp)':>12s}")
for k in np.argsort(-contrib):
    print(f"{items[k]:18s} {w[k]:7.3f} {r[k]:9.3f} {contrib[k]:12.3f}")
print(f"{'total inflation':18s} {'':7s} {'':9s} {contrib.sum():12.3f}")
```

Output

```
sum of weights = 1.000000
Laspeyres  100 <w, r>             = 120.173
Laspeyres  100 <p1,q0>/<p0,q0>    = 120.173
Paasche    100 <p1,q1>/<p0,q1>    = 119.944
Fisher     sqrt(L * P)            = 120.058

item               weight  relative  contrib(pp)
housing/utilities   0.203     1.300        6.078
food                0.162     1.240        3.890
education           0.162     1.200        3.241
transport           0.086     1.350        3.025
health              0.081     1.150        1.216
furnishings         0.086     1.120        1.037
clothing            0.113     1.080        0.908
recreation          0.038     1.090        0.340
communication       0.049     1.050        0.243
alcohol/tobacco     0.019     1.100        0.194
total inflation                           20.173
```

Policy Lens: Computing a consumer price index

In the simulated basket the headline Laspeyres inflation is 20.2%, of which housing and utilities contribute about 6.1 percentage points despite a weight of only 0.20, because its relative price rose by 30%. The Paasche index is lower (119.9): households substituted away from the items whose prices rose most (transport quantities fell by 10%), and the fixed-basket Laspeyres index ignores this *substitution bias*. Real CPIs are computed in two stages: elementary aggregates (geometric or arithmetic means of individual price quotes) and then an inner product with expenditure weights from a household budget survey, often chained across base years. Each stage is a linear-algebra operation on large arrays, and the choice of weights (whose expenditure pattern?) is a policy decision with distributional consequences: a plutocratic CPI weights households by their spending, so it reflects the basket of richer households (ILO et al., 2004).

⋆ Key Concept: Inner products, norms and angles

The inner product $\mathbf{x}^\top \mathbf{y}$ is the workhorse of data science: weighted means, totals, predictions and price indices are all inner products. Norms measure size (ℓ_2 length, ℓ_1 total absolute size, ℓ_∞ worst case) and all are equivalent on $\mathbb{R}^n$ up to constants depending on n. The Cauchy–Schwarz inequality $|\mathbf{x}^\top \mathbf{y}| \le \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2$ makes angles, cosine similarity and correlation well defined, and implies the triangle inequality.

Exercises

1.2.1 Theory Prove that $\|\cdot\|_1$ and $\|\cdot\|_\infty$ satisfy the three norm axioms. Then show that neither is induced by an inner product, by checking that the *parallelogram law* $\|\mathbf{x}+\mathbf{y}\|^2 + \|\mathbf{x}-\mathbf{y}\|^2 = 2\|\mathbf{x}\|^2 + 2\|\mathbf{y}\|^2$ fails for $\mathbf{x} = \mathbf{e}_1$, $\mathbf{y} = \mathbf{e}_2$.

1.2.2 Theory Show that $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{W}} = \mathbf{x}^\top \mathbf{W} \mathbf{y}$ is an inner product if $\mathbf{W}$ is symmetric positive definite. Write down the corresponding Cauchy–Schwarz inequality and interpret it for $\mathbf{W} = \operatorname{diag}(\mathbf{w})$ with survey weights $\mathbf{w}$.

1.2.3 Theory (Hölder and the dual norm.) Show that $|\mathbf{x}^\top \mathbf{y}| \le \|\mathbf{x}\|_1 \, \|\mathbf{y}\|_\infty$, and that for fixed $\mathbf{y}$ the maximum of $\mathbf{x}^\top \mathbf{y}$ over $\|\mathbf{x}\|_1 \le 1$ equals $\|\mathbf{y}\|_\infty$.

1.2.4 Python Generate 1000 points uniformly in $[-1, 1]^p$ and compute the ratio of the largest to the smallest pairwise Euclidean distance for $p = 2, 10, 100, 1000$. Plot the ratio against p and explain the "concentration of distances" you observe. What does it imply for nearest-neighbour methods in high dimensions?

1.2.5 Python Extend `cpi_inner_product.py` to compute a chained Laspeyres index over three periods (weights updated each period) and compare it with the fixed-base index. Also compute the *Jevons* elementary index $\prod_k r_k^{1/m}$ via logarithms.

1.2.6 Policy Using the simulated basket, compute the Laspeyres index separately for a "poor" household whose expenditure shares are 0.45 on food and 0.10 on education, with the remaining 0.45 spread proportionally over the other items. By how many points does its inflation differ from the headline? Discuss implications for indexing social transfers to the CPI.

1.2.7 ★ Graduate Let L and P be the Laspeyres and Paasche indices. Show that L/P can be written as a ratio involving the weighted covariance between price relatives and quantity relatives q_{1k}/q_{0k} under the weights $\mathbf{w}$ (Bortkiewicz's relation), and deduce the sign condition under which $L > P$.

1.3 Matrices

A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is a rectangular array (a_{ij}), but its real importance is as a *linear map* $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ from $\mathbb{R}^n$ to $\mathbb{R}^m$. Every linear map between finite-dimensional spaces is of this form, so matrices are the language of linear models, of linear transformations of data and of linear economic systems.

1.3.1 Matrix products and their four interpretations

For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times r}$ the product $\mathbf{C} = \mathbf{A}\mathbf{B} \in \mathbb{R}^{m\times r}$ has entries $c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$. The inner dimensions must agree. Write $\mathbf{a}_k$ for the kth column of $\mathbf{A}$, $\tilde{\mathbf{a}}_i^\top$ for its ith row, and similarly $\mathbf{b}_j$, $\tilde{\mathbf{b}}_k^\top$ for $\mathbf{B}$.

Proposition 1.11 (Four views of a matrix product)**.** *(i)* Entry view*:* $c_{ij} = \tilde{\mathbf{a}}_i^\top \mathbf{b}_j$*, an inner product of a row of* $\mathbf{A}$ *and a column of* $\mathbf{B}$*.*

(ii) Column view*: the* j*th column of* $\mathbf{C}$ *is* $\mathbf{A}\mathbf{b}_j = \sum_k b_{kj}\mathbf{a}_k$*, a linear combination of the columns of* $\mathbf{A}$*.*

(iii) Row view*: the* i*th row of* $\mathbf{C}$ *is* $\tilde{\mathbf{a}}_i^\top \mathbf{B} = \sum_k a_{ik}\tilde{\mathbf{b}}_k^\top$*, a combination of the rows of* $\mathbf{B}$*.*

(iv) Outer-product view*:* $\mathbf{C} = \sum_{k=1}^{n} \mathbf{a}_k \tilde{\mathbf{b}}_k^\top$*, a sum of* n *rank-one matrices.*

Proof. All four are regroupings of the double sum defining c_{ij}. For (iv), the (i, j) entry of the outer product $\mathbf{a}_k\tilde{\mathbf{b}}_k^\top$ is $a_{ik}b_{kj}$; summing over k gives c_{ij}. Views (ii) and (iii) fix j or i respectively and collect terms. □

Each view answers a different question. The column view shows that $\operatorname{col}(\mathbf{AB}) \subseteq \operatorname{col}(\mathbf{A})$, so $\operatorname{rank}(\mathbf{AB}) \le \min\{\operatorname{rank}\mathbf{A}, \operatorname{rank}\mathbf{B}\}$. The outer-product view underlies low-rank approximation and the singular value decomposition (Chapter 3), and it explains why $\mathbf{X}^\top\mathbf{X} = \sum_{i=1}^{n} \mathbf{x}_i\mathbf{x}_i^\top$ can be accumulated one observation at a time. Matrix multiplication is associative and distributes over addition, but it is *not commutative*: in general $\mathbf{AB} \neq \mathbf{BA}$, even when both are defined.

The script `ch01/matrix_products.py` computes a 3×4 product by all four views (a list comprehension of inner products, `np.column_stack` of matrix–vector products, `np.vstack` of row products and a sum of `np.outer` terms) and checks the transpose, inverse, orthogonality and idempotency rules discussed below:

Output of `ch01/matrix_products.py`

```
C = AB =
 [[-2.  2. -4.  2.]
 [-3.  3. -6.  3.]
 [ 1.  0.  6. -1.]]
all four views agree: True
rank of C (a sum of 2 outer products) = 2
(MN)^T = N^T M^T : True
MN = NM ?        : False
(MN)^-1 = N^-1 M^-1: True
Q^T Q = I        : True   ||Qv|| - ||v|| = -8.9e-16
det(Q)           = -1.0
P symmetric      : True   P^2 = P : True
trace(P) = 3.0  = rank(X) = 3
eigenvalues of P : [0. 0. 0. 1. 1. 1.]
```

1.3.2 Special matrices

Definition 1.12. A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is

- *diagonal* if $a_{ij} = 0$ for $i \neq j$, written $\operatorname{diag}(a_{11}, \dots, a_{nn})$;
- *symmetric* if $\mathbf{A}^\top = \mathbf{A}$;
- *orthogonal* if $\mathbf{A}^\top\mathbf{A} = \mathbf{A}\mathbf{A}^\top = \mathbf{I}$, i.e. $\mathbf{A}^{-1} = \mathbf{A}^\top$;
- *upper (lower) triangular* if $a_{ij} = 0$ for $i > j$ ($i < j$);
- *idempotent* if $\mathbf{A}^2 = \mathbf{A}$; an idempotent symmetric matrix is an *orthogonal projection*;
- *positive (semi)definite* if it is symmetric and $\mathbf{x}^\top\mathbf{A}\mathbf{x} > 0$ (≥ 0) for all $\mathbf{x} \neq \mathbf{0}$.

Multiplying by a diagonal matrix rescales: $\operatorname{diag}(\mathbf{d})\mathbf{A}$ scales the rows of $\mathbf{A}$, $\mathbf{A}\operatorname{diag}(\mathbf{d})$ its columns. Covariance matrices are symmetric positive semidefinite. Triangular systems $\mathbf{T}\mathbf{x} = \mathbf{b}$ are solved by forward or back substitution in $O(n^2)$ operations, which is why factorisations into triangular pieces (**LU**, Cholesky, **QR**) are the basis of numerical linear algebra (Golub and Van Loan, 2013).

Proposition 1.13. *(a) An orthogonal* $\mathbf{Q}$ *preserves inner products and lengths:* $\langle \mathbf{Q}\mathbf{x}, \mathbf{Q}\mathbf{y}\rangle = \langle \mathbf{x}, \mathbf{y}\rangle$*, and* $\det \mathbf{Q} = \pm 1$*. (b) The eigenvalues of an idempotent matrix are* 0 *or* 1*, and if* $\mathbf{P}$ *is an orthogonal projection then* $\mathbf{I} - \mathbf{P}$ *is one too.*

Proof. (a) $(\mathbf{Q}\mathbf{x})^\top(\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top\mathbf{Q}^\top\mathbf{Q}\mathbf{y} = \mathbf{x}^\top\mathbf{y}$; and $1 = \det(\mathbf{Q}^\top\mathbf{Q}) = (\det \mathbf{Q})^2$. (b) If $\mathbf{P}\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$ then $\lambda\mathbf{v} = \mathbf{P}\mathbf{v} = \mathbf{P}^2\mathbf{v} = \lambda^2\mathbf{v}$, so $\lambda \in \{0, 1\}$. Also $(\mathbf{I}-\mathbf{P})^2 = \mathbf{I} - 2\mathbf{P} + \mathbf{P}^2 = \mathbf{I} - \mathbf{P}$ and $(\mathbf{I}-\mathbf{P})^\top = \mathbf{I} - \mathbf{P}$. □

The "hat matrix" $\mathbf{P} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ of least squares is the leading example: the listing confirms that it is symmetric and idempotent, with eigenvalues 0 and 1 and trace equal to $\operatorname{rank}\mathbf{X} = 3$.

1.3.3 Transpose and inverse rules

The transpose reverses products, $(\mathbf{A}\mathbf{B})^\top = \mathbf{B}^\top\mathbf{A}^\top$, since the (i, j) entry of both sides is $\sum_k a_{jk}b_{ki}$. A square $\mathbf{A}$ is *invertible* if there is $\mathbf{A}^{-1}$ with $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$; this holds iff $\det \mathbf{A} \neq 0$ iff the columns are linearly independent. For invertible $\mathbf{A}, \mathbf{B}$,

$$(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}, \qquad (\mathbf{A}^\top)^{-1} = (\mathbf{A}^{-1})^\top, \qquad (c\mathbf{A})^{-1} = c^{-1}\mathbf{A}^{-1}. \tag{1.5}$$

The first follows from $(\mathbf{A}\mathbf{B})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}\mathbf{I}\mathbf{A}^{-1} = \mathbf{I}$. A useful consequence is the *Sherman–Morrison* formula for a rank-one update,

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^\top)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^\top\mathbf{A}^{-1}}{1 + \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u}}, \qquad 1 + \mathbf{v}^\top\mathbf{A}^{-1}\mathbf{u} \neq 0, \tag{1.6}$$

which lets a regression be updated in $O(p^2)$ operations when one observation is added.

△ Caution: Do not invert matrices to solve equations

The formula $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ is a mathematical statement, not an algorithm. Computing an explicit inverse costs about three times as much as solving the system, and is less accurate. In code write `np.linalg.solve(XtX, Xty)` or, better still, use a least-squares routine based on the **QR** factorisation (`np.linalg.lstsq`), which never forms $\mathbf{X}^\top\mathbf{X}$ at all; see Chapter 2 and Golub and Van Loan (2013, Chapter 5).

1.3.4 Block matrices and the partitioned inverse

Matrices can be partitioned into blocks, and blocks multiply like scalars provided the dimensions conform (and the order of factors is kept). Partitioning is how we reason about subsets of regressors, about the conditional distribution of part of a Gaussian vector, and about instrumental variables.

Theorem 1.14 (Partitioned inverse). *Let* $\mathbf{M} = \begin{pmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{pmatrix}$ *with* $\mathbf{A}$ *and the* Schur complement $\mathbf{S} = \mathbf{D} - \mathbf{C}\mathbf{A}^{-1}\mathbf{B}$ *invertible. Then* $\det \mathbf{M} = \det \mathbf{A} \cdot \det \mathbf{S}$ *and*

$$\mathbf{M}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{B}\mathbf{S}^{-1}\mathbf{C}\mathbf{A}^{-1} & -\mathbf{A}^{-1}\mathbf{B}\mathbf{S}^{-1} \\ -\mathbf{S}^{-1}\mathbf{C}\mathbf{A}^{-1} & \mathbf{S}^{-1} \end{pmatrix}.$$

Proof. Block Gaussian elimination gives the factorisation

$$\mathbf{M} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{C}\mathbf{A}^{-1} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{S} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{A}^{-1}\mathbf{B} \\ \mathbf{0} & \mathbf{I} \end{pmatrix},$$

which is checked by multiplying out. The outer factors are triangular with unit diagonal, so have determinant 1, giving $\det \mathbf{M} = \det \mathbf{A} \det \mathbf{S}$. Their inverses are obtained by negating the off-diagonal block. Inverting the three factors in reverse order and multiplying gives the stated formula. □

The script `ch01/block_inverse.py` implements Theorem 1.14 for a 5×5 positive definite matrix partitioned as $2 + 3$, and checks the trace and determinant identities of the next subsection:

Output of `ch01/block_inverse.py`

```
block formula equals inv(M): True
max abs difference        : 1.11e-16
det(M) = 7.052223,  det(A) det(S) = 7.052223
lower-right block of inv(M) equals inv(S): True
tr(PQ) = -5.831260, tr(QP) = -5.831260
tr(M) = 7.587384 = sum of eigenvalues 7.587384
det(M) = 7.052223 = prod of eigenvalues 7.052223
det(MN) - det(M)det(N) = 0.0e+00
det(3M) / det(M) = 243.0  (= 3^5 = 243)
det(1000 I_200) overflows: inf   slogdet gives log|det| = 1381.551
```

Graduate Perspective: Schur complements everywhere

The lower-right block of $\mathbf{M}^{-1}$ is $\mathbf{S}^{-1}$. Applied to $\mathbf{M} = \mathbf{X}^\top\mathbf{X}$ with $\mathbf{X} = (\mathbf{X}_1, \mathbf{X}_2)$ this gives $\mathbf{S} = \mathbf{X}_2^\top(\mathbf{I} - \mathbf{P}_1)\mathbf{X}_2$, which is the algebra behind the Frisch–Waugh–Lovell theorem: the coefficients on $\mathbf{X}_2$ are those from regressing residualised $\mathbf{y}$ on residualised $\mathbf{X}_2$. Applied to a covariance matrix $\mathbf{\Sigma}$, the Schur complement $\mathbf{\Sigma}_{22} - \mathbf{\Sigma}_{21}\mathbf{\Sigma}_{11}^{-1}\mathbf{\Sigma}_{12}$ is the conditional covariance of a Gaussian vector (Chapter 5). See Hansen (2022, Chapter 3) and Magnus and Neudecker (2019).

1.3.5 Trace and determinant

The *trace* of a square matrix is $\operatorname{tr} \mathbf{A} = \sum_i a_{ii}$. It is linear and satisfies the cyclic property

$$\operatorname{tr}(\mathbf{A}\mathbf{B}) = \sum_i \sum_k a_{ik} b_{ki} = \sum_k \sum_i b_{ki} a_{ik} = \operatorname{tr}(\mathbf{B}\mathbf{A}), \tag{1.7}$$

valid whenever both products are square, even if $\mathbf{AB}$ and $\mathbf{BA}$ have different sizes. Hence $\operatorname{tr}(\mathbf{ABC}) = \operatorname{tr}(\mathbf{CAB})$, and for a scalar $\mathbf{x}^\top\mathbf{Ax} = \operatorname{tr}(\mathbf{Axx}^\top)$, the trick used to compute expectations of quadratic forms. The trace equals the sum of the eigenvalues; for the hat matrix it counts the parameters, so $\operatorname{tr}\mathbf{P}$ is used as the "effective degrees of freedom" of smoothers and ridge regression.

The *determinant* is the unique function of square matrices that is multilinear in the columns, alternating (changes sign when two columns are swapped) and equals 1 at $\mathbf{I}$. Its key properties are $\det(\mathbf{AB}) = \det\mathbf{A}\det\mathbf{B}$, $\det\mathbf{A}^\top = \det\mathbf{A}$, $\det(c\mathbf{A}) = c^n \det\mathbf{A}$ and $\det\mathbf{A} = \prod_i \lambda_i$. Geometrically, $|\det\mathbf{A}|$ is the factor by which the linear map $\mathbf{x} \mapsto \mathbf{Ax}$ scales areas (volumes in $\mathbb{R}^n$), and the sign records whether orientation is reversed (Fig. 1.3). A zero determinant means the map squashes space into a lower-dimensional subspace: the matrix is singular.

Output of `ch01/det_geometry.py`

```
A = [[2, 1], [0.5, 1.5]]   det =  2.500  image area = 2.500
B = [[1, 2], [0.5, 1]]     det =  0.000  image area = 0.000
B is singular: its columns are parallel, the square collapses to a line
```

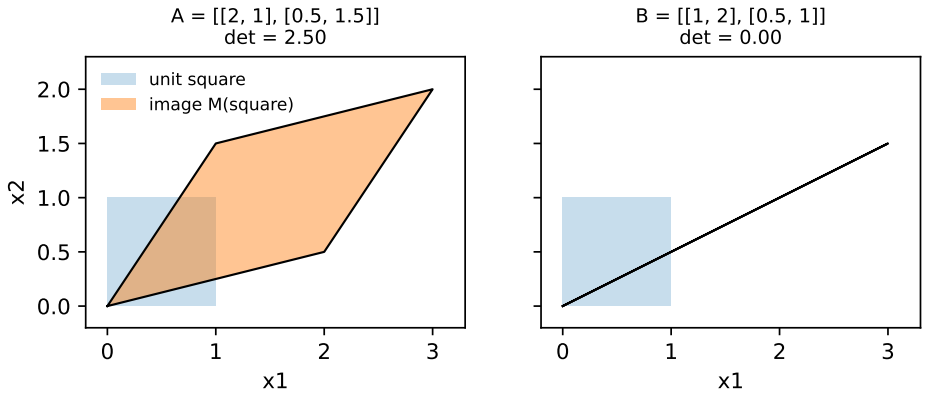


Figure 1.3: The image of the unit square under two linear maps. Left: $\det\mathbf{A} = 2.5$ and the parallelogram has area 2.5. Right: $\det\mathbf{B} = 0$ and the square collapses onto a line.

△ Caution: Determinants are poor measures of near-singularity

$\det(10^{-1}\mathbf{I}_{100}) = 10^{-100}$, yet this matrix is perfectly conditioned; and $\det(1000\,\mathbf{I}_{200})$ overflows to `inf` in double precision, as the listing shows. Use the condition number $\operatorname{cond}(\mathbf{A})$ (Chapter 2) to judge near-singularity, and use `np.linalg.slogdet` for log-determinants in Gaussian log-likelihoods.

Historical Note: Sylvester, Cayley and the word "matrix"

Determinants are older than matrices: they were studied by Leibniz and Seki Takakazu in the late seventeenth century, and Cauchy gave the word "determinant" its modern meaning in 1812. The word *matrix* (Latin for "womb") was coined by James Joseph Sylvester in 1850 for a rectangular array from which determinants could be "born" (Sylvester, 1850). It was his friend Arthur Cayley who, in *A memoir on the theory of matrices* (1858), first treated matrices as algebraic objects in their own right, defining their addition, multiplication and inverse, observing that multiplication is not commutative, and stating the Cayley–Hamilton theorem (Cayley, 1858).

1.3.6 Vectorised computation and broadcasting in NumPy

NumPy implements matrix algebra on `ndarrays`. The operator `@` is the matrix product, while `*` is *elementwise* (Hadamard) multiplication, written $\mathbf{A} \odot \mathbf{B}$. When an elementwise operation combines arrays of different shapes, NumPy *broadcasts* them using two rules: shapes are compared from the right, and two dimensions are compatible if they are equal or one of them is 1 (a missing dimension counts as 1); a dimension of length 1 is virtually repeated to match. Thus an (n, p) array minus a $(p,)$ array subtracts the vector from every row, and an (n, p) array times an $(n, 1)$ array scales every row.

Listing 1.4: Broadcasting: centring, row scaling, pairwise distances `ch01/broadcasting.py`

```python
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Vectorised computation and NumPy broadcasting rules, with a classic pitfall.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
X = rng.normal(loc=[10, 50, 3], scale=[2, 15, 1], size=(5, 3)).round(1)
print("X (5 x 3) =\n", X)
mu = X.mean(axis=0)                       # shape (3,)
Xc = X - mu                               # (5,3) - (3,) -> row-wise subtraction
print("column means after centring:", np.round(Xc.mean(axis=0), 12) + 0.0)
w = np.array([0.1, 0.2, 0.3, 0.2, 0.2])   # one weight per row
print("weighted column means w^T X :", np.round(w @ X, 3))
Xw = X * w[:, None]                       # (5,3) * (5,1) scales each row
print("row scaling equals diag(w) X:", np.allclose(Xw, np.diag(w) @ X))

# outer product and pairwise distances without loops
a, b = np.arange(1, 4), np.arange(1, 5)
print("outer product a[:,None]*b[None,:] =\n", a[:, None] * b[None, :])
diff = X[:, None, :] - X[None, :, :]      # shape (5, 5, 3)
Dist = np.sqrt((diff**2).sum(axis=2))
print("pairwise Euclidean distances (rounded):\n", Dist.round(1))

# pitfall: a column vector (n,1) minus a flat vector (n,) gives an (n,n) matrix
y = rng.standard_normal(5)
yhat = y.reshape(-1, 1)                   # e.g. a prediction returned as (5,1)
resid = y - yhat
print("shape of y - yhat when yhat is (5,1):", resid.shape, " <- silent bug!")
print("correct: y - yhat.ravel() has shape", (y - yhat.ravel()).shape)
try:
    X + np.ones(5)                        # (5,3) + (5,) is incompatible
except ValueError as err:
    print("ValueError:", err)
```

Output

```
X (5 x 3) =
 [[ 9.4 48.2  2.7]
 [ 8.6 53.8  2.4]
 [ 9.5 46.8  4.1]
 [ 7.6 33.2  2.8]
 [ 7.8 55.2  3.5]]
column means after centring: [0. 0. 0.]
weighted column means w^T X : [ 8.59 47.3   3.24]
```

```
row scaling equals diag(w) X: True
outer product a[:,None]*b[None,:] =
 [[ 1  2  3  4]
 [ 2  4  6  8]
 [ 3  6  9 12]]
pairwise Euclidean distances (rounded):
 [[ 0.   5.7  2.  15.1  7.2]
 [ 5.7  0.   7.3 20.6  2. ]
 [ 2.   7.3  0.  13.8  8.6]
 [15.1 20.6 13.8  0.  22. ]
 [ 7.2  2.   8.6 22.   0. ]]
shape of y - yhat when yhat is (5,1): (5, 5)  <- silent bug!
correct: y - yhat.ravel() has shape (5,)
ValueError: operands could not be broadcast together with shapes (5,3) (5,)
```

△ Caution: The $(n,)$ versus $(n,1)$ trap

If `y` has shape `(n,)` and `yhat` has shape `(n,1)`, then `y - yhat` silently broadcasts to an (n,n) matrix, and `np.mean((y - yhat)**2)` returns a meaningless "mean squared error" without any warning. Many model-fitting libraries return predictions as column arrays. Check shapes with `assert y.shape == yhat.shape`, or flatten with `.ravel()`.

1.3.7 A policy example: the Leontief input–output model

Divide an economy into m sectors. Let x_j be the gross output of sector j and $a_{ij} \geq 0$ the amount of sector i's output used to produce one unit of sector j's output (the *technical coefficients*). If f_i is final demand for sector i (households, government, exports), market clearing requires

$$x_i = \sum_{j=1}^{m} a_{ij} x_j + f_i \quad (i = 1, \dots, m), \qquad \text{i.e.} \qquad (\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{f}. \tag{1.8}$$

The *Leontief inverse* $\mathbf{L} = (\mathbf{I} - \mathbf{A})^{-1}$ maps final demand to the gross output required to satisfy it, directly and through all rounds of intermediate purchases. Its entry ℓ_{ij} is the output of sector i needed per unit of final demand for sector j, and the column sum $\sum_i \ell_{ij}$ is sector j's *output multiplier*.

Theorem 1.15 (Productive economies)**.** *If $\mathbf{A} \geq 0$ (entrywise) and every column sum of $\mathbf{A}$ is less than 1, then $\mathbf{I} - \mathbf{A}$ is invertible,*

$$(\mathbf{I} - \mathbf{A})^{-1} = \sum_{k=0}^{\infty} \mathbf{A}^k = \mathbf{I} + \mathbf{A} + \mathbf{A}^2 + \cdots \geq 0,$$

and the series converges; hence every non-negative final demand $\mathbf{f}$ is met by a non-negative output $\mathbf{x} = \mathbf{L}\mathbf{f}$.

Proof. Let $\|\mathbf{A}\|_1 = \max_j \sum_i |a_{ij}| =: \alpha < 1$, the maximum column sum, which is a matrix norm satisfying $\|\mathbf{AB}\|_1 \leq \|\mathbf{A}\|_1 \, \|\mathbf{B}\|_1$ (Chapter 2). Then $\left\|\mathbf{A}^k\right\|_1 \leq \alpha^k$, so the partial sums $\mathbf{S}_K = \sum_{k=0}^{K} \mathbf{A}^k$ form a Cauchy sequence and converge to some $\mathbf{S}$. Since $(\mathbf{I} - \mathbf{A})\mathbf{S}_K = \mathbf{I} - \mathbf{A}^{K+1} \to \mathbf{I}$, we get $(\mathbf{I} - \mathbf{A})\mathbf{S} = \mathbf{I}$. Each $\mathbf{A}^k \geq 0$, so $\mathbf{S} \geq 0$. □

The series has an economic reading: $\mathbf{f}$ is the direct requirement, $\mathbf{Af}$ the first round of inputs, $\mathbf{A}^2\mathbf{f}$ the inputs needed to produce those inputs, and so on.

Listing 1.5: A three-sector input–output model (illustrative coefficients) ch01/leontief.py

```python
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# A three-sector Leontief input-output model with illustrative coefficients.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

sectors = ["agriculture", "industry", "services"]
# technical coefficients a_ij: input from sector i per unit output of sector j
# (hypothetical numbers for teaching, not an official input-output table)
A = np.array([[0.15, 0.10, 0.05],
              [0.20, 0.30, 0.15],
              [0.10, 0.20, 0.25]])
f = np.array([400.0, 600.0, 900.0])      # final demand (GHS million)
I = np.eye(3)
L = np.linalg.inv(I - A)                 # Leontief inverse
x = np.linalg.solve(I - A, f)            # gross output: (I - A) x = f
print("column sums of A:", A.sum(axis=0), "(all < 1: productive economy)")
print("Leontief inverse L = (I - A)^-1 =\n", L.round(4))
print("gross output x =", x.round(2))
print("check x - Ax = f:", np.allclose(x - A @ x, f))
mult = L.sum(axis=0)                     # output multipliers
for s, m in zip(sectors, mult):
    print(f"output multiplier of {s:12s} = {m:.4f}")

# policy shock: +100 in final demand for services (e.g. public health spending)
df = np.array([0.0, 0.0, 100.0])
dx = L @ df
print("change in output by sector:", dx.round(2), " total =", dx.sum().round(2))

# employment: labour coefficients (jobs per GHS million of output, hypothetical)
ell = np.array([12.0, 4.0, 7.0])
print("jobs created by the shock = ell^T L df =", round(ell @ dx, 1))

# Neumann series: L = I + A + A^2 + ... (converges because rho(A) < 1)
rho = max(abs(np.linalg.eigvals(A)))
S, term = I.copy(), I.copy()
for k in range(1, 31):
    term = term @ A
    S += term
print(f"spectral radius of A = {rho:.4f}")
print("max |L - sum_{k<=30} A^k| = %.2e" % np.abs(L - S).max())
```

Output

```
column sums of A: [0.45 0.6  0.45] (all < 1: productive economy)
Leontief inverse L = (I - A)^-1 =
 [[1.2414 0.2132 0.1254]
 [0.4138 1.5862 0.3448]
 [0.2759 0.4514 1.442 ]]
gross output x = [ 737.3  1427.59 1679.  ]
check x - Ax = f: True
output multiplier of agriculture  = 1.9310
output multiplier of industry     = 2.2508
output multiplier of services     = 1.9122
change in output by sector: [ 12.54  34.48 144.2 ]  total = 191.22
```

```
jobs created by the shock = ell^T L df = 1297.8
spectral radius of A = 0.5152
max |L - sum_{k<=30} A^k| = 1.25e-09
```

Policy Lens: Input–output multipliers for impact assessment

Governments and development partners use input–output multipliers to estimate the economy-wide effect of a spending programme, a new factory or an export shock. In the illustrative economy, an extra GHS 100 million of final demand for services raises gross output by about GHS 191 million (services' output multiplier is 1.91), including GHS 34 million in industry, and with hypothetical labour coefficients creates about 1,300 jobs. Such numbers depend on strong assumptions: fixed coefficients (no substitution), no capacity constraints and no price effects, and they count gross output, which double-counts intermediate inputs. Multipliers summed across many projects can therefore greatly overstate the true value added. Computable general equilibrium models relax these assumptions (Miller and Blair, 2009).

Historical Note: Wassily Leontief

Wassily Leontief published the first input–output table for the United States in 1936 (Leontief, 1936), followed by *The Structure of American Economy, 1919–1929* (1941). The model was one of the first large-scale applications of matrix algebra to economic policy, and its solution required inverting matrices on early computing equipment. Leontief received the 1973 Nobel Memorial Prize in Economic Sciences "for the development of the input-output method and for its application to important economic problems". Today national statistics offices and the United Nations compile supply-and-use and input–output tables as part of the System of National Accounts.

⋆ Key Concept: Matrices as maps and as computations

A matrix product can be read entrywise (inner products), by columns (combinations of columns), by rows, or as a sum of outer products. Transposes and inverses reverse the order of products. Partitioned matrices and the Schur complement give block inverses and determinants. The trace is cyclic; the determinant is a volume-scaling factor. In code, use `@` for products, `solve` rather than `inv`, and broadcasting rather than loops, while checking shapes.

Exercises

1.3.1 Theory Prove that the product of two upper-triangular matrices is upper triangular, that the inverse of an invertible upper-triangular matrix is upper triangular, and that the determinant of a triangular matrix is the product of its diagonal entries.

1.3.2 Theory Verify the Sherman–Morrison formula (1.6) by multiplication. Use it to show how the OLS estimator changes when one observation $(\mathbf{x}_0, y_0)$ is added to the sample.

1.3.3 Theory Show that if $\mathbf{P}$ is an orthogonal projection then $\operatorname{tr}\mathbf{P} = \operatorname{rank}\mathbf{P}$. (Hint: use the eigenvalues, or write $\mathbf{P} = \mathbf{U}\mathbf{U}^\top$ with orthonormal columns $\mathbf{U}$ and use the cyclic property of the trace.)

1.3.4 Python Write a function `back_substitution(U, b)` for an upper-triangular system using only one Python loop over rows (vectorise the inner sum) and compare its result and speed with `scipy.linalg.solve_triangular` for $n = 2000$.

1.3.5 Python Use the partitioned-inverse formula to verify numerically the Frisch–Waugh–Lovell theorem: regress a simulated $\mathbf{y}$ on $\mathbf{X} = (\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2)$ and show that the coefficient on $\mathbf{x}_2$ equals that from regressing residualised $\mathbf{y}$ on residualised $\mathbf{x}_2$.

1.3.6 Policy In `leontief.py`, compute the *backward linkages* (column sums of $\mathbf{L}$) and *forward linkages* (row sums of $\mathbf{L}$). Which sector is the best target for a demand stimulus that maximises gross output? Which maximises employment? Explain why the two rankings can differ.

1.3.7 ★ Graduate Let $\mathbf{A} \geq 0$ be square with spectral radius $\rho(\mathbf{A}) < 1$ but some column sums greater than 1. Show that $\sum_{k} \mathbf{A}^k$ still converges to $(\mathbf{I} - \mathbf{A})^{-1}$. (Hint: for any $\epsilon > 0$ there is an induced matrix norm with $\|\mathbf{A}\| \leq \rho(\mathbf{A}) + \epsilon$.) Relate the condition $\rho(\mathbf{A}) < 1$ to the Hawkins–Simon condition.

1.4 Data as Matrices

Once data are stored in a matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, descriptive statistics become matrix expressions. This is not just elegant: matrix expressions can be differentiated (Chapter 12), analysed for rank and conditioning (Chapter 2) and computed by optimised libraries.

1.4.1 The design matrix

In a linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ the matrix $\mathbf{X}$ is called the *design matrix*. It usually contains a column of ones for the intercept, numerical covariates, and indicator (dummy) columns for categorical variables: a categorical variable with G levels is encoded by $G - 1$ dummies when an intercept is present, since all G dummies sum to $\mathbf{1}_n$ and would make the columns linearly dependent. Interactions are elementwise products of columns and polynomial terms are powers. The prediction for observation i is the inner product $\mathbf{x}_i^{\top}\boldsymbol{\beta}$; the vector of predictions is the column combination $\mathbf{X}\boldsymbol{\beta}$.

1.4.2 Means, the centring matrix and standardisation

The vector of column means is

$$\bar{\mathbf{x}} = \frac{1}{n}\mathbf{X}^{\top}\mathbf{1}_n = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i \in \mathbb{R}^p.$$

Subtracting the mean from each row gives the centred data matrix $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_n\bar{\mathbf{x}}^{\top}$. Since $\mathbf{1}_n\bar{\mathbf{x}}^{\top} = n^{-1}\mathbf{1}_n\mathbf{1}_n^{\top}\mathbf{X}$, centring is multiplication by a matrix:

$$\tilde{\mathbf{X}} = \mathbf{H}\mathbf{X}, \qquad \mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}_n\mathbf{1}_n^{\top}. \tag{1.9}$$

Proposition 1.16 (Properties of the centring matrix)**.** $\mathbf{H}$ *is symmetric and idempotent,* $\mathbf{H}\mathbf{1}_n = \mathbf{0}$, $\operatorname{tr}\mathbf{H} = n - 1$ *and* $\operatorname{rank}\mathbf{H} = n - 1$. *It is the orthogonal projection onto the subspace* $\{\mathbf{v} \in \mathbb{R}^n : \mathbf{1}_n^{\top}\mathbf{v} = 0\}$ *of vectors with mean zero.*

Proof. Write $\mathbf{J} = n^{-1}\mathbf{1}_n\mathbf{1}_n^{\top}$. Then $\mathbf{J}^{\top} = \mathbf{J}$ and $\mathbf{J}^2 = n^{-2}\mathbf{1}_n(\mathbf{1}_n^{\top}\mathbf{1}_n)\mathbf{1}_n^{\top} = \mathbf{J}$, so $\mathbf{J}$ is the orthogonal projection onto $\operatorname{span}\{\mathbf{1}_n\}$ and $\mathbf{H} = \mathbf{I} - \mathbf{J}$ is the complementary projection (Proposition 1.13). $\mathbf{H}\mathbf{1}_n = \mathbf{1}_n - \mathbf{1}_n = \mathbf{0}$, and $\operatorname{tr}\mathbf{H} = n - \operatorname{tr}\mathbf{J} = n - n^{-1}\mathbf{1}_n^{\top}\mathbf{1}_n = n - 1$, which equals the rank of a projection. □

The trace $n - 1$ is precisely why the sample variance divides by $n - 1$: centring uses up one degree of freedom. *Standardisation* divides each centred column by its standard deviation, $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{D}^{-1/2}$ with $\mathbf{D} = \operatorname{diag}(s_{11}, \dots, s_{pp})$, so that every column has mean 0 and variance 1.

Standardisation makes coefficients comparable across variables measured in different units and is essential before penalised regression, principal components and k-means, whose results otherwise depend on arbitrary choices of units (cedis versus thousands of cedis).

1.4.3 Sample covariance and correlation matrices

Definition 1.17. The *sample covariance matrix* and *sample correlation matrix* of $\mathbf{X}$ are

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^{\top}\mathbf{H}\mathbf{X} = \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^{\top}, \qquad \mathbf{R} = \mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{-1/2}, \tag{1.10}$$

with entries $s_{jk} = (n-1)^{-1}\sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$ and $r_{jk} = s_{jk}/\sqrt{s_{jj}s_{kk}}$.

The two expressions for $\mathbf{S}$ agree because $\mathbf{H}$ is a symmetric idempotent matrix: $\mathbf{X}^{\top}\mathbf{H}\mathbf{X} = \mathbf{X}^{\top}\mathbf{H}^{\top}\mathbf{H}\mathbf{X} = \tilde{\mathbf{X}}^{\top}\tilde{\mathbf{X}}$, and the outer-product view of Proposition 1.11 turns $\tilde{\mathbf{X}}^{\top}\tilde{\mathbf{X}}$ into the sum over observations.

Proposition 1.18. *$\mathbf{S}$ and $\mathbf{R}$ are symmetric positive semidefinite, $r_{jj} = 1$ and $|r_{jk}| \le 1$. $\mathbf{S}$ is positive definite iff $\tilde{\mathbf{X}}$ has full column rank p, which requires $n \ge p + 1$.*

Proof. For any $\mathbf{a} \in \mathbb{R}^p$, $\mathbf{a}^{\top}\mathbf{S}\mathbf{a} = (n-1)^{-1}\left\|\tilde{\mathbf{X}}\mathbf{a}\right\|_2^2 \ge 0$, with equality iff $\tilde{\mathbf{X}}\mathbf{a} = \mathbf{0}$. Hence $\mathbf{S}$ is positive definite iff the columns of $\tilde{\mathbf{X}}$ are independent; since they lie in the $(n-1)$-dimensional space $\{\mathbf{v} : \mathbf{1}_n^{\top}\mathbf{v} = 0\}$, at most $n-1$ of them can be independent. $\mathbf{R} = \mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{-1/2}$ is congruent to $\mathbf{S}$ and so is also positive semidefinite. Finally r_{jk} is the cosine of the angle between centred columns j and k, so $|r_{jk}| \le 1$ by Cauchy–Schwarz. □

In particular, when $p > n - 1$ (more variables than observations, common in genomics, text and macroeconomic panels with many indicators) the sample covariance matrix is singular and cannot be inverted; regularisation is required (Chapter 8).

1.4.4 From pandas to NumPy

Survey and administrative data usually arrive as tables with named, heterogeneously typed columns, handled in Python by `pandas`. The method `df.to_numpy()` returns the underlying array, after which the matrix formulas apply. Two details matter. First, categorical and string columns must be encoded (for example with `pd.get_dummies`) before conversion, or the result has `dtype=object` and arithmetic becomes slow or fails. Second, missing values (`NaN`) propagate through every sum and product, so a single missing entry makes a whole row of $\mathbf{S}$ undefined; deal with missingness explicitly (Chapter 13).

Listing 1.6: Centring matrix, covariance and correlation of a simulated survey `ch01/data_matrix.py`

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Centring matrix, sample covariance and correlation from a data matrix.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import pandas as pd

rng = np.random.default_rng(803)
n = 500
# simulated household survey (illustrative only)
school = rng.integers(0, 19, n).astype(float)
hhsize = rng.poisson(4.5, n) + 1.0
```

```
dist = rng.gamma(2.0, 4.0, n)                          # km to nearest market
loginc = 1.2 + 0.09 * school - 0.04 * dist + 0.05 * hhsize \
    + rng.normal(0, 0.45, n)
df = pd.DataFrame({"log_income": loginc, "schooling": school,
                   "hh_size": hhsize, "dist_market": dist})
print(df.describe().loc[["mean", "std"]].round(3))
X = df.to_numpy()                                      # n x p float64 array
print("X:", X.shape, X.dtype)

one = np.ones((n, 1))
H = np.eye(n) - one @ one.T / n                        # centring matrix
print("H symmetric:", np.allclose(H, H.T), " H idempotent:", np.allclose(H @ H, H),
      " trace(H) =", round(np.trace(H), 8))
print("H X equals X - column means:", np.allclose(H @ X, X - X.mean(axis=0)))

S = X.T @ H @ X / (n - 1)                              # sample covariance
Xc = X - X.mean(axis=0)
S_fast = Xc.T @ Xc / (n - 1)                           # same, without the n x n H
print("S matches np.cov:", np.allclose(S, np.cov(X, rowvar=False)),
      "  matches fast version:", np.allclose(S, S_fast))
d = np.sqrt(np.diag(S))
R = S / np.outer(d, d)                                 # D^-1/2 S D^-1/2
Z = Xc / d                                             # standardised data
print("R matches np.corrcoef:", np.allclose(R, np.corrcoef(X, rowvar=False)),
      "  Z^T Z/(n-1) = R:", np.allclose(Z.T @ Z / (n - 1), R))
print("correlation matrix R:")
print(pd.DataFrame(R, index=df.columns, columns=df.columns).round(3))
print("eigenvalues of S (all >= 0):", np.linalg.eigvalsh(S).round(4))
print("memory of H: %.1f MB versus X: %.3f MB" % (H.nbytes / 1e6, X.nbytes / 1e6))
```

Output

```
      log_income  schooling  hh_size  dist_market
mean       1.925      8.708    5.436        7.950
std        0.708      5.503    2.157        5.776
X: (500, 4) float64
H symmetric: True  H idempotent: True  trace(H) = 499.0
H X equals X - column means: True
S matches np.cov: True   matches fast version: True
R matches np.corrcoef: True   Z^T Z/(n-1) = R: True
correlation matrix R:
             log_income  schooling  hh_size  dist_market
log_income        1.000      0.703    0.121       -0.250
schooling         0.703      1.000   -0.067        0.080
hh_size           0.121     -0.067    1.000        0.021
dist_market      -0.250      0.080    0.021        1.000
eigenvalues of S (all >= 0): [ 0.1877  4.6409 29.1663 34.8117]
memory of H: 2.0 MB versus X: 0.016 MB
```

△ Caution: Formulas versus algorithms, again

$\mathbf{S} = \mathbf{X}^{\top}\mathbf{H}\mathbf{X}/(n-1)$ is the right *formula*, but forming the $n \times n$ matrix $\mathbf{H}$ is a terrible *algorithm*: for $n = 500$ it already occupies 2 MB (versus 16 kB for $\mathbf{X}$), and for a census of $n = 10^7$ it would need 800 TB. Always centre by broadcasting, `Xc = X - X.mean(axis=0)`, which costs $O(np)$

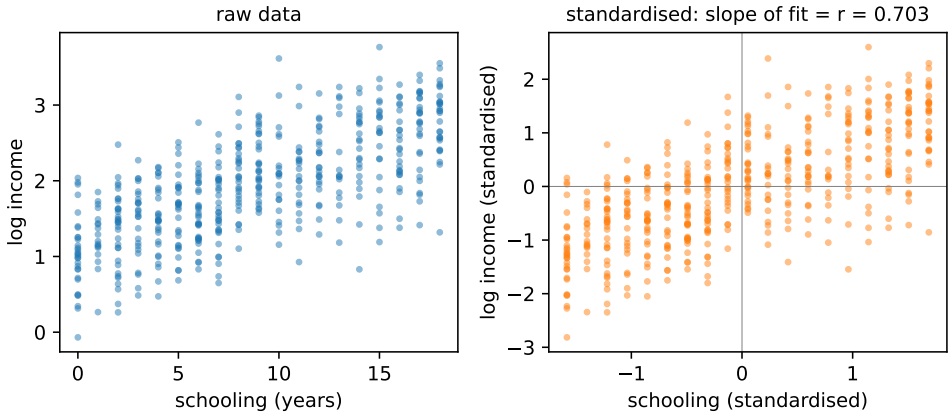


Figure 1.4: Simulated household survey: log income against years of schooling, raw (left) and standardised (right). Standardisation moves the cloud to the origin and rescales the axes; the least-squares slope of the standardised variables equals the correlation coefficient (script `ch01/standardise_fig.py`).

operations and no extra memory beyond $\tilde{\mathbf{X}}$.

Policy Lens: Standardised indicators and composite indices

Composite indices such as multidimensional poverty or human development indices combine indicators measured in different units (years of schooling, life expectancy, income). Each indicator is first normalised (by z-scores, as here, or by min–max scaling onto $[0,1]$), then aggregated with weights, an inner product again. The choice of normalisation is not neutral: z-scores depend on the sample, so adding or removing districts changes every district's score, whereas fixed "goalposts" (pre-specified minima and maxima) give scores that are comparable across years. The correlation matrix $\mathbf{R}$ of the indicators shows which dimensions are redundant and is the starting point for principal-component weights (Chapter 3).

1.4.5 The Kronecker product

Definition 1.19. For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{r\times s}$ the *Kronecker product* is the $mr \times ns$ block matrix $\mathbf{A} \otimes \mathbf{B} = (a_{ij}\mathbf{B})$. The operator $\operatorname{vec}(\mathbf{A})$ stacks the columns of $\mathbf{A}$ into one vector of length mn.

The main rules, valid whenever the products are defined, are the mixed-product rule $(\mathbf{A}\otimes\mathbf{B})(\mathbf{C}\otimes\mathbf{D}) = (\mathbf{AC})\otimes(\mathbf{BD})$, $(\mathbf{A}\otimes\mathbf{B})^{-1} = \mathbf{A}^{-1}\otimes\mathbf{B}^{-1}$, $(\mathbf{A}\otimes\mathbf{B})^{\top} = \mathbf{A}^{\top}\otimes\mathbf{B}^{\top}$ and the identity

$$\operatorname{vec}(\mathbf{AXB}) = (\mathbf{B}^{\top}\otimes\mathbf{A})\operatorname{vec}(\mathbf{X}), \tag{1.11}$$

which converts matrix equations into ordinary linear systems and is central to matrix calculus (Chapter 12). In panel data with N units observed for T periods, stacked by unit, the unit dummies are $\mathbf{I}_N \otimes \mathbf{1}_T$ and the period dummies $\mathbf{1}_N \otimes \mathbf{I}_T$; the covariance of errors that are correlated within units is $\mathbf{I}_N \otimes \boldsymbol{\Sigma}_T$.

The script `ch01/kronecker.py` checks these rules numerically (using `np.kron` and `M.reshape(-1, order="F")` for vec) and builds the panel dummy matrices:

Output of `ch01/kronecker.py`

```
A kron B =
 [[0 1 0 2]
 [1 0 2 0]
 [0 3 0 4]
 [3 0 4 0]]
mixed product rule  : True
inverse rule        : True
vec(AXB) identity   : True
district dummies D = I_N kron 1_T, shape (12, 3)
[[1 1 1 1 0 0 0 0 0 0 0 0]
 [0 0 0 0 1 1 1 1 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 1 1 1]]
year dummies 1_N kron I_T, shape (12, 4)
within transform Q y removes district means: True
```

Historical Note: Whose product?

The Kronecker product is named after Leopold Kronecker, but the determinant identity $\det(\mathbf{A}\otimes\mathbf{B}) = (\det\mathbf{A})^n(\det\mathbf{B})^m$ for $\mathbf{A}\in\mathbb{R}^{m\times m}$, $\mathbf{B}\in\mathbb{R}^{n\times n}$ was published by Johann Georg Zehfuss in 1858. In a short historical survey, Henderson et al. (1983) trace how the name of Kronecker became attached to the product and argue that Zehfuss deserves at least part of the credit; the name "Zehfuss product" is occasionally used.

⋆ Key Concept: Descriptive statistics are matrix algebra

For a data matrix $\mathbf{X}$: means are $n^{-1}\mathbf{X}^\top\mathbf{1}$, centring is $\mathbf{HX}$ with $\mathbf{H}=\mathbf{I}-n^{-1}\mathbf{1}\mathbf{1}^\top$ (a projection of trace $n-1$), the covariance is $\mathbf{S}=\mathbf{X}^\top\mathbf{H}\mathbf{X}/(n-1)$, and the correlation is $\mathbf{R}=\mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{-1/2}$. Both are positive semidefinite and are singular when $p\geq n$. Use these formulas for reasoning, and broadcasting for computing.

Exercises

1.4.1 Theory Show that $\bar{\mathbf{x}}$ minimises $\sum_{i=1}^n\|\mathbf{x}_i-\mathbf{c}\|_2^2$ over $\mathbf{c}\in\mathbb{R}^p$, and that the minimum equals $(n-1)\,\mathrm{tr}\,\mathbf{S}$.

1.4.2 Theory Let $\mathbf{Y}=\mathbf{X}\mathbf{B}+\mathbf{1}_n\mathbf{c}^\top$ be an affine transformation of the data (a change of units and origin). Show that $\mathbf{S}_Y=\mathbf{B}^\top\mathbf{S}_X\mathbf{B}$. Deduce that correlations are invariant to positive rescaling of each variable but not to general linear transformations.

1.4.3 Theory Prove the mixed-product rule for Kronecker products, and use it to show that the eigenvalues of $\mathbf{A}\otimes\mathbf{B}$ are the products $\lambda_i(\mathbf{A})\mu_j(\mathbf{B})$.

1.4.4 Python Load `sklearn.datasets.load_diabetes(as_frame=True)`. Compute $\mathbf{S}$ and $\mathbf{R}$ with matrix formulas and compare them with `df.cov()` and `df.corr()`. What do you notice about the column scaling of this data set, and how does it affect $\mathbf{S}$?

1.4.5 Python Show numerically that the sample covariance can be computed in a single pass over rows as $\sum_i\mathbf{x}_i\mathbf{x}_i^\top$ and $\sum_i\mathbf{x}_i$. Then add 10^8 to every entry of $\mathbf{X}$ and compare this "one-pass" covariance with `np.cov`. Explain the result after reading Section 1.5.

1.4.6 Policy A district development index averages three standardised indicators. Simulate 50 districts, compute the index with z-scores, then drop the five best-off districts and recompute. How many of the remaining districts change rank? Repeat with fixed goalposts

and comment on which method a planning ministry should prefer for monitoring progress over time.

1.4.7 ★ Graduate Let $\mathbf{X} \in \mathbb{R}^{n \times p}$ have i.i.d. standard normal entries, with $p/n \to \gamma \in (0, 1)$. The eigenvalues of $\mathbf{S}$ do *not* all converge to 1. Simulate the eigenvalue histogram for $n = 2000$, $p = 500$ and compare it with the Marchenko–Pastur support $[(1 - \sqrt{\gamma})^2, (1 + \sqrt{\gamma})^2]$. What does this imply for covariance estimation in large macroeconomic panels? (Vershynin, 2018)

1.5 Floating-Point Arithmetic and Computational Precision

Mathematics works in $\mathbb{R}$, an uncountable set. A computer has finitely many bit patterns, so it can represent only finitely many numbers. Almost every real number must therefore be *rounded* when it is stored, and almost every arithmetic operation must round its result. This section explains the IEEE 754 standard that governs this rounding on essentially every modern processor, a simple model for reasoning about its errors, and the handful of situations in which those errors become large enough to matter in data analysis (Goldberg, 1991; Higham, 2002; Overton, 2001).

1.5.1 IEEE 754 double precision

A *binary floating-point number* has the form

$$x = (-1)^s \times (1.f_1 f_2 \ldots f_{t-1})_2 \times 2^e, \qquad e_{\min} \le e \le e_{\max}, \tag{1.12}$$

with sign bit s, a *significand* (mantissa) of t binary digits whose leading digit is an implicit 1, and an integer *exponent* e. IEEE 754 *double precision* (`float64`, the default in NumPy, R, MATLAB and Stata's `double`) uses 64 bits: 1 sign bit, 11 exponent bits and 52 fraction bits, so $t = 53$ and $e_{\min} = -1022$, $e_{\max} = 1023$. Single precision (`float32`) uses 32 bits with $t = 24$.

> **Definition 1.20** (Machine epsilon and unit roundoff)**.** *Machine epsilon* $\varepsilon_{\text{mach}} = 2^{1-t}$ is the distance from 1 to the next larger floating-point number. The *unit roundoff* is $u = \varepsilon_{\text{mach}}/2 = 2^{-t}$. In double precision $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$ and $u \approx 1.11 \times 10^{-16}$; in single precision $\varepsilon_{\text{mach}} = 2^{-23} \approx 1.19 \times 10^{-7}$.

Thus a double carries about 15–16 significant decimal digits, a single about 7. The *spacing* between adjacent doubles is $\varepsilon_{\text{mach}}\, 2^e$: it is relative, not absolute. Near 1 it is 2.2×10^{-16}; near 10^{16} it is 2, so $10^{16} + 1$ cannot be represented and rounds back to 10^{16}. The largest double is about 1.8×10^{308}; results beyond it *overflow* to `inf`. The smallest normalised positive double is about 2.2×10^{-308}; below it the standard provides *subnormal* numbers (with leading digit 0 and reduced precision) down to about 4.9×10^{-324}, after which results *underflow* to zero. Invalid operations such as $\infty - \infty$ or $0/0$ produce `nan` ("not a number"), which propagates through subsequent arithmetic and is not equal to anything, not even itself.

Listing 1.7: Inside IEEE 754: epsilon, spacing, overflow, underflow `ch01/float_basics.py`

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# IEEE 754 double precision: representation, spacing, overflow and underflow.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

print("0.1 + 0.2 == 0.3 ?", 0.1 + 0.2 == 0.3)
print("0.1 + 0.2 =", repr(0.1 + 0.2), "  0.1 stored as", f"{0.1:.20f}")
print("0.1 in hexadecimal:", (0.1).hex())
```

```python
for t in (np.float64, np.float32):
    fi = np.finfo(t)
    print(f"{t.__name__:8s} eps = {fi.eps:.3e}  max = {fi.max:.3e}"
          f"  tiny = {fi.tiny:.3e}  digits = {fi.precision}")
print("unit roundoff u = eps/2 = %.3e" % (np.finfo(float).eps / 2))

# spacing between neighbouring doubles grows with magnitude
for x in [1.0, 1e3, 1e8, 1e16]:
    print(f"spacing(ulp) at {x:8.0e}: {np.spacing(x):.3e}")
print("1e16 + 1 - 1e16 =", 1e16 + 1 - 1e16)
print("2**53 + 1 == 2**53 as floats ?", float(2**53) + 1 == float(2**53))
print("1 + eps/2 == 1 ?", 1.0 + np.finfo(float).eps / 2 == 1.0)

# overflow, underflow and special values
with np.errstate(over="ignore", invalid="ignore"):
    big = np.exp(np.array([709.0, 710.0]))
    print("exp(709), exp(710) =", big)
    print("inf - inf =", big[1] - big[1], "  1/inf =", 1 / big[1])
print("5e-324 (smallest subnormal) =", 5e-324, "  5e-324 / 2 =", 5e-324 / 2)
print("nan == nan ?", np.nan == np.nan, "  np.isnan(nan):", np.isnan(np.nan))

# integers do not overflow gracefully either
a = np.array([2**62], dtype=np.int64)
print("int64: 2**62 * 2 =", (a * 2)[0], "  (wrap-around, no warning)")
print("40000.7 cast to int16 ->", np.array([40000.7]).astype(np.int16)[0],
      "  (cf. Ariane 5)")

# comparing floats
print("np.isclose(0.1 + 0.2, 0.3):", np.isclose(0.1 + 0.2, 0.3))
print("np.isclose(1e-10, 2e-10)  :", np.isclose(1e-10, 2e-10),
      " <- default atol=1e-8 dominates!")
print("np.isclose(1e-10, 2e-10, atol=0):", np.isclose(1e-10, 2e-10, atol=0))
```

Output

```
0.1 + 0.2 == 0.3 ? False
0.1 + 0.2 = 0.30000000000000004   0.1 stored as 0.1000000000000000555
0.1 in hexadecimal: 0x1.999999999999ap-4
float64  eps = 2.220e-16  max = 1.798e+308  tiny = 2.225e-308  digits = 15
float32  eps = 1.192e-07  max = 3.403e+38  tiny = 1.175e-38  digits = 6
unit roundoff u = eps/2 = 1.110e-16
spacing(ulp) at    1e+00: 2.220e-16
spacing(ulp) at    1e+03: 1.137e-13
spacing(ulp) at    1e+08: 1.490e-08
spacing(ulp) at    1e+16: 2.000e+00
1e16 + 1 - 1e16 = 0.0
2**53 + 1 == 2**53 as floats ? True
1 + eps/2 == 1 ? True
exp(709), exp(710) = [8.21840746e+307             inf]
inf - inf = nan   1/inf = 0.0
5e-324 (smallest subnormal) = 5e-324   5e-324 / 2 = 0.0
nan == nan ? False   np.isnan(nan): True
int64: 2**62 * 2 = -9223372036854775808   (wrap-around, no warning)
40000.7 cast to int16 -> -25536   (cf. Ariane 5)
np.isclose(0.1 + 0.2, 0.3): True
np.isclose(1e-10, 2e-10)  : True  <- default atol=1e-8 dominates!
```

```
np.isclose(1e-10, 2e-10, atol=0): False
```

The decimal number 0.1 has the infinite binary expansion $0.000110011001100\ldots_2$, so it is stored with a relative error of about 5×10^{-17}; the hexadecimal output `0x1.999999999999ap-4` shows the repeating pattern of 9s (binary 1001) and the final rounding up to `a`. Hence `0.1 + 0.2 == 0.3` is `False`. Integer arithmetic in NumPy has a different failure mode: fixed-width integers *wrap around* silently on overflow, and an out-of-range float-to-int conversion produces garbage.

Historical Note: IEEE 754, William Kahan, and the Ariane 5

Before 1985 every computer manufacturer had its own floating-point formats and rounding rules, so the same program could give different answers on different machines. The IEEE Standard for Binary Floating-Point Arithmetic, IEEE 754-1985, fixed the formats, required correctly rounded $+, -, \times, \div$ and $\sqrt{}$, and introduced infinities, NaNs and gradual underflow (IEEE, 1985); it was revised in 2008 and 2019. Its principal architect, William Kahan of the University of California, Berkeley, received the 1989 ACM Turing Award for his fundamental contributions to numerical analysis. The cost of ignoring numerical limits was demonstrated on 4 June 1996, when the first Ariane 5 rocket broke up about 40 seconds after lift-off. The inquiry board found that software reused from Ariane 4 converted a 64-bit floating-point value related to the horizontal velocity into a 16-bit signed integer; on the faster Ariane 5 trajectory the value was too large, the conversion raised an unhandled exception, and both inertial reference units shut down (Lions et al., 1996).

1.5.2 The standard model of rounding error

Let $\mathrm{fl}(x)$ denote the floating-point number nearest to x (round to nearest, ties to even).

Theorem 1.21 (Rounding error model). *If x lies within the normalised range then*

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \le u.$$

IEEE 754 requires that for $\circ \in \{+, -, \times, \div\}$ the computed result is the exactly rounded true result, so

$$\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta), \qquad |\delta| \le u. \tag{1.13}$$

Proof sketch. Let $2^e \le |x| < 2^{e+1}$. In this binade the floating-point numbers are equally spaced with gap 2^{e+1-t}, so the nearest one is within half a gap: $|\mathrm{fl}(x) - x| \le 2^{e-t} \le 2^{-t}\,|x| = u\,|x|$. Setting $\delta = (\mathrm{fl}(x) - x)/x$ gives the claim. The statement for operations is then the definition of correct rounding applied to the exact result $x \circ y$. □

The model says that each *individual* operation is accurate to a relative error of u. The danger lies in how errors combine. Three consequences recur throughout the book.

(i) Floating-point addition is commutative but *not associative*: $(a+b)+c \neq a+(b+c)$ in general, so the order of a summation affects the answer (Section 1.6 returns to the reproducibility consequences).

(ii) Errors accumulate. A product of n numbers computed left to right satisfies $\mathrm{fl}(\prod_i x_i) = \prod_i x_i\,(1+\theta_n)$ with $|\theta_n| \le \gamma_n := nu/(1-nu)$, a bound that grows linearly in n.

(iii) The *condition* of the problem determines how much an input perturbation is amplified. For a differentiable f, the relative condition number $\kappa_f(x) = \left|x f'(x)/f(x)\right|$ satisfies $\left|f(x(1+\delta)) - f(x)\right| / \left|f(x)\right| \approx \kappa_f(x)\,|\delta|$.

An algorithm is *backward stable* if its computed answer is the exact answer for slightly perturbed data (perturbations of order u). For such an algorithm the forward error is at most about $\kappa \times u$:

well-conditioned problems solved by stable algorithms give accurate answers, and nothing can rescue an ill-conditioned problem except reformulation or more precision.

1.5.3 Catastrophic cancellation

Subtraction of two nearby numbers is the classic way to lose accuracy. Suppose $\hat{x} = x(1+\delta_1)$ and $\hat{y} = y(1+\delta_2)$ carry small relative errors (from rounding or from measurement). Even if the subtraction itself is exact,

$$\frac{\left|(\hat{x}-\hat{y})-(x-y)\right|}{\left|x-y\right|} = \frac{\left|x\delta_1 - y\delta_2\right|}{\left|x-y\right|} \le \max\{|\delta_1|,|\delta_2|\}\,\frac{|x|+|y|}{\left|x-y\right|}. \tag{1.14}$$

The factor $(|x|+|y|)/|x-y|$ is the condition number of subtraction; it is enormous when $x \approx y$. The subtraction does not *create* error, it *reveals* the errors already present in the leading digits that cancel. Computing $(e^x-1)/x$ for $x = 10^{-12}$ subtracts 1 from $\mathrm{fl}(e^x) = 1 + 10^{-12}(1+\delta)$, where $|\delta|$ is up to $u/10^{-12} \approx 10^{-4}$: four digits are correct, twelve are lost, exactly as the error model $\varepsilon_{\text{mach}}/x$ in Fig. 1.5 predicts. The cure is algebraic rewriting into a form without the subtraction: library functions `np.expm1` and `np.log1p` compute e^x-1 and $\log(1+x)$ accurately for small x; $1-\cos x = 2\sin^2(x/2)$; and the small root of a quadratic is computed from the product of the roots, $x_2 = c/x_1$.

Listing 1.8: Catastrophic cancellation and its cures ch01/cancellation.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Catastrophic cancellation and how algebraic rewriting cures it.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

# (exp(x) - 1)/x -> 1 as x -> 0: naive formula versus expm1
print(f"{'x':>8s} {'naive (e^x-1)/x':>18s} {'expm1(x)/x':>18s}")
for x in [1e-4, 1e-8, 1e-12, 1e-15, 1e-16]:
    print(f"{x:8.0e} {(np.exp(x) - 1) / x:18.15f} {np.expm1(x) / x:18.15f}")

# (1 - cos x)/x^2 -> 1/2: rewrite as 2 sin^2(x/2)/x^2
x = 1.2e-8
print("\n(1-cos x)/x^2      at x=1.2e-8:", (1 - np.cos(x)) / x**2)
print("2 sin^2(x/2)/x^2   at x=1.2e-8:", 2 * np.sin(x / 2)**2 / x**2)

# roots of x^2 + b x + c with b >> c: textbook versus stable formula
b, c = 1e8, 1.0
disc = np.sqrt(b * b - 4 * c)
r_naive = (-b + disc) / 2
q = -0.5 * (b + np.sign(b) * disc)
r_stable = c / q
print("\nsmall root, textbook formula :", r_naive)
print("small root, stable c/q       :", r_stable, "  (true value about -1e-8)")
```

Output

```
       x    naive (e^x-1)/x         expm1(x)/x
   1e-04  1.000050001667141  1.000050001666708
   1e-08  0.999999993922529  1.000000005000000
   1e-12  1.000088900582341  1.000000000000500
   1e-15  1.110223024625157  1.000000000000000
   1e-16  0.000000000000000  1.000000000000000
```

```
(1-cos x)/x^2      at x=1.2e-8: 0.7709882115452477
2 sin^2(x/2)/x^2   at x=1.2e-8: 0.5

small root, textbook formula : -7.450580596923828e-09
small root, stable c/q       : -1e-08   (true value about -1e-8)
```

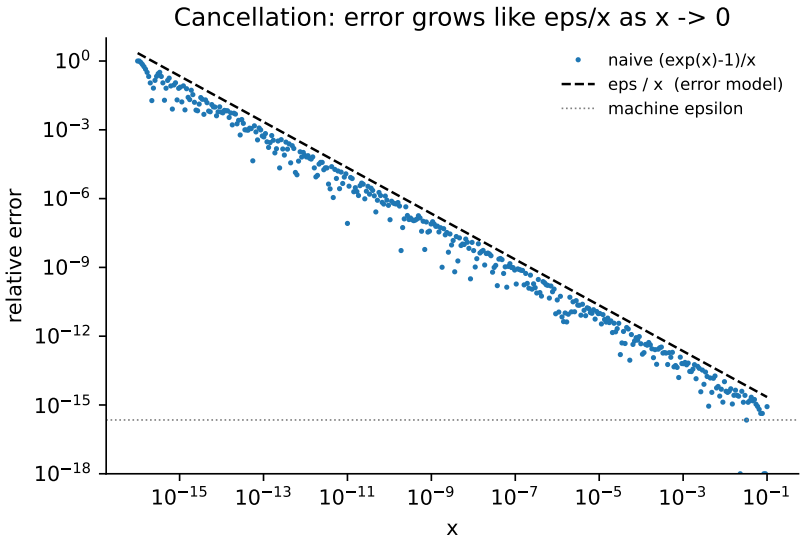


Figure 1.5: Relative error of the naive formula $(e^x - 1)/x$ computed in double precision. The error follows the model $\varepsilon_{\text{mach}}/x$ (dashed) and reaches 100% near $x = 10^{-16}$, while `np.expm1(x)/x` is accurate to machine precision throughout (script `ch01/cancellation_fig.py`).

△ Caution: Differences of large, nearly equal quantities

Cancellation lurks wherever a small quantity is computed as the difference of large ones: the change in a price index between two months computed from two separately rounded levels, a residual sum of squares computed as $\mathbf{y}^\top\mathbf{y} - \hat{\boldsymbol{\beta}}^\top\mathbf{X}^\top\mathbf{y}$, a finite-difference derivative $(f(x+h) - f(x))/h$ with tiny h, or the one-pass variance formula below. Whenever you see such a difference, ask whether it can be computed directly.

1.5.4 Overflow, underflow and the log-sum-exp trick

Probabilities in statistical models are often products of many factors or ratios of exponentials. Two numerically fragile expressions appear everywhere: the *log-sum-exp* $\operatorname{lse}(\mathbf{v}) = \log \sum_{k=1}^{K} e^{v_k}$ and the *softmax* $\sigma(\mathbf{v})_k = e^{v_k}/\sum_l e^{v_l}$. They define the multinomial logit model of discrete choice (transport mode, occupation, migration destination), the normalising constant of mixture models, and the output layer of every classification neural network. With $v_k \approx 800$, e^{v_k} overflows; with $v_k \approx -800$ it underflows to 0 and $\log 0 = -\infty$.

Proposition 1.22 (Log-sum-exp). *For any $c \in \mathbb{R}$, $\operatorname{lse}(\mathbf{v}) = c + \operatorname{lse}(\mathbf{v} - c\mathbf{1})$ and $\sigma(\mathbf{v}) = \sigma(\mathbf{v} - c\mathbf{1})$. With $c = m := \max_k v_k$,*

$$m \le \operatorname{lse}(\mathbf{v}) \le m + \log K,$$

and the shifted computation $m + \log \sum_k e^{v_k - m}$ cannot overflow and loses no significant terms.

Proof. $\log \sum_k e^{v_k} = \log\left(e^c \sum_k e^{v_k - c}\right) = c + \log \sum_k e^{v_k - c}$; the softmax identity follows because e^{-c}

cancels from numerator and denominator. After shifting by m every exponent is ≤ 0 and the largest equals 0, so each term lies in $(0, 1]$ and their sum lies in $[1, K]$, giving the bounds and ruling out both overflow and the loss of the dominant term. □

For the binary logit log-likelihood $\ell(\beta) = \sum_i [y_i \eta_i - \log(1 + e^{\eta_i})]$ with $\eta_i = \mathbf{x}_i^\top \beta$, the term $\log(1 + e^{\eta})$ (the "softplus") should be computed as `np.logaddexp(0, eta)`, which applies the same shift; computing $p_i = 1/(1 + e^{-\eta_i})$ first and then $\log p_i$ and $\log(1 - p_i)$ fails as soon as some p_i rounds to exactly 0 or 1.

Listing 1.9: Stable softmax, log-sum-exp and logit log-likelihood ch01/logsumexp.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Stable softmax, log-sum-exp and logit log-likelihoods.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy.special import logsumexp, expit

# utilities of 4 transport modes for one traveller (simulated, large scale)
v = np.array([800.0, 805.0, 790.0, 802.0])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(v) / np.exp(v).sum()
print("naive softmax  :", naive)


def softmax(v):
    """Shift by the maximum before exponentiating: exact and safe."""
    z = np.exp(v - v.max())
    return z / z.sum()


def lse(v):
    m = v.max()
    return m + np.log(np.exp(v - m).sum())


print("stable softmax :", softmax(v).round(6))
print("sum of probs   :", softmax(v).sum())
print("log-sum-exp    : ours = %.10f   scipy = %.10f" % (lse(v), logsumexp(v)))
print("log P(mode 2)  = v_2 - lse(v) = %.6f" % (v[1] - lse(v)))
v_small = np.array([-800.0, -805.0, -790.0])
with np.errstate(divide="ignore"):
    print("naive log-sum-exp of very negative utilities:",
          np.log(np.exp(v_small).sum()), "  stable:", round(lse(v_small), 6))

# binary logit log-likelihood: log(1 + exp(eta)) via logaddexp
rng = np.random.default_rng(803)
n = 1000
x = rng.normal(0, 1, n)
eta = 40 * x                                    # a very well separated model
y = (rng.uniform(size=n) < expit(eta)).astype(float)
with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
    p = 1 / (1 + np.exp(-eta))
    ll_naive = np.sum(y * np.log(p) + (1 - y) * np.log(1 - p))
ll_stable = np.sum(y * eta - np.logaddexp(0, eta))
print("binary logit log-likelihood, naive  :", ll_naive)
print("binary logit log-likelihood, stable :", round(ll_stable, 6))
print("log1p(1e-20) =", np.log1p(1e-20), "  log(1 + 1e-20) =", np.log(1 + 1e-20))
```

Output

```
naive softmax  : [nan nan nan nan]
stable softmax : [0.006377 0.946499 0.       0.047123]
sum of probs   : 1.0000000000000002
log-sum-exp    : ours = 805.0549855249   scipy = 805.0549855249
log P(mode 2)  = v_2 - lse(v) = -0.054986
naive log-sum-exp of very negative utilities: -inf   stable: -789.999954
binary logit log-likelihood, naive  : nan
binary logit log-likelihood, stable : -38.233852
log1p(1e-20) = 1e-20   log(1 + 1e-20) = 0.0
```

Graduate Perspective: Accuracy of log-sum-exp and softmax

Blanchard et al. (2021) give a rounding-error analysis of the shifted algorithms, showing that they are accurate (the computed result has a relative error bounded by a modest multiple of u times the condition number of the problem) and comparing several variants used in practice. The same shifting idea underlies the forward–backward algorithm for hidden Markov models (scaled recursions), the computation of marginal likelihoods in Bayesian model comparison, and importance-sampling weights in Chapter 10, where one always works with log-weights and normalises with log-sum-exp.

1.5.5 Summation error and compensated (Kahan) summation

Sums are the most common computation in statistics. Let $\hat{S}_k = \mathrm{fl}(\hat{S}_{k-1} + x_k)$ be the *recursive* (left-to-right) sum. Applying (1.13) at every step and collecting terms gives the classical bound (Higham, 2002, Chapter 4)

$$\left|\hat{S}_n - \sum_{i=1}^{n} x_i\right| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad \gamma_{n-1} = \frac{(n-1)u}{1-(n-1)u} \approx nu. \tag{1.15}$$

The error can grow linearly with n; it is largest relative to the answer when the terms have mixed signs (the sum is small compared with $\sum |x_i|$) or when a running total is much larger than the terms being added, which then lose their low-order digits. Two remedies are standard. *Pairwise* summation adds neighbours, then pairs of pairs, and so on, replacing γ_{n-1} by $\gamma_{\lceil \log_2 n \rceil}$; NumPy's `np.sum` uses a blocked version of it. *Compensated summation* keeps a running correction for the low-order bits lost at each step.

Algorithm 1 Kahan compensated summation

```
s ← 0, c ← 0                          ▷ c holds the negative of the lost low-order part
for i = 1, ..., n do
    y ← x_i − c                       ▷ correct the new term
    t ← s + y                         ▷ low-order digits of y are lost here
    c ← (t − s) − y                   ▷ algebraically zero; numerically, the lost part
    s ← t
end for
return s
```

Kahan's algorithm satisfies $\left|\hat{S}_n - S_n\right| \le \left(2u + O(nu^2)\right) \sum_i |x_i|$: the leading error term no longer depends on n (Kahan, 1965; Higham, 1993). Note that an "optimising" compiler that simplifies $(t - s) - y$ to 0 destroys the algorithm, a reminder that floating-point expressions must not be

rearranged as if they were real-number algebra. Python's `math.fsum` goes further and returns the correctly rounded sum.

Listing 1.10: Naive, pairwise, Kahan and exactly rounded summation ch01/summation.py

```python
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Rounding error in long sums: naive loop, Kahan, pairwise (NumPy) and fsum.
# (c) 2026 Stephen E. Moore. MIT Licence.
import math
import numpy as np


def naive_sum(x, zero=0.0):
    s = zero
    for xi in x:
        s += xi
    return s


def kahan_sum(x, zero=0.0):
    """Compensated summation (Kahan, 1965)."""
    s, c = zero, zero
    for xi in x:
        y = xi - c            # correct the next term by the lost low part
        t = s + y             # big + small: low-order digits of y are lost
        c = (t - s) - y       # recover what was lost (algebraically zero)
        s = t
    return s


n = 1_000_000
x = np.full(n, 0.1)
exact = 100_000.0                       # the intended sum of the decimal 0.1s
print(f"summing 0.1 one million times (target {exact})")
for name, val in [("naive loop", naive_sum(x)), ("Kahan", kahan_sum(x)),
                  ("np.sum (pairwise)", np.sum(x)), ("math.fsum", math.fsum(x))]:
    print(f"  {name:18s} {val:.10f}   error {val - exact: .3e}")

x32 = x.astype(np.float32)
print("same data in float32:")
print(f"  {'naive loop':18s} {float(naive_sum(x32, np.float32(0))):.4f}")
print(f"  {'Kahan':18s} {float(kahan_sum(x32, np.float32(0))):.4f}")
print(f"  {'np.sum (pairwise)':18s} {float(np.sum(x32)):.4f}")

# order matters: a large total plus many small amounts (e.g. a tax ledger)
amounts = np.r_[1e16, np.ones(10_000)]
print("1e16 then 10,000 ones, left to right:", naive_sum(amounts) - 1e16)
print("small ones first                    :", naive_sum(amounts[::-1]) - 1e16)
print("math.fsum (correctly rounded)       :", math.fsum(amounts) - 1e16)
```

Output

```
summing 0.1 one million times (target 100000.0)
  naive loop         100000.0000013329   error  1.333e-06
  Kahan              100000.0000000000   error  0.000e+00
  np.sum (pairwise)  99999.9999999998   error -2.037e-10
```

```
  math.fsum          100000.0000000000   error  0.000e+00
same data in float32:
  naive loop         100958.3438
  Kahan              100000.0000
  np.sum (pairwise)  100000.0859
1e16 then 10,000 ones, left to right: 0.0
small ones first                    : 10000.0
math.fsum (correctly rounded)       : 10000.0
```

In double precision, a million additions of 0.1 drift by about 1.3×10^{-6}, comfortably inside the worst-case bound (1.15) of about 10^{-5}, while pairwise summation is several thousand times more accurate; in single precision the naive loop is off by almost 1%, while Kahan's algorithm is exact to the printed digits. The last three lines show that adding 10,000 ones to 10^{16} one at a time has *no effect at all*, because each 1 is below half the spacing of doubles near 10^{16}.

Historical Note: The Vancouver Stock Exchange index

In January 1982 the Vancouver Stock Exchange launched a new index with a starting value of 1000.000. It was recomputed after every trade, about three thousand times a day, and each new value was *truncated* (not rounded) to three decimal places. Every update therefore lost on average 0.0005 index points. By November 1983 the index stood at around 520, even though the market had not fallen; when the error was discovered and the index was recomputed correctly, its value was close to 1100. The simulation below reproduces the mechanism with made-up price changes.

Output of `ch01/vancouver.py`: truncation versus rounding

```
index updates simulated       : 1,232,000
final index, full precision   :    978.929
final index, rounded (3 dp)   :    979.487
final index, truncated (3 dp) :    369.388
truncation drift              :   -609.541
predicted drift ~ -0.0005 x m :     -616.0
```

The lesson is that *unbiased* rounding errors (round to nearest) tend to cancel, growing like $\sqrt{m}$ for m operations if they behave like independent random errors, whereas *biased* errors (truncation, or always rounding halves up) grow like m.

1.5.6 Computing a variance: one pass, two passes, or Welford

Proposition 1.2 gives two algebraically equivalent formulas for the sum of squared deviations $SS = \sum_i (x_i - \bar{x})^2$:

$$\text{two-pass: } SS = \sum_i (x_i - \bar{x})^2, \qquad \text{one-pass (“textbook”): } SS = \sum_i x_i^2 - \frac{1}{n}\Big(\sum_i x_i\Big)^2 .$$

The one-pass formula is attractive because it needs only one sweep through the data, and it appears in many textbooks and some old software. It is numerically disastrous when the mean is large relative to the standard deviation: it subtracts two large, nearly equal numbers, each of which carries a rounding error of order $u \sum_i x_i^2$. By (1.14) the relative error in SS is of order

$$u \, \frac{\sum_i x_i^2}{SS} = u\Big(1 + \frac{n\bar{x}^2}{SS}\Big) \approx u\,\Big(1 + \frac{\bar{x}^2}{s^2}\Big), \tag{1.16}$$

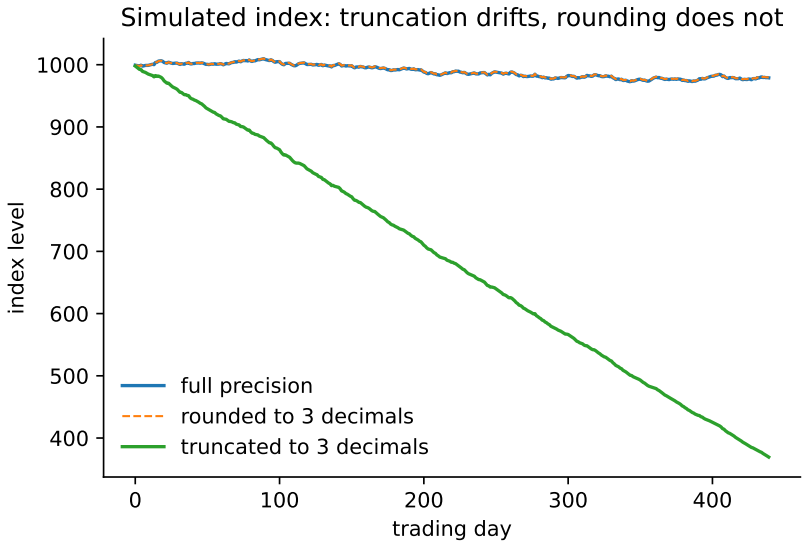


Figure 1.6: A simulated index updated 2800 times per trading day. Rounding to three decimals (dashed) tracks the exact index; truncation loses about 0.0005 points per update, a systematic drift of more than 600 points over 22 months.

where $s^2 = SS/(n-1)$. The quantity $\kappa = \sqrt{1 + n\bar{x}^2/SS}$ is the condition number of the variance computation (Chan et al., 1983). Incomes recorded in cedis with mean 10^6 and standard deviation 1 (for instance, a narrow band of salaries recorded to the cedi) have $\bar{x}^2/s^2 = 10^{12}$, so about 12 of 16 digits are lost; at mean 10^8 all are lost and the result can even be *negative*.

The two-pass algorithm first computes $\bar{x}$, then sums squared deviations; its error is of order nu (or $u \log n$ with pairwise sums), essentially independent of $\bar{x}$. When the data can be read only once (streaming data, or an administrative file too large for memory), *Welford's algorithm* updates the mean and the sum of squares together (Welford, 1962):

$$\bar{x}_k = \bar{x}_{k-1} + \frac{x_k - \bar{x}_{k-1}}{k}, \qquad M_k = M_{k-1} + (x_k - \bar{x}_{k-1})(x_k - \bar{x}_k), \tag{1.17}$$

with $\bar{x}_0 = M_0 = 0$, so that $M_n = SS$. To verify the second recursion, write $d = x_k - \bar{x}_{k-1}$, so that $x_k - \bar{x}_k = d(k-1)/k$; adding x_k to a sample of size $k-1$ changes the sum of squared deviations by $d^2(k-1)/k$, as a direct expansion shows. Welford's recursion works with deviations from a running mean and so avoids the cancellation.

Listing 1.11: Three variance algorithms under a growing offset ch01/variance_algorithms.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# One-pass (textbook), two-pass and Welford variance under a large offset.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n = 10_000
z = rng.standard_normal(n)
z = (z - z.mean()) / z.std(ddof=1)        # exact sample variance 1 (to ~1e-16)


def one_pass(x):
    """Textbook 'computing formula': (sum x^2 - n xbar^2)/(n - 1)."""
    s, s2 = x.sum(), (x * x).sum()
    return (s2 - s * s / x.size) / (x.size - 1)
```

```
16
17
18 def two_pass(x):
19     d = x - x.mean()
20     return (d @ d) / (x.size - 1)
21
22
23 def welford(x):
24     """Single pass, numerically stable update (Welford, 1962)."""
25     mean, m2 = 0.0, 0.0
26     for k, xk in enumerate(x, start=1):
27         delta = xk - mean
28         mean += delta / k
29         m2 += delta * (xk - mean)
30     return m2 / (len(x) - 1)
31
32
33 offsets = [0, 1e2, 1e4, 1e6, 1e7, 1e8, 1e9]
34 print(f"{'offset':>8s} {'one-pass':>14s} {'two-pass':>12s} {'Welford':>12s}")
35 for c in offsets:
36     x = c + z                              # e.g. incomes measured in cedis
37     v = [one_pass(x), two_pass(x), welford(x.tolist())]
38     print(f"{c:8.0e} {v[0]:14.6f} {v[1]:12.9f} {v[2]:12.9f}")
39 x32 = (1e4 + z).astype(np.float32)
40 print("float32, offset 1e4: one-pass = %.4f, two-pass = %.6f"
41       % (one_pass(x32), two_pass(x32)))
```

Output

```
  offset       one-pass     two-pass      Welford
   0e+00       1.000000  1.000000000  1.000000000
   1e+02       1.000000  1.000000000  1.000000000
   1e+04       1.000000  1.000000000  1.000000000
   1e+06       1.000100  1.000000000  1.000000000
   1e+07       0.998500  1.000000000  1.000000000
   1e+08       0.000000  1.000000000  1.000000001
   1e+09       0.000000  1.000000000  0.999999999
float32, offset 1e4: one-pass = -9.6898, two-pass = 0.999995
```

△ Caution: A negative variance is a symptom, not a curiosity

In `float32` with an offset of only 10^4, the one-pass formula returned a *negative* variance. Any code that computes variances, covariance matrices or R^2 via "sum of squares minus n times mean squared" (including hand-written SQL aggregates on large databases and spreadsheet formulas) is exposed to this error. Centre first, or use Welford's update; if data must be aggregated in one pass, at least subtract a rough provisional mean (such as the first observation) from every value.

1.5.7 Comparing floats; single versus double precision

Because computed values carry rounding errors, testing `a == b` for floats is almost always wrong. NumPy's `np.isclose(a, b)` tests

$$|a - b| \le \texttt{atol} + \texttt{rtol} \cdot |b|, \qquad \texttt{rtol} = 10^{-5},\ \texttt{atol} = 10^{-8}\ \text{(defaults)},$$

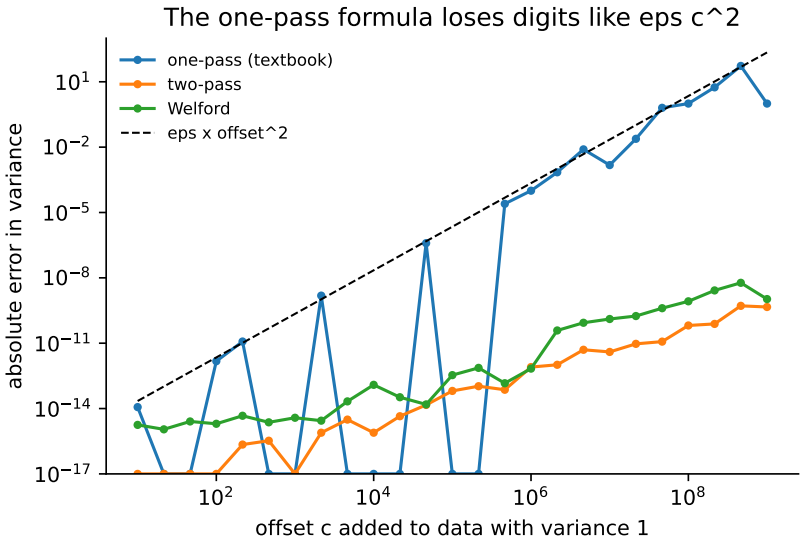


Figure 1.7: Absolute error in the computed variance of data with variance 1 after adding an offset c. The one-pass formula follows $\varepsilon_{\text{mach}} c^2$ (dashed) until it has no correct digits; the two-pass and Welford algorithms remain accurate (script `ch01/variance_fig.py`).

and `np.allclose` applies it to whole arrays. Two features deserve care. The absolute tolerance dominates for small numbers: by default 10^{-10} and 2×10^{-10} are "close", which is wrong if the quantities are, say, probabilities of rare events; set `atol=0` for purely relative comparison. And the test is asymmetric in a and b; Python's `math.isclose` uses a symmetric relative test with `rel_tol=1e-9`. The right tolerance is a modelling decision: it should reflect the conditioning of the computation and the precision actually needed.

Single precision halves memory and doubles throughput on vector units and graphics processors, and deep learning routinely trains in `float32` or even 16-bit formats, because stochastic gradients are noisy anyway. For statistics and econometrics, double precision should be the default: with only 7 significant digits, sums of millions of terms, sample variances, cross-products $\mathbf{X}^\top\mathbf{X}$ and solutions of moderately ill-conditioned systems lose most or all of their accuracy. Mixed-precision methods (storing data in `float32` but accumulating in `float64`) offer a compromise.

The script `ch01/rounding_money.py` contrasts Python's binary `round` with the `decimal` module (`Decimal("2.665").quantize(Decimal("0.01"), ROUND_HALF_UP)`), computes VAT on simulated invoices held as integer pesewas, and rounds published growth contributions:

Output of `ch01/rounding_money.py`

```
round(0.5), round(1.5), round(2.5) = 0 2 2   (round half to even)
round(2.675, 2) = 2.67   because 2.675 is stored as 2.6749999999999982236
Decimal('2.665') half-up   -> 2.67
Decimal('2.665') half-even -> 2.66
Decimal(2.675) (from float) = 2.674999999999999822364316059974953532218933105468 75

VAT rounded per line   : 37502474 pesewas
VAT rounded on total   : 37502175 pesewas
difference             : 299 pesewas

sector contributions, rounded to 1 dp: [0.7 1.  1.  0.2]  sum = 2.9
aggregate growth rounded to 1 dp     : 3.0
```

Policy Lens: Rounding in official statistics and tax computation

Rounding is a policy choice as well as a numerical one. (1) *Money should not be stored in binary floating point*: amounts are kept as integers of the smallest unit (pesewas, cents) or as decimal numbers, and rounding rules (half up, half even) are set by law or regulation; Python's built-in `round` uses round-half-to-even, and `round(2.675, 2)` gives 2.67 because 2.675 is stored slightly below itself. (2) *Where* rounding happens matters: in the simulated ledger, rounding VAT on each of 10,000 invoice lines rather than on the total collects 299 pesewas more, because rounding halves up is biased; summed over millions of transactions such choices have revenue consequences. (3) Published tables are rounded independently, so *components need not add up to totals*: contributions to growth of $0.7 + 1.0 + 1.0 + 0.2 = 2.9$ points sit beside an aggregate of 3.0%. Statistics offices either publish a note to that effect or use controlled rounding. Analysts who re-use published rounded figures as inputs should expect such discrepancies and should never "correct" them by forcing components to sum.

⋆ Key Concept: Floating-point arithmetic in one paragraph

A double has 53 significant bits (≈ 16 decimal digits), unit roundoff $u \approx 1.1 \times 10^{-16}$ and range about $10^{\pm 308}$. Every operation obeys $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$, $|\delta| \le u$, so individual operations are accurate, but errors accumulate (sums: $\approx nu$), are amplified by ill-conditioning, and are exposed by cancellation. Rewrite formulas to avoid subtracting nearly equal numbers (`expm1`, `log1p`, two-pass or Welford variance), shift exponentials (log-sum-exp), sum carefully (pairwise, Kahan, `fsum`), compare with tolerances (`np.isclose`), and prefer `float64`.

Exercises

1.5.1 Theory Show that with round-to-nearest, $\mathrm{fl}(\mathrm{fl}(a+b)+c)$ and $\mathrm{fl}(a+\mathrm{fl}(b+c))$ can differ, by finding three doubles for which they do. Then prove that for the product of n numbers computed left to right, $\mathrm{fl}(\prod_i x_i) = \prod_i x_i(1+\theta_n)$ with $|\theta_n| \le \gamma_n$, assuming no overflow or underflow.

1.5.2 Theory Derive the relative condition numbers of $f(x) = \log x$, $f(x) = e^x$ and $f(x) = x - 1$. Where is each ill-conditioned? Explain why $\log(1+x)$ for small x is a well-conditioned *problem* that the naive *algorithm* `np.log(1 + x)` solves badly.

1.5.3 Theory Prove the Welford recursion (1.17) by showing that $SS_k - SS_{k-1} = \frac{k-1}{k}(x_k - \bar{x}_{k-1})^2$. Extend it to a recursion for the sample covariance of two variables, and to a formula for combining the means and sums of squares of two sub-samples (useful for parallel computation).

1.5.4 Python Compute $f(x) = \sqrt{x^2+1} - x$ for $x = 10^k$, $k = 0, \ldots, 9$, both directly and via the algebraically equivalent $1/(\sqrt{x^2+1} + x)$. Plot both relative errors and explain the result with (1.14).

1.5.5 Python Implement pairwise summation recursively and compare its error with naive, Kahan and `np.sum` on $n = 10^6$ uniform random numbers stored in `float32`, using `math.fsum` on the `float64` values as the reference. Plot error against n on log–log axes and estimate the growth rates.

1.5.6 Policy A social-protection programme pays each of 1.5 million beneficiaries a transfer equal to 12.5% of a reference amount that varies by household, rounded to the nearest pesewa. Simulate reference amounts and compare total payments under half-up, half-even and truncation rules. How large is the difference over a year of monthly payments, and which rule would you recommend to the ministry?

1.5.7 ★ Graduate Read Chan et al. (1983). Implement their pairwise ("updating") algorithm for the variance, which combines sub-sample statistics, and compare its accuracy and speed with Welford's algorithm on 10^7 observations with offset 10^8. Discuss which algorithm suits distributed computation over a partitioned administrative database.

1.6 Computational Cost

Precision is one constraint on computation; time and memory are the others. A method that is statistically ideal but needs a week of computing, or more memory than the machine has, is not a method we can use. This section develops the vocabulary for estimating cost before writing code.

1.6.1 Flop counts

A *flop* is one floating-point addition, subtraction, multiplication or division. Counting flops gives a machine-independent measure of the work in a numerical algorithm; combined with a machine's speed (in flops per second) it yields a time estimate.

Proposition 1.23 (Costs of basic operations)**.** *For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times r}$: the inner product $\mathbf{x}^\top\mathbf{y}$ costs $2n-1$ flops; the matrix–vector product $\mathbf{Ax}$ costs $m(2n-1) \approx 2mn$; and the matrix product $\mathbf{AB}$ costs $mr(2n-1) \approx 2mnr$.*

Proof. An inner product requires n multiplications and $n-1$ additions. $\mathbf{Ax}$ consists of m inner products of length n (entry view), and $\mathbf{AB}$ of mr such inner products. □

Table 1.2 lists the leading-order costs of the operations used most in this book (Golub and Van Loan, 2013; Trefethen and Bau, 1997). Only the leading term matters for large problems, which is why costs are quoted in $O(\cdot)$ form.

Table 1.2: Leading-order flop counts for dense matrix computations ($\mathbf{A} \in \mathbb{R}^{n\times n}$, $\mathbf{X} \in \mathbb{R}^{n\times p}$, $n \geq p$).

Operation	Flops	Order
inner product, $\mathbf{x}^\top\mathbf{y}$	$2n$	$O(n)$
matrix–vector, $\mathbf{Ax}$	$2n^2$	$O(n^2)$
triangular solve, $\mathbf{Tx} = \mathbf{b}$	n^2	$O(n^2)$
matrix product, $\mathbf{AB}$	$2n^3$	$O(n^3)$
LU factorisation / `solve`	$\frac{2}{3}n^3$	$O(n^3)$
Cholesky factorisation	$\frac{1}{3}n^3$	$O(n^3)$
explicit inverse, $\mathbf{A}^{-1}$	$2n^3$	$O(n^3)$
cross-product $\mathbf{X}^\top\mathbf{X}$ (symmetric)	np^2	$O(np^2)$
least squares by QR	$2np^2 - \frac{2}{3}p^3$	$O(np^2)$
symmetric eigen-decomposition	$\approx 9n^3$	$O(n^3)$

Two practical consequences follow. First, *the order of multiplication matters*. For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\mathbf{x} \in \mathbb{R}^n$, $(\mathbf{AB})\mathbf{x}$ costs $2n^3 + 2n^2$ flops but $\mathbf{A}(\mathbf{Bx})$ only $4n^2$: for $n = 10^4$ that is 2×10^{12} versus 4×10^8 operations, minutes versus milliseconds. Similarly $\mathbf{X}^\top\mathbf{y}$ should be computed as `X.T @ y`, never as a product involving an $n\times n$ matrix such as $\mathbf{H}$ or the hat matrix $\mathbf{P}$. Second, for regression with $n \gg p$ the cost $O(np^2)$ is *linear* in the number of observations, so least squares scales well to large samples; the bottleneck appears when p is large.

Graduate Perspective: Beyond n^3

Strassen showed in 1969 that two $n \times n$ matrices can be multiplied in $O(n^{\log_2 7}) \approx O(n^{2.81})$ operations by a clever recursive scheme (Strassen, 1969), and the theoretical exponent has since been pushed below 2.38. These algorithms are rarely used in practice: their constants are large and their numerical stability is weaker. Of much greater practical importance for data science are *randomised* algorithms, which use random projections to compute approximate low-rank factorisations of huge matrices in close to $O(np \log k)$ time, and iterative methods (conjugate gradients, stochastic gradient descent) whose cost per iteration is one pass over the data (Chapter 8).

1.6.2 Vectorisation versus Python loops

Flop counts are not the whole story. A Python `for` loop executes each iteration in the interpreter, which checks types and allocates objects for every number; this costs tens of nanoseconds per element, independent of how simple the arithmetic is. A vectorised NumPy call hands the whole array to compiled code, and matrix products are passed to an optimised BLAS library that uses vector instructions, several cores and careful management of the memory cache. The difference is typically two to four orders of magnitude.

The next script restricts the linear-algebra library to a single thread (by setting environment variables before NumPy is imported), so that the measured scaling is not distorted by multithreading.

Listing 1.12: Loops versus vectorised code; cubic cost of dense algebra `ch01/timing_vectorise.py`

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Python loops versus vectorised NumPy, and the O(n^3) cost of matrix products.
# (c) 2026 Stephen E. Moore. MIT Licence.
import os
os.environ["OPENBLAS_NUM_THREADS"] = "1"   # one thread: clean scaling
os.environ["OMP_NUM_THREADS"] = "1"
import time
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False})
rng = np.random.default_rng(803)


def best_time(f, repeat=3):
    """Best wall-clock time of several runs (reduces timing noise)."""
    out = []
    for _ in range(repeat):
        t0 = time.perf_counter()
        f()
        out.append(time.perf_counter() - t0)
    return min(out)


x = rng.standard_normal(1_000_000)
t_loop = best_time(lambda: sum(xi * xi for xi in x), repeat=1)
t_vec = best_time(lambda: x @ x)
print(f"sum of squares, n = 1e6: loop {t_loop:.3f}s, vectorised {t_vec:.5f}s,"
      f" speed-up ~{t_loop / t_vec:,.0f}x")
```

```

sizes = np.array([200, 400, 800, 1200, 1600])
t_mm, t_solve = [], []
for n in sizes:
    M = rng.standard_normal((n, n))
    b = rng.standard_normal(n)
    np.linalg.solve(M, b)                    # warm-up call
    t_mm.append(best_time(lambda: M @ M, repeat=5))
    t_solve.append(best_time(lambda: np.linalg.solve(M, b), repeat=5))
t_mm, t_solve = np.array(t_mm), np.array(t_solve)
print(f"{'n':>6s} {'matmul (s)':>11s} {'GFLOP/s':>8s} {'solve (s)':>10s}")
for n, a, s in zip(sizes, t_mm, t_solve):
    print(f"{n:6d} {a:11.4f} {2 * float(n)**3 / a / 1e9:8.1f} {s:10.4f}")
slope = np.polyfit(np.log(sizes[1:]), np.log(t_mm[1:]), 1)[0]
print(f"fitted log-log slope for matmul time: {slope:.2f} (theory: 3)")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(sizes, t_mm, "o-", label="matrix product A @ B  (2n^3 flops)")
ax.loglog(sizes, t_solve, "s-", label="solve Ax = b  (2n^3/3 flops)")
ref = t_mm[-1] * (sizes / sizes[-1]) ** 3
ax.loglog(sizes, ref, "k--", lw=1, label="slope 3 reference")
ax.set_xlabel("matrix dimension n")
ax.set_ylabel("seconds (best of 5)")
ax.set_title("Dense linear algebra cost grows like n^3")
ax.legend(frameon=False, fontsize=8)
plt.savefig("ch01_timing.pdf", bbox_inches="tight")
```

Output

```
sum of squares, n = 1e6: loop 0.217s, vectorised 0.00084s, speed-up ~257x
     n  matmul (s)  GFLOP/s  solve (s)
   200      0.0006     28.7     0.0004
   400      0.0044     29.3     0.0026
   800      0.0352     29.1     0.0183
  1200      0.1314     26.3     0.0638
  1600      0.3954     20.7     0.1722
fitted log-log slope for matmul time: 3.22 (theory: 3)
```

Timings are noisy: they depend on the processor, the BLAS library, the number of threads and other programs running at the same time. Report the best (or median) of several repetitions, and draw conclusions from *scaling* (slopes on a log–log plot) rather than from single numbers. The achieved rate of tens of GFLOP/s for matrix products, against perhaps 10^7 elementary operations per second for interpreted loops, is the reason vectorised code is the norm in scientific Python (Harris et al., 2020).

1.6.3 Memory

Memory is often the binding constraint. A `float64` occupies 8 bytes, so a dense $n \times p$ matrix needs $8np$ bytes; a laptop with 16 GB of memory holds at most about 2×10^9 doubles, and in practice much less because intermediate results (a centred copy, $\mathbf{X}^\top\mathbf{X}$, temporaries created by expressions such as `(X - mu) / sd`) also need space. Remedies include in-place operations (`X -= mu`), processing data in chunks with streaming algorithms such as Welford's, smaller types where precision permits (`float32`, `int8` for dummy variables) and sparse storage.

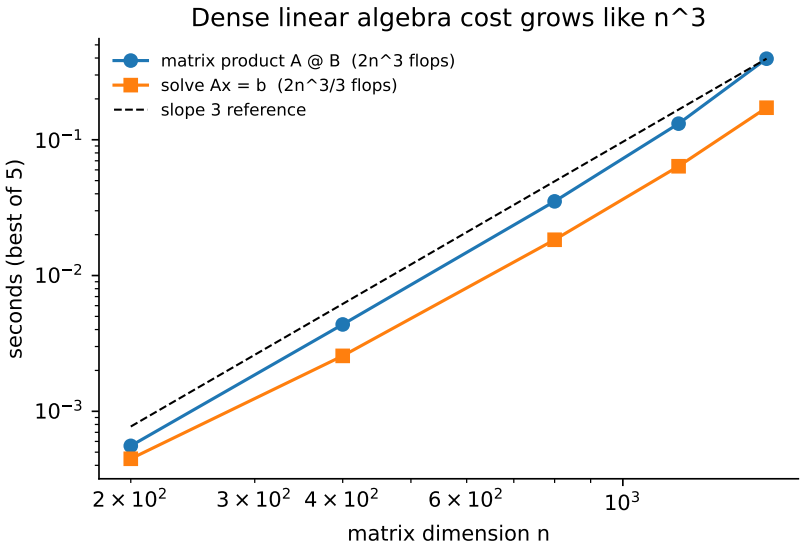


Figure 1.8: Wall-clock time of dense matrix products and linear solves on log–log axes. The slope is roughly the theoretical value 3; for small n, fixed overheads and multithreading distort the picture, and individual points are noisy. Timings depend on the machine (and on whatever else it is running); the slope does not.

△ Caution: Integer types and silent overflow

Integer arrays in NumPy have fixed width and wrap around on overflow without any warning (Section 1.5). On Windows, NumPy versions before 2.0 use 32-bit integers by default, so `np.array([1200])**3 * 2` silently overflows the range $\pm 2.1 \times 10^9$ and becomes negative. A first draft of `timing_vectorise.py` printed negative GFLOP/s rates for exactly this reason. Convert counts to `float` or specify `dtype=np.int64` before computing large products of integers, such as flop counts, populations times rates, or monetary amounts in pesewas.

1.6.4 Sparse matrices for large administrative data

Many matrices in policy analytics are *sparse*: most entries are zero. Dummy-variable design matrices (one non-zero per row per categorical variable), employer–employee links, input–output tables at a fine level of disaggregation, trade networks, and transition matrices between geographical units are typical. Storing the zeros is wasteful, and multiplying by them is wasted work.

Definition 1.24 (Compressed sparse row format)**.** The CSR representation of $\mathbf{A} \in \mathbb{R}^{m \times n}$ with nnz non-zero entries stores three arrays: `data` (the non-zero values, row by row), `indices` (their column indices) and `indptr` (length $m+1$; row i occupies positions `indptr`$[i]$ to `indptr`$[i+1]-1$). Storage is $O(\text{nnz} + m)$ and a matrix–vector product costs $2\,\text{nnz}$ flops instead of $2mn$.

Listing 1.13: A sparse employer–employee matrix (simulated register) ch01/sparse_admin.py

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Sparse matrices for a simulated employer-employee administrative dataset.
# (c) 2026 Stephen E. Moore. MIT Licence.
import time
import numpy as np
```

```python
import scipy.sparse as sp

rng = np.random.default_rng(803)
n_workers, n_firms, n_dist = 500_000, 20_000, 250
firm = rng.integers(0, n_firms, n_workers)            # employer of each worker
district = rng.integers(0, n_dist, n_firms)[firm]     # firm location
wage = rng.lognormal(7.0, 0.6, n_workers)             # simulated monthly wage

# incidence matrix F: F[i, j] = 1 if worker i is employed by firm j
F = sp.csr_matrix((np.ones(n_workers), (np.arange(n_workers), firm)),
                  shape=(n_workers, n_firms))
dense_gb = n_workers * n_firms * 8 / 1e9
sparse_mb = (F.data.nbytes + F.indices.nbytes + F.indptr.nbytes) / 1e6
print(f"F is {F.shape[0]:,} x {F.shape[1]:,} with {F.nnz:,} non-zeros"
      f" (density {F.nnz / np.prod(F.shape):.1e})")
print(f"dense storage would need {dense_gb:,.0f} GB; CSR uses {sparse_mb:.1f} MB")

t0 = time.perf_counter()
size = F.T @ np.ones(n_workers)                       # workers per firm
mean_wage = (F.T @ wage) / np.maximum(size, 1)        # average wage per firm
t1 = time.perf_counter()
print(f"firm sizes and mean wages via F^T x in {1e3 * (t1 - t0):.1f} ms")
print(f"largest firm: {int(size.max())} workers; mean firm size {size.mean():.1f}")
check = np.bincount(firm, weights=wage, minlength=n_firms) / np.maximum(size, 1)
print("agrees with np.bincount:", np.allclose(mean_wage, check))

# district dummy design matrix and its cross-product
D = sp.csr_matrix((np.ones(n_workers), (np.arange(n_workers), district)),
                  shape=(n_workers, n_dist))
DtD = (D.T @ D).toarray()
print("D^T D is diagonal:", np.count_nonzero(DtD - np.diag(np.diag(DtD))) == 0,
      "  diagonal = workers per district, e.g.", np.diag(DtD)[:4].astype(int))
coef = (D.T @ np.log(wage)) / np.diag(DtD)            # OLS on dummies = means
print("OLS on district dummies = district means of log wage:",
      np.allclose(coef[0], np.log(wage)[district == 0].mean()))

# worker-to-worker co-employment network among a 2,000-worker sample
S = F[:2000]
Co = S @ S.T                                          # co-worker indicator
print(f"co-employment matrix (2000 x 2000): {Co.nnz:,} non-zeros,"
      f" format {Co.format}")
```

Output

```
F is 500,000 x 20,000 with 500,000 non-zeros (density 3.5e-04)
dense storage would need 80 GB; CSR uses 8.0 MB
firm sizes and mean wages via F^T x in 7.3 ms
largest firm: 46 workers; mean firm size 25.0
agrees with np.bincount: True
D^T D is diagonal: True   diagonal = workers per district, e.g. [1900 2120 2050 2040]
OLS on district dummies = district means of log wage: True
co-employment matrix (2000 x 2000): 2,220 non-zeros, format csr
```

Policy Lens: Large administrative data sets

Tax records, social-security contributions, health insurance claims, school censuses and mobile money records are increasingly used to complement surveys. They are large (n in the millions) and naturally sparse in their relational structure. In the simulated register of 500,000 workers and 20,000 firms, the dense worker-by-firm incidence matrix would need 80 GB, but the CSR version needs 8 MB, and firm sizes and average wages for all firms are computed in milliseconds as $\mathbf{F}^\top\mathbf{1}$ and $\mathbf{F}^\top\mathbf{w}$. Linked employer–employee data of this kind underlie two-way fixed-effects models of wage determination, whose normal equations are sparse linear systems with millions of unknowns, solvable only with sparse or iterative methods (Davis, 2006). Two cautions apply: such data are confidential, so analysis must follow the data-protection law and the statistics office's access rules; and administrative coverage (formal workers only) is itself a source of bias.

△ Caution: Operations that destroy sparsity

Centring a sparse matrix ($\mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$), inverting it, or forming certain products makes it dense: the inverse of a sparse matrix is generally dense, and $\mathbf{F}\mathbf{F}^\top$ for a worker–firm matrix has a dense block for every large firm. Keep sparse matrices sparse: absorb means into the model rather than centring, solve with sparse factorisations (`scipy.sparse.linalg`) rather than inverting, and check `.nnz` before calling `.toarray()`.

1.6.5 Reproducibility

A computational result is *reproducible* if someone else (or you, a year later) can obtain the same numbers from the same data and code. For policy analysis, where numbers feed into budgets and public debate, this is a professional obligation (Peng, 2011). The main ingredients are:

(i) *Seeds.* Create one generator, `rng = np.random.default_rng(803)`, at the top of a script and pass it to every function that needs randomness. For parallel work, derive independent streams with `np.random.SeedSequence(803).spawn(k)` rather than using seeds $1, 2, \ldots, k$.

(ii) *Environments.* Record the versions of Python and of every package (a `requirements.txt` or conda `environment.yml`); algorithms and defaults change between versions.

(iii) *Determinism of arithmetic.* Because floating-point addition is not associative, multithreaded reductions or a different BLAS library can change the last digits of a result. Results should therefore be compared with tolerances, and tables should report only meaningful digits.

(iv) *Scripts, not clicks.* The full pipeline from raw data to published table should be a script under version control, as in this book's `run_all.py` (Wilson et al., 2014).

Listing 1.14: Seeds, independent streams, versions and summation order `ch01/reproducibility.py`

```
# DEP 803 -- Chapter 1: Notation, Vectors, Matrices and Computational Precision
# Seeds, independent random streams, recorded versions and summation order.
# (c) 2026 Stephen E. Moore. MIT Licence.
import sys
import numpy as np
import scipy
import pandas as pd

# 1. the same seed gives the same draws
a = np.random.default_rng(803).normal(size=3)
b = np.random.default_rng(803).normal(size=3)
c = np.random.default_rng(804).normal(size=3)
```

```python
print("seed 803:", a.round(6))
print("seed 803 again identical:", np.array_equal(a, b),
      "  seed 804 differs:", not np.array_equal(a, c))

# 2. independent streams for parallel workers (e.g. bootstrap replications)
ss = np.random.SeedSequence(803)
streams = [np.random.default_rng(s) for s in ss.spawn(3)]
print("first draw of 3 spawned streams:",
      [round(g.uniform(), 6) for g in streams])

# 3. record the computational environment alongside results
print("python", sys.version.split()[0], "| numpy", np.__version__,
      "| scipy", scipy.__version__, "| pandas", pd.__version__)

# 4. floating-point addition is not associative: order changes the answer
rng = np.random.default_rng(803)
x = rng.lognormal(10, 2, 1_000_000)                  # skewed "incomes"
s1 = np.sum(x)
s2 = np.sum(rng.permutation(x))
s3 = np.sum(np.sort(x))
print("total, original order : %.6f" % s1)
print("total, shuffled order : %.6f" % s2)
print("total, sorted order   : %.6f" % s3)
print("relative differences  : %.1e, %.1e" % ((s2 - s1) / s1, (s3 - s1) / s1))
print("(0.1 + 0.2) + 0.3 == 0.1 + (0.2 + 0.3) ?",
      (0.1 + 0.2) + 0.3 == 0.1 + (0.2 + 0.3))
```

Output

```
seed 803: [-0.298183 -0.122465 -0.253373]
seed 803 again identical: True   seed 804 differs: True
first draw of 3 spawned streams: [0.160771, 0.296841, 0.682341]
python 3.9.13 | numpy 1.23.5 | scipy 1.9.3 | pandas 1.5.2
total, original order : 162075748960.387360
total, shuffled order : 162075748960.387329
total, sorted order   : 162075748960.387360
relative differences  : -1.9e-16, 0.0e+00
(0.1 + 0.2) + 0.3 == 0.1 + (0.2 + 0.3) ? False
```

⋆ Key Concept: Cost, memory and reproducibility

Count flops before computing: inner products $O(n)$, matrix–vector products $O(n^2)$, dense products, solves and decompositions $O(n^3)$, least squares $O(np^2)$; order matrix products to keep intermediate results small. Vectorised NumPy is orders of magnitude faster than Python loops. Dense storage costs $8np$ bytes; sparse (CSR) storage costs $O(\text{nnz})$ and is essential for administrative data. Fix seeds, record environments, and remember that summation order can change the last digits.

Exercises

1.6.1 Theory Show that computing $\mathbf{x}^\top \mathbf{A}\mathbf{y}$ for $\mathbf{A} \in \mathbb{R}^{n\times n}$ costs about $2n^2$ flops whichever way it is parenthesised, but that for $\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_k\mathbf{x}$ the cost ranges from $O(kn^2)$ to $O(kn^3)$ depending on the order. For a chain of matrices of sizes 10×1000, 1000×10, 10×1000, find the optimal parenthesisation.

1.6.2 Theory Derive the leading-order flop count $\frac{2}{3}n^3$ for Gaussian elimination by summing the work at each elimination step, $\sum_{k=1}^{n-1} 2(n-k)^2$.

1.6.3 Python Time `np.linalg.inv(A) @ b` against `np.linalg.solve(A, b)` for $n = 500, 1000, 2000$, and compare the residuals $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|$ for a matrix with condition number about 10^{10} (construct it as $\mathbf{U}\,\mathrm{diag}(\mathbf{s})\mathbf{V}^\top$ with random orthogonal $\mathbf{U}, \mathbf{V}$).

1.6.4 Python Extend `sparse_admin.py`: add a second categorical variable (occupation, 300 levels) and construct the sparse design matrix $(\mathbf{1}, \mathbf{D}_{\text{district}}, \mathbf{D}_{\text{occ}})$ dropping one level of each. Solve the least-squares problem for log wages with `scipy.sparse.linalg.lsqr` and compare the time with a dense solve on a 50,000-worker subsample.

1.6.5 Python Write a seeded script that computes $\mathbf{X}^\top\mathbf{X}$ for a large random $\mathbf{X}$ and run it twice, with the environment variable `OMP_NUM_THREADS` set to 1 and then to 8. Do the results agree to the last bit? What tolerance would you use in an automated test?

1.6.6 Policy A ministry wants monthly district-level unemployment rates from a register of 8 million contributors linked to 260 districts and 40,000 employers. Estimate the memory needed for dense and sparse versions of the contributor-by-district and contributor-by-employer matrices, and the time to compute all district rates. Draft a short note on what must be documented so that the published series is reproducible.

1.6.7 ★ Graduate Two-way fixed-effects (AKM) wage models $\log w_{it} = \alpha_i + \psi_{J(i,t)} + \varepsilon_{it}$ lead to normal equations with a sparse matrix whose structure is the worker–firm graph. Explain why the system is singular unless one restricts to a connected set of firms, how connected components can be found from the sparse incidence matrix, and why iterative solvers such as conjugate gradients are preferred to factorisation.

Chapter Summary

- **Notation.** Indices i (observations), j (variables), t (time); bold lower-case column vectors, bold capitals for matrices, hats for estimators. The data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$ has rows $\mathbf{x}_i^\top$ (records) and columns $\mathbf{x}^{(j)}$ (variables). Sums are reductions along axes; products of many small numbers belong on the log scale. O, o, Θ describe growth; order-of-magnitude arithmetic predicts cost.
- **Vectors.** Inner products give weighted means, totals, predictions and price indices (Laspeyres $= 100\,\langle \mathbf{w}, \mathbf{r}\rangle$). The ℓ_1, ℓ_2, ℓ_∞ norms are equivalent up to factors depending on n. Cauchy–Schwarz, $|\mathbf{x}^\top\mathbf{y}| \le \|\mathbf{x}\|\,\|\mathbf{y}\|$, makes angles, cosine similarity and correlation well defined and implies the triangle inequality.
- **Matrices.** A product has entry, column, row and outer-product interpretations. $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$. Orthogonal matrices preserve length; projections are symmetric idempotent with eigenvalues $0, 1$ and trace equal to rank. The Schur complement gives block inverses and determinants; $\mathrm{tr}(\mathbf{AB}) = \mathrm{tr}(\mathbf{BA})$; $|\det \mathbf{A}|$ scales volume. The Leontief inverse $(\mathbf{I} - \mathbf{A})^{-1} = \sum_k \mathbf{A}^k$ yields output multipliers.
- **Data as matrices.** $\mathbf{H} = \mathbf{I} - n^{-1}\mathbf{1}\mathbf{1}^\top$ centres; $\mathbf{S} = \mathbf{X}^\top\mathbf{H}\mathbf{X}/(n-1)$ and $\mathbf{R} = \mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{-1/2}$ are positive semidefinite and singular if $p \ge n$. Kronecker products encode panel structures and $\mathrm{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes \mathbf{A})\,\mathrm{vec}\,\mathbf{X}$.
- **Floating point.** Doubles carry ≈ 16 digits ($u \approx 1.1 \times 10^{-16}$); $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$. Cancellation exposes earlier errors; rewrite formulas (`expm1`, `log1p`), use log-sum-exp, pairwise or Kahan sums, two-pass or Welford variances, tolerance-based comparisons and integer or decimal arithmetic for money.

- **Cost.** Dense products and solves are $O(n^3)$, least squares $O(np^2)$; vectorise, order products sensibly, use sparse storage for administrative data, and make every result reproducible (seeds, environments, scripts).

Further Reading

The linear algebra of this chapter is developed with a data-science emphasis in Strang (2019) and Deisenroth et al. (2020, Chapters 2–3); Murphy (2022, Chapter 7) gives a compact review of the vector and matrix results used in probabilistic machine learning, and Géron (2022, Chapter 2) shows the pandas–NumPy workflow on a full project. Economists will find Simon and Blume (1994) a gentle introduction and Magnus and Neudecker (2019) the standard advanced reference for matrix algebra in econometrics; Hansen (2022, Appendix A) collects the matrix results needed for regression. For computation, Golub and Van Loan (2013) and Trefethen and Bau (1997) are the classic texts on numerical linear algebra, and Higham (2002) is the definitive account of rounding-error analysis, including summation and variance algorithms. Goldberg (1991) and Overton (2001) are readable introductions to IEEE arithmetic, and McCullough and Vinod (1999) documents how numerical inaccuracy has affected published econometric results. Miller and Blair (2009) is the standard reference on input–output analysis and ILO et al. (2004) on consumer price indices. Probabilistic modelling software of the kind used in Matsuura (2022) relies throughout on log-scale computation and log-sum-exp; and the large-panel forecasting exercises of Goulet Coulombe et al. (2022) illustrate why the scaling of data matrices and the cost of computation matter in applied macroeconomics.

Chapter 2

Linear Algebra for Modelling: Rank, Projections and Decompositions

Learning Objectives

After working through this chapter you should be able to:

- describe a matrix through its four fundamental subspaces and decide, from ranks alone, whether a linear system has no, one or infinitely many solutions;
- state and prove the rank–nullity theorem and use it to diagnose identification, perfect multicollinearity and the dummy variable trap in regression designs;
- compute and interpret numerical rank with a tolerance, and explain why the answer depends on that tolerance;
- construct orthogonal projections, prove the properties of the hat matrix P and the annihilator M, and derive the ANOVA decomposition and R^2;
- interpret least squares as an orthogonal projection, prove the Frisch–Waugh–Lovell theorem and use it to implement fixed-effects estimators by demeaning;
- choose between LU, Cholesky, QR and SVD for a given computational task, and explain why QR is preferred to the normal equations on ill-conditioned designs;
- define the Moore–Penrose pseudoinverse through the four Penrose conditions and use it for minimum-norm least squares when $p > n$.

Almost every model a data scientist or policy analyst fits contains a linear algebra problem at its core. A Mincer wage regression on a household survey, a fixed-effects evaluation of a district-level programme, an input–output model of the economy, a principal-component asset index, a ridge regression with hundreds of predictors, and the reconciliation of the production, expenditure and income estimates of GDP are all, once the economics has been written down, statements about vectors, subspaces and matrices. The questions we care about in policy work map directly onto linear-algebraic ones:

- *Can the effect be learned from these data at all?* That is a question about the rank of the design matrix (identification).
- *What part of the outcome is explained by the regressors, and what is left over?* That is a question about orthogonal projection onto a column space.
- *What does "controlling for region" actually do?* The Frisch–Waugh–Lovell theorem gives an exact geometric answer: it projects region out of everything.
- *Will the computer give me the right number?* That depends on how the algebra is organised:

on decompositions (LU, Cholesky, QR, SVD) and on condition numbers.

- *What happens when there are more variables than observations?* The pseudoinverse selects one solution among infinitely many, and this choice is the door to regularisation and to modern "interpolating" machine learning.

This chapter develops these ideas at the level needed for the rest of the course. The notation follows Chapter 1; eigenvalues and the singular value decomposition are treated in depth in Chapter 3, and the calculus of matrix functions in Chapter 12. Our main references are Strang (2019), Golub and Van Loan (2013) and Trefethen and Bau (1997) for the mathematics and computation, and Hansen (2022) and Davidson and MacKinnon (2004) for the econometrics. Throughout, data sets are *simulated*; numbers that look like Ghanaian statistics are illustrations, not estimates.

2.1 Linear Systems and Vector Spaces

2.1.1 Three ways to read $A\mathbf{x} = \mathbf{b}$

Let $A \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. The system $A\mathbf{x} = \mathbf{b}$ can be read in three complementary ways.

1. **Row picture.** Each of the m equations $\mathbf{a}_i^\top \mathbf{x} = b_i$ is a hyperplane in $\mathbb{R}^n$; a solution is a point lying on all m hyperplanes.
2. **Column picture.** Writing $A = [\mathbf{a}^{(1)}, \dots, \mathbf{a}^{(n)}]$,
$$A\mathbf{x} = x_1 \mathbf{a}^{(1)} + x_2 \mathbf{a}^{(2)} + \cdots + x_n \mathbf{a}^{(n)}, \tag{2.1}$$
so solving the system means expressing $\mathbf{b}$ as a linear combination of the columns of A. In a regression the columns are variables and $\mathbf{x}$ holds the coefficients: the column picture is the one statisticians use.
3. **Map picture.** $\mathbf{x} \mapsto A\mathbf{x}$ is a linear map $\mathbb{R}^n \to \mathbb{R}^m$. Solving the system asks whether $\mathbf{b}$ lies in the image of this map and, if so, which inputs are sent to it.

The column and map pictures lead directly to the subspaces studied below.

2.1.2 Vector spaces, subspaces and span

We work in $\mathbb{R}^n$ throughout, but the definitions apply verbatim to any real vector space (spaces of functions, of matrices, or of random variables with finite variance).

Definition 2.1 (Subspace)**.** A non-empty set $S \subseteq \mathbb{R}^n$ is a *subspace* if it is closed under addition and scalar multiplication: $\mathbf{u}, \mathbf{v} \in S$ and $c \in \mathbb{R}$ imply $\mathbf{u} + \mathbf{v} \in S$ and $c\mathbf{u} \in S$. In particular every subspace contains $\mathbf{0}$.

Definition 2.2 (Span, linear independence, basis, dimension)**.** Let $\mathbf{v}_1, \dots, \mathbf{v}_k \in \mathbb{R}^n$.

- Their *span* is $\operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\} = \{c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k : c_i \in \mathbb{R}\}$, the smallest subspace containing them.
- They are *linearly independent* if $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ implies $c_1 = \cdots = c_k = 0$; otherwise they are *linearly dependent*.
- They form a *basis* of a subspace S if they are linearly independent and span S.
- The *dimension* $\dim S$ is the number of vectors in any basis of S.

The definition of dimension makes sense only because of the following fact.

Theorem 2.3 (Invariance of dimension)**.** *If $\mathbf{v}_1, \dots, \mathbf{v}_k$ are linearly independent and lie in $\operatorname{span}\{\mathbf{w}_1, \dots, \mathbf{w}_\ell\}$, then $k \le \ell$. Consequently any two bases of a subspace have the same number of elements.*

Proof sketch. Put $W = [\mathbf{w}_1, \dots, \mathbf{w}_\ell]$. Each $\mathbf{v}_j = W\mathbf{c}_j$ for some $\mathbf{c}_j \in \mathbb{R}^\ell$, so $V = [\mathbf{v}_1, \dots, \mathbf{v}_k] = WC$ with $C \in \mathbb{R}^{\ell \times k}$. If $k > \ell$, the homogeneous system $C\mathbf{z} = \mathbf{0}$ has more unknowns than equations and therefore (Gaussian elimination leaves at least one free variable) a non-zero solution $\mathbf{z}$. Then $V\mathbf{z} = WC\mathbf{z} = \mathbf{0}$, contradicting the independence of the $\mathbf{v}_j$. Applying the result in both directions to two bases gives equal sizes. □

Example 2.4 (Dependence in a design matrix)**.** Let a survey record, for each household, total expenditure E, food expenditure F and non-food expenditure N, with $E = F + N$ by construction. The three columns $\mathbf{E}, \mathbf{F}, \mathbf{N} \in \mathbb{R}^n$ satisfy $1 \cdot \mathbf{E} - 1 \cdot \mathbf{F} - 1 \cdot \mathbf{N} = \mathbf{0}$, so they are linearly dependent and span a space of dimension at most two. Any regression that includes all three runs into the problem studied in Section 2.2.

2.1.3 The four fundamental subspaces

Definition 2.5 (Fundamental subspaces)**.** For $A \in \mathbb{R}^{m \times n}$ define

$$\begin{aligned}
\operatorname{col}(A) &= \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m && \text{(column space, image)},\\
\operatorname{null}(A) &= \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}\} \subseteq \mathbb{R}^n && \text{(null space, kernel)},\\
\operatorname{col}(A^\top) &= \{A^\top\mathbf{y} : \mathbf{y} \in \mathbb{R}^m\} \subseteq \mathbb{R}^n && \text{(row space)},\\
\operatorname{null}(A^\top) &= \{\mathbf{y} \in \mathbb{R}^m : A^\top\mathbf{y} = \mathbf{0}\} \subseteq \mathbb{R}^m && \text{(left null space)}.
\end{aligned}$$

The *rank* of A is $\operatorname{rank}(A) = \dim \operatorname{col}(A)$.

Each of the four sets is a subspace (check closure). The column space and the left null space live in the "data space" $\mathbb{R}^m$ (one coordinate per observation); the row space and the null space live in the "parameter space" $\mathbb{R}^n$ (one coordinate per coefficient).

Theorem 2.6 (Fundamental theorem of linear algebra)**.** *Let $A \in \mathbb{R}^{m \times n}$ have rank r. Then*

1. $\dim \operatorname{col}(A) = \dim \operatorname{col}(A^\top) = r$ *(row rank equals column rank);*
2. $\dim \operatorname{null}(A) = n - r$ *and* $\dim \operatorname{null}(A^\top) = m - r$*;*
3. $\operatorname{null}(A)$ *is the orthogonal complement of* $\operatorname{col}(A^\top)$ *in* $\mathbb{R}^n$*, and* $\operatorname{null}(A^\top)$ *is the orthogonal complement of* $\operatorname{col}(A)$ *in* $\mathbb{R}^m$*.*

Proof. (3) If $A\mathbf{x} = \mathbf{0}$ then every row $\mathbf{a}_i$ of A satisfies $\mathbf{a}_i^\top\mathbf{x} = 0$, so $\mathbf{x}$ is orthogonal to every linear combination of rows, that is, to $\operatorname{col}(A^\top)$. Conversely, if $\mathbf{x} \perp \operatorname{col}(A^\top)$ then $\mathbf{x} \perp \mathbf{a}_i$ for each i, so $A\mathbf{x} = \mathbf{0}$. The statement for $A^\top$ is the same argument applied to $A^\top$.

(1) Let the columns of $C \in \mathbb{R}^{m \times r}$ be a basis of $\operatorname{col}(A)$. Every column of A is a combination of these, so $A = CR$ for some $R \in \mathbb{R}^{r \times n}$. Then every row of A is a combination of the r rows of R, hence $\dim \operatorname{col}(A^\top) \le r = \dim \operatorname{col}(A)$. Applying the same reasoning to $A^\top$ gives the reverse inequality.

(2) This is the rank–nullity theorem, proved as Theorem 2.9 below; the second identity is rank–nullity for $A^\top$ combined with (1). □

⋆ Key Concept: The four subspaces

A matrix $A \in \mathbb{R}^{m\times n}$ of rank r splits the parameter space as $\mathbb{R}^n = \mathrm{col}(A^\top) \oplus \mathrm{null}(A)$ (dimensions r and $n-r$) and the data space as $\mathbb{R}^m = \mathrm{col}(A) \oplus \mathrm{null}(A^\top)$ (dimensions r and $m-r$), with the two pieces orthogonal in each case. A maps the row space one-to-one onto the column space and sends the null space to $\mathbf{0}$. In regression language: $\mathrm{col}(X)$ is the set of all possible fitted values, $\mathrm{null}(X^\top)$ is where residuals live, and $\mathrm{null}(X)$ contains the coefficient directions that the data cannot see.

2.1.4 Existence and uniqueness of solutions

Theorem 2.7 (Solvability; Rouché–Capelli)**.** *Let $A \in \mathbb{R}^{m\times n}$ and $\mathbf{b} \in \mathbb{R}^m$.*

1. *(Existence) $A\mathbf{x} = \mathbf{b}$ has a solution iff $\mathbf{b} \in \mathrm{col}(A)$ iff $\mathrm{rank}[A\ \mathbf{b}] = \mathrm{rank}(A)$ iff $\mathbf{b} \perp \mathrm{null}(A^\top)$.*
2. *(Uniqueness) If a solution $\mathbf{x}_p$ exists, the set of all solutions is the affine set $\mathbf{x}_p + \mathrm{null}(A)$; the solution is unique iff $\mathrm{null}(A) = \{\mathbf{0}\}$ iff $\mathrm{rank}(A) = n$ (full column rank).*

Proof. (1) The first equivalence is the column picture (2.1). Appending $\mathbf{b}$ to A leaves the column space unchanged exactly when $\mathbf{b}$ is already in it. The last equivalence follows from Theorem 2.6(3). (2) If $A\mathbf{x}_p = \mathbf{b}$ and $A\mathbf{x} = \mathbf{b}$ then $A(\mathbf{x} - \mathbf{x}_p) = \mathbf{0}$, so $\mathbf{x} - \mathbf{x}_p \in \mathrm{null}(A)$; conversely $A(\mathbf{x}_p + \mathbf{z}) = \mathbf{b}$ for any $\mathbf{z} \in \mathrm{null}(A)$. □

The certificate in part (1) is worth remembering: a system is inconsistent precisely when there is a vector $\mathbf{y}$ with $A^\top\mathbf{y} = \mathbf{0}$ but $\mathbf{y}^\top\mathbf{b} \neq 0$. Such a $\mathbf{y}$ is a combination of the equations that reads "0 = non-zero".

Example 2.8 (A small system)**.** Take

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 3 \\ 7 \end{pmatrix}.$$

The second column is twice the first, and columns one and three are independent, so $\mathrm{rank}(A) = 2 = m$: the columns span $\mathbb{R}^2$ and the system is solvable for every $\mathbf{b}$. Since $n - r = 1$, the solution set is a line. Elimination gives $x_3 = 1$ and $x_1 + 2x_2 = 2$, so $\mathbf{x} = (2, 0, 1)^\top + t(-2, 1, 0)^\top$, $t \in \mathbb{R}$; the direction $(-2, 1, 0)^\top$ spans $\mathrm{null}(A)$ and is orthogonal to both rows of A, as Theorem 2.6 predicts.

2.1.5 Computing the four subspaces

In exact arithmetic one finds bases by Gaussian elimination (reduced row echelon form). In floating point the reliable tool is the singular value decomposition $A = U\Sigma V^\top$, treated fully in Chapter 3. If $\sigma_1 \geq \cdots \geq \sigma_r > 0$ are the non-zero singular values, then the first r columns of U are an orthonormal basis of $\mathrm{col}(A)$, the remaining $m - r$ columns of U a basis of $\mathrm{null}(A^\top)$, the first r columns of V a basis of $\mathrm{col}(A^\top)$ and the remaining $n - r$ columns of V a basis of $\mathrm{null}(A)$. Listing 2.1 builds a 4×5 matrix of rank two, extracts the four subspaces, verifies their dimensions and orthogonality, and classifies systems by comparing $\mathrm{rank}(A)$ with $\mathrm{rank}[A\ \mathbf{b}]$.

Listing 2.1: Four fundamental subspaces and solvability ch02/subspaces.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Four fundamental subspaces of a rank-deficient matrix and solvability of Ax = b
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
```

```python
m, n, r = 4, 5, 2
A = rng.integers(-3, 4, size=(m, r)) @ rng.integers(-3, 4, size=(r, n))
A = A.astype(float)
print("A =\n", A)

U, s, Vt = np.linalg.svd(A)
tol = max(A.shape) * np.finfo(float).eps * s[0]
rk = int(np.sum(s > tol))
col_basis = U[:, :rk]          # basis of col(A)   (subspace of R^m)
lnull_basis = U[:, rk:]        # basis of null(A^T)
row_basis = Vt[:rk].T          # basis of col(A^T) (subspace of R^n)
null_basis = Vt[rk:].T         # basis of null(A)

print(f"\nsingular values: {np.round(s, 4)}")
print(f"rank r = {rk}, m = {m}, n = {n}")
print(f"dim col(A)   = {col_basis.shape[1]}   dim null(A^T) = {lnull_basis.shape[1]}"
      f"   (sum = m = {m})")
print(f"dim row(A)   = {row_basis.shape[1]}   dim null(A)   = {null_basis.shape[1]}"
      f"   (sum = n = {n})")
print(f"max |A N|          = {np.abs(A @ null_basis).max():.2e}")
print(f"max |row^T null|   = {np.abs(row_basis.T @ null_basis).max():.2e}")
print(f"max |col^T lnull|  = {np.abs(col_basis.T @ lnull_basis).max():.2e}")


def classify(A, b):
    """Existence / uniqueness of solutions of Ax = b via ranks."""
    rA = np.linalg.matrix_rank(A)
    rAb = np.linalg.matrix_rank(np.column_stack([A, b]))
    if rAb > rA:
        return "no solution (b not in col(A))"
    if rA == A.shape[1]:
        return "unique solution"
    return f"infinitely many solutions ({A.shape[1] - rA} free parameters)"


b_in = A @ rng.normal(size=n)              # consistent right-hand side
b_out = b_in + lnull_basis[:, 0]           # add a left-null-space component
print("\nb in col(A)    :", classify(A, b_in))
print("b not in col(A):", classify(A, b_out))
X_full = rng.normal(size=(6, 3))
print("tall full-rank :", classify(X_full, X_full @ np.ones(3)))

# Any solution = particular solution + element of null(A)
x_p = np.linalg.lstsq(A, b_in, rcond=None)[0]
x_alt = x_p + null_basis @ rng.normal(size=n - rk)
print(f"\n||A x_p - b||   = {np.linalg.norm(A @ x_p - b_in):.2e}")
print(f"||A x_alt - b|| = {np.linalg.norm(A @ x_alt - b_in):.2e}")
print(f"||x_p|| = {np.linalg.norm(x_p):.4f} <= ||x_alt|| = "
      f"{np.linalg.norm(x_alt):.4f}")
```

Output

```
A =
 [[-4.  2.  2. -4. -4.]
 [-6.  3.  3. -6. -6.]
 [ 5. -4. -2.  6.  4.]
```

```
 [ 1.  1. -1.  0.  2.]]

singular values: [16.6793  2.7929  0.      0.    ]
rank r = 2, m = 4, n = 5
dim col(A)   = 2   dim null(A^T) = 2   (sum = m = 4)
dim row(A)   = 2   dim null(A)   = 3   (sum = n = 5)
max |A N|          = 3.77e-15
max |row^T null|   = 1.49e-16
max |col^T lnull|  = 4.68e-17

b in col(A)    : infinitely many solutions (3 free parameters)
b not in col(A): no solution (b not in col(A))
tall full-rank : unique solution

||A x_p - b||   = 6.84e-15
||A x_alt - b|| = 1.12e-14
||x_p|| = 0.7150 <= ||x_alt|| = 1.5174
```

The output confirms $2+2=4=m$ and $2+3=5=n$. For a consistent right-hand side there are three free parameters, and the solution returned by `np.linalg.lstsq` has the smallest norm among all solutions; we will see why in Section 2.6.

△ Caution: Rank is not a continuous function

In exact arithmetic the rank of a matrix is an integer that jumps when entries change by an arbitrarily small amount: $\left(\begin{smallmatrix}1 & 1\\ 1 & 1+\delta\end{smallmatrix}\right)$ has rank 2 for every $\delta \neq 0$ and rank 1 at $\delta = 0$. A computer, which stores $1+\delta$ with relative error about $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$, cannot distinguish these cases when $|\delta|$ is tiny. "What is the rank of my data matrix?" is therefore always a question with a tolerance attached (Section 2.2.4).

Policy Lens: Accounting identities as linear systems

Many accounting frameworks used in policy analysis are linear systems. In a Leontief input–output model, gross outputs $\mathbf{x}$ satisfy $\mathbf{x} = A\mathbf{x} + \mathbf{d}$, where A holds technical coefficients and $\mathbf{d}$ is final demand, so $(\mathbf{I} - A)\mathbf{x} = \mathbf{d}$. Existence and uniqueness of a non-negative solution for every final-demand scenario is exactly the statement that $\mathbf{I} - A$ is invertible with a non-negative inverse; the inverse $(\mathbf{I} - A)^{-1}$ is the matrix of output multipliers used to assess the economy-wide effect of, say, a public investment programme (Leontief, 1936). We solve such a model in Section 2.5.

Exercises

2.1.1 Theory Prove that $\operatorname{col}(A)$ and $\operatorname{null}(A)$ are subspaces. Show that the union of two subspaces is in general not a subspace, but their intersection and their sum $S+T=\{\mathbf{s}+\mathbf{t} : \mathbf{s}\in S, \mathbf{t}\in T\}$ are.

2.1.2 Theory Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$. Prove that $\operatorname{col}(AB) \subseteq \operatorname{col}(A)$ and $\operatorname{null}(B) \subseteq \operatorname{null}(AB)$, and deduce $\operatorname{rank}(AB) \le \min\{\operatorname{rank}(A), \operatorname{rank}(B)\}$.

2.1.3 Theory Find bases for all four fundamental subspaces of $A = \left(\begin{smallmatrix}1 & 0 & 2 & 1\\ 0 & 1 & 1 & 1\\ 1 & 1 & 3 & 2\end{smallmatrix}\right)$ by hand. Verify the dimension counts of Theorem 2.6 and give a vector $\mathbf{b}$ for which $A\mathbf{x}=\mathbf{b}$ is inconsistent, together with the certificate $\mathbf{y}$.

2.1.4 Python Modify Listing 2.1 to compute bases of the four subspaces with `scipy.linalg`.

↪ `null_space` and `scipy.linalg.orth`, and check that the projector onto $\mathrm{col}(A)$ obtained from your basis agrees with the one obtained from the SVD.

2.1.5 Python Generate 200 random 6×8 matrices of the form $A = BC$ with $B \in \mathbb{R}^{6\times k}$, $C \in \mathbb{R}^{k\times 8}$ Gaussian, for $k = 1, \dots, 6$. Verify numerically that $\mathrm{rank}(A) = k$ in every case and that $\dim \mathrm{null}(A) + \mathrm{rank}(A) = 8$.

2.1.6 Policy A regional planning office models the budgets of m districts as $A\mathbf{x} = \mathbf{b}$, where $\mathbf{x}$ holds three unit costs of public services to be inferred, $\mathbf{b}$ the district budgets and A the service volumes. Explain in terms of the four subspaces what it means (i) when the system is inconsistent and (ii) when it has infinitely many solutions. Which situation is typical with $m = 30$ districts, and what should the analyst do?

2.1.7 ★ Graduate Let $S \subseteq \mathbb{R}^n$ be a subspace of dimension k. Show that there is a matrix A with $S = \mathrm{null}(A)$ and a matrix B with $S = \mathrm{col}(B)$, and give their minimal sizes. Interpret: every set of linear restrictions on a parameter vector can be written either as "equations" or as a "parametrisation".

2.2 Rank and Identification

2.2.1 The rank–nullity theorem

Theorem 2.9 (Rank–nullity)**.** *For every $A \in \mathbb{R}^{m\times n}$,*

$$\mathrm{rank}(A) + \dim \mathrm{null}(A) = n. \tag{2.2}$$

Proof. Let $k = \dim \mathrm{null}(A)$ and let $\mathbf{v}_1, \dots, \mathbf{v}_k$ be a basis of $\mathrm{null}(A)$. Extend it to a basis $\mathbf{v}_1, \dots, \mathbf{v}_k, \mathbf{w}_1, \dots, \mathbf{w}_{n-k}$ of $\mathbb{R}^n$ (possible by repeatedly adding vectors not in the current span). We show that $A\mathbf{w}_1, \dots, A\mathbf{w}_{n-k}$ is a basis of $\mathrm{col}(A)$, which gives $\mathrm{rank}(A) = n - k$.

Spanning. Any $\mathbf{x} \in \mathbb{R}^n$ can be written $\mathbf{x} = \sum_i c_i \mathbf{v}_i + \sum_j d_j \mathbf{w}_j$, so $A\mathbf{x} = \sum_j d_j A\mathbf{w}_j$ because $A\mathbf{v}_i = \mathbf{0}$.

Independence. Suppose $\sum_j d_j A\mathbf{w}_j = \mathbf{0}$. Then $A\big(\sum_j d_j \mathbf{w}_j\big) = \mathbf{0}$, so $\sum_j d_j \mathbf{w}_j \in \mathrm{null}(A)$ and therefore $\sum_j d_j \mathbf{w}_j = \sum_i c_i \mathbf{v}_i$ for some c_i. Since the $\mathbf{v}$'s and $\mathbf{w}$'s together are linearly independent, all d_j (and c_i) are zero. □

The theorem is a conservation law: each of the n input dimensions is either "transmitted" to the output (counted by the rank) or "lost" in the null space. Two consequences are used constantly in econometrics.

Corollary 2.10. *For $X \in \mathbb{R}^{n\times k}$: $\mathrm{null}(X^\top X) = \mathrm{null}(X)$, hence $\mathrm{rank}(X^\top X) = \mathrm{rank}(X)$. In particular $X^\top X$ is invertible iff X has full column rank k, and then $X^\top X$ is symmetric positive definite.*

Proof. If $X\mathbf{v} = \mathbf{0}$ then $X^\top X\mathbf{v} = \mathbf{0}$. Conversely, $X^\top X\mathbf{v} = \mathbf{0}$ implies $0 = \mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2$, so $X\mathbf{v} = \mathbf{0}$. Equal null spaces have equal dimensions, and rank–nullity (with the same $n = k$ columns) gives equal ranks. If $\mathrm{rank}(X) = k$, then $\mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2 > 0$ for every $\mathbf{v} \neq \mathbf{0}$. □

Corollary 2.11. *If $X \in \mathbb{R}^{n\times k}$ with $k > n$ then $\dim \mathrm{null}(X) \geq k - n > 0$: with more regressors than observations the coefficients can never be pinned down by the data alone.*

2.2.2 Full column rank and identification

Consider the linear model

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad X \in \mathbb{R}^{n\times k}, \tag{2.3}$$

where we observe $(\mathbf{y}, X)$. Two parameter vectors β and β' are *observationally equivalent* if $X\beta = X\beta'$: they produce exactly the same systematic part of the outcome and no amount of data on $\mathbf{y}$ can tell them apart.

Definition 2.12 (Identification in the linear model)**.** The coefficient vector β is *identified* (given the design X) if $X\beta = X\beta'$ implies $\beta = \beta'$. A linear function $\ell^\top\beta$ is identified (*estimable*) if $X\beta = X\beta'$ implies $\ell^\top\beta = \ell^\top\beta'$.

Theorem 2.13 (Rank condition)**.** *(a) β is identified iff* $\text{null}(X) = \{\mathbf{0}\}$ *iff* $\text{rank}(X) = k$ *iff $X^\top X$ is invertible. (b) $\ell^\top\beta$ is identified iff $\ell \in \text{col}(X^\top)$, the row space of X.*

Proof. (a) $X\beta = X\beta'$ iff $\beta - \beta' \in \text{null}(X)$; the remaining equivalences are Theorem 2.9 and Corollary 2.10. (b) If $\ell = X^\top\mathbf{a}$ then $\ell^\top\beta = \mathbf{a}^\top(X\beta)$ depends on β only through $X\beta$. Conversely, by Theorem 2.6 write $\ell = \ell_r + \ell_0$ with $\ell_r \in \text{col}(X^\top)$ and $\ell_0 \in \text{null}(X)$. If $\ell_0 \neq \mathbf{0}$, then β and $\beta + \ell_0$ are observationally equivalent but $\ell^\top(\beta + \ell_0) - \ell^\top\beta = \|\ell_0\|^2 \neq 0$. □

In population terms the same statement reads: with $\mathbb{E}[\mathbf{x}_i\varepsilon_i] = \mathbf{0}$, the moment equation $\mathbb{E}[\mathbf{x}_i\mathbf{x}_i^\top]\beta = \mathbb{E}[\mathbf{x}_i y_i]$ has a unique solution iff $\mathbb{E}[\mathbf{x}_i\mathbf{x}_i^\top]$ is non-singular (Hansen, 2022, Ch. 2). The sample design matrix having full column rank is the finite-sample counterpart.

⋆ Key Concept: Identification is a rank condition

In a linear model, the data determine $X\beta$, never β directly. The coefficients are identified exactly when X has full column rank, so that X maps the parameter space one-to-one into the data space. When the rank is deficient, only the linear combinations $\ell^\top\beta$ with ℓ in the row space of X are learnable; everything in $\text{null}(X)$ is invisible to any estimator.

2.2.3 Perfect multicollinearity and the dummy variable trap

Perfect multicollinearity means that some column of X is an exact linear combination of the others, i.e. $\text{null}(X) \neq \{\mathbf{0}\}$. The classic example arises with categorical regressors. Suppose household i lives in region $g(i) \in \{1, \dots, G\}$ and let $D \in \{0,1\}^{n\times G}$ be the matrix of region indicators, $D_{ig} = 1$ iff $g(i) = g$. Every row of D contains exactly one 1, so

$$D\mathbf{1}_G = \mathbf{1}_n. \tag{2.4}$$

If the regression also contains an intercept, the design $X = [\mathbf{1}_n,\ \mathbf{educ},\ D]$ satisfies $X\mathbf{c} = \mathbf{0}$ with $\mathbf{c} = (1, 0, -1, \dots, -1)^\top$: this is the *dummy variable trap*. By rank–nullity, $\text{rank}(X) = k - 1$ and the intercept and region effects are not separately identified. The three standard remedies are all re-parametrisations that remove the null direction:

1. drop one region (the *base* or reference category); the remaining dummy coefficients are then differences from the base region;
2. drop the intercept and keep all G dummies; each coefficient is a region-specific intercept;
3. keep everything but impose a linear restriction such as $\sum_g \alpha_g = 0$ ("effects coding"), which selects one point of the affine solution set.

All three give the same fitted values and the same coefficients on the non-regional regressors; they differ only in which estimable functions are reported.

Policy Lens: The dummy trap with all regions of Ghana

Since the creation of six new regions in 2018–2019, Ghana has 16 administrative regions. An analyst who constructs one indicator per region from a household survey and adds them all

to a regression with an intercept has 18 columns but rank 17 (Listing 2.2). Some packages silently drop a column (and which one may depend on the order of the data), others return a "minimum-norm" solution, and the naive formula $(X^\top X)^{-1}X^\top \mathbf{y}$ fails or returns numerical garbage. A report that compares "the Upper East effect" across two such analyses may thus be comparing different estimable functions. Always state the base region, and check $\operatorname{rank}(X)$ before interpreting coefficients.

2.2.4 Numerical rank

Because rank is discontinuous, software computes a *numerical rank*: the number of singular values above a tolerance,

$$\operatorname{rank}_\tau(A) = \#\{i : \sigma_i(A) > \tau\}. \tag{2.5}$$

NumPy's `np.linalg.matrix_rank` uses by default $\tau = \sigma_1 \max(m, n)\, \varepsilon_{\text{mach}}$, a threshold at the level of rounding errors in computing the SVD; a user-supplied `tol` replaces it. The justification is a classical result: $\sigma_{r+1}(A)$ is the distance (in the spectral norm) from A to the nearest matrix of rank at most r (the Eckart–Young theorem, Chapter 3). Thus $\operatorname{rank}_\tau(A) = r$ means "A is within τ of a rank-r matrix, but not within τ of a matrix of rank $r-1$". Whether τ should reflect rounding error or measurement error in the data is a modelling decision, not a computational one.

Listing 2.2: Rank, the dummy trap and numerical rank `ch02/dummy_trap.py`

```python
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Rank, the dummy variable trap and numerical rank in a simulated survey
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, G = 800, 16                        # households, regions (Ghana has 16)
region = rng.integers(0, G, size=n)
educ = rng.normal(9, 3, size=n)       # years of schooling (simulated)
D = np.eye(G)[region]                 # n x G matrix of region dummies
one = np.ones((n, 1))


def report(name, X):
    r = np.linalg.matrix_rank(X)
    k = X.shape[1]
    nullity = k - r
    flag = "full column rank" if nullity == 0 else f"RANK DEFICIENT by {nullity}"
    print(f"{name:<34s} k = {k:2d}  rank = {r:2d}  -> {flag}")


report("const + educ", np.column_stack([one, educ]))
report("const + educ + all 16 dummies", np.column_stack([one, educ, D]))
report("const + educ + 15 dummies", np.column_stack([one, educ, D[:, 1:]]))
report("no const + educ + 16 dummies", np.column_stack([educ, D]))

# The null vector of the trapped design: (1, 0, -1, ..., -1)
X_trap = np.column_stack([one, educ, D])
c = np.r_[1.0, 0.0, -np.ones(G)]
print(f"\n||X_trap c|| = {np.linalg.norm(X_trap @ c):.1e}  (exact dependence)")

# Rank-nullity check: rank + dim null = number of columns
_, s, Vt = np.linalg.svd(X_trap)
tol = s.max() * max(X_trap.shape) * np.finfo(float).eps
```

```
null_dim = int(np.sum(s <= tol))
print(f"rank + nullity = {np.linalg.matrix_rank(X_trap)} + {null_dim} = "
      f"{X_trap.shape[1]}")

# Near collinearity: numerical rank depends on the tolerance
print("\nNear-collinear regressor  x3 = x1 + x2 + delta * noise")
x1, x2 = rng.normal(size=n), rng.normal(size=n)
for delta in [1e-2, 1e-8, 1e-13, 1e-16]:
    X = np.column_stack([one, x1, x2, x1 + x2 + delta * rng.normal(size=n)])
    s = np.linalg.svd(X, compute_uv=False)
    r_def = np.linalg.matrix_rank(X)
    r_loose = np.linalg.matrix_rank(X, tol=1e-6 * s[0])
    print(f"delta = {delta:7.0e}  s_min/s_max = {s[-1] / s[0]:9.2e}  "
          f"rank(default) = {r_def}  rank(tol=1e-6) = {r_loose}")
```

Output

```
const + educ                       k =  2  rank =  2  -> full column rank
const + educ + all 16 dummies      k = 18  rank = 17  -> RANK DEFICIENT by 1
const + educ + 15 dummies          k = 17  rank = 17  -> full column rank
no const + educ + 16 dummies       k = 17  rank = 17  -> full column rank

||X_trap c|| = 0.0e+00  (exact dependence)
rank + nullity = 17 + 1 = 18

Near-collinear regressor  x3 = x1 + x2 + delta * noise
delta =   1e-02  s_min/s_max =  3.46e-03  rank(default) = 4  rank(tol=1e-6) = 4
delta =   1e-08  s_min/s_max =  3.32e-09  rank(default) = 4  rank(tol=1e-6) = 3
delta =   1e-13  s_min/s_max =  3.46e-14  rank(default) = 3  rank(tol=1e-6) = 3
delta =   1e-16  s_min/s_max =  2.58e-16  rank(default) = 3  rank(tol=1e-6) = 3
```

The second part of the output is the important one. With $\delta = 10^{-8}$ the fourth regressor differs from $x_1 + x_2$ by noise eight orders of magnitude smaller than the signal. At machine precision the matrix has full rank ($\sigma_4/\sigma_1 \approx 3 \times 10^{-9}$ is far above $\varepsilon_{\text{mach}}$), so the coefficients are *technically* identified; with any tolerance reflecting real measurement error, they are not. This is *near* (imperfect) multicollinearity: the estimator exists but its variance, which is proportional to the inverse of the small singular values squared, is enormous.

△ Caution: Software does not stop you

`np.linalg.lstsq` and `np.linalg.pinv` return an answer for a rank-deficient design without any warning other than the returned rank; `np.linalg.solve(X.T @ X, X.T @ y)` may raise `LinAlgError` or may return huge, meaningless coefficients, depending on rounding. Check the rank (and the condition number) of every design matrix you build, especially designs generated by code: interactions, dummies and polynomial terms are the usual sources of hidden dependencies.

2.2.5 Identification of a programme effect

The rank condition is a modelling tool, not only a numerical check. Suppose a programme (say, a school-feeding or cash-transfer scheme) is rolled out to *all* households in five of the sixteen regions, and we estimate

$$y_i = \tau T_i + \gamma\, \text{educ}_i + \alpha_{g(i)} + \varepsilon_i,$$

with region fixed effects α_g to absorb regional differences in prices, climate and infrastructure. Since $T_i = \sum_{g \le 5} D_{ig}$, the column T is a combination of dummy columns: τ is not identified. Listing 2.3 shows what this means concretely.

Listing 2.3: A programme effect collinear with region effects ch02/identification.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Identification: a programme effect that is collinear with region effects
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, G, tau = 1200, 16, 0.30            # households, regions, true effect
region = rng.integers(0, G, size=n)
treated_regions = np.arange(5)        # programme rolled out region-wide
T = np.isin(region, treated_regions).astype(float)
educ = rng.normal(9, 3, size=n)
alpha = rng.normal(0, 0.4, size=G)    # unobserved regional effects
y = 1.0 + tau * T + 0.08 * educ + alpha[region] + rng.normal(0, 0.5, n)

D = np.eye(G)[region]                 # all 16 region dummies, no constant
X = np.column_stack([T, educ, D])     # T = sum of first five dummy columns
print(f"columns = {X.shape[1]}, rank = {np.linalg.matrix_rank(X)}")

# Two different exact least-squares solutions
b1 = np.linalg.lstsq(X, y, rcond=None)[0]          # minimum-norm solution
c = np.r_[1.0, 0.0, -1.0 * np.isin(np.arange(G), treated_regions)]  # Xc=0
b2 = b1 + 2.0 * c
print(f"||X c|| = {np.linalg.norm(X @ c):.1e}")
print(f"tau-hat from solution 1 = {b1[0]: .4f}")
print(f"tau-hat from solution 2 = {b2[0]: .4f}")
print(f"max |fitted1 - fitted2| = {np.abs(X @ b1 - X @ b2).max():.1e}")

# An estimable function: l' beta is identified iff l is in row(X)
_, s, Vt = np.linalg.svd(X, full_matrices=False)
r = int(np.sum(s > s[0] * 1e-10))
P_row = Vt[:r].T @ Vt[:r]             # projector onto row(X)
for name, l in [("tau alone", np.r_[1, 0, np.zeros(G)]),
                ("educ slope", np.r_[0, 1, np.zeros(G)]),
                ("tau + alpha_1", np.r_[1, 0, 1, np.zeros(G - 1)])]:
    gap = np.linalg.norm(l - P_row @ l)
    ok = "identified" if gap < 1e-8 else "NOT identified"
    print(f"{name:<14s} distance from row(X) = {gap:.3f}  -> {ok}")

# A second (pre-programme) survey round restores identification
y0 = 1.0 + 0.08 * educ + alpha[region] + rng.normal(0, 0.5, n)
Y = np.r_[y0, y]
post = np.r_[np.zeros(n), np.ones(n)]
Xp = np.column_stack([post * np.r_[T, T], post, np.r_[educ, educ],
                      np.vstack([D, D])])
bp = np.linalg.lstsq(Xp, Y, rcond=None)[0]
print(f"\nTwo rounds: columns = {Xp.shape[1]}, rank = "
      f"{np.linalg.matrix_rank(Xp)}, tau-hat (DiD) = {bp[0]:.4f}")
```

Output

```
columns = 18, rank = 17
||X c|| = 0.0e+00
tau-hat from solution 1 =  1.1466
tau-hat from solution 2 =  3.1466
max |fitted1 - fitted2| = 4.4e-16
tau alone      distance from row(X) = 0.408  -> NOT identified
educ slope     distance from row(X) = 0.000  -> identified
tau + alpha_1  distance from row(X) = 0.000  -> identified

Two rounds: columns = 19, rank = 19, tau-hat (DiD) = 0.2872
```

Two exact least-squares solutions give $\hat{\tau} = 1.15$ and $\hat{\tau} = 3.15$ with identical fitted values; indeed every real number is a least-squares value of $\hat{\tau}$. The *estimable* functions, such as γ or $\tau + \alpha_1$ (the mean outcome level in a treated region), lie in the row space of X. The programme effect itself does not: it is confounded with the regional effects of the treated regions. Adding a pre-programme survey round changes the design. The difference-in-differences model with a "post" indicator and an interaction now has full column rank and recovers $\hat{\tau} = 0.29$, close to the true value 0.30. Identification came from a change in the *design* (a second period), not from a better algorithm.

Policy Lens: Programme effects that cannot be separated from place

When a programme is assigned at the level of a region or district, any regional fixed effect at the same level absorbs it completely. Evaluations of geographically targeted policies (regional development funds, district-wide health interventions) therefore need variation *within* the fixed-effect units: timing of roll-out (panels and difference-in-differences), eligibility thresholds (regression discontinuity), or assignment at a finer level than the controls. The rank of the design matrix is the first diagnostic of whether a proposed evaluation strategy can work at all.

Historical Note: Frisch, confluence analysis and the first Nobel Prize in Economics

The Norwegian economist Ragnar Frisch (1895–1973), who coined the word *econometrics* and was the first editor of *Econometrica*, studied exact and near linear dependencies among economic variables in his 1934 monograph *Statistical Confluence Analysis by Means of Complete Regression Systems*; the term *multicollinearity* is usually traced to this work. In 1969 Frisch and Jan Tinbergen shared the first Sveriges Riksbank Prize in Economic Sciences in Memory of Alfred Nobel, "for having developed and applied dynamic models for the analysis of economic processes". The identification problem for systems of equations was formalised shortly afterwards by the Cowles Commission, notably by Koopmans (1949).

Graduate Perspective: Rank conditions throughout econometrics

Identification is a rank condition far beyond OLS. In the instrumental-variables model $y = \mathbf{x}^\top \boldsymbol{\beta} + \varepsilon$ with instruments $\mathbf{z}$ satisfying $\mathbb{E}[\mathbf{z}\varepsilon] = \mathbf{0}$, the moment equations $\mathbb{E}[\mathbf{z}\mathbf{x}^\top]\boldsymbol{\beta} = \mathbb{E}[\mathbf{z}y]$ identify $\boldsymbol{\beta}$ iff $\operatorname{rank}\mathbb{E}[\mathbf{z}\mathbf{x}^\top] = k$, the *rank condition*; the *order condition* $\dim \mathbf{z} \geq k$ is necessary but not sufficient (Wooldridge, 2010, Ch. 5). In simultaneous-equation systems the rank condition is imposed on sub-matrices of the structural coefficients (Koopmans, 1949). In weak-instrument problems, the matrix has full rank but is close to rank-deficient: this is the population analogue of the $\delta = 10^{-8}$ row in Listing 2.2, and it has equally unpleasant statistical consequences.

Exercises

2.2.1 Theory Prove $\operatorname{rank}(A+B) \le \operatorname{rank}(A) + \operatorname{rank}(B)$ and $\operatorname{rank}(AB) \ge \operatorname{rank}(A) + \operatorname{rank}(B) - n$ for $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times p}$ (Sylvester's inequality). *Hint:* apply rank–nullity to A restricted to $\operatorname{col}(B)$.

2.2.2 Theory Show that if X contains an intercept and G region dummies, then $\operatorname{null}(X)$ is spanned by $\mathbf{c} = (1, 0, -\mathbf{1}_G^\top)^\top$ (with the regressor educ in position 2), provided $[\mathbf{1}, \mathbf{educ}]$ has rank two and each region is non-empty. What is $\operatorname{null}(X)$ if one region has no observations?

2.2.3 Theory In the trapped model $y_i = \mu + \gamma\,\mathrm{educ}_i + \alpha_{g(i)} + \varepsilon_i$, use Theorem 2.13(b) to decide which of the following are estimable: μ; $\alpha_2 - \alpha_1$; $\mu + \alpha_3$; $\sum_g \alpha_g$; γ.

2.2.4 Python Add to Listing 2.2 a set of 5 "ecological zone" dummies, where each region belongs to exactly one zone. How large is the rank deficiency of $[\mathbf{1}, \mathbf{educ}, D_{\text{region}}, D_{\text{zone}}]$? Find a basis of the null space numerically and interpret each basis vector.

2.2.5 Python Reproduce the near-collinearity experiment of Listing 2.2 and, for each δ, compute the OLS estimate of the coefficient on x_3 over 200 simulated outcome vectors $y = 1 + x_1 + x_2 + x_3 + \varepsilon$. Plot its standard deviation against δ on a log–log scale and explain the slope.

2.2.6 Policy A ministry wants to evaluate a teacher-incentive programme that was introduced in all districts of three regions in the same year. Data are available for two years before and two years after the programme. Write down a regression that identifies the programme effect, verify the rank condition by a short simulation, and explain which additional assumption (beyond rank) is needed for the identified coefficient to be a causal effect.

2.2.7 ★ Graduate Let $X = [X_1, X_2]$. Show that the coefficients on X_2 are identified (even if those on X_1 are not) iff $\operatorname{col}(X_1) \cap \operatorname{col}(X_2) = \{\mathbf{0}\}$ and X_2 has full column rank. Relate this to the statement that fixed-effects models identify slopes on time-varying regressors but not on time-invariant ones.

2.3 Orthogonality and Projections

2.3.1 Inner products and orthogonal complements

On $\mathbb{R}^n$ we use the Euclidean inner product $\langle \mathbf{u}, \mathbf{v}\rangle = \mathbf{u}^\top\mathbf{v}$ and norm $\|\mathbf{u}\| = \sqrt{\mathbf{u}^\top\mathbf{u}}$. Vectors are *orthogonal*, $\mathbf{u} \perp \mathbf{v}$, if $\mathbf{u}^\top\mathbf{v} = 0$. For a centred data vector, $\mathbf{u}^\top\mathbf{v}/n$ is a sample covariance, so orthogonality of centred variables is zero sample correlation: geometry and statistics speak the same language.

Definition 2.14 (Orthogonal complement)**.** For a subspace $S \subseteq \mathbb{R}^n$, $S^\perp = \{\mathbf{v} \in \mathbb{R}^n : \mathbf{v}^\top\mathbf{s} = 0\ \forall \mathbf{s} \in S\}$.

Theorem 2.15 (Orthogonal decomposition)**.** *For every subspace $S \subseteq \mathbb{R}^n$: $S^\perp$ is a subspace, $S \cap S^\perp = \{\mathbf{0}\}$, $\dim S + \dim S^\perp = n$, $(S^\perp)^\perp = S$, and every $\mathbf{y} \in \mathbb{R}^n$ has a unique decomposition $\mathbf{y} = \mathbf{s} + \mathbf{t}$ with $\mathbf{s} \in S$, $\mathbf{t} \in S^\perp$.*

Proof. Let the columns of $A \in \mathbb{R}^{n\times k}$ be a basis of S. Then $S^\perp = \operatorname{null}(A^\top)$, so by rank–nullity $\dim S^\perp = n - k$. If $\mathbf{v} \in S \cap S^\perp$ then $\mathbf{v}^\top\mathbf{v} = 0$, so $\mathbf{v} = \mathbf{0}$. Combining a basis of S with a basis of $S^\perp$ gives n vectors which are independent (a vanishing combination splits into a part in S and a part in $S^\perp$ that are negatives of each other, so both vanish), hence a basis of $\mathbb{R}^n$; this proves existence of the decomposition, and uniqueness follows from $S \cap S^\perp = \{\mathbf{0}\}$. Finally $S \subseteq (S^\perp)^\perp$ and both have dimension k. □

2.3.2 Orthogonal projection onto a subspace

Theorem 2.16 (Projection theorem)**.** *Let $S \subseteq \mathbb{R}^n$ be a subspace and $\mathbf{y} \in \mathbb{R}^n$. There is a unique $\hat{\mathbf{y}} \in S$ with*

$$\|\mathbf{y} - \hat{\mathbf{y}}\| = \min_{\mathbf{s} \in S} \|\mathbf{y} - \mathbf{s}\|,$$

and it is characterised by the orthogonality condition $\mathbf{y} - \hat{\mathbf{y}} \perp S$.

Proof. Let $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{t}$ be the decomposition of Theorem 2.15. For any $\mathbf{s} \in S$, $\hat{\mathbf{y}} - \mathbf{s} \in S$ is orthogonal to $\mathbf{t}$, so by Pythagoras

$$\|\mathbf{y} - \mathbf{s}\|^2 = \|\mathbf{t} + (\hat{\mathbf{y}} - \mathbf{s})\|^2 = \|\mathbf{t}\|^2 + \|\hat{\mathbf{y}} - \mathbf{s}\|^2 \;\geq\; \|\mathbf{y} - \hat{\mathbf{y}}\|^2,$$

with equality iff $\mathbf{s} = \hat{\mathbf{y}}$. Conversely, if $\mathbf{s}^* \in S$ satisfies $\mathbf{y} - \mathbf{s}^* \perp S$, then $\mathbf{y} = \mathbf{s}^* + (\mathbf{y} - \mathbf{s}^*)$ is the orthogonal decomposition, so $\mathbf{s}^* = \hat{\mathbf{y}}$ by uniqueness. □

Now let $S = \operatorname{col}(X)$ with $X \in \mathbb{R}^{n \times k}$ of full column rank. Writing $\hat{\mathbf{y}} = X\mathbf{b}$, the orthogonality condition $X^\top(\mathbf{y} - X\mathbf{b}) = \mathbf{0}$ gives $\mathbf{b} = (X^\top X)^{-1} X^\top \mathbf{y}$ (invertibility by Corollary 2.10), hence

$$\hat{\mathbf{y}} = P\mathbf{y}, \qquad P = X(X^\top X)^{-1} X^\top, \qquad M = \mathbf{I}_n - P. \tag{2.6}$$

P is called the *hat matrix* or *projection matrix* (it "puts the hat on $\mathbf{y}$"), and M the *annihilator* or *residual maker*.

Theorem 2.17 (Properties of P and M)**.** *Let $X \in \mathbb{R}^{n \times k}$ have rank k. Then*

1. *$P^\top = P$, $P^2 = P$, $M^\top = M$, $M^2 = M$, and $PM = MP = 0$;*
2. *$PX = X$, $MX = 0$; $\operatorname{col}(P) = \operatorname{col}(X)$ and $\operatorname{col}(M) = \operatorname{col}(X)^\perp = \operatorname{null}(X^\top)$;*
3. *the eigenvalues of P are 1 (multiplicity k) and 0 (multiplicity $n - k$);*
4. *$\operatorname{tr}(P) = \operatorname{rank}(P) = k$ and $\operatorname{tr}(M) = \operatorname{rank}(M) = n - k$;*
5. *$0 \leq P_{ii} \leq 1$ for every i.*

Proof. (1) $P^\top = X(X^\top X)^{-1} X^\top = P$ since $X^\top X$ is symmetric; $P^2 = X(X^\top X)^{-1}(X^\top X)(X^\top X)^{-1} X^\top = P$. Then $M^2 = \mathbf{I} - 2P + P^2 = M$ and $PM = P - P^2 = 0$. (2) $PX = X(X^\top X)^{-1} X^\top X = X$, so $MX = 0$. Clearly $\operatorname{col}(P) \subseteq \operatorname{col}(X)$ and $\operatorname{col}(X) = \operatorname{col}(PX) \subseteq \operatorname{col}(P)$. For $\mathbf{y} \in \mathbb{R}^n$, $X^\top M\mathbf{y} = (MX)^\top \mathbf{y} = \mathbf{0}$, so $\operatorname{col}(M) \subseteq \operatorname{null}(X^\top)$; and if $X^\top \mathbf{y} = \mathbf{0}$ then $P\mathbf{y} = \mathbf{0}$ and $M\mathbf{y} = \mathbf{y}$. (3) If $P\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$, then $\lambda\mathbf{v} = P\mathbf{v} = P^2\mathbf{v} = \lambda^2\mathbf{v}$, so $\lambda \in \{0, 1\}$. As P is symmetric it is orthogonally diagonalisable, and the eigenspace for 1 is $\operatorname{col}(P) = \operatorname{col}(X)$, of dimension k. (4) The trace is the sum of the eigenvalues. Directly: $\operatorname{tr}(P) = \operatorname{tr}\big((X^\top X)^{-1} X^\top X\big) = \operatorname{tr}(\mathbf{I}_k) = k$, using $\operatorname{tr}(AB) = \operatorname{tr}(BA)$. (5) $P_{ii} = (P^2)_{ii} = \sum_j P_{ij}^2 \geq P_{ii}^2$. □

The next result says that symmetry and idempotency *characterise* orthogonal projections, whatever the subspace.

Theorem 2.18 (Orthogonal projectors)**.** *A matrix $Q \in \mathbb{R}^{n \times n}$ is the orthogonal projector onto $\operatorname{col}(Q)$ iff $Q^2 = Q$ and $Q^\top = Q$. The orthogonal projector onto a given subspace is unique.*

Proof. If Q is the orthogonal projector onto S, then $Q\mathbf{s} = \mathbf{s}$ for $\mathbf{s} \in S$, so $Q^2 = Q$; and for any $\mathbf{u}, \mathbf{v}$, $\mathbf{u}^\top Q\mathbf{v} = (Q\mathbf{u})^\top (Q\mathbf{v}) = \mathbf{u}^\top Q^\top \mathbf{v}$ since $\mathbf{u} - Q\mathbf{u} \perp Q\mathbf{v}$ and $\mathbf{v} - Q\mathbf{v} \perp Q\mathbf{u}$; hence $Q = Q^\top$. Conversely, if $Q^2 = Q = Q^\top$, write $\mathbf{y} = Q\mathbf{y} + (\mathbf{I} - Q)\mathbf{y}$. For any $Q\mathbf{z} \in \operatorname{col}(Q)$, $(Q\mathbf{z})^\top (\mathbf{I} - Q)\mathbf{y} = \mathbf{z}^\top (Q - Q^2)\mathbf{y} = 0$, so this is the orthogonal decomposition of $\mathbf{y}$ relative to $\operatorname{col}(Q)$. Uniqueness follows from uniqueness of the decomposition. □

A consequence is that P depends on X only through $\operatorname{col}(X)$: any re-parametrisation $X \mapsto XA$ with A invertible (rescaling variables, changing the base category of a set of dummies, replacing x_1, x_2 by $x_1 + x_2, x_1 - x_2$) leaves fitted values and residuals unchanged.

★ Key Concept: Hat matrix and annihilator

$P = X(X^\top X)^{-1}X^\top$ projects orthogonally onto the space of fitted values $\mathrm{col}(X)$, and $M = \mathbf{I} - P$ onto the residual space $\mathrm{col}(X)^\perp$. Both are symmetric and idempotent, $PM = 0$, $\operatorname{tr} P = k$ and $\operatorname{tr} M = n - k$. The trace identity is where the "degrees of freedom" $n - k$ in $s^2 = \mathbf{e}^\top\mathbf{e}/(n-k)$ come from (Chapter 6).

△ Caution: Projections that are not orthogonal, and projections that are too big

(i) A matrix with $Q^2 = Q$ but $Q \neq Q^\top$ is an *oblique* projector: it projects onto $\mathrm{col}(Q)$ along a direction that is not orthogonal. Weighted and instrumental variables estimators produce such projectors (e.g. $X(Z^\top X)^{-1}Z^\top$), and Pythagoras fails for them. (ii) Never form P explicitly for real data: with $n = 10^5$ survey households, P has 10^{10} entries (80 GB). Compute $P\mathbf{y}$ as $X\mathbf{b}$, and the diagonal P_{ii} from a thin QR factorisation (Section 2.4).

2.3.3 Pythagoras and the ANOVA decomposition

Since $\mathbf{y} = P\mathbf{y} + M\mathbf{y}$ with $(P\mathbf{y})^\top(M\mathbf{y}) = \mathbf{y}^\top PM\mathbf{y} = 0$,

$$\|\mathbf{y}\|^2 = \|P\mathbf{y}\|^2 + \|M\mathbf{y}\|^2 . \tag{2.7}$$

If X contains a column of ones, then $\mathbf{1} \in \mathrm{col}(X)$ and the projector $P_{\mathbf{1}} = \mathbf{1}\mathbf{1}^\top/n$ onto the constant satisfies $PP_{\mathbf{1}} = P_{\mathbf{1}}$. Applying (2.7) to the centred vector $(\mathbf{I} - P_{\mathbf{1}})\mathbf{y} = \mathbf{y} - \bar{y}\mathbf{1}$ gives the analysis-of-variance identity

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{ESS}} + \underbrace{\sum_i e_i^2}_{\text{RSS}}, \qquad R^2 = \frac{\text{ESS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}. \tag{2.8}$$

(The cross term vanishes because $\hat{\mathbf{y}} - \bar{y}\mathbf{1} \in \mathrm{col}(X)$ and $\mathbf{e} \perp \mathrm{col}(X)$.) Geometrically, $R^2 = \cos^2\theta$, where θ is the angle between the centred outcome and its projection onto the centred column space. Without an intercept, (2.8) fails and the "centred" R^2 can be negative or misleading.

Listing 2.4: Hat matrix, annihilator and the ANOVA decomposition ch02/projection.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Hat matrix P and annihilator M: symmetry, idempotency, trace and Pythagoras
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, k = 60, 4
X = np.column_stack([np.ones(n), rng.normal(size=(n, k - 1))])
y = X @ np.array([2.0, 1.0, -0.5, 0.0]) + rng.normal(size=n)

P = X @ np.linalg.solve(X.T @ X, X.T)     # hat (projection) matrix
M = np.eye(n) - P                         # annihilator (residual maker)

print("Property                    P           M")
print(f"symmetric  max|A - A^T|  {np.abs(P - P.T).max():9.1e}  "
      f"{np.abs(M - M.T).max():9.1e}")
print(f"idempotent max|A^2 - A|  {np.abs(P @ P - P).max():9.1e}  "
      f"{np.abs(M @ M - M).max():9.1e}")
print(f"trace                    {np.trace(P):9.4f}  {np.trace(M):9.4f}")
print(f"rank                     {np.linalg.matrix_rank(P):9d}  "
```

```
      f"{np.linalg.matrix_rank(M):9d}")
ev = np.linalg.eigvalsh(P)
print(f"eigenvalues of P: {np.sum(np.isclose(ev, 1))} ones, "
      f"{np.sum(np.isclose(ev, 0, atol=1e-10))} zeros")
print(f"max|PM| = {np.abs(P @ M).max():.1e},  max|MX| = {np.abs(M @ X).max():.1e}")

yhat, e = P @ y, M @ y
print(f"\nyhat' e = {yhat @ e: .2e}   (orthogonal pieces)")
print(f"||y||^2 = {y @ y:.4f}   ||yhat||^2 + ||e||^2 = {yhat @ yhat + e @ e:.4f}")

# ANOVA decomposition about the mean (X contains a constant)
ybar = y.mean()
TSS = np.sum((y - ybar) ** 2)
ESS = np.sum((yhat - ybar) ** 2)
RSS = e @ e
print(f"TSS = {TSS:.4f}  ESS + RSS = {ESS + RSS:.4f}  R^2 = {ESS / TSS:.4f}")

# Projection onto a single vector a: P_a = a a' / a'a
a = X[:, 1]
Pa = np.outer(a, a) / (a @ a)
print(f"\nrank-one projector: trace = {np.trace(Pa):.4f}, "
      f"idempotent err = {np.abs(Pa @ Pa - Pa).max():.1e}")
# Oblique (non-orthogonal) projector: idempotent but not symmetric
Q = np.array([[1.0, 1.0], [0.0, 0.0]])
print(f"oblique Q: Q^2 = Q? {np.allclose(Q @ Q, Q)}, symmetric? "
      f"{np.allclose(Q, Q.T)}")
```

Output

```
Property                      P           M
symmetric  max|A - A^T|    3.1e-17    3.1e-17
idempotent max|A^2 - A|    5.6e-17    4.4e-16
trace                       4.0000    56.0000
rank                             4         56
eigenvalues of P: 4 ones, 56 zeros
max|PM| = 3.5e-17,  max|MX| = 4.4e-16

yhat' e = -1.86e-14   (orthogonal pieces)
||y||^2 = 389.0032   ||yhat||^2 + ||e||^2 = 389.0032
TSS = 149.0310  ESS + RSS = 149.0310  R^2 = 0.5687

rank-one projector: trace = 1.0000, idempotent err = 2.8e-17
oblique Q: Q^2 = Q? True, symmetric? False
```

All identities of Theorem 2.17 hold to rounding error. The rank-one projector $\mathbf{a}\mathbf{a}^\top/\mathbf{a}^\top\mathbf{a}$ is the building block of everything that follows: projecting onto a single direction is what both Gram–Schmidt and Householder methods do repeatedly.

2.3.4 Orthonormal bases and Gram–Schmidt

If the columns of $Q \in \mathbb{R}^{n\times k}$ are orthonormal ($Q^\top Q = \mathbf{I}_k$), the projector onto $\mathrm{col}(Q)$ is simply $P = QQ^\top$ and the coordinates of the projection are inner products: $P\mathbf{y} = \sum_j q_j(q_j^\top \mathbf{y})$. Orthonormal bases turn least squares into a sequence of one-dimensional projections. The Gram–Schmidt

process constructs one from any basis $\mathbf{a}_1, \dots, \mathbf{a}_k$:

$$\mathbf{v}_j = \mathbf{a}_j - \sum_{i<j} (q_i^\top \mathbf{a}_j)\, q_i, \qquad q_j = \mathbf{v}_j / \|\mathbf{v}_j\|, \qquad j = 1, \dots, k. \tag{2.9}$$

By induction $\operatorname{span}\{q_1, \dots, q_j\} = \operatorname{span}\{\mathbf{a}_1, \dots, \mathbf{a}_j\}$ for every j, and $\mathbf{v}_j \neq \mathbf{0}$ exactly because $\mathbf{a}_j$ is not in the span of the previous columns (linear independence). Collecting the coefficients $r_{ij} = q_i^\top \mathbf{a}_j$ ($i < j$) and $r_{jj} = \|\mathbf{v}_j\|$ gives the factorisation

$$A = QR, \qquad Q^\top Q = \mathbf{I}_k, \qquad R \text{ upper triangular with } r_{jj} > 0, \tag{2.10}$$

the (thin) *QR decomposition*. In regression terms, $\mathbf{v}_j$ is the residual from regressing $\mathbf{a}_j$ on the previous columns: Gram–Schmidt *is* sequential partialling-out, a fact we exploit in the Frisch–Waugh–Lovell theorem.

Algorithm 2 Modified Gram–Schmidt (MGS) for $A \in \mathbb{R}^{n \times k}$

$V \leftarrow A$
for $j = 1, \dots, k$ **do**
  $r_{jj} \leftarrow \|\mathbf{v}_j\|$; $q_j \leftarrow \mathbf{v}_j / r_{jj}$
  for $\ell = j+1, \dots, k$ **do**
    $r_{j\ell} \leftarrow q_j^\top \mathbf{v}_\ell$; $\mathbf{v}_\ell \leftarrow \mathbf{v}_\ell - r_{j\ell} q_j$ ▷ remove q_j from the *updated* columns
  end for
end for

In exact arithmetic classical Gram–Schmidt (CGS, formula (2.9)) and the modified version of Algorithm 2 are identical. In floating point they are not. CGS computes all coefficients $q_i^\top \mathbf{a}_j$ from the *original* column, so the rounding errors in earlier q_i are never corrected, and the computed Q can lose orthogonality completely; MGS projects the *current* residual, which corrects much of the error. The loss of orthogonality $\|Q^\top Q - \mathbf{I}\|$ grows like $\varepsilon_{\text{mach}} \kappa^2$ for CGS (until it saturates) and like $\varepsilon_{\text{mach}} \kappa$ for MGS, where κ is the condition number of A (Björck, 1996). Householder QR (Section 2.5) produces a Q that is orthogonal to machine precision regardless of κ. Listing 2.5 and Fig. 2.1 show the three regimes.

Listing 2.5: Loss of orthogonality: CGS, MGS and Householder ch02/gram_schmidt.py

```python
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Classical vs modified Gram-Schmidt vs Householder QR: loss of orthogonality
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt


def cgs(A):
    """Classical Gram-Schmidt: project the ORIGINAL column on earlier q's."""
    m, n = A.shape
    Q, R = np.zeros((m, n)), np.zeros((n, n))
    for j in range(n):
        R[:j, j] = Q[:, :j].T @ A[:, j]
        v = A[:, j] - Q[:, :j] @ R[:j, j]
        R[j, j] = np.linalg.norm(v)
        Q[:, j] = v / R[j, j]
    return Q, R


def mgs(A):
```

```python
    """Modified Gram-Schmidt: remove each q from the UPDATED remaining columns."""
    V = A.astype(float).copy()
    m, n = A.shape
    Q, R = np.zeros((m, n)), np.zeros((n, n))
    for j in range(n):
        R[j, j] = np.linalg.norm(V[:, j])
        Q[:, j] = V[:, j] / R[j, j]
        R[j, j + 1:] = Q[:, j] @ V[:, j + 1:]
        V[:, j + 1:] -= np.outer(Q[:, j], R[j, j + 1:])
    return Q, R


def test_matrix(m, n, kappa, rng):
    """Random m x n matrix with prescribed 2-norm condition number."""
    U, _ = np.linalg.qr(rng.normal(size=(m, n)))
    V, _ = np.linalg.qr(rng.normal(size=(n, n)))
    return U @ np.diag(np.logspace(0, -np.log10(kappa), n)) @ V.T


rng = np.random.default_rng(803)
kappas = np.logspace(1, 15, 15)
loss = {"Classical GS": [], "Modified GS": [], "Householder (np.linalg.qr)": []}
print(" cond(A)     CGS        MGS     Householder   ||A - QR||/||A|| (MGS)")
for i, kap in enumerate(kappas):
    A = test_matrix(80, 12, kap, rng)
    for name, f in zip(loss, [cgs, mgs, np.linalg.qr]):
        Q, R = f(A)
        loss[name].append(np.linalg.norm(Q.T @ Q - np.eye(12), 2))
    Qm, Rm = mgs(A)
    if i % 3 == 0:
        res = np.linalg.norm(A - Qm @ Rm) / np.linalg.norm(A)
        print(f"{kap:8.0e}  " + "  ".join(f"{loss[k][-1]:9.1e}" for k in loss)
              + f"     {res:8.1e}")

plt.figure(figsize=(6, 3.8))
for (name, vals), mk in zip(loss.items(), ["o", "s", "^"]):
    plt.loglog(kappas, vals, marker=mk, ms=4, label=name)
plt.loglog(kappas, 1e-16 * kappas, "k:", lw=1, label=r"$\epsilon\,\kappa$")
plt.loglog(kappas, 1e-16 * kappas ** 2, "k--", lw=1, label=r"$\epsilon\,\kappa^2$")
plt.ylim(1e-17, 10)
plt.xlabel(r"condition number $\kappa(A)$")
plt.ylabel(r"$\|Q^\top Q - I\|_2$")
plt.legend(fontsize=8)
plt.grid(alpha=0.3, which="major")
plt.tight_layout()
plt.savefig("ch02_gram_schmidt.pdf", bbox_inches="tight")
```

Output

```
 cond(A)     CGS        MGS     Householder   ||A - QR||/||A|| (MGS)
   1e+01    1.0e-15    1.2e-15    6.8e-16      1.3e-16
   1e+04    1.2e-10    9.0e-13    9.8e-16      1.2e-16
   1e+07    2.2e-05    2.5e-10    7.0e-16      1.3e-16
   1e+10    2.2e+00    7.7e-07    1.0e-15      1.2e-16
   1e+13    4.0e+00    4.1e-04    7.7e-16      1.0e-16
```

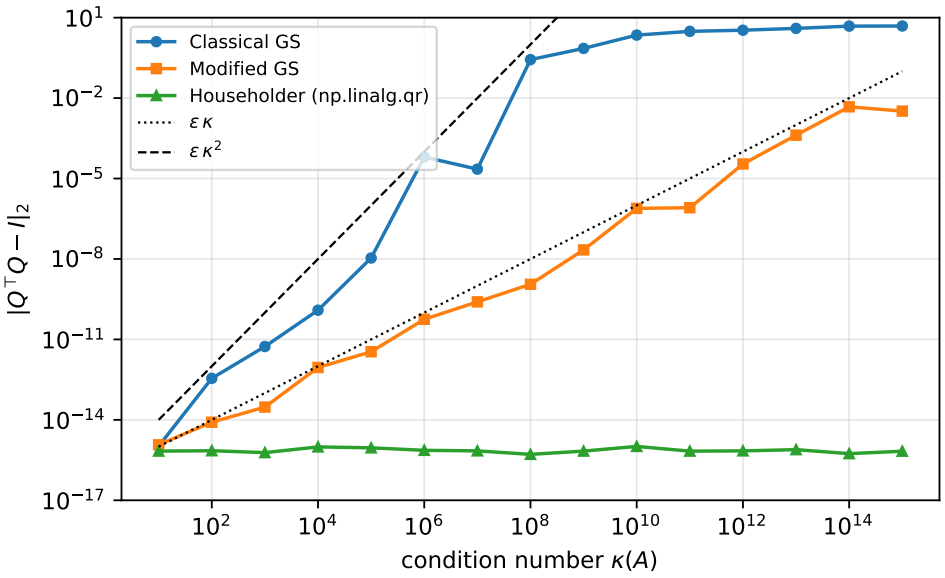


Figure 2.1: Loss of orthogonality $\|Q^\top Q - \mathbf{I}\|_2$ of the computed Q for 80×12 matrices with prescribed condition number. Classical Gram–Schmidt follows $\varepsilon_{\text{mach}}\kappa^2$ until Q is no longer orthogonal at all; modified Gram–Schmidt follows $\varepsilon_{\text{mach}}\kappa$; Householder QR stays at machine precision. In all three cases the product QR reproduces A to machine precision: the factorisation is accurate, but only Householder delivers an orthogonal Q.

Historical Note: Gram, Schmidt and an older idea

The Danish actuary and mathematician Jørgen Pedersen Gram (1850–1916) used successive orthogonalisation in an 1883 paper on series expansions by the method of least squares (Gram, 1883), and Erhard Schmidt (1876–1959) gave the now-standard recursive formulation for functions in his 1907 work on integral equations (Schmidt, 1907). Equivalent computations appear, in other guises, in still earlier least-squares work. That the "modified" ordering is numerically superior was established in the 1960s, in the work of Rice and of Björck (1967).

Exercises

2.3.1 Theory Let $X = [X_1, X_2]$ with full column rank and let P, P_1 be the projectors onto $\operatorname{col}(X)$ and $\operatorname{col}(X_1)$. Prove that $PP_1 = P_1P = P_1$, that $P - P_1$ is an orthogonal projector, and identify its range. Deduce that R^2 cannot decrease when a regressor is added.

2.3.2 Theory Show that if Q is an orthogonal projector then $\|Q\mathbf{y}\| \le \|\mathbf{y}\|$, and that $\mathbf{I} - 2Q$ is orthogonal and symmetric. What is its geometric meaning?

2.3.3 Theory Prove that R^2 equals the squared sample correlation between $\mathbf{y}$ and $\hat{\mathbf{y}}$ when X contains an intercept. Give a counterexample without an intercept.

2.3.4 Python Implement classical Gram–Schmidt with *reorthogonalisation* (apply the projection step twice, "CGS2") and add it to Fig. 2.1. How does its loss of orthogonality compare with MGS and Householder?

2.3.5 Python Using `load_diabetes` from scikit-learn, compute the hat matrix of the regression of the target on the ten features plus a constant via a thin QR, and verify numerically every statement of Theorem 2.17. Which observation has the largest P_{ii}?

2.3.6 ★ Graduate Let W be symmetric positive definite and define the W-inner product $\langle \mathbf{u}, \mathbf{v} \rangle_W = \mathbf{u}^\top W \mathbf{v}$. Derive the W-orthogonal projector onto $\operatorname{col}(X)$, show that it is idempotent but not

symmetric in the usual sense, and relate it to the generalised least squares estimator.

2.3.7 Policy A poverty analyst regresses household consumption on a set of asset indicators and reports $R^2 = 0.62$. A colleague adds a second set of asset indicators that are linear combinations of the first plus measurement noise, and R^2 rises to 0.66. Using the projection interpretation, explain why this increase is not evidence of better targeting, and propose a better comparison.

2.4 Least Squares as Projection

2.4.1 The normal equations

Given $\mathbf{y} \in \mathbb{R}^n$ and $X \in \mathbb{R}^{n \times k}$, the least-squares problem is

$$\min_{\mathbf{b} \in \mathbb{R}^k} S(\mathbf{b}) = \|\mathbf{y} - X\mathbf{b}\|^2 . \tag{2.11}$$

By the projection theorem (Theorem 2.16) with $S = \operatorname{col}(X)$, the minimum is attained at every $\mathbf{b}$ with $X\mathbf{b} = P\mathbf{y}$, and these are exactly the solutions of the *normal equations*

$$X^\top X\, \mathbf{b} = X^\top \mathbf{y}, \tag{2.12}$$

which simply state that the residual $\mathbf{e} = \mathbf{y} - X\mathbf{b}$ is orthogonal to every column of X. The same equations follow from calculus: $S(\mathbf{b}) = \mathbf{y}^\top\mathbf{y} - 2\mathbf{b}^\top X^\top \mathbf{y} + \mathbf{b}^\top X^\top X \mathbf{b}$ has gradient $-2X^\top\mathbf{y} + 2X^\top X\mathbf{b}$ and Hessian $2X^\top X \succeq 0$, so S is convex and its stationary points are its minimisers (Chapters 8 and 12).

Theorem 2.19 (Existence and uniqueness of least squares)**.** *The normal equations* (2.12) *are always consistent. The fitted values* $\hat{\mathbf{y}} = P\mathbf{y}$ *and residuals* $\mathbf{e} = M\mathbf{y}$ *are always unique. The coefficient vector is unique,* $\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$*, iff* X *has full column rank.*

Proof. $X^\top \mathbf{y} \in \operatorname{col}(X^\top) = \operatorname{col}(X^\top X)$ (the last equality because both have dimension $\operatorname{rank} X$ by Corollary 2.10, and $\operatorname{col}(X^\top X) \subseteq \operatorname{col}(X^\top)$), so a solution exists. Uniqueness of $\hat{\mathbf{y}}$ is the projection theorem; uniqueness of $\mathbf{b}$ holds iff $\operatorname{null}(X^\top X) = \operatorname{null}(X) = \{\mathbf{0}\}$. □

The projection picture is summarised in Fig. 2.2.

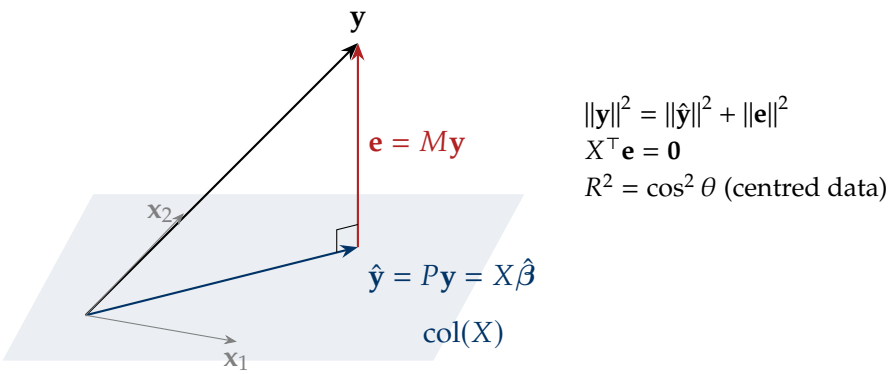


Figure 2.2: Least squares as orthogonal projection: the fitted vector is the point of $\operatorname{col}(X)$ closest to $\mathbf{y}$, and the residual is orthogonal to the plane spanned by the regressors.

2.4.2 Algebraic properties of residuals

From $X^\top \mathbf{e} = \mathbf{0}$ follow the familiar properties of OLS residuals, all of them consequences of geometry rather than of any statistical assumption:

1. residuals are orthogonal to every regressor and to the fitted values: $X^\top \mathbf{e} = \mathbf{0}$, $\hat{\mathbf{y}}^\top \mathbf{e} = 0$;
2. if X contains an intercept, $\sum_i e_i = 0$, the regression hyperplane passes through the means, $\bar{y} = \bar{\mathbf{x}}^\top \hat{\boldsymbol{\beta}}$, and the mean of $\hat{y}_i$ equals $\bar{y}$;
3. $\mathbf{e} = M\mathbf{y} = M(X\boldsymbol{\beta} + \boldsymbol{\varepsilon}) = M\boldsymbol{\varepsilon}$: the residual is the error projected onto $\operatorname{col}(X)^\perp$, so $\mathbf{e}^\top \mathbf{e} = \boldsymbol{\varepsilon}^\top M \boldsymbol{\varepsilon} \le \boldsymbol{\varepsilon}^\top \boldsymbol{\varepsilon}$;
4. $\mathbb{E}[\mathbf{e}^\top \mathbf{e} \mid X] = \sigma^2 \operatorname{tr}(M) = \sigma^2 (n-k)$ under $\mathbb{E}[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top \mid X] = \sigma^2 \mathbf{I}$ (proved in Chapter 6).

The third property explains why residuals "look better" than errors: projection removes k dimensions of noise.

2.4.3 R^2 and its limits

From (2.8), $R^2 \in [0,1]$ when an intercept is included, and R^2 is non-decreasing in the set of regressors because projecting onto a larger subspace cannot increase the residual (Exercise 2.3.1). With $k = n$ linearly independent regressors, $R^2 = 1$ regardless of the data. The adjusted $\bar{R}^2 = 1 - \frac{\mathrm{RSS}/(n-k)}{\mathrm{TSS}/(n-1)}$ penalises dimension by using the trace identity of Theorem 2.17 for both numerator and denominator. Neither is a measure of causal validity, and in prediction problems out-of-sample error is the relevant criterion (Hastie et al., 2009, Ch. 7).

Historical Note: Legendre, Gauss and the priority dispute

The method of least squares was first published by Adrien-Marie Legendre in 1805, in an appendix to his work on the orbits of comets, where he also coined the name *méthode des moindres carrés*. In 1809, in *Theoria motus corporum coelestium*, Carl Friedrich Gauss presented the method with a probabilistic justification (normally distributed errors) and stated that he had been using it since 1795. Gauss's fame in astronomy rested partly on his prediction of the orbit of Ceres, discovered by Giuseppe Piazzi on 1 January 1801 and lost in the glare of the Sun after a few weeks of observation; using Gauss's computations, astronomers recovered it at the end of 1801. Legendre objected strongly to Gauss's claim of priority. Stigler (1981) reviews the evidence for and against Gauss's claim; whatever its merits, Legendre's publication is the one that first made the method known.

2.4.4 The Frisch–Waugh–Lovell theorem

Partition the regressors as $X = [X_1, X_2]$ with $X_1 \in \mathbb{R}^{n \times k_1}$, $X_2 \in \mathbb{R}^{n \times k_2}$, and write the OLS fit as

$$\mathbf{y} = X_1 \hat{\boldsymbol{\beta}}_1 + X_2 \hat{\boldsymbol{\beta}}_2 + \mathbf{e}. \tag{2.13}$$

Let $P_1 = X_1 (X_1^\top X_1)^{-1} X_1^\top$ and $M_1 = \mathbf{I} - P_1$; $M_1 \mathbf{v}$ is the residual from regressing $\mathbf{v}$ on X_1.

Theorem 2.20 (Frisch–Waugh–Lovell). *If $X = [X_1, X_2]$ has full column rank, then*

$$\hat{\boldsymbol{\beta}}_2 = (X_2^\top M_1 X_2)^{-1} X_2^\top M_1 \mathbf{y} = \left(\tilde{X}_2^\top \tilde{X}_2\right)^{-1} \tilde{X}_2^\top \tilde{\mathbf{y}}, \qquad \tilde{X}_2 = M_1 X_2,\ \tilde{\mathbf{y}} = M_1 \mathbf{y}, \tag{2.14}$$

and the residuals from regressing $\tilde{\mathbf{y}}$ on $\tilde{X}_2$ are identical to the residuals $\mathbf{e}$ of the full regression.

Proof. Premultiply (2.13) by M_1. Since $M_1 X_1 = 0$, and since $\mathbf{e} \perp \operatorname{col}(X) \supseteq \operatorname{col}(X_1)$ implies $P_1 \mathbf{e} = \mathbf{0}$, i.e. $M_1 \mathbf{e} = \mathbf{e}$, we get

$$M_1 \mathbf{y} = M_1 X_2 \hat{\boldsymbol{\beta}}_2 + \mathbf{e}. \tag{2.15}$$

Multiply by $X_2^\top$: $X_2^\top M_1 \mathbf{y} = X_2^\top M_1 X_2 \hat{\boldsymbol{\beta}}_2 + X_2^\top \mathbf{e}$, and $X_2^\top \mathbf{e} = \mathbf{0}$ by the normal equations. The matrix $X_2^\top M_1 X_2 = \tilde{X}_2^\top \tilde{X}_2$ is invertible: if $\tilde{X}_2 \mathbf{c} = \mathbf{0}$ then $X_2 \mathbf{c} \in \operatorname{col}(X_1)$, i.e. $X_2 \mathbf{c} = X_1 \mathbf{d}$ for some $\mathbf{d}$, and full

column rank of X forces $\mathbf{c} = \mathbf{0}$. This gives (2.14), using $M_1^\top M_1 = M_1$. Finally, (2.15) expresses $\tilde{\mathbf{y}}$ as $\tilde{X}_2\hat{\boldsymbol{\beta}}_2$ plus a vector $\mathbf{e}$ orthogonal to $\mathrm{col}(\tilde{X}_2)$ (because $\tilde{X}_2^\top \mathbf{e} = X_2^\top M_1 \mathbf{e} = X_2^\top \mathbf{e} = \mathbf{0}$), which is the orthogonal decomposition defining the residual of that regression. □

⋆ Key Concept: "Controlling for" means projecting out

The coefficient on X_2 in a multiple regression equals the coefficient from regressing the part of $\mathbf{y}$ not explained by X_1 on the part of X_2 not explained by X_1. "Holding X_1 constant" is not a thought experiment performed by the estimator; it is the orthogonal projection M_1 applied to both sides. The identifying variation for $\boldsymbol{\beta}_2$ is exactly the variation in X_2 that is orthogonal to X_1, so if $M_1 X_2 \approx 0$, there is (almost) nothing to learn from.

Two remarks. First, it suffices to residualise X_2: $(\tilde{X}_2^\top \tilde{X}_2)^{-1}\tilde{X}_2^\top \mathbf{y}$ gives the same $\hat{\boldsymbol{\beta}}_2$, because $\tilde{X}_2^\top M_1 = \tilde{X}_2^\top$; but residualising only $\mathbf{y}$ does *not* work. Second, the theorem is purely algebraic, so it holds for any estimator defined by least squares, including penalised least squares with penalty only on $\boldsymbol{\beta}_2$.

2.4.5 Fixed effects as demeaning

Consider a panel of workers $i = 1, \dots, N$ observed in years $t = 1, \dots, T$, with

$$y_{it} = \beta x_{it} + \alpha_i + \gamma_t + \varepsilon_{it}, \tag{2.16}$$

where α_i is an unobserved worker effect (ability, location, family background) that may be correlated with x_{it}. The least-squares dummy-variable (LSDV) estimator regresses y on x, year dummies and N worker dummies $D \in \{0,1\}^{NT \times N}$. Take $X_1 = D$ in Theorem 2.20. Since $D^\top D = T\,\mathbf{I}_N$,

$$P_D \mathbf{v} = D(D^\top D)^{-1} D^\top \mathbf{v} = (\text{vector of worker means } \bar{v}_i), \qquad (M_D \mathbf{v})_{it} = v_{it} - \bar{v}_i.$$

Hence the LSDV slope equals the slope of the *within* regression of the demeaned outcome $y_{it} - \bar{y}_i$ on the demeaned regressors. This replaces a regression with $N + T$ columns by one with T columns (`np.bincount` computes all group means in one pass), and it is how every fixed-effects routine works in practice.

Listing 2.6 simulates a wage panel of $N = 400$ workers in 16 regions over $T = 5$ years, in which training hours are higher for workers with higher wage potential. Pooled OLS is badly biased upwards; region fixed effects remove only the regional part of the confounding; worker fixed effects recover $\beta = 0.10$. LSDV and demeaning agree to all printed digits and their residuals coincide, exactly as the theorem says.

Listing 2.6: Frisch–Waugh–Lovell: LSDV equals the within estimator `ch02/fwl_panel.py`

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Frisch-Waugh-Lovell: fixed effects by dummies (LSDV) = within (demeaning)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
N, T, G, beta = 400, 5, 16, 0.10          # workers, years, regions, true slope
region = rng.integers(0, G, size=N)
mu_region = rng.normal(0, 0.5, size=G)     # regional wage levels (simulated)
alpha = mu_region[region] + rng.normal(0, 0.3, size=N)   # worker effects
ids = np.repeat(np.arange(N), T)
year = np.tile(np.arange(T), N)
```

```
# training hours are higher where wages are high: correlated with alpha
x = 2.0 * alpha[ids] + rng.normal(0, 1, size=N * T)
y = beta * x + alpha[ids] + 0.03 * year + rng.normal(0, 0.2, size=N * T)

Dw = np.eye(N)[ids]                          # N*T x N worker dummies
Dt = np.eye(T)[year][:, 1:]                  # year dummies (base year dropped)
ols = lambda X, v: np.linalg.lstsq(X, v, rcond=None)[0]

b_pool = ols(np.column_stack([np.ones(N * T), x, Dt]), y)[1]
X_lsdv = np.column_stack([x, Dt, Dw])
coef_lsdv = ols(X_lsdv, y)
e_lsdv = y - X_lsdv @ coef_lsdv


def demean(v, g):
    """Subtract group means: this is M_D v for the group dummy matrix D."""
    means = np.bincount(g, weights=v) / np.bincount(g)
    return v - means[g]


yw, xw = demean(y, ids), demean(x, ids)
Dtw = np.column_stack([demean(c, ids) for c in Dt.T])
coef_w = ols(np.column_stack([xw, Dtw]), yw)
e_w = yw - np.column_stack([xw, Dtw]) @ coef_w
b_reg = ols(np.column_stack([demean(x, region[ids]), Dt]),
            demean(y, region[ids]))[0]

print(f"design sizes: LSDV {X_lsdv.shape},  within {(N * T, 1 + T - 1)}")
print(f"true beta                     = {beta:.4f}")
print(f"pooled OLS (ignores alpha_i)  = {b_pool:.4f}")
print(f"region fixed effects only     = {b_reg:.4f}")
print(f"LSDV with {N} worker dummies  = {coef_lsdv[0]:.6f}")
print(f"within (demeaned, FWL)        = {coef_w[0]:.6f}")
print(f"max |e_LSDV - e_within|       = {np.abs(e_lsdv - e_w).max():.1e}")
print(f"year effects equal?           {np.allclose(coef_lsdv[1:T], coef_w[1:])}")
dof = N * T - N - (T - 1) - 1
print(f"residual dof: correct = {dof}, naive within = {N * T - T}")

# Figure: pooled scatter vs within scatter (subsample of 6 regions)
sel = np.isin(region[ids], np.arange(6))
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.4), sharey=False)
for g in range(6):
    m = sel & (region[ids] == g)
    ax[0].scatter(x[m], y[m], s=5, alpha=0.5)
    ax[1].scatter(xw[m], yw[m], s=5, alpha=0.5)
for a, (xx, yy, b, ttl) in zip(ax, [(x, y, b_pool, "pooled OLS"),
                                    (xw, yw, coef_w[0], "within (demeaned)")]):
    grid = np.linspace(xx[sel].min(), xx[sel].max(), 2)
    c0 = yy[sel].mean() - b * xx[sel].mean()
    a.plot(grid, c0 + b * grid, "k-", lw=2)
    a.set_title(f"{ttl}: slope = {b:.3f}", fontsize=10)
    a.set_xlabel("training hours (simulated)")
ax[0].set_ylabel("log wage (simulated)")
fig.tight_layout()
fig.savefig("ch02_fwl_panel.pdf", bbox_inches="tight")
```

Output

```
design sizes: LSDV (2000, 405),  within (2000, 5)
true beta                      = 0.1000
pooled OLS (ignores alpha_i)   = 0.4054
region fixed effects only      = 0.2307
LSDV with 400 worker dummies   = 0.101862
within (demeaned, FWL)         = 0.101862
max |e_LSDV - e_within|        = 8.7e-15
year effects equal?            True
residual dof: correct = 1595, naive within = 1995
```

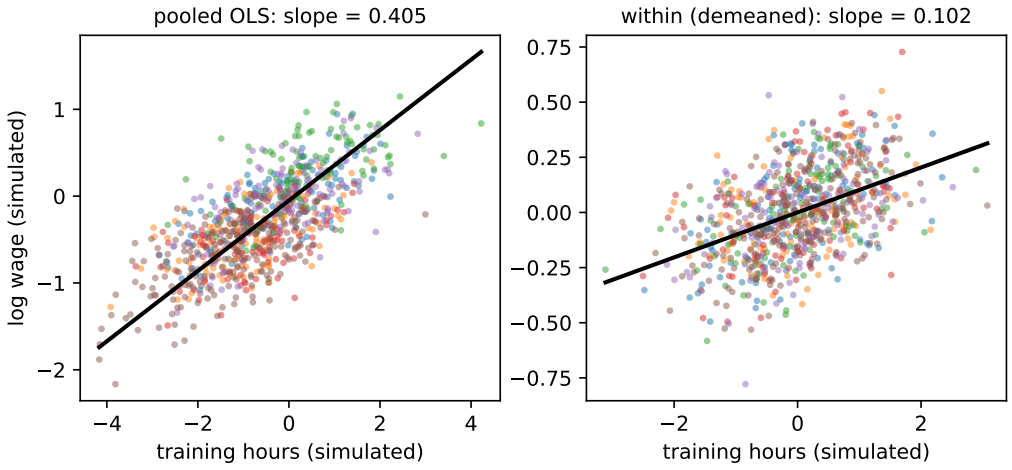


Figure 2.3: Simulated wage panel (six of sixteen regions shown). Left: pooled data, where workers with higher fixed wage levels also train more, so the pooled slope mixes the causal effect with the correlation between x_{it} and α_i. Right: after demeaning within workers (the projection M_D), the slope is the within-worker effect 0.10.

△ Caution: Degrees of freedom after demeaning

The within regression has T columns, but the model has $N+T$ parameters. Software that runs OLS on demeaned data and divides the RSS by $NT-T$ (here 1995) instead of $NT-N-T$ (1595) underestimates standard errors; in short panels the error is large ($\sqrt{1995/1595} \approx 1.12$ here). The residuals are identical, but $\mathrm{tr}(M)$ must count the projected-out dummies. Clustered standard errors raise further issues (Cameron and Trivedi, 2005).

Policy Lens: Fixed effects in district-level policy evaluation

Ghana's Metropolitan, Municipal and District Assemblies differ in many persistent ways (remoteness, soil quality, ethnic composition, administrative capacity) that also affect which districts receive programmes first. With a panel of district outcomes, district fixed effects project out every time-invariant district characteristic, observed or not; year effects project out national shocks. By FWL, the programme effect is then estimated only from *changes* within districts relative to the national trend. Two consequences follow directly from the geometry: (i) any time-invariant district attribute (e.g. "coastal") has $M_D\mathbf{x} = \mathbf{0}$ and its coefficient is not identified; (ii) if roll-out timing is almost the same everywhere, $M_D X_2$ is small and the estimate is imprecise, however many districts there are.

Historical Note: Frisch, Waugh and Lovell

Frisch and Waugh (1933) showed in the first volume of *Econometrica* that regressing detrended series on each other gives the same coefficients as including a time trend in the regression. Michael Lovell, studying seasonal adjustment, proved the general result for arbitrary sets of regressors (Lovell, 1963), and later gave a short textbook proof (Lovell, 2008). Frederick Waugh was an agricultural economist at the US Department of Agriculture.

Graduate Perspective: FWL and double machine learning

In the partially linear model $y = \theta d + g(\mathbf{z}) + \varepsilon$, Robinson (1988) showed that θ can be estimated at the parametric rate by regressing $y - \mathbb{E}[y \mid \mathbf{z}]$ on $d - \mathbb{E}[d \mid \mathbf{z}]$: a nonparametric version of FWL, with the linear projection M_1 replaced by a conditional expectation. Double/debiased machine learning (Chernozhukov et al., 2018) estimates both conditional expectations with flexible learners (random forests, lasso, neural networks), uses sample splitting (cross-fitting) to avoid overfitting bias, and then applies exactly the residual-on-residual regression of Theorem 2.20. This is now a standard tool for policy evaluation with many controls.

2.4.6 Leverage

The diagonal elements $h_{ii} = P_{ii} = \mathbf{x}_i^\top (X^\top X)^{-1} \mathbf{x}_i$ of the hat matrix are the *leverages*: $\partial \hat{y}_i / \partial y_i = h_{ii}$, so h_{ii} measures how much observation i pulls its own fitted value towards itself (Hoaglin and Welsch, 1978).

Proposition 2.21 (Properties of leverage). *Let $X \in \mathbb{R}^{n \times k}$ have full column rank.*

1. *$0 \le h_{ii} \le 1$ and $\sum_i h_{ii} = k$, so the average leverage is k/n.*
2. *If X contains an intercept, $h_{ii} \ge 1/n$, and $h_{ii} = \frac{1}{n} + (\mathbf{x}_i - \bar{\mathbf{x}})_*^\top (\tilde{X}^\top \tilde{X})^{-1} (\mathbf{x}_i - \bar{\mathbf{x}})_*$, where $*$ denotes the non-constant regressors and $\tilde{X}$ their centred design: leverage grows with the Mahalanobis distance of $\mathbf{x}_i$ from the centre.*
3. *If $X = QR$ is a thin QR factorisation, $h_{ii} = \|\mathbf{q}_i\|^2$, where $\mathbf{q}_i^\top$ is the i-th row of Q.*
4. *The leave-one-out residual satisfies $y_i - \mathbf{x}_i^\top \hat{\boldsymbol{\beta}}_{(-i)} = e_i / (1 - h_{ii})$.*

Proof sketch. (1) is Theorem 2.17(4),(5). (2) Apply FWL with $X_1 = \mathbf{1}$: $P = P_{\mathbf{1}} + P_{\tilde{X}}$ where $\tilde{X} = M_{\mathbf{1}} X_*$, and take diagonals. (3) $P = QQ^\top$. (4) Removing row i changes $X^\top X$ by the rank-one matrix $\mathbf{x}_i \mathbf{x}_i^\top$; the Sherman–Morrison formula $(A - \mathbf{u}\mathbf{u}^\top)^{-1} = A^{-1} + A^{-1}\mathbf{u}\mathbf{u}^\top A^{-1} / (1 - \mathbf{u}^\top A^{-1} \mathbf{u})$ with $A = X^\top X$, $\mathbf{u} = \mathbf{x}_i$ yields $\hat{\boldsymbol{\beta}}_{(-i)} = \hat{\boldsymbol{\beta}} - (X^\top X)^{-1} \mathbf{x}_i e_i / (1 - h_{ii})$, and the claim follows by multiplying by $\mathbf{x}_i^\top$. □

Part (4) is the basis of the closed-form leave-one-out cross-validation error $\frac{1}{n} \sum_i \{e_i / (1 - h_{ii})\}^2$ for linear smoothers, and of influence measures such as Cook's distance (Belsley et al., 1980). A point with $h_{ii} = 1$ is fitted exactly whatever its outcome: its residual is zero and it contributes nothing to the RSS. This happens, for instance, when a dummy variable is non-zero for a single observation.

Listing 2.7: Leverage from a thin QR factorisation ch02/leverage.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Leverage h_ii = diagonal of the hat matrix, computed stably from QR
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n = 200
```

```python
hh_size = rng.poisson(4, size=n) + 1.0             # simulated household size
income = 2.0 + 0.3 * hh_size + rng.normal(0, 0.5, n)
hh_size[0], income[0] = 25.0, 4.0                  # one atypical household
X = np.column_stack([np.ones(n), hh_size])
k = X.shape[1]

Q, R = np.linalg.qr(X)                             # P = Q Q^T
h = np.sum(Q ** 2, axis=1)                         # h_ii = ||q_i||^2
print(f"sum of leverages = {h.sum():.4f}  (= k = {k})")
print(f"min h = {h.min():.4f} (>= 1/n = {1 / n:.4f}),  max h = {h.max():.4f}")
print(f"average k/n = {k / n:.4f};  rule of thumb flags h > 2k/n = {2 * k / n:.3f}")
print(f"flagged observations: {np.flatnonzero(h > 2 * k / n)}")

b_all = np.linalg.lstsq(X, income, rcond=None)[0]
b_drop = np.linalg.lstsq(X[1:], income[1:], rcond=None)[0]
print(f"\nslope with obs 0    = {b_all[1]:.4f}")
print(f"slope without obs 0 = {b_drop[1]:.4f}")

# Leave-one-out residual from a single fit: e_i / (1 - h_ii)
e = income - X @ b_all
loo_formula = e[0] / (1 - h[0])
loo_direct = income[0] - X[0] @ b_drop
print(f"LOO residual: formula {loo_formula:.4f}, refit {loo_direct:.4f}")

# A dummy for a region with a single observation has leverage exactly one
Xd = np.column_stack([X, np.eye(n)[:, 5]])
hd = np.sum(np.linalg.qr(Xd)[0] ** 2, axis=1)
print(f"\nleverage of the lone obs in its own dummy: h = {hd[5]:.6f}")
```

Output

```
sum of leverages = 2.0000  (= k = 2)
min h = 0.0050 (>= 1/n = 0.0050),  max h = 0.3191
average k/n = 0.0100;  rule of thumb flags h > 2k/n = 0.020
flagged observations: [  0  41  68  76 100 109 111]

slope with obs 0    = 0.2105
slope without obs 0 = 0.2967
LOO residual: formula -5.4398, refit -5.4398

leverage of the lone obs in its own dummy: h = 1.000000
```

The atypical household (size 25) has leverage 0.32, more than thirty times the average; including it lowers the estimated slope from 0.30 to 0.21. The leave-one-out formula reproduces the refitted residual without refitting.

Exercises

2.4.1 Theory Show that $\hat{\boldsymbol{\beta}}$ is invariant (up to the corresponding linear transformation) under $X \mapsto XA$ for invertible A, and that $\hat{\mathbf{y}}$ and $\mathbf{e}$ do not change at all. What does this imply for changing the base category of a set of dummies?

2.4.2 Theory Prove the "only residualise X_2" remark following Theorem 2.20, and show by a 2×2 example that regressing $M_1\mathbf{y}$ on X_2 (without residualising X_2) generally gives a different coefficient.

2.4.3 Theory Use FWL to show that in the simple regression $y_i = a + bx_i + e_i$, $\hat{b} = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sum_i (x_i - \bar{x})^2$, and that adding a linear time trend to a regression of one trending series on another is equivalent to detrending both (the original Frisch–Waugh result).

2.4.4 Python Extend Listing 2.6 to an *unbalanced* panel (drop each observation independently with probability 0.3). Verify that LSDV still equals one-way demeaning for the slope when year effects are omitted, but that with year effects one-way demeaning of y and x alone is no longer enough. What is the correct FWL recipe?

2.4.5 Python Compute leave-one-out cross-validation errors for polynomial regressions of degree 1 to 8 on a simulated data set using only one fit per degree (Proposition 2.21(4)), and check against brute-force refitting.

2.4.6 Policy In a district panel, a programme was introduced in 2015 in some districts and in 2018 in others. A researcher also wants to include "distance to the regional capital" as a control in a district fixed-effects regression. Explain, using M_D, what happens to that coefficient, and how its interaction with a time trend could still be identified.

2.4.7 ★ Graduate (Two-way fixed effects.) For a *balanced* panel show that the two-way within transformation $\ddot{v}_{it} = v_{it} - \bar{v}_{i\cdot} - \bar{v}_{\cdot t} + \bar{v}_{\cdot\cdot}$ is the orthogonal projection onto the complement of $\mathrm{col}([D_{\text{worker}}, D_{\text{year}}])$. Why does this fail for unbalanced panels, and how do iterative demeaning (alternating projections) algorithms overcome it?

2.4.8 ★ Graduate Recent work on staggered difference-in-differences shows that the two-way fixed-effects coefficient is a weighted average of comparisons, some with negative weights. Using FWL, write the two-way fixed-effects estimator as $\sum_{it} w_{it} y_{it}$ with $w_{it} \propto \ddot{D}_{it}$, and explain why already treated units can act as controls.

2.5 Matrix Decompositions

A matrix decomposition writes A as a product of factors with simple structure (triangular, orthogonal, diagonal), so that solving systems, computing determinants or fitting least squares reduces to cheap operations on the factors. The rule of numerical linear algebra is: *factor once, then solve*; never compute an explicit inverse unless the inverse itself is wanted (Higham, 2002, Ch. 14).

2.5.1 LU decomposition

Gaussian elimination on a square A subtracts multiples ℓ_{ij} of row j from rows $i > j$ to create zeros below the diagonal. Recording the multipliers gives $A = LU$ with L unit lower triangular and U upper triangular. To avoid division by small pivots, rows are interchanged (*partial pivoting*), giving

$$PA = LU, \qquad |\ell_{ij}| \le 1, \tag{2.17}$$

with P a permutation matrix. Then $A\mathbf{x} = \mathbf{b}$ is solved by forward substitution $L\mathbf{z} = P\mathbf{b}$ followed by back substitution $U\mathbf{x} = \mathbf{z}$.

Theorem 2.22 (Existence of LU)**.** *Every square matrix has a factorisation $PA = LU$ with partial pivoting. If all leading principal sub-matrices of A are non-singular, $A = LU$ without pivoting, with L unit lower triangular, and this factorisation is unique.*

Proof sketch. Induction on the size. Write $A = \begin{pmatrix} a_{11} & \mathbf{w}^\top \\ \mathbf{v} & B \end{pmatrix}$ with $a_{11} \neq 0$ (after a row swap if necessary). Then

$$A = \begin{pmatrix} 1 & \mathbf{0}^\top \\ \mathbf{v}/a_{11} & \mathbf{I} \end{pmatrix} \begin{pmatrix} a_{11} & \mathbf{w}^\top \\ \mathbf{0} & B - \mathbf{v}\mathbf{w}^\top / a_{11} \end{pmatrix},$$

and the *Schur complement* $B - \mathbf{v}\mathbf{w}^{\top}/a_{11}$ is factored recursively. If no non-zero pivot exists in a column, that column is already eliminated and one moves on. Uniqueness: if $L_1U_1 = L_2U_2$ with non-singular factors, then $L_2^{-1}L_1 = U_2U_1^{-1}$ is both unit lower and upper triangular, hence $\mathbf{I}$. □

Cost. Elimination costs about $\frac{2}{3}n^3$ floating-point operations (flops); each subsequent solve costs $2n^2$. For $n = 1000$ that is roughly 7×10^8 flops for the factorisation but only 2×10^6 per right-hand side, so analysing many scenarios with the same matrix is nearly free once A is factored. Gaussian elimination with partial pivoting is backward stable in practice, although the theoretical growth factor can be as large as 2^{n-1} (Higham, 2002, Ch. 9).

2.5.2 Cholesky decomposition

Theorem 2.23 (Cholesky). *A symmetric matrix $A \in \mathbb{R}^{n\times n}$ is positive definite iff it can be written $A = LL^{\top}$ with L lower triangular with positive diagonal entries. This factorisation is unique.*

Proof. If $A = LL^{\top}$ with L non-singular, then $\mathbf{x}^{\top}A\mathbf{x} = \|L^{\top}\mathbf{x}\|^2 > 0$ for $\mathbf{x} \neq \mathbf{0}$. Conversely let A be SPD and write $A = \left(\begin{smallmatrix} a_{11} & \mathbf{v}^{\top} \\ \mathbf{v} & B \end{smallmatrix}\right)$. Then $a_{11} = \mathbf{e}_1^{\top}A\mathbf{e}_1 > 0$; put $\ell_{11} = \sqrt{a_{11}}$ and $\boldsymbol{\ell}_{21} = \mathbf{v}/\ell_{11}$. The Schur complement $S = B - \mathbf{v}\mathbf{v}^{\top}/a_{11}$ is SPD: for $\mathbf{z} \neq \mathbf{0}$ choose $x_1 = -\mathbf{v}^{\top}\mathbf{z}/a_{11}$, then $\mathbf{z}^{\top}S\mathbf{z} = (x_1, \mathbf{z}^{\top})A(x_1, \mathbf{z}^{\top})^{\top} > 0$. By induction $S = L_SL_S^{\top}$ and

$$A = \begin{pmatrix} \ell_{11} & \mathbf{0}^{\top} \\ \boldsymbol{\ell}_{21} & L_S \end{pmatrix}\begin{pmatrix} \ell_{11} & \boldsymbol{\ell}_{21}^{\top} \\ \mathbf{0} & L_S^{\top} \end{pmatrix}.$$

Each step is forced (positive square roots), which gives uniqueness. □

Cholesky costs $\frac{1}{3}n^3$ flops, half of LU, needs no pivoting for SPD matrices and is unconditionally stable. It is the method of choice for SPD systems: normal equations $X^{\top}X\mathbf{b} = X^{\top}\mathbf{y}$ (when they are well conditioned), ridge regression $(X^{\top}X + \lambda\mathbf{I})\mathbf{b} = X^{\top}\mathbf{y}$, Gaussian-process and kriging systems, and Newton steps with positive definite Hessians (Chapter 8). Because the algorithm breaks down (a non-positive pivot) exactly when A is not positive definite, attempting a Cholesky factorisation is also the cheapest test of definiteness.

Simulating correlated variables. If $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$ and $\boldsymbol{\Sigma} = LL^{\top}$, then $\mathbf{x} = \boldsymbol{\mu} + L\mathbf{z}$ satisfies $\mathrm{Cov}(\mathbf{x}) = L\,\mathbf{I}\,L^{\top} = \boldsymbol{\Sigma}$, so $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. This is how correlated shocks are generated in Monte Carlo studies, stress tests and Bayesian samplers (Chapter 10). Any square root $\boldsymbol{\Sigma} = BB^{\top}$ works (for instance the symmetric root from an eigendecomposition), but the Cholesky factor is the cheapest and, being triangular, has a recursive interpretation: the j-th variable is built from the first j shocks only.

Listing 2.8: LU for an input–output model; Cholesky for simulation ch02/lu_cholesky.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# LU for repeated solves (input-output model) and Cholesky for simulation
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import linalg

rng = np.random.default_rng(803)

# --- LU: a small Leontief input-output model (illustrative coefficients) ---
sectors = ["agric", "industry", "services"]
A = np.array([[0.15, 0.10, 0.05],      # input of row sector per unit
              [0.20, 0.30, 0.15],      # of column sector's output
              [0.10, 0.20, 0.25]])
```

```
L_mat = np.eye(3) - A
P, L, U = linalg.lu(L_mat)
print("LU of (I - A):  max|PLU - (I - A)| =",
      f"{np.abs(P @ L @ U - L_mat).max():.1e}")
lu_piv = linalg.lu_factor(L_mat)       # factor once ...
demands = np.array([[100, 80, 120],    # ... solve for many scenarios
                    [110, 80, 120],
                    [100, 95, 130]], dtype=float)
for d in demands:
    x = linalg.lu_solve(lu_piv, d)
    print(f"final demand {d} -> gross output {np.round(x, 1)}")

# --- Cholesky: simulate correlated (income, consumption, savings) shocks ---
sd = np.array([1.0, 0.8, 0.5])
R = np.array([[1.0, 0.7, 0.4],
              [0.7, 1.0, -0.2],
              [0.4, -0.2, 1.0]])
Sigma = np.outer(sd, sd) * R
Lc = np.linalg.cholesky(Sigma)         # Sigma = Lc Lc^T, Lc lower triangular
print("\nCholesky factor L:\n", np.round(Lc, 4))
Z = rng.standard_normal(size=(100_000, 3))
Xsim = Z @ Lc.T                        # rows are draws from N(0, Sigma)
print("target correlation (upper):", np.round(R[np.triu_indices(3, 1)], 3))
print("sample correlation (upper):",
      np.round(np.corrcoef(Xsim.T)[np.triu_indices(3, 1)], 3))

# Cholesky is a cheap positive-definiteness test
bad = np.array([[1.0, 0.9, 0.9], [0.9, 1.0, -0.9], [0.9, -0.9, 1.0]])
print("\neigenvalues of 'bad' correlation matrix:",
      np.round(np.linalg.eigvalsh(bad), 3))
try:
    np.linalg.cholesky(bad)
except np.linalg.LinAlgError as err:
    print("np.linalg.cholesky(bad) fails:", err)

# Solving an SPD system with Cholesky (as in the normal equations)
b = rng.normal(size=3)
c_and_low = linalg.cho_factor(Sigma)
x = linalg.cho_solve(c_and_low, b)
print(f"\nSPD solve residual ||Sigma x - b|| = {np.linalg.norm(Sigma @ x - b):.1e}")
```

Output

```
LU of (I - A):  max|PLU - (I - A)| = 2.8e-17
final demand [100.  80. 120.] -> gross output [156.2 209.7 236.7]
final demand [110.  80. 120.] -> gross output [168.7 213.8 239.5]
final demand [100.  95. 130.] -> gross output [160.7 236.9 257.9]

Cholesky factor L:
 [[ 1.      0.      0.    ]
 [ 0.56    0.5713  0.    ]
 [ 0.2    -0.3361  0.3115]]
target correlation (upper): [ 0.7  0.4 -0.2]
sample correlation (upper): [ 0.698  0.4   -0.203]

eigenvalues of 'bad' correlation matrix: [-0.8  1.9  1.9]
```

```
np.linalg.cholesky(bad) fails: Matrix is not positive definite

SPD solve residual ||Sigma x - b|| = 5.6e-17
```

The first block factors the Leontief matrix $\mathbf{I} - A$ of an illustrative three-sector economy once and evaluates three final-demand scenarios. Increasing final demand for agriculture by 10 units raises gross agricultural output by about 12.5 units and also raises the output of industry and services: these are the output multipliers. The second block generates 10^5 draws with a prescribed correlation matrix, and shows that a matrix with all entries in $[-1, 1]$ and unit diagonal need not be a valid correlation matrix: the Cholesky attempt fails because one eigenvalue is negative.

Historical Note: André-Louis Cholesky

André-Louis Cholesky (1875–1918) was a French artillery officer and geodesist who developed his method around 1910 to solve the normal equations arising in the least-squares adjustment of geodetic survey networks. He was killed in the final months of the First World War, in 1918. His method was published posthumously in 1924 by a fellow officer, Commandant Benoît (Benoît, 1924). Its use spread first in geodesy and only later in numerical analysis; Brezinski (2006) recounts his life and work.

△ Caution: Do not invert matrices to solve systems

`np.linalg.inv(A) @ b` costs about three times as much as `np.linalg.solve(A, b)` and is less accurate; `np.linalg.inv(X.T @ X) @ X.T @ y` combines this with the squaring of the condition number discussed below. Use `solve`, the factor-and-solve pairs `scipy.linalg.lu_factor` ↪ `/lu_solve` and `cho_factor/cho_solve` for repeated solves, and `np.linalg.lstsq` or QR for least squares.

2.5.3 QR decomposition by Householder reflections

Gram–Schmidt computes Q column by column, which loses orthogonality (Fig. 2.1). Householder's idea is the reverse: apply a sequence of *orthogonal* transformations to A until it becomes upper triangular.

Definition 2.24 (Householder reflector)**.** For $\mathbf{v} \neq \mathbf{0}$, $H = \mathbf{I} - 2\mathbf{v}\mathbf{v}^\top/(\mathbf{v}^\top\mathbf{v})$ is the reflection across the hyperplane $\mathbf{v}^\perp$. It is symmetric, orthogonal ($H^\top H = H^2 = \mathbf{I}$) and has eigenvalues -1 (once) and $+1$ ($n-1$ times).

Given $\mathbf{x} \in \mathbb{R}^m$, the choice $\mathbf{v} = \mathbf{x} + \operatorname{sign}(x_1)\,\|\mathbf{x}\|\,\mathbf{e}_1$ gives $H\mathbf{x} = -\operatorname{sign}(x_1)\,\|\mathbf{x}\|\,\mathbf{e}_1$: the reflector maps $\mathbf{x}$ onto the first coordinate axis. (The sign is chosen to avoid subtracting nearly equal numbers when forming $\mathbf{v}$.) Applying reflectors $H_1, \dots, H_k$ that zero the sub-diagonal part of columns $1, \dots, k$ in turn gives

$$H_k \cdots H_2 H_1 A = \begin{pmatrix} R \\ 0 \end{pmatrix}, \qquad A = Q \begin{pmatrix} R \\ 0 \end{pmatrix}, \quad Q = H_1 H_2 \cdots H_k \in \mathbb{R}^{m \times m}. \tag{2.18}$$

Q is never formed explicitly: one stores the vectors $\mathbf{v}_j$ and applies $H_j\mathbf{y} = \mathbf{y} - 2\mathbf{v}_j(\mathbf{v}_j^\top\mathbf{y})$ in $O(m)$ operations each. Because every step is an exact orthogonal transformation of the current matrix (up to rounding in a single reflector), the computed factors are those of a nearby matrix $A + \delta A$ with $\|\delta A\| \le c\,\varepsilon_{\text{mach}}\,\|A\|$: Householder QR is *backward stable* (Higham, 2002, Ch. 19).

2.5.4 Least squares via QR

Write $Q = [Q_1, Q_2]$ with $Q_1 \in \mathbb{R}^{n \times k}$. Since orthogonal matrices preserve norms,

$$\|\mathbf{y} - X\mathbf{b}\|^2 = \left\|Q^\top \mathbf{y} - Q^\top X\mathbf{b}\right\|^2 = \left\|Q_1^\top \mathbf{y} - R\mathbf{b}\right\|^2 + \left\|Q_2^\top \mathbf{y}\right\|^2 . \tag{2.19}$$

The second term does not depend on $\mathbf{b}$, and the first is zero for the solution of the triangular system

$$R\hat{\boldsymbol{\beta}} = Q_1^\top \mathbf{y}, \qquad \mathrm{RSS} = \left\|Q_2^\top \mathbf{y}\right\|^2 , \qquad P = Q_1 Q_1^\top . \tag{2.20}$$

The normal equations never appear. This matters because of conditioning.

Proposition 2.25 (Squaring the condition number). *For X of full column rank, $\kappa_2(X^\top X) = \kappa_2(X)^2$, where $\kappa_2(X) = \sigma_{\max}(X)/\sigma_{\min}(X)$.*

Proof. If $X = U\Sigma V^\top$ is the thin SVD, $X^\top X = V\Sigma^2 V^\top$, so the singular values of $X^\top X$ are the squares σ_i^2. □

A backward stable solver for a system with condition number κ delivers a relative error of order $\kappa\,\varepsilon_{\mathrm{mach}}$ (Chapter 1). Solving the normal equations therefore gives errors of order $\kappa(X)^2\varepsilon_{\mathrm{mach}}$, whereas Householder QR gives errors of order $\varepsilon_{\mathrm{mach}}\big(\kappa(X) + \kappa(X)^2 \tan\theta\big)$, where θ is the angle between $\mathbf{y}$ and $\mathrm{col}(X)$ (Higham, 2002, Ch. 20). When residuals are small relative to $\mathbf{y}$, QR is accurate to $\kappa\,\varepsilon_{\mathrm{mach}}$; when the fit is poor, both are limited by κ^2, but QR is never worse. With $\kappa(X) = 10^8$ (not unusual for raw polynomial or interaction designs), the normal equations can lose all sixteen digits.

Listing 2.9 implements Householder QR from scratch and compares it with the normal equations (solved by Cholesky) and with `np.linalg.lstsq` (SVD-based) on polynomial regressions $y = \sum_{j=0}^{d} b_j t^j$ with $t \in [0, 1]$. The monomial basis $1, t, t^2, \ldots$ is notoriously ill conditioned because high powers of t look alike on $[0, 1]$. The data are constructed with zero residual, so the exact solution is known.

Listing 2.9: Householder QR versus the normal equations `ch02/qr_vs_normal.py`

```python
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Householder QR vs normal equations for ill-conditioned polynomial regression
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import linalg


def householder_qr(A):
    """Return R and the Householder vectors (Q kept in factored form)."""
    R = A.astype(float).copy()
    m, n = R.shape
    vs = []
    for j in range(n):
        x = R[j:, j]
        v = x.copy()
        v[0] += np.copysign(np.linalg.norm(x), x[0])   # avoid cancellation
        v /= np.linalg.norm(v)
        R[j:, j:] -= 2.0 * np.outer(v, v @ R[j:, j:])  # apply H = I - 2vv^T
        vs.append(v)
    return np.triu(R[:n]), vs


def apply_qt(vs, b):
```

```
    """Compute Q^T b using the stored Householder vectors."""
    b = b.astype(float).copy()
    for j, v in enumerate(vs):
        b[j:] -= 2.0 * v * (v @ b[j:])
    return b


def ls_householder(X, y):
    R, vs = householder_qr(X)
    return linalg.solve_triangular(R, apply_qt(vs, y)[:X.shape[1]])


def ls_normal(X, y):
    return linalg.cho_solve(linalg.cho_factor(X.T @ X), X.T @ y)


rng = np.random.default_rng(803)
n = 100
t = np.linspace(0, 1, n)
degrees = np.arange(1, 14)
errs = {"normal equations (Cholesky)": [], "Householder QR (own code)": [],
        "np.linalg.lstsq (SVD)": []}
conds = []
print("deg   cond(X)    normal eq.  Householder    lstsq")
for d in degrees:
    X = np.vander(t, d + 1, increasing=True)          # 1, t, t^2, ..., t^d
    beta = rng.normal(size=d + 1)
    y = X @ beta                                       # exact fit: zero residual
    conds.append(np.linalg.cond(X))
    sols = [ls_normal(X, y), ls_householder(X, y),
            np.linalg.lstsq(X, y, rcond=None)[0]]
    for key, b in zip(errs, sols):
        errs[key].append(np.linalg.norm(b - beta) / np.linalg.norm(beta))
    if d % 2 == 1:
        print(f"{d:3d}  {conds[-1]:9.2e}  " +
              "  ".join(f"{errs[k][-1]:10.2e}" for k in errs))

conds = np.array(conds)
plt.figure(figsize=(6, 3.8))
for (key, v), mk in zip(errs.items(), ["o", "s", "^"]):
    plt.semilogy(degrees, np.maximum(v, 1e-17), marker=mk, ms=4, label=key)
plt.semilogy(degrees, 1.1e-16 * conds, "k:", lw=1, label=r"$\epsilon\,\kappa(X)$")
plt.semilogy(degrees, np.minimum(1.1e-16 * conds ** 2, 1e3), "k--", lw=1,
             label=r"$\epsilon\,\kappa(X)^2$")
plt.xlabel("polynomial degree $d$")
plt.ylabel("relative coefficient error")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("ch02_qr_vs_normal.pdf", bbox_inches="tight")
```

Output

```
deg   cond(X)    normal eq.  Householder    lstsq
  1   4.35e+00    4.58e-16    6.89e-16    5.78e-16
  3   1.21e+02    5.10e-14    5.70e-16    1.76e-15
```

```
 5   3.69e+03    1.74e-10    6.25e-14    4.69e-14
 7   1.16e+05    1.31e-07    5.68e-13    8.25e-13
 9   3.72e+06    5.17e-06    5.69e-12    1.82e-11
11   1.21e+08    6.44e-02    2.49e-09    1.29e-09
13   3.95e+09    1.58e+00    3.02e-08    8.20e-09
```

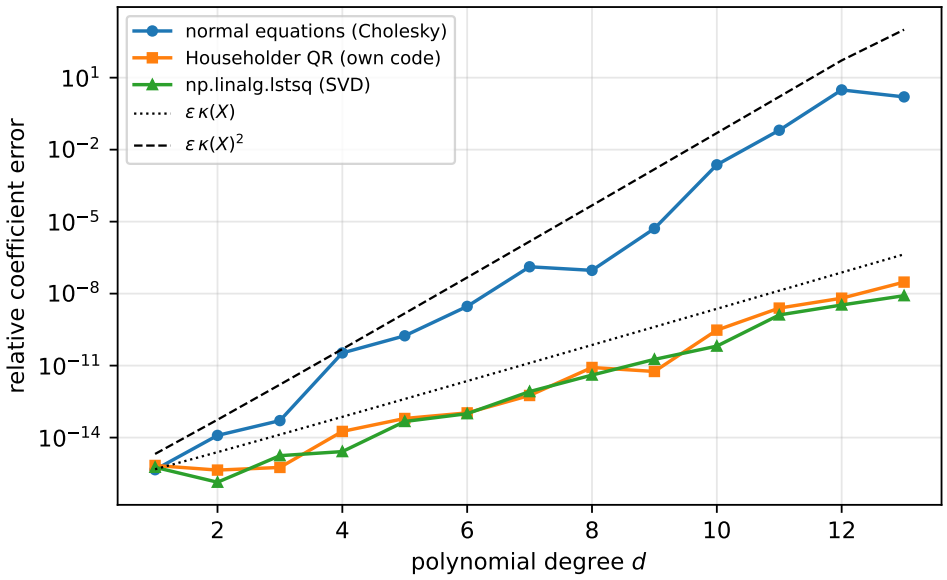


Figure 2.4: Relative error in the coefficients of a polynomial regression of degree d ($n = 100$ points on $[0, 1]$, exact data). The normal equations track $\varepsilon_{\text{mach}}\kappa(X)^2$ and are useless by $d \approx 11$; Householder QR and the SVD track $\varepsilon_{\text{mach}}\kappa(X)$.

At degree 11, $\kappa(X) \approx 1.2 \times 10^8$: QR and SVD are accurate to about nine digits, while the normal equations give coefficients with only one correct digit, and at degree 13 none. The remedy in practice is twofold: use QR (or SVD), and improve the conditioning of the design itself, by centring and scaling regressors or using orthogonal polynomials.

⋆ Key Concept: Choosing a decomposition

Square system $A\mathbf{x} = \mathbf{b}$	LU with partial pivoting (`np.linalg.solve`, `lu_factor`); $\frac{2}{3}n^3$ flops.
SPD system	Cholesky (`cho_factor`); $\frac{1}{3}n^3$ flops; doubles as a definiteness test and as a simulator of $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$.
Least squares, full rank	Householder QR; $2nk^2 - \frac{2}{3}k^3$ flops; error $\propto \kappa(X)$ for small residuals.
Rank-deficient or nearly so	SVD (`lstsq`, `pinv`); about $2nk^2 + 11k^3$ flops; reveals the numerical rank.
Normal equations	Cheapest ($nk^2 + \frac{1}{3}k^3$), acceptable only for well-conditioned designs ($\kappa(X) \ll 10^8$).

The flop counts (leading terms, for $X \in \mathbb{R}^{n\times k}$, $n \geq k$) are from Golub and Van Loan (2013, Sec. 5.3). For $n \gg k$ all least-squares methods cost $O(nk^2)$ and the normal equations are about twice as fast as QR; the price of the speed is the squared condition number.

Historical Note: Alston Householder

Alston S. Householder (1904–1993), an American mathematician who directed mathematics research at Oak Ridge National Laboratory, introduced elementary reflectors for triangularising matrices in a two-page paper in 1958 (Householder, 1958). Together with the Givens rotations of the same decade and the backward error analysis of J. H. Wilkinson, his work transformed numerical linear algebra from a collection of recipes into a discipline with provable stability guarantees. The Householder Symposium and the Householder Prize in numerical linear algebra are named after him.

Graduate Perspective: Updating, sparsity and big data

For streaming data, QR factors can be *updated* when rows are added or deleted (Givens rotations), so that rolling-window regressions, as in real-time forecasting (Goulet Coulombe et al., 2022), need not refactor from scratch. Large fixed-effects designs are sparse; sparse QR and iterative methods such as LSQR (Paige and Saunders, 1982) solve them without ever forming $X^\top X$. Randomised sketching (multiply X by a random $s \times n$ matrix with $k < s \ll n$, then solve the small problem) gives approximate least squares in time nearly linear in the size of X.

Exercises

2.5.1 Theory Prove that a Householder reflector is symmetric and orthogonal, and verify that $\mathbf{v} = \mathbf{x} + \operatorname{sign}(x_1)\,\|\mathbf{x}\|\,\mathbf{e}_1$ gives $H\mathbf{x} = -\operatorname{sign}(x_1)\,\|\mathbf{x}\|\,\mathbf{e}_1$.

2.5.2 Theory Show that $\det A = \pm \prod_i u_{ii}$ when $PA = LU$, and that $\log\det \boldsymbol{\Sigma} = 2\sum_i \log \ell_{ii}$ when $\boldsymbol{\Sigma} = LL^\top$. Why is the second formula the right way to evaluate a Gaussian log-likelihood?

2.5.3 Theory Prove that if A is SPD then the thin QR factor R of X with $X^\top X = A$ is, up to signs of its rows, the transpose of the Cholesky factor of A.

2.5.4 Python Time `np.linalg.solve(A, B)` against `np.linalg.inv(A) @ B` for $n = 200, 400, 800, 1600$ and 50 right-hand sides, and compare the residuals $\|AX - B\|$. Fit the exponent of n in the running time.

2.5.5 Python Repeat Listing 2.9 with the regressor t rescaled to $[-1, 1]$ and with Legendre polynomials (`np.polynomial.legendre.legvander`). Report $\kappa(X)$ and the errors of the three methods at degree 13.

2.5.6 Python Use the Cholesky factor to simulate 10^4 draws of (log income, log consumption) for 500 households with correlation 0.8, then estimate the correlation and its sampling distribution. What goes wrong if you use the *upper* factor $L^\top$ instead of L in $\mathbf{x} = L\mathbf{z}$?

2.5.7 Policy Using the input–output model in Listing 2.8, compute the full matrix of output multipliers $(\mathbf{I} - A)^{-1}$ column by column with one LU factorisation. Which sector has the largest total multiplier, and how should a planning ministry interpret this number in choosing where to direct a fixed public investment budget?

2.5.8 ★ Graduate Show that the error of the normal equations cannot be repaired by iterative refinement in working precision, but that *semi-normal equations* $R^\top R\mathbf{b} = X^\top \mathbf{y}$ with R from QR plus one step of refinement achieve QR-level accuracy (Björck, 1996). Test this numerically on the degree-11 polynomial design.

2.6 Generalised Inverses

When X is rank-deficient (the dummy trap, $p > n$, collinear programme indicators), $(X^\top X)^{-1}$ does not exist, yet least-squares problems still have solutions, and software returns one of them.

Which one, and why? The answer is the Moore–Penrose pseudoinverse.

2.6.1 The four Penrose conditions

Definition 2.26 (Moore–Penrose pseudoinverse)**.** For $A \in \mathbb{R}^{m\times n}$, a matrix $A^+ \in \mathbb{R}^{n\times m}$ is a *Moore–Penrose pseudoinverse* of A if

$$(1)\ AA^+A = A, \quad (2)\ A^+AA^+ = A^+, \quad (3)\ (AA^+)^\top = AA^+, \quad (4)\ (A^+A)^\top = A^+A. \tag{2.21}$$

Condition (1) alone defines a *generalised inverse* (a "g-inverse", written A^-), which always exists but is not unique; any A^- gives *a* solution $\mathbf{x} = A^-\mathbf{b}$ of a consistent system. Conditions (3) and (4) make AA^+ and A^+A *orthogonal* projectors, and (2) makes A^+ "as small as possible".

Theorem 2.27 (Existence and uniqueness)**.** *Every $A \in \mathbb{R}^{m\times n}$ has exactly one Moore–Penrose pseudoinverse. If $A = U_r\Sigma_r V_r^\top$ is the compact SVD with $\Sigma_r = \operatorname{diag}(\sigma_1, \dots, \sigma_r)$, $\sigma_r > 0$, then*

$$A^+ = V_r\Sigma_r^{-1}U_r^\top. \tag{2.22}$$

Proof. Existence. With $U_r^\top U_r = V_r^\top V_r = \mathbf{I}_r$, $AA^+ = U_rU_r^\top$ and $A^+A = V_rV_r^\top$ are symmetric, and $AA^+A = U_rU_r^\top U_r\Sigma_rV_r^\top = A$; similarly $A^+AA^+ = A^+$.

Uniqueness. Let B and C both satisfy (1)–(4). Using (2), (3), (1) for C, and (3) again,

$$B = BAB = B(AB)^\top = BB^\top A^\top = BB^\top(ACA)^\top = B(AB)^\top(AC)^\top = BABAC = BAC.$$

Similarly, using (2), (4), (1) for B and (4),

$$C = CAC = (CA)^\top C = A^\top C^\top C = (ABA)^\top C^\top C = (BA)^\top(CA)^\top C = BACAC = BAC.$$

Hence $B = C$. □

Proposition 2.28 (Special cases and projections)**.**

1. *If A is square and invertible, $A^+ = A^{-1}$.*
2. *If A has full column rank, $A^+ = (A^\top A)^{-1}A^\top$ (a left inverse); if A has full row rank, $A^+ = A^\top(AA^\top)^{-1}$ (a right inverse).*
3. *AA^+ is the orthogonal projector onto $\operatorname{col}(A)$ and A^+A the orthogonal projector onto the row space $\operatorname{col}(A^\top)$.*
4. *$(A^+)^+ = A$, $(A^\top)^+ = (A^+)^\top$, and $(A^\top A)^+A^\top = A^+$; but in general $(AB)^+ \neq B^+A^+$.*

In particular, for a full-column-rank design, $\hat{\boldsymbol{\beta}} = X^+\mathbf{y}$ and $P = XX^+$: the pseudoinverse generalises everything we did in Section 2.4.

2.6.2 Minimum-norm least squares

Theorem 2.29 (Minimum-norm least-squares solution)**.** *For any $A \in \mathbb{R}^{m\times n}$ and $\mathbf{b} \in \mathbb{R}^m$, the set of least-squares solutions of $\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|$ is*

$$\mathcal{S} = \{A^+\mathbf{b} + (\mathbf{I} - A^+A)\mathbf{z} : \mathbf{z} \in \mathbb{R}^n\},$$

and $\mathbf{x}^+ = A^+\mathbf{b}$ is its unique element of minimum Euclidean norm. Equivalently, $\mathbf{x}^+$ is the unique least-squares solution lying in the row space $\operatorname{col}(A^\top)$.

Proof. $\mathbf{x}$ is a least-squares solution iff $A\mathbf{x} = AA^+\mathbf{b}$ (the projection of $\mathbf{b}$ onto $\operatorname{col}(A)$, Proposition 2.28(3) and Theorem 2.16). Clearly $A\mathbf{x}^+ = AA^+\mathbf{b}$, so the solution set is $\mathbf{x}^+ + \operatorname{null}(A)$, and $\operatorname{null}(A) = \operatorname{col}(\mathbf{I} - A^+A)$ because $\mathbf{I} - A^+A$ is the orthogonal projector onto $\operatorname{col}(A^\top)^\perp = \operatorname{null}(A)$. Now $\mathbf{x}^+ = A^+\mathbf{b} = A^+AA^+\mathbf{b} \in \operatorname{col}(A^+A) = \operatorname{col}(A^\top)$, which is orthogonal to $\operatorname{null}(A)$, so for $\mathbf{w} \in \operatorname{null}(A)$, $\|\mathbf{x}^+ + \mathbf{w}\|^2 = \|\mathbf{x}^+\|^2 + \|\mathbf{w}\|^2$, minimised only at $\mathbf{w} = \mathbf{0}$. □

⋆ Key Concept: What software returns for rank-deficient regressions

`np.linalg.lstsq` and `np.linalg.pinv` return the minimum-norm solution $X^{+}\mathbf{y}$: among all coefficient vectors with the same (optimal) fitted values, the one with no component in $\text{null}(X)$. Estimable functions $\ell^{\top}\beta$ with $\ell \in \text{col}(X^{\top})$ take the same value at every least-squares solution (Theorem 2.13); everything else about the minimum-norm solution, such as the value $\hat{\tau} = 1.15$ in Listing 2.3, is a convention, not an estimate.

Historical Note: E. H. Moore and Roger Penrose

Eliakim Hastings Moore (1862–1932) of the University of Chicago described a "general reciprocal" of a matrix in a 1920 abstract (Moore, 1920), but the idea attracted little attention, partly because of Moore's idiosyncratic notation. The geodesist Arne Bjerhammar rediscovered generalised inverses in 1951 in connection with least-squares adjustment. Roger Penrose (b. 1931), then a graduate student at Cambridge, independently characterised the inverse by the four equations (2.21) and proved its uniqueness (Penrose, 1955). Penrose later became famous for his work in mathematical physics and shared the 2020 Nobel Prize in Physics for work on black holes. Ben-Israel and Greville (2003) is the standard reference on generalised inverses.

2.6.3 Computing the pseudoinverse

Listing 2.10 computes A^{+} for a 6×5 matrix of rank three, checks the four Penrose conditions, confirms that `pinv` and `lstsq` agree, and verifies the geometry of Theorem 2.29: adding a null-space vector leaves the residual unchanged but increases the norm.

Listing 2.10: Penrose conditions and minimum-norm least squares `ch02/pinv_penrose.py`

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Moore-Penrose pseudoinverse: Penrose conditions and minimum-norm least squares
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
A = rng.normal(size=(6, 3)) @ rng.normal(size=(3, 5))   # 6 x 5, rank 3
Ap = np.linalg.pinv(A)

# pinv from the SVD: A^+ = V diag(1/s_i) U^T over the non-zero s_i
U, s, Vt = np.linalg.svd(A, full_matrices=False)
r = int(np.sum(s > s[0] * 1e-12))
Ap_svd = Vt[:r].T @ np.diag(1 / s[:r]) @ U[:, :r].T
print(f"rank(A) = {r},  max|pinv - SVD formula| = {np.abs(Ap - Ap_svd).max():.1e}")

checks = {"(1) A A+ A   = A ": A @ Ap @ A - A,
          "(2) A+ A A+  = A+": Ap @ A @ Ap - Ap,
          "(3) (A A+)^T = A A+": (A @ Ap).T - A @ Ap,
          "(4) (A+ A)^T = A+ A": (Ap @ A).T - Ap @ A}
for name, D in checks.items():
    print(f"Penrose {name:22s} max error = {np.abs(D).max():.1e}")

# Naive inverse of A^T A fails for rank-deficient A
print(f"cond(A^T A) = {np.linalg.cond(A.T @ A):.1e}  (singular in exact arithmetic)")

# Minimum-norm least squares: all minimisers are x+ + null(A)
b = rng.normal(size=6)
x_plus = Ap @ b
x_lstsq, res, rk, sv = np.linalg.lstsq(A, b, rcond=None)
```

```python
N = Vt[r:].T                          # basis of null(A)
x_other = x_plus + N @ rng.normal(size=N.shape[1])
print(f"\n||x_pinv - x_lstsq|| = {np.linalg.norm(x_plus - x_lstsq):.1e}")
print(f"residual norms: x+ {np.linalg.norm(A @ x_plus - b):.6f}, "
      f"x+ + null {np.linalg.norm(A @ x_other - b):.6f}")
print(f"solution norms: x+ {np.linalg.norm(x_plus):.4f}, "
      f"x+ + null {np.linalg.norm(x_other):.4f}")
print(f"x+ in row(A)?  ||(I - A+A) x+|| = "
      f"{np.linalg.norm(x_plus - Ap @ A @ x_plus):.1e}")

# Underdetermined p > n: interpolation with the minimum-norm solution
n, p = 20, 50
X = rng.normal(size=(n, p))
y = X[:, :5] @ np.ones(5) + 0.1 * rng.normal(size=n)
b_mn = np.linalg.pinv(X) @ y
print(f"\np = {p} > n = {n}: training residual = "
      f"{np.linalg.norm(X @ b_mn - y):.1e} (interpolates)")
lam = 1e-8
b_ridge = np.linalg.solve(X.T @ X + lam * np.eye(p), X.T @ y)
print(f"ridge, lambda = {lam:.0e}: ||b_ridge - b_minnorm|| = "
      f"{np.linalg.norm(b_ridge - b_mn):.1e}")
```

Output

```
rank(A) = 3,  max|pinv - SVD formula| = 2.8e-17
Penrose (1) A A+ A   = A       max error = 8.9e-16
Penrose (2) A+ A A+  = A+      max error = 1.7e-16
Penrose (3) (A A+)^T = A A+    max error = 3.1e-16
Penrose (4) (A+ A)^T = A+ A    max error = 3.7e-16
cond(A^T A) = 5.7e+16  (singular in exact arithmetic)

||x_pinv - x_lstsq|| = 1.8e-16
residual norms: x+ 0.876272, x+ + null 0.876272
solution norms: x+ 0.2924, x+ + null 0.9843
x+ in row(A)?  ||(I - A+A) x+|| = 1.3e-16

p = 50 > n = 20: training residual = 1.0e-14 (interpolates)
ridge, lambda = 1e-08: ||b_ridge - b_minnorm|| = 8.0e-07
```

△ Caution: The pseudoinverse is discontinuous

$\left(\begin{smallmatrix}1&0\\0&\delta\end{smallmatrix}\right)^{+} = \left(\begin{smallmatrix}1&0\\0&1/\delta\end{smallmatrix}\right)$ for $\delta \neq 0$, but $\left(\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right)^{+} = \left(\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right)$. Small singular values are inverted into huge ones, so in floating point A^{+} is computed with a cut-off: `np.linalg.pinv` treats singular values below `rcond`$\times\sigma_1$ as zero (default `rcond=1e-15`), and `np.linalg.lstsq(X, y, rcond=None)` uses $\varepsilon_{\text{mach}} \max(m, n)$. Changing the cut-off changes the numerical rank and hence the answer. This is truncated-SVD regularisation in disguise; choose the cut-off deliberately when the design is nearly rank deficient.

2.6.4 Underdetermined problems: $p > n$

In modern data science it is common to have more predictors than observations: night-time light and satellite features for a few hundred districts, text features of policy documents, or thousands of SNPs for a few hundred patients. With $X \in \mathbb{R}^{n \times p}$, $p > n$ and $\operatorname{rank} X = n$, the

system $X\mathbf{b} = \mathbf{y}$ has infinitely many exact solutions (Corollary 2.11): least squares *interpolates* the data, and the minimum-norm solution is $\hat{\mathbf{b}} = X^\top(XX^\top)^{-1}\mathbf{y}$. Two facts link this solution to the rest of the course.

Proposition 2.30 (Ridge converges to minimum norm)**.** *For every X and $\lambda > 0$, $\hat{\mathbf{b}}_\lambda = (X^\top X + \lambda\mathbf{I})^{-1}X^\top\mathbf{y}$ is well defined, and $\hat{\mathbf{b}}_\lambda \to X^+\mathbf{y}$ as $\lambda \downarrow 0$.*

Proof. $X^\top X + \lambda\mathbf{I}$ is SPD for $\lambda > 0$. With $X = U\Sigma V^\top$ (full SVD), $(X^\top X + \lambda\mathbf{I})^{-1}X^\top = V\,\mathrm{diag}\big(\sigma_i/(\sigma_i^2 + \lambda)\big)U^\top$, with entries 0 where $\sigma_i = 0$. As $\lambda \downarrow 0$, $\sigma_i/(\sigma_i^2 + \lambda) \to 1/\sigma_i$ for $\sigma_i > 0$, which is (2.22). □

Second, gradient descent on $\|\mathbf{y} - X\mathbf{b}\|^2$ started at $\mathbf{b} = \mathbf{0}$ never leaves $\mathrm{col}(X^\top)$ (each gradient $-2X^\top(\mathbf{y} - X\mathbf{b})$ lies there), so if it converges it converges to the minimum-norm interpolant: an algorithm can regularise implicitly. These observations are the starting point for ridge regression and the lasso (Chapters 8 and 9) and for the statistical analysis of interpolating models.

Listing 2.11 fits min-norm least squares with the first p of $d = 200$ Gaussian features to $n = 40$ observations, for p from 2 to 200, and records test error. The result (Fig. 2.5) is the *double descent* curve of Belkin et al. (2019): the error explodes near the interpolation threshold $p = n$, where $XX^\top$ (or $X^\top X$) is nearly singular and the minimum-norm solution has a huge norm, and then *decreases* again as p grows, because among more and more interpolating solutions the one of minimum norm becomes smoother.

Listing 2.11: Minimum-norm least squares across $p = n$ ch02/min_norm_descent.py

```python
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Test error of minimum-norm least squares as p crosses n (double descent)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n, d, n_test, reps, sigma = 40, 200, 500, 60, 0.5
beta = 1 / np.sqrt(np.arange(1, d + 1))          # decaying true coefficients
beta *= 2.0 / np.linalg.norm(beta)
ps = np.unique(np.r_[np.arange(2, 36, 3), np.arange(36, 46), 50, 60, 80,
                     100, 130, 160, 200])
risk = np.zeros((reps, ps.size))
norms = np.zeros((reps, ps.size))
for r in range(reps):
    X = rng.normal(size=(n, d))
    y = X @ beta + sigma * rng.normal(size=n)
    Xt = rng.normal(size=(n_test, d))
    yt = Xt @ beta + sigma * rng.normal(size=n_test)
    for j, p in enumerate(ps):
        b = np.linalg.lstsq(X[:, :p], y, rcond=None)[0]   # min-norm if p > n
        risk[r, j] = np.mean((yt - Xt[:, :p] @ b) ** 2)
        norms[r, j] = np.linalg.norm(b)
med = np.median(risk, axis=0)
print("  p   p/n   median test MSE   median ||b||")
for p, m, nb in zip(ps, med, np.median(norms, axis=0)):
    if p in (5, 20, 35, 38, 40, 42, 45, 60, 100, 200):
        print(f"{p:4d}  {p / n:4.2f}  {m:14.3f}  {nb:12.3f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.semilogy(ps / n, med, "o-", ms=4, label="median test MSE")
ax.axvline(1.0, color="k", ls=":", lw=1)
ax.axhline(sigma ** 2 + beta @ beta, color="gray", ls="--", lw=1,
```

```
            label="null model (b = 0)")
ax.set_xlabel("p / n  (number of regressors / sample size)")
ax.set_ylabel("test mean squared error")
ax.text(1.05, med.max() / 2, "interpolation\nthreshold p = n", fontsize=8)
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("ch02_min_norm_descent.pdf", bbox_inches="tight")
```

Output

```
  p   p/n   median test MSE   median ||b||
  5  0.12           3.057         1.487
 20  0.50           3.292         2.076
 35  0.88          10.700         3.555
 38  0.95          23.108         4.881
 40  1.00          74.464         8.789
 42  1.05          19.126         4.407
 45  1.12          11.176         3.376
 60  1.50           4.376         2.066
100  2.50           3.234         1.389
200  5.00           3.421         0.934
```

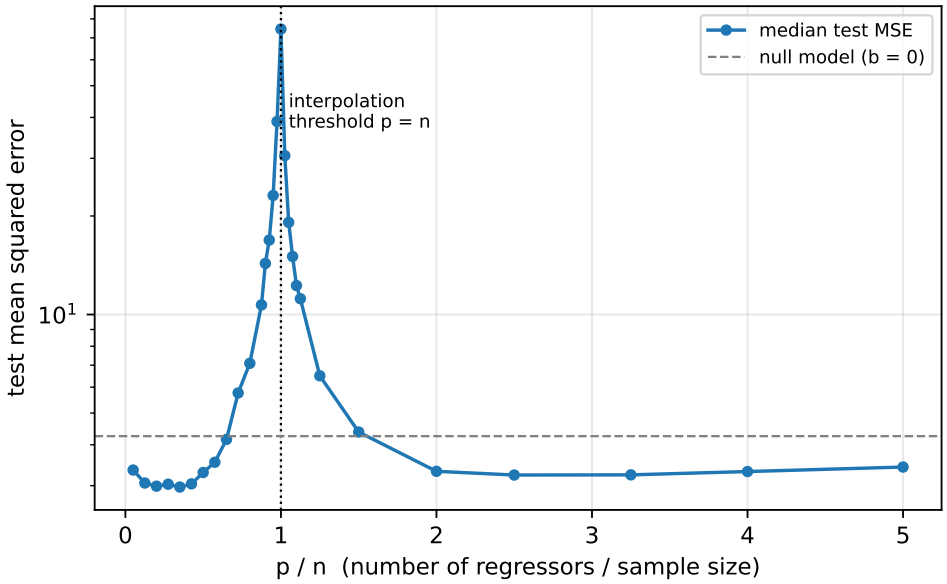


Figure 2.5: Median test error (over 60 simulated data sets) of minimum-norm least squares as a function of the ratio p/n of regressors to observations, $n = 40$. The peak at $p = n$ is caused by the smallest singular value of X approaching zero; beyond it, the minimum-norm interpolant improves again (double descent).

Graduate Perspective: Ridgeless regression and benign overfitting

Hastie et al. (2022) derive the exact asymptotic risk of minimum-norm ("ridgeless") least squares as $n, p \to \infty$ with $p/n \to \gamma$, using random matrix theory: the risk diverges at $\gamma = 1$ because the smallest singular value of $X/\sqrt{n}$ tends to $\left|1 - \sqrt{\gamma}\right|$ (the Marchenko–Pastur law),

and for $\gamma > 1$ it can decrease. Whether interpolation is "benign" depends on the spectrum of the feature covariance. This literature explains part of the success of over-parametrised neural networks, and it is built entirely on the objects of this section: X^{+}, the SVD and projections (Vershynin, 2018).

2.6.5 Constrained least squares and national accounts balancing

Preliminary estimates of national accounts aggregates come from different sources (production surveys, household expenditure surveys, tax and payroll records) and do not satisfy the accounting identities: GDP by production, expenditure and income differ. Stone et al. (1942) proposed to adjust the vector of preliminary estimates $\mathbf{x}_0$ as little as possible, in a metric that respects their reliability, subject to the linear constraints $A\mathbf{x} = \mathbf{0}$:

$$\min_{\mathbf{x}} (\mathbf{x}-\mathbf{x}_0)^{\top} V^{-1} (\mathbf{x}-\mathbf{x}_0) \quad \text{s.t.} \quad A\mathbf{x} = \mathbf{0} \qquad \Longrightarrow \qquad \mathbf{x}^{*} = \mathbf{x}_0 - V A^{\top} (A V A^{\top})^{+} A \mathbf{x}_0, \tag{2.23}$$

where V is the (diagonal) matrix of prior variances. The solution is the V^{-1}-orthogonal projection of $\mathbf{x}_0$ onto $\mathrm{null}(A)$ (the Lagrangian derivation is in Chapter 9). If the constraints are redundant (here: production = expenditure, production = income *and* expenditure = income), $AVA^{\top}$ is singular and the inverse must be replaced by the pseudoinverse; since $A\mathbf{x}_0 \in \mathrm{col}(A) = \mathrm{col}(AVA^{\top})$, the result is the same as with a non-redundant subset of constraints.

Listing 2.12: Balancing illustrative national accounts ch02/balancing.py

```
# DEP 803 -- Chapter 2: Linear Algebra for Modelling
# Least-squares balancing of (illustrative) national accounts with pinv
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

names = ["VA agric", "VA indus", "VA serv", "C house", "G govt", "I invest",
         "NX net ex", "COMP empl", "GOS surpl"]
x0 = np.array([210., 330., 480., 700., 150., 230., -40., 520., 470.])
cv = np.array([.08, .04, .04, .03, .01, .10, .15, .05, .08])  # rel. std errors
V = np.diag((cv * np.abs(x0)) ** 2)        # prior variances of the estimates
prod = np.r_[1, 1, 1, 0, 0, 0, 0, 0, 0]    # production approach
expd = np.r_[0, 0, 0, 1, 1, 1, 1, 0, 0]    # expenditure approach
inc = np.r_[0, 0, 0, 0, 0, 0, 0, 1, 1]     # income approach
A = np.vstack([prod - expd, prod - inc, expd - inc])   # A x = 0 (redundant!)
print(f"constraints: {A.shape[0]}, rank(A) = {np.linalg.matrix_rank(A)}")
print("GDP before:  production {:.1f}, expenditure {:.1f}, income {:.1f}"
      .format(prod @ x0, expd @ x0, inc @ x0))

S = A @ V @ A.T                            # singular because rank(A) = 2
print(f"cond(A V A^T) = {np.linalg.cond(S):.1e}")
x_star = x0 - V @ A.T @ np.linalg.pinv(S) @ (A @ x0)
print("GDP after:   production {:.1f}, expenditure {:.1f}, income {:.1f}"
      .format(prod @ x_star, expd @ x_star, inc @ x_star))
print(f"max |A x*| = {np.abs(A @ x_star).max():.1e}")

# Same answer using only the two independent constraints
A2 = A[:2]
x2 = x0 - V @ A2.T @ np.linalg.solve(A2 @ V @ A2.T, A2 @ x0)
print(f"max |x* - x*(2 constraints)| = {np.abs(x_star - x2).max():.1e}\n")
print(f"{'item':<10s} {'prelim':>8s} {'balanced':>9s} {'change':>8s} {'cv':>5s}")
for nm, a, b, c in zip(names, x0, x_star, cv):
    print(f"{nm:<10s} {a:8.1f} {b:9.1f} {b - a:8.1f} {c:5.2f}")
```

Output

```
constraints: 3, rank(A) = 2
GDP before:  production 1020.0, expenditure 1040.0, income 990.0
cond(A V A^T) = 3.8e+16
GDP after:   production 1022.0, expenditure 1022.0, income 1022.0
max |A x*| = 5.7e-14
max |x* - x*(2 constraints)| = 5.7e-14

item         prelim  balanced   change    cv
VA agric      210.0     210.7      0.7  0.08
VA indus      330.0     330.4      0.4  0.04
VA serv       480.0     480.9      0.9  0.04
C house       700.0     692.1     -7.9  0.03
G govt        150.0     150.0     -0.0  0.01
I invest      230.0     220.6     -9.4  0.10
NX net ex     -40.0     -40.6     -0.6  0.15
COMP empl     520.0     530.4     10.4  0.05
GOS surpl     470.0     491.7     21.7  0.08
```

Policy Lens: Reliability-weighted reconciliation

In the (illustrative) accounts above, the preliminary expenditure estimate exceeds the income estimate by 50 units. Balancing moves every estimate, but the adjustments are proportional to the prior variances: government consumption (relative error 1%) is essentially unchanged, while the large and less reliable items (gross operating surplus, compensation of employees, investment) absorb most of the discrepancy. The balanced GDP of 1022 is not the average of the three approaches; it is a precision-weighted compromise. The same projection underlies the balancing of supply–use tables, the benchmarking of quarterly to annual accounts, and the reconciliation of regional to national totals; the choice of V is a statistical judgement that should be documented, because it determines who "absorbs" the statistical discrepancy.

Think About It

The minimum-norm solution treats all coefficients symmetrically: it penalises $\sum_j b_j^2$. If the regressors are measured in different units (cedis and thousands of cedis, years and months), the "minimum-norm" solution changes when you rescale them, although the fitted values do not. What does this imply for reporting a pseudoinverse solution, and why do ridge and lasso implementations standardise the regressors first?

Exercises

2.6.1 Theory Verify the four Penrose conditions for $A^+ = (A^\top A)^{-1}A^\top$ when A has full column rank, and for $A^+ = A^\top(AA^\top)^{-1}$ when it has full row rank. Compute $\mathbf{x}^+$ for the single equation $x_1 + 2x_2 + 2x_3 = 9$ and interpret it geometrically.

2.6.2 Theory Prove parts (3) and (4) of Proposition 2.28, and find 2×2 matrices A, B with $(AB)^+ \neq B^+A^+$.

2.6.3 Theory Show that $G = A^+$ is the unique matrix such that $G\mathbf{b}$ is the minimum-norm least-squares solution for *every* $\mathbf{b}$. Deduce that any linear estimator that returns a least-squares solution of minimum norm must coincide with `np.linalg.pinv`.

2.6.4 Python Implement A^+ from `np.linalg.svd` with a user-chosen relative cut-off and apply

it to the near-collinear designs of Listing 2.2. Plot the norm of $\hat{\mathbf{b}}$ and the in-sample RSS against the cut-off and explain the trade-off.

2.6.5 Python Modify Listing 2.11 to fit ridge regression with $\lambda \in \{10^{-3}, 10^{-1}, 1\}$ instead of minimum-norm least squares. How does the peak at $p = n$ change? Compare with the optimal ridge penalty chosen by leave-one-out cross-validation.

2.6.6 Policy Extend Listing 2.12 with a regional dimension: three regions whose value added must add to the national total, plus the three national identities. Write the constraint matrix, check its rank, balance the system, and discuss how the choice of V for the regional estimates affects regional growth rates reported to policymakers.

2.6.7 ★ Graduate Show that gradient descent $\mathbf{b}_{t+1} = \mathbf{b}_t + \eta X^\top(\mathbf{y} - X\mathbf{b}_t)$ with $\mathbf{b}_0 = \mathbf{0}$ and $0 < \eta < 2/\sigma_1^2$ converges to $X^+\mathbf{y}$ for any X. Express $\mathbf{b}_t$ in the SVD basis and show that early stopping acts like a filter $1 - (1 - \eta\sigma_i^2)^t$ on the singular directions, comparable to ridge's $\sigma_i^2/(\sigma_i^2 + \lambda)$.

Chapter Summary

- A matrix $A \in \mathbb{R}^{m\times n}$ of rank r has four fundamental subspaces: $\operatorname{col}(A) \perp \operatorname{null}(A^\top)$ in $\mathbb{R}^m$ and $\operatorname{col}(A^\top) \perp \operatorname{null}(A)$ in $\mathbb{R}^n$, of dimensions $r, m-r, r, n-r$. $A\mathbf{x} = \mathbf{b}$ is solvable iff $\mathbf{b} \in \operatorname{col}(A)$; solutions are unique iff $\operatorname{null}(A) = \{\mathbf{0}\}$.
- Rank–nullity, $\operatorname{rank}(A) + \dim\operatorname{null}(A) = n$, turns identification into counting: $\boldsymbol{\beta}$ is identified iff X has full column rank; otherwise only $\boldsymbol{\ell}^\top\boldsymbol{\beta}$ with $\boldsymbol{\ell} \in \operatorname{col}(X^\top)$ is estimable. The dummy trap and region-level programmes are rank deficiencies; numerical rank always needs a tolerance.
- $P = X(X^\top X)^{-1}X^\top$ and $M = \mathbf{I} - P$ are symmetric idempotent projectors with $\operatorname{tr} P = k$, $\operatorname{tr} M = n - k$; Pythagoras gives TSS = ESS + RSS and R^2.
- OLS is the orthogonal projection of $\mathbf{y}$ on $\operatorname{col}(X)$. The Frisch–Waugh–Lovell theorem says that controlling for X_1 means applying M_1 to everything; fixed effects are demeaning, and leverages h_{ii} are the diagonal of P.
- LU ($\frac{2}{3}n^3$) solves square systems, Cholesky ($\frac{1}{3}n^3$) SPD systems and generates correlated draws, Householder QR solves least squares stably; the normal equations square the condition number and fail on ill-conditioned designs.
- The Moore–Penrose pseudoinverse, defined by four conditions and computed from the SVD, gives the minimum-norm least-squares solution; it is the $\lambda \to 0$ limit of ridge and the solution behind interpolation and double descent.

Further Reading

Strang (2019) is the most accessible treatment of the four subspaces and of the factorisations $A = LU$, QR, $U\Sigma V^\top$ from a data-science viewpoint, and Strang (1993) is a short essay on the fundamental theorem. Deisenroth et al. (2020) (Chs. 2–4) covers the same material with machine-learning applications, as do the linear-algebra chapters of Murphy (2022, Ch. 7) and the linear-regression chapters of Géron (2022, Ch. 4), which discuss the normal equations versus SVD-based solvers in scikit-learn. Trefethen and Bau (1997) is the best introduction to the numerical side (projectors, QR, Householder, conditioning and stability), Golub and Van Loan (2013) the standard reference and Higham (2002) the authority on rounding-error analysis; for least squares specifically see Björck (1996). On the econometric side, Davidson and MacKinnon (2004) build their whole treatment of regression on projections and the FWL theorem, and

Hansen (2022) and Wooldridge (2010) discuss identification, fixed effects and leverage. Angrist and Pischke (2009) explain why regression "controls" are projections and how to interpret them causally. For generalised inverses see Ben-Israel and Greville (2003); for the high-dimensional behaviour of minimum-norm least squares, Hastie et al. (2022) and Belkin et al. (2019). The forecasting study of Goulet Coulombe et al. (2022) uses many of these tools (ridge, factor models, rolling regressions) in a macroeconomic setting, and the Bayesian models of Matsuura (2022) are fitted in Stan, which parametrises covariance matrices through their Cholesky factors.

Chapter 3

Eigenvalues, the SVD, PCA and Conditioning

Learning Objectives

After working through this chapter you should be able to:

1. define eigenvalues and eigenvectors, compute them from the characteristic polynomial for small matrices, and use diagonalisation to evaluate matrix powers;
2. analyse a finite Markov chain (for example, labour-market transitions) through its spectrum, find its stationary distribution and state the Perron–Frobenius theorem;
3. state and sketch the proof of the spectral theorem, and test a symmetric matrix for positive (semi)definiteness in three different ways;
4. interpret covariance matrices geometrically through ellipsoids and the Mahalanobis distance;
5. implement power iteration, inverse iteration and Rayleigh-quotient iteration, explain their convergence rates, and compute PageRank;
6. derive the singular value decomposition, relate it to the eigen-decompositions of $\mathbf{X}^\top\mathbf{X}$ and $\mathbf{X}\mathbf{X}^\top$, and use the Eckart–Young–Mirsky theorem for low-rank approximation;
7. derive principal component analysis as variance maximisation, compute it stably via the SVD, and build an asset (wealth) index and a diffusion index;
8. define the condition number, prove the perturbation bound for linear systems, and diagnose multicollinearity with variance inflation factors and eigenvalues.

Chapter 2 treated a matrix as a *static* object: a table of numbers, a linear map, a set of columns spanning a subspace. This chapter asks a more dynamic question: *what does a matrix do when it acts repeatedly, and in which directions does it act most strongly?* The answers are given by eigenvalues and eigenvectors for square matrices, and by singular values and singular vectors for arbitrary rectangular data matrices. Together they form the most widely used toolkit in applied mathematics.

For a data scientist the reasons are immediate. Principal component analysis (PCA), the workhorse of dimension reduction, is an eigenvalue problem for a covariance matrix. Google's original PageRank is the leading eigenvector of a huge stochastic matrix. Recommender systems, latent semantic analysis and image compression are low-rank singular value decompositions. Spectral clustering, graph neural networks and the analysis of gradient descent all depend on the spectrum of some matrix. The numerical reliability of every regression you will ever run is governed by a ratio of singular values, the *condition number*.

For an economist the same tools appear under different names. Labour-market flows between formal work, informal work and unemployment are a Markov chain whose long-run unemployment rate is an eigenvector. The stability of a vector autoregression depends on whether its companion matrix has eigenvalues inside the unit circle. A household survey without consumption data can still rank households by wealth through the first principal component of their asset ownership (Filmer and Pritchett, 2001). Central banks summarise hundreds of indicators by a handful of "diffusion indexes" – principal components – and use them to forecast inflation (Stock and Watson, 2002a,b); Goulet Coulombe et al. (2022) show that such factor structures remain a core ingredient of modern machine-learning forecasts. And the familiar complaint that "the coefficients are insignificant because the regressors are collinear" is, precisely, a statement about small eigenvalues of a correlation matrix.

The chapter has six sections. Section 3.1 introduces eigenvalues, diagonalisation and Markov chains. Section 3.2 specialises to symmetric matrices, the setting of covariance matrices and quadratic forms. Section 3.3 explains how eigenvalues are actually computed. Section 3.4 develops the singular value decomposition and low-rank approximation, Section 3.5 builds PCA and factor models on it, and Section 3.6 closes with conditioning, multicollinearity and the motivation for ridge regression. Throughout, Strang (2019) and Deisenroth et al. (2020) are good companions; Golub and Van Loan (2013) and Trefethen and Bau (1997) are the numerical references, and Murphy (2022, ch. 20) and Géron (2022, ch. 8) treat PCA from a machine-learning perspective.

3.1 Eigenvalues and Eigenvectors

3.1.1 Definitions and the characteristic polynomial

Definition 3.1 (Eigenvalue and eigenvector)**.** Let $A \in \mathbb{R}^{n\times n}$ (or $\mathbb{C}^{n\times n}$). A scalar $\lambda \in \mathbb{C}$ is an *eigenvalue* of A if there is a nonzero vector $\mathbf{v} \in \mathbb{C}^n$ with

$$A\mathbf{v} = \lambda\mathbf{v}. \tag{3.1}$$

The vector $\mathbf{v}$ is an *eigenvector* and $(\lambda, \mathbf{v})$ an *eigenpair*. The set of all eigenvalues is the *spectrum* $\sigma(A)$, and $\rho(A) = \max_{\lambda\in\sigma(A)} |\lambda|$ is the *spectral radius*. The *eigenspace* of λ is $\operatorname{null}(A - \lambda\mathbf{I})$.

Geometrically, an eigenvector is a direction that A does not rotate: it is only stretched (by $|\lambda| > 1$), shrunk ($|\lambda| < 1$), or reversed ($\lambda < 0$). Since $\mathbf{v} \neq \mathbf{0}$ solves $(A - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$, the matrix $A - \lambda\mathbf{I}$ must be singular, which gives the algebraic characterisation.

Proposition 3.2 (Characteristic polynomial)**.** *λ is an eigenvalue of A if and only if $p_A(\lambda) = \det(\lambda\mathbf{I} - A) = 0$. The* characteristic polynomial *p_A is monic of degree n,*

$$p_A(\lambda) = \lambda^n - \operatorname{tr}(A)\,\lambda^{n-1} + \cdots + (-1)^n \det(A), \tag{3.2}$$

so A has exactly n eigenvalues $\lambda_1, \ldots, \lambda_n$ in $\mathbb{C}$ counted with (algebraic) multiplicity, and

$$\operatorname{tr}(A) = \sum_{i=1}^{n} \lambda_i, \qquad \det(A) = \prod_{i=1}^{n} \lambda_i.$$

Proof. $A\mathbf{v} = \lambda\mathbf{v}$ for some $\mathbf{v} \neq \mathbf{0}$ iff $\operatorname{null}(\lambda\mathbf{I} - A) \neq \{\mathbf{0}\}$ iff $\lambda\mathbf{I} - A$ is singular iff its determinant is zero. Expanding the determinant by the Leibniz formula, the only term containing λ^n or λ^{n-1} is the product of the diagonal entries $\prod_i(\lambda - a_{ii})$, which gives the first two coefficients; setting $\lambda = 0$ gives the constant term $\det(-A) = (-1)^n \det A$. By the fundamental theorem of algebra $p_A(\lambda) = \prod_i(\lambda - \lambda_i)$; comparing coefficients gives the trace and determinant identities. □

Example 3.3 (A 2×2 matrix)**.** For $A = \left(\begin{smallmatrix} 4 & 1 \\ 2 & 3 \end{smallmatrix}\right)$, $p_A(\lambda) = \lambda^2 - 7\lambda + 10 = (\lambda - 2)(\lambda - 5)$. For $\lambda = 5$, $(A - 5\mathbf{I})\mathbf{v} = \mathbf{0}$ reads $-v_1 + v_2 = 0$, so $\mathbf{v}_1 = (1, 1)^\top$; for $\lambda = 2$, $2v_1 + v_2 = 0$ gives $\mathbf{v}_2 = (1, -2)^\top$. Check: $\operatorname{tr} A = 7 = 5 + 2$ and $\det A = 10 = 5 \cdot 2$.

△ Caution: Never compute eigenvalues from the characteristic polynomial

The characteristic polynomial is a theoretical device. Polynomial roots are extraordinarily sensitive to their coefficients: Wilkinson's famous example $\prod_{k=1}^{20}(x - k)$ has roots that move into the complex plane when a single coefficient is perturbed by about 2^{-23} (Wilkinson, 1963). Numerical software does the reverse – `np.roots` computes polynomial roots as eigenvalues of a companion matrix using the QR algorithm of Section 3.3. In practice: use `np.linalg.eig/eigh`, never `np.roots(np.poly(A))`.

3.1.2 Diagonalisation and matrix powers

Definition 3.4 (Similarity and diagonalisability)**.** Matrices A and B are *similar* if $B = S^{-1}AS$ for an invertible S; similar matrices have the same characteristic polynomial, hence the same eigenvalues. A is *diagonalisable* if it is similar to a diagonal matrix, i.e.

$$A = V\mathbf{\Lambda}V^{-1}, \qquad \mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \dots, \lambda_n), \tag{3.3}$$

where the columns of V are eigenvectors.

Theorem 3.5. *Eigenvectors belonging to distinct eigenvalues are linearly independent. Consequently a matrix with n distinct eigenvalues is diagonalisable. More generally, A is diagonalisable if and only if, for every eigenvalue, the geometric multiplicity* $\dim\operatorname{null}(A - \lambda\mathbf{I})$ *equals the algebraic multiplicity.*

Proof of the first claim. Suppose $\mathbf{v}_1, \dots, \mathbf{v}_k$ are eigenvectors for distinct $\lambda_1, \dots, \lambda_k$ and, for a contradiction, that k is the smallest number for which a nontrivial relation $\sum_{i=1}^{k} c_i\mathbf{v}_i = \mathbf{0}$ exists (so all $c_i \neq 0$). Applying $A - \lambda_k\mathbf{I}$ gives $\sum_{i=1}^{k-1} c_i(\lambda_i - \lambda_k)\mathbf{v}_i = \mathbf{0}$, a nontrivial relation among $k - 1$ vectors because $\lambda_i \neq \lambda_k$. This contradicts the minimality of k. □

The main payoff of (3.3) is that powers and functions of A act on eigenvalues alone:

$$A^k = V\mathbf{\Lambda}^k V^{-1}, \qquad \exp(A) = V\operatorname{diag}(e^{\lambda_1}, \dots, e^{\lambda_n})V^{-1}. \tag{3.4}$$

Writing $\mathbf{x}_0 = \sum_i c_i\mathbf{v}_i$ in the eigenbasis, $A^k\mathbf{x}_0 = \sum_i c_i\lambda_i^k\mathbf{v}_i$. As $k \to \infty$ the term with the largest $|\lambda_i|$ dominates; this simple observation drives the long-run behaviour of Markov chains, linear difference equations, vector autoregressions and the power method.

⋆ Key Concept: Eigen-decomposition decouples dynamics

In the eigenbasis a linear system $\mathbf{x}_{t+1} = A\mathbf{x}_t$ becomes n independent scalar equations $c_{i,t+1} = \lambda_i c_{i,t}$. The system is stable iff $\rho(A) < 1$; the slowest-decaying mode has rate $|\lambda|$ closest to 1; a shock to that mode has half-life $\ln(0.5)/\ln|\lambda|$ periods. This is exactly how the stability of a VAR is checked: all eigenvalues of its companion matrix must lie strictly inside the unit circle.

Not every matrix is diagonalisable. The Jordan block $J = \left(\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right)$ has the double eigenvalue 2 but only the one-dimensional eigenspace $\operatorname{span}\{(1, 0)^\top\}$. Every square matrix is similar to a block-diagonal *Jordan form*, but the Jordan form is discontinuous in the entries of A and is never computed numerically; the stable alternative is the *Schur decomposition* $A = QTQ^*$ with Q unitary and T upper triangular (Golub and Van Loan, 2013, ch. 7). Listing 3.1 illustrates the ideas numerically.

Listing 3.1: Eigenpairs, diagonalisation and matrix powers ch03/eig_basics.py

```python
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Characteristic polynomial, eigenpairs, diagonalisation and matrix powers.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

np.set_printoptions(precision=4, suppress=True)
A = np.array([[4.0, 1.0],
              [2.0, 3.0]])

# Characteristic polynomial det(lambda I - A) = lambda^2 - tr(A) lambda + det(A)
coef = np.poly(A)                       # coefficients, highest power first
print("char. poly coefficients :", coef)
print("roots of polynomial     :", np.sort(np.roots(coef)))

lam, V = np.linalg.eig(A)               # columns of V are eigenvectors
order = np.argsort(lam)
lam, V = lam[order], V[:, order]
print("eigenvalues (eig)       :", lam)
print("trace = sum, det = prod :", np.trace(A), lam.sum(),
      round(np.linalg.det(A), 10), lam.prod())

# Check A v = lambda v for each pair
resid = np.linalg.norm(A @ V - V * lam, axis=0)
print("residuals ||Av - lv||   :", resid)

# Diagonalisation A = V D V^{-1} and the power A^10
D = np.diag(lam)
Vinv = np.linalg.inv(V)
print("||A - V D V^-1||        :", np.linalg.norm(A - V @ D @ Vinv))
k = 10
Ak_eig = V @ np.diag(lam ** k) @ Vinv
Ak_dir = np.linalg.matrix_power(A, k)
print("A^10 via eigen:\n", Ak_eig)
print("max |difference| from matrix_power:", np.abs(Ak_eig - Ak_dir).max())

# A defective (non-diagonalisable) matrix: Jordan block
J = np.array([[2.0, 1.0],
              [0.0, 2.0]])
lamJ, VJ = np.linalg.eig(J)
print("Jordan block eigenvalues:", lamJ)
print("eigenvector matrix rank :", np.linalg.matrix_rank(VJ, tol=1e-8))
print("cond(VJ) (huge => defective):", f"{np.linalg.cond(VJ):.2e}")
```

Output

```
char. poly coefficients : [ 1. -7. 10.]
roots of polynomial     : [2. 5.]
eigenvalues (eig)       : [2. 5.]
trace = sum, det = prod : 7.0 7.0 10.0 10.0
residuals ||Av - lv||   : [0. 0.]
||A - V D V^-1||        : 0.0
A^10 via eigen:
 [[6510758. 3254867.]
 [6509734. 3255891.]]
```

```
max |difference| from matrix_power: 0.0
Jordan block eigenvalues: [2. 2.]
eigenvector matrix rank : 1
cond(VJ) (huge => defective): 4.50e+15
```

Note the last two lines: for the defective matrix J, `eig` still returns two eigenvector columns, but they are numerically parallel, and the condition number of V is of order $1/\varepsilon_{\text{mach}}$. A huge $\text{cond}(V)$ is the practical signature of a (nearly) defective matrix, and it warns that formulas such as $A^k = V\boldsymbol{\Lambda}^k V^{-1}$ will be inaccurate.

Historical Note: From celestial mechanics to "eigen"

Eigenvalue problems first arose in eighteenth-century mechanics: Euler and Lagrange studied the principal axes of rotating bodies and the small oscillations of mechanical systems, and Laplace and Lagrange analysed the "secular" (long-period) perturbations of planetary orbits. In 1829 Augustin-Louis Cauchy proved that the roots of the resulting *secular equation* for a symmetric system are all real – in modern language, that real symmetric matrices have real eigenvalues – and also showed that the principal axes can be chosen orthogonal. The prefix "eigen" (German for "own" or "characteristic") comes from David Hilbert, who in his 1904 work on linear integral equations spoke of *Eigenfunktionen* and *Eigenwerte*; English writers adopted the hybrid words "eigenvalue" and "eigenvector" in the following decades, alongside older terms such as "characteristic root" and "latent root" still common in econometrics.

3.1.3 Markov chains and labour-market transitions

A (time-homogeneous, finite) *Markov chain* on states $\{1, \dots, n\}$ is described by a *transition matrix* P with $p_{ij} = \mathbb{P}(s_{t+1} = j \mid s_t = i) \geq 0$ and rows summing to one, $P\mathbf{1} = \mathbf{1}$. Such a P is called (row-)*stochastic*. If $\mathbf{x}_t^\top$ is the row vector of probabilities (or population shares) over states at time t, then

$$\mathbf{x}_{t+1}^\top = \mathbf{x}_t^\top P, \qquad \text{so} \qquad \mathbf{x}_t^\top = \mathbf{x}_0^\top P^t. \tag{3.5}$$

Long-run behaviour is therefore governed by the spectrum of P. Two facts follow at once: $P\mathbf{1} = \mathbf{1}$ says that 1 is an eigenvalue (with right eigenvector $\mathbf{1}$), and since every row of P is a probability vector, $\|P\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_\infty$, so $\rho(P) \leq 1$.

Definition 3.6 (Stationary distribution)**.** A probability vector $\boldsymbol{\pi}$ ($\pi_i \geq 0$, $\sum_i \pi_i = 1$) is *stationary* for P if $\boldsymbol{\pi}^\top P = \boldsymbol{\pi}^\top$, i.e. $P^\top \boldsymbol{\pi} = \boldsymbol{\pi}$: $\boldsymbol{\pi}$ is a *left* eigenvector of P for the eigenvalue 1, normalised to sum to one.

When does $\mathbf{x}_0^\top P^t$ converge to a unique $\boldsymbol{\pi}$ regardless of the starting point? The answer is one of the most beautiful theorems of matrix theory.

Theorem 3.7 (Perron–Frobenius)**.** *Let $A \in \mathbb{R}^{n \times n}$ be a nonnegative matrix.*

(i) (Perron, 1907) If A is strictly positive, then $\rho(A)$ is a simple eigenvalue of A, it has an eigenvector with strictly positive entries, and every other eigenvalue satisfies $|\lambda| < \rho(A)$.

(ii) (Frobenius, 1912) If A is irreducible *(its directed graph is strongly connected), then $\rho(A)$ is a simple eigenvalue with a strictly positive eigenvector; if in addition A is* primitive *($A^m > 0$ entrywise for some m, equivalently irreducible and aperiodic), then $|\lambda| < \rho(A)$ for every other eigenvalue.*

We omit the proof, which can be found in Horn and Johnson (2013, ch. 8). Applied to a stochastic matrix (for which $\rho(P) = 1$) the theorem yields the fundamental result on Markov chains.

Corollary 3.8 (Convergence of a regular Markov chain)**.** *If P is stochastic and primitive, there is a unique stationary distribution $\boldsymbol{\pi} > 0$, and for every initial distribution $\mathbf{x}_0$, $\mathbf{x}_0^\top P^t \to \boldsymbol{\pi}^\top$. If P is diagonalisable with eigenvalues $1 = \lambda_1 > |\lambda_2| \geq \cdots$, then $\left\|\mathbf{x}_0^\top P^t - \boldsymbol{\pi}^\top\right\| = O(|\lambda_2|^t)$.*

Proof sketch. Write $P = V\boldsymbol{\Lambda}V^{-1}$ and let $\mathbf{w}_i^\top$ denote the rows of V^{-1} (left eigenvectors). Then $P^t = \sum_i \lambda_i^t \mathbf{v}_i \mathbf{w}_i^\top$. The $i = 1$ term is $\mathbf{1}\boldsymbol{\pi}^\top$ (because $\mathbf{v}_1 = \mathbf{1}$ and $\mathbf{w}_1 = \boldsymbol{\pi}$ under the normalisation $\mathbf{w}_1^\top \mathbf{v}_1 = 1$), and every other term decays like $|\lambda_i|^t \leq |\lambda_2|^t$. Hence $\mathbf{x}_0^\top P^t = (\mathbf{x}_0^\top \mathbf{1})\boldsymbol{\pi}^\top + O(|\lambda_2|^t) = \boldsymbol{\pi}^\top + O(|\lambda_2|^t)$. □

The quantity $1 - |\lambda_2|$ is the *spectral gap*; a large gap means fast *mixing*. Listing 3.2 applies these results to a *simulated* quarterly transition matrix between formal employment, informal employment and unemployment. The numbers are illustrative, chosen only to mimic the qualitative features of labour markets with a large informal sector (high persistence of formal jobs, frequent exits from unemployment into informal work); they are not estimates for any country.

Listing 3.2: A simulated three-state labour-market Markov chain ch03/markov_labour.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# A simulated labour-market Markov chain: eigenvalues and stationary distribution.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
states = ["Formal", "Informal", "Unemployed"]
# Simulated quarterly transition matrix, rows = origin, columns = destination
P = np.array([[0.94, 0.04, 0.02],
              [0.05, 0.85, 0.10],
              [0.08, 0.32, 0.60]])
assert np.allclose(P.sum(axis=1), 1.0)

# Stationary distribution: left eigenvector of P for eigenvalue 1
lam, W = np.linalg.eig(P.T)
order = np.argsort(-np.abs(lam))
lam, W = lam[order], W[:, order]
pi = np.real(W[:, 0] / W[:, 0].sum())
print("eigenvalues of P      :", np.round(np.real(lam), 4))
print("stationary pi         :", np.round(pi, 4))
print("check pi P = pi       :", np.allclose(pi @ P, pi))
print("half-life of shocks   : %.1f quarters"
      % (np.log(0.5) / np.log(np.abs(lam[1]))))

# Distribution path from "everyone unemployed"
x = np.array([0.0, 0.0, 1.0])
path = [x]
for t in range(40):
    x = x @ P
    path.append(x)
path = np.array(path)
print("after 4 quarters      :", np.round(path[4], 4))
print("after 40 quarters     :", np.round(path[40], 4))
gap = np.abs(path - pi).sum(axis=1)
print("TV gap at t=10, 20    : %.2e  %.2e" % (gap[10] / 2, gap[20] / 2))

# Simulate 2000 workers for 200 quarters and compare long-run shares
```

```python
n, T = 2000, 200
s = rng.integers(0, 3, size=n)
cum = P.cumsum(axis=1)
for t in range(T):
    u = rng.random(n)
    s = (u[:, None] > cum[s]).sum(axis=1)
shares = np.bincount(s, minlength=3) / n
print("simulated shares T=200:", np.round(shares, 4))
print("expected spell (quarters):",
      dict(zip(states, np.round(1 / (1 - np.diag(P)), 1))))

fig, ax = plt.subplots(figsize=(6, 3.8))
for j, name in enumerate(states):
    ax.plot(path[:, j], lw=2, label=name)
    ax.axhline(pi[j], ls=":", color="gray", lw=1)
ax.set_xlabel("quarter t")
ax.set_ylabel("share of labour force")
ax.set_title("Convergence of x0 P^t to the stationary distribution")
ax.legend(frameon=False)
plt.savefig("ch03_markov_convergence.pdf", bbox_inches="tight")
```

Output

```
eigenvalues of P      : [1.     0.8837 0.5063]
stationary pi         : [0.4878 0.3902 0.122 ]
check pi P = pi       : True
half-life of shocks   : 5.6 quarters
after 4 quarters      : [0.2238 0.5482 0.228 ]
after 40 quarters     : [0.4848 0.3926 0.1226]
TV gap at t=10, 20    : 1.24e-01  3.60e-02
simulated shares T=200: [0.5075 0.383  0.1095]
expected spell (quarters): {'Formal': 16.7, 'Informal': 6.7, 'Unemployed': 2.5}
```

Several lessons are visible. The stationary distribution implies a long-run unemployment rate of about 12% and an informal share of about 39%. The second eigenvalue 0.884 implies that deviations from steady state have a half-life of about 5.6 quarters (Fig. 3.1): even after a large shock that throws everyone out of work, the distribution is close to its long-run value within about five years. The diagonal of P gives expected spell lengths $1/(1-p_{ii})$, and the simulation of 2000 individual workers reproduces π up to sampling noise – the ergodic theorem for Markov chains at work.

Policy Lens: Labour-market flows and the steady-state unemployment rate

Labour economists decompose movements in unemployment into *flows*: job-finding rates and separation rates between states. The steady-state unemployment rate implied by current flow rates is exactly the stationary distribution of a transition matrix estimated from panel data such as rotating labour-force surveys. In a two-state model with separation rate s and job-finding rate f the formula collapses to $u^{*} = s/(s+f)$ (Exercise **3.1.3**). With an informal sector the matrix is 3×3 and policy questions become eigenvector comparisons: how much would the long-run informal share fall if a formalisation programme raised the informal-to-formal transition probability by five percentage points? Compute π before and after. The second eigenvalue tells you how long the transition will take – an important input into the political economy of any reform whose benefits arrive slowly.

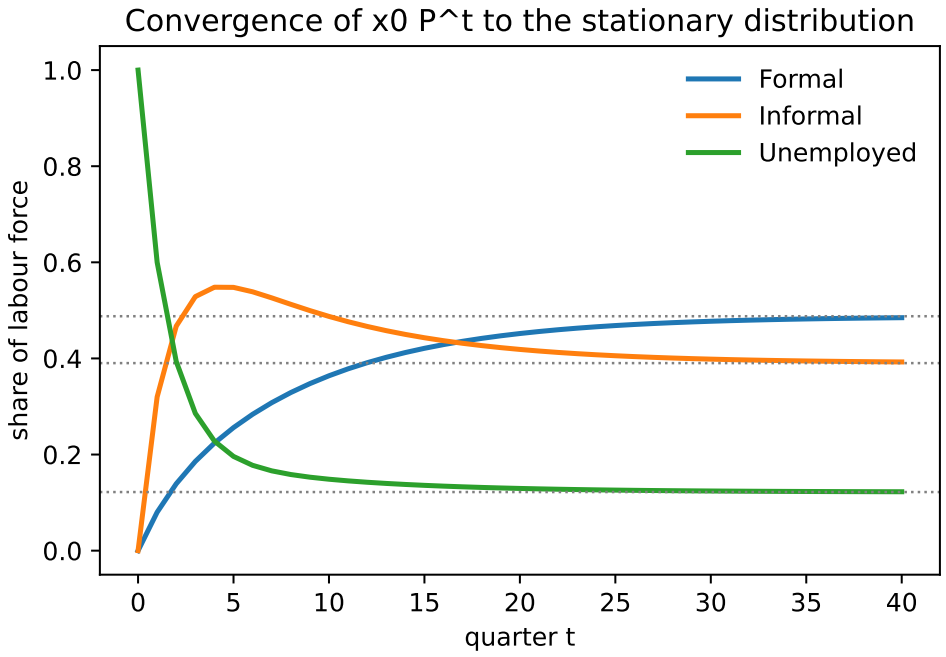


Figure 3.1: Simulated labour-market chain started with everyone unemployed. Shares converge geometrically to the stationary distribution (dotted lines) at rate $|\lambda_2|^t \approx 0.884^t$.

Historical Note: Markov and Perron–Frobenius

Andrei Andreyevich Markov introduced chains of dependent trials in 1906, in a paper extending the weak law of large numbers to variables that are not independent; in 1913 he famously applied the idea to the sequence of vowels and consonants in Pushkin's *Eugene Onegin*. The matrix theory came independently. Oskar Perron proved his theorem on positive matrices in 1907, motivated by continued fractions, and Georg Frobenius extended it to nonnegative irreducible matrices in 1912. The two strands met in the 1930s, and Perron–Frobenius theory now underlies Markov chain theory, input–output economics (Leontief), stable population theory in demography and the PageRank algorithm (Section 3.3.4).

Exercises

3.1.1 Theory Prove that if $A\mathbf{v} = \lambda\mathbf{v}$ then $A^k\mathbf{v} = \lambda^k\mathbf{v}$, $p(A)\mathbf{v} = p(\lambda)\mathbf{v}$ for any polynomial p, and, if A is invertible, $A^{-1}\mathbf{v} = \lambda^{-1}\mathbf{v}$. Deduce that a matrix is invertible iff $0 \notin \sigma(A)$.

3.1.2 Theory Show that A and $A^\top$ have the same eigenvalues but in general different eigenvectors. Show that a triangular matrix has its diagonal entries as eigenvalues. Find a 2×2 real matrix with no real eigenvalues and interpret it geometrically.

3.1.3 Theory Two-state labour market. Let $P = \begin{pmatrix} 1-s & s \\ f & 1-f \end{pmatrix}$ with states (employed, unemployed) and $0 < s, f < 1$. Find both eigenvalues of P, show that the stationary unemployment rate is $u^* = s/(s+f)$, and show that the deviation $u_t - u^*$ decays like $(1-s-f)^t$.

3.1.4 Python Using `markov_labour.py`, compute P^t for $t = 1, 4, 20, 100$ via the eigen-decomposition and via `np.linalg.matrix_power`, and confirm that every row of P^{100} is (numerically) $\boldsymbol{\pi}^\top$. Plot $\log \|\mathbf{x}_t - \boldsymbol{\pi}\|_1$ against t and estimate its slope; compare it with $\log|\lambda_2|$.

3.1.5 Python Policy A formalisation programme raises p_{21} (informal $\to$ formal) from 0.05 to 0.10, with the change taken out of p_{22}. Compute the new stationary distribution, the new

spectral gap and half-life, and the expected number of quarters before the formal share is within one percentage point of its new steady state, starting from the old one.

3.1.6 ★ Graduate Let P be a periodic chain, e.g. $P = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. Compute its eigenvalues, show that $\mathbf{x}_0^\top P^t$ does not converge in general, but that the Cesàro average $\frac{1}{T}\sum_{t<T} \mathbf{x}_0^\top P^t$ does. Which hypothesis of Corollary 3.8 fails? Show that the "lazy" chain $\frac{1}{2}(\mathbf{I} + P)$ is primitive.

3.1.7 ★ Graduate Policy (Reversibility.) A chain is *reversible* if $\pi_i p_{ij} = \pi_j p_{ji}$ for all i, j. Show that $D^{1/2} P D^{-1/2}$ is then symmetric, where $D = \operatorname{diag}(\boldsymbol{\pi})$, so P has real eigenvalues. Is the simulated labour-market chain reversible? What would reversibility mean economically for gross flows between formal and informal work?

3.2 Symmetric Matrices, Quadratic Forms and Covariance

Covariance matrices, Hessians of smooth functions, Gram matrices $\mathbf{X}^\top\mathbf{X}$, graph Laplacians and kernel matrices are all symmetric. Symmetry buys three things that general matrices lack: real eigenvalues, orthogonal eigenvectors, and a guaranteed diagonalisation that is perfectly conditioned.

3.2.1 The spectral theorem

Theorem 3.9 (Spectral theorem). *If $A \in \mathbb{R}^{n\times n}$ is symmetric ($A = A^\top$), then all its eigenvalues are real and there is an orthogonal matrix Q ($Q^\top Q = \mathbf{I}$) such that*

$$A = Q\boldsymbol{\Lambda}Q^\top = \sum_{i=1}^{n} \lambda_i \, \mathbf{Q}_{:i}\mathbf{Q}_{:i}^\top, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \dots, \lambda_n). \tag{3.6}$$

Equivalently, $\mathbb{R}^n$ has an orthonormal basis of eigenvectors of A, and A is a weighted sum of rank-one orthogonal projections.

Proof sketch. Real eigenvalues. If $A\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \in \mathbb{C}^n \setminus \{\mathbf{0}\}$, then $\bar{\mathbf{v}}^\top A\mathbf{v} = \lambda \|\mathbf{v}\|^2$. Taking complex conjugates and using $A = \bar{A} = A^\top$, $\bar{\mathbf{v}}^\top A\mathbf{v} = \overline{\bar{\mathbf{v}}^\top A\mathbf{v}}$ is real, so λ is real.

Orthogonality. If $A\mathbf{v} = \lambda\mathbf{v}$, $A\mathbf{w} = \mu\mathbf{w}$ with $\lambda \neq \mu$, then $\lambda\mathbf{w}^\top\mathbf{v} = \mathbf{w}^\top A\mathbf{v} = (A\mathbf{w})^\top\mathbf{v} = \mu\mathbf{w}^\top\mathbf{v}$, so $\mathbf{w}^\top\mathbf{v} = 0$.

Diagonalisation, by induction on n. Take a real unit eigenvector $\mathbf{q}_1$ for some eigenvalue λ_1 (it exists since λ_1 is real) and complete it to an orthogonal matrix $Q_1 = [\mathbf{q}_1 \; W]$. Then

$$Q_1^\top A Q_1 = \begin{pmatrix} \lambda_1 & \mathbf{q}_1^\top A W \\ W^\top A\mathbf{q}_1 & W^\top A W \end{pmatrix} = \begin{pmatrix} \lambda_1 & \mathbf{0}^\top \\ \mathbf{0} & W^\top A W \end{pmatrix},$$

because $W^\top A\mathbf{q}_1 = \lambda_1 W^\top\mathbf{q}_1 = \mathbf{0}$ and the matrix is symmetric. The block $W^\top A W$ is symmetric of size $n-1$, so by induction it equals $Q_2\boldsymbol{\Lambda}_2 Q_2^\top$; combining, $Q = Q_1 \operatorname{diag}(1, Q_2)$ diagonalises A. □

A second, variational characterisation of the eigenvalues is often more useful in statistics.

Theorem 3.10 (Rayleigh quotient and Courant–Fischer). *Let A be symmetric with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ and orthonormal eigenvectors $\mathbf{q}_1, \dots, \mathbf{q}_n$. For $\mathbf{x} \neq \mathbf{0}$ define the* Rayleigh quotient *$R_A(\mathbf{x}) = \mathbf{x}^\top A\mathbf{x} / \mathbf{x}^\top\mathbf{x}$. Then*

$$\lambda_n \leq R_A(\mathbf{x}) \leq \lambda_1, \qquad \lambda_1 = \max_{\mathbf{x}\neq\mathbf{0}} R_A(\mathbf{x}), \qquad \lambda_k = \max_{\mathbf{x}\perp\mathbf{q}_1,\dots,\mathbf{q}_{k-1}} R_A(\mathbf{x}),$$

with the maxima attained at the eigenvectors. More generally (Courant–Fischer) $\lambda_k = \max_{\dim S = k} \min_{\mathbf{x}\in S\setminus\{\mathbf{0}\}} R_A(\mathbf{x})$.

Proof of the first claims. Write $\mathbf{x} = Q\mathbf{y}$. Then $R_A(\mathbf{x}) = \sum_i \lambda_i y_i^2 / \sum_i y_i^2$, a weighted average of the eigenvalues with weights $y_i^2/\|\mathbf{y}\|^2$. It is maximised by putting all weight on λ_1 ($\mathbf{y} = \mathbf{e}_1$, $\mathbf{x} = \mathbf{q}_1$). Restricting $\mathbf{x} \perp \mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ forces $y_1 = \cdots = y_{k-1} = 0$, and the same argument gives λ_k. □

Theorem 3.10 is the mathematical heart of PCA (Section 3.5): the direction of maximal variance of a data cloud is the top eigenvector of its covariance matrix.

3.2.2 Quadratic forms and definiteness

A *quadratic form* is a function $q(\mathbf{x}) = \mathbf{x}^\top A\mathbf{x}$ with A symmetric (any square B can be replaced by its symmetric part $(B + B^\top)/2$ without changing q). In the eigenbasis, $q(\mathbf{x}) = \sum_i \lambda_i y_i^2$ with $\mathbf{y} = Q^\top\mathbf{x}$: a quadratic form is a sum of squares along orthogonal axes, weighted by eigenvalues.

Definition 3.11 (Definiteness)**.** A symmetric A is *positive definite* (PD, $A \succ 0$) if $\mathbf{x}^\top A\mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$; *positive semidefinite* (PSD, $A \succeq 0$) if $\mathbf{x}^\top A\mathbf{x} \geq 0$; *negative (semi)definite* if $-A$ is; and *indefinite* otherwise.

Theorem 3.12 (Tests for positive definiteness)**.** *For symmetric $A \in \mathbb{R}^{n\times n}$ the following are equivalent:*

(a) $A \succ 0$;

(b) all eigenvalues of A are strictly positive;

(c) (Sylvester's criterion) all leading principal minors $\det(A_{1:k,1:k})$, $k = 1, \dots, n$, *are strictly positive;*

(d) A has a Cholesky factorisation $A = LL^\top$ with L lower triangular with positive diagonal;

(e) $A = B^\top B$ for some B with linearly independent columns.

For PSD, (b) becomes $\lambda_i \geq 0$ and (e) drops the independence requirement; but (c) must be replaced by all *principal minors being nonnegative – leading ones are not enough.*

Proof of (a)⇔(b)⇔(e). (a)⇒(b): take $\mathbf{x} = \mathbf{q}_i$, so $\lambda_i = \mathbf{q}_i^\top A\mathbf{q}_i > 0$. (b)⇒(a): $\mathbf{x}^\top A\mathbf{x} = \sum_i \lambda_i y_i^2 > 0$ unless $\mathbf{y} = Q^\top\mathbf{x} = \mathbf{0}$. (b)⇒(e): $B = \Lambda^{1/2}Q^\top$. (e)⇒(a): $\mathbf{x}^\top B^\top B\mathbf{x} = \|B\mathbf{x}\|^2 > 0$ when $B\mathbf{x} \neq \mathbf{0}$, which holds for $\mathbf{x} \neq \mathbf{0}$ by independence of the columns. For (c) and (d) see Golub and Van Loan (2013, ch. 4). □

⋆ Key Concept: Definiteness = signs of eigenvalues

PD matrices are the matrix analogue of positive numbers. Hessians that are PD characterise strict local minima (Chapter 8); covariance matrices are always PSD and are PD exactly when no linear combination of the variables is constant; $\mathbf{X}^\top\mathbf{X}$ is PD exactly when $\mathbf{X}$ has full column rank, which is the identification condition for OLS. In code the cheapest reliable PD test is to attempt a Cholesky factorisation.

3.2.3 Covariance matrices, ellipsoids and the Mahalanobis distance

Let $\mathbf{x} \in \mathbb{R}^p$ be a random vector with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma} = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$. For any $\mathbf{w}$, $\mathrm{Var}(\mathbf{w}^\top\mathbf{x}) = \mathbf{w}^\top\boldsymbol{\Sigma}\mathbf{w} \geq 0$, so $\boldsymbol{\Sigma}$ is PSD. By the spectral theorem $\boldsymbol{\Sigma} = Q\boldsymbol{\Lambda}Q^\top$, and the transformed vector $\mathbf{y} = Q^\top(\mathbf{x} - \boldsymbol{\mu})$ has $\mathrm{Cov}(\mathbf{y}) = \boldsymbol{\Lambda}$: *rotating to the eigenbasis decorrelates the variables*, and λ_i is the variance in direction $\mathbf{q}_i$.

Definition 3.13 (Mahalanobis distance)**.** For PD $\boldsymbol{\Sigma}$, the Mahalanobis distance of $\mathbf{x}$ from $\boldsymbol{\mu}$ is

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})} = \Big(\sum_{i=1}^{p} \frac{y_i^2}{\lambda_i}\Big)^{1/2}, \qquad \mathbf{y} = Q^\top(\mathbf{x}-\boldsymbol{\mu}). \tag{3.7}$$

The level sets $\{\mathbf{x} : d_M(\mathbf{x}) = c\}$ are ellipsoids centred at $\boldsymbol{\mu}$ whose axes point along the eigenvectors $\mathbf{q}_i$ with semi-axis lengths $c\sqrt{\lambda_i}$. The Mahalanobis distance measures how many "standard deviations" $\mathbf{x}$ is from the centre *in the metric of the data*: a point can be close in Euclidean distance yet far in Mahalanobis distance if it lies along a direction of low variance. If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ then $d_M^2(\mathbf{x}) \sim \chi^2_p$ (Chapter 5), which gives a principled multivariate outlier test. The geometry is shown in Fig. 3.2 and computed in Listing 3.3 for a *simulated* household survey of (centred) log income and log consumption.

Listing 3.3: Spectral decomposition, PD tests and Mahalanobis distance ch03/spectral_quadform.py

```python
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Spectral theorem, definiteness tests and Mahalanobis ellipses for a covariance.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
np.set_printoptions(precision=4, suppress=True)

# Simulated household survey: log income and log consumption (centred)
Sigma_true = np.array([[1.00, 0.72],
                       [0.72, 0.64]])
X = rng.multivariate_normal([0, 0], Sigma_true, size=500)
S = np.cov(X, rowvar=False)
lam, Q = np.linalg.eigh(S)            # ascending eigenvalues, orthonormal Q
print("sample covariance S:\n", S)
print("eigenvalues        :", lam)
print("Q'Q = I ?          :", np.allclose(Q.T @ Q, np.eye(2)))
print("||S - Q L Q'||     : %.2e" % np.linalg.norm(S - Q @ np.diag(lam) @ Q.T))

# Three tests of positive definiteness
minors = [np.linalg.det(S[:k, :k]) for k in (1, 2)]
print("leading minors     :", np.round(minors, 4))
L = np.linalg.cholesky(S)             # succeeds only if S is PD
print("Cholesky factor L  :\n", L)
B = np.array([[1.0, 2.0], [2.0, 1.0]])   # indefinite
print("eig of indefinite B:", np.linalg.eigvalsh(B))
try:
    np.linalg.cholesky(B)
except np.linalg.LinAlgError as err:
    print("Cholesky of B fails:", err)

# Mahalanobis vs Euclidean distance for two candidate points
Sinv = np.linalg.inv(S)
pts = np.array([[1.5, 1.5], [1.5, -0.5]])
for p in pts:
    d_e = np.linalg.norm(p)
    d_m = np.sqrt(p @ Sinv @ p)
    print(f"point {p}: Euclid = {d_e:.3f}, Mahalanobis = {d_m:.3f}")
d2 = np.einsum("ij,jk,ik->i", X, Sinv, X)
```

```
print("share with d^2 > 5.99 (chi2_2 95%%): %.3f" % np.mean(d2 > 5.991))

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.scatter(X[:, 0], X[:, 1], s=6, alpha=0.4, color="gray")
t = np.linspace(0, 2 * np.pi, 200)
circle = np.vstack([np.cos(t), np.sin(t)])
for c in (1, 2, 3):
    ell = Q @ np.diag(np.sqrt(lam)) @ circle * c
    ax.plot(ell[0], ell[1], lw=1.5, color="C0")
for j in range(2):
    v = Q[:, j] * 2 * np.sqrt(lam[j])
    ax.annotate("", xy=v, xytext=(0, 0),
                arrowprops=dict(arrowstyle="->", color="C3", lw=2))
ax.plot(*pts.T, "k*", ms=12)
ax.set_aspect("equal")
ax.set_xlabel("log income (centred)")
ax.set_ylabel("log consumption (centred)")
ax.set_title("Mahalanobis ellipses x' S^-1 x = c^2, c = 1, 2, 3")
plt.savefig("ch03_mahalanobis.pdf", bbox_inches="tight")
```

Output

```
sample covariance S:
 [[0.9556 0.7042]
 [0.7042 0.63  ]]
eigenvalues        : [0.07   1.5156]
Q'Q = I ?          : True
||S - Q L Q'||     : 1.92e-16
leading minors     : [0.9556 0.1061]
Cholesky factor L  :
 [[0.9775 0.    ]
 [0.7204 0.3333]]
eig of indefinite B: [-1.  3.]
Cholesky of B fails: Matrix is not positive definite
point [1.5 1.5]: Euclid = 2.121, Mahalanobis = 1.938
point [ 1.5 -0.5]: Euclid = 1.581, Mahalanobis = 5.056
share with d^2 > 5.99 (chi2_2 95%): 0.050
```

The point $(1.5, 1.5)$ is further from the origin in Euclidean terms (2.12 versus 1.58), yet its Mahalanobis distance is less than 2 while that of $(1.5, -0.5)$ exceeds 5. Income and consumption move together; a household with high income but low consumption is unusual *given the correlation*. The eigenvalues 1.52 and 0.07 show that almost all variation lies along one direction – a first hint of dimension reduction.

△ Caution: Inverting near-singular covariance matrices

$\boldsymbol{\Sigma}^{-1}$ weights each direction by $1/\lambda_i$. When some eigenvalues are tiny – many correlated variables, or p close to the sample size n – the sample covariance is nearly singular, d_M is dominated by noise in the smallest eigen-directions, and outlier tests become unreliable. When $p > n$ the sample covariance is singular and $\hat{\boldsymbol{\Sigma}}^{-1}$ does not exist. Remedies are shrinkage estimators such as $(1-\alpha)\hat{\boldsymbol{\Sigma}} + \alpha \frac{\operatorname{tr}\hat{\boldsymbol{\Sigma}}}{p}\mathbf{I}$ (Ledoit and Wolf, 2004), which lift the small eigenvalues, or working in the span of the leading principal components.

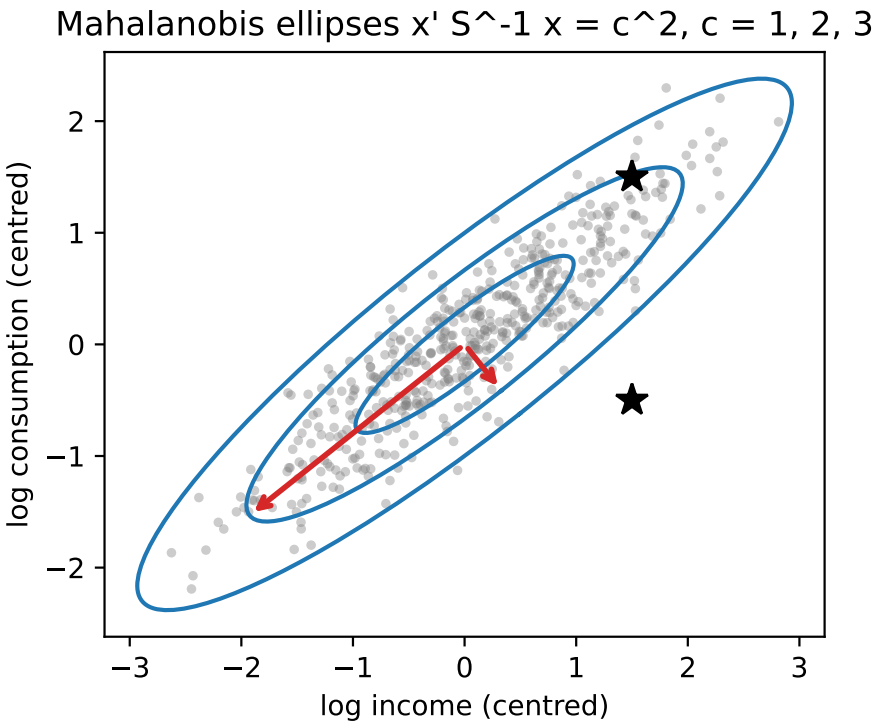


Figure 3.2: Mahalanobis ellipses for a simulated income–consumption covariance. Arrows are the eigenvectors scaled by $2\sqrt{\lambda_i}$. The two stars have similar Euclidean norms, but the point $(1.5, -0.5)$, which has high income and low consumption, lies far outside the $c = 3$ ellipse.

Graduate Perspective: Matrix functions and whitening

The spectral theorem lets us define $f(A) = Q\,f(\Lambda)Q^\top$ for any function f on the spectrum. Important examples are $\Sigma^{1/2}$, $\Sigma^{-1/2}$ and $\log \Sigma$. *Whitening* $\mathbf{z} = \Sigma^{-1/2}(\mathbf{x} - \boldsymbol{\mu})$ gives $\mathrm{Cov}(\mathbf{z}) = \mathbf{I}$, and $d_M(\mathbf{x}) = \|\mathbf{z}\|$; ZCA whitening in deep learning, the GLS transformation in econometrics and the Mahalanobis matching estimator in programme evaluation (Imbens and Rubin, 2015) all rest on it. Note that $\Sigma^{-1/2}$ is not the only whitening matrix: any W with $W^\top W = \Sigma^{-1}$ works (for example $W = L^{-1}$ from the Cholesky factor), and different choices give rotated versions of $\mathbf{z}$.

Exercises

3.2.1 Theory Prove that if A is symmetric and $Q^\top A Q = \Lambda$, then $\mathrm{tr}(A) = \sum_i \lambda_i$, $\|A\|_F^2 = \sum_i \lambda_i^2$ and $\|A\|_2 = \max_i |\lambda_i|$.

3.2.2 Theory Show that $\mathbf{X}^\top\mathbf{X}$ is PSD for every $\mathbf{X} \in \mathbb{R}^{n\times p}$ and that it is PD iff $\mathrm{rank}\,\mathbf{X} = p$. Show also that $\mathbf{X}^\top\mathbf{X}$ and $\mathbf{X}\mathbf{X}^\top$ have the same nonzero eigenvalues.

3.2.3 Theory Give a 2×2 symmetric matrix whose leading principal minors are both nonnegative but which is not PSD, showing why Sylvester's criterion needs *all* principal minors in the semidefinite case.

3.2.4 Theory Show that if $\mathbf{x} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$ with $\Sigma \succ 0$ then $d_M^2(\mathbf{x}) \sim \chi^2_p$. (Hint: whiten.) Hence derive the 95% outlier threshold 5.99 used for $p = 2$ in Listing 3.3.

3.2.5 Python Generate $n = 50$ observations of $p = 40$ correlated variables, compute the sample covariance and plot its eigenvalues against the true ones. Repeat with the Ledoit–Wolf

shrinkage estimator from `sklearn.covariance`. What happens to the smallest eigenvalues, and to the share of observations flagged as Mahalanobis outliers?

3.2.6 ★ Graduate Prove the Courant–Fischer theorem, and deduce Weyl's inequality: for symmetric A, E, $|\lambda_k(A+E) - \lambda_k(A)| \le \|E\|_2$. Why does this say that the eigenvalues of symmetric matrices are *perfectly conditioned*, whereas eigenvectors may not be?

3.2.7 Policy A targeting agency flags households whose (income, consumption) pair has Mahalanobis distance above the 99% χ^2_2 quantile for re-interview. Discuss what kinds of households are flagged, why Euclidean distance on standardised variables would flag different households, and the risks of this rule when consumption is measured with error.

3.3 Computing Eigenvalues

Since eigenvalues are roots of a degree-n polynomial, and by the Abel–Ruffini theorem there is no formula in radicals for the roots of a general polynomial of degree five or more, *every* eigenvalue algorithm for $n \ge 5$ must be iterative. This section develops the simplest iterations, which are also the ones used at web scale, and then explains what `np.linalg.eig` does.

3.3.1 Power iteration

Algorithm 3 Power iteration

Require: $A \in \mathbb{R}^{n\times n}$, starting vector $\mathbf{x}_0$ with $\|\mathbf{x}_0\| = 1$

1: **for** $k = 1, 2, \dots$ **do**
2: $\quad \mathbf{y}_k \leftarrow A\mathbf{x}_{k-1}$
3: $\quad \mathbf{x}_k \leftarrow \mathbf{y}_k / \|\mathbf{y}_k\|$
4: $\quad \rho_k \leftarrow \mathbf{x}_k^{\top} A \mathbf{x}_k$ ▷ Rayleigh quotient estimate of λ_1
5: **end for**

Theorem 3.14 (Convergence of power iteration)**.** *Let A be diagonalisable with eigenvalues ordered $|\lambda_1| > |\lambda_2| \ge \cdots \ge |\lambda_n|$ and eigenvectors $\mathbf{v}_i$, and suppose $\mathbf{x}_0 = \sum_i c_i \mathbf{v}_i$ with $c_1 \neq 0$. Then $\mathbf{x}_k$ converges in direction to $\pm\mathbf{v}_1$ with*

$$\operatorname{dist}(\mathbf{x}_k, \operatorname{span}\{\mathbf{v}_1\}) = O\!\left(\left|\lambda_2/\lambda_1\right|^{k}\right).$$

If A is symmetric, the Rayleigh quotient converges twice as fast: $\left|\rho_k - \lambda_1\right| = O(|\lambda_2/\lambda_1|^{2k})$.

Proof. Up to normalisation, $\mathbf{x}_k \propto A^k \mathbf{x}_0 = c_1 \lambda_1^k \big(\mathbf{v}_1 + \sum_{i\ge 2} (c_i/c_1)(\lambda_i/\lambda_1)^k \mathbf{v}_i\big)$, and every ratio $|\lambda_i/\lambda_1|^k \le |\lambda_2/\lambda_1|^k \to 0$. For symmetric A with orthonormal eigenvectors, write $\mathbf{x}_k = \mathbf{q}_1 \cos\theta_k + \mathbf{u}\sin\theta_k$ with $\mathbf{u} \perp \mathbf{q}_1$ a unit vector and $\sin\theta_k = O(|\lambda_2/\lambda_1|^k)$. Then $\rho_k = \lambda_1 \cos^2\theta_k + (\mathbf{u}^{\top} A \mathbf{u}) \sin^2\theta_k$, so $\left|\rho_k - \lambda_1\right| \le 2\|A\|_2 \sin^2\theta_k = O(|\lambda_2/\lambda_1|^{2k})$: an $O(\varepsilon)$ error in the eigenvector gives an $O(\varepsilon^2)$ error in the eigenvalue because $\mathbf{q}_1$ is a stationary point of the Rayleigh quotient. □

The theorem says that the speed is controlled entirely by the *eigenvalue ratio* $|\lambda_2/\lambda_1|$, and that each iteration costs only one matrix–vector product – for a sparse matrix with m nonzeros that is $O(m)$ work, which is why power iteration remains viable for matrices with billions of rows. Listing 3.4 verifies the rate on symmetric 50×50 matrices with prescribed spectra.

Listing 3.4: Power iteration, inverse iteration and Rayleigh-quotient iteration
ch03/power_iteration.py

```python
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Power iteration, Rayleigh quotient and inverse iteration; convergence rates.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)


def make_sym(eigs, rng):
    """Symmetric matrix with prescribed eigenvalues and random eigenvectors."""
    Q, _ = np.linalg.qr(rng.standard_normal((len(eigs), len(eigs))))
    return Q @ np.diag(eigs) @ Q.T, Q


def power_iteration(A, x0, iters):
    x = x0 / np.linalg.norm(x0)
    rq = []
    for _ in range(iters):
        y = A @ x
        x = y / np.linalg.norm(y)
        rq.append(x @ A @ x)              # Rayleigh quotient
    return x, np.array(rq)


n, iters = 50, 60
x0 = rng.standard_normal(n)
fig, ax = plt.subplots(figsize=(6, 3.8))
for ratio in (0.5, 0.8, 0.95):
    eigs = np.concatenate([[10.0, 10.0 * ratio],
                           rng.uniform(0, 10.0 * ratio * 0.9, n - 2)])
    A, Q = make_sym(eigs, rng)
    x, rq = power_iteration(A, x0, iters)
    err = np.abs(rq - 10.0)
    vec_err = 1 - abs(x @ Q[:, 0])
    ax.semilogy(err, lw=2, label=f"|l2/l1| = {ratio}")
    ax.semilogy(10 * ratio ** (2 * np.arange(iters)), "k:", lw=0.8)
    print(f"ratio {ratio:4.2f}: RQ error after {iters} its = {err[-1]:.2e}, "
          f"1-|cos| = {vec_err:.2e}")
ax.set_xlabel("iteration k")
ax.set_ylabel("|Rayleigh quotient - l1|")
ax.set_title("Power iteration: error ~ (l2/l1)^(2k) for symmetric A")
ax.legend(frameon=False)
plt.savefig("ch03_power_convergence.pdf", bbox_inches="tight")

# Inverse iteration with a shift: find the eigenvalue closest to mu
eigs = np.arange(1.0, n + 1.0)
A, Q = make_sym(eigs, rng)
mu = 17.3
x = x0 / np.linalg.norm(x0)
M = A - mu * np.eye(n)
for k in range(8):
    x = np.linalg.solve(M, x)
    x /= np.linalg.norm(x)
```

```
    print(f"inverse iteration k={k + 1}: RQ = {x @ A @ x:.12f}")

# Rayleigh quotient iteration: update the shift every step (cubic rate)
x = Q[:, 30] + 0.03 * rng.standard_normal(n)
x /= np.linalg.norm(x)
for k in range(5):
    rho = x @ A @ x
    print(f"RQI k={k}: rho = {rho:.15f}")
    x = np.linalg.solve(A - rho * np.eye(n), x)
    x /= np.linalg.norm(x)
```

Output

```
ratio 0.50: RQ error after 60 its = 3.55e-15, 1-|cos| = 0.00e+00
ratio 0.80: RQ error after 60 its = 6.14e-11, 1-|cos| = 1.54e-11
ratio 0.95: RQ error after 60 its = 2.07e-02, 1-|cos| = 2.09e-02
inverse iteration k=1: RQ = 17.115652051672
inverse iteration k=2: RQ = 17.027092864696
inverse iteration k=3: RQ = 17.005270242356
inverse iteration k=4: RQ = 17.000984239529
inverse iteration k=5: RQ = 17.000181655154
inverse iteration k=6: RQ = 17.000033412167
inverse iteration k=7: RQ = 17.000006139433
inverse iteration k=8: RQ = 17.000001127784
RQI k=0: rho = 30.802462909572359
RQI k=1: rho = 30.999708977431222
RQI k=2: rho = 30.999999999975792
RQI k=3: rho = 30.999999999999979
RQI k=4: rho = 30.999999999999968
```

3.3.2 Shifts, inverse iteration and the Rayleigh quotient

Power iteration finds only the dominant eigenvalue, and slowly if the ratio is near one. Both limitations are removed by applying it to a transformed matrix with the same eigenvectors. If $A\mathbf{v} = \lambda\mathbf{v}$ then $(A - \mu\mathbf{I})^{-1}\mathbf{v} = (\lambda - \mu)^{-1}\mathbf{v}$. Hence the eigenvalue of A *closest to the shift* μ becomes the dominant eigenvalue of $(A - \mu\mathbf{I})^{-1}$, and the convergence ratio is

$$\frac{|\lambda_J - \mu|}{|\lambda_K - \mu|}, \qquad \lambda_J = \text{closest}, \ \lambda_K = \text{second closest to } \mu.$$

This is *inverse iteration*: each step solves a linear system $(A - \mu\mathbf{I})\mathbf{y}_k = \mathbf{x}_{k-1}$ (factorise once, then reuse the factorisation). In the listing, with eigenvalues $1, 2, \ldots, 50$ and $\mu = 17.3$, the ratio is $0.3/0.7 \approx 0.43$ for the eigenvector and 0.18 for the Rayleigh quotient, which is what we observe.

Rayleigh-quotient iteration (RQI) updates the shift at every step, $\mu_k = \rho_k$. Because the shift approaches the eigenvalue, the ratio tends to zero and convergence is *cubic* for symmetric matrices: the number of correct digits roughly triples per step, as the last five lines of output show (30.80, then 30.99971, then 30.999999999976). The price is a new factorisation at every step, and the eigenvalue found depends on the starting vector.

△ Caution: Near-singular systems in inverse iteration are a feature

When μ is very close to an eigenvalue, $A - \mu\mathbf{I}$ is nearly singular and the solve $(A - \mu\mathbf{I})\mathbf{y} = \mathbf{x}$ is badly conditioned (Section 3.6). Remarkably this does not harm inverse iteration: the

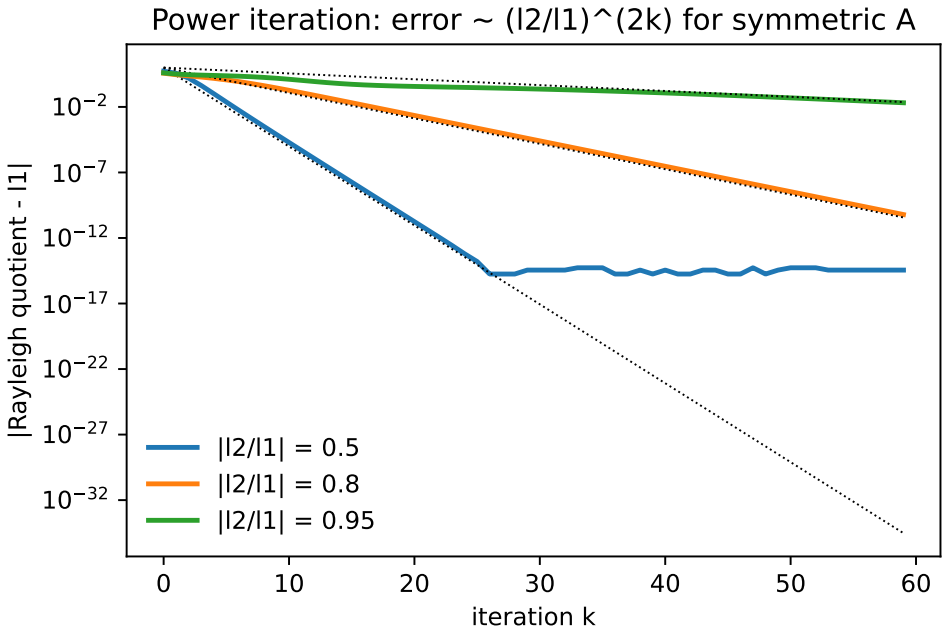


Figure 3.3: Error of the Rayleigh quotient in power iteration for three eigenvalue ratios. Dotted lines are the theoretical rate $(\lambda_2/\lambda_1)^{2k}$. With ratio 0.95 sixty iterations leave an error of order 10^{-2}; with ratio 0.5 machine precision is reached in about 25.

error in $\mathbf{y}$ is large but lies almost entirely in the direction of the wanted eigenvector, which is exactly the direction we normalise and keep (Trefethen and Bau, 1997, Lecture 27). Do not "fix" it by perturbing the shift.

3.3.3 The QR algorithm and library routines

Production eigen-solvers use the *QR algorithm*. In its basic form: set $A_0 = A$, and for $k = 0, 1, \dots$ factor $A_k = Q_k R_k$ and form $A_{k+1} = R_k Q_k = Q_k^{\top} A_k Q_k$. Every A_k is orthogonally similar to A, and under mild conditions A_k converges to an upper triangular (Schur) form whose diagonal holds the eigenvalues. The QR algorithm is in fact simultaneous power iteration on a whole basis, made numerically stable by re-orthogonalisation. Practical versions (i) first reduce A by orthogonal similarity to Hessenberg form (tridiagonal if A is symmetric), costing $O(n^3)$ once and making each QR step $O(n^2)$ (or $O(n)$); (ii) use shifts, the matrix analogue of RQI, giving quadratic or cubic convergence; and (iii) deflate converged eigenvalues. The whole computation costs about $10n^3$ flops for a general matrix and fewer for a symmetric one (Golub and Van Loan, 2013, ch. 7–8).

NumPy exposes two families of LAPACK drivers:

- `np.linalg.eig` (`geev`): general square matrices; returns possibly complex eigenvalues in no particular order and eigenvectors normalised to unit length but *not* orthogonal in general;
- `np.linalg.eigh` (`syevd`): symmetric/Hermitian matrices; uses only the lower triangle, returns *real* eigenvalues in *ascending* order and an orthonormal eigenvector matrix; faster and more accurate. `eigvalsh` skips the eigenvectors.

`scipy.sparse.linalg.eigs/eigsh` use the Arnoldi/Lanczos methods (Krylov-subspace refinements of the power method) to find a few eigenpairs of large sparse matrices.

⋆ Key Concept: Which eigen-solver?

Symmetric (covariance, Gram, Hessian, Laplacian)? Use `eigh` – and remember the eigenvalues come in *ascending* order. General square matrix (transition matrix, VAR companion)? Use `eig` and expect complex output. Only a few eigenpairs of a huge sparse matrix? Use `eigsh`/`eigs` or power iteration. Rectangular data matrix? You want the SVD (Section 3.4), not an eigen-decomposition of $\mathbf{X}^\top\mathbf{X}$.

3.3.4 PageRank on a simulated trade network

Consider n nodes (web pages, or here districts linked by trade) and let M be the *column-stochastic* matrix with M_{ji} the share of node i's outgoing links (or flows) that go to node j. A node is "important" if important nodes send much to it: the importance vector $\mathbf{r}$ should satisfy $\mathbf{r} = M\mathbf{r}$. To guarantee a unique positive solution, PageRank (Brin and Page, 1998; Page et al., 1999) replaces M by the *Google matrix*

$$G = \alpha M + \frac{1-\alpha}{n}\mathbf{1}\mathbf{1}^\top, \qquad 0 < \alpha < 1, \tag{3.8}$$

after replacing columns of dangling nodes (no out-links) by $\mathbf{1}/n$. G is strictly positive and column-stochastic, so by Perron's theorem it has a unique positive eigenvector $\mathbf{r}$ with $G\mathbf{r} = \mathbf{r}$, $\mathbf{1}^\top\mathbf{r} = 1$, and all other eigenvalues satisfy $|\lambda| \le \alpha$ (Langville and Meyer, 2006). Hence power iteration converges at rate at least α^k whatever the network – with $\alpha = 0.85$, about 0.07 decimal digits per iteration in the worst case, and in practice much faster. $\mathbf{r}$ is the stationary distribution of a "random surfer" who follows a link with probability α and teleports uniformly otherwise.

Listing 3.5: PageRank of a simulated inter-district trade network; eig versus eigh `ch03/pagerank.py`

```python
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# PageRank of a simulated inter-district trade network; eig versus eigh.
# (c) 2026 Stephen E. Moore. MIT Licence.
import time
import numpy as np

rng = np.random.default_rng(803)
n = 10
names = [f"D{i:02d}" for i in range(n)]
# Simulated directed trade links: W[i, j] = value shipped from i to j.
# District D00 is a hub port that many districts ship to.
W = rng.gamma(0.6, 1.0, size=(n, n)) * (rng.random((n, n)) < 0.4)
W[:, 0] += rng.uniform(1.0, 3.0, n)
np.fill_diagonal(W, 0.0)
W[7, :] = 0.0                          # D07 is a "dangling" node (no exports)

# Column-stochastic link matrix: M[j, i] = share of i's flows going to j
out = W.sum(axis=1)
M = np.where(out[:, None] > 0, W / np.where(out > 0, out, 1)[:, None], 1 / n).T
alpha = 0.85
G = alpha * M + (1 - alpha) / n * np.ones((n, n))   # the Google matrix

# Power iteration
r = np.full(n, 1 / n)
for k in range(1, 200):
    r_new = G @ r
    if np.abs(r_new - r).sum() < 1e-12:
        break
    r = r_new
```

```python
print(f"power iteration converged in {k} steps")

# Dense eigen-solver for comparison
lam, V = np.linalg.eig(G)
i1 = np.argmax(np.abs(lam))
v = np.real(V[:, i1])
v = v / v.sum()
lam2 = np.sort(np.abs(lam))[-2]
print(f"lambda_1 = {np.real(lam[i1]):.6f}, |lambda_2| = {lam2:.4f}"
      f" (bound alpha = {alpha})")
print("max |r_power - r_eig| = %.2e" % np.abs(r_new - v).max())
rank = np.argsort(-v)
print("rank district PageRank  in-flow share")
inflow = W.sum(axis=0) / W.sum()
for pos, i in enumerate(rank[:5], start=1):
    print(f"{pos:4d}   {names[i]}    {v[i]:.4f}     {inflow[i]:.4f}")

# eig versus eigh on a symmetric 600 x 600 matrix
B = rng.standard_normal((600, 600))
S = (B + B.T) / 2
t0 = time.perf_counter(); l_g, V_g = np.linalg.eig(S); t1 = time.perf_counter()
l_h, V_h = np.linalg.eigh(S); t2 = time.perf_counter()
print(f"eig : {t1 - t0:.3f}s, dtype {l_g.dtype}, "
      f"orthogonality err {np.abs(V_g.T @ V_g - np.eye(600)).max():.1e}")
print(f"eigh: {t2 - t1:.3f}s, dtype {l_h.dtype}, "
      f"orthogonality err {np.abs(V_h.T @ V_h - np.eye(600)).max():.1e}")
print("same spectrum:", np.allclose(np.sort(l_g.real), l_h))
```

Output

```
power iteration converged in 25 steps
lambda_1 = 1.000000, |lambda_2| = 0.3107 (bound alpha = 0.85)
max |r_power - r_eig| = 2.45e-14
rank district PageRank  in-flow share
   1   D00    0.3109     0.4828
   2   D05    0.1547     0.0431
   3   D04    0.1142     0.1006
   4   D03    0.1042     0.1116
   5   D02    0.1004     0.0799
eig : 0.553s, dtype float64, orthogonality err 5.9e-12
eigh: 0.120s, dtype float64, orthogonality err 3.2e-15
same spectrum: True
```

The hub district D00 ranks first, but the ranking is not simply in-flow: D05 receives only 4% of all flows yet ranks second, because a large share of the *hub's* exports go to it. Importance is recursive. The final block of output compares `eig` and `eigh` on a symmetric 600×600 matrix: same spectrum, but `eigh` is several times faster and its eigenvectors are orthogonal to machine precision (timings depend on the machine).

Policy Lens: Network centrality for trade and financial policy

Eigenvector centrality and PageRank are used to identify systemically important nodes: banks in interbank-exposure networks (where a central regulator asks which failure would spread furthest), countries or sectors in input–output and trade networks, and districts in

regional-development planning (which market towns most improve access when a road is upgraded). Two cautions apply. First, centrality depends on the normalisation of flows (shares versus values) and on α; report robustness to both. Second, the leading eigenvector summarises *one* notion of importance; for shock propagation the whole matrix $(\mathbf{I} - \alpha M)^{-1}$ (a Leontief-type inverse) is usually more informative.

Historical Note: From von Mises to Google

The power method is usually credited to Richard von Mises and Hilda Pollaczek-Geiringer, who described it in 1929. Inverse iteration was introduced by Helmut Wielandt in 1944. The QR algorithm was discovered independently by John G. F. Francis (published 1961–62) and Vera Kublanovskaya (1961), and is listed among the ten most influential algorithms of the twentieth century. In 1998 Sergey Brin and Lawrence Page, then graduate students at Stanford, described the PageRank vector of the web graph, computed by the power method; the eigenvector ideas were older – Edmund Landau had proposed a similar ranking for chess tournaments in 1895, and sociologists and economists had used eigenvector centrality and input–output multipliers for decades.

Exercises

3.3.1 Theory Show that if $\mathbf{x}_0$ has $c_1 = 0$ in the eigen-expansion, power iteration in exact arithmetic never finds $\mathbf{v}_1$. Why does this almost never matter in floating point?

3.3.2 Theory Prove that $\lambda \in \sigma(A)$ iff $(\lambda - \mu)^{-1} \in \sigma((A - \mu\mathbf{I})^{-1})$ with the same eigenvectors, and derive the convergence ratio of inverse iteration.

3.3.3 Theory For the Google matrix (3.8), prove that every eigenvalue $\lambda \neq 1$ of G satisfies $|\lambda| \leq \alpha$. (Hint: use $\mathbf{1}^\top G = \mathbf{1}^\top$ to show that a right eigenvector $\mathbf{x}$ for $\lambda \neq 1$ satisfies $\mathbf{1}^\top \mathbf{x} = 0$, hence $G\mathbf{x} = \alpha M \mathbf{x}$.)

3.3.4 Python Implement the basic (unshifted) QR algorithm in ten lines and apply it to a random symmetric 6×6 matrix. Plot the size of the largest subdiagonal entry against iteration and relate the decay of each subdiagonal entry to ratios $|\lambda_{i+1}/\lambda_i|$.

3.3.5 Python In `pagerank.py`, recompute the ranking for $\alpha \in \{0.5, 0.7, 0.85, 0.95, 0.99\}$. Record the number of power iterations and Kendall's τ between rankings. Which districts are sensitive to α, and why?

3.3.6 ★ Graduate (Deflation.) After computing $(\lambda_1, \mathbf{q}_1)$ of a symmetric A, show that $A - \lambda_1 \mathbf{q}_1 \mathbf{q}_1^\top$ has eigenvalues $0, \lambda_2, \ldots, \lambda_n$ with the same eigenvectors. Use this to compute the top three eigenpairs by repeated power iteration, and discuss how errors in $\mathbf{q}_1$ propagate to later eigenpairs.

3.3.7 Policy A ministry wants to choose two districts for new market infrastructure to maximise access across the trade network. Compare choosing the top two PageRank districts with choosing the top two by total in-flow in the simulated network. Construct a small example in which the two rules disagree and argue which is more defensible.

3.4 The Singular Value Decomposition

Eigen-decompositions apply only to square matrices, and are well behaved only for symmetric ones. Data matrices are rectangular – n observations by p variables – and we would like a decomposition that exists for *every* matrix and uses orthogonal bases. That is the singular value decomposition, which Strang (2019) calls the climax of linear algebra.

3.4.1 Existence and geometry

Theorem 3.15 (Singular value decomposition). *Every $\mathbf{X} \in \mathbb{R}^{n\times p}$ can be written as*

$$\mathbf{X} = U\Sigma V^\top = \sum_{j=1}^{r} \sigma_j \, \mathbf{u}_j \mathbf{v}_j^\top, \tag{3.9}$$

where $U \in \mathbb{R}^{n\times n}$ and $V \in \mathbb{R}^{p\times p}$ are orthogonal, $\Sigma \in \mathbb{R}^{n\times p}$ is zero except for the diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(n,p)} \geq 0$, and $r = \operatorname{rank}\mathbf{X}$ is the number of nonzero singular values. *The columns $\mathbf{u}_j$ and $\mathbf{v}_j$ are the left and right* singular vectors.

Proof. The matrix $\mathbf{X}^\top\mathbf{X} \in \mathbb{R}^{p\times p}$ is symmetric PSD, so by the spectral theorem $\mathbf{X}^\top\mathbf{X} = V\mathbf{\Lambda}V^\top$ with $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$. Let r be the number of positive eigenvalues; then $r = \operatorname{rank}(\mathbf{X}^\top\mathbf{X}) = \operatorname{rank}\mathbf{X}$. Define $\sigma_j = \sqrt{\lambda_j}$ and, for $j \leq r$, $\mathbf{u}_j = \mathbf{X}\mathbf{v}_j/\sigma_j$. These are orthonormal:

$$\mathbf{u}_i^\top \mathbf{u}_j = \frac{\mathbf{v}_i^\top \mathbf{X}^\top \mathbf{X}\mathbf{v}_j}{\sigma_i\sigma_j} = \frac{\lambda_j \mathbf{v}_i^\top \mathbf{v}_j}{\sigma_i\sigma_j} = \delta_{ij}.$$

Complete $\{\mathbf{u}_1, \ldots, \mathbf{u}_r\}$ to an orthonormal basis of $\mathbb{R}^n$. For $j \leq r$, $\mathbf{X}\mathbf{v}_j = \sigma_j\mathbf{u}_j$ by construction; for $j > r$, $\|\mathbf{X}\mathbf{v}_j\|^2 = \lambda_j = 0$, so $\mathbf{X}\mathbf{v}_j = \mathbf{0}$. Hence $\mathbf{X}V = U\Sigma$, i.e. $\mathbf{X} = U\Sigma V^\top$. □

The proof also gives the relation with eigen-decompositions:

$$\mathbf{X}^\top\mathbf{X} = V(\Sigma^\top\Sigma)V^\top, \qquad \mathbf{X}\mathbf{X}^\top = U(\Sigma\Sigma^\top)U^\top, \tag{3.10}$$

so the right singular vectors are eigenvectors of $\mathbf{X}^\top\mathbf{X}$, the left singular vectors are eigenvectors of $\mathbf{X}\mathbf{X}^\top$, and the nonzero eigenvalues of both are σ_j^2. For a *symmetric PSD* matrix the SVD and the spectral decomposition coincide; for a symmetric indefinite matrix, $\sigma_j = |\lambda_j|$.

Geometry. Read (3.9) from right to left: any linear map is a rotation (or reflection) $V^\top$, followed by an axis-aligned stretch Σ, followed by another rotation U. The unit sphere in $\mathbb{R}^p$ is mapped to an ellipsoid in $\mathbb{R}^n$ whose semi-axes are $\sigma_j\mathbf{u}_j$. Hence

$$\|\mathbf{X}\|_2 = \max_{\|\mathbf{v}\|=1} \|\mathbf{X}\mathbf{v}\| = \sigma_1, \qquad \min_{\|\mathbf{v}\|=1} \|\mathbf{X}\mathbf{v}\| = \sigma_p \ (n \geq p), \qquad \|\mathbf{X}\|_F^2 = \sum_j \sigma_j^2.$$

The four fundamental subspaces. The SVD supplies orthonormal bases for all four subspaces of Chapter 2 at once: $\operatorname{col}(\mathbf{X}) = \operatorname{span}\{\mathbf{u}_1, \ldots, \mathbf{u}_r\}$, $\operatorname{null}(\mathbf{X}^\top) = \operatorname{span}\{\mathbf{u}_{r+1}, \ldots, \mathbf{u}_n\}$, $\operatorname{col}(\mathbf{X}^\top) = \operatorname{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_r\}$ and $\operatorname{null}(\mathbf{X}) = \operatorname{span}\{\mathbf{v}_{r+1}, \ldots, \mathbf{v}_p\}$. It also gives the *pseudo-inverse* $\mathbf{X}^+ = V\Sigma^+U^\top$ with $\Sigma^+ = \operatorname{diag}(1/\sigma_1, \ldots, 1/\sigma_r, 0, \ldots)$, and the minimum-norm least-squares solution $\hat{\boldsymbol{\beta}} = \mathbf{X}^+\mathbf{y}$ even when $\mathbf{X}$ is rank-deficient.

★ Key Concept: Thin SVD in practice

For $n \gg p$ use the thin (economy) SVD $\mathbf{X} = U_p\Sigma_pV^\top$ with $U_p \in \mathbb{R}^{n\times p}$: `np.linalg.svd(X, full_matrices=False)`, which returns U, the vector of singular values *in descending order*, and $V^\top$ (not V!). Rank, norm, condition number, pseudo-inverse, projections and PCA all come from this one factorisation.

Listing 3.6: The SVD, its relation to eigen-decompositions, and Eckart–Young `ch03/svd_basics.py`

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# The SVD, its link to eig(X'X) and eig(XX'), and the Eckart-Young theorem.
```

```python
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
np.set_printoptions(precision=4, suppress=True)
n, p = 8, 4
X = rng.standard_normal((n, p)) @ np.diag([3.0, 2.0, 1.0, 0.2])

U, s, Vt = np.linalg.svd(X, full_matrices=False)    # thin SVD
print("shapes U, s, Vt     :", U.shape, s.shape, Vt.shape)
print("singular values     :", s)
print("||X - U S V'||      : %.2e" % np.linalg.norm(X - U @ np.diag(s) @ Vt))
print("U'U = I, V'V = I    :", np.allclose(U.T @ U, np.eye(p)),
      np.allclose(Vt @ Vt.T, np.eye(p)))

# Relation to eigen-decompositions
ev_XtX = np.linalg.eigvalsh(X.T @ X)[::-1]
ev_XXt = np.linalg.eigvalsh(X @ X.T)[::-1]
print("sqrt eig(X'X)       :", np.sqrt(ev_XtX))
print("eig(XX') (top 5)    :", ev_XXt[:5])
print("s^2                 :", s ** 2)

# Rank and condition number from the singular values
print("rank, cond(X)       :", np.linalg.matrix_rank(X), round(s[0] / s[-1], 3),
      round(np.linalg.cond(X), 3))

# Eckart-Young-Mirsky: best rank-k approximation error
print(" k  ||X-Xk||_2  s_(k+1)   ||X-Xk||_F  sqrt(sum s_j^2, j>k)")
for k in range(1, p):
    Xk = U[:, :k] @ np.diag(s[:k]) @ Vt[:k]
    e2 = np.linalg.norm(X - Xk, 2)
    eF = np.linalg.norm(X - Xk, "fro")
    print(f"{k:2d}  {e2:9.4f}  {s[k]:8.4f}   {eF:9.4f}   "
          f"{np.sqrt((s[k:] ** 2).sum()):9.4f}")

# A random rank-2 competitor never does better
best = np.linalg.norm(X - U[:, :2] @ np.diag(s[:2]) @ Vt[:2], "fro")
errs = []
for _ in range(2000):
    A = rng.standard_normal((n, 2)) @ rng.standard_normal((2, p))
    # optimal scaling of the competitor's column space (least squares)
    Qa, _ = np.linalg.qr(A)
    errs.append(np.linalg.norm(X - Qa @ (Qa.T @ X), "fro"))
print("best rank-2 error %.4f; best of 2000 random subspaces %.4f"
      % (best, min(errs)))
```

Output

```
shapes U, s, Vt     : (8, 4) (4,) (4, 4)
singular values     : [7.424  4.0029 2.1981 0.2601]
||X - U S V'||      : 5.15e-15
U'U = I, V'V = I    : True True
sqrt eig(X'X)       : [7.424  4.0029 2.1981 0.2601]
eig(XX') (top 5)    : [55.1163 16.0233  4.8317  0.0677  0.    ]
s^2                 : [55.1163 16.0233  4.8317  0.0677]
rank, cond(X)       : 4 28.543 28.543
```

```
  k  ||X-Xk||_2  s_(k+1)   ||X-Xk||_F  sqrt(sum s_j^2, j>k)
  1     4.0029    4.0029      4.5741      4.5741
  2     2.1981    2.1981      2.2135      2.2135
  3     0.2601    0.2601      0.2601      0.2601
best rank-2 error 2.2135; best of 2000 random subspaces 3.5269
```

3.4.2 Low-rank approximation: the Eckart–Young–Mirsky theorem

Truncating the sum (3.9) after k terms gives $\mathbf{X}_k = \sum_{j=1}^{k} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top = U_k \Sigma_k V_k^\top$, a rank-$k$ matrix. It is the best one.

Theorem 3.16 (Eckart–Young–Mirsky). *For $k < r$ and any matrix B with* rank $B \le k$,

$$\|\mathbf{X} - B\|_2 \ge \|\mathbf{X} - \mathbf{X}_k\|_2 = \sigma_{k+1}, \qquad \|\mathbf{X} - B\|_F \ge \|\mathbf{X} - \mathbf{X}_k\|_F = \Big(\sum_{j>k} \sigma_j^2 \Big)^{1/2}.$$

The same optimality holds in every unitarily invariant norm (Mirsky, 1960).

Proof for the spectral norm. The equalities follow from $\mathbf{X} - \mathbf{X}_k = \sum_{j>k} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top$. Now let rank $B \le k$. Then $\dim \operatorname{null}(B) \ge p - k$, while $W = \operatorname{span}\{\mathbf{v}_1, \dots, \mathbf{v}_{k+1}\}$ has dimension $k+1$, so the two subspaces intersect nontrivially: pick a unit vector $\mathbf{w} \in \operatorname{null}(B) \cap W$, $\mathbf{w} = \sum_{j \le k+1} c_j \mathbf{v}_j$ with $\sum c_j^2 = 1$. Then

$$\|\mathbf{X} - B\|_2^2 \ge \|(\mathbf{X} - B)\mathbf{w}\|^2 = \|\mathbf{X}\mathbf{w}\|^2 = \sum_{j=1}^{k+1} \sigma_j^2 c_j^2 \ge \sigma_{k+1}^2.$$

The Frobenius case follows by applying Weyl's inequality for singular values, $\sigma_{i+j-1}(\mathbf{X}) \le \sigma_i(\mathbf{X} - B) + \sigma_j(B)$, with $j = k+1$ (so $\sigma_{k+1}(B) = 0$), giving $\sigma_i(\mathbf{X} - B) \ge \sigma_{i+k}(\mathbf{X})$ for every i, and summing squares. □

The output of Listing 3.6 checks both identities and shows that the best of 2000 random rank-2 subspaces is far from the optimum. The theorem has a statistical reading: $\mathbf{X}_k$ is the least-squares fit of the data by k "latent factors", and the residual energy is the tail sum of squared singular values. This is precisely PCA.

3.4.3 Image compression and truncated SVD

An $m \times n$ grey-scale image is a matrix. Storing $\mathbf{X}_k$ requires $k(m + n + 1)$ numbers instead of mn. Listing 3.7 builds an 80×80 image by tiling 100 of the 8×8 handwritten-digit images bundled with scikit-learn (`load_digits`) and compresses it.

Listing 3.7: Low-rank approximation of a digit mosaic ch03/svd_digits.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Low-rank image compression of a mosaic built from sklearn's digits data.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
imgs = digits.images[:100]                # 100 images of 8 x 8 pixels
# Tile them into an 80 x 80 mosaic (10 x 10 grid of digits)
mosaic = imgs.reshape(10, 10, 8, 8).transpose(0, 2, 1, 3).reshape(80, 80)
```

```
U, s, Vt = np.linalg.svd(mosaic, full_matrices=False)

m, n = mosaic.shape
print(f"image {m} x {n}, numerical rank {np.linalg.matrix_rank(mosaic)}")
print(" k   storage   ratio  rel.Frob.err  energy kept")
energy = np.cumsum(s ** 2) / np.sum(s ** 2)
ranks = (1, 5, 10, 20, 30)
approx = {}
for k in ranks:
    Ak = (U[:, :k] * s[:k]) @ Vt[:k]
    approx[k] = Ak
    store = k * (m + n + 1)
    rel = np.linalg.norm(mosaic - Ak) / np.linalg.norm(mosaic)
    print(f"{k:3d}  {store:7d}  {store / (m * n):6.3f}   {rel:8.4f}"
          f"     {energy[k - 1]:.4f}")
k90 = np.searchsorted(energy, 0.90) + 1
print(f"rank needed for 90% of squared Frobenius norm: {k90}")

fig, axes = plt.subplots(1, 4, figsize=(8.5, 2.6))
for ax, k in zip(axes, (5, 10, 20)):
    ax.imshow(approx[k], cmap="gray_r")
    ax.set_title(f"rank {k}")
for ax in axes[:3]:
    ax.set_xticks([]); ax.set_yticks([])
axes[3].imshow(mosaic, cmap="gray_r")
axes[3].set_title(f"original (rank {np.linalg.matrix_rank(mosaic)})")
axes[3].set_xticks([]); axes[3].set_yticks([])
plt.savefig("ch03_svd_digits.pdf", bbox_inches="tight")

fig, ax = plt.subplots(figsize=(6, 3.4))
ax.semilogy(np.arange(1, len(s) + 1), s, "o-", ms=3)
ax.set_xlabel("index j")
ax.set_ylabel("singular value s_j (log scale)")
ax.set_title("Singular value spectrum of the digit mosaic")
plt.savefig("ch03_svd_spectrum.pdf", bbox_inches="tight")
```

Output

```
image 80 x 80, numerical rank 66
 k   storage   ratio  rel.Frob.err  energy kept
  1       161   0.025     0.5509     0.6965
  5       805   0.126     0.4461     0.8010
 10      1610   0.252     0.3545     0.8743
 20      3220   0.503     0.2154     0.9536
 30      4830   0.755     0.1255     0.9843
rank needed for 90% of squared Frobenius norm: 13
```

Two observations matter beyond images. First, *the decay of the singular values determines compressibility* (Fig. 3.5). Smooth images, term–document matrices and panels of macroeconomic series are approximately low rank; a mosaic of distinct digits much less so. Second, the rank-1 term already captures most of the squared Frobenius norm, because all pixels are nonnegative and the first singular pair captures the *mean* pattern; this is why PCA centres the data first.

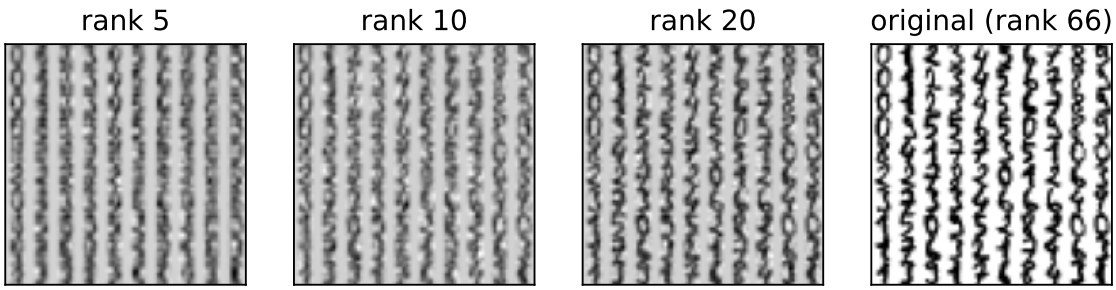


Figure 3.4: Rank-k approximations $\mathbf{X}_k$ of an 80×80 mosaic of handwritten digits from scikit-learn's `load_digits`. Digits become legible around $k = 20$.

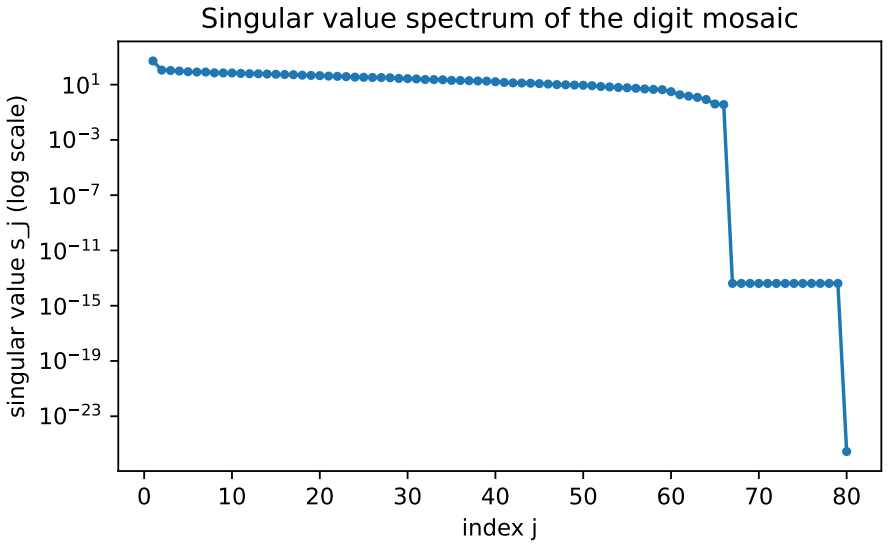


Figure 3.5: Singular values of the digit mosaic on a log scale. The slow decay explains why the mosaic, unlike a smooth photograph, compresses only moderately well.

△ Caution: Do not form $\mathbf{X}^\top\mathbf{X}$ to get the SVD

Computing singular values as square roots of the eigenvalues of $\mathbf{X}^\top\mathbf{X}$ is mathematically correct but numerically poor: forming $\mathbf{X}^\top\mathbf{X}$ squares the condition number (Section 3.6), so singular values smaller than about $\sqrt{\varepsilon_{\text{mach}}}\,\sigma_1 \approx 10^{-8}\sigma_1$ are lost entirely. Library SVD routines work on $\mathbf{X}$ directly.

Historical Note: Beltrami, Jordan, Eckart and Young, Golub and Kahan

The SVD was discovered independently by Eugenio Beltrami (1873) and Camille Jordan (1874), both working on bilinear forms; James Joseph Sylvester rediscovered it for real square matrices in 1889, and Erhard Schmidt (1907) proved the infinite-dimensional version for integral operators, together with the optimal low-rank approximation property. The latter was rediscovered for matrices by the psychometricians Carl Eckart and Gale Young in 1936 (Eckart and Young, 1936), and generalised to all unitarily invariant norms by Leon Mirsky in 1960 (Stewart, 1993). The SVD became a practical tool only with the 1965 algorithm of Gene Golub and William Kahan (Golub and Kahan, 1965), which first reduces $\mathbf{X}$ to bidiagonal form by Householder reflections; with the later Golub–Reinsch (1970) refinement it remains the basis of LAPACK's routines.

Graduate Perspective: Randomised SVD and matrix completion

For very large matrices one needs only the top k singular triplets. Randomised algorithms (Halko et al., 2011) multiply $\mathbf{X}$ by a random $p \times (k+q)$ Gaussian matrix Ω, orthonormalise $Y = \mathbf{X}\Omega$ (optionally after a few power iterations $(\mathbf{X}\mathbf{X}^\top)^s\mathbf{X}\Omega$ to sharpen the spectrum), and compute the small SVD of $Q^\top\mathbf{X}$. Their error is close to σ_{k+1} with high probability – a beautiful application of the high-dimensional probability of Vershynin (2018). Scikit-learn's `TruncatedSVD` and `PCA(svd_solver="randomized")` use this. Low-rank structure also underlies *matrix completion* (recommender systems) and the "synthetic control" and interactive fixed-effects estimators of panel econometrics, where an unobserved low-rank matrix of factors times loadings is estimated by nuclear-norm penalisation.

Exercises

3.4.1 Theory Prove that $\|\mathbf{X}\|_2 = \sigma_1$ and $\|\mathbf{X}\|_F^2 = \sum_j \sigma_j^2$. Deduce $\|\mathbf{X}\|_2 \le \|\mathbf{X}\|_F \le \sqrt{r}\,\|\mathbf{X}\|_2$.

3.4.2 Theory Show that $\mathbf{X}^+ = V\Sigma^+U^\top$ satisfies the four Moore–Penrose conditions, that $\mathbf{X}\mathbf{X}^+$ is the orthogonal projector onto $\operatorname{col}(\mathbf{X})$, and that $\hat{\boldsymbol{\beta}} = \mathbf{X}^+\mathbf{y}$ is the least-squares solution of minimum Euclidean norm.

3.4.3 Theory Show that for a square matrix $|\det \mathbf{X}| = \prod_j \sigma_j$, and that the singular values of a symmetric matrix are the absolute values of its eigenvalues. Give a 2×2 non-symmetric example where singular values and absolute eigenvalues differ.

3.4.4 Python Using `svd_digits.py`, plot the relative Frobenius error against storage ratio $k(m+n+1)/(mn)$. Repeat for a smooth synthetic image, e.g. $x_{ij} = \sin(i/10)\cos(j/15) + 0.5\exp(-((i-40)^2+(j-40)^2)/200)$, and explain the difference via the singular value spectra.

3.4.5 Python Implement the randomised SVD of the Graduate Perspective box with $k = 10$ and oversampling $q = 5$ on the digits mosaic. Compare its top ten singular values and run time with `np.linalg.svd`, with and without one power iteration.

3.4.6 ★ Graduate Prove the Frobenius-norm part of Theorem 3.16 without Weyl's inequality, by showing that for any orthonormal $W \in \mathbb{R}^{p\times k}$, $\|\mathbf{X} - \mathbf{X}WW^\top\|_F^2 = \|\mathbf{X}\|_F^2 - \operatorname{tr}(W^\top\mathbf{X}^\top\mathbf{X}W)$ and maximising the trace with Ky Fan's theorem.

3.4.7 Policy A statistics office stores a large district-by-month price matrix and proposes releasing only a rank-5 approximation to protect respondent confidentiality and save space. Discuss what is preserved and what is lost, whether a low-rank release protects confidentiality, and how you would choose k.

3.5 Principal Component Analysis

3.5.1 Variance maximisation

Let $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^p$ be observations with sample mean $\bar{\mathbf{x}}$, and let $\mathbf{X}_c \in \mathbb{R}^{n\times p}$ be the *centred* data matrix with rows $(\mathbf{x}_i - \bar{\mathbf{x}})^\top$. The sample covariance is $S = \mathbf{X}_c^\top\mathbf{X}_c/(n-1)$. PCA seeks the linear combination $z = \mathbf{w}^\top\mathbf{x}$ with the largest variance, subject to a normalisation (otherwise the variance is unbounded):

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \widehat{\operatorname{Var}}(\mathbf{w}^\top\mathbf{x}) = \arg\max_{\|\mathbf{w}\|=1} \mathbf{w}^\top S\mathbf{w}. \tag{3.11}$$

Introducing a Lagrange multiplier (Chapter 9), $\mathcal{L} = \mathbf{w}^\top S\mathbf{w} - \lambda(\mathbf{w}^\top\mathbf{w} - 1)$, the first-order condition $2S\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0}$ is the eigen-equation $S\mathbf{w} = \lambda\mathbf{w}$, and at any such point the objective

equals $\mathbf{w}^\top S\mathbf{w} = \lambda$. So the maximiser is the eigenvector of the largest eigenvalue – exactly the Rayleigh-quotient result of Theorem 3.10.

Theorem 3.17 (Principal components). *Let $S = Q\Lambda Q^\top$ with $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$. For $k = 1, \ldots, p$ the direction maximising $\mathbf{w}^\top S\mathbf{w}$ over unit vectors orthogonal to $\mathbf{w}_1, \ldots, \mathbf{w}_{k-1}$ is $\mathbf{w}_k = \mathbf{q}_k$, with maximal variance λ_k. The* principal components *$z_{ik} = \mathbf{q}_k^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ are uncorrelated in the sample, with variances λ_k. The total variance is preserved: $\sum_k \lambda_k = \operatorname{tr} S = \sum_j s_{jj}$.*

Equivalently – and this is Pearson's original 1901 formulation – the first k principal directions span the k-dimensional affine subspace that minimises the sum of squared *orthogonal* distances from the data points. By Eckart–Young that subspace is spanned by the top k right singular vectors of $\mathbf{X}_c$: maximising retained variance and minimising reconstruction error are the same problem.

3.5.2 PCA via the SVD and explained variance

Write the thin SVD $\mathbf{X}_c = U\Sigma V^\top$. Then $S = \mathbf{X}_c^\top \mathbf{X}_c/(n-1) = V\frac{\Sigma^2}{n-1}V^\top$, so

$$\text{loadings} = V, \qquad \lambda_k = \frac{\sigma_k^2}{n-1}, \qquad \text{scores } Z = \mathbf{X}_c V = U\Sigma. \tag{3.12}$$

The *explained variance ratio* of component k is $\lambda_k / \sum_j \lambda_j = \sigma_k^2 / \sum_j \sigma_j^2$. A *scree plot* shows these ratios in decreasing order; one looks for an "elbow" after which components explain little. The SVD route is preferred over `eigh(S)` for the numerical reason of the previous caution box, and because it delivers the scores directly.

⋆ Key Concept: PCA in four lines

(1) Centre (and usually standardise) the columns. (2) Compute the thin SVD $\mathbf{X}_c = U\Sigma V^\top$. (3) Loadings are the columns of V, scores are $U\Sigma$, variances are $\sigma_k^2/(n-1)$. (4) Choose k by the scree plot, a cumulative-variance target, cross-validation of reconstruction error, or (for factor models) an information criterion such as Bai and Ng (2002).

Loadings and biplots. The loading v_{jk} tells how much variable j contributes to component k; with standardised variables, $\sqrt{\lambda_k} v_{jk}$ is the correlation between variable j and component k. A *biplot* overlays the scores of observations (points) and the loadings of variables (arrows) on the first two components, so that clusters of observations can be read off against the variables that separate them (Fig. 3.7).

To standardise or not. PCA on the covariance matrix is not scale invariant: multiplying one variable by 1000 (changing cedis to pesewas, say) can make it dominate the first component. PCA on the *correlation* matrix – equivalently on standardised data – treats all variables equally. Use covariance PCA when all variables share a meaningful common unit (pixel intensities, yields on bonds of different maturities); use correlation PCA when units are arbitrary (macroeconomic indicators, survey items).

△ Caution: PCA pitfalls

(i) *Sign indeterminacy*: $\mathbf{v}_k$ and $-\mathbf{v}_k$ are equally valid; fix a convention (e.g. make loadings sum to a positive number) before interpreting or comparing across samples. (ii) *Rotation within equal eigenvalues*: if $\lambda_k \approx \lambda_{k+1}$ the individual components are poorly determined, though their span is stable. (iii) *Leakage*: in forecasting or prediction, PCA must be fitted on the training sample only – or re-estimated in real time, as in Listing 3.8. (iv) *Variance is not*

relevance: the components with the largest variance need not be the ones most correlated with an outcome of interest.

3.5.3 PCA of a simulated macroeconomic panel

Listing 3.8 simulates $T = 240$ months of $N = 40$ indicators driven by two latent AR(1) factors – a "real activity" factor loading on the first 25 series and a "price pressure" factor on the remaining 15, with some mixed series – plus idiosyncratic noise, and with series measured in very different units. The data are simulated and do not describe any real economy.

Listing 3.8: PCA of a simulated macro panel and a diffusion-index forecast ch03/pca_macro.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# PCA of a simulated macro panel; scree plot; diffusion-index inflation forecast.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
T, N, r = 240, 40, 2                    # 20 years of monthly data, 40 series
# Two latent AR(1) factors: "real activity" and "price pressure"
F = np.zeros((T, r))
for t in range(1, T):
    F[t] = np.array([0.8, 0.9]) * F[t - 1] + rng.standard_normal(r)
Lam = np.zeros((N, r))
Lam[:25, 0] = rng.uniform(0.5, 1.5, 25)            # activity block
Lam[25:, 1] = rng.uniform(0.5, 1.5, 15)            # price block
Lam[20:30] += rng.uniform(-0.4, 0.4, (10, r))      # some mixed series
scale = rng.uniform(0.5, 20.0, N)                  # different units
X = (F @ Lam.T + rng.standard_normal((T, N)) * 1.2) * scale

# PCA via the SVD of the standardised panel
Z = (X - X.mean(0)) / X.std(0, ddof=1)
U, s, Vt = np.linalg.svd(Z, full_matrices=False)
evr = s ** 2 / np.sum(s ** 2)
print("explained variance ratio (first 6):", np.round(evr[:6], 3))
print("cumulative (first 3)              :", np.round(np.cumsum(evr)[:3], 3))
Fhat = U[:, :r] * s[:r] / np.sqrt(T - 1)            # principal component scores
for j in range(r):
    beta, *_ = np.linalg.lstsq(np.c_[np.ones(T), Fhat], F[:, j], rcond=None)
    res = F[:, j] - np.c_[np.ones(T), Fhat] @ beta
    r2 = 1 - res.var() / F[:, j].var()
    print(f"R^2 of true factor {j + 1} on 2 PCs: {r2:.3f}")

# Without standardisation the high-variance series dominate PC1
Xc = X - X.mean(0)
_, s_raw, Vt_raw = np.linalg.svd(Xc, full_matrices=False)
top = np.argsort(-np.abs(Vt_raw[0]))[:3]
print("unstandardised PC1 top series:", top, "scales:", np.round(scale[top], 1))
print("unstandardised EVR(PC1): %.3f" % (s_raw[0] ** 2 / np.sum(s_raw ** 2)))

# Diffusion-index forecast of an inflation series (series 35), h = 1
y = Z[:, 35]
start = 120
err_ar, err_di = [], []
for t in range(start, T - 1):
```

```
    U_t, s_t, _ = np.linalg.svd(Z[:t + 1], full_matrices=False)
    f_t = U_t[:, :r] * s_t[:r]                    # factors estimated in real time
    Xar = np.c_[np.ones(t), y[:t]]
    Xdi = np.c_[Xar, f_t[:t]]
    b_ar, *_ = np.linalg.lstsq(Xar, y[1:t + 1], rcond=None)
    b_di, *_ = np.linalg.lstsq(Xdi, y[1:t + 1], rcond=None)
    err_ar.append(y[t + 1] - np.r_[1, y[t]] @ b_ar)
    err_di.append(y[t + 1] - np.r_[1, y[t], f_t[t]] @ b_di)
rmse_ar = np.sqrt(np.mean(np.square(err_ar)))
rmse_di = np.sqrt(np.mean(np.square(err_di)))
print(f"out-of-sample RMSE: AR(1) {rmse_ar:.3f}, AR + 2 factors {rmse_di:.3f}"
      f", ratio {rmse_di / rmse_ar:.3f}")

fig, ax = plt.subplots(figsize=(6, 3.6))
k = np.arange(1, 16)
ax.bar(k, evr[:15], color="C0", alpha=0.7, label="share of variance")
ax.plot(k, np.cumsum(evr)[:15], "o-", color="C3", label="cumulative")
ax.set_xlabel("principal component")
ax.set_ylabel("explained variance ratio")
ax.set_title("Scree plot: simulated panel of 40 macro indicators")
ax.legend(frameon=False)
plt.savefig("ch03_scree_macro.pdf", bbox_inches="tight")
```

Output

```
explained variance ratio (first 6): [0.527 0.192 0.019 0.018 0.016 0.015]
cumulative (first 3)              : [0.527 0.719 0.739]
R^2 of true factor 1 on 2 PCs: 0.985
R^2 of true factor 2 on 2 PCs: 0.979
unstandardised PC1 top series: [ 5 19 18] scales: [19.3 16.8 16.1]
unstandardised EVR(PC1): 0.621
out-of-sample RMSE: AR(1) 0.783, AR + 2 factors 0.666, ratio 0.851
```

The scree plot (Fig. 3.6) has a sharp elbow at two components, which together explain about 72% of the variance, and the two estimated components span the true factors almost exactly ($R^2 \approx 0.98$). Without standardisation, by contrast, the first "component" is mainly the three series with the largest measurement scale. Note that PCA recovers the factor *space*, not the individual factors: any rotation of $(\hat F_1, \hat F_2)$ fits equally well, which is why we regress each true factor on both components.

3.5.4 Factor models and diffusion indexes

The simulation is an instance of the *approximate factor model*

$$x_{it} = \boldsymbol{\lambda}_i^{\top} \mathbf{f}_t + e_{it}, \qquad i = 1, \dots, N,\; t = 1, \dots, T, \tag{3.13}$$

with $r \ll N$ common factors $\mathbf{f}_t$, loadings $\boldsymbol{\lambda}_i$ and weakly correlated idiosyncratic errors. In matrix form $\mathbf{X} = F\Lambda^{\top} + E$, a low-rank matrix plus noise. When the factors are pervasive (each affects a non-negligible fraction of series), the r largest eigenvalues of $\mathbf{X}^{\top}\mathbf{X}/(NT)$ stay bounded away from zero while the rest vanish, and the principal components consistently estimate the factor space as $N, T \to \infty$ (Stock and Watson, 2002a; Bai and Ng, 2002).

Stock and Watson (2002b) called the estimated factors *diffusion indexes* and proposed the two-step forecast

$$\hat y_{t+h} = \hat\alpha + \hat\beta(L)\, y_t + \hat\gamma^{\top} \hat{\mathbf{f}}_t,$$

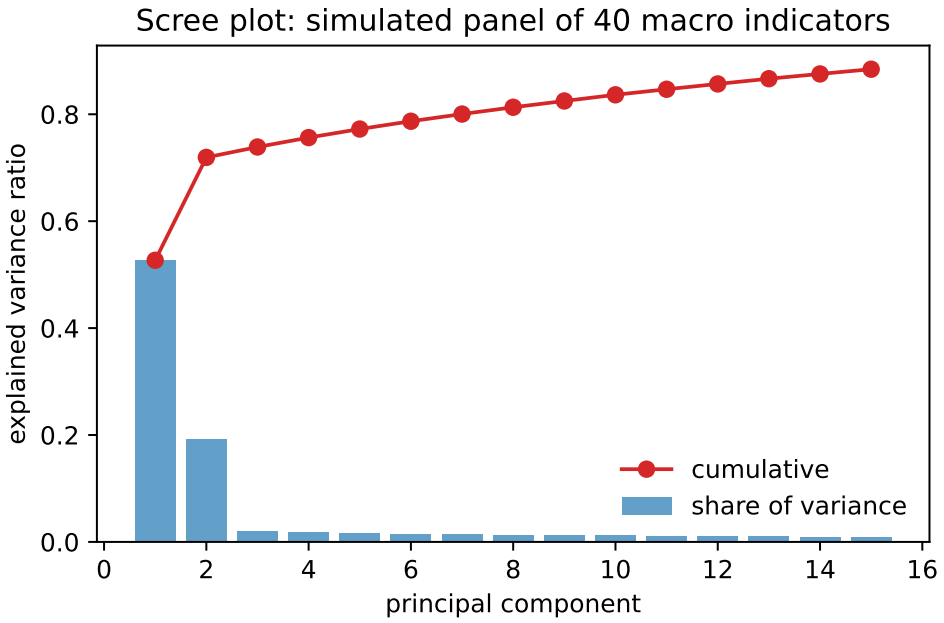


Figure 3.6: Scree plot for the simulated panel of 40 standardised macroeconomic indicators. The elbow after two components correctly reveals the two-factor structure.

an autoregression augmented with a few principal components of a large panel. In our simulated example, re-estimating the factors in real time at every forecast origin reduces the one-step out-of-sample RMSE of an inflation-type series by about 15% relative to a pure AR(1).

Policy Lens: Diffusion indexes for nowcasting inflation

Central banks and finance ministries monitor hundreds of monthly indicators – prices, money, credit, exchange rates, commodity prices, activity surveys – that arrive at different times. Factor models compress this information into a few indexes that can be updated as each data release arrives ("nowcasting"). Goulet Coulombe et al. (2022) find that factors remain among the most useful inputs even for nonlinear machine-learning forecasters: much of the gain from "big data" in macroeconomic forecasting comes from the dimension reduction that PCA provides, and nonlinear methods add most in periods of high uncertainty. For a policy institution the practical lessons are to standardise and transform series to stationarity before extracting factors, to re-estimate factors in real time (no look-ahead), and to compare against simple autoregressive benchmarks in honest pseudo-out-of-sample exercises.

3.5.5 A wealth index from asset ownership

Many household surveys in low- and middle-income countries, notably the Demographic and Health Surveys (DHS), record ownership of durable goods and housing characteristics but not consumption or income. Filmer and Pritchett (2001) proposed using the first principal component of standardised asset indicators as a proxy for long-run household wealth; the DHS wealth index is built in this spirit. If $a_{ij} \in \{0, 1\}$ indicates that household i owns asset j, with sample mean $\bar{a}_j$ and standard deviation s_j, the index is

$$w_i = \sum_{j=1}^{p} \frac{v_{1j}}{s_j} (a_{ij} - \bar{a}_j), \tag{3.14}$$

where $\mathbf{v}_1$ is the leading eigenvector of the correlation matrix of the indicators. The *score weight* v_{1j}/s_j makes owning a rare asset (small s_j) count more.

Listing 3.9 simulates a survey of 3000 households in which ownership of twelve assets depends on an unobserved latent wealth variable and on rural residence through a probit model. Because the data are simulated, we can check the index against the latent wealth that generated them – something that is never possible with real data.

Listing 3.9: A Filmer–Pritchett asset index on a simulated household survey ch03/pca_asset_index.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# A DHS-style asset index (Filmer-Pritchett) on a simulated household survey.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 3000
assets = ["electricity", "piped water", "flush toilet", "television",
          "refrigerator", "radio", "mobile phone", "motorbike", "car",
          "finished floor", "bicycle", "earth floor"]
# Simulated: ownership probability = Phi(a + b * wealth + c * rural)
a = np.array([0.3, -0.4, -0.9, -0.2, -1.0, 0.6, 1.0, -0.8, -1.9, 0.2, -0.3, -0.3])
b = np.array([1.1, 1.0, 1.2, 1.2, 1.3, 0.4, 0.8, 0.5, 1.0, 1.0, -0.1, -1.0])
c = np.array([-.6, -.6, -.5, -.3, -.3, 0.1, -.2, 0.3, -.2, -.4, 0.6, 0.5])
wealth = rng.standard_normal(n)                       # latent, unobserved
rural = (rng.random(n) < 0.5).astype(float)
A = (rng.random((n, 12)) < stats.norm.cdf(a + np.outer(wealth, b)
                                          + np.outer(rural, c))).astype(float)

mu, sd = A.mean(0), A.std(0, ddof=1)
Z = (A - mu) / sd
R = np.corrcoef(A, rowvar=False)
lam, V = np.linalg.eigh(R)
lam, V = lam[::-1], V[:, ::-1]
v1 = V[:, 0] * np.sign(V[:, 0].sum())                 # fix the sign convention
print("eigenvalues of R (first 4):", np.round(lam[:4], 3),
      " PC1 share: %.3f" % (lam[0] / 12))
print(f"{'asset':15s} own%   PC1 loading  score weight (v/sd)")
for j in np.argsort(-v1):
    print(f"{assets[j]:15s} {100 * mu[j]:5.1f}  {v1[j]:8.3f}    "
          f"{v1[j] / sd[j]:8.3f}")

index = Z @ v1                                        # Filmer-Pritchett index
count = A.sum(1) - 2 * A[:, -1]                       # naive count (earth floor -)
for name, x in [("PCA index", index), ("asset count", count)]:
    rho = stats.spearmanr(x, wealth).correlation
    poor_true = wealth <= np.quantile(wealth, 0.4)
    # exactly 40% flagged; ties broken by survey order (households are iid)
    poor_hat = stats.rankdata(x, method="ordinal") <= 0.4 * n
    hit = np.mean(poor_hat[poor_true])
    print(f"{name:12s}: Spearman with latent wealth {rho:.3f}; "
          f"poorest-40% correctly targeted {hit:.3f}")
q = np.digitize(index, np.quantile(index, [0.2, 0.4, 0.6, 0.8]))
print("rural share by index quintile:",
      np.round([rural[q == k].mean() for k in range(5)], 3))

```

```
scores = Z @ V[:, :2]
fig, axes = plt.subplots(1, 2, figsize=(8.5, 3.6))
o = np.argsort(v1)
axes[0].barh(np.array(assets)[o], v1[o], color="C0")
axes[0].set_xlabel("PC1 loading")
axes[0].set_title("Asset index weights")
idx = rng.choice(n, 400, replace=False)
axes[1].scatter(scores[idx, 0], scores[idx, 1], s=5, c=wealth[idx],
                cmap="viridis", alpha=0.6)
for j in range(12):
    axes[1].arrow(0, 0, 4 * V[j, 0], 4 * V[j, 1], color="C3", width=0.01)
    if abs(V[j, 1]) > 0.3 or abs(V[j, 0]) > 0.35:
        axes[1].text(4.3 * V[j, 0], 4.3 * V[j, 1], assets[j], fontsize=7)
axes[1].set_xlabel("PC1 score"); axes[1].set_ylabel("PC2 score")
axes[1].set_title("Biplot (colour = latent wealth)")
plt.tight_layout()
plt.savefig("ch03_asset_index.pdf", bbox_inches="tight")
```

Output

```
eigenvalues of R (first 4): [3.685 1.052 0.957 0.898]  PC1 share: 0.307
asset           own%   PC1 loading  score weight (v/sd)
television       40.8     0.361       0.734
refrigerator     24.2     0.349       0.816
piped water      31.9     0.340       0.729
electricity      49.7     0.338       0.675
flush toilet     23.2     0.335       0.793
finished floor   49.0     0.333       0.665
mobile phone     76.7     0.253       0.598
car               8.2     0.234       0.852
motorbike        29.1     0.196       0.432
radio            73.3     0.136       0.307
bicycle          50.0    -0.062      -0.125
earth floor      49.6    -0.341      -0.681
PCA index   : Spearman with latent wealth 0.882; poorest-40% correctly targeted 0.822
asset count : Spearman with latent wealth 0.863; poorest-40% correctly targeted 0.791
rural share by index quintile: [0.631 0.543 0.486 0.463 0.407]
```

The first component explains about 31% of the variance – typical of real asset indices, and not a sign of failure: binary indicators are noisy. Its loadings have the expected pattern (Fig. 3.7), its Spearman correlation with the latent wealth is 0.88, and it correctly identifies about 82% of the poorest 40% of households, better than a simple asset count. The rural share falls steadily across index quintiles. But notice the second component: it is driven by assets whose ownership depends on rural residence rather than wealth, a reminder that each component is a *statistical* summary whose economic meaning must be argued.

Policy Lens: Asset indices for targeting

Asset indices are used to rank households for targeted transfers, fee waivers and health-insurance exemptions, and to report outcomes by "wealth quintile" in DHS reports and the SDG monitoring framework. Proxy-means tests used by many cash-transfer programmes are cousins of the asset index (they typically use regression weights predicting consumption rather than PCA weights). Good practice includes: (i) checking robustness of quintile assignment to the asset list and to separate urban and rural indices (McKenzie, 2005); (ii)

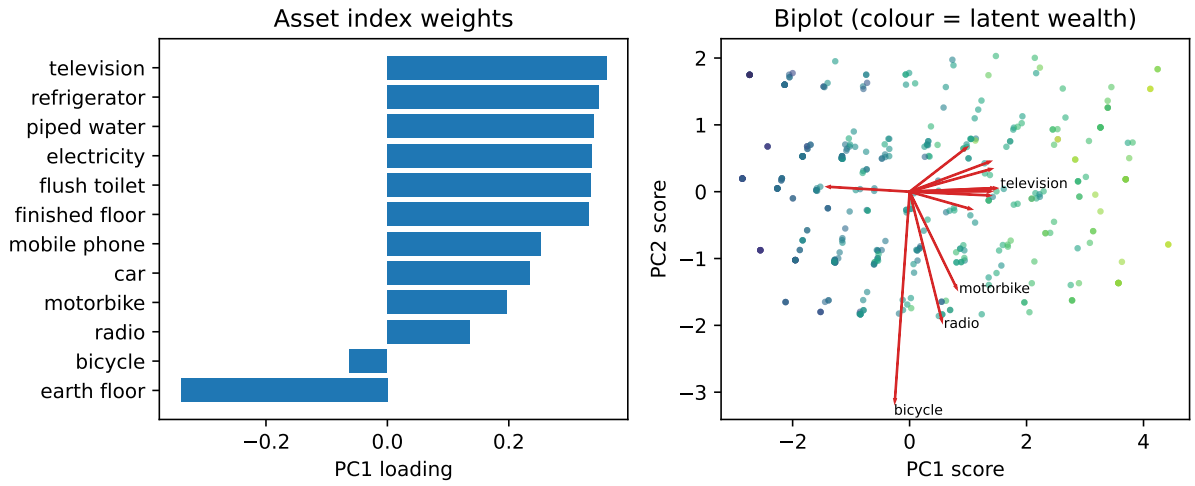


Figure 3.7: Left: first principal component loadings for the simulated asset data; "modern" assets load positively and earth floors negatively. Right: biplot of 400 households on the first two components, coloured by the (normally unobservable) latent wealth. The second component picks up assets more common in rural areas (bicycle, radio, motorbike) rather than wealth.

reporting inclusion and exclusion errors, as in the targeting rate above; (iii) remembering that the index measures a relative ranking of asset-based living standards, not income, and responds slowly to shocks – so it is better suited to identifying the chronically poor than the transiently poor.

Historical Note: Pearson, Hotelling and the naming of components

Karl Pearson introduced the method in 1901 as the problem of finding "lines and planes of closest fit" to points in space – the orthogonal-regression formulation (Pearson, 1901). Harold Hotelling developed it independently in 1933 (Hotelling, 1933) as the analysis of a complex of statistical variables into *principal components*, giving the variance-maximisation derivation, the name, and the power-iteration-like method for computing the components. Factor analysis, with which PCA is often confused, originated earlier still with Charles Spearman's 1904 work on general intelligence. The approximate factor model and its large-N, large-T theory are much more recent, associated with Chamberlain and Rothschild (1983), and Stock and Watson and Bai and Ng in the early 2000s.

Graduate Perspective: When does PCA find the signal? Spiked models

In the spiked covariance model $\boldsymbol{\Sigma} = \mathbf{I} + \sum_k \theta_k \mathbf{q}_k \mathbf{q}_k^\top$ with $p/n \to \gamma$, random matrix theory shows a phase transition: if $\theta_k > \sqrt{\gamma}$ the top sample eigenvalue separates from the Marchenko–Pastur bulk and its eigenvector is correlated with $\mathbf{q}_k$; if $\theta_k < \sqrt{\gamma}$ the sample eigenvector is asymptotically orthogonal to the truth (Johnstone, 2001; Baik et al., 2005; Vershynin, 2018). For factor models this is the distinction between *strong* (pervasive) and *weak* factors. For sparse loadings, sparse PCA adds an ℓ_1 penalty to (3.11), and nonlinear analogues include kernel PCA and autoencoders (Murphy, 2022, ch. 20).

Exercises

3.5.1 Theory Prove Theorem 3.17 by induction using Theorem 3.10, and show that the scores of different components are uncorrelated in the sample.

3.5.2 Theory Show that the first k principal directions minimise $\sum_i \|(\mathbf{x}_i - \bar{\mathbf{x}}) - WW^\top(\mathbf{x}_i - \bar{\mathbf{x}})\|^2$

over orthonormal $W \in \mathbb{R}^{p\times k}$ (Pearson's formulation), and that the minimum equals $(n-1)\sum_{j>k}\lambda_j$.

3.5.3 Theory Let R be the $p \times p$ equicorrelation matrix with off-diagonal entries $\rho > 0$. Find its eigenvalues and eigenvectors, and interpret the first principal component. What fraction of variance does it explain? Relate the answer to a one-factor model.

3.5.4 Python Reproduce `pca_macro.py` with `sklearn.decomposition.PCA` and confirm the explained-variance ratios. Then implement the Bai and Ng (2002) criterion $IC_{p2}(k) = \ln V(k) + k\frac{N+T}{NT}\ln\min(N,T)$, with $V(k)$ the mean squared residual of a k-factor fit, and check that it selects $r = 2$.

3.5.5 Python In `pca_asset_index.py`, construct separate urban and rural indices and a pooled index. How many households change wealth quintile? Compute the targeting rate of each.

3.5.6 ★ Graduate Simulate the factor model with one factor whose loadings are nonzero for only m of $N = 200$ series. Plot the correlation between the estimated and true factor against m for $T = 100$ and $T = 400$. Relate what you see to the spiked-model phase transition.

3.5.7 Policy You are asked to rank 50 districts by "development" using 15 indicators (literacy, electrification, distance to health facility, ...). Explain how you would use PCA, whether you would standardise, how you would handle indicators where higher is worse, and why the weights PCA chooses may not reflect policy priorities. Compare with an equally weighted index.

3.6 Conditioning

A problem is *well conditioned* if small changes in the data produce small changes in the answer, and *ill conditioned* otherwise. Conditioning is a property of the *problem*, not of the algorithm; the corresponding property of an algorithm is *stability* (Higham, 2002). A stable algorithm applied to an ill-conditioned problem can still give an inaccurate answer, and no algorithm can do better than the conditioning allows.

3.6.1 The condition number

Definition 3.18 (Condition number)**.** For invertible $A \in \mathbb{R}^{n\times n}$ and a matrix norm induced by a vector norm, $\kappa(A) = \|A\|\,\left\|A^{-1}\right\|$. In the 2-norm, using the SVD,

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}. \tag{3.15}$$

For rectangular $\mathbf{X}$ of full column rank, $\kappa_2(\mathbf{X}) = \sigma_1/\sigma_p$. By convention $\kappa = \infty$ for singular matrices. NumPy: `np.linalg.cond(A)` (2-norm by default).

Always $\kappa(A) \geq 1$, with equality for orthogonal matrices (in the 2-norm). For symmetric PD matrices $\kappa_2(A) = \lambda_{\max}/\lambda_{\min}$. Geometrically, κ_2 is the eccentricity of the ellipsoid A maps the unit sphere to; and $1/\kappa_2(A) = \min\{\|E\|_2/\|A\|_2 : A + E \text{ singular}\}$ is the relative distance to the nearest singular matrix (an application of Eckart–Young). Determinants say nothing about conditioning: $\det(10^{-3}\mathbf{I}_{10}) = 10^{-30}$ but $\kappa = 1$.

Theorem 3.19 (Perturbation bound for linear systems)**.** *Let A be invertible, $A\mathbf{x} = \mathbf{b}$ with $\mathbf{b} \neq \mathbf{0}$.*

(a) If $A(\mathbf{x}+\delta\mathbf{x}) = \mathbf{b}+\delta\mathbf{b}$, *then*

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \le \kappa(A)\,\frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}, \tag{3.16}$$

and the bound is attained for suitable $\mathbf{b}$ *and* $\delta\mathbf{b}$.

(b) If $(A+\delta A)(\mathbf{x}+\delta\mathbf{x}) = \mathbf{b}$ *and* $\kappa(A)\,\|\delta A\|/\|A\| < 1$, *then*

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \le \frac{\kappa(A)\,\|\delta A\|/\|A\|}{1-\kappa(A)\,\|\delta A\|/\|A\|}.$$

Proof. (a) Subtracting $A\mathbf{x} = \mathbf{b}$ gives $A\,\delta\mathbf{x} = \delta\mathbf{b}$, so $\|\delta\mathbf{x}\| \le \left\|A^{-1}\right\|\|\delta\mathbf{b}\|$. Also $\|\mathbf{b}\| = \|A\mathbf{x}\| \le \|A\|\,\|\mathbf{x}\|$, i.e. $1/\|\mathbf{x}\| \le \|A\|/\|\mathbf{b}\|$. Multiplying the two inequalities gives (3.16). In the 2-norm, equality holds when $\mathbf{b} = \mathbf{u}_1$ (so $\mathbf{x} = \mathbf{v}_1/\sigma_1$ and $\|\mathbf{b}\| = \sigma_1\|\mathbf{x}\|$) and $\delta\mathbf{b} = \epsilon\mathbf{u}_n$ (so $\delta\mathbf{x} = \epsilon\mathbf{v}_n/\sigma_n$ and $\|\delta\mathbf{x}\| = \|\delta\mathbf{b}\|/\sigma_n$).

(b) Expanding, $A\,\delta\mathbf{x} = -\delta A(\mathbf{x}+\delta\mathbf{x})$, so $\|\delta\mathbf{x}\| \le \left\|A^{-1}\right\|\|\delta A\|(\|\mathbf{x}\|+\|\delta\mathbf{x}\|)$. Writing $\eta = \left\|A^{-1}\right\|\|\delta A\| = \kappa(A)\,\|\delta A\|/\|A\| < 1$ and rearranging, $\|\delta\mathbf{x}\|(1-\eta) \le \eta\,\|\mathbf{x}\|$. □

★ Key Concept: The rule of thumb $\kappa \cdot \varepsilon_{\text{mach}}$

A backward-stable solver (Gaussian elimination with partial pivoting, QR, Cholesky) computes the exact solution of a problem perturbed by a relative amount of order $\varepsilon_{\text{mach}} \approx 1.1\times10^{-16}$ (Chapter 2). By Theorem 3.19, the relative error of the computed solution is then of order $\kappa(A)\,\varepsilon_{\text{mach}}$. **You lose about** $\log_{10}\kappa(A)$ **significant digits.** With $\kappa = 10^{10}$ you have about six correct digits; with $\kappa \ge 10^{16}$, possibly none.

Listing 3.10 first constructs the worst case of (3.16) for a nearly singular 2×2 matrix – the amplification equals $\kappa_2(A) = 40002$ exactly – and then turns to a classic ill-conditioned family.

3.6.2 The Hilbert matrix

The Hilbert matrix H_n has entries $h_{ij} = 1/(i+j-1)$. It is symmetric PD – it is the Gram matrix $h_{ij} = \int_0^1 t^{i-1}t^{j-1}\,dt$ of the monomials $1, t, \ldots, t^{n-1}$ on $[0,1]$ – and its condition number grows like $e^{3.5n}$. It is therefore exactly the matrix of normal equations for fitting a polynomial by least squares with the monomial basis on uniformly spread data, which is why such fits are notoriously unstable.

Listing 3.10: Condition numbers, the Hilbert matrix and least squares ch03/conditioning_hilbert.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Condition numbers, the perturbation bound, Hilbert matrices, LS conditioning.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import hilbert

rng = np.random.default_rng(803)
eps = np.finfo(float).eps

# 1. The perturbation bound ||dx||/||x|| <= kappa ||db||/||b|| is attainable
A = np.array([[1.0, 1.0], [1.0, 1.0001]])
U, s, Vt = np.linalg.svd(A)
kappa = s[0] / s[-1]
b = U[:, 0]                       # b along the top left singular vector
db = 1e-6 * U[:, -1]              # perturbation along the bottom one
x = np.linalg.solve(A, b)
```

```
dx = np.linalg.solve(A, b + db) - x
amp = (np.linalg.norm(dx) / np.linalg.norm(x)) / (np.linalg.norm(db)
                                                  / np.linalg.norm(b))
print(f"kappa_2(A) = {kappa:.1f}; observed amplification = {amp:.1f}")

# 2. Hilbert matrices H_ij = 1/(i+j-1): solve H x = H 1
print(" n   cond_2(H)   rel. error   kappa*eps")
ns = np.arange(2, 15)
conds, errs = [], []
for n in ns:
    H = hilbert(n)
    x_true = np.ones(n)
    x_hat = np.linalg.solve(H, H @ x_true)
    c = np.linalg.cond(H)
    e = np.linalg.norm(x_hat - x_true) / np.linalg.norm(x_true)
    conds.append(c); errs.append(e)
    if n % 2 == 0:
        print(f"{n:2d}  {c:10.2e}  {e:10.2e}  {c * eps:10.2e}")

# 3. Least squares: normal equations square the condition number
t = np.linspace(0, 1, 100)
X = np.vander(t, 9, increasing=True)          # polynomial regression, degree 8
beta = rng.standard_normal(9)
y = X @ beta
b_ne = np.linalg.solve(X.T @ X, X.T @ y)
Q, R = np.linalg.qr(X)
b_qr = np.linalg.solve(R, Q.T @ y)
print(f"cond(X) = {np.linalg.cond(X):.2e}, "
      f"cond(X'X) = {np.linalg.cond(X.T @ X):.2e}")
for name, bh in [("normal equations", b_ne), ("QR", b_qr)]:
    print(f"{name:17s}: rel. coef. error "
          f"{np.linalg.norm(bh - beta) / np.linalg.norm(beta):.2e}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.semilogy(ns, conds, "o-", label="cond_2(H_n)")
ax.semilogy(ns, errs, "s-", label="relative error of solve")
ax.semilogy(ns, np.array(conds) * eps, "k:", label="cond * eps_mach")
ax.axhline(1.0, color="gray", lw=0.8)
ax.set_xlabel("dimension n")
ax.set_ylabel("log scale")
ax.set_title("Hilbert matrices: error tracks cond(H) * eps_mach")
ax.legend(frameon=False)
plt.savefig("ch03_hilbert.pdf", bbox_inches="tight")
```

Output

```
kappa_2(A) = 40002.0; observed amplification = 40002.0
 n   cond_2(H)   rel. error   kappa*eps
 2    1.93e+01    5.66e-16    4.28e-15
 4    1.55e+04    3.37e-14    3.44e-12
 6    1.50e+07    4.08e-11    3.32e-09
 8    1.53e+10    6.74e-08    3.39e-06
10    1.60e+13    2.68e-05    3.56e-03
12    1.64e+16    1.78e-01    3.65e+00
14    3.21e+17    1.00e+01    7.14e+01
cond(X) = 6.56e+05, cond(X'X) = 4.31e+11
```

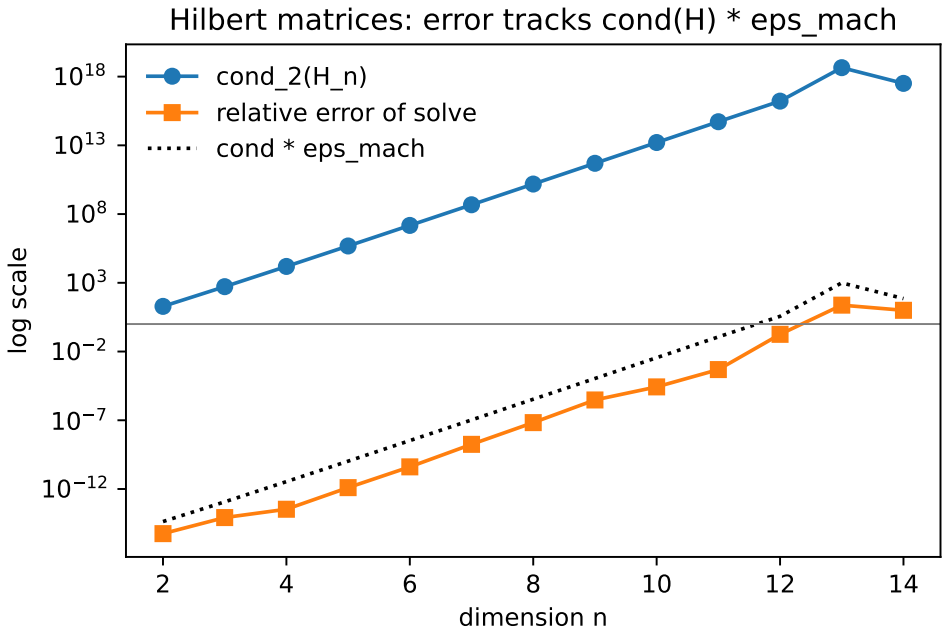


Figure 3.8: Hilbert matrices: the relative error of `np.linalg.solve` grows in step with $\kappa_2(H_n)$ and stays below $\kappa_2(H_n)\varepsilon_{\text{mach}}$. For $n \geq 12$, $\kappa_2 > 1/\varepsilon_{\text{mach}}$ and the computed solution has no correct digits.

```
normal equations : rel. coef. error 3.28e-06
QR               : rel. coef. error 1.23e-11
```

Figure 3.8 confirms the rule of thumb. Note that the *computed* condition number saturates near 10^{17}-10^{18} for $n \geq 13$: once $\kappa \gtrsim 1/\varepsilon_{\text{mach}}$ the smallest singular value is itself buried in rounding error, and even the condition number cannot be computed accurately.

3.6.3 Conditioning of least squares

For $\min_\beta \|\mathbf{y} - \mathbf{X}\beta\|$ with $\mathbf{X}$ of full column rank, the sensitivity of $\hat{\boldsymbol{\beta}}$ to perturbations of $\mathbf{X}$ and $\mathbf{y}$ is governed by $\kappa(\mathbf{X})$ when the residual is small, and by $\kappa(\mathbf{X})^2 \tan\theta$ when it is large, where θ is the angle between $\mathbf{y}$ and $\operatorname{col}(\mathbf{X})$ (Golub and Van Loan, 2013, ch. 5). Whatever the residual, the normal equations $\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\top\mathbf{y}$ have matrix condition number

$$\kappa_2(\mathbf{X}^\top\mathbf{X}) = \kappa_2(\mathbf{X})^2, \tag{3.17}$$

because the singular values of $\mathbf{X}^\top\mathbf{X}$ are σ_j^2. Solving the normal equations therefore *squares* the error amplification even when the problem itself is well conditioned, while QR or the SVD work with $\mathbf{X}$ directly. In the listing, a degree-8 polynomial regression has $\kappa(\mathbf{X}) \approx 6.6 \times 10^5$; the normal equations lose about five more digits than QR (3×10^{-6} versus 10^{-11} relative error). This is why `np.linalg.lstsq` uses the SVD and regression software uses QR.

△ Caution: Never invert $X^\top X$ explicitly

Writing `np.linalg.inv(X.T @ X) @ X.T @ y` is the textbook formula but the worst numerical choice: it squares the condition number *and* adds the error of an explicit inverse. Use `np.linalg.lstsq(X, y)`, or `scipy.linalg.solve` with `assume_a="pos"` if you already have a well-conditioned Gram matrix. Centring and scaling regressors (and using orthogonal

polynomials instead of raw powers) often reduces $\kappa(\mathbf{X})$ by orders of magnitude at no cost to interpretation.

3.6.4 Multicollinearity and variance inflation factors

In the linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\beta}+\boldsymbol{\varepsilon}$ with $\mathrm{Var}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$, $\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$. Statistical imprecision and numerical ill-conditioning are two faces of the same small eigenvalues. Suppose the regressors are standardised so that $\mathbf{X}^\top\mathbf{X}/(n-1) = R$, their correlation matrix, and write $R = Q\boldsymbol{\Lambda}Q^\top$. Then

$$\mathrm{Var}(\hat{\boldsymbol{\beta}}) = \frac{\sigma^2}{n-1}R^{-1} = \frac{\sigma^2}{n-1}\sum_{k=1}^{p}\frac{1}{\lambda_k}\,\mathbf{q}_k\mathbf{q}_k^\top. \tag{3.18}$$

A small eigenvalue λ_k inflates the variance of every coefficient that loads on $\mathbf{q}_k$: the data contain almost no information about the linear combination $\mathbf{q}_k^\top\boldsymbol{\beta}$.

Definition 3.20 (Variance inflation factor)**.** For regressor j, $\mathrm{VIF}_j = 1/(1-R_j^2)$, where R_j^2 is the coefficient of determination from regressing x_j on the other regressors (with an intercept). It is the factor by which $\mathrm{Var}(\hat{\beta}_j)$ exceeds its value under orthogonal regressors.

Proposition 3.21 (VIFs and eigenvalues)**.** *With R the correlation matrix of the regressors,*

$$\mathrm{VIF}_j = (R^{-1})_{jj} = \sum_{k=1}^{p}\frac{q_{jk}^2}{\lambda_k}, \qquad \sum_{j=1}^{p}\mathrm{VIF}_j = \mathrm{tr}(R^{-1}) = \sum_{k=1}^{p}\frac{1}{\lambda_k}, \qquad \max_j \mathrm{VIF}_j \le \frac{1}{\lambda_{\min}}.$$

Proof. Partition $R = \begin{pmatrix} 1 & \mathbf{r}^\top \\ \mathbf{r} & R_{-j} \end{pmatrix}$ with variable j first. By the block-inverse (Schur complement) formula, $(R^{-1})_{11} = 1/(1-\mathbf{r}^\top R_{-j}^{-1}\mathbf{r})$, and $\mathbf{r}^\top R_{-j}^{-1}\mathbf{r}$ is exactly the R^2 of the population regression of the standardised x_j on the others. The eigen-expression follows from $R^{-1} = Q\boldsymbol{\Lambda}^{-1}Q^\top$; the trace identity by summing over j; and the bound because $\sum_k q_{jk}^2 = 1$, so VIF_j is a weighted average of the $1/\lambda_k$. □

Belsley-style diagnostics (Belsley et al., 1980) complement VIFs with the *condition indices* $\sqrt{\lambda_{\max}/\lambda_k}$ of the (column-scaled) design and the eigenvector $\mathbf{q}_k$ attached to the smallest eigenvalue, which identifies *which* variables are involved in a near-dependency. Common rules of thumb flag VIFs above 10 and condition indices above 30, but these are conventions, not theorems.

Listing 3.11: Multicollinearity among simulated policy covariates ch03/vif_collinearity.py

```
# DEP 803 -- Chapter 3: Eigenvalues, the SVD, PCA and Conditioning
# Multicollinearity among simulated policy covariates: VIF, eigenvalues, ridge.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
names = ["educ_spend", "health_spend", "road_density", "pupil_teacher"]


def simulate(n, rho, rng):
    """Simulated district covariates: education and health spending share a
    common budget component; correlation between them approximately rho."""
```

```
    budget = rng.standard_normal(n)
    e1 = np.sqrt(rho) * budget + np.sqrt(1 - rho) * rng.standard_normal(n)
    e2 = np.sqrt(rho) * budget + np.sqrt(1 - rho) * rng.standard_normal(n)
    road = 0.3 * budget + rng.standard_normal(n)
    ptr = -0.2 * e1 + rng.standard_normal(n)
    return np.c_[e1, e2, road, ptr]


n = 200
X = simulate(n, 0.95, rng)
R = np.corrcoef(X, rowvar=False)
vif = np.diag(np.linalg.inv(R))
# VIF_j = 1 / (1 - R_j^2) from the auxiliary regression of x_j on the others
Xs = (X - X.mean(0)) / X.std(0)
for j, name in enumerate(names):
    others = np.c_[np.ones(n), np.delete(Xs, j, axis=1)]
    g, *_ = np.linalg.lstsq(others, Xs[:, j], rcond=None)
    r2 = 1 - np.var(Xs[:, j] - others @ g) / np.var(Xs[:, j])
    print(f"{name:13s} VIF = {vif[j]:6.2f}   1/(1-R^2) = {1 / (1 - r2):6.2f}")
lam, V = np.linalg.eigh(R)
print("eigenvalues of R :", np.round(lam, 4))
print("sum VIF = sum 1/lambda:", round(vif.sum(), 3), round((1 / lam).sum(), 3))
print("condition indices sqrt(lmax/l):", np.round(np.sqrt(lam[-1] / lam), 2))
print("smallest eigvec  :", np.round(V[:, 0], 3))

# Sampling spread of OLS vs ridge coefficients as collinearity rises
beta = np.array([1.0, 1.0, 0.5, -0.5])
rhos = np.array([0.0, 0.5, 0.8, 0.9, 0.95, 0.98, 0.99])
k_ridge = 10.0
sd_ols, sd_rdg = [], []
for rho in rhos:
    b_o, b_r = [], []
    for rep in range(500):
        Xr = simulate(n, rho, rng)
        y = Xr @ beta + 2.0 * rng.standard_normal(n)
        G = Xr.T @ Xr
        b_o.append(np.linalg.solve(G, Xr.T @ y))
        b_r.append(np.linalg.solve(G + k_ridge * np.eye(4), Xr.T @ y))
    sd_ols.append(np.std(np.array(b_o)[:, 0]))
    sd_rdg.append(np.std(np.array(b_r)[:, 0]))
print(" rho   sd(OLS b1)  sd(ridge b1)  1/sqrt(1-rho^2)")
for r_, so, sr in zip(rhos, sd_ols, sd_rdg):
    print(f"{r_:4.2f}   {so:8.3f}    {sr:8.3f}      {1 / np.sqrt(1 - r_ ** 2):6.2f}")
G = X.T @ X
print(f"cond(X'X) = {np.linalg.cond(G):.1f};"
      f" cond(X'X + {k_ridge:.0f} I) = "
      f"{np.linalg.cond(G + k_ridge * np.eye(4)):.1f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(rhos, sd_ols, "o-", label="OLS")
ax.plot(rhos, sd_rdg, "s-", label=f"ridge (k = {k_ridge:.0f})")
ax.set_xlabel("correlation between education and health spending")
ax.set_ylabel("sampling sd of coefficient on educ_spend")
ax.set_title("Multicollinearity inflates variance; ridge stabilises")
ax.legend(frameon=False)
plt.savefig("ch03_vif_ridge.pdf", bbox_inches="tight")
```

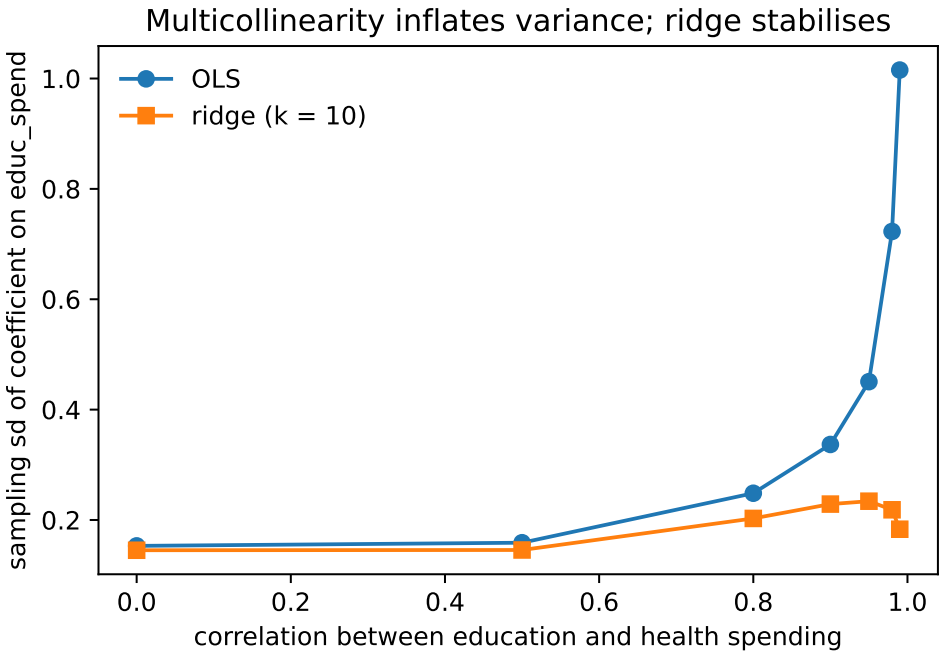


Figure 3.9: Sampling standard deviation of the coefficient on (simulated) education spending as its correlation with health spending rises, for OLS and for ridge regression with $k = 10$ (500 replications, $n = 200$).

Output

```
educ_spend    VIF =  11.75   1/(1-R^2) =  11.75
health_spend  VIF =  11.91   1/(1-R^2) =  11.91
road_density  VIF =   1.16   1/(1-R^2) =   1.16
pupil_teacher VIF =   1.12   1/(1-R^2) =   1.12
eigenvalues of R : [0.0434 0.7047 0.9408 2.3111]
sum VIF = sum 1/lambda: 25.945 25.945
condition indices sqrt(lmax/l): [7.3  1.81 1.57 1.  ]
smallest eigvec  : [-0.704  0.71  -0.025 -0.011]
 rho   sd(OLS b1)  sd(ridge b1)  1/sqrt(1-rho^2)
0.00      0.153       0.145        1.00
0.50      0.159       0.146        1.15
0.80      0.248       0.203        1.67
0.90      0.337       0.229        2.29
0.95      0.451       0.234        3.20
0.98      0.723       0.218        5.03
0.99      1.015       0.183        7.09
cond(X'X) = 53.0; cond(X'X + 10 I) = 25.0
```

The simulated district data have education and health spending sharing a common budget component (correlation 0.95). Both have VIFs near 12 – computed identically from $(R^{-1})_{jj}$ and from auxiliary regressions – while road density and the pupil–teacher ratio are almost unaffected. The smallest eigenvalue 0.043 has eigenvector $\approx (-0.70, 0.71, 0, 0)$: the data say almost nothing about the *difference* between the two spending effects, although their sum is well determined. The Monte Carlo table shows the sampling standard deviation of the education coefficient growing roughly like $1/\sqrt{1-\rho^2} = \sqrt{\text{VIF}}$.

3.6.5 Ridge regression as a remedy

Ridge regression (Hoerl and Kennard, 1970) replaces $\mathbf{X}^\top\mathbf{X}$ by $\mathbf{X}^\top\mathbf{X} + k\mathbf{I}$:

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + k\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y} = \sum_{j=1}^{p} \frac{\sigma_j}{\sigma_j^2 + k}\,(\mathbf{u}_j^\top\mathbf{y})\,\mathbf{v}_j.$$

Each eigenvalue is shifted by k, so $\kappa_2(\mathbf{X}^\top\mathbf{X} + k\mathbf{I}) = (\sigma_1^2 + k)/(\sigma_p^2 + k)$ falls (from 53 to 25 in the listing), and the directions with small σ_j are *shrunk* by the factor $\sigma_j^2/(\sigma_j^2 + k)$ instead of amplified by $1/\sigma_j$. The price is bias. In Fig. 3.9 the ridge standard deviation stays almost flat as $\rho \to 1$. The derivation of the ridge estimator as a penalised least-squares problem, its bias–variance trade-off and the choice of k by cross-validation are developed in Chapter 12; its Bayesian interpretation (a Gaussian prior on $\boldsymbol{\beta}$) is taken up with estimation in Chapter 7. *Principal components regression*, which keeps only the components with large σ_j (a hard threshold rather than ridge's smooth shrinkage), links this section back to Section 3.5.

Policy Lens: Collinear policy covariates

Policy variables frequently move together: education and health budgets are set in the same appropriation process; minimum wages and inflation rise together; districts with roads also have electricity. Regressing an outcome on all of them yields large standard errors, unstable signs and coefficients that change dramatically when one variable is added – not because the effects are absent but because the data cannot separate them. Before concluding that "education spending has no effect", inspect VIFs and the eigenvector of the smallest eigenvalue: it tells you which contrast is unidentified. Remedies are substantive rather than mechanical – collect variation that separates the policies (a reform that changed one and not the other), report the well-determined combination (total social spending), or use shrinkage when prediction rather than causal interpretation is the goal. Dropping a variable simply to lower a VIF changes the estimand and can introduce omitted-variable bias.

Historical Note: Turing, von Neumann and the birth of numerical analysis

The first rigorous error analyses of matrix computation were written just as electronic computers appeared. John von Neumann and Herman Goldstine's 1947 paper on inverting matrices of high order analysed Gaussian elimination for symmetric positive definite matrices and identified the ratio of extreme eigenvalues as the quantity controlling error growth. In 1948 Alan Turing, in "Rounding-off errors in matrix processes", introduced the term *condition number*, defining measures he called the N- and M-condition numbers, and also popularised the LU factorisation viewpoint (Turing, 1948; von Neumann and Goldstine, 1947). James Wilkinson's subsequent backward error analysis explained why Gaussian elimination with pivoting works so well in practice. The Hilbert matrix, which David Hilbert studied in 1894 in connection with approximation by polynomials, became the standard test case for ill-conditioning.

Exercises

3.6.1 Theory Prove $\kappa_2(A) = \sigma_1/\sigma_n$, $\kappa_2(Q) = 1$ for orthogonal Q, $\kappa(AB) \le \kappa(A)\kappa(B)$, and $\kappa_2(\mathbf{X}^\top\mathbf{X}) = \kappa_2(\mathbf{X})^2$.

3.6.2 Theory Show that $\min\{\|E\|_2 : A + E \text{ singular}\} = \sigma_n(A)$, so that $1/\kappa_2(A)$ is the relative distance from A to the set of singular matrices.

3.6.3 Theory Prove Proposition 3.21 in detail, including the Schur-complement step. Deduce that with two regressors of correlation ρ, $\mathrm{VIF}_1 = \mathrm{VIF}_2 = 1/(1-\rho^2)$ and the eigenvalues of R are $1 \pm \rho$.

3.6.4 Python For $n = 3, \dots, 12$ fit a degree-$(n-1)$ polynomial to 100 points on $[0, 1]$ using (i) raw monomials, (ii) monomials of the centred and scaled variable $2t - 1$, and (iii) Legendre polynomials (`np.polynomial.legendre.legvander`). Plot $\kappa(\mathbf{X})$ for each basis.

3.6.5 Python Extend `vif_collinearity.py` to compute the bias, variance and MSE of the ridge coefficient on education spending for $k \in \{0, 1, 3, 10, 30, 100\}$ at $\rho = 0.95$. Which k minimises the MSE of $\hat{\beta}_1$? Of $\hat{\beta}_1 + \hat{\beta}_2$?

3.6.6 ★ Graduate Using the SVD, derive the bias and variance of the ridge estimator and show that there always exists $k > 0$ for which $\mathrm{MSE}(\hat{\boldsymbol{\beta}}_{\mathrm{ridge}}) < \mathrm{MSE}(\hat{\boldsymbol{\beta}}_{\mathrm{OLS}})$ (the Hoerl–Kennard existence theorem). Why does this not contradict the Gauss–Markov theorem?

3.6.7 ★ Graduate Policy In a regression of district learning outcomes on education and health spending with $\rho = 0.95$, show that $\hat{\beta}_1 + \hat{\beta}_2$ has a much smaller standard error than either coefficient. Write a short note to a budget committee explaining what the data can and cannot say about reallocating funds from health to education.

Think About It

A colleague reports that the first principal component of 30 district indicators "explains only 25% of the variance, so PCA is useless here". Another reports 95% and concludes the index is excellent. What would you want to know about the indicators, their scaling and the eigenvalue spectrum before agreeing with either?

Chapter Summary

- An eigenpair satisfies $A\mathbf{v} = \lambda\mathbf{v}$; eigenvalues are the roots of $\det(\lambda\mathbf{I} - A)$, with $\sum \lambda_i = \operatorname{tr} A$ and $\prod \lambda_i = \det A$. Diagonalisation $A = V\boldsymbol{\Lambda}V^{-1}$ turns powers and dynamics into scalar problems; stability requires $\rho(A) < 1$.
- A primitive stochastic matrix has a unique stationary distribution (a left Perron eigenvector); convergence is geometric at rate $|\lambda_2|$, which gives half-lives of labour-market shocks.
- Symmetric matrices have real eigenvalues and orthonormal eigenvectors, $A = Q\boldsymbol{\Lambda}Q^{\top}$. Definiteness is the sign pattern of the eigenvalues; Cholesky is the practical test. Covariance matrices define Mahalanobis ellipsoids with axes $\sqrt{\lambda_i}\mathbf{q}_i$.
- Power iteration converges at rate $|\lambda_2/\lambda_1|$ (squared for the Rayleigh quotient of symmetric matrices); shifts and inverse iteration target any eigenvalue; RQI converges cubically; libraries use the shifted QR algorithm. PageRank is a Perron vector computed by power iteration. Use `eigh` for symmetric matrices.
- Every matrix has an SVD $\mathbf{X} = U\Sigma V^{\top}$; σ_j^2 are the eigenvalues of $\mathbf{X}^{\top}\mathbf{X}$ and $\mathbf{X}\mathbf{X}^{\top}$. Truncating the SVD gives the best rank-k approximation (Eckart–Young–Mirsky).
- PCA finds orthogonal directions of maximal variance: eigenvectors of the covariance (or correlation) matrix, computed via the SVD of the centred data. Standardisation matters. Asset indices and diffusion indexes are first principal components.
- $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ bounds the amplification of relative errors; expect to lose $\log_{10}\kappa$ digits. Normal equations square κ. Multicollinearity means small eigenvalues of the correlation matrix: $\mathrm{VIF}_j = (R^{-1})_{jj}$. Ridge regression shifts the eigenvalues by k and trades variance for bias.

Further Reading

Strang (2019) is the most readable account of eigenvalues, the SVD and low-rank approximation with data-science applications, and Deisenroth et al. (2020, ch. 4 and 10) covers matrix decompositions and PCA at a level close to this chapter. For machine-learning perspectives on PCA, probabilistic PCA and factor analysis see Murphy (2022, ch. 20) and Géron (2022, ch. 8); Hastie et al. (2009, sec. 14.5) treats principal components, curves and surfaces. Jolliffe (2002) is the standard monograph on PCA, and Jolliffe and Cadima (2016) a short modern review. The numerical side – the QR algorithm, Lanczos methods, the Golub–Kahan SVD and perturbation theory – is covered in Trefethen and Bau (1997) (an excellent first course) and Golub and Van Loan (2013) (the reference); Higham (2002) is definitive on conditioning and stability, and Horn and Johnson (2013) on matrix theory including Perron–Frobenius. Stewart (1993) tells the early history of the SVD. On Markov chains see Levin and Peres (2017), and on PageRank Langville and Meyer (2006). For factor models and diffusion indexes start with Stock and Watson (2002a,b) and Bai and Ng (2002), and read Goulet Coulombe et al. (2022) for how factors interact with machine-learning forecasters. On asset indices see Filmer and Pritchett (2001) and McKenzie (2005). Collinearity diagnostics are developed in Belsley et al. (1980), and their econometric interpretation in Hansen (2022).

Part II

Probability and Statistical Models

Chapter 4

Probability Foundations, Random Variables and Transformations

Learning Objectives

After working through this chapter you should be able to:

- define a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, explain why a σ-algebra is needed, and derive the elementary consequences of Kolmogorov's axioms (complements, monotonicity, inclusion–exclusion, continuity);
- compute and interpret conditional probabilities, apply the law of total probability and Bayes' theorem, and recognise the base-rate fallacy in targeting and diagnostic problems;
- work with random variables through their CDF, PMF, PDF and quantile function, including mixed distributions, and use the empirical CDF as a data-driven estimate of F;
- manipulate joint, marginal and conditional distributions, test independence, and separate marginals from dependence with copulas (Sklar's theorem);
- derive the distribution of transformed random variables by the CDF method, the change-of-variables formula (with Jacobians), convolution and order statistics;
- generate random variables reproducibly in NumPy and prove why inverse transform sampling, Box–Muller and acceptance–rejection work.

Every statement a data scientist makes about uncertainty—a 95% prediction interval for inflation, a poverty headcount with a margin of error, the probability that a household selected by a targeting formula is truly poor—rests on the mathematical theory of probability. This chapter lays that foundation. We begin with the axiomatic definition of probability due to Kolmogorov, because it is the common language of statistics, econometrics and machine learning: once probability is defined as a *measure* on a collection of events, all of the rules used in later chapters (expectation in Chapter 6, estimation in Chapter 7, Monte Carlo in Chapter 10) follow from three axioms.

For economic policy the payoff is immediate. Social-protection programmes rely on imperfect proxies of welfare, so they make two kinds of errors: they *include* households that are not poor and *exclude* households that are. These are conditional probabilities, and confusing $\mathbb{P}(\text{poor} \mid \text{selected})$ with $\mathbb{P}(\text{selected} \mid \text{poor})$ is one of the most common—and most consequential—reasoning errors in applied work. Similarly, household incomes are almost never normal, but their logarithms often are approximately so; knowing how densities transform under $y = e^x$ is what lets us move between the two scales correctly. Finally, modern policy analysis is increasingly computational:

microsimulation models, bootstrap standard errors and Bayesian samplers all consume streams of pseudo-random numbers. Section 4.6 explains where those numbers come from and why a seed is part of a reproducible analysis.

Throughout, the mathematical development is paired with Python. Each section contains programs whose output is reproduced verbatim (listings that produce a figure are shown up to the end of the computations; the plotting commands that follow are in the script files), and every number in this chapter comes from a *simulated* population: none of the figures should be read as statistics of Ghana or of any other country. Standard references for the material are Casella and Berger (2002), Wasserman (2004), Grimmett and Stirzaker (2020) and, at the measure-theoretic level, Billingsley (1995); Murphy (2022, Ch. 2) gives the machine-learning view.

4.1 Probability Spaces

4.1.1 Sample spaces and events

A *random experiment* is any process whose outcome is not known in advance: rolling a die, drawing a household at random from a sampling frame, observing next month's inflation rate. The set of all possible outcomes is the *sample space* Ω; an individual outcome is written $\omega \in \Omega$. Subsets of Ω are *events*.

Example 4.1 (Sample spaces). (a) One roll of a die: $\Omega = \{1,2,3,4,5,6\}$; the event "even" is $A = \{2,4,6\}$.

(b) Selecting one household from a frame of N households: $\Omega = \{1,\dots,N\}$; the event "household is poor" is the set of indices whose consumption is below the line.

(c) Monthly inflation: $\Omega = \mathbb{R}$; the event "inflation within the target band [6%, 10%]" is the interval $[6,10]$.

(d) An infinite sequence of coin tosses: $\Omega = \{0,1\}^{\mathbb{N}}$, an uncountable set. Events such as "the relative frequency of heads converges to $1/2$" live here.

Set operations on events have a direct reading: $A \cup B$ is "A or B occurs", $A \cap B$ (also written AB) is "both occur", $A^c = \Omega \setminus A$ is "A does not occur", and $A \cap B = \varnothing$ means A and B are *disjoint* (mutually exclusive). De Morgan's laws, $\left(\bigcup_i A_i\right)^c = \bigcap_i A_i^c$ and $\left(\bigcap_i A_i\right)^c = \bigcup_i A_i^c$, are used constantly.

4.1.2 Why we need a σ-algebra

On a finite sample space we can assign a probability to every subset. On $\Omega = \mathbb{R}$ or $\Omega = \{0,1\}^{\mathbb{N}}$ this is impossible in general: there is no way to assign a "length" to *every* subset of $[0,1]$ that is countably additive and translation invariant (the Vitali construction, see Billingsley, 1995, Sec. 3). The resolution is to restrict attention to a well-behaved collection of events that is closed under the operations we need.

Definition 4.2 (σ-algebra). A collection $\mathcal{F}$ of subsets of Ω is a *σ-algebra* (or σ-field) if

(i) $\Omega \in \mathcal{F}$;

(ii) $A \in \mathcal{F} \Rightarrow A^c \in \mathcal{F}$ (closed under complements);

(iii) $A_1, A_2, \dots \in \mathcal{F} \Rightarrow \bigcup_{i=1}^{\infty} A_i \in \mathcal{F}$ (closed under countable unions).

By De Morgan, a σ-algebra is also closed under countable intersections, and it always contains $\varnothing$. The smallest example is $\{\varnothing, \Omega\}$ (we know nothing); the largest is the power set 2^{Ω} (we can ask about everything). On $\mathbb{R}$ the standard choice is the *Borel σ-algebra* $\mathcal{B}(\mathbb{R})$, the smallest σ-algebra

containing all intervals $(a, b]$. It contains every set a practitioner will ever meet—open and closed sets, countable sets, limits of intervals—while excluding pathological sets.

Think About It

A σ-algebra can be read as an *information set*. If an analyst only observes whether a household is above or below the poverty line, the events she can evaluate form $\{\varnothing, P, P^c, \Omega\}$, where P is "poor". Observing the full consumption value generates a much richer σ-algebra. Conditional expectation (Chapter 6) is defined relative to such information sets, which is why the concept matters in econometrics and finance.

4.1.3 Kolmogorov's axioms

Definition 4.3 (Probability measure; probability space)**.** Let $\mathcal{F}$ be a σ-algebra on Ω. A function $\mathbb{P} : \mathcal{F} \to \mathbb{R}$ is a *probability measure* if

(K1) $\mathbb{P}(A) \geq 0$ for every $A \in \mathcal{F}$ (non-negativity);

(K2) $\mathbb{P}(\Omega) = 1$ (normalisation);

(K3) for every sequence of pairwise disjoint events $A_1, A_2, \ldots \in \mathcal{F}$,

$$\mathbb{P}\Big(\bigcup_{i=1}^{\infty} A_i\Big) = \sum_{i=1}^{\infty} \mathbb{P}(A_i) \qquad \text{(countable additivity).} \tag{4.1}$$

The triple $(\Omega, \mathcal{F}, \mathbb{P})$ is a *probability space*.

⋆ Key Concept: The probability space

A probability model has three ingredients: *what can happen* (Ω), *what we can ask about* ($\mathcal{F}$), and *how likely each askable event is* ($\mathbb{P}$). Every rule of probability used in this course is a consequence of the three axioms (K1)–(K3). When a model seems paradoxical, the first diagnostic question is: which of the three ingredients has been specified ambiguously?

Historical Note: From gambling to measure theory

Systematic probability reasoning is usually dated to the 1654 correspondence between Blaise Pascal and Pierre de Fermat on the *problem of points*: how to divide the stakes fairly when a game of chance is interrupted before it ends. A century earlier Gerolamo Cardano had written on games of chance in his *Liber de ludo aleae*, which was published only in 1663, long after his death. Pierre-Simon Laplace systematised the classical theory in his *Théorie analytique des probabilités* (1812) and the more popular *Essai philosophique sur les probabilités* (1814). The modern axiomatic foundation came with Andrey Kolmogorov's monograph *Grundbegriffe der Wahrscheinlichkeitsrechnung* (1933), which built probability on Lebesgue's measure theory and answered the call in Hilbert's sixth problem (1900) for an axiomatic treatment of probability.

4.1.4 Consequences of the axioms

Proposition 4.4 (Elementary properties)**.** *For events $A, B \in \mathcal{F}$:*

(a) $\mathbb{P}(\varnothing) = 0$ *and finite additivity holds;*

(b) $\mathbb{P}(A^c) = 1 - \mathbb{P}(A)$*;*

(c) if $A \subseteq B$ *then* $\mathbb{P}(A) \leq \mathbb{P}(B)$*, and* $\mathbb{P}(B \setminus A) = \mathbb{P}(B) - \mathbb{P}(A)$*;*

(d) $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$;

(e) (Boole's inequality) $\mathbb{P}(\bigcup_i A_i) \le \sum_i \mathbb{P}(A_i)$ *for any countable family.*

Proof. (a) Take $A_1 = \Omega$ and $A_i = \varnothing$ for $i \ge 2$ in (K3): $1 = 1 + \sum_{i \ge 2} \mathbb{P}(\varnothing)$, so $\mathbb{P}(\varnothing) = 0$. Padding a finite disjoint family with empty sets gives finite additivity. (b) $\Omega = A \cup A^c$ disjointly, so $1 = \mathbb{P}(A) + \mathbb{P}(A^c)$. (c) $B = A \cup (B \setminus A)$ disjointly, so $\mathbb{P}(B) = \mathbb{P}(A) + \mathbb{P}(B \setminus A) \ge \mathbb{P}(A)$ by (K1). (d) $A \cup B = A \cup (B \setminus A)$ and $B = (A \cap B) \cup (B \setminus A)$, both disjoint; subtract. (e) Define $B_1 = A_1$ and $B_i = A_i \setminus \bigcup_{j<i} A_j$. The B_i are disjoint, have the same union as the A_i, and $B_i \subseteq A_i$; apply (K3) and (c). □

Property (d) generalises to any finite number of events.

Theorem 4.5 (Inclusion–exclusion)**.** *For events* $A_1, \dots, A_n$,

$$\mathbb{P}\Big(\bigcup_{i=1}^{n} A_i\Big) = \sum_i \mathbb{P}(A_i) - \sum_{i<j} \mathbb{P}(A_i A_j) + \sum_{i<j<k} \mathbb{P}(A_i A_j A_k) - \cdots + (-1)^{n+1} \mathbb{P}(A_1 \cdots A_n). \tag{4.2}$$

Proof sketch. By induction on n: apply (d) to $A = \bigcup_{i<n} A_i$ and $B = A_n$, and use the induction hypothesis on both $\mathbb{P}(\bigcup_{i<n} A_i)$ and $\mathbb{P}(\bigcup_{i<n} (A_i A_n))$. Alternatively, write $\mathbf{1}_{\cup A_i} = 1 - \prod_i (1 - \mathbf{1}_{A_i})$, expand the product and take expectations (Chapter 6). □

Example 4.6 (The matching problem)**.** A statistics office randomly assigns n enumerators to n districts, one each. Each enumerator has a home district among the n. Let A_i be the event that enumerator i is sent home. For any k indices, $\mathbb{P}(A_{i_1} \cdots A_{i_k}) = (n-k)!/n!$, and there are $\binom{n}{k}$ such terms, so (4.2) gives

$$\mathbb{P}(\text{at least one match}) = \sum_{k=1}^{n} (-1)^{k+1} \binom{n}{k} \frac{(n-k)!}{n!} = \sum_{k=1}^{n} \frac{(-1)^{k+1}}{k!} \longrightarrow 1 - e^{-1} \approx 0.632.$$

Remarkably, the answer hardly depends on n once $n \ge 5$.

Listing 4.1: Inclusion–exclusion versus simulation for the matching problem ch04/ch04_inclexcl.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Matching problem: inclusion-exclusion formula versus simulation.
# (c) 2026 Stephen E. Moore. MIT Licence.
import math
import numpy as np

rng = np.random.default_rng(803)


def p_match_exact(n):
    """P(at least one fixed point of a random permutation of n items)."""
    return sum((-1) ** (k + 1) / math.factorial(k) for k in range(1, n + 1))


def p_match_sim(n, reps):
    """Monte Carlo estimate using random permutations (vectorised)."""
    perms = np.argsort(rng.random((reps, n)), axis=1)
    return np.mean((perms == np.arange(n)).any(axis=1))


print("n enumerators are randomly assigned to n districts.")
print("Event: at least one enumerator is sent to his or her home district.\n")
```

```python
print(f"{'n':>4} {'incl-excl':>11} {'simulated':>11} {'1 - 1/e':>9}")
for n in [2, 3, 5, 10, 50]:
    print(f"{n:>4} {p_match_exact(n):11.6f} {p_match_sim(n, 100_000):11.6f}"
          f" {1 - math.exp(-1):9.6f}")

# Continuity from below: A_n = {first six occurs within n rolls} increases
print("\nContinuity: P(first six within n rolls) = 1 - (5/6)^n increases to 1")
for n in [1, 5, 10, 25, 50]:
    print(f"  n = {n:>2}: {1 - (5 / 6) ** n:.6f}")
```

Output

```
n enumerators are randomly assigned to n districts.
Event: at least one enumerator is sent to his or her home district.

   n   incl-excl   simulated   1 - 1/e
   2    0.500000    0.503450  0.632121
   3    0.666667    0.667120  0.632121
   5    0.633333    0.635330  0.632121
  10    0.632121    0.631990  0.632121
  50    0.632121    0.631330  0.632121

Continuity: P(first six within n rolls) = 1 - (5/6)^n increases to 1
  n =  1: 0.166667
  n =  5: 0.598122
  n = 10: 0.838494
  n = 25: 0.989517
  n = 50: 0.999890
```

The second half of the listing illustrates the following continuity property, which is equivalent to countable additivity (given finite additivity) and is used whenever we pass to a limit inside a probability.

Theorem 4.7 (Continuity of probability)**.** *If $A_1 \subseteq A_2 \subseteq \cdots$ and $A = \bigcup_n A_n$, then $\mathbb{P}(A_n) \uparrow \mathbb{P}(A)$. If $B_1 \supseteq B_2 \supseteq \cdots$ and $B = \bigcap_n B_n$, then $\mathbb{P}(B_n) \downarrow \mathbb{P}(B)$.*

Proof. Let $C_1 = A_1$ and $C_n = A_n \setminus A_{n-1}$. The C_n are disjoint with $\bigcup_{k \le n} C_k = A_n$ and $\bigcup_k C_k = A$. By (K3), $\mathbb{P}(A) = \sum_{k=1}^{\infty} \mathbb{P}(C_k) = \lim_n \sum_{k \le n} \mathbb{P}(C_k) = \lim_n \mathbb{P}(A_n)$. The decreasing case follows by taking complements. □

In the listing, $A_n = \{\text{first six within } n \text{ rolls}\}$ increases to the event "a six eventually appears", whose probability is therefore $\lim_n [1 - (5/6)^n] = 1$. Continuity is also what makes a CDF right-continuous (Theorem 4.16 below).

△ Caution: Disjoint is not independent

Students frequently confuse *disjoint* events ($A \cap B = \varnothing$) with *independent* events ($\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$, Section 4.2). If A and B are disjoint and both have positive probability, they are maximally *dependent*: learning that A occurred tells you for certain that B did not. Likewise, adding probabilities $\mathbb{P}(A) + \mathbb{P}(B)$ is only valid for disjoint events; otherwise use inclusion–exclusion.

4.1.5 Interpretations of probability

The axioms say how probabilities must *behave*, not what they *mean*. Two interpretations dominate applied work.

Frequentist. $\mathbb{P}(A)$ is the long-run relative frequency of A in independent repetitions of the experiment. If $f_n(A)$ is the proportion of the first n trials in which A occurs, then $f_n(A) \to \mathbb{P}(A)$. In the frequentist view this is a *definition*; in Kolmogorov's framework it is a *theorem*—the strong law of large numbers, proved in Chapter 6. Note that relative frequencies satisfy the axioms exactly: $f_n(A) \geq 0$, $f_n(\Omega) = 1$ and $f_n(A \cup B) = f_n(A) + f_n(B)$ for disjoint A, B.

Subjective (Bayesian). $\mathbb{P}(A)$ is a coherent degree of belief that A is true, which may be held about one-off events—"the central bank will cut rates next meeting"—for which no long-run frequency exists. De Finetti's *Dutch book* argument shows that beliefs which violate the axioms expose the holder to a set of bets that lose money for sure; coherence forces the axioms. Bayesian statistics (Gelman et al., 2013; Matsuura, 2022) updates such beliefs with Bayes' theorem (Section 4.2).

Both interpretations use the same mathematics, so nothing in this chapter depends on which one you prefer. The relative-frequency idea is, however, the engine of simulation: we estimate a probability by generating many independent repetitions on a computer and counting.

Listing 4.2: Relative frequencies of a six in five independent sequences of die rolls
ch04/ch04_relfreq.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Relative frequencies of an event settle down as the number of trials grows.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n_max, n_paths = 10_000, 5
rolls = rng.integers(1, 7, size=(n_paths, n_max))      # fair six-sided die
is_six = (rolls == 6)
n = np.arange(1, n_max + 1)
running = np.cumsum(is_six, axis=1) / n                 # relative frequency path

print("Relative frequency of the event {six} (true value 1/6 = 0.1667)")
print(f"{'n':>7}" + "".join(f"  path{j + 1}" for j in range(n_paths)))
for k in [10, 100, 1_000, 10_000]:
    row = "".join(f"{running[j, k - 1]:8.4f}" for j in range(n_paths))
    print(f"{k:>7}" + row)

# Event algebra on the same sample: A = {even}, B = {>= 4}
x = rolls[0]
A, B = (x % 2 == 0), (x >= 4)
pA, pB, pAB, pAuB = A.mean(), B.mean(), (A & B).mean(), (A | B).mean()
print("\nAddition rule on path 1 (relative frequencies obey it exactly):")
print(f"  f(A) + f(B) - f(A and B) = {pA + pB - pAB:.4f}")
print(f"  f(A or B)                = {pAuB:.4f}   (true 4/6 = {4 / 6:.4f})")
```

Output

```
Relative frequency of the event {six} (true value 1/6 = 0.1667)
      n  path1  path2  path3  path4  path5
     10  0.2000  0.3000  0.1000  0.2000  0.1000
    100  0.1800  0.1300  0.1900  0.2100  0.2000
   1000  0.1680  0.1620  0.1710  0.1530  0.1770
  10000  0.1699  0.1629  0.1724  0.1630  0.1600

Addition rule on path 1 (relative frequencies obey it exactly):
  f(A) + f(B) - f(A and B) = 0.6618
  f(A or B)                = 0.6618   (true 4/6 = 0.6667)
```

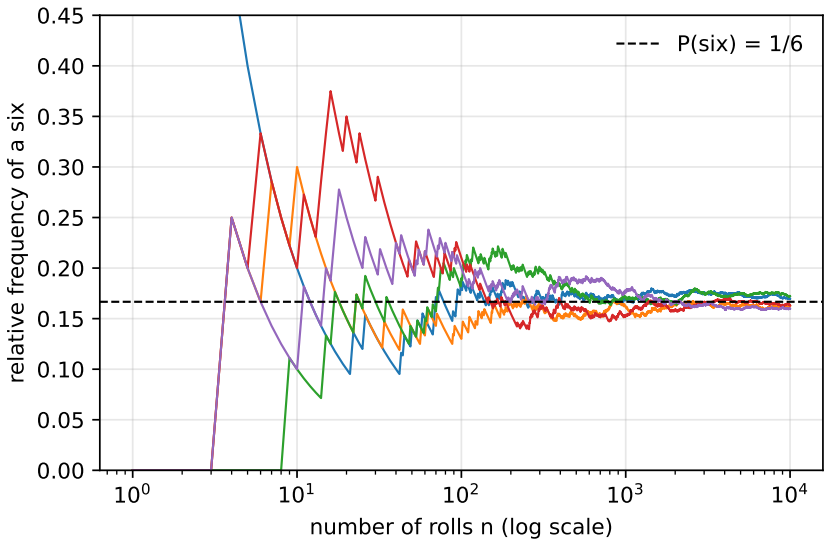


Figure 4.1: Running relative frequency of a six in five independent sequences of 10,000 simulated rolls. Early fluctuations are large; all paths settle near $1/6$ at the rate $1/\sqrt{n}$ (Chapter 6).

Figure 4.1 shows two facts at once. First, after 10 rolls the relative frequency can be far from $1/6$; small samples are noisy. Second, the paths converge, but slowly: the typical error after n rolls is of order $\sqrt{p(1-p)/n}$, so a hundred times more rolls buy only one more decimal digit of accuracy. This $1/\sqrt{n}$ rate governs all Monte Carlo methods in Chapter 10.

Policy Lens: Risk, insurance and the frequentist view

Weather-index insurance for smallholder farmers pays out when, for example, seasonal rainfall at a reference station falls below a trigger. The premium is set from an estimate of $\mathbb{P}(\text{rainfall} < \text{trigger})$, usually a relative frequency over a few decades of station records. With 30 years of data and a true payout probability of 0.1, the standard error of the frequency estimate is $\sqrt{0.1 \times 0.9/30} \approx 0.055$—more than half the quantity being estimated. Pricing, reinsurance and the design of the trigger must therefore treat the probability itself as uncertain, which is precisely where Bayesian reasoning and the bootstrap (Chapter 11) enter.

Exercises

4.1.1 Theory Let $\mathcal{F}$ be a σ-algebra. Prove that $\varnothing \in \mathcal{F}$, that $\mathcal{F}$ is closed under countable intersections and that $A, B \in \mathcal{F} \Rightarrow A \setminus B \in \mathcal{F}$.

4.1.2 Theory Let $\Omega = \{1,2,3,4\}$. Write down the smallest σ-algebra containing $\{1\}$ and $\{2\}$. How many elements does it have? Generalise: if a partition of Ω has k cells, how many events does the σ-algebra it generates contain?

4.1.3 Theory Prove Bonferroni's inequality $\mathbb{P}(\bigcap_{i=1}^{n} A_i) \geq 1 - \sum_{i=1}^{n} \mathbb{P}(A_i^c)$. Explain how it justifies testing each of m hypotheses at level α/m.

4.1.4 Python Modify `ch04_inclexcl.py` to compute the probability of *exactly* k matches for $n = 10$ and compare it with the Poisson(1) probabilities $e^{-1}/k!$. Explain the agreement.

4.1.5 Python Using `ch04_relfreq.py`, estimate how many rolls are needed for the relative frequency of a six to lie within 0.005 of $1/6$ in 95% of 1,000 simulated paths. Compare with the prediction $n \approx (1.96)^2 p(1-p)/0.005^2$.

4.1.6 ★ Graduate Show that a finitely additive $\mathbb{P}$ on a σ-algebra is countably additive if and only if it is continuous from below. Why did Kolmogorov insist on countable rather than finite additivity?

4.1.7 Policy A government designs a weather-index insurance product whose trigger is hit, historically, in 4 of the last 30 seasons. (a) Give the frequency estimate of the payout probability and its standard error. (b) Discuss two reasons why the historical frequency may be a poor guide to the future, and how a subjective probability could legitimately incorporate them.

4.2 Conditional Probability and Bayes' Theorem

4.2.1 Definition and the multiplication rule

Learning that an event B has occurred changes the relevant sample space from Ω to B. Probabilities must then be renormalised so that B has probability one.

Definition 4.8 (Conditional probability)**.** If $\mathbb{P}(B) > 0$, the *conditional probability of A given B* is

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}. \tag{4.3}$$

For fixed B, the map $A \mapsto \mathbb{P}(A \mid B)$ is itself a probability measure on $(\Omega, \mathcal{F})$: it is non-negative, gives $\mathbb{P}(\Omega \mid B) = 1$ and inherits countable additivity from $\mathbb{P}$. Hence every result of Section 4.1 holds conditionally, e.g. $\mathbb{P}(A^c \mid B) = 1 - \mathbb{P}(A \mid B)$. Rearranging (4.3) gives the *multiplication rule*

$$\mathbb{P}(A_1 A_2 \cdots A_n) = \mathbb{P}(A_1)\,\mathbb{P}(A_2 \mid A_1)\,\mathbb{P}(A_3 \mid A_1 A_2) \cdots \mathbb{P}(A_n \mid A_1 \cdots A_{n-1}), \tag{4.4}$$

the *chain rule* of probability. It is the basis of sequential sampling designs and of autoregressive models in machine learning, which factorise the probability of a sequence into one-step-ahead conditionals.

Example 4.9 (Two-stage sampling)**.** A survey selects 2 enumeration areas (EAs) at random from 10, of which 3 are urban, without replacement. By the chain rule, $\mathbb{P}(\text{both urban}) = \mathbb{P}(U_1)\mathbb{P}(U_2 \mid U_1) = \frac{3}{10} \cdot \frac{2}{9} = \frac{1}{15}$.

4.2.2 The law of total probability

Theorem 4.10 (Law of total probability)**.** *Let $B_1, B_2, \dots$ be a (finite or countable) partition of Ω with $\mathbb{P}(B_i) > 0$ for all i. Then for any event A,*

$$\mathbb{P}(A) = \sum_i \mathbb{P}(A \mid B_i)\,\mathbb{P}(B_i). \tag{4.5}$$

Proof. $A = \bigcup_i (A \cap B_i)$ with the pieces disjoint, so by (K3) $\mathbb{P}(A) = \sum_i \mathbb{P}(A \cap B_i) = \sum_i \mathbb{P}(A \mid B_i)\mathbb{P}(B_i)$. □

The law of total probability is the formal version of "averaging over strata". A national poverty rate is the population-weighted average of regional poverty rates; an unconditional default rate is the average of default rates across credit grades.

4.2.3 Independence and conditional independence

Definition 4.11 (Independence)**.** Events A and B are *independent*, written $A \perp B$, if $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$. A family $\{A_i\}_{i \in I}$ is *mutually independent* if $\mathbb{P}(\bigcap_{i \in J} A_i) = \prod_{i \in J} \mathbb{P}(A_i)$ for every finite $J \subseteq I$.

When $\mathbb{P}(B) > 0$, independence is equivalent to $\mathbb{P}(A \mid B) = \mathbb{P}(A)$: learning B does not change the probability of A. Pairwise independence does *not* imply mutual independence (Exercise **4.2.2**).

Definition 4.12 (Conditional independence)**.** A and B are *conditionally independent given* C ($\mathbb{P}(C) > 0$), written $A \perp B \mid C$, if $\mathbb{P}(A \cap B \mid C) = \mathbb{P}(A \mid C)\,\mathbb{P}(B \mid C)$.

Neither independence nor conditional independence implies the other. Two rapid tests on the same patient are strongly dependent (both tend to be positive when the patient is ill) but may be roughly independent *given* the true disease status. Conversely, two independent causes become dependent once we condition on a common effect—the "explaining away" phenomenon that underlies collider bias in causal inference.

4.2.4 Bayes' theorem

Theorem 4.13 (Bayes' theorem)**.** *Let $B_1, B_2, \dots$ be a partition of Ω with $\mathbb{P}(B_i) > 0$, and let $\mathbb{P}(A) > 0$. Then*

$$\mathbb{P}(B_j \mid A) = \frac{\mathbb{P}(A \mid B_j)\,\mathbb{P}(B_j)}{\sum_i \mathbb{P}(A \mid B_i)\,\mathbb{P}(B_i)}. \tag{4.6}$$

Proof. By (4.3) twice, $\mathbb{P}(B_j \mid A)\mathbb{P}(A) = \mathbb{P}(A \cap B_j) = \mathbb{P}(A \mid B_j)\mathbb{P}(B_j)$; expand $\mathbb{P}(A)$ with (4.5). □

In Bayesian language, $\mathbb{P}(B_j)$ is the *prior*, $\mathbb{P}(A \mid B_j)$ the *likelihood*, and $\mathbb{P}(B_j \mid A)$ the *posterior*. For two hypotheses H and H^c, the *odds form* is especially transparent:

$$\underbrace{\frac{\mathbb{P}(H \mid A)}{\mathbb{P}(H^c \mid A)}}_{\text{posterior odds}} = \underbrace{\frac{\mathbb{P}(A \mid H)}{\mathbb{P}(A \mid H^c)}}_{\text{likelihood ratio}} \times \underbrace{\frac{\mathbb{P}(H)}{\mathbb{P}(H^c)}}_{\text{prior odds}}. \tag{4.7}$$

Historical Note: Bayes, Price and Laplace

The Reverend Thomas Bayes (c. 1701–1761) solved a special case of the "inverse" problem—inferring the chance of an event from observed frequencies—in an essay found among his papers after his death. Richard Price edited it and communicated it to the Royal Society, which published it in the

Philosophical Transactions in 1763 (Bayes, 1763). Laplace, apparently unaware of Bayes' work, stated the principle in general form in 1774 and used it extensively, including in the *Théorie analytique* (1812), to analyse data such as the ratio of male to female births. The name "Bayesian" for the approach became common only in the twentieth century.

⋆ Key Concept: Never invert a conditional probability without the base rate

$\mathbb{P}(A \mid B)$ and $\mathbb{P}(B \mid A)$ are different numbers, linked by Bayes' theorem through the base rates $\mathbb{P}(A)$ and $\mathbb{P}(B)$. A test with 95% sensitivity and 95% specificity can still produce mostly false positives when the condition is rare. Ignoring the prior is the *base-rate fallacy*; in court it is called the prosecutor's fallacy.

4.2.5 Diagnostic tests: a malaria rapid test

Consider a malaria rapid diagnostic test (RDT) with sensitivity $\mathbb{P}(+ \mid \text{ill}) = 0.95$ and specificity $\mathbb{P}(- \mid \text{healthy}) = 0.95$; these are illustrative values, not those of any particular product. The *positive predictive value* is, by (4.6),

$$\mathrm{PPV}(p) = \mathbb{P}(\text{ill} \mid +) = \frac{0.95\,p}{0.95\,p + 0.05\,(1-p)},$$

where p is the prevalence among those tested.

Listing 4.3: Predictive values of a diagnostic test as functions of prevalence ch04/ch04_malaria.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Diagnostic test: positive and negative predictive values versus prevalence.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
sens, spec = 0.95, 0.95                    # illustrative test characteristics


def ppv(p):
    return sens * p / (sens * p + (1 - spec) * (1 - p))


def npv(p):
    return spec * (1 - p) / (spec * (1 - p) + (1 - sens) * p)


print(f"Rapid test with sensitivity {sens} and specificity {spec}")
print(f"{'prevalence':>10} {'PPV':>7} {'NPV':>7} {'PPV sim':>8}")
N = 1_000_000
for p in [0.01, 0.05, 0.10, 0.30, 0.50]:
    ill = rng.random(N) < p
    pos = np.where(ill, rng.random(N) < sens, rng.random(N) < 1 - spec)
    print(f"{p:10.2f} {ppv(p):7.3f} {npv(p):7.3f} {ill[pos].mean():8.3f}")

# Two positive tests, conditionally independent given true status
p = 0.05
post1 = ppv(p)
post2 = sens * post1 / (sens * post1 + (1 - spec) * (1 - post1))
print(f"\nPrevalence 0.05: P(ill | +) = {post1:.3f}, P(ill | +,+) = {post2:.3f}")
```

Output

```
Rapid test with sensitivity 0.95 and specificity 0.95
prevalence     PPV     NPV  PPV sim
      0.01   0.161   0.999    0.159
      0.05   0.500   0.997    0.500
      0.10   0.679   0.994    0.680
      0.30   0.891   0.978    0.890
      0.50   0.950   0.950    0.950

Prevalence 0.05: P(ill | +) = 0.500, P(ill | +,+) = 0.950
```

At 1% prevalence (say, screening in a low-transmission season) only about 16% of positives are truly infected; at 30% prevalence (testing febrile patients in a high-transmission area) the PPV is 89%. The test is the same; only the prior has changed. The last line of the output shows sequential updating: if a second test is conditionally independent of the first given the true status, the posterior after one positive becomes the prior for the second, and the probability of infection rises from 0.50 to 0.95.

4.2.6 The base-rate fallacy in poverty targeting

Many cash-transfer programmes—LEAP-type programmes for extremely poor households among them—select beneficiaries, at least in part, with a *proxy means test* (PMT): a score predicted from easily verified household characteristics (dwelling materials, assets, demographics) by a regression fitted on survey data. Households whose predicted consumption falls below a cut-off are eligible. Because the prediction is imperfect, two errors arise, and both are conditional probabilities:

$$\text{inclusion error} = \mathbb{P}(\text{non-poor} \mid \text{selected}) = 1 - \text{PPV},$$
$$\text{exclusion error} = \mathbb{P}(\text{not selected} \mid \text{poor}) = 1 - \text{sensitivity}.$$

These have different denominators—beneficiaries versus the poor—and therefore respond differently to programme design. The listing below simulates a stylised population of 200,000 households (not real data): log consumption is normal, 25% are poor, and the PMT score equals log consumption plus independent prediction error.

Listing 4.4: Inclusion and exclusion errors of a simulated proxy means test ch04/ch04_targeting.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Proxy means test (PMT) targeting in a simulated population: Bayes in action.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
N = 200_000                                  # simulated households
log_c = rng.normal(0.0, 0.6, N)              # log consumption per adult (simulated)
z = np.quantile(log_c, 0.25)                 # poverty line: 25% are poor
poor = log_c < z
pmt = log_c + rng.normal(0.0, 0.35, N)       # PMT score = noisy prediction
cut = np.quantile(pmt, 0.20)                 # programme covers 20% of households
selected = pmt < cut

sens = selected[poor].mean()                 # P(selected | poor)
spec = (~selected[~poor]).mean()             # P(not selected | non-poor)
```

```python
ppv = poor[selected].mean()                 # P(poor | selected)
prev = poor.mean()
ppv_bayes = sens * prev / (sens * prev + (1 - spec) * (1 - prev))
print(f"Prevalence of poverty  P(poor)              = {prev:.3f}")
print(f"Coverage               P(sel)               = {selected.mean():.3f}")
print(f"Sensitivity            P(sel | poor)        = {sens:.3f}")
print(f"Specificity            P(not sel | nonpoor) = {spec:.3f}")
print(f"P(poor | sel), direct count                 = {ppv:.3f}")
print(f"P(poor | sel), Bayes' theorem               = {ppv_bayes:.3f}")
print(f"Inclusion error  = 1 - P(poor | sel)        = {1 - ppv:.3f}")
print(f"Exclusion error  = 1 - P(sel | poor)        = {1 - sens:.3f}")

print("\nSame PMT accuracy, different coverage of the programme:")
print(f"{'coverage':>9} {'inclusion':>10} {'exclusion':>10}")
for cov in [0.10, 0.20, 0.25, 0.30, 0.40]:
    s = pmt < np.quantile(pmt, cov)
    print(f"{cov:9.2f} {1 - poor[s].mean():10.3f} {1 - s[poor].mean():10.3f}")
```

Output

```
Prevalence of poverty  P(poor)              = 0.250
Coverage               P(sel)               = 0.200
Sensitivity            P(sel | poor)        = 0.639
Specificity            P(not sel | nonpoor) = 0.946
P(poor | sel), direct count                 = 0.798
P(poor | sel), Bayes' theorem               = 0.798
Inclusion error  = 1 - P(poor | sel)        = 0.202
Exclusion error  = 1 - P(sel | poor)        = 0.361

Same PMT accuracy, different coverage of the programme:
 coverage  inclusion  exclusion
     0.10      0.077      0.631
     0.20      0.202      0.361
     0.25      0.265      0.265
     0.30      0.326      0.191
     0.40      0.431      0.089
```

The direct count and Bayes' theorem agree exactly, as they must. With 20% coverage and a 25% poverty rate the programme *cannot* reach all the poor, so exclusion error is at least $1 - 0.20/0.25 = 20\%$ even with a perfect PMT. The coverage table shows the fundamental trade-off: expanding the programme lowers exclusion error but raises inclusion error. At coverage equal to the poverty rate (25%) the two errors coincide.

Policy Lens: Targeting errors are a design choice

Policy debates often quote a single "targeting accuracy". The simulation shows why this is incomplete: for a given PMT, the budget (coverage) moves the programme along a trade-off curve between inclusion and exclusion errors. A ministry that cares more about not missing poor households (exclusion) should accept more leakage (inclusion), and vice versa; the choice is normative, not statistical. Reporting both errors, with their correct denominators, is the minimum standard for evaluating a targeting mechanism; see Brown et al. (2018) for evidence that PMT exclusion errors can be large in practice.

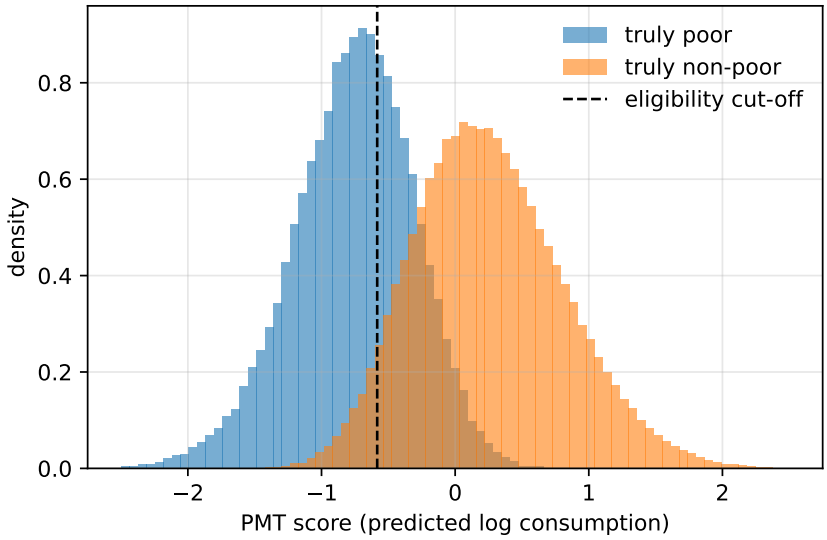


Figure 4.2: Distribution of the PMT score for truly poor and non-poor households in the simulated population. Poor households to the right of the cut-off are excluded; non-poor households to the left are included.

△ Caution: Which denominator?

"30% of beneficiaries are non-poor" and "30% of the poor are beneficiaries" are completely different statements. Before quoting an error rate, write it as $\mathbb{P}(\cdot \mid \cdot)$ and check what is being conditioned on. Reports that compare programmes using differently defined error rates are a common source of confusion.

4.2.7 The Monty Hall problem

A game-show contestant picks one of three doors; a car is behind one, goats behind the other two. The host, who knows where the car is, opens a *different* door showing a goat and offers a switch. Should the contestant switch? Let C_i be "car behind door i", the contestant picks door 1, and H_3 is "host opens door 3". Under the standard protocol (host always opens a goat door, choosing at random when both unchosen doors hide goats), $\mathbb{P}(H_3 \mid C_1) = 1/2$, $\mathbb{P}(H_3 \mid C_2) = 1$, $\mathbb{P}(H_3 \mid C_3) = 0$, and Bayes gives

$$\mathbb{P}(C_2 \mid H_3) = \frac{1 \cdot \tfrac{1}{3}}{\tfrac{1}{2} \cdot \tfrac{1}{3} + 1 \cdot \tfrac{1}{3} + 0} = \frac{2}{3}.$$

Switching wins with probability 2/3. With k doors and a host who opens $k-2$ goat doors, switching wins with probability $1 - 1/k$.

Listing 4.5: Simulating the Monty Hall game with k doors ch04/ch04_montyhall.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Monty Hall problem: simulate sticking versus switching with k doors.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)


def play(k, reps):
```

```
    """Host opens all but one of the unchosen doors, never revealing the car."""
    car = rng.integers(0, k, reps)
    pick = rng.integers(0, k, reps)
    # remaining closed door: the car if pick != car, else a random goat door
    other = rng.integers(0, k - 1, reps)
    other = other + (other >= pick)         # uniform over doors != pick
    remaining = np.where(pick != car, car, other)
    return np.mean(pick == car), np.mean(remaining == car)


print(f"{'doors':>5} {'stick sim':>10} {'stick exact':>12} "
      f"{'switch sim':>11} {'switch exact':>13}")
for k in [3, 4, 10, 100]:
    stick, switch = play(k, 200_000)
    print(f"{k:>5} {stick:10.4f} {1 / k:12.4f} {switch:11.4f} {1 - 1 / k:13.4f}")
```

Output

```
doors  stick sim  stick exact  switch sim  switch exact
    3     0.3343       0.3333      0.6657        0.6667
    4     0.2510       0.2500      0.7490        0.7500
   10     0.0999       0.1000      0.9001        0.9000
  100     0.0099       0.0100      0.9901        0.9900
```

△ Caution: Conditioning on the information mechanism

The 2/3 answer depends on the host's protocol. If the host opened a door *at random* and it happened to show a goat, the posterior would be 1/2. In data analysis the same issue appears as selection: what we observe depends on how data were generated or filtered (survey non-response, administrative records of programme applicants only). Always model the mechanism that produced the observation, not just the observation.

4.2.8 Preview: naive Bayes classification

Bayes' theorem with many predictors $\mathbf{x} = (x_1, \dots, x_d)$ requires the likelihood $\mathbb{P}(\mathbf{x} \mid y)$, whose number of parameters grows exponentially in d. The *naive Bayes* classifier assumes the features are conditionally independent given the class,

$$\mathbb{P}(y \mid \mathbf{x}) \propto \mathbb{P}(y) \prod_{j=1}^{d} \mathbb{P}(x_j \mid y),$$

so that only d one-dimensional conditionals per class must be estimated. The listing fits a naive Bayes poverty classifier to four binary deprivation indicators in a simulated survey by simple counting (with Laplace smoothing) and checks it against scikit-learn's `BernoulliNB` (Pedregosa et al., 2011).

Listing 4.6: A naive Bayes poverty classifier built from conditional frequencies
ch04/ch04_naive_bayes.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Naive Bayes poverty classifier built from counts (simulated survey).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```python
from sklearn.naive_bayes import BernoulliNB

rng = np.random.default_rng(803)
n = 20_000
poor = rng.random(n) < 0.3
names = ["thatch roof", "no phone", "no toilet", "size > 6"]
p_given_poor = np.array([0.55, 0.45, 0.60, 0.40])      # P(x_j = 1 | poor)
p_given_rich = np.array([0.15, 0.10, 0.20, 0.15])      # P(x_j = 1 | non-poor)
probs = np.where(poor[:, None], p_given_poor, p_given_rich)
X = (rng.random((n, 4)) < probs).astype(int)
train, test = slice(0, 15_000), slice(15_000, n)

# "Fitting" = estimating a prior and four conditional probabilities per class
yt, Xt = poor[train], X[train]
prior = yt.mean()
th1 = (Xt[yt].sum(0) + 1) / (yt.sum() + 2)             # Laplace smoothing
th0 = (Xt[~yt].sum(0) + 1) / ((~yt).sum() + 2)
print(f"Estimated prior P(poor) = {prior:.3f}")
for j, nm in enumerate(names):
    print(f"  P({nm:<11} | poor) = {th1[j]:.3f}   | non-poor) = {th0[j]:.3f}")


def posterior(Xn):
    l1 = np.log(prior) + (Xn * np.log(th1) + (1 - Xn) * np.log(1 - th1)).sum(1)
    l0 = np.log(1 - prior) + (Xn * np.log(th0)
                              + (1 - Xn) * np.log(1 - th0)).sum(1)
    return 1 / (1 + np.exp(l0 - l1))


post = posterior(X[test])
acc = np.mean((post > 0.5) == poor[test])
sk = BernoulliNB(alpha=1.0).fit(Xt, yt)
post_sk = sk.predict_proba(X[test])[:, 1]
print(f"\nTest accuracy (threshold 0.5)         = {acc:.3f}")
print(f"Max |ours - sklearn BernoulliNB|      = {np.max(abs(post - post_sk)):.2e}")
print("Posterior P(poor | all four deprivations) = "
      f"{posterior(np.ones((1, 4)))[0]:.3f}")
print("Posterior P(poor | none of them)          = "
      f"{posterior(np.zeros((1, 4)))[0]:.3f}")
```

Output

```
Estimated prior P(poor) = 0.306
  P(thatch roof | poor) = 0.555   | non-poor) = 0.151
  P(no phone    | poor) = 0.442   | non-poor) = 0.102
  P(no toilet   | poor) = 0.606   | non-poor) = 0.204
  P(size > 6    | poor) = 0.396   | non-poor) = 0.143

Test accuracy (threshold 0.5)         = 0.829
Max |ours - sklearn BernoulliNB|      = 3.33e-16
Posterior P(poor | all four deprivations) = 0.983
Posterior P(poor | none of them)          = 0.048
```

The hand-built posterior matches scikit-learn to machine precision: "training" naive Bayes *is* estimating a prior and class-conditional probabilities. Here the data were simulated to satisfy conditional independence, so the model is correctly specified; with real survey indicators (e.g.

roof material and toilet type are correlated even within poverty classes) naive Bayes double-counts evidence and its posteriors are typically overconfident, although its classifications can remain good (Murphy, 2022, Sec. 9.3).

Policy Lens: Conditional probabilities in survey design

Household surveys routinely use skip patterns: a module on health expenditure is asked only of households that report an illness. Estimates from the module are then conditional on the filter question, $\mathbb{P}(\cdot \mid \text{ill})$, and must be combined with $\mathbb{P}(\text{ill})$ through the law of total probability to obtain population quantities. Stratified designs similarly produce stratum-specific estimates $\mathbb{P}(A \mid B_h)$ that are weighted by population shares $\mathbb{P}(B_h)$—equation (4.5) with survey weights.

Exercises

4.2.1 Theory Prove that for fixed B with $\mathbb{P}(B) > 0$, $\mathbb{P}(\cdot \mid B)$ satisfies Kolmogorov's axioms. Deduce $\mathbb{P}(A \cup C \mid B) = \mathbb{P}(A \mid B) + \mathbb{P}(C \mid B) - \mathbb{P}(AC \mid B)$.

4.2.2 Theory Toss two fair coins. Let $A = \{\text{first is H}\}$, $B = \{\text{second is H}\}$, $C = \{\text{exactly one H}\}$. Show that A, B, C are pairwise independent but not mutually independent.

4.2.3 Theory Show that if $A \perp B$ then $A \perp B^c$, $A^c \perp B^c$. Construct events with $A \perp B \mid C$ but $A \not\perp B$, and events with $A \perp B$ but $A \not\perp B \mid C$ (explaining away).

4.2.4 Python Extend `ch04_malaria.py`: find the prevalence at which the PPV equals 0.5 as a function of specificity (with sensitivity fixed at 0.95), and plot it. Show analytically that this prevalence is $(1 - \text{spec})/(\text{sens} + 1 - \text{spec})$.

4.2.5 Python In `ch04_targeting.py`, vary the standard deviation of the PMT prediction error from 0 to 0.8 at 25% coverage. Plot inclusion and exclusion errors against it. What is the error at zero noise, and why?

4.2.6 Policy A PMT has sensitivity 0.70 and specificity 0.90. Compute the inclusion error in a district where the poverty rate is 50% and in one where it is 10%. A ministry proposes to use the same cut-off nationally. Discuss the implication for the composition of beneficiaries across districts.

4.2.7 ★ Graduate Generalise the odds form (4.7) to n conditionally independent pieces of evidence and show that the log posterior odds is the log prior odds plus a sum of log likelihood ratios. Relate this to logistic regression and explain why naive Bayes and logistic regression form a generative–discriminative pair (Ng and Jordan, 2002).

4.3 Random Variables and Distributions

4.3.1 Random variables as measurable functions

Data are numbers, but outcomes need not be: a sampled household is an $\omega \in \Omega$, and its consumption, size and poverty status are *numbers attached to* ω.

Definition 4.14 (Random variable). A *random variable* on $(\Omega, \mathcal{F}, \mathbb{P})$ is a function $X : \Omega \to \mathbb{R}$ such that

$$\{X \le x\} := \{\omega \in \Omega : X(\omega) \le x\} \in \mathcal{F} \qquad \text{for every } x \in \mathbb{R}. \tag{4.8}$$

Condition (4.8), called *measurability*, guarantees that every statement we want to make about X—"$X \le x$", "$a < X \le b$", "$X \in B$" for Borel B—is an event with a well-defined probability. The *distribution* (or law) of X is the probability measure $P_X(B) = \mathbb{P}(X \in B)$ on $(\mathbb{R}, \mathcal{B}(\mathbb{R}))$. Once P_X is known, the underlying Ω can usually be forgotten; this is what lets us say "let $X \sim \mathcal{N}(0,1)$" without specifying a sample space. Sums, products, continuous functions and limits of random variables are again random variables.

Graduate Perspective: Why measurability matters

In applied work non-measurable functions never arise by accident, so measurability may seem pedantic. It becomes essential in three places: (i) when defining conditional expectation $\mathbb{E}[Y \mid \mathcal{G}]$ as a $\mathcal{G}$-measurable random variable, the foundation of martingale theory and rational-expectations econometrics; (ii) in empirical-process theory, where the supremum over an uncountable class of functions (as in Theorem 4.20 below) need not be measurable and outer probabilities are used (van der Vaart, 1998); and (iii) in stochastic processes indexed by continuous time.

4.3.2 The cumulative distribution function

Definition 4.15 (CDF)**.** The *cumulative distribution function* of X is $F_X(x) = \mathbb{P}(X \le x)$, $x \in \mathbb{R}$.

Theorem 4.16 (Properties of a CDF)**.** *A function $F : \mathbb{R} \to [0,1]$ is the CDF of some random variable if and only if*

(i) *F is non-decreasing;*

(ii) *$\lim_{x \to -\infty} F(x) = 0$ and $\lim_{x \to \infty} F(x) = 1$;*

(iii) *F is right-continuous: $\lim_{h \downarrow 0} F(x+h) = F(x)$.*

Moreover $\mathbb{P}(a < X \le b) = F(b) - F(a)$ and $\mathbb{P}(X = x) = F(x) - F(x^-)$, the size of the jump at x.

Proof of necessity. (i) If $x < y$ then $\{X \le x\} \subseteq \{X \le y\}$; use monotonicity. (ii) $\{X \le -n\} \downarrow \varnothing$ and $\{X \le n\} \uparrow \Omega$; apply Theorem 4.7. (iii) $\{X \le x + 1/n\} \downarrow \{X \le x\}$; continuity from above. Similarly $\{X \le x - 1/n\} \uparrow \{X < x\}$, giving $F(x^-) = \mathbb{P}(X < x)$ and the jump formula. Sufficiency is proved by constructing $X = F^{-1}(U)$ with U uniform; see Theorem 4.32. $\square$

The CDF characterises the distribution completely: two random variables with the same CDF have the same law (they are *equal in distribution*, $X \overset{d}{=} Y$), even if they are different functions on Ω.

4.3.3 Discrete and continuous distributions

Definition 4.17 (PMF and PDF)**.** X is *discrete* if it takes values in a countable set $\{x_1, x_2, \dots\}$; its *probability mass function* is $p(x_i) = \mathbb{P}(X = x_i)$, with $\sum_i p(x_i) = 1$ and $F(x) = \sum_{x_i \le x} p(x_i)$. X is (absolutely) *continuous* if there is a function $f \ge 0$, the *probability density function*, with

$$F(x) = \int_{-\infty}^{x} f(t)\,dt \quad \text{for all } x, \qquad \text{so that} \qquad \mathbb{P}(X \in B) = \int_B f(t)\,dt. \tag{4.9}$$

At points where f is continuous, $f(x) = F'(x)$.

△ Caution: A density is not a probability

For continuous X, $\mathbb{P}(X = x) = 0$ for every x, and $f(x)$ can exceed 1 (the Uniform$(0, 0.1)$ density equals 10). The correct reading is $\mathbb{P}(x < X \le x + h) \approx f(x)\,h$ for small h. Densities also depend on the units of measurement: if income is measured in thousands rather than units of currency, the density values are multiplied by 1,000 (Section 4.5).

Familiar examples include the Bernoulli, binomial and Poisson PMFs and the uniform, exponential and normal densities; Chapter 5 studies these families systematically.

4.3.4 The quantile function

Policy statements are frequently phrased in terms of quantiles: "the poorest 20%", the median wage, the 5% value-at-risk of a portfolio.

Definition 4.18 (Quantile function)**.** The *quantile function* (generalised inverse) of a CDF F is

$$F^{-1}(p) = \inf\{x \in \mathbb{R} : F(x) \ge p\}, \qquad 0 < p < 1. \tag{4.10}$$

If F is continuous and strictly increasing, F^{-1} is the ordinary inverse. In general F^{-1} is non-decreasing and left-continuous, flat parts of F become jumps of F^{-1}, and jumps of F become flat parts of F^{-1}. The key property, used repeatedly below, is the *Galois inequality*

$$F^{-1}(p) \le x \iff p \le F(x) \qquad \text{for all } p \in (0,1),\ x \in \mathbb{R}. \tag{4.11}$$

(*Proof:* if $p \le F(x)$ then x belongs to the set in (4.10), so $F^{-1}(p) \le x$. Conversely, by right-continuity the infimum is attained, $F(F^{-1}(p)) \ge p$, so $F^{-1}(p) \le x$ implies $F(x) \ge F(F^{-1}(p)) \ge p$.)

4.3.5 The empirical CDF and the Glivenko–Cantelli theorem

Given data $X_1, \dots, X_n \overset{\text{iid}}{\sim} F$, the natural estimate of F puts mass $1/n$ on each observation.

Definition 4.19 (Empirical CDF)**.** $F_n(x) = \dfrac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{X_i \le x\}$.

For each fixed x, $nF_n(x) \sim \text{Binomial}(n, F(x))$, so $\mathbb{E}F_n(x) = F(x)$ and $\operatorname{Var} F_n(x) = F(x)(1 - F(x))/n$; by the law of large numbers $F_n(x) \to F(x)$. The remarkable fact is that convergence is *uniform* in x.

Theorem 4.20 (Glivenko–Cantelli)**.** *If $X_1, X_2, \dots \overset{iid}{\sim} F$, then*

$$D_n := \sup_{x \in \mathbb{R}} |F_n(x) - F(x)| \xrightarrow{a.s.} 0.$$

Proof sketch. For continuous F, fix k and choose points $x_j = F^{-1}(j/k)$, $j = 1, \dots, k-1$. Between consecutive grid points both F_n and F are monotone and F rises by $1/k$, so $D_n \le \max_j \left|F_n(x_j) - F(x_j)\right| + 1/k$. The maximum is over finitely many points, each converging almost surely by the strong law; hence $\limsup D_n \le 1/k$ a.s. for every k. □

The Dvoretzky–Kiefer–Wolfowitz (DKW) inequality, in Massart's sharp form, quantifies the rate: $\mathbb{P}(D_n > \varepsilon) \le 2e^{-2n\varepsilon^2}$. Setting the right side to α gives a distribution-free confidence band $F_n \pm \sqrt{\log(2/\alpha)/(2n)}$. Kolmogorov showed that $\sqrt{n}D_n$ has a limiting distribution that does not

depend on F (for continuous F); its use as a test statistic is the Kolmogorov–Smirnov test used throughout this chapter's code.

Listing 4.7: Empirical CDFs of simulated incomes and the sup-distance D_n ch04/ch04_ecdf.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Empirical CDF of simulated incomes and the Glivenko-Cantelli theorem.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
F = stats.lognorm(s=0.8, scale=np.exp(1.0))       # "true" income distribution


def ecdf(sample):
    xs = np.sort(sample)
    return xs, np.arange(1, xs.size + 1) / xs.size


def sup_distance(sample):
    """D_n = sup_x |F_n(x) - F(x)|, attained at the jump points."""
    xs, fn = ecdf(sample)
    Fx = F.cdf(xs)
    return max(np.max(fn - Fx), np.max(Fx - (fn - 1 / xs.size)))


print(f"{'n':>7} {'D_n':>8} {'sqrt(n) D_n':>12} {'DKW 95% band':>13}")
for n in [20, 200, 2_000, 20_000, 200_000]:
    d = sup_distance(F.rvs(size=n, random_state=rng))
    eps = np.sqrt(np.log(2 / 0.05) / (2 * n))      # Dvoretzky-Kiefer-Wolfowitz
    print(f"{n:>7} {d:8.4f} {np.sqrt(n) * d:12.3f} {eps:13.4f}")

x = F.rvs(size=200, random_state=rng)
print("\nQuantiles of a sample of n = 200 versus the true quantile function")
for p in [0.10, 0.25, 0.50, 0.75, 0.90]:
    print(f"  p = {p:.2f}: sample {np.quantile(x, p):6.2f}   true {F.ppf(p):6.2f}")
```

Output

```
      n      D_n  sqrt(n) D_n  DKW 95% band
     20   0.2532        1.132        0.3037
    200   0.0351        0.496        0.0960
   2000   0.0149        0.665        0.0304
  20000   0.0042        0.599        0.0096
 200000   0.0015        0.678        0.0030

Quantiles of a sample of n = 200 versus the true quantile function
  p = 0.10: sample   1.13   true   0.98
  p = 0.25: sample   1.71   true   1.58
  p = 0.50: sample   3.21   true   2.72
  p = 0.75: sample   5.16   true   4.66
  p = 0.90: sample   9.54   true   7.58
```

The column $\sqrt{n}\,D_n$ stays roughly constant while D_n shrinks, confirming the $1/\sqrt{n}$ rate, and

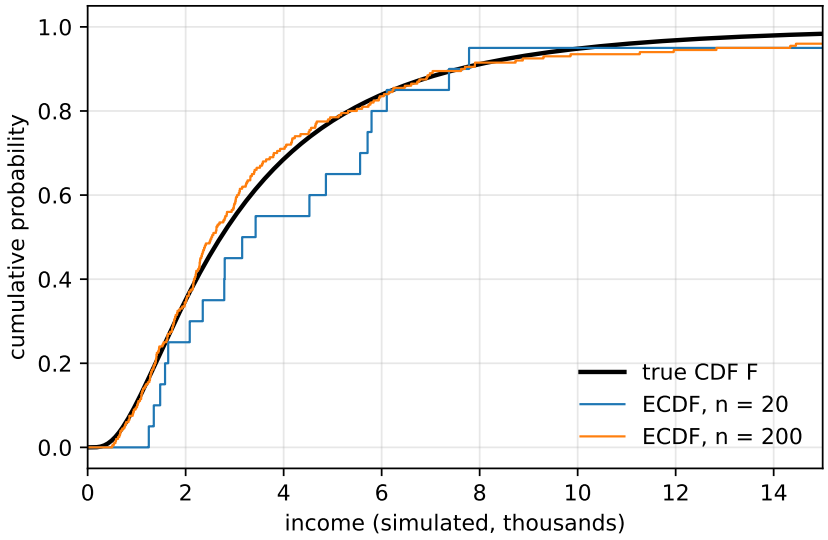


Figure 4.3: Empirical CDFs from simulated log-normal income samples of sizes 20 and 200 against the true CDF. The ECDF is a step function with jumps of $1/n$ at the data points.

every D_n lies within the DKW band. The sample quantiles for $n = 200$ are noticeably off in the upper tail: estimating the 90th percentile of a skewed income distribution from a small sample is hard, because few observations lie there.

Historical Note: 1933: a remarkable year

The uniform convergence of the empirical distribution function was proved for continuous F by Valery Glivenko and extended to arbitrary F by Francesco Paolo Cantelli, both in 1933 in the Italian *Giornale dell'Istituto Italiano degli Attuari*. In the same journal and the same year, Kolmogorov derived the limiting distribution of $\sqrt{n}D_n$; Nikolai Smirnov later developed the two-sample version and tables, giving the Kolmogorov–Smirnov test its name. 1933 was also the year of Kolmogorov's *Grundbegriffe*.

⋆ Key Concept: The ECDF is the data

The empirical distribution F_n is the nonparametric estimate of F, and many statistics are just functionals of it: the sample mean is $\int x\, dF_n(x)$, sample quantiles are $F_n^{-1}(p)$, and the bootstrap (Chapter 11) resamples from F_n. The Glivenko–Cantelli theorem is why "plugging in" F_n for F works in large samples.

4.3.6 Mixed distributions: zero-inflated expenditure

Not every random variable is discrete or continuous. In household surveys a large fraction of households report *zero* expenditure on health, education or remittances in the reference period, while positive amounts are continuously distributed. Let $\pi_0 = \mathbb{P}(X = 0)$ and let G be the continuous CDF of spending among spenders. Then

$$F(x) = \pi_0 \, \mathbf{1}\{x \geq 0\} + (1 - \pi_0)\, G(x), \tag{4.12}$$

a *mixture* of a point mass at zero and a continuous distribution. F has a jump of size π_0 at 0, so X has neither a PMF nor a density. By the Lebesgue decomposition, every CDF can be written as a mixture of a discrete, an absolutely continuous and a (pathological) singular part (Billingsley, 1995, Sec. 31).

Listing 4.8: A zero-inflated expenditure distribution: CDF, quantiles and moments
ch04/ch04_mixed.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# A mixed distribution: zero-inflated household health expenditure (simulated).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
pi0 = 0.40                                    # P(no health spending last month)
pos = stats.lognorm(s=1.0, scale=np.exp(3.0))  # spending given spending > 0


def cdf(x):
    """F(x) = pi0 * 1{x >= 0} + (1 - pi0) * G(x): a jump of size pi0 at zero."""
    x = np.asarray(x, dtype=float)
    return np.where(x < 0, 0.0, pi0 + (1 - pi0) * pos.cdf(np.maximum(x, 0)))


def quantile(p):
    """Generalised inverse F^{-1}(p) = inf{x : F(x) >= p}."""
    p = np.asarray(p, dtype=float)
    q = pos.ppf(np.clip((p - pi0) / (1 - pi0), 0, 1))
    return np.where(p <= pi0, 0.0, q)


n = 100_000
spend = np.where(rng.random(n) < pi0, 0.0, pos.rvs(size=n, random_state=rng))
print(f"P(X = 0): model {pi0:.3f}   sample {np.mean(spend == 0):.3f}")
print(f"P(X <= 50): model {cdf(50.0):.3f}   sample {np.mean(spend <= 50):.3f}")
mean_model = (1 - pi0) * pos.mean()
print(f"E[X]:  model {mean_model:7.2f}   sample {spend.mean():7.2f}")
m_pos = spend[spend > 0].mean()
print(f"E[X | X > 0]: model {pos.mean():7.2f}   sample {m_pos:7.2f}")
print("\n  p   quantile F^-1(p)   sample quantile")
for p in [0.25, 0.40, 0.50, 0.75, 0.95]:
    print(f"{p:4.2f}   {quantile(p):12.2f}   {np.quantile(spend, p):12.2f}")
print("\nNo density exists: the point x = 0 carries positive probability.")
```

Output

```
P(X = 0): model 0.400   sample 0.402
P(X <= 50): model 0.891   sample 0.890
E[X]:  model   19.87   sample   19.81
E[X | X > 0]: model   33.12   sample   33.12

  p   quantile F^-1(p)   sample quantile
0.25           0.00           0.00
0.40           0.00           0.00
0.50           7.63           7.66
0.75          24.79          24.75
0.95          80.08          80.54

No density exists: the point x = 0 carries positive probability.
```

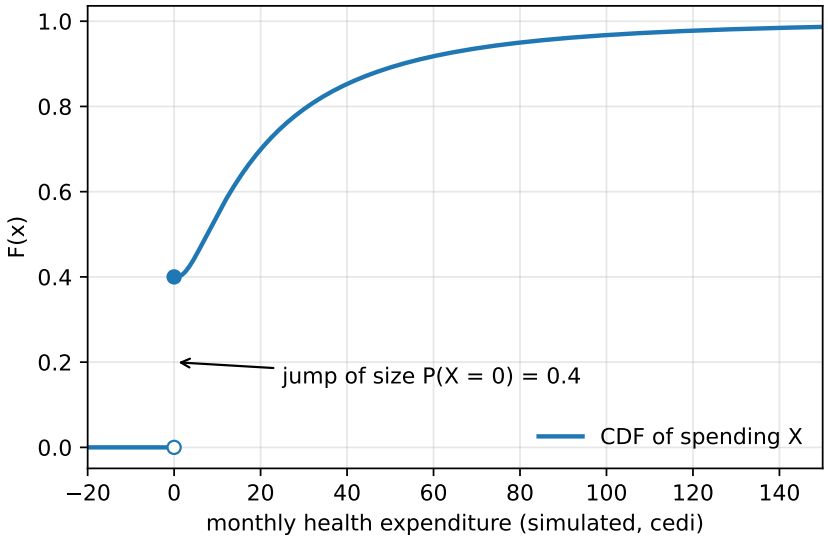


Figure 4.4: CDF of simulated monthly health expenditure with 40% zeros. The jump at zero is the point mass $\mathbb{P}(X = 0)$; the generalised inverse maps every $p \le 0.4$ to zero.

The quantile function (4.10) handles the jump automatically: every quantile up to the 40th percentile is zero, and the median spending is about 7.6 (simulated currency units) although mean spending is about 20. The two conditional means—overall and among spenders—answer different policy questions (average burden on all households versus burden on those who use services), and confusing them is a frequent error in reports.

Policy Lens: Zeros are information

Treating zero health spending as "missing" or taking logs after dropping zeros discards the *extensive margin*: whether households use care at all. Catastrophic-expenditure indicators, two-part models and hurdle models (Wooldridge, 2010, Ch. 17) are all built on the mixed distribution (4.12). A fee-exemption policy may act mainly on π_0 (more people seek care) rather than on G (what users spend).

Exercises

4.3.1 Theory Prove that if X is a random variable and $g : \mathbb{R} \to \mathbb{R}$ is non-decreasing, then $g(X)$ is a random variable. (Hint: show $\{g(X) \le y\}$ is of the form $\{X \le c\}$ or $\{X < c\}$.)

4.3.2 Theory Let F be a CDF. Prove that F has at most countably many jumps. (Hint: how many jumps of size greater than $1/k$ can there be?)

4.3.3 Theory Prove the Galois inequality (4.11) in detail, including the claim that the infimum in (4.10) is attained.

4.3.4 Python Write a function `ecdf_band(x, alpha)` that returns the ECDF and the DKW band. By simulation from a log-normal, estimate the coverage of the 95% band for $n = 50$ and $n = 500$. Is the band conservative?

4.3.5 Python For the zero-inflated model of `ch04_mixed.py`, derive $\mathrm{Var}(X)$ analytically in terms of π_0 and the moments of G, and check it against the simulation.

4.3.6 ★ Graduate Show that $\sqrt{n}\,(F_n(x) - F(x)) \xrightarrow{d} \mathcal{N}(0, F(x)(1 - F(x)))$ for fixed x, and compute the limiting covariance of $\sqrt{n}(F_n(x) - F(x))$ and $\sqrt{n}(F_n(y) - F(y))$ for $x < y$. Explain why the limit process is a Brownian bridge composed with F.

4.3.7 Policy A health ministry reports that "mean out-of-pocket spending rose by 10%". Using the decomposition $\mathbb{E}X = (1-\pi_0)\mathbb{E}[X \mid X>0]$, describe two very different scenarios consistent with this statement and the policy conclusions each would suggest.

4.4 Joint, Marginal and Conditional Distributions

Economic questions are almost always about several variables at once: how consumption moves with income, whether poverty is concentrated in large households, how shocks to different crops co-move. A *random vector* $\mathbf{X} = (X_1, \dots, X_d)^\top$ is a vector of random variables on the same probability space; its *joint CDF* is $F(x_1, \dots, x_d) = \mathbb{P}(X_1 \le x_1, \dots, X_d \le x_d)$.

4.4.1 Discrete joint distributions

For discrete (X, Y) the *joint PMF* is $p(x,y) = \mathbb{P}(X=x, Y=y)$. The *marginal* PMFs are obtained by summing out the other variable, and the *conditional* PMF by (4.3):

$$p_X(x) = \sum_y p(x,y), \qquad p_{Y\mid X}(y \mid x) = \frac{p(x,y)}{p_X(x)} \quad (p_X(x)>0). \tag{4.13}$$

The listing below works with an illustrative joint PMF of household-size class and poverty status (the numbers are invented for teaching purposes).

Listing 4.9: Joint, marginal and conditional PMFs of household size and poverty
ch04/ch04_joint_discrete.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Joint, marginal and conditional PMFs: household size and poverty status.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import pandas as pd

rng = np.random.default_rng(803)
sizes = ["1-2", "3-4", "5-6", "7+"]
# Illustrative joint PMF p(x, y): rows = size class, columns = (non-poor, poor)
P = np.array([[0.150, 0.020],
              [0.260, 0.060],
              [0.200, 0.110],
              [0.090, 0.110]])
assert np.isclose(P.sum(), 1.0)
joint = pd.DataFrame(P, index=sizes, columns=["non-poor", "poor"])
px, py = P.sum(axis=1), P.sum(axis=0)        # marginals
print("Joint PMF with marginals")
tab = joint.copy()
tab["P(size)"] = px
print(tab.round(3).to_string())
print(f"P(poor) = {py[1]:.3f}\n")

cond = pd.DataFrame({"P(poor | size)": P[:, 1] / px,
                     "P(size | poor)": P[:, 1] / py[1]}, index=sizes)
print(cond.round(3).to_string())
indep = np.outer(px, py)
print(f"\nmax |p(x,y) - p(x)p(y)| = {np.max(abs(P - indep)):.3f}  -> dependent")

# Sampling from the joint PMF and recovering it by cross-tabulation
n = 50_000
cells = rng.choice(P.size, size=n, p=P.ravel())
```

```python
size_i, poor_i = np.unravel_index(cells, P.shape)
ct = pd.crosstab(np.array(sizes)[size_i], poor_i, normalize=True)
print(f"max |empirical - true| over 8 cells (n = {n}) = "
      f"{np.max(abs(ct.values - P)):.4f}")
```

Output

```
Joint PMF with marginals
     non-poor  poor  P(size)
1-2      0.15  0.02     0.17
3-4      0.26  0.06     0.32
5-6      0.20  0.11     0.31
7+       0.09  0.11     0.20
P(poor) = 0.300

     P(poor | size)  P(size | poor)
1-2           0.118           0.067
3-4           0.188           0.200
5-6           0.355           0.367
7+            0.550           0.367

max |p(x,y) - p(x)p(y)| = 0.050  -> dependent
max |empirical - true| over 8 cells (n = 50000) = 0.0034
```

The two conditional columns answer different questions. $\mathbb{P}(\text{poor} \mid \text{size } 7+) = 0.55$ is the poverty *rate* among large households (a targeting question); $\mathbb{P}(\text{size } 7+ \mid \text{poor}) = 0.367$ is the *composition* of the poor (a programme design question: what share of beneficiaries would be large households). Both are needed, and they are linked by Bayes' theorem. The final line checks that sampling from the joint PMF and cross-tabulating recovers it, an instance of Glivenko–Cantelli in a finite space.

4.4.2 Continuous joint distributions

(X, Y) is jointly continuous with density $f(x, y) \geq 0$ if $\mathbb{P}((X, Y) \in A) = \iint_A f(x, y)\, dx\, dy$ for Borel $A \subseteq \mathbb{R}^2$. Then

$$f_X(x) = \int_{-\infty}^{\infty} f(x, y)\, dy, \qquad f_{Y|X}(y \mid x) = \frac{f(x, y)}{f_X(x)} \quad (f_X(x) > 0). \tag{4.14}$$

The conditional density is defined even though $\mathbb{P}(X = x) = 0$: it is the limit of $\mathbb{P}(Y \leq y \mid x < X \leq x + h)$ as $h \downarrow 0$, differentiated in y. Equivalently, $f(x, y) = f_X(x) f_{Y|X}(y \mid x)$, the continuous chain rule, and $f_X(x \mid y) \propto f_{Y|X}(y \mid x) f_X(x)$, the continuous Bayes' theorem that underlies all of Bayesian inference (Gelman et al., 2013).

△ Caution: Conditioning on a probability-zero event

Because $\mathbb{P}(X = x) = 0$, the conditional density $f_{Y|X}(\cdot \mid x)$ is only determined up to sets of x of probability zero, and it depends on *how* we approach the event. The Borel–Kolmogorov paradox shows that conditioning a uniform point on a sphere on lying on a great circle gives different answers depending on whether the circle is described as a meridian or as the equator. In practice: condition on random *variables*, not on isolated events, and specify the conditioning variable explicitly.

4.4.3 Independence of random variables

Definition 4.21 (Independent random variables)**.** X and Y are independent if $\mathbb{P}(X \in A, Y \in B) = \mathbb{P}(X \in A)\mathbb{P}(Y \in B)$ for all Borel A, B. Equivalently, $F(x, y) = F_X(x)F_Y(y)$ for all x, y; for discrete variables $p(x, y) = p_X(x)p_Y(y)$; for continuous variables $f(x, y) = f_X(x)f_Y(y)$ (almost everywhere).

A useful criterion: $X \perp Y$ if and only if the joint density factorises as $f(x, y) = g(x)h(y)$ on a *product* support. If $g(X)$ and $h(Y)$ are functions of independent variables, they are independent too. In the household-size table, $\max \left| p(x, y) - p_X(x)p_Y(y) \right| = 0.05$, so size and poverty are dependent.

⋆ Key Concept: Joint, marginal, conditional

The joint distribution determines everything: marginals by summing or integrating out, conditionals by dividing joint by marginal ($f_{Y|X} = f / f_X$), and independence by checking whether the joint factorises. The converse fails: marginals alone do *not* determine the joint distribution—the missing ingredient is the dependence structure (the copula).

4.4.4 Covariance: a preview

The simplest summary of dependence is the *covariance* $\operatorname{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$ and its scale-free version, the correlation $\operatorname{Corr}(X, Y) = \operatorname{Cov}(X, Y) / \sqrt{\operatorname{Var} X \operatorname{Var} Y} \in [-1, 1]$. Independence implies zero covariance (when the moments exist), but not conversely: if $X \sim \mathcal{N}(0, 1)$ and $Y = X^2$, then $\operatorname{Cov}(X, Y) = \mathbb{E}X^3 = 0$ although Y is a function of X. Covariance and correlation are treated fully in Chapter 6; here we use them descriptively.

4.4.5 Simulating income and consumption

A workhorse model in welfare analysis treats log income Y_1 and log consumption Y_2 as bivariate normal, $(Y_1, Y_2)^\top \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$. Completing the square in the bivariate normal density shows that the conditional distribution is again normal:

$$Y_2 \mid Y_1 = y \;\sim\; \mathcal{N}\!\left(\mu_2 + \rho\frac{\sigma_2}{\sigma_1}(y - \mu_1),\; \sigma_2^2(1 - \rho^2)\right). \tag{4.15}$$

The conditional mean is linear in y with slope $\rho\sigma_2/\sigma_1$—in this model, the income elasticity of consumption—and the conditional variance does not depend on y.

Listing 4.10: Simulated bivariate normal log income and log consumption ch04/ch04_bivariate.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Simulated bivariate distribution of log income and log consumption.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
mu = np.array([7.0, 6.8])                    # means of (log income, log cons.)
s1, s2, rho = 0.70, 0.50, 0.80
Sigma = np.array([[s1**2, rho * s1 * s2],
                  [rho * s1 * s2, s2**2]])
n = 5_000
Y = rng.multivariate_normal(mu, Sigma, size=n)
```

```
y1, y2 = Y[:, 0], Y[:, 1]

print("Sample moments of (log income, log consumption)")
print(f"  means       {y1.mean():.3f}  {y2.mean():.3f}   (true 7.000  6.800)")
print(f"  covariance  {np.cov(y1, y2)[0, 1]:.4f}           "
      f"(true {Sigma[0, 1]:.4f})")
print(f"  correlation {np.corrcoef(y1, y2)[0, 1]:.4f}           (true {rho:.4f})")
inc, con = np.exp(y1), np.exp(y2)
print(f"  corr. of levels exp(Y1), exp(Y2): {np.corrcoef(inc, con)[0, 1]:.4f}")

# Conditional distribution of Y2 given Y1 = y: normal, linear mean
slope = rho * s2 / s1
csd = s2 * np.sqrt(1 - rho**2)
print(f"\nE[Y2 | Y1 = y] = {mu[1]:.2f} + {slope:.4f} (y - {mu[0]:.2f}),"
      f"  sd(Y2 | Y1) = {csd:.4f}")
print(f"{'income band':>14} {'mean Y2 (sim)':>14} {'theory':>8} {'sd (sim)':>9}")
for lo in [6.0, 6.5, 7.0, 7.5]:
    m = (y1 >= lo) & (y1 < lo + 0.1)
    centre = y1[m].mean()
    print(f"[{lo:.1f}, {lo + 0.1:.1f}) {y2[m].mean():14.3f} "
          f"{mu[1] + slope * (centre - mu[0]):8.3f} {y2[m].std():9.3f}")
```

Output

```
Sample moments of (log income, log consumption)
  means       7.013  6.810   (true 7.000  6.800)
  covariance  0.2751           (true 0.2800)
  correlation 0.8011           (true 0.8000)
  corr. of levels exp(Y1), exp(Y2): 0.7720

E[Y2 | Y1 = y] = 6.80 + 0.5714 (y - 7.00),  sd(Y2 | Y1) = 0.3000
   income band  mean Y2 (sim)   theory  sd (sim)
[6.0, 6.1)          6.243    6.258     0.305
[6.5, 6.6)          6.563    6.545     0.332
[7.0, 7.1)          6.844    6.830     0.304
[7.5, 7.6)          7.098    7.112     0.299
```

The binned conditional means and standard deviations match (4.15). Note that the correlation of the *levels* e^{Y_1} and e^{Y_2} (0.77) differs from the correlation of the logs (0.80): Pearson correlation is not invariant under nonlinear monotone transformations. This motivates copulas.

4.4.6 Copulas and Sklar's theorem

A joint distribution mixes two kinds of information: the behaviour of each variable on its own (the marginals) and the way they move together (the dependence). Copulas separate the two.

Definition 4.22 (Copula)**.** A d-dimensional *copula* is a joint CDF C on $[0,1]^d$ whose marginals are all Uniform$(0,1)$.

Theorem 4.23 (Sklar's theorem)**.** *Let F be a joint CDF with marginals $F_1, \dots, F_d$. There exists a copula C such that*

$$F(x_1, \dots, x_d) = C\big(F_1(x_1), \dots, F_d(x_d)\big) \qquad \text{for all } \mathbf{x} \in \mathbb{R}^d. \tag{4.16}$$

If the marginals are continuous, C is unique. Conversely, for any copula C and any univariate CDFs F_j,

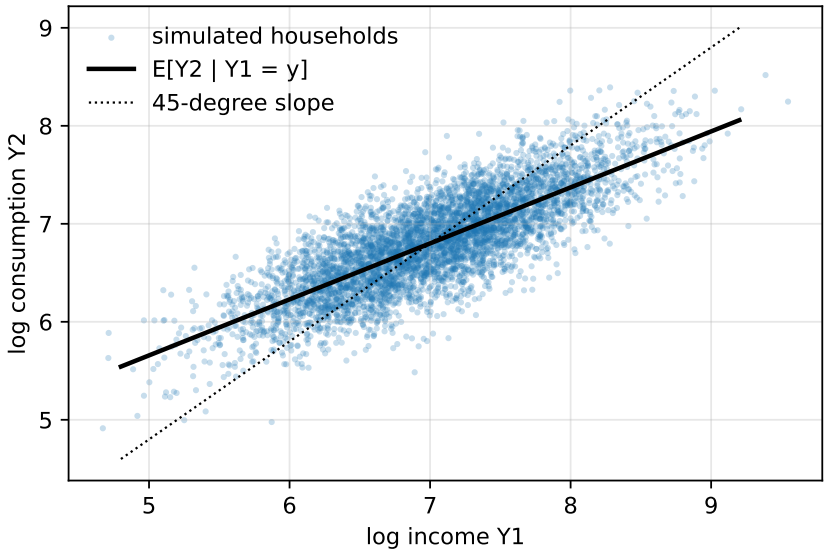


Figure 4.5: Simulated log income and log consumption for 5,000 households with the conditional mean line (4.15). The slope $0.57 < 1$ reflects consumption smoothing: consumption varies less than income.

(4.16) *defines a valid joint CDF.*

Proof sketch for continuous marginals. Let $U_j = F_j(X_j)$. By the probability integral transform (Theorem 4.31), each $U_j \sim \text{Uniform}(0,1)$, so the joint CDF C of $(U_1, \ldots, U_d)$ is a copula. Since F_j is continuous, $\{X_j \le x_j\} = \{U_j \le F_j(x_j)\}$ up to null sets, giving (4.16). For the converse, take $\mathbf{U} \sim C$ and set $X_j = F_j^{-1}(U_j)$; by Theorem 4.32, $X_j \sim F_j$. □

The converse is a recipe for simulation: draw from a copula, then push each coordinate through the desired marginal quantile function. The *Gaussian copula* takes C to be the copula of a multivariate normal with correlation matrix R. Rank correlations (Spearman's ρ_S, Kendall's τ) depend only on the copula, so they are invariant to monotone transformations of the margins; for the Gaussian copula $\rho_S = (6/\pi)\arcsin(r/2)$ and $\tau = (2/\pi)\arcsin r$.

Listing 4.11: A Gaussian copula with log-normal and exponential marginals ch04/ch04_copula.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Gaussian copula: same dependence, different marginals (Sklar's theorem).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n, r = 4_000, 0.7
Z = rng.multivariate_normal([0, 0], [[1, r], [r, 1]], size=n)
U = stats.norm.cdf(Z)                          # copula sample: uniform marginals
income = stats.lognorm(s=0.8, scale=np.exp(1.0)).ppf(U[:, 0])
loss = stats.expon(scale=2.0).ppf(U[:, 1])  # e.g. shock-related losses

print("Marginal checks (KS p-values against Uniform(0,1)):")
for j in range(2):
    print(f"  U{j + 1}: p = {stats.kstest(U[:, j], 'uniform').pvalue:.3f}")
rows = [("normal scores Z", Z[:, 0], Z[:, 1]),
```

```
        ("copula scale U", U[:, 0], U[:, 1]),
        ("income, loss", income, loss)]
print(f"\n{'scale':<17} {'Pearson':>8} {'Spearman':>9} {'Kendall':>8}")
for name, a, b in rows:
    pr = np.corrcoef(a, b)[0, 1]
    sp = stats.spearmanr(a, b).correlation
    kt = stats.kendalltau(a, b).correlation
    print(f"{name:<17} {pr:8.3f} {sp:9.3f} {kt:8.3f}")
print(f"\nTheory: Spearman = (6/pi) asin(r/2) = {6 / np.pi * np.arcsin(r / 2):.3f},"
      f" Kendall = (2/pi) asin(r) = {2 / np.pi * np.arcsin(r):.3f}")
```

Output

```
Marginal checks (KS p-values against Uniform(0,1)):
  U1: p = 0.082
  U2: p = 0.283

scale              Pearson  Spearman  Kendall
normal scores Z      0.698     0.686    0.495
copula scale U       0.685     0.686    0.495
income, loss         0.648     0.686    0.495

Theory: Spearman = (6/pi) asin(r/2) = 0.683, Kendall = (2/pi) asin(r) = 0.494
```

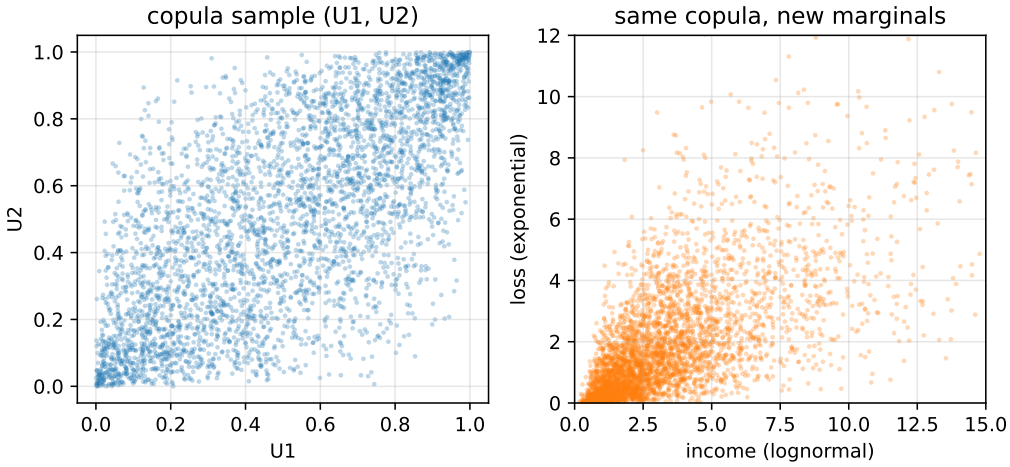


Figure 4.6: Left: a sample from the Gaussian copula with $r = 0.7$. Right: the same uniforms pushed through log-normal (income) and exponential (loss) quantile functions. The dependence structure, and hence the rank correlations, are unchanged.

The Spearman and Kendall columns are identical on all three scales, whereas Pearson's correlation changes with the marginals.

Graduate Perspective: Copulas in risk and policy modelling

Copulas let modellers combine well-understood marginals (fitted separately, perhaps from different data sources) with a dependence structure, a common strategy in insurance, credit-risk and climate-impact modelling, and in synthetic-data generation for microsimulation. The Gaussian copula has zero *tail dependence*: extreme events in one margin become asymptotically independent of extremes in the other. Its use for pricing portfolios of correlated mortgage

defaults before 2008 is a cautionary tale about this property (MacKenzie and Spears, 2014). Student-t, Clayton and Gumbel copulas allow joint extremes; Nelsen (2006) is the standard monograph.

Historical Note: Sklar and the name "copula"

Abe Sklar introduced copulas, and proved the theorem that bears his name, in a short 1959 note in the *Publications de l'Institut de Statistique de l'Université de Paris*, on a problem that Maurice Fréchet and others had studied: how a multivariate distribution is related to its one-dimensional margins (Sklar, 1959). The Latin word *copula* means a link or tie. The concept remained a specialist tool until the 1990s, when it was taken up widely in finance and actuarial science.

Exercises

4.4.1 Theory Let (X, Y) have density $f(x, y) = 2$ on the triangle $0 < x < y < 1$. Find the marginal densities and $f_{Y|X}(y \mid x)$. Are X and Y independent? Explain why the factorisation criterion fails even though f is constant.

4.4.2 Theory Derive (4.15) by completing the square in the exponent of the bivariate normal density.

4.4.3 Theory Show that $M(u, v) = \min(u, v)$, $W(u, v) = \max(u + v - 1, 0)$ and $\Pi(u, v) = uv$ are copulas. Which random vectors (U, V) do they describe? (These are the Fréchet–Hoeffding bounds and the independence copula.)

4.4.4 Python Using `ch04_joint_discrete.py`, compute the poverty rate implied by drawing a household at random and then a person at random within it, assuming the household-size classes contain on average 1.5, 3.5, 5.5 and 8 persons. Why does the person-level poverty rate exceed the household-level rate?

4.4.5 Python Replace the Gaussian copula in `ch04_copula.py` by a Student-t copula with 3 degrees of freedom and the same r. Estimate $\mathbb{P}(U_2 > 0.99 \mid U_1 > 0.99)$ for both copulas and comment.

4.4.6 ★ Graduate Prove that Kendall's τ depends only on the copula: $\tau = 4 \int_{[0,1]^2} C(u, v)\, dC(u, v) - 1$.

4.4.7 Policy In a simulated model, droughts reduce both crop income and livestock income. Explain, with reference to tail dependence, why an insurance scheme calibrated with a Gaussian copula fitted to normal years could under-estimate the probability that both income sources collapse at once, and what this means for the reserves a government should hold.

4.5 Transformations of Random Variables

Data analysis is full of transformations: logs of incomes, ratios such as budget shares, totals across household members, the largest flood in thirty years. If X has a known distribution and $Y = g(X)$, what is the distribution of Y? Three general tools answer the question: the CDF method, the change-of-variables (Jacobian) formula, and convolution.

4.5.1 The CDF method

The most general approach writes the event $\{Y \le y\}$ in terms of X:

$$F_Y(y) = \mathbb{P}(g(X) \le y) = \mathbb{P}\big(X \in g^{-1}((-\infty, y])\big), \tag{4.17}$$

and then differentiates if a density is wanted.

Example 4.24 (Square of a standard normal)**.** Let $Z \sim \mathcal{N}(0,1)$ and $Y = Z^2$. For $y > 0$, $F_Y(y) = \mathbb{P}(-\sqrt{y} \le Z \le \sqrt{y}) = 2\Phi(\sqrt{y}) - 1$, so $f_Y(y) = \phi(\sqrt{y})/\sqrt{y} = (2\pi y)^{-1/2} e^{-y/2}$, the χ^2_1 density. Note that $g(z) = z^2$ is not one-to-one; the CDF method handles this without difficulty.

4.5.2 Change of variables in one dimension

Theorem 4.25 (Univariate change of variables)**.** *Let X have density f_X, supported on an open interval I, and let $g : I \to \mathbb{R}$ be strictly monotone and continuously differentiable with $g' \neq 0$ on I. Then $Y = g(X)$ has density*

$$f_Y(y) = f_X\big(g^{-1}(y)\big)\left|\frac{d}{dy} g^{-1}(y)\right| = \frac{f_X(x)}{\big|g'(x)\big|}\bigg|_{x = g^{-1}(y)} \qquad y \in g(I), \tag{4.18}$$

and $f_Y(y) = 0$ otherwise.

Proof. Suppose first g is increasing. Then $\{g(X) \le y\} = \{X \le g^{-1}(y)\}$, so $F_Y(y) = F_X(g^{-1}(y))$. Differentiating with the chain rule gives $f_Y(y) = f_X(g^{-1}(y))\,(g^{-1})'(y)$, and $(g^{-1})'(y) = 1/g'(g^{-1}(y)) > 0$. If g is decreasing, $\{g(X) \le y\} = \{X \ge g^{-1}(y)\}$, so $F_Y(y) = 1 - F_X(g^{-1}(y))$ and $f_Y(y) = -f_X(g^{-1}(y))(g^{-1})'(y)$, which is positive because $(g^{-1})' < 0$. Both cases are captured by the absolute value. □

The factor $\big|(g^{-1})'(y)\big|$ is the local stretching of the axis: probability is conserved, $f_Y(y)\,\big|dy\big| = f_X(x)\,|dx|$, so where g stretches intervals the density thins out.

Example 4.26 (Linear transformations)**.** If $Y = a + bX$ with $b \neq 0$, then $f_Y(y) = f_X((y-a)/b)/|b|$. In particular, $X \sim \mathcal{N}(\mu, \sigma^2)$ iff $(X - \mu)/\sigma \sim \mathcal{N}(0,1)$, and changing income units from cedis to thousands of cedis ($b = 10^{-3}$) multiplies density values by 10^3.

4.5.3 Log-normal incomes

If log income $X = \log Y \sim \mathcal{N}(\mu, \sigma^2)$, then income $Y = e^X$ is *log-normal*. With $g(x) = e^x$, $g^{-1}(y) = \log y$ and $(g^{-1})'(y) = 1/y$, (4.18) gives

$$f_Y(y) = \frac{1}{y\,\sigma\sqrt{2\pi}} \exp\left\{-\frac{(\log y - \mu)^2}{2\sigma^2}\right\}, \qquad y > 0. \tag{4.19}$$

Quantiles transform directly, because g is increasing: the median of Y is e^{μ}. Means do *not*: $\mathbb{E}Y = e^{\mu + \sigma^2/2} > e^{\mathbb{E}X}$ (a consequence of Jensen's inequality, Chapter 6), $\operatorname{Var} Y = (e^{\sigma^2} - 1)e^{2\mu + \sigma^2}$, and the mode is $e^{\mu - \sigma^2}$, so mode < median < mean: the right skew typical of income data.

Listing 4.12: Log-normal incomes: change-of-variables density versus simulation
ch04/ch04_lognormal.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Change of variables: log-normal incomes from normal log-incomes.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
mu, sigma = 1.0, 0.8                          # parameters of log income X
X = rng.normal(mu, sigma, 200_000)
Y = np.exp(X)                                 # income Y = g(X) = exp(X)
```

```python

def f_Y(y):
    """f_Y(y) = f_X(log y) |d log y / dy| = f_X(log y) / y."""
    return stats.norm.pdf(np.log(y), mu, sigma) / y


print("Income Y = exp(X), X ~ N(1.0, 0.8^2)")
print(f"{'quantity':<12} {'formula':>9} {'simulated':>10}")
rows = [("mean", np.exp(mu + sigma**2 / 2), Y.mean()),
        ("median", np.exp(mu), np.median(Y)),
        ("variance", (np.exp(sigma**2) - 1) * np.exp(2 * mu + sigma**2), Y.var()),
        ("P(Y <= 2)", stats.norm.cdf((np.log(2) - mu) / sigma), np.mean(Y <= 2))]
for name, a, b in rows:
    print(f"{name:<12} {a:9.4f} {b:10.4f}")
print(f"mode exp(mu - sigma^2) = {np.exp(mu - sigma**2):.4f}"
      "  < median < mean (right skew)")
# The density formula integrates to one and agrees with scipy.stats.lognorm
g = np.linspace(1e-4, 80, 400_001)
print(f"integral of f_Y over (0, 80)        = {np.trapz(f_Y(g), g):.6f}")
ln = stats.lognorm(s=sigma, scale=np.exp(mu))
print(f"max |f_Y - scipy lognorm.pdf|       = {np.max(abs(f_Y(g) - ln.pdf(g))):.2e}")
```

Output

```
Income Y = exp(X), X ~ N(1.0, 0.8^2)
quantity       formula  simulated
mean            3.7434     3.7454
median          2.7183     2.7165
variance       12.5626    12.4680
P(Y <= 2)       0.3507     0.3509
mode exp(mu - sigma^2) = 1.4333  < median < mean (right skew)
integral of f_Y over (0, 80)        = 0.999988
max |f_Y - scipy lognorm.pdf|       = 1.94e-16
```

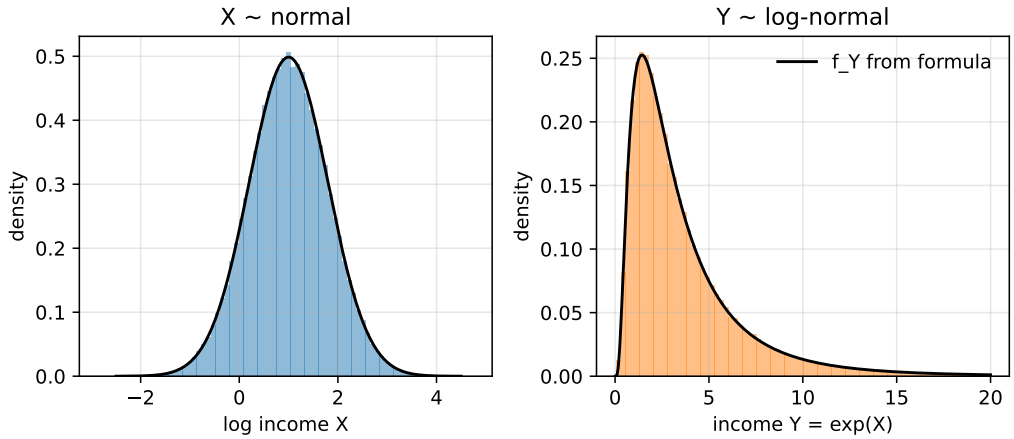


Figure 4.7: Left: simulated normal log incomes. Right: the implied incomes $Y = e^X$ (truncated at 20 for display) with the density (4.19). A symmetric distribution on the log scale becomes strongly right-skewed on the level scale.

△ Caution: Retransformation bias

If a regression is estimated for $\log Y$ and the fitted value $\widehat{\log Y}$ is simply exponentiated, the result estimates the conditional *median* of Y, not its mean. Under log-normal errors the mean requires the factor $e^{\sigma^2/2}$; with $\sigma = 0.8$ this is $e^{0.32} \approx 1.38$, a 38% difference. Poverty-mapping and microsimulation models that predict log consumption must therefore simulate, or correct, rather than exponentiate point predictions.

4.5.4 Multivariate change of variables

Theorem 4.27 (Jacobian formula)**.** *Let* $\mathbf{X}$ *have density* $f_{\mathbf{X}}$ *on an open set* $A \subseteq \mathbb{R}^d$*, and let* $\mathbf{g} : A \to B$ *be a continuously differentiable bijection onto an open set* B *with differentiable inverse* $\mathbf{h} = \mathbf{g}^{-1}$*. Then* $\mathbf{Y} = \mathbf{g}(\mathbf{X})$ *has density*

$$f_{\mathbf{Y}}(\mathbf{y}) = f_{\mathbf{X}}\big(\mathbf{h}(\mathbf{y})\big)\,\left|\det J_{\mathbf{h}}(\mathbf{y})\right|, \qquad J_{\mathbf{h}}(\mathbf{y}) = \left[\frac{\partial h_i}{\partial y_j}(\mathbf{y})\right]_{i,j=1}^{d}, \quad \mathbf{y} \in B. \tag{4.20}$$

Proof sketch. For a Borel set $D \subseteq B$, $\mathbb{P}(\mathbf{Y} \in D) = \mathbb{P}(\mathbf{X} \in \mathbf{h}(D)) = \int_{\mathbf{h}(D)} f_{\mathbf{X}}(\mathbf{x})\,d\mathbf{x}$. The multivariate substitution rule of calculus, $\mathbf{x} = \mathbf{h}(\mathbf{y})$, replaces $d\mathbf{x}$ by $|\det J_{\mathbf{h}}(\mathbf{y})|\,d\mathbf{y}$, giving $\int_D f_{\mathbf{X}}(\mathbf{h}(\mathbf{y}))\,|\det J_{\mathbf{h}}(\mathbf{y})|\,d\mathbf{y}$. Since this holds for all D, the integrand is the density of $\mathbf{Y}$. □

The determinant measures how $\mathbf{h}$ scales volumes (Chapter 2). For an affine map $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$ with invertible $\mathbf{A}$, $f_{\mathbf{Y}}(\mathbf{y}) = f_{\mathbf{X}}(\mathbf{A}^{-1}(\mathbf{y}-\mathbf{b}))/|\det \mathbf{A}|$; applied to $\mathbf{X} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ this yields the multivariate normal density with covariance $\mathbf{A}\mathbf{A}^{\top}$, the basis of simulating correlated normals via a Cholesky factor.

Example 4.28 (Household total and earner share)**.** Two earners have independent incomes $X_1, X_2 \overset{\text{iid}}{\sim} \text{Exp}(\lambda)$. Let $T = X_1 + X_2$ (household income) and $S = X_1/T$ (share of earner 1). The inverse map is $x_1 = st$, $x_2 = (1-s)t$, with

$$J_{\mathbf{h}} = \begin{pmatrix} \partial x_1/\partial t & \partial x_1/\partial s \\ \partial x_2/\partial t & \partial x_2/\partial s \end{pmatrix} = \begin{pmatrix} s & t \\ 1-s & -t \end{pmatrix}, \qquad |\det J_{\mathbf{h}}| = t.$$

Hence $f_{T,S}(t,s) = \lambda^2 e^{-\lambda t} \cdot t = \left[\lambda^2 t e^{-\lambda t}\right] \cdot 1$ on $(0,\infty)\times(0,1)$. The density factorises on a product set, so $T \sim \text{Gamma}(2,\lambda)$, $S \sim \text{Uniform}(0,1)$, and—surprisingly—*the share is independent of the total*.

Listing 4.13: Verifying the Jacobian calculation for total and share ch04/ch04_jacobian.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Multivariate change of variables: total and share of two exponential incomes.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
lam, n = 0.5, 200_000
X1 = rng.exponential(1 / lam, n)            # income of earner 1
X2 = rng.exponential(1 / lam, n)            # income of earner 2
T = X1 + X2                                 # household total
S = X1 / T                                  # share of earner 1

# Jacobian method: (T, S) has density lam^2 t exp(-lam t) * 1 on (0,inf)x(0,1)
# so T ~ Gamma(2, 1/lam), S ~ Uniform(0, 1) and T, S are independent.
```

```python
print("Kolmogorov-Smirnov p-values")
p_T = stats.kstest(T, "gamma", (2, 0, 2)).pvalue
print(f"  T against Gamma(2, scale=2): {p_T:.3f}")
print(f"  S against Uniform(0, 1)    : {stats.kstest(S, 'uniform').pvalue:.3f}")
print(f"\ncorr(T, S)   = {np.corrcoef(T, S)[0, 1]: .4f}")
print(f"corr(T, S^2) = {np.corrcoef(T, S**2)[0, 1]: .4f}")
# Independence check on rectangles: P(T<=t, S<=s) vs P(T<=t)P(S<=s)
print(f"\n{'t':>4} {'s':>5} {'joint':>8} {'product':>8}")
for t, s in [(2, 0.25), (4, 0.5), (8, 0.9)]:
    joint = np.mean((T <= t) & (S <= s))
    print(f"{t:>4} {s:5.2f} {joint:8.4f} {np.mean(T <= t) * np.mean(S <= s):8.4f}")
```

Output

```
Kolmogorov-Smirnov p-values
  T against Gamma(2, scale=2): 0.978
  S against Uniform(0, 1)    : 0.115

corr(T, S)   =  0.0005
corr(T, S^2) =  0.0001

   t     s    joint  product
   2  0.25   0.0661   0.0661
   4  0.50   0.2975   0.2976
   8  0.90   0.8162   0.8160
```

4.5.5 Sums of independent variables and convolution

Totals—household income summed over members, district output summed over firms, aggregate claims on an insurance pool—are sums. If $X \perp Y$ have densities f_X and f_Y, set $S = X + Y$. Conditioning on X,

$$f_S(s) = \int_{-\infty}^{\infty} f_X(x)\, f_Y(s - x)\, dx =: (f_X * f_Y)(s), \tag{4.21}$$

the *convolution* of the densities. (Proof: apply Theorem 4.27 to $(X, Y) \mapsto (X, X + Y)$, whose Jacobian determinant is 1, then integrate out X.) For integer valued variables the analogue is $p_S(s) = \sum_k p_X(k) p_Y(s - k)$. Convolution is commutative and associative, so the density of $X_1 + \cdots + X_n$ is the n-fold convolution power.

Example 4.29 (Sums of exponentials)**.** If $X_1, X_2 \overset{\text{iid}}{\sim} \text{Exp}(\lambda)$, then for $s > 0$, $f_S(s) = \int_0^s \lambda e^{-\lambda x} \lambda e^{-\lambda(s-x)} dx = \lambda^2 s e^{-\lambda s}$, in agreement with Example 4.28. By induction the sum of k independent $\text{Exp}(\lambda)$ variables is $\text{Gamma}(k, \lambda)$, with density $\lambda^k s^{k-1} e^{-\lambda s}/(k-1)!$. Likewise sums of independent normals are normal and sums of independent Poissons are Poisson (Chapter 5).

Listing 4.14: Numerical convolution of densities and PMFs ch04/ch04_convolution.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Sums of independent variables: numerical convolution versus simulation.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

```

```
rng = np.random.default_rng(803)
h = 0.01
x = np.arange(0, 30, h)
f_exp = stats.expon(scale=2.0).pdf(x)        # waiting time, mean 2

# Density of S_k = sum of k iid Exp(mean 2) by repeated discrete convolution
dens = {1: f_exp}
for k in range(2, 6):
    dens[k] = np.convolve(dens[k - 1], f_exp)[: x.size] * h
print("Sum of k iid exponentials (mean 2) is Gamma(k, scale 2)")
print(f"{'k':>2} {'max|conv - gamma|':>18} {'P(S>10) conv':>13} {'P(S>10) sim':>12}")
for k in range(1, 6):
    err = np.max(abs(dens[k] - stats.gamma(k, scale=2.0).pdf(x)))
    tail = dens[k][x > 10].sum() * h
    sim = np.mean(rng.exponential(2.0, (200_000, k)).sum(1) > 10)
    print(f"{k:>2} {err:18.4f} {tail:13.4f} {sim:12.4f}")

# Discrete convolution: total of two dice
die = np.full(6, 1 / 6)
two = np.convolve(die, die)
print("\nPMF of the sum of two dice (values 2..12):")
print("  " + " ".join(f"{36 * p:.0f}" for p in two) + "   (times 1/36)")
```

Output

```
Sum of k iid exponentials (mean 2) is Gamma(k, scale 2)
 k  max|conv - gamma|  P(S>10) conv  P(S>10) sim
 1             0.0000        0.0067       0.0069
 2             0.0025        0.0404       0.0406
 3             0.0014        0.1247       0.1250
 4             0.0014        0.2657       0.2665
 5             0.0014        0.4425       0.4394

PMF of the sum of two dice (values 2..12):
  1 2 3 4 5 6 5 4 3 2 1   (times 1/36)
```

The discrete convolution with `np.convolve` reproduces the gamma densities to within the grid error, and the dice example gives the familiar triangular PMF $1, 2, \ldots, 6, \ldots, 1$ over 36. Figure 4.8 shows the shape converging towards a normal curve; the central limit theorem (Chapter 6) makes this precise, and characteristic functions—Fourier transforms, which turn convolutions into products—are the standard tool for proving it.

⋆ Key Concept: Three ways to transform a distribution

(1) The CDF method (4.17) always works: express $\{g(X) \le y\}$ in terms of X. **(2)** For smooth bijections, multiply the old density by the absolute Jacobian determinant of the *inverse* map (4.20). **(3)** For sums of independent variables, convolve (4.21). In all three, probability is conserved; only its spread over the new axis changes.

4.5.6 Order statistics: minima and maxima

Let $X_1, \ldots, X_n \overset{\text{iid}}{\sim} F$ and write the sorted sample as $X_{(1)} \le \cdots \le X_{(n)}$. Extremes matter in policy: the first default in a loan portfolio, the worst drought in a planning horizon, the largest claim on a disaster fund.

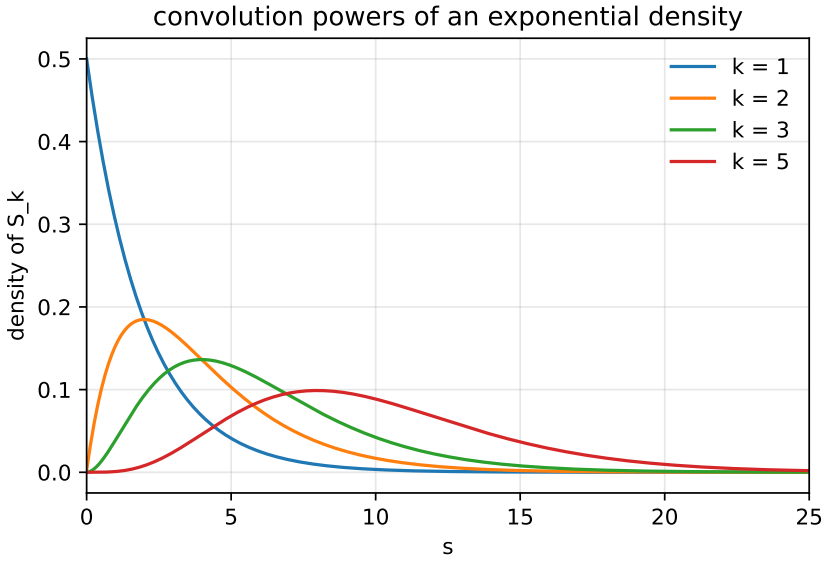


Figure 4.8: Convolution powers of the exponential density (mean 2), computed numerically. As k grows the density of the sum becomes more symmetric and bell-shaped, a first glimpse of the central limit theorem.

Proposition 4.30 (Distribution of the maximum and minimum)**.**

$$F_{X_{(n)}}(x) = F(x)^n, \qquad F_{X_{(1)}}(x) = 1 - \big(1 - F(x)\big)^n. \tag{4.22}$$

More generally, $\mathbb{P}(X_{(k)} \le x) = \sum_{j=k}^{n} \binom{n}{j} F(x)^j (1 - F(x))^{n-j}$.

Proof. $\{X_{(n)} \le x\} = \bigcap_i \{X_i \le x\}$ and $\{X_{(1)} > x\} = \bigcap_i \{X_i > x\}$; use independence. For $X_{(k)}$, $X_{(k)} \le x$ iff at least k of the X_i are $\le x$, and the number that are is Binomial$(n, F(x))$. □

For Exp(λ) variables, $1 - F_{X_{(1)}}(x) = e^{-n\lambda x}$, so the minimum is Exp$(n\lambda)$: the waiting time to the first of n independent defaults shrinks like $1/n$. For uniforms, $X_{(n)}$ has CDF x^n and mean $n/(n+1)$.

Listing 4.15: Minima and maxima of iid samples ch04/ch04_order_stats.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Order statistics: minimum and maximum of n iid variables.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
reps = 100_000

# Minimum of n iid Exp(rate lam) is Exp(rate n lam): first of n loans to default
lam = 0.1                                       # default hazard per month
print("Months until the FIRST default among n loans, each Exp(rate 0.1)")
print(f"{'n':>4} {'E[min] sim':>11} {'1/(n lam)':>10}")
for n in [1, 5, 20, 100]:
    m = rng.exponential(1 / lam, (reps, n)).min(axis=1)
    print(f"{n:>4} {m.mean():11.3f} {1 / (n * lam):10.3f}")

# Maximum of n iid U(0,1): CDF x^n, mean n/(n+1)
```

```
print("\nMaximum of n iid Uniform(0,1): P(max <= 0.9) = 0.9^n")
print(f"{'n':>4} {'sim':>8} {'exact':>8} {'E[max] sim':>11} {'n/(n+1)':>8}")
for n in [2, 10, 50]:
    M = rng.random((reps, n)).max(axis=1)
    print(f"{n:>4} {np.mean(M <= 0.9):8.4f} {0.9**n:8.4f} {M.mean():11.4f}"
          f" {n / (n + 1):8.4f}")

# Worst rainfall shortfall over a 30-year horizon (annual maxima, N(0,1))
n = 30
Mx = rng.standard_normal((reps, n)).max(axis=1)
print(f"\nMax of 30 iid N(0,1): mean {Mx.mean():.3f}, "
      f"P(max > 2) = {np.mean(Mx > 2):.3f}  (exact 1 - Phi(2)^30 = "
      f"{1 - stats.norm.cdf(2) ** 30:.3f})")
```

Output

```
Months until the FIRST default among n loans, each Exp(rate 0.1)
   n  E[min] sim  1/(n lam)
   1       9.999     10.000
   5       2.000      2.000
  20       0.498      0.500
 100       0.100      0.100

Maximum of n iid Uniform(0,1): P(max <= 0.9) = 0.9^n
   n      sim    exact  E[max] sim  n/(n+1)
   2   0.8121   0.8100      0.6651   0.6667
  10   0.3487   0.3487      0.9091   0.9091
  50   0.0054   0.0052      0.9803   0.9804

Max of 30 iid N(0,1): mean 2.042, P(max > 2) = 0.498  (exact 1 - Phi(2)^30 = 0.499)
```

The last line is sobering: if annual rainfall anomalies were iid standard normal, a two-sigma event would occur at least once in a 30-year horizon with probability about one half. An event that is "rare" in any given year is close to certain over the lifetime of a dam or a pension fund.

4.5.7 The probability integral transform

Theorem 4.31 (Probability integral transform)**.** *If X has a continuous CDF F, then $U = F(X) \sim \text{Uniform}(0,1)$.*

Proof. For $u \in (0,1)$, by (4.11), $\mathbb{P}(F(X) < u) = \mathbb{P}(X < F^{-1}(u)) = F(F^{-1}(u)^-)$. Since F is continuous, this equals $F(F^{-1}(u)) = u$ (continuity and right-continuity give $F(F^{-1}(u)) = u$). Hence $\mathbb{P}(U < u) = u$, and as $\mathbb{P}(U = u) = 0$ for continuous F also $\mathbb{P}(U \le u) = u$. □

The theorem fails for discrete F, since then $F(X)$ takes only finitely or countably many values. The PIT has two major uses. First, it underlies the Kolmogorov–Smirnov test and the copula construction in Theorem 4.23. Second, it is the standard tool for checking *density forecasts*: if a forecaster issues predictive CDFs F_t and the outcomes are y_t, then the PIT values $u_t = F_t(y_t)$ should look iid uniform when the forecasts are correct (Diebold et al., 1998).

Listing 4.16: PIT histograms for calibrated and miscalibrated density forecasts ch04/ch04_pit.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Probability integral transform: calibrated and miscalibrated forecasts.
```

```python
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 5_000
# Outcomes (e.g. monthly inflation surprises) drawn from a Student-t, df = 5
y = stats.t(df=5).rvs(size=n, random_state=rng)
models = {"correct: t(5)": stats.t(df=5),
          "too narrow: N(0, 0.8^2)": stats.norm(0, 0.8),
          "too wide: N(0, 2^2)": stats.norm(0, 2.0)}
print(f"{'forecast model':<24} {'KS p-value':>11} {'P(PIT<0.05)':>12}"
      f" {'P(PIT>0.95)':>12}")
pits = {}
for name, m in models.items():
    u = m.cdf(y)                              # PIT values U = F(Y)
    pits[name] = u
    p = stats.kstest(u, "uniform").pvalue
    print(f"{name:<24} {p:11.3g} {np.mean(u < 0.05):12.3f}"
          f" {np.mean(u > 0.95):12.3f}")
print("A calibrated forecast puts 5% of PIT values in each tail.")
```

Output

```
forecast model            KS p-value  P(PIT<0.05)  P(PIT>0.95)
correct: t(5)                  0.482        0.045        0.051
too narrow: N(0, 0.8^2)     6.62e-28        0.122        0.129
too wide: N(0, 2^2)         8.19e-81        0.008        0.012
A calibrated forecast puts 5% of PIT values in each tail.
```

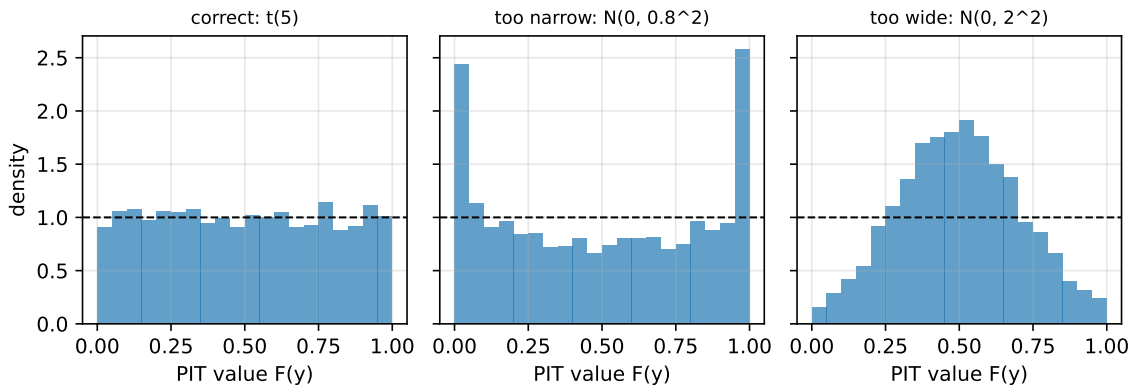


Figure 4.9: PIT histograms for outcomes drawn from a Student-t_5 distribution. Correct forecast: flat. Too narrow (over-confident) forecast: U-shaped, too many outcomes in the tails. Too wide: hump-shaped.

Policy Lens: Are the fan charts calibrated?

Several central banks publish inflation fan charts, i.e. density forecasts. A U-shaped PIT histogram computed from past fan charts and realised inflation signals that the bands were

too narrow—the bank was over-confident about its forecasts—while a hump shape signals excessive caution. Evaluating the calibration of probabilistic forecasts in this way is part of the forecast-evaluation toolkit used in the machine-learning forecasting literature reviewed by Goulet Coulombe et al. (2022).

Exercises

4.5.1 Theory Let $X \sim \text{Uniform}(0,1)$. Find the density of $Y = -\log X$ and of $Y = X^2$ using Theorem 4.25, and verify your answers with the CDF method.

4.5.2 Theory Let $X, Y \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$. Use Theorem 4.27 with polar coordinates to prove that $R^2 = X^2 + Y^2$ (exponential with mean 2) and the angle $\Theta \sim \text{Uniform}(0, 2\pi)$ are independent. (This is the Box–Muller theorem of Section 4.6.)

4.5.3 Theory Derive the density of $X_{(k)}$ for an iid sample from a continuous density f: $f_{(k)}(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1}(1-F(x))^{n-k} f(x)$. Show that for uniforms $X_{(k)} \sim \text{Beta}(k, n-k+1)$.

4.5.4 Python Using `np.convolve`, compute the PMF of the total number of sixes in $n = 1, \dots, 30$ rolls by repeatedly convolving the Bernoulli(1/6) PMF, and compare with `scipy.stats.binom`. Then compute the PMF of the sum of 10 dice and plot it.

4.5.5 Python Simulate the retransformation bias: generate $\log Y = 1 + 0.5x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.8^2)$, fit the regression of $\log Y$ on x by least squares, and compare the mean of $e^{\widehat{\log Y}}$ with the mean of Y, with and without the factor $e^{\hat{\sigma}^2/2}$.

4.5.6 ★ Graduate Let $\mathbf{X} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ and let $\mathbf{g}$ be a composition of invertible maps with tractable Jacobians. Explain how (4.20) gives the log-likelihood of a *normalising flow*, and why flow architectures are designed so that $\det J$ is triangular.

4.5.7 Policy A disaster-risk fund covers 40 districts. Suppose the annual loss in each district is independent Exp with mean 1 (simulated units). Compute the distribution of (a) the total loss and (b) the largest district loss. Which quantity should determine the fund's reserves, and how would positive dependence between districts change your answer?

4.6 Generating Random Variables

Every simulation in this chapter began with a call such as `rng.random(n)`. This section explains what such a call does and how uniform numbers are turned into draws from any target distribution. The material is the foundation for Monte Carlo methods in Chapter 10 and resampling in Chapter 11.

4.6.1 Pseudo-random number generators

A computer is deterministic, so it cannot produce truly random numbers without an external physical source. A *pseudo-random number generator* (PRNG) is a deterministic recursion

$$s_{k+1} = T(s_k), \qquad u_k = G(s_k) \in [0,1),$$

on a finite state space, started from a *seed* s_0. Because the state space is finite the sequence is eventually periodic; a good generator has an astronomically long period and outputs that pass stringent statistical tests of uniformity and independence (for instance the TestU01 batteries of L'Ecuyer and Simard, 2007).

Middle-square. John von Neumann's early method squares a $2m$-digit number and keeps the middle $2m$ digits. It is easy to understand and easy to break: sequences quickly fall into short cycles or collapse to zero.

Linear congruential generators (LCGs). $x_{k+1} = (ax_k + c) \bmod m$, $u_k = x_k/m$. With well-chosen constants LCGs are fast and have period up to m, but their successive tuples lie on a small number of hyperplanes. The infamous RANDU generator ($a = 65539$, $c = 0$, $m = 2^{31}$), widely distributed in the 1960s and 1970s, satisfies $x_{k+2} = 6x_{k+1} - 9x_k \pmod{2^{31}}$ because $65539^2 = (2^{16} + 3)^2 \equiv 6 \cdot 65539 - 9 \pmod{2^{31}}$. Consequently $9u_k - 6u_{k+1} + u_{k+2}$ is always an integer, and all triples (u_k, u_{k+1}, u_{k+2}) lie on just 15 parallel planes in the unit cube.

Modern generators. The Mersenne Twister MT19937 (Matsumoto and Nishimura, 1998) has period $2^{19937} - 1$ and was the default in Python's `random` module and in legacy NumPy. Since version 1.17, NumPy's recommended interface is `np.random.default_rng`, which returns a `Generator` backed by `PCG64`, a member of O'Neill's permuted congruential generator family (O'Neill, 2014): a 128-bit LCG whose output is scrambled by a permutation function, combining speed, a small state, period 2^{128} and excellent statistical quality.

Listing 4.17: Middle-square, the RANDU defect, and reproducible streams with PCG64
ch04/ch04_prng.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Pseudo-random number generators: middle-square, RANDU, and NumPy's PCG64.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt


def middle_square(x, steps):
    """von Neumann: square a 4-digit number and keep the middle 4 digits."""
    out = [x]
    for _ in range(steps):
        x = (x * x // 100) % 10_000
        out.append(x)
    return out


seq = middle_square(1234, 60)
first_repeat = next(i for i, v in enumerate(seq) if v in seq[:i])
print("Middle-square from 1234:", " ".join(f"{v:04d}" for v in seq[:10]), "...")
print(f"  value {seq[first_repeat]:04d} recurs at step {first_repeat}: "
      "the sequence has fallen into a short cycle")

# RANDU: x_{k+1} = 65539 x_k mod 2^31  (widely used in the 1960s-70s)
m = 2**31
x = np.empty(30_002, dtype=np.int64)
x[0] = 1
for k in range(1, x.size):
    x[k] = (65539 * x[k - 1]) % m
lhs = (x[2:] - 6 * x[1:-1] + 9 * x[:-2]) % m
print(f"\nRANDU: x[k+2] - 6 x[k+1] + 9 x[k] = 0 (mod 2^31) for all k? "
      f"{bool(np.all(lhs == 0))}")
u = x / m
c_randu = 9 * u[:-2] - 6 * u[1:-1] + u[2:]

```

```python
# Modern practice: one seeded Generator (PCG64), independent child streams
rng = np.random.default_rng(803)
print(f"\nbit generator: {type(rng.bit_generator).__name__}")
a = np.random.default_rng(803).random(3)
b = np.random.default_rng(803).random(3)
print("same seed, same stream:", np.round(a, 6), np.array_equal(a, b))
children = np.random.SeedSequence(803).spawn(3)
streams = [np.random.default_rng(s) for s in children]
print("spawned streams, first draw each:",
      np.round([g.random() for g in streams], 6))
v = rng.random(30_002)
c_pcg = 9 * v[:-2] - 6 * v[1:-1] + v[2:]
```

Output

```
Middle-square from 1234: 1234 5227 3215 3362 3030 1809 2724 4201 6484 0422 ...
  value 0000 recurs at step 57: the sequence has fallen into a short cycle

RANDU: x[k+2] - 6 x[k+1] + 9 x[k] = 0 (mod 2^31) for all k? True

bit generator: PCG64
same seed, same stream: [0.545206 0.507696 0.392677] True
spawned streams, first draw each: [0.160771 0.296841 0.682341]
```

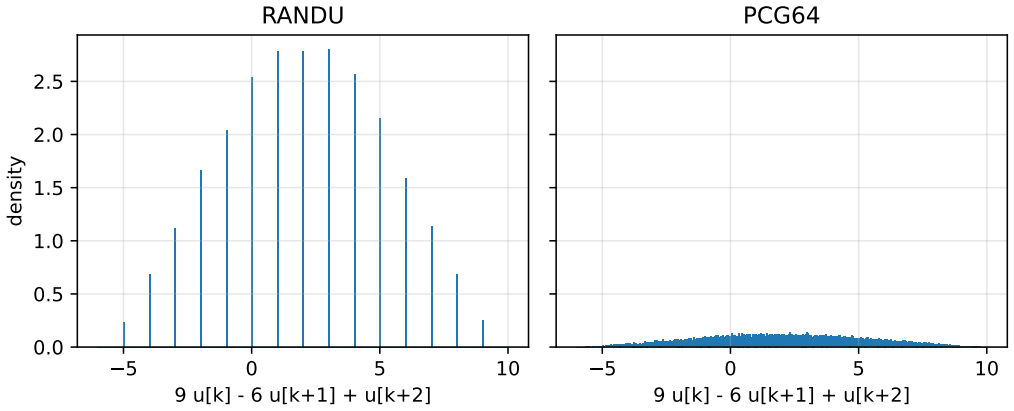


Figure 4.10: Histogram of $9u_k - 6u_{k+1} + u_{k+2}$. For RANDU all mass sits on integers—the 15 planes—while for PCG64 the combination has a smooth density.

Historical Note: A brief history of pseudo-randomness

John von Neumann proposed the middle-square method in the second half of the 1940s, in connection with the first electronic Monte Carlo computations; his remarks to a 1949 symposium were published in 1951 in a National Bureau of Standards volume, together with a famous warning that arithmetical procedures can never produce *truly* random digits (von Neumann, 1951). D. H. Lehmer proposed the multiplicative congruential method at the same 1949 symposium. The lattice structure of RANDU and other LCGs was exposed by George Marsaglia's 1968 paper "Random numbers fall mainly in the planes" (Marsaglia, 1968). Makoto Matsumoto and Takuji Nishimura published the Mersenne Twister in 1998, and Melissa O'Neill introduced the PCG family in a 2014 technical report.

4.6.2 Seeds and reproducibility

The seed determines the entire stream. Re-running `np.random.default_rng(803)` reproduces exactly the same numbers, which is what makes every output in this book reproducible. Good practice:

- create *one* `Generator` at the top of a script and pass it to functions, rather than reseeding repeatedly or relying on hidden global state (`np.random.seed`);
- for parallel or independent simulations use `np.random.SeedSequence(seed).spawn(k)`, which derives statistically independent child streams;
- record the seed, the library versions and the generator in published replication files.

△ Caution: Do not seed-hack

Re-running an analysis with different seeds until a bootstrap confidence interval excludes zero, or until a randomised assignment "looks balanced", invalidates the probability statements the simulation was meant to support. Fix the seed *before* looking at results, report it, and check that conclusions are stable across a few seeds. If they are not, increase the number of replications.

4.6.3 Inverse transform sampling

Given uniforms, how do we draw from an arbitrary distribution F? The most fundamental method inverts the CDF.

Theorem 4.32 (Inverse transform)**.** *Let F be any CDF with generalised inverse F^{-1} (4.10), and let $U \sim \text{Uniform}(0,1)$. Then $X = F^{-1}(U)$ has CDF F.*

Proof. By the Galois inequality (4.11), $\{F^{-1}(U) \le x\} = \{U \le F(x)\}$ for every x. Hence $\mathbb{P}(X \le x) = \mathbb{P}(U \le F(x)) = F(x)$. □

The proof is two lines, but the theorem is completely general: it covers continuous, discrete and mixed distributions alike, and it proves the sufficiency part of Theorem 4.16.

Example 4.33 (Three inverse-transform samplers)**.** (a) *Exponential.* $F(x) = 1 - e^{-\lambda x}$ gives $F^{-1}(u) = -\log(1-u)/\lambda$.

(b) *Pareto top incomes.* $F(x) = 1 - (x_m/x)^a$ for $x \ge x_m$ gives $F^{-1}(u) = x_m(1-u)^{-1/a}$. Pareto tails are the standard model of top incomes and wealth.

(c) *Discrete.* For values $v_1 < \cdots < v_K$ with cumulative probabilities $c_k = \sum_{j \le k} p_j$, $F^{-1}(u) = v_k$ where k is the smallest index with $c_k \ge u$; `np.searchsorted` finds k for a whole vector of u at once.

Listing 4.18: Inverse transform sampling for exponential, Pareto and discrete targets
`ch04/ch04_inverse.py`

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Inverse transform sampling: exponential, Pareto and a discrete distribution.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
n = 500_000
U = rng.random(n)
```

```python

# Exponential with rate lam: F^{-1}(u) = -log(1 - u) / lam
lam = 0.25
E = -np.log1p(-U) / lam
print(f"Exponential(rate 0.25): mean {E.mean():.3f} (4.000), "
      f"var {E.var():.3f} (16.000)")

# Pareto top incomes: F(x) = 1 - (xm/x)^a for x >= xm, F^{-1}(u) = xm (1-u)^(-1/a)
xm, a = 50.0, 2.5
Pa = xm * (1 - U) ** (-1 / a)
print(f"Pareto(xm=50, a=2.5):   mean {Pa.mean():.2f} ({a * xm / (a - 1):.2f}), "
      f"P(X > 200) {np.mean(Pa > 200):.4f} ({(xm / 200) ** a:.4f})")
print(f"  KS p-value vs scipy pareto: "
      f"{stats.kstest(Pa[:20_000], 'pareto', (a, 0, xm)).pvalue:.3f}")

# Discrete: household size with an arbitrary PMF, via the generalised inverse
size = np.arange(1, 9)
pmf = np.array([0.10, 0.14, 0.17, 0.18, 0.15, 0.11, 0.08, 0.07])
cdf = np.cumsum(pmf)
idx = np.searchsorted(cdf, U, side="right").clip(max=size.size - 1)
draw = size[idx]
freq = np.bincount(draw, minlength=9)[1:] / n
print("\nHousehold size  " + " ".join(f"{s:>6d}" for s in size))
print("target PMF      " + " ".join(f"{p:6.3f}" for p in pmf))
print("sample freq.    " + " ".join(f"{f:6.3f}" for f in freq))
print(f"mean size: target {np.dot(size, pmf):.3f}, sample {draw.mean():.3f}")
```

Output

```
Exponential(rate 0.25): mean 3.997 (4.000), var 15.993 (16.000)
Pareto(xm=50, a=2.5):   mean 83.33 (83.33), P(X > 200) 0.0311 (0.0312)
  KS p-value vs scipy pareto: 0.140

Household size       1      2      3      4      5      6      7      8
target PMF       0.100  0.140  0.170  0.180  0.150  0.110  0.080  0.070
sample freq.     0.100  0.140  0.170  0.180  0.150  0.110  0.080  0.070
mean size: target 4.140, sample 4.139
```

⋆ Key Concept: Uniforms are universal

Any distribution on $\mathbb{R}$ can be simulated from a single Uniform$(0,1)$ draw via $F^{-1}(U)$, and the probability integral transform runs the map backwards. Together, Theorems 4.31 and 4.32 say that all continuous distributions are "the same" up to a monotone change of scale—the idea behind copulas, quantile regression and the Kolmogorov–Smirnov test.

4.6.4 The Box–Muller transform

The normal CDF has no closed-form inverse, so the inverse transform requires numerical approximation of Φ^{-1}. Box and Muller (1958) found an exact alternative.

Theorem 4.34 (Box–Muller). *Let* $U_1, U_2 \overset{iid}{\sim} \text{Uniform}(0,1)$, $R = \sqrt{-2\log U_1}$ *and* $\Theta = 2\pi U_2$. *Then* $Z_1 = R\cos\Theta$ *and* $Z_2 = R\sin\Theta$ *are independent* $\mathcal{N}(0,1)$.

Proof. By the inverse transform, $R^2 = -2\log U_1$ is exponential with mean 2, i.e. has density

$\frac{1}{2}e^{-r^2/2}$ in the variable r^2, and $\Theta \sim \text{Uniform}(0, 2\pi)$ independently. In polar coordinates, the pair (R, Θ) has density $re^{-r^2/2} \cdot \frac{1}{2\pi}$ on $(0, \infty) \times (0, 2\pi)$. The map $(r, \theta) \mapsto (z_1, z_2) = (r\cos\theta, r\sin\theta)$ has inverse Jacobian determinant $1/r$, so by Theorem 4.27

$$f_{Z_1,Z_2}(z_1, z_2) = \frac{1}{2\pi} e^{-(z_1^2+z_2^2)/2} = \phi(z_1)\,\phi(z_2),$$

which factorises into two standard normal densities. □

Listing 4.19: The Box–Muller transform and moment checks ch04/ch04_boxmuller.py

```python
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Box-Muller transform: two uniforms in, two independent standard normals out.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 500_000
U1, U2 = rng.random(n), rng.random(n)
R = np.sqrt(-2 * np.log1p(-U1))              # radius: R^2 ~ Exp(mean 2)
theta = 2 * np.pi * U2                       # angle ~ Uniform(0, 2 pi)
Z1, Z2 = R * np.cos(theta), R * np.sin(theta)

print(f"{'statistic':<20} {'Z1':>8} {'Z2':>8} {'N(0,1)':>8}")
for name, f, t in [("mean", np.mean, 0.0), ("variance", np.var, 1.0),
                   ("skewness", stats.skew, 0.0),
                   ("excess kurtosis", stats.kurtosis, 0.0)]:
    print(f"{name:<20} {f(Z1):8.4f} {f(Z2):8.4f} {t:8.4f}")
print(f"corr(Z1, Z2)         {np.corrcoef(Z1, Z2)[0, 1]:8.4f}")
print(f"corr(Z1^2, Z2^2)     {np.corrcoef(Z1**2, Z2**2)[0, 1]:8.4f}")
print(f"KS p-value Z1: {stats.kstest(Z1[:20_000], 'norm').pvalue:.3f}, "
      f"Z2: {stats.kstest(Z2[:20_000], 'norm').pvalue:.3f}")
print(f"P(|Z1| > 1.96) = {np.mean(abs(Z1) > 1.96):.4f}  (0.0500)")
```

Output

```
statistic                 Z1       Z2   N(0,1)
mean                  0.0009  -0.0001   0.0000
variance              0.9993   0.9993   1.0000
skewness             -0.0025  -0.0004   0.0000
excess kurtosis       0.0004  -0.0074   0.0000
corr(Z1, Z2)          0.0003
corr(Z1^2, Z2^2)      0.0027
KS p-value Z1: 0.103, Z2: 0.705
P(|Z1| > 1.96) = 0.0500  (0.0500)
```

In the listing we use $1-U_1$ via `np.log1p(-U1)`, because `rng.random` can return exactly 0 but never 1. Production libraries generally use faster methods (the polar method or the ziggurat algorithm, which NumPy's `standard_normal` uses), but Box–Muller remains the cleanest illustration of the Jacobian formula at work.

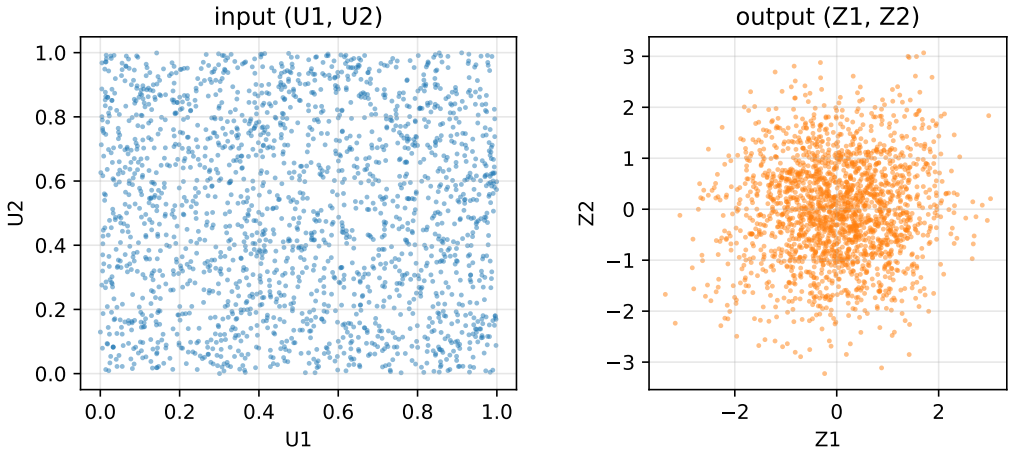


Figure 4.11: Box–Muller maps a uniform sample in the unit square (left) to a sample of independent standard normals (right).

4.6.5 Acceptance–rejection sampling

When neither F^{-1} nor a clever transformation is available, we can sample from an easier *proposal* density g and keep only some draws.

Theorem 4.35 (Acceptance–rejection)**.** *Suppose $f(x) \le M g(x)$ for all x, with $M < \infty$. Repeat: draw $X \sim g$ and $V \sim \text{Uniform}(0,1)$ independently, and accept X if $V \le f(X)/(Mg(X))$. The accepted value has density f, and the probability of acceptance per proposal is $1/M$.*

Proof. For any Borel set B,

$$\mathbb{P}(X \in B,\ \text{accept}) = \int_B g(x)\,\frac{f(x)}{M g(x)}\,dx = \frac{1}{M}\int_B f(x)\,dx.$$

With $B = \mathbb{R}$, $\mathbb{P}(\text{accept}) = 1/M$. Dividing, $\mathbb{P}(X \in B \mid \text{accept}) = \int_B f(x)\,dx$. Since trials are independent, the number of proposals until acceptance is geometric with mean M. □

Geometrically, $(X, V M g(X))$ is uniform under the curve Mg; keeping the points under f leaves a sample uniform under f, whose x-coordinates have density f. The listing samples from a Beta(2.7, 6.3) density (a plausible model for a budget share) with a uniform proposal.

Listing 4.20: Acceptance–rejection sampling from a Beta density ch04/ch04_accept_reject.py

```
# DEP 803 -- Chapter 4: Probability Foundations, Random Variables and Transformations
# Acceptance-rejection sampling from a Beta(2.7, 6.3) density.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
target = stats.beta(2.7, 6.3)               # e.g. a share of budget on food
mode = (2.7 - 1) / (2.7 + 6.3 - 2)
M = target.pdf(mode)                        # envelope M * g with g = U(0,1)
n = 200_000
X = rng.random(n)                           # proposals from g
V = rng.random(n)
accept = V * M <= target.pdf(X)             # accept if V <= f(X) / (M g(X))
```

```python
draws = X[accept]

print(f"envelope constant M = sup f/g      = {M:.4f}")
print(f"acceptance rate: theory 1/M        = {1 / M:.4f}")
print(f"                 simulated         = {accept.mean():.4f}")
print(f"accepted draws                     = {draws.size}")
print(f"mean: target {target.mean():.4f}, sample {draws.mean():.4f}")
print(f"var : target {target.var():.5f}, sample {draws.var():.5f}")
print(f"KS p-value (first 20000 draws)     = "
      f"{stats.kstest(draws[:20_000], target.cdf).pvalue:.3f}")
```

Output

```
envelope constant M = sup f/g      = 2.6697
acceptance rate: theory 1/M        = 0.3746
                 simulated         = 0.3736
accepted draws                     = 74718
mean: target 0.3000, sample 0.2986
var : target 0.02100, sample 0.02073
KS p-value (first 20000 draws)     = 0.165
```

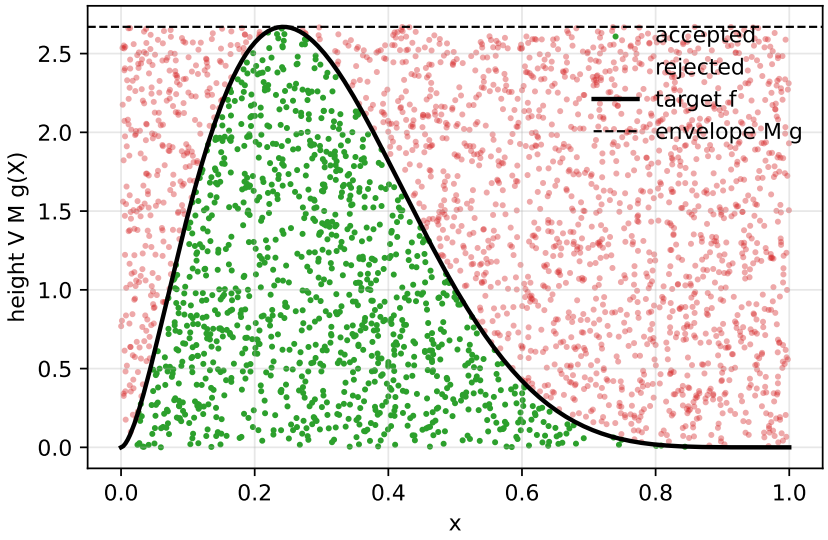


Figure 4.12: Acceptance–rejection with a flat envelope. Points below the target density are accepted; the acceptance rate equals the area under f divided by the area under the envelope, $1/M \approx 0.37$.

Graduate Perspective: Beyond acceptance–rejection

The efficiency $1/M$ collapses in high dimensions: for a d-dimensional target the best envelope constant typically grows exponentially in d. This is why modern Bayesian computation relies on Markov chain Monte Carlo, which produces dependent draws without an envelope, and on importance sampling, which reweights instead of rejecting (Robert and Casella, 2004). Both are developed in Chapter 10; Stan's Hamiltonian Monte Carlo (Matsuura, 2022) is a prominent example.

Policy Lens: Microsimulation needs reproducible randomness

Tax-benefit microsimulation models simulate how a reform changes each household's disposable income, often with random elements (take-up of a benefit, imputed incomes, behavioural responses). Because results are summarised by small differences between a baseline and a reform scenario, both scenarios should use the *same* random numbers (common random numbers), so that differences reflect the policy and not simulation noise. Seeded, spawnable generators make this straightforward and make the published estimates replicable.

Exercises

4.6.1 Theory Prove that an LCG with $c = 0$ and modulus $m = 2^e$ cannot have full period m. Show that $65539^2 \equiv 6 \cdot 65539 - 9 \pmod{2^{31}}$ and deduce the RANDU recurrence.

4.6.2 Theory Give the inverse-transform sampler for the Weibull CDF $F(x) = 1 - e^{-(x/\beta)^k}$, the logistic CDF $F(x) = 1/(1 + e^{-x})$ and the geometric PMF $p(k) = (1-p)^{k-1}p$, $k = 1, 2, \ldots$ (show that $\lceil \log U / \log(1-p) \rceil$ works).

4.6.3 Theory In Theorem 4.35, show that the optimal M for $f = \text{Beta}(\alpha, \beta)$ $(\alpha, \beta \geq 1)$ and $g = \text{Uniform}(0, 1)$ is the density at the mode. For the standard normal target and a Cauchy proposal, find the optimal M.

4.6.4 Python Implement the middle-square method for 4-digit seeds and compute, for every seed from 0000 to 9999, the number of steps until a value repeats. Report the maximum and the median. What does this say about the method?

4.6.5 Python Sample from the half-normal density $f(x) = \sqrt{2/\pi}\, e^{-x^2/2}$, $x > 0$, by acceptance–rejection with an Exp(1) proposal. Derive M, compare the empirical and theoretical acceptance rates, and attach a random sign to obtain normals.

4.6.6 ★ Graduate Explain why the outputs of independently seeded generators can overlap, and how `SeedSequence.spawn` avoids this. Discuss reproducibility issues when the same code is run with a different NumPy version or on a GPU.

4.6.7 Policy A ministry simulates the cost of a new child grant by drawing take-up decisions for each eligible household. Design a simulation in which baseline and reform scenarios use common random numbers, and show by simulation that the standard deviation of the estimated cost *difference* is much smaller than with independent streams.

Chapter Summary

- A probability space $(\Omega, \mathcal{F}, \mathbb{P})$ specifies outcomes, the events we can ask about (a σ-algebra) and a countably additive measure with $\mathbb{P}(\Omega) = 1$. Complements, inclusion–exclusion, Boole's inequality and continuity follow from Kolmogorov's three axioms; relative frequencies obey the same rules and converge to probabilities.
- Conditional probability renormalises to the conditioning event. The law of total probability averages over a partition; Bayes' theorem inverts conditionals using base rates. Inclusion and exclusion errors in targeting, and predictive values of diagnostic tests, are conditional probabilities with different denominators.
- A random variable is a measurable function; its law is captured by the CDF, which is non-decreasing, right-continuous and has limits 0 and 1. PMFs, PDFs and mixed distributions (such as zero-inflated expenditure) are special cases; the quantile function

is the generalised inverse. The ECDF converges uniformly to F (Glivenko–Cantelli).

- Joint distributions yield marginals by summing or integrating and conditionals by division. Sklar's theorem separates marginals from dependence; rank correlations depend only on the copula.
- Transformations are handled by the CDF method, the Jacobian formula (probability is conserved, density is rescaled), convolution for sums, and explicit formulas for order statistics. The probability integral transform maps any continuous variable to a uniform.
- Simulation starts from seeded PRNGs (NumPy's PCG64); inverse transform, Box–Muller and acceptance–rejection turn uniforms into draws from target distributions.

Further Reading

For a rigorous yet readable development of the material in this chapter see Casella and Berger (2002, Chs. 1–4) and Wasserman (2004, Chs. 1–3); Grimmett and Stirzaker (2020) has a wealth of exercises, and Billingsley (1995) is the standard measure-theoretic treatment. Murphy (2022, Ch. 2) reviews probability from the machine-learning perspective, including Bayes' rule and transformations of random variables, and treats naive Bayes classifiers in Ch. 9; Matsuura (2022) introduces the Bayesian interpretation that underlies Stan, and Gelman et al. (2013) develops it fully. Classification, the main machine-learning use of conditional probabilities, is introduced practically in Géron (2022, Ch. 3), with precision and recall playing the roles of PPV and sensitivity; see also Hastie et al. (2009). Kolmogorov (1933) and Bayes (1763) remain rewarding historical reading. The evaluation of probabilistic and machine-learning forecasts in macroeconomics is discussed in Goulet Coulombe et al. (2022), and PIT-based density-forecast evaluation in Diebold et al. (1998). On copulas, Nelsen (2006) is the standard introduction. On targeting, see Brown et al. (2018). For random number generation, Robert and Casella (2004) cover inverse-transform and acceptance–rejection methods in depth, L'Ecuyer and Simard (2007) describe statistical testing of generators, and O'Neill (2014) explains the design of the PCG family used by NumPy (Harris et al., 2020).

Chapter 5

Common Distributions in Data Science and Econometrics; Likelihood

Learning Objectives

After studying this chapter you should be able to:

- state the probability mass or density function, support, mean and variance of the Bernoulli, binomial, Poisson, geometric, negative binomial, multinomial, uniform, normal, exponential, gamma, beta, log-normal, Pareto and Weibull distributions, and choose among them for a given data-generating story;
- prove the Poisson limit theorem, the memorylessness of the exponential distribution and the gamma–Poisson mixture representation of the negative binomial;
- compute Lorenz curves, Gini coefficients and poverty headcounts implied by a parametric income distribution;
- derive the chi-square, Student t and F distributions from normal building blocks and explain why they govern regression inference;
- manipulate the multivariate normal: affine maps, marginals, conditionals (via the Schur complement), Mahalanobis distance and Cholesky simulation;
- write a distribution in exponential-family form, read off its sufficient statistic and moments from the log-partition function, and fit logistic and Poisson regressions as generalised linear models;
- define the likelihood, distinguish it from a probability, derive least squares as Gaussian maximum likelihood, apply the factorisation theorem, check fitted models with QQ plots, and perform a conjugate beta–binomial Bayesian update.

Every statistical statement we make about a policy is a statement about a *distribution*. When a ministry asks how many households fall below a poverty line, it is asking for the value of a cumulative distribution function at one point. When a health service plans staffing for rural clinics, it needs a model for the number of visits a household makes in a year — a distribution on the non-negative integers. When a labour economist asks whether the long-term unemployed find it harder to get a job, she is asking about the shape of the hazard function of a duration distribution. And when an analyst reports a regression coefficient with a p-value, the p-value is a tail probability of a Student t distribution whose existence rests on the normal distribution of the errors.

This chapter is a working catalogue of the distributions that appear again and again in data science and econometrics, organised not as a list to be memorised but as a small number

of families connected by limits, mixtures and transformations. The Poisson is a limit of the binomial; the negative binomial is a Poisson whose rate is itself random; the chi-square is a sum of squared normals; the t is a normal divided by the square root of an independent chi-square; the log-normal is the exponential of a normal. Most of these distributions belong to a single algebraic family — the *exponential family* — and that observation is what makes logistic regression, Poisson regression and ordinary least squares special cases of one framework, the *generalised linear model*.

The chapter closes with the concept that turns a distribution into a tool for learning from data: the *likelihood*. Once we fix the observed data and view the density as a function of the parameters, we obtain the object that underlies maximum likelihood estimation (developed in Chapter 7), likelihood-ratio tests, information criteria and Bayesian inference. We build on the probability foundations of Chapter 4 and the linear algebra of Chapters 2 and 3; expectations, moment generating functions and limit theorems are developed further in Chapter 6. Good companions are Murphy (2022, Chs. 2–3), Casella and Berger (2002, Chs. 2–3) and, for the Bayesian preview, Matsuura (2022).

Throughout, we use `scipy.stats`, whose "frozen" distribution objects expose `pmf`/`pdf`, `cdf`, `sf` (survival function), `ppf` (quantile function), `rvs` (random draws), `stats` (moments) and, for continuous families, `fit` (maximum likelihood). All data in this chapter are *simulated* unless stated otherwise; the numbers illustrate methods and are not statistics of Ghana or of any other country.

5.1 Discrete Distributions

A discrete random variable X takes values in a countable set and is described by its *probability mass function* (PMF) $p(k) = \mathbb{P}(X = k)$. Discrete distributions are the natural models for binary outcomes (employed or not, enrolled or not), for counts (clinic visits, crimes reported, firms entering a market) and for categorical choices (which sector a new firm enters).

5.1.1 Bernoulli and binomial

Definition 5.1 (Bernoulli and binomial)**.** $X \sim \text{Bern}(p)$ if $\mathbb{P}(X = 1) = p = 1 - \mathbb{P}(X = 0)$, $p \in [0, 1]$. If $X_1, \dots, X_n \overset{\text{iid}}{\sim} \text{Bern}(p)$ then $S = \sum_{i=1}^{n} X_i \sim \text{Bin}(n, p)$, with PMF

$$\mathbb{P}(S = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k = 0, 1, \dots, n. \tag{5.1}$$

Because $X_i^2 = X_i$, $\mathbb{E}X_i = p$ and $\operatorname{Var} X_i = p - p^2 = p(1-p)$; by independence and linearity, $\mathbb{E}S = np$ and $\operatorname{Var} S = np(1-p)$. The variance is largest at $p = 1/2$: a survey estimating a proportion near one half needs the largest sample for a given precision, which is why sample-size calculations use $p = 1/2$ as the conservative default.

5.1.2 The Poisson distribution and the law of rare events

Definition 5.2 (Poisson)**.** $X \sim \text{Pois}(\lambda)$, $\lambda > 0$, if $\mathbb{P}(X = k) = e^{-\lambda}\lambda^k / k!$ for $k = 0, 1, 2, \dots$

Using $\sum_k \lambda^k / k! = e^{\lambda}$, one obtains $\mathbb{E}X = \lambda$ and $\mathbb{E}[X(X-1)] = \lambda^2$, hence $\operatorname{Var} X = \lambda$: *the Poisson mean equals its variance* (*equidispersion*). The Poisson arises whenever events occur "rarely and independently", as the next theorem makes precise.

Theorem 5.3 (Poisson limit theorem). *Let $S_n \sim \mathrm{Bin}(n, p_n)$ with $np_n \to \lambda \in (0, \infty)$. Then for every fixed $k \geq 0$,*

$$\lim_{n\to\infty} \mathbb{P}(S_n = k) = e^{-\lambda}\frac{\lambda^k}{k!}.$$

Proof. Write $\lambda_n = np_n$. For fixed k,

$$\mathbb{P}(S_n = k) = \frac{n(n-1)\cdots(n-k+1)}{k!}\,\frac{\lambda_n^k}{n^k}\Big(1-\frac{\lambda_n}{n}\Big)^{n-k} = \frac{\lambda_n^k}{k!}\underbrace{\prod_{j=0}^{k-1}\Big(1-\frac{j}{n}\Big)}_{\to 1}\underbrace{\Big(1-\frac{\lambda_n}{n}\Big)^{n}}_{\to e^{-\lambda}}\underbrace{\Big(1-\frac{\lambda_n}{n}\Big)^{-k}}_{\to 1}.$$

The product has a fixed number k of factors each tending to one, and $\lambda_n^k \to \lambda^k$. For the middle factor, $n\log(1-\lambda_n/n) = -\lambda_n + O(\lambda_n^2/n) \to -\lambda$ using $\log(1-x) = -x + O(x^2)$, so $(1-\lambda_n/n)^n \to e^{-\lambda}$. □

Pointwise convergence of the PMF implies convergence in total variation (Scheffé's lemma), and Le Cam (1960) gave a sharp quantitative version: if $S = \sum_i X_i$ with independent $X_i \sim \mathrm{Bern}(p_i)$ and $\lambda = \sum_i p_i$, then

$$d_{TV}\big(\mathcal{L}(S), \mathrm{Pois}(\lambda)\big) = \tfrac{1}{2}\sum_{k\geq 0}\big|\mathbb{P}(S=k) - e^{-\lambda}\lambda^k/k!\big| \;\leq\; \sum_i p_i^2. \tag{5.2}$$

For $p_i = \lambda/n$ the bound is λ^2/n. Notice that the p_i need *not* be equal: the Poisson approximates the total count of many rare, independent and *heterogeneous* events. That is the deep reason it fits counts of road accidents in a district, of crimes reported to a police station in a week, or of new firm registrations in a month: each of many potential events has a small probability, and they are roughly independent.

Listing 5.1: The Poisson limit and the horse-kick data ch05/poisson_limit.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Poisson limit of the binomial and Bortkiewicz's horse-kick data
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

lam = 2.0
k = np.arange(0, 60)
print("Total variation distance between Bin(n, lam/n) and Poisson(lam):")
for n in [5, 10, 50, 100, 1000, 10000]:
    diff = stats.binom.pmf(k, n, lam / n) - stats.poisson.pmf(k, lam)
    tv = 0.5 * np.abs(diff).sum()
    print(f"  n = {n:6d}   TV = {tv:.5f}   bound lam^2/n = {lam**2 / n:.5f}")

# Bortkiewicz (1898): deaths by horse kick, 10 Prussian corps x 20 years
deaths = np.array([0, 1, 2, 3, 4])
corps_years = np.array([109, 65, 22, 3, 1])
N = corps_years.sum()
mean = (deaths * corps_years).sum() / N
var = ((deaths - mean) ** 2 * corps_years).sum() / (N - 1)
expected = N * stats.poisson.pmf(deaths, mean)
print(f"\nHorse kicks: N = {N}, mean = {mean:.3f}, variance = {var:.3f}")
print(" deaths  observed  expected(Poisson)")
for d, o, e in zip(deaths, corps_years, expected):
    print(f"{d:7d}{o:10d}{e:12.1f}")
```

Output

```
Total variation distance between Bin(n, lam/n) and Poisson(lam):
  n =      5   TV = 0.12488   bound lam^2/n = 0.80000
  n =     10   TV = 0.05220   bound lam^2/n = 0.40000
  n =     50   TV = 0.00927   bound lam^2/n = 0.08000
  n =    100   TV = 0.00457   bound lam^2/n = 0.04000
  n =   1000   TV = 0.00045   bound lam^2/n = 0.00400
  n =  10000   TV = 0.00005   bound lam^2/n = 0.00040

Horse kicks: N = 200, mean = 0.610, variance = 0.611
 deaths  observed  expected(Poisson)
      0       109             108.7
      1        65              66.3
      2        22              20.2
      3         3               4.1
      4         1               0.6
```

The total variation distance falls like $1/n$, comfortably inside Le Cam's bound. The second part reproduces the most famous data set in the history of the Poisson distribution.

Historical Note: Poisson, Bortkiewicz and the Prussian cavalry

Siméon-Denis Poisson derived the distribution as a limit of the binomial in his 1837 *Recherches sur la probabilité des jugements* (Poisson, 1837), a treatise on the reliability of jury verdicts. The result attracted little attention until Ladislaus von Bortkiewicz's 1898 monograph *Das Gesetz der kleinen Zahlen* ("The law of small numbers") (Bortkiewicz, 1898) showed how well it described rare events. His best-known example counts soldiers killed by horse kicks in Prussian army corps over 20 years (1875–1894); the frequently reproduced subset of 10 corps gives 200 corps-years with 109, 65, 22, 3 and 1 corps-years showing 0, 1, 2, 3 and 4 deaths. Its mean (0.61) and variance (0.61) coincide almost exactly, and the Poisson expected frequencies in the output above match the observed ones remarkably well.

5.1.3 Geometric and negative binomial

Definition 5.4 (Geometric and negative binomial)**.** In a sequence of independent Bern(p) trials, the number of trials T up to and including the first success is *geometric*: $\mathbb{P}(T = k) = (1-p)^{k-1}p$, $k = 1, 2, \dots$, with $\mathbb{E}T = 1/p$ and $\operatorname{Var} T = (1-p)/p^2$. The number Y of failures before the r-th success is *negative binomial*, $Y \sim \mathrm{NB}(r, p)$:

$$\mathbb{P}(Y = k) = \binom{k+r-1}{k} p^r (1-p)^k, \qquad k = 0, 1, \dots, \tag{5.3}$$

with $\mathbb{E}Y = r(1-p)/p$ and $\operatorname{Var} Y = r(1-p)/p^2$. The formula makes sense for any real $r > 0$ once the binomial coefficient is written as $\Gamma(k+r)/(\Gamma(r)\,k!)$.

The geometric distribution is the discrete analogue of the exponential: it is memoryless, $\mathbb{P}(T > s + t \mid T > s) = \mathbb{P}(T > t)$ (Exercise 5.1.2). A useful reparametrisation of the negative binomial in econometrics is by its mean $\mu = r(1-p)/p$:

$$\mathbb{E}Y = \mu, \qquad \operatorname{Var} Y = \mu + \frac{\mu^2}{r} = \mu + \alpha\mu^2, \quad \alpha = 1/r. \tag{5.4}$$

The variance exceeds the mean, and α measures *overdispersion*; the Poisson is the limit $\alpha \to 0$ ($r \to \infty$).

Proposition 5.5 (Negative binomial as a gamma–Poisson mixture). *If $U \sim \text{Gamma}(r, \text{rate } r)$ (mean 1, variance $1/r$) and, conditional on U, $Y \mid U \sim \text{Pois}(\mu U)$, then marginally $Y \sim \text{NB}(r, p)$ with $p = r/(r+\mu)$.*

Proof. Integrate out U:

$$\mathbb{P}(Y=k) = \int_0^\infty e^{-\mu u}\frac{(\mu u)^k}{k!}\,\frac{r^r u^{r-1}e^{-ru}}{\Gamma(r)}\,\mathrm{d}u = \frac{\mu^k r^r}{k!\,\Gamma(r)}\int_0^\infty u^{k+r-1}e^{-(\mu+r)u}\,\mathrm{d}u,$$

and the last integral equals $\Gamma(k+r)/(\mu+r)^{k+r}$, so

$$\mathbb{P}(Y=k) = \frac{\Gamma(k+r)}{k!\,\Gamma(r)}\left(\frac{r}{r+\mu}\right)^r\left(\frac{\mu}{r+\mu}\right)^k,$$

which is (5.3) with $p = r/(r+\mu)$. Alternatively, by the laws of total expectation and variance, $\mathbb{E}Y = \mathbb{E}[\mu U] = \mu$ and $\operatorname{Var} Y = \mathbb{E}[\mu U] + \operatorname{Var}(\mu U) = \mu + \mu^2/r$. □

The mixture interpretation is what makes the negative binomial so useful in policy work: households differ in unobserved health status, so their clinic-visit *rates* differ; even if each household's visits are Poisson given its own rate, the population of visit counts is overdispersed. Greenwood and Yule (1920) used exactly this argument to model "accident proneness" and repeated attacks of disease.

5.1.4 The multinomial distribution

Definition 5.6 (Multinomial). If each of n independent trials falls into one of J categories with probabilities $\pi_1, \dots, \pi_J$ ($\sum_j \pi_j = 1$), the vector of counts $\boldsymbol{N} = (N_1, \dots, N_J)$ has PMF

$$\mathbb{P}(\boldsymbol{N} = \boldsymbol{n}) = \frac{n!}{n_1! \cdots n_J!}\prod_{j=1}^{J}\pi_j^{n_j}, \qquad \sum_j n_j = n,$$

with $\mathbb{E}N_j = n\pi_j$, $\operatorname{Var} N_j = n\pi_j(1-\pi_j)$ and $\operatorname{Cov}(N_j, N_l) = -n\pi_j\pi_l$ for $j \neq l$.

The negative covariances reflect the constraint $\sum_j N_j = n$: the covariance matrix $n(\operatorname{diag}(\boldsymbol{\pi}) - \boldsymbol{\pi}\boldsymbol{\pi}^\top)$ is singular, with null vector $\mathbf{1}$. Each margin is binomial, $N_j \sim \text{Bin}(n, \pi_j)$. The multinomial is the likelihood behind multinomial logit models of occupational or sectoral choice.

⋆ Key Concept: Choosing a count model

- Fixed number of independent yes/no trials: **binomial**, $\text{Var} = np(1-p) < \mathbb{E}$.
- Many rare, independent opportunities: **Poisson**, $\text{Var} = \mathbb{E}$ (equidispersion).
- Unobserved heterogeneity in rates: **negative binomial**, $\text{Var} = \mu + \alpha\mu^2 > \mathbb{E}$.
- Waiting for the first success: **geometric** (memoryless).
- Counts across J categories with a fixed total: **multinomial**.

The variance-to-mean ratio is the first diagnostic to compute for any count variable.

5.1.5 Computing with `scipy.stats`

Listing 5.2: Moments and PMFs of discrete distributions ch05/discrete_pmfs.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Means, variances and PMFs of the main discrete distributions with scipy.stats
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# (name, frozen distribution, closed-form mean, closed-form variance)
p, n, lam, r = 0.3, 20, 4.0, 3
dists = [
    ("Bernoulli(0.3)", stats.bernoulli(p), p, p * (1 - p)),
    ("Binomial(20,0.3)", stats.binom(n, p), n * p, n * p * (1 - p)),
    ("Poisson(4)", stats.poisson(lam), lam, lam),
    ("Geometric(0.3)", stats.geom(p), 1 / p, (1 - p) / p**2),
    ("NegBin(3,0.3)", stats.nbinom(r, p), r * (1 - p) / p, r * (1 - p) / p**2),
]
print(f"{'distribution':18s}{'mean':>8s}{'formula':>9s}{'var':>9s}{'formula':>9s}")
for name, d, m, v in dists:
    print(f"{name:18s}{d.mean():8.3f}{m:9.3f}{d.var():9.3f}{v:9.3f}")
print("note: scipy geom counts trials (support 1,2,...);")
print("      scipy nbinom counts failures before the r-th success.")

# multinomial: sector choice of 10 new firms with probs (0.5, 0.3, 0.2)
mn = stats.multinomial(10, [0.5, 0.3, 0.2])
print("P(5,3,2) under Multinomial(10; .5,.3,.2) =", round(mn.pmf([5, 3, 2]), 4))
print("Cov matrix of the counts:")
print(np.round(mn.cov(), 2))

fig, axes = plt.subplots(1, 3, figsize=(9, 3), sharey=True)
k = np.arange(0, 16)
for ax, (name, d, _, _) in zip(axes, [dists[1], dists[2], dists[4]]):
    ax.bar(k, d.pmf(k), color="#1f4e79", width=0.7)
    ax.axvline(d.mean(), color="#cc9900", lw=2, label="mean")
    ax.set_title(name, fontsize=10)
    ax.set_xlabel("k")
axes[0].set_ylabel("P(X = k)")
axes[0].legend(frameon=False)
fig.tight_layout()
plt.savefig("ch05_discrete_pmfs.pdf", bbox_inches="tight")
```

Output

```
distribution          mean  formula      var  formula
Bernoulli(0.3)       0.300    0.300    0.210    0.210
Binomial(20,0.3)     6.000    6.000    4.200    4.200
Poisson(4)           4.000    4.000    4.000    4.000
Geometric(0.3)       3.333    3.333    7.778    7.778
NegBin(3,0.3)        7.000    7.000   23.333   23.333
note: scipy geom counts trials (support 1,2,...);
      scipy nbinom counts failures before the r-th success.
P(5,3,2) under Multinomial(10; .5,.3,.2) = 0.085
Cov matrix of the counts:
```

```
[[ 2.5 -1.5 -1. ]
 [-1.5  2.1 -0.6]
 [-1.  -0.6  1.6]]
```

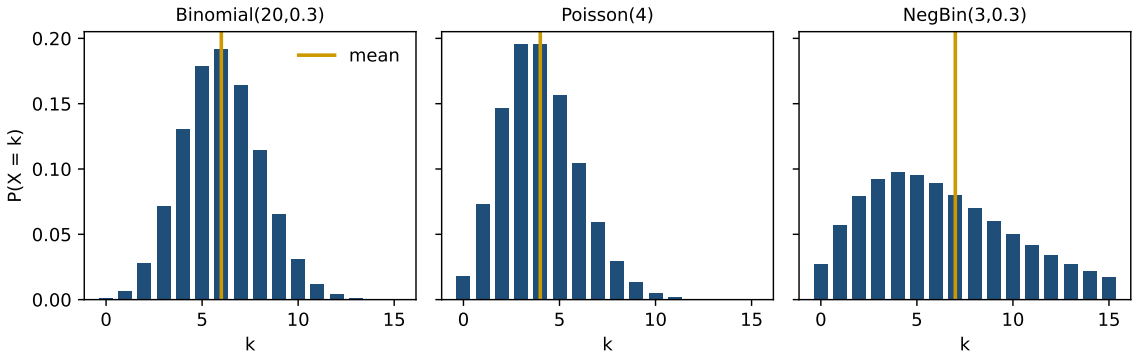


Figure 5.1: PMFs of Bin(20, 0.3), Pois(4) and NB(3, 0.3); the vertical line marks the mean. The negative binomial has a much longer right tail than the Poisson.

△ Caution: Parametrisations differ between sources

`scipy.stats.geom` counts *trials* (support $1, 2, \ldots$), whereas many textbooks count *failures* (support $0, 1, \ldots$). `scipy.stats.nbinom(n, p)` counts failures before the n-th success, while econometrics software usually parametrises the negative binomial by (μ, α). The exponential and gamma are specified by a *scale* in SciPy (`scale = 1/rate`). Always check the mean of a frozen distribution against the formula you have in mind before using it.

5.1.6 Count data in policy: overdispersion in clinic visits

Listing 5.3 simulates a household survey in which each household has a latent "frailty" $U \sim \text{Gamma}(1.5, \text{rate } 1.5)$ and makes $\text{Pois}(3U)$ clinic visits per year. We then fit a Poisson and a negative binomial by the method of moments, using (5.4) to solve $\hat{r} = \bar{y}^2/(s^2 - \bar{y})$.

Listing 5.3: Overdispersed clinic visits ch05/overdispersion.py

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Simulated clinic visits: Poisson versus negative binomial (gamma-Poisson mixture)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 5000                          # simulated households
mu, r = 3.0, 1.5                  # mean visits per year, NB size (shape)
frailty = rng.gamma(shape=r, scale=1 / r, size=n)   # mean 1, var 1/r
visits = rng.poisson(mu * frailty)

m, v = visits.mean(), visits.var(ddof=1)
print(f"sample mean = {m:.3f}, sample variance = {v:.3f}, ratio = {v / m:.2f}")
print(f"theory: Var = mu + mu^2/r = {mu + mu**2 / r:.3f}")

```

```python
# method-of-moments negative binomial: Var = m + m^2/r  =>  r = m^2/(v - m)
r_hat = m**2 / (v - m)
p_hat = r_hat / (r_hat + m)       # scipy nbinom(r, p) has mean r(1-p)/p
print(f"MoM estimates: r = {r_hat:.3f}, p = {p_hat:.3f}")

k = np.arange(0, 16)
obs = np.array([(visits == j).mean() for j in k])
pois = stats.poisson.pmf(k, m)
nb = stats.nbinom.pmf(k, r_hat, p_hat)
print(" k   observed  Poisson  NegBin")
for j in [0, 1, 2, 5, 10, 15]:
    print(f"{j:2d}{obs[j]:10.4f}{pois[j]:9.4f}{nb[j]:8.4f}")
ll_p = stats.poisson.logpmf(visits, m).sum()
ll_nb = stats.nbinom.logpmf(visits, r_hat, p_hat).sum()
print(f"log-likelihood: Poisson = {ll_p:.1f}, NegBin = {ll_nb:.1f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.bar(k, obs, color="#bbbbbb", label="simulated data")
ax.plot(k, pois, "o-", color="#1f4e79", label="Poisson fit")
ax.plot(k, nb, "s--", color="#cc9900", label="negative binomial fit")
ax.set_xlabel("clinic visits per household per year")
ax.set_ylabel("proportion")
ax.legend(frameon=False)
plt.savefig("ch05_overdispersion.pdf", bbox_inches="tight")
```

Output

```
sample mean = 2.936, sample variance = 8.734, ratio = 2.97
theory: Var = mu + mu^2/r = 9.000
MoM estimates: r = 1.487, p = 0.336
 k   observed  Poisson  NegBin
 0    0.1970   0.0531  0.1977
 1    0.1976   0.1558  0.1952
 2    0.1574   0.2288  0.1611
 5    0.0652   0.0965  0.0674
10    0.0124   0.0007  0.0118
15    0.0016   0.0000  0.0018
log-likelihood: Poisson = -12960.5, NegBin = -11075.3
```

The variance is three times the mean; the Poisson predicts 5% zero-visit households where 20% are observed, and almost no household with ten or more visits. The log-likelihood improves by almost 1900 units — an overwhelming difference (Fig. 5.2).

Policy Lens: Count models of service usage

Health ministries plan staff, drugs and budgets from expected utilisation. Overdispersion has two practical consequences. First, *capacity*: a Poisson model understates the probability of very high-use households (chronic patients) and thus the tail of demand. Second, *inference*: standard errors from a Poisson regression that ignores overdispersion are too small, so the effect of, say, a user-fee exemption on visits will look more precisely estimated than it is. The same logic applies to crime counts by police district and to firm entries by municipality. Standard references are Cameron and Trivedi (2013) and Cameron and Trivedi (2005, Ch. 20).

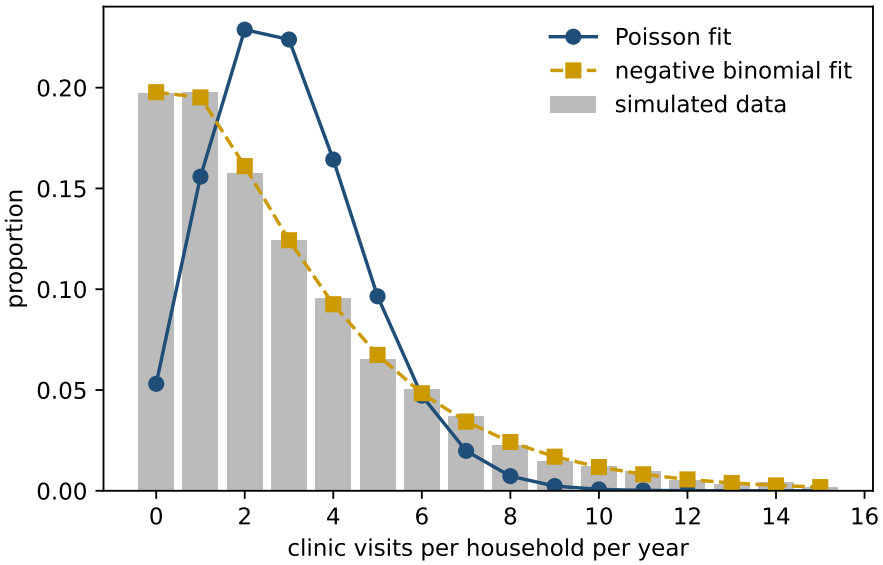


Figure 5.2: Simulated clinic visits. The Poisson fit (same mean) badly underpredicts zeros and large counts; the negative binomial matches both.

Exercises

5.1.1 Theory Derive $\mathbb{E}X$ and $\operatorname{Var} X$ for $X \sim \text{Pois}(\lambda)$ using the factorial moment $\mathbb{E}[X(X-1)]$, and show that the probability generating function is $G(s) = \mathbb{E}s^X = e^{\lambda(s-1)}$. Use G to prove that the sum of independent $\text{Pois}(\lambda_1)$ and $\text{Pois}(\lambda_2)$ variables is $\text{Pois}(\lambda_1 + \lambda_2)$.

5.1.2 Theory Prove that the geometric distribution is memoryless and that it is the *only* memoryless distribution on $\{1, 2, \dots\}$.

5.1.3 Theory Let $N \sim \text{Pois}(\lambda)$ be the number of firms entering a market in a year, and suppose each entrant is independently foreign-owned with probability q. Show that the numbers of foreign and domestic entrants are independent Poisson variables with means λq and $\lambda(1-q)$ (*Poisson thinning*).

5.1.4 Python Modify Listing 5.1 to use *heterogeneous* probabilities $p_i = 2\lambda i/(n(n+1))$, $i = 1, \dots, n$, which still sum to λ. Compute the total variation distance to $\text{Pois}(\lambda)$ (the exact distribution of S can be obtained by repeated convolution with `np.convolve`) and compare it with Le Cam's bound $\sum_i p_i^2$.

5.1.5 Python Using the multinomial in Listing 5.2, verify by simulation (10^5 draws) that the covariance matrix of the counts is $n(\operatorname{diag}(\boldsymbol{\pi}) - \boldsymbol{\pi}\boldsymbol{\pi}^\top)$ and that its smallest eigenvalue is zero. What is the eigenvector?

5.1.6 ★ Graduate Show that the method-of-moments estimator $\hat{r} = \bar{y}^2/(s^2 - \bar{y})$ in Listing 5.3 is undefined or negative when $s^2 \le \bar{y}$. Simulate 1000 samples of size $n = 50$ from $\text{NB}(r = 5, \mu = 2)$ and report how often this happens. What does this imply for estimating overdispersion in small samples?

5.1.7 Policy A district records weekly burglary counts with mean 4.1 and variance 11.3. (a) Estimate the negative binomial (μ, α). (b) Under the Poisson and under the negative binomial, compute the probability of a week with 10 or more burglaries. (c) A police chief proposes to flag any week with 10 or more burglaries as a "crime wave". How often would this rule raise a false alarm under each model?

5.2 Continuous Distributions

A continuous random variable has a density f with $\mathbb{P}(a < X \le b) = \int_a^b f(x)\,\mathrm{d}x$, cumulative distribution function (CDF) $F(x) = \mathbb{P}(X \le x)$, survival function $S(x) = 1 - F(x)$ and quantile function $F^{-1}(u) = \inf\{x : F(x) \ge u\}$. The quantile function is the workhorse of simulation: if $U \sim \mathrm{Unif}(0,1)$ then $F^{-1}(U)$ has CDF F (*inverse transform sampling*).

5.2.1 Uniform and normal

$X \sim \mathrm{Unif}(a,b)$ has density $1/(b-a)$ on $[a,b]$, mean $(a+b)/2$ and variance $(b-a)^2/12$. By the probability integral transform, $F(X) \sim \mathrm{Unif}(0,1)$ for any continuous X; this is why p-values are uniform under the null hypothesis and why QQ plots work (Section 5.6).

Definition 5.7 (Normal). $X \sim \mathcal{N}(\mu, \sigma^2)$ if $f(x) = (2\pi\sigma^2)^{-1/2}\exp\{-(x-\mu)^2/(2\sigma^2)\}$, $x \in \mathbb{R}$. The standard normal $Z \sim \mathcal{N}(0,1)$ has density ϕ and CDF Φ, and $X = \mu + \sigma Z$.

The normal is closed under affine maps and under sums of independent variables, its moment generating function is $M(t) = \exp(\mu t + \sigma^2 t^2/2)$, and by the central limit theorem (Chapter 6) it is the limiting distribution of standardised sums. Among all distributions with given mean and variance it has maximal entropy — a formal sense in which it is the "least informative" assumption beyond two moments.

Historical Note: From de Moivre to Gauss

Abraham de Moivre, a French Huguenot living in London, obtained the normal curve in 1733 as an approximation to binomial probabilities for large n, in a short Latin pamphlet later reprinted in the second edition of his *Doctrine of Chances* (1738). Carl Friedrich Gauss, in his 1809 *Theoria motus* on the orbits of celestial bodies (Gauss, 1809), argued that if the arithmetic mean is the best estimate of a quantity measured with error, then the errors must follow the normal law — and he used this to justify the method of least squares, which Legendre had published in 1805. Laplace soon connected the two threads through his central limit theorem. The story is told in detail by Stigler (1986).

5.2.2 Exponential: memorylessness and unemployment durations

Definition 5.8 (Exponential). $T \sim \mathrm{Exp}(\lambda)$ (rate $\lambda > 0$) if $f(t) = \lambda e^{-\lambda t}$ for $t \ge 0$. Then $S(t) = e^{-\lambda t}$, $\mathbb{E}T = 1/\lambda$ and $\mathrm{Var}\,T = 1/\lambda^2$.

For a duration T with density f and survival S, the *hazard function*

$$h(t) = \lim_{\Delta \downarrow 0} \frac{\mathbb{P}(t \le T < t + \Delta \mid T \ge t)}{\Delta} = \frac{f(t)}{S(t)} = -\frac{\mathrm{d}}{\mathrm{d}t}\log S(t) \tag{5.5}$$

is the instantaneous exit rate among those still "at risk"; conversely $S(t) = \exp\{-\int_0^t h(u)\,\mathrm{d}u\}$. In labour economics T is the length of an unemployment spell and $h(t)$ the job-finding rate after t months of unemployment (Lancaster, 1990).

Theorem 5.9 (Memorylessness). *A non-negative continuous random variable T with $S(t) > 0$ for all t satisfies $\mathbb{P}(T > s + t \mid T > s) = \mathbb{P}(T > t)$ for all $s, t \ge 0$ if and only if T is exponential. Equivalently, the exponential is the only distribution with a constant hazard.*

Proof. ($\Leftarrow$) $\mathbb{P}(T > s+t \mid T > s) = e^{-\lambda(s+t)}/e^{-\lambda s} = e^{-\lambda t}$. ($\Rightarrow$) Memorylessness means $S(s+t) = S(s)S(t)$. Let $g = \log S$, which is monotone and satisfies Cauchy's equation $g(s+t) = g(s) + g(t)$.

Then $g(m/n) = (m/n)g(1)$ for all positive rationals, and monotonicity extends this to $g(t) = tg(1)$ for all $t \geq 0$. Hence $S(t) = e^{-\lambda t}$ with $\lambda = -g(1) > 0$. □

Memorylessness is a strong behavioural statement: a job seeker unemployed for a year would have exactly the same chance of finding work next month as one who lost her job yesterday. Empirically, exit rates often *fall* with duration — because of skill depreciation, employer screening on duration, discouragement, or simply because the most employable leave first (dynamic selection on unobserved heterogeneity). This is called *negative duration dependence*.

5.2.3 Gamma and Weibull

Definition 5.10 (Gamma and Weibull). $X \sim \text{Gamma}(\alpha, \beta)$ (shape $\alpha > 0$, rate $\beta > 0$) has density $f(x) = \beta^\alpha x^{\alpha-1} e^{-\beta x}/\Gamma(\alpha)$, $x > 0$, with $\mathbb{E}X = \alpha/\beta$ and $\text{Var}\, X = \alpha/\beta^2$. T is Weibull with shape $k > 0$ and scale $\lambda > 0$ if $S(t) = \exp\{-(t/\lambda)^k\}$; its hazard is

$$h(t) = \frac{k}{\lambda}\left(\frac{t}{\lambda}\right)^{k-1}, \tag{5.6}$$

decreasing for $k < 1$, constant for $k = 1$ (exponential) and increasing for $k > 1$.

The gamma is the distribution of a sum of α independent $\text{Exp}(\beta)$ variables when α is an integer (the Erlang distribution), and $\text{Gamma}(\nu/2, \text{rate } 1/2)$ is the chi-square distribution with ν degrees of freedom (Section 5.3). It is the conjugate prior for a Poisson rate and the mixing distribution of Proposition 5.5. The Weibull, named after the Swedish engineer Waloddi Weibull (Weibull, 1951) who popularised it for the strength of materials, is the simplest model with a monotone hazard and a default parametric model in duration analysis.

Listing 5.4: Memorylessness and Weibull hazards ch05/unemployment_duration.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Memorylessness of the exponential and Weibull hazards for unemployment spells
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 200_000
expo = rng.exponential(scale=6.0, size=n)            # mean spell 6 months
weib = stats.weibull_min(c=0.7, scale=5.0).rvs(n, random_state=rng)

print("P(T > s + 3 | T > s) for simulated spells (months), s = 0, 3, 6, 12")
for name, T in [("exponential", expo), ("Weibull k=0.7", weib)]:
    row = [((T > s + 3).sum() / (T > s).sum()) for s in [0, 3, 6, 12]]
    print(f"  {name:14s}" + "  ".join(f"{c:.3f}" for c in row))
print(f"  exponential theory exp(-3/6) = {np.exp(-0.5):.3f}")

# hazard h(t) = f(t) / S(t)
t = np.linspace(0.05, 24, 300)
fig, ax = plt.subplots(figsize=(6, 3.8))
for c, lab in [(0.7, "Weibull k=0.7 (negative duration dependence)"),
               (1.0, "Weibull k=1 = exponential (constant)"),
               (1.5, "Weibull k=1.5 (positive duration dependence)")]:
    d = stats.weibull_min(c=c, scale=6.0)
    ax.plot(t, d.pdf(t) / d.sf(t), lw=2, label=lab)
```

```
ax.set_ylim(0, 0.6)
ax.set_xlabel("months unemployed")
ax.set_ylabel("hazard of leaving unemployment")
ax.legend(frameon=False, fontsize=8)
plt.savefig("ch05_hazards.pdf", bbox_inches="tight")
```

Output

```
P(T > s + 3 | T > s) for simulated spells (months), s = 0, 3, 6, 12
  exponential   0.606  0.605  0.609  0.610
  Weibull k=0.7 0.497  0.646  0.687  0.731
  exponential theory exp(-3/6) = 0.607
```

For the exponential, the conditional probability of remaining unemployed a further three months is $e^{-0.5} \approx 0.607$ regardless of elapsed duration. For the Weibull with $k = 0.7$ it *rises* from 0.50 to 0.73: the longer someone has been unemployed, the more likely they are to remain so (Fig. 5.3).

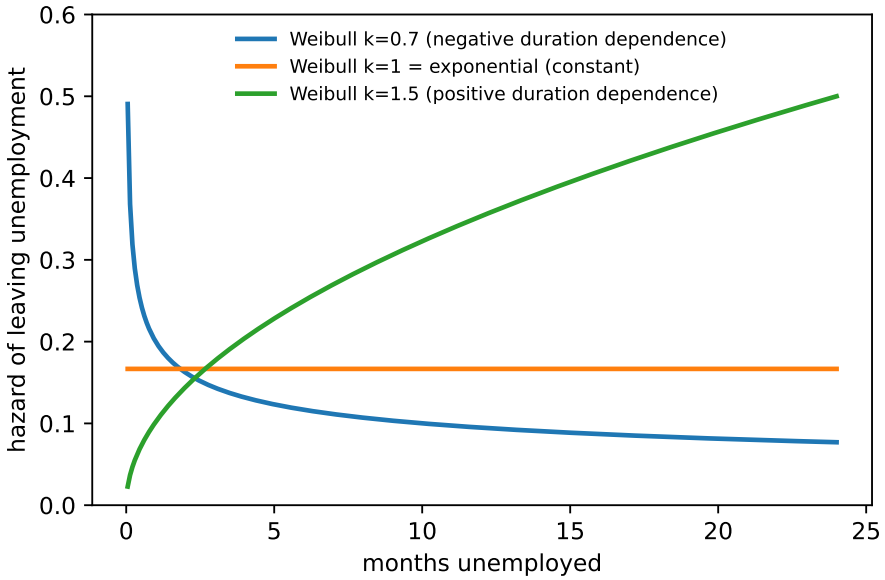


Figure 5.3: Weibull hazard functions with scale 6 months. Shape $k < 1$ gives negative duration dependence; $k = 1$ is the memoryless exponential.

△ Caution: Duration dependence or heterogeneity?

A falling *population* hazard does not imply that any individual's hazard falls. If each person has a constant (exponential) hazard λ_i, but λ_i varies across people, the high-λ people leave first and the aggregate hazard declines (Exercise 5.2.4). Distinguishing true state dependence from this selection effect is a central identification problem in the econometrics of durations (Lancaster, 1990); it matters for policy because only true duration dependence justifies targeting help at the long-term unemployed *because* of their duration.

5.2.4 Beta: a model for proportions

Definition 5.11 (Beta). $X \sim \text{Beta}(a, b)$, $a, b > 0$, has density $f(x) = x^{a-1}(1-x)^{b-1}/B(a,b)$ on $(0,1)$, where $B(a,b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$. Its mean is $a/(a+b)$ and its variance $ab/\{(a+b)^2(a+b+1)\}$.

The beta family is flexible on the unit interval: Beta$(1,1)$ is uniform, $a = b < 1$ gives a U-shape, and $a \neq b$ gives skewed shapes (Fig. 5.6). It models proportions — the share of a district's budget spent on education, the vote share of a party, a school's pass rate — and is the conjugate prior for a binomial probability (Section 5.6).

5.2.5 Log-normal and Pareto: income and wealth

Definition 5.12 (Log-normal). Y is log-normal, $\log Y \sim \mathcal{N}(\mu, \sigma^2)$, if $f(y) = \{y\sigma\sqrt{2\pi}\}^{-1}\exp\{-(\log y - \mu)^2/(2\sigma^2)\}$, $y > 0$. Then

$$\text{median} = e^{\mu}, \qquad \mathbb{E}Y = e^{\mu+\sigma^2/2}, \qquad \operatorname{Var} Y = (e^{\sigma^2}-1)e^{2\mu+\sigma^2}.$$

The mean exceeds the median, increasingly so as σ grows. A multiplicative story generates log-normality: if income evolves as $Y_t = Y_{t-1}(1+\varepsilon_t)$ with small independent shocks, then $\log Y_t$ is approximately a sum of independent terms and hence approximately normal (Gibrat's law of proportionate effect). Aitchison and Brown (1957) is the classic treatment of its economic uses.

Definition 5.13 (Pareto). X is Pareto with scale $x_m > 0$ and tail index $\alpha > 0$ if $S(x) = \mathbb{P}(X > x) = (x/x_m)^{-\alpha}$ for $x \geq x_m$. Its density is $\alpha x_m^{\alpha} x^{-\alpha-1}$; $\mathbb{E}X = \alpha x_m/(\alpha-1)$ if $\alpha > 1$ (infinite otherwise) and $\operatorname{Var} X < \infty$ only if $\alpha > 2$.

The defining property of a Pareto tail is *scale invariance*: $\mathbb{P}(X > tx \mid X > x) = t^{-\alpha}$ for every threshold $x \geq x_m$, so $\log S(x)$ is linear in $\log x$ with slope $-\alpha$. A related identity is that the mean income of those above any threshold x is $\frac{\alpha}{\alpha-1}x$ — with $\alpha = 2$, the average income of people earning more than any given amount is twice that amount. Empirical income distributions are typically log-normal-like in the body and Pareto-like in the top tail (Atkinson et al., 2011).

Historical Note: Pareto, Lorenz and Gini

Vilfredo Pareto, an Italian engineer turned economist who succeeded Léon Walras at Lausanne, reported in his *Cours d'économie politique* (1896–97) (Pareto, 1896) that plotting the logarithm of the number of people with income above x against $\log x$ gave nearly straight lines for tax data from several countries and periods. In 1905 the American economist Max O. Lorenz, then a graduate student, proposed the curve that now bears his name for comparing the concentration of wealth (Lorenz, 1905). In 1912 the Italian statistician Corrado Gini introduced his mean-difference index of variability in *Variabilità e mutabilità* (Gini, 1912); its connection to the area under the Lorenz curve made it the most widely reported inequality measure in the world.

5.2.6 Lorenz curves and the Gini coefficient from a distribution

Let $Y \geq 0$ have CDF F, quantile function F^{-1} and finite mean m. The *Lorenz curve* gives the share of total income held by the poorest fraction p of the population:

$$L(p) = \frac{1}{m}\int_0^p F^{-1}(u)\,\mathrm{d}u, \qquad p \in [0,1]. \tag{5.7}$$

L is increasing and convex with $L(0)=0$, $L(1)=1$ and $L(p)\le p$; the diagonal $L(p)=p$ is perfect equality. The *Gini coefficient* is twice the area between the diagonal and L:

$$G = 1 - 2\int_0^1 L(p)\,\mathrm{d}p = \frac{\mathbb{E}\,|Y_1 - Y_2|}{2m}, \tag{5.8}$$

where Y_1, Y_2 are independent copies of Y (the second form is Gini's mean difference).

Proposition 5.14 (Lorenz curve and Gini of the log-normal and Pareto). *(i) If* $\log Y \sim \mathcal{N}(\mu,\sigma^2)$ *then* $L(p)=\Phi(\Phi^{-1}(p)-\sigma)$ *and* $G = 2\Phi(\sigma/\sqrt{2})-1$.

(ii) If Y is Pareto with $\alpha>1$ then $L(p)=1-(1-p)^{1-1/\alpha}$ and $G=1/(2\alpha-1)$.
In both cases inequality depends only on the shape parameter (σ or α), not on the scale: multiplying all incomes by a constant leaves L and G unchanged.

Proof. (i) Write $Y=e^{\mu+\sigma Z}$. Then $F^{-1}(u)=e^{\mu+\sigma\Phi^{-1}(u)}$ and, substituting $z=\Phi^{-1}(u)$,

$$L(p)=\frac{1}{e^{\mu+\sigma^2/2}}\int_{-\infty}^{\Phi^{-1}(p)} e^{\mu+\sigma z}\phi(z)\,\mathrm{d}z = \int_{-\infty}^{\Phi^{-1}(p)}\phi(z-\sigma)\,\mathrm{d}z = \Phi(\Phi^{-1}(p)-\sigma),$$

completing the square $\sigma z - z^2/2 = -(z-\sigma)^2/2+\sigma^2/2$. For G, use the mean difference: a direct computation (Exercise 5.2.3) gives $\mathbb{E}\,|Y_1-Y_2| = 2m\{2\Phi(\sigma/\sqrt{2})-1\}$. (ii) $F^{-1}(u)=x_m(1-u)^{-1/\alpha}$ and $m=\alpha x_m/(\alpha-1)$, so $L(p)=\frac{\alpha-1}{\alpha}\int_0^p (1-u)^{-1/\alpha}\,\mathrm{d}u = 1-(1-p)^{1-1/\alpha}$, and $G=1-2\int_0^1 L = 1-2\{1-\alpha/(2\alpha-1)\}=1/(2\alpha-1)$. □

For a sample $y_{(1)}\le\cdots\le y_{(n)}$ the empirical analogue of (5.8) is $\hat G = \frac{2\sum_i i\,y_{(i)}}{n\sum_i y_i} - \frac{n+1}{n}$.

Listing 5.5: Log-normal incomes, Lorenz curve, Gini and poverty ch05/income_inequality.py

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Simulated household incomes: log-normal fit, Lorenz curve, Gini, poverty headcount
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 20_000                                  # simulated survey households
mu, sigma = np.log(12_000), 0.8             # parameters of log income
income = rng.lognormal(mu, sigma, n)        # annual income, simulated units


def gini(y):
    """Sample Gini coefficient via the sorted-rank formula."""
    y = np.sort(y)
    k = np.arange(1, len(y) + 1)
    return 2 * (k * y).sum() / (len(y) * y.sum()) - (len(y) + 1) / len(y)


# maximum likelihood fit of a log-normal (location fixed at zero)
s_hat, _, scale_hat = stats.lognorm.fit(income, floc=0)
print(f"true   mu = {mu:.4f}, sigma = {sigma:.4f}")
print(f"fitted mu = {np.log(scale_hat):.4f}, sigma = {s_hat:.4f}")
print(f"mean   = {income.mean():8.1f}  (theory {np.exp(mu + sigma**2 / 2):8.1f})")
print(f"median = {np.median(income):8.1f}  (theory {np.exp(mu):8.1f})")

g_theory = 2 * stats.norm.cdf(sigma / np.sqrt(2)) - 1
```

```python
print(f"Gini: sample = {gini(income):.4f}, log-normal formula = {g_theory:.4f}")

z = 6_000                                   # an illustrative poverty line
h_model = stats.norm.cdf((np.log(z) - np.log(scale_hat)) / s_hat)
print(f"poverty headcount at z = {z}: sample = {(income < z).mean():.4f}, "
      f"model = {h_model:.4f}")
gap = np.mean(np.maximum(z - income, 0) / z)
print(f"poverty gap index (FGT1) = {gap:.4f}")

# Lorenz curve: cumulative share of income held by the poorest fraction p
y = np.sort(income)
p = np.arange(1, n + 1) / n
L = np.cumsum(y) / y.sum()
pp = np.linspace(0, 1, 200)
L_theory = stats.norm.cdf(stats.norm.ppf(pp) - sigma)
print(f"bottom 40% share = {L[int(0.4 * n) - 1]:.4f}, "
      f"top 10% share = {1 - L[int(0.9 * n) - 1]:.4f}")

fig, ax = plt.subplots(figsize=(4.6, 4.2))
ax.plot([0, 1], [0, 1], "k--", lw=1, label="perfect equality")
ax.plot(p, L, color="#1f4e79", lw=2.5, label="simulated sample")
ax.plot(pp, L_theory, color="#cc9900", lw=1.5, ls=":", label="log-normal theory")
ax.fill_between(p, L, p, color="#1f4e79", alpha=0.12)
ax.set_xlabel("cumulative share of households")
ax.set_ylabel("cumulative share of income")
ax.legend(frameon=False, fontsize=8)
plt.savefig("ch05_lorenz.pdf", bbox_inches="tight")
```

Output

```
true   mu = 9.3927, sigma = 0.8000
fitted mu = 9.3885, sigma = 0.7901
mean   =  16347.2  (theory  16525.5)
median =  11913.7  (theory  12000.0)
Gini: sample = 0.4249, log-normal formula = 0.4284
poverty headcount at z = 6000: sample = 0.1924, model = 0.1916
poverty gap index (FGT1) = 0.0599
bottom 40% share = 0.1478, top 10% share = 0.3122
```

Policy Lens: Inequality and poverty from a fitted income distribution

Given a poverty line z, the *headcount ratio* is simply $H = F(z)$. Under a log-normal model $H = \Phi\big((\log z - \mu)/\sigma\big)$, so the effects of growth and redistribution can be read off directly: uniform growth at rate g shifts μ to $\mu + \log(1+g)$ and lowers H, while a mean-preserving fall in inequality lowers σ (and lowers H whenever z is below the median). Both H and the poverty-gap index $\mathbb{E}[\max(z - Y, 0)/z]$ are members of the Foster–Greer–Thorbecke family (Foster et al., 1984). In the simulated survey above, the model-based headcount (0.192) agrees with the sample headcount (0.192). Parametric models are particularly valuable when only grouped data (decile shares) are published, because a two-parameter distribution can be fitted to a handful of Lorenz-curve points. Their danger is the top tail: a log-normal fitted to the body of the distribution understates the income share of the very rich.

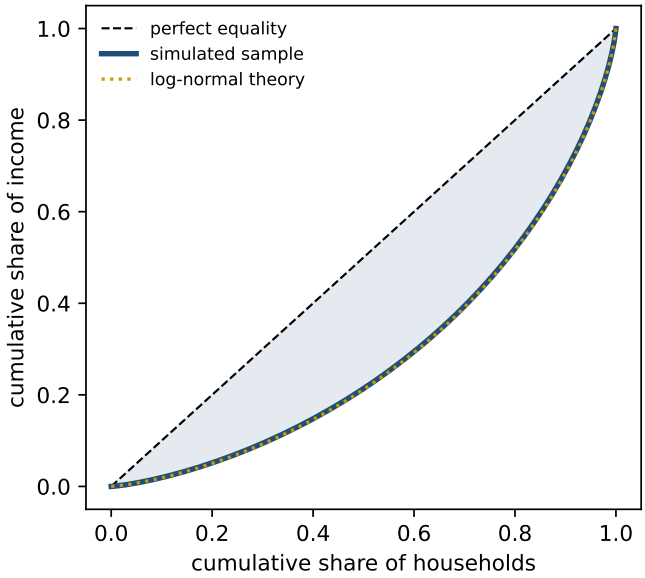


Figure 5.4: Lorenz curve of a simulated log-normal income sample ($\sigma = 0.8$). The shaded area equals $G/2$; the dotted curve is $\Phi(\Phi^{-1}(p) - \sigma)$.

Listing 5.6: Pareto tails and the Hill estimator ch05/pareto_tail.py

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Pareto tails of top incomes: log-log survival plot and the Hill estimator
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n = 20_000
logn = rng.lognormal(np.log(12_000), 0.8, n)          # log-normal body
alpha, xm = 2.0, 60_000.0                             # Pareto top tail
par = xm * (1 + rng.pareto(alpha, n))                 # classical Pareto
# mixture: 95% log-normal, 5% Pareto top earners (simulated)
income = np.where(rng.random(n) < 0.95, logn, par)


def hill(x, k):
    """Hill estimator of the tail index from the k largest observations."""
    xs = np.sort(x)[::-1]
    return 1.0 / np.mean(np.log(xs[:k] / xs[k]))


for k in [100, 200, 500, 1000]:
    print(f"Hill estimate of alpha with k = {k:4d}: {hill(income, k):.3f}")
print(f"true Pareto tail index alpha = {alpha}")
print(f"Pareto mean x_m*alpha/(alpha-1) = {xm * alpha / (alpha - 1):.0f}")
top1 = np.sort(income)[-n // 100:].sum() / income.sum()
print(f"top 1% income share: mixture = {top1:.4f}, "
      f"log-normal only = {np.sort(logn)[-n // 100:].sum() / logn.sum():.4f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
for x, lab, col in [(logn, "log-normal only", "#888888"),
                    (income, "log-normal + Pareto tail", "#1f4e79")]:
```

```
    xs = np.sort(x)
    surv = 1 - np.arange(len(xs)) / len(xs)
    ax.loglog(xs, surv, lw=2, color=col, label=lab)
grid = np.logspace(np.log10(xm), 6.5, 50)
ax.loglog(grid, 0.05 * (grid / xm) ** (-alpha), "--", color="#cc9900",
          label="reference line, slope = -alpha")
ax.set_ylim(1e-4, 1.1)
ax.set_xlabel("income (simulated units)")
ax.set_ylabel("P(Income > x)")
ax.legend(frameon=False, fontsize=8)
plt.savefig("ch05_pareto_tail.pdf", bbox_inches="tight")
```

Output

```
Hill estimate of alpha with k =  100: 2.011
Hill estimate of alpha with k =  200: 2.219
Hill estimate of alpha with k =  500: 2.211
Hill estimate of alpha with k = 1000: 2.246
true Pareto tail index alpha = 2.0
Pareto mean x_m*alpha/(alpha-1) = 120000
top 1% income share: mixture = 0.1255, log-normal only = 0.0628
```

The Hill estimator (Hill, 1975) of the tail index, using the k largest observations $x_{(1)} \geq x_{(2)} \geq \dots$, is $\hat{\alpha}_k = \left\{ k^{-1} \sum_{i=1}^{k} \log(x_{(i)}/x_{(k+1)}) \right\}^{-1}$ — the maximum likelihood estimator of α for Pareto data above the threshold $x_{(k+1)}$. It is accurate when k is small enough that only tail observations are used, and biased as k grows and the log-normal body contaminates the sample: the familiar bias–variance trade-off of tail estimation. Adding a 5% Pareto top tail doubles the top-1% income share relative to the log-normal alone (Fig. 5.5).

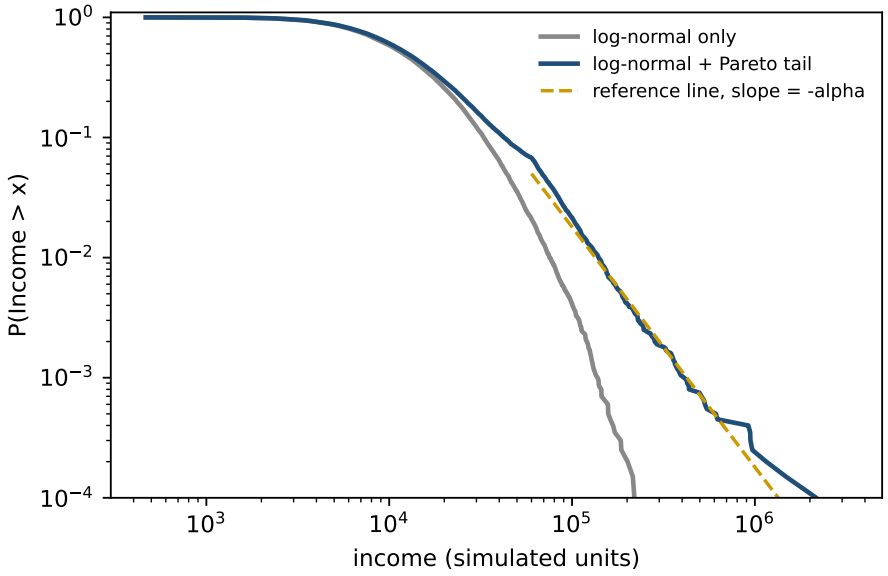


Figure 5.5: Empirical survival functions on log–log axes. A log-normal tail bends down; a Pareto tail is a straight line with slope $-\alpha$.

5.2.7 A catalogue in code

Listing 5.7: Moments of continuous distributions ch05/continuous_pdfs.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Densities and moments of common continuous distributions with scipy.stats
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

dists = {
    "Uniform(0,1)": stats.uniform(0, 1),
    "Normal(0,1)": stats.norm(0, 1),
    "Exponential(rate 2)": stats.expon(scale=1 / 2),
    "Gamma(3, rate 2)": stats.gamma(a=3, scale=1 / 2),
    "Beta(2,5)": stats.beta(2, 5),
    "LogNormal(0,0.5)": stats.lognorm(s=0.5, scale=1.0),
    "Pareto(alpha 3)": stats.pareto(b=3),
    "Weibull(k 1.5)": stats.weibull_min(c=1.5),
}
print(f"{'distribution':21s}{'mean':>8s}{'var':>8s}{'skew':>8s}{'median':>8s}")
for name, d in dists.items():
    mean, var, skew = d.stats(moments="mvs")
    print(f"{name:21s}{float(mean):8.3f}{float(var):8.3f}"
          f"{float(skew):8.3f}{d.median():8.3f}")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.4))
x = np.linspace(0.001, 4, 400)
for name in ["Exponential(rate 2)", "Gamma(3, rate 2)", "LogNormal(0,0.5)",
             "Weibull(k 1.5)"]:
    axes[0].plot(x, dists[name].pdf(x), lw=2, label=name)
axes[0].set_xlabel("x")
axes[0].set_ylabel("density")
axes[0].legend(frameon=False, fontsize=8)
u = np.linspace(0.001, 0.999, 400)
for a, b in [(1, 1), (2, 5), (5, 2), (0.5, 0.5)]:
    axes[1].plot(u, stats.beta(a, b).pdf(u), lw=2, label=f"Beta({a},{b})")
axes[1].set_xlabel("proportion")
axes[1].set_ylim(0, 3.5)
axes[1].legend(frameon=False, fontsize=8)
fig.tight_layout()
plt.savefig("ch05_continuous_pdfs.pdf", bbox_inches="tight")
```

Output

```
distribution            mean     var    skew  median
Uniform(0,1)           0.500   0.083   0.000   0.500
Normal(0,1)            0.000   1.000   0.000   0.000
Exponential(rate 2)    0.500   0.250   2.000   0.347
Gamma(3, rate 2)       1.500   0.750   1.155   1.337
Beta(2,5)              0.286   0.026   0.596   0.264
LogNormal(0,0.5)       1.133   0.365   1.750   1.000
Pareto(alpha 3)        1.500   0.750     nan   1.260
Weibull(k 1.5)         0.903   0.376   1.072   0.783
```

The Pareto with $\alpha = 3$ has a nan skewness because its third moment is infinite ($\mathbb{E}X^r < \infty$ if and only if $r < \alpha$).

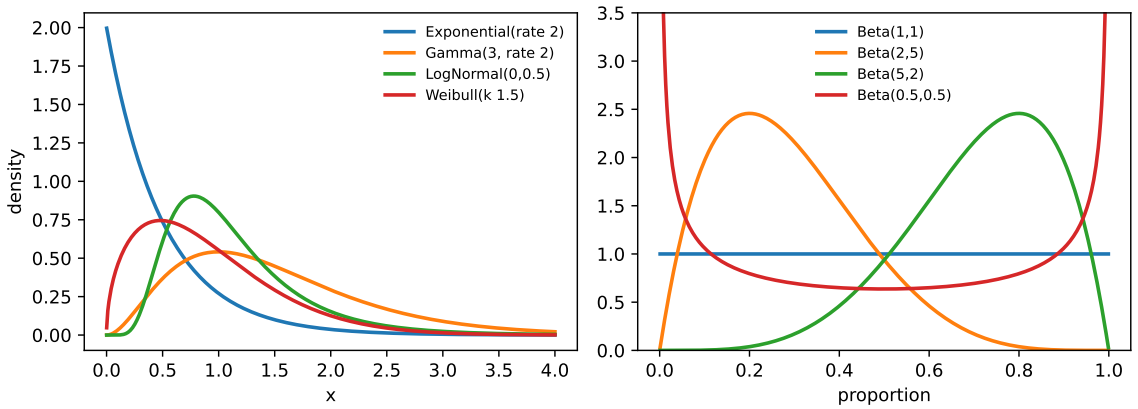


Figure 5.6: Left: right-skewed densities for positive quantities (durations, incomes). Right: the flexibility of the beta family on $(0, 1)$.

⋆ Key Concept: Matching distributions to data-generating stories

- Additive accumulation of many small effects ⇒ **normal**.
- Multiplicative accumulation (proportional growth) ⇒ **log-normal**.
- Scale-invariant tails (top incomes, firm sizes, city sizes) ⇒ **Pareto**.
- Waiting times: constant hazard ⇒ **exponential**; monotone hazard ⇒ **Weibull**; sums of waiting times ⇒ **gamma**.
- Proportions on $(0, 1)$ ⇒ **beta**.

Inequality measures (Lorenz, Gini) are functionals of the distribution and, for these families, depend only on shape parameters.

Exercises

5.2.1 Theory Show that if $U \sim \text{Unif}(0, 1)$ then $-\log(U)/\lambda \sim \text{Exp}(\lambda)$ and $\lambda(-\log U)^{1/k}$ is Weibull(k, λ). Derive the Weibull mean $\lambda\Gamma(1 + 1/k)$.

5.2.2 Theory Prove that the minimum of independent $\text{Exp}(\lambda_1), \ldots, \text{Exp}(\lambda_J)$ durations is $\text{Exp}(\sum_j \lambda_j)$ and that the probability that spell j ends first is $\lambda_j / \sum_l \lambda_l$. Interpret in terms of competing exits from unemployment (to employment, to inactivity, to migration).

5.2.3 Theory Complete the proof of Proposition 5.14(i) by computing $\mathbb{E}\,|Y_1 - Y_2|$ for independent log-normals. *Hint:* write $\mathbb{E}\,|Y_1 - Y_2| = 2\mathbb{E}[(Y_1 - Y_2)^+]$ and use the formula for $\mathbb{E}[e^W \mathbf{1}\{W > c\}]$ with W normal.

5.2.4 Theory Suppose a fraction π of job seekers have hazard λ_H and the rest $\lambda_L < \lambda_H$, each constant. Show that the population hazard $h(t) = f(t)/S(t)$ is strictly decreasing in t and tends to λ_L.

5.2.5 Python Fit a Pareto distribution by maximum likelihood to the incomes above 60,000 in Listing 5.6 (use `stats.pareto.fit(x, floc=0, fscale=60000)`). Compare with the Hill estimator, and plot $\hat{\alpha}_k$ against k (a "Hill plot").

5.2.6 Python Using Listing 5.5, compute the headcount, poverty gap and squared poverty gap (FGT_2) after (a) a uniform 10% income increase and (b) a reduction of σ from 0.8 to 0.7 holding the mean constant (adjust μ accordingly). Which policy reduces FGT_2 more?

5.2.7 ★ Graduate The log-normal Gini depends only on σ, so $\hat{G} = 2\Phi(\hat{\sigma}/\sqrt{2}) - 1$ is a parametric estimator. Use the delta method to derive its asymptotic standard error when $\hat{\sigma}$ is the MLE, and compare by simulation with the sampling variability of the nonparametric $\hat{G}$ when the data are (a) log-normal and (b) the log-normal–Pareto mixture. Which is more robust to misspecification?

5.2.8 Policy Grouped data published by a statistics office report that the poorest 40% hold 15% of income. Assuming a log-normal model, solve for σ, then report the implied Gini coefficient and the top-10% share. Discuss what information is lost.

5.3 Sampling Distributions: Chi-square, Student t and F

The distributions of this section are rarely models for raw data. They are the distributions of *statistics* — sample variances, t-ratios, F-statistics — computed from normal samples, and they are the reason every regression table contains standard errors and p-values.

5.3.1 The chi-square distribution

Definition 5.15 (Chi-square)**.** If $Z_1, \dots, Z_\nu \overset{\text{iid}}{\sim} \mathcal{N}(0,1)$, then $Q = \sum_{i=1}^{\nu} Z_i^2$ has the chi-square distribution with ν degrees of freedom, $Q \sim \chi^2_\nu$.

Proposition 5.16. *$\chi^2_\nu = \mathrm{Gamma}(\nu/2, \text{rate } 1/2)$, with density $f(q) = q^{\nu/2-1}e^{-q/2}/\{2^{\nu/2}\Gamma(\nu/2)\}$, mean ν and variance 2ν.*

Proof. For $\nu = 1$: $\mathbb{P}(Z^2 \le q) = 2\Phi(\sqrt{q}) - 1$, so the density is $\phi(\sqrt{q})/\sqrt{q} = (2\pi)^{-1/2} q^{-1/2} e^{-q/2}$, the $\mathrm{Gamma}(1/2, 1/2)$ density (using $\Gamma(1/2) = \sqrt{\pi}$). The MGF of $\mathrm{Gamma}(\alpha, \beta)$ is $(1 - t/\beta)^{-\alpha}$; a sum of ν independent $\mathrm{Gamma}(1/2, 1/2)$ variables therefore has MGF $(1 - 2t)^{-\nu/2}$, which is that of $\mathrm{Gamma}(\nu/2, 1/2)$. The moments follow from $\mathbb{E} = \alpha/\beta$, $\mathrm{Var} = \alpha/\beta^2$. □

More generally, if $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$ and $\boldsymbol{P}$ is a symmetric idempotent matrix (an orthogonal projection) of rank r, then $\mathbf{z}^\top \boldsymbol{P} \mathbf{z} \sim \chi^2_r$: diagonalise $\boldsymbol{P} = \boldsymbol{U}\,\mathrm{diag}(\mathbf{1}_r, \mathbf{0})\boldsymbol{U}^\top$ and note $\boldsymbol{U}^\top \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$. This is the engine behind the next theorem.

Theorem 5.17 (Sample mean and variance of a normal sample)**.** *Let $X_1, \dots, X_n \overset{iid}{\sim} \mathcal{N}(\mu, \sigma^2)$, $\bar{X} = n^{-1}\sum X_i$ and $S^2 = (n-1)^{-1}\sum (X_i - \bar{X})^2$. Then (i) $\bar{X} \sim \mathcal{N}(\mu, \sigma^2/n)$; (ii) $(n-1)S^2/\sigma^2 \sim \chi^2_{n-1}$; (iii) $\bar{X}$ and S^2 are independent.*

Proof. Let $\mathbf{z} = (\mathbf{X} - \mu\mathbf{1})/\sigma \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$, $\boldsymbol{P}_1 = n^{-1}\mathbf{1}\mathbf{1}^\top$ and $\boldsymbol{M} = \mathbf{I} - \boldsymbol{P}_1$. Both are orthogonal projections, of ranks 1 and $n-1$, with $\boldsymbol{P}_1\boldsymbol{M} = \mathbf{0}$. Then $(n-1)S^2/\sigma^2 = \mathbf{z}^\top \boldsymbol{M}\mathbf{z} \sim \chi^2_{n-1}$, proving (ii). The vectors $\boldsymbol{P}_1\mathbf{z}$ and $\boldsymbol{M}\mathbf{z}$ are jointly normal with covariance $\boldsymbol{P}_1\boldsymbol{M}^\top = \mathbf{0}$, hence independent (Section 5.4); $\bar{X}$ is a function of the first and S^2 of the second, proving (iii). □

The theorem is a special case of Cochran (1934)'s theorem on partitioning a sum of squares into independent chi-square components — the mathematical basis of the analysis of variance. Independence of $\bar{X}$ and S^2 *characterises* the normal distribution: for no other distribution are they independent.

5.3.2 Student's t distribution

Definition 5.18 (Student t)**.** If $Z \sim \mathcal{N}(0,1)$ and $Q \sim \chi^2_\nu$ are independent, $T = Z/\sqrt{Q/\nu}$ has the t distribution with ν degrees of freedom, with density

$$f(t) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)}\left(1 + \frac{t^2}{\nu}\right)^{-(\nu+1)/2}, \qquad t \in \mathbb{R}. \tag{5.9}$$

The density follows by conditioning on Q: given $Q = q$, $T \sim \mathcal{N}(0, \nu/q)$, and integrating against the χ^2_ν density gives (5.9) (Exercise **5.3.1**). Equivalently, T is a *scale mixture of normals*: a normal whose precision is gamma-distributed. The t has polynomial tails, $f(t) \sim c\,|t|^{-(\nu+1)}$, so only moments of order less than ν exist; $\operatorname{Var} T = \nu/(\nu-2)$ for $\nu > 2$. As $\nu \to \infty$, $Q/\nu \xrightarrow{p} 1$ and $T \xrightarrow{d} \mathcal{N}(0,1)$.

By Theorem 5.17, $\sqrt{n}(\bar{X} - \mu)/\sigma \sim \mathcal{N}(0,1)$ is independent of $(n-1)S^2/\sigma^2 \sim \chi^2_{n-1}$, so the unknown σ cancels in

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{\sqrt{n}(\bar{X} - \mu)/\sigma}{\sqrt{S^2/\sigma^2}} \sim t_{n-1}. \tag{5.10}$$

This is a *pivotal quantity*: its distribution does not depend on unknown parameters.

Historical Note: "Student" at the Guinness brewery

William Sealy Gosset was a chemist and statistician at the Guinness brewery in Dublin, where experiments on barley and hops yielded only small samples. Treating S as if it were σ gave intervals that were too narrow, and in "The probable error of a mean", published in *Biometrika* in 1908 under the pseudonym "Student" (Student, 1908), he derived the distribution of the ratio (with a combination of mathematics and simulation by hand on anthropometric data). Guinness permitted publication only anonymously. R. A. Fisher later gave a rigorous derivation and recast the statistic in the form t now used.

5.3.3 The F distribution

Definition 5.19 (F)**.** If $Q_1 \sim \chi^2_{d_1}$ and $Q_2 \sim \chi^2_{d_2}$ are independent, then $F = (Q_1/d_1)/(Q_2/d_2) \sim F_{d_1,d_2}$. Its mean is $d_2/(d_2-2)$ for $d_2 > 2$.

Immediate consequences: $1/F \sim F_{d_2,d_1}$ and, if $T \sim t_\nu$, $T^2 \sim F_{1,\nu}$. The F distribution is named in honour of Fisher, who studied the closely related distribution of $\frac{1}{2}\log F$; the letter F was introduced by George Snedecor in the 1930s.

5.3.4 Why these distributions govern regression inference

Consider the classical normal linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I}_n)$, with $\mathbf{X}$ of full column rank k. Let $\boldsymbol{P} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ and $\boldsymbol{M} = \mathbf{I} - \boldsymbol{P}$ (Chapter 2). Then $\hat{\boldsymbol{\beta}} - \boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1})$ and the residual sum of squares is $\boldsymbol{\varepsilon}^\top\boldsymbol{M}\boldsymbol{\varepsilon}$, so

$$\frac{\mathrm{RSS}}{\sigma^2} \sim \chi^2_{n-k}, \qquad s^2 = \frac{\mathrm{RSS}}{n-k},$$

independent of $\hat{\boldsymbol{\beta}}$ because $\boldsymbol{M}\mathbf{X} = \mathbf{0}$. Consequently

$$\frac{\hat{\beta}_j - \beta_j}{s\sqrt{[(\mathbf{X}^\top\mathbf{X})^{-1}]_{jj}}} \sim t_{n-k}, \qquad \frac{(\boldsymbol{R}\hat{\boldsymbol{\beta}} - \boldsymbol{r})^\top[\boldsymbol{R}(\mathbf{X}^\top\mathbf{X})^{-1}\boldsymbol{R}^\top]^{-1}(\boldsymbol{R}\hat{\boldsymbol{\beta}} - \boldsymbol{r})/q}{s^2} \sim F_{q,n-k} \tag{5.11}$$

under $H_0 : \boldsymbol{R\beta} = \boldsymbol{r}$ with q restrictions. The t and F tests of every regression package are exact under normal errors; without normality they are justified asymptotically, because $t_{n-k} \to \mathcal{N}(0,1)$ and $qF_{q,n-k} \xrightarrow{d} \chi^2_q$ (Hansen, 2022, Ch. 5).

Listing 5.8: Building χ^2, t and F from normals ch05/sampling_dists.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Build chi-square, t and F variables from normals and compare with scipy.stats
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
R, k, m = 200_000, 5, 10
Z = rng.standard_normal((R, k))
chi2_k = (Z**2).sum(axis=1)                       # chi-square(k)
chi2_m = (rng.standard_normal((R, m)) ** 2).sum(axis=1)
t_k = rng.standard_normal(R) / np.sqrt(chi2_k / k)   # Student t(k)
F_km = (chi2_k / k) / (chi2_m / m)                # F(k, m)

print("95th percentiles: simulated vs exact")
for name, x, d in [("chi2(5)", chi2_k, stats.chi2(k)),
                   ("t(5)", t_k, stats.t(k)),
                   ("F(5,10)", F_km, stats.f(k, m))]:
    print(f"  {name:8s} {np.quantile(x, 0.95):7.3f}  {d.ppf(0.95):7.3f}"
          f"   mean {x.mean():6.3f} vs {d.mean():6.3f}")

# t(k)^2 is F(1,k)
print(f"P(t(5)^2 > 3) = {(t_k**2 > 3).mean():.4f},  "
      f"F(1,5) tail = {stats.f(1, k).sf(3):.4f}")

# Cochran: (n-1) S^2 / sigma^2 ~ chi2(n-1) for normal samples of size n
n, sigma = 8, 2.0
X = rng.normal(5.0, sigma, size=(R, n))
q = (n - 1) * X.var(axis=1, ddof=1) / sigma**2
print(f"(n-1)S^2/sigma^2: mean {q.mean():.3f} (theory {n - 1}), "
      f"var {q.var():.3f} (theory {2 * (n - 1)})")
print(f"corr(Xbar, S^2) = {np.corrcoef(X.mean(1), X.var(1, ddof=1))[0, 1]:.4f}")

print("two-sided 5% critical values of t(df) versus z = 1.960")
print("  " + "  ".join(f"df={d}: {stats.t(d).ppf(0.975):.3f}"
                       for d in [2, 5, 10, 30, 120]))

fig, ax = plt.subplots(figsize=(6, 3.8))
x = np.linspace(-5, 5, 400)
ax.plot(x, stats.norm.pdf(x), "k", lw=2.5, label="N(0,1)")
for df, col in [(1, "#b22222"), (3, "#cc9900"), (10, "#1f4e79")]:
    lab = f"t({df})" + (" = Cauchy" if df == 1 else "")
    ax.plot(x, stats.t(df).pdf(x), lw=1.8, color=col, label=lab)
ax.set_xlabel("x")
ax.set_ylabel("density")
ax.legend(frameon=False)
plt.savefig("ch05_t_vs_normal.pdf", bbox_inches="tight")
```

Output

```
95th percentiles: simulated vs exact
  chi2(5)   11.042   11.070   mean  4.997 vs  5.000
  t(5)       2.031    2.015   mean  0.005 vs  0.000
  F(5,10)    3.335    3.326   mean  1.253 vs  1.250
P(t(5)^2 > 3) = 0.1436,  F(1,5) tail = 0.1438
(n-1)S^2/sigma^2: mean 6.998 (theory 7), var 13.935 (theory 14)
corr(Xbar, S^2) = -0.0009
two-sided 5% critical values of t(df) versus z = 1.960
  df=2: 4.303  df=5: 2.571  df=10: 2.228  df=30: 2.042  df=120: 1.980
```

⋆ Key Concept: The normal family tree

$$Z \sim \mathcal{N}(0,1) \;\Rightarrow\; \sum_{i=1}^{\nu} Z_i^2 \sim \chi^2_\nu, \qquad \frac{Z}{\sqrt{\chi^2_\nu/\nu}} \sim t_\nu, \qquad \frac{\chi^2_{d_1}/d_1}{\chi^2_{d_2}/d_2} \sim F_{d_1,d_2}, \qquad t^2_\nu \sim F_{1,\nu}.$$

The numerator and denominator must be *independent*; in regression this independence comes from the orthogonality $\boldsymbol{PM} = \mathbf{0}$ of the fitted values and residuals.

5.3.5 The Cauchy distribution: a warning

The t_1 distribution is the standard *Cauchy*, with density $1/\{\pi(1+x^2)\}$; it is also the distribution of the ratio Z_1/Z_2 of two independent standard normals. Its tails are so heavy that $\mathbb{E}|X| = \infty$: the mean does not exist. Worse, the characteristic function is $e^{-|t|}$, so the sample mean of n iid Cauchy variables has characteristic function $(e^{-|t|/n})^n = e^{-|t|}$ — *the average of n Cauchy draws is again standard Cauchy*. Averaging does not help at all; the law of large numbers and the central limit theorem both fail because their moment conditions fail.

Listing 5.9: Running means under the Cauchy distribution ch05/cauchy_mean.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# The Cauchy warning: running sample means that never settle down
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n = 100_000
idx = np.arange(1, n + 1)
normal = rng.standard_normal(n)
cauchy = rng.standard_normal(n) / rng.standard_normal(n)   # ratio of normals

run_norm = np.cumsum(normal) / idx
run_cauchy = np.cumsum(cauchy) / idx
print("    n    mean(normal)   mean(Cauchy)   median(Cauchy)")
for m in [10, 100, 1000, 10_000, 100_000]:
    print(f"{m:6d}{run_norm[m - 1]:14.4f}{run_cauchy[m - 1]:15.4f}"
          f"{np.median(cauchy[:m]):16.4f}")

# the mean of n Cauchy draws is again standard Cauchy: check its IQR
means = (rng.standard_normal((20_000, 50))
         / rng.standard_normal((20_000, 50))).mean(axis=1)
q1, q3 = np.quantile(means, [0.25, 0.75])
```

```
print(f"IQR of means of 50 Cauchy draws = [{q1:.3f}, {q3:.3f}] (theory [-1, 1])")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.semilogx(idx, run_cauchy, color="#b22222", lw=1.2, label="Cauchy")
ax.semilogx(idx, run_norm, color="#1f4e79", lw=1.8, label="normal")
ax.axhline(0, color="k", lw=0.6)
ax.set_xlabel("sample size n")
ax.set_ylabel("running sample mean")
ax.legend(frameon=False)
plt.savefig("ch05_cauchy_mean.pdf", bbox_inches="tight")
```

Output

```
     n    mean(normal)   mean(Cauchy)   median(Cauchy)
    10         -0.2281        -3.9651          -0.8939
   100         -0.0756      -906.7327          -0.3524
  1000          0.0069       -91.4873           0.0275
 10000         -0.0081        -9.2081           0.0014
100000          0.0005        -1.8387          -0.0040
IQR of means of 50 Cauchy draws = [-1.028, 0.963] (theory [-1, 1])
```

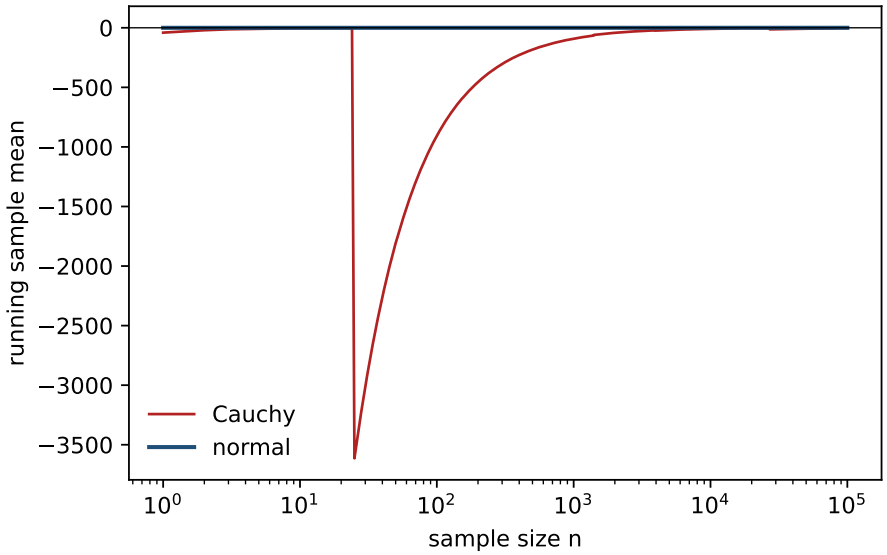


Figure 5.7: Running sample means. The normal mean converges; the Cauchy mean jumps whenever a huge observation arrives and never settles. The sample median, by contrast, is consistent.

△ Caution: Heavy tails break averages

Ratios of noisy quantities (for example, a ratio of two estimated growth rates, or a cost per beneficiary when the number of beneficiaries is noisy and can be near zero) can behave like Cauchy variables. Means, OLS and standard errors are then unreliable, and robust summaries — medians, trimmed means, quantile regression — should be used. Always look at the tails of a distribution before averaging it.

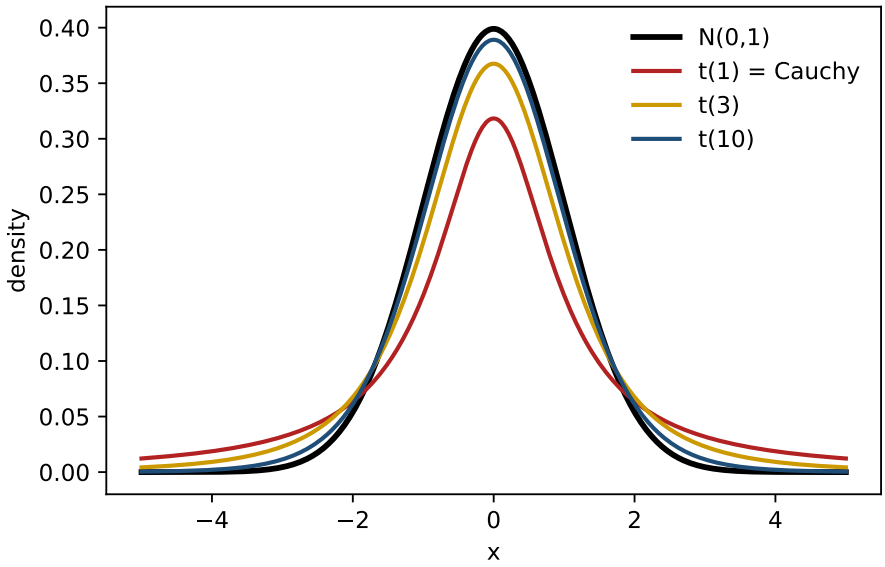


Figure 5.8: Student t densities approach the normal as the degrees of freedom grow; t_1 is the Cauchy.

Graduate Perspective: Pivots, exactness and asymptotics

Exact finite-sample t and F distributions require normal, homoskedastic errors — an assumption almost never credible in microeconometrics. Modern practice relies instead on asymptotic normality with heteroskedasticity-robust (sandwich) or cluster-robust standard errors (Hansen, 2022, Ch. 7), or on the bootstrap (Chapter 11). The t_{n-k} critical values are still used as a mild small-sample correction. With few clusters, however, even cluster-robust t-tests over-reject badly, and wild-cluster bootstrap or t critical values with $G-1$ degrees of freedom are recommended.

Exercises

5.3.1 Theory Derive the t_ν density (5.9) by writing the joint density of (Z, Q), changing variables to (T, Q) and integrating out Q.

5.3.2 Theory Show that if $T \sim t_\nu$ then $T^2 \sim F_{1,\nu}$, and that if $F \sim F_{d_1,d_2}$ then $d_1 F/(d_1 F + d_2) \sim \text{Beta}(d_1/2, d_2/2)$.

5.3.3 Theory Prove that the ratio Z_1/Z_2 of independent standard normals is standard Cauchy, and that $\mathbb{E}|X| = \infty$ for the Cauchy distribution.

5.3.4 Python Verify Theorem 5.17 fails without normality: repeat the last part of Listing 5.8 with exponential data and report the variance of $(n-1)S^2/\sigma^2$ and the correlation between $\bar{X}$ and S^2. Explain both numbers.

5.3.5 Python Simulate a regression with $n = 15$, $k = 3$ and t_3-distributed errors. Estimate the actual rejection rate of the nominal 5% t-test of a true null hypothesis using t_{12} critical values, and compare with normal errors.

5.3.6 ★ Graduate Let $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$ and $\boldsymbol{A}$ symmetric. Prove that $\mathbf{z}^\top \boldsymbol{A} \mathbf{z} \sim \chi^2_r$ if and only if $\boldsymbol{A}$ is idempotent of rank r. (Hint: compare the MGF $\prod_i (1-2t\lambda_i)^{-1/2}$ with $(1-2t)^{-r/2}$.)

5.3.7 Policy An evaluation reports "cost per job created" in 40 districts, computed as programme spending divided by the estimated number of jobs, where the latter is noisy and sometimes

close to zero. Explain, with reference to the Cauchy distribution, why the average cost per job across districts may be meaningless, and propose a better summary.

5.4 The Multivariate Normal Distribution

5.4.1 Definition and density

Definition 5.20 (Multivariate normal)**.** A random vector $\mathbf{X} \in \mathbb{R}^d$ is multivariate normal, $\mathbf{X} \sim \mathcal{N}_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, if $\boldsymbol{a}^\top \mathbf{X}$ is univariate normal for every $\boldsymbol{a} \in \mathbb{R}^d$. Equivalently, $\mathbf{X} = \boldsymbol{\mu} + \boldsymbol{A}\mathbf{z}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_m)$ and $\boldsymbol{A}\boldsymbol{A}^\top = \boldsymbol{\Sigma}$. If $\boldsymbol{\Sigma}$ is positive definite, $\mathbf{X}$ has density

$$f(\mathbf{x}) = (2\pi)^{-d/2} \det(\boldsymbol{\Sigma})^{-1/2} \exp\left\{-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right\}. \tag{5.12}$$

The density is constant on the ellipsoids $(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) = c^2$, whose axes are the eigenvectors of $\boldsymbol{\Sigma}$ with half-lengths $c\sqrt{\lambda_j}$ (Chapter 3). The definition via linear combinations also covers *degenerate* normals with singular $\boldsymbol{\Sigma}$, which concentrate on an affine subspace and have no density — for instance the vector of OLS residuals, which lies in the $(n-k)$-dimensional space $\mathrm{col}(\mathbf{X})^\perp$.

Theorem 5.21 (Affine transformations)**.** *If $\mathbf{X} \sim \mathcal{N}_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, $\boldsymbol{B} \in \mathbb{R}^{m\times d}$ and $\mathbf{b} \in \mathbb{R}^m$, then $\boldsymbol{B}\mathbf{X} + \mathbf{b} \sim \mathcal{N}_m(\boldsymbol{B}\boldsymbol{\mu} + \mathbf{b},\ \boldsymbol{B}\boldsymbol{\Sigma}\boldsymbol{B}^\top)$.*

Proof. For any $\boldsymbol{a} \in \mathbb{R}^m$, $\boldsymbol{a}^\top(\boldsymbol{B}\mathbf{X}+\mathbf{b}) = (\boldsymbol{B}^\top\boldsymbol{a})^\top\mathbf{X} + \boldsymbol{a}^\top\mathbf{b}$ is univariate normal, so $\boldsymbol{B}\mathbf{X}+\mathbf{b}$ is multivariate normal; its mean and covariance follow from linearity of expectation and $\mathrm{Cov}(\boldsymbol{B}\mathbf{X}) = \boldsymbol{B}\boldsymbol{\Sigma}\boldsymbol{B}^\top$. □

Two consequences are used constantly. *Marginals:* taking $\boldsymbol{B}$ to select a subvector shows that every subvector of a normal vector is normal, with the corresponding sub-blocks of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. *Independence:* for jointly normal vectors, zero covariance implies independence, because if $\boldsymbol{\Sigma}$ is block diagonal the density (5.12) factorises. (Marginal normality of each component is *not* enough: see Exercise **5.4.1**.)

5.4.2 Conditional distributions and the Schur complement

Partition $\mathbf{X} = (\mathbf{X}_1, \mathbf{X}_2)$ with $\mathbf{X}_1 \in \mathbb{R}^p$, $\mathbf{X}_2 \in \mathbb{R}^q$ and

$$\boldsymbol{\mu} = \begin{pmatrix}\boldsymbol{\mu}_1\\ \boldsymbol{\mu}_2\end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix}\boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12}\\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22}\end{pmatrix}, \qquad \boldsymbol{\Sigma}_{22} \succ 0.$$

Theorem 5.22 (Conditional distribution)**.** $\mathbf{X}_1 \mid \mathbf{X}_2 = \mathbf{x}_2 \sim \mathcal{N}_p\left(\boldsymbol{\mu}_{1|2},\ \boldsymbol{\Sigma}_{1|2}\right)$ *with*

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2), \qquad \boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}. \tag{5.13}$$

The conditional mean is linear in $\mathbf{x}_2$ and the conditional covariance $\boldsymbol{\Sigma}_{1|2}$ — the Schur complement *of $\boldsymbol{\Sigma}_{22}$ in $\boldsymbol{\Sigma}$ — does not depend on $\mathbf{x}_2$.*

Proof. Let $\boldsymbol{W} = \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}$ and define $\boldsymbol{E} = \mathbf{X}_1 - \boldsymbol{\mu}_1 - \boldsymbol{W}(\mathbf{X}_2 - \boldsymbol{\mu}_2)$. The pair $(\boldsymbol{E}, \mathbf{X}_2)$ is an affine function of $\mathbf{X}$ and hence jointly normal, and

$$\mathrm{Cov}(\boldsymbol{E}, \mathbf{X}_2) = \boldsymbol{\Sigma}_{12} - \boldsymbol{W}\boldsymbol{\Sigma}_{22} = \mathbf{0},$$

so $\boldsymbol{E}$ is independent of $\mathbf{X}_2$. Its distribution is $\mathcal{N}(\mathbf{0}, \mathrm{Cov}(\boldsymbol{E}))$ with

$$\mathrm{Cov}(\boldsymbol{E}) = \boldsymbol{\Sigma}_{11} - \boldsymbol{W}\boldsymbol{\Sigma}_{21} - \boldsymbol{\Sigma}_{12}\boldsymbol{W}^\top + \boldsymbol{W}\boldsymbol{\Sigma}_{22}\boldsymbol{W}^\top = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}.$$

Since $\mathbf{X}_1 = \boldsymbol{\mu}_1 + \boldsymbol{W}(\mathbf{X}_2 - \boldsymbol{\mu}_2) + \boldsymbol{E}$ and $\boldsymbol{E}$ is independent of $\mathbf{X}_2$, conditioning on $\mathbf{X}_2 = \mathbf{x}_2$ fixes the first two terms and leaves $\boldsymbol{E}$ unchanged, giving (5.13).

For an algebraic view through the density, use the block factorisation

$$\boldsymbol{\Sigma} = \begin{pmatrix} \mathbf{I} & \boldsymbol{W} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \boldsymbol{\Sigma}_{1|2} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \boldsymbol{W}^\top & \mathbf{I} \end{pmatrix},$$

which gives $\det \boldsymbol{\Sigma} = \det \boldsymbol{\Sigma}_{1|2} \det \boldsymbol{\Sigma}_{22}$ and

$$(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) = (\mathbf{x}_1 - \boldsymbol{\mu}_{1|2})^\top \boldsymbol{\Sigma}_{1|2}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_{1|2}) + (\mathbf{x}_2 - \boldsymbol{\mu}_2)^\top \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2).$$

Hence $f(\mathbf{x}_1, \mathbf{x}_2) = f_{1|2}(\mathbf{x}_1 \mid \mathbf{x}_2)\, f_2(\mathbf{x}_2)$ with $f_{1|2}$ the $\mathcal{N}(\boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2})$ density. □

5.4.3 Relation to linear regression

When $p = 1$, (5.13) says that $\mathbb{E}[X_1 \mid \mathbf{X}_2] = \beta_0 + \boldsymbol{\beta}^\top \mathbf{X}_2$ with $\boldsymbol{\beta} = \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21}$ and $\beta_0 = \mu_1 - \boldsymbol{\beta}^\top \boldsymbol{\mu}_2$ — precisely the population least-squares coefficients $\boldsymbol{\beta} = \operatorname{Var}(\mathbf{X}_2)^{-1} \operatorname{Cov}(\mathbf{X}_2, X_1)$. Under joint normality, therefore, the *best linear predictor* coincides with the *conditional expectation*, the regression error $E = X_1 - \mathbb{E}[X_1 \mid \mathbf{X}_2]$ is normal, independent of the regressors, and homoskedastic with variance $\sigma^2_{1|2} = \Sigma_{11} - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21}$. The population R^2 is $1 - \sigma^2_{1|2} / \Sigma_{11}$. Without joint normality, the linear projection still exists but need not equal the conditional mean, and heteroskedasticity is typical (Hansen, 2022, Ch. 2).

5.4.4 Mahalanobis distance

The quadratic form in the exponent defines the *Mahalanobis distance*

$$D(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}, \tag{5.14}$$

which measures how far $\mathbf{x}$ is from the centre in units adapted to the spread and correlation of the distribution. Writing $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top$ and $\mathbf{z} = \boldsymbol{L}^{-1}(\mathbf{X} - \boldsymbol{\mu})$, we have $D^2 = \|\mathbf{z}\|^2$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$, so $D^2 \sim \chi^2_d$. Thus the region $\{D^2 \le \chi^2_{d,0.95}\}$ contains 95% of the probability, which yields a principled multivariate outlier rule.

Historical Note: Mahalanobis and the generalised distance

Prasanta Chandra Mahalanobis, founder of the Indian Statistical Institute (1931) and a principal architect of India's National Sample Survey, introduced his generalised distance in 1936 (Mahalanobis, 1936), motivated by anthropometric comparisons between population groups. He later played a central role in Indian economic planning; the Second Five-Year Plan (1956–61) was based on his growth model.

5.4.5 Simulation via the Cholesky factor

Definition 5.20 is also an algorithm: compute the Cholesky factorisation $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top$ (lower triangular $\boldsymbol{L}$, Chapter 2), draw $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ and set $\mathbf{X} = \boldsymbol{\mu} + \boldsymbol{L}\mathbf{z}$. Then $\operatorname{Cov}(\mathbf{X}) = \boldsymbol{L}\boldsymbol{L}^\top = \boldsymbol{\Sigma}$ by Theorem 5.21. The Cholesky factor is the cheapest square root ($d^3/3$ flops) and the triangular solve $\boldsymbol{L}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ gives stable Mahalanobis distances without forming $\boldsymbol{\Sigma}^{-1}$.

Listing 5.10 simulates a three-dimensional vector (log income, years of schooling, household size) for a stylised population, checks the conditional formula against draws near a conditioning point, and computes Mahalanobis distances.

Listing 5.10: Multivariate normal: simulation, conditioning, Mahalanobis ch05/mvn_cholesky.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Multivariate normal: Cholesky simulation, conditional moments, Mahalanobis distance
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
# (log income, years of schooling, household size) -- simulated
mu = np.array([9.4, 8.0, 5.0])
Sigma = np.array([[0.64, 1.20, -0.30],
                  [1.20, 9.00, -0.90],
                  [-0.30, -0.90, 4.00]])
L = np.linalg.cholesky(Sigma)                 # Sigma = L L^T
n = 400_000
X = mu + rng.standard_normal((n, 3)) @ L.T    # rows are N(mu, Sigma) draws
print("max |sample cov - Sigma| =", np.abs(np.cov(X.T) - Sigma).max().round(4))

# conditional distribution of X1 (log income) given X2 = 12, X3 = 4
a = np.array([12.0, 4.0])
S11, S12, S22 = Sigma[0, 0], Sigma[0, 1:], Sigma[1:, 1:]
w = np.linalg.solve(S22, S12)                 # regression coefficients
cmean = mu[0] + w @ (a - mu[1:])
cvar = S11 - S12 @ w                          # Schur complement
near = np.all(np.abs(X[:, 1:] - a) < 0.15, axis=1)
print(f"E[X1 | X2=12, X3=4]: formula {cmean:.4f}, "
      f"simulation {X[near, 0].mean():.4f} ({near.sum()} draws)")
print(f"Var[X1 | ...]      : formula {cvar:.4f}, simulation "
      f"{X[near, 0].var():.4f}")
print("population regression coefficients Sigma22^-1 Sigma21 =", w.round(4))

# Mahalanobis distance: D^2 ~ chi-square(3)
d2 = np.einsum("ij,ij->i", X - mu, np.linalg.solve(Sigma, (X - mu).T).T)
print(f"P(D^2 > chi2_3(0.95)) = {(d2 > stats.chi2(3).ppf(0.95)).mean():.4f}")
x0 = np.array([11.0, 6.0, 5.0])
d0 = np.sqrt((x0 - mu) @ np.linalg.solve(Sigma, x0 - mu))
print(f"household (11, 6, 5): Euclidean dist {np.linalg.norm(x0 - mu):.3f}, "
      f"Mahalanobis {d0:.3f}")

# figure: (log income, schooling) scatter with Mahalanobis contours
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.scatter(X[:3000, 1], X[:3000, 0], s=3, alpha=0.3, color="#1f4e79")
S2 = Sigma[np.ix_([1, 0], [1, 0])]
vals, vecs = np.linalg.eigh(S2)
th = np.linspace(0, 2 * np.pi, 200)
for c in [1, 2, 3]:
    pts = c * vecs @ np.diag(np.sqrt(vals)) @ np.vstack([np.cos(th), np.sin(th)])
    ax.plot(mu[1] + pts[0], mu[0] + pts[1], color="#cc9900", lw=1.5)
ax.scatter([x0[1]], [x0[0]], color="#b22222", zorder=3, label="household x0")
xs = np.linspace(-2, 18, 50)
ax.plot(xs, mu[0] + S2[0, 1] / S2[0, 0] * (xs - mu[1]), "k--", lw=1,
        label="E[log income | schooling]")
ax.set_xlabel("years of schooling (simulated)")
ax.set_ylabel("log income (simulated)")
ax.legend(frameon=False, fontsize=8, loc="lower right")
```

```
plt.savefig("ch05_mvn_contours.pdf", bbox_inches="tight")
```

Output

```
max |sample cov - Sigma| = 0.033
E[X1 | X2=12, X3=4]: formula 9.9610, simulation 9.9350 (389 draws)
Var[X1 | ...]      : formula 0.4717, simulation 0.4698
population regression coefficients Sigma22^-1 Sigma21 = [ 0.1287 -0.046 ]
P(D^2 > chi2_3(0.95)) = 0.0499
household (11, 6, 5): Euclidean dist 2.561, Mahalanobis 2.787
```

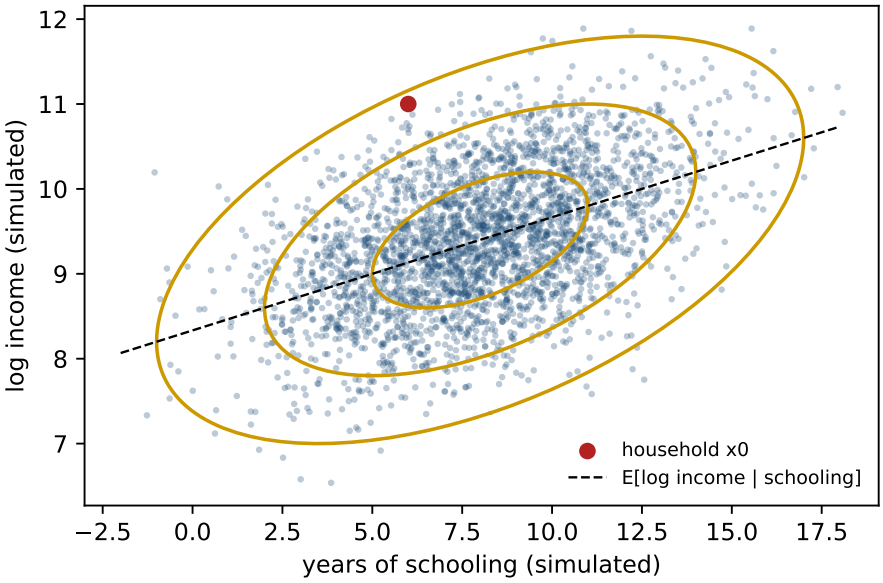


Figure 5.9: Simulated (schooling, log income) pairs with Mahalanobis contours $D = 1, 2, 3$ and the conditional-mean line $\mathbb{E}[\text{log income} \mid \text{schooling}]$, which is the population regression line.

The household $(11, 6, 5)$ is only 2.56 units from the mean in Euclidean terms, but its Mahalanobis distance (2.79) is larger: it combines *high* income with *low* schooling, which runs against the positive correlation (Fig. 5.9).

⋆ Key Concept: Multivariate normal toolkit

- Affine maps of normals are normal: $\boldsymbol{B}\mathbf{X} + \mathbf{b} \sim \mathcal{N}(\boldsymbol{B}\boldsymbol{\mu} + \mathbf{b}, \boldsymbol{B}\boldsymbol{\Sigma}\boldsymbol{B}^{\top})$.
- Marginals are read off sub-blocks; zero covariance $\Leftrightarrow$ independence (jointly normal case only).
- Conditionals: mean $\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2)$ (linear), covariance = Schur complement (constant).
- $D^2 = (\mathbf{X} - \boldsymbol{\mu})^{\top}\boldsymbol{\Sigma}^{-1}(\mathbf{X} - \boldsymbol{\mu}) \sim \chi^2_d$; simulate with $\boldsymbol{\mu} + \boldsymbol{L}\mathbf{z}$.

Graduate Perspective: Gaussian conditioning everywhere

Theorem 5.22 is arguably the most used formula in modern statistics. It is the update step of the Kalman filter for state-space macro models, the predictive equation of Gaussian-process

regression (Murphy, 2022, Ch. 17), the basis of multiple imputation of missing survey variables under normality, and (through the Schur complement) the reason the inverse covariance matrix encodes *partial* correlations: $[\boldsymbol{\Sigma}^{-1}]_{jl} = 0$ iff X_j and X_l are conditionally independent given the rest — the foundation of Gaussian graphical models.

Exercises

5.4.1 Theory Let $Z \sim \mathcal{N}(0,1)$ and $W = SZ$ where $S = \pm 1$ with probability $1/2$ independently of Z. Show that $W \sim \mathcal{N}(0,1)$, $\operatorname{Cov}(Z,W) = 0$, yet Z and W are not independent and (Z,W) is not bivariate normal.

5.4.2 Theory For the bivariate normal with correlation ρ, specialise (5.13) to show $\mathbb{E}[X_1 \mid X_2] = \mu_1 + \rho\frac{\sigma_1}{\sigma_2}(X_2 - \mu_2)$ and $\operatorname{Var}(X_1 \mid X_2) = \sigma_1^2(1-\rho^2)$. Interpret "regression to the mean".

5.4.3 Theory Prove the block-inverse formula: the upper-left block of $\boldsymbol{\Sigma}^{-1}$ is $\boldsymbol{\Sigma}_{1|2}^{-1}$. Deduce that conditional precision equals a sub-block of the joint precision matrix.

5.4.4 Python Extend Listing 5.10 to compute the conditional distribution of (log income, household size) given schooling = 12 and verify it by simulation. Compare the conditional correlation with the unconditional one.

5.4.5 Python Generate 500 draws from $\mathcal{N}_5(\mathbf{0}, \boldsymbol{\Sigma})$ with $\Sigma_{jl} = 0.7^{|j-l|}$, contaminate 10 rows by adding 3 to the first coordinate and subtracting 3 from the second, and show that a Mahalanobis rule with threshold $\chi^2_{5,0.99}$ detects them while univariate $|z| > 3$ rules do not.

5.4.6 ★ Graduate Show that the OLS coefficients from regressing X_1 on $(1, \mathbf{X}_2)$ in a large sample from a joint normal converge to $\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$ and that the residual variance converges to the Schur complement. Then show by simulation that for a non-normal joint distribution (e.g. log income log-normal and schooling uniform with a nonlinear dependence) the conditional mean is not linear while the linear projection is still well defined.

5.4.7 Policy A targeting agency computes a Mahalanobis distance of each household's vector of asset indicators from the mean of known poor households and classifies as "poor" those within a threshold. Discuss the implicit distributional assumption and how binary asset indicators violate it.

5.5 The Exponential Family and Generalised Linear Models

5.5.1 Canonical form

Definition 5.23 (Exponential family)**.** A family of densities or PMFs $\{f(y \mid \boldsymbol{\theta})\}$ is an exponential family if it can be written

$$f(y \mid \boldsymbol{\eta}) = h(y)\exp\!\big\{\boldsymbol{\eta}^{\top}\mathbf{T}(y) - A(\boldsymbol{\eta})\big\}, \tag{5.15}$$

where $\mathbf{T}(y) \in \mathbb{R}^s$ is the *sufficient statistic*, $\boldsymbol{\eta} = \boldsymbol{\eta}(\boldsymbol{\theta})$ the *natural (canonical) parameter*, $h(y) \geq 0$ the *base measure*, and

$$A(\boldsymbol{\eta}) = \log \int h(y)\exp\{\boldsymbol{\eta}^{\top}\mathbf{T}(y)\}\,\mathrm{d}y$$

the *log-partition* (cumulant) function, which normalises the density. The set $\{\boldsymbol{\eta} : A(\boldsymbol{\eta}) < \infty\}$ is the natural parameter space. Crucially, the support of f does not depend on the parameter.

Example 5.24 (Members of the family).

- *Bernoulli:* $p^y(1-p)^{1-y} = \exp\{y \log \frac{p}{1-p} + \log(1-p)\}$, so $T(y) = y$, $\eta = \log \frac{p}{1-p}$ (the *logit*), $A(\eta) = \log(1+e^\eta)$, $h = 1$.
- *Poisson:* $e^{-\lambda}\lambda^y/y! = \frac{1}{y!}\exp\{y \log \lambda - \lambda\}$, so $\eta = \log \lambda$, $A(\eta) = e^\eta$, $h(y) = 1/y!$.
- *Normal, known* σ^2: $\eta = \mu/\sigma^2$, $T(y) = y$, $A(\eta) = \sigma^2\eta^2/2$, $h(y) = e^{-y^2/(2\sigma^2)}/\sqrt{2\pi\sigma^2}$.
- *Normal, both unknown:* $\boldsymbol{T}(y) = (y, y^2)$, $\boldsymbol{\eta} = (\mu/\sigma^2, -1/(2\sigma^2))$, $A = -\eta_1^2/(4\eta_2) - \frac{1}{2}\log(-2\eta_2)$.
- *Exponential:* $\lambda e^{-\lambda y}$ has $\eta = -\lambda$, $A = -\log(-\eta)$.
- *Gamma, beta, binomial (fixed n), negative binomial (fixed r), multinomial, multivariate normal* are also members. The Cauchy, the t, the uniform$(0, \theta)$ and the Pareto with unknown x_m are not (the last two because the support depends on the parameter).

5.5.2 Moments from the log-partition function

Theorem 5.25 (Cumulants of the sufficient statistic). *In the interior of the natural parameter space, A is convex and infinitely differentiable, and*

$$\nabla A(\boldsymbol{\eta}) = \mathbb{E}_{\boldsymbol{\eta}}[\boldsymbol{T}(Y)], \qquad \nabla^2 A(\boldsymbol{\eta}) = \operatorname{Cov}_{\boldsymbol{\eta}}[\boldsymbol{T}(Y)]. \tag{5.16}$$

More generally, the cumulant generating function of $\boldsymbol{T}(Y)$ is $K(\boldsymbol{t}) = A(\boldsymbol{\eta}+\boldsymbol{t}) - A(\boldsymbol{\eta})$.

Proof. Differentiate the identity $e^{A(\boldsymbol{\eta})} = \int h(y) e^{\boldsymbol{\eta}^\top \boldsymbol{T}(y)}\,\mathrm{d}y$ under the integral sign (justified by dominated convergence in the interior): $e^A \nabla A = \int \boldsymbol{T}(y) h(y) e^{\boldsymbol{\eta}^\top \boldsymbol{T}(y)}\,\mathrm{d}y$, i.e. $\nabla A = \mathbb{E}[\boldsymbol{T}]$. Differentiating again, $\nabla^2 A = \mathbb{E}[\boldsymbol{T}\boldsymbol{T}^\top] - \mathbb{E}[\boldsymbol{T}]\mathbb{E}[\boldsymbol{T}]^\top = \operatorname{Cov}(\boldsymbol{T})$, which is positive semidefinite, so A is convex. For the CGF, $\mathbb{E}e^{\boldsymbol{t}^\top \boldsymbol{T}} = \int h\, e^{(\boldsymbol{\eta}+\boldsymbol{t})^\top \boldsymbol{T} - A(\boldsymbol{\eta})} = e^{A(\boldsymbol{\eta}+\boldsymbol{t}) - A(\boldsymbol{\eta})}$. □

For the Poisson, $A(\eta) = e^\eta$ gives $\mathbb{E}Y = \operatorname{Var} Y = e^\eta = \lambda$ in one line; for the Bernoulli, $A'(\eta) = e^\eta/(1+e^\eta) = p$ and $A''(\eta) = p(1-p)$. In a one-parameter family the mean $\mu = A'(\eta)$ is strictly increasing in η (since $A'' = \operatorname{Var} T > 0$), so the map $\eta \mapsto \mu$ is invertible and the variance can be written as a function of the mean, the *variance function* $V(\mu) = A''(\eta(\mu))$: $V(\mu) = \mu$ for the Poisson, $\mu(1-\mu)$ for the Bernoulli, σ^2 (constant) for the normal.

Listing 5.11: Moments from derivatives of the log-partition function ch05/expfam_moments.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Exponential family: moments from derivatives of the log-partition function
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

# natural parameter eta, log-partition A(eta), and the mean/variance it implies
families = {
    "Bernoulli(p=0.3)": (np.log(0.3 / 0.7), lambda e: np.log1p(np.exp(e)),
                         stats.bernoulli(0.3)),
    "Poisson(lam=4)": (np.log(4.0), lambda e: np.exp(e), stats.poisson(4.0)),
    "Exponential(rate=2)": (-2.0, lambda e: -np.log(-e), stats.expon(scale=0.5)),
    "Geometric(p=0.3)*": (np.log(0.7), lambda e: -np.log1p(-np.exp(e)),
                          stats.geom(0.3, loc=-1)),
}
h = 1e-4
print(f"{'family':22s}{'eta':>8s}{'A1(eta)':>9s}{'mean':>8s}"
      f"{'A2(eta)':>9s}{'var':>8s}")
for name, (eta, A, d) in families.items():
```

```
    d1 = (A(eta + h) - A(eta - h)) / (2 * h)          # A'(eta)
    d2 = (A(eta + h) - 2 * A(eta) + A(eta - h)) / h**2  # A''(eta)
    print(f"{name:22s}{eta:8.3f}{d1:9.4f}{d.mean():8.4f}{d2:9.4f}{d.var():8.4f}")
print("* geometric counted as failures before first success (support 0,1,...)")

# sufficiency in action: the Poisson MLE depends on the data only via sum(y)
rng = np.random.default_rng(803)
y1 = rng.poisson(4.0, 50)
y2 = np.zeros(50, dtype=int)
y2[0] = y1.sum()                  # a very different sample, same total
print(f"two samples with T = sum(y) = {y1.sum()} and {y2.sum()}:")
for lam in np.linspace(3, 5, 5):
    diff = stats.poisson.logpmf(y1, lam).sum() - stats.poisson.logpmf(y2, lam).sum()
    print(f"  lam = {lam:.2f}  loglik difference = {diff:.6f}")
```

Output

```
family                    eta  A1(eta)    mean  A2(eta)     var
Bernoulli(p=0.3)       -0.847   0.3000  0.3000   0.2100  0.2100
Poisson(lam=4)          1.386   4.0000  4.0000   4.0000  4.0000
Exponential(rate=2)    -2.000   0.5000  0.5000   0.2500  0.2500
Geometric(p=0.3)*      -0.357   2.3333  2.3333   7.7778  7.7778
* geometric counted as failures before first success (support 0,1,...)
two samples with T = sum(y) = 199 and 199:
  lam = 3.00  loglik difference = 672.742299
  lam = 3.50  loglik difference = 672.742299
  lam = 4.00  loglik difference = 672.742299
  lam = 4.50  loglik difference = 672.742299
  lam = 5.00  loglik difference = 672.742299
```

The second part of the output previews Section 5.6: two Poisson samples with the same total $\sum y_i$ have log-likelihoods differing by a constant that does not depend on λ, so they carry identical information about λ. The total is *sufficient*.

Historical Note: Koopman, Darmois, Pitman — and Nelder and Wedderburn

The exponential family emerged from the search for distributions admitting a fixed-dimensional sufficient statistic, a concept introduced by Fisher in 1922. Independently, Georges Darmois (1935), Bernard Koopman (1936) and Edwin Pitman (1936) showed that, under regularity conditions (in particular, support not depending on the parameter), a family admits a sufficient statistic of fixed dimension for every sample size only if it is of the form (5.15) (Darmois, 1935; Koopman, 1936; Pitman, 1936); the result is known as the Pitman–Koopman–Darmois theorem. In 1972 John Nelder and Robert Wedderburn, at Rothamsted Experimental Station, unified regression for normal, binomial, Poisson and gamma responses as *generalised linear models* fitted by a single algorithm (Nelder and Wedderburn, 1972); their work led to the GLIM software and the influential book by McCullagh and Nelder (1989).

5.5.3 Generalised linear models

Definition 5.26 (GLM). A generalised linear model has three components:

(i) a *random component*: $Y_i \mid \mathbf{x}_i$ independent from a one-parameter exponential family (possibly with a dispersion parameter ϕ) with mean μ_i;

(ii) a *linear predictor* $\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$;

(iii) a *link function* g, monotone and differentiable, with $g(\mu_i) = \eta_i$.

The *canonical link* is the one that makes the linear predictor equal to the natural parameter, $g = (A')^{-1}$.

Response	Distribution	Canonical link $g(\mu)$	Inverse link	$V(\mu)$
Continuous	Normal	identity μ	η	1
Binary	Bernoulli	logit $\log \frac{\mu}{1-\mu}$	$1/(1+e^{-\eta})$	$\mu(1-\mu)$
Count	Poisson	log $\log \mu$	e^{η}	μ
Positive, skewed	Gamma	inverse $-1/\mu$	$-1/\eta$	μ^2

Non-canonical links are common: the *probit* $g = \Phi^{-1}$ for binary data, or the log link with gamma responses (health expenditures). The link maps the restricted range of μ (e.g. $(0,1)$ or $(0,\infty)$) onto $\mathbb{R}$, which is why OLS on a binary or count outcome — the identity link — can produce impossible predictions.

5.5.4 Logistic and Poisson regression; the score and IRLS

With the canonical link the log-likelihood of a GLM (with $\phi = 1$) is

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \big\{ y_i \, \mathbf{x}_i^\top \boldsymbol{\beta} - A(\mathbf{x}_i^\top \boldsymbol{\beta}) \big\} + \text{const}, \tag{5.17}$$

with score and Hessian

$$\nabla \ell(\boldsymbol{\beta}) = \sum_i (y_i - \mu_i)\mathbf{x}_i = \mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu}), \qquad \nabla^2 \ell(\boldsymbol{\beta}) = -\sum_i A''(\eta_i)\mathbf{x}_i\mathbf{x}_i^\top = -\mathbf{X}^\top \boldsymbol{W} \mathbf{X}, \tag{5.18}$$

where $\boldsymbol{W} = \operatorname{diag}(V(\mu_i))$. Since $\boldsymbol{W} \succ 0$, ℓ is concave: any stationary point is the global maximum, and the MLE solves the *moment conditions* $\mathbf{X}^\top(\mathbf{y} - \hat{\boldsymbol{\mu}}) = \mathbf{0}$ — residuals orthogonal to regressors, exactly as in OLS. In logistic regression $\mu_i = 1/(1+e^{-\mathbf{x}_i^\top \boldsymbol{\beta}})$, $V = \mu(1-\mu)$; in Poisson regression $\mu_i = e^{\mathbf{x}_i^\top \boldsymbol{\beta}}$, $V = \mu$. Coefficients have multiplicative interpretations: e^{β_j} is an odds ratio in the logit model and a rate ratio in the Poisson model.

Newton's method (Chapter 8) applied to (5.18) gives $\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + (\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu})$, which can be rearranged as a weighted least-squares problem,

$$\boldsymbol{\beta}^{(t+1)} = (\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{W} \mathbf{z}, \qquad z_i = \eta_i + \frac{y_i - \mu_i}{V(\mu_i)}, \tag{5.19}$$

with weights and *working response* $\mathbf{z}$ evaluated at $\boldsymbol{\beta}^{(t)}$: *iteratively reweighted least squares* (IRLS). At convergence, $(\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1}$ estimates the covariance matrix of $\hat{\boldsymbol{\beta}}$ (the inverse Fisher information; Chapter 7).

Listing 5.12: Logistic and Poisson regression by IRLS ch05/glm_irls.py

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Logistic and Poisson regression as GLMs fitted by IRLS, checked with scikit-learn
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from sklearn.linear_model import LogisticRegression, PoissonRegressor

rng = np.random.default_rng(803)
n = 4000
```

```python
# simulated households: years of schooling of head, distance to clinic (km)
school = rng.uniform(0, 16, n)
dist = rng.exponential(5.0, n)
X = np.column_stack([np.ones(n), school, dist])

# binary outcome: child enrolled in secondary school (logit link)
beta_logit = np.array([-1.0, 0.25, -0.10])
y_bin = rng.binomial(1, 1 / (1 + np.exp(-X @ beta_logit)))
# count outcome: clinic visits per year (log link)
beta_pois = np.array([1.2, 0.03, -0.08])
y_cnt = rng.poisson(np.exp(X @ beta_pois))


def irls(X, y, family, iters=25):
    """Iteratively reweighted least squares for canonical-link GLMs."""
    b = np.zeros(X.shape[1])
    for it in range(iters):
        eta = X @ b
        mu = 1 / (1 + np.exp(-eta)) if family == "logit" else np.exp(eta)
        w = mu * (1 - mu) if family == "logit" else mu    # Var(y) = A''(eta)
        z = eta + (y - mu) / w                             # working response
        b_new = np.linalg.solve(X.T @ (w[:, None] * X), X.T @ (w * z))
        if np.max(np.abs(b_new - b)) < 1e-10:
            return b_new, it + 1, w
        b = b_new
    return b, iters, w


for fam, y, beta in [("logit", y_bin, beta_logit), ("log", y_cnt, beta_pois)]:
    b, its, w = irls(X, y, fam)
    se = np.sqrt(np.diag(np.linalg.inv(X.T @ (w[:, None] * X))))
    if fam == "logit":
        sk = LogisticRegression(penalty="none", tol=1e-10, max_iter=1000)
        sk.fit(X[:, 1:], y)
    else:
        sk = PoissonRegressor(alpha=0.0, tol=1e-10, max_iter=1000)
        sk.fit(X[:, 1:], y)
    sk_b = np.r_[np.ravel(sk.intercept_), np.ravel(sk.coef_)]
    print(f"{fam} link: IRLS converged in {its} iterations")
    print("   true    " + " ".join(f"{v:9.4f}" for v in beta))
    print("   IRLS    " + " ".join(f"{v:9.4f}" for v in b))
    print("   s.e.    " + " ".join(f"{v:9.4f}" for v in se))
    print("   sklearn" + " ".join(f"{v:9.4f}" for v in sk_b))
b_logit = irls(X, y_bin, "logit")[0]
print(f"odds ratio for +4 years of schooling = {np.exp(4 * b_logit[1]):.3f}")
b_pois = irls(X, y_cnt, "log")[0]
print(f"rate ratio for +10 km distance = {np.exp(10 * b_pois[2]):.3f}")
```

Output

```
logit link: IRLS converged in 6 iterations
   true     -1.0000    0.2500   -0.1000
   IRLS     -0.9294    0.2578   -0.1254
   s.e.      0.0801    0.0094    0.0083
   sklearn  -0.9294    0.2578   -0.1254
log link: IRLS converged in 8 iterations
```

```
    true      1.2000    0.0300   -0.0800
    IRLS      1.2118    0.0294   -0.0833
    s.e.      0.0216    0.0020    0.0026
    sklearn   1.2118    0.0294   -0.0833
odds ratio for +4 years of schooling = 2.805
rate ratio for +10 km distance = 0.435
```

IRLS converges in 6–8 iterations (Newton's quadratic convergence) and reproduces scikit-learn's unpenalised estimates to four decimals. In this simulated survey, four more years of the household head's schooling multiply the odds of secondary enrolment by about 2.8, and each additional 10 km to the nearest clinic multiplies the expected number of visits by 0.44.

Policy Lens: Logistic and Poisson regression in programme evaluation

Binary outcomes (enrolment, vaccination, employment, take-up of a programme) and counts (visits, arrests, births, new businesses) are the bread and butter of policy evaluation. GLMs respect the range of the outcome and give interpretable multiplicative effects. Two cautions: odds ratios are not risk ratios unless the outcome is rare, so effects on probabilities should be reported as *average marginal effects* $n^{-1}\sum_i \partial\mu_i/\partial x_{ij} = n^{-1}\sum_i \beta_j V(\mu_i)$ (for the canonical link); and the Poisson MLE remains consistent for $\boldsymbol{\beta}$ under overdispersion as long as the conditional mean is correctly specified, provided standard errors are computed with the sandwich formula (Wooldridge, 2010, Ch. 18).

⋆ Key Concept: Exponential family and GLMs

- Canonical form $h(y)\exp\{\boldsymbol{\eta}^\top \boldsymbol{T}(y) - A(\boldsymbol{\eta})\}$: $\boldsymbol{T}$ is sufficient, $\nabla A = \mathbb{E}\boldsymbol{T}$, $\nabla^2 A = \operatorname{Cov}\boldsymbol{T}$, A convex.
- GLM = exponential-family response + linear predictor + link; canonical link sets $\eta = \mathbf{x}^\top\boldsymbol{\beta}$ equal to the natural parameter.
- Canonical-link score $\mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu})$, Hessian $-\mathbf{X}^\top \boldsymbol{W}\mathbf{X}$: concave log-likelihood, fitted by IRLS (Newton).
- OLS, logistic and Poisson regression are three instances of one model.

△ Caution: Separation and non-existence of the MLE

If a regressor (or combination) perfectly separates $y = 1$ from $y = 0$ — for example, every household within 1 km of a school enrols — the logistic log-likelihood increases without bound as $|\beta_j| \to \infty$ and the MLE does not exist. IRLS then fails to converge or reports huge coefficients and standard errors. Similar problems arise in Poisson regression when a category has only zero counts. Remedies include penalisation (ridge or Firth-type), dropping or merging categories, or reporting the separation itself as a finding.

Exercises

5.5.1 Theory Write the binomial(n, p) with fixed n, the negative binomial with fixed r, and the gamma with both parameters unknown in canonical form. Identify $\boldsymbol{T}$, $\boldsymbol{\eta}$, A and h, and use Theorem 5.25 to recover their means and variances.

5.5.2 Theory Show that the beta family is a two-parameter exponential family with sufficient statistic $(\log y, \log(1-y))$, and that the uniform$(0, \theta)$ family is not an exponential family.

5.5.3 Theory Derive the IRLS update (5.19) from Newton's method. Show that for the normal distribution with identity link IRLS converges in one step to OLS.

5.5.4 Python Add a probit fit to Listing 5.12 by maximising the Bernoulli log-likelihood with $\mu = \Phi(\eta)$ via `scipy.optimize.minimize`. Compare $\hat\beta_{\text{logit}}/\hat\beta_{\text{probit}}$ with the folklore factor ≈ 1.6, and compare average marginal effects.

5.5.5 Python Simulate overdispersed counts (negative binomial, $\alpha = 1$) with the same mean model as in Listing 5.12. Fit Poisson regression by IRLS, and compare the model-based standard errors $(\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1/2}$ with sandwich standard errors $(\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1} \mathbf{X}^\top \operatorname{diag}(\hat e_i^2) \mathbf{X} (\mathbf{X}^\top \boldsymbol{W} \mathbf{X})^{-1}$ and with the Monte Carlo standard deviation of $\hat\beta$ over 500 replications.

5.5.6 ★ Graduate Show that for a canonical-link GLM with an intercept, $\sum_i \hat\mu_i = \sum_i y_i$. Interpret this "calibration-in-the-large" property for a risk score used to allocate a budget across districts.

5.5.7 ★ Graduate Construct a data set with quasi-complete separation and show what IRLS does. Then add an ℓ_2 penalty $\frac{\lambda}{2}\|\beta\|^2$ and derive the modified IRLS update.

5.5.8 Policy A ministry models the number of new firm registrations per municipality with Poisson regression on log population, a broadband indicator and distance to the regional capital. Explain why log population is often included as an *offset* (coefficient fixed at one), what the broadband coefficient means as a rate ratio, and how you would check for overdispersion.

5.6 Likelihood

5.6.1 Definition: likelihood versus probability

Definition 5.27 (Likelihood)**.** Let $\mathbf{y}$ be observed data with density or PMF $f(\mathbf{y} \mid \boldsymbol\theta)$, $\boldsymbol\theta \in \Theta$. The *likelihood function* is

$$\mathcal{L}(\boldsymbol\theta; \mathbf{y}) = f(\mathbf{y} \mid \boldsymbol\theta), \qquad \boldsymbol\theta \in \Theta, \tag{5.20}$$

regarded as a function of $\boldsymbol\theta$ with $\mathbf{y}$ *fixed*. The log-likelihood is $\ell(\boldsymbol\theta) = \log \mathcal{L}(\boldsymbol\theta; \mathbf{y})$. For iid data $y_1, \dots, y_n$,

$$\mathcal{L}(\boldsymbol\theta) = \prod_{i=1}^{n} f(y_i \mid \boldsymbol\theta), \qquad \ell(\boldsymbol\theta) = \sum_{i=1}^{n} \log f(y_i \mid \boldsymbol\theta). \tag{5.21}$$

The same formula $f(\mathbf{y} \mid \boldsymbol\theta)$ is read in two ways. As a function of $\mathbf{y}$ for fixed $\boldsymbol\theta$, it is a probability distribution: it sums or integrates to one over $\mathbf{y}$. As a function of $\boldsymbol\theta$ for fixed $\mathbf{y}$, it is a likelihood: it measures how well each parameter value explains the data actually observed, and it does *not* integrate to one over $\boldsymbol\theta$ (and need not be integrable at all). For example, with $y = 7$ successes in $n = 10$ trials, $\mathcal{L}(p) = \binom{10}{7} p^7 (1-p)^3$ has $\int_0^1 \mathcal{L}(p)\,\mathrm{d}p = 1/11$. Only *ratios* $\mathcal{L}(\boldsymbol\theta_1)/\mathcal{L}(\boldsymbol\theta_2)$ are meaningful; multiplicative constants not involving $\boldsymbol\theta$ (such as $\binom{10}{7}$ or $h(\mathbf{y})$ in an exponential family) can be dropped.

Logs are taken for three reasons: products become sums, so the law of large numbers applies to $n^{-1}\ell$; the log-likelihood of an exponential family is concave in the natural parameter; and numerically, a product of thousands of densities underflows to zero in floating point while the sum of their logs is perfectly representable (`stats.norm.logpdf`, not `np.log(stats.norm.pdf(...))`).

Historical Note: Fisher and the word "likelihood"

R. A. Fisher proposed maximising what we now call the likelihood in a 1912 undergraduate paper, but at first did not distinguish it clearly from inverse (Bayesian) probability. In his 1921 paper on the correlation coefficient in *Metron* (Fisher, 1921) he introduced the term "likelihood" precisely to emphasise that it is *not* a probability over parameters, and in the 1922 *Philosophical Transactions* paper "On the mathematical foundations of theoretical statistics" (Fisher, 1922) he set out the concepts of consistency, efficiency, sufficiency and maximum likelihood estimation that frame much of modern statistics.

5.6.2 The likelihood principle

Definition 5.28 (Likelihood principle)**.** If two experiments, with the same parameter $\boldsymbol{\theta}$, yield data $\mathbf{y}_1$ and $\mathbf{y}_2$ such that $\mathcal{L}_1(\boldsymbol{\theta}; \mathbf{y}_1) = c\, \mathcal{L}_2(\boldsymbol{\theta}; \mathbf{y}_2)$ for all $\boldsymbol{\theta}$ and some $c > 0$ not depending on $\boldsymbol{\theta}$, then $\mathbf{y}_1$ and $\mathbf{y}_2$ carry the same evidence about $\boldsymbol{\theta}$.

The classic illustration: one survey team interviews $n = 12$ households and finds $y = 3$ that use a service (binomial sampling), while another keeps interviewing until it finds the third user, which happens at household 12 (negative binomial sampling). The likelihoods are $\binom{12}{3} p^3 (1-p)^9$ and $\binom{11}{2} p^3 (1-p)^9$ — proportional — so likelihood-based and Bayesian inferences agree. Yet the frequentist p-values for $H_0 : p = 1/2$ differ, because they depend on outcomes that *could* have occurred under each stopping rule. Birnbaum (1962) showed that the likelihood principle follows from the more widely accepted sufficiency and conditionality principles, a result that has been debated ever since. Bayesian inference obeys the likelihood principle; p-values and confidence intervals generally do not.

5.6.3 Normal regression: maximum likelihood is least squares

Proposition 5.29. *In the model* $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i$, $\varepsilon_i \overset{iid}{\sim} \mathcal{N}(0, \sigma^2)$, *the log-likelihood is*

$$\ell(\boldsymbol{\beta}, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \lVert \mathbf{y} - \mathbf{X}\boldsymbol{\beta} \rVert^2 . \tag{5.22}$$

It is maximised by the OLS estimator $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ *and by* $\hat{\sigma}^2_{\mathrm{ML}} = \mathrm{RSS}/n$, *and the maximised value is* $\ell_{\max} = -\frac{n}{2}\{\log(2\pi\, \mathrm{RSS}/n) + 1\}$.

Proof. For any fixed σ^2, $\boldsymbol{\beta}$ enters only through $-\lVert \mathbf{y} - \mathbf{X}\boldsymbol{\beta} \rVert^2 / (2\sigma^2)$, so maximising over $\boldsymbol{\beta}$ is minimising the sum of squares, whose solution is OLS for every σ^2. Substituting, the *profile* log-likelihood is $-\frac{n}{2}\log(2\pi\sigma^2) - \mathrm{RSS}/(2\sigma^2)$; setting its derivative $-\frac{n}{2\sigma^2} + \frac{\mathrm{RSS}}{2\sigma^4}$ to zero gives $\sigma^2 = \mathrm{RSS}/n$, and substitution gives $\ell_{\max}$. □

Listing 5.13 checks this numerically on the `diabetes` data bundled with scikit-learn, maximising (5.22) with a general-purpose optimiser (parametrising $\log \sigma$ to keep $\sigma > 0$).

Listing 5.13: Gaussian likelihood and least squares `ch05/normal_reg_ls.py`

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Gaussian regression likelihood: numerical MLE equals least squares
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize
from sklearn.datasets import load_diabetes

X0, y = load_diabetes(return_X_y=True)        # bundled scikit-learn data
```

```python
X = np.column_stack([np.ones(len(y)), X0[:, [2, 3, 8]]])   # bmi, bp, s5
n, k = X.shape


def negloglik(theta):
    """Negative Gaussian log-likelihood; theta = (beta, log sigma)."""
    beta, log_s = theta[:-1], theta[-1]
    r = y - X @ beta
    return (n * log_s + 0.5 * n * np.log(2 * np.pi)
            + 0.5 * (r @ r) * np.exp(-2 * log_s))


start = np.r_[np.zeros(k), np.log(y.std())]
res = optimize.minimize(negloglik, start, method="Powell",
                        options={"xtol": 1e-10, "ftol": 1e-14, "maxiter": 50_000})
res = optimize.minimize(negloglik, res.x, method="BFGS")   # polish
beta_ml, s_ml = res.x[:-1], np.exp(res.x[-1])
beta_ls, *_ = np.linalg.lstsq(X, y, rcond=None)
rss = np.sum((y - X @ beta_ls) ** 2)

print(f"{'coef':8s}{'MLE (numeric)':>14s}{'OLS (lstsq)':>14s}")
for name, a, b in zip(["const", "bmi", "bp", "s5"], beta_ml, beta_ls):
    print(f"{name:8s}{a:14.4f}{b:14.4f}")
print(f"sigma_ML = {s_ml:.4f},  sqrt(RSS/n) = {np.sqrt(rss / n):.4f},  "
      f"sqrt(RSS/(n-k)) = {np.sqrt(rss / (n - k)):.4f}")
print(f"max loglik = {-res.fun:.3f}; formula -n/2 (log(2 pi RSS/n) + 1) = "
      f"{-n / 2 * (np.log(2 * np.pi * rss / n) + 1):.3f}")
```

Output

```
coef     MLE (numeric)   OLS (lstsq)
const          152.1335      152.1335
bmi            603.0784      603.0784
bp             262.2720      262.2720
s5             543.8712      543.8712
sigma_ML = 55.5252,  sqrt(RSS/n) = 55.5252,  sqrt(RSS/(n-k)) = 55.7782
max loglik = -2402.613; formula -n/2 (log(2 pi RSS/n) + 1) = -2402.613
```

Note that $\hat\sigma_{\mathrm{ML}} = \sqrt{\mathrm{RSS}/n}$ is smaller than the usual $s = \sqrt{\mathrm{RSS}/(n-k)}$: the MLE of σ^2 is biased downward by the factor $(n-k)/n$ (Theorem 5.17 and Section 5.3). The expression for $\ell_{\max}$ is what enters AIC and BIC for linear regression.

5.6.4 The logistic likelihood and likelihood surfaces

For binary y_i with $\mathbb{P}(y_i = 1 \mid \mathbf{x}_i) = \mu_i = \sigma(\mathbf{x}_i^\top \boldsymbol\beta)$, $\sigma(t) = 1/(1+e^{-t})$,

$$\ell(\boldsymbol\beta) = \sum_{i} \big\{ y_i \log \mu_i + (1-y_i)\log(1-\mu_i) \big\} = \sum_{i} \big\{ y_i \eta_i - \log(1+e^{\eta_i}) \big\}, \quad \eta_i = \mathbf{x}_i^\top \boldsymbol\beta, \tag{5.23}$$

which is (5.17) with $A(\eta) = \log(1+e^{\eta})$. Its negative is the *cross-entropy* or *log loss* minimised by every classifier in machine learning that outputs probabilities (Murphy, 2022, Ch. 10); numerically $\log(1+e^{\eta})$ should be computed as `np.logaddexp(0, eta)` to avoid overflow.

Plotting ℓ over a grid makes the geometry visible. Contours of the log-likelihood near the maximum are approximately ellipses whose shape is given by the Hessian (the observed

information), and the set $\{\boldsymbol{\theta} : 2[\ell(\hat{\boldsymbol{\theta}}) - \ell(\boldsymbol{\theta})] \le \chi^2_{d,0.95}\}$ is an approximate 95% *likelihood-ratio confidence region* (Wilks' theorem, Chapter 7).

Listing 5.14: Likelihood contours for normal and logistic models ch05/likelihood_surface.py

```python
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Log-likelihood surfaces: normal (mu, sigma) and logistic regression (b0, b1)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats, optimize

rng = np.random.default_rng(803)
y = rng.normal(2.0, 1.5, 40)                        # simulated sample

# normal log-likelihood on a grid
M, S = np.meshgrid(np.linspace(1, 3.2, 200), np.linspace(0.9, 2.6, 200))
ll_norm = stats.norm.logpdf(y[:, None, None], M, S).sum(axis=0)
i, j = np.unravel_index(ll_norm.argmax(), ll_norm.shape)
print(f"normal: grid max at mu = {M[i, j]:.3f}, sigma = {S[i, j]:.3f}")
print(f"        closed form mu = {y.mean():.3f}, sigma = {y.std(ddof=0):.3f}")

# logistic regression log-likelihood
n = 150
x = rng.normal(0, 1, n)
yb = rng.binomial(1, 1 / (1 + np.exp(-(-0.5 + 1.2 * x))))


def negll(b):
    eta = b[0] + b[1] * x
    return -(yb * eta - np.logaddexp(0, eta)).sum()


res = optimize.minimize(negll, np.zeros(2), method="BFGS")
print(f"logistic: MLE b0 = {res.x[0]:.3f}, b1 = {res.x[1]:.3f}, "
      f"max loglik = {-res.fun:.2f}")
B0, B1 = np.meshgrid(np.linspace(-1.5, 0.5, 150), np.linspace(0.2, 2.4, 150))
eta = B0[None] + B1[None] * x[:, None, None]
ll_log = (yb[:, None, None] * eta - np.logaddexp(0, eta)).sum(axis=0)
# likelihood-ratio 95% region: 2 (l_max - l) <= chi2_2(0.95)
crit = stats.chi2(2).ppf(0.95) / 2
print(f"95% LR region: loglik >= max - {crit:.3f}")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
lev = ll_norm.max() - np.array([8, 4, 2, 1, 0.5])
axes[0].contour(M, S, ll_norm, levels=np.sort(lev), cmap="viridis")
axes[0].plot(y.mean(), y.std(), "r*", ms=10)
axes[0].set_xlabel("mu")
axes[0].set_ylabel("sigma")
axes[0].set_title("normal log-likelihood", fontsize=10)
lev2 = -res.fun - np.array([10, 5, crit, 1])
axes[1].contour(B0, B1, ll_log, levels=np.sort(lev2), cmap="viridis")
axes[1].contour(B0, B1, ll_log, levels=[-res.fun - crit], colors="#b22222")
axes[1].plot(*res.x, "r*", ms=10)
axes[1].set_xlabel("b0")
axes[1].set_ylabel("b1")
axes[1].set_title("logistic log-likelihood (red: 95% LR region)", fontsize=10)
```

```
fig.tight_layout()
plt.savefig("ch05_likelihood_contours.pdf", bbox_inches="tight")
```

Output

```
normal: grid max at mu = 1.663, sigma = 1.037
        closed form mu = 1.658, sigma = 1.040
logistic: MLE b0 = -0.814, b1 = 1.044, max loglik = -80.93
95% LR region: loglik >= max - 2.996
```

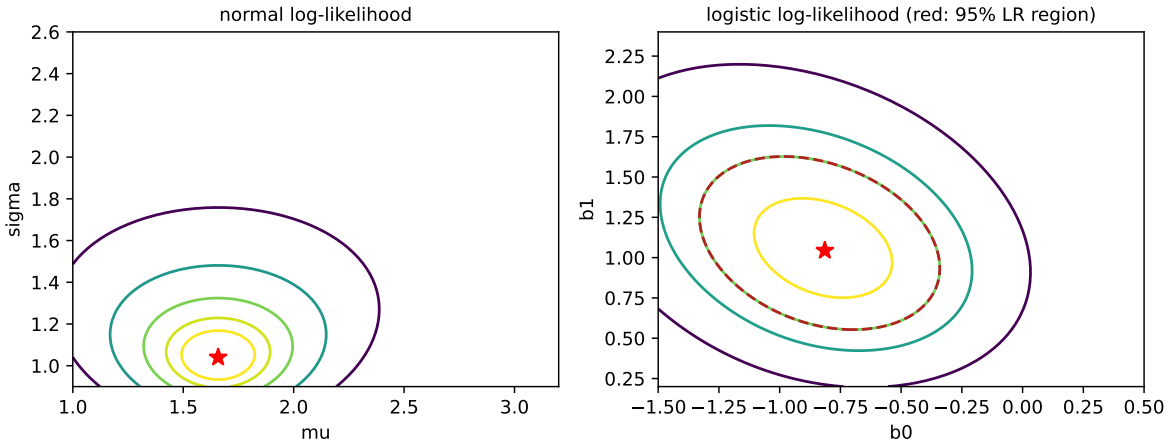


Figure 5.10: Log-likelihood contours. Left: normal sample, parameters (μ, σ); the contours are skewed in σ. Right: logistic regression; the red contour is the 95% likelihood-ratio region $\ell \geq \ell_{\max} - \chi^2_{2,0.95}/2$. Stars mark the MLEs.

△ Caution: A likelihood is not a posterior

It is tempting to read $\mathcal{L}(\boldsymbol{\theta})$ as "the probability that $\boldsymbol{\theta}$ is the true parameter". It is not: it is a probability of the *data*, it does not integrate to one over $\boldsymbol{\theta}$, and it changes under reparametrisation differently from a density (the MLE is invariant, $\widehat{g(\theta)} = g(\hat\theta)$, but a density of θ would acquire a Jacobian). Turning a likelihood into probabilities about $\boldsymbol{\theta}$ requires a prior — see the Bayesian preview below.

5.6.5 Sufficiency and the factorisation theorem

Definition 5.30 (Sufficient statistic)**.** A statistic $\boldsymbol{T}(\mathbf{y})$ is *sufficient* for $\boldsymbol{\theta}$ if the conditional distribution of $\mathbf{y}$ given $\boldsymbol{T}(\mathbf{y})$ does not depend on $\boldsymbol{\theta}$.

Intuitively, once $\boldsymbol{T}$ is known, the remaining detail of the data is pure noise as far as $\boldsymbol{\theta}$ is concerned: one could regenerate a data set with the same distribution from $\boldsymbol{T}$ alone without knowing $\boldsymbol{\theta}$.

Theorem 5.31 (Fisher–Neyman factorisation)**.** *$\boldsymbol{T}(\mathbf{y})$ is sufficient for $\boldsymbol{\theta}$ if and only if there are non-negative functions g and h with*

$$f(\mathbf{y} \mid \boldsymbol{\theta}) = g\big(\boldsymbol{T}(\mathbf{y}), \boldsymbol{\theta}\big)\, h(\mathbf{y}) \qquad \text{for all } \mathbf{y}, \boldsymbol{\theta}. \tag{5.24}$$

Proof (discrete case). ($\Rightarrow$) If $\boldsymbol{T}$ is sufficient, $f(\mathbf{y} \mid \boldsymbol{\theta}) = \mathbb{P}_{\boldsymbol{\theta}}(\boldsymbol{T} = \boldsymbol{T}(\mathbf{y}))\,\mathbb{P}(\mathbf{y} \mid \boldsymbol{T} = \boldsymbol{T}(\mathbf{y}))$; the first factor is $g(\boldsymbol{T}(\mathbf{y}), \boldsymbol{\theta})$ and the second, free of $\boldsymbol{\theta}$ by sufficiency, is $h(\mathbf{y})$. ($\Leftarrow$) Suppose (5.24). For $\mathbf{y}$ with $\boldsymbol{T}(\mathbf{y}) = \boldsymbol{t}$,

$$\mathbb{P}_{\boldsymbol{\theta}}(\mathbf{y} \mid \boldsymbol{T} = \boldsymbol{t}) = \frac{g(\boldsymbol{t}, \boldsymbol{\theta}) h(\mathbf{y})}{\sum_{\mathbf{y}' : \boldsymbol{T}(\mathbf{y}') = \boldsymbol{t}} g(\boldsymbol{t}, \boldsymbol{\theta}) h(\mathbf{y}')} = \frac{h(\mathbf{y})}{\sum_{\mathbf{y}' : \boldsymbol{T}(\mathbf{y}') = \boldsymbol{t}} h(\mathbf{y}')},$$

which does not depend on $\boldsymbol{\theta}$. The continuous case is analogous with densities, or follows from measure-theoretic arguments (Casella and Berger, 2002, Sec. 6.2). □

Consequently the likelihood depends on the data only through $\boldsymbol{T}$: by (5.24), $\mathcal{L}(\boldsymbol{\theta}; \mathbf{y}) \propto g(\boldsymbol{T}(\mathbf{y}), \boldsymbol{\theta})$. For an iid sample from the exponential family (5.15),

$$\prod_i f(y_i \mid \boldsymbol{\eta}) = \Big\{ \prod_i h(y_i) \Big\} \exp\Big\{ \boldsymbol{\eta}^{\top} \sum_i \boldsymbol{T}(y_i) - n A(\boldsymbol{\eta}) \Big\},$$

so $\sum_i \boldsymbol{T}(y_i)$ is sufficient: $\sum y_i$ for the Bernoulli and Poisson, $(\sum y_i, \sum y_i^2)$ for the normal, $(\sum \log y_i, \sum y_i)$ for the gamma. Its dimension does not grow with n — the property that the Pitman–Koopman–Darmois theorem shows is characteristic of the family.

⋆ Key Concept: Likelihood essentials

- $\mathcal{L}(\boldsymbol{\theta}) = f(\mathbf{y} \mid \boldsymbol{\theta})$ with $\mathbf{y}$ fixed; only ratios matter; work with $\ell = \log \mathcal{L}$, a sum over iid observations.
- Gaussian likelihood $\Rightarrow$ least squares; Bernoulli likelihood $\Rightarrow$ log loss; Poisson likelihood $\Rightarrow$ Poisson regression.
- Factorisation: $f = g(\boldsymbol{T}, \boldsymbol{\theta}) h(\mathbf{y}) \Leftrightarrow \boldsymbol{T}$ sufficient; in exponential families $\sum_i \boldsymbol{T}(y_i)$ is sufficient.
- $\mathcal{L}$ is not a probability distribution over $\boldsymbol{\theta}$; a prior is needed for that.

5.6.6 Model checking: fitting with `.fit` and QQ plots

A likelihood is only as good as the model behind it. The practical workflow is: propose several candidate families, fit each by maximum likelihood, compare them by maximised log-likelihood penalised for complexity (AIC $= 2k - 2\ell_{\max}$), and *look* at the fit. Every continuous `scipy.stats` distribution has a `fit` method returning MLEs of shape, location and scale; fixing `floc=0` is essential for positive data, otherwise SciPy also estimates a shift.

A *quantile–quantile (QQ) plot* graphs the sorted data $y_{(i)}$ against the fitted quantiles $\hat{F}^{-1}\big((i - \tfrac{1}{2})/n\big)$. If the model is right, the points lie near the 45-degree line (by the probability integral transform, $\hat{F}(y_{(i)})$ should look like uniform order statistics); curvature at the ends diagnoses tails that are too light or too heavy.

Listing 5.15: Fitting duration models and QQ plots `ch05/fit_qq.py`

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Model checking: fit candidate duration models with .fit, compare AIC, QQ plots
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
# simulated unemployment spells (months) from a gamma model with shape 1.8
spells = stats.gamma(a=1.8, scale=4.0).rvs(800, random_state=rng)
```

```python

cands = {"exponential": stats.expon, "gamma": stats.gamma,
         "Weibull": stats.weibull_min, "log-normal": stats.lognorm}
fits = {}
print(f"{'model':12s}{'params (shape, loc, scale)':>34s}{'loglik':>10s}{'AIC':>9s}")
for name, fam in cands.items():
    par = fam.fit(spells, floc=0)           # MLE with location fixed at 0
    ll = fam.logpdf(spells, *par).sum()
    kpar = len(par) - 1                     # loc was not estimated
    fits[name] = fam(*par)
    ps = ", ".join(f"{v:.3f}" for v in par)
    print(f"{name:12s}{ps:>34s}{ll:10.1f}{2 * kpar - 2 * ll:9.1f}")

ks = stats.kstest(spells, fits["exponential"].cdf)
print(f"KS test vs fitted exponential: D = {ks.statistic:.3f}, "
      f"p = {ks.pvalue:.2g} (nominal: params were estimated)")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
pr = (np.arange(1, len(spells) + 1) - 0.5) / len(spells)
xs = np.sort(spells)
for ax, name in zip(axes, ["exponential", "gamma"]):
    q = fits[name].ppf(pr)
    ax.scatter(q, xs, s=6, color="#1f4e79")
    lim = [0, max(q.max(), xs.max())]
    ax.plot(lim, lim, color="#cc9900", lw=1.5)
    ax.set_title(f"QQ plot: {name} fit", fontsize=10)
    ax.set_xlabel("theoretical quantile")
    ax.set_ylabel("sample quantile")
fig.tight_layout()
plt.savefig("ch05_qqplots.pdf", bbox_inches="tight")
```

Output

```
model               params (shape, loc, scale)    loglik      AIC
exponential                        0.000, 7.006   -2357.4   4716.8
gamma                  1.803, 0.000, 3.885   -2287.5   4579.1
Weibull                1.410, 0.000, 7.712   -2290.0   4584.0
log-normal             0.866, 0.000, 5.179   -2335.4   4674.8
KS test vs fitted exponential: D = 0.127, p = 1.2e-11 (nominal: params were estimated)
```

The gamma (the true model) has the smallest AIC, with the Weibull close behind; the exponential is decisively rejected, both by AIC and by the visible curvature of its QQ plot (Fig. 5.11). The Kolmogorov–Smirnov p-value is labelled "nominal" because using estimated parameters makes the standard KS distribution conservative; a parametric bootstrap (Chapter 11) gives correct p-values.

Policy Lens: Durations of unemployment

The shape of the fitted duration distribution has direct policy content. A Weibull shape $\hat{k} < 1$ (or a gamma shape below one) means falling exit rates: benefits that run out after a fixed period, or activation programmes, can be targeted at the duration at which exit rates collapse. A shape above one suggests that job seekers become *more* likely to leave the longer they search, as when benefit exhaustion approaches. Because unobserved heterogeneity mimics negative duration dependence (Caution in Section 5.2), applied work typically uses mixture

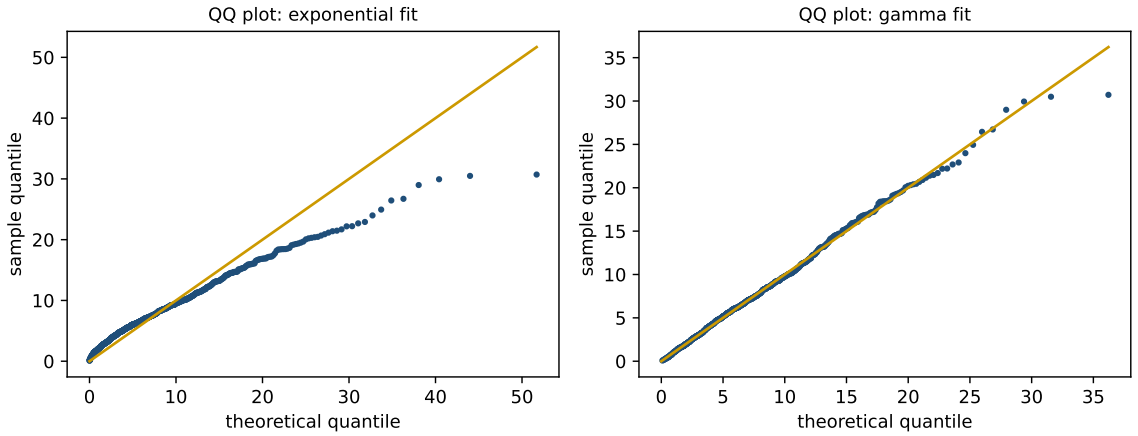


Figure 5.11: QQ plots for simulated unemployment spells. The exponential fit (left) has too heavy an upper tail and too many very short spells; the gamma fit (right) lies on the line.

models and censored likelihoods — spells still in progress at the survey date contribute $S(t_i)$ rather than $f(t_i)$ to the likelihood (Lancaster, 1990).

5.6.7 A Bayesian preview: prior times likelihood

Bayes' theorem (Chapter 4) turns a likelihood into a probability distribution over parameters once a prior $p(\boldsymbol{\theta})$ is specified:

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{\mathcal{L}(\boldsymbol{\theta}; \mathbf{y})\, p(\boldsymbol{\theta})}{\int \mathcal{L}(\boldsymbol{\theta}'; \mathbf{y}) p(\boldsymbol{\theta}') \, \mathrm{d}\boldsymbol{\theta}'} \;\propto\; \mathcal{L}(\boldsymbol{\theta}; \mathbf{y})\, p(\boldsymbol{\theta}). \tag{5.25}$$

Posterior $\propto$ likelihood $\times$ prior. The data enter only through the likelihood, which is why Bayesian inference obeys the likelihood principle. When the prior and posterior belong to the same family, the prior is *conjugate*.

Proposition 5.32 (Beta–binomial conjugacy)**.** *If* $\theta \sim \mathrm{Beta}(a, b)$ *and* $y \mid \theta \sim \mathrm{Bin}(n, \theta)$ *then* $\theta \mid y \sim \mathrm{Beta}(a + y,\ b + n - y)$*, with posterior mean*

$$\mathbb{E}[\theta \mid y] = \frac{a + y}{a + b + n} = w\,\frac{a}{a + b} + (1 - w)\,\frac{y}{n}, \qquad w = \frac{a + b}{a + b + n}. \tag{5.26}$$

Proof. $p(\theta \mid y) \propto \theta^{y}(1-\theta)^{n-y} \cdot \theta^{a-1}(1-\theta)^{b-1} = \theta^{a+y-1}(1-\theta)^{b+n-y-1}$, the kernel of $\mathrm{Beta}(a+y, b+n-y)$; the normalising constant must be $1/B(a+y, b+n-y)$. The mean formula is the beta mean, rearranged. □

The posterior mean is a weighted average of the prior mean and the MLE y/n; the prior acts like $a + b$ "pseudo-observations", and as $n \to \infty$ the weight $w \to 0$ and the data dominate. Matsuura (2022, Chs. 2–3) develops this view of modelling — write down the likelihood, choose priors, compute the posterior — and shows how Stan handles non-conjugate models by Markov chain Monte Carlo (Chapter 10).

Listing 5.16: Beta prior, binomial likelihood, beta posterior ch05/beta_binomial.py

```
# DEP 803 -- Chapter 5: Common Distributions in Data Science and Econometrics
# Bayesian preview: beta prior x binomial likelihood = beta posterior
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```python
import matplotlib.pyplot as plt
from scipy import stats

# simulated pilot survey: 14 of 40 sampled households take up a cash transfer
n, y = 40, 14
a0, b0 = 4.0, 12.0                # prior: take-up around 25% (prior mean 0.25)
grid = np.linspace(0, 1, 2001)
prior = stats.beta(a0, b0).pdf(grid)
lik = stats.binom.pmf(y, n, grid)
post_grid = prior * lik
post_grid /= np.trapz(post_grid, grid)       # normalise numerically
post = stats.beta(a0 + y, b0 + n - y)         # conjugate update

print(f"MLE y/n = {y / n:.4f}")
print(f"prior mean = {a0 / (a0 + b0):.4f}, posterior mean = {post.mean():.4f}")
w = (a0 + b0) / (a0 + b0 + n)
print(f"posterior mean = w*prior mean + (1-w)*MLE with w = {w:.4f}")
print(f"max |grid posterior - Beta({a0 + y:.0f},{b0 + n - y:.0f}) pdf| = "
      f"{np.abs(post_grid - post.pdf(grid)).max():.2e}")
lo, hi = post.ppf([0.025, 0.975])
print(f"95% credible interval: [{lo:.4f}, {hi:.4f}]")
print(f"P(take-up > 0.30 | data) = {post.sf(0.30):.4f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(grid, prior, color="#888888", lw=2, label="prior Beta(4,12)")
ax.plot(grid, lik / np.trapz(lik, grid), color="#1f4e79", lw=2, ls="--",
        label="likelihood (rescaled)")
ax.plot(grid, post.pdf(grid), color="#cc9900", lw=2.5, label="posterior Beta(18,38)")
ax.set_xlim(0, 0.8)
ax.set_xlabel("take-up rate theta")
ax.set_ylabel("density")
ax.legend(frameon=False)
plt.savefig("ch05_beta_binomial.pdf", bbox_inches="tight")
```

Output

```
MLE y/n = 0.3500
prior mean = 0.2500, posterior mean = 0.3214
posterior mean = w*prior mean + (1-w)*MLE with w = 0.2857
max |grid posterior - Beta(18,38) pdf| = 4.09e-14
95% credible interval: [0.2068, 0.4481]
P(take-up > 0.30 | data) = 0.6224
```

The grid computation of prior × likelihood, normalised numerically, coincides with the conjugate Beta(18, 38) posterior to rounding error. The posterior delivers statements a programme manager actually wants — "given the pilot, the probability that take-up exceeds 30% is about 0.62" — which a p-value cannot provide.

Graduate Perspective: From likelihood to modern inference

The likelihood is the hub connecting much of the rest of this book. Maximising it gives MLEs, whose asymptotic normality with covariance equal to the inverse Fisher information (Chapter 7) underlies Wald, score and likelihood-ratio tests. Penalising it gives ridge and lasso (Gaussian or Laplace priors in disguise), AIC and BIC. Integrating it against a prior

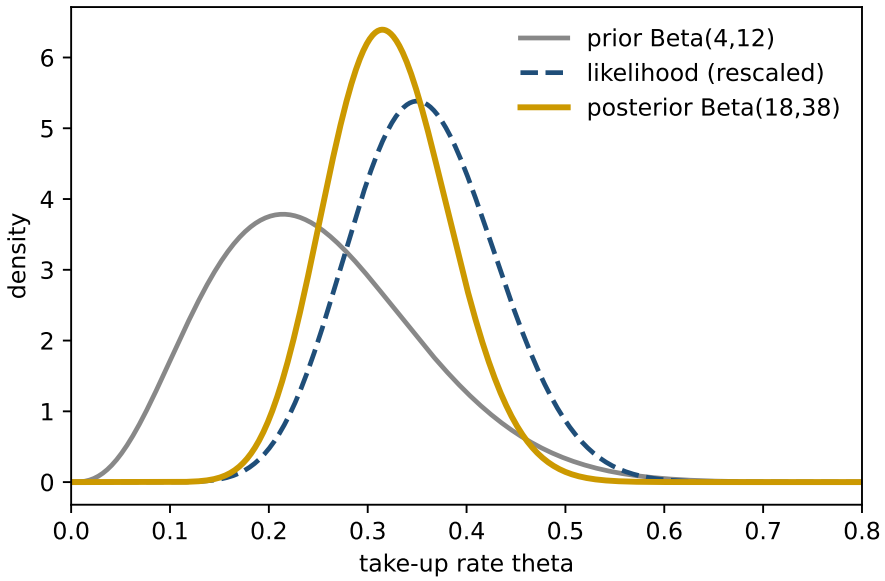


Figure 5.12: Bayesian updating of a programme take-up rate: the posterior (gold) is a compromise between the prior (grey) and the rescaled likelihood (blue).

gives Bayesian inference and marginal likelihoods for model comparison; when integrals are intractable, MCMC and variational inference take over. When the likelihood itself is intractable (agent-based or structural models), one turns to simulated likelihood, indirect inference or approximate Bayesian computation. Pawitan (2001) is an excellent book-length treatment of likelihood-based inference.

Exercises

5.6.1 Theory Using the factorisation theorem, find a sufficient statistic for (a) Unif$(0,\theta)$; (b) $\mathcal{N}(\mu,\sigma^2)$ with both parameters unknown; (c) the Pareto with known x_m; (d) the Weibull with known shape k.

5.6.2 Theory Show that the MLE of λ for an iid exponential sample in which m of the n spells are right-censored (still in progress) at times c_j is $\hat\lambda = d/\tau$, where d is the number of completed spells and τ the total time observed (completed plus censored). Explain why ignoring censoring biases the estimated mean duration downward.

5.6.3 Theory Verify that the binomial and negative binomial likelihoods in the likelihood-principle example are proportional, then compute the one-sided p-values for $H_0: p = 1/2$ against $p < 1/2$ under each sampling scheme.

5.6.4 Python Extend Listing 5.14 to plot the *profile* log-likelihood of μ in the normal model, $\ell_p(\mu) = \max_\sigma \ell(\mu,\sigma)$, and read off the 95% LR interval $\{\mu : 2[\ell_p(\hat\mu) - \ell_p(\mu)] \le 3.84\}$. Compare with the t interval.

5.6.5 Python Generate 500 incomes from the log-normal–Pareto mixture of Listing 5.6. Fit log-normal, gamma and Weibull models with `fit(floc=0)`, compare AIC, and draw QQ plots on log scales. Which feature of the data does every candidate miss?

5.6.6 ★ Graduate Prove that the MLE is invariant under reparametrisation: if $\hat\theta$ maximises $\mathcal{L}(\theta)$ and $\psi = g(\theta)$ for a bijection g, then $\hat\psi = g(\hat\theta)$. Show by example that the posterior mode is *not* invariant.

5.6.7 ★ Graduate Derive the posterior for a Poisson rate λ under a Gamma(a, b) prior with n iid counts. Show that the posterior predictive distribution of a new count is negative binomial, and connect this to Proposition 5.5.

5.6.8 Policy A pilot of a school-feeding programme in 8 schools finds that 61 of 150 sampled pupils had been absent at least once in the last week. A previous national survey suggests an absence rate around 30% (treat as a Beta$(9, 21)$ prior). Compute the posterior, a 90% credible interval and the probability that the absence rate exceeds 35%. How would you explain the result to a district education officer? What would change if the 150 pupils were clustered within 8 schools?

Chapter Summary

- **Discrete:** binomial (fixed trials), Poisson (rare events; limit of the binomial, with Le Cam's bound $\sum p_i^2$), geometric (memoryless waiting), negative binomial (gamma–Poisson mixture; Var $= \mu + \alpha\mu^2$), multinomial (categories). The variance-to-mean ratio diagnoses overdispersion in counts such as clinic visits.
- **Continuous:** the exponential is the unique memoryless duration; the Weibull allows monotone hazards; the gamma sums waiting times; the beta models proportions; the log-normal and Pareto model incomes, with Lorenz curves and Gini coefficients depending only on σ or α.
- **Sampling distributions:** χ^2, t and F are built from independent normals; in the normal linear model the orthogonality of fitted values and residuals delivers exact t and F tests. The Cauchy shows that averages can fail completely.
- **Multivariate normal:** closed under affine maps; marginals from sub-blocks; conditionals with linear mean and Schur-complement covariance — so the conditional mean is the population regression; $D^2 \sim \chi^2_d$; simulate via Cholesky.
- **Exponential family:** $h(y)e^{\boldsymbol{\eta}^\top \boldsymbol{T}(y) - A(\boldsymbol{\eta})}$, moments from derivatives of A, sufficient statistic $\sum \boldsymbol{T}(y_i)$. GLMs combine it with a linear predictor and a link; logistic and Poisson regression are fitted by IRLS.
- **Likelihood:** $f(\mathbf{y} \mid \boldsymbol{\theta})$ viewed as a function of $\boldsymbol{\theta}$; Gaussian likelihood gives least squares; the factorisation theorem characterises sufficiency; models are checked by AIC and QQ plots; prior × likelihood gives the posterior, e.g. beta–binomial.

Further Reading

For the distributions themselves and their relationships, Casella and Berger (2002, Chs. 3–5) and Wasserman (2004, Chs. 2–3) are the standard graduate references, and Murphy (2022, Ch. 2) gives a machine-learning perspective including the exponential family (Sec. 3.4) and GLMs (Ch. 12). Matsuura (2022) is the course text for Bayesian modelling built on likelihoods, with worked Stan and Python examples; Géron (2022, Ch. 4) develops logistic and softmax regression from the log loss, and Goulet Coulombe et al. (2022) illustrates the role of loss functions and distributional assumptions in macroeconomic forecasting. Count-data econometrics is covered by Cameron and Trivedi (2013) and Cameron and Trivedi (2005); duration models by Lancaster (1990); income distributions and inequality by Aitchison and Brown (1957) and Atkinson et al. (2011); GLMs by McCullagh and Nelder (1989); and likelihood theory by Pawitan (2001). Stigler (1986) tells the history of the normal distribution and least squares, and Hansen (2022) connects the normal regression model to modern econometric practice.

Chapter 6

Expectation, Covariance, Limit Theorems and Concentration

Learning Objectives

After working through this chapter you should be able to:

1. define the expectation of discrete, continuous and general random variables, use linearity and the law of the unconscious statistician (LOTUS), and prove Jensen's inequality;
2. interpret Jensen's inequality economically through risk aversion, certainty equivalents and the welfare value of insurance;
3. compute and interpret moments, moment generating functions and characteristic functions, and know when each exists;
4. manipulate variances, covariances and covariance matrices, prove that covariance matrices are positive semidefinite, and compute the variance of a portfolio or a composite index;
5. distinguish correlation from dependence and from causation, and explain Simpson's paradox;
6. state and prove the law of iterated expectations, the law of total variance and the best-predictor property of the conditional expectation function, and relate them to regression and average treatment effects;
7. prove Markov's and Chebyshev's inequalities and the weak law of large numbers, and explain the four main modes of convergence and how they are related;
8. state the central limit theorem (Lindeberg–Lévy, Lyapunov, Lindeberg, multivariate), the Berry–Esseen bound, Slutsky's theorem and the delta method, and use them to build standard errors for ratios and elasticities;
9. derive Hoeffding and Chernoff bounds, use them to determine survey sample sizes and to control many simultaneous estimates, and describe concentration of measure in high dimension.

A household survey records the consumption of a few thousand households; a statistics office then publishes a national poverty rate, a mean income and a margin of error. A central bank averages a hundred forecasters and calls the result the consensus. A machine-learning model is trained by minimising an *average* loss over a training sample and is judged by its average loss on new data. A cash-transfer programme is evaluated by comparing the *average* outcome of

beneficiaries with that of a control group. In every case an unknown population quantity, an *expectation*, is replaced by a sample average, and somebody must say how close the two are likely to be.

This chapter supplies the mathematics behind that substitution. Section 6.1 defines expectation and its main tools (linearity, LOTUS, Jensen's inequality, moment generating functions). Section 6.2 studies second moments: variance, covariance, covariance matrices, and the gap between correlation, dependence and causation. Section 6.3 develops conditional expectation, the single most important concept for regression and for policy evaluation. Section 6.4 proves that sample averages converge to expectations (the law of large numbers) and catalogues the ways in which random quantities can converge. Section 6.5 describes the shape of the error, the central limit theorem, together with the delta method that turns it into standard errors for almost any statistic. Finally Section 6.6 gives non-asymptotic guarantees, concentration inequalities, which are the language of modern statistical learning theory and high-dimensional data science (Vershynin, 2018).

The material builds directly on the probability spaces, random variables and distributions of Chapters 4 and 5, and it is the foundation for estimation theory (Chapter 7), Monte Carlo methods (Chapter 10) and resampling (Chapter 11). Good companions are Casella and Berger (2002), Wasserman (2004), Hansen (2022, Ch. 2) and, for the machine-learning perspective, Murphy (2022, Ch. 2).

6.1 Expectation

The expectation of a random variable is its probability-weighted average value: the centre of mass of its distribution. It is the population counterpart of the sample mean, the quantity a regression models, the objective a learning algorithm minimises and the welfare criterion of expected-utility theory.

6.1.1 Definition: discrete, continuous and general

Definition 6.1 (Expectation)**.** Let X be a random variable on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$.

(i) If X is discrete with support $\{x_1, x_2, \dots\}$ and pmf $p(x_j) = \mathbb{P}(X = x_j)$, then $\mathbb{E}[X] = \sum_j x_j\, p(x_j)$, provided $\sum_j |x_j|\, p(x_j) < \infty$.

(ii) If X is continuous with density f, then $\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, f(x)\, dx$, provided $\int |x|\, f(x)\, dx < \infty$.

(iii) In general $\mathbb{E}[X] = \int_\Omega X(\omega)\, \mathbb{P}(d\omega)$, the Lebesgue integral of X with respect to $\mathbb{P}$, defined below.

When the absolute sum or integral is finite we say X is *integrable* and write $X \in L^1$.

The general definition unifies the discrete and continuous cases (and mixtures of them, such as expenditure with a mass point at zero). The construction has three steps, which are worth knowing because they explain *why* expectation behaves as it does.

1. For a *simple* random variable $X = \sum_{k=1}^{m} a_k \mathbf{1}_{A_k}$ (finitely many values a_k on events A_k) define $\mathbb{E}[X] = \sum_k a_k \mathbb{P}(A_k)$.
2. For $X \geq 0$ define $\mathbb{E}[X] = \sup\{\mathbb{E}[S] : S \text{ simple},\ 0 \leq S \leq X\}$; this may be $+\infty$. Concretely one may take $S_m = \min\{\lfloor 2^m X \rfloor / 2^m,\ m\}$, which increases to X, and $\mathbb{E}[X] = \lim_m \mathbb{E}[S_m]$.
3. For general X write $X = X^+ - X^-$ with $X^+ = \max(X, 0)$, $X^- = \max(-X, 0)$, and set $\mathbb{E}[X] = \mathbb{E}[X^+] - \mathbb{E}[X^-]$ whenever at least one of the two is finite.

The "Lebesgue intuition" is that we slice the *range* of X into horizontal strips and weight each value by the probability of the set where X takes it, instead of slicing the domain as the Riemann integral does. A useful consequence for non-negative variables is the *tail formula*

$$\mathbb{E}[X] = \int_0^\infty \mathbb{P}(X > t)\,dt \qquad (X \ge 0), \tag{6.1}$$

which follows from writing $X = \int_0^\infty \mathbf{1}\{X > t\}\,dt$ and exchanging expectation and integral (Tonelli's theorem). For integer-valued $X \ge 0$ it reads $\mathbb{E}[X] = \sum_{k \ge 1} \mathbb{P}(X \ge k)$.

Example 6.2 (An expectation that does not exist)**.** For the standard Cauchy density $f(x) = 1/\{\pi(1+x^2)\}$, $\int_0^\infty x f(x)\,dx = \lim_{M\to\infty} \log(1+M^2)/(2\pi) = \infty$, and likewise for the negative half. Hence $\mathbb{E}[X^+] = \mathbb{E}[X^-] = \infty$ and $\mathbb{E}[X]$ is undefined, even though the density is symmetric about zero. We will see in Section 6.4 that sample means of Cauchy data never settle down.

Example 6.3 (St Petersburg game)**.** A fair coin is tossed until the first head; if this happens on toss k the prize is 2^k. Then $\mathbb{E}[\text{prize}] = \sum_{k\ge1} 2^k 2^{-k} = \infty$, yet few people would pay more than a modest sum to play. The resolution proposed by Daniel Bernoulli, that people value *utility* rather than money, leads directly to Jensen's inequality below: $\mathbb{E}[\log(\text{prize})] = \sum_k k \log 2\, 2^{-k} = 2\log 2 < \infty$.

Historical Note: Huygens, the value of a chance, and the Bernoullis

The idea of expectation predates probability itself. In their 1654 correspondence Blaise Pascal and Pierre de Fermat solved the "problem of points": how to divide the stakes of an interrupted game fairly. Christiaan Huygens, having heard of the problem in Paris, wrote the first printed treatise on probability, *De ratiociniis in ludo aleae* ("On reasoning in games of chance", 1657), organised around the *value* of a chance, which is what we now call expected value. The St Petersburg game was posed by Nicolaus Bernoulli in 1713, and his cousin Daniel Bernoulli's resolution through logarithmic utility (published in the *Commentarii* of the St Petersburg Academy, 1738) is the origin of expected-utility theory and of the concept of risk aversion.

6.1.2 Linearity and the law of the unconscious statistician

Theorem 6.4 (Properties of expectation)**.** *For integrable X, Y and constants a, b:*

(a) (linearity) $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, *whether or not X and Y are independent;*

(b) (monotonicity) if $X \le Y$ *almost surely then* $\mathbb{E}[X] \le \mathbb{E}[Y]$; *in particular* $|\mathbb{E}[X]| \le \mathbb{E}|X|$;

(c) (independence) if X and Y are independent then $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$.

Proof sketch. For simple random variables all three claims are finite-sum identities; for (a), write X and Y on a common partition $\{A_k \cap B_l\}$. They extend to non-negative variables through increasing simple approximations and the monotone convergence theorem, and to general integrable variables through $X = X^+ - X^-$. For (c) with simple $X = \sum_k a_k \mathbf{1}_{A_k}$ and $Y = \sum_l b_l \mathbf{1}_{B_l}$, independence gives $\mathbb{E}[XY] = \sum_{k,l} a_k b_l \mathbb{P}(A_k)\mathbb{P}(B_l) = \mathbb{E}[X]\mathbb{E}[Y]$. □

Linearity is astonishingly useful precisely because it needs no independence. The expected number of matches when n letters are put at random into n envelopes is $\sum_{i=1}^n \mathbb{P}(\text{letter } i \text{ matched}) = n \cdot (1/n) = 1$, although the match events are dependent.

Theorem 6.5 (Law of the unconscious statistician, LOTUS)**.** *If X has pmf p or density f and g is measurable with* $\mathbb{E}\,|g(X)| < \infty$, *then*

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \qquad \text{or} \qquad \mathbb{E}[g(X)] = \int g(x) f(x)\,dx.$$

More generally, for a random vector $\mathbf{X}$ with density f, $\mathbb{E}[g(\mathbf{X})] = \int g(\mathbf{x}) f(\mathbf{x})\,d\mathbf{x}$.

Proof (discrete case). Let $Y = g(X)$. Grouping the support of X according to the value of g, $\mathbb{E}[Y] = \sum_y y\mathbb{P}(Y = y) = \sum_y y \sum_{x:g(x)=y} p(x) = \sum_x g(x)p(x)$; absolute convergence justifies the rearrangement. The continuous case follows from the change-of-variables formula for Lebesgue integrals, $\int g \, d(\mathbb{P} \circ X^{-1}) = \int g(X) \, d\mathbb{P}$. □

The name is a joke at the expense of those who use the formula without noticing it is a theorem: we do *not* need the distribution of $Y = g(X)$ to compute $\mathbb{E}[Y]$. Listing 6.1 computes the mean of a lognormal income $Y = e^Z$, $Z \sim \mathcal{N}(\mu, s^2)$, three ways: from the density of Y, from LOTUS applied to Z, and by Monte Carlo. It also shows two facts we will explain with Jensen's inequality: $\mathbb{E}[1/X] > 1/\mathbb{E}[X]$ and $\mathbb{E}[\log Y] < \log \mathbb{E}[Y]$.

Listing 6.1: Expectation by summation, quadrature, LOTUS and Monte Carlo
ch06/ch06_expectation_lotus.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Expectations computed three ways: exact sums, quadrature (LOTUS), Monte Carlo.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import integrate, stats

rng = np.random.default_rng(803)

# 1. Discrete: household size X on {1,...,8} (simulated pmf)
x = np.arange(1, 9)
p = np.array([0.08, 0.12, 0.17, 0.18, 0.16, 0.12, 0.10, 0.07])
EX = np.sum(x * p)
EX2 = np.sum(x**2 * p)                      # LOTUS with g(x) = x^2
print("Household size (discrete)")
print(f"  E[X] = {EX:.4f}  E[X^2] = {EX2:.4f}  Var(X) = {EX2 - EX**2:.4f}")
print(f"  E[1/X] = {np.sum(p / x):.4f}  versus 1/E[X] = {1 / EX:.4f}")

# 2. Continuous: lognormal income Y = exp(Z), Z ~ N(mu, s^2)
mu, s = 7.0, 0.8
fy = stats.lognorm(s=s, scale=np.exp(mu)).pdf
fz = stats.norm(mu, s).pdf
EY_exact = np.exp(mu + s**2 / 2)
# (a) definition: integrate y * f_Y(y)  (split range for accuracy)
EY_def = sum(integrate.quad(lambda y: y * fy(y), a, b)[0]
             for a, b in [(0, 1e3), (1e3, 1e4), (1e4, np.inf)])
# (b) LOTUS: integrate exp(z) * f_Z(z), no density of Y needed
EY_lotus = integrate.quad(lambda z: np.exp(z) * fz(z),
                          mu - 12 * s, mu + 12 * s)[0]
# (c) Monte Carlo with a standard error
y = np.exp(rng.normal(mu, s, size=1_000_000))
print("\nLognormal income, mu = 7, s = 0.8 (simulated units)")
print(f"  exact  E[Y] = exp(mu + s^2/2) = {EY_exact:10.3f}")
print(f"  quad   int y f_Y(y) dy        = {EY_def:10.3f}")
print(f"  LOTUS  int e^z f_Z(z) dz      = {EY_lotus:10.3f}")
print(f"  Monte Carlo (n = 1e6)         = {y.mean():10.3f}"
      f"  (s.e. {y.std(ddof=1) / np.sqrt(y.size):.3f})")
print(f"  E[log Y] = {np.log(y).mean():.4f}  log E[Y] = {np.log(EY_exact):.4f}")
```

Output

```
Household size (discrete)
  E[X] = 4.3300  E[X^2] = 22.6700  Var(X) = 3.9211
  E[1/X] = 0.3167  versus 1/E[X] = 0.2309

Lognormal income, mu = 7, s = 0.8 (simulated units)
```

```
exact  E[Y] = exp(mu + s^2/2) =   1510.204
quad   int y f_Y(y) dy        =   1510.204
LOTUS  int e^z f_Z(z) dz      =   1510.204
Monte Carlo (n = 1e6)         =   1508.809  (s.e. 1.428)
E[log Y] = 6.9989  log E[Y] = 7.3200
```

The Monte Carlo estimate differs from the exact value $e^{\mu+s^2/2} \approx 1510.2$ by about one standard error. The standard error printed by the program, $s_Y/\sqrt{n}$, anticipates the central limit theorem of Section 6.5.

6.1.3 Jensen's inequality, risk aversion and certainty equivalents

Theorem 6.6 (Jensen's inequality)**.** *Let $\varphi : I \to \mathbb{R}$ be convex on an interval I, and let X take values in I with X and $\varphi(X)$ integrable. Then*

$$\varphi(\mathbb{E}[X]) \le \mathbb{E}[\varphi(X)]. \tag{6.2}$$

If φ is concave the inequality reverses. If φ is strictly convex, equality holds if and only if X is almost surely constant.

Proof. Let $\mu = \mathbb{E}[X] \in I$. A convex function lies above each of its supporting lines: there is a slope c (any element of the subdifferential at μ, e.g. the right derivative) such that $\varphi(x) \ge \varphi(\mu) + c(x-\mu)$ for all $x \in I$. Substituting $x = X$ and taking expectations, using monotonicity and linearity, $\mathbb{E}[\varphi(X)] \ge \varphi(\mu) + c(\mathbb{E}[X] - \mu) = \varphi(\mu)$. If φ is strictly convex the supporting-line inequality is strict for $x \neq \mu$, so equality forces $\mathbb{P}(X \neq \mu) = 0$. □

Many familiar inequalities are special cases: $\varphi(x) = x^2$ gives $\mathbb{E}[X^2] \ge (\mathbb{E}X)^2$, i.e. $\operatorname{Var}(X) \ge 0$; $\varphi(x) = 1/x$ on $(0,\infty)$ gives $\mathbb{E}[1/X] \ge 1/\mathbb{E}[X]$ (the household size example above); $\varphi = \log$ (concave) gives $\mathbb{E}[\log Y] \le \log \mathbb{E}[Y]$, which with a uniform distribution on $\{y_1, \dots, y_n\}$ is the arithmetic–geometric mean inequality.

Expected utility and the certainty equivalent. Let W be a random income and u an increasing utility function. A decision maker who ranks lotteries by $\mathbb{E}[u(W)]$ is *risk averse* if u is concave; by Jensen, $\mathbb{E}[u(W)] \le u(\mathbb{E}[W])$: the lottery is worth less than its mean received for sure. The *certainty equivalent* CE and *risk premium* π are defined by

$$u(\mathrm{CE}) = \mathbb{E}[u(W)], \qquad \pi = \mathbb{E}[W] - \mathrm{CE} \ \ge 0. \tag{6.3}$$

A second-order Taylor expansion of both sides around $\mu = \mathbb{E}[W]$ gives the Arrow–Pratt approximation

$$\pi \approx \tfrac{1}{2} A(\mu)\, \operatorname{Var}(W), \qquad A(w) = -\frac{u''(w)}{u'(w)}, \tag{6.4}$$

so the premium is proportional to the variance and to the coefficient of absolute risk aversion A. For constant relative risk aversion (CRRA), $u(c) = c^{1-\gamma}/(1-\gamma)$ ($\gamma \neq 1$) or $u(c) = \log c$ ($\gamma = 1$), and lognormal consumption $\log C \sim \mathcal{N}(\mu, s^2)$, LOTUS and the normal MGF give the exact result

$$\mathrm{CE} = \exp\{\mu + (1-\gamma)s^2/2\}, \qquad \frac{\mathrm{CE}}{\mathbb{E}[C]} = e^{-\gamma s^2/2}. \tag{6.5}$$

Listing 6.2 checks (6.5) by Monte Carlo and then studies a stylised farm household whose income is 1000 in a normal year and 400 in a drought year (probability 0.2). Figure 6.1 shows the geometry: the chord joining the two outcomes lies below the utility curve, so expected utility lies below the utility of expected income, and the certainty equivalent lies to the left of the mean.

Listing 6.2: Jensen's inequality, certainty equivalents and insurance ch06/ch06_jensen_ce.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Jensen's inequality, certainty equivalents and the value of insurance (CRRA).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)


def u(c, g):
    """CRRA utility with relative risk aversion g."""
    return np.log(c) if g == 1 else c**(1 - g) / (1 - g)


def u_inv(v, g):
    return np.exp(v) if g == 1 else ((1 - g) * v)**(1 / (1 - g))


# 1. Lognormal consumption: CE has the closed form exp(mu + (1-g) s^2 / 2)
mu, s = 7.0, 0.6
c = np.exp(rng.normal(mu, s, size=1_000_000))
print("Lognormal consumption: E[C] =", f"{c.mean():.1f}")
print(f"{'gamma':>6}{'CE (MC)':>10}{'CE exact':>10}{'premium':>9}{'% of E[C]':>11}")
for g in [0.5, 1, 2, 3]:
    ce = u_inv(np.mean(u(c, g)), g)
    exact = np.exp(mu + (1 - g) * s**2 / 2)
    rp = c.mean() - ce
    print(f"{g:6.1f}{ce:10.1f}{exact:10.1f}{rp:9.1f}{100 * rp / c.mean():10.1f}%")

# 2. A farm household: income 1000 in a normal year, 400 in a drought (prob 0.2)
w = np.array([1000.0, 400.0])
pr = np.array([0.8, 0.2])
g = 2
Ew = pr @ w
ce = u_inv(pr @ u(w, g), g)
print(f"\nFarm income: E[W] = {Ew:.1f}, CE (gamma = 2) = {ce:.1f}")
print(f"  risk premium                  = {Ew - ce:.1f}")
print(f"  actuarially fair premium      = {w[0] - Ew:.1f}")
print(f"  max premium for full insurance = {w[0] - ce:.1f}")

# Figure: log utility, chord, E[u(W)] < u(E[W]) and the certainty equivalent
cc = np.linspace(300, 1100, 300)
ul = np.log(w)
Eu = pr @ ul
ce1 = np.exp(Eu)
plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(cc, np.log(cc), color="navy", lw=2, label="u(w) = log w")
ax.plot(w, ul, "o--", color="gray", label="chord")
ax.plot(Ew, Eu, "s", color="firebrick", label="E[u(W)]")
ax.plot(Ew, np.log(Ew), "^", color="darkgreen", label="u(E[W])")
ax.hlines(Eu, 300, Ew, color="firebrick", lw=0.6, ls=":")
ax.vlines([ce1, Ew], 5.7, [Eu, np.log(Ew)], colors="k", lw=0.6, ls=":")
ax.text(ce1 - 75, 5.73, "CE")
ax.text(Ew + 10, 5.73, "E[W]")
ax.set_xlabel("income w")
ax.set_ylabel("utility")
ax.set_ylim(5.7, 7.05)
ax.legend(loc="upper left", frameon=False)
ax.set_title("Jensen: E[u(W)] <= u(E[W]) for concave u")
plt.savefig("ch06_jensen.pdf", bbox_inches="tight")
```

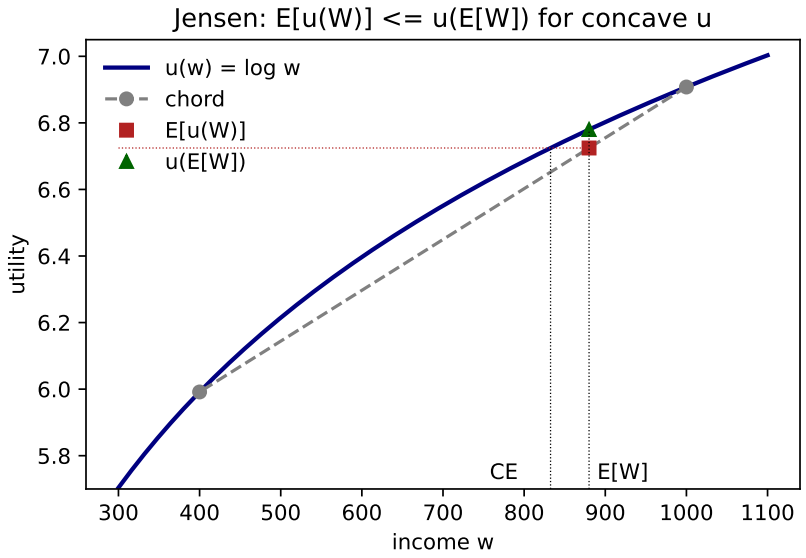


Figure 6.1: Jensen's inequality for log utility. The expected utility of the risky farm income (square) lies on the chord, below the utility of the mean income (triangle). The certainty equivalent CE is the sure income with the same utility; the gap $\mathbb{E}[W] - \text{CE}$ is the risk premium.

```
62 print(f"\nlog utility: CE = {ce1:.1f}, premium = {Ew - ce1:.1f}")
```

Output

```
Lognormal consumption: E[C] = 1311.9
 gamma   CE (MC)  CE exact  premium  % of E[C]
   0.5    1198.9    1199.9    113.0       8.6%
   1.0    1095.8    1096.6    216.2      16.5%
   2.0     915.6     916.0    396.4      30.2%
   3.0     765.2     765.1    546.7      41.7%

Farm income: E[W] = 880.0, CE (gamma = 2) = 769.2
  risk premium                   = 110.8
  actuarially fair premium       = 120.0
  max premium for full insurance = 230.8

log utility: CE = 832.6, premium = 47.4
```

With $\gamma = 2$ the household would accept a sure income of 769.2 in place of a lottery whose mean is 880. Full insurance replaces the lottery by the sure amount $1000 - P$, where P is the premium; it is worth buying whenever $1000 - P \geq \text{CE}$, i.e. $P \leq 230.8$, almost twice the actuarially fair premium of 120. The difference, 110.8, is the welfare gain from removing risk, which is the economic rationale for insurance, social protection and precautionary savings.

Policy Lens: Jensen's inequality and the welfare cost of risk

Because $\mathbb{E}[u(W)] < u(\mathbb{E}[W])$ for concave u, policies that leave mean income unchanged but reduce its variability (weather-index insurance, public works guarantees, buffer stocks, counter-cyclical transfers) raise welfare. Equation (6.4) lets an analyst put a money value on this: with relative risk aversion γ and coefficient of variation κ, the premium is roughly $\frac{1}{2}\gamma\kappa^2$ of mean income. A farm household with $\gamma = 2$ and $\kappa = 0.3$ would give up about 9% of

mean income to eliminate the risk. The same logic underlies Atkinson's inequality index, in which a concave social welfare function penalises dispersion of incomes across people rather than across states of nature.

△ Caution: $\mathbb{E}[g(X)] \neq g(\mathbb{E}[X])$: retransformation bias

A regression of $\log Y$ on $\mathbf{x}$ estimates $\mathbb{E}[\log Y \mid \mathbf{x}]$; exponentiating the fitted value does *not* estimate $\mathbb{E}[Y \mid \mathbf{x}]$. By Jensen it underestimates it; for normal errors with variance σ^2 the correction factor is $e^{\sigma^2/2}$, and Duan (1983) proposed a non-parametric "smearing" correction. With $\sigma = 0.8$ the naive back-transform is 27% too low, a serious error for a revenue projection or a cost–benefit analysis.

6.1.4 Moments, moment generating and characteristic functions

Definition 6.7 (Moments)**.** For $k \geq 1$, the k-th (raw) moment of X is $\mathbb{E}[X^k]$ and the k-th central moment is $\mathbb{E}[(X-\mu)^k]$ with $\mu = \mathbb{E}[X]$, whenever $\mathbb{E}\,|X|^k < \infty$. The variance is the second central moment; the *skewness* and *(excess) kurtosis* are $\mathbb{E}[(X-\mu)^3]/\sigma^3$ and $\mathbb{E}[(X-\mu)^4]/\sigma^4 - 3$.

If $\mathbb{E}\,|X|^k < \infty$ then $\mathbb{E}\,|X|^j < \infty$ for all $j < k$: apply Jensen to the convex function $t \mapsto t^{k/j}$ to obtain *Lyapunov's inequality* $(\mathbb{E}\,|X|^j)^{1/j} \leq (\mathbb{E}\,|X|^k)^{1/k}$. Heavy-tailed laws possess only finitely many moments: a Pareto law with tail index α has $\mathbb{E}[X^k] < \infty$ iff $k < \alpha$, and a Student-t with ν degrees of freedom has moments of order $k < \nu$ only.

Definition 6.8 (MGF and characteristic function)**.** The *moment generating function* (MGF) of X is $M_X(t) = \mathbb{E}[e^{tX}]$ for those $t \in \mathbb{R}$ where the expectation is finite. The *characteristic function* (CF) is $\phi_X(t) = \mathbb{E}[e^{itX}] = \mathbb{E}[\cos tX] + i\,\mathbb{E}[\sin tX]$, defined for all $t \in \mathbb{R}$.

Theorem 6.9 (Key properties)**.** *(a) If M_X is finite on an open interval around 0 then all moments exist, $M_X^{(k)}(0) = \mathbb{E}[X^k]$, and M_X determines the distribution of X uniquely.*

(b) ϕ_X always exists, $|\phi_X(t)| \leq 1$, $\phi_X(0) = 1$, and ϕ_X determines the distribution uniquely.

(c) If X and Y are independent then $M_{X+Y} = M_X M_Y$ and $\phi_{X+Y} = \phi_X \phi_Y$; also $M_{a+bX}(t) = e^{at} M_X(bt)$.

(d) (Lévy's continuity theorem) $X_n \xrightarrow{d} X$ if and only if $\phi_{X_n}(t) \to \phi_X(t)$ for every t.

Proof of (a), moments, and (c). If $M_X(t) < \infty$ for $|t| < h$, then $e^{|tx|} \leq e^{tx} + e^{-tx}$ is integrable and dominates every partial sum of $\sum_k (tx)^k/k!$, so by dominated convergence $M_X(t) = \sum_{k \geq 0} t^k \mathbb{E}[X^k]/k!$ for $|t| < h$; matching Taylor coefficients gives $M_X^{(k)}(0) = \mathbb{E}[X^k]$. For (c), $e^{t(X+Y)} = e^{tX} e^{tY}$ is a product of independent variables, so Theorem 6.4(c) applies. Uniqueness (a)–(b) and continuity (d) rest on Fourier inversion; see Billingsley (1995, Sec. 26). □

For instance $X \sim \mathcal{N}(\mu, \sigma^2)$ has $M_X(t) = \exp(\mu t + \sigma^2 t^2/2)$, so sums of independent normals are normal; a Poisson(λ) has $M(t) = \exp\{\lambda(e^t - 1)\}$, so sums of independent Poissons are Poisson. The logarithm $K_X(t) = \log M_X(t)$ is the *cumulant generating function*; its derivatives at zero are the cumulants (mean, variance, third central moment, …), and cumulants of independent sums *add*. We will use K_X again in Section 6.6 to derive Chernoff bounds. Listing 6.3 recovers Poisson moments by numerical differentiation of the MGF and shows that the Cauchy law, which has no MGF, nevertheless has a well-behaved characteristic function $\phi(t) = e^{-|t|}$.

Listing 6.3: Moments from the MGF; characteristic functions ch06/ch06_mgf_cf.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Moments from the MGF by differentiation; characteristic functions when MGFs fail.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# Poisson(lam): M(t) = exp(lam (e^t - 1)); derivatives at 0 give raw moments
lam = 3.0
M = lambda t: np.exp(lam * (np.exp(t) - 1))
h = 1e-3
m1 = (M(h) - M(-h)) / (2 * h)
m2 = (M(h) - 2 * M(0) + M(-h)) / h**2
print("Poisson(3) via central differences of the MGF")
print(f"  M'(0)  = {m1:.4f}  (exact E[X]   = {lam:.4f})")
print(f"  M''(0) = {m2:.4f}  (exact E[X^2] = {lam + lam**2:.4f})")
x = rng.poisson(lam, size=200_000)
for t in [0.2, 0.5, 1.0]:
    print(f"  t = {t:3.1f}: empirical MGF {np.mean(np.exp(t * x)):9.4f}"
          f"  exact {M(t):9.4f}")

# Cauchy: no MGF (E[e^{tX}] = inf for t != 0) but phi(t) = exp(-|t|) exists
xc = rng.standard_cauchy(size=200_000)
print("\nStandard Cauchy: empirical characteristic function")
for t in [0.5, 1.0, 2.0]:
    phi_hat = np.mean(np.exp(1j * t * xc))
    print(f"  t = {t:3.1f}: Re phi_hat = {phi_hat.real:7.4f}"
          f"  Im = {phi_hat.imag:7.4f}  exact exp(-|t|) = {np.exp(-t):.4f}")
print("  empirical MGF at t = 0.1 for growing n (does not settle):")
with np.errstate(over="ignore"):
    for n in [10**3, 10**4, 10**5, 2 * 10**5]:
        print(f"    n = {n:6d}: {np.mean(np.exp(0.1 * xc[:n])):.3e}")
```

Output

```
Poisson(3) via central differences of the MGF
  M'(0)  = 3.0000  (exact E[X]   = 3.0000)
  M''(0) = 12.0000  (exact E[X^2] = 12.0000)
  t = 0.2: empirical MGF    1.9423  exact    1.9430
  t = 0.5: empirical MGF    7.0030  exact    7.0018
  t = 1.0: empirical MGF  175.7194  exact  173.2690

Standard Cauchy: empirical characteristic function
  t = 0.5: Re phi_hat =  0.6066  Im =  0.0012  exact exp(-|t|) = 0.6065
  t = 1.0: Re phi_hat =  0.3676  Im =  0.0003  exact exp(-|t|) = 0.3679
  t = 2.0: Re phi_hat =  0.1335  Im = -0.0014  exact exp(-|t|) = 0.1353
  empirical MGF at t = 0.1 for growing n (does not settle):
    n =   1000: 2.288e+06
    n =  10000: 3.165e+241
    n = 100000: inf
    n = 200000: inf
```

The empirical MGF of the Cauchy sample explodes as n grows (there is no finite target to converge to), whereas the empirical characteristic function matches $e^{-|t|}$ to two or three decimals: $\cos(tX)$ is bounded, so its average always converges.

Graduate Perspective: Exchanging limits and expectations

Much of asymptotic theory consists of justifying $\lim_n \mathbb{E}[X_n] = \mathbb{E}[\lim_n X_n]$. The three workhorse results of Lebesgue integration are: *monotone convergence* (if $0 \le X_n \uparrow X$ then $\mathbb{E}X_n \uparrow \mathbb{E}X$); *Fatou's lemma* ($\mathbb{E}[\liminf X_n] \le \liminf \mathbb{E}[X_n]$ for $X_n \ge 0$); and *dominated convergence* (if $X_n \to X$ a.s. and $|X_n| \le Z$ with $\mathbb{E}Z < \infty$ then $\mathbb{E}X_n \to \mathbb{E}X$). Uniform integrability, $\lim_{M\to\infty} \sup_n \mathbb{E}[|X_n|\mathbf{1}\{|X_n| > M\}] = 0$, is the sharp condition under which convergence in probability implies convergence of means (Billingsley, 1995, Sec. 16). These results also justify differentiating under the integral sign, as in the proof above and in the score identities of maximum likelihood (Chapter 7).

⋆ Key Concept: Expectation

$\mathbb{E}[X] = \int X \, d\mathbb{P}$ is a linear, monotone functional; LOTUS lets us compute $\mathbb{E}[g(X)]$ from the law of X alone. Jensen's inequality $\varphi(\mathbb{E}X) \le \mathbb{E}\varphi(X)$ for convex φ is the mathematics of risk aversion: for concave utility the certainty equivalent lies below the mean, and the gap measures the welfare cost of risk. MGFs (when they exist) and characteristic functions (always) encode all moments and characterise distributions; they turn sums of independent variables into products.

Exercises

6.1.1 Theory Prove the tail formula (6.1) for a non-negative integer-valued X by writing $X = \sum_{k\ge1} \mathbf{1}\{X \ge k\}$. Use it to compute the mean of a geometric distribution on $\{1, 2, \dots\}$ with success probability p.

6.1.2 Theory Let $X > 0$ with $\mathbb{E}[X] = \mu$. Use Jensen's inequality to rank $\mathbb{E}[\sqrt{X}]$, $\sqrt{\mu}$, $\mathbb{E}[X^2]$, μ^2, $\mathbb{E}[e^{-X}]$ and $e^{-\mu}$. Which rankings need additional assumptions?

6.1.3 Theory Derive the Arrow–Pratt approximation (6.4) by expanding $u(\text{CE}) = u(\mu - \pi)$ to first order and $\mathbb{E}[u(W)]$ to second order around μ. Then derive the exact formula (6.5).

6.1.4 Python Modify Listing 6.2 to plot the maximum premium a farm household would pay for full insurance as a function of $\gamma \in [0, 6]$ and of the drought probability $q \in [0.05, 0.4]$ (a contour plot). Where is demand for insurance most sensitive to risk aversion?

6.1.5 Python Use LOTUS and numerical quadrature to compute $\mathbb{E}[\max(z - Y, 0)/z]$ (the poverty gap index) for lognormal income Y with $\mu = 7$, $s = 0.8$ and poverty line $z = 800$. Check your answer by Monte Carlo and report a standard error.

6.1.6 ★ Graduate Show that if $M_X(t) < \infty$ for $|t| < h$ then $\mathbb{P}(|X| > x) \le Ce^{-cx}$ for some constants $C, c > 0$ (exponential tails). Conclude that the lognormal distribution, which has all moments, has no MGF on any interval $(0, h)$. Is the lognormal determined by its moments? (Look up Heyde's example.)

6.1.7 Policy A government is choosing between two crop-support schemes with the same expected cost: (i) a fixed transfer of 60 to every farm household each year, (ii) a payment of 300 in drought years only (probability 0.2). Using the farm example with $\gamma = 2$, compute the certainty equivalent income under each scheme and explain which is better for welfare and why. What does the comparison assume about the households' access to credit and savings?

6.2 Variance, covariance and correlation

The mean locates a distribution; second moments describe its spread and how two or more variables move together. They are the raw material of portfolio theory, of composite indicators, of principal components (Chapter 3) and of least squares.

6.2.1 Definitions and basic properties

Definition 6.10 (Variance, covariance, correlation)**.** For $X, Y \in L^2$ (finite second moments) with means μ_X, μ_Y:

$$\operatorname{Var}(X) = \mathbb{E}[(X-\mu_X)^2] = \mathbb{E}[X^2]-\mu_X^2, \qquad \operatorname{Cov}(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)] = \mathbb{E}[XY]-\mu_X\mu_Y,$$

and, if both variances are positive, $\operatorname{Corr}(X,Y) = \operatorname{Cov}(X,Y)/\sqrt{\operatorname{Var}(X)\operatorname{Var}(Y)}$. The standard deviation is $\sigma_X = \sqrt{\operatorname{Var}(X)}$.

Proposition 6.11 (Properties)**.** *For $X, Y, Z \in L^2$ and constants a, b, c:*

(a) $\operatorname{Var}(aX+b) = a^2\operatorname{Var}(X)$*, and* $\operatorname{Var}(X) = 0$ *iff X is a.s. constant;*

(b) covariance is symmetric and bilinear: $\operatorname{Cov}(aX+bY, Z) = a\operatorname{Cov}(X,Z) + b\operatorname{Cov}(Y,Z)$*;*

(c) $\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X,Y)$*;*

(d) (Cauchy–Schwarz) $|\operatorname{Cov}(X,Y)| \le \sigma_X\sigma_Y$*, so* $-1 \le \operatorname{Corr}(X,Y) \le 1$*, with* $|\operatorname{Corr}(X,Y)| = 1$ *iff* $Y = a + bX$ *a.s. for some $b \neq 0$;*

(e) if X and Y are independent then $\operatorname{Cov}(X,Y) = 0$*.*

Proof. (a)–(c) follow from linearity of expectation. For (d), let $\tilde X = X - \mu_X$, $\tilde Y = Y - \mu_Y$ and consider $q(t) = \mathbb{E}[(\tilde Y - t\tilde X)^2] = \sigma_Y^2 - 2t\operatorname{Cov}(X,Y) + t^2\sigma_X^2 \ge 0$ for all t. A non-negative quadratic has non-positive discriminant: $4\operatorname{Cov}(X,Y)^2 - 4\sigma_X^2\sigma_Y^2 \le 0$. Equality holds iff $q(t^*) = 0$ for some t^*, i.e. $\tilde Y = t^*\tilde X$ a.s. Part (e) is Theorem 6.4(c) applied to $\tilde X, \tilde Y$. □

Correlation is invariant to changes of location and scale (with positive scale factors), which is why it is preferred for comparing associations measured in different units. It measures *linear* association only; we return to this point shortly.

6.2.2 Covariance matrices

For a random vector $\mathbf{X} = (X_1, \dots, X_p)^\top$ with mean $\boldsymbol{\mu} = \mathbb{E}[\mathbf{X}]$ the *covariance matrix* is

$$\boldsymbol{\Sigma} = \operatorname{Cov}(\mathbf{X}) = \mathbb{E}[(\mathbf{X}-\boldsymbol{\mu})(\mathbf{X}-\boldsymbol{\mu})^\top], \qquad \Sigma_{jk} = \operatorname{Cov}(X_j, X_k).$$

Linearity gives the two rules used constantly in econometrics and machine learning: for a fixed vector $\mathbf{w}$ and matrix $\mathbf{A}$,

$$\operatorname{Var}(\mathbf{w}^\top\mathbf{X}) = \mathbf{w}^\top\boldsymbol{\Sigma}\mathbf{w} = \sum_{j} w_j^2\Sigma_{jj} + 2\sum_{j<k} w_j w_k \Sigma_{jk}, \qquad \operatorname{Cov}(\mathbf{AX}+\mathbf{b}) = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top. \tag{6.6}$$

Theorem 6.12 (Covariance matrices are positive semidefinite)**.** *Every covariance matrix $\boldsymbol{\Sigma}$ is symmetric and positive semidefinite. It is positive definite unless some non-trivial linear combination $\mathbf{w}^\top\mathbf{X}$ is almost surely constant. Conversely, every symmetric PSD matrix is the covariance matrix of some random vector.*

Proof. Symmetry is clear. For any $\mathbf{w} \in \mathbb{R}^p$, by (6.6), $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} = \mathrm{Var}(\mathbf{w}^\top \mathbf{X}) = \mathbb{E}[(\mathbf{w}^\top(\mathbf{X} - \boldsymbol{\mu}))^2] \geq 0$. If $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} = 0$ for some $\mathbf{w} \neq \mathbf{0}$ then $\mathrm{Var}(\mathbf{w}^\top \mathbf{X}) = 0$, so $\mathbf{w}^\top \mathbf{X}$ is a.s. constant. Conversely, if $\boldsymbol{\Sigma}$ is symmetric PSD, write $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$ (e.g. a Cholesky or symmetric square-root factor, Chapter 3) and take $\mathbf{X} = \mathbf{L}\mathbf{z}$ with $\mathbf{z}$ a vector of independent standard normals; then $\mathrm{Cov}(\mathbf{X}) = \mathbf{L}\mathbf{I}\mathbf{L}^\top = \boldsymbol{\Sigma}$. □

The sample covariance matrix $\hat{\boldsymbol{\Sigma}} = (n-1)^{-1} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$ is the covariance matrix of the empirical distribution (up to the factor $n/(n-1)$), so it is PSD too; it is singular whenever $n \leq p$.

Portfolios and composite indices. A portfolio with weights $\mathbf{w}$ on assets with return covariance $\boldsymbol{\Sigma}$ has variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$; so does a composite index $\sum_j w_j X_j$ built from standardised indicators (a human development or asset index, a financial conditions index). Positive covariances *add* to the variance; ignoring them understates the uncertainty of the index. Minimising $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ subject to $\mathbf{1}^\top \mathbf{w} = 1$ (a Lagrangian exercise, Chapter 9) gives the minimum-variance weights $\mathbf{w}^\star = \boldsymbol{\Sigma}^{-1}\mathbf{1} / (\mathbf{1}^\top \boldsymbol{\Sigma}^{-1} \mathbf{1})$.

Listing 6.4: Covariance matrices, composite indices and a non-PSD trap
ch06/ch06_covariance_psd.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Covariance matrices are PSD; the variance of a composite index is w' S w.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# Simulated district indicators: schooling, health, assets, water
names = ["school", "health", "assets", "water"]
R = np.array([[1.0, 0.6, 0.5, 0.3],
              [0.6, 1.0, 0.4, 0.4],
              [0.5, 0.4, 1.0, 0.2],
              [0.3, 0.4, 0.2, 1.0]])
sd = np.array([1.0, 0.8, 1.5, 1.2])
S = np.outer(sd, sd) * R                       # population covariance
X = rng.multivariate_normal(np.zeros(4), S, size=5000)
S_hat = np.cov(X, rowvar=False)
print("eigenvalues of S_hat:", np.round(np.linalg.eigvalsh(S_hat), 3))

w = np.full(4, 0.25)                           # equal-weight composite index
index = X @ w
print(f"Var(index): w'S w = {w @ S @ w:.4f}  w'S_hat w = {w @ S_hat @ w:.4f}"
      f"  sample = {index.var(ddof=1):.4f}")
print(f"  ignoring covariances: sum w_i^2 s_ii = {np.sum(w**2 * np.diag(S)):.4f}")

# Minimum-variance weights (sum to one): w* proportional to S^{-1} 1
one = np.ones(4)
wmin = np.linalg.solve(S, one)
wmin /= wmin.sum()
print("min-variance weights:", dict(zip(names, np.round(wmin, 3))))
print(f"  variance at w*: {wmin @ S @ wmin:.4f}")

# A 'correlation matrix' assembled from inconsistent pairwise estimates
B = np.array([[1.0, 0.9, 0.9],
              [0.9, 1.0, -0.9],
              [0.9, -0.9, 1.0]])
lam, V = np.linalg.eigh(B)
print("\ninconsistent matrix eigenvalues:", np.round(lam, 3))
v = V[:, 0]
print(f"  'variance' of v'X for v = {np.round(v, 3)}: {v @ B @ v:.3f}  (< 0!)")
```

Output

```
eigenvalues of S_hat: [0.285 0.654 1.312 3.007]
Var(index): w'S w = 0.6849  w'S_hat w = 0.6861  sample = 0.6861
  ignoring covariances: sum w_i^2 s_ii = 0.3331
min-variance weights: {'school': 0.182, 'health': 0.615, 'assets': 0.03, 'water': 0.173}
  variance at w*: 0.5618

inconsistent matrix eigenvalues: [-0.8  1.9  1.9]
  'variance' of v'X for v = [-0.577  0.577  0.577]: -0.800  (< 0!)
```

The equal-weight index has variance 0.685, more than double the 0.333 we would report if we ignored the covariances. The last block shows a matrix with unit diagonal and entries in $[-1, 1]$ that is *not* a valid correlation matrix: if X_1 is strongly positively correlated with both X_2 and X_3, then X_2 and X_3 cannot be strongly negatively correlated. The negative eigenvalue would imply a negative variance for $X_2 + X_3 - X_1$.

△ Caution: Pairwise-deleted and hand-assembled covariance matrices

When missing values are handled by computing each covariance from the observations available for that pair ("pairwise deletion"), or when correlations are taken from different studies, the resulting matrix need not be PSD. Portfolio optimisers, Cholesky factorisations, Mahalanobis distances and Gaussian likelihoods then fail or produce nonsense. Always check the smallest eigenvalue; if it is negative, project onto the PSD cone (clip negative eigenvalues) or, better, use a coherent estimator such as complete-case or EM estimation, or a shrinkage estimator.

6.2.3 Correlation versus dependence

Independence implies zero covariance (Proposition 6.11(e)), but the converse is false. Covariance only detects *linear* association.

> **Example 6.13** (Uncorrelated but dependent)**.** Let $X \sim \mathcal{N}(0, 1)$ and $Y = X^2$. Then $\mathrm{Cov}(X, Y) = \mathbb{E}[X^3] - \mathbb{E}[X]\mathbb{E}[X^2] = 0$ because odd moments of a symmetric law vanish, yet Y is a deterministic function of X: knowing X tells us Y exactly. Independence would require $\mathbb{P}(X \in A, Y \in B) = \mathbb{P}(X \in A)\mathbb{P}(Y \in B)$ for all events, which fails, e.g., for $A = \{|x| > 1.5\}$, $B = \{y > 2\}$: since $A \subset B$, $\mathbb{P}(A \cap B) = \mathbb{P}(A) = 0.134$, whereas $\mathbb{P}(A)\mathbb{P}(B) = 0.134 \times 0.157 = 0.021$. Meanwhile $\mathrm{Corr}(|X|, Y) \approx 0.94$: a transformation of X reveals the dependence that the linear correlation misses.

There is one important case in which zero correlation *does* imply independence: if (X, Y) is *jointly* normal, $\mathrm{Cov}(X, Y) = 0$ makes the joint density factorise (Chapter 5). Two marginally normal variables that are not jointly normal can be uncorrelated and dependent (take $Y = SX$ with $S = \pm 1$ an independent random sign; then $\mathrm{Cov}(X, Y) = 0$ yet $|Y| = |X|$). Rank correlations share the blind spot of Pearson's coefficient, but mutual information, the maximal correlation $\sup_{f,g} \mathrm{Corr}(f(X), g(Y))$ and distance correlation are zero *only* under independence.

6.2.4 Correlation versus causation, and Simpson's paradox

A non-zero correlation between a policy variable T and an outcome Y may arise because T causes Y, because Y causes T, or because a third variable drives both. In policy data the third possibility is the rule rather than the exception: programmes are targeted at needy places, people select into training, and governments raise spending when growth slows.

Simpson's paradox is the most striking form: an association that holds within every subgroup

can reverse when the groups are pooled. Write the pooled covariance using the law of total covariance (proved in Section 6.3, Theorem 6.16):

$$\operatorname{Cov}(T, Y) = \underbrace{\mathbb{E}[\operatorname{Cov}(T, Y \mid G)]}_{\text{within groups}} + \underbrace{\operatorname{Cov}(\mathbb{E}[T \mid G], \mathbb{E}[Y \mid G])}_{\text{between groups}}. \tag{6.7}$$

If groups with high average T have low average Y, the between-group term can be negative and large enough to overturn positive within-group covariances. Listing 6.5 simulates a training programme that raises earnings by 0.8 per hour in every region, but which is targeted at a poorer region; Fig. 6.2 shows the result.

Listing 6.5: Simpson's paradox in a simulated training programme ch06/ch06_simpson.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Simpson's paradox: a training programme helps within regions, 'hurts' pooled.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
# Simulated: hours of training T; earnings Y; region 0 (richer) or 1 (poorer).
# The programme targets the poorer region, which trains more and earns less.
n = 600
region = rng.integers(0, 2, size=n)
base = np.where(region == 0, 60.0, 30.0)
T = np.where(region == 0, rng.uniform(0, 10, n), rng.uniform(10, 25, n))
Y = base + 0.8 * T + rng.normal(0, 4, n)


def slope(x, y):
    return np.cov(x, y)[0, 1] / np.var(x, ddof=1)


print(f"pooled slope of Y on T:        {slope(T, Y): .3f}")
for r, lab in [(0, "region A"), (1, "region B")]:
    m = region == r
    print(f"slope within {lab}:         {slope(T[m], Y[m]): .3f}  (n = {m.sum()})")
# Adjusting for region: regress Y on T and a region dummy
Z = np.column_stack([np.ones(n), T, region])
b = np.linalg.lstsq(Z, Y, rcond=None)[0]
print(f"slope on T controlling for region: {b[1]: .3f}")

plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
cols = ["tab:blue", "tab:orange"]
for r, lab in [(0, "region A (richer)"), (1, "region B (poorer)")]:
    m = region == r
    ax.scatter(T[m], Y[m], s=8, alpha=0.5, color=cols[r], label=lab)
    tt = np.linspace(T[m].min(), T[m].max(), 2)
    ax.plot(tt, Y[m].mean() + slope(T[m], Y[m]) * (tt - T[m].mean()),
            color=cols[r], lw=2)
tt = np.linspace(0, 25, 2)
ax.plot(tt, Y.mean() + slope(T, Y) * (tt - T.mean()), "k--", lw=2,
        label="pooled fit")
ax.set_xlabel("training hours T (simulated)")
ax.set_ylabel("earnings Y (simulated)")
ax.legend(frameon=False, fontsize=9)
plt.savefig("ch06_simpson.pdf", bbox_inches="tight")
```

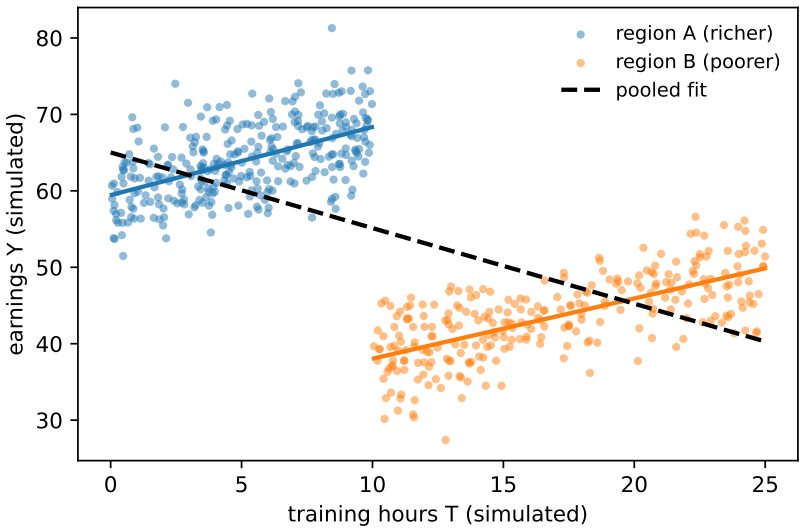


Figure 6.2: Simpson's paradox (simulated data). Within each region the slope of earnings on training hours is positive (about 0.8); pooling the regions gives a negative slope because the poorer region receives more training.

Output

```
pooled slope of Y on T:        -0.989
slope within region A:          0.891  (n = 323)
slope within region B:          0.788  (n = 277)
slope on T controlling for region:  0.821
```

Conditioning on the region recovers the within-region effect. But conditioning is not a universal cure: whether one *should* condition on a variable depends on the causal structure. Conditioning on a confounder (a common cause of T and Y) removes bias; conditioning on a mediator (a variable on the causal path from T to Y) or a collider (a common effect of T and Y) introduces it. Statistics alone, without a causal model, cannot decide which aggregation is "right" (Pearl, 2009).

Historical Note: Galton, Pearson, Yule and Simpson

Francis Galton introduced "co-relation" in 1888 while studying the relation between the lengths of different body parts, building on his studies of the heredity of stature that had led him to regression (Section 6.3). Karl Pearson gave the product-moment formula its modern mathematical treatment in the 1890s, and George Udny Yule noted as early as 1903 that associations in pooled contingency tables can mislead. The reversal phenomenon is named after Edward H. Simpson, whose 1951 paper in the *Journal of the Royal Statistical Society* discussed it for $2 \times 2 \times 2$ tables (Simpson, 1951); it had also been described earlier by Pearson and Yule.

⋆ Key Concept: Second moments

$\mathrm{Var}(\mathbf{w}^\top \mathbf{X}) = \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$: covariances matter for any sum, portfolio or index. Covariance matrices are symmetric PSD, and a matrix that is not PSD cannot be a covariance matrix. Correlation measures *linear* association: independence $\Rightarrow$ zero correlation, but not conversely (except under joint normality). Correlation is not causation, and pooled associations can reverse within groups (Simpson's paradox) because of the between-group term in the law of total covariance.

Exercises

6.2.1 Theory Show that $\operatorname{Var}(\bar{X}_n) = \sigma^2/n$ for an iid sample, and that for an *equicorrelated* sample ($\operatorname{Corr}(X_i, X_j) = \rho$, $i \neq j$) $\operatorname{Var}(\bar{X}_n) = \sigma^2\{1 + (n-1)\rho\}/n$. What happens as $n \to \infty$ when $\rho > 0$? Relate this to the design effect of cluster sampling.

6.2.2 Theory Prove that the correlation matrix $\mathbf{R} = \mathbf{D}^{-1/2}\boldsymbol{\Sigma}\mathbf{D}^{-1/2}$, $\mathbf{D} = \operatorname{diag}(\boldsymbol{\Sigma})$, is PSD, and find the set of ρ for which the 3×3 equicorrelation matrix with off-diagonal entries ρ is a valid correlation matrix.

6.2.3 Theory Let $X \sim \mathcal{N}(0,1)$ and S an independent random sign. Show that $Y = SX$ is $\mathcal{N}(0,1)$, that $\operatorname{Cov}(X, Y) = 0$, and that (X, Y) is not bivariate normal.

6.2.4 Python Using the covariance matrix in Listing 6.4, plot the variance of the index $\mathbf{w}^\top \mathbf{X}$ as $\mathbf{w}$ ranges over the simplex for the first three indicators (fix $w_4 = 0$). Mark the minimum-variance point and the equal-weight point.

6.2.5 Python Implement the "clip the negative eigenvalues" repair of the matrix $\mathbf{B}$ in Listing 6.4, rescale it to unit diagonal, and report how far (in Frobenius norm) the repaired matrix is from $\mathbf{B}$.

6.2.6 ★ Graduate Verify the decomposition (6.7) numerically for the Simpson simulation, reporting the within and between components. Then construct a simulation in which conditioning on a *collider* creates a spurious negative correlation between two independent variables (e.g. admission to a selective programme based on the sum of two scores).

6.2.7 Policy A ministry ranks 16 districts on an equal-weight index of four indicators measured with sampling error whose covariance matrix is $\boldsymbol{\Sigma}/100$ (with $\boldsymbol{\Sigma}$ from Listing 6.4). Simulate the probability that two districts whose true index values differ by 0.1 are ranked in the wrong order. How would you communicate this uncertainty in a published league table?

6.3 Conditional expectation

Regression, forecasting, classification and policy evaluation all ask the same question: *what is the average of Y among units with characteristics $X = x$?* The answer is the conditional expectation function, the central object of applied econometrics (Angrist and Pischke, 2009; Hansen, 2022).

6.3.1 Definition and basic rules

Definition 6.14 (Conditional expectation)**.** If (X, Y) is discrete, $\mathbb{E}[Y \mid X = x] = \sum_y y\, \mathbb{P}(Y = y \mid X = x)$ for $\mathbb{P}(X = x) > 0$. If it has joint density $f(x, y)$ and marginal $f_X(x) > 0$, $\mathbb{E}[Y \mid X = x] = \int y\, f(x, y)/f_X(x)\, dy$. The function $m(x) = \mathbb{E}[Y \mid X = x]$ is the *conditional expectation function* (CEF), and the random variable $\mathbb{E}[Y \mid X] = m(X)$ is the conditional expectation of Y given X. In general, for $Y \in L^1$, $\mathbb{E}[Y \mid X]$ is the (a.s. unique) random variable of the form $m(X)$ such that

$$\mathbb{E}\big[(Y - m(X))\, h(X)\big] = 0 \qquad \text{for every bounded measurable } h. \tag{6.8}$$

The general definition says that $Y - \mathbb{E}[Y \mid X]$ is orthogonal to every function of X. Its existence follows from the Radon–Nikodym theorem; for $Y \in L^2$ it is the orthogonal projection of Y onto the closed subspace of square-integrable functions of X (Theorem 6.17). The usual rules follow from (6.8):

(a) linearity: $\mathbb{E}[aY + bZ \mid X] = a\mathbb{E}[Y \mid X] + b\mathbb{E}[Z \mid X]$;

(b) taking out what is known: $\mathbb{E}[g(X)Y \mid X] = g(X)\mathbb{E}[Y \mid X]$;

(c) independence: if Y is independent of X then $\mathbb{E}[Y \mid X] = \mathbb{E}[Y]$;

(d) conditional Jensen: $\varphi(\mathbb{E}[Y \mid X]) \le \mathbb{E}[\varphi(Y) \mid X]$ for convex φ.

6.3.2 The law of iterated expectations and the law of total variance

Theorem 6.15 (Law of iterated expectations, LIE)**.** *For $Y \in L^1$, $\mathbb{E}\big[\mathbb{E}[Y \mid X]\big] = \mathbb{E}[Y]$. More generally (tower property), if Z is a function of X then $\mathbb{E}\big[\mathbb{E}[Y \mid X] \mid Z\big] = \mathbb{E}[Y \mid Z]$.*

Proof. In the continuous case, by LOTUS and Fubini,

$$\mathbb{E}[m(X)] = \int \Big(\int y \, \frac{f(x,y)}{f_X(x)} \, dy \Big) f_X(x)\, dx = \iint y \, f(x,y)\, dy\, dx = \mathbb{E}[Y].$$

In general take $h \equiv 1$ in (6.8). For the tower property, let $W = \mathbb{E}[Y \mid X]$; for bounded h, $\mathbb{E}[(W - \mathbb{E}[Y \mid Z])h(Z)] = \mathbb{E}[(W - Y)h(Z)] + \mathbb{E}[(Y - \mathbb{E}[Y \mid Z])h(Z)]$. The first term is zero by (6.8) because $h(Z)$ is a function of X, and the second is zero by the definition of $\mathbb{E}[Y \mid Z]$. □

In words: the overall average is the average of the group averages, weighted by group size, $\mathbb{E}[Y] = \sum_r \mathbb{P}(R = r)\mathbb{E}[Y \mid R = r]$. It is how national means are assembled from regional survey estimates, and it is the basis of post-stratification and of many proofs in this book.

Theorem 6.16 (Law of total variance and total covariance)**.** *For $Y, Z \in L^2$,*

$$\begin{aligned} \operatorname{Var}(Y) &= \mathbb{E}\big[\operatorname{Var}(Y \mid X)\big] + \operatorname{Var}\big(\mathbb{E}[Y \mid X]\big), \\ \operatorname{Cov}(Y, Z) &= \mathbb{E}\big[\operatorname{Cov}(Y, Z \mid X)\big] + \operatorname{Cov}\big(\mathbb{E}[Y \mid X], \mathbb{E}[Z \mid X]\big), \end{aligned} \tag{6.9}$$

where $\operatorname{Var}(Y \mid X) = \mathbb{E}[(Y - \mathbb{E}[Y \mid X])^2 \mid X]$ and similarly for $\operatorname{Cov}(Y, Z \mid X)$.

Proof. Write $m = \mathbb{E}[Y \mid X]$, $n = \mathbb{E}[Z \mid X]$ and $(Y - \mathbb{E}Y)(Z - \mathbb{E}Z) = [(Y - m) + (m - \mathbb{E}Y)][(Z - n) + (n - \mathbb{E}Z)]$. Expand and take expectations. The term $\mathbb{E}[(Y - m)(Z - n)] = \mathbb{E}[\operatorname{Cov}(Y, Z \mid X)]$ by the LIE; the term $\mathbb{E}[(m - \mathbb{E}Y)(n - \mathbb{E}Z)]$ equals $\operatorname{Cov}(m, n)$ because $\mathbb{E}m = \mathbb{E}Y$ by the LIE. The cross terms vanish: e.g. $\mathbb{E}[(Y - m)(n - \mathbb{E}Z)] = 0$ by (6.8), since $n - \mathbb{E}Z$ is a function of X. Setting $Z = Y$ gives (6.9). □

The variance of Y splits into a *within-group* part (average conditional variance) and a *between-group* part (variance of the conditional means). The ratio $\operatorname{Var}(\mathbb{E}[Y \mid X])/\operatorname{Var}(Y)$ is the population R^2 of the CEF, the largest share of the variance of Y that any function of X can explain. Listing 6.6 verifies both laws in a simulated three-region household survey.

Listing 6.6: Iterated expectations and the within/between decomposition
ch06/ch06_iterated_expectations.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Law of iterated expectations and law of total variance (within/between).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
# Simulated household survey: region shares and region-specific log incomes
regions = ["North", "Middle", "Coast"]
share = np.array([0.25, 0.45, 0.30])
mu = np.array([7.2, 7.8, 8.1])          # mean of log income by region
sig = np.array([0.7, 0.6, 0.8])
n = 200_000
r = rng.choice(3, size=n, p=share)
```

```
y = rng.normal(mu[r], sig[r])            # log income

cond_mean = np.array([y[r == k].mean() for k in range(3)])
cond_var = np.array([y[r == k].var() for k in range(3)])
p_hat = np.bincount(r) / n
print(f"{'region':>8}{'P(R=r)':>9}{'E[Y|R]':>9}{'Var(Y|R)':>10}")
for k in range(3):
    print(f"{regions[k]:>8}{p_hat[k]:9.3f}{cond_mean[k]:9.3f}{cond_var[k]:10.3f}")

lie = p_hat @ cond_mean
within = p_hat @ cond_var
between = p_hat @ (cond_mean - lie)**2
print(f"\nE[Y] directly           = {y.mean():.4f}")
print(f"E[E[Y|R]] (LIE)         = {lie:.4f}")
print(f"Var(Y) directly         = {y.var():.4f}")
print(f"E[Var(Y|R)] (within)    = {within:.4f}")
print(f"Var(E[Y|R]) (between)   = {between:.4f}")
print(f"within + between        = {within + between:.4f}")
print(f"share explained by region (R^2 of CEF) = {between / y.var():.3f}")
```

Output

```
  region   P(R=r)   E[Y|R]  Var(Y|R)
   North    0.250    7.201     0.486
  Middle    0.450    7.796     0.357
   Coast    0.299    8.104     0.637

E[Y] directly           = 7.7394
E[E[Y|R]] (LIE)         = 7.7394
Var(Y) directly         = 0.5872
E[Var(Y|R)] (within)    = 0.4734
Var(E[Y|R]) (between)   = 0.1138
within + between        = 0.5872
share explained by region (R^2 of CEF) = 0.194
```

Policy Lens: Within- and between-region inequality

The decomposition (6.9) applied to log consumption answers a classic question of spatial policy: how much of national inequality is *between* regions (and might be addressed by regional transfers or infrastructure) and how much is *within* regions (and requires household-level instruments)? In the simulated survey only 19% of the variance of log income is between regions. Real decompositions typically find that within-region inequality dominates, a finding that cautions against relying on geographic targeting alone. Decomposable inequality measures such as the Theil index extend this idea beyond the variance of logs.

Historical Note: Galton's quincunx and "regression"

Francis Galton built the quincunx, a board on which falling shot bounces off rows of pins into bins, to illustrate how many small independent deflections produce a normal curve; he described it in *Natural Inheritance* (1889). He extended the device to two stages to explain why a normal population stays normal from one generation to the next. In "Regression towards mediocrity in hereditary stature" (1886) he observed that the mean height of children of tall parents, while above average, is closer to the mean than the parents' heights: the conditional mean $\mathbb{E}[Y \mid X = x]$ has a slope smaller than one in standard units. The word "regression" has stayed with the conditional expectation ever since. The modern measure-theoretic definition of conditional expectation is due to Andrey Kolmogorov's *Grundbegriffe der Wahrscheinlichkeitsrechnung* (1933).

6.3.3 The CEF as the best mean-square predictor

Theorem 6.17 (CEF decomposition and best prediction)**.** *Let $Y \in L^2$ and $m(X) = \mathbb{E}[Y \mid X]$. Then $Y = m(X) + e$ with $\mathbb{E}[e \mid X] = 0$, so e is uncorrelated with every function of X. Moreover, for every function g with $g(X) \in L^2$,*

$$\mathbb{E}\big[(Y - g(X))^2\big] = \mathbb{E}\big[(Y - m(X))^2\big] + \mathbb{E}\big[(m(X) - g(X))^2\big], \tag{6.10}$$

so m minimises the mean-squared prediction error, uniquely up to a.s. equality.

Proof. $\mathbb{E}[e \mid X] = \mathbb{E}[Y \mid X] - m(X) = 0$ by rule (b). Write $Y - g = (Y - m) + (m - g)$ and expand: the cross term $2\mathbb{E}[e\,(m(X) - g(X))]$ vanishes by (6.8) (or by the LIE and rule (b): $\mathbb{E}[e\,h(X)] = \mathbb{E}[h(X)\mathbb{E}[e \mid X]] = 0$). The last term in (6.10) is non-negative and zero only if $g(X) = m(X)$ a.s. □

Equation (6.10) is a Pythagorean theorem in the Hilbert space L^2: $m(X)$ is the orthogonal projection of Y onto the space of functions of X. This is why squared-error machine learning (regression trees, random forests, neural networks with quadratic loss) is, at the population level, estimation of the CEF; other losses target other functionals (the conditional median for absolute loss, conditional quantiles for the check loss).

6.3.4 Linear projection versus the CEF

Definition 6.18 (Best linear predictor)**.** For $\mathbf{X} \in \mathbb{R}^k$ (including a constant) with $\mathbb{E}[\mathbf{X}\mathbf{X}^\top]$ positive definite, the *linear projection* of Y on $\mathbf{X}$ is $\mathbf{X}^\top\boldsymbol{\beta}$ with

$$\boldsymbol{\beta} = \arg\min_{\mathbf{b}} \mathbb{E}\big[(Y - \mathbf{X}^\top\mathbf{b})^2\big] = \big(\mathbb{E}[\mathbf{X}\mathbf{X}^\top]\big)^{-1}\mathbb{E}[\mathbf{X}Y]. \tag{6.11}$$

With a scalar regressor and an intercept, the slope is $\mathrm{Cov}(X, Y)/\mathrm{Var}(X)$.

The first-order condition is $\mathbb{E}[\mathbf{X}(Y - \mathbf{X}^\top\boldsymbol{\beta})] = \mathbf{0}$: the projection error $u = Y - \mathbf{X}^\top\boldsymbol{\beta}$ is uncorrelated with the regressors, but *not* in general with nonlinear functions of them. Two facts link the projection to the CEF (Angrist and Pischke, 2009, Ch. 3):

(i) if the CEF is linear, $m(\mathbf{x}) = \mathbf{x}^\top\mathbf{b}$, then $\mathbf{b} = \boldsymbol{\beta}$;

(ii) in all cases $\boldsymbol{\beta}$ is the best linear *approximation* to the CEF: $\boldsymbol{\beta} = \arg\min_{\mathbf{b}} \mathbb{E}[(m(\mathbf{X}) - \mathbf{X}^\top\mathbf{b})^2]$. This follows from (6.10) with $g(\mathbf{X}) = \mathbf{X}^\top\mathbf{b}$, because the first term on the right does not depend on $\mathbf{b}$.

OLS estimates $\boldsymbol{\beta}$; it therefore always estimates *something* meaningful, the best linear approximation to the CEF, even when the CEF is nonlinear. Listing 6.7 illustrates with a convex schooling–earnings relation.

Listing 6.7: The CEF, binned means and the linear projection ch06/ch06_cef_blp.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# The CEF minimises mean-squared error; the linear projection is its best line.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
# Simulated: X = years of schooling (0-16), Y = log earnings, nonlinear CEF
n = 50_000
x = rng.uniform(0, 16, n)
```

```
cef = lambda t: 1.0 + 0.02 * t + 0.006 * t**2      # convex returns
y = cef(x) + rng.normal(0, 0.3 + 0.02 * x)         # heteroskedastic noise

# Linear projection from moments: b = Cov(X,Y)/Var(X), a = E[Y] - b E[X]
b = np.cov(x, y)[0, 1] / np.var(x, ddof=1)
a = y.mean() - b * x.mean()
# Nonparametric CEF estimate: means within 16 bins
edges = np.linspace(0, 16, 17)
idx = np.clip(np.digitize(x, edges) - 1, 0, 15)
bin_mean = np.bincount(idx, weights=y) / np.bincount(idx)

mse = lambda pred: np.mean((y - pred)**2)
print(f"linear projection: a = {a:.4f}, b = {b:.4f}")
print(f"MSE of true CEF          : {mse(cef(x)):.5f}")
print(f"MSE of binned CEF        : {mse(bin_mean[idx]):.5f}")
print(f"MSE of linear projection : {mse(a + b * x):.5f}")
print(f"MSE of constant E[Y]     : {mse(y.mean()):.5f}")
# Orthogonality: the CEF error is uncorrelated with ANY function of X
corr = lambda a, b: np.corrcoef(a, b)[0, 1]
e = y - cef(x)
print(f"CEF error:  corr(e, X) = {corr(e, x): .4f}  corr(e, X^2) = "
      f"{corr(e, x**2): .4f}  corr(e, sin X) = {corr(e, np.sin(x)): .4f}")
u = y - a - b * x
print(f"projection: corr(u, X) = {corr(u, x): .4f}  corr(u, X^2) = "
      f"{corr(u, x**2): .4f}")

plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
sub = rng.choice(n, 1500, replace=False)
ax.scatter(x[sub], y[sub], s=4, alpha=0.3, color="gray", label="data (subsample)")
t = np.linspace(0, 16, 200)
ax.plot(t, cef(t), color="navy", lw=2, label="CEF E[Y|X]")
ax.plot(t, a + b * t, color="firebrick", lw=2, ls="--", label="linear projection")
mid = (edges[:-1] + edges[1:]) / 2
ax.plot(mid, bin_mean, "o", color="darkgreen", ms=4, label="binned means")
ax.set_xlabel("years of schooling X (simulated)")
ax.set_ylabel("log earnings Y")
ax.legend(frameon=False, fontsize=9)
plt.savefig("ch06_cef.pdf", bbox_inches="tight")
```

Output

```
linear projection: a = 0.7452, b = 0.1156
MSE of true CEF          : 0.21903
MSE of binned CEF        : 0.22011
MSE of linear projection : 0.23225
MSE of constant E[Y]     : 0.51711
CEF error:  corr(e, X) = -0.0044  corr(e, X^2) = -0.0039  corr(e, sin X) =  0.0040
projection: corr(u, X) = -0.0000  corr(u, X^2) =  0.0596
```

The CEF error is (up to sampling noise) uncorrelated with X, X^2 and $\sin X$; the projection error is exactly uncorrelated with X (by construction) but correlated with X^2, revealing the curvature the line misses. The MSE ordering $0.219 < 0.220 < 0.232 < 0.517$ is (6.10) in action.

△ Caution: Mean independence versus uncorrelatedness

$\mathbb{E}[e \mid X] = 0$ (the CEF error) is much stronger than $\mathbb{E}[Xe] = 0$ (the projection error). Unbiasedness of OLS for a *structural* or causal parameter requires the former to hold for the

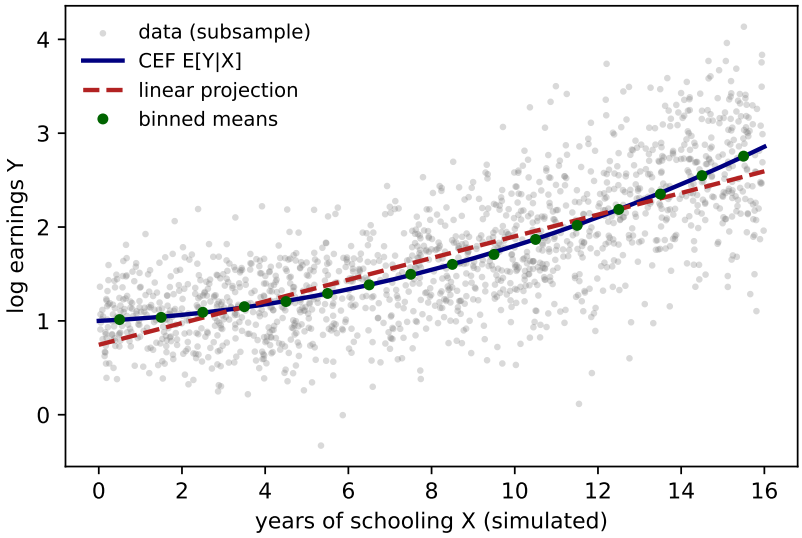


Figure 6.3: A convex CEF (solid), its estimate by binned means (dots) and the linear projection (dashed) in simulated schooling–earnings data. The projection is the best straight-line approximation to the CEF, weighted by the density of X.

structural error, which is an assumption about the world, not a property of least squares. A regression always produces residuals orthogonal to the regressors; that tells us nothing about whether omitted variables are correlated with them.

6.3.5 Regression and average treatment effects

Let $D \in \{0, 1\}$ indicate participation in a programme and let Y_1, Y_0 be the *potential outcomes* with and without it, so the observed outcome is $Y = DY_1 + (1 - D)Y_0$. The average treatment effect is $\text{ATE} = \mathbb{E}[Y_1 - Y_0]$ and the effect on the treated is $\text{ATT} = \mathbb{E}[Y_1 - Y_0 \mid D = 1]$. Since D is binary, the CEF of Y given D is linear, and the OLS slope on D equals the difference in means:

$$\mathbb{E}[Y \mid D = 1] - \mathbb{E}[Y \mid D = 0] = \underbrace{\mathbb{E}[Y_1 - Y_0 \mid D = 1]}_{\text{ATT}} + \underbrace{\mathbb{E}[Y_0 \mid D = 1] - \mathbb{E}[Y_0 \mid D = 0]}_{\text{selection bias}}. \tag{6.12}$$

Under random assignment D is independent of (Y_0, Y_1), so by rule (c) the selection bias vanishes and ATT = ATE: the difference in means identifies the ATE.

Listing 6.8: Difference in means, OLS on a dummy, and selection bias ch06/ch06_ate_regression.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Regression on a treatment dummy = difference of conditional means (ATE intuition).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n = 20_000
# Potential outcomes for a simulated cash-transfer programme
ability = rng.normal(size=n)
y0 = 50 + 8 * ability + rng.normal(0, 5, n)
y1 = y0 + 6 + 2 * ability                       # heterogeneous effect
print(f"true ATE = E[Y1 - Y0] = {np.mean(y1 - y0):.3f}")


def ols_dummy(d, y):
    Z = np.column_stack([np.ones(len(d)), d])
```

```
    return np.linalg.lstsq(Z, y, rcond=None)[0][1]


# (a) Randomised assignment: D independent of (Y0, Y1)
d = rng.integers(0, 2, n)
y = np.where(d == 1, y1, y0)
print("\nRandomised assignment")
print(f"  E[Y|D=1] - E[Y|D=0] = {y[d == 1].mean() - y[d == 0].mean():.3f}")
print(f"  OLS slope on D      = {ols_dummy(d, y):.3f}")

# (b) Self-selection: high-ability people enrol more often
d = (ability + rng.normal(size=n) > 0.5).astype(int)
y = np.where(d == 1, y1, y0)
naive = y[d == 1].mean() - y[d == 0].mean()
att = np.mean((y1 - y0)[d == 1])
sel = y0[d == 1].mean() - y0[d == 0].mean()
print("\nSelf-selection on ability")
print(f"  naive difference    = {naive:.3f}")
print(f"  = ATT {att:.3f} + selection bias {sel:.3f}")
print(f"  (ATT exceeds ATE because gains rise with ability)")
```

Output

```
true ATE = E[Y1 - Y0] = 5.990

Randomised assignment
  E[Y|D=1] - E[Y|D=0] = 5.847
  OLS slope on D      = 5.847

Self-selection on ability
  naive difference    = 16.423
  = ATT 7.445 + selection bias 8.977
  (ATT exceeds ATE because gains rise with ability)
```

Policy Lens: Why randomised evaluations work

Equation (6.12) is the algebra behind randomised controlled trials of cash transfers, school feeding or agricultural extension. In the simulation, self-selection of able individuals nearly triples the naive estimate (16.4 against a true ATE of 6.0), because those who enrol would have earned more anyway. Randomisation makes the comparison group a valid counterfactual. When randomisation is impossible, the same conditional-expectation algebra guides the alternatives: conditioning on covariates ("selection on observables"), instrumental variables, differences-in-differences and regression discontinuity designs (Angrist and Pischke, 2009).

⋆ Key Concept: Conditional expectation

$\mathbb{E}[Y \mid X]$ is the function of X whose error is orthogonal to every function of X. It satisfies the LIE $\mathbb{E}[\mathbb{E}[Y \mid X]] = \mathbb{E}[Y]$ and the variance decomposition $\operatorname{Var} Y = \mathbb{E}\operatorname{Var}(Y \mid X) + \operatorname{Var}\mathbb{E}(Y \mid X)$, and it is the best predictor of Y in mean square. The linear projection (what OLS estimates) is the best linear approximation to the CEF. With a binary treatment the regression slope is a difference of conditional means, which equals the ATE when treatment is randomly assigned.

Exercises

6.3.1 Theory Let N be the number of households in a village (random, mean ν, variance τ^2) and let household consumptions $Y_1, Y_2, \dots$ be iid with mean μ, variance σ^2, independent of N. Use the LIE and the law of total variance to show that total village consumption $S = \sum_{i=1}^{N} Y_i$ has $\mathbb{E}S = \nu\mu$ and $\operatorname{Var} S = \nu\sigma^2 + \mu^2\tau^2$.

6.3.2 Theory Prove that the conditional variance satisfies $\mathbb{E}[\operatorname{Var}(Y \mid X)] \le \operatorname{Var}(Y)$ and give an example in which $\operatorname{Var}(Y \mid X = x) > \operatorname{Var}(Y)$ for some x.

6.3.3 Theory Prove facts (i) and (ii) about the linear projection, and derive the slope formula $\operatorname{Cov}(X, Y)/\operatorname{Var}(X)$ from (6.11).

6.3.4 Python Replace the convex CEF in Listing 6.7 by $m(x) = 1 + 0.5\sin(x/2)$ and $X \sim$ Beta-distributed on $[0, 16]$ with most mass at low schooling. Show numerically that the projection slope changes with the distribution of X although the CEF does not.

6.3.5 Python Fit a regression tree (scikit-learn `DecisionTreeRegressor`) of increasing depth to the data in Listing 6.7, and plot its test MSE against depth together with the MSE of the true CEF. Explain the floor.

6.3.6 ★ Graduate Let (X, Y) be bivariate normal with correlation ρ. Derive $\mathbb{E}[Y \mid X]$ and $\operatorname{Var}(Y \mid X)$, and use them to explain Galton's regression to the mean in standard units. Why is regression to the mean a threat to before–after evaluations of programmes targeted at districts with unusually bad outcomes?

6.3.7 Policy In Listing 6.8 add a baseline measure of ability to the regression under self-selection. Does the coefficient on D now estimate the ATE or the ATT? Relate your answer to effect heterogeneity and to the weights OLS places on different strata (Angrist and Pischke, 2009, Sec. 3.3).

6.4 Inequalities and the law of large numbers

Why should a sample average be close to the population mean? The answer comes in two steps: simple inequalities that bound tail probabilities by moments, and the law of large numbers that they imply.

6.4.1 Markov's and Chebyshev's inequalities

Theorem 6.19 (Markov's inequality). *If $X \ge 0$ and $a > 0$, then $\mathbb{P}(X \ge a) \le \mathbb{E}[X]/a$. More generally, for any increasing $\varphi \ge 0$ with $\varphi(a) > 0$, $\mathbb{P}(X \ge a) \le \mathbb{E}[\varphi(X)]/\varphi(a)$.*

Proof. Pointwise, $a\mathbf{1}\{X \ge a\} \le X$ because the left side is 0 when $X < a$ and $a \le X$ otherwise. Taking expectations gives $a\mathbb{P}(X \ge a) \le \mathbb{E}X$. For the general form, apply the same argument to $\varphi(X)$ and the event $\{\varphi(X) \ge \varphi(a)\} \supseteq \{X \ge a\}$. □

Theorem 6.20 (Chebyshev's inequality). *If $\operatorname{Var}(X) = \sigma^2 < \infty$ and $\mu = \mathbb{E}X$, then for every $k > 0$*

$$\mathbb{P}(|X - \mu| \ge k) \le \frac{\sigma^2}{k^2}, \qquad \text{equivalently} \qquad \mathbb{P}(|X - \mu| \ge t\sigma) \le \frac{1}{t^2}.$$

Proof. Apply Markov's inequality to the non-negative variable $(X - \mu)^2$ and the level k^2: $\mathbb{P}(|X - \mu| \ge k) = \mathbb{P}((X - \mu)^2 \ge k^2) \le \mathbb{E}[(X - \mu)^2]/k^2$. □

Both inequalities are *distribution-free*: they need only one moment. The price is that they are usually loose, as Listing 6.9 shows, although Chebyshev's bound is attained by a three-point

distribution and so cannot be improved without further assumptions. Applying Markov to $e^{\lambda X}$ instead of X^2 yields the far sharper exponential bounds of Section 6.6.

Listing 6.9: Exact tails versus Markov and Chebyshev bounds ch06/ch06_markov_chebyshev.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# How tight are Markov's and Chebyshev's inequalities? Exact tails versus bounds.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

# Markov: P(X >= a) <= E[X]/a for X >= 0. Exponential(1) has E[X] = 1.
print("Markov, X ~ Exponential(1)")
print(f"{'a':>5}{'exact':>12}{'bound 1/a':>12}")
for a in [1, 2, 4, 8]:
    print(f"{a:5d}{stats.expon.sf(a):12.5f}{1 / a:12.5f}")

# Chebyshev: P(|X - mu| >= k sigma) <= 1/k^2
dists = {"Normal": stats.norm(),
         "Uniform": stats.uniform(-np.sqrt(3), 2 * np.sqrt(3)),
         "Laplace": stats.laplace(scale=1 / np.sqrt(2)),
         "t(3)": stats.t(3, scale=1 / np.sqrt(3))}
print("\nChebyshev: P(|X - mu| >= k sd) for unit-variance laws")
print(f"{'k':>5}" + "".join(f"{d:>10}" for d in dists) + f"{'1/k^2':>9}")
for k in [1.5, 2, 3, 4]:
    row = "".join(f"{2 * D.sf(k):10.5f}" for D in dists.values())
    print(f"{k:5.1f}{row}{1 / k**2:9.4f}")

# Chebyshev is attained by a three-point law: +-k w.p. 1/(2k^2), 0 otherwise
k = 2.0
vals = np.array([-k, 0.0, k])
pr = np.array([1 / (2 * k**2), 1 - 1 / k**2, 1 / (2 * k**2)])
print(f"\nthree-point law: Var = {pr @ vals**2:.3f},"
      f" P(|X| >= {k}) = {pr[0] + pr[2]:.3f} = 1/k^2")
```

Output

```
Markov, X ~ Exponential(1)
    a       exact    bound 1/a
    1     0.36788      1.00000
    2     0.13534      0.50000
    4     0.01832      0.25000
    8     0.00034      0.12500

Chebyshev: P(|X - mu| >= k sd) for unit-variance laws
    k    Normal   Uniform   Laplace      t(3)    1/k^2
  1.5   0.13361   0.13397   0.11987   0.08051   0.4444
  2.0   0.04550   0.00000   0.05911   0.04052   0.2500
  3.0   0.00270   0.00000   0.01437   0.01385   0.1111
  4.0   0.00006   0.00000   0.00349   0.00617   0.0625

three-point law: Var = 1.000, P(|X| >= 2.0) = 0.250 = 1/k^2
```

Historical Note: Bienaymé, Chebyshev and Markov

The inequality now called Chebyshev's was stated by Irénée-Jules Bienaymé in 1853 in work on least squares; Pafnuty Chebyshev proved it independently, with a simple proof, in 1867, and used it to prove a general law of large numbers for independent variables with bounded variances. Bienaymé is also credited with the identity $\mathrm{Var}(\sum X_i) = \sum \mathrm{Var}(X_i)$ for uncorrelated variables. Chebyshev's

student Andrey Markov gave the first-moment inequality and extended the law of large numbers to dependent sequences, work that led him to Markov chains (Chapter 10).

6.4.2 Modes of convergence

Let $X_1, X_2, \dots$ and X be random variables on a common probability space (for convergence in distribution this is not needed).

Definition 6.21 (Modes of convergence). (i) *Almost surely*: $X_n \xrightarrow{\text{a.s.}} X$ if $\mathbb{P}(\{\omega : X_n(\omega) \to X(\omega)\}) = 1$.

(ii) *In probability*: $X_n \xrightarrow{p} X$ if $\mathbb{P}(|X_n - X| > \varepsilon) \to 0$ for every $\varepsilon > 0$.

(iii) *In mean square* (L^2): $X_n \xrightarrow{L^2} X$ if $\mathbb{E}[(X_n - X)^2] \to 0$ (more generally in L^r if $\mathbb{E}\,|X_n - X|^r \to 0$).

(iv) *In distribution*: $X_n \xrightarrow{d} X$ if $F_{X_n}(x) \to F_X(x)$ at every continuity point x of F_X.

Theorem 6.22 (Relationships).

$$X_n \xrightarrow{a.s.} X \implies X_n \xrightarrow{p} X \impliedby X_n \xrightarrow{L^2} X, \qquad X_n \xrightarrow{p} X \implies X_n \xrightarrow{d} X,$$

and if $X_n \xrightarrow{d} c$ for a constant c then $X_n \xrightarrow{p} c$. No other implication holds in general.

Proof of the main implications. $L^2 \Rightarrow$ probability: by Markov, $\mathbb{P}(|X_n - X| > \varepsilon) \le \mathbb{E}[(X_n - X)^2]/\varepsilon^2 \to 0$. A.s. $\Rightarrow$ probability: the events $B_n = \bigcup_{m \ge n} \{|X_m - X| > \varepsilon\}$ decrease to a set contained in $\{X_n \not\to X\}$, which has probability zero, so $\mathbb{P}(|X_n - X| > \varepsilon) \le \mathbb{P}(B_n) \to 0$ by continuity of probability. Probability $\Rightarrow$ distribution: for continuity points x and any $\varepsilon > 0$, $F_{X_n}(x) \le F_X(x + \varepsilon) + \mathbb{P}(|X_n - X| > \varepsilon)$ and $F_X(x - \varepsilon) \le F_{X_n}(x) + \mathbb{P}(|X_n - X| > \varepsilon)$; let $n \to \infty$, then $\varepsilon \downarrow 0$. Distribution to a constant: $\mathbb{P}(|X_n - c| > \varepsilon) \le F_{X_n}(c - \varepsilon) + 1 - F_{X_n}(c + \varepsilon/2) \to 0$. □

Counterexamples are instructive. On $\Omega = [0,1]$ with uniform $\mathbb{P}$, let $X_n = n\mathbf{1}_{[0,1/n]}$: then $X_n \to 0$ a.s. and in probability, but $\mathbb{E}[X_n^2] = n \to \infty$, so not in L^2. The "typewriter" sequence of indicators of intervals of length $1/k$ sweeping repeatedly across $[0,1]$ converges to 0 in probability and in L^2 but at no point ω almost surely. And if $X \sim \mathcal{N}(0,1)$ then $X_n = -X$ converges in distribution to X but not in probability.

△ Caution: Convergence in distribution is about laws, not values

$X_n \xrightarrow{d} X$ says that the *distribution* of X_n approaches that of X; it says nothing about X_n and X being close as random variables, and it does not imply that moments converge ($\mathbb{E}X_n \to \mathbb{E}X$ can fail without uniform integrability). The CLT is a statement of this weak kind, which is why it supports approximate confidence intervals but not statements about individual outcomes.

6.4.3 The weak and strong laws of large numbers

Theorem 6.23 (Weak law of large numbers, WLLN). *Let $X_1, X_2, \dots$ be uncorrelated with common mean μ and common variance $\sigma^2 < \infty$, and let $\bar{X}_n = n^{-1}\sum_{i=1}^n X_i$. Then $\bar{X}_n \xrightarrow{L^2} \mu$ and $\bar{X}_n \xrightarrow{p} \mu$;*

explicitly, for every $\varepsilon > 0$,

$$\mathbb{P}\big(\big|\bar{X}_n - \mu\big| \geq \varepsilon\big) \leq \frac{\sigma^2}{n\varepsilon^2} \to 0. \tag{6.13}$$

Proof. By linearity $\mathbb{E}\bar{X}_n = \mu$, and because the X_i are uncorrelated (Bienaymé), $\operatorname{Var}(\bar{X}_n) = n^{-2}\sum_i \operatorname{Var}(X_i) = \sigma^2/n$. Hence $\mathbb{E}[(\bar{X}_n - \mu)^2] = \sigma^2/n \to 0$, and Chebyshev's inequality gives (6.13). □

The proof uses only second moments and no independence beyond zero correlation. With more work the variance assumption can be dropped entirely.

Theorem 6.24 (Strong law of large numbers, Kolmogorov)**.** *If* $X_1, X_2, \ldots$ *are iid, then* $\bar{X}_n \xrightarrow{a.s.} \mu$ *if and only if* $\mathbb{E}\,|X_1| < \infty$ *and* $\mu = \mathbb{E}X_1$*. (Khinchin's weak law:* $\mathbb{E}\,|X_1| < \infty$ *also suffices for* $\bar{X}_n \xrightarrow{p} \mu$*.)*

The strong law says that, with probability one, the *whole path* of running means converges, which is what justifies interpreting probabilities as long-run frequencies and Monte Carlo averages as approximations to integrals. A proof under the extra assumption $\mathbb{E}X_1^4 < \infty$ is short (Exercise **6.4.3**); the general proof uses truncation and Kolmogorov's maximal inequality (Billingsley, 1995, Sec. 22).

Listing 6.10 draws 10^5 observations from four laws and tracks the running mean (Fig. 6.4). For the Bernoulli and exponential laws the path settles quickly. For the Pareto law with tail index 1.5 the mean (3) exists but the variance is infinite: the strong law still holds, but convergence is slow and punctuated by jumps when a huge observation arrives. For the Cauchy law there is no mean and the running mean never settles: in fact $\bar{X}_n$ has exactly the same standard Cauchy distribution for every n, as the characteristic function shows, $\phi_{\bar{X}_n}(t) = \phi(t/n)^n = e^{-|t|}$.

Listing 6.10: Running means and the failure of the LLN for Cauchy data ch06/ch06_running_means.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Running sample means: the LLN at work, and its failure for the Cauchy law.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
N = 100_000
n = np.arange(1, N + 1)
alpha = 1.5                                        # Pareto tail index
draws = {
    "Bernoulli(0.3)": (rng.random(N) < 0.3).astype(float),
    "Exponential(1)": rng.exponential(1.0, N),
    "Pareto(1.5)": rng.pareto(alpha, N) + 1.0,      # mean 3, infinite variance
    "Cauchy": rng.standard_cauchy(N),
}
means = {"Bernoulli(0.3)": 0.3, "Exponential(1)": 1.0,
         "Pareto(1.5)": alpha / (alpha - 1), "Cauchy": np.nan}

print(f"{'law':>15}{'mu':>7}{'n=100':>9}{'n=1e3':>9}{'n=1e4':>9}{'n=1e5':>9}")
run = {}
for lab, x in draws.items():
    run[lab] = np.cumsum(x) / n
    vals = "".join(f"{run[lab][m - 1]:9.3f}" for m in [100, 1000, 10000, N])
    print(f"{lab:>15}{means[lab]:7.2f}{vals}")

plt.rcParams.update({"font.size": 10})
fig, axes = plt.subplots(2, 2, figsize=(6.4, 4.4), sharex=True)
for ax, lab in zip(axes.ravel(), draws):
    ax.plot(n, run[lab], lw=1, color="navy")
```

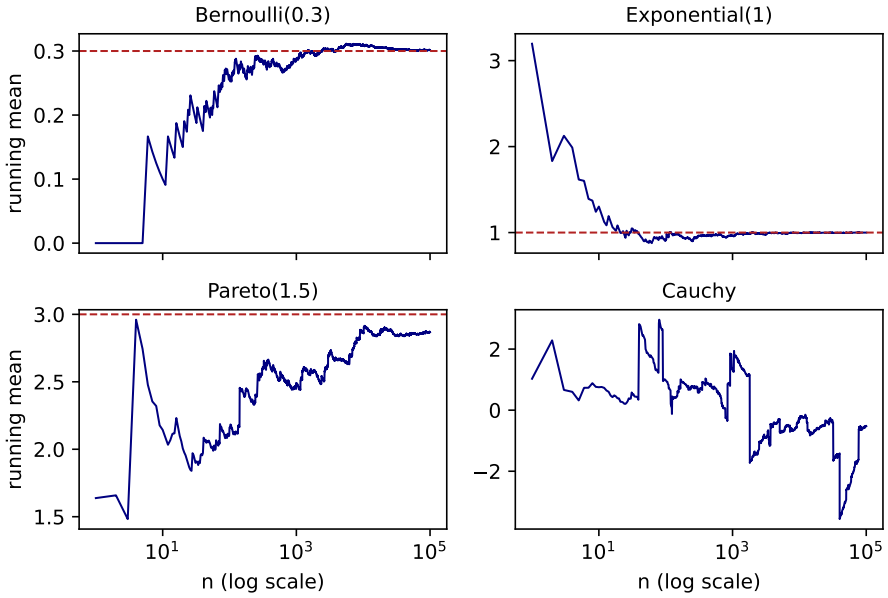


Figure 6.4: Running means $\bar{X}_n$ for four laws (log scale for n; dashed lines are the true means). Heavy tails slow convergence (Pareto, $\alpha = 1.5$); without a mean there is no convergence at all (Cauchy).

```
    if not np.isnan(means[lab]):
        ax.axhline(means[lab], color="firebrick", ls="--", lw=1)
    ax.set_xscale("log")
    ax.set_title(lab, fontsize=10)
for ax in axes[1]:
    ax.set_xlabel("n (log scale)")
for ax in axes[:, 0]:
    ax.set_ylabel("running mean")
fig.tight_layout()
plt.savefig("ch06_lln.pdf", bbox_inches="tight")
```

Output

```
           law      mu    n=100    n=1e3    n=1e4    n=1e5
Bernoulli(0.3)    0.30    0.260    0.286    0.309    0.302
Exponential(1)    1.00    0.918    0.960    0.996    0.999
   Pareto(1.5)    3.00    2.161    2.491    2.863    2.868
        Cauchy     nan    0.972    1.323   -0.248   -0.526
```

Convergence in probability, quantified. Listing 6.11 estimates $\mathbb{P}(|\bar{X}_n - \mu| > 0.1)$ for exponential data by Monte Carlo and compares it with Chebyshev's bound (6.13). The bound is valid but conservative by orders of magnitude for large n; the CLT and concentration inequalities of the next two sections explain the true rate, which is exponential rather than $1/n$.

Listing 6.11: Convergence in probability: Monte Carlo versus Chebyshev ch06/ch06_llln_prob.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Convergence in probability of the sample mean: Monte Carlo versus Chebyshev.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```

rng = np.random.default_rng(803)
reps, eps = 20_000, 0.1
mu, var = 1.0, 1.0                    # Exponential(1)
print(f"P(|Xbar_n - mu| > {eps}) for Exponential(1), {reps} replications")
print(f"{'n':>6}{'Monte Carlo':>13}{'Chebyshev':>11}{'MSE':>10}{'var/n':>9}")
for n in [10, 50, 100, 500, 1000, 2000]:
    # the sum of n Exponential(1) draws is Gamma(n, 1): sample it directly
    xbar = rng.gamma(n, 1.0, size=reps) / n
    p = np.mean(np.abs(xbar - mu) > eps)
    cheb = min(1.0, var / (n * eps**2))
    mse = np.mean((xbar - mu)**2)
    print(f"{n:6d}{p:13.4f}{cheb:11.4f}{mse:10.5f}{var / n:9.5f}")
```

Output

```
P(|Xbar_n - mu| > 0.1) for Exponential(1), 20000 replications
     n  Monte Carlo  Chebyshev       MSE    var/n
    10       0.7523     1.0000   0.09912  0.10000
    50       0.4778     1.0000   0.01998  0.02000
   100       0.3190     1.0000   0.01010  0.01000
   500       0.0248     0.2000   0.00200  0.00200
  1000       0.0013     0.1000   0.00099  0.00100
  2000       0.0001     0.0500   0.00049  0.00050
```

6.4.4 Consistency

Definition 6.25 (Consistency)**.** An estimator $\hat{\theta}_n$ of a parameter θ is *(weakly) consistent* if $\hat{\theta}_n \xrightarrow{p} \theta$ and *strongly consistent* if $\hat{\theta}_n \xrightarrow{\text{a.s.}} \theta$.

The LLN makes every sample moment consistent for its population counterpart. Combined with the *continuous mapping theorem* (if $Z_n \xrightarrow{p} c$ and g is continuous at c then $g(Z_n) \xrightarrow{p} g(c)$; Theorem 6.30), it makes smooth functions of sample moments consistent as well: the sample variance $\overline{X^2}_n - \bar{X}_n^2$, the sample standard deviation, the correlation coefficient, the OLS estimator $\hat{\boldsymbol{\beta}} = (n^{-1}\sum \mathbf{x}_i\mathbf{x}_i^\top)^{-1} n^{-1}\sum \mathbf{x}_i y_i \xrightarrow{p} \boldsymbol{\beta}$ of (6.11), and ratio estimators such as mean food shares. Note that consistency is compatible with bias in finite samples ($\mathbb{E}s < \sigma$ by Jensen's inequality, since the square root is concave).

Policy Lens: The law of large numbers behind official statistics and insurance

A national survey of a few thousand randomly selected households can estimate a mean for millions because of the LLN; the accuracy depends on the sample size n, not on the population size (apart from a finite-population correction). The same law makes pooled insurance possible: a scheme covering n independent farms has average claims per farm with variance σ^2/n, so its per-member risk shrinks with membership, while a scheme covering farms hit by the *same* drought gains little, because the covariances in $\operatorname{Var}(\bar{X}_n) = \sigma^2\{1 + (n-1)\rho\}/n$ do not average out. Covariate shocks are the reason why crop insurance needs reinsurance or public backing.

Historical Note: From Bernoulli's golden theorem to Kolmogorov

Jacob Bernoulli proved the first law of large numbers, for the relative frequency of successes in independent trials, in *Ars Conjectandi*, published posthumously in 1713; he called it his "golden theorem" and even computed how many trials suffice for a given accuracy. Poisson coined the phrase *la loi des grands nombres* in the 1830s. Émile Borel proved a strong law for Bernoulli trials in 1909, and Andrey Kolmogorov established the strong law for iid variables with a finite mean around 1930, presenting it in his 1933 *Grundbegriffe*, the book that founded probability on measure theory.

⋆ Key Concept: Laws of large numbers

Markov: $\mathbb{P}(X \geq a) \leq \mathbb{E}X/a$; Chebyshev: $\mathbb{P}(|X-\mu| \geq k) \leq \sigma^2/k^2$. Together with $\operatorname{Var}(\bar{X}_n) = \sigma^2/n$ they prove the weak law $\bar{X}_n \xrightarrow{p} \mu$. The strong law gives $\bar{X}_n \xrightarrow{\text{a.s.}} \mu$ for iid data iff $\mathbb{E}|X| < \infty$. Almost-sure and L^2 convergence each imply convergence in probability, which implies convergence in distribution. Without a finite mean (Cauchy) averaging does not help; with infinite variance (Pareto, $\alpha \leq 2$) it helps slowly.

Exercises

6.4.1 Theory Use Markov's inequality with $\varphi(x) = x^r$ to show $\mathbb{P}(|X| \geq a) \leq \mathbb{E}|X|^r / a^r$, and show by example that for each a the bound can be attained.

6.4.2 Theory (One-sided Chebyshev, Cantelli.) Show that $\mathbb{P}(X - \mu \geq k) \leq \sigma^2/(\sigma^2 + k^2)$ for $k > 0$ by applying Markov to $(X - \mu + u)^2$ and optimising over $u \geq 0$.

6.4.3 Theory Prove the strong law under $\mathbb{E}X_1^4 < \infty$: show that $\mathbb{E}[(\sum_{i=1}^n (X_i - \mu))^4] \leq Cn^2$, deduce $\sum_n \mathbb{P}(|\bar{X}_n - \mu| > \varepsilon) < \infty$, and apply the Borel–Cantelli lemma.

6.4.4 Theory Prove that $X_n = n\mathbf{1}_{[0,1/n]}$ on $[0,1]$ converges almost surely but not in L^1, and construct the typewriter sequence explicitly, showing it converges in probability but not almost surely.

6.4.5 Python Repeat Listing 6.10 for 200 independent paths of the Pareto(1.5) running mean and plot the 5%, 50% and 95% quantiles of $\bar{X}_n$ across paths against n. At what rate does the band shrink? (Compare with $n^{-1/3} = n^{1/\alpha - 1}$.)

6.4.6 ★ Graduate Let $X_1, X_2, \ldots$ be a stationary AR(1) process $X_t = \phi X_{t-1} + u_t$ with $|\phi| < 1$ and iid u_t of variance σ_u^2. Show that $\operatorname{Var}(\bar{X}_n) \approx \sigma_u^2/\{n(1-\phi)^2\}$ and that the WLLN still holds. What is the "effective sample size" of a persistent macroeconomic time series with $\phi = 0.9$?

6.4.7 Policy A crop-insurance pool has n member farms with iid losses of mean 100 and standard deviation 150, plus a common drought component inducing correlation ρ between any two farms. For $\rho \in \{0, 0.1, 0.3\}$ compute the standard deviation of the average loss per farm for $n = 10, 100, 10^4$, and explain why the pool cannot diversify away covariate risk.

6.5 The central limit theorem

The LLN says $\bar{X}_n - \mu \to 0$. The central limit theorem (CLT) says how fast and with what shape: the error is of order $\sigma/\sqrt{n}$ and approximately normal, *whatever* the distribution of the data, provided the variance is finite. This is the basis of almost every standard error, confidence interval and p-value in applied work.

6.5.1 Lindeberg–Lévy, Lyapunov and Lindeberg

Theorem 6.26 (Lindeberg–Lévy CLT)**.** *If $X_1, X_2, \dots$ are iid with mean μ and variance $0 < \sigma^2 < \infty$, then*

$$Z_n = \frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} = \frac{\sum_{i=1}^n (X_i - \mu)}{\sigma\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1). \tag{6.14}$$

Proof sketch via characteristic functions. Let $Y_i = (X_i - \mu)/\sigma$, with $\mathbb{E}Y_i = 0$, $\mathbb{E}Y_i^2 = 1$ and CF ϕ. Because $\mathbb{E}Y_i^2 < \infty$, ϕ is twice differentiable with $\phi(s) = 1 - s^2/2 + o(s^2)$ as $s \to 0$. By independence,

$$\phi_{Z_n}(t) = \phi\big(t/\sqrt{n}\big)^n = \Big(1 - \frac{t^2}{2n} + o\big(\tfrac{1}{n}\big)\Big)^n \longrightarrow e^{-t^2/2},$$

the CF of $\mathcal{N}(0,1)$. Lévy's continuity theorem (Theorem 6.9(d)) completes the proof. □

The theorem extends to independent but non-identically distributed summands, which is what regression with heterogeneous units requires. Let X_i be independent with means μ_i, variances σ_i^2, and $s_n^2 = \sum_{i=1}^n \sigma_i^2$.

Theorem 6.27 (Lindeberg and Lyapunov CLTs)**.** *If the* Lindeberg condition

$$\frac{1}{s_n^2} \sum_{i=1}^n \mathbb{E}\big[(X_i - \mu_i)^2 \mathbf{1}\{|X_i - \mu_i| > \varepsilon s_n\}\big] \to 0 \quad \text{for every } \varepsilon > 0$$

holds, then $\sum_{i=1}^n (X_i - \mu_i)/s_n \xrightarrow{d} \mathcal{N}(0,1)$. A sufficient condition is Lyapunov's condition*: for some $\delta > 0$, $s_n^{-(2+\delta)} \sum_{i=1}^n \mathbb{E}\,|X_i - \mu_i|^{2+\delta} \to 0$.*

Lindeberg's condition says that no single summand contributes a non-negligible share of the total variance. It fails, for example, when one huge firm dominates an industry aggregate or one region dominates a national total; then the aggregate inherits the (possibly non-normal) distribution of the dominant unit. Feller showed that the condition is also necessary when $\max_i \sigma_i^2/s_n^2 \to 0$.

6.5.2 How good is the normal approximation? Berry–Esseen

Theorem 6.28 (Berry–Esseen)**.** *If the X_i are iid with $\rho = \mathbb{E}\,|X_1 - \mu|^3 < \infty$, then*

$$\sup_x \big|\mathbb{P}(Z_n \le x) - \Phi(x)\big| \le \frac{C\rho}{\sigma^3\sqrt{n}},$$

with an absolute constant C; the best known value is $C \le 0.4748$ (Shevtsova, 2011).

The bound decays like $n^{-1/2}$ and grows with the standardised third absolute moment, a measure of skewness and tail weight. Listing 6.12 simulates standardised means of exponential data (skewness 2) and of Bernoulli(0.1) data, computes the Kolmogorov–Smirnov distance to $\mathcal{N}(0,1)$, and compares it with the Berry–Esseen bound.

Listing 6.12: Sampling distribution of standardised means ch06/ch06_clt_sampling.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Sampling distribution of standardised means; KS distance and Berry-Esseen bound.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```

rng = np.random.default_rng(803)
reps = 50_000
C_BE = 0.4748                          # Shevtsova (2011) constant, iid case


def ks_to_normal(z):
    return stats.kstest(z, "norm").statistic


# Exponential(1): mu = sd = 1, rho = E|X - mu|^3 = 12/e - 2
rho_exp = 12 / np.e - 2
# Bernoulli(0.1): rho = p q (p^2 + q^2), sd = sqrt(p q)
p = 0.1
q = 1 - p
rho_ber, sd_ber = p * q * (p**2 + q**2), np.sqrt(p * q)
print(f"{'n':>5}{'KS exp':>9}{'BE exp':>9}{'KS bern':>9}{'BE bern':>9}"
      f"{'skew exp':>10}")
Z = {}
for n in [5, 30, 300]:
    xbar = rng.gamma(n, 1.0, size=reps) / n                  # Exponential mean
    z = np.sqrt(n) * (xbar - 1.0)
    Z[n] = z
    zb = (rng.binomial(n, p, size=reps) / n - p) / (sd_ber / np.sqrt(n))
    be_exp = C_BE * rho_exp / np.sqrt(n)
    be_ber = C_BE * rho_ber / (sd_ber**3 * np.sqrt(n))
    print(f"{n:5d}{ks_to_normal(z):9.4f}{be_exp:9.4f}{ks_to_normal(zb):9.4f}"
          f"{be_ber:9.4f}{stats.skew(z):10.3f}")
print("(skewness of the mean is 2/sqrt(n): "
      + ", ".join(f"{2 / np.sqrt(n):.3f}" for n in [5, 30, 300]) + ")")

plt.rcParams.update({"font.size": 10})
fig, axes = plt.subplots(1, 3, figsize=(6.6, 2.6), sharey=True)
t = np.linspace(-4, 4, 300)
for ax, n in zip(axes, Z):
    ax.hist(Z[n], bins=80, range=(-4, 4), density=True, color="lightsteelblue")
    ax.plot(t, stats.norm.pdf(t), color="firebrick", lw=1.5)
    ax.set_title(f"n = {n}", fontsize=10)
    ax.set_xlabel("sqrt(n)(Xbar - mu)/sd")
axes[0].set_ylabel("density")
fig.tight_layout()
plt.savefig("ch06_clt.pdf", bbox_inches="tight")
```

Output

```
    n   KS exp   BE exp  KS bern  BE bern  skew exp
    5   0.0616   0.5127   0.3618   0.5804     0.897
   30   0.0246   0.2093   0.1458   0.2369     0.350
  300   0.0092   0.0662   0.0484   0.0749     0.123
(skewness of the mean is 2/sqrt(n): 0.894, 0.365, 0.115)
```

The bound is respected and is conservative by a factor of seven to eight for the exponential case. For the Bernoulli(0.1) case the distance stays large longer: the distribution is discrete (a lattice), which alone forces a distance of order $1/\sqrt{npq}$, and it is skewed. A popular rule of thumb ("$n \geq 30$ is enough") is clearly not a theorem.

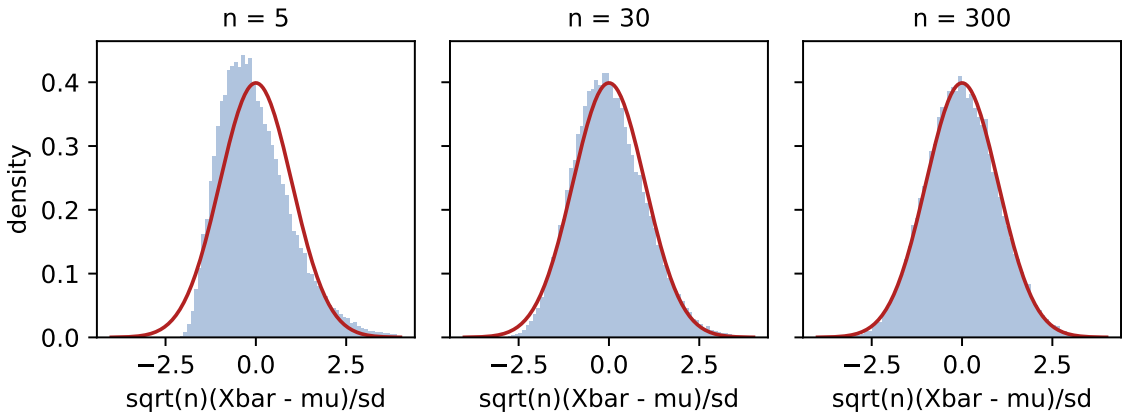


Figure 6.5: Standardised sample means of Exponential(1) data for $n = 5, 30, 300$ with the $\mathcal{N}(0,1)$ density. The right skew (skewness $2/\sqrt{n}$) fades slowly.

Historical Note: De Moivre, Laplace, Lyapunov, Lindeberg and Pólya

Abraham de Moivre derived the normal approximation to the binomial distribution in a short Latin paper of 1733 (*Approximatio ad summam terminorum binomii* $(a+b)^n$ *in seriem expansi*), later included in the second edition of his *Doctrine of Chances* (1738). Pierre-Simon Laplace extended it to sums of general independent errors in a memoir read in 1810 and in the *Théorie analytique des probabilités* (1812). The first rigorous proof under general conditions was Aleksandr Lyapunov's (1901), using third moments; Jarl Waldemar Lindeberg gave his condition in 1922, and William Feller (1935) showed its necessity. The name "central limit theorem" (*zentraler Grenzwertsatz*) is due to George Pólya (1920), who meant that the theorem is of central importance in probability. Andrew Berry (1941) and Carl-Gustav Esseen (1942) independently obtained the rate bound.

6.5.3 Multivariate CLT, Slutsky's theorem and continuous mapping

Theorem 6.29 (Multivariate CLT)**.** *If* $\mathbf{X}_1, \mathbf{X}_2, \dots \in \mathbb{R}^p$ *are iid with mean* $\boldsymbol{\mu}$ *and covariance* $\boldsymbol{\Sigma}$ *(finite), then* $\sqrt{n}(\bar{\mathbf{X}}_n - \boldsymbol{\mu}) \xrightarrow{d} \mathcal{N}_p(\mathbf{0}, \boldsymbol{\Sigma})$.

The proof reduces to the univariate case by the *Cramér–Wold device*: a random vector $\mathbf{Y}_n \xrightarrow{d} \mathbf{Y}$ iff $\mathbf{w}^\top \mathbf{Y}_n \xrightarrow{d} \mathbf{w}^\top \mathbf{Y}$ for every fixed $\mathbf{w}$, and $\sqrt{n}\,\mathbf{w}^\top(\bar{\mathbf{X}}_n - \boldsymbol{\mu}) \xrightarrow{d} \mathcal{N}(0, \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w})$ by Theorem 6.26.

Theorem 6.30 (Continuous mapping and Slutsky)**.** *(a) (Continuous mapping) If* $\mathbf{Y}_n \to \mathbf{Y}$ *almost surely, in probability or in distribution, and* g *is continuous (on a set of probability one under* $\mathbf{Y}$*), then* $g(\mathbf{Y}_n) \to g(\mathbf{Y})$ *in the same mode.*

(b) (Slutsky) If $Y_n \xrightarrow{d} Y$ *and* $W_n \xrightarrow{p} c$ *for a constant* c*, then* $Y_n + W_n \xrightarrow{d} Y + c$*,* $W_n Y_n \xrightarrow{d} cY$ *and, if* $c \neq 0$*,* $Y_n / W_n \xrightarrow{d} Y/c$.

The most important consequence is that we may replace σ by a consistent estimate: since $s_n \xrightarrow{p} \sigma$ (Section 6.4), Slutsky gives

$$T_n = \frac{\sqrt{n}(\bar{X}_n - \mu)}{s_n} = \frac{\sigma}{s_n} Z_n \xrightarrow{d} \mathcal{N}(0,1), \tag{6.15}$$

which justifies the large-sample interval $\bar{X}_n \pm 1.96\, s_n/\sqrt{n}$ for any law with finite variance.

6.5.4 The delta method

Policy quantities are rarely plain means. Food shares are ratios of means, elasticities are ratios of log changes, poverty and inequality measures are nonlinear functionals. The delta method transfers the CLT from the sample moments to smooth functions of them.

Theorem 6.31 (Delta method). *Suppose* $\sqrt{n}(\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}) \xrightarrow{d} \mathcal{N}_p(\mathbf{0}, \boldsymbol{\Sigma})$ *and* $g : \mathbb{R}^p \to \mathbb{R}$ *is differentiable at* $\boldsymbol{\theta}$ *with gradient* $\nabla g(\boldsymbol{\theta}) \neq \mathbf{0}$. *Then*

$$\sqrt{n}\big(g(\hat{\boldsymbol{\theta}}_n) - g(\boldsymbol{\theta})\big) \xrightarrow{d} \mathcal{N}\big(0,\ \nabla g(\boldsymbol{\theta})^\top \boldsymbol{\Sigma}\, \nabla g(\boldsymbol{\theta})\big). \tag{6.16}$$

For vector-valued g *the variance is* $\mathbf{G}\boldsymbol{\Sigma}\mathbf{G}^\top$ *with* $\mathbf{G}$ *the Jacobian.*

Proof sketch. Differentiability gives $g(\hat{\boldsymbol{\theta}}_n) - g(\boldsymbol{\theta}) = \nabla g(\boldsymbol{\theta})^\top(\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}) + r_n$ with $r_n = o(\|\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}\|)$. Multiply by $\sqrt{n}$. The leading term converges to $\nabla g(\boldsymbol{\theta})^\top \mathbf{Z}$ with $\mathbf{Z} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$, by the continuous mapping theorem, and its variance is $\nabla g^\top \boldsymbol{\Sigma} \nabla g$ by (6.6). The remainder satisfies $\sqrt{n}\, r_n = o(1) \cdot \sqrt{n}\, \|\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}\|$, the product of a term tending to zero in probability and a term that is bounded in probability (it converges in distribution), so $\sqrt{n}\, r_n \xrightarrow{p} 0$ and Slutsky's theorem finishes the argument. □

Example 6.32 (Ratio of means). Let $\hat{\boldsymbol{\theta}} = (\bar{X}, \bar{Y})$ estimate (μ_X, μ_Y) and $R = \mu_Y/\mu_X$ (e.g. the aggregate food share: total food spending over total spending). With $g(a, b) = b/a$, $\nabla g = (-\mu_Y/\mu_X^2,\ 1/\mu_X)$ and

$$\operatorname{Var}(\hat{R}) \approx \frac{1}{n\mu_X^2}\big(\sigma_Y^2 - 2R\,\sigma_{XY} + R^2\sigma_X^2\big) = \frac{\operatorname{Var}(Y - RX)}{n\mu_X^2}.$$

The classical ratio-estimator variance of survey sampling is exactly this formula.

Example 6.33 (Arc elasticity). With mean quantities $\bar{q}_0, \bar{q}_1$ from two independent survey rounds at known prices p_0, p_1, the arc price elasticity is $\hat{\eta} = \log(\bar{q}_1/\bar{q}_0)/\log(p_1/p_0)$. Since $d\log q = dq/q$, the delta method gives $\operatorname{se}(\hat{\eta}) \approx \{\sigma_1^2/(n_1 q_1^2) + \sigma_0^2/(n_0 q_0^2)\}^{1/2} / \big|\log(p_1/p_0)\big|$: the coefficients of variation of the two means, scaled by the size of the price change.

Listing 6.13: Delta-method standard errors for a ratio and an elasticity ch06/ch06_delta_method.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Delta-method standard errors for a ratio of means and an arc elasticity.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
reps, n = 5000, 400


def survey(size):
    """Simulated households: total expenditure X and food expenditure Y."""
    x = np.exp(rng.normal(6.5, 0.6, size))
    y = 0.15 * x + 3 * np.sqrt(x) * np.exp(rng.normal(0, 0.3, size))
    return x, y


# 1. Food share R = E[Y]/E[X]; gradient of g(a, b) = b/a is (-b/a^2, 1/a)
R_hat, se_hat = np.empty(reps), np.empty(reps)
for r in range(reps):
    x, y = survey(n)
```

```
    mx, my = x.mean(), y.mean()
    S = np.cov(x, y)
    grad = np.array([-my / mx**2, 1 / mx])
    R_hat[r] = my / mx
    se_hat[r] = np.sqrt(grad @ S @ grad / n)
xb, yb = survey(2_000_000)
R0 = yb.mean() / xb.mean()
cover = np.mean(np.abs(R_hat - R0) <= 1.96 * se_hat)
print("Ratio of means (aggregate food share), n = 400")
print(f"  true R = {R0:.4f}, mean of R_hat = {R_hat.mean():.4f}")
print(f"  Monte Carlo sd of R_hat = {R_hat.std():.5f}")
print(f"  mean delta-method se    = {se_hat.mean():.5f}")
print(f"  coverage of R_hat +- 1.96 se = {cover:.3f}")

# 2. Arc price elasticity from two independent survey rounds
p0, p1 = 10.0, 12.0                          # prices (known)
q0_mu, q1_mu, sd_q = 50.0, 46.0, 15.0        # mean quantities, sd
eta0 = np.log(q1_mu / q0_mu) / np.log(p1 / p0)
q0 = rng.normal(q0_mu, sd_q, size=(reps, n)).mean(axis=1)
q1 = rng.normal(q1_mu, sd_q, size=(reps, n)).mean(axis=1)
eta = np.log(q1 / q0) / np.log(p1 / p0)
se_delta = np.sqrt(sd_q**2 / (n * q1_mu**2) + sd_q**2 / (n * q0_mu**2))
se_delta /= np.log(p1 / p0)
print("\nArc price elasticity  eta = log(q1/q0)/log(p1/p0)")
print(f"  true eta = {eta0:.4f}, mean of eta_hat = {eta.mean():.4f}")
print(f"  Monte Carlo sd = {eta.std():.4f}, delta-method se = {se_delta:.4f}")
print(f"  95% interval: {eta0:.3f} +- {1.96 * se_delta:.3f}")
```

Output

```
Ratio of means (aggregate food share), n = 400
  true R = 0.2563, mean of R_hat = 0.2564
  Monte Carlo sd of R_hat = 0.00261
  mean delta-method se    = 0.00257
  coverage of R_hat +- 1.96 se = 0.946

Arc price elasticity  eta = log(q1/q0)/log(p1/p0)
  true eta = -0.4573, mean of eta_hat = -0.4574
  Monte Carlo sd = 0.1218, delta-method se = 0.1215
  95% interval: -0.457 +- 0.238
```

In both cases the delta-method standard error matches the Monte Carlo standard deviation to within a few percent, and the nominal 95% interval for the ratio covers the truth in 94.6% of replications. The second example carries a lesson of its own: a 20% price change and samples of 400 give an elasticity of -0.46 ± 0.24, too imprecise to distinguish "inelastic" from "very inelastic".

Policy Lens: Standard errors for elasticities and poverty measures

Tax and subsidy reform analysis relies on demand elasticities, and the welfare cost of a tax is roughly proportional to the elasticity; an interval of -0.46 ± 0.24 implies a factor of three in the estimated deadweight loss. The delta method gives such intervals at negligible computational cost. The same tool yields standard errors for poverty measures with an estimated poverty line (e.g. 60% of median consumption), for Gini coefficients and for ratios of regional means; survey software implements it under the name *linearisation*, combining the gradient with the design-based covariance of the underlying totals. The bootstrap (Chapter 11) is the main alternative when g is not smooth.

6.5.5 Heavy tails slow convergence

The CLT requires a finite variance; Berry–Esseen requires a finite third moment to give a rate. Income and wealth distributions often have Pareto upper tails with tail index α between about 1.5 and 3 (Chapter 5). For $\alpha \le 2$ the variance is infinite and the classical CLT fails; for $2 < \alpha < 3$ it holds but without the $n^{-1/2}$ rate. Listing 6.14 measures the actual coverage of the textbook interval $\bar{x} \pm 1.96 s/\sqrt{n}$ for Pareto incomes.

Listing 6.14: Coverage of CLT intervals for Pareto incomes ch06/ch06_pareto_clt.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Heavy tails slow the CLT: coverage of mean intervals for Pareto incomes.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
reps = 2000
sizes = [50, 200, 1000, 5000]
print("Coverage of xbar +- 1.96 s/sqrt(n) for Pareto(alpha) incomes, x_m = 1")
print(f"{'alpha':>6}{'mean':>7}" + "".join(f"{'n=' + str(n):>9}" for n in sizes))
Zs = {}
for a in [1.5, 2.5, 4.0, 8.0]:
    mu = a / (a - 1)
    row = ""
    for n in sizes:
        x = rng.pareto(a, size=(reps, n)) + 1.0
        xbar, s = x.mean(axis=1), x.std(axis=1, ddof=1)
        z = (xbar - mu) / (s / np.sqrt(n))
        row += f"{np.mean(np.abs(z) <= 1.96):9.3f}"
        if n == 1000:
            Zs[a] = z
    print(f"{a:6.1f}{mu:7.3f}{row}")
print("(alpha = 1.5: infinite variance, the classical CLT does not apply)")

plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
t = np.linspace(-6, 4, 400)
for a, col in [(1.5, "firebrick"), (2.5, "darkorange"), (8.0, "navy")]:
    ax.hist(Zs[a], bins=100, range=(-6, 4), density=True, histtype="step",
            lw=1.4, color=col, label=f"alpha = {a}")
ax.plot(t, stats.norm.pdf(t), "k--", lw=1.2, label="N(0,1)")
ax.set_xlabel("studentised mean, n = 1000")
ax.set_ylabel("density")
ax.legend(frameon=False)
plt.savefig("ch06_pareto.pdf", bbox_inches="tight")
```

Output

```
Coverage of xbar +- 1.96 s/sqrt(n) for Pareto(alpha) incomes, x_m = 1
 alpha   mean     n=50    n=200   n=1000   n=5000
   1.5  3.000    0.680    0.726    0.767    0.788
   2.5  1.667    0.850    0.897    0.936    0.933
   4.0  1.333    0.887    0.940    0.943    0.953
   8.0  1.143    0.918    0.947    0.959    0.952
(alpha = 1.5: infinite variance, the classical CLT does not apply)
```

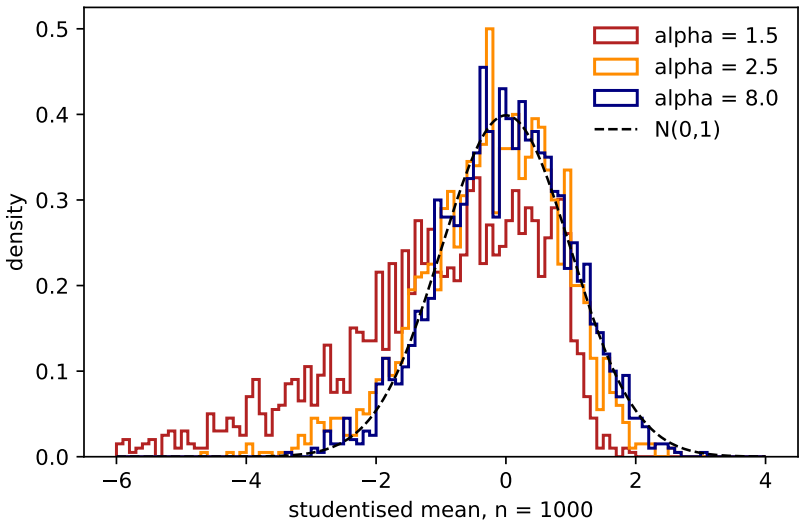


Figure 6.6: Studentised means of Pareto samples ($n = 1000$). For $\alpha = 8$ the normal approximation is good; for $\alpha = 2.5$ the distribution is skewed to the left (a large sample mean usually comes with a large s); for $\alpha = 1.5$ it is far from normal.

△ Caution: "Large" n depends on the tails

With $\alpha = 2.5$, an interval for mean income from 50 households covers the truth only 85% of the time, and with $\alpha = 1.5$ even 5000 households give 79% coverage. Under-coverage is caused by samples that happen to miss the rare very rich households: their mean and their standard deviation are both too small. Remedies include stratifying and oversampling the top of the distribution, combining surveys with tax data, modelling the upper tail parametrically, or targeting more robust parameters (the median, trimmed means, quantile shares).

Graduate Perspective: Beyond the classical CLT

If $\mathbb{P}(|X| > x) \sim cx^{-\alpha}$ with $0 < \alpha < 2$, suitably centred sums normalised by $n^{1/\alpha}$ (not $\sqrt{n}$) converge to a non-normal *α-stable* law: the generalised CLT of Lévy, Gnedenko and Kolmogorov. For dependent data, CLTs for martingale differences and for mixing processes underpin time-series econometrics and require heteroskedasticity- and autocorrelation-consistent (HAC) variances. Empirical-process theory (Donsker's theorem) gives CLTs for whole distribution functions, the basis of Kolmogorov–Smirnov tests and of the bootstrap (van der Vaart, 1998).

⋆ Key Concept: Central limit theorem and delta method

For iid data with finite variance, $\sqrt{n}(\bar{X}_n - \mu)/\sigma \xrightarrow{d} \mathcal{N}(0,1)$; Lindeberg's condition extends this to heterogeneous independent summands, Berry–Esseen bounds the error by $C\rho/(\sigma^3\sqrt{n})$, and the multivariate CLT follows by Cramér–Wold. Slutsky lets us plug in consistent estimates, so $\bar{X} \pm 1.96 s/\sqrt{n}$ is asymptotically valid. The delta method gives $\operatorname{Var} g(\hat{\boldsymbol{\theta}}) \approx \nabla g^{\top} \boldsymbol{\Sigma} \nabla g / n$. Heavy tails make "large n" very large.

Exercises

6.5.1 Theory Verify Lyapunov's condition with $\delta = 1$ for independent Bernoulli(p_i) variables with $p_i \in [a, 1-a]$, $0 < a < 1/2$. Show it fails if $p_i = 1/i^2$.

6.5.2 Theory Prove the delta method for $g(\theta) = \theta^2$ when $\theta = 0$. What goes wrong, and what is the limit distribution of $n\bar{X}_n^2$ (the "second-order" delta method)?

6.5.3 Theory Use the delta method to show that for a Bernoulli(p) sample $\sqrt{n}\,(2\arcsin\sqrt{\hat{p}} - 2\arcsin\sqrt{p}) \xrightarrow{d} \mathcal{N}(0,1)$: the arcsine transformation is *variance-stabilising*.

6.5.4 Python Extend Listing 6.12 to the lognormal with $s = 1.5$ and plot the KS distance against n on log–log axes. Estimate the slope. Is it $-1/2$?

6.5.5 Python Compute the delta-method standard error of the coefficient of variation $\hat{\sigma}/\hat{\mu}$ of exponential data, and check its accuracy by simulation for $n = 50, 500$.

6.5.6 ★ Graduate (Poverty gap with an estimated line.) Let the poverty line be $z = 0.6\,\xi$ where ξ is the population median of income, and let $P_1 = \mathbb{E}[\max(z - Y, 0)/z]$. Derive the influence function of $\hat{P}_1$ (which must include a term from estimating the median) and use it to obtain a standard error. Check it by simulation with lognormal incomes.

6.5.7 Policy A statistics office publishes the mean income of the top decile from a survey of 8000 households. Using a Pareto upper tail with $\alpha = 2.2$, simulate the coverage of the usual 95% interval for the top-decile mean. Recommend a sampling or estimation strategy and justify it with your simulation.

6.6 Concentration inequalities

The CLT is an asymptotic statement: it says the normal approximation becomes accurate *eventually*, but not for which n, and it says little about far tails or about many estimates at once. Concentration inequalities give explicit, finite-sample bounds of the form $\mathbb{P}(|\bar{X}_n - \mu| \ge \varepsilon) \le 2e^{-cn\varepsilon^2}$. They are the working tool of statistical learning theory, randomised algorithms and high-dimensional statistics (Vershynin, 2018; Boucheron et al., 2013).

6.6.1 The Chernoff method

Markov's inequality applied to $e^{\lambda S}$ for $\lambda > 0$ gives, for any random variable S with MGF M_S,

$$\mathbb{P}(S \ge t) \le \inf_{\lambda > 0} e^{-\lambda t} M_S(\lambda) = \exp\Big\{ -\sup_{\lambda > 0} \big(\lambda t - K_S(\lambda)\big) \Big\}, \tag{6.17}$$

where $K_S = \log M_S$ is the cumulant generating function. If $S = \sum_{i=1}^n X_i$ with independent X_i, then $K_S = \sum_i K_{X_i}$: independence turns the bound for a sum into a product of individual bounds. That is the whole trick, and it explains why exponential bounds are available exactly when MGFs are.

Theorem 6.34 (Chernoff bound for Bernoulli means). *If $X_1, \dots, X_n \overset{iid}{\sim}$ Bernoulli(p) and $p < a < 1$, then*

$$\mathbb{P}(\bar{X}_n \ge a) \le \exp\{-n\,\mathrm{KL}(a \,\|\, p)\}, \qquad \mathrm{KL}(a \,\|\, p) = a \log \frac{a}{p} + (1-a)\log\frac{1-a}{1-p}.$$

Proof. Here $K_X(\lambda) = \log(1 - p + pe^{\lambda})$. Minimising $-\lambda n a + n K_X(\lambda)$ over λ gives $e^{\lambda^*} = a(1-p)/\{p(1-a)\}$, and substitution into (6.17) yields $-n\,\mathrm{KL}(a\|p)$. □

The exponent $\mathrm{KL}(a\|p)$, the Kullback–Leibler divergence, is in fact the exact exponential rate (Cramér's large-deviation theorem), so the Chernoff bound is tight on the log scale.

6.6.2 Hoeffding's inequality and sub-Gaussian variables

Lemma 6.35 (Hoeffding's lemma)**.** *If $a \le X \le b$ almost surely, then $\mathbb{E}\big[e^{\lambda(X-\mathbb{E}X)}\big] \le \exp\{\lambda^2(b-a)^2/8\}$ for all $\lambda \in \mathbb{R}$.*

Proof sketch. Let $\psi(\lambda) = \log \mathbb{E}e^{\lambda(X-\mathbb{E}X)}$. Then $\psi(0) = \psi'(0) = 0$, and $\psi''(\lambda)$ is the variance of X under the exponentially tilted law with density proportional to $e^{\lambda x}$. Any variable supported on $[a,b]$ has variance at most $(b-a)^2/4$, so $\psi'' \le (b-a)^2/4$, and Taylor's theorem gives $\psi(\lambda) \le \lambda^2(b-a)^2/8$. □

Theorem 6.36 (Hoeffding's inequality)**.** *Let $X_1, \dots, X_n$ be independent with $a_i \le X_i \le b_i$, and $\mu = \mathbb{E}\bar{X}_n$. For $\varepsilon > 0$,*

$$\mathbb{P}(\bar{X}_n - \mu \ge \varepsilon) \le \exp\Big\{-\frac{2n^2\varepsilon^2}{\sum_i (b_i-a_i)^2}\Big\}, \qquad \mathbb{P}\big(\big|\bar{X}_n - \mu\big| \ge \varepsilon\big) \le 2e^{-2n\varepsilon^2} \ \text{ if } X_i \in [0,1]. \tag{6.18}$$

Proof. By (6.17), independence and Lemma 6.35, $\mathbb{P}(\sum_i (X_i - \mathbb{E}X_i) \ge n\varepsilon) \le \exp\{-\lambda n\varepsilon + \lambda^2 \sum_i (b_i-a_i)^2/8\}$. Minimise over λ: $\lambda^* = 4n\varepsilon/\sum_i (b_i-a_i)^2$. The two-sided bound follows by applying the result to $-X_i$ and adding (a union bound). □

Hoeffding's inequality needs no knowledge of the variance, only of the range; it is therefore conservative when the variance is small (e.g. rare events, $p = 0.1$). Bernstein's inequality repairs this by using the variance as well: $\mathbb{P}(\bar{X}_n - \mu \ge \varepsilon) \le \exp\{-n\varepsilon^2/(2\sigma^2 + 2M\varepsilon/3)\}$ for $|X_i - \mathbb{E}X_i| \le M$.

Definition 6.37 (Sub-Gaussian random variable)**.** A centred random variable X is *sub-Gaussian with variance proxy σ^2* if $\mathbb{E}e^{\lambda X} \le e^{\lambda^2\sigma^2/2}$ for all $\lambda \in \mathbb{R}$. Equivalently (up to constants), $\mathbb{P}(|X| \ge t) \le 2e^{-t^2/(2\sigma^2)}$, or $\|X\|_{\psi_2} = \inf\{K > 0 : \mathbb{E}e^{X^2/K^2} \le 2\} < \infty$.

Gaussian variables (proxy σ^2) and bounded variables (proxy $(b-a)^2/4$, by Hoeffding's lemma) are sub-Gaussian; exponential, Poisson and lognormal variables are not (their tails are heavier). Sums of independent sub-Gaussians are sub-Gaussian with proxies adding, which gives the general Hoeffding bound $\mathbb{P}\big(\big|\bar{X}_n - \mu\big| \ge \varepsilon\big) \le 2\exp\{-n\varepsilon^2/(2\sigma^2)\}$ (Vershynin, 2018, Ch. 2).

6.6.3 Concentration bounds versus the CLT

Listing 6.15 compares, for Bernoulli means, the exact binomial tail with the CLT approximation and with the Chebyshev, Hoeffding and Chernoff bounds.

Listing 6.15: Exact tails, CLT, Chebyshev, Hoeffding and Chernoff ch06/ch06_concentration_bounds.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Tail of a Bernoulli mean: exact, CLT, Chebyshev, Hoeffding and Chernoff (KL).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats


def kl(a, p):
    """Kullback-Leibler divergence between Bernoulli(a) and Bernoulli(p)."""
    return a * np.log(a / p) + (1 - a) * np.log((1 - a) / (1 - p))

```

```
13
14 eps = 0.05
15 ns = np.arange(20, 2001, 10)
16 print(f"Upper tail P(Xbar - p >= {eps}) for Bernoulli(p) means")
17 for p in [0.5, 0.1]:
18     print(f"\np = {p}")
19     print(f"{'n':>6}{'exact':>11}{'CLT':>11}{'Chebyshev':>11}{'Hoeffding':>11}"
20           f"{'Chernoff':>11}")
21     for n in [100, 400, 1000, 2000]:
22         k = int(np.ceil(n * (p + eps) - 1e-9))
23         exact = stats.binom.sf(k - 1, n, p)
24         clt = stats.norm.sf(eps * np.sqrt(n / (p * (1 - p))))
25         cheb = min(1, p * (1 - p) / (n * eps**2))     # two-sided bound
26         hoef = np.exp(-2 * n * eps**2)
27         cher = np.exp(-n * kl(p + eps, p))
28         print(f"{n:6d}{exact:11.2e}{clt:11.2e}{cheb:11.2e}{hoef:11.2e}"
29               f"{cher:11.2e}")
30
31 p = 0.1
32 k = np.ceil(ns * (p + eps) - 1e-9)
33 plt.rcParams.update({"font.size": 10})
34 fig, ax = plt.subplots(figsize=(6, 3.8))
35 ax.semilogy(ns, stats.binom.sf(k - 1, ns, p), "k", lw=2, label="exact")
36 ax.semilogy(ns, stats.norm.sf(eps * np.sqrt(ns / (p * (1 - p)))), "--",
37             label="CLT approximation")
38 ax.semilogy(ns, np.minimum(1, p * (1 - p) / (ns * eps**2)), ":",
39             label="Chebyshev")
40 ax.semilogy(ns, np.exp(-2 * ns * eps**2), "-.", label="Hoeffding")
41 ax.semilogy(ns, np.exp(-ns * kl(p + eps, p)), lw=1.5, label="Chernoff (KL)")
42 ax.set_xlabel("sample size n")
43 ax.set_ylabel("P(Xbar - p >= 0.05)")
44 ax.set_title("Bernoulli(0.1) mean: bounds versus exact tail")
45 ax.set_ylim(1e-12, 2)
46 ax.legend(frameon=False, fontsize=9)
47 plt.savefig("ch06_concentration.pdf", bbox_inches="tight")
```

Output

```
Upper tail P(Xbar - p >= 0.05) for Bernoulli(p) means

p = 0.5
     n      exact        CLT  Chebyshev  Hoeffding   Chernoff
   100   1.84e-01   1.59e-01   1.00e+00   6.07e-01   6.06e-01
   400   2.55e-02   2.28e-02   2.50e-01   1.35e-01   1.35e-01
  1000   8.65e-04   7.83e-04   1.00e-01   6.74e-03   6.68e-03
  2000   4.23e-06   3.87e-06   5.00e-02   4.54e-05   4.46e-05

p = 0.1
     n      exact        CLT  Chebyshev  Hoeffding   Chernoff
   100   7.26e-02   4.78e-02   3.60e-01   6.07e-01   2.94e-01
   400   1.05e-03   4.29e-04   9.00e-02   1.35e-01   7.49e-03
  1000   4.49e-07   6.80e-08   3.60e-02   6.74e-03   4.86e-06
  2000   1.57e-12   4.54e-14   1.80e-02   4.54e-05   2.36e-11
```

Three lessons emerge. (i) The CLT approximation is the most accurate for moderate tail probabilities, but it is not a bound: for $p = 0.1$ it *underestimates* the exact tail, by a factor of 35 at $n = 2000$, because the skewed binomial has a heavier right tail than the normal. (ii) Chebyshev's $1/n$ rate is hopeless for small probabilities. (iii) For $p = 0.5$ Hoeffding and Chernoff coincide and are within a factor of ten of the truth; for $p = 0.1$ Chernoff, which adapts to the variance through

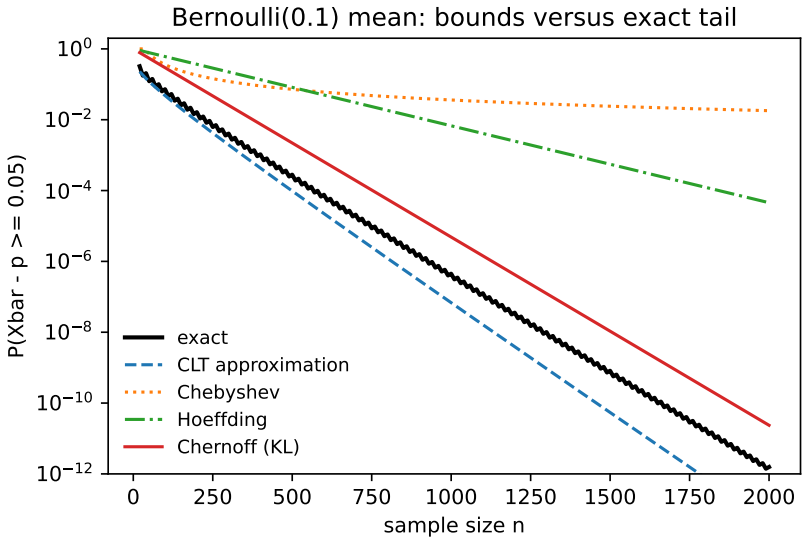


Figure 6.7: Upper tail $\mathbb{P}(\bar{X}_n - p \geq 0.05)$ for Bernoulli(0.1) means. Chebyshev decays like $1/n$; Hoeffding and Chernoff decay exponentially. The CLT approximation is close to the exact tail for moderate probabilities but is not a bound, and underestimates the far tail.

the KL divergence, is far sharper than Hoeffding.

6.6.4 Survey sample size determination

To estimate a proportion p (a poverty rate, an immunisation rate, a vote share) within $\pm\varepsilon$ with probability at least $1-\delta$ from a simple random sample, we can invert any of the bounds:

$$n_{\text{CLT}} = \frac{z_{1-\delta/2}^2\, p(1-p)}{\varepsilon^2}, \qquad n_{\text{Cheb}} = \frac{p(1-p)}{\delta\varepsilon^2}, \qquad n_{\text{Hoeff}} = \frac{\log(2/\delta)}{2\varepsilon^2}. \tag{6.19}$$

With the worst case $p = 1/2$, $\varepsilon = 0.03$ and $\delta = 0.05$, these are 1068, 5556 and 2050. The CLT formula is the one used in practice, with $z_{0.975} = 1.96$; it is approximate but accurate for proportions not too close to 0 or 1. Hoeffding's guarantee is *exact* for every n, at the price of about twice the sample. The dependence on the confidence level is the key difference: Hoeffding's n grows only like $\log(1/\delta)$, Chebyshev's like $1/\delta$.

Listing 6.16: Sample sizes for a proportion and for ten regions at once `ch06/ch06_sample_size.py`

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Survey sample sizes for a proportion: CLT, Chebyshev, Hoeffding, union bound.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)


def n_clt(eps, delta, p=0.5):
    return np.ceil(stats.norm.ppf(1 - delta / 2)**2 * p * (1 - p) / eps**2)


def n_cheb(eps, delta, p=0.5):
    return np.ceil(p * (1 - p) / (delta * eps**2))


```

```python
def n_hoef(eps, delta):
    return np.ceil(np.log(2 / delta) / (2 * eps**2))


print("Sample size for margin eps with confidence 1 - delta (worst case p=0.5)")
print(f"{'eps':>6}{'delta':>7}{'CLT':>8}{'Chebyshev':>11}{'Hoeffding':>11}")
for eps, delta in [(0.03, 0.05), (0.02, 0.05), (0.01, 0.05), (0.02, 0.01)]:
    print(f"{eps:6.2f}{delta:7.2f}{n_clt(eps, delta):8.0f}"
          f"{n_cheb(eps, delta):11.0f}{n_hoef(eps, delta):11.0f}")

# Simultaneous precision in m = 10 regions: union bound uses delta/m each
m, eps, delta, deff = 10, 0.03, 0.05, 2.0
print(f"\nm = {m} regions, eps = {eps}, overall delta = {delta}")
print(f"  per region, CLT, delta:      {n_clt(eps, delta):6.0f}")
print(f"  per region, CLT, delta/m:    {n_clt(eps, delta / m):6.0f}")
print(f"  per region, Hoeffding, d/m:  {n_hoef(eps, delta / m):6.0f}")
print(f"  x design effect {deff} (cluster sample): "
      f"{deff * n_clt(eps, delta / m):6.0f}")

# Check by simulation: simulated regional poverty rates, simple random samples
p_true = rng.uniform(0.15, 0.55, size=m)
reps = 20_000
for lab, n in [("delta", n_clt(eps, delta)), ("delta/m", n_clt(eps, delta / m))]:
    phat = rng.binomial(int(n), p_true, size=(reps, m)) / n
    miss_any = np.mean(np.any(np.abs(phat - p_true) > eps, axis=1))
    print(f"  n per region = {n:5.0f} ({lab:>7}): "
          f"P(some region misses by > eps) = {miss_any:.3f}")
```

Output

```
Sample size for margin eps with confidence 1 - delta (worst case p=0.5)
   eps  delta     CLT  Chebyshev  Hoeffding
  0.03   0.05    1068       5556       2050
  0.02   0.05    2401      12500       4612
  0.01   0.05    9604      50000      18445
  0.02   0.01    4147      62500       6623

m = 10 regions, eps = 0.03, overall delta = 0.05
  per region, CLT, delta:        1068
  per region, CLT, delta/m:      2189
  per region, Hoeffding, d/m:    3329
  x design effect 2.0 (cluster sample):   4378
  n per region =  1068 (  delta): P(some region misses by > eps) = 0.325
  n per region =  2189 (delta/m): P(some region misses by > eps) = 0.032
```

Many estimates at once: the union bound. A national survey that must publish poverty rates for m regions faces a simultaneous question: what is the probability that *at least one* regional estimate misses by more than ε? Boole's inequality (the union bound) $\mathbb{P}(\bigcup_j A_j) \le \sum_j \mathbb{P}(A_j)$ shows that it suffices to run each region at level δ/m. The simulation above confirms that sizing each region for $\delta = 0.05$ gives a 32.5% chance that some region misses, while sizing it for $\delta/m = 0.005$ brings the simultaneous failure rate to 3.2%. Thanks to the logarithm in Hoeffding's bound, the cost is modest: the sample size grows like $\log(m/\delta)$.

Policy Lens: Designing a national living standards survey (simulated example)

Suppose a survey in the style of a living standards measurement study must estimate poverty in each of 10 regions to within ±3 percentage points with 95% *simultaneous* confidence.

The CLT with a Bonferroni correction requires about 2200 completed interviews per region under simple random sampling. Real surveys sample clusters (enumeration areas) and then households within them; because households in the same cluster are alike, the variance is inflated by the *design effect* deff $\approx 1 + (b-1)\rho$, where b is the number of households per cluster and ρ the intra-cluster correlation (Section 6.2, Exercise 6.2.1). With deff $= 2$ the requirement doubles to about 4400 per region, or 44,000 nationally, before allowing for non-response. Sample-size arithmetic of this kind determines survey budgets. (All numbers here are illustrative, not those of any actual survey.)

Policy Lens: What a poll's "margin of error" means

The reported margin of error of an opinion poll of $n = 1000$ is usually $1.96\sqrt{0.25/1000} \approx 3.1$ percentage points: a CLT-based 95% half-width for a single proportion near $1/2$ under simple random sampling. It does not cover non-response bias, coverage error or question wording, it is larger for differences between two candidates (roughly double for the lead, since the two shares are negatively correlated) and for subgroups, and with many subgroup comparisons some will look "significant" by chance.

Listing 6.17: Family-wise error, Bonferroni and the maximum of m means ch06/ch06_union_bound.py

```
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# Many hypotheses: family-wise error, the union bound and the max of m means.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
alpha, n, reps = 0.05, 400, 2000
print("m independent true-null z-tests on Bernoulli(0.5) means, n = 400")
print(f"{'m':>5}{'FWER':>8}{'1-(1-a)^m':>11}{'m*a':>7}{'FWER Bonf':>11}"
      f"{'E max|dev|':>12}{'Hoeffding':>11}")
for m in [1, 5, 20, 100, 500]:
    phat = rng.binomial(n, 0.5, size=(reps, m)) / n
    z = (phat - 0.5) / np.sqrt(0.25 / n)
    fwer = np.mean(np.any(np.abs(z) > 1.96, axis=1))
    crit = stats.norm.ppf(1 - alpha / (2 * m))          # Bonferroni cut-off
    bonf = np.mean(np.any(np.abs(z) > crit, axis=1))
    maxdev = np.mean(np.max(np.abs(phat - 0.5), axis=1))
    hoef = np.sqrt(np.log(2 * m / alpha) / (2 * n))      # 95% bound on max
    print(f"{m:5d}{fwer:8.3f}{1 - (1 - alpha)**m:11.3f}{min(1, m * alpha):7.2f}"
          f"{bonf:11.3f}{maxdev:12.4f}{hoef:11.4f}")
print("Hoeffding column: max deviation bound holding with prob >= 0.95")
```

Output

```
m independent true-null z-tests on Bernoulli(0.5) means, n = 400
    m    FWER  1-(1-a)^m    m*a  FWER Bonf  E max|dev|  Hoeffding
    1   0.051      0.050   0.05      0.051      0.0202     0.0679
    5   0.226      0.226   0.25      0.051      0.0388     0.0814
   20   0.627      0.642   1.00      0.037      0.0536     0.0914
  100   0.993      0.994   1.00      0.044      0.0682     0.1018
  500   1.000      1.000   1.00      0.056      0.0807     0.1113
Hoeffding column: max deviation bound holding with prob >= 0.95
```

The output shows the multiple-testing problem in its starkest form: with 20 true null

hypotheses each tested at 5%, the chance of at least one false discovery is 63%; with 100 it is near certainty. The Bonferroni cut-off $z_{1-\alpha/(2m)}$ restores family-wise control. The expected largest deviation among m means grows like $\sqrt{\log m/n}$, matching the Hoeffding–union bound $\sqrt{\log(2m/\alpha)/(2n)}$.

△ Caution: Searching over many specifications

The union bound is the formal version of a familiar warning: if an analyst tries many outcomes, subgroups, controls or model specifications and reports the most significant, the nominal p-value is meaningless. Pre-analysis plans, Bonferroni or Holm corrections, false-discovery-rate control and sample splitting are the remedies. In machine learning the same logic, with m replaced by the size (or complexity) of the hypothesis class, gives generalisation bounds: the gap between training and test error is of order $\sqrt{\log m/n}$ for a class of m models.

6.6.5 High-dimensional phenomena

Concentration has surprising geometric consequences in high dimension (Vershynin, 2018, Ch. 3). Let $\mathbf{X} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$. Then $\|\mathbf{X}\|^2 = \sum_{j=1}^{d} X_j^2$ has mean d and variance $2d$, so $\|\mathbf{X}\|/\sqrt{d} \to 1$; much more is true: $\|\mathbf{X}\| - \sqrt{d}$ is sub-Gaussian with a variance proxy that does *not* depend on d,

$$\mathbb{P}\Big(\big|\|\mathbf{X}\| - \sqrt{d}\big| \geq t\Big) \leq 2e^{-ct^2}. \tag{6.20}$$

A Gaussian vector in high dimension lives in a thin shell of radius $\sqrt{d}$ and width $O(1)$. Similarly, if $\mathbf{X}, \mathbf{Y}$ are independent and uniformly distributed on the unit sphere then $\langle \mathbf{X}, \mathbf{Y} \rangle$ is approximately $\mathcal{N}(0, 1/d)$: random directions are nearly orthogonal, and by a union bound even among k random vectors the largest $|\cos|$ is only of order $\sqrt{\log k/d}$.

Listing 6.18: Norm concentration and near-orthogonality of random vectors ch06/ch06_high_dim.py

```python
# DEP 803 -- Chapter 6: Expectation, Covariance, Limit Theorems and Concentration
# High-dimensional concentration: norms of Gaussian vectors, near-orthogonality.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
k = 200                                   # number of random vectors
print(f"{'d':>6}{'mean |X|/sqrt(d)':>18}{'sd |X|':>9}{'mean |cos|':>12}"
      f"{'max |cos|':>11}{'1/sqrt(d)':>11}")
cosines = {}
for d in [2, 10, 100, 1000, 10000]:
    X = rng.normal(size=(k, d))
    norms = np.linalg.norm(X, axis=1)
    U = X / norms[:, None]
    C = U @ U.T
    off = C[np.triu_indices(k, 1)]        # all k(k-1)/2 pairwise cosines
    cosines[d] = off
    print(f"{d:6d}{np.mean(norms / np.sqrt(d)):18.4f}{norms.std():9.4f}"
          f"{np.mean(np.abs(off)):12.4f}{np.max(np.abs(off)):11.4f}"
          f"{1 / np.sqrt(d):11.4f}")
print("(sd of |X| stays near 1/sqrt(2) = 0.7071 in every dimension)")

plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
for d, col in [(10, "firebrick"), (100, "darkorange"), (1000, "navy")]:
```

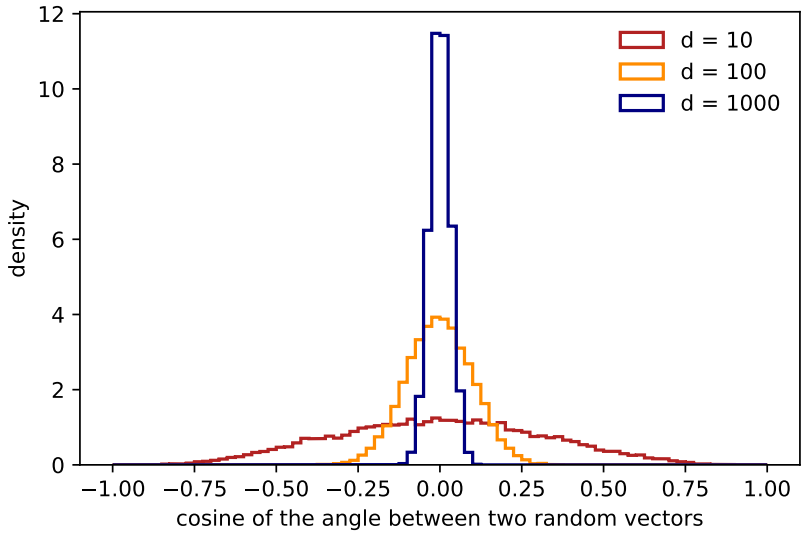


Figure 6.8: Cosines of angles between 200 random Gaussian vectors in dimension d. As d grows the cosines concentrate around 0 with spread $1/\sqrt{d}$: random high-dimensional vectors are nearly orthogonal.

```
    ax.hist(cosines[d], bins=80, range=(-1, 1), density=True, histtype="step",
            lw=1.5, color=col, label=f"d = {d}")
ax.set_xlabel("cosine of the angle between two random vectors")
ax.set_ylabel("density")
ax.legend(frameon=False)
plt.savefig("ch06_highdim.pdf", bbox_inches="tight")
```

Output

```
     d  mean |X|/sqrt(d)   sd |X|  mean |cos|  max |cos|  1/sqrt(d)
     2            0.9071   0.6418      0.6348     1.0000     0.7071
    10            0.9533   0.6363      0.2581     0.9127     0.3162
   100            0.9882   0.6828      0.0803     0.3944     0.1000
  1000            0.9992   0.7330      0.0252     0.1411     0.0316
 10000            0.9998   0.7261      0.0080     0.0399     0.0100
(sd of |X| stays near 1/sqrt(2) = 0.7071 in every dimension)
```

Graduate Perspective: Concentration of measure in data science

These facts drive several modern methods. (i) *Random projections*: the Johnson–Lindenstrauss lemma states that k points can be mapped into $O(\varepsilon^{-2}\log k)$ dimensions by a random linear map while preserving all pairwise distances within a factor $1 \pm \varepsilon$; the proof is norm concentration plus a union bound. (ii) *Distances lose contrast*: for iid features, nearest and farthest neighbours become almost equidistant, which hurts naive k-nearest-neighbour methods. (iii) *Random matrices*: the sample covariance matrix of n observations in d dimensions is a poor estimate of $\boldsymbol{\Sigma}$ unless $n \gg d$, and its eigenvalues spread out according to the Marchenko–Pastur law, which matters for portfolio optimisation and principal components of large macroeconomic panels (Chapter 3). Vershynin (2018) and Wainwright (2019) are the standard graduate references.

Historical Note: Chernoff, Hoeffding and the rise of non-asymptotic bounds

Sergei Bernstein derived exponential inequalities for sums of independent variables in the 1920s. Herman Chernoff's 1952 paper in the *Annals of Mathematical Statistics*, on the asymptotic efficiency of tests based on sums of observations, introduced the exponential bound (6.17) (Chernoff, 1952); Chernoff later credited the idea of the bound to Herman Rubin. Wassily Hoeffding's 1963 paper in the *Journal of the American Statistical Association* gave the inequality for bounded variables that bears his name (Hoeffding, 1963). Their use exploded with computational learning theory in the 1980s and with high-dimensional statistics in the 2000s.

⋆ Key Concept: Concentration

The Chernoff method, Markov applied to $e^{\lambda S}$ and optimised over λ, turns MGF bounds into exponential tail bounds. Hoeffding: for $[0,1]$-valued independent variables $\mathbb{P}(|\bar{X}_n - \mu| \geq \varepsilon) \leq 2e^{-2n\varepsilon^2}$, valid for every n. Sample sizes from concentration bounds scale like $\log(1/\delta)/\varepsilon^2$, and with m simultaneous targets like $\log(m/\delta)/\varepsilon^2$ (union bound). In high dimension, norms concentrate and random vectors are nearly orthogonal.

Exercises

6.6.1 Theory Derive the Chernoff bound for a sum of n iid $\mathcal{N}(0,\sigma^2)$ variables and show it gives $\mathbb{P}(\bar{X}_n \geq \varepsilon) \leq e^{-n\varepsilon^2/(2\sigma^2)}$. Compare with the exact tail $1 - \Phi(\sqrt{n}\varepsilon/\sigma)$.

6.6.2 Theory Prove that a variable on $[a,b]$ has variance at most $(b-a)^2/4$, and complete the proof of Hoeffding's lemma.

6.6.3 Theory Show that for $X \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$, $\mathbb{E}\,\|X\|^2 = d$ and $\operatorname{Var}\|X\|^2 = 2d$, and use the inequality $\left|\|X\| - \sqrt{d}\right| \leq \left|\|X\|^2 - d\right| / \sqrt{d}$ with Chebyshev to show $\|X\|/\sqrt{d} \xrightarrow{p} 1$.

6.6.4 Python Add Bernstein's bound to Listing 6.15 and to its figure. For $p = 0.1$, by how much does it improve on Hoeffding at $n = 1000$?

6.6.5 Python Implement a Johnson–Lindenstrauss random projection of $k = 500$ points in $d = 10{,}000$ dimensions to $r \in \{50, 200, 800\}$ dimensions and report the worst relative distortion of pairwise distances.

6.6.6 ★ Graduate (Generalisation bound.) A classifier is selected from a finite class $\mathcal{H}$ of m rules by minimising the training error on n iid examples with 0–1 loss. Use Hoeffding and the union bound to show that, with probability at least $1-\delta$, every $h \in \mathcal{H}$ has $\left|\text{test error} - \text{training error}\right| \leq \sqrt{\log(2m/\delta)/(2n)}$. What does this imply for selecting a targeting rule among 1000 candidate proxy-means formulas with $n = 5000$ households?

6.6.7 Policy A health ministry wants district-level immunisation coverage for $m = 50$ districts within ± 5 percentage points, with 90% simultaneous confidence, using a cluster survey with design effect 1.5. Compute the required number of children per district using (a) the CLT with Bonferroni and (b) Hoeffding with the union bound, and discuss which you would defend to a budget committee.

6.6.8 Policy A poll of 1000 voters shows candidate A at 48% and B at 45%. Using the multinomial covariance $\operatorname{Cov}(\hat{p}_A, \hat{p}_B) = -p_A p_B / n$, compute the standard error of the lead $\hat{p}_A - \hat{p}_B$ and test whether the lead is significant.

Chapter Summary

- **Expectation** is the Lebesgue integral $\int X \, d\mathbb{P}$; it is linear (with no independence needed), monotone, and computable by LOTUS. Jensen's inequality $\varphi(\mathbb{E}X) \leq \mathbb{E}\varphi(X)$ for convex φ underlies risk aversion, certainty equivalents, the welfare value of insurance and retransformation bias.
- **MGFs and characteristic functions** generate moments, characterise laws and turn independent sums into products; the CF always exists, the MGF only for light-tailed laws.
- **Second moments**: $\operatorname{Var}(\mathbf{w}^\top \mathbf{X}) = \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$; covariance matrices are PSD. Zero correlation does not imply independence, correlation does not imply causation, and Simpson's paradox arises from the between-group term of the law of total covariance.
- **Conditional expectation**: the LIE, the law of total variance and the best-predictor property of the CEF; OLS estimates the best linear approximation to the CEF; with random assignment a difference in means identifies the ATE.
- **LLN**: Markov and Chebyshev give the weak law; Kolmogorov's strong law holds iff $\mathbb{E}|X| < \infty$. Know the four modes of convergence and their implications. Averages of Cauchy data never converge; heavy tails slow convergence.
- **CLT**: $\sqrt{n}(\bar{X}_n - \mu)/\sigma \xrightarrow{d} \mathcal{N}(0,1)$, with Lindeberg/Lyapunov extensions, the Berry–Esseen rate $C\rho/(\sigma^3\sqrt{n})$, a multivariate version, Slutsky's theorem and the delta method $\nabla g^\top \boldsymbol{\Sigma} \nabla g / n$ for standard errors of ratios, elasticities and poverty measures.
- **Concentration**: Chernoff's method and Hoeffding's inequality give finite-sample exponential bounds; sample sizes scale like $\log(m/\delta)/\varepsilon^2$ for m simultaneous targets; in high dimension norms concentrate and random vectors are nearly orthogonal.

Further Reading

Casella and Berger (2002, Ch. 2–5) and Wasserman (2004, Ch. 3–6) cover expectation, inequalities and limit theorems at the level of this chapter; Grimmett and Stirzaker (2020) and Billingsley (1995) give the measure-theoretic treatment, including proofs of the strong law, Lévy's continuity theorem and the Lindeberg–Feller CLT. van der Vaart (1998) is the standard reference for Slutsky's theorem, the delta method and their extensions. For conditional expectation as the foundation of regression and policy evaluation, read Hansen (2022, Ch. 2–3), Angrist and Pischke (2009, Ch. 2–3) and Wooldridge (2010); on causal reasoning and Simpson's paradox see Pearl (2009). Concentration inequalities are treated thoroughly by Boucheron et al. (2013), Wainwright (2019) and, with an emphasis on high-dimensional geometry and data science, Vershynin (2018). For the history, Stigler (1986) and Fischer (2011) are excellent.

Among the course readings, Murphy (2022, Ch. 2) reviews expectation, covariance, the LLN and the CLT from a machine-learning perspective and Matsuura (2022) uses the same tools for Bayesian modelling and Monte Carlo summaries of posteriors. Géron (2022) shows where averages of losses and their concentration appear in model training and validation, and Goulet Coulombe et al. (2022) is a good example of how forecast evaluation relies on averages of losses whose sampling variability must be taken seriously.

Chapter 7

Estimation Principles: Moments, Maximum Likelihood and Information Geometry

Learning Objectives

After working through this chapter you should be able to:

- define an estimator, decompose its mean squared error into squared bias and variance, and evaluate bias, variance, consistency and efficiency by Monte Carlo;
- construct method-of-moments and generalised method of moments (GMM) estimators, including two-step efficient GMM, the Hansen J-test and instrumental variables;
- derive maximum likelihood estimators in closed form, compute them numerically with `scipy.optimize.minimize`, and recognise convergence failures such as separation;
- prove that the score has mean zero, state and prove the Cramér–Rao bound, and compute standard errors from observed and expected information and from the sandwich formula;
- carry out and compare Wald, likelihood-ratio and score tests;
- interpret the MLE as minimum Kullback–Leibler projection, explain the Fisher information as a Riemannian metric and implement a natural-gradient step;
- compute posterior distributions (conjugate and grid approximations), relate MAP estimation to penalised likelihood and ridge regression, and state the Bernstein–von Mises theorem.

Every number a data scientist or policy analyst reports — a poverty headcount, a return to schooling, a programme take-up elasticity, the coefficient in a forecasting model — is an *estimate*: the value of some function of data that is meant to approximate an unknown feature of the world. The previous chapters gave us the language of random variables (Chapter 4), the standard families of distributions (Chapter 5) and the laws of large numbers and central limit theorems (Chapter 6). This chapter turns that machinery into *principles* for constructing estimators and for judging how good they are.

There are essentially three such principles, and this chapter studies all of them.

1. **Moments.** Choose the parameter so that population moments implied by the model match their sample counterparts. Generalised to *moment conditions* $\mathbb{E}[g(X, \theta)] = 0$, this principle (GMM) is the workhorse of empirical economics, because it needs only a few economically motivated restrictions rather than a full probability model. Ordinary least squares and

instrumental variables are special cases.

2. **Likelihood.** Choose the parameter that makes the observed data most probable. Maximum likelihood (ML) is efficient when the model is correct, invariant under reparametrisation, and underlies logistic regression, Poisson regression and essentially every "loss function" used in machine learning. When the model is wrong, ML still estimates something well defined — the member of the model that is closest to the truth in Kullback–Leibler divergence — and *information geometry* gives that statement a precise geometric meaning.
3. **Bayes.** Treat the parameter as uncertain, combine a prior with the likelihood, and report the posterior distribution. Bayesian estimation formalises the use of prior information (earlier survey rounds, expert knowledge), produces regularised estimates, and is implemented at scale by probabilistic programming languages such as Stan (Matsuura, 2022).

For data science, these principles explain *why* common algorithms work: minimising cross-entropy is maximum likelihood, weight decay is a Gaussian prior, and the natural gradient is steepest ascent in the Fisher geometry. For economic policy, they underpin the credibility of evidence: whether a return-to-schooling estimate is biased by unobserved ability, whether the standard error in a ministry report survives a misspecified model, and how a small pilot can be combined with prior evidence. Throughout, all data are *simulated* (for example "a simulated household survey") unless they are one of the datasets bundled with scikit-learn; none of the numbers should be read as statistics of Ghana or any other country.

7.1 Estimators and Their Properties

7.1.1 Statistical models, estimators and estimates

Definition 7.1 (Statistical model)**.** A *statistical model* is a family of probability distributions $\mathcal{P} = \{P_\theta : \theta \in \Theta\}$ for the data $X = (X_1, \dots, X_n)$, indexed by a parameter θ in a parameter space Θ. The model is *parametric* if $\Theta \subseteq \mathbb{R}^p$ for a fixed p, and *correctly specified* if the true data distribution P_0 equals P_{θ_0} for some $\theta_0 \in \Theta$. The parameter is *identified* if $P_\theta = P_{\theta'}$ implies $\theta = \theta'$.

Definition 7.2 (Estimator and estimate)**.** An *estimator* of θ (or of a functional $\psi = \psi(\theta)$) is a measurable function $\hat{\theta}_n = T(X_1, \dots, X_n)$ of the data that does not depend on unknown parameters. An *estimate* is the number $T(x_1, \dots, x_n)$ obtained by applying the estimator to a particular realised sample.

The distinction matters: the estimator is a *random variable* with a sampling distribution; the estimate is a single draw from that distribution. All statements about bias, variance and confidence refer to the estimator, that is, to the procedure applied across hypothetical repeated samples.

Definition 7.3 (Bias, variance, MSE)**.** For a scalar estimator $\hat{\theta}$ of θ, with expectations taken under P_θ,

$$\operatorname{Bias}_\theta(\hat{\theta}) = \mathbb{E}_\theta[\hat{\theta}] - \theta, \qquad \operatorname{Var}_\theta(\hat{\theta}) = \mathbb{E}_\theta\big[(\hat{\theta} - \mathbb{E}_\theta\hat{\theta})^2\big], \qquad \operatorname{MSE}_\theta(\hat{\theta}) = \mathbb{E}_\theta\big[(\hat{\theta} - \theta)^2\big].$$

$\hat{\theta}$ is *unbiased* if $\operatorname{Bias}_\theta(\hat{\theta}) = 0$ for all $\theta \in \Theta$. For a vector estimator the MSE matrix is $\mathbb{E}_\theta[(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})^\top]$.

Theorem 7.4 (Bias–variance decomposition)**.** *If* $\mathbb{E}_\theta[\hat\theta^2] < \infty$, *then* $\mathrm{MSE}_\theta(\hat\theta) = \mathrm{Bias}_\theta(\hat\theta)^2 + \mathrm{Var}_\theta(\hat\theta)$.

Proof. Write $m = \mathbb{E}_\theta\hat\theta$. Then $\hat\theta - \theta = (\hat\theta - m) + (m - \theta)$ and

$$\mathbb{E}_\theta(\hat\theta - \theta)^2 = \mathbb{E}_\theta(\hat\theta - m)^2 + 2(m-\theta)\,\mathbb{E}_\theta(\hat\theta - m) + (m-\theta)^2.$$

The middle term vanishes because $\mathbb{E}_\theta(\hat\theta - m) = 0$, and the remaining terms are $\mathrm{Var}_\theta(\hat\theta)$ and $\mathrm{Bias}_\theta(\hat\theta)^2$. □

The vector version reads $\mathbb{E}[(\hat{\boldsymbol\theta} - \boldsymbol\theta)(\hat{\boldsymbol\theta} - \boldsymbol\theta)^\top] = \mathrm{Var}(\hat{\boldsymbol\theta}) + \mathbf{b}\mathbf{b}^\top$ with $\mathbf{b}$ the bias vector; the proof is identical. The decomposition is the starting point for regularisation: accepting a little bias can buy a large reduction in variance, a theme that returns with ridge regression in Section 7.6.

Example 7.5 (Three variance estimators)**.** Let $X_i \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$ and $SS = \sum_i (X_i - \bar X)^2$, so that $SS/\sigma^2 \sim \chi^2_{n-1}$. Consider $\hat\sigma^2_d = SS/d$ for $d \in \{n-1, n, n+1\}$. Using $\mathbb{E}[\chi^2_{n-1}] = n-1$ and $\mathrm{Var}[\chi^2_{n-1}] = 2(n-1)$,

$$\mathrm{Bias}(\hat\sigma^2_d) = \sigma^2\frac{n-1-d}{d}, \qquad \mathrm{Var}(\hat\sigma^2_d) = \frac{2(n-1)\sigma^4}{d^2}, \qquad \mathrm{MSE}(\hat\sigma^2_d) = \sigma^4\,\frac{2(n-1) + (n-1-d)^2}{d^2}.$$

The unbiased S^2 ($d = n-1$) has the *largest* MSE of the three; the MLE ($d = n$) is biased downwards but better; and $d = n+1$ minimises the MSE among all estimators of the form SS/d (Exercise **7.1.2**). Listing 7.1 checks these formulae by simulation.

Listing 7.1: Monte Carlo bias, variance and MSE ch07/est_bias_variance.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Monte Carlo bias, variance and MSE of three estimators of a variance.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
sigma2, R = 4.0, 40000          # true variance, Monte Carlo replications
divisors = {"n-1 (S^2)": 1, "n (MLE)": 0, "n+1": -1}   # divisor = n - ddof

print(f"True sigma^2 = {sigma2}, normal data, R = {R} replications")
print(f"{'n':>4} {'divisor':>10} {'bias':>8} {'var':>8} {'MSE':>8}"
      f" {'bias^2+var':>11} {'MSE theory':>11}")
for n in (5, 20, 100):
    x = rng.normal(0.0, np.sqrt(sigma2), size=(R, n))
    ss = ((x - x.mean(axis=1, keepdims=True)) ** 2).sum(axis=1)
    for name, ddof in divisors.items():
        d = n - ddof
        est = ss / d
        bias = est.mean() - sigma2
        var = est.var()
        mse = np.mean((est - sigma2) ** 2)
        # exact MSE: SS/sigma^2 ~ chi^2_{n-1}
        theory = sigma2**2 * (2 * (n - 1) + (n - 1 - d) ** 2) / d**2
        print(f"{n:>4} {name:>10} {bias:8.3f} {var:8.3f} {mse:8.3f}"
              f" {bias**2 + var:11.3f} {theory:11.3f}")
```

Output

```
True sigma^2 = 4.0, normal data, R = 40000 replications
   n    divisor     bias      var      MSE  bias^2+var  MSE theory
   5  n-1 (S^2)   -0.002    8.036    8.036       8.036       8.000
   5    n (MLE)   -0.802    5.143    5.786       5.786       5.760
   5        n+1   -1.335    3.571    5.353       5.353       5.333
  20  n-1 (S^2)    0.000    1.681    1.681       1.681       1.684
  20    n (MLE)   -0.200    1.517    1.557       1.557       1.560
  20        n+1   -0.381    1.376    1.521       1.521       1.524
 100  n-1 (S^2)   -0.004    0.322    0.322       0.322       0.323
 100    n (MLE)   -0.044    0.316    0.318       0.318       0.318
 100        n+1   -0.083    0.310    0.317       0.317       0.317
```

The columns "MSE" and "bias^2+var" agree to all printed digits (the decomposition is an algebraic identity that also holds for the empirical distribution of the R replications), and both match the exact formula to Monte Carlo error. With $n = 5$ the differences are large; with $n = 100$ they are negligible: all three estimators are *asymptotically equivalent*.

△ Caution: Unbiasedness is not a virtue in itself

Unbiasedness is not preserved by nonlinear transformations: S^2 is unbiased for σ^2 but S is biased for σ. It can also conflict with common sense: for $X \sim \text{Poisson}(\lambda)$ the only unbiased estimator of $e^{-2\lambda}$ is $(-1)^X$, which is negative half of the time. Policy reports should judge estimators by MSE, by the coverage of their intervals and by robustness, not by unbiasedness alone.

7.1.2 Consistency, asymptotic normality and efficiency

Exact finite-sample distributions are available only in special models. Most of the theory of estimation is therefore *asymptotic*: it describes the behaviour of $\hat{\theta}_n$ as $n \to \infty$ and uses that description as an approximation for the sample size at hand.

Definition 7.6 (Consistency and asymptotic normality)**.** A sequence of estimators $\hat{\theta}_n$ is *(weakly) consistent* for θ if $\hat{\theta}_n \xrightarrow{p} \theta$ under P_θ for every $\theta \in \Theta$. It is $\sqrt{n}$*-asymptotically normal* with asymptotic variance $V(\theta)$ if

$$\sqrt{n}\,(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}\big(0, V(\theta)\big).$$

In that case we write $\hat{\theta}_n \overset{a}{\sim} \mathcal{N}(\theta, V/n)$ and call $\sqrt{\hat{V}/n}$ the (asymptotic) *standard error*.

If $\text{Bias}(\hat{\theta}_n) \to 0$ and $\text{Var}(\hat{\theta}_n) \to 0$, then $\hat{\theta}_n$ is consistent, because Markov's inequality gives $\mathbb{P}(|\hat{\theta}_n - \theta| > \varepsilon) \le \text{MSE}(\hat{\theta}_n)/\varepsilon^2$; the converse is false, since a consistent estimator need not even have a finite variance. Two tools from Chapter 6 do most of the work in asymptotic arguments: the *continuous mapping theorem* (if $\hat{\theta}_n \xrightarrow{p} \theta$ and g is continuous at θ then $g(\hat{\theta}_n) \xrightarrow{p} g(\theta)$) and the *delta method*:

$$\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}(0, V) \implies \sqrt{n}\big(g(\hat{\theta}_n) - g(\theta)\big) \xrightarrow{d} \mathcal{N}\big(0,\, g'(\theta)^2 V\big) \tag{7.1}$$

for g differentiable at θ (with gradient sandwich $\nabla g^\top V \nabla g$ in the vector case). Together with Slutsky's theorem they allow us to replace unknown asymptotic variances by consistent estimates.

Definition 7.7 (Relative efficiency)**.** If two estimators satisfy $\sqrt{n}(\hat{\theta}_j - \theta) \xrightarrow{d} \mathcal{N}(0, V_j)$, $j = 1, 2$, the *asymptotic relative efficiency* of $\hat{\theta}_2$ with respect to $\hat{\theta}_1$ is $\mathrm{ARE}(\hat{\theta}_2, \hat{\theta}_1) = V_1/V_2$. An ARE of 0.64 means that $\hat{\theta}_2$ needs about $1/0.64 \approx 1.57$ times as many observations to achieve the same precision.

Example 7.8 (Mean versus median)**.** Let X_i be iid with density f symmetric about θ and variance σ^2. The sample mean satisfies $\sqrt{n}(\bar{X} - \theta) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$. The sample median M_n satisfies $\sqrt{n}(M_n - \theta) \xrightarrow{d} \mathcal{N}\big(0, 1/(4 f(\theta)^2)\big)$ whenever $f(\theta) > 0$, even if $\sigma^2 = \infty$. Hence $\mathrm{ARE}(M_n, \bar{X}) = 4 f(\theta)^2 \sigma^2$, which equals

- $2/\pi \approx 0.637$ for the normal distribution (the mean wins);
- $4 \cdot (0.3676)^2 \cdot 3 \approx 1.62$ for Student's t_3 (the median wins);
- ∞ for the Cauchy distribution, for which $\bar{X}$ has the same Cauchy distribution as a single observation and is not consistent at all.

7.1.3 Monte Carlo evaluation of estimators

When analytic results are unavailable, the properties of an estimator can be *estimated* by simulation (Chapter 10): choose a data-generating process with known θ, draw R samples of size n, compute $\hat{\theta}^{(1)}, \ldots, \hat{\theta}^{(R)}$, and estimate bias, variance, MSE and the coverage of interval procedures by the corresponding averages over replications. The Monte Carlo standard error of an estimated coverage $\hat{c}$ is $\sqrt{\hat{c}(1-\hat{c})/R}$, about 0.005 for $R = 2000$ and $c = 0.95$. One $R \times n$ array of draws and an `axis=1` reduction replace an explicit loop.

Listing 7.2 compares the mean and median under three distributions. Since the Cauchy mean has no variance, we measure spread by the interquartile range (IQR) of the sampling distribution, scaled by $\sqrt{n}$ so that a $\sqrt{n}$-consistent estimator gives a stable number.

Listing 7.2: Mean versus median under light and heavy tails (plotting code omitted)
ch07/mean_vs_median.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Sample mean versus sample median as estimators of a centre: light vs heavy tails.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
R = 20000
draws = {"normal": lambda size: rng.normal(size=size),
         "t(3)": lambda size: rng.standard_t(3, size=size),
         "Cauchy": lambda size: rng.standard_cauchy(size=size)}

def iqr(v):
    q1, q3 = np.percentile(v, [25, 75])
    return q3 - q1

print("Spread of estimators of the centre (true centre = 0)")
print("sqrt(n) * IQR of the sampling distribution, R = 20000")
print(f"{'model':>7} {'n':>5} {'mean':>9} {'median':>9} {'ratio':>7}")
for name, draw in draws.items():
    for n in (25, 400):
        x = draw((R, n))
```

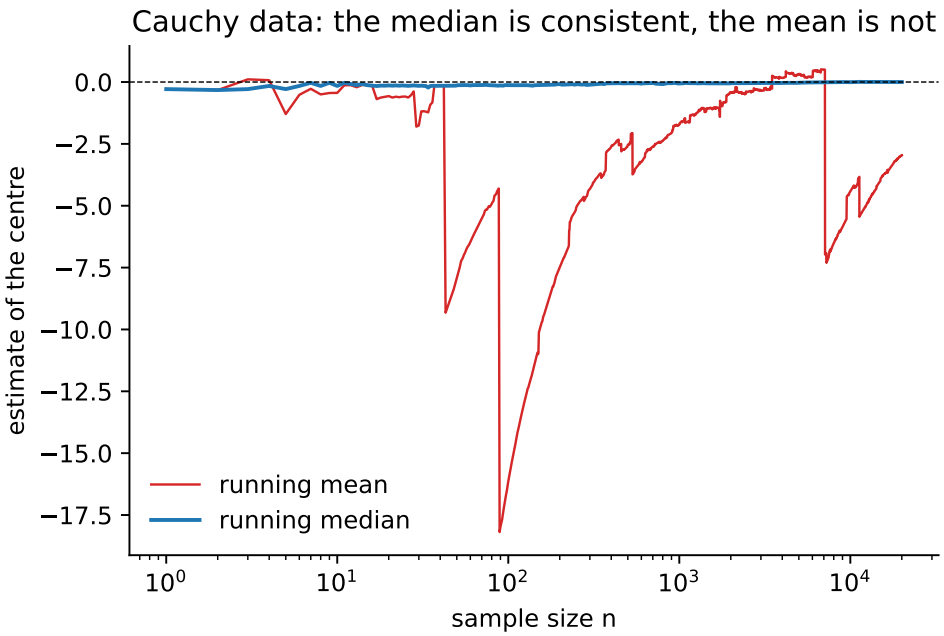


Figure 7.1: Running mean and running median of 20 000 standard Cauchy draws. The median settles at the centre 0; the mean never does (Listing 7.2).

```
        m, md = x.mean(axis=1), np.median(x, axis=1)
        a, b = np.sqrt(n) * iqr(m), np.sqrt(n) * iqr(md)
        print(f"{name:>7} {n:>5} {a:9.3f} {b:9.3f} {a / b:7.2f}")
print("Theory (normal): ARE(median, mean) = 2/pi =", round(2 / np.pi, 3))
```

Output

```
Spread of estimators of the centre (true centre = 0)
sqrt(n) * IQR of the sampling distribution, R = 20000
  model     n      mean    median   ratio
 normal    25     1.333     1.664    0.80
 normal   400     1.351     1.689    0.80
   t(3)    25     2.131     1.834    1.16
   t(3)   400     2.304     1.841    1.25
 Cauchy    25     9.891     2.133    4.64
 Cauchy   400    40.604     2.129   19.08
Theory (normal): ARE(median, mean) = 2/pi = 0.637
```

For normal data the ratio of IQRs is $0.80 = \sqrt{2/\pi}$, as predicted by Example 7.8. Under t_3 the median is about 25% tighter. Under the Cauchy distribution the scaled IQR of the mean grows like $\sqrt{n}$ (from 9.9 to 40.6 as n grows sixteenfold), exactly as it must if $\bar{X}$ does not concentrate at all, while the median is perfectly well behaved. Figure 7.1 shows a single Cauchy sequence: the running mean is repeatedly thrown off by single enormous observations.

⋆ Key Concept: Judging an estimator

An estimator is a procedure, judged by its sampling distribution. The MSE splits into squared bias plus variance (Theorem 7.4). In large samples the relevant properties are consistency

($\hat\theta_n \xrightarrow{p} \theta$), asymptotic normality ($\sqrt{n}(\hat\theta_n - \theta) \xrightarrow{d} \mathcal{N}(0, V)$) and efficiency (small V). Efficiency rankings depend on the data distribution: the mean is optimal for normal data and useless for Cauchy data. When theory is silent, *simulate*.

Policy Lens: Mean or median income?

Household income and consumption distributions are right-skewed with heavy upper tails. A survey-based *mean* income is sensitive to a handful of very rich respondents (and to top-coding and data-entry errors), while the *median* is stable. The two estimate different population quantities, so the choice is first a question of *what one wants to measure*; but when either would do — for instance when tracking a "typical" household over time — the median's robustness under heavy tails, demonstrated in Listing 7.2, is a strong practical argument.

Historical Note: Fisher and the criteria of estimation

The vocabulary of this section — *consistency*, *efficiency* and *sufficiency* — was introduced by R. A. Fisher in his 1922 paper "On the mathematical foundations of theoretical statistics" in the *Philosophical Transactions of the Royal Society* (Fisher, 1922). Fisher there separated the problems of *specification* (choosing the model), *estimation* and *distribution*, a division still used today. Earlier, F. Y. Edgeworth had studied the "probable errors of frequency-constants" (1908–09) and anticipated parts of the efficiency theory; see Stigler (2007) for the history.

Exercises

7.1.1 Theory Prove the vector bias–variance decomposition $\mathbb{E}[(\hat{\boldsymbol\theta} - \boldsymbol\theta)(\hat{\boldsymbol\theta} - \boldsymbol\theta)^\top] = \operatorname{Var}(\hat{\boldsymbol\theta}) + \mathbf{b}\mathbf{b}^\top$, where $\mathbf{b} = \mathbb{E}\hat{\boldsymbol\theta} - \boldsymbol\theta$. Deduce that the trace of the MSE matrix is the expected squared Euclidean error.

7.1.2 Theory Using the formulae in Example 7.5, show that among estimators of the form SS/d the MSE is minimised at $d = n + 1$. Is $SS/(n+1)$ admissible among *all* estimators of σ^2? (State what you can prove and what you cannot.)

7.1.3 Theory Let $X_i \overset{\text{iid}}{\sim} \text{Uniform}(0, \theta)$. Show that $\hat\theta_1 = 2\bar X$ is unbiased with variance $\theta^2/(3n)$ and that $\hat\theta_2 = \max_i X_i$ has $\mathbb{E}\hat\theta_2 = n\theta/(n+1)$ and $\operatorname{MSE}(\hat\theta_2) = 2\theta^2/((n+1)(n+2))$. Which converges faster? Is $\hat\theta_2$ asymptotically normal?

7.1.4 Python Modify Listing 7.1 to use t_5 data rescaled to variance 4. Which divisor now minimises the MSE for $n = 20$? Explain using the formula $\operatorname{Var}(S^2) = \sigma^4\{2/(n-1) + \kappa/n\}$, where κ is the excess kurtosis.

7.1.5 Python Estimate by simulation the coverage of the nominal 95% interval $\bar X \pm 1.96 S/\sqrt{n}$ for lognormal data ($\log X \sim \mathcal{N}(0,1)$) with $n \in \{20, 100, 500\}$. Report Monte Carlo standard errors. What does this imply for confidence intervals around mean household consumption?

7.1.6 Policy A statistics office reports mean monthly household consumption from a survey of 2000 households. A journalist notes that removing the top five households changes the estimate by 8%. Write a short memo explaining, in terms of the sampling distribution of the mean under heavy tails, why this happens and what alternative summaries (median, trimmed mean, mean of log consumption) estimate.

7.2 The Method of Moments and GMM

7.2.1 The classical method of moments

Suppose the first p moments $\mu_k(\theta) = \mathbb{E}_\theta[X^k]$, $k = 1, \dots, p$, determine $\theta \in \mathbb{R}^p$. The sample moments $\hat{\mu}_k = n^{-1}\sum_i X_i^k$ are consistent for $\mu_k(\theta_0)$ by the law of large numbers.

Definition 7.9 (Method-of-moments estimator)**.** The *method-of-moments (MoM) estimator* $\tilde{\theta}_n$ solves the p equations $\mu_k(\tilde{\theta}_n) = \hat{\mu}_k$, $k = 1, \dots, p$. Equivalently, with $\mu(\theta) = (\mu_1(\theta), \dots, \mu_p(\theta))^\top$, $\tilde{\theta}_n = \mu^{-1}(\hat{\mu})$. Central moments (mean and variance) may be used instead of raw moments.

Proposition 7.10 (Properties of MoM)**.** *If μ is a bijection with continuously differentiable inverse near $\mu(\theta_0)$ and $\mathbb{E}X^{2p} < \infty$, then $\tilde{\theta}_n \xrightarrow{p} \theta_0$ and $\sqrt{n}(\tilde{\theta}_n - \theta_0) \xrightarrow{d} \mathcal{N}\big(0,\ D\,\Sigma\,D^\top\big)$, where $\Sigma = \operatorname{Var}\big((X, X^2, \dots, X^p)^\top\big)$ and D is the Jacobian of μ^{-1} at $\mu(\theta_0)$.*

Proof. Consistency follows from the LLN and the continuous mapping theorem. The multivariate CLT gives $\sqrt{n}(\hat{\mu} - \mu(\theta_0)) \xrightarrow{d} \mathcal{N}(0, \Sigma)$, and the delta method (7.1) applied to μ^{-1} yields the result. □

Example 7.11 (Gamma and beta)**.** *Gamma.* If $X \sim \text{Gamma}(k, \vartheta)$ (shape k, scale ϑ), then $\mathbb{E}X = k\vartheta$ and $\operatorname{Var} X = k\vartheta^2$. Solving $\bar{X} = k\vartheta$ and $\hat{s}^2 = k\vartheta^2$ gives

$$\tilde{k} = \bar{X}^2/\hat{s}^2, \qquad \tilde{\vartheta} = \hat{s}^2/\bar{X}.$$

Beta. If $X \sim \text{Beta}(a, b)$ then $m = \mathbb{E}X = a/(a+b)$ and $v = \operatorname{Var} X = m(1-m)/(a+b+1)$. Hence $a + b = m(1-m)/v - 1 =: c$ and

$$\tilde{a} = \bar{X}\,\tilde{c}, \qquad \tilde{b} = (1 - \bar{X})\,\tilde{c}, \qquad \tilde{c} = \bar{X}(1-\bar{X})/\hat{s}^2 - 1.$$

The beta family is a natural model for budget shares and proportions; the gamma family for positive, right-skewed quantities such as consumption or rainfall.

Listing 7.3: Method of moments for gamma and beta data ch07/mom_gamma_beta.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Method-of-moments estimators for the gamma and beta families.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

def mom_gamma(x):
    m, v = x.mean(), x.var()
    return m**2 / v, v / m                    # shape k, scale theta

def mom_beta(x):
    m, v = x.mean(), x.var()
    c = m * (1 - m) / v - 1
    return m * c, (1 - m) * c                 # a, b

# Simulated household consumption (gamma) and budget shares (beta)
k0, th0, a0, b0, n, R = 2.5, 1.6, 2.0, 5.0, 400, 5000
xg = rng.gamma(k0, th0, size=n)
xb = rng.beta(a0, b0, size=n)
```

```python
print(f"One sample of n = {n}")
print("  gamma: true (k, theta) = (%.2f, %.2f), MoM = (%.3f, %.3f)"
      % ((k0, th0) + mom_gamma(xg)))
print("  beta : true (a, b)     = (%.2f, %.2f), MoM = (%.3f, %.3f)"
      % ((a0, b0) + mom_beta(xb)))

# Sampling properties over R replications
G = rng.gamma(k0, th0, size=(R, n))
B = rng.beta(a0, b0, size=(R, n))
mg, vg = G.mean(axis=1), G.var(axis=1)
mb, vb = B.mean(axis=1), B.var(axis=1)
k_hat, th_hat = mg**2 / vg, vg / mg
c = mb * (1 - mb) / vb - 1
a_hat, b_hat = mb * c, (1 - mb) * c
print(f"\nMonte Carlo over R = {R} samples of size n = {n}")
print(f"{'param':>6} {'true':>6} {'mean est':>9} {'bias':>8} {'sd':>7}")
for name, t, e in (("k", k0, k_hat), ("theta", th0, th_hat),
                   ("a", a0, a_hat), ("b", b0, b_hat)):
    print(f"{name:>6} {t:6.2f} {e.mean():9.3f} {e.mean() - t:8.3f}"
          f" {e.std():7.3f}")
```

Output

```
One sample of n = 400
  gamma: true (k, theta) = (2.50, 1.60), MoM = (2.432, 1.631)
  beta : true (a, b)     = (2.00, 5.00), MoM = (1.965, 4.989)

Monte Carlo over R = 5000 samples of size n = 400
 param   true  mean est     bias      sd
     k   2.50     2.530    0.030   0.213
 theta   1.60     1.593   -0.007   0.144
     a   2.00     2.017    0.017   0.144
     b   5.00     5.042    0.042   0.380
```

The estimators are nearly unbiased at $n = 400$. They are, however, *not* efficient: in Section 7.3 we will see that the gamma MLE uses $\sum \log X_i$ as well as $\sum X_i$ and has a smaller variance (Exercise 7.3.4). MoM estimators are nonetheless valuable: they are quick, often closed form, and excellent starting values for numerical likelihood maximisation.

Historical Note: Pearson, Weldon and the crabs

The method of moments was introduced by Karl Pearson in his 1894 paper "Contributions to the mathematical theory of evolution" (Pearson, 1894). The zoologist W. F. R. Weldon had measured the ratio of forehead breadth to body length for 1000 female shore crabs from the Bay of Naples and found an asymmetric distribution; he suspected the population was a mixture of two types. Pearson fitted a mixture of two normal distributions — five parameters — by equating the first five sample moments to their theoretical values, which required solving a polynomial equation of degree nine by hand. Fisher later criticised the method's inefficiency, and the debate between Pearson's moments and Fisher's likelihood shaped much of early twentieth-century statistics.

7.2.2 Generalised method of moments

Economic theory rarely delivers a full probability model, but it often delivers *moment conditions*: orthogonality between an error and variables known at the time of a decision, or between an unobserved shock and an instrument.

Definition 7.12 (GMM)**.** Let $g(X,\theta)\in\mathbb{R}^L$ satisfy the population moment condition

$$\mathbb{E}\big[g(X_i,\theta_0)\big]=\mathbf{0},\qquad \theta_0\in\Theta\subseteq\mathbb{R}^p,\quad L\geq p, \tag{7.2}$$

and let $\bar g_n(\theta)=n^{-1}\sum_i g(X_i,\theta)$. For a symmetric positive definite weighting matrix W (possibly estimated, $\hat W\xrightarrow{p}W$) the *GMM estimator* is

$$\hat\theta_{\mathrm{GMM}}=\arg\min_{\theta\in\Theta}\; Q_n(\theta),\qquad Q_n(\theta)=\bar g_n(\theta)^\top\,\hat W\,\bar g_n(\theta).$$

The model is *just identified* if $L=p$ (then $\bar g_n(\hat\theta)=0$ and W is irrelevant) and *over-identified* if $L>p$.

Theorem 7.13 (Asymptotics of GMM; Hansen, 1982)**.** *Under identification ($\mathbb{E}g(X,\theta)=0$ only at θ_0), smoothness and moment conditions, $\hat\theta_{\mathrm{GMM}}\xrightarrow{p}\theta_0$ and*

$$\sqrt{n}(\hat\theta_{\mathrm{GMM}}-\theta_0)\xrightarrow{d}\mathcal{N}\Big(0,\;(G^\top WG)^{-1}G^\top WSWG\,(G^\top WG)^{-1}\Big),$$

where $G=\mathbb{E}[\partial g(X,\theta_0)/\partial\theta^\top]$ ($L\times p$, full column rank) and $S=\mathbb{E}[g(X,\theta_0)g(X,\theta_0)^\top]$. The asymptotic variance is minimised (in the positive semidefinite order) by $W=S^{-1}$, giving $(G^\top S^{-1}G)^{-1}$.

Proof sketch. The first-order condition is $\hat G^\top\hat W\bar g_n(\hat\theta)=0$ with $\hat G=\partial\bar g_n/\partial\theta^\top$. A mean-value expansion $\bar g_n(\hat\theta)=\bar g_n(\theta_0)+\bar G(\hat\theta-\theta_0)$ gives $\sqrt{n}(\hat\theta-\theta_0)=-(\hat G^\top\hat W\bar G)^{-1}\hat G^\top\hat W\sqrt{n}\,\bar g_n(\theta_0)$. By the CLT $\sqrt{n}\,\bar g_n(\theta_0)\xrightarrow{d}\mathcal{N}(0,S)$, and $\hat G,\bar G\xrightarrow{p}G$; Slutsky's theorem gives the sandwich form. For optimality, with $A=(G^\top WG)^{-1}G^\top W$ and $B=(G^\top S^{-1}G)^{-1}G^\top S^{-1}$ one checks $(A-B)SB^\top=0$, so $ASA^\top=BSB^\top+(A-B)S(A-B)^\top\geq BSB^\top$. □

Since S depends on θ_0, efficient GMM is implemented in *two steps*: (i) compute a preliminary consistent $\hat\theta^{(1)}$ with some W (e.g. $W=I$ or the 2SLS weight below); (ii) estimate $\hat S=n^{-1}\sum_i g(X_i,\hat\theta^{(1)})g(X_i,\hat\theta^{(1)})^\top$ and re-estimate with $\hat W=\hat S^{-1}$. Iterating to convergence, or minimising the *continuously updated* criterion, gives estimators with the same first-order asymptotics but different finite-sample behaviour.

Theorem 7.14 (Hansen's J-test of over-identifying restrictions)**.** *If (7.2) holds and $\hat\theta$ is efficient two-step GMM, then $J_n=n\,\bar g_n(\hat\theta)^\top\hat S^{-1}\bar g_n(\hat\theta)\xrightarrow{d}\chi^2_{L-p}$.*

A large J_n indicates that *not all* moment conditions can hold simultaneously. The test cannot tell which condition fails, and it has no power against a violation that biases all just-identified sub-estimators in the same way.

7.2.3 OLS and instrumental variables as moment estimators

Consider the linear model $y_i=\mathbf{x}_i^\top\boldsymbol\beta+u_i$ with $\mathbf{x}_i\in\mathbb{R}^p$.

- **OLS** uses the p moment conditions $\mathbb{E}[\mathbf{x}_iu_i]=\mathbf{0}$. The sample analogue $n^{-1}\sum_i\mathbf{x}_i(y_i-\mathbf{x}_i^\top\boldsymbol\beta)=\mathbf{0}$ is the normal equations, so $\hat{\boldsymbol\beta}_{\mathrm{OLS}}=(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ is a just-identified MoM estimator.
- If some regressors are *endogenous*, $\mathbb{E}[\mathbf{x}_iu_i]\neq\mathbf{0}$, but we have $L\geq p$ **instruments** $\mathbf{z}_i$ with $\mathbb{E}[\mathbf{z}_iu_i]=\mathbf{0}$ (exogeneity) and $\operatorname{rank}\mathbb{E}[\mathbf{z}_i\mathbf{x}_i^\top]=p$ (relevance), GMM uses $g_i(\boldsymbol\beta)=\mathbf{z}_i(y_i-\mathbf{x}_i^\top\boldsymbol\beta)$.

Because g is linear in $\boldsymbol\beta$, the GMM criterion is quadratic and the minimiser is explicit:

$$\hat{\boldsymbol\beta}(W)=\big(\mathbf{X}^\top\mathbf{Z}\,W\,\mathbf{Z}^\top\mathbf{X}\big)^{-1}\mathbf{X}^\top\mathbf{Z}\,W\,\mathbf{Z}^\top\mathbf{y}. \tag{7.3}$$

With $L = p$ this reduces to the *IV estimator* $(\mathbf{Z}^\top\mathbf{X})^{-1}\mathbf{Z}^\top\mathbf{y}$ for any W. With $W = (\mathbf{Z}^\top\mathbf{Z}/n)^{-1}$ it is *two-stage least squares* (2SLS), which is efficient GMM under conditional homoskedasticity ($S = \sigma^2\mathbb{E}[\mathbf{z}\mathbf{z}^\top]$). Under heteroskedasticity, two-step GMM with $\hat{S} = n^{-1}\sum_i \hat{u}_i^2\mathbf{z}_i\mathbf{z}_i^\top$ is (weakly) more efficient, and the robust sandwich of Theorem 7.13 must be used for 2SLS standard errors.

A simulated returns-to-schooling study. We simulate a labour-force survey in which unobserved *ability* raises both schooling and log wages, so that schooling is endogenous and OLS is biased upwards. Two instruments shift schooling but are excluded from the wage equation: distance to the nearest secondary school (in the spirit of Card, 1995) and exposure to a school-building programme (in the spirit of Duflo, 2001). The wage error is heteroskedastic. The true return is 0.08 log points per year. A third candidate instrument, a household asset index, is correlated with ability and therefore *invalid*.

Listing 7.4: OLS, IV, 2SLS and two-step GMM for returns to schooling ch07/gmm_iv_schooling.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Returns to schooling with endogenous schooling: OLS, 2SLS and two-step GMM.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
n = 3000
# Simulated labour-force survey (NOT real data)
ability = rng.normal(size=n)                    # unobserved by the analyst
dist = rng.uniform(0, 10, size=n)               # km to nearest secondary school
prog = rng.binomial(1, 0.4, size=n)             # exposed to school-building drive
v = rng.normal(size=n)
school = 9 - 0.25 * dist + 1.5 * prog + 1.0 * ability + v
u = rng.normal(size=n) * (0.3 + 0.1 * np.abs(school - 9))   # heteroskedastic
logw = 1.0 + 0.08 * school + 0.25 * ability + u             # true return 0.08

def gmm(y, X, Z, W):
    """Linear GMM: minimise gbar' W gbar with g_i = z_i (y_i - x_i'b)."""
    A = X.T @ Z @ W @ Z.T @ X
    return np.linalg.solve(A, X.T @ Z @ W @ Z.T @ y)

def robust_gmm_cov(y, X, Z, W, b):
    e = y - X @ b
    S = (Z * e[:, None] ** 2).T @ Z / n          # (1/n) sum e_i^2 z_i z_i'
    G = Z.T @ X / n
    M = np.linalg.inv(G.T @ W @ G)
    return M @ G.T @ W @ S @ W @ G @ M / n, S

def j_test(y, X, Z, b, S):
    gbar = Z.T @ (y - X @ b) / n
    J = n * gbar @ np.linalg.solve(S, gbar)
    df = Z.shape[1] - X.shape[1]
    return J, df, stats.chi2.sf(J, df)

one = np.ones(n)
X = np.column_stack([one, school])
Z_ols = X                                       # OLS: instruments = regressors
Z1 = np.column_stack([one, dist])               # just identified IV
Z2 = np.column_stack([one, dist, prog])         # over-identified

```

```python
print(f"Simulated data, n = {n}, true return to schooling = 0.080")
print(f"{'estimator':<26} {'beta_s':>8} {'robust se':>10}")
rows = [("OLS", Z_ols, None), ("IV (distance)", Z1, None),
        ("2SLS (distance, programme)", Z2, None)]
for name, Z, _ in rows:
    W = np.linalg.inv(Z.T @ Z / n)              # 2SLS weighting
    b = gmm(logw, X, Z, W)
    V, S = robust_gmm_cov(logw, X, Z, W, b)
    print(f"{name:<26} {b[1]:8.4f} {np.sqrt(V[1, 1]):10.4f}")

# Two-step efficient GMM: W = S^{-1} from first-step residuals
b1 = gmm(logw, X, Z2, np.linalg.inv(Z2.T @ Z2 / n))
_, S1 = robust_gmm_cov(logw, X, Z2, np.eye(3), b1)
W2 = np.linalg.inv(S1)
b2 = gmm(logw, X, Z2, W2)
V2, S2 = robust_gmm_cov(logw, X, Z2, W2, b2)
print(f"{'two-step efficient GMM':<26} {b2[1]:8.4f} {np.sqrt(V2[1, 1]):10.4f}")
J, df, p = j_test(logw, X, Z2, b2, S1)
print(f"Hansen J (valid instruments)   = {J:6.3f}, df = {df}, p = {p:.3f}")

# An invalid 'instrument': household asset index correlated with ability
assets = rng.normal(size=n) + 0.5 * ability
Z3 = np.column_stack([one, dist, assets])
b1 = gmm(logw, X, Z3, np.linalg.inv(Z3.T @ Z3 / n))
_, S1 = robust_gmm_cov(logw, X, Z3, np.eye(3), b1)
b3 = gmm(logw, X, Z3, np.linalg.inv(S1))
J, df, p = j_test(logw, X, Z3, b3, S1)
print(f"Two-step GMM with assets: beta_s = {b3[1]:.4f}")
print(f"Hansen J (invalid instrument)  = {J:6.2f}, df = {df}, p = {p:.2g}")

# First-stage strength
coef, *_ = np.linalg.lstsq(Z2, school, rcond=None)
rss1 = np.sum((school - Z2 @ coef) ** 2)
rss0 = np.sum((school - school.mean()) ** 2)
Fstat = ((rss0 - rss1) / 2) / (rss1 / (n - 3))
print(f"First-stage F (distance, programme) = {Fstat:.1f}")
```

Output

```
Simulated data, n = 3000, true return to schooling = 0.080
estimator                    beta_s  robust se
OLS                          0.1525     0.0063
IV (distance)                0.0686     0.0134
2SLS (distance, programme)   0.0704     0.0092
two-step efficient GMM       0.0705     0.0092
Hansen J (valid instruments)   =  0.040, df = 1, p = 0.841
Two-step GMM with assets: beta_s = 0.1349
Hansen J (invalid instrument)  = 121.83, df = 1, p = 2.5e-28
First-stage F (distance, programme) = 890.8
```

OLS overstates the return by almost a factor of two (0.153), because the ability bias $\mathrm{Cov}(s, 0.25a)/\mathrm{Var}(s)$ is large. All instrument-based estimates are within about one standard error of 0.08. Adding the programme instrument to distance reduces the standard error from 0.013 to 0.009; two-step GMM gives practically the same answer as 2SLS here, because the heteroskedasticity is mild relative to the instrument strength. The J statistic (0.04, $p = 0.84$) gives

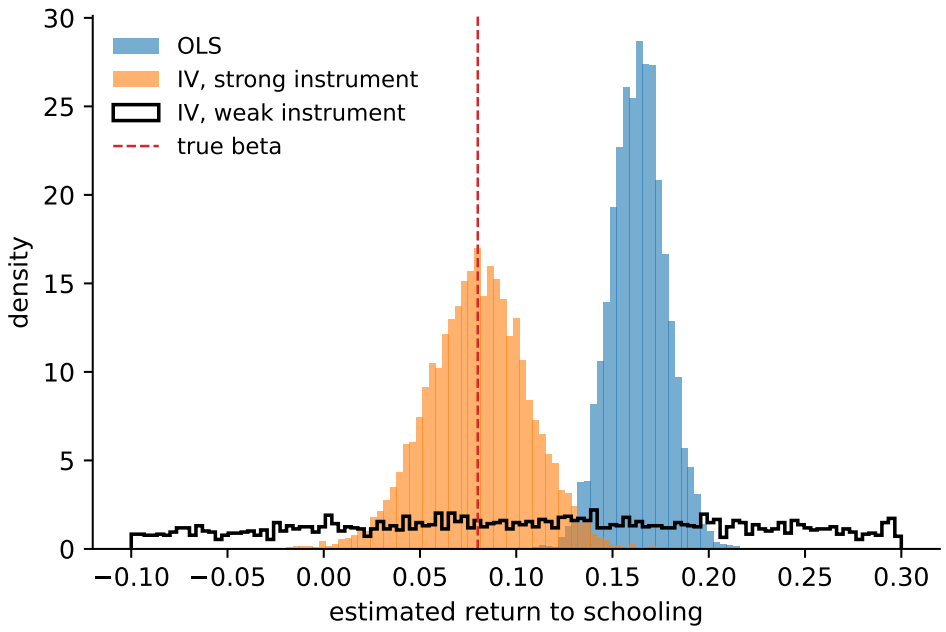


Figure 7.2: Monte Carlo sampling distributions ($R = 5000$, $n = 500$) of OLS and IV estimates of a return to schooling of 0.08. OLS is precise but centred at the wrong value. IV with a strong instrument is centred correctly but more dispersed. With a weak instrument (first-stage slope 0.08) the IV distribution is extremely dispersed and median-biased towards OLS (Listing 7.5).

no evidence against the over-identifying restriction. With the invalid asset index the estimate is pulled back towards OLS (0.135) and the J-test rejects emphatically ($J = 122$).

Listing 7.5: Sampling distributions of OLS and IV (plotting code omitted) `ch07/iv_montecarlo.py`

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Sampling distributions of OLS and IV under endogeneity, strong vs weak instrument.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
R, n, beta = 5000, 500, 0.08

def simulate(pi):
    """Return OLS and IV slope estimates for R samples; pi = first-stage slope."""
    a = rng.normal(size=(R, n))
    z = rng.normal(size=(R, n))
    s = pi * z + a + rng.normal(size=(R, n))
    y = beta * s + 0.25 * a + 0.5 * rng.normal(size=(R, n))
    sc = s - s.mean(axis=1, keepdims=True)
    yc = y - y.mean(axis=1, keepdims=True)
    zc = z - z.mean(axis=1, keepdims=True)
    ols = (sc * yc).sum(1) / (sc * sc).sum(1)
    iv = (zc * yc).sum(1) / (zc * sc).sum(1)
    return ols, iv

print(f"R = {R} samples of n = {n}; true beta = {beta}")
print(f"{'design':>8} {'estimator':>9} {'median':>8} {'IQR':>8}")
res = {}
```

```
for label, pi in (("strong", 1.0), ("weak", 0.08)):
    ols, iv = simulate(pi)
    res[label] = (ols, iv)
    for name, e in (("OLS", ols), ("IV", iv)):
        q1, q2, q3 = np.percentile(e, [25, 50, 75])
        print(f"{label:>8} {name:>9} {q2:8.4f} {q3 - q1:8.4f}")
```

Output

```
R = 5000 samples of n = 500; true beta = 0.08
  design estimator   median      IQR
  strong       OLS   0.1635   0.0188
  strong        IV   0.0802   0.0343
    weak       OLS   0.2043   0.0231
    weak        IV   0.1091   0.4018
```

Figure 7.2 displays the classic trade-off: consistency is bought with variance. With a strong instrument the IV interquartile range is about 1.8 times that of OLS. With a weak instrument the IQR is twenty times larger and the median (0.109) is pulled towards the OLS probability limit.

△ Caution: Weak instruments

The asymptotic normal approximation for IV and GMM can be very poor when instruments are only weakly correlated with the endogenous regressor: estimates are biased towards OLS, standard errors are unreliable and tests over-reject (Staiger and Stock, 1997). Always report the first-stage F statistic (a common rule of thumb asks for $F > 10$ with one endogenous regressor; the simulated study above has $F \approx 891$), and consider weak-instrument-robust inference (e.g. Anderson–Rubin tests) when it is small.

⋆ Key Concept: Moment conditions

GMM needs only $\mathbb{E}[g(X, \theta_0)] = 0$, not a full likelihood. Its estimator minimises $\bar{g}_n^\top W \bar{g}_n$; the efficient weight is $W = S^{-1}$, giving asymptotic variance $(G^\top S^{-1} G)^{-1}$. OLS ($\mathbf{z} = \mathbf{x}$), IV ($L = p$) and 2SLS ($W \propto (\mathbf{Z}^\top \mathbf{Z})^{-1}$) are special cases. Over-identification allows the J-test, which checks the *compatibility* of the moment conditions, not their truth.

Policy Lens: Returns to education and school construction

Whether expanding schooling raises earnings is central to education budgets. Simple OLS of log wages on years of schooling confounds the effect of schooling with that of ability and family background. Instrumental-variable designs exploit sources of variation in schooling that plausibly do not affect wages directly: distance to school (Card, 1995), the quarter of birth combined with compulsory schooling laws (Angrist and Krueger, 1991), or the regional and cohort intensity of a school-construction programme (Duflo, 2001). The estimated effect is then a *local* average for those whose schooling responds to the instrument (Angrist and Pischke, 2009) — often exactly the population a policy expansion would reach. Every such design rests on an exclusion restriction that must be argued, not merely tested.

Historical Note: Hansen and GMM

Lars Peter Hansen's 1982 *Econometrica* paper "Large sample properties of generalized method of moments estimators" (Hansen, 1982) unified many existing estimators (least squares, instrumental

variables, two-stage least squares) under one framework and provided the asymptotic theory and the test of over-identifying restrictions. GMM became the standard tool for estimating rational-expectations models of asset prices and consumption from Euler equations, where theory specifies orthogonality conditions but not the distribution of shocks. Hansen shared the 2013 Nobel memorial prize in economic sciences with Eugene Fama and Robert Shiller for their empirical analysis of asset prices.

Exercises

7.2.1 Theory Derive the MoM estimators of Example 7.11. For the gamma family, use Proposition 7.10 to derive the asymptotic variance of $\tilde{k}$ in terms of k only. (Hint: you need the third and fourth moments of the gamma distribution.)

7.2.2 Theory Show that (7.3) minimises the GMM criterion and that it does not depend on W when $L = p$ and $\mathbf{Z}^\top\mathbf{X}$ is invertible. Show that with $W = (\mathbf{Z}^\top\mathbf{Z})^{-1}$ it equals OLS of $\mathbf{y}$ on the first-stage fitted values $\hat{\mathbf{X}} = \mathbf{Z}(\mathbf{Z}^\top\mathbf{Z})^{-1}\mathbf{Z}^\top\mathbf{X}$.

7.2.3 Theory In the model $y = \beta s + \gamma a + u$, $s = \pi z + a + v$ (all variables mean zero, a, z, u, v independent with unit variances except $\operatorname{Var} u = \sigma_u^2$), compute the probability limits of OLS and IV. Check your answer against the "strong" row of Listing 7.5.

7.2.4 Python Simulate the null distribution of the J statistic in the valid-instrument design of Listing 7.4 using $R = 2000$ replications with $n = 200$ and $n = 3000$. Compare its rejection rate at 5% with the nominal size. Repeat with the invalid asset instrument to estimate power.

7.2.5 ★ Graduate *Weak-instrument asymptotics.* In Listing 7.5 set the first-stage coefficient to $\pi = c/\sqrt{n}$ and show by simulation that the distribution of $\hat{\beta}_{IV}$ does not collapse as n grows. Relate the result to the Staiger and Stock (1997) framework and explain why the usual delta-method argument fails.

7.2.6 Policy A ministry evaluates a school-feeding programme with district-level variation in roll-out timing as an instrument for school attendance. List the moment conditions implied, explain what the exclusion restriction requires in words, and give two plausible channels by which it could fail. Could a J-test detect them?

7.3 Maximum Likelihood Estimation

7.3.1 Likelihood, log-likelihood and the score

Definition 7.15 (Likelihood and MLE)**.** Let $X_1, \dots, X_n$ be independent with density or mass function $f(x;\theta)$. The *likelihood* is the function of θ

$$L_n(\theta) = \prod_{i=1}^{n} f(X_i;\theta), \qquad \ell_n(\theta) = \log L_n(\theta) = \sum_{i=1}^{n} \log f(X_i;\theta).$$

A *maximum likelihood estimator* (MLE) is any $\hat{\theta}_n \in \arg\max_{\theta\in\Theta} \ell_n(\theta)$. For a regular model with $\hat{\theta}_n$ in the interior of Θ, it solves the *score equations*

$$U_n(\hat{\theta}_n) = \nabla_\theta \ell_n(\hat{\theta}_n) = \sum_{i=1}^{n} s(X_i;\hat{\theta}_n) = \mathbf{0}, \qquad s(x;\theta) = \nabla_\theta \log f(x;\theta). \tag{7.4}$$

The likelihood is *not* a probability distribution over θ; it measures the relative support the data give to different parameter values. Only ratios $L_n(\theta_1)/L_n(\theta_2)$ are meaningful, so multiplicative constants not involving θ may be dropped. Note also that the score equations (7.4) are *moment conditions*: since $\mathbb{E}_{\theta_0}[s(X;\theta_0)] = 0$ (Theorem 7.21), the MLE is the just-identified MoM estimator based on the score.

7.3.2 Closed-form examples

Example 7.16 (Four classical MLEs). 1. *Bernoulli* ($x \in \{0,1\}$): $\ell(p) = \sum x_i \log p + (n - \sum x_i)\log(1-p)$; $\ell'(p) = \sum x_i/p - (n-\sum x_i)/(1-p) = 0$ gives $\hat p = \bar x$.

2. *Poisson*: $\ell(\lambda) = \sum x_i \log\lambda - n\lambda - \sum \log x_i!$; $\ell'(\lambda) = \sum x_i/\lambda - n = 0$ gives $\hat\lambda = \bar x$.

3. *Normal*: $\ell(\mu,\sigma) = -n\log\sigma - \sum (x_i-\mu)^2/(2\sigma^2)$+const; the score equations give $\hat\mu = \bar x$ and $\hat\sigma^2 = n^{-1}\sum (x_i - \bar x)^2$ (the biased divisor of Example 7.5).

4. *Exponential* with rate λ: $\ell(\lambda) = n\log\lambda - \lambda\sum x_i$; $\hat\lambda = 1/\bar x$.

In each case the second derivative is negative at the root, which is therefore the global maximum (the log-likelihoods are concave in these parametrisations).

Theorem 7.17 (Invariance). *If $\hat\theta$ is an MLE of θ and $\psi = g(\theta)$ for any function g, then $g(\hat\theta)$ is an MLE of ψ, where the likelihood of ψ is defined as the* profile $L^*(\psi) = \sup_{\theta: g(\theta)=\psi} L(\theta)$.

Proof. For every ψ, $L^*(\psi) \le \sup_\theta L(\theta) = L(\hat\theta) = L^*(g(\hat\theta))$, the last equality because $\hat\theta$ belongs to the set $\{\theta : g(\theta) = g(\hat\theta)\}$. □

Invariance is very convenient for policy quantities: the MLE of the probability that a household makes no clinic visits under a Poisson model is $e^{-\hat\lambda}$; the MLE of the median waiting time under an exponential model is $\log 2/\hat\lambda$; the MLE of the odds of take-up is $\hat p/(1-\hat p)$. Listing 7.6 checks the closed forms against a general-purpose optimiser and illustrates invariance.

Listing 7.6: Closed-form and numerical MLEs; invariance ch07/mle_closed_form.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Closed-form MLEs checked against numerical maximisation; the invariance property.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize, stats

rng = np.random.default_rng(803)
n = 250
takeup = rng.binomial(1, 0.35, size=n)          # programme take-up (0/1)
visits = rng.poisson(2.4, size=n)               # clinic visits per household
income = rng.normal(5.0, 1.2, size=n)           # log income
wait = rng.exponential(1 / 0.5, size=n)         # waiting time, rate 0.5


def num_mle(negll, x0, bounds):
    return optimize.minimize(negll, x0, bounds=bounds, method="L-BFGS-B").x


norm_fit = num_mle(lambda t: -stats.norm.logpdf(income, t[0], t[1]).sum(),
                   [0.0, 1.0], [(None, None), (1e-6, None)])
fits = [
    ("Bernoulli p", takeup.mean(),
     num_mle(lambda t: -stats.bernoulli.logpmf(takeup, t[0]).sum(), [0.5],
```

```
            [(1e-6, 1 - 1e-6)])[0]),
    ("Poisson lambda", visits.mean(),
     num_mle(lambda t: -stats.poisson.logpmf(visits, t[0]).sum(), [1.0],
            [(1e-6, None)])[0]),
    ("Normal mu", income.mean(), norm_fit[0]),
    ("Normal sigma", income.std(ddof=0), norm_fit[1]),
    ("Exponential rate", 1 / wait.mean(),
     num_mle(lambda t: -stats.expon.logpdf(wait, scale=1 / t[0]).sum(),
            [1.0], [(1e-6, None)])[0]),
]
print(f"{'parameter':<17} {'closed form':>12} {'numerical':>12} {'diff':>10}")
for name, cf, nu in fits:
    print(f"{name:<17} {cf:12.6f} {nu:12.6f} {cf - nu:10.2e}")

# Invariance: the MLE of g(theta) is g(theta_hat)
lam = visits.mean()
rate = 1 / wait.mean()
p = takeup.mean()
print("\nInvariance of the MLE")
print(f"  P(no visits)  : exp(-lambda_hat) = {np.exp(-lam):.4f}"
      f"   (sample share of zeros = {np.mean(visits == 0):.4f})")
print(f"  median wait   : log(2)/rate_hat  = {np.log(2) / rate:.4f}"
      f"   (sample median = {np.median(wait):.4f})")
print(f"  odds of uptake: p/(1-p)          = {p / (1 - p):.4f}")
```

Output

```
parameter          closed form    numerical       diff
Bernoulli p           0.340000     0.340000   3.30e-09
Poisson lambda        2.376000     2.376000   2.47e-08
Normal mu             5.064261     5.064261   1.34e-07
Normal sigma          1.145504     1.145504  -2.79e-08
Exponential rate      0.524303     0.524304  -2.35e-08

Invariance of the MLE
  P(no visits)  : exp(-lambda_hat) = 0.0929   (sample share of zeros = 0.1000)
  median wait   : log(2)/rate_hat  = 1.3220   (sample median = 1.3404)
  odds of uptake: p/(1-p)          = 0.5152
```

The numerical and closed-form estimates agree to about 10^{-7}, the tolerance of the optimiser. Note the comparison of model-based and purely empirical estimates: the Poisson model implies 9.3% zeros, the sample has 10.0%; such comparisons are simple but useful model checks.

7.3.3 Numerical MLE

Outside simple families the score equations have no closed-form solution and we maximise ℓ_n numerically, i.e. *minimise* $-\ell_n$ with `scipy.optimize.minimize` (see Chapter 8 for the algorithms). Good practice:

1. **Reparametrise to remove constraints**: optimise over $\log \sigma$, $\log k$ or $\operatorname{logit}(p)$ rather than imposing bounds. By invariance the MLE is unchanged.
2. **Supply the analytic gradient** (`jac=`) when it is cheap; it improves both speed and accuracy. Check it against finite differences once.
3. **Use numerically stable formulas**, e.g. `np.logaddexp(0, eta)` for $\log(1+e^{\eta})$ and `scipy.stats`

↪ `.norm.logcdf` instead of `np.log(norm.cdf(...))`.

4. **Start well** (MoM estimates, OLS on a transformed outcome) and check convergence flags, gradient norms and, ideally, several starting values.

Example 7.18 (Gamma MLE). For Gamma(k, ϑ), $\ell(k, \vartheta) = (k-1)\sum \log x_i - \sum x_i/\vartheta - n \log \Gamma(k) - nk \log \vartheta$. The score equations are

$$\frac{\partial \ell}{\partial \vartheta} = \frac{\sum x_i}{\vartheta^2} - \frac{nk}{\vartheta} = 0 \;\Rightarrow\; k\vartheta = \bar{x}, \qquad \frac{\partial \ell}{\partial k} = \sum \log x_i - n\psi(k) - n \log \vartheta = 0,$$

with $\psi = \Gamma'/\Gamma$ the digamma function. Substituting $\vartheta = \bar{x}/k$ leaves the one-dimensional equation $\log k - \psi(k) = \log \bar{x} - \overline{\log x}$, which has no closed-form solution. The MLE thus matches the mean exactly (like MoM) but uses the mean of $\log x$ instead of the variance.

Listing 7.7: Numerical gamma MLE in log-parameters (plotting code omitted)
ch07/mle_gamma_numeric.py

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Numerical MLE for the gamma distribution: log-parametrisation, MoM start, contours.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize, special, stats

rng = np.random.default_rng(803)
x = rng.gamma(2.5, 1.6, size=300)       # simulated household consumption
n, sx, slx = x.size, x.sum(), np.log(x).sum()


def negll(t):
    """Negative gamma log-likelihood in (log k, log theta)."""
    k, th = np.exp(t)
    return -((k - 1) * slx - sx / th - n * special.gammaln(k)
             - n * k * np.log(th))


def grad(t):
    k, th = np.exp(t)
    dk = slx - n * special.digamma(k) - n * np.log(th)
    dth = sx / th**2 - n * k / th
    return -np.array([dk * k, dth * th])       # chain rule for log-params


m, v = x.mean(), x.var()
start = np.log([m**2 / v, v / m])              # method-of-moments start
res = optimize.minimize(negll, start, jac=grad, method="BFGS")
k_hat, th_hat = np.exp(res.x)
print(f"MoM start       : k = {m**2 / v:.4f}, theta = {v / m:.4f}")
print(f"MLE (BFGS)      : k = {k_hat:.4f}, theta = {th_hat:.4f}")
print(f"  converged = {res.success}, iterations = {res.nit},"
      f" max |grad| = {np.abs(res.jac).max():.1e}")
kf, _, thf = stats.gamma.fit(x, floc=0)
print(f"scipy gamma.fit : k = {kf:.4f}, theta = {thf:.4f}")
print(f"Score eq. check : k*theta = {k_hat * th_hat:.4f},"
      f" mean(x) = {m:.4f}")
```

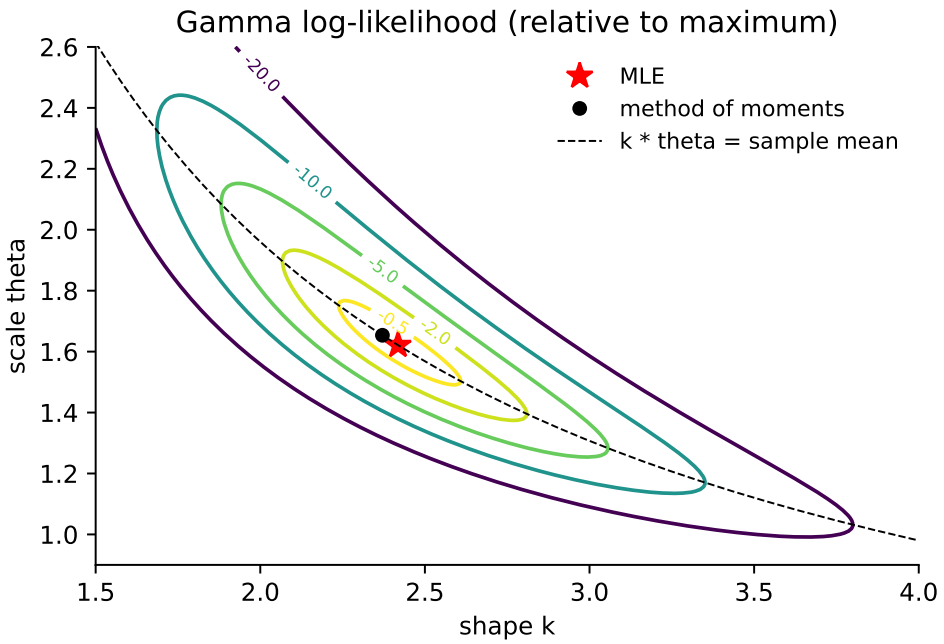


Figure 7.3: Contours of the gamma log-likelihood (relative to its maximum) for $n = 300$ simulated observations, with the MLE, the method-of-moments estimate and the ridge $k\vartheta = \bar{x}$ (Listing 7.7).

Output

```
MoM start       : k = 2.3710, theta = 1.6537
MLE (BFGS)      : k = 2.4185, theta = 1.6212
  converged = True, iterations = 6, max |grad| = 7.2e-07
scipy gamma.fit : k = 2.4185, theta = 1.6212
Score eq. check : k*theta = 3.9208, mean(x) = 3.9208
```

BFGS converges in six iterations from the MoM start and agrees with `scipy.stats.gamma.fit`. Figure 7.3 shows the long, curved ridge of the log-likelihood along $k\vartheta = \bar{x}$: the shape and scale are strongly negatively correlated, which is why a parametrisation in (mean, shape) is often preferred in regression models.

7.3.4 Likelihood-based regression: logit, probit and Poisson

Most applied ML is *conditional*: we model $f(y_i \mid \mathbf{x}_i; \boldsymbol{\beta})$ and maximise $\sum_i \log f(y_i \mid \mathbf{x}_i; \boldsymbol{\beta})$, treating the regressors as given.

Binary outcomes. For $y_i \in \{0, 1\}$ let $\mathbb{P}(y_i = 1 \mid \mathbf{x}_i) = F(\eta_i)$ with linear index $\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$ and a distribution function F. Then

$$\ell(\boldsymbol{\beta}) = \sum_{i} \big\{ y_i \log F(\eta_i) + (1 - y_i) \log\big(1 - F(\eta_i)\big) \big\}.$$

The *logit* model uses $F(\eta) = 1/(1 + e^{-\eta})$, for which the log-likelihood simplifies to $\sum_i \{y_i \eta_i - \log(1 + e^{\eta_i})\}$, with

$$U(\boldsymbol{\beta}) = \mathbf{X}^\top (\mathbf{y} - \mathbf{p}), \qquad \nabla^2 \ell(\boldsymbol{\beta}) = -\mathbf{X}^\top \operatorname{diag}\{p_i(1 - p_i)\}\mathbf{X}, \qquad p_i = F(\eta_i).$$

The Hessian is negative definite whenever $\mathbf{X}$ has full rank, so ℓ is strictly concave and any interior stationary point is the unique global maximum. The *probit* model uses $F = \Phi$, the

standard normal distribution function; it corresponds to a latent utility $y^{*} = \eta + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$, $y = \mathbf{1}\{y^{*} > 0\}$. Since the logistic distribution has standard deviation $\pi/\sqrt{3} \approx 1.8$, logit coefficients are roughly 1.6–1.8 times probit coefficients; *marginal effects* $\partial \mathbb{P}(y=1 \mid \mathbf{x})/\partial x_j = F'(\eta)\beta_j$ are much more comparable.

Count outcomes with exposure. If y_i counts events (clinic visits, crimes, loan defaults) over an exposure E_i (population, person-years, number of loans), the Poisson regression model with *offset* $\log E_i$ is

$$y_i \mid \mathbf{x}_i \sim \text{Poisson}(\mu_i), \qquad \log \mu_i = \log E_i + \mathbf{x}_i^{\top}\boldsymbol{\beta},$$

so that $\exp(\beta_j)$ is a *rate ratio*: the multiplicative change in events per unit of exposure. The log-likelihood $\sum_i \{y_i \log \mu_i - \mu_i\}$ is concave in $\boldsymbol{\beta}$, with score $\mathbf{X}^{\top}(\mathbf{y} - \boldsymbol{\mu})$.

Listing 7.8: Logit, probit and Poisson regression by numerical ML ch07/mle_glm.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Numerical MLE: logit and probit take-up models; Poisson demand with exposure.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize, stats, special
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(803)
n = 2000
# Simulated programme take-up survey (NOT real data)
income = rng.normal(0, 1, n)                    # standardised log income
dist = rng.uniform(0, 5, n)                     # km to enrolment office
female = rng.binomial(1, 0.5, n)                # female-headed household
X = np.column_stack([np.ones(n), income, dist, female])
b_true = np.array([0.5, -0.6, -0.4, 0.5])
y = rng.binomial(1, special.expit(X @ b_true))


def logit_nll(b):
    eta = X @ b
    return np.sum(np.logaddexp(0, eta) - y * eta)   # stable log(1 + e^eta)


def logit_grad(b):
    return X.T @ (special.expit(X @ b) - y)


def probit_nll(b):
    eta = X @ b
    return -np.sum(y * stats.norm.logcdf(eta)
                   + (1 - y) * stats.norm.logcdf(-eta))


b0 = np.zeros(4)
r_logit = optimize.minimize(logit_nll, b0, jac=logit_grad, method="BFGS")
r_probit = optimize.minimize(probit_nll, b0, method="BFGS")
sk = LogisticRegression(penalty="none", max_iter=1000).fit(X[:, 1:], y)
sk_b = np.r_[sk.intercept_, sk.coef_.ravel()]
names = ["const", "income", "distance", "female"]
print(f"Take-up model (simulated), n = {n}")
print(f"{'':>9} {'true':>7} {'logit':>8} {'sklearn':>8} {'probit':>8}"
      f" {'1.6*probit':>10}")
```

```
for j, nm in enumerate(names):
    print(f"{nm:>9} {b_true[j]:7.2f} {r_logit.x[j]:8.4f} {sk_b[j]:8.4f}"
          f" {r_probit.x[j]:8.4f} {1.6 * r_probit.x[j]:10.4f}")
print(f"log-lik: logit = {-r_logit.fun:.2f}, probit = {-r_probit.fun:.2f}")
# Average marginal effect of distance on the take-up probability
p = special.expit(X @ r_logit.x)
ame_l = np.mean(p * (1 - p)) * r_logit.x[2]
ame_p = np.mean(stats.norm.pdf(X @ r_probit.x)) * r_probit.x[2]
print(f"AME of +1 km distance: logit {ame_l:.4f}, probit {ame_p:.4f}")

# Poisson regression with exposure: clinic visits in m districts
m = 120
pop = rng.integers(5, 80, m) * 1000              # district population
urban = rng.binomial(1, 0.4, m)
fee = rng.uniform(0, 2, m)                       # user fee (simulated units)
Xd = np.column_stack([np.ones(m), urban, fee])
g_true = np.array([-1.0, 0.3, -0.35])            # log visits per person-year
visits = rng.poisson(pop * np.exp(Xd @ g_true))
off = np.log(pop)


def pois_nll(g):
    eta = off + Xd @ g
    return np.sum(np.exp(eta) - visits * eta)


def pois_grad(g):
    return Xd.T @ (np.exp(off + Xd @ g) - visits)


r_p = optimize.minimize(pois_nll, np.zeros(3), jac=pois_grad, method="BFGS")
print(f"\nPoisson demand with log(population) offset, m = {m} districts")
for nm, t, e in zip(["const", "urban", "fee"], g_true, r_p.x):
    print(f"{nm:>9}: true {t:6.2f}, MLE {e:8.4f}, rate ratio {np.exp(e):.4f}")
```

Output

```
Take-up model (simulated), n = 2000
             true    logit  sklearn   probit 1.6*probit
    const    0.50   0.5774   0.5774   0.3489     0.5583
   income   -0.60  -0.6016  -0.6016  -0.3647    -0.5835
 distance   -0.40  -0.4478  -0.4478  -0.2721    -0.4353
   female    0.50   0.4792   0.4792   0.2957     0.4731
log-lik: logit = -1209.26, probit = -1209.18
AME of +1 km distance: logit -0.0935, probit -0.0937

Poisson demand with log(population) offset, m = 120 districts
    const: true  -1.00, MLE  -0.9979, rate ratio 0.3687
    urban: true   0.30, MLE   0.2982, rate ratio 1.3474
      fee: true  -0.35, MLE  -0.3517, rate ratio 0.7035
```

Our hand-coded logit agrees with scikit-learn's unpenalised `LogisticRegression` to four decimals. The probit coefficients are about 1/1.6 of the logit ones, yet the log-likelihoods are almost identical and the average marginal effect (AME) of an extra kilometre is -0.094 in both: each additional kilometre to the enrolment office lowers the take-up probability by about nine

percentage points. In the Poisson model the fee coefficient implies a rate ratio of 0.70: in this simulated design one more unit of user fee is associated with 30% fewer visits per person.

Policy Lens: Take-up and demand models

Low take-up of social programmes (cash transfers, health insurance, fertiliser subsidies) is a recurrent policy problem. A logit or probit model of take-up on distance, information, income and household composition identifies which barriers are associated with non-participation; AMEs translate coefficients into percentage-point changes that can be compared with the cost of, say, mobile enrolment units. Similarly, Poisson regression with population exposure is the natural model for *demand for a public service* across districts — clinic visits, court cases, water-point use — and its rate ratios answer questions such as "by how much would visits fall if a user fee were introduced?" These are associations unless the regressors are as good as randomly assigned; causal claims need the designs of Section 7.2.

7.3.5 When maximum likelihood fails: separation

The MLE need not exist. In a logit model, if some linear combination of the regressors perfectly separates the $y_i = 1$ and $y_i = 0$ observations, the likelihood increases monotonically along that direction towards its supremum 1, which is never attained (Albert and Anderson, 1984).

Listing 7.9: Complete separation in a logit model ch07/mle_separation.py

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Complete separation: the logit MLE does not exist and optimisers drift away.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize, special
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(803)
n = 40
x = np.sort(rng.normal(size=n))
y = (x > 0.1).astype(float)          # perfectly separated outcome
X = np.column_stack([np.ones(n), x])


def nll(b):
    eta = X @ b
    return np.sum(np.logaddexp(0, eta) - y * eta)


def grad(b):
    return X.T @ (special.expit(X @ b) - y)


print("Complete separation: y = 1 exactly when x > 0.1")
print(f"{'maxiter':>8} {'b0':>9} {'b1':>9} {'neg loglik':>11} {'success':>8}")
for it in (10, 50, 200, 1000):
    r = optimize.minimize(nll, np.zeros(2), jac=grad, method="BFGS",
                          options={"maxiter": it, "gtol": 1e-12})
    print(f"{it:>8} {r.x[0]:9.2f} {r.x[1]:9.2f} {r.fun:11.2e}"
          f" {str(r.success):>8}")
print(f"implied boundary -b0/b1 at maxiter = 1000: {-r.x[0] / r.x[1]:.4f}")
for C in (1.0, 100.0):
    sk = LogisticRegression(C=C).fit(x[:, None], y)
```

```
    print(f"sklearn L2-penalised, C = {C:>5}: b0 = {sk.intercept_[0]:6.2f},"
          f" b1 = {sk.coef_[0, 0]:6.2f}")
```

Output

```
Complete separation: y = 1 exactly when x > 0.1
 maxiter        b0        b1  neg loglik  success
      10     -1.54     20.50    1.07e+00    False
      50    -75.64    996.36    3.60e-12    False
     200    -79.60   1048.51    9.03e-13     True
    1000    -79.60   1048.51    9.03e-13     True
implied boundary -b0/b1 at maxiter = 1000: 0.0759
sklearn L2-penalised, C =   1.0: b0 =  -0.50, b1 =   2.71
sklearn L2-penalised, C = 100.0: b0 =  -1.10, b1 =  13.84
```

The optimiser drives the slope above 1000, the negative log-likelihood to 10^{-12} and — this is the dangerous part — eventually *reports success*, because the gradient is numerically zero. Only the ratio $-\hat{\beta}_0/\hat{\beta}_1 \approx 0.076$, the location of the separating point, is meaningful. Penalisation (the default in scikit-learn, which is a Gaussian prior; Section 7.6) or Firth's bias-reduced likelihood (Firth, 1993) give finite estimates.

△ Caution: Read the optimiser output critically

A "converged" flag means only that a numerical stopping rule was met. Warning signs of a non-existent or poorly identified MLE include huge coefficients or standard errors, fitted probabilities of exactly 0 or 1, a Hessian that is nearly singular, and estimates that change with the starting value or the tolerance. Separation is common with rare outcomes, many dummy variables or small samples — precisely the situation of many programme pilots.

⋆ Key Concept: Maximum likelihood

The MLE maximises $\ell_n(\theta) = \sum_i \log f(X_i;\theta)$ and, in regular problems, solves the score equations $\sum_i s(X_i;\hat{\theta}) = 0$. It is invariant: the MLE of $g(\theta)$ is $g(\hat{\theta})$. Logit, probit and Poisson regression are MLEs whose log-likelihoods are concave. Numerically: reparametrise, give gradients, use stable formulas, start at MoM values and *always* check for non-existence (separation) and flat directions.

Historical Note: From "absolute criterion" to likelihood

As a Cambridge undergraduate, R. A. Fisher published "On an absolute criterion for fitting frequency curves" (Fisher, 1912), proposing to choose parameters that maximise the joint probability of the observations. He introduced the name *likelihood* in the early 1920s, insisting that it is not a probability, and developed the theory of the method in 1922 and 1925 (Fisher, 1922, 1925). Special cases had appeared earlier, for instance in Gauss's derivation of least squares from the normal error law (Stigler, 2007).

Exercises

7.3.1 Theory Derive the MLEs in Example 7.16, verify that each is a global maximum, and find the MLE of θ for $X_i \overset{\text{iid}}{\sim} \text{Uniform}(0,\theta)$. Why do the score equations fail in the uniform case?

7.3.2 Theory Show that the logit log-likelihood is concave and that the Poisson regression log-likelihood with offset is concave in $\boldsymbol{\beta}$. Show that in the logit model with an intercept,

the MLE satisfies $\sum_i \hat{p}_i = \sum_i y_i$.

7.3.3 Theory For the gamma model, prove that $\log k - \psi(k)$ is strictly decreasing on $(0,\infty)$ from $+\infty$ to 0, and deduce that the MLE exists and is unique whenever the x_i are not all equal. (Hint: Jensen's inequality gives $\log \bar{x} > \overline{\log x}$.)

7.3.4 Python Using $R = 2000$ replications of the gamma design in Listing 7.7, compare the Monte Carlo standard deviations of the MoM and ML estimators of k. Estimate the ARE of MoM relative to ML.

7.3.5 ★ Graduate *Quasi-complete separation.* Construct a data set in which separation holds with ties at the boundary. Show by experiment that some but not all coefficients diverge. Implement Firth's penalised likelihood $\ell(\beta) + \tfrac{1}{2}\log\det I(\beta)$ for the logit model and show that it gives finite estimates.

7.3.6 Policy Using the Poisson model in Listing 7.8, compute the predicted total number of visits across the 120 districts if the user fee were abolished, and the percentage change. Which assumptions does this counterfactual prediction require?

7.4 Fisher Information and Asymptotic Theory

7.4.1 The score and the information identity

Throughout this section we work under the following conditions, which hold for all the models of Section 7.3 except the uniform.

> **Assumption 7.19** (Regularity)**.** (R1) Θ is an open subset of $\mathbb{R}^p$ and θ_0 is identified; (R2) the support $\{x : f(x;\theta) > 0\}$ does not depend on θ; (R3) $\log f(x;\theta)$ is three times continuously differentiable in θ; (R4) differentiation with respect to θ may be interchanged with integration over x (twice) for $\int f(x;\theta)\,dx = 1$; (R5) the information matrix below is finite and positive definite; (R6) the third derivatives are dominated by an integrable function in a neighbourhood of θ_0.

> **Definition 7.20** (Fisher information)**.** The *Fisher information* (per observation) is the variance matrix of the score,
>
> $$I(\theta) = \mathbb{E}_\theta\left[s(X;\theta)s(X;\theta)^\top\right], \qquad s(x;\theta) = \nabla_\theta \log f(x;\theta),$$
>
> and the information in an iid sample of size n is $nI(\theta)$.

Theorem 7.21 (Score identities)**.** *Under Assumption 7.19,* (i) $\mathbb{E}_\theta[s(X;\theta)] = \mathbf{0}$, *and* (ii) $I(\theta) = -\mathbb{E}_\theta\left[\nabla^2_\theta \log f(X;\theta)\right]$ (the information identity).

Proof. (i) Differentiate $\int f(x;\theta)\,dx = 1$ under the integral sign:

$$\mathbf{0} = \int \nabla_\theta f(x;\theta)\,dx = \int \frac{\nabla_\theta f(x;\theta)}{f(x;\theta)}\, f(x;\theta)\,dx = \mathbb{E}_\theta[s(X;\theta)].$$

(ii) Differentiate $\int s(x;\theta) f(x;\theta)\,dx = \mathbf{0}$ once more with respect to $\theta^\top$:

$$\mathbf{0} = \int \nabla^2_\theta \log f(x;\theta)\, f(x;\theta)\,dx + \int s(x;\theta)\,\nabla_\theta f(x;\theta)^\top\,dx = \mathbb{E}_\theta[\nabla^2_\theta \log f] + \mathbb{E}_\theta[s\,s^\top],$$

using $\nabla_\theta f = f\,s$ in the second integral. □

The information identity says that the *curvature* of the expected log-likelihood equals the *variance* of the score. Both views are useful: high information means a sharply peaked log-likelihood and precisely estimable parameters.

Example 7.22 (Information in standard models)**.** Bernoulli: $I(p) = 1/\{p(1-p)\}$. Poisson: $I(\lambda) = 1/\lambda$. Exponential rate: $I(\lambda) = 1/\lambda^2$. Normal (μ, σ): $I = \mathrm{diag}(1/\sigma^2, 2/\sigma^2)$. Cauchy location: $I(\theta) = 1/2$. Logit regression (conditional on $\mathbf{X}$): $\mathcal{I}_n(\beta) = \mathbf{X}^\top \mathrm{diag}\{p_i(1-p_i)\}\mathbf{X}$; Poisson regression: $\mathcal{I}_n(\beta) = \mathbf{X}^\top \mathrm{diag}\{\mu_i\}\mathbf{X}$.

7.4.2 The Cramér–Rao lower bound

Theorem 7.23 (Cramér–Rao)**.** *Under Assumption 7.19 with $p = 1$, let $T = T(X_1, \dots, X_n)$ satisfy $\mathbb{E}_\theta T = \psi(\theta)$ with ψ differentiable and $\frac{d}{d\theta}\int T \prod f = \int T \, \frac{d}{d\theta}\prod f$. Then*

$$\mathrm{Var}_\theta(T) \;\geq\; \frac{\psi'(\theta)^2}{n I(\theta)}.$$

In particular every unbiased estimator of θ has variance at least $1/(nI(\theta))$.

Proof. Let $U = \sum_i s(X_i;\theta)$ be the sample score, so $\mathbb{E}U = 0$ and $\mathrm{Var}\, U = nI(\theta)$ by Theorem 7.21 and independence. Differentiating $\psi(\theta) = \int T\, L_n \, dx$ gives

$$\psi'(\theta) = \int T \, \frac{\partial L_n}{\partial \theta}\, dx = \mathbb{E}_\theta[TU] = \mathrm{Cov}_\theta(T, U),$$

since $\mathbb{E}U = 0$. The Cauchy–Schwarz inequality gives $\psi'(\theta)^2 = \mathrm{Cov}(T,U)^2 \leq \mathrm{Var}(T)\,\mathrm{Var}(U) = \mathrm{Var}(T)\, nI(\theta)$. □

Equality holds if and only if $T - \psi(\theta)$ is a (non-random multiple) of U, which happens exactly in one-parameter exponential families with T the sufficient statistic. In the multi-parameter case, $\mathrm{Var}(T) \succeq \nabla\psi^\top \{nI(\theta)\}^{-1} \nabla\psi$ for an unbiased estimator T of $\psi(\theta) \in \mathbb{R}$.

Example 7.24 (Attaining the bound)**.** For Poisson data $\mathrm{Var}(\bar X) = \lambda/n = 1/(nI(\lambda))$: the sample mean is *efficient*. For exponential data, $\hat\lambda = 1/\bar X$ is biased, $\mathbb{E}\hat\lambda = n\lambda/(n-1)$, and its variance $n^2\lambda^2/\{(n-1)^2(n-2)\}$ exceeds λ^2/n; the bound is attained only asymptotically.

Historical Note: Rao, Cramér and the information inequality

The inequality is named after C. R. Rao, who proved it in a 1945 paper in the *Bulletin of the Calcutta Mathematical Society* (Rao, 1945), and Harald Cramér, who gave it in his 1946 book *Mathematical Methods of Statistics* (Cramér, 1946). It was found independently by Maurice Fréchet (1943) and Georges Darmois (1945), and is sometimes called the Fréchet–Darmois–Cramér–Rao inequality. The same 1945 paper of Rao, written when he was 24, also contains the Rao–Blackwell theorem and proposes using the Fisher information as a Riemannian metric on the parameter space — the seed of information geometry (Section 7.5).

7.4.3 Consistency and asymptotic normality of the MLE

Theorem 7.25 (Large-sample theory of the MLE)**.** *Let $X_i \overset{iid}{\sim} f(\cdot;\theta_0)$ and let Assumption 7.19 hold. Suppose in addition that Θ is compact or ℓ_n is concave (so that a global maximiser is well behaved). Then there is a sequence of MLEs with*

$$\hat\theta_n \xrightarrow{\;p\;} \theta_0 \qquad \text{and} \qquad \sqrt{n}(\hat\theta_n - \theta_0) \xrightarrow{\;d\;} \mathcal{N}\big(\mathbf{0},\; I(\theta_0)^{-1}\big).$$

Thus the MLE is asymptotically unbiased and attains the Cramér–Rao bound: it is asymptotically efficient.

Proof sketch. Consistency. By the LLN, $n^{-1}\ell_n(\theta) \xrightarrow{p} M(\theta) = \mathbb{E}_{\theta_0} \log f(X;\theta)$ for each θ. By the information inequality of Section 7.5, $M(\theta_0) - M(\theta) = \mathrm{KL}(f_{\theta_0} \,\|\, f_\theta) > 0$ for $\theta \neq \theta_0$, so θ_0 is the unique maximiser of the limit. If the convergence is uniform in θ (compactness plus continuity and a dominating function) or ℓ_n is concave, the maximiser of $n^{-1}\ell_n$ converges to the maximiser of M.

Asymptotic normality. Expand the score equation around θ_0:

$$\mathbf{0} = \frac{1}{\sqrt{n}} U_n(\hat\theta_n) = \frac{1}{\sqrt{n}} U_n(\theta_0) + \left[\frac{1}{n}\nabla^2 \ell_n(\bar\theta_n)\right] \sqrt{n}(\hat\theta_n - \theta_0),$$

with $\bar\theta_n$ between $\hat\theta_n$ and θ_0 (row by row). By Theorem 7.21 and the CLT, $n^{-1/2} U_n(\theta_0) \xrightarrow{d} \mathcal{N}(0, I(\theta_0))$. By the LLN, consistency and (R6), $n^{-1}\nabla^2 \ell_n(\bar\theta_n) \xrightarrow{p} -I(\theta_0)$. Hence $\sqrt{n}(\hat\theta_n - \theta_0) = I(\theta_0)^{-1} n^{-1/2} U_n(\theta_0) + o_p(1) \xrightarrow{d} \mathcal{N}(0, I^{-1} I I^{-1}) = \mathcal{N}(0, I^{-1})$. □

The representation $\sqrt{n}(\hat\theta_n - \theta_0) \approx I(\theta_0)^{-1} n^{-1/2} \sum_i s(X_i;\theta_0)$ says that the MLE is asymptotically an *average* of the *influence function* $I^{-1}s$. This is the key to the sandwich formula below.

7.4.4 Observed versus expected information; standard errors

Two estimates of the information are natural:

$$\text{expected: } nI(\hat\theta_n), \qquad \text{observed: } J_n(\hat\theta_n) = -\nabla^2 \ell_n(\hat\theta_n) = -\sum_i \nabla^2_\theta \log f(X_i;\hat\theta_n).$$

Both are consistent for $nI(\theta_0)$ after division by n. Standard errors are the square roots of the diagonal of $J_n(\hat\theta)^{-1}$ (or $\{nI(\hat\theta)\}^{-1}$), and a Wald interval is $\hat\theta_j \pm 1.96\ \mathrm{se}(\hat\theta_j)$. For canonical exponential-family models (logit, Poisson regression, normal mean) the two coincide at the MLE. When they differ, Efron and Hinkley (1978) argue for the observed information: it adapts to the particular sample and approximates the *conditional* variance of $\hat\theta$ given ancillary features of the data. The Cauchy location model is the textbook illustration.

Listing 7.10: CRLB checks; observed vs expected information ch07/fisher_crlb.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Cramer-Rao bound check and observed vs expected information (Cauchy location).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize

rng = np.random.default_rng(803)
R = 20000

# Part A: Poisson mean and exponential rate versus the Cramer-Rao bound
lam, n = 2.0, 50
xbar = rng.poisson(lam, size=(R, n)).mean(axis=1)
print(f"Part A: R = {R} samples of size n = {n}")
print(f"  Poisson mean    : Var(xbar) = {xbar.var():.5f},"
      f" CRLB lam/n = {lam / n:.5f}")
rate = 0.5
rhat = 1 / rng.exponential(1 / rate, size=(R, n)).mean(axis=1)
print(f"  Exponential rate: Var(1/xbar) = {rhat.var():.6f},"
      f" CRLB rate^2/n = {rate**2 / n:.6f}")
print(f"                    bias = {rhat.mean() - rate:.4f}"
      f" (theory rate/(n-1) = {rate / (n - 1):.4f})")

```

```python
# Part B: Cauchy location; expected information n/2, observed varies
n, R2 = 20, 4000


def mle_cauchy(x):
    nll = lambda t: np.sum(np.log1p((x - t) ** 2))
    return optimize.minimize_scalar(nll, bracket=(np.median(x) - 1,
                                                  np.median(x) + 1)).x


def obs_info(x, t):
    d = x - t
    return np.sum(2 * (1 - d**2) / (1 + d**2) ** 2)


est, J = np.empty(R2), np.empty(R2)
for r in range(R2):
    x = rng.standard_cauchy(n)
    est[r] = mle_cauchy(x)
    J[r] = obs_info(x, est[r])
z_exp = est / np.sqrt(2 / n)                # standardise by expected info
z_obs = est * np.sqrt(J)                    # standardise by observed info
print(f"\nPart B: Cauchy location, n = {n}, R = {R2}")
print(f"  Var(MLE) = {est.var():.4f}; 1/I(theta) = 2/n = {2 / n:.4f}")
print(f"  observed information: min {J.min():.2f}, median"
      f" {np.median(J):.2f}, max {J.max():.2f} (expected {n / 2:.1f})")
print("  coverage of 95% Wald intervals:")
print(f"    expected information: {np.mean(np.abs(z_exp) < 1.96):.3f}")
print(f"    observed information: {np.mean(np.abs(z_obs) < 1.96):.3f}")
# conditional behaviour: split by observed information
lo = J < np.median(J)
print("  Var(MLE | J below median) = %.4f, mean 1/J = %.4f"
      % (est[lo].var(), np.mean(1 / J[lo])))
print("  Var(MLE | J above median) = %.4f, mean 1/J = %.4f"
      % (est[~lo].var(), np.mean(1 / J[~lo])))
```

Output

```
Part A: R = 20000 samples of size n = 50
  Poisson mean    : Var(xbar) = 0.03966, CRLB lam/n = 0.04000
  Exponential rate: Var(1/xbar) = 0.005398, CRLB rate^2/n = 0.005000
                    bias = 0.0103 (theory rate/(n-1) = 0.0102)

Part B: Cauchy location, n = 20, R = 4000
  Var(MLE) = 0.1150; 1/I(theta) = 2/n = 0.1000
  observed information: min 0.87, median 10.66, max 23.97 (expected 10.0)
  coverage of 95% Wald intervals:
    expected information: 0.928
    observed information: 0.938
  Var(MLE | J below median) = 0.1486, mean 1/J = 0.1365
  Var(MLE | J above median) = 0.0811, mean 1/J = 0.0756
```

Part A confirms that $\bar{X}$ attains the bound for the Poisson mean, while $1/\bar{X}$ exceeds it by about 8% at $n = 50$ and has the predicted bias $\lambda/(n-1)$. In Part B, the observed information varies enormously across samples (from 0.87 to 24, against an expected value of 10). Samples with low

observed information really do produce less precise estimates (conditional variance 0.149 versus 0.081), and intervals based on the observed information cover slightly better.

Standard errors from the Hessian in practice. For a model fitted by numerical optimisation, compute the Hessian of $-\ell_n$ at $\hat\theta$ analytically, by automatic differentiation, or by finite differences of the analytic gradient, and invert it. Listing 7.11 does this for a logit take-up model fitted by Newton–Raphson and compares with the inverse-Hessian approximation returned by BFGS and with the Monte Carlo standard deviation of the estimator.

Listing 7.11: Logit standard errors: Hessian, BFGS and simulation ch07/logit_se.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Standard errors of a logit MLE: analytic Hessian, BFGS approximation, simulation.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import optimize, special

rng = np.random.default_rng(803)
n, R = 500, 2000
b_true = np.array([0.5, -0.6, -0.4])
names = ["const", "income", "distance"]


def make_x():
    return np.column_stack([np.ones(n), rng.normal(size=n),
                            rng.uniform(0, 5, size=n)])


def newton_logit(X, y, tol=1e-10):
    """Newton-Raphson (= Fisher scoring for the canonical logit link)."""
    b = np.zeros(X.shape[1])
    for _ in range(50):
        p = special.expit(X @ b)
        H = (X * (p * (1 - p))[:, None]).T @ X      # information matrix
        step = np.linalg.solve(H, X.T @ (y - p))
        b += step
        if np.max(np.abs(step)) < tol:
            break
    p = special.expit(X @ b)
    H = (X * (p * (1 - p))[:, None]).T @ X
    return b, H


# One data set: compare analytic and BFGS-approximate standard errors
X = make_x()
y = rng.binomial(1, special.expit(X @ b_true))
b, H = newton_logit(X, y)
se_h = np.sqrt(np.diag(np.linalg.inv(H)))
nll = lambda c: np.sum(np.logaddexp(0, X @ c) - y * (X @ c))
gr = lambda c: X.T @ (special.expit(X @ c) - y)
rb = optimize.minimize(nll, np.zeros(3), jac=gr, method="BFGS")
se_bfgs = np.sqrt(np.diag(rb.hess_inv))

# Monte Carlo: sampling sd and coverage of Wald intervals
B = np.empty((R, 3))
cover = np.zeros(3)
for r in range(R):
```

```
    Xr = make_x()
    yr = rng.binomial(1, special.expit(Xr @ b_true))
    br, Hr = newton_logit(Xr, yr)
    B[r] = br
    cover += np.abs(br - b_true) < 1.96 * np.sqrt(np.diag(np.linalg.inv(Hr)))
print(f"Logit take-up model, n = {n}; Monte Carlo R = {R}")
print(f"{'':>9} {'b_hat':>8} {'se Hess':>8} {'se BFGS':>8} {'MC sd':>8}"
      f" {'cover':>7}")
for j, nm in enumerate(names):
    print(f"{nm:>9} {b[j]:8.4f} {se_h[j]:8.4f} {se_bfgs[j]:8.4f}"
          f" {B[:, j].std():8.4f} {cover[j] / R:7.3f}")
```

Output

```
Logit take-up model, n = 500; Monte Carlo R = 2000
             b_hat  se Hess  se BFGS    MC sd   cover
     const  0.5362   0.1879   0.1868   0.1904   0.959
    income -0.4590   0.1019   0.1001   0.1071   0.946
  distance -0.4236   0.0694   0.0689   0.0711   0.948
```

The Hessian-based standard errors match the Monte Carlo standard deviations closely and the Wald intervals have coverage within Monte Carlo error (≈ 0.005) of 95%.

△ Caution: `hess_inv` is not a covariance matrix estimate

The `hess_inv` attribute returned by `minimize(..., method="BFGS")` is the optimiser's *quasi-Newton approximation*, built from gradient differences along the optimisation path. It can be accurate (as above, within 2%) but it can also be badly wrong, especially after few iterations or when started near the optimum; L-BFGS-B returns only a low-rank operator. For reported standard errors, always evaluate the Hessian at $\hat\theta$ separately.

7.4.5 Misspecification: quasi-MLE and the sandwich

Suppose we maximise $\ell_n(\theta) = \sum_i \log f(X_i;\theta)$ although the data come from a density f_0 that is *not* in the model. The estimator is then called a *quasi-* or *pseudo-MLE*. Its limit is the *pseudo-true* value $\theta^* = \arg\max_\theta \mathbb{E}_0 \log f(X;\theta)$ (Section 7.5), and the expansion in the proof of Theorem 7.25 still applies, except that the two matrices

$$A = -\mathbb{E}_0\big[\nabla^2_\theta \log f(X;\theta^*)\big], \qquad B = \mathbb{E}_0\big[s(X;\theta^*)s(X;\theta^*)^\top\big]$$

are no longer equal.

Theorem 7.26 (Huber–White sandwich). *Under regularity conditions, $\sqrt{n}(\hat\theta_n - \theta^*) \xrightarrow{d} \mathcal{N}(0, A^{-1}BA^{-1})$, and $A^{-1}BA^{-1}$ is consistently estimated by $\hat A^{-1}\hat B\hat A^{-1}$ with $\hat A = -n^{-1}\nabla^2\ell_n(\hat\theta)$ and $\hat B = n^{-1}\sum_i s(X_i;\hat\theta)s(X_i;\hat\theta)^\top$.*

If the model is correct, $A = B = I(\theta_0)$ and the sandwich collapses to I^{-1}. The robust ("heteroskedasticity-consistent") OLS covariance $(\mathbf{X}^\top\mathbf{X})^{-1}\big(\sum_i \hat u_i^2 \mathbf{x}_i\mathbf{x}_i^\top\big)(\mathbf{X}^\top\mathbf{X})^{-1}$ of White (1980) is the special case of the normal linear model. A remarkable fact makes the sandwich especially useful for counts: in the Poisson model the score is $\mathbf{x}_i(y_i - e^{\mathbf{x}_i^\top\beta})$, so the pseudo-true value equals the true β whenever the *conditional mean* is correctly specified, $\mathbb{E}[y \mid \mathbf{x}] = e^{\mathbf{x}^\top\beta}$, whatever the distribution of y (Gouriéroux et al., 1984). Poisson QMLE with sandwich standard errors is therefore a robust estimator for any non-negative outcome with a log-linear mean, including trade flows (Santos Silva and Tenreyro, 2006).

Listing 7.12: Poisson QMLE: model-based versus sandwich standard errors ch07/sandwich_qmle.py

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Poisson quasi-MLE with overdispersed counts: model vs sandwich standard errors.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, R = 400, 3000
g_true = np.array([0.8, 0.3, -0.35])


def poisson_fit(X, y):
    """Poisson MLE by Newton-Raphson; returns b, A = -Hessian, B = sum s s'."""
    b = np.zeros(X.shape[1])
    b[0] = np.log(y.mean())
    for _ in range(50):
        mu = np.exp(X @ b)
        A = (X * mu[:, None]).T @ X
        step = np.linalg.solve(A, X.T @ (y - mu))
        b += step
        if np.max(np.abs(step)) < 1e-10:
            break
    mu = np.exp(X @ b)
    A = (X * mu[:, None]).T @ X
    s = X * (y - mu)[:, None]                    # individual scores
    return b, A, s.T @ s


def simulate(alpha):
    """Negative binomial counts with E[y|x] = exp(x'g), Var = mu + alpha mu^2."""
    X = np.column_stack([np.ones(n), rng.binomial(1, 0.4, n),
                         rng.uniform(0, 2, n)])
    mu = np.exp(X @ g_true)
    lam = mu * rng.gamma(1 / alpha, alpha, n) if alpha > 0 else mu
    return X, rng.poisson(lam)


print(f"Poisson (quasi-)MLE, n = {n}, R = {R}; coverage of 95% intervals"
      " for the fee slope")
print(f"{'alpha':>6} {'MC sd':>8} {'model se':>9} {'sandwich':>9}"
      f" {'cov model':>10} {'cov sand':>9}")
for alpha in (0.0, 0.5, 1.5):
    est, se_m, se_s = np.empty(R), np.empty(R), np.empty(R)
    for r in range(R):
        X, y = simulate(alpha)
        b, A, Bm = poisson_fit(X, y)
        Ai = np.linalg.inv(A)
        est[r] = b[2]
        se_m[r] = np.sqrt(Ai[2, 2])
        se_s[r] = np.sqrt((Ai @ Bm @ Ai)[2, 2])
    err = np.abs(est - g_true[2])
    print(f"{alpha:6.1f} {est.std():8.4f} {se_m.mean():9.4f}"
          f" {se_s.mean():9.4f} {np.mean(err < 1.96 * se_m):10.3f}"
          f" {np.mean(err < 1.96 * se_s):9.3f}")
print(f"mean of fee slope estimates (alpha = 1.5): {est.mean():.4f}"
      f" (true {g_true[2]})")
```

Output

```
Poisson (quasi-)MLE, n = 400, R = 3000; coverage of 95% intervals for the fee slope
 alpha    MC sd  model se  sandwich  cov model  cov sand
   0.0   0.0671    0.0651    0.0648      0.947     0.945
   0.5   0.0900    0.0653    0.0901      0.845     0.947
   1.5   0.1269    0.0654    0.1255      0.689     0.946
mean of fee slope estimates (alpha = 1.5): -0.3484 (true -0.35)
```

With Poisson data ($\alpha = 0$) both standard errors are correct. With negative binomial overdispersion, $\mathrm{Var}(y \mid \mathbf{x}) = \mu + \alpha\mu^2$, the point estimates remain unbiased but the model-based standard error stays at 0.065 while the true sampling standard deviation rises to 0.127: nominal 95% intervals cover only 69% of the time. The sandwich tracks the true variability and restores correct coverage.

Policy Lens: Robust standard errors in policy reports

A policy report that states "the fee effect is significant at the 1% level" is making a claim about a standard error. If the model's variance assumption is wrong — as it almost always is for consumption, earnings and count data — model-based standard errors can be far too small, producing spurious precision and overconfident recommendations. Reporting sandwich (heteroskedasticity-robust) standard errors, and cluster-robust versions when households are sampled in villages or schools (see Chapter 11), is the minimum standard of credible quantitative policy analysis.

Historical Note: Huber and White

Peter Huber (Huber, 1967) derived the sandwich variance for estimators that maximise a criterion other than the true log-likelihood. Halbert White brought the idea into econometrics with his heteroskedasticity-consistent covariance estimator (White, 1980) and his theory of misspecified ML (White, 1982), which also proposed the information-matrix test of $A = B$.

7.4.6 The trinity: Wald, likelihood-ratio and score tests

To test $H_0 : \theta = \theta_0$ against $H_1 : \theta \neq \theta_0$ (with $\theta \in \mathbb{R}^p$) three statistics are standard:

$$\begin{aligned} \text{Wald:} \quad & W_n = n(\hat\theta - \theta_0)^\top I(\hat\theta)(\hat\theta - \theta_0), \\ \text{Likelihood ratio:} \quad & LR_n = 2\{\ell_n(\hat\theta) - \ell_n(\theta_0)\}, \\ \text{Score (Lagrange multiplier):} \quad & S_n = \tfrac{1}{n} U_n(\theta_0)^\top I(\theta_0)^{-1} U_n(\theta_0). \end{aligned}$$

Theorem 7.27 (Wilks; asymptotic equivalence)**.** *Under H_0 and Assumption 7.19, W_n, LR_n and S_n all converge in distribution to χ^2_p, and their pairwise differences are $o_p(1)$. More generally, for a composite null that imposes r restrictions, the three statistics (with the restricted MLE in place of θ_0) converge to χ^2_r.*

Proof sketch for LR_n. A second-order expansion of $\ell_n(\theta_0)$ about $\hat\theta$, where $U_n(\hat\theta) = 0$, gives $LR_n = n(\hat\theta - \theta_0)^\top\{-n^{-1}\nabla^2\ell_n(\bar\theta)\}(\hat\theta - \theta_0) = W_n + o_p(1)$. With $Z_n = \sqrt{n}\, I^{1/2}(\hat\theta - \theta_0) \xrightarrow{d} \mathcal{N}(0, I_p)$, $W_n = Z_n^\top Z_n + o_p(1) \xrightarrow{d} \chi^2_p$. Likewise $n^{-1/2}U_n(\theta_0) = I\sqrt{n}(\hat\theta - \theta_0) + o_p(1)$ shows $S_n = W_n + o_p(1)$. □

The tests differ in what they need: the Wald test needs only the unrestricted fit, the score test only the restricted fit (attractive when the unrestricted model is hard to estimate), and the LR test both. The LR and score tests are *invariant* to reparametrisation; the Wald test is not. For the

exponential rate, with $\hat{\lambda} = 1/\bar{X}$,

$$W^{(\lambda)} = n(1-\lambda_0\bar{X})^2 = S, \quad W^{(\mu)} = n\Big(1-\frac{1}{\lambda_0\bar{X}}\Big)^2, \quad LR = 2n\big\{\log(\hat{\lambda}/\lambda_0) - 1 + \lambda_0\bar{X}\big\},$$

where $W^{(\mu)}$ is the Wald test of the same hypothesis written as $\mu = 1/\lambda_0$ for the mean $\mu = 1/\lambda$.

Listing 7.13: Simulated null distributions of the trinity (plotting code omitted) ch07/trinity_tests.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Null distributions of Wald, likelihood-ratio and score tests (exponential rate).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
lam0, R = 1.0, 50000
crit = stats.chi2.ppf(0.95, 1)


def trinity(x, lam0):
    n = x.shape[1]
    xbar = x.mean(axis=1)
    lhat = 1 / xbar
    wald_r = n * (lhat - lam0) ** 2 / lhat**2        # rate scale, I = 1/lam^2
    mu0 = 1 / lam0
    wald_m = n * (xbar - mu0) ** 2 / xbar**2         # mean scale, I = 1/mu^2
    lr = 2 * n * (np.log(lhat / lam0) - 1 + lam0 * xbar)
    score = n * (1 - lam0 * xbar) ** 2               # U^2 / I(lam0)
    return wald_r, wald_m, lr, score


print(f"H0: rate = {lam0}; R = {R}; nominal size 0.05 (chi2_1 crit"
      f" {crit:.3f})")
print(f"{'n':>5} {'Wald(rate)':>11} {'Wald(mean)':>11} {'LR':>8} {'score':>8}")
for n in (5, 10, 30, 200):
    x = rng.exponential(1 / lam0, size=(R, n))
    rej = [np.mean(t > crit) for t in trinity(x, lam0)]
    print(f"{n:>5} {rej[0]:11.4f} {rej[1]:11.4f} {rej[2]:8.4f} {rej[3]:8.4f}")

# Power at a local alternative (n = 30, true rate 1.4)
x = rng.exponential(1 / 1.4, size=(R, 30))
pw = [np.mean(t > crit) for t in trinity(x, lam0)]
print("power, n = 30, rate = 1.4: Wald(rate) %.3f, Wald(mean) %.3f,"
      " LR %.3f, score %.3f" % tuple(pw))
```

Output

```
H0: rate = 1.0; R = 50000; nominal size 0.05 (chi2_1 crit 3.841)
    n  Wald(rate)  Wald(mean)       LR    score
    5      0.0443      0.1328   0.0562   0.0443
   10      0.0452      0.0957   0.0517   0.0452
   30      0.0484      0.0654   0.0501   0.0484
  200      0.0488      0.0521   0.0493   0.0488
power, n = 30, rate = 1.4: Wald(rate) 0.303, Wald(mean) 0.591, LR 0.428, score 0.303
```

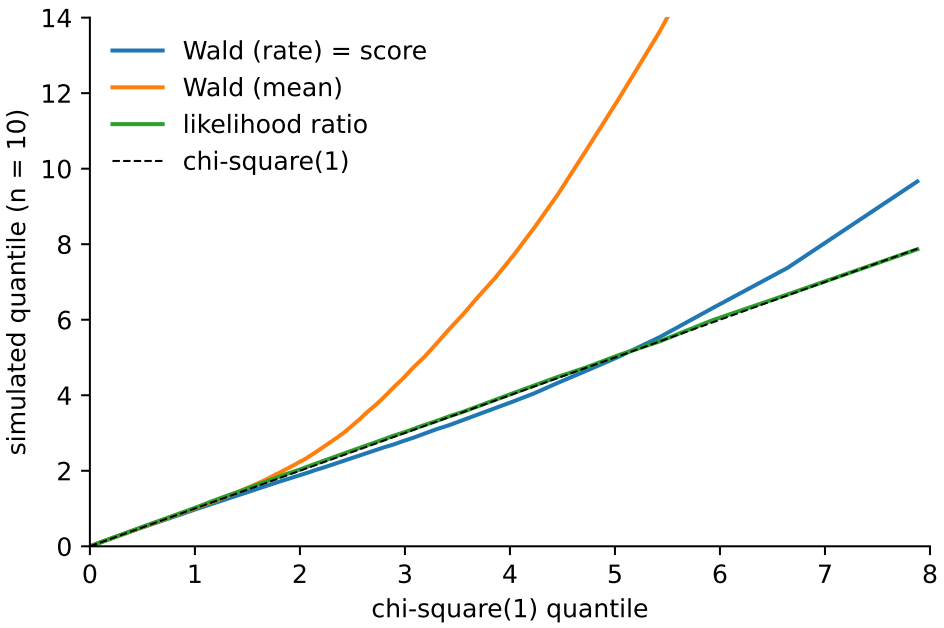


Figure 7.4: Quantiles of the Wald (two parametrisations), likelihood-ratio and score statistics for $H_0 : \lambda = 1$ with $n = 10$ exponential observations, against χ^2_1 quantiles. The LR statistic is almost exactly χ^2_1; the Wald test in the mean parametrisation is badly oversized (Listing 7.13).

The LR test has size close to nominal even for tiny samples (0.056 at $n = 5$, 0.052 at $n = 10$), the rate-scale Wald/score test is slightly conservative, and the mean-scale Wald test rejects 13% of the time at $n = 5$ when it should reject 5%. (Its apparently high power at $n = 30$ is an artefact of this oversizing.) All four agree at $n = 200$, as Theorem 7.27 predicts.

Historical Note: Wald, Wilks and Rao

Samuel Wilks proved in 1938 that minus twice the log likelihood ratio is asymptotically chi-squared (Wilks, 1938); Abraham Wald developed the test based on the unrestricted estimate in 1943 (Wald, 1943); and C. R. Rao proposed the score test in 1948 (Rao, 1948). The score test was rediscovered in econometrics as the Lagrange multiplier test (Aitchison and Silvey, 1958; Silvey, 1959), because the multiplier of the constraint in the restricted maximisation is a linear function of the score.

Graduate Perspective: Beyond regular iid models

Theorem 7.25 is the simplest member of a large family. Newey and McFadden (1994) give general consistency and asymptotic-normality theorems for *extremum estimators* (maximisers of a criterion), covering GMM, QMLE and nonlinear least squares in one framework. Le Cam's theory of local asymptotic normality (van der Vaart, 1998) explains why I^{-1} is a lower bound for *all* regular estimators, not only unbiased ones (the Hájek–Le Cam convolution theorem). Non-regular cases — parameters on the boundary (variance components equal to zero), support depending on θ, or unidentified parameters under the null (testing the number of mixture components) — give non-χ^2 limits and are common in applied work.

⋆ Key Concept: Information and inference

The score has mean zero and variance $I(\theta)$, which also equals the expected negative Hessian. No unbiased estimator beats $1/(nI)$ (Cramér–Rao), and the MLE attains it asymptotically: $\hat{\theta} \overset{a}{\sim} \mathcal{N}(\theta_0, \{nI(\theta_0)\}^{-1})$. Standard errors come from the inverse observed (or expected)

information; under misspecification use the sandwich $A^{-1}BA^{-1}$. Wald, LR and score tests are asymptotically equivalent χ^2 tests but can differ markedly in small samples; the LR test is parametrisation-invariant.

Exercises

7.4.1 Theory Compute the Fisher information for each model in Example 7.22 using both $\mathbb{E}[s^2]$ and $-\mathbb{E}[\partial^2 \log f]$. For the normal model, show that the information for (μ, σ^2) is $\mathrm{diag}(1/\sigma^2, 1/(2\sigma^4))$ and explain the relation to $\mathrm{diag}(1/\sigma^2, 2/\sigma^2)$ for (μ, σ) via the Jacobian of the reparametrisation.

7.4.2 Theory Show that the information identity fails for $X \sim \mathrm{Uniform}(0, \theta)$ and that the estimator $\frac{n+1}{n}\max_i X_i$ has variance of order n^{-2}, "beating" the Cramér–Rao rate. Which regularity condition is violated?

7.4.3 Theory Prove that equality in the Cramér–Rao bound for an unbiased estimator T of θ requires $\partial \ell_n/\partial\theta = nI(\theta)\,(T-\theta)$, and deduce that f must be of one-parameter exponential-family form.

7.4.4 Python Repeat the trinity simulation for the Bernoulli model, $H_0: p = 0.1$, with $n \in \{20, 50, 200\}$. Explain the irregular rejection rates in terms of the discreteness of $\sum y_i$, and compare with the exact binomial test.

7.4.5 ★ Graduate *One-step estimators.* Starting from a $\sqrt{n}$-consistent estimator $\tilde\theta$ (e.g. the sample median for Cauchy location), show that one Newton step $\tilde\theta + J_n(\tilde\theta)^{-1} U_n(\tilde\theta)$ is asymptotically efficient. Verify by simulation using the Cauchy design of Listing 7.10.

7.4.6 Policy A policy report on a health-insurance subsidy fits a Poisson model to monthly outpatient visits ($n = 1200$ households) and reports a model-based standard error of 0.04 for the subsidy coefficient; the sample variance of visits is four times the mean. Using the logic of Listing 7.12, estimate roughly how large the robust standard error is likely to be and explain how the policy conclusion might change.

7.5 Information Geometry Basics

Information geometry studies statistical models as geometric objects: each parameter value θ is a point, the model $\{f_\theta\}$ is a manifold, and the divergence between distributions — rather than the Euclidean distance between parameter vectors — determines how far apart two points are. This section introduces the two central objects: the Kullback–Leibler divergence and the Fisher–Rao metric.

7.5.1 The Kullback–Leibler divergence

Definition 7.28 (KL divergence)**.** For densities (or mass functions) f and g with $g(x) > 0$ wherever $f(x) > 0$,

$$\mathrm{KL}(f \,\|\, g) = \mathbb{E}_f\left[\log \frac{f(X)}{g(X)}\right] = \int f(x) \log \frac{f(x)}{g(x)}\, dx.$$

If f puts mass where g does not, $\mathrm{KL}(f\|g) = +\infty$.

Theorem 7.29 (Gibbs' inequality)**.** $\mathrm{KL}(f\|g) \geq 0$*, with equality if and only if* $f = g$ *almost everywhere.*

Proof. The function $-\log$ is strictly convex, so Jensen's inequality gives

$$\mathrm{KL}(f\|g) = \mathbb{E}_f\Big[-\log\frac{g(X)}{f(X)}\Big] \;\geq\; -\log\mathbb{E}_f\Big[\frac{g(X)}{f(X)}\Big] = -\log\int_{\{f>0\}} g(x)\,dx \;\geq\; -\log 1 = 0.$$

Equality in Jensen's inequality for a strictly convex function requires $g(X)/f(X)$ to be constant f-almost surely; equality in the last step requires $\int_{\{f>0\}} g = 1$. Together these force $g = f$ almost everywhere. □

KL is *not* a distance: it is asymmetric and violates the triangle inequality. For two normal distributions

$$\mathrm{KL}\big(\mathcal{N}(\mu_1,\sigma_1^2)\,\|\,\mathcal{N}(\mu_2,\sigma_2^2)\big) = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2+(\mu_1-\mu_2)^2}{2\sigma_2^2} - \frac{1}{2}. \tag{7.5}$$

7.5.2 Maximum likelihood as minimum KL

Write $\mathrm{KL}(f_0\|f_\theta) = \mathbb{E}_0\log f_0(X) - \mathbb{E}_0\log f_\theta(X)$. The first term does not depend on θ, so

$$\theta^* = \arg\min_\theta \mathrm{KL}(f_0\,\|\,f_\theta) = \arg\max_\theta \mathbb{E}_0\log f_\theta(X). \tag{7.6}$$

The MLE maximises the sample analogue $n^{-1}\sum_i \log f_\theta(X_i) = \mathbb{E}_{\hat P_n}\log f_\theta$, where $\hat P_n$ is the empirical distribution. In words:

- the MLE minimises the *cross-entropy* $H(\hat P_n, f_\theta) = -\mathbb{E}_{\hat P_n}\log f_\theta$, i.e. it is the member of the model closest, in the KL sense, to the empirical distribution (for discrete data, literally $\arg\min_\theta \mathrm{KL}(\hat P_n\|f_\theta)$);
- if the model is correct, $\theta^* = \theta_0$ by Gibbs' inequality — this is the identification step in the consistency proof of Theorem 7.25;
- if the model is wrong, the MLE converges to the *pseudo-true* value θ^*, the KL projection of the truth onto the model, and its sampling variance is the sandwich of Theorem 7.26.

Example 7.30 (An exponential model for gamma data)**.** If the data are $\mathrm{Gamma}(3,1)$ but we fit an exponential model, $\mathbb{E}_0\log f_\lambda = \log\lambda - \lambda\mathbb{E}_0 X$, maximised at $\lambda^* = 1/\mathbb{E}_0 X = 1/3$. The fitted model reproduces the mean but not the shape; the residual divergence $\mathrm{KL}(f_0\|f_{\lambda^*}) = -H(f_0) + \log\mathbb{E}_0 X + 1$ measures the cost of misspecification.

Listing 7.14: KL divergence and pseudo-true parameters (plotting code omitted)
ch07/kl_pseudo_true.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# KL divergence: closed form, local Fisher approximation, and pseudo-true parameters.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate, stats

rng = np.random.default_rng(803)


def kl_normal(m1, s1, m2, s2):
    return np.log(s2 / s1) + (s1**2 + (m1 - m2) ** 2) / (2 * s2**2) - 0.5

```

```python

# Part A: closed form vs numerical integration; asymmetry
f, g = stats.norm(0, 1), stats.norm(1, 2)
num = integrate.quad(lambda x: f.pdf(x) * (f.logpdf(x) - g.logpdf(x)),
                     -np.inf, np.inf)[0]
print("Part A: KL divergences between normals")
print(f"  KL(N(0,1) || N(1,4)) closed form {kl_normal(0, 1, 1, 2):.5f},"
      f" quadrature {num:.5f}")
print(f"  KL(N(1,4) || N(0,1)) closed form {kl_normal(1, 2, 0, 1):.5f}"
      "  (KL is not symmetric)")

# Local approximation KL(p, p + d) ~ d^2 I(p) / 2 for the Bernoulli
p = 0.3
print("  Bernoulli p = 0.3: KL(p || p+d) vs d^2 I(p)/2, I(p) = 1/(p(1-p))")
for d in (0.1, 0.03, 0.01):
    q = p + d
    kl = p * np.log(p / q) + (1 - p) * np.log((1 - p) / (1 - q))
    approx = d**2 / (2 * p * (1 - p))
    print(f"    d = {d:5.2f}: KL = {kl:.3e}, approx = {approx:.3e},"
          f" ratio = {kl / approx:.4f}")

# Part B: misspecified exponential model fitted to gamma(3, 1) data
k0 = 3.0
f0 = stats.gamma(k0)
lam_star = 1 / f0.mean()                      # pseudo-true rate
print("\nPart B: exponential model fitted to gamma(shape 3) data")
print(f"  pseudo-true rate 1/E[X] = {lam_star:.4f}")
for n in (50, 500, 5000, 50000):
    x = f0.rvs(size=n, random_state=rng)
    print(f"  n = {n:>6}: MLE rate = {1 / x.mean():.4f}")
lams = np.linspace(0.1, 1.0, 300)
# KL(f0 || Exp(lam)) = -H(f0) - E log f_lam = -H(f0) - log lam + lam E[X]
kl_curve = -f0.entropy() - np.log(lams) + lams * f0.mean()
print(f"  KL(f0 || best exponential) = {kl_curve.min():.4f} nats"
      " (misspecification gap)")
```

Output

```
Part A: KL divergences between normals
  KL(N(0,1) || N(1,4)) closed form 0.44315, quadrature 0.44315
  KL(N(1,4) || N(0,1)) closed form 1.30685  (KL is not symmetric)
  Bernoulli p = 0.3: KL(p || p+d) vs d^2 I(p)/2, I(p) = 1/(p(1-p))
    d =  0.10: KL = 2.160e-02, approx = 2.381e-02, ratio = 0.9072
    d =  0.03: KL = 2.069e-03, approx = 2.143e-03, ratio = 0.9654
    d =  0.01: KL = 2.352e-04, approx = 2.381e-04, ratio = 0.9877

Part B: exponential model fitted to gamma(shape 3) data
  pseudo-true rate 1/E[X] = 0.3333
  n =     50: MLE rate = 0.3624
  n =    500: MLE rate = 0.3337
  n =   5000: MLE rate = 0.3386
  n =  50000: MLE rate = 0.3324
  KL(f0 || best exponential) = 0.2510 nats (misspecification gap)
```

The quadrature confirms (7.5) and the asymmetry (0.443 versus 1.307 nats). The MLE of the exponential rate converges to 1/3, not to any "true" rate — there is none — and the best

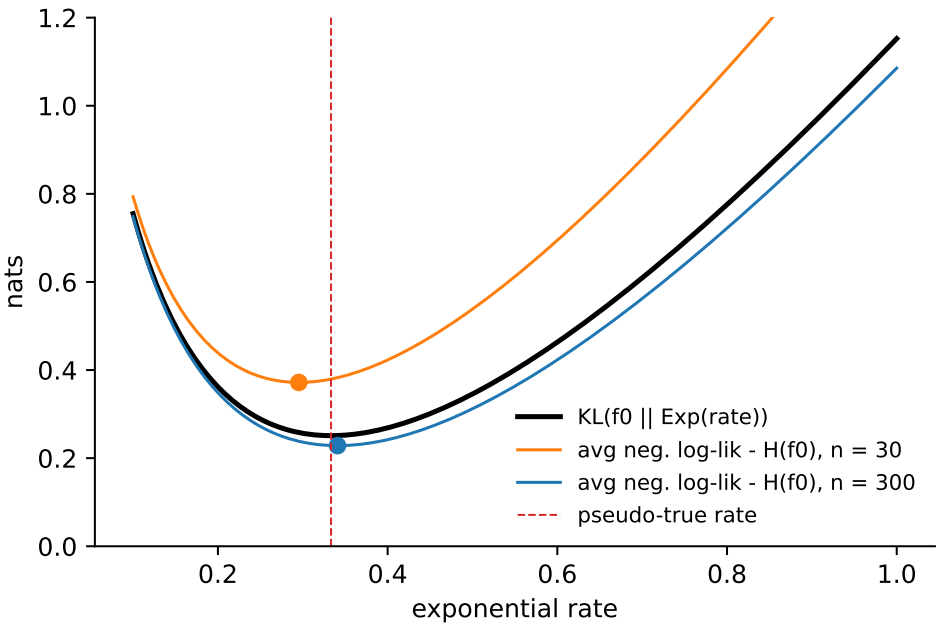


Figure 7.5: Population KL divergence from Gamma(3, 1) to the exponential model (black) and its sample analogues, the average negative log-likelihood shifted by the entropy $H(f_0)$, for $n = 30$ and $n = 300$. The minimisers (dots) are the MLEs; they approach the pseudo-true rate $1/3$ (Listing 7.14).

exponential approximation is 0.25 nats away from the truth.

△ Caution: The direction of KL matters

Minimising $\mathrm{KL}(f_0 \| f_\theta)$ ("forward" KL, as ML does) penalises models that put little mass where the data are; the fit tends to *cover* all of the data, even at the cost of placing mass where there are none. Minimising $\mathrm{KL}(f_\theta \| f_0)$ ("reverse" KL, as variational inference does) penalises mass where f_0 is small and tends to lock onto a single mode. Which direction is appropriate depends on the use of the fitted model.

Historical Note: Kullback, Leibler and information

Solomon Kullback and Richard Leibler, both working on cryptanalysis for the US government, introduced the divergence in their 1951 paper "On information and sufficiency" in the *Annals of Mathematical Statistics* (Kullback and Leibler, 1951), calling it the mean information for discrimination between two hypotheses. It is now also known as *relative entropy*.

7.5.3 Fisher information as a Riemannian metric

Theorem 7.31 (Local quadratic form of KL). *Under Assumption 7.19, as* $\delta \to \mathbf{0}$,

$$\mathrm{KL}(f_\theta \,\|\, f_{\theta+\delta}) = \tfrac{1}{2}\,\delta^\top I(\theta)\,\delta + O(\|\delta\|^3).$$

Proof. Expand $\log f_{\theta+\delta}(x) = \log f_\theta(x) + \delta^\top s(x;\theta) + \tfrac{1}{2}\delta^\top \nabla^2 \log f_\theta(x)\,\delta + O(\|\delta\|^3)$. Taking $\mathbb{E}_\theta$ of $\log f_\theta - \log f_{\theta+\delta}$ and using $\mathbb{E}_\theta s = \mathbf{0}$ and $-\mathbb{E}_\theta \nabla^2 \log f_\theta = I(\theta)$ (Theorem 7.21) gives the result. □

Thus, *infinitesimally*, KL is a squared distance with metric tensor $I(\theta)$: the statistical "length" of a small displacement $d\theta$ is $ds^2 = d\theta^\top I(\theta)\,d\theta$. A Riemannian metric that varies from point to point is exactly what is needed, because one unit of μ means very different things when $\sigma = 0.1$

and when $\sigma = 10$: for the normal model $ds^2 = (d\mu^2 + 2\,d\sigma^2)/\sigma^2$, the metric of the hyperbolic plane. Listing 7.14 (Part A) checks Theorem 7.31 numerically for the Bernoulli: the ratio of KL to $\delta^2 I(p)/2$ approaches 1 as $\delta \to 0$.

Reparametrisation invariance. If $\theta = \theta(\eta)$ with Jacobian $\mathbf{J} = \partial\theta/\partial\eta^\top$, the chain rule for the score gives

$$I_\eta(\eta) = \mathbf{J}^\top I_\theta\big(\theta(\eta)\big)\,\mathbf{J}, \tag{7.7}$$

so $ds^2 = d\eta^\top I_\eta\, d\eta = d\theta^\top I_\theta\, d\theta$: lengths, and therefore the geometry, do not depend on the coordinates used. Quantities that are built from $I(\theta)$ in this covariant way — the volume element $\sqrt{\det I(\theta)}\,d\theta$, the natural gradient below, the LR and score tests — inherit this invariance. The Wald test and ordinary gradient descent do not.

> **Definition 7.32** (Jeffreys prior)**.** The *Jeffreys prior* is $\pi_J(\theta) \propto \sqrt{\det I(\theta)}$. By (7.7), $\sqrt{\det I_\eta} = \sqrt{\det I_\theta}\,|\det \mathbf{J}|$, which is exactly the change-of-variables rule for densities: the Jeffreys prior is the same distribution whatever parametrisation is used to express it. It is the "uniform" distribution with respect to the Fisher–Rao volume.

For the Bernoulli, $\pi_J(p) \propto p^{-1/2}(1-p)^{-1/2}$, the Beta$(\frac{1}{2}, \frac{1}{2})$ distribution; for a Poisson mean, $\pi_J(\lambda) \propto \lambda^{-1/2}$; for a normal scale, $\pi_J(\sigma) \propto 1/\sigma$ (improper). We use the Bernoulli case in Section 7.6.

7.5.4 The natural gradient

Ordinary gradient ascent $\theta \leftarrow \theta + \eta\nabla\ell(\theta)$ moves in the direction of steepest ascent *per unit of Euclidean distance* in parameter space — a choice that depends on the parametrisation. The steepest ascent per unit of *statistical* distance solves

$$\max_{\delta}\ \ell(\theta+\delta) \quad \text{subject to} \quad \tfrac{1}{2}\,\delta^\top I(\theta)\,\delta = \varepsilon.$$

Linearising ℓ and using a Lagrange multiplier gives $\delta \propto I(\theta)^{-1}\nabla\ell(\theta)$, the *natural gradient* (Amari, 1998):

$$\theta \leftarrow \theta + \eta\, I(\theta)^{-1}\nabla_\theta \bar{\ell}_n(\theta), \qquad \bar{\ell}_n = n^{-1}\ell_n. \tag{7.8}$$

With $\eta = 1$ this is *Fisher scoring*, Fisher's own algorithm for computing MLEs; for canonical GLMs it coincides with Newton–Raphson (as in Listing 7.11). The natural gradient is invariant to smooth reparametrisations to first order and automatically adapts the step to the local curvature.

Listing 7.15: Ordinary versus natural gradient for the normal model (plotting code omitted)
ch07/natural_gradient.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# Ordinary vs natural gradient ascent for the normal log-likelihood in (mu, sigma).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
x = rng.normal(3.0, 0.5, size=200)
m1, m2 = x.mean(), x.var()            # MLE: mu = m1, sigma = sqrt(m2)


def avg_ll(mu, s):
    return -np.log(s) - (m2 + (m1 - mu) ** 2) / (2 * s**2)

```

```python

def grad(t):
    mu, s = t
    d2 = m2 + (m1 - mu) ** 2
    return np.array([(m1 - mu) / s**2, -1 / s + d2 / s**3])


def fisher_inv(t):
    s = t[1]
    return np.diag([s**2, s**2 / 2])   # inverse of diag(1/s^2, 2/s^2)


def ascend(t0, eta, natural, iters=5000):
    t, path = np.array(t0, float), [np.array(t0, float)]
    for k in range(iters):
        g = grad(t)
        step = fisher_inv(t) @ g if natural else g
        t = t + eta * step
        path.append(t.copy())
        if np.linalg.norm(t - [m1, np.sqrt(m2)]) < 1e-6:
            return np.array(path), k + 1
        if not np.all(np.isfinite(t)) or t[1] <= 0:
            return np.array(path), np.inf
    return np.array(path), iters


start = (1.0, 0.4)
print(f"MLE: mu = {m1:.4f}, sigma = {np.sqrt(m2):.4f}; start = {start}")
print(f"{'method':<18} {'step':>6} {'iterations to 1e-6':>20}")
paths = {}
for eta in (0.01, 0.05, 0.2):
    p, k = ascend(start, eta, natural=False)
    paths[("ordinary", eta)] = p
    print(f"{'ordinary gradient':<18} {eta:6.2f} {k:>20}")
for eta in (0.2, 0.5, 1.0):
    p, k = ascend(start, eta, natural=True)
    paths[("natural", eta)] = p
    print(f"{'natural gradient':<18} {eta:6.2f} {k:>20}")
```

Output

```
MLE: mu = 2.9797, sigma = 0.5087; start = (1.0, 0.4)
method               step   iterations to 1e-6
ordinary gradient    0.01                  582
ordinary gradient    0.05                  210
ordinary gradient    0.20                  441
natural gradient     0.20                   71
natural gradient     0.50                   26
natural gradient     1.00                    8
```

Starting with σ too small, the ordinary gradient $\partial\bar{\ell}/\partial\sigma \propto \sigma^{-3}$ is enormous, so any step size large enough to be fast far from the optimum is destabilising near the start: the number of iterations is 210–582 and is *not* monotone in the step size. The natural gradient rescales by $I^{-1} = \mathrm{diag}(\sigma^2, \sigma^2/2)$, and with $\eta = 1$ converges in 8 iterations. In deep learning, approximations to the natural gradient (K-FAC) and adaptive methods such as Adam can be read as attempts to

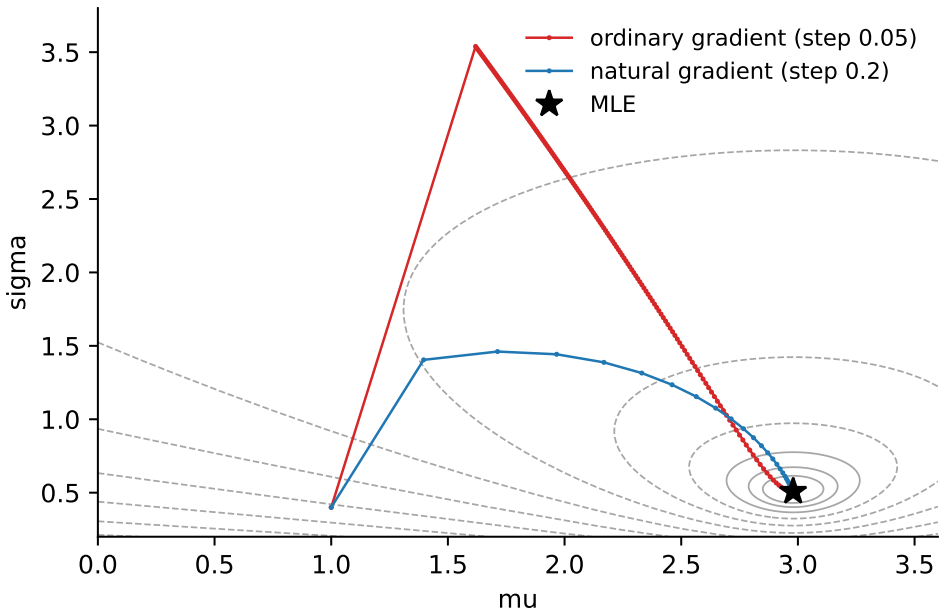


Figure 7.6: Paths of ordinary and natural gradient ascent on the normal average log-likelihood in (μ, σ), starting at $(1, 0.4)$. The ordinary gradient is huge at small σ and overshoots to $\sigma \approx 3.5$ before crawling back; the natural gradient takes a smooth, direct route (Listing 7.15).

precondition by an estimate of the information matrix (Chapter 8).

7.5.5 Cross-entropy loss in machine learning

A classifier that outputs class probabilities $q_\theta(k \mid \mathbf{x})$, $k = 1, \dots, K$, is trained by minimising the *cross-entropy* (log) loss

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log q_\theta(y_i \mid \mathbf{x}_i) = \frac{1}{n}\sum_{i=1}^{n} H\big(\delta_{y_i},\, q_\theta(\cdot \mid \mathbf{x}_i)\big),$$

where δ_y is the one-hot distribution. This is *exactly* the negative average conditional log-likelihood of a categorical model; with $K = 2$ and a linear index it is logistic regression. Hence everything in this chapter applies: minimising cross-entropy is ML, it estimates the KL projection of the true conditional distribution onto the model class, and its minimiser is calibrated only if the model is (approximately) correct (Murphy, 2022; Goodfellow et al., 2016). The *squared-error* loss is likewise the negative log-likelihood of a Gaussian model with fixed variance.

Graduate Perspective: Dually flat geometry and beyond

Amari (1985) and Amari and Nagaoka (2000) showed that exponential families carry not one but a pair of *dual affine connections*, in which the natural and the mean parameters are both "straight" coordinates. In this dually flat geometry the KL divergence is a Bregman divergence, there is a Pythagorean theorem for KL projections, and the MLE in an exponential family is the projection of the empirical distribution onto the model along a straight line in mean coordinates. These ideas underpin the analysis of the EM algorithm, of variational inference and of mirror descent; Nielsen (2020) gives an accessible introduction.

★ Key Concept: Geometry of estimation

$\mathrm{KL}(f\|g) \geq 0$ with equality iff $f = g$ (Jensen). The MLE minimises KL from the empirical distribution to the model, so it converges to the KL projection θ^* of the truth — the true parameter if the model is right, a pseudo-true value otherwise. Locally $\mathrm{KL}(f_\theta\|f_{\theta+\delta}) \approx \frac{1}{2}\delta^\top I(\theta)\delta$: the Fisher information is a Riemannian metric, invariant under reparametrisation. It yields the Jeffreys prior $\sqrt{\det I}$ and the natural gradient $I^{-1}\nabla\ell$. Cross-entropy loss is negative log-likelihood.

Exercises

7.5.1 Theory Derive (7.5). Find KL between two Poisson distributions and between two exponential distributions, and verify Theorem 7.31 for each.

7.5.2 Theory Prove (7.7). Compute the Fisher information of the Bernoulli model in the log-odds parametrisation $\eta = \log\{p/(1-p)\}$ directly and via (7.7).

7.5.3 Theory Show that the Fisher–Rao distance between Bernoulli(p_1) and Bernoulli(p_2) is $2\left|\arcsin\sqrt{p_1} - \arcsin\sqrt{p_2}\right|$. (Hint: change variables to $\phi = \arcsin\sqrt{p}$ and show that I_ϕ is constant.) Relate this to the classical variance-stabilising transformation for proportions.

7.5.4 Python Extend Listing 7.15 by running ordinary gradient ascent in the coordinates $(\mu, \log\sigma)$ and natural gradient ascent in both coordinate systems. Plot the four paths in the (μ,σ) plane and comment on (approximate) invariance.

7.5.5 ★ Graduate Show that for a regular exponential family $f(x;\theta) = h(x)\exp\{\theta^\top T(x) - A(\theta)\}$, $I(\theta) = \nabla^2 A(\theta)$ and $\mathrm{KL}(f_{\theta_1}\|f_{\theta_2}) = A(\theta_2) - A(\theta_1) - \nabla A(\theta_1)^\top(\theta_2-\theta_1)$, the Bregman divergence of A. Deduce that the MLE solves $\nabla A(\hat\theta) = \bar T$.

7.5.6 Policy A forecasting team evaluates two models of the probability that a district experiences a food-price spike next month, using the average log score (negative cross-entropy) on hold-out data. Explain why the log score is a *proper* scoring rule, how it relates to KL divergence, and why it punishes confident wrong forecasts much more heavily than the Brier score does.

7.6 Bayesian Estimation

7.6.1 Prior, likelihood and posterior

In the Bayesian approach the parameter is uncertain and that uncertainty is described by a probability distribution. Given a *prior* density $\pi(\theta)$ and the likelihood $L_n(\theta) = f(x_{1:n}\mid\theta)$, Bayes' theorem (Chapter 4) gives the *posterior*

$$\pi(\theta\mid x_{1:n}) = \frac{L_n(\theta)\,\pi(\theta)}{\int L_n(\vartheta)\pi(\vartheta)\,d\vartheta} \;\propto\; L_n(\theta)\,\pi(\theta). \tag{7.9}$$

The denominator, the *marginal likelihood* or *evidence*, does not depend on θ. The posterior combines both sources of information and is the complete answer to the estimation problem; point and interval summaries are derived from it.

Definition 7.33 (Bayesian summaries)**.** The *posterior mean* $\mathbb{E}[\theta\mid x]$ minimises the posterior expected squared error; the *posterior median* minimises the expected absolute error; the *maximum a posteriori* (MAP) estimate is $\hat\theta_{\mathrm{MAP}} = \arg\max_\theta\{\ell_n(\theta) + \log\pi(\theta)\}$. A 95% *credible*

> *interval* is any set C with $\mathbb{P}(\theta \in C \mid x) = 0.95$; the *equal-tailed* interval uses the 2.5% and 97.5% posterior quantiles.

A credible interval is a direct probability statement about θ given the data — the statement that is often (incorrectly) attached to a frequentist confidence interval. The price is the need to specify a prior.

Choosing a prior. Priors may be *informative* (earlier survey rounds, meta-analyses), *weakly informative* (e.g. $\mathcal{N}(0, 2.5^2)$ on a standardised logit coefficient) or *reference* priors such as Jeffreys' (Section 7.5). *Conjugate* priors give posteriors in the same family.

7.6.2 Conjugate examples

Example 7.34 (Three conjugate pairs).

1. *Beta–binomial.* With $y \mid p \sim \text{Bin}(n, p)$ and $p \sim \text{Beta}(a, b)$, $\pi(p \mid y) \propto p^{y}(1-p)^{n-y} p^{a-1}(1-p)^{b-1}$, so $p \mid y \sim \text{Beta}(a + y,\ b + n - y)$, with mean $(a+y)/(a+b+n)$ and mode $(a+y-1)/(a+b+n-2)$. The prior acts like a prior successes and b prior failures.

2. *Gamma–Poisson.* With $y_i \mid \lambda \overset{\text{iid}}{\sim} \text{Poisson}(\lambda)$ and $\lambda \sim \text{Gamma}(\alpha, \text{rate } \beta)$, $\lambda \mid y \sim \text{Gamma}(\alpha + \sum y_i,\ \beta + n)$.

3. *Normal–normal.* With $x_i \mid \mu \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$ (σ known) and $\mu \sim \mathcal{N}(\mu_0, \tau_0^2)$, $\mu \mid x \sim \mathcal{N}(\mu_n, \tau_n^2)$ with
$$\frac{1}{\tau_n^2} = \frac{1}{\tau_0^2} + \frac{n}{\sigma^2}, \qquad \mu_n = w\,\bar{x} + (1-w)\,\mu_0, \quad w = \frac{n/\sigma^2}{1/\tau_0^2 + n/\sigma^2}.$$
Precisions add, and the posterior mean *shrinks* the sample mean towards the prior mean with a data weight w that tends to 1 as $n \to \infty$.

Listing 7.16: Conjugate updating and Bernstein–von Mises ch07/bayes_conjugate.py

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Conjugate Bayesian updating: beta-binomial, gamma-Poisson, normal-normal.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)


def report(name, post, mle, se, mode):
    lo, hi = post.ppf([0.025, 0.975])
    print(f"{name:<22} {post.mean():8.4f} {mode:8.4f} [{lo:.4f}, {hi:.4f}]"
          f"  MLE {mle:.4f} +/- {1.96 * se:.4f}")


print(f"{'model (simulated)':<22} {'post mean':>8} {'MAP':>8}"
      f" {'95% credible interval':>20}")
# 1. Beta-binomial: programme take-up in a pilot of 40 households
a0, b0, n = 2.0, 8.0, 40                # prior mean 0.2 from earlier rounds
y = rng.binomial(n, 0.35)
post = stats.beta(a0 + y, b0 + n - y)
mode = (a0 + y - 1) / (a0 + b0 + n - 2)
ph = y / n
report(f"take-up, y={y}/{n}", post, ph, np.sqrt(ph * (1 - ph) / n), mode)
```

```
jeff = stats.beta(0.5 + y, 0.5 + n - y)
report("  same, Jeffreys prior", jeff, ph, np.sqrt(ph * (1 - ph) / n),
       (y - 0.5) / (n - 1))

# 2. Gamma-Poisson: weekly malaria cases at a clinic, 12 weeks
a0, r0, m = 20.0, 2.0, 12               # prior mean 10 cases per week
cases = rng.poisson(13.0, m)
post = stats.gamma(a0 + cases.sum(), scale=1 / (r0 + m))
lam = cases.mean()
report("cases/week, 12 weeks", post, lam, np.sqrt(lam / m),
       (a0 + cases.sum() - 1) / (r0 + m))

# 3. Normal-normal: mean log income, known sigma = 1.2
mu0, tau0, sig, k = 5.0, 0.5, 1.2, 30
x = rng.normal(5.6, sig, k)
prec = 1 / tau0**2 + k / sig**2
mpost = (mu0 / tau0**2 + x.sum() / sig**2) / prec
post = stats.norm(mpost, 1 / np.sqrt(prec))
report("mean log income, k=30", post, x.mean(), sig / np.sqrt(k), mpost)
w = (k / sig**2) / prec
print(f"  shrinkage weight on the data = {w:.3f}")

# Bernstein-von Mises: posterior sd / MLE se -> 1 as n grows (beta-binomial)
print("\nBeta(2, 8) prior, true p = 0.35:  n, posterior sd / Wald se,"
      " |post mean - MLE|")
for n in (10, 40, 200, 1000, 5000):
    y = rng.binomial(n, 0.35)
    post = stats.beta(2 + y, 8 + n - y)
    ph = y / n
    se = np.sqrt(ph * (1 - ph) / n)
    print(f"  n = {n:>5}: {post.std() / se:.4f}, {abs(post.mean() - ph):.4f}")
```

Output

```
model (simulated)      post mean      MAP 95% credible interval
take-up, y=14/40         0.3200   0.3125 [0.1995, 0.4542]  MLE 0.3500 +/- 0.1478
  same, Jeffreys prior   0.3537   0.3462 [0.2167, 0.5041]  MLE 0.3500 +/- 0.1478
cases/week, 12 weeks    12.3571  12.2857 [10.5843, 14.2652]  MLE 12.7500 +/- 2.0203
mean log income, k=30    5.5925   5.5925 [5.1992, 5.9858]  MLE 5.7062 +/- 0.4294
  shrinkage weight on the data = 0.839

Beta(2, 8) prior, true p = 0.35:  n, posterior sd / Wald se, |post mean - MLE|
  n =    10: 0.6901, 0.2000
  n =    40: 0.8661, 0.0300
  n =   200: 0.9689, 0.0069
  n =  1000: 0.9936, 0.0014
  n =  5000: 0.9987, 0.0003
```

In the take-up pilot (14 of 40 households), the Beta(2, 8) prior, centred on 0.2 from earlier rounds, pulls the posterior mean from the MLE 0.35 to 0.32, and the credible interval is narrower than the Wald interval because the prior adds the equivalent of ten observations. With the Jeffreys prior the posterior mean is essentially the MLE. In the normal–normal case the data receive weight 0.84. The last block illustrates the Bernstein–von Mises theorem below: as n grows, the posterior standard deviation approaches the frequentist standard error and the posterior mean approaches the MLE.

Policy Lens: Borrowing strength from earlier evidence

Programme pilots are often small. A Bayesian analysis lets a ministry combine a 40-household pilot with evidence from earlier rounds or neighbouring regions in a transparent way: the prior is stated explicitly, its weight ("ten households' worth") can be debated, and the sensitivity of the conclusions to it can be reported. The posterior probability that take-up exceeds a break-even threshold, $\mathbb{P}(p > p^{\dagger} \mid \text{data})$, is often exactly the quantity a decision-maker needs.

7.6.3 MAP estimation as penalised likelihood

Taking logarithms in (7.9),

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \left\{ \ell_n(\theta) + \log \pi(\theta) \right\} :$$

the log prior acts as a *penalty*. In the linear model $\mathbf{y} \mid \boldsymbol{\beta} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \sigma^2\mathbf{I})$ with prior $\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \tau^2\mathbf{I})$, the negative log-posterior is $\frac{1}{2\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2$+const, so

$$\hat{\boldsymbol{\beta}}_{\text{MAP}} = \mathbb{E}[\boldsymbol{\beta} \mid \mathbf{y}] = (\mathbf{X}^{\top}\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^{\top}\mathbf{y}, \qquad \lambda = \sigma^2/\tau^2, \tag{7.10}$$

which is *ridge regression* (Hoerl and Kennard, 1970). A Laplace prior gives the lasso, and the default L_2 penalty of scikit-learn's logistic regression (inverse strength C) is a $\mathcal{N}(0, C)$ prior on the slopes. Listing 7.17 verifies (7.10) on the diabetes data bundled with scikit-learn.

Listing 7.17: MAP with a Gaussian prior equals ridge regression ch07/map_ridge.py

```
# DEP 803 -- Chapter 7: Estimation Principles
# MAP with a Gaussian prior is ridge regression (diabetes data bundled in sklearn).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Ridge

X, y = load_diabetes(return_X_y=True)
X = (X - X.mean(0)) / X.std(0)          # standardise the 10 predictors
y = y - y.mean()
n, p = X.shape
sigma2 = 54.0**2                        # noise variance, treated as known
print(f"Diabetes data: n = {n}, p = {p}; sigma = 54 treated as known")
print(f"{'prior sd tau':>13} {'lambda':>9} {'||b_MAP - b_ridge||':>20}"
      f" {'||b_MAP||':>10}")
for tau in (100.0, 10.0, 3.0, 1.0):
    lam = sigma2 / tau**2
    # posterior: b | y ~ N(m, V), V = (X'X/s2 + I/tau2)^{-1}, m = V X'y/s2
    V = np.linalg.inv(X.T @ X / sigma2 + np.eye(p) / tau**2)
    m = V @ X.T @ y / sigma2
    br = Ridge(alpha=lam, fit_intercept=False).fit(X, y).coef_
    print(f"{tau:13.1f} {lam:9.1f} {np.linalg.norm(m - br):20.2e}"
          f" {np.linalg.norm(m):10.2f}")
ols = np.linalg.lstsq(X, y, rcond=None)[0]
print(f"{'(flat prior)':>13} {0.0:9.1f} {'OLS':>20} {np.linalg.norm(ols):10.2f}")
tau = 3.0
V = np.linalg.inv(X.T @ X / sigma2 + np.eye(p) / tau**2)
m = V @ X.T @ y / sigma2
print("Posterior for the bmi coefficient (tau = 3): mean %.2f, sd %.2f"
      % (m[2], np.sqrt(V[2, 2])))
```

Output

```
Diabetes data: n = 442, p = 10; sigma = 54 treated as known
 prior sd tau    lambda  ||b_MAP - b_ridge||  ||b_MAP||
        100.0       0.3             1.31e-12      62.71
         10.0      29.2             1.74e-13      39.24
          3.0     324.0             3.77e-14      26.91
          1.0    2916.0             7.85e-15       9.33
 (flat prior)       0.0                  OLS      65.54
Posterior for the bmi coefficient (tau = 3): mean 16.29, sd 2.11
```

The posterior mean and scikit-learn's ridge solution agree to machine precision for every prior scale. As the prior tightens (τ from 100 to 1) the coefficient vector shrinks from essentially the OLS norm (65.5) to 9.3. Unlike the penalised point estimate, the Bayesian analysis also delivers a posterior standard deviation (2.1 for the bmi coefficient).

△ Caution: MAP is not invariant

Unlike the MLE (Theorem 7.17), the MAP estimate depends on the parametrisation, because a density transforms with a Jacobian: the mode of the posterior of $\log \sigma$ is not the logarithm of the mode of the posterior of σ. The posterior median and quantile-based credible intervals *are* invariant under monotone transformations. Report MAP estimates with care, and never treat a penalised point estimate as a full Bayesian analysis.

7.6.4 The Bernstein–von Mises theorem

Theorem 7.35 (Bernstein–von Mises). *Let $X_i \overset{iid}{\sim} f(\cdot;\theta_0)$ in a regular parametric model (Assumption 7.19 plus mild conditions ensuring that the posterior concentrates), and let the prior have a continuous, positive density at θ_0. Then*

$$\sup_B \left| \Pi(\theta \in B \mid X_{1:n}) - \mathcal{N}\big(\hat\theta_n,\ \{nI(\theta_0)\}^{-1}\big)(B) \right| \xrightarrow{p} 0,$$

where $\hat\theta_n$ is the MLE (or any efficient estimator). That is, the posterior is asymptotically normal, centred at the MLE, with the inverse Fisher information as covariance.

A precise statement and proof are in van der Vaart (1998, Ch. 10). Two consequences are important in practice. First, in large regular problems Bayesian credible intervals and frequentist confidence intervals essentially coincide, and the prior "washes out". Second, the theorem can fail when its assumptions fail — many parameters relative to n, parameters on a boundary, weak identification, or misspecification (in which case the posterior concentrates at θ^* with the *non-robust* covariance A^{-1} rather than the sandwich). It is precisely in small samples and high-dimensional problems that the prior matters most.

7.6.5 Grid approximation

When the parameter has one or two dimensions, the posterior can be computed by brute force: evaluate $\log L_n(\theta) + \log \pi(\theta)$ on a fine grid, exponentiate after subtracting the maximum (to avoid underflow), and normalise so that the grid weights sum to one. Means, standard deviations, quantiles and posterior probabilities are then weighted sums. We apply this to a logit take-up model with intercept a and distance slope b for a pilot of only 30 simulated households, with independent $\mathcal{N}(0, 2.5^2)$ priors.

Listing 7.18: Grid approximation of a logit posterior (plotting code omitted) ch07/bayes_grid.py

```python
# DEP 803 -- Chapter 7: Estimation Principles
# Grid approximation of a two-parameter logit posterior: MAP, MLE, posterior mean.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import special

rng = np.random.default_rng(803)
n = 30                                  # small pilot survey (simulated)
dist = rng.uniform(0, 5, n)             # km to enrolment office
y = rng.binomial(1, special.expit(1.0 - 0.6 * dist))

a = np.linspace(-3, 5, 401)             # intercept grid
b = np.linspace(-2.5, 1.0, 401)         # slope grid
A, B = np.meshgrid(a, b, indexing="ij")
eta = A[..., None] + B[..., None] * dist          # (401, 401, n)
loglik = np.sum(y * eta - np.logaddexp(0, eta), axis=-1)
tau = 2.5                               # N(0, tau^2) priors on a and b
logprior = -(A**2 + B**2) / (2 * tau**2)
logpost = loglik + logprior
post = np.exp(logpost - logpost.max())
post /= post.sum()                      # normalise on the grid

pa, pb = post.sum(axis=1), post.sum(axis=0)       # marginals
mean_a, mean_b = pa @ a, pb @ b
sd_b = np.sqrt(pb @ (b - mean_b) ** 2)
cdf_b = np.cumsum(pb)
lo, hi = b[np.searchsorted(cdf_b, 0.025)], b[np.searchsorted(cdf_b, 0.975)]
i, j = np.unravel_index(np.argmax(logpost), logpost.shape)
k, l = np.unravel_index(np.argmax(loglik), loglik.shape)
p_neg = pb[b < 0].sum()
print(f"Logit take-up posterior on a {a.size} x {b.size} grid, n = {n}")
print(f"  MLE  (grid)      : a = {a[k]:6.3f}, b = {b[l]:6.3f}")
print(f"  MAP  (grid)      : a = {a[i]:6.3f}, b = {b[j]:6.3f}")
print(f"  posterior mean   : a = {mean_a:6.3f}, b = {mean_b:6.3f}")
print(f"  posterior sd of b = {sd_b:.3f}; 95% credible interval for b:"
      f" [{lo:.3f}, {hi:.3f}]")
print(f"  P(b < 0 | data)  = {p_neg:.4f}")
# Posterior predictive take-up probability at 4 km from the office
pr4 = np.sum(post * special.expit(A + 4 * B))
print(f"  posterior predictive P(take-up | 4 km) = {pr4:.3f}")
```

Output

```
Logit take-up posterior on a 401 x 401 grid, n = 30
  MLE  (grid)      : a =  0.480, b = -0.557
  MAP  (grid)      : a =  0.420, b = -0.531
  posterior mean   : a =  0.488, b = -0.595
  posterior sd of b = 0.343; 95% credible interval for b: [-1.310, 0.038]
  P(b < 0 | data)  = 0.9661
  posterior predictive P(take-up | 4 km) = 0.154
```

With $n = 30$ the posterior is skewed (Fig. 7.7); the MAP lies between the MLE and the prior mode 0, while the posterior mean of the slope (-0.595) is further from zero than both, because

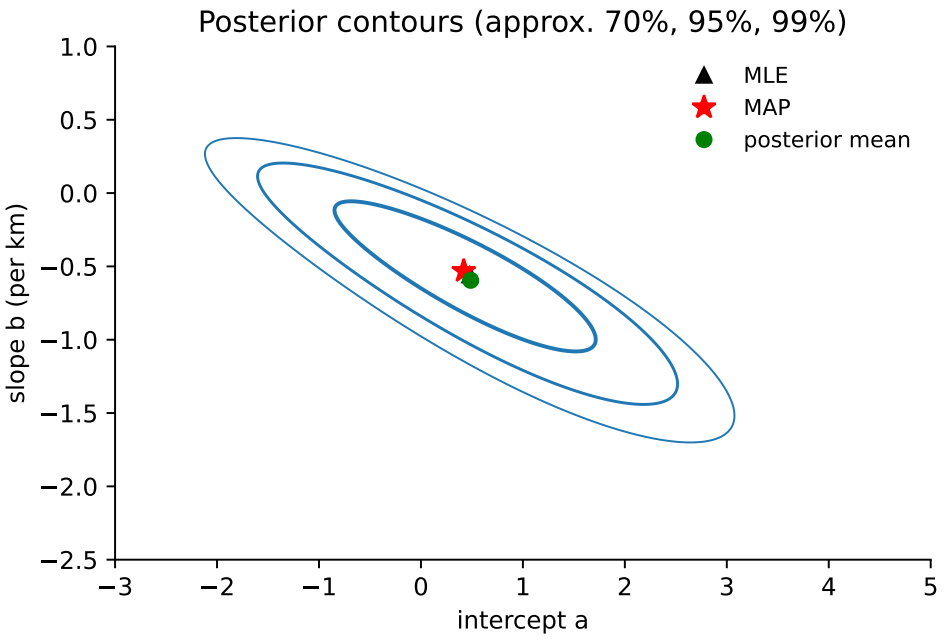


Figure 7.7: Posterior of the intercept and distance slope in a logit take-up model with $n = 30$ simulated households, computed on a 401×401 grid. The posterior is clearly non-elliptical and the MLE, MAP and posterior mean differ (Listing 7.18).

the posterior has a long tail towards steep negative slopes. The 95% credible interval for b just includes 0, yet the posterior probability that distance *reduces* take-up is 0.97 — a statement with a direct decision-theoretic interpretation. The posterior predictive take-up at 4 km, 0.154, averages the logistic curve over parameter uncertainty.

△ Caution: The curse of dimensionality for grids

A grid with 400 points per dimension needs 400^p evaluations: feasible for $p = 2$ (1.6×10^5), impossible for $p = 10$ (10^{26}). Beyond two or three parameters we need Monte Carlo methods — importance sampling, Markov chain Monte Carlo and Hamiltonian Monte Carlo (Chapter 10) — or analytic approximations such as the Laplace approximation (Theorem 7.35) and variational inference (Section 7.5).

7.6.6 Probabilistic programming

Probabilistic programming languages let the analyst write the model — data, parameters, priors and likelihood — in a few declarative lines, and leave the computation of the posterior to a general-purpose algorithm. Stan (Carpenter et al., 2017) uses the No-U-Turn variant of Hamiltonian Monte Carlo, which exploits the gradient of the log posterior. Matsuura (2022), one of the course texts, develops Bayesian statistical modelling with Stan from simple regressions to hierarchical, time-series and spatial models. The logit model of Listing 7.18 would be written in Stan as follows (shown for reference only; Stan is *not* required for this course).

```
data       { int<lower=0> N; vector[N] dist; array[N] int<lower=0, upper=1> y; }
parameters { real a; real b; }
model      { a ~ normal(0, 2.5); b ~ normal(0, 2.5);
             y ~ bernoulli_logit(a + b * dist); }
```

From Python the model can be sampled with CmdStanPy; posterior summaries are then averages over the draws.

Graduate Perspective: Hierarchical models and empirical Bayes

The natural next step is a *hierarchical* model: district-level take-up rates $p_j \sim \text{Beta}(a, b)$ with (a, b) themselves estimated, so that small districts borrow strength from the whole country. Estimating (a, b) by maximising the marginal likelihood is *empirical Bayes*; placing a hyperprior on them is fully Bayesian. Both yield shrinkage estimators that dominate the separate district MLEs in total MSE — the Stein phenomenon — and are central to small-area estimation of poverty (Gelman et al., 2013; Efron and Hastie, 2016).

Historical Note: Jeffreys and the revival of Bayesian inference

Harold Jeffreys, a Cambridge geophysicist, defended inverse (Bayesian) probability against Fisher's criticisms in his *Theory of Probability* (1939). In a 1946 paper in the *Proceedings of the Royal Society* (Jeffreys, 1946) he proposed the prior proportional to the square root of the determinant of the Fisher information, chosen precisely because it is invariant under reparametrisation. Interest in Bayesian methods grew through the work of Savage, de Finetti and Lindley in the 1950s and 1960s, but routine application to realistic models became possible only with MCMC algorithms in the 1990s and probabilistic programming languages such as BUGS and later Stan.

⋆ Key Concept: Bayesian estimation

Posterior $\propto$ likelihood $\times$ prior. Conjugate pairs (beta–binomial, gamma–Poisson, normal–normal) give closed forms in which the prior acts like pseudo-observations and the posterior mean shrinks the MLE towards the prior mean. MAP = penalised ML: a Gaussian prior gives ridge. Credible intervals are direct probability statements about θ. By Bernstein–von Mises, regular posteriors become $\mathcal{N}(\hat{\theta}, \{nI\}^{-1})$ in large samples. Low-dimensional posteriors can be computed on a grid; otherwise use MCMC or Stan.

Exercises

7.6.1 Theory Derive the three conjugate posteriors in Example 7.34. For the beta–binomial, show that the posterior mean is a weighted average of the prior mean and the MLE, and find the weight.

7.6.2 Theory Derive (7.10) by completing the square, and show that the posterior covariance is $\sigma^2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}$. What prior gives the lasso as its MAP estimator?

7.6.3 Python Using the gamma–Poisson model of Listing 7.16, compute the posterior predictive distribution of next week's malaria cases (a negative binomial) and the posterior probability that weekly cases exceed 15. Check your answer by simulation.

7.6.4 Python Rerun Listing 7.18 with $n \in \{30, 100, 1000\}$ (keeping the same DGP). Compare the posterior standard deviation of b with the standard error from the inverse Hessian at the MLE, and plot the posterior contours. Relate what you see to Theorem 7.35.

7.6.5 ★ Graduate *Prior sensitivity.* For the pilot in Listing 7.16, compute $\mathbb{P}(p > 0.3 \mid y)$ under the priors Beta(2, 8), Beta(1, 1), Beta($\frac{1}{2}, \frac{1}{2}$) and Beta(20, 80). Write a short paragraph on how such a sensitivity analysis should be reported. Then derive the empirical Bayes estimator of (a, b) from a simulated set of 50 district pilots.

7.6.6 Policy A cash-transfer programme is cost-effective if take-up exceeds 30%. A 40-household pilot finds 14 participants. Using a prior based on a neighbouring region's take-up of 20% (equivalent to 10 households), give the posterior probability that the programme is cost-effective, and discuss whether the decision should depend on this probability alone.

Chapter Summary

- An **estimator** is a procedure with a sampling distribution; MSE = Bias^2 + Var. Large-sample properties — consistency, asymptotic normality, efficiency — are evaluated by theory or by Monte Carlo. Efficiency depends on the data distribution: the median beats the mean under heavy tails.
- The **method of moments** matches sample and population moments. **GMM** minimises $\bar{g}_n^\top W \bar{g}_n$ for moment conditions $\mathbb{E}g(X, \theta_0) = 0$; the efficient weight is S^{-1}; the J-test checks over-identifying restrictions. OLS, IV and 2SLS are GMM estimators; IV removes endogeneity bias at the cost of variance, and weak instruments destroy the normal approximation.
- The **MLE** maximises $\sum \log f(X_i; \theta)$, solves the score equations, is invariant to reparametrisation and underlies logit, probit and Poisson regression. Numerical MLE needs reparametrisation, gradients, stable formulas and vigilance against non-existence (separation).
- The score has mean zero and variance $I(\theta) = -\mathbb{E}\nabla^2 \log f$. The **Cramér–Rao** bound $1/(nI)$ limits unbiased estimators and the MLE attains it asymptotically. Standard errors come from the observed information or, under misspecification, the **sandwich** $A^{-1}BA^{-1}$. Wald, LR and score tests are asymptotically χ^2 and equivalent, but not in small samples.
- **Information geometry**: KL divergence is non-negative (Jensen); the MLE is the KL projection of the empirical distribution onto the model and converges to a pseudo-true value under misspecification; the Fisher information is a Riemannian metric that gives the Jeffreys prior and the natural gradient. Cross-entropy loss is negative log-likelihood.
- **Bayesian estimation**: posterior $\propto$ likelihood $\times$ prior; conjugate updating shrinks towards the prior; MAP is penalised likelihood (ridge = Gaussian prior); Bernstein–von Mises reconciles Bayesian and frequentist answers in large regular models; grids, MCMC and Stan compute posteriors in practice.

Further Reading

Casella and Berger (2002) (Chs. 7 and 10) and Wasserman (2004) (Chs. 9–11) are the standard graduate introductions to point estimation, the Cramér–Rao bound, asymptotic theory of the MLE and the trinity of tests; van der Vaart (1998) gives the rigorous asymptotic theory, including Le Cam's theory and the Bernstein–von Mises theorem. For econometric applications — GMM, instrumental variables, quasi-maximum likelihood and robust inference — Hansen (2022), Hayashi (2000), Wooldridge (2010) and Cameron and Trivedi (2005) are excellent, with Newey and McFadden (1994) the reference for the general theory of extremum estimators and Angrist and Pischke (2009) for the interpretation of IV estimates in policy evaluation. Murphy (2022, Chs. 4–5) presents MLE, MAP, Bayesian estimation and the cross-entropy view from the machine-learning side, and Géron (2022, Ch. 4) shows the same loss functions at work in scikit-learn. Matsuura (2022) is the course text on Bayesian modelling with Stan, and Gelman et al. (2013) the standard reference on applied Bayesian data analysis. Goulet Coulombe et al. (2022) show, in macroeconomic forecasting, how the choice of loss function and of regularisation — both likelihood and prior ideas in disguise — shapes the gains from machine learning. For information geometry, Amari and Nagaoka (2000) and Nielsen (2020) are good starting points, and Amari (1998) introduces the natural gradient. Stigler (2007) tells the history of maximum likelihood, and Efron and Hastie (2016) places all of these ideas in the context of modern computer-age statistics.

Part III

Optimisation

Chapter 8

Optimisation: Convexity, Gradients, Hessians, Newton and Stochastic Gradient Methods

Learning Objectives

After working through this chapter you should be able to:

1. formulate estimation and learning problems as *empirical risk minimisation*, recognise OLS, maximum likelihood, least absolute deviations and Huber regression as M-estimators, and state when a minimiser exists (Weierstrass, coercivity);
2. compute gradients, Jacobians and Hessians, use Taylor's theorem to prove the first- and second-order optimality conditions, classify stationary points, and verify analytic gradients with finite differences;
3. prove the first- and second-order characterisations of convexity, use strong convexity and L-smoothness to quantify the difficulty of a problem, and show that the logit log-likelihood is concave;
4. implement gradient descent with fixed steps, Armijo backtracking, heavy-ball momentum and Nesterov acceleration, prove the $O(1/k)$ and linear convergence rates, and explain the role of the condition number and of feature scaling;
5. derive Newton's method, prove its local quadratic convergence, apply Newton–Raphson/IRLS to a logit model and Gauss–Newton to nonlinear least squares, and use BFGS and L-BFGS through `scipy.optimize.minimize`;
6. explain stochastic approximation, implement minibatch SGD with learning-rate schedules and Adam in NumPy, read loss curves, and connect these tools to `SGDClassifier` and to neural network training.

Almost every quantitative method in this book ends with the word "minimise". The ordinary least squares estimator of Chapter 2 minimises a sum of squared residuals; the maximum likelihood estimators of Chapter 7 maximise a log-likelihood; principal components (Chapter 3) maximise a variance; a lasso regression, a random forest split, a neural network and a structural model of household demand are all defined as the solution of an optimisation problem. For a small linear model the solution has a closed form and the optimisation is invisible. As soon as the model is nonlinear (a logit for programme take-up, a diffusion curve for mobile-money adoption, a discrete-choice model of school enrolment) or the data set is large (an administrative register with millions of records), the analyst must choose an *algorithm*, and that choice determines

whether the estimate is computed in seconds, in hours, or not at all.

This chapter develops the mathematics that makes this choice rational. We first describe the problems (Section 8.1) and the calculus needed to characterise their solutions (Section 8.2). Convexity (Section 8.3) is the single most important structural property: for convex problems every local solution is global and simple algorithms come with guarantees. We then study the three families of algorithms that dominate practice: gradient descent and its accelerated variants (Section 8.4), Newton and quasi-Newton methods (Section 8.5), and stochastic gradient methods (Section 8.6), which power modern machine learning. Constrained problems (budget constraints, non-negativity, the lasso) are the subject of Chapter 9; this chapter treats unconstrained problems.

Throughout, the emphasis is the one a policy analyst needs: an estimate is only as credible as the optimiser that produced it. A logit that "converged" to a saddle point, a structural model stuck in a local optimum, or a machine-learning model trained with a badly chosen learning rate can each lead to wrong policy conclusions. Knowing *why* algorithms work also tells you how they fail. Our main references are Nocedal and Wright (2006) and Boyd and Vandenberghe (2004), with Bottou et al. (2018) for stochastic methods and Murphy (2022, Ch. 8) and Géron (2022, Ch. 4 and 11) for the machine-learning perspective.

8.1 Optimisation in data science and econometrics

8.1.1 The generic problem

An *unconstrained optimisation problem* asks for

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^p} f(\boldsymbol{\theta}), \tag{8.1}$$

where $f : \mathbb{R}^p \to \mathbb{R}$ is the *objective* (also *loss*, *cost* or *criterion*). Maximisation is the same problem with f replaced by $-f$; we always write problems as minimisations.

Definition 8.1 (Local and global minimisers)**.** A point $\boldsymbol{\theta}^\star$ is a *global minimiser* of f if $f(\boldsymbol{\theta}^\star) \le f(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$. It is a *local minimiser* if there is $r > 0$ such that $f(\boldsymbol{\theta}^\star) \le f(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$ with $\|\boldsymbol{\theta} - \boldsymbol{\theta}^\star\| < r$; it is a *strict* local minimiser if the inequality is strict for $\boldsymbol{\theta} \neq \boldsymbol{\theta}^\star$. The value $f^\star = \inf_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$ is the *optimal value*; the set of minimisers is written $\arg\min f$.

The infimum always exists in $[-\infty, \infty)$, but it need not be attained: $f(\theta) = e^\theta$ has $f^\star = 0$ and no minimiser. Existence is therefore a genuine question, and for estimation it is a practical one (see the caution box on logit separation below).

8.1.2 Empirical risk minimisation and M-estimators

In data science the objective usually has the form of an average over observations.

Definition 8.2 (Empirical risk; M-estimator)**.** Let $z_1, \dots, z_n$ be data (for example $z_i = (y_i, \mathbf{x}_i)$) and $\rho(z, \boldsymbol{\theta})$ a *loss* measuring how badly parameter $\boldsymbol{\theta}$ fits observation z. The *empirical risk* is

$$R_n(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \rho(z_i, \boldsymbol{\theta}), \tag{8.2}$$

and an *M-estimator* ("M" for "minimisation" or "maximum-likelihood type") is any $\hat{\boldsymbol{\theta}}_n \in \arg\min_{\boldsymbol{\theta}} R_n(\boldsymbol{\theta})$. The population counterpart is the *risk* $R(\boldsymbol{\theta}) = \mathbb{E}[\rho(Z, \boldsymbol{\theta})]$; under regularity conditions $\hat{\boldsymbol{\theta}}_n \xrightarrow{p} \boldsymbol{\theta}_0 = \arg\min R$.

The four estimators below are the workhorses of applied econometrics; all are M-estimators for a linear index $\mathbf{x}_i^\top \boldsymbol{\beta}$ with residual $r_i = y_i - \mathbf{x}_i^\top \boldsymbol{\beta}$.

Estimator	loss ρ	estimates
OLS	$\tfrac{1}{2}r^2$	conditional mean $\mathbb{E}[y \mid \mathbf{x}]$
LAD (median regression)	$\lvert r \rvert$	conditional median
Huber regression	$\rho_c(r)$, see (8.3)	robust location
Maximum likelihood	$-\log f(y_i \mid \mathbf{x}_i; \boldsymbol{\theta})$	parameters of a model density

where the Huber loss with tuning constant $c > 0$ is

$$\rho_c(r) = \begin{cases} \tfrac{1}{2}r^2, & \lvert r \rvert \le c, \\ c\,\lvert r \rvert - \tfrac{1}{2}c^2, & \lvert r \rvert > c. \end{cases} \tag{8.3}$$

Gaussian maximum likelihood with known variance reproduces OLS, and Laplace (double-exponential) maximum likelihood reproduces LAD, so the table illustrates a general principle: *choosing a loss is choosing what feature of the conditional distribution you estimate and how much you trust the tails*. Least absolute deviations dates back to Boscovich and Laplace in the eighteenth century and was put on a modern footing as quantile regression by Koenker and Bassett (1978); the Huber loss was introduced by Huber (1964), and the tuning constant $c = 1.345$ (in units of the error standard deviation) gives 95% efficiency relative to OLS when the errors really are normal.

Example 8.3 (Three M-estimators of the returns to schooling)**.** Listing 8.1 simulates a survey of $n = 400$ workers with log earnings $\log w_i = 1 + 0.08\,educ_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, 0.3^2)$, and contaminates 5% of the records with gross recording errors (log earnings inflated by 2 to 4). Each estimator minimises its own empirical risk numerically. OLS is pulled upwards in the intercept by the outliers; LAD and Huber, whose losses grow only linearly in the tails (Fig. 8.1), stay close to the truth.

Listing 8.1: OLS, LAD and Huber regression as numerical M-estimators `ch08/ch08_mestimators.py`

```
# DEP 803 -- Chapter 8: Optimisation
# M-estimators (OLS, LAD, Huber) on a simulated earnings survey with outliers
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

rng = np.random.default_rng(803)
n = 400
educ = rng.uniform(0, 16, n)                       # years of schooling
logw = 1.0 + 0.08 * educ + rng.normal(0, 0.3, n)   # simulated log earnings
bad = rng.choice(n, 20, replace=False)             # 5% gross recording errors
logw[bad] += rng.uniform(2.0, 4.0, 20)
X = np.column_stack([np.ones(n), educ])


def huber(r, c=1.345):
    a = np.abs(r)
    return np.where(a <= c, 0.5 * r**2, c * a - 0.5 * c**2)


losses = {
    "OLS (squared)": lambda r: 0.5 * r**2,
```

```
    "LAD (absolute)": lambda r: np.abs(r),
    "Huber (c=1.345)": lambda r: huber(r / 0.3),   # residual scaled by sigma
}
b0 = np.linalg.lstsq(X, logw, rcond=None)[0]       # starting value
print(f"{'estimator':<18}{'intercept':>10}{'slope':>9}{'iters':>7}{'risk':>10}")
print(f"{'truth':<18}{1.0:>10.4f}{0.08:>9.4f}")
for name, rho in losses.items():
    risk = lambda b, rho=rho: np.mean(rho(logw - X @ b))
    res = minimize(risk, b0, method="Nelder-Mead",
                   options={"xatol": 1e-8, "fatol": 1e-10, "maxiter": 4000})
    b = res.x
    print(f"{name:<18}{b[0]:>10.4f}{b[1]:>9.4f}{res.nit:>7d}{res.fun:>10.5f}")

# figure: the three loss functions rho(r)
r = np.linspace(-3, 3, 400)
plt.figure(figsize=(6, 3.8))
plt.plot(r, 0.5 * r**2, label=r"squared $\frac{1}{2}r^2$")
plt.plot(r, np.abs(r), label=r"absolute $|r|$")
plt.plot(r, huber(r), label="Huber, c = 1.345")
plt.ylim(0, 4)
plt.gca().set(xlabel="residual r", ylabel=r"loss $\rho(r)$",
              title="Loss functions behind three M-estimators")
plt.legend(frameon=False)
plt.savefig("ch08_losses.pdf", bbox_inches="tight")
```

Output

```
estimator          intercept    slope  iters      risk
truth                 1.0000   0.0800
OLS (squared)         1.1459   0.0788     47   0.24193
LAD (absolute)        1.0361   0.0785     75   0.36251
Huber (c=1.345)       1.0257   0.0791     54   1.01320
```

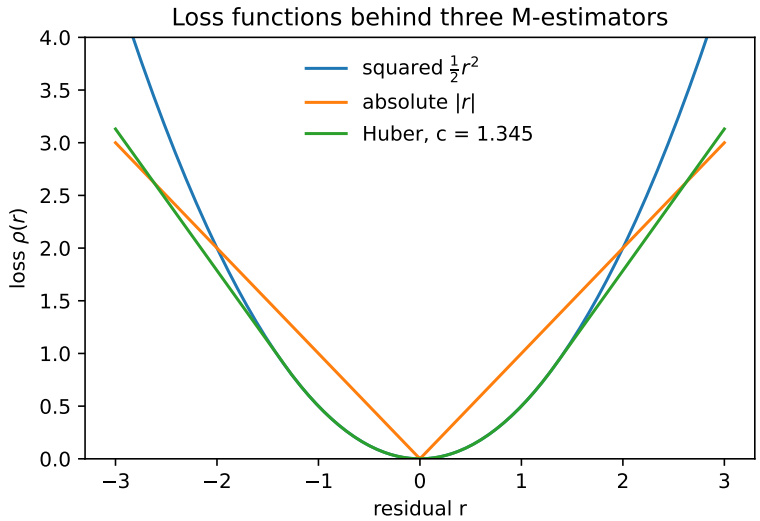


Figure 8.1: Loss functions of three M-estimators. The squared loss grows quadratically, so a single large residual dominates the risk; the absolute and Huber losses grow linearly and bound the influence of outliers. The Huber loss is quadratic (smooth) near zero.

Two remarks on Listing 8.1. First, the derivative-free Nelder–Mead method was used because the LAD loss is not differentiable at $r = 0$; for smooth problems derivative-based methods are far more efficient (Section 8.5). Second, the "risk" column is not comparable across rows: each estimator minimises a *different* function, which is exactly the point.

⋆ Key Concept: Estimation is optimisation

An M-estimator is *defined* as the minimiser of an empirical risk $R_n(\boldsymbol{\theta}) = n^{-1}\sum_i \rho(z_i, \boldsymbol{\theta})$. The loss ρ encodes the statistical target (mean, median, robust location, likelihood); the optimisation algorithm is merely the means of computing the minimiser. Statistical properties (consistency, standard errors) belong to the estimator; speed, accuracy and failure modes belong to the algorithm. Keep the two apart.

8.1.3 Existence of minimisers

Theorem 8.4 (Weierstrass extreme value theorem)**.** *If $K \subset \mathbb{R}^p$ is non-empty and compact (closed and bounded) and $f : K \to \mathbb{R}$ is continuous, then f attains its minimum and its maximum on K.*

Proof. Let $f^\star = \inf_K f \in [-\infty, \infty)$ and choose $\boldsymbol{\theta}_k \in K$ with $f(\boldsymbol{\theta}_k) \to f^\star$. Because K is bounded, the Bolzano–Weierstrass theorem gives a convergent subsequence $\boldsymbol{\theta}_{k_j} \to \boldsymbol{\theta}^\star$, and $\boldsymbol{\theta}^\star \in K$ because K is closed. Continuity gives $f(\boldsymbol{\theta}^\star) = \lim_j f(\boldsymbol{\theta}_{k_j}) = f^\star$; in particular $f^\star > -\infty$. The maximum is the minimum of $-f$. □

Estimation problems are rarely posed on a compact set, so we need a version for $\mathbb{R}^p$.

Definition 8.5 (Coercivity)**.** $f : \mathbb{R}^p \to \mathbb{R}$ is *coercive* if $f(\boldsymbol{\theta}) \to \infty$ whenever $\|\boldsymbol{\theta}\| \to \infty$.

Corollary 8.6. *A continuous coercive function $f : \mathbb{R}^p \to \mathbb{R}$ has a global minimiser.*

Proof. Fix any $\boldsymbol{\theta}_0$. By coercivity there is r with $f(\boldsymbol{\theta}) > f(\boldsymbol{\theta}_0)$ whenever $\|\boldsymbol{\theta}\| > r$. The sublevel set $S = \{\boldsymbol{\theta} : f(\boldsymbol{\theta}) \le f(\boldsymbol{\theta}_0)\}$ is closed (by continuity), contained in the ball of radius r, hence compact and non-empty. By Theorem 8.4 f has a minimiser on S, and points outside S have larger values. □

Example 8.7 (Existence for OLS, LAD and Huber)**.** Let $\mathbf{X}$ have full column rank. Then $\|\mathbf{X}\boldsymbol{\beta}\| \ge \sigma_{\min}\|\boldsymbol{\beta}\|$ with $\sigma_{\min} > 0$ the smallest singular value (Chapter 3), so $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\| \ge \sigma_{\min}\|\boldsymbol{\beta}\| - \|\mathbf{y}\| \to \infty$. Each of the squared, absolute and Huber losses is continuous and grows at least linearly in $|r|$, so all three empirical risks are coercive and have minimisers. Without full rank, coercivity fails along null($\mathbf{X}$) and minimisers exist but are not unique.

△ Caution: Maximum likelihood may not exist: separation in logit models

For the logit model the negative log-likelihood $\sum_i [\log(1 + e^{\mathbf{x}_i^\top \boldsymbol{\beta}}) - y_i \mathbf{x}_i^\top \boldsymbol{\beta}]$ is *not* coercive if the data are *separable*: if some $\boldsymbol{\beta}_0$ satisfies $\mathbf{x}_i^\top \boldsymbol{\beta}_0 > 0$ whenever $y_i = 1$ and $\mathbf{x}_i^\top \boldsymbol{\beta}_0 < 0$ whenever $y_i = 0$, then the likelihood increases forever along $t\boldsymbol{\beta}_0$, $t \to \infty$, and no MLE exists (Albert and Anderson, 1984). In small programme evaluations (for example a binary covariate that perfectly predicts take-up in one district) software will report huge coefficients and standard errors, or "convergence" after hitting an iteration cap. Always check for separation before interpreting a logit.

8.1.4 Local versus global optima

For non-convex objectives an algorithm that only looks at local information (values, gradients, Hessians near the current point) can only be expected to find a *local* minimiser. A cheap, widely used safeguard is *multi-start*: run a local method from many starting values and keep the best result. Listing 8.2 applies this to the one-dimensional function $f(x) = 0.1x^2 + \sin 3x + \frac{1}{2}\cos 5x$, which has many local minima.

Listing 8.2: Multi-start local search on a non-convex function ch08/ch08_weierstrass.py

```
# DEP 803 -- Chapter 8: Optimisation
# Local versus global minima: multi-start search on a non-convex risk
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy.optimize import minimize


def f(x):
    """A non-convex one-dimensional 'risk' with several local minima."""
    return 0.1 * x**2 + np.sin(3 * x) + 0.5 * np.cos(5 * x)


rng = np.random.default_rng(803)
starts = rng.uniform(-6, 6, 12)
found = []
for s in starts:
    res = minimize(lambda z: f(z[0]), [s], method="BFGS")
    found.append((s, res.x[0], res.fun))
print(f"{'start':>8}{'local min x*':>14}{'f(x*)':>10}")
for s, xs, fs in sorted(found, key=lambda t: t[0]):
    print(f"{s:>8.3f}{xs:>14.4f}{fs:>10.4f}")
xs = np.array([t[1] for t in found])
uniq = np.unique(np.round(xs, 3))
print("distinct local minima found:", len(uniq))
best = min(found, key=lambda t: t[2])
grid = np.linspace(-6, 6, 200001)
print(f"best multi-start: x = {best[1]:.4f}, f = {best[2]:.4f}")
print(f"grid search     : x = {grid[np.argmin(f(grid))]:.4f}, "
      f"f = {f(grid).min():.4f}")
```

Output

```
   start  local min x*     f(x*)
  -4.101       -4.4820    0.7816
  -3.397       -2.1739    0.1735
  -1.288       -0.5789   -1.4376
  -1.118       -0.5789   -1.4376
  -0.535       -0.5789   -1.4376
   0.092       -0.5789   -1.4376
   0.510        1.7314   -0.9461
   0.542        1.7314   -0.9461
   1.185        1.7314   -0.9461
   2.030        1.7314   -0.9461
   2.328        1.7314   -0.9461
   4.059        3.3177    0.2783
distinct local minima found: 5
```

```
best multi-start: x = -0.5789, f = -1.4376
grid search     : x = -0.5789, f = -1.4376
```

Twelve random starts found five distinct local minima; the starts nearest the origin reached the global one, confirmed by a fine grid. In $p = 1$ a grid is cheap; in $p = 20$ a grid with 100 points per axis has 10^{40} points, which is why convexity (Section 8.3) is so valuable: it rules out the problem altogether.

Historical Note: Fermat and the method of maxima and minima

Around 1636–1637 Pierre de Fermat circulated a manuscript, *Methodus ad disquirendam maximam et minimam*, describing how to find extreme values of a polynomial quantity: compare the value at x with the value at a slightly perturbed point $x + e$, treat the two as "adequal", divide by e and then set e to zero. In modern language this is setting the derivative to zero, decades before Newton and Leibniz developed calculus. Fermat applied the idea to geometric problems such as tangents to curves and, later, to the refraction of light (the principle of least time). The condition $\nabla f(\boldsymbol{\theta}^\star) = \mathbf{0}$ proved in Theorem 8.13 is still called *Fermat's rule*.

Policy Lens: Estimating a structural demand model

Structural models of demand, such as random-coefficient logit models of choice between health facilities, schools or food products (Berry et al., 1995), are estimated by minimising a GMM or likelihood criterion over parameters that enter nonlinearly through market-share inversions. The criterion is typically non-convex, evaluation is expensive, and the literature contains well-documented cases where published estimates changed materially when the optimisation was redone with tighter tolerances, better starting values and analytic gradients. For policy simulations (the effect of a fee abolition, a tax on sugary drinks, the opening of a new clinic) the counterfactual is only as reliable as the optimum. Good practice: analytic gradients (Section 8.2), multiple starting values, tight convergence tolerances, and a report of the gradient norm and Hessian eigenvalues at the solution.

Exercises

8.1.1 Theory Show that the minimiser of $\sum_i (y_i - m)^2$ over $m \in \mathbb{R}$ is the sample mean and that any minimiser of $\sum_i |y_i - m|$ is a sample median. What is the set of minimisers of the second problem when n is even?

8.1.2 Theory Show that the Huber loss ρ_c is continuously differentiable with derivative $\psi_c(r) = \rho_c'(r) = \max\{-c, \min(c, r)\}$, but not twice differentiable at $|r| = c$. Show that Gaussian MLE with known variance and Laplace MLE with known scale are equivalent to OLS and LAD respectively.

8.1.3 Theory Give an example of a continuous function on $\mathbb{R}$ that is bounded below but has no minimiser, and of a continuous function on a bounded but open interval with no minimiser. Which hypothesis of Theorem 8.4 fails in each case?

8.1.4 Python Modify Listing 8.1 to vary the share of contaminated records from 0% to 20%. Plot the estimated slope and intercept against the contamination share for the three estimators. At what share does OLS become unreliable for the intercept?

8.1.5 Python In Listing 8.2, how many random starts are needed (on average over 100 replications) for multi-start BFGS to find the global minimum with probability 0.95? Relate the answer to the width of the global minimum's basin of attraction.

8.1.6 Policy A district health directorate's data set contains a binary indicator "village has a community health worker" and every such village had full immunisation take-up. Explain what happens when a logit of take-up on this indicator is estimated, why the MLE does not exist, and two remedies (penalised likelihood, exact or Firth-type methods, or redefining the model).

8.1.7 ★ Graduate Let $\rho(z, \boldsymbol{\theta})$ be continuous in $\boldsymbol{\theta}$ and suppose $\boldsymbol{\theta}$ ranges over a compact set Θ. Using Theorem 8.4, prove that an M-estimator exists for every sample. Explain why consistency arguments for M-estimators (uniform convergence of R_n to R, identification) are usually stated on compact parameter spaces (see van der Vaart, 1998, Ch. 5).

8.2 Multivariate calculus for optimisation

The algorithms of this chapter use local information: function values, first derivatives and sometimes second derivatives. This section collects the calculus they rest on and proves the classical optimality conditions. Matrix-calculus rules for differentiating expressions such as $\boldsymbol{\beta}^\top \mathbf{A}\boldsymbol{\beta}$ are developed systematically in Chapter 12 and in Magnus and Neudecker (2019).

8.2.1 Gradients, directional derivatives and Jacobians

Definition 8.8 (Gradient, directional derivative, Jacobian)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ be differentiable at $\boldsymbol{\theta}$. The *gradient* is the column vector $\nabla f(\boldsymbol{\theta}) = (\partial f/\partial\theta_1, \dots, \partial f/\partial\theta_p)^\top$. The *directional derivative* in direction $\mathbf{v}$ is

$$D_{\mathbf{v}} f(\boldsymbol{\theta}) = \lim_{t \to 0} \frac{f(\boldsymbol{\theta} + t\mathbf{v}) - f(\boldsymbol{\theta})}{t} = \nabla f(\boldsymbol{\theta})^\top \mathbf{v}.$$

For a vector-valued map $\mathbf{F} : \mathbb{R}^p \to \mathbb{R}^m$ the *Jacobian* $J_{\mathbf{F}}(\boldsymbol{\theta})$ is the $m \times p$ matrix with entries $\partial F_i/\partial\theta_j$; row i is $\nabla F_i^\top$.

Proposition 8.9 (Steepest descent direction)**.** *If $\nabla f(\boldsymbol{\theta}) \neq \mathbf{0}$, the unit vector that minimises $D_{\mathbf{v}} f(\boldsymbol{\theta})$ over $\|\mathbf{v}\| = 1$ is $\mathbf{v}^\star = -\nabla f(\boldsymbol{\theta})/\|\nabla f(\boldsymbol{\theta})\|$, and the minimal value is $-\|\nabla f(\boldsymbol{\theta})\|$.*

Proof. By Cauchy–Schwarz, $\nabla f^\top \mathbf{v} \geq -\|\nabla f\|\,\|\mathbf{v}\| = -\|\nabla f\|$, with equality iff $\mathbf{v}$ is a negative multiple of ∇f. □

The gradient is orthogonal to the level set $\{f = f(\boldsymbol{\theta})\}$ and points "uphill"; the negative gradient is the basis of gradient descent (Section 8.4). Note that "steepest" depends on the norm used to measure $\mathbf{v}$: steepest descent in the norm $\|\mathbf{v}\|_{\mathbf{B}} = (\mathbf{v}^\top \mathbf{B}\mathbf{v})^{1/2}$ gives the direction $-\mathbf{B}^{-1}\nabla f$, which with $\mathbf{B} = \nabla^2 f$ is Newton's direction (Section 8.5).

Example 8.10 (Gradients of standard losses)**.** For least squares $f(\boldsymbol{\beta}) = \frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$: $\nabla f = -\frac{1}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$ and $\nabla^2 f = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$. For the logit average negative log-likelihood $f(\boldsymbol{\beta}) = \frac{1}{n}\sum_i [\log(1 + e^{\mathbf{x}_i^\top\boldsymbol{\beta}}) - y_i \mathbf{x}_i^\top\boldsymbol{\beta}]$, write $\sigma(z) = 1/(1 + e^{-z})$ and $p_i = \sigma(\mathbf{x}_i^\top\boldsymbol{\beta})$. Since $\sigma' = \sigma(1 - \sigma)$,

$$\nabla f(\boldsymbol{\beta}) = \frac{1}{n}\mathbf{X}^\top(\mathbf{p} - \mathbf{y}), \qquad \nabla^2 f(\boldsymbol{\beta}) = \frac{1}{n}\mathbf{X}^\top\mathbf{W}\mathbf{X}, \quad \mathbf{W} = \operatorname{diag}\{p_i(1 - p_i)\}. \tag{8.4}$$

The residual form "$\mathbf{X}^\top$(prediction − outcome)" is shared by all generalised linear models with canonical link.

8.2.2 Hessians and Taylor's theorem

Definition 8.11 (Hessian)**.** If f is twice differentiable, its *Hessian* $\nabla^2 f(\boldsymbol{\theta})$ is the $p \times p$ matrix of second partial derivatives $\partial^2 f / \partial\theta_i \partial\theta_j$, i.e. the Jacobian of the gradient. If the second partials are continuous ($f \in C^2$), the Hessian is symmetric (Schwarz–Clairaut theorem).

Theorem 8.12 (Taylor's theorem in several variables)**.** *Let $f \in C^2$ on an open set containing the segment $[\boldsymbol{\theta}, \boldsymbol{\theta} + \mathbf{h}]$. Then there is $t \in (0,1)$ with*

$$f(\boldsymbol{\theta} + \mathbf{h}) = f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top \mathbf{h} + \tfrac{1}{2} \mathbf{h}^\top \nabla^2 f(\boldsymbol{\theta} + t\mathbf{h})\, \mathbf{h}, \tag{8.5}$$

and, as $\mathbf{h} \to \mathbf{0}$, $f(\boldsymbol{\theta} + \mathbf{h}) = f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^\top \mathbf{h} + \tfrac{1}{2} \mathbf{h}^\top \nabla^2 f(\boldsymbol{\theta}) \mathbf{h} + o(\|\mathbf{h}\|^2)$. Also $\nabla f(\boldsymbol{\theta} + \mathbf{h}) = \nabla f(\boldsymbol{\theta}) + \int_0^1 \nabla^2 f(\boldsymbol{\theta} + s\mathbf{h}) \mathbf{h}\, ds$.

Proof sketch. Apply the one-variable Taylor theorem with Lagrange remainder to $\phi(s) = f(\boldsymbol{\theta} + s\mathbf{h})$ on $[0,1]$: $\phi'(s) = \nabla f(\boldsymbol{\theta} + s\mathbf{h})^\top \mathbf{h}$ and $\phi''(s) = \mathbf{h}^\top \nabla^2 f(\boldsymbol{\theta} + s\mathbf{h}) \mathbf{h}$ by the chain rule. The $o(\|\mathbf{h}\|^2)$ form follows from continuity of $\nabla^2 f$; the integral form is the fundamental theorem of calculus applied to each component of $s \mapsto \nabla f(\boldsymbol{\theta} + s\mathbf{h})$. □

Every method in this chapter is built on a Taylor model: gradient descent on the first-order model plus a quadratic penalty, Newton's method on the full second-order model.

8.2.3 Optimality conditions

Theorem 8.13 (First-order necessary condition; Fermat's rule)**.** *If $\boldsymbol{\theta}^\star$ is a local minimiser of f and f is differentiable at $\boldsymbol{\theta}^\star$, then $\nabla f(\boldsymbol{\theta}^\star) = \mathbf{0}$.*

Proof. Suppose $\mathbf{g} = \nabla f(\boldsymbol{\theta}^\star) \neq \mathbf{0}$ and take $\mathbf{v} = -\mathbf{g}$. Then $D_{\mathbf{v}} f(\boldsymbol{\theta}^\star) = -\|\mathbf{g}\|^2 < 0$, so $f(\boldsymbol{\theta}^\star + t\mathbf{v}) = f(\boldsymbol{\theta}^\star) - t\|\mathbf{g}\|^2 + o(t) < f(\boldsymbol{\theta}^\star)$ for all small $t > 0$, contradicting local minimality. □

A point with $\nabla f = \mathbf{0}$ is a *stationary* (or *critical*) point. Stationary points include minima, maxima and *saddle points*; second-order information distinguishes them.

Theorem 8.14 (Second-order conditions)**.** *Let $f \in C^2$ near $\boldsymbol{\theta}^\star$.*

(a) (Necessary) If $\boldsymbol{\theta}^\star$ is a local minimiser, then $\nabla f(\boldsymbol{\theta}^\star) = \mathbf{0}$ and $\nabla^2 f(\boldsymbol{\theta}^\star)$ is positive semidefinite.

(b) (Sufficient) If $\nabla f(\boldsymbol{\theta}^\star) = \mathbf{0}$ and $\nabla^2 f(\boldsymbol{\theta}^\star)$ is positive definite, then $\boldsymbol{\theta}^\star$ is a strict local minimiser.

Proof. (a) For any $\mathbf{v}$ and small t, Theorem 8.12 with $\nabla f(\boldsymbol{\theta}^\star) = \mathbf{0}$ gives $0 \le f(\boldsymbol{\theta}^\star + t\mathbf{v}) - f(\boldsymbol{\theta}^\star) = \frac{t^2}{2} \mathbf{v}^\top \nabla^2 f(\boldsymbol{\theta}^\star) \mathbf{v} + o(t^2)$. Divide by t^2 and let $t \to 0$: $\mathbf{v}^\top \nabla^2 f(\boldsymbol{\theta}^\star) \mathbf{v} \ge 0$.

(b) Let $\lambda > 0$ be the smallest eigenvalue of $\nabla^2 f(\boldsymbol{\theta}^\star)$, so $\mathbf{h}^\top \nabla^2 f(\boldsymbol{\theta}^\star) \mathbf{h} \ge \lambda \|\mathbf{h}\|^2$ (Chapter 3). By Taylor, $f(\boldsymbol{\theta}^\star + \mathbf{h}) - f(\boldsymbol{\theta}^\star) \ge \frac{\lambda}{2} \|\mathbf{h}\|^2 + o(\|\mathbf{h}\|^2)$, which is strictly positive for $0 < \|\mathbf{h}\| < r$ and some $r > 0$. □

The gap between (a) and (b) is real: $f(x) = x^3$ and $f(x) = x^4$ both have $f'(0) = f''(0) = 0$; the first has no minimum at 0, the second does. When the Hessian is singular the second-order test is inconclusive.

Definition 8.15 (Saddle point)**.** A stationary point $\boldsymbol{\theta}^\star$ is a *saddle point* if it is neither a local minimiser nor a local maximiser. If $\nabla^2 f(\boldsymbol{\theta}^\star)$ has both positive and negative eigenvalues, $\boldsymbol{\theta}^\star$ is a saddle: f increases along eigenvectors with positive eigenvalues and decreases along those with negative eigenvalues.

Example 8.16 (Classifying stationary points)**.** Let $f(x,y) = x^3 - 3x + y^2 - \tfrac{1}{2}xy^2$. Then $\nabla f = (3x^2 - 3 - \tfrac{1}{2}y^2,\ y(2-x))^\top$ and $\nabla^2 f = \begin{pmatrix} 6x & -y \\ -y & 2-x \end{pmatrix}$. The equation $y(2-x)=0$ gives $y=0$ (then $x=\pm 1$) or $x=2$ (then $y^2=18$). At $(1,0)$ the Hessian is $\mathrm{diag}(6,1)\succ 0$: a strict local minimum. At $(-1,0)$ it is $\mathrm{diag}(-6,3)$: a saddle. At $(2,\pm 3\sqrt{2})$ the eigenvalues are $6\pm\sqrt{54}$, of opposite sign: saddles. Listing 8.3 finds these points numerically by solving $\nabla f = \mathbf{0}$ from many starts and classifies them by Hessian eigenvalues; Fig. 8.2 shows the contours. Note that $(1,0)$ is *not* a global minimum: $f \to -\infty$ as $x \to -\infty$.

Listing 8.3: Finding and classifying stationary points ch08/ch08_critical.py

```python
# DEP 803 -- Chapter 8: Optimisation
# Locate and classify stationary points with the gradient and Hessian
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import root


def f(v):
    x, y = v
    return x**3 - 3 * x + y**2 - 2 * x * y**2 / 4


def grad(v):
    x, y = v
    return np.array([3 * x**2 - 3 - 0.5 * y**2, 2 * y - x * y])


def hess(v):
    x, y = v
    return np.array([[6 * x, -y], [-y, 2 - x]])


rng = np.random.default_rng(803)
pts = []
for s in rng.uniform(-3, 3, size=(60, 2)):
    sol = root(grad, s, jac=hess)
    sol.x[np.abs(sol.x) < 1e-10] = 0.0
    if sol.success and not any(np.allclose(sol.x, q, atol=1e-6) for q in pts):
        pts.append(sol.x)
print(f"{'x':>8}{'y':>8}{'f':>9}   Hessian eigenvalues    type")
for q in sorted(pts, key=lambda t: (t[0], t[1])):
    lam = np.linalg.eigvalsh(hess(q))
    kind = ("local min" if lam.min() > 0 else
            "local max" if lam.max() < 0 else
            "saddle" if lam.min() < 0 < lam.max() else "degenerate")
    print(f"{q[0]:>8.3f}{q[1]:>8.3f}{f(q):>9.3f}   "
          f"[{lam[0]:>7.3f}, {lam[1]:>7.3f}]   {kind}")

xx, yy = np.meshgrid(np.linspace(-2.5, 3.5, 300), np.linspace(-3, 3, 300))
zz = f((xx, yy))
plt.figure(figsize=(6, 3.8))
cs = plt.contour(xx, yy, zz, levels=np.linspace(-4, 6, 26), cmap="viridis")
for q in pts:
    lam = np.linalg.eigvalsh(hess(q))
    mk = "o" if lam.min() > 0 else ("^" if lam.max() < 0 else "x")
```

```
    plt.plot(q[0], q[1], mk, color="crimson", ms=8, mew=2)
plt.gca().set(xlabel="x", ylabel="y",
              title="Stationary points: o min, ^ max, x saddle")
plt.savefig("ch08_critical.pdf", bbox_inches="tight")
```

Output

```
      x       y        f   Hessian eigenvalues    type
 -1.000   0.000    2.000   [ -6.000,   3.000]   saddle
  1.000   0.000   -2.000   [  1.000,   6.000]   local min
  2.000  -4.243    2.000   [ -1.348,  13.348]   saddle
  2.000   4.243    2.000   [ -1.348,  13.348]   saddle
```

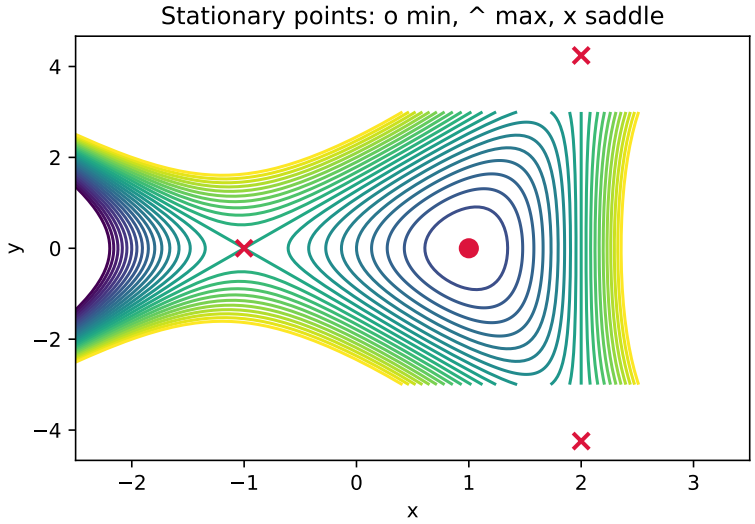


Figure 8.2: Contours of $f(x, y) = x^3 - 3x + y^2 - \frac{1}{2}xy^2$ with its stationary points: one strict local minimum ($\circ$) and three saddle points ($\times$). Contours cross at the saddle $(-1, 0)$.

⋆ Key Concept: Optimality conditions

At a local minimiser of a smooth f: $\nabla f = \mathbf{0}$ (first order) and $\nabla^2 f \succeq 0$ (second order, necessary). Conversely $\nabla f = \mathbf{0}$ and $\nabla^2 f \succ 0$ guarantee a strict local minimiser. Hessian eigenvalues of mixed sign signal a saddle point. Algorithms that only drive $\|\nabla f\|$ to zero can stop at saddles; reporting the Hessian eigenvalues at a reported optimum is cheap insurance.

8.2.4 Numerical gradients and gradient checking

Analytic gradients are fast and accurate, but they are also a frequent source of bugs. Finite differences provide an independent check. With $\mathbf{e}_j$ the j-th unit vector,

$$\text{forward:}\quad \frac{\partial f}{\partial \theta_j} \approx \frac{f(\boldsymbol{\theta} + h\mathbf{e}_j) - f(\boldsymbol{\theta})}{h}, \qquad \text{truncation error } \tfrac{h}{2}\left|f''\right| = O(h), \tag{8.6}$$

$$\text{central:}\quad \frac{\partial f}{\partial \theta_j} \approx \frac{f(\boldsymbol{\theta} + h\mathbf{e}_j) - f(\boldsymbol{\theta} - h\mathbf{e}_j)}{2h}, \qquad \text{truncation error } \tfrac{h^2}{6}\left|f'''\right| = O(h^2). \tag{8.7}$$

The truncation errors follow from Theorem 8.12 (for the central formula the even-order terms cancel). But each function value carries a rounding error of order $\varepsilon_{\text{mach}}\left|f\right|$ (see Chapter 1), and

dividing a difference of two such values by h amplifies it to order $\varepsilon_{\text{mach}}\,|f|\,/h$. Balancing the two error sources:

Proposition 8.17 (Optimal finite-difference step). *Minimising* $\frac{h}{2}M_2 + \frac{2\varepsilon_{\text{mach}}|f|}{h}$ *gives* $h^\star \propto \sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$ *and error* $O(\sqrt{\varepsilon_{\text{mach}}}) \approx 10^{-8}$ *for the forward difference; minimising* $\frac{h^2}{6}M_3 + \frac{\varepsilon_{\text{mach}}|f|}{h}$ *gives* $h^\star \propto \varepsilon_{\text{mach}}^{1/3} \approx 6\times 10^{-6}$ *and error* $O(\varepsilon_{\text{mach}}^{2/3}) \approx 10^{-11}$ *for the central difference (constants depending on* $|f|$ *and the derivative bounds* M_2, M_3 *omitted).*

Gradient checking compares the analytic gradient $\mathbf{g}_a$ with a central-difference gradient $\mathbf{g}_{fd}$ through the relative error $\|\mathbf{g}_a - \mathbf{g}_{fd}\|\,/\|\mathbf{g}_a\|$; values below about 10^{-6} indicate a correct gradient, values of order one indicate a bug. Listing 8.4 applies this to the logit gradient (8.4) and to a deliberately buggy version that forgets the factor $1/n$, and then scans h over thirteen orders of magnitude.

Listing 8.4: Finite differences and gradient checking for the logit loss ch08/ch08_gradcheck.py

```python
# DEP 803 -- Chapter 8: Optimisation
# Finite-difference gradients and gradient checking for the logit loss
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n, p = 500, 4
X = np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))])
beta_true = np.array([-0.5, 1.0, -0.8, 0.3])
y = (rng.uniform(size=n) < 1 / (1 + np.exp(-X @ beta_true))).astype(float)


def loss(b):
    """Average negative log-likelihood of the logit model."""
    z = X @ b
    return np.mean(np.logaddexp(0, z) - y * z)


def grad(b):
    """Analytic gradient: X'(sigma(Xb) - y) / n."""
    return X.T @ (1 / (1 + np.exp(-X @ b)) - y) / n


def fd_grad(f, b, h, central=True):
    g = np.zeros_like(b)
    for j in range(b.size):
        e = np.zeros_like(b)
        e[j] = h
        g[j] = (f(b + e) - f(b - e)) / (2 * h) if central else \
            (f(b + e) - f(b)) / h
    return g


b = rng.normal(size=p)
ga = grad(b)
print("analytic gradient :", np.round(ga, 8))
print("central FD, h=1e-5:", np.round(fd_grad(loss, b, 1e-5), 8))
rel = np.linalg.norm(fd_grad(loss, b, 1e-5) - ga) / np.linalg.norm(ga)
```

```
print(f"relative error    : {rel:.2e}  (pass if < 1e-6)")


def bad_grad(b):          # a typical bug: forgot to divide by n
    return X.T @ (1 / (1 + np.exp(-X @ b)) - y)


rel_bad = np.linalg.norm(fd_grad(loss, b, 1e-5) - bad_grad(b)) / \
    np.linalg.norm(bad_grad(b))
print(f"buggy gradient    : relative error {rel_bad:.2e}  -> FAIL")

hs = np.logspace(-14, -1, 60)
err_f = [np.linalg.norm(fd_grad(loss, b, h, False) - ga) for h in hs]
err_c = [np.linalg.norm(fd_grad(loss, b, h, True) - ga) for h in hs]
print(f"best h forward : {hs[np.argmin(err_f)]:.1e}, error {min(err_f):.1e}")
print(f"best h central : {hs[np.argmin(err_c)]:.1e}, error {min(err_c):.1e}")

plt.figure(figsize=(6, 3.8))
plt.loglog(hs, err_f, "o-", ms=3, label="forward difference, O(h)")
plt.loglog(hs, err_c, "s-", ms=3, label="central difference, O(h^2)")
plt.gca().set(xlabel="step h", ylabel="error in gradient (2-norm)",
              title="Truncation error vs rounding error")
plt.legend(frameon=False)
plt.savefig("ch08_fd_error.pdf", bbox_inches="tight")
```

Output

```
analytic gradient : [ 0.15215757 -0.03638254  0.12453259 -0.26523193]
central FD, h=1e-5: [ 0.15215757 -0.03638254  0.12453259 -0.26523193]
relative error    : 2.75e-11  (pass if < 1e-6)
buggy gradient    : relative error 9.98e-01  -> FAIL
best h forward : 2.5e-08, error 1.9e-09
best h central : 6.5e-06, error 8.8e-12
```

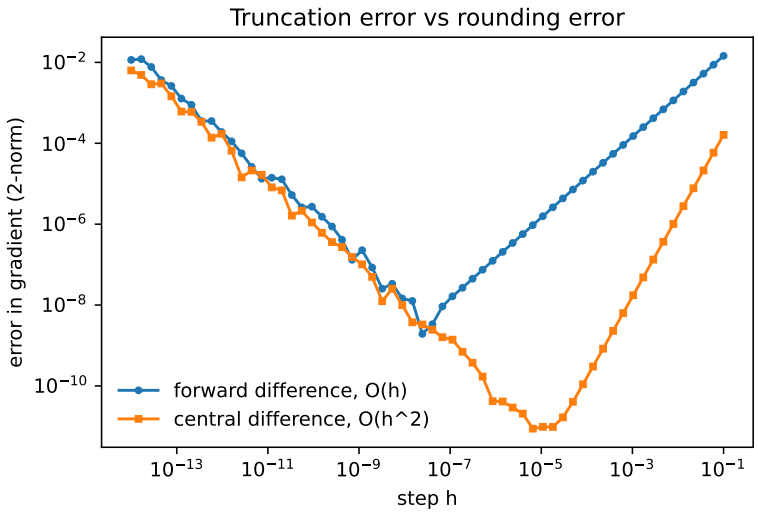


Figure 8.3: Error of finite-difference gradients of the logit loss as a function of the step h. For large h the truncation error dominates (slopes 1 and 2 on the log–log scale); for small h rounding error dominates and grows like $1/h$. The optimal steps agree with Proposition 8.17.

The best steps found empirically ($h \approx 2.5 \times 10^{-8}$ forward, 6.5×10^{-6} central) and the corresponding errors ($\approx 10^{-9}$ and $\approx 10^{-11}$) match Proposition 8.17; Fig. 8.3 shows the characteristic V shape.

△ Caution: Finite differences: the step is not "as small as possible"

Choosing $h = 10^{-12}$ "to be accurate" gives *worse* gradients than $h = 10^{-6}$, because rounding error grows like $\varepsilon_{\text{mach}}/h$. Finite-difference gradients also cost $p + 1$ (forward) or $2p$ (central) function evaluations per gradient, which is prohibitive when p is large, and they are inaccurate for objectives that are themselves computed with noise (simulation-based estimators, objectives with internal iterative solvers and loose tolerances). Default derivative-based optimisers in SciPy fall back on forward differences when no `jac` is supplied; supplying an analytic gradient often changes both speed and the final answer.

Graduate Perspective: Complex-step differentiation and automatic differentiation

For real-analytic f, $f'(x) = \operatorname{Im} f(x + ih)/h + O(h^2)$ involves no subtraction, so h can be as small as 10^{-20} and the derivative is accurate to machine precision (Squire and Trapp, 1998). More generally, *automatic differentiation* (AD) applies the chain rule to the elementary operations of a program and delivers exact (to rounding) derivatives. In reverse mode, the gradient of a scalar function of p parameters costs a small constant multiple of one function evaluation, independently of p (Baydin et al., 2018). Reverse-mode AD ("back-propagation") is what makes training neural networks with millions of parameters feasible, and it is built into Stan (Matsuura, 2022), PyTorch, JAX and TensorFlow.

Exercises

8.2.1 Theory Derive the gradient and Hessian of the logit loss in Example 8.10 from first principles. Show that the Hessian of the Poisson regression loss $\frac{1}{n}\sum_i [e^{\mathbf{x}_i^\top \boldsymbol{\beta}} - y_i \mathbf{x}_i^\top \boldsymbol{\beta}]$ is $\frac{1}{n}\mathbf{X}^\top \operatorname{diag}(e^{\mathbf{x}_i^\top \boldsymbol{\beta}})\mathbf{X}$.

8.2.2 Theory Prove that for $f \in C^1$ the gradient $\nabla f(\boldsymbol{\theta})$ is orthogonal to the tangent of any differentiable curve lying in the level set $\{f = f(\boldsymbol{\theta})\}$ through $\boldsymbol{\theta}$.

8.2.3 Theory Let $f(x, y) = (y - x^2)(y - 2x^2)$ (Peano's example). Show that $(0, 0)$ is a local minimum of f along every straight line through the origin but is not a local minimum of f. Which part of Theorem 8.14 explains why the second-order test is silent?

8.2.4 Python Extend Listing 8.4 to check the *Hessian* (8.4) by central differences of the analytic gradient. What is the best step and achievable accuracy? Relate your answer to Proposition 8.17.

8.2.5 Python Implement complex-step differentiation for the logit loss (use `np.log1p(np.exp(z))` with complex `z`) and reproduce Fig. 8.3 with a third curve. Explain the result.

8.2.6 Python Find and classify all stationary points of $f(x, y) = x^4 + y^4 - 4xy$ analytically and with a modified version of Listing 8.3.

8.2.7 ★ Graduate Let $f(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}$ with $\mathbf{A}$ symmetric and indefinite. Show that gradient descent started at a random point converges to the saddle at $\mathbf{0}$ with probability zero. (This is the simplest case of a general result for gradient methods, see Lee et al., 2016.)

8.3 Convexity

Convexity is to optimisation what linearity is to algebra: the property under which a complete, efficient theory exists. Boyd and Vandenberghe (2004) is the standard reference; here we develop what is needed for the algorithms that follow.

8.3.1 Convex sets and convex functions

Definition 8.18 (Convex set)**.** A set $C \subseteq \mathbb{R}^p$ is *convex* if $\theta\mathbf{x} + (1-\theta)\mathbf{y} \in C$ for all $\mathbf{x}, \mathbf{y} \in C$ and $\theta \in [0,1]$: the segment between any two points of C lies in C.

Examples: subspaces, half-spaces $\{\mathbf{x} : \mathbf{a}^\top\mathbf{x} \le b\}$, balls of any norm, the probability simplex $\{\mathbf{x} \ge \mathbf{0}, \mathbf{1}^\top\mathbf{x} = 1\}$, the cone of positive semidefinite matrices, and budget sets $\{\mathbf{x} \ge \mathbf{0} : \mathbf{p}^\top\mathbf{x} \le m\}$. Intersections of convex sets are convex; unions generally are not.

Definition 8.19 (Convex, strictly convex and concave functions)**.** $f : \mathbb{R}^p \to \mathbb{R}$ is *convex* if for all $\mathbf{x}, \mathbf{y}$ and $\theta \in [0,1]$

$$f(\theta\mathbf{x} + (1-\theta)\mathbf{y}) \le \theta f(\mathbf{x}) + (1-\theta) f(\mathbf{y}), \tag{8.8}$$

strictly convex if the inequality is strict for $\mathbf{x} \ne \mathbf{y}$ and $\theta \in (0,1)$, and *concave* if $-f$ is convex. Equivalently, f is convex iff its *epigraph* $\{(\mathbf{x}, t) : t \ge f(\mathbf{x})\}$ is a convex set.

Inequality (8.8) says that the chord lies above the graph. Applied repeatedly it gives *Jensen's inequality* $f(\sum_i \theta_i \mathbf{x}_i) \le \sum_i \theta_i f(\mathbf{x}_i)$ for convex weights, and in its probabilistic form $f(\mathbb{E}X) \le \mathbb{E} f(X)$ (Chapter 6).

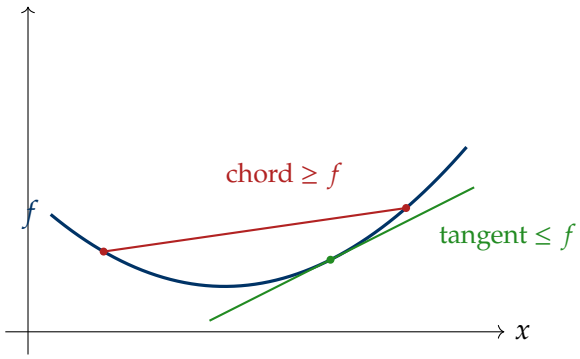


Figure 8.4: A convex function lies below its chords (definition (8.8)) and above its tangents (first-order condition, Theorem 8.20).

8.3.2 First- and second-order characterisations

Theorem 8.20 (First-order characterisation)**.** *Let f be differentiable on $\mathbb{R}^p$. Then f is convex if and only if*

$$f(\mathbf{y}) \ge f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y} - \mathbf{x}) \qquad \text{for all } \mathbf{x}, \mathbf{y}. \tag{8.9}$$

Proof. ($\Rightarrow$) For $t \in (0,1]$, convexity gives $f(\mathbf{x} + t(\mathbf{y}-\mathbf{x})) \le (1-t)f(\mathbf{x}) + t f(\mathbf{y})$, i.e. $\frac{f(\mathbf{x}+t(\mathbf{y}-\mathbf{x})) - f(\mathbf{x})}{t} \le f(\mathbf{y}) - f(\mathbf{x})$. Letting $t \downarrow 0$ the left side tends to $\nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x})$.

($\Leftarrow$) Let $\mathbf{z} = \theta\mathbf{x} + (1-\theta)\mathbf{y}$. Applying (8.9) at $\mathbf{z}$ twice, $f(\mathbf{x}) \ge f(\mathbf{z}) + \nabla f(\mathbf{z})^\top(\mathbf{x}-\mathbf{z})$ and $f(\mathbf{y}) \ge f(\mathbf{z}) + \nabla f(\mathbf{z})^\top(\mathbf{y}-\mathbf{z})$. Multiply by θ and $1-\theta$ and add: the gradient terms cancel because $\theta(\mathbf{x}-\mathbf{z}) + (1-\theta)(\mathbf{y}-\mathbf{z}) = \mathbf{0}$, leaving $\theta f(\mathbf{x}) + (1-\theta) f(\mathbf{y}) \ge f(\mathbf{z})$. □

Corollary 8.21 (Stationary points of convex functions are global minimisers)**.** *If f is convex and differentiable and $\nabla f(\mathbf{x}^\star) = \mathbf{0}$, then $\mathbf{x}^\star$ is a global minimiser. If f is strictly convex, the global minimiser (if it exists) is unique.*

Proof. Put $\mathbf{x} = \mathbf{x}^\star$ in (8.9): $f(\mathbf{y}) \geq f(\mathbf{x}^\star)$ for all $\mathbf{y}$. If $\mathbf{x}_1 \neq \mathbf{x}_2$ were two minimisers of a strictly convex f, then $f(\frac{1}{2}\mathbf{x}_1 + \frac{1}{2}\mathbf{x}_2) < f^\star$, a contradiction. □

More generally, without differentiability, *every local minimiser of a convex function is global*: if $f(\mathbf{y}) < f(\mathbf{x}^\star)$ for some $\mathbf{y}$, then by (8.8) the points $\mathbf{x}^\star + t(\mathbf{y} - \mathbf{x}^\star)$ have values $< f(\mathbf{x}^\star)$ for all small $t > 0$, contradicting local minimality. This is the deepest reason why convexity matters: *a local search suffices*.

Theorem 8.22 (Second-order characterisation)**.** *Let $f \in C^2(\mathbb{R}^p)$. Then f is convex if and only if $\nabla^2 f(\mathbf{x}) \succeq 0$ for all $\mathbf{x}$. If $\nabla^2 f(\mathbf{x}) \succ 0$ for all $\mathbf{x}$, then f is strictly convex.*

Proof. ($\Leftarrow$) By (8.5), $f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y} - \mathbf{x}) + \frac{1}{2}(\mathbf{y} - \mathbf{x})^\top \nabla^2 f(\mathbf{z})(\mathbf{y} - \mathbf{x})$ for some $\mathbf{z}$ on the segment; the last term is ≥ 0 (resp. > 0 if $\mathbf{y} \neq \mathbf{x}$ and the Hessian is positive definite), so (8.9) holds (strictly), and Theorem 8.20 applies.

($\Rightarrow$) Fix $\mathbf{x}, \mathbf{v}$ and let $\phi(t) = f(\mathbf{x} + t\mathbf{v})$, a convex function of one variable. By (8.9) applied to ϕ at t and at 0, $\phi(t) \geq \phi(0) + \phi'(0)t$ and $\phi(0) \geq \phi(t) - \phi'(t)t$; adding, $(\phi'(t) - \phi'(0))t \geq 0$, so ϕ' is non-decreasing and $\phi''(0) = \mathbf{v}^\top \nabla^2 f(\mathbf{x})\mathbf{v} \geq 0$. □

The converse of the strict statement fails: $f(x) = x^4$ is strictly convex but $f''(0) = 0$.

8.3.3 Operations that preserve convexity

Checking the definition directly is tedious; in practice one builds convex functions from simple pieces. The following are proved directly from (8.8) (Boyd and Vandenberghe, 2004, Sec. 3.2):

(i) *non-negative weighted sums*: if $f_1, \ldots, f_m$ are convex and $w_j \geq 0$ then $\sum_j w_j f_j$ is convex (so empirical risks with convex losses are convex);

(ii) *affine pre-composition*: if f is convex then $g(\boldsymbol{\beta}) = f(\mathbf{A}\boldsymbol{\beta} + \mathbf{b})$ is convex (so $\rho(y_i - \mathbf{x}_i^\top \boldsymbol{\beta})$ is convex in $\boldsymbol{\beta}$ whenever ρ is convex);

(iii) *pointwise maximum*: $\max_j f_j$ and $\sup_\alpha f_\alpha$ of convex functions are convex (e.g. the hinge loss $\max(0, 1 - y\mathbf{x}^\top\boldsymbol{\beta})$ and the largest eigenvalue of a symmetric matrix);

(iv) *composition*: if $h : \mathbb{R} \to \mathbb{R}$ is convex and non-decreasing and g is convex, then $h \circ g$ is convex (e.g. $e^{g(\mathbf{x})}$);

(v) *norms* are convex, and so is $\|\mathbf{x}\|^2$; adding a ridge penalty $\frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ makes any convex risk strictly convex.

By (i), (ii) and (v), OLS, LAD, Huber, ridge, lasso and logistic regression (next subsection) all have convex objectives.

8.3.4 Strong convexity and smoothness

Convexity is qualitative. Rates of convergence of algorithms depend on two quantitative constants: how strongly the function curves upwards (at least) and how fast its gradient can change (at most).

Definition 8.23 (L-smoothness and μ-strong convexity)**.** A differentiable f is *L-smooth* if $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. It is *μ-strongly convex* ($\mu > 0$) if

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y} - \mathbf{x}) + \tfrac{\mu}{2}\|\mathbf{y} - \mathbf{x}\|^2 \qquad \text{for all } \mathbf{x}, \mathbf{y}. \tag{8.10}$$

For $f \in C^2$: f is convex and L-smooth iff $\mathbf{0} \preceq \nabla^2 f(\mathbf{x}) \preceq L\mathbf{I}$, and μ-strongly convex iff $\nabla^2 f(\mathbf{x}) \succeq \mu\mathbf{I}$, for all $\mathbf{x}$. The ratio $\kappa = L/\mu \geq 1$ is the *condition number* of the problem.

Lemma 8.24 (Descent lemma)**.** *If f is L-smooth, then for all $\mathbf{x}, \mathbf{y}$*

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) + \tfrac{L}{2}\|\mathbf{y}-\mathbf{x}\|^2 . \tag{8.11}$$

Proof. By the integral form of Taylor's theorem and Cauchy–Schwarz, $f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) = \int_0^1 [\nabla f(\mathbf{x}+t(\mathbf{y}-\mathbf{x})) - \nabla f(\mathbf{x})]^\top(\mathbf{y}-\mathbf{x})\,dt \leq \int_0^1 Lt\,\|\mathbf{y}-\mathbf{x}\|^2\,dt = \tfrac{L}{2}\|\mathbf{y}-\mathbf{x}\|^2$. □

So an L-smooth, μ-strongly convex function is sandwiched between two quadratics with curvatures μ and L around every point. Two consequences used repeatedly below:

Proposition 8.25. *Let f be μ-strongly convex with minimiser $\mathbf{x}^\star$. Then for all $\mathbf{x}$,*

$$f(\mathbf{x}) - f^\star \leq \frac{1}{2\mu}\left\|\nabla f(\mathbf{x})\right\|^2 \qquad \text{and} \qquad \left\|\mathbf{x}-\mathbf{x}^\star\right\| \leq \frac{1}{\mu}\left\|\nabla f(\mathbf{x})\right\|. \tag{8.12}$$

If f is L-smooth, also $f(\mathbf{x}) - f^\star \geq \frac{1}{2L}\left\|\nabla f(\mathbf{x})\right\|^2$.

Proof. Minimising the right side of (8.10) over $\mathbf{y}$ (at $\mathbf{y} = \mathbf{x} - \nabla f(\mathbf{x})/\mu$) gives $f(\mathbf{y}) \geq f(\mathbf{x}) - \left\|\nabla f(\mathbf{x})\right\|^2/(2\mu)$ for every $\mathbf{y}$, in particular for $\mathbf{y} = \mathbf{x}^\star$. For the second inequality, put $\mathbf{y} = \mathbf{x}^\star$ and $\mathbf{x}$ in (8.10) and use $f^\star \leq f(\mathbf{x})$ and Cauchy–Schwarz: $0 \geq f^\star - f(\mathbf{x}) \geq -\left\|\nabla f(\mathbf{x})\right\|\|\mathbf{x}-\mathbf{x}^\star\| + \frac{\mu}{2}\|\mathbf{x}-\mathbf{x}^\star\|^2$. The last claim follows from (8.11) with $\mathbf{y} = \mathbf{x} - \nabla f(\mathbf{x})/L$: $f^\star \leq f(\mathbf{y}) \leq f(\mathbf{x}) - \left\|\nabla f(\mathbf{x})\right\|^2/(2L)$. □

The first inequality in (8.12) (the *Polyak–Łojasiewicz* inequality) says that a small gradient certifies a small optimality gap; it is the key to linear convergence in Section 8.4. The second justifies the usual stopping rule "stop when $\left\|\nabla f\right\|$ is small" for strongly convex problems.

Example 8.26 (Ridge regression)**.** For $f(\boldsymbol{\beta}) = \frac{1}{2n}\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ the Hessian is the constant matrix $\frac{1}{n}\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$, so $\mu = \lambda_{\min}(\mathbf{X}^\top\mathbf{X}/n) + \lambda$ and $L = \lambda_{\max}(\mathbf{X}^\top\mathbf{X}/n) + \lambda$. The condition number is a property of the *design*: badly scaled or nearly collinear covariates (Chapter 3) make κ large, and both the ridge penalty and standardisation reduce it.

8.3.5 Concavity of the logit log-likelihood

Proposition 8.27. *The logit log-likelihood $\ell(\boldsymbol{\beta}) = \sum_i [y_i \mathbf{x}_i^\top\boldsymbol{\beta} - \log(1 + e^{\mathbf{x}_i^\top\boldsymbol{\beta}})]$ is concave. It is strictly concave if $\mathbf{X}$ has full column rank. Its negative average $f = -\ell/n$ is L-smooth with $L = \lambda_{\max}(\mathbf{X}^\top\mathbf{X})/(4n)$.*

Proof. By (8.4), $-\nabla^2\ell = \mathbf{X}^\top\mathbf{W}\mathbf{X}$ with $\mathbf{W} = \operatorname{diag}\{p_i(1-p_i)\}$ and $0 < p_i(1-p_i) \leq \frac{1}{4}$. For any $\mathbf{v}$, $\mathbf{v}^\top\mathbf{X}^\top\mathbf{W}\mathbf{X}\mathbf{v} = \sum_i p_i(1-p_i)(\mathbf{x}_i^\top\mathbf{v})^2 \geq 0$, with equality only if $\mathbf{X}\mathbf{v} = \mathbf{0}$, i.e. $\mathbf{v} = \mathbf{0}$ under full rank. Apply Theorem 8.22. Finally $\mathbf{v}^\top\mathbf{X}^\top\mathbf{W}\mathbf{X}\mathbf{v} \leq \frac{1}{4}\|\mathbf{X}\mathbf{v}\|^2 \leq \frac{1}{4}\lambda_{\max}(\mathbf{X}^\top\mathbf{X})\|\mathbf{v}\|^2$. □

Consequently any stationary point of the logit likelihood is *the* MLE, and Newton's method (Section 8.5) cannot get stuck at a spurious local optimum. The logit is not *globally* strongly convex, however: $p_i(1-p_i) \to 0$ as $\left|\mathbf{x}_i^\top\boldsymbol{\beta}\right| \to \infty$, which is the algebraic face of the separation problem of Section 8.1. The same argument shows that the probit, Poisson and other canonical-link GLM log-likelihoods are concave (Murphy, 2022, Ch. 10).

Listing 8.5 verifies these facts numerically, computes μ, L and κ for a ridge problem with one badly scaled covariate, checks Jensen's inequality for the softplus function $\log(1+e^x)$, and examines utility functions (next subsection).

Listing 8.5: Numerical checks of convexity, smoothness and quasi-concavity
ch08/ch08_convexity.py

```
# DEP 803 -- Chapter 8: Optimisation
# Numerical convexity checks: logit Hessian, ridge constants mu and L
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, p = 1000, 5
X = np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))])
X[:, 1] *= 10                                   # a badly scaled covariate
y = (rng.uniform(size=n) < 0.4).astype(float)


def logit_hessian(b):
    """Hessian of the average logit negative log-likelihood."""
    pr = 1 / (1 + np.exp(-X @ b))
    w = pr * (1 - pr)
    return (X * w[:, None]).T @ X / n


print("1. Logit NLL: smallest Hessian eigenvalue at random points")
for k in range(5):
    b = rng.normal(scale=2, size=p)
    lam = np.linalg.eigvalsh(logit_hessian(b))
    print(f"   point {k}: min eig = {lam.min():.3e}, max eig = {lam.max():.3e}")

print("2. Ridge risk (1/2n)||y - Xb||^2 + (lam/2)||b||^2")
G = X.T @ X / n
eig = np.linalg.eigvalsh(G)
for lam in [0.0, 0.1, 1.0]:
    mu, L = eig.min() + lam, eig.max() + lam
    print(f"   lam = {lam:>4}: mu = {mu:8.4f}, L = {L:8.4f}, "
          f"kappa = L/mu = {L / mu:8.1f}")
Xs = X.copy()
Xs[:, 1:] = (Xs[:, 1:] - Xs[:, 1:].mean(0)) / Xs[:, 1:].std(0)
e2 = np.linalg.eigvalsh(Xs.T @ Xs / n)
print(f"   after standardising: kappa = {e2.max() / e2.min():.2f}")

print("3. Jensen check for f(x) = log(1 + exp(x)) on random pairs")
f = lambda t: np.logaddexp(0, t)
a, b = rng.normal(size=(2, 100000)) * 5
th = rng.uniform(size=100000)
gap = th * f(a) + (1 - th) * f(b) - f(th * a + (1 - th) * b)
print(f"   min of theta f(a) + (1-theta) f(b) - f(mix) = {gap.min():.2e}")

print("4. Cobb-Douglas u = x1^0.3 x2^0.5: concave? quasi-concave?")
def hess_cd(x1, x2, a=0.3, c=0.5):
    u = x1**a * x2**c
    return np.array([[a * (a - 1) * u / x1**2, a * c * u / (x1 * x2)],
                     [a * c * u / (x1 * x2), c * (c - 1) * u / x2**2]])
lam = np.linalg.eigvalsh(hess_cd(2.0, 3.0))
print(f"   Hessian eigenvalues at (2,3): {lam.round(4)} -> concave")
lam = np.linalg.eigvalsh(hess_cd(2.0, 3.0, a=0.9, c=0.8))
print(f"   with a=0.9, c=0.8 (sum > 1): {lam.round(4)} -> not concave")
x1 = rng.uniform(0.1, 5, (2, 50000))
```

```
x2 = rng.uniform(0.1, 5, (2, 50000))
u = lambda s, t: s**0.9 * t**0.8
mix1, mix2 = th[:50000] * x1[0] + (1 - th[:50000]) * x1[1], \
    th[:50000] * x2[0] + (1 - th[:50000]) * x2[1]
qc = u(mix1, mix2) - np.minimum(u(x1[0], x2[0]), u(x1[1], x2[1]))
print(f"   but min u(mix) - min(u_a, u_b) = {qc.min():.2e} >= 0"
      " -> quasi-concave")
```

Output

```
1. Logit NLL: smallest Hessian eigenvalue at random points
   point 0: min eig = 4.142e-03, max eig = 4.069e-01
   point 1: min eig = 9.566e-04, max eig = 1.783e-02
   point 2: min eig = 1.781e-03, max eig = 1.104e-01
   point 3: min eig = 1.983e-03, max eig = 1.887e-01
   point 4: min eig = 2.114e-02, max eig = 8.806e-01
2. Ridge risk (1/2n)||y - Xb||^2 + (lam/2)||b||^2
   lam =  0.0: mu =   0.9297, L =  99.8013, kappa = L/mu =    107.4
   lam =  0.1: mu =   1.0297, L =  99.9013, kappa = L/mu =     97.0
   lam =  1.0: mu =   1.9297, L = 100.8013, kappa = L/mu =     52.2
   after standardising: kappa = 1.19
3. Jensen check for f(x) = log(1 + exp(x)) on random pairs
   min of theta f(a) + (1-theta) f(b) - f(mix) = 1.05e-11
4. Cobb-Douglas u = x1^0.3 x2^0.5: concave? quasi-concave?
   Hessian eigenvalues at (2,3): [-0.1451 -0.0261] -> concave
   with a=0.9, c=0.8 (sum > 1): [-0.6299  0.4489] -> not concave
   but min u(mix) - min(u_a, u_b) = 1.23e-05 >= 0 -> quasi-concave
```

One covariate with a standard deviation of 10 instead of 1 pushes κ above 100; standardising brings it to about 1.2. We shall see in Section 8.4 that the number of gradient descent iterations is proportional to κ.

8.3.6 Quasi-concavity and utility functions

Economic theory often needs a weaker property than concavity.

Definition 8.28 (Quasi-concavity). $u : C \to \mathbb{R}$ on a convex set C is *quasi-concave* if $u(\theta \mathbf{x} + (1-\theta)\mathbf{y}) \geq \min\{u(\mathbf{x}), u(\mathbf{y})\}$ for all $\mathbf{x}, \mathbf{y} \in C$, $\theta \in [0,1]$; equivalently, every upper contour set $\{\mathbf{x} : u(\mathbf{x}) \geq a\}$ is convex. It is *strictly* quasi-concave if the inequality is strict for $\mathbf{x} \neq \mathbf{y}$, $\theta \in (0,1)$.

Every concave function is quasi-concave, and any non-decreasing transformation $h \circ u$ of a quasi-concave u is quasi-concave. Since utility is ordinal (only the ranking of bundles matters), quasi-concavity, which survives monotone transformations, is the natural assumption: it says that indifference curves are convex to the origin, i.e. consumers prefer mixtures to extremes. Concavity, which does not survive monotone transformations, is a cardinal property.

Example 8.29 (Cobb–Douglas utility). Let $u(x_1, x_2) = x_1^a x_2^c$ on $\mathbb{R}^2_{++}$ with $a, c > 0$. Since $\log u = a \log x_1 + c \log x_2$ is concave and $u = \exp(\log u)$ is an increasing transformation, u is quasi-concave for *all* $a, c > 0$. But u is concave only when $a + c \leq 1$: the Hessian has diagonal entries $a(a-1)u/x_1^2$, $c(c-1)u/x_2^2$ and determinant $ac(1-a-c)u^2/(x_1 x_2)^2$, which is negative when $a + c > 1$. Part 4 of Listing 8.5 confirms both statements for $(a, c) = (0.9, 0.8)$.

Strict quasi-concavity of utility together with a convex budget set guarantees that the

consumer's problem has at most one solution, so demand functions are well defined; the Lagrangian and Karush–Kuhn–Tucker machinery for such problems is developed in Chapter 9 (see also Simon and Blume, 1994, Ch. 21).

⋆ Key Concept: Why convexity is the dividing line

For a convex objective: every local minimiser is global, the set of minimisers is convex, a stationary point is optimal, and first-order methods have global convergence guarantees whose speed is governed by $\kappa = L/\mu$. For non-convex objectives none of this holds in general. Check convexity with the Hessian ($\nabla^2 f \succeq 0$ *everywhere*) or by building f from convexity-preserving operations.

△ Caution: Pointwise checks do not prove convexity

A positive definite Hessian *at the optimum* proves only a strict local minimum, not convexity. Sampling the Hessian at random points (as in Listing 8.5) can refute convexity but never prove it. And quasi-concavity is *not* preserved under addition: the sum of two quasi-concave functions need not be quasi-concave, so aggregate welfare built from quasi-concave individual utilities can have non-convex upper contour sets.

Historical Note: Jensen, Minkowski and the birth of convex analysis

The Danish engineer and mathematician Johan Jensen, who worked for the Copenhagen telephone company, introduced convex functions through the midpoint inequality $f(\frac{x+y}{2}) \le \frac{f(x)+f(y)}{2}$ and proved the inequality that bears his name in a 1906 paper in *Acta Mathematica* (Jensen, 1906). Hermann Minkowski had developed the geometry of convex bodies in the 1890s in connection with number theory. The systematic theory of convex functions and duality, *convex analysis*, was built in the twentieth century by Werner Fenchel and R. Tyrrell Rockafellar, whose 1970 monograph *Convex Analysis* remains the standard reference.

Policy Lens: Cost-minimising service delivery

A ministry wants to allocate N community health workers across districts to minimise total expected cost of unmet need. If the cost of unmet need in district d is a convex, decreasing function $c_d(n_d)$ of workers assigned (diminishing returns), the total cost $\sum_d c_d(n_d)$ is convex by rule (i), and with the convex constraint $\sum_d n_d = N$ the optimum is characterised by equalised marginal costs $c_d'(n_d)$ across districts, a condition any local search will find. If instead there are fixed set-up costs (opening a facility), the cost function becomes non-convex and the problem acquires many local optima, typically requiring integer programming. Recognising which case one is in is the first step of any optimisation-based policy design; constraints are treated in Chapter 9.

Exercises

8.3.1 Theory Prove that a convex function $f : (a, b) \to \mathbb{R}$ on an open interval is continuous (hint: use (8.8) to bound the difference quotients of f). Give an example of a convex function on $[0, 1]$ that is discontinuous at an endpoint.

8.3.2 Theory Prove rules (i)–(iv) on operations preserving convexity directly from the definition. Use them to show that the hinge loss, the Huber loss and the softplus $\log(1 + e^x)$ are convex.

8.3.3 Theory Show that f is μ-strongly convex iff $f - \frac{\mu}{2}\|\cdot\|^2$ is convex, and that a strongly convex continuous function on $\mathbb{R}^p$ is coercive and therefore has a unique minimiser.

8.3.4 Theory Show that the CES utility $u(\mathbf{x}) = (\sum_j \alpha_j x_j^r)^{1/r}$ with $\alpha_j > 0$ and $r < 1$, $r \neq 0$, is quasi-concave on $\mathbb{R}^p_{++}$. Is it concave?

8.3.5 Python For the design of Listing 8.5, plot κ of the ridge problem as a function of the scale factor $s \in [1, 100]$ of the second covariate and of λ. Interpret the result in terms of eigenvalues of $\mathbf{X}^\top\mathbf{X}/n$.

8.3.6 Python Plot the minimum Hessian eigenvalue of the logit loss along the ray $t\hat{\boldsymbol{\beta}}$, $t \in [0, 20]$. Explain the decay and relate it to the separation problem.

8.3.7 Policy Write down the cost-minimisation problem of the Policy Lens with $c_d(n) = a_d/(1+n)$ and show that the optimal allocation satisfies $n_d + 1 \propto \sqrt{a_d}$. Interpret: how should workers be allocated between a district with twice the need of another?

8.3.8 ★ Graduate Show that the log-likelihood of the probit model is concave in $\boldsymbol{\beta}$. (Hint: the key fact is log-concavity of Φ and $1-\Phi$; prove that $\frac{d^2}{dz^2}\log\Phi(z) < 0$.)

8.4 Gradient descent

8.4.1 The algorithm

Gradient descent (GD) moves repeatedly in the steepest-descent direction of Proposition 8.9:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k), \qquad k = 0, 1, 2, \ldots, \tag{8.13}$$

with *step size* (in machine learning, *learning rate*) $\alpha_k > 0$. Two interpretations explain the choice of α_k. First, by the descent lemma (8.11), $\mathbf{x}_{k+1}$ minimises the quadratic upper bound $f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top(\mathbf{y} - \mathbf{x}_k) + \frac{1}{2\alpha}\|\mathbf{y} - \mathbf{x}_k\|^2$ when $\alpha = 1/L$; so the step $1/L$ is the largest one that is guaranteed safe. Second, a GD step is an explicit Euler step for the *gradient flow* ODE $\dot{\mathbf{x}} = -\nabla f(\mathbf{x})$, whose trajectories descend f continuously; too large an Euler step overshoots.

Lemma 8.30 (Sufficient decrease)**.** *If f is L-smooth and $\alpha = 1/L$, then $f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \frac{1}{2L}\|\nabla f(\mathbf{x}_k)\|^2$. More generally, any $0 < \alpha < 2/L$ gives $f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \alpha(1 - \frac{L\alpha}{2})\|\nabla f(\mathbf{x}_k)\|^2$.*

Proof. Put $\mathbf{y} = \mathbf{x}_k - \alpha\nabla f(\mathbf{x}_k)$ in (8.11): $f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \alpha\|\nabla f(\mathbf{x}_k)\|^2 + \frac{L\alpha^2}{2}\|\nabla f(\mathbf{x}_k)\|^2$. □

Summing the sufficient-decrease inequality over k shows that, for any L-smooth f bounded below (convex or not), $\sum_k \|\nabla f(\mathbf{x}_k)\|^2 \le 2L(f(\mathbf{x}_0) - f^\star) < \infty$, so $\min_{j<k}\|\nabla f(\mathbf{x}_j)\| = O(1/\sqrt{k})$: GD always finds approximately stationary points. For convex functions we can say much more.

8.4.2 Convergence rates

Theorem 8.31 (GD on convex L-smooth functions)**.** *Let f be convex and L-smooth with a minimiser $\mathbf{x}^\star$. Gradient descent with $\alpha = 1/L$ satisfies*

$$f(\mathbf{x}_k) - f^\star \le \frac{L\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2k}, \qquad k \ge 1. \tag{8.14}$$

Proof. Write $\mathbf{g}_k = \nabla f(\mathbf{x}_k)$. By Lemma 8.30 and convexity (8.9) with $\mathbf{y} = \mathbf{x}^\star$,

$$f(\mathbf{x}_{k+1}) \le f(\mathbf{x}_k) - \tfrac{1}{2L}\|\mathbf{g}_k\|^2 \le f^\star + \mathbf{g}_k^\top(\mathbf{x}_k - \mathbf{x}^\star) - \tfrac{1}{2L}\|\mathbf{g}_k\|^2 .$$

Expanding $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{x}_k - \mathbf{x}^\star - \mathbf{g}_k/L\|^2$ shows that

$$\mathbf{g}_k^\top(\mathbf{x}_k - \mathbf{x}^\star) - \tfrac{1}{2L}\|\mathbf{g}_k\|^2 = \tfrac{L}{2}\left(\|\mathbf{x}_k - \mathbf{x}^\star\|^2 - \|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2\right).$$

Hence $f(\mathbf{x}_{j+1}) - f^\star \le \frac{L}{2}(\|\mathbf{x}_j - \mathbf{x}^\star\|^2 - \|\mathbf{x}_{j+1} - \mathbf{x}^\star\|^2)$. Summing over $j = 0, \dots, k-1$ the right side telescopes to at most $\frac{L}{2}\|\mathbf{x}_0 - \mathbf{x}^\star\|^2$. Because $f(\mathbf{x}_j)$ is non-increasing (Lemma 8.30), the left side is at least $k\,(f(\mathbf{x}_k) - f^\star)$. □

The $O(1/k)$ rate is *sublinear*: to gain one more digit of accuracy one needs ten times as many iterations. Strong convexity improves this dramatically.

Theorem 8.32 (Linear convergence under strong convexity)**.** *If f is L-smooth and μ-strongly convex, GD with $\alpha = 1/L$ satisfies*

$$f(\mathbf{x}_k) - f^\star \le \left(1 - \frac{1}{\kappa}\right)^k \big(f(\mathbf{x}_0) - f^\star\big), \qquad \kappa = \frac{L}{\mu}. \tag{8.15}$$

Proof. Combine Lemma 8.30 with the Polyak–Łojasiewicz inequality (8.12), $\|\mathbf{g}_k\|^2 \ge 2\mu(f(\mathbf{x}_k) - f^\star)$: $f(\mathbf{x}_{k+1}) - f^\star \le f(\mathbf{x}_k) - f^\star - \frac{\mu}{L}(f(\mathbf{x}_k) - f^\star)$. Iterate. □

"Linear" convergence means the error falls by a constant factor per iteration, i.e. linearly on a log scale. To reach accuracy ε requires about $\kappa \log(1/\varepsilon)$ iterations: *the condition number is the price of the problem*. For quadratics we can compute everything exactly.

Theorem 8.33 (GD on quadratics)**.** *Let $f(\mathbf{x}) = \frac{1}{2}(\mathbf{x}-\mathbf{x}^\star)^\top \mathbf{A}(\mathbf{x}-\mathbf{x}^\star)$ with $\mathbf{A}$ symmetric and eigenvalues $0 < \mu = \lambda_1 \le \cdots \le \lambda_p = L$. GD with constant step α satisfies $\|\mathbf{x}_k - \mathbf{x}^\star\| \le q(\alpha)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|$ with $q(\alpha) = \max\{|1 - \alpha\mu|, |1 - \alpha L|\}$. It converges for every $\mathbf{x}_0$ iff $0 < \alpha < 2/L$. The best constant step is $\alpha^\star = 2/(\mu + L)$, giving*

$$q(\alpha^\star) = \frac{\kappa - 1}{\kappa + 1}. \tag{8.16}$$

Proof. Since $\nabla f(\mathbf{x}) = \mathbf{A}(\mathbf{x} - \mathbf{x}^\star)$, the error $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^\star$ obeys $\mathbf{e}_{k+1} = (\mathbf{I} - \alpha\mathbf{A})\mathbf{e}_k$, so $\mathbf{e}_k = (\mathbf{I} - \alpha\mathbf{A})^k \mathbf{e}_0$. Expanding $\mathbf{e}_0$ in the orthonormal eigenvectors $\mathbf{u}_i$ of $\mathbf{A}$ (Chapter 3), $\mathbf{e}_k = \sum_i (1 - \alpha\lambda_i)^k c_i \mathbf{u}_i$, so $\|\mathbf{e}_k\| \le \max_i |1 - \alpha\lambda_i|^k \, \|\mathbf{e}_0\|$ with equality when $\mathbf{e}_0$ is an eigenvector of the extreme eigenvalue. As $1 - \alpha\lambda$ is linear in λ, the maximum over $[\mu, L]$ is attained at an endpoint, which gives $q(\alpha)$; and $|1 - \alpha L| < 1$ iff $\alpha < 2/L$. q is minimised where $1 - \alpha\mu = -(1 - \alpha L)$, i.e. $\alpha = 2/(\mu + L)$, with value $1 - 2\mu/(\mu + L) = (L - \mu)/(L + \mu)$. □

8.4.3 The condition number and zig-zagging

The proof shows what happens geometrically. Along the steep eigen-direction ($\lambda = L$) the step is so large that the iterate overshoots and changes sign each iteration ($1 - \alpha L \approx -1$); along the flat direction ($\lambda = \mu$) progress is tiny ($1 - \alpha\mu \approx 1$). The trajectory zig-zags across the narrow valley while creeping along it (Fig. 8.5). Listing 8.6 confirms (8.16): the iteration counts with $\alpha = 2/(\mu + L)$ match $\lceil \log(10^{-8})/\log q \rceil$ exactly, and grow linearly in κ.

Listing 8.6: Gradient descent on quadratics with increasing condition number
ch08/ch08_gd_quadratic.py

```
# DEP 803 -- Chapter 8: Optimisation
# Gradient descent on quadratics: condition number, rates and zig-zag
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt


def gd(A, x0, step, tol=1e-8, maxit=200000):
    """GD on f(x) = x'Ax/2; returns iterates until ||x|| < tol*||x0||."""
    x, path = x0.copy(), [x0.copy()]
```

```
    for k in range(maxit):
        x = x - step * (A @ x)
        path.append(x.copy())
        if np.linalg.norm(x) < tol * np.linalg.norm(x0):
            break
    return np.array(path)


x0 = np.array([1.0, 1.0])
print(f"{'kappa':>7}{'iters 1/L':>11}{'iters 2/(mu+L)':>16}"
      f"{'rate (k-1)/(k+1)':>18}{'predicted':>11}")
for kappa in [1, 10, 100, 1000]:
    A = np.diag([1.0, kappa])
    n1 = len(gd(A, x0, 1 / kappa)) - 1
    n2 = len(gd(A, x0, 2 / (1 + kappa))) - 1
    rate = (kappa - 1) / (kappa + 1)
    pred = np.ceil(np.log(1e-8) / np.log(rate)) if kappa > 1 else 1
    print(f"{kappa:>7}{n1:>11}{n2:>16}{rate:>18.4f}{pred:>11.0f}")

# zig-zag picture for kappa = 10 with x0 chosen on the "bad" direction
A = np.diag([1.0, 10.0])
x0 = np.array([10.0, 1.0])
path = gd(A, x0, 2 / 11)[:25]
mom = [x0.copy()]
x, v = x0.copy(), np.zeros(2)
beta, alpha = ((np.sqrt(10) - 1) / (np.sqrt(10) + 1))**2, \
    4 / (1 + np.sqrt(10))**2
for k in range(24):
    v = beta * v - alpha * (A @ x)
    x = x + v
    mom.append(x.copy())
mom = np.array(mom)
print(f"after 24 steps: GD ||x|| = {np.linalg.norm(path[-1]):.2e}, "
      f"heavy-ball ||x|| = {np.linalg.norm(mom[-1]):.2e}")

xx, yy = np.meshgrid(np.linspace(-11, 11, 300), np.linspace(-3.5, 3.5, 300))
plt.figure(figsize=(6, 3.8))
plt.contour(xx, yy, 0.5 * (xx**2 + 10 * yy**2), levels=15, cmap="Greys",
            linewidths=0.7)
plt.plot(path[:, 0], path[:, 1], "o-", ms=3, label="gradient descent")
plt.plot(mom[:, 0], mom[:, 1], "s-", ms=3, label="heavy-ball momentum")
plt.gca().set(xlabel="x1 (flat direction, eigenvalue 1)",
              ylabel="x2 (steep, eigenvalue 10)", title="Zig-zag of GD on an elongated
    ↪ quadratic (kappa = 10)")
plt.legend(frameon=False, loc="upper right")
plt.savefig("ch08_zigzag.pdf", bbox_inches="tight")
```

Output

```
  kappa  iters 1/L  iters 2/(mu+L)  rate (k-1)/(k+1)  predicted
      1          1               1            0.0000          1
     10        172              92            0.8182         92
    100       1799             922            0.9802        922
   1000      18066            9211            0.9980       9211
after 24 steps: GD ||x|| = 8.14e-02, heavy-ball ||x|| = 1.95e-05
```

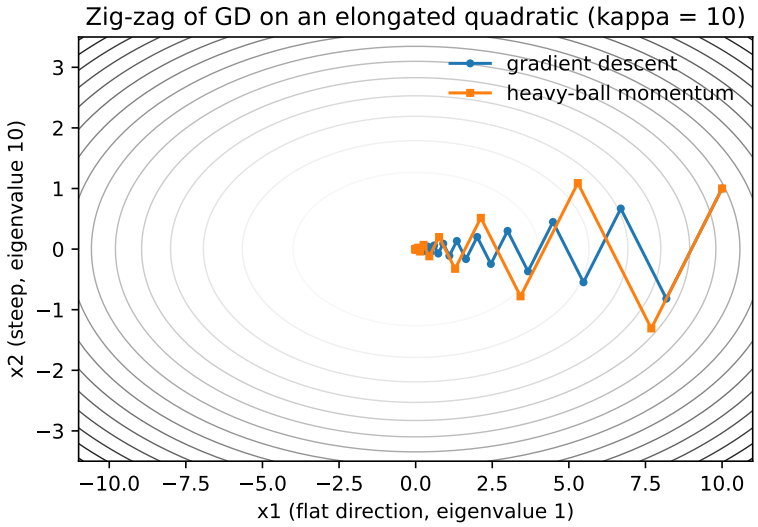


Figure 8.5: Gradient descent with the optimal constant step on $f = \frac{1}{2}(x_1^2 + 10x_2^2)$ zig-zags across the valley; heavy-ball momentum (Section 8.4.5) travels along the valley floor and reaches the minimum far sooner.

⋆ Key Concept: Condition number governs first-order methods

For an L-smooth, μ-strongly convex objective, gradient descent needs $O(\kappa \log(1/\varepsilon))$ iterations, with $\kappa = L/\mu$ the ratio of largest to smallest Hessian eigenvalue. For least squares $\kappa = \mathrm{cond}(\mathbf{X}^\top\mathbf{X}) = \mathrm{cond}(\mathbf{X})^2$ (Chapter 3), so poorly scaled or collinear covariates slow GD down quadratically in $\mathrm{cond}(\mathbf{X})$. Momentum methods reduce the dependence to $\sqrt{\kappa}$; Newton's method removes it locally.

Historical Note: Cauchy and the method of steepest descent

In a three-page note to the Paris Academy of Sciences in 1847, Augustin-Louis Cauchy proposed solving systems of equations arising in astronomical computations by minimising a sum of squares and moving, at each step, in the direction in which the function decreases fastest (Cauchy, 1847). He also suggested choosing the step length by a one-dimensional minimisation, the idea behind modern line searches. The method was rediscovered several times; its convergence was analysed rigorously in the twentieth century, including by Haskell Curry (1944) and by Leonid Kantorovich, whose inequality yields the rate $((\kappa-1)/(\kappa+1))^2$ for steepest descent with exact line search on quadratics.

8.4.4 Choosing the step: backtracking and the Armijo condition

The constant L is rarely known, and a global L may be far too pessimistic locally. A *line search* chooses α_k adaptively. The *Armijo* (sufficient decrease) condition for a descent direction $\mathbf{d}_k$ (one with $\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k < 0$) is

$$f(\mathbf{x}_k + \alpha \mathbf{d}_k) \le f(\mathbf{x}_k) + c\,\alpha\,\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k, \qquad c \in (0,1), \tag{8.17}$$

typically $c = 10^{-4}$ for Newton-type methods and $c = \frac{1}{2}$ for gradient descent (Armijo, 1966). *Backtracking* starts from a trial step and halves it until (8.17) holds (Algorithm 4).

Proposition 8.34. *If f is L-smooth and $\mathbf{d} = -\nabla f(\mathbf{x})$, the Armijo condition with $c \le \frac{1}{2}$ holds for every $\alpha \le 1/L$. Hence backtracking terminates with $\alpha \ge \min\{\bar\alpha, \tau/L\}$, and Theorems 8.31 and 8.32 hold with L replaced by $\max\{1/\bar\alpha, L/\tau\}$.*

Algorithm 4 Gradient descent with Armijo backtracking

Require: $\mathbf{x}_0$, initial trial step $\bar{\alpha} > 0$, $c \in (0,1)$, shrink factor $\tau \in (0,1)$, tolerance ε

for $k = 0, 1, 2, \dots$ **do**
  $\mathbf{g} \leftarrow \nabla f(\mathbf{x}_k)$; **if** $\|\mathbf{g}\| \le \varepsilon$ **stop**
  $\alpha \leftarrow \bar{\alpha}$
  while $f(\mathbf{x}_k - \alpha \mathbf{g}) > f(\mathbf{x}_k) - c\,\alpha\,\|\mathbf{g}\|^2$ **do**
    $\alpha \leftarrow \tau \alpha$ ▷ backtrack
  end while
  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \mathbf{g}$
end for

Proof. From the proof of Lemma 8.30, $f(\mathbf{x} - \alpha\mathbf{g}) \le f(\mathbf{x}) - \alpha(1 - \tfrac{L\alpha}{2})\,\|\mathbf{g}\|^2 \le f(\mathbf{x}) - \tfrac{\alpha}{2}\,\|\mathbf{g}\|^2$ when $\alpha \le 1/L$. □

Practical line searches (in SciPy's BFGS, for example) also impose a *curvature* condition that rules out steps that are too short; together with (8.17) these are the *Wolfe conditions* (Nocedal and Wright, 2006, Ch. 3).

8.4.5 Momentum and Nesterov acceleration

Zig-zagging suggests averaging successive steps, so that oscillating components cancel and consistent components accumulate. Polyak's *heavy-ball method* (Polyak, 1964) adds a fraction β of the previous displacement:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1}). \tag{8.18}$$

Nesterov's accelerated gradient (Nesterov, 1983) evaluates the gradient at the extrapolated point instead:

$$\mathbf{y}_k = \mathbf{x}_k + \beta_k(\mathbf{x}_k - \mathbf{x}_{k-1}), \qquad \mathbf{x}_{k+1} = \mathbf{y}_k - \tfrac{1}{L}\nabla f(\mathbf{y}_k). \tag{8.19}$$

Theorem 8.35 (Accelerated rates; statement)**.** *Let f be convex and L-smooth.*

(a) Nesterov's method with $\beta_k = \frac{k-1}{k+2}$ satisfies $f(\mathbf{x}_k) - f^\star \le \frac{2L\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{(k+1)^2}$.

(b) If f is also μ-strongly convex, Nesterov's method with constant $\beta = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ satisfies $f(\mathbf{x}_k) - f^\star \le (1 - 1/\sqrt{\kappa})^k \big(f(\mathbf{x}_0) - f^\star + \tfrac{\mu}{2}\|\mathbf{x}_0 - \mathbf{x}^\star\|^2\big)$.

(c) For quadratics with $\mu\mathbf{I} \preceq \mathbf{A} \preceq L\mathbf{I}$, the heavy-ball method with $\alpha = 4/(\sqrt{L}+\sqrt{\mu})^2$ and $\beta = \big(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\big)^2$ converges at the asymptotic rate $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ per iteration.

Proofs use Lyapunov ("estimate sequence") arguments; see Nesterov (2018, Ch. 2). The effect is to replace κ by $\sqrt{\kappa}$ in iteration counts: for $\kappa = 10^4$, from about 10^4 iterations per digit to about 10^2. Heavy-ball's guarantee in (c) does not extend to all strongly convex functions: there are smooth strongly convex examples on which it fails to converge (Lessard et al., 2016), which is one reason Nesterov's scheme is preferred in theory.

Graduate Perspective: Optimal first-order methods

Nesterov (2018) shows that for any method whose iterates lie in the span of previously computed gradients there exist L-smooth convex functions (in high enough dimension) for which $f(\mathbf{x}_k) - f^\star \ge \frac{3L\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{32(k+1)^2}$, and μ-strongly convex ones for which $\|\mathbf{x}_k - \mathbf{x}^\star\|^2$ decays no faster than $\big(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\big)^{2k}\|\mathbf{x}_0 - \mathbf{x}^\star\|^2$. Nesterov's method attains these lower bounds up to

constants and is therefore *optimal* among first-order methods. Faster convergence requires more information (second derivatives, as in Section 8.5) or more structure (e.g. finite sums, exploited by variance-reduced stochastic methods, Section 8.6).

8.4.6 An econometric example and feature scaling

Listing 8.7 fits a least-squares regression of a simulated household outcome on household size, land (acres) and years of schooling of the household head, *without* rescaling. The raw design has $\kappa = \mathrm{cond}(\mathbf{X}^\top\mathbf{X}/n) \approx 1755$. Plain GD with step $1/L$ and GD with Armijo backtracking (which can take somewhat larger steps) are still far from the optimum after 3000 iterations; heavy-ball and Nesterov reach 10^{-8} in a few hundred (Fig. 8.6). After standardising the covariates ($\kappa \approx 1.1$), plain GD needs four iterations.

Listing 8.7: GD, backtracking, heavy-ball and Nesterov on least squares `ch08/ch08_gd_compare.py`

```
# DEP 803 -- Chapter 8: Optimisation
# GD, Armijo backtracking, heavy-ball and Nesterov on least squares
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n = 2000
# simulated household survey: size, land (acres), years of schooling of head
hh = rng.poisson(4, n) + 1.0
land = rng.gamma(2.0, 1.5, n)
sch = rng.uniform(0, 18, n)
X = np.column_stack([np.ones(n), hh, land, sch])
y = X @ np.array([2.0, 0.3, 0.5, 0.1]) + rng.normal(0, 1, n)


def make(X):
    f = lambda b: 0.5 * np.mean((y - X @ b)**2)
    g = lambda b: -X.T @ (y - X @ b) / len(y)
    eig = np.linalg.eigvalsh(X.T @ X / len(y))
    return f, g, eig.min(), eig.max()


def run(X, method, iters=3000):
    f, g, mu, L = make(X)
    fstar = f(np.linalg.lstsq(X, y, rcond=None)[0])
    b = np.zeros(X.shape[1])
    b_prev, hist = b.copy(), []
    t = 1.0
    for k in range(iters):
        hist.append(f(b) - fstar)
        if method == "GD 1/L":
            b = b - g(b) / L
        elif method == "GD + Armijo":
            gb, t = g(b), min(1.0, 4 * t)      # backtracking line search
            while f(b - t * gb) > f(b) - 0.5 * t * gb @ gb:
                t *= 0.5
            b = b - t * gb
        elif method == "heavy ball":
            q = np.sqrt(L / mu)
            a, m = 4 / (np.sqrt(L) + np.sqrt(mu))**2, ((q - 1) / (q + 1))**2
```

```
            b, b_prev = b - a * g(b) + m * (b - b_prev), b
        elif method == "Nesterov":
            q = np.sqrt(L / mu)
            z = b + (q - 1) / (q + 1) * (b - b_prev)
            b, b_prev = z - g(z) / L, b
    return np.maximum(np.array(hist), 1e-16), L / mu


methods = ["GD 1/L", "GD + Armijo", "heavy ball", "Nesterov"]
plt.figure(figsize=(6, 3.8))
print(f"{'method':<14}{'gap after 3000':>16}{'iters to 1e-8':>15}")
for m in methods:
    h, kappa = run(X, m)
    hit = str(np.argmax(h < 1e-8)) if (h < 1e-8).any() else "> 3000"
    print(f"{m:<14}{h[-1]:>16.2e}{hit:>15}")
    plt.semilogy(h, label=m)
print(f"condition number of X'X/n (raw): {kappa:,.0f}")

Xs = X.copy()
Xs[:, 1:] = (X[:, 1:] - X[:, 1:].mean(0)) / X[:, 1:].std(0)
h, kappa = run(Xs, "GD 1/L", 200)
print(f"standardised: kappa = {kappa:.2f}, GD 1/L reaches 1e-8 in "
      f"{np.argmax(h < 1e-8)} iterations")
plt.gca().set(xlabel="iteration k", ylabel=r"$f(\beta_k) - f^\star$")
plt.ylim(1e-14, 1e2)
plt.legend(frameon=False)
plt.title("First-order methods on raw (unscaled) least squares")
plt.savefig("ch08_gd_rates.pdf", bbox_inches="tight")
```

Output

```
method          gap after 3000  iters to 1e-8
GD 1/L                4.98e-03         > 3000
GD + Armijo           2.85e-07         > 3000
heavy ball            1.00e-16            360
Nesterov              1.00e-16            444
condition number of X'X/n (raw): 1,755
standardised: kappa = 1.11, GD 1/L reaches 1e-8 in 4 iterations
```

Standardising the covariates is a change of variables $\boldsymbol{\beta} = \mathbf{D}^{-1}\tilde{\boldsymbol{\beta}}$ (plus a shift for centring) that does not change the fitted values or the optimum, only the geometry of the problem seen by the algorithm. It is a crude *preconditioner*: GD on the rescaled problem equals GD with the scaled step $\mathbf{D}^{-2}\nabla f$ on the original one. Newton's method in Section 8.5 uses the best possible (local) preconditioner, the Hessian itself.

△ Caution: Step sizes: too large diverges, too small stalls

For a quadratic, any constant step $\alpha > 2/L$ makes GD diverge geometrically along the top eigen-direction (Theorem 8.33); a "loss going to infinity" or `nan` in training usually means the learning rate is too large for the current scaling. Conversely, a tiny step gives slow progress that is easily mistaken for convergence. Stopping when f "stops changing" is unreliable on flat valleys; monitor $\|\nabla f\|$ and, for strongly convex problems, use (8.12) to translate it into a bound on the optimality gap.

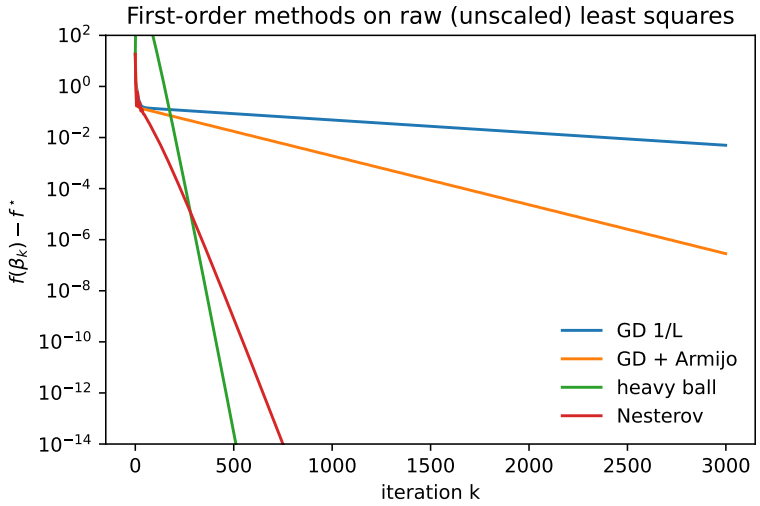


Figure 8.6: Optimality gap of four first-order methods on an unscaled least-squares problem ($\kappa \approx 1755$). GD and GD with backtracking converge linearly but slowly; the accelerated methods' slopes reflect the $\sqrt{\kappa}$ dependence. (The early rise of heavy ball is a transient typical of momentum.)

Exercises

8.4.1 Theory Show that for an L-smooth convex f the GD iterates with $\alpha = 1/L$ satisfy $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\| \le \|\mathbf{x}_k - \mathbf{x}^\star\|$ (Fejér monotonicity). Deduce that the iterates stay in a bounded set.

8.4.2 Theory Prove that under the assumptions of Theorem 8.32 the iterates themselves converge linearly: $\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \le \kappa(1 - 1/\kappa)^k \, \|\mathbf{x}_0 - \mathbf{x}^\star\|^2$.

8.4.3 Theory For the quadratic $f = \tfrac{1}{2}(x_1^2 + \kappa x_2^2)$ and steepest descent with *exact* line search, compute the step α_k in closed form and show that from $\mathbf{x}_0 = (\kappa, 1)$ the iterates satisfy $\mathbf{x}_k = \left(\tfrac{\kappa-1}{\kappa+1}\right)^k (\kappa, (-1)^k)$.

8.4.4 Python Reproduce Fig. 8.5 for $\kappa = 50$ with (a) the optimal constant step, (b) exact line search, (c) heavy ball and (d) Nesterov. Count the iterations needed to reach $\|\mathbf{x}_k\| < 10^{-6}$ for each.

8.4.5 Python In Listing 8.7, replace the constant step $1/L$ by $1.9/L$ and by $2.05/L$. What happens, and why? Then run GD on centred but *unscaled* covariates and compare κ with the raw and standardised versions. Which matters more here, centring or scaling?

8.4.6 Policy A data team fits a regression of household consumption on income measured in cedis (values around 10^4) and household size (values around 5) by gradient descent and reports that "the algorithm converged after 10 000 iterations" with a coefficient on household size close to its starting value of zero. Diagnose the problem and propose two remedies.

8.4.7 ★ Graduate Prove Theorem 8.35(c): write the heavy-ball recursion for a quadratic as a linear system $\mathbf{z}_{k+1} = \mathbf{T}\mathbf{z}_k$ in $\mathbf{z}_k = (\mathbf{e}_k, \mathbf{e}_{k-1})$, reduce it to 2×2 blocks using the eigenvectors of $\mathbf{A}$, and show that with the stated α, β every block has spectral radius $\sqrt{\beta}$.

8.5 Newton and quasi-Newton methods

8.5.1 Newton's method

Gradient descent uses a first-order model of f and a fixed curvature guess $1/\alpha$. Newton's method uses the full second-order Taylor model $m_k(\mathbf{d}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{d} + \tfrac{1}{2}\mathbf{d}^\top \nabla^2 f(\mathbf{x}_k)\mathbf{d}$. If $\nabla^2 f(\mathbf{x}_k) \succ 0$, m_k is minimised where $\nabla m_k(\mathbf{d}) = \mathbf{0}$, giving the *Newton step*

$$\mathbf{d}_k = -\left[\nabla^2 f(\mathbf{x}_k)\right]^{-1} \nabla f(\mathbf{x}_k), \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + t_k \mathbf{d}_k, \tag{8.20}$$

with $t_k = 1$ for the *pure* method. Equivalently, Newton's method applies the Newton–Raphson root-finding iteration to the first-order condition $\nabla f(\mathbf{x}) = \mathbf{0}$. In practice one never forms the inverse: one solves $\nabla^2 f(\mathbf{x}_k)\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ by a Cholesky factorisation (Chapter 2), which also detects a Hessian that is not positive definite. Three properties distinguish Newton's method:

(i) it minimises a quadratic in *one* step, whatever its condition number;

(ii) it is *affine invariant*: if $\mathbf{x} = \mathbf{B}\mathbf{y}$ with $\mathbf{B}$ invertible, Newton on $g(\mathbf{y}) = f(\mathbf{B}\mathbf{y})$ produces exactly the transformed Newton iterates on f, so rescaling covariates has no effect (contrast Listing 8.7);

(iii) near a minimiser with positive definite Hessian it converges *quadratically*.

Theorem 8.36 (Local quadratic convergence). *Let $f \in C^2$ with $\nabla f(\mathbf{x}^\star) = \mathbf{0}$, $\nabla^2 f(\mathbf{x}^\star) \succeq \mu \mathbf{I}$ for some $\mu > 0$, and Hessian Lipschitz with constant M: $\left\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\right\| \le M \|\mathbf{x} - \mathbf{y}\|$. If $\|\mathbf{x}_0 - \mathbf{x}^\star\| \le \mu/(2M)$, the pure Newton iterates satisfy*

$$\left\|\mathbf{x}_{k+1} - \mathbf{x}^\star\right\| \le \frac{M}{\mu} \left\|\mathbf{x}_k - \mathbf{x}^\star\right\|^2, \tag{8.21}$$

so they converge to $\mathbf{x}^\star$ and the number of correct digits roughly doubles at every iteration.

Proof sketch. Write $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^\star$ and $\mathbf{H}_k = \nabla^2 f(\mathbf{x}_k)$. By the integral form of Taylor's theorem, $\nabla f(\mathbf{x}_k) = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}^\star) = \int_0^1 \nabla^2 f(\mathbf{x}^\star + s\mathbf{e}_k)\mathbf{e}_k \, ds$. Hence

$$\mathbf{e}_{k+1} = \mathbf{e}_k - \mathbf{H}_k^{-1}\nabla f(\mathbf{x}_k) = \mathbf{H}_k^{-1} \int_0^1 \left[\mathbf{H}_k - \nabla^2 f(\mathbf{x}^\star + s\mathbf{e}_k)\right]\mathbf{e}_k \, ds.$$

The integrand is bounded by $M(1-s)\|\mathbf{e}_k\|^2$, whose integral is $\tfrac{M}{2}\|\mathbf{e}_k\|^2$. If $\|\mathbf{e}_k\| \le \mu/(2M)$ then $\mathbf{H}_k \succeq \nabla^2 f(\mathbf{x}^\star) - M\|\mathbf{e}_k\|\mathbf{I} \succeq \tfrac{\mu}{2}\mathbf{I}$, so $\left\|\mathbf{H}_k^{-1}\right\| \le 2/\mu$. Together: $\|\mathbf{e}_{k+1}\| \le \tfrac{M}{\mu}\|\mathbf{e}_k\|^2 \le \tfrac{1}{2}\|\mathbf{e}_k\|$, so the iterates stay in the ball and the bound applies at every step. □

8.5.2 Damping and globalisation

Quadratic convergence is *local*. Far from the solution the quadratic model can be a poor guide and the pure Newton step can overshoot catastrophically, even for a convex function. The classical example is $f(x) = \sqrt{1 + x^2}$, strictly convex with minimum at 0. Here $f'(x)/f''(x) = x(1 + x^2)$, so the pure Newton map is $x \mapsto -x^3$: it converges (cubically) if $|x_0| < 1$ and diverges if $|x_0| > 1$. *Damped Newton* chooses t_k by Armijo backtracking (8.17) along the Newton direction; once the iterates enter the region of Theorem 8.36, the full step $t_k = 1$ is accepted and quadratic convergence resumes. Listing 8.8 shows both behaviours.

Listing 8.8: Pure versus damped Newton on $f(x) = \sqrt{1 + x^2}$ ch08/ch08_newton_damping.py

```
# DEP 803 -- Chapter 8: Optimisation
# Pure Newton can diverge; damped Newton with backtracking is globally safe
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```

f = lambda x: np.sqrt(1 + x**2)          # convex, minimum at x = 0
g = lambda x: x / np.sqrt(1 + x**2)
h = lambda x: (1 + x**2) ** -1.5


def newton(x, damped, iters=6):
    out = [x]
    for k in range(iters):
        d = -g(x) / h(x)                 # Newton direction (= -x - x^3 here)
        t = 1.0
        if damped:                       # Armijo backtracking
            while f(x + t * d) > f(x) + 1e-4 * t * g(x) * d:
                t *= 0.5
        x = x + t * d
        out.append(x)
    return out


for x0 in [0.5, 1.5]:
    for damped in [False, True]:
        xs = newton(x0, damped)
        lab = "damped" if damped else "pure  "
        print(f"x0 = {x0}, {lab}: " + " ".join(f"{v:9.2e}" for v in xs))
```

Output

```
x0 = 0.5, pure  :  5.00e-01 -1.25e-01  1.95e-03 -7.45e-09  0.00e+00  0.00e+00  0.00e+00
x0 = 0.5, damped:  5.00e-01 -1.25e-01  1.95e-03 -7.45e-09  0.00e+00  0.00e+00  0.00e+00
x0 = 1.5, pure  :  1.50e+00 -3.38e+00  3.84e+01 -5.68e+04  1.83e+14 -6.17e+42 2.35e+128
x0 = 1.5, damped:  1.50e+00 -9.38e-01  8.24e-01 -5.59e-01  1.75e-01 -5.37e-03  1.55e-07
```

For non-convex f the Hessian may be indefinite, and $\mathbf{d}_k$ need not be a descent direction. Standard remedies are *modified Newton* (add $\tau\mathbf{I}$ to the Hessian until the Cholesky factorisation succeeds) and *trust-region* methods, which minimise the quadratic model over a ball $\|\mathbf{d}\| \le \Delta_k$ and adapt Δ_k according to how well the model predicted the actual decrease (Nocedal and Wright, 2006, Ch. 4); `trust-exact` below is one.

Historical Note: Newton, Raphson and Simpson

Isaac Newton described an iterative scheme for solving the cubic $x^3 - 2x - 5 = 0$ in *De analysi* (written 1669, published 1711) and in the *Method of Fluxions* (written 1671, published 1736). His version successively corrected a polynomial rather than updating x directly. Joseph Raphson, in *Analysis aequationum universalis* (1690), gave the simpler update of x itself that we now use; hence "Newton–Raphson". Neither formulation used derivatives explicitly. Thomas Simpson (1740) stated the method in terms of fluxions (derivatives), applied it to systems of two equations, and noted that it could be used to solve optimisation problems by setting the gradient to zero (Ypma, 1995).

8.5.3 Newton–Raphson and IRLS for logistic regression

For the logit model, with $\mathbf{p} = \sigma(\mathbf{X}\boldsymbol{\beta})$ and $\mathbf{W} = \operatorname{diag}\{p_i(1-p_i)\}$, the gradient and Hessian of the negative log-likelihood are $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})$ and $\mathbf{X}^\top\mathbf{W}\mathbf{X}$ (8.4). The Newton step can be rearranged as

$$\boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k + (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top(\mathbf{y} - \mathbf{p}) = (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{W}\mathbf{z}, \qquad \mathbf{z} = \mathbf{X}\boldsymbol{\beta}_k + \mathbf{W}^{-1}(\mathbf{y} - \mathbf{p}), \tag{8.22}$$

which is a *weighted least-squares* regression of the "working response" $\mathbf{z}$ on $\mathbf{X}$ with weights $\mathbf{W}$. Because the weights are recomputed at every step, the algorithm is called *iteratively reweighted least squares* (IRLS); it is how `glm` routines fit generalised linear models. For canonical links the observed and expected information coincide, so Newton–Raphson and Fisher scoring (Chapter 7) are the same algorithm.

Listing 8.9 fits a logit of programme take-up on a *simulated* beneficiary survey of 5000 households: take-up depends on log income (negatively), a female-headed household indicator and the distance to the district office. Newton converges from $\beta_0 = \mathbf{0}$ in five iterations; the ratio $\|\mathbf{g}_k\| / \|\mathbf{g}_{k-1}\|^2$ stays bounded, the signature of quadratic convergence. The inverse Hessian at the optimum gives the usual MLE standard errors (Cameron and Trivedi, 2005, Ch. 5), and the estimates coincide with scikit-learn's unpenalised fit.

Listing 8.9: Newton–Raphson (IRLS) for a logit of programme take-up `ch08/ch08_newton_logit.py`

```
# DEP 803 -- Chapter 8: Optimisation
# Newton-Raphson / IRLS for a logit of programme take-up (simulated data)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(803)
n = 5000
# simulated beneficiary survey: does the household take up a subsidy?
loginc = rng.normal(7.5, 0.8, n)          # log monthly income
female = rng.binomial(1, 0.45, n)         # female-headed household
dist = rng.gamma(2.0, 3.0, n)             # km to the district office
X = np.column_stack([np.ones(n), loginc - 7.5, female, dist])
names = ["const", "loginc", "female", "dist_km"]
beta_true = np.array([0.8, -0.9, 0.5, -0.12])
y = (rng.uniform(size=n) < 1 / (1 + np.exp(-X @ beta_true))).astype(float)


def newton(b, tol=1e-10, maxit=50):
    hist = []
    for k in range(maxit):
        p = 1 / (1 + np.exp(-X @ b))
        g = X.T @ (p - y)                       # gradient of the NLL
        hist.append(np.linalg.norm(g))
        W = p * (1 - p)
        H = (X * W[:, None]).T @ X              # Hessian X'WX
        if hist[-1] < tol:
            break
        b = b - np.linalg.solve(H, g)           # Newton step = IRLS update
    return b, H, hist


b_hat, H, hist = newton(np.zeros(4))
print("Newton iterations: ||gradient||_k and ratio g_k / g_(k-1)^2")
for k, gk in enumerate(hist):
    ratio = f"{gk / hist[k - 1]**2:9.2e}" if 0 < k and hist[k - 1] < 1 else ""
    print(f"  k = {k}: {gk:10.3e}   {ratio}")
se = np.sqrt(np.diag(np.linalg.inv(H)))
sk = LogisticRegression(penalty="none", fit_intercept=False, tol=1e-10,
                        max_iter=1000).fit(X, y)
print(f"{'':>9}{'truth':>8}{'Newton':>9}{'se':>8}{'sklearn':>9}")
```

```python
for j in range(4):
    print(f"{names[j]:>9}{beta_true[j]:>8.3f}{b_hat[j]:>9.4f}{se[j]:>8.4f}"
          f"{sk.coef_[0, j]:>9.4f}")

# gradient descent on the same objective, step 1/L with L = lambda_max(X'X)/4
L = np.linalg.eigvalsh(X.T @ X).max() / 4
b, gd_hist = np.zeros(4), []
for k in range(5000):
    g = X.T @ (1 / (1 + np.exp(-X @ b)) - y)
    gd_hist.append(np.linalg.norm(g))
    b -= g / L
print(f"GD with step 1/L: ||grad|| after 5000 iters = {gd_hist[-1]:.2e}")
print(f"max |b_GD - b_Newton| = {np.abs(b - b_hat).max():.2e}")

plt.figure(figsize=(6, 3.8))
plt.semilogy(gd_hist, label="gradient descent (step 1/L)")
plt.semilogy(range(len(hist)), hist, "o-", label="Newton / IRLS")
plt.xscale("symlog", linthresh=10)
plt.gca().set(xlabel="iteration (symlog scale)", ylabel="norm of gradient",
              title="Logit take-up model: Newton vs gradient descent")
plt.legend(frameon=False)
plt.savefig("ch08_newton_conv.pdf", bbox_inches="tight")
```

Output

```
Newton iterations: ||gradient||_k and ratio g_k / g_(k-1)^2
  k = 0:  7.415e+02
  k = 1:  1.064e+02
  k = 2:  6.459e+00
  k = 3:  2.500e-02
  k = 4:  3.477e-07    5.57e-04
  k = 5:  4.077e-13    3.37e+00
            truth   Newton      se  sklearn
    const   0.800   0.8505  0.0610   0.8505
   loginc  -0.900  -0.8892  0.0429  -0.8892
   female   0.500   0.4503  0.0627   0.4503
  dist_km  -0.120  -0.1215  0.0078  -0.1215
GD with step 1/L: ||grad|| after 5000 iters = 4.97e-05
max |b_GD - b_Newton| = 1.92e-07
```

The cost comparison is instructive. A Newton iteration forms $\mathbf{X}^\top\mathbf{W}\mathbf{X}$ ($O(np^2)$) and solves a $p \times p$ system ($O(p^3)$); a GD iteration costs $O(np)$. With $p = 4$ Newton is overwhelmingly better. With $p = 10^5$ features (text or image data) a single Newton step is infeasible, and first-order or quasi-Newton methods take over.

8.5.4 Gauss–Newton for nonlinear least squares

Many models in economics are fitted by *nonlinear least squares* (NLS): $f(\boldsymbol{\theta}) = \tfrac{1}{2}\,\|\mathbf{r}(\boldsymbol{\theta})\|^2$ with residuals $r_i(\boldsymbol{\theta}) = y_i - m(\mathbf{x}_i;\boldsymbol{\theta})$. With $\mathbf{J}$ the Jacobian of $\mathbf{r}$,

$$\nabla f = \mathbf{J}^\top \mathbf{r}, \qquad \nabla^2 f = \mathbf{J}^\top\mathbf{J} + \sum_{i} r_i\,\nabla^2 r_i .$$

The *Gauss–Newton* method drops the second term, which is small when the residuals are small or the model is nearly linear, and solves $(\mathbf{J}^\top\mathbf{J})\mathbf{d} = -\mathbf{J}^\top\mathbf{r}$, i.e. the *linear* least-squares problem

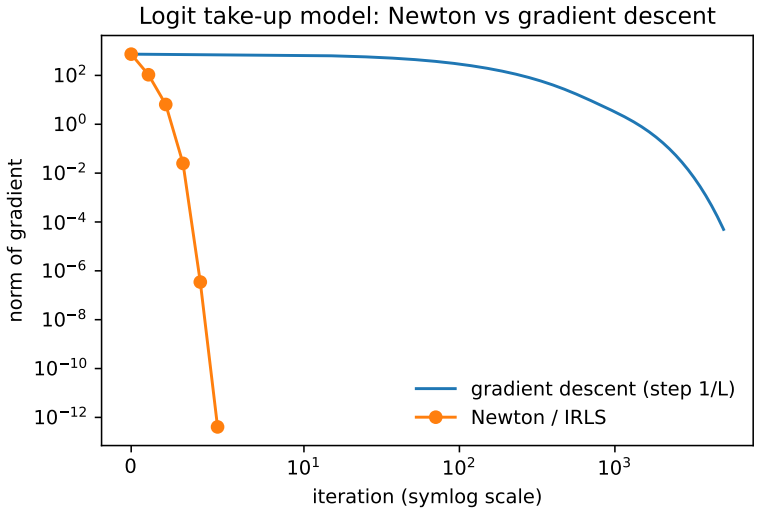


Figure 8.7: Gradient norm for the take-up logit: Newton/IRLS reaches 10^{-13} in 5 iterations, gradient descent with step $1/L$ needs thousands of iterations to reach 10^{-4}, because the unscaled covariate "distance in km" makes the problem ill-conditioned.

$\min_{\mathbf{d}} \|\mathbf{J}\mathbf{d} + \mathbf{r}\|^2$ (best solved by QR, Chapter 2). It needs only first derivatives, $\mathbf{J}^\top\mathbf{J}$ is always positive semidefinite, and convergence is fast (quadratic for zero-residual problems, linear with a small constant otherwise). The *Levenberg–Marquardt* method (Levenberg, 1944; Marquardt, 1963) solves $(\mathbf{J}^\top\mathbf{J} + \lambda_k \mathbf{I})\mathbf{d} = -\mathbf{J}^\top\mathbf{r}$, interpolating between Gauss–Newton ($\lambda_k \to 0$) and short gradient steps (λ_k large); it is a trust-region method and the default in `scipy.optimize.least_squares(method="lm")`.

Listing 8.10 fits a logistic *diffusion curve* $m(t) = K/(1 + e^{-r(t-t_0)})$ (ceiling K, speed r, inflection month t_0) to 48 months of *simulated* adoption shares of a mobile-money service. From a poor starting curve, Gauss–Newton converges in about six iterations to the same estimates as Levenberg–Marquardt (Fig. 8.8).

Listing 8.10: Gauss–Newton for a logistic diffusion curve ch08/ch08_gauss_newton.py

```
# DEP 803 -- Chapter 8: Optimisation
# Gauss-Newton for a logistic adoption curve (simulated mobile-money uptake)
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares

rng = np.random.default_rng(803)
t = np.arange(48.0)                                  # months since launch
K, r, t0 = 0.62, 0.18, 22.0                          # true parameters
share = K / (1 + np.exp(-r * (t - t0))) + rng.normal(0, 0.015, t.size)


def model(th):
    return th[0] / (1 + np.exp(-th[1] * (t - th[2])))


def resid(th):
    return share - model(th)


def jac(th):                                         # Jacobian of residuals
```

```
    e = np.exp(-th[1] * (t - th[2]))
    d = 1 + e
    return -np.column_stack([1 / d, th[0] * (t - th[2]) * e / d**2,
                             -th[0] * th[1] * e / d**2])


start = np.array([0.5, 0.1, 15.0])                  # starting guess
th = start.copy()
print(f"{'it':>3}{'K':>9}{'r':>9}{'t0':>9}{'SSR':>12}")
for k in range(15):
    rk, J = resid(th), jac(th)
    print(f"{k:>3}{th[0]:>9.4f}{th[1]:>9.4f}{th[2]:>9.4f}{rk @ rk:>12.4e}")
    step = np.linalg.lstsq(J, -rk, rcond=None)[0]   # solve J d = -r (LS)
    th = th + step
    if np.linalg.norm(step) < 1e-9:
        break
lm = least_squares(resid, start, jac=jac, method="lm")
print("scipy least_squares (LM):", np.round(lm.x, 4), " nfev =", lm.nfev)

plt.figure(figsize=(6, 3.8))
plt.plot(t, share, "o", ms=4, label="simulated adoption share")
plt.plot(t, model(th), lw=2, label="Gauss-Newton fit")
plt.plot(t, model(start), "--", label="starting curve")
plt.gca().set(xlabel="months since launch", ylabel="share of adults using service",
              title="Nonlinear least squares: logistic diffusion curve")
plt.legend(frameon=False)
plt.savefig("ch08_gauss_newton.pdf", bbox_inches="tight")
```

Output

```
 it        K        r       t0         SSR
  0   0.5000   0.1000  15.0000  5.6312e-01
  1   0.6776   0.1281  29.7206  3.8122e-01
  2   0.5329   0.1588  19.3463  1.4416e-01
  3   0.6254   0.1722  23.0573  1.3001e-02
  4   0.6183   0.1823  22.1395  6.2493e-03
  5   0.6196   0.1828  22.1910  6.2111e-03
  6   0.6196   0.1828  22.1911  6.2111e-03
  7   0.6196   0.1828  22.1911  6.2111e-03
  8   0.6196   0.1828  22.1911  6.2111e-03
scipy least_squares (LM): [ 0.6196  0.1828 22.1911]  nfev = 8
```

8.5.5 Quasi-Newton methods: BFGS and L-BFGS

Quasi-Newton methods keep the fast local convergence of Newton without computing Hessians. They maintain an approximation $\mathbf{B}_k \approx \nabla^2 f(\mathbf{x}_k)$ (or $\mathbf{H}_k \approx [\nabla^2 f(\mathbf{x}_k)]^{-1}$) updated from gradient differences. With $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$, Taylor's theorem gives $\mathbf{y}_k \approx \nabla^2 f\, \mathbf{s}_k$, which motivates the *secant condition* $\mathbf{B}_{k+1}\mathbf{s}_k = \mathbf{y}_k$. Among all symmetric matrices satisfying it, the BFGS update chooses the inverse approximation closest to $\mathbf{H}_k$ in a weighted Frobenius norm:

$$\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top)\mathbf{H}_k(\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top, \qquad \rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}. \tag{8.23}$$

The search direction is $\mathbf{d}_k = -\mathbf{H}_k \nabla f(\mathbf{x}_k)$ with a Wolfe line search, which guarantees the *curvature condition* $\mathbf{y}_k^\top \mathbf{s}_k > 0$, and with it that $\mathbf{H}_{k+1}$ stays positive definite. Each iteration costs $O(p^2)$, and

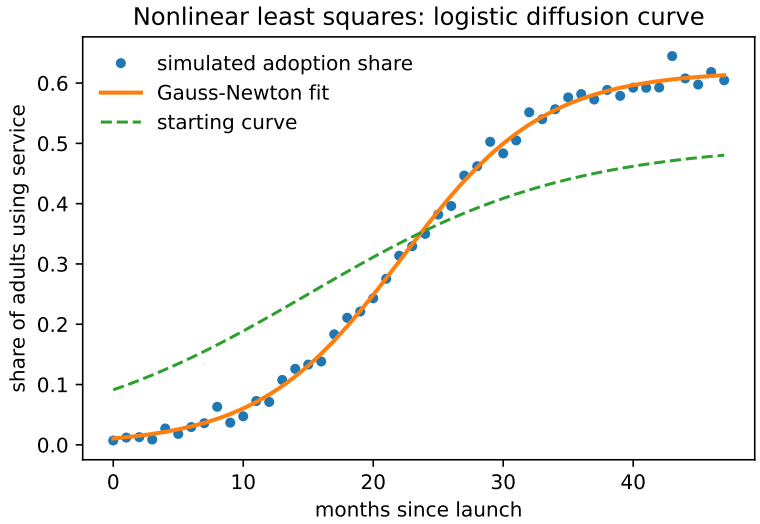


Figure 8.8: Nonlinear least-squares fit of a logistic diffusion curve to simulated adoption data.

under standard conditions BFGS converges *superlinearly*: $\|\mathbf{e}_{k+1}\|/\|\mathbf{e}_k\| \to 0$ (Nocedal and Wright, 2006, Ch. 6).

For large p even storing $\mathbf{H}_k$ is too expensive. *Limited-memory BFGS* (L-BFGS) (Liu and Nocedal, 1989) stores only the last m pairs $(\mathbf{s}_j, \mathbf{y}_j)$ (typically $m = 5$–20) and computes $\mathbf{H}_k \nabla f$ implicitly by a "two-loop recursion" in $O(mp)$ operations. L-BFGS is the default solver of scikit-learn's `LogisticRegression`, of Stan's optimiser (Matsuura, 2022), and of many structural estimation codes.

Listing 8.11 compares the methods of `scipy.optimize.minimize` on the 10-dimensional Rosenbrock function (a non-convex "banana valley") and on a logit with $n = 10\,000$, $p = 20$. Derivative-free Nelder–Mead exhausts its budget of 3000 evaluations on both problems without converging. Among gradient methods, iteration counts fall from CG to BFGS to Newton-type methods, but *time* depends on the cost per iteration: on the logit, where a Hessian is cheap, all gradient-based methods finish in milliseconds.

Listing 8.11: Comparing optimisers in `scipy.optimize.minimize` ch08/ch08_bfgs.py

```
# DEP 803 -- Chapter 8: Optimisation
# Comparing scipy.optimize.minimize methods: iterations, evaluations, time
# (c) 2026 Stephen E. Moore. MIT Licence.
import time
import numpy as np
from scipy.optimize import minimize, rosen, rosen_der, rosen_hess

rng = np.random.default_rng(803)
n, p = 10000, 20
X = np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))])
beta = rng.normal(scale=0.3, size=p)
y = (rng.uniform(size=n) < 1 / (1 + np.exp(-X @ beta))).astype(float)


def f(b):
    z = X @ b
    return np.mean(np.logaddexp(0, z) - y * z)


```

```python
def g(b):
    return X.T @ (1 / (1 + np.exp(-X @ b)) - y) / n


def H(b):
    pr = 1 / (1 + np.exp(-X @ b))
    return (X * (pr * (1 - pr))[:, None]).T @ X / n


problems = {"Rosenbrock d=10": (rosen, rosen_der, rosen_hess, np.zeros(10)),
            "logit n=10000, p=20": (f, g, H, np.zeros(p))}
methods = ["Nelder-Mead", "CG", "BFGS", "L-BFGS-B", "Newton-CG", "trust-exact"]
for pname, (fun, jac, hess, x0) in problems.items():
    print(pname)
    print(f"  {'method':<12}{'nit':>6}{'nfev':>7}{'njev':>6}{'f*':>13}"
          f"{'ms':>8}")
    for m in methods:
        kw = {}
        if m != "Nelder-Mead":
            kw["jac"] = jac
        if m in ("Newton-CG", "trust-exact"):
            kw["hess"] = hess
        opts = {"maxiter": 3000, "maxfev": 3000} if m == "Nelder-Mead" \
            else {}
        t0 = time.perf_counter()
        res = minimize(fun, x0, method=m, options=opts, **kw)
        ms = 1000 * (time.perf_counter() - t0)
        nj = getattr(res, "njev", 0)
        print(f"  {m:<12}{res.nit:>6}{res.nfev:>7}{nj:>6}{res.fun:>13.6e}"
              f"{ms:>8.0f}")
```

Output

```
Rosenbrock d=10
  method         nit   nfev  njev           f*      ms
  Nelder-Mead   2175   3000     0 1.230938e+00     133
  CG             285    464   464 7.392342e-11      54
  BFGS            61     84    84 6.987591e-16      16
  L-BFGS-B        58     77    77 1.291175e-10      11
  Newton-CG       52     62    62 7.862469e-12      20
  trust-exact     26     27    25 1.562729e-18      16
logit n=10000, p=20
  method         nit   nfev  njev           f*      ms
  Nelder-Mead   2438   3000     0 6.250046e-01    2146
  CG               6     13    13 6.129771e-01      35
  BFGS            11     12    12 6.129771e-01      35
  L-BFGS-B         5      6     6 6.129771e-01      18
  Newton-CG        5      5     5 6.129771e-01      31
  trust-exact      3      4     4 6.129771e-01      33
```

△ Caution: Comparing optimisers fairly

Wall-clock times are machine- and load-dependent (rerun Listing 8.11 and they will change); iteration counts are not comparable across methods because an "iteration" of Newton-CG may involve many Hessian–vector products. Compare function, gradient and Hessian

evaluations, final gradient norm and achieved objective, and always check the `success` flag and `message` returned by `minimize`. A method that "converges" with a large gradient norm has merely stopped.

Historical Note: Variable-metric methods: Davidon, Fletcher, Powell and BFGS

Working at Argonne National Laboratory on long numerical minimisations, the physicist William Davidon devised a "variable metric" method that built up curvature information from gradients. His 1959 technical report (Davidon, 1959) circulated widely but appeared in a journal only in 1991, in the first issue of the *SIAM Journal on Optimization*. Roger Fletcher and Michael Powell analysed and popularised the method in 1963 (Fletcher and Powell, 1963) (the DFP method). In 1970 four authors independently proposed the update (8.23): Charles Broyden, Roger Fletcher, Donald Goldfarb and David Shanno (Broyden, 1970; Fletcher, 1970; Goldfarb, 1970; Shanno, 1970), hence BFGS. Dong Liu and Jorge Nocedal's limited-memory version followed in 1989 (Liu and Nocedal, 1989).

Policy Lens: Calibrating a model to match moments

Many policy models (microsimulation models of tax–benefit systems, dynamic models of schooling and labour supply, agent-based epidemic models) are *calibrated* or estimated by the method of simulated moments: choose $\boldsymbol{\theta}$ to minimise $Q(\boldsymbol{\theta}) = (\hat{\mathbf{m}} - \mathbf{m}_S(\boldsymbol{\theta}))^\top \boldsymbol{\Omega} (\hat{\mathbf{m}} - \mathbf{m}_S(\boldsymbol{\theta}))$, the distance between moments in the data and moments simulated from the model (McFadden, 1989). Because $\mathbf{m}_S$ is computed by simulation, Q can be noisy and non-smooth; holding the random draws fixed across evaluations (common random numbers, Chapter 10) is essential for gradient-based methods. A typical robust workflow is a derivative-free or multi-start global search, followed by BFGS or Gauss–Newton (treating $\hat{\mathbf{m}} - \mathbf{m}_S(\boldsymbol{\theta})$ as residuals) from the best points, with the Hessian at the optimum checked for positive definiteness before standard errors are reported.

⋆ Key Concept: Choosing a second-order method

Newton's method is affine invariant and converges quadratically near a strict minimiser, at a cost of $O(np^2 + p^3)$ per step: ideal for small and medium p (logit, probit, Poisson, most structural models). Damping or trust regions make it globally reliable. Gauss–Newton/Levenberg–Marquardt exploit least-squares structure. BFGS gives superlinear convergence from gradients only, and L-BFGS scales to large p. For well-conditioned convex problems of moderate size, "L-BFGS with analytic gradients" is an excellent default.

Exercises

8.5.1 Theory Prove that Newton's method is affine invariant: if $g(\mathbf{y}) = f(\mathbf{B}\mathbf{y})$ with $\mathbf{B}$ invertible and $\mathbf{x}_0 = \mathbf{B}\mathbf{y}_0$, then the Newton iterates satisfy $\mathbf{x}_k = \mathbf{B}\mathbf{y}_k$ for all k. Show that gradient descent is not affine invariant.

8.5.2 Theory Derive the IRLS form (8.22) of the Newton step. Derive the analogous working response and weights for Poisson regression with log link.

8.5.3 Theory Verify that the BFGS update (8.23) satisfies the secant condition $\mathbf{H}_{k+1}\mathbf{y}_k = \mathbf{s}_k$ and that $\mathbf{H}_{k+1} \succ 0$ whenever $\mathbf{H}_k \succ 0$ and $\mathbf{y}_k^\top \mathbf{s}_k > 0$.

8.5.4 Python Add a probit model to Listing 8.9: derive the gradient and Hessian, implement Newton's method, and compare the estimated marginal effect of distance at the mean of the covariates with the logit one.

8.5.5 Python Implement BFGS from (8.23) with Armijo backtracking (restart with $\mathbf{H} = \mathbf{I}$ if

$\mathbf{y}_k^\top \mathbf{s}_k \le 0$) and compare its iteration count with SciPy's on the Rosenbrock function. Then implement the L-BFGS two-loop recursion (Nocedal and Wright, 2006, Alg. 7.4) with $m = 5$.

8.5.6 Policy In Listing 8.10, re-estimate the diffusion curve using only the first 18 months of data. How precise are the estimates of the ceiling K (use $\hat\sigma^2(\mathbf{J}^\top\mathbf{J})^{-1}$ for the covariance)? What does this imply for forecasting the eventual reach of a new financial-inclusion programme early in its roll-out?

8.5.7 ★ Graduate Show that for a zero-residual NLS problem ($\mathbf{r}(\boldsymbol\theta^\star) = \mathbf{0}$, $\mathbf{J}$ of full rank) Gauss–Newton converges quadratically, while for a problem with large residuals it converges only linearly with a rate governed by $\big\|(\mathbf{J}^\top\mathbf{J})^{-1}\sum_i r_i \nabla^2 r_i\big\|$ at $\boldsymbol\theta^\star$.

8.6 Stochastic gradient descent

8.6.1 From gradient descent to stochastic approximation

An empirical risk $f(\boldsymbol\theta) = \frac{1}{n}\sum_{i=1}^n f_i(\boldsymbol\theta)$ with $f_i = \rho(z_i, \cdot)$ has gradient $\frac{1}{n}\sum_i \nabla f_i$, whose cost grows linearly in n. With $n = 10^7$ records, one gradient evaluation reads the whole register. Yet far from the optimum a rough direction is enough. *Stochastic gradient descent* (SGD) replaces the full gradient by an unbiased estimate from a random *minibatch* $\mathcal{B}_k \subset \{1,\dots,n\}$ of size B:

$$\boldsymbol\theta_{k+1} = \boldsymbol\theta_k - \alpha_k \mathbf{g}_k, \qquad \mathbf{g}_k = \frac{1}{B}\sum_{i\in\mathcal{B}_k} \nabla f_i(\boldsymbol\theta_k), \qquad \mathbb{E}[\mathbf{g}_k \mid \boldsymbol\theta_k] = \nabla f(\boldsymbol\theta_k). \tag{8.24}$$

Each update costs $O(Bp)$ instead of $O(np)$. One pass through the data is an *epoch*; in practice the data are reshuffled each epoch and partitioned into consecutive minibatches.

SGD is an instance of *stochastic approximation*. Robbins and Monro (1951) considered finding the root $\boldsymbol\theta^\star$ of a function $M(\boldsymbol\theta) = \mathbb{E}[H(\boldsymbol\theta, \xi)]$ that can only be observed with noise, through the recursion $\boldsymbol\theta_{k+1} = \boldsymbol\theta_k - \alpha_k H(\boldsymbol\theta_k, \xi_k)$. Applied to $H = \nabla\rho$, so that $M = \nabla R$ is the gradient of the *population* risk, it minimises R directly from a stream of fresh observations.

Theorem 8.37 (Robbins–Monro conditions; statement)**.** *Suppose f is μ-strongly convex and L-smooth, $\mathbb{E}[\mathbf{g}_k \mid \boldsymbol\theta_k] = \nabla f(\boldsymbol\theta_k)$, and $\mathbb{E}[\|\mathbf{g}_k - \nabla f(\boldsymbol\theta_k)\|^2 \mid \boldsymbol\theta_k] \le \sigma^2$. If the step sizes satisfy*

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \qquad \sum_{k=0}^{\infty} \alpha_k^2 < \infty, \tag{8.25}$$

then $\boldsymbol\theta_k \to \boldsymbol\theta^\star$ almost surely and in mean square. With $\alpha_k = \frac{c}{k+k_0}$, $c > 1/\mu$ and k_0 large enough that $\alpha_0 \le 1/L$, $\mathbb{E}[f(\boldsymbol\theta_k)] - f^\star = O(1/k)$.

The first condition lets the iterates travel any distance; the second makes the accumulated noise $\sum_k \alpha_k(\mathbf{g}_k - \nabla f)$ a convergent martingale. Proofs are in Bottou et al. (2018, Sec. 4). The $O(1/k)$ rate is much slower than GD's linear rate per *iteration*, but each iteration is n/B times cheaper.

8.6.2 Constant step sizes and the noise floor

With a constant step SGD does not converge: it converges *to a neighbourhood*.

Theorem 8.38 (SGD with constant step)**.** *Under the assumptions of Theorem 8.37 and a constant step $\alpha \le 1/L$,*

$$\mathbb{E}[f(\boldsymbol\theta_k)] - f^\star \le (1-\alpha\mu)^k\big(f(\boldsymbol\theta_0) - f^\star\big) + \frac{\alpha L \sigma^2}{2\mu}. \tag{8.26}$$

Proof. By the descent lemma, $f(\boldsymbol{\theta}_{k+1}) \le f(\boldsymbol{\theta}_k) - \alpha \nabla f(\boldsymbol{\theta}_k)^\top \mathbf{g}_k + \frac{L\alpha^2}{2}\|\mathbf{g}_k\|^2$. Take conditional expectations, using $\mathbb{E}\|\mathbf{g}_k\|^2 = \|\nabla f\|^2 + \mathbb{E}\|\mathbf{g}_k - \nabla f\|^2 \le \|\nabla f\|^2 + \sigma^2$ and $\alpha L \le 1$:

$$\mathbb{E}[f(\boldsymbol{\theta}_{k+1}) \mid \boldsymbol{\theta}_k] \le f(\boldsymbol{\theta}_k) - \tfrac{\alpha}{2}\|\nabla f(\boldsymbol{\theta}_k)\|^2 + \tfrac{L\alpha^2\sigma^2}{2} \le f(\boldsymbol{\theta}_k) - \alpha\mu\big(f(\boldsymbol{\theta}_k) - f^\star\big) + \tfrac{L\alpha^2\sigma^2}{2},$$

by (8.12). Subtract $f^\star$, take total expectations, and unroll the recursion $e_{k+1} \le (1-\alpha\mu)e_k + \frac{L\alpha^2\sigma^2}{2}$, using $\sum_{j}(1-\alpha\mu)^j \le \frac{1}{\alpha\mu}$. □

The bound has a *transient* term that decays linearly, as in GD, and a *noise floor* $\alpha L\sigma^2/(2\mu)$ proportional to the step size and to the gradient variance. Halving α halves the floor but slows the transient. A minibatch of size B divides σ^2 by B (sampling with replacement), and so lowers the floor at B times the cost per step. These trade-offs motivate *learning-rate schedules*: a large step early (fast transient) followed by smaller steps (lower floor). Common choices are step decay (divide by 10 every few epochs), $\alpha_k \propto 1/\sqrt{k}$ or $1/k$, and cosine schedules. A complementary device is *Polyak–Ruppert averaging*: report $\bar{\boldsymbol{\theta}}_k = \frac{1}{k}\sum_{j\le k}\boldsymbol{\theta}_j$, which attains the optimal asymptotic variance with slowly decaying steps (Polyak and Juditsky, 1992).

Listing 8.12 fits a logit by minibatch SGD to a *simulated* administrative register of $n = 200\,000$ individuals and $p = 10$ covariates, three epochs, under four schedules, measuring the excess loss relative to the exact optimum (computed by Newton's method).

Listing 8.12: Minibatch SGD and learning-rate schedules for a large logit ch08/ch08_sgd.py

```
# DEP 803 -- Chapter 8: Optimisation
# Minibatch SGD for a logit on a large simulated administrative register
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
n, p = 200_000, 10
X = np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))])
beta_true = np.concatenate([[-1.0], rng.normal(scale=0.5, size=p - 1)])
y = (rng.uniform(size=n) < 1 / (1 + np.exp(-X @ beta_true))).astype(float)


def loss(b):
    z = X @ b
    return np.mean(np.logaddexp(0, z) - y * z)


def minibatch_grad(b, idx):
    Xb = X[idx]
    return Xb.T @ (1 / (1 + np.exp(-Xb @ b)) - y[idx]) / idx.size


# reference optimum by Newton's method (10 iterations are plenty)
b_star = np.zeros(p)
for _ in range(10):
    pr = 1 / (1 + np.exp(-X @ b_star))
    H = (X * (pr * (1 - pr))[:, None]).T @ X / n
    b_star -= np.linalg.solve(H, X.T @ (pr - y) / n)
f_star = loss(b_star)


def sgd(schedule, batch=64, epochs=3, seed=1):
    r = np.random.default_rng(seed)
```

```
    b, k, curve = np.zeros(p), 0, []
    steps_per_epoch = n // batch
    for ep in range(epochs):
        perm = r.permutation(n)                  # reshuffle every epoch
        for s in range(steps_per_epoch):
            idx = perm[s * batch:(s + 1) * batch]
            b -= schedule(k) * minibatch_grad(b, idx)
            k += 1
            if k % 250 == 0:
                curve.append(loss(b) - f_star)
    return b, np.array(curve)


schedules = {
    "constant 0.5": lambda k: 0.5,
    "constant 0.05": lambda k: 0.05,
    "1/sqrt(k) decay": lambda k: 0.5 / np.sqrt(1 + k / 100),
    "step decay": lambda k: 0.5 * 0.1 ** (k // 3125),
}
print(f"n = {n:,}, batch 64, 3 epochs = {3 * n // 64:,} updates")
print(f"{'schedule':<17}{'excess loss':>13}{'||b - b*||':>12}")
plt.figure(figsize=(6, 3.8))
for name, sch in schedules.items():
    b, curve = sgd(sch)
    print(f"{name:<17}{loss(b) - f_star:>13.2e}"
          f"{np.linalg.norm(b - b_star):>12.4f}")
    plt.semilogy(250 * np.arange(1, curve.size + 1), np.abs(curve) + 1e-9,
                 label=name, lw=1)
for batch in [8, 512]:
    b, _ = sgd(schedules["constant 0.5"], batch=batch)
    print(f"constant 0.5, batch {batch:>3}: excess loss "
          f"{loss(b) - f_star:.2e}")
plt.gca().set(xlabel="SGD update k", ylabel="excess loss  f(b_k) - f*",
              title="Learning-rate schedules and the SGD noise floor")
plt.legend(frameon=False)
plt.savefig("ch08_sgd_schedules.pdf", bbox_inches="tight")
```

Output

```
n = 200,000, batch 64, 3 epochs = 9,375 updates
schedule          excess loss  ||b - b*||
constant 0.5          1.99e-03      0.1727
constant 0.05         4.61e-04      0.0815
1/sqrt(k) decay       4.74e-04      0.0827
step decay            1.74e-05      0.0146
constant 0.5, batch   8: excess loss 1.87e-02
constant 0.5, batch 512: excess loss 5.44e-04
```

The results line up with Theorem 8.38. With $\alpha = 0.5$ the excess loss hovers around 2×10^{-3}; reducing the step tenfold lowers the floor about fourfold; step decay, which cuts the step by 10 after each epoch, reaches 1.7×10^{-5}, a hundred times better than the constant step, in the same number of updates. Larger minibatches lower the floor too (batch 8: 1.9×10^{-2}; batch 512: 5.4×10^{-4}), but each update costs more.

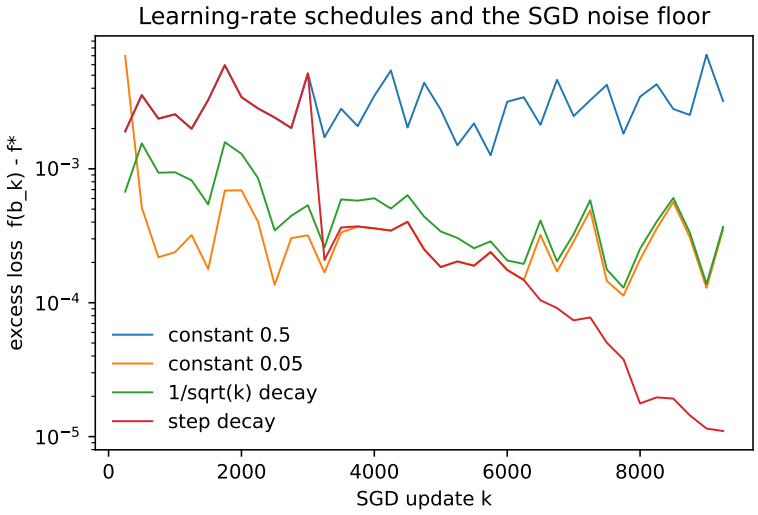


Figure 8.9: Excess loss of minibatch SGD under four learning-rate schedules. A constant large step stalls at a high noise floor; a constant small step has a lower floor; step decay combines a fast start with a low floor, as predicted by (8.26).

Historical Note: Robbins, Monro, Kiefer and Wolfowitz

Herbert Robbins and his student Sutton Monro published "A stochastic approximation method" in the *Annals of Mathematical Statistics* in 1951 (Robbins and Monro, 1951). Their motivating example was finding the dose level at which a response has a given probability, using only noisy observations at chosen doses. A year later Jack Kiefer and Jacob Wolfowitz adapted the idea to *maximising* a regression function using finite differences of noisy evaluations (Kiefer and Wolfowitz, 1952), a forerunner of today's zeroth-order and simulation-based optimisation. Stochastic approximation was developed further in control and signal processing (for example the least-mean-squares adaptive filter of Widrow and Hoff, 1960) and returned to centre stage when Bottou (2010) and others showed that SGD is the method of choice for large-scale learning.

8.6.3 Momentum and adaptive methods: AdaGrad, RMSProp, Adam

SGD inherits the conditioning problems of GD, and noise makes line searches unreliable. Four modifications dominate practice. All operations on vectors below are elementwise.

$$
\begin{aligned}
\text{momentum:}\quad & \mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \mathbf{g}_k, \quad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \mathbf{v}_{k+1};\\
\text{AdaGrad:}\quad & \mathbf{s}_{k+1} = \mathbf{s}_k + \mathbf{g}_k^2, \quad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\, \mathbf{g}_k / (\sqrt{\mathbf{s}_{k+1}} + \epsilon);\\
\text{RMSProp:}\quad & \mathbf{s}_{k+1} = \gamma \mathbf{s}_k + (1-\gamma)\mathbf{g}_k^2, \quad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\, \mathbf{g}_k / (\sqrt{\mathbf{s}_{k+1}} + \epsilon);\\
\text{Adam:}\quad & \mathbf{m}_{k+1} = \beta_1 \mathbf{m}_k + (1-\beta_1)\mathbf{g}_k, \quad \mathbf{v}_{k+1} = \beta_2 \mathbf{v}_k + (1-\beta_2)\mathbf{g}_k^2,\\
& \hat{\mathbf{m}} = \frac{\mathbf{m}_{k+1}}{1-\beta_1^{k+1}}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}_{k+1}}{1-\beta_2^{k+1}}, \quad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\, \hat{\mathbf{m}} / (\sqrt{\hat{\mathbf{v}}} + \epsilon).
\end{aligned}
$$

Momentum is the stochastic heavy ball (8.18); it averages noisy gradients over roughly $1/(1-\beta)$ steps, so its effective step is $\alpha/(1-\beta)$. AdaGrad (Duchi et al., 2011) gives each coordinate its own step, inversely proportional to the root of its accumulated squared gradients: coordinates with large gradients (steep or badly scaled directions) get small steps, and rare features get large ones. Its steps decay forever, which suits convex problems; RMSProp, proposed by Geoffrey Hinton in lecture notes, replaces the sum by an exponential moving average so that steps do not vanish. Adam (Kingma and Ba, 2015) combines an exponential average of gradients (momentum, $\beta_1 = 0.9$) with RMSProp scaling ($\beta_2 = 0.999$); the division by $1-\beta^{k+1}$ corrects the bias of averages

initialised at zero. These methods act as cheap *diagonal preconditioners*: they approximately undo bad scaling of individual coordinates, though not correlation between them.

Listing 8.13 implements all five methods in NumPy and applies them to a *simulated* logit whose covariates have standard deviations ranging from 0.2 to 20, so that κ is large. Plain SGD and momentum (whose effective step $0.0002/(1-0.9)$ equals SGD's) make almost no progress, because a step small enough to be stable in the steep coordinate is useless in the flat ones. The adaptive methods are insensitive to the scaling and reach excess losses 25 to 300 times smaller (Fig. 8.10).

Listing 8.13: Momentum, AdaGrad, RMSProp and Adam in NumPy ch08/ch08_adam.py

```
# DEP 803 -- Chapter 8: Optimisation
# Momentum, AdaGrad, RMSProp and Adam in NumPy; comparison with sklearn
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import SGDClassifier, LogisticRegression

rng = np.random.default_rng(803)
n, p = 50_000, 8
X = rng.normal(size=(n, p)) * np.array([1, 1, 1, 5, 5, 20, 0.2, 0.2])
X = np.column_stack([np.ones(n), X])                # badly scaled features
beta = np.array([-0.5, 0.8, -0.6, 0.4, 0.1, -0.1, 0.02, 3.0, -2.0])
y = (rng.uniform(size=n) < 1 / (1 + np.exp(-X @ beta))).astype(float)


def loss(b):
    z = X @ b
    return np.mean(np.logaddexp(0, z) - y * z)


def grad(b, idx):
    return X[idx].T @ (1 / (1 + np.exp(-X[idx] @ b)) - y[idx]) / idx.size


class Adam:
    """Kingma and Ba (2015) with bias correction."""
    def __init__(self, dim, lr=0.01, b1=0.9, b2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, b1, b2, eps
        self.m, self.v, self.t = np.zeros(dim), np.zeros(dim), 0

    def step(self, g):
        self.t += 1
        self.m = self.b1 * self.m + (1 - self.b1) * g
        self.v = self.b2 * self.v + (1 - self.b2) * g**2
        mhat = self.m / (1 - self.b1**self.t)
        vhat = self.v / (1 - self.b2**self.t)
        return -self.lr * mhat / (np.sqrt(vhat) + self.eps)


class Simple:
    """SGD, momentum, AdaGrad and RMSProp as one small class."""
    def __init__(self, dim, kind, lr):
        self.kind, self.lr = kind, lr
        self.v, self.s = np.zeros(dim), np.zeros(dim)

```

```
    def step(self, g):
        if self.kind == "SGD":
            return -self.lr * g
        if self.kind == "momentum":
            self.v = 0.9 * self.v - self.lr * g
            return self.v
        if self.kind == "AdaGrad":
            self.s += g**2
        else:                                          # RMSProp
            self.s = 0.9 * self.s + 0.1 * g**2
        return -self.lr * g / (np.sqrt(self.s) + 1e-8)


opts = {"SGD": lambda: Simple(p + 1, "SGD", 0.002),
        "momentum": lambda: Simple(p + 1, "momentum", 0.0002),
        "AdaGrad": lambda: Simple(p + 1, "AdaGrad", 0.1),
        "RMSProp": lambda: Simple(p + 1, "RMSProp", 0.005),
        "Adam": lambda: Adam(p + 1, lr=0.005)}
ref = LogisticRegression(penalty="none", fit_intercept=False, max_iter=5000,
                         tol=1e-10).fit(X, y)
f_star = loss(ref.coef_[0])
print(f"{'optimiser':<10}{'excess loss':>13}{'max |b - b*|':>14}")
plt.figure(figsize=(6, 3.8))
for name, make in opts.items():
    r = np.random.default_rng(7)
    opt, b, curve = make(), np.zeros(p + 1), []
    for ep in range(4):
        perm = r.permutation(n)
        for s in range(0, n, 100):
            b += opt.step(grad(b, perm[s:s + 100]))
            if (s // 100) % 25 == 0:
                curve.append(loss(b) - f_star)
    print(f"{name:<10}{loss(b) - f_star:>13.2e}"
          f"{np.abs(b - ref.coef_[0]).max():>14.4f}")
    plt.semilogy(np.abs(curve) + 1e-7, label=name, lw=1)
plt.gca().set(xlabel="minibatch update (x25), batch size 100",
              ylabel="excess training loss", title="Adaptive methods on a badly scaled
    ↪ logit")
plt.legend(frameon=False, ncol=2)
plt.savefig("ch08_adam.pdf", bbox_inches="tight")

sk = SGDClassifier(loss="log_loss", penalty=None, learning_rate="optimal",
                   fit_intercept=False, max_iter=20, tol=None,
                   random_state=0).fit(X, y)
print(f"sklearn SGDClassifier, raw X   : excess loss "
      f"{loss(sk.coef_[0]) - f_star:.2e}")
m, s = X[:, 1:].mean(0), X[:, 1:].std(0)            # standardise, refit
sk2 = SGDClassifier(loss="log_loss", penalty=None, max_iter=20, tol=None,
                    random_state=0).fit((X[:, 1:] - m) / s, y)
c = sk2.coef_[0] / s                                 # back to raw units
b_sk = np.concatenate([[sk2.intercept_[0] - c @ m], c])
print(f"sklearn SGDClassifier, scaled X: excess loss "
      f"{loss(b_sk) - f_star:.2e}")
```

Output

```
optimiser   excess loss  max |b - b*|
SGD            6.24e-02        2.9637
momentum       6.24e-02        2.9640
AdaGrad        3.61e-04        0.0364
RMSProp        2.62e-03        0.0493
Adam           2.03e-04        0.0213
sklearn SGDClassifier, raw X   : excess loss 6.47e-02
sklearn SGDClassifier, scaled X: excess loss 1.90e-03
```

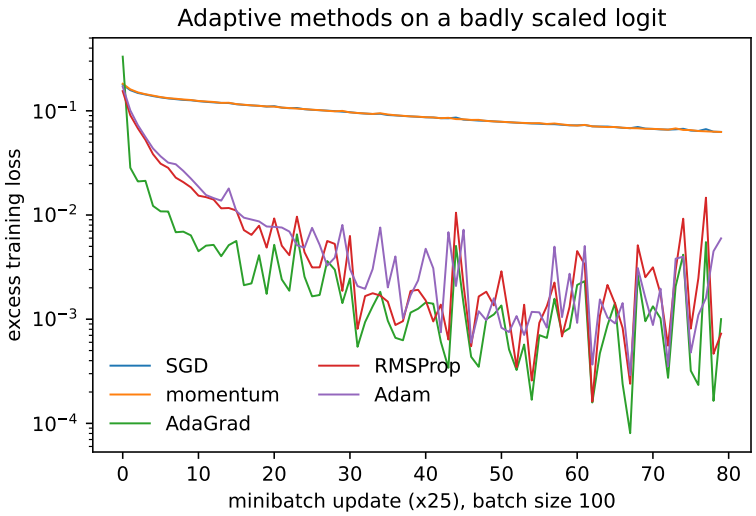


Figure 8.10: Training-loss curves for five stochastic optimisers on a badly scaled logit (excess loss over the exact optimum, log scale). SGD and momentum curves overlap; adaptive methods reach a far lower loss.

8.6.4 Practical diagnostics and software

Loss curves (training loss, and validation loss for prediction problems, against updates or epochs) are the main diagnostic for stochastic optimisation (Géron, 2022, Ch. 11):

- a loss that explodes or becomes `nan`: learning rate too large or unscaled features;
- a loss that falls and then fluctuates at a plateau: noise floor, so decay the learning rate or increase the batch size;
- a loss that falls very slowly and smoothly: learning rate too small or ill-conditioning, so rescale features or use an adaptive method;
- training loss falling while validation loss rises: over-fitting, a statistical rather than an optimisation problem (Chapter 11).

A useful habit is the *learning-rate range test*: run a few hundred updates for learning rates on a log grid and pick one somewhat below the value at which the loss starts to diverge.

Scikit-learn's `SGDClassifier` and `SGDRegressor` implement SGD for linear models (with `loss="log_loss"` for logistic regression, `"hinge"` for linear SVMs, `"squared_error"` for least squares), with optional ℓ_1 / ℓ_2 penalties, averaging and several schedules. The last lines of Listing 8.13 show why the documentation insists on standardising features first: on the raw data `SGDClassifier` stalls like our plain SGD, while after `StandardScaler`-type scaling it is thirty times closer to the

optimum. Neural networks are trained by exactly these algorithms, with gradients computed by back-propagation (Rumelhart et al., 1986); Keras and PyTorch offer SGD with (Nesterov) momentum, RMSProp and Adam, and Géron (2022, Ch. 11) gives practical guidance on choosing among them.

△ Caution: Pitfalls of stochastic optimisation

(i) Not shuffling: if records are sorted (by region, by date), consecutive minibatches are unrepresentative and SGD drifts. (ii) Judging convergence from a single noisy minibatch loss: use a moving average or the full-data loss on a schedule. (iii) Treating Adam as tuning-free: its learning rate still matters, and there are simple convex problems on which Adam with constant hyperparameters does not converge (Reddi et al., 2018). (iv) Using SGD estimates for inference: a stopped SGD run is not the exact M-estimator, and its sampling variability is not the one reported by MLE standard-error formulas unless the optimisation error is negligible.

Policy Lens: Large administrative datasets

Tax records, social-protection registries, mobile-phone metadata, health insurance claims and satellite-image pixels can have tens of millions of rows. For a logit or a linear model with a few hundred covariates, Newton's method needs $O(np^2)$ operations per iteration and the data may not fit in memory; minibatch SGD reads the data in chunks and makes useful progress after a fraction of an epoch. A typical design for, say, targeting a cash-transfer programme with a proxy-means test estimated on a national register: standardise covariates, run a few epochs of SGD or Adam with a decaying schedule to get close, then (if inference is needed) polish with a few Newton or L-BFGS iterations on the full data or a large subsample, and report the full-data gradient norm. Out-of-core and distributed variants follow the same logic.

Graduate Perspective: Variance reduction and non-convex landscapes

For finite sums, the noise floor can be removed without decaying steps. SVRG (Johnson and Zhang, 2013) periodically computes a full gradient $\nabla f(\tilde{\boldsymbol{\theta}})$ and uses $\nabla f_i(\boldsymbol{\theta}_k) - \nabla f_i(\tilde{\boldsymbol{\theta}}) + \nabla f(\tilde{\boldsymbol{\theta}})$, an unbiased estimate whose variance vanishes as $\boldsymbol{\theta}_k, \tilde{\boldsymbol{\theta}} \to \boldsymbol{\theta}^\star$; for strongly convex problems it converges linearly at a per-epoch cost comparable to SGD. For non-convex objectives such as neural network losses, the theory changes: guarantees are for small gradient norm, saddle points (not local minima) are the main obstacle in high dimension, and gradient noise helps escape them (Lee et al., 2016). Why SGD-trained over-parameterised networks generalise well ("implicit regularisation") is an active research area (see Murphy, 2022).

⋆ Key Concept: Stochastic gradients

SGD trades exact gradients for cheap unbiased ones. With constant step α it converges linearly to a noise floor of order $\alpha\sigma^2/B$; Robbins–Monro step sizes ($\sum \alpha_k = \infty$, $\sum \alpha_k^2 < \infty$) or step-decay schedules drive the error to zero; minibatches and averaging reduce variance. Adaptive methods (AdaGrad, RMSProp, Adam) rescale each coordinate and are robust to bad scaling, but standardising features remains the first step.

Exercises

8.6.1 Theory Show that the minibatch gradient in (8.24) with indices drawn uniformly with replacement is unbiased and that its covariance is $\frac{1}{B}$ times that of a single sampled gradient.

What changes when sampling is without replacement?

8.6.2 Theory Which of the schedules $\alpha_k = c$, $c/\sqrt{k+1}$, $c/(k+1)$ and $c/(k+1)^2$ satisfy the Robbins–Monro conditions (8.25)? For the one that fails the first condition, show that SGD can stop short of the optimum even without noise.

8.6.3 Theory Show that Adam's bias correction is exact for a constant gradient: if $\mathbf{g}_k \equiv \mathbf{g}$, then $\hat{\mathbf{m}} = \mathbf{g}$ and $\hat{\mathbf{v}} = \mathbf{g}^2$ for every k, so the first Adam step has size α in every coordinate, whatever the scale of $\mathbf{g}$.

8.6.4 Python In Listing 8.12, add Polyak–Ruppert averaging of the iterates over the last epoch with constant step 0.5. How does the excess loss of the average compare with that of the final iterate? Plot the excess loss against batch size $B \in \{8, 16, \dots, 1024\}$ at fixed number of *epochs*, and at fixed number of *updates*.

8.6.5 Python Run a learning-rate range test for Adam on the problem of Listing 8.13 (α from 10^{-5} to 1) and plot the loss after 200 updates. Then standardise the covariates and repeat for plain SGD. Discuss.

8.6.6 Policy A social-protection agency wants to update a proxy-means-test logit monthly on a register of 15 million households. Write a one-page technical note recommending an estimation strategy (algorithm, scaling, schedule, stopping rule, how to obtain standard errors), justifying each choice by results in this chapter.

8.6.7 ★ Graduate Implement SVRG for the logit of Listing 8.12 and show empirically that it converges linearly with a constant step. Explain, using Theorem 8.38, why the variance-reduced estimate removes the noise floor.

Chapter Summary

- Estimators and learning methods are minimisers of empirical risks; the loss determines the statistical target, the algorithm only computes it. Minimisers exist for continuous coercive objectives (Weierstrass); logit MLE can fail to exist under separation.
- Optimality conditions: $\nabla f = \mathbf{0}$ is necessary; $\nabla^2 f \succeq 0$ is necessary and $\nabla^2 f \succ 0$ (with $\nabla f = \mathbf{0}$) sufficient for a local minimum; mixed-sign Hessian eigenvalues indicate a saddle. Check analytic gradients by central differences with $h \approx 10^{-5}$–10^{-6}.
- Convexity makes every local minimum global. f is convex iff it lies above its tangents, iff $\nabla^2 f \succeq 0$. Strong convexity μ and smoothness L define the condition number $\kappa = L/\mu$. The logit log-likelihood is concave; Cobb–Douglas utility is always quasi-concave but concave only with non-increasing returns.
- Gradient descent converges at rate $O(1/k)$ for convex L-smooth f and linearly, at rate $1 - 1/\kappa$, under strong convexity; ill-conditioning causes zig-zagging. Line searches choose steps automatically; momentum and Nesterov acceleration reduce the dependence on κ to $\sqrt{\kappa}$; feature scaling is a cheap preconditioner.
- Newton's method is affine invariant and locally quadratically convergent; damping makes it robust. IRLS is Newton for GLMs; Gauss–Newton and Levenberg–Marquardt solve NLS; BFGS and L-BFGS give superlinear convergence from gradients alone.
- SGD uses cheap unbiased gradient estimates; constant steps reach a noise floor proportional to $\alpha\sigma^2$; Robbins–Monro schedules, minibatches and averaging control the noise; Adam and its relatives adapt per-coordinate steps. Loss curves are the main diagnostic.

Further Reading

Nocedal and Wright (2006) is the standard graduate text on numerical optimisation: line searches and trust regions (Ch. 3–4), quasi-Newton and L-BFGS (Ch. 6–7) and nonlinear least squares (Ch. 10). Boyd and Vandenberghe (2004) develops convex analysis and its applications; Chapters 2–3 and 9 match Sections 8.3 to 8.5. Nesterov (2018) proves the complexity bounds and accelerated methods of Section 8.4. For stochastic and large-scale methods, the survey by Bottou et al. (2018) is the best single reference. On the course reading list, Murphy (2022, Ch. 8) covers all six sections from a machine-learning perspective; Géron (2022, Ch. 4 and 11) gives a hands-on treatment of gradient descent variants, learning schedules and optimisers for neural networks; Matsuura (2022) discusses the optimisation and gradient-based sampling behind Stan's estimates; and Goulet Coulombe et al. (2022) illustrates why regularised, nonlinear (and hence optimisation-heavy) models are valuable for macroeconomic forecasting. For econometric applications of M-estimation and numerical optimisation see Cameron and Trivedi (2005, Ch. 5 and 10), Wooldridge (2010, Ch. 12) and Hansen (2022); for the mathematical economics of concavity and quasi-concavity, Simon and Blume (1994, Ch. 21). Chapter 9 continues with constrained optimisation, Lagrange multipliers and the KKT conditions.

Chapter 9

Constrained Optimisation and Lagrange Multipliers in Policy Problems

Learning Objectives

After studying this chapter you should be able to:

1. state and prove (in sketch) the Lagrange multiplier theorem, explain the role of the constraint qualification and interpret the tangency condition geometrically;
2. derive Marshallian demand for Cobb–Douglas and CES preferences by hand and reproduce the results with `scipy.optimize`;
3. interpret multipliers as shadow prices, prove the envelope theorem and use the bordered Hessian to check second-order conditions;
4. write down and solve Karush–Kuhn–Tucker (KKT) conditions, including corner solutions, and know when LICQ or Slater's condition guarantees them;
5. form the Lagrange dual of a problem, prove weak duality, solve linear programmes with `linprog` and read dual values as prices;
6. recognise restricted least squares, ridge, lasso and the support vector machine as constrained optimisation problems, and implement projected gradient, penalty and barrier methods;
7. formulate and solve policy problems – budget allocation, equity-weighted transfers, poverty-gap targeting, Ramsey taxation – and analyse the sensitivity of their solutions.

Almost every interesting policy question is a *constrained* optimisation problem. A District Assembly wants the largest possible improvement in education, health and agricultural outcomes, but its Common Fund allocation is fixed. A social protection agency wants to reduce poverty as much as possible, but it has a fixed budget and cannot observe household incomes perfectly. A nutrition programme wants the cheapest food basket that still meets requirements for energy, protein and micronutrients. A Ministry of Finance must raise a given amount of revenue while doing as little damage as possible to household welfare, and it must do so within a fiscal rule that caps the deficit. In each case the decision maker chooses $\mathbf{x}$ to maximise or minimise an objective $f(\mathbf{x})$, but only among the $\mathbf{x}$ that satisfy constraints $h_j(\mathbf{x}) = c_j$ or $g_k(\mathbf{x}) \le 0$.

Chapter 8 treated unconstrained optimisation: set the gradient to zero and check curvature. Constraints change the picture in two fundamental ways. First, at a constrained optimum the gradient of the objective is generally *not* zero; it is balanced by the gradients of the binding constraints. Second, the balancing weights – the *Lagrange multipliers* – are not mere algebraic devices. They measure how much the optimal value would improve if a constraint were relaxed

slightly. In economics they are *shadow prices*: the marginal utility of income, the marginal social value of an extra cedi of budget, the cost of an extra gram of protein in a least-cost diet, the marginal cost of public funds. For a policy analyst, the multiplier is often more informative than the solution itself, because it tells us what a constraint is costing us.

Constrained optimisation is equally central to data science. Restricted least squares imposes economic theory on a regression; ridge regression and the lasso are least squares with a constraint on the size of the coefficients; the support vector machine is a quadratic programme whose dual reveals the support vectors; and many machine learning algorithms handle constraints by projection, penalties or barriers. Duality theory, which turns a constrained problem into a problem about prices, underpins linear programming, convex optimisation and modern interior-point solvers.

The chapter is organised as follows. Section 9.1 develops the method of Lagrange multipliers for equality constraints and applies it to consumer demand. Section 9.2 interprets multipliers through the envelope theorem and studies second-order conditions. Section 9.3 extends the theory to inequality constraints via the KKT conditions. Section 9.4 develops duality and linear programming, culminating in a least-cost nutritious diet. Section 9.5 turns to constrained estimation in statistics and machine learning, and Section 9.6 brings everything together in policy optimisation problems. Throughout, all numbers are produced by simulated data or by datasets bundled with scikit-learn; none of the simulated figures should be read as statistics of Ghana or of any other country.

9.1 Equality Constraints and the Lagrangian

9.1.1 The problem and its geometry

We consider the equality-constrained problem

$$\max_{\mathbf{x}\in\mathbb{R}^n} \; f(\mathbf{x}) \qquad \text{subject to} \qquad h_j(\mathbf{x}) = c_j, \quad j = 1, \dots, m, \tag{9.1}$$

with $m < n$ and $f, h_1, \dots, h_m$ continuously differentiable. Write $\mathbf{h}(\mathbf{x}) = (h_1(\mathbf{x}), \dots, h_m(\mathbf{x}))^\top$ and $\mathbf{c} = (c_1, \dots, c_m)^\top$. The *feasible set* $S = \{\mathbf{x} : \mathbf{h}(\mathbf{x}) = \mathbf{c}\}$ is, under mild conditions, a smooth $(n-m)$-dimensional surface in $\mathbb{R}^n$. Minimisation problems are handled by maximising $-f$.

> **Definition 9.1** (Local constrained maximum)**.** A point $\mathbf{x}^* \in S$ is a *local constrained maximum* of f on S if there is $\varepsilon > 0$ such that $f(\mathbf{x}^*) \geq f(\mathbf{x})$ for all $\mathbf{x} \in S$ with $\|\mathbf{x} - \mathbf{x}^*\| < \varepsilon$.

The geometry is easiest to see with $n = 2$ and a single constraint. Picture the level curves $\{f = k\}$ of the objective (indifference curves, iso-outcome curves) and the constraint curve $\{h = c\}$ (a budget line). Moving along the constraint curve, the value of f changes at rate $\nabla f(\mathbf{x})^\top \mathbf{v}$, where $\mathbf{v}$ is a tangent vector of the constraint curve. At a point where this directional derivative is non-zero we can move along the constraint in one direction and increase f, so the point is not optimal. At an optimum, therefore, $\nabla f(\mathbf{x}^*)$ must be orthogonal to every tangent direction of the constraint. Since $\nabla h(\mathbf{x}^*)$ is also orthogonal to the tangent directions (it is normal to the level set of h), the two gradients must be *parallel*:

$$\nabla f(\mathbf{x}^*) = \lambda \, \nabla h(\mathbf{x}^*) \quad \text{for some scalar } \lambda.$$

Equivalently, the level curve of f through $\mathbf{x}^*$ is *tangent* to the constraint curve. In consumer theory this is the familiar statement that the indifference curve is tangent to the budget line: the marginal rate of substitution equals the price ratio.

⋆ Key Concept: Tangency

At a regular constrained optimum, the gradient of the objective lies in the span of the gradients of the constraints: $\nabla f(\mathbf{x}^*) = \sum_j \lambda_j \nabla h_j(\mathbf{x}^*)$. Geometrically, no feasible direction of motion changes f to first order, so the level set of f is tangent to the feasible surface. The multipliers λ_j are the weights that make the balance exact.

9.1.2 The Lagrange multiplier theorem

Define the *Lagrangian*

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \sum_{j=1}^{m} \lambda_j \big(h_j(\mathbf{x}) - c_j\big) = f(\mathbf{x}) - \boldsymbol{\lambda}^{\top}\big(\mathbf{h}(\mathbf{x}) - \mathbf{c}\big). \tag{9.2}$$

Setting $\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{0}$ and $\nabla_{\boldsymbol{\lambda}}\mathcal{L} = \mathbf{0}$ gives $n + m$ equations in $n + m$ unknowns:

$$\nabla f(\mathbf{x}) = \sum_{j=1}^{m} \lambda_j \nabla h_j(\mathbf{x}), \qquad \mathbf{h}(\mathbf{x}) = \mathbf{c}. \tag{9.3}$$

Theorem 9.2 (Lagrange multiplier theorem)**.** *Let $f, h_1, \dots, h_m : \mathbb{R}^n \to \mathbb{R}$ be C^1 and let $\mathbf{x}^*$ be a local constrained maximum (or minimum) of f on $S = \{\mathbf{x} : \mathbf{h}(\mathbf{x}) = \mathbf{c}\}$. Suppose the* constraint qualification *holds at $\mathbf{x}^*$: the $m \times n$ Jacobian $D\mathbf{h}(\mathbf{x}^*)$ has full row rank m, that is, $\nabla h_1(\mathbf{x}^*), \dots, \nabla h_m(\mathbf{x}^*)$ are linearly independent. Then there exists a unique $\boldsymbol{\lambda}^* \in \mathbb{R}^m$ such that $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ satisfies (9.3).*

Proof sketch via the implicit function theorem. Because $D\mathbf{h}(\mathbf{x}^*)$ has rank m we can reorder the variables and write $\mathbf{x} = (\mathbf{u}, \mathbf{w})$ with $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{w} \in \mathbb{R}^{n-m}$, such that the $m \times m$ block $D_{\mathbf{u}}\mathbf{h}(\mathbf{x}^*)$ is invertible. By the implicit function theorem there is a C^1 function $\mathbf{u} = \phi(\mathbf{w})$, defined near $\mathbf{w}^*$, with $\mathbf{h}(\phi(\mathbf{w}), \mathbf{w}) = \mathbf{c}$ and

$$D\phi(\mathbf{w}^*) = -\big[D_{\mathbf{u}}\mathbf{h}\big]^{-1} D_{\mathbf{w}}\mathbf{h},$$

all derivatives evaluated at $\mathbf{x}^*$. Near $\mathbf{x}^*$ the feasible set is exactly the graph of ϕ, so $\mathbf{w}^*$ is a local *unconstrained* optimum of $F(\mathbf{w}) = f(\phi(\mathbf{w}), \mathbf{w})$. Hence

$$\mathbf{0}^{\top} = DF(\mathbf{w}^*) = D_{\mathbf{u}} f \, D\phi + D_{\mathbf{w}} f = -D_{\mathbf{u}} f \, [D_{\mathbf{u}}\mathbf{h}]^{-1} D_{\mathbf{w}}\mathbf{h} + D_{\mathbf{w}} f.$$

Define the row vector $\boldsymbol{\lambda}^{*\top} = D_{\mathbf{u}} f \, [D_{\mathbf{u}}\mathbf{h}]^{-1}$. Then $D_{\mathbf{u}} f = \boldsymbol{\lambda}^{*\top} D_{\mathbf{u}}\mathbf{h}$ by construction, and the display above says $D_{\mathbf{w}} f = \boldsymbol{\lambda}^{*\top} D_{\mathbf{w}}\mathbf{h}$. Stacking the two blocks gives $\nabla f(\mathbf{x}^*)^{\top} = \boldsymbol{\lambda}^{*\top} D\mathbf{h}(\mathbf{x}^*)$, which is (9.3). Uniqueness follows because the rows of $D\mathbf{h}(\mathbf{x}^*)$ are linearly independent. □

The proof shows what the constraint qualification buys: it lets us solve the constraints locally for m of the variables, reducing the problem to an unconstrained one in $n - m$ variables. When it fails, the feasible set can have a cusp or a degenerate point, and the theorem can fail.

△ Caution: The constraint qualification is not a technicality

Minimise $f(x_1, x_2) = x_1$ subject to $h(x_1, x_2) = x_2^2 - x_1^3 = 0$. The feasible set is the cusp $x_1 = |x_2|^{2/3} \geq 0$, so the minimum is at the origin. But $\nabla f(\mathbf{0}) = (1, 0)^{\top}$ while $\nabla h(\mathbf{0}) = (0, 0)^{\top}$; no λ satisfies $\nabla f = \lambda \nabla h$. Solving the Lagrange equations blindly would miss the true optimum. Always check that the constraint gradients are linearly independent at the candidate points, and examine separately any feasible points where they are not.

The theorem gives *necessary* conditions. Solutions of (9.3) are *critical points* of the Lagrangian; they may be maxima, minima or saddle points on S. Sufficient conditions (Section 9.2) use second

derivatives, or global arguments: if f is concave and the constraints are linear, every critical point is a global maximum. A further practical route is the *Weierstrass theorem*: a continuous function on a compact set attains its maximum, so if S is compact and the constraint qualification holds everywhere on S, the global maximum is the best of the critical points.

Historical Note: Lagrange and the multiplier

Joseph-Louis Lagrange (1736–1813) introduced multipliers in his *Mécanique analytique* (1788), where they represent the forces needed to keep a mechanical system on its constraint surface – the first "shadow prices" were physical reaction forces. In the *Théorie des fonctions analytiques* (1797) he applied the same device to the general problem of finding maxima and minima of a function whose variables are linked by equations. The rigorous treatment based on the implicit function theorem came much later, with the development of analysis in the nineteenth and twentieth centuries.

9.1.3 Consumer utility maximisation

The canonical economic application is the consumer problem

$$\max_{\mathbf{x}\geq \mathbf{0}} \; u(\mathbf{x}) \quad \text{subject to} \quad \mathbf{p}^{\top}\mathbf{x} = m, \tag{9.4}$$

where $\mathbf{p} \gg \mathbf{0}$ is the price vector and $m > 0$ is income. We postpone the non-negativity constraints to Section 9.3 and look for interior solutions. The solution $\mathbf{x}(\mathbf{p}, m)$ is the *Marshallian (uncompensated) demand*; the optimal value $v(\mathbf{p}, m) = u(\mathbf{x}(\mathbf{p}, m))$ is the *indirect utility function*.

Example 9.3 (Cobb–Douglas demand)**.** Let $u(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i \log x_i$ with $\alpha_i > 0$, $\sum_i \alpha_i = 1$ (a monotone transformation of $\prod_i x_i^{\alpha_i}$, which leaves the solution unchanged). The Lagrangian is $\mathcal{L} = \sum_i \alpha_i \log x_i - \lambda(\mathbf{p}^{\top}\mathbf{x} - m)$ and the first-order conditions are

$$\frac{\alpha_i}{x_i} = \lambda p_i, \quad i = 1, \dots, n, \qquad \mathbf{p}^{\top}\mathbf{x} = m.$$

Hence $p_i x_i = \alpha_i / \lambda$. Summing over i gives $m = 1/\lambda$, so

$$x_i(\mathbf{p}, m) = \frac{\alpha_i m}{p_i}, \qquad \lambda = \frac{1}{m}.$$

Each good receives the constant budget share α_i, and the multiplier is the marginal (log-)utility of income. The constraint qualification holds because $\nabla(\mathbf{p}^{\top}\mathbf{x}) = \mathbf{p} \neq \mathbf{0}$.

Example 9.4 (CES demand)**.** Let $u(\mathbf{x}) = \left(\sum_i a_i x_i^{\rho}\right)^{1/\rho}$ with $a_i > 0$, $\rho < 1$, $\rho \neq 0$, and elasticity of substitution $\sigma = 1/(1-\rho)$. Maximising the monotone transform $\sum_i a_i x_i^{\rho}$ instead, the first-order conditions give $a_i \rho x_i^{\rho-1} = \lambda p_i$, so for any two goods

$$\frac{x_i}{x_k} = \left(\frac{a_i\, p_k}{a_k\, p_i}\right)^{\sigma}.$$

Writing $x_i = x_1 (a_i/a_1)^{\sigma} (p_1/p_i)^{\sigma}$ and substituting into the budget constraint yields

$$x_i(\mathbf{p}, m) = \frac{a_i^{\sigma} p_i^{-\sigma}}{\sum_k a_k^{\sigma} p_k^{1-\sigma}}\, m. \tag{9.5}$$

As $\sigma \to 1$ we recover Cobb–Douglas shares; as $\sigma \to \infty$ goods become perfect substitutes and demand concentrates on the good with the highest a_i/p_i; as $\sigma \to 0$ we approach Leontief (fixed proportions). The own-price elasticity is $\partial \log x_i / \partial \log p_i = -\sigma + (\sigma - 1) s_i$, where $s_i = p_i x_i / m$ is the budget share.

Two properties of Marshallian demand follow immediately from the structure of (9.4): *homogeneity of degree zero*, $\mathbf{x}(t\mathbf{p}, tm) = \mathbf{x}(\mathbf{p}, m)$ for $t > 0$ (the constraint set does not change when all prices and income are scaled), and *Walras' law*, $\mathbf{p}^{\top}\mathbf{x}(\mathbf{p}, m) = m$ when preferences are locally non-satiated.

9.1.4 Solving numerically in Python

Listing 9.1 verifies the tangency condition for a two-good Cobb–Douglas consumer and draws Fig. 9.1. At the optimum the two gradients are exactly proportional, with ratio λ; at another point on the budget line the angle between them is about 15° and the directional derivative of u along the budget line is non-zero, so moving along the line raises utility.

Listing 9.1: Tangency of level set and constraint (plotting code omitted)
ch09/ch09_lagrange_geometry.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Geometry of the Lagrange condition: tangency of level set and budget line.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

a, p1, p2, m = 0.4, 2.0, 5.0, 100.0          # Cobb-Douglas share, prices, income


def u(x1, x2):
    return x1**a * x2**(1 - a)


def grad_u(x1, x2):
    return np.array([a * u(x1, x2) / x1, (1 - a) * u(x1, x2) / x2])


grad_g = np.array([p1, p2])                  # gradient of g(x) = p1 x1 + p2 x2
xs = np.array([a * m / p1, (1 - a) * m / p2])  # Marshallian demand
lam = grad_u(*xs)[0] / p1

print("Optimum x* = (%.2f, %.2f),  u(x*) = %.4f" % (xs[0], xs[1], u(*xs)))
print("grad u(x*) =", np.round(grad_u(*xs), 4), "  grad g =", grad_g)
print("ratio grad u / grad g (componentwise):", np.round(grad_u(*xs) / grad_g, 4))
print("Lagrange multiplier lambda = %.4f" % lam)

xb = np.array([35.0, (m - p1 * 35.0) / p2])  # another point on the budget line
gu = grad_u(*xb)
cosang = gu @ grad_g / (np.linalg.norm(gu) * np.linalg.norm(grad_g))
t = np.array([p2, -p1]) / np.hypot(p1, p2)   # tangent direction of budget line
print("\nAt xb = (%.1f, %.1f): u = %.4f" % (xb[0], xb[1], u(*xb)))
print("angle between grad u and grad g = %.2f degrees" %
      np.degrees(np.arccos(cosang)))
print("directional derivative of u along budget line = %.4f" % (gu @ t))
```

Output

```
Optimum x* = (20.00, 12.00),  u(x*) = 14.7204
grad u(x*) = [0.2944 0.736 ]   grad g = [2. 5.]
ratio grad u / grad g (componentwise): [0.1472 0.1472]
Lagrange multiplier lambda = 0.1472

At xb = (35.0, 6.0): u = 12.1484
angle between grad u and grad g = 15.28 degrees
```

```
directional derivative of u along budget line = -0.3223
```

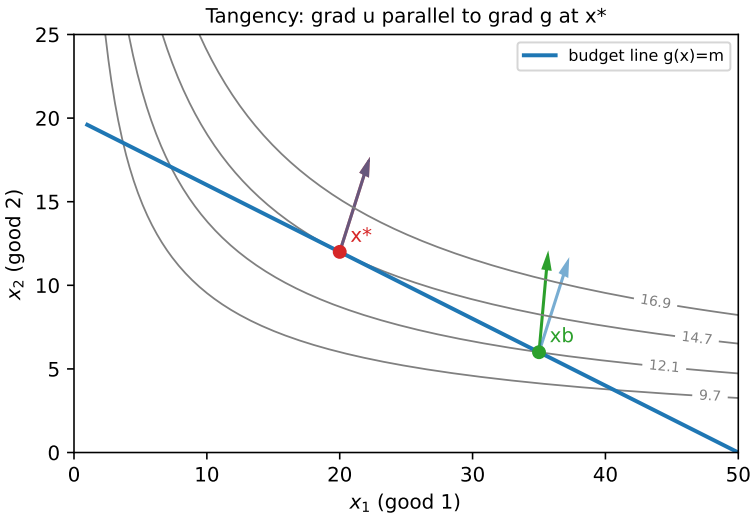


Figure 9.1: Indifference curves of $u = x_1^{0.4} x_2^{0.6}$ and the budget line $2x_1 + 5x_2 = 100$. At $\mathbf{x}^*$ the utility gradient (red) is parallel to the constraint gradient (blue); at $\mathbf{x}_b$ the two gradients point in different directions and the indifference curve cuts the budget line.

There are two numerical strategies. The first solves the *optimisation problem* directly with a constrained solver such as SLSQP (sequential least squares programming), available through `scipy.optimize.minimize`. The second solves the *system of first-order conditions* (9.3) with a root finder such as `scipy.optimize.root`, which also returns the multiplier. Listing 9.2 does both for a three-good Cobb–Douglas consumer and compares the CES closed form (9.5) with SLSQP for several elasticities of substitution.

Listing 9.2: Marshallian demand: closed form and numerics ch09/ch09_consumer_demand.py

```python
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Marshallian demand (Cobb-Douglas, CES): closed form vs scipy.optimize.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy.optimize import minimize, root

p = np.array([2.0, 5.0, 4.0])                # prices (GHS per unit)
m = 120.0                                    # household budget (GHS)
alpha = np.array([0.3, 0.5, 0.2])            # Cobb-Douglas shares (sum to 1)

# --- 1. Cobb-Douglas: closed form x_i = alpha_i m / p_i ----------------
x_cd = alpha * m / p
lam_cd = 1 / m                               # for u = sum alpha_i log x_i

# --- 2. Direct numerical maximisation with SLSQP -----------------------
negu = lambda x: -np.sum(alpha * np.log(x))
budget = {"type": "eq", "fun": lambda x: m - p @ x, "jac": lambda x: -p}
res = minimize(negu, x0=np.full(3, 5.0), method="SLSQP", constraints=[budget],
               bounds=[(1e-6, None)] * 3, tol=1e-12)


# --- 3. Solve the Lagrange first-order system F(x, lambda) = 0 ---------
def foc(z):
    x, lam = z[:3], z[3]
```

```
    return np.r_[alpha / x - lam * p, m - p @ x]


sol = root(foc, x0=np.r_[np.full(3, 5.0), 0.1])
print("Cobb-Douglas, p =", p, " m =", m)
print("  closed form  x =", np.round(x_cd, 6), " lambda =", round(lam_cd, 6))
print("  SLSQP        x =", np.round(res.x, 6))
print("  root(FOC)    x =", np.round(sol.x[:3], 6), " lambda =",
      round(sol.x[3], 6))
print("  budget shares p_i x_i / m =", np.round(p * x_cd / m, 3))

# --- 4. CES utility u = (sum a_i x_i^r)^(1/r), sigma = 1/(1-r) ---------
a = np.array([0.5, 0.3, 0.2])


def ces_demand(p, m, sigma):
    w = a**sigma * p**(1 - sigma)
    return m * a**sigma * p**(-sigma) / w.sum()


print("\nCES demand (closed form vs SLSQP):")
print("  sigma    x1(cf)   x1(num)   x2(cf)   x3(cf)   max|diff|")
for sigma in [0.5, 0.8, 2.0, 5.0]:
    r = 1 - 1 / sigma
    negces = lambda x: -np.log(np.sum(a * x**r)) / r
    out = minimize(negces, x0=np.full(3, 5.0), method="SLSQP",
                   constraints=[budget], bounds=[(1e-8, None)] * 3, tol=1e-12)
    xc = ces_demand(p, m, sigma)
    print("  %5.2f  %8.4f  %8.4f  %8.4f  %8.4f   %.1e" %
          (sigma, xc[0], out.x[0], xc[1], xc[2], np.max(np.abs(xc - out.x))))

# own-price elasticity of good 1 demand (numerical, CES sigma = 2)
h = 1e-5
dp = np.array([h, 0, 0])
e11 = (np.log(ces_demand(p + dp, m, 2.0)[0]) -
       np.log(ces_demand(p - dp, m, 2.0)[0])) / (np.log(p[0] + h) -
                                                 np.log(p[0] - h))
s1 = (a**2 * p**(-1))[0] / np.sum(a**2 * p**(-1))   # expenditure share
print("\nCES sigma=2: own-price elasticity of good 1 = %.4f" % e11)
print("theory -sigma + (sigma-1) s1                = %.4f" % (-2 + s1))
```

Output

```
Cobb-Douglas, p = [2. 5. 4.]  m = 120.0
  closed form  x = [18. 12.  6.]  lambda = 0.008333
  SLSQP        x = [18.000012 11.999992  6.000003]
  root(FOC)    x = [18. 12.  6.]  lambda = 0.008333
  budget shares p_i x_i / m = [0.3 0.5 0.2]

CES demand (closed form vs SLSQP):
  sigma    x1(cf)   x1(num)   x2(cf)   x3(cf)   max|diff|
   0.50   19.2359   19.2359    9.4236    8.6025   7.5e-06
   0.80   25.5310   25.5310    8.1515    7.0452   1.9e-05
   2.00   49.0196   49.0196    2.8235    1.9608   2.2e-05
   5.00   59.8426   59.8426    0.0477    0.0191   2.3e-06

CES sigma=2: own-price elasticity of good 1 = -1.1830
theory -sigma + (sigma-1) s1                = -1.1830
```

All three methods agree to the solver tolerance. Note how strongly the elasticity of substitution

matters: with $\sigma = 5$ almost the whole budget goes to good 1, which has the highest weight-to-price ratio, while with $\sigma = 0.5$ the consumer keeps a balanced basket. The numerical own-price elasticity agrees with the formula $-\sigma + (\sigma - 1)s_1$.

△ Caution: Numerical pitfalls

SLSQP returns a point satisfying the KKT conditions to a tolerance, not a certified global optimum. Supply analytic gradients (`jac=`) when you can, use bounds to keep the iterates inside the domain of log or of fractional powers, and rescale problems so that variables and constraints are of similar magnitude. Root-finding on the first-order system finds *any* critical point, including minima and saddle points; it must be followed by a second-order check.

Exercises

9.1.1 Theory Prove that if f is concave on $\mathbb{R}^n$ and $\mathbf{h}(\mathbf{x}) = \mathbf{A}\mathbf{x}$ is linear, then any $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ satisfying (9.3) gives a global maximum of f on $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{c}\}$. *Hint:* use $f(\mathbf{x}) \le f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top(\mathbf{x} - \mathbf{x}^*)$.

9.1.2 Theory Maximise $f(x, y) = xy$ subject to $x^2 + y^2 = 2$. Find all critical points of the Lagrangian, classify them using values of f, and explain why the constraint qualification holds everywhere on the circle.

9.1.3 Theory Derive the CES demand (9.5) in full detail and show that the own-price elasticity equals $-\sigma + (\sigma - 1)s_i$. What are the limits as $\sigma \to 0$ and $\sigma \to \infty$?

9.1.4 Python Modify Listing 9.2 to compute the indirect utility $v(\mathbf{p}, m)$ for CES preferences numerically and verify Roy's identity $x_i = -(\partial v/\partial p_i)/(\partial v/\partial m)$ by finite differences.

9.1.5 Python The minimum-variance portfolio problem minimises $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ subject to $\mathbf{1}^\top \mathbf{w} = 1$. Derive the closed form $\mathbf{w}^* = \boldsymbol{\Sigma}^{-1}\mathbf{1} / (\mathbf{1}^\top \boldsymbol{\Sigma}^{-1}\mathbf{1})$ and check it with SLSQP for a simulated 5×5 covariance matrix. What is the multiplier?

9.1.6 ★ Graduate Let $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}\mathbf{x}$ with $\mathbf{A}$ symmetric. Show that the critical points of f on the unit sphere $\|\mathbf{x}\| = 1$ are eigenvectors of $\mathbf{A}$ and that the multipliers are the eigenvalues (compare Chapter 3). Deduce the Rayleigh quotient characterisation of the largest eigenvalue.

9.1.7 Policy A household survey shows that a household spends its whole food budget of GHS 600 per month on three food groups with shares 0.45, 0.35 and 0.20. Assuming Cobb–Douglas preferences, predict the effect of a 20% increase in the price of the first group on the quantities of all three groups. Which assumption of the model is most questionable for a poor household, and how would CES preferences change your answer?

9.2 Interpretation of Multipliers

9.2.1 Shadow prices and the value function

Embed problem (9.1) in a family indexed by the right-hand sides $\mathbf{c}$ and define the *value function*

$$V(\mathbf{c}) = \max_{\mathbf{x}} \{ f(\mathbf{x}) : \mathbf{h}(\mathbf{x}) = \mathbf{c} \}.$$

Suppose that for $\mathbf{c}$ near $\mathbf{c}_0$ the optimum $\mathbf{x}^*(\mathbf{c})$ and multiplier $\boldsymbol{\lambda}^*(\mathbf{c})$ are differentiable functions of $\mathbf{c}$ (sufficient conditions are given in Theorem 9.6 below). Then

$$\frac{\partial V}{\partial c_j}(\mathbf{c}) = \lambda_j^*(\mathbf{c}), \qquad j = 1, \dots, m. \tag{9.6}$$

The multiplier is the rate at which the optimal value increases when the jth constraint is relaxed by one unit. If f is measured in cedis of welfare and c_j is a budget, λ_j is the welfare return to the marginal cedi; if c_j is a nutrient requirement and f is (minus) cost, λ_j is the marginal cost of the nutrient. Such a number is called a *shadow price*: it is the price at which the decision maker would be indifferent to buying or selling a little more of the constrained resource, even when no market for it exists.

9.2.2 The envelope theorem

Formula (9.6) is a special case of a general and extremely useful result. Consider a parametric problem

$$V(\theta) = \max_{\mathbf{x}} \; f(\mathbf{x}, \theta) \quad \text{s.t.} \quad \mathbf{h}(\mathbf{x}, \theta) = \mathbf{0},$$

with Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \theta) = f(\mathbf{x}, \theta) - \boldsymbol{\lambda}^{\top}\mathbf{h}(\mathbf{x}, \theta)$.

Theorem 9.5 (Envelope theorem). *Suppose f and $\mathbf{h}$ are C^1, that for θ in an open interval the problem has a solution $\mathbf{x}^*(\theta)$ with multiplier $\boldsymbol{\lambda}^*(\theta)$, both differentiable in θ, and that the constraint qualification holds. Then*

$$\frac{dV}{d\theta} = \frac{\partial \mathcal{L}}{\partial \theta}\big(\mathbf{x}^*(\theta), \boldsymbol{\lambda}^*(\theta), \theta\big) = \frac{\partial f}{\partial \theta} - \sum_j \lambda_j^* \frac{\partial h_j}{\partial \theta}.$$

Proof. Since $\mathbf{h}(\mathbf{x}^*(\theta), \theta) = \mathbf{0}$ for all θ, we have $V(\theta) = f(\mathbf{x}^*(\theta), \theta) = \mathcal{L}(\mathbf{x}^*(\theta), \boldsymbol{\lambda}^*(\theta), \theta)$. By the chain rule,

$$\frac{dV}{d\theta} = \nabla_{\mathbf{x}}\mathcal{L}^{\top}\frac{d\mathbf{x}^*}{d\theta} + \nabla_{\boldsymbol{\lambda}}\mathcal{L}^{\top}\frac{d\boldsymbol{\lambda}^*}{d\theta} + \frac{\partial \mathcal{L}}{\partial \theta}.$$

The first term vanishes because $\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{0}$ at the optimum (the Lagrange condition), and the second vanishes because $\nabla_{\boldsymbol{\lambda}}\mathcal{L} = -\mathbf{h} = \mathbf{0}$ (feasibility). Only the direct effect $\partial\mathcal{L}/\partial\theta$ remains. □

Taking $\theta = c_j$ and $\mathbf{h}(\mathbf{x}, \mathbf{c}) = \mathbf{h}(\mathbf{x}) - \mathbf{c}$ gives $\partial\mathcal{L}/\partial c_j = \lambda_j$, which is (9.6). The intuition is that, at an optimum, the indirect effect of re-optimising $\mathbf{x}$ is second order: the decision maker was already at the top of the hill, so small adjustments of $\mathbf{x}$ have no first-order effect on the value.

⋆ Key Concept: Multipliers are shadow prices

By the envelope theorem, $\lambda_j^* = \partial V/\partial c_j$: the multiplier on a constraint is the marginal value of relaxing it. A large multiplier flags a constraint that is expensive; a zero multiplier (Section 9.3) flags one that does not bind. Units matter: λ_j is measured in units of the objective per unit of the constraint.

Marginal utility of income and Roy's identity. For the consumer problem with $\mathcal{L} = u(\mathbf{x}) - \lambda(\mathbf{p}^{\top}\mathbf{x} - m)$, the envelope theorem gives $\partial v/\partial m = \lambda$ (the *marginal utility of income*) and $\partial v/\partial p_i = -\lambda x_i$. Dividing yields *Roy's identity*, $x_i(\mathbf{p}, m) = -(\partial v/\partial p_i)/(\partial v/\partial m)$. For the log Cobb–Douglas utility of Example 9.3, $v = \log m + \text{const}$, so $\lambda = 1/m$: an extra cedi is worth less, in utility terms, to a richer household. Note that λ depends on the cardinal representation of u and is not comparable across households without further (welfare-theoretic) assumptions – a point we return to with equity weights in Section 9.6.

Marginal value of a public budget. Consider a district allocating a budget B (in GHS million) among programmes with outcome functions $f_i(x_i) = a_i \log(1 + x_i/b_i)$ – concave, so returns diminish. The Lagrange conditions $a_i/(b_i + x_i) = \lambda$ say that, at the optimum, the *marginal* outcome per cedi is equalised across funded programmes; their common value is $\lambda = V'(B)$. Listing 9.3 checks this numerically and Fig. 9.2 shows the value function and its tangent.

Listing 9.3: Envelope theorem, comparative statics and bordered Hessian (plotting code omitted)
ch09/ch09_envelope.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Multipliers as shadow prices: envelope theorem and bordered Hessian.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq

# Simulated district programme: outcome f_i(x) = a_i log(1 + x_i / b_i)
names = ["education", "health", "agric. ext."]
a = np.array([3.0, 4.0, 2.0])                # scale of returns (simulated)
b = np.array([1.0, 2.0, 0.5])                # saturation (GHS million)


def allocate(B):
    """KKT solution x_i = max(0, a_i/lam - b_i), lam found by bisection."""
    lam = brentq(lambda L: np.maximum(a / L - b, 0).sum() - B, 1e-8, 1e3,
                 xtol=1e-14)
    return np.maximum(a / lam - b, 0), lam


def V(B):
    return np.sum(a * np.log1p(allocate(B)[0] / b))


B0 = 6.0
x0, lam0 = allocate(B0)
h = 1e-4
dV = (V(B0 + h) - V(B0 - h)) / (2 * h)
print("Budget B = %.1f (GHS million), simulated returns" % B0)
for n, xi, mi in zip(names, x0, a / (b + x0)):
    print("  %-12s x = %6.3f   marginal return = %.4f" % (n, xi, mi))
print("Lagrange multiplier lambda       = %.6f" % lam0)
print("Finite-difference dV/dB          = %.6f" % dV)

# comparative statics: dx/dB (all programmes interior here)
dx = (allocate(B0 + h)[0] - allocate(B0 - h)[0]) / (2 * h)
print("dx/dB (numerical)                =", np.round(dx, 4),
      " sum = %.4f" % dx.sum())
print("closed form a_i / sum(a)         =", np.round(a / a.sum(), 4))
print("dlambda/dB: numerical %.5f, closed form -lam^2/sum(a) = %.5f" %
      ((allocate(B0 + h)[1] - allocate(B0 - h)[1]) / (2 * h),
       -lam0**2 / a.sum()))

# ---- bordered Hessian second-order check ------------------------------
H = np.diag(-a / (b + x0)**2)                # Hessian of Lagrangian in x
g = np.ones(3)                               # gradient of the constraint
Hb = np.block([[np.zeros((1, 1)), g[None, :]], [g[:, None], H]])
minors = [np.linalg.det(Hb[:r, :r]) for r in (3, 4)]
print("\nBordered Hessian leading minors of order 3, 4:", np.round(minors, 4))
print("signs (+, -) as required for a constrained maximum:",
      bool(minors[0] > 0 and minors[1] < 0))
```

Output

```
Budget B = 6.0 (GHS million), simulated returns
  education    x =  2.167   marginal return = 0.9474
  health       x =  2.222   marginal return = 0.9474
  agric. ext.  x =  1.611   marginal return = 0.9474
```

```
Lagrange multiplier lambda       = 0.947368
Finite-difference dV/dB          = 0.947368
dx/dB (numerical)                = [0.3333 0.4444 0.2222]  sum = 1.0000
closed form a_i / sum(a)         = [0.3333 0.4444 0.2222]
dlambda/dB: numerical -0.09972, closed form -lam^2/sum(a) = -0.09972

Bordered Hessian leading minors of order 3, 4: [ 0.5235 -0.3021]
signs (+, -) as required for a constrained maximum: True
```

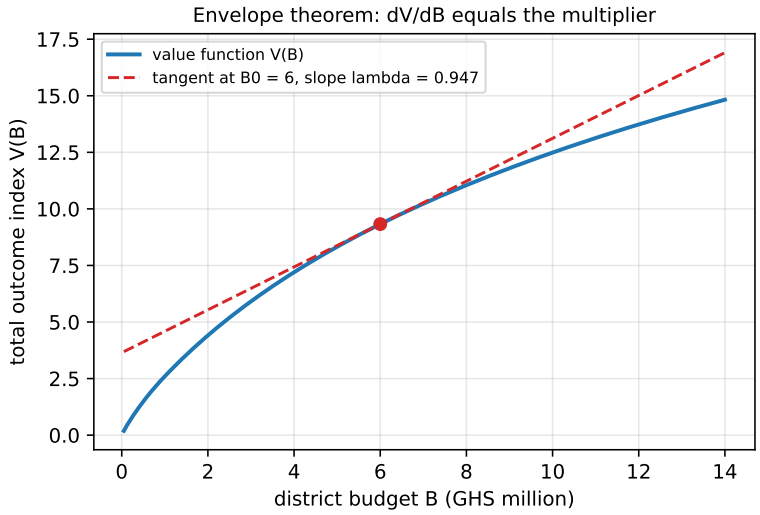


Figure 9.2: Value function $V(B)$ of the simulated district allocation problem. The tangent at $B_0 = 6$ has slope equal to the Lagrange multiplier, as predicted by the envelope theorem. Concavity of V reflects diminishing returns to the budget.

Policy Lens: The shadow price of a fiscal rule

Fiscal rules cap the deficit, the debt ratio or the growth of expenditure. If spending decisions are made optimally subject to the rule, the multiplier on the rule is the welfare value of one more cedi of permitted spending. Comparing it with the cost of an extra cedi of borrowing – interest plus any risk premium – tells the analyst whether the rule is "tight" in an economically meaningful sense. A multiplier of $\lambda = 0.95$ outcome points per cedi in our simulated district, for example, can be compared with the outcome cost of the debt service that relaxing the rule would require. Shadow prices therefore make the trade-offs implicit in a rule explicit, which is exactly what a budget committee needs; they do *not* by themselves say the rule is wrong, because rules also serve credibility goals that the model omits.

9.2.3 Comparative statics

How does the optimum move when a parameter changes? Write the first-order conditions as $\mathbf{F}(\mathbf{x}, \boldsymbol{\lambda}; \theta) = \mathbf{0}$, where $\mathbf{F} = (\nabla_{\mathbf{x}} \mathcal{L},\ -\mathbf{h})$. If the Jacobian

$$\mathbf{J} = \frac{\partial \mathbf{F}}{\partial (\mathbf{x}, \boldsymbol{\lambda})} = \begin{pmatrix} \nabla^2_{\mathbf{x}\mathbf{x}} \mathcal{L} & -D\mathbf{h}^{\top} \\ -D\mathbf{h} & \mathbf{0} \end{pmatrix}$$

is non-singular, the implicit function theorem gives

$$\frac{d(\mathbf{x}^*, \boldsymbol{\lambda}^*)}{d\theta} = -\mathbf{J}^{-1} \frac{\partial \mathbf{F}}{\partial \theta}. \tag{9.7}$$

This is the method of *comparative statics* systematised by Samuelson. In the district example the solution is $x_i = a_i/\lambda - b_i$ with $\lambda = \sum_i a_i/(B + \sum_i b_i)$, so $dx_i/dB = a_i/\sum_k a_k$ and $d\lambda/dB = -\lambda^2/\sum_k a_k < 0$; the listing confirms both by finite differences. The second derivative of the value function, $V''(B) = d\lambda/dB$, is negative: the marginal value of the budget falls as the budget grows.

Historical Note: Samuelson, Hotelling and comparative statics

Paul Samuelson's *Foundations of Economic Analysis* (Samuelson, 1947) recast much of economic theory as the study of constrained maximisation problems and their comparative statics, deriving testable sign restrictions from second-order conditions. Earlier, Harold Hotelling (1932) had used the envelope-type relation between profit functions and supply functions now known as Hotelling's lemma, and his 1931 analysis of exhaustible resources interpreted the scarcity rent of a resource as a shadow price that must grow at the rate of interest. Samuelson received the Nobel Memorial Prize in Economic Sciences in 1970.

9.2.4 Second-order conditions and the bordered Hessian

First-order conditions do not distinguish maxima from minima. The right second-order object is the Hessian of the *Lagrangian*, not of f, restricted to the tangent space of the constraints $T = \{\mathbf{v} \in \mathbb{R}^n : D\mathbf{h}(\mathbf{x}^*)\mathbf{v} = \mathbf{0}\}$.

Theorem 9.6 (Second-order sufficient conditions)**.** *Let $f, \mathbf{h}$ be C^2 and let $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ satisfy (9.3) with $D\mathbf{h}(\mathbf{x}^*)$ of full rank. If $\mathbf{v}^\top \nabla^2_{\mathbf{xx}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*)\mathbf{v} < 0$ for all non-zero $\mathbf{v} \in T$, then $\mathbf{x}^*$ is a strict local constrained maximum. (Replace < 0 by > 0 for a minimum.) Moreover, the Jacobian $\mathbf{J}$ is then non-singular, so $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ is a differentiable function of the parameters.*

Proof sketch. Parametrise S near $\mathbf{x}^*$ by a curve $\mathbf{x}(t)$ with $\mathbf{x}(0) = \mathbf{x}^*$, $\mathbf{x}'(0) = \mathbf{v} \in T$. Along the curve $f(\mathbf{x}(t)) = \mathcal{L}(\mathbf{x}(t), \boldsymbol{\lambda}^*)$, because $\mathbf{h}$ is constant. Differentiating twice at $t = 0$ and using $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$, $\frac{d^2}{dt^2} f(\mathbf{x}(t))|_{t=0} = \mathbf{v}^\top \nabla^2_{\mathbf{xx}} \mathcal{L}\, \mathbf{v} < 0$, so f has a strict local maximum along every feasible curve. A compactness argument on the unit sphere of T makes the bound uniform. For non-singularity of $\mathbf{J}$, suppose $\mathbf{J}(\mathbf{v}, \mathbf{w}) = \mathbf{0}$; then $D\mathbf{h}\mathbf{v} = \mathbf{0}$ and $\mathbf{v}^\top \nabla^2 \mathcal{L} \mathbf{v} = \mathbf{v}^\top D\mathbf{h}^\top \mathbf{w} = 0$, forcing $\mathbf{v} = \mathbf{0}$, and then $D\mathbf{h}^\top \mathbf{w} = \mathbf{0}$ forces $\mathbf{w} = \mathbf{0}$ by the rank condition. □

The definiteness of a quadratic form on a subspace can be checked with determinants. Form the *bordered Hessian*

$$\bar{\mathbf{H}} = \begin{pmatrix} \mathbf{0}_{m \times m} & D\mathbf{h}(\mathbf{x}^*) \\ D\mathbf{h}(\mathbf{x}^*)^\top & \nabla^2_{\mathbf{xx}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*) \end{pmatrix} \in \mathbb{R}^{(m+n)\times(m+n)}.$$

Let $\bar{H}_r$ denote the leading principal minor of order r. Then (assuming the first m columns of $D\mathbf{h}$ are linearly independent):

- $\mathbf{x}^*$ is a strict local *maximum* if the last $n - m$ leading principal minors $\bar{H}_{2m+1}, \dots, \bar{H}_{m+n}$ alternate in sign, with $\operatorname{sign} \bar{H}_{m+r} = (-1)^r$ for $r = m+1, \dots, n$;
- $\mathbf{x}^*$ is a strict local *minimum* if those minors all have the sign $(-1)^m$.

With one constraint and three variables, a maximum requires $\bar{H}_3 > 0$ and $\bar{H}_4 < 0$, which is what the listing finds. For two variables and one constraint the single condition is $\bar{H}_3 > 0$ for a maximum – the familiar textbook criterion (Simon and Blume, 1994).

△ Caution: Which Hessian?

A common error is to check the Hessian of f alone, or the Hessian of $\mathcal{L}$ on all of $\mathbb{R}^n$. Neither is the right object. The function $f(x, y) = xy$ is not concave, yet on the line $x + y = 2$ it has a strict maximum at $(1, 1)$, because the Hessian of the Lagrangian is negative on the tangent direction $(1, -1)$. Conversely, a concave objective with a non-linear equality constraint need not have a maximum at a critical point: the curvature of the constraint enters through the

multiplier term $-\sum \lambda_j \nabla^2 h_j$.

Graduate Perspective: Sensitivity in convex optimisation

In convex optimisation the envelope theorem has a global, non-smooth version: for a convex problem satisfying Slater's condition, the optimal value $p^*(\mathbf{u})$ of the problem with perturbed constraints $g_k(\mathbf{x}) \le u_k$ is convex in $\mathbf{u}$, and the optimal dual vector $\boldsymbol{\mu}^*$ satisfies $p^*(\mathbf{u}) \ge p^*(\mathbf{0}) - \boldsymbol{\mu}^{*\top}\mathbf{u}$ for all $\mathbf{u}$ – the multiplier is a *subgradient* of the value function, valid even where p^* is not differentiable (Boyd and Vandenberghe, 2004, Section 5.6). This is the foundation of sensitivity reports in LP solvers and of decomposition methods that coordinate sub-problems through prices.

Exercises

9.2.1 Theory Prove the envelope theorem for an unconstrained problem $V(\theta) = \max_{\mathbf{x}} f(\mathbf{x}, \theta)$ and give an example where V is differentiable but $\mathbf{x}^*(\theta)$ is not unique. Does the conclusion still hold?

9.2.2 Theory For the district problem with $f_i = a_i \log(1 + x_i/b_i)$ and all programmes funded, derive $\lambda(B) = \sum_i a_i/(B + \sum_i b_i)$ and $V(B)$ in closed form. Verify $V'(B) = \lambda(B)$ directly.

9.2.3 Theory Show that for $f(x, y) = xy$ subject to $x + y = 2$ the bordered Hessian criterion identifies $(1, 1)$ as a maximum, while the Hessian of f is indefinite.

9.2.4 Python Use (9.7) to compute $d\mathbf{x}^*/d\mathbf{p}$ for the three-good Cobb–Douglas consumer of Listing 9.2, and compare with the closed form. Check that the Slutsky matrix is symmetric and negative semi-definite.

9.2.5 Python Replace the log returns in Listing 9.3 by $f_i(x) = a_i(1 - e^{-x/b_i})$. Recompute the allocation, the multiplier and its finite-difference check, and plot $\lambda(B)$.

9.2.6 ★ Graduate (*Le Chatelier principle.*) Show that adding a constraint that is satisfied at the original optimum makes the response of $\mathbf{x}^*$ to its own price smaller in absolute value. Use the comparative-statics formula and the fact that restricting a negative definite form to a subspace preserves definiteness.

9.2.7 Policy A regional health directorate faces a fixed wage bill and a fixed drug budget. Write down a model in which both constraints bind and interpret the two multipliers. If the estimated multiplier on drugs is three times that on wages, what reallocation would you recommend, and what additional information would you need before acting on it?

9.3 Inequality Constraints and the KKT Conditions

9.3.1 The general problem

Budgets are rarely equalities – a district may underspend – and quantities cannot be negative. We therefore consider the general nonlinear programme in *standard (minimisation) form*

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad g_k(\mathbf{x}) \le 0 \ (k = 1, \dots, p), \qquad h_j(\mathbf{x}) = 0 \ (j = 1, \dots, m). \tag{9.8}$$

A maximisation problem $\max f$ becomes $\min(-f)$, and a constraint $g(\mathbf{x}) \ge c$ becomes $c - g(\mathbf{x}) \le 0$. At a feasible point $\mathbf{x}$, the inequality g_k is *active* (binding) if $g_k(\mathbf{x}) = 0$ and *inactive* (slack) if $g_k(\mathbf{x}) < 0$; write $\mathcal{A}(\mathbf{x}) = \{k : g_k(\mathbf{x}) = 0\}$ for the active set. Inactive constraints play no role locally: a small

move in any direction keeps them satisfied. Active constraints behave like equalities in directions that would violate them, but impose nothing in directions that move into the interior.

Intuitively, at a local minimum there must be no direction $\mathbf{d}$ that decreases f ($\nabla f^{\top}\mathbf{d} < 0$) while keeping the equalities ($D\mathbf{h}\,\mathbf{d} = \mathbf{0}$) and moving into the feasible side of each active inequality ($\nabla g_k^{\top}\mathbf{d} \le 0$). By Farkas' lemma – the theorem of the alternative for linear inequalities – the non-existence of such a direction is equivalent to $-\nabla f$ lying in the cone generated by the active inequality gradients plus the span of the equality gradients. Writing this out gives the Karush–Kuhn–Tucker conditions.

Definition 9.7 (KKT conditions)**.** A point $\mathbf{x}^*$ with multipliers $\boldsymbol{\mu}^* \in \mathbb{R}^p$, $\boldsymbol{\lambda}^* \in \mathbb{R}^m$ satisfies the *KKT conditions* for (9.8) if

$$
\begin{aligned}
&\text{(stationarity)} && \nabla f(\mathbf{x}^*) + \sum_{k=1}^{p} \mu_k^* \nabla g_k(\mathbf{x}^*) + \sum_{j=1}^{m} \lambda_j^* \nabla h_j(\mathbf{x}^*) = \mathbf{0},\\
&\text{(primal feasibility)} && g_k(\mathbf{x}^*) \le 0, \quad h_j(\mathbf{x}^*) = 0,\\
&\text{(dual feasibility)} && \mu_k^* \ge 0,\\
&\text{(complementary slackness)} && \mu_k^*\, g_k(\mathbf{x}^*) = 0 \quad \text{for all } k.
\end{aligned}
$$

Equivalently, $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$ for the Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\mu}^{\top}\mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^{\top}\mathbf{h}(\mathbf{x})$.

⋆ Key Concept: Complementary slackness

For each inequality, *either* the constraint binds ($g_k = 0$) *or* its multiplier is zero ($\mu_k = 0$) – or both. A slack constraint has zero shadow price: relaxing a constraint that does not bind is worth nothing. A binding constraint has a non-negative multiplier, whose sign records the direction in which the constraint pushes. In economics: a resource that is not fully used has a zero price; a good that is not produced earns at most zero profit.

9.3.2 Constraint qualifications and the KKT theorem

Definition 9.8 (LICQ and Slater)**.** The *linear independence constraint qualification* (LICQ) holds at a feasible $\mathbf{x}^*$ if the gradients $\{\nabla g_k(\mathbf{x}^*) : k \in \mathcal{A}(\mathbf{x}^*)\} \cup \{\nabla h_j(\mathbf{x}^*)\}_{j=1}^{m}$ are linearly independent. For a *convex* problem – f and all g_k convex, all h_j affine – *Slater's condition* holds if there is a strictly feasible point $\tilde{\mathbf{x}}$ with $g_k(\tilde{\mathbf{x}}) < 0$ for all k and $\mathbf{h}(\tilde{\mathbf{x}}) = \mathbf{0}$.

Theorem 9.9 (Karush–Kuhn–Tucker)**.** *Let f, g_k, h_j be C^1.*

1. *(*Necessity.*) If $\mathbf{x}^*$ is a local minimum of (9.8) and LICQ holds at $\mathbf{x}^*$, there exist unique multipliers $(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ such that the KKT conditions hold.*
2. *(*Sufficiency for convex problems.*) If the problem is convex and $(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ satisfies the KKT conditions, then $\mathbf{x}^*$ is a global minimum.*
3. *(*Convex necessity.*) If the problem is convex and satisfies Slater's condition, then every global minimum satisfies the KKT conditions for some multipliers.*

Proof sketch. (1) Under LICQ, the set of feasible directions at $\mathbf{x}^*$ is (to first order) the cone $C = \{\mathbf{d} : \nabla g_k^{\top}\mathbf{d} \le 0 \ (k \in \mathcal{A}),\ D\mathbf{h}\,\mathbf{d} = \mathbf{0}\}$, because LICQ allows us – through the implicit function theorem, as in Theorem 9.2 – to construct a feasible curve tangent to any $\mathbf{d} \in C$. Optimality implies $\nabla f^{\top}\mathbf{d} \ge 0$ for all $\mathbf{d} \in C$. Farkas' lemma states that this holds if and only if $-\nabla f = \sum_{k \in \mathcal{A}} \mu_k \nabla g_k + \sum_j \lambda_j \nabla h_j$ with $\mu_k \ge 0$. Setting $\mu_k = 0$ for inactive k gives complementary

slackness; uniqueness follows from linear independence. (2) For convex f and g_k and affine h_j, the Lagrangian $\mathcal{L}(\cdot, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ is convex in $\mathbf{x}$ because $\boldsymbol{\mu}^* \geq \mathbf{0}$, so stationarity makes $\mathbf{x}^*$ its global minimiser. For any feasible $\mathbf{x}$, $f(\mathbf{x}) \geq f(\mathbf{x}) + \boldsymbol{\mu}^{*\top}\mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^{*\top}\mathbf{h}(\mathbf{x}) = \mathcal{L}(\mathbf{x}, \cdot) \geq \mathcal{L}(\mathbf{x}^*, \cdot) = f(\mathbf{x}^*)$, using $\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$ and complementary slackness. (3) follows from strong duality (Theorem 9.13). □

LICQ is the inequality analogue of the rank condition in Theorem 9.2. Weaker qualifications exist – the Mangasarian–Fromovitz condition requires only a direction strictly into the interior of the active inequalities – and for *linear* constraints no qualification is needed at all, which is why linear programmes always have multipliers (Nocedal and Wright, 2006, Chapter 12).

Historical Note: Karush, Kuhn and Tucker

The conditions now called KKT first appeared in William Karush's 1939 master's thesis at the University of Chicago, *Minima of Functions of Several Variables with Inequalities as Side Conditions*, which was never published as a paper. Fritz John (1948) derived a version without a constraint qualification, at the cost of a multiplier on the objective that may be zero. Harold Kuhn and Albert Tucker rediscovered the conditions and published them in "Nonlinear Programming" at the Second Berkeley Symposium (1951) (Kuhn and Tucker, 1951); the paper launched nonlinear programming as a field. Karush's priority was widely acknowledged only later, and the name "Karush–Kuhn–Tucker" became standard from the 1970s (Kjeldsen, 2000).

9.3.3 Corner solutions

In the consumer problem with non-negativity, write the constraints as $\mathbf{p}^\top\mathbf{x} - m \leq 0$ and $-x_i \leq 0$. With multipliers $\lambda \geq 0$ and $\nu_i \geq 0$, stationarity for $\min(-u)$ reads $-\partial u/\partial x_i + \lambda p_i - \nu_i = 0$, and complementary slackness gives $\nu_i x_i = 0$. Eliminating ν_i:

$$\frac{\partial u}{\partial x_i} \leq \lambda p_i, \qquad \text{with equality if } x_i > 0. \tag{9.9}$$

A good is bought only if its marginal utility per cedi reaches the marginal utility of income. Goods whose marginal utility per cedi at zero consumption, $u_i(\mathbf{0})/p_i$, is too low are not bought at all – a *corner solution*. Cobb–Douglas utility never produces corners (marginal utility is infinite at zero), which is one reason it is analytically convenient and empirically limited: survey data are full of zeros.

Example 9.10 (Worked example A: one active constraint)**.** Minimise $f = (x_1 - 2)^2 + (x_2 - 2)^2$ subject to $g_1 = x_1 + x_2 - 2 \leq 0$, $g_2 = x_1 - 1.5 \leq 0$, $x_1, x_2 \geq 0$. The unconstrained minimiser $(2, 2)$ is infeasible, so try g_1 active and the rest slack: stationarity gives $2(x_i - 2) + \mu_1 = 0$, so $x_1 = x_2 = 2 - \mu_1/2$; $x_1 + x_2 = 2$ gives $\mu_1 = 2$ and $\mathbf{x}^* = (1, 1)$. Check: $g_2 = -0.5 < 0$ and $x_i = 1 > 0$, so their multipliers are zero; $\mu_1 = 2 \geq 0$. All KKT conditions hold, and since the problem is convex, $(1, 1)$ is the global minimum. The multiplier says that increasing the right-hand side of g_1 from 2 to $2 + \delta$ lowers the minimum by about 2δ.

Example 9.11 (Worked example B: quasi-linear utility)**.** Maximise $\alpha \log x_1 + x_2$ subject to $p_1 x_1 + p_2 x_2 \leq m$, $x_1, x_2 \geq 0$. Good 2 has constant marginal utility 1. If $x_2 > 0$, (9.9) gives $\lambda = 1/p_2$ and $x_1 = \alpha p_2/p_1$, which leaves $m - \alpha p_2$ for good 2 – feasible only if $m > \alpha p_2$. If $m \leq \alpha p_2$ we guess $x_2 = 0$: then $x_1 = m/p_1$, $\lambda = \alpha/m$, and the condition for good 2 is $1 \leq \lambda p_2$, i.e. $\nu_2 = \alpha p_2/m - 1 \geq 0$, which holds precisely when $m \leq \alpha p_2$. The Engel curves have a kink at $m = \alpha p_2$: below it all income goes to good 1. This is a stylised model of a poor household that spends everything on food until a threshold income is reached.

The general strategy for solving KKT systems by hand is *active-set enumeration*: guess which constraints bind, solve the resulting equations, and check the inequalities (primal feasibility of the slack constraints, non-negativity of the multipliers of the binding ones). With p inequalities there are up to 2^p guesses; economic intuition usually eliminates most of them. Numerical

solvers such as SLSQP and interior-point methods automate this search. Listing 9.4 checks both worked examples with SLSQP and recovers the multipliers from the stationarity condition by least squares; Fig. 9.3 illustrates them.

Listing 9.4: KKT examples checked with SLSQP (plotting code omitted) ch09/ch09_kkt_examples.py

```python
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# KKT conditions: hand-solved examples checked numerically with SLSQP.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize


def kkt_multipliers(grad_f, grads_active):
    """Solve grad f + sum mu_j grad g_j = 0 for mu (least squares)."""
    G = np.column_stack(grads_active)
    mu, *_ = np.linalg.lstsq(G, -grad_f, rcond=None)
    return mu, np.linalg.norm(G @ mu + grad_f)


# ---- Example A: min (x1-2)^2 + (x2-2)^2, x1+x2 <= 2, x1 <= 1.5, x >= 0 -----
f = lambda x: (x[0] - 2)**2 + (x[1] - 2)**2
df = lambda x: np.array([2 * (x[0] - 2), 2 * (x[1] - 2)])
cons = [{"type": "ineq", "fun": lambda x: 2 - x[0] - x[1]},     # g1 <= 0
        {"type": "ineq", "fun": lambda x: 1.5 - x[0]}]          # g2 <= 0
res = minimize(f, x0=[0.0, 0.0], jac=df, method="SLSQP", constraints=cons,
               bounds=[(0, None), (0, None)], tol=1e-12)
x = res.x
slack = np.array([2 - x.sum(), 1.5 - x[0], x[0], x[1]])
print("Example A (by hand: x* = (1, 1), mu1 = 2, others 0)")
print("  SLSQP x* =", np.round(x, 6), " f* = %.6f" % res.fun)
print("  slacks of (g1, g2, x1>=0, x2>=0):", np.round(slack, 6))
mu, resid = kkt_multipliers(df(x), [np.array([1.0, 1.0])])
print("  multiplier on active g1: mu1 = %.6f  (stationarity residual %.1e)"
      % (mu[0], resid))

# ---- Example B: quasi-linear utility, corner solution --------------------
# max alpha*log(x1) + x2  s.t.  p1 x1 + p2 x2 <= m,  x1, x2 >= 0
alpha, p1, p2 = 3.0, 1.0, 2.0


def demand_hand(m):
    if m > alpha * p2:                                  # interior
        return np.array([alpha * p2 / p1, (m - alpha * p2) / p2]), 1 / p2
    return np.array([m / p1, 0.0]), alpha / m           # corner: x2 = 0


print("\nExample B: quasi-linear utility, threshold income alpha*p2 = %.1f"
      % (alpha * p2))
print("     m   x1(hand) x2(hand)  x1(SLSQP) x2(SLSQP)  lambda   mu2(x2>=0)")
for m in [2.0, 4.0, 6.0, 8.0, 12.0]:
    xh, lam = demand_hand(m)
    r = minimize(lambda z: -(alpha * np.log(z[0]) + z[1]), x0=[0.5, 0.5],
                 method="SLSQP", bounds=[(1e-9, None), (0, None)],
                 constraints=[{"type": "ineq",
                               "fun": lambda z: m - p1 * z[0] - p2 * z[1]}],
                 tol=1e-12)
    mu2 = max(lam * p2 - 1, 0.0)                        # dual feasibility
    print("  %5.1f  %8.3f %8.3f  %9.3f %9.3f  %7.4f  %8.4f" %
          (m, *xh, *r.x, lam, mu2))

```

```
# ---- Example C: failure of LICQ (Kuhn-Tucker cusp) ------------------------
# min -x1  s.t.  x2 - (1 - x1)^3 <= 0,  -x2 <= 0 ; optimum at (1, 0)
xc = np.array([1.0, 0.0])
g_act = [np.array([3 * (1 - xc[0])**2, 1.0]), np.array([0.0, -1.0])]
mu, resid = kkt_multipliers(np.array([-1.0, 0.0]), g_act)
print("\nExample C (cusp): best multipliers", np.round(mu, 3),
      " stationarity residual = %.3f -> no KKT point" % resid)
```

Output

```
Example A (by hand: x* = (1, 1), mu1 = 2, others 0)
  SLSQP x* = [1. 1.]  f* = 2.000000
  slacks of (g1, g2, x1>=0, x2>=0): [-0.   0.5  1.   1. ]
  multiplier on active g1: mu1 = 2.000000  (stationarity residual 6.3e-16)

Example B: quasi-linear utility, threshold income alpha*p2 = 6.0
     m   x1(hand) x2(hand)  x1(SLSQP) x2(SLSQP)  lambda   mu2(x2>=0)
   2.0     2.000    0.000      2.000     0.000   1.5000    2.0000
   4.0     4.000    0.000      4.000     0.000   0.7500    0.5000
   6.0     6.000    0.000      6.000     0.000   0.5000    0.0000
   8.0     6.000    1.000      6.000     1.000   0.5000    0.0000
  12.0     6.000    3.000      6.000     3.000   0.5000    0.0000

Example C (cusp): best multipliers [0. 0.]  stationarity residual = 1.000 -> no KKT point
```

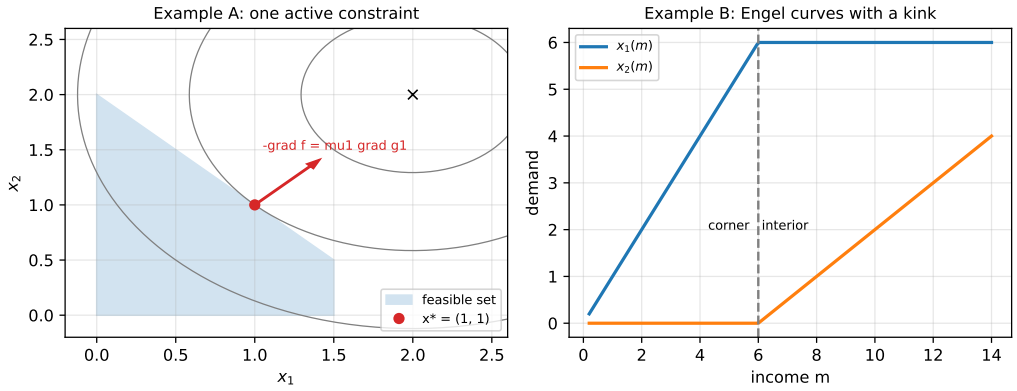


Figure 9.3: Left: example A. The feasible set is shaded; at $\mathbf{x}^* = (1, 1)$ the negative gradient of f points along the outward normal of the single active constraint. Right: example B. Engel curves of the quasi-linear consumer have a kink at $m = \alpha p_2 = 6$, where the corner solution $x_2 = 0$ gives way to an interior one.

Example C in the listing is the classical cusp of Kuhn and Tucker: minimise $-x_1$ subject to $x_2 \le (1 - x_1)^3$ and $x_2 \ge 0$. The minimiser is $(1, 0)$, but both active gradients are vertical there, $(0, 1)$ and $(0, -1)$, so they are linearly dependent and cannot combine to give $-\nabla f = (1, 0)$. LICQ fails and no KKT multipliers exist even though the point is optimal.

△ Caution: Sign conventions for multipliers

Textbooks, solvers and economists use different conventions: $\mathcal{L} = f - \lambda^\top(\mathbf{h} - \mathbf{c})$ for maximisation versus $\mathcal{L} = f + \boldsymbol{\mu}^\top \mathbf{g}$ for minimisation with $\mathbf{g} \le \mathbf{0}$; and solvers may report multipliers ("marginals", "duals") with either sign. In `scipy.optimize.linprog` with the HiGHS method, the marginals of $\le$ constraints in a minimisation are ≤ 0, being the derivative

of the optimal value with respect to the right-hand side. Always verify the sign by perturbing a constraint and re-solving.

Exercises

9.3.1 Theory Solve by hand: minimise $x_1^2 + x_2^2$ subject to $x_1 + 2x_2 \geq 4$, $x_1 \geq 1$. Enumerate the four possible active sets, identify the KKT point, and interpret the multiplier.

9.3.2 Theory Show that in a convex problem the set of KKT multipliers at a minimiser is the same for every minimiser. *Hint:* use the saddle-point characterisation of Section 9.4.

9.3.3 Theory Derive the KKT conditions for the consumer with utility $u(\mathbf{x}) = \sum_i a_i \log(x_i + \gamma_i)$, $\gamma_i > 0$ (Stone–Geary type with a shift) and show that the demand has the "water-filling" form $x_i = \max\{0, a_i/(\lambda p_i) - \gamma_i\}$.

9.3.4 Python Implement active-set enumeration: write a function that, for a convex quadratic programme with linear inequalities, loops over all active sets, solves the equality-constrained subproblem in closed form and returns the KKT point. Test it on example A and on a random problem with five constraints, checking against SLSQP.

9.3.5 Python In example B, give good 2 diminishing marginal utility by setting $u = \alpha \log x_1 + x_2 - \beta x_2^2$ with $\beta = 0.05$. Find the demand numerically over a grid of incomes and plot the Engel curves. Where are the kinks, and is there an income beyond which the budget constraint stops binding?

9.3.6 ★ Graduate Construct a problem in which LICQ fails but the Mangasarian–Fromovitz condition holds, and show that KKT multipliers exist but are not unique. Explain why non-uniqueness of multipliers matters for sensitivity analysis.

9.3.7 Policy A farmer allocates 4 hectares among maize, cassava and soybean with concave revenue functions and a labour constraint. Formulate the problem with non-negativity, and explain in words what it means when the KKT multiplier on soybean area is strictly positive. What extension service might change that?

9.4 Duality and Linear Programming

9.4.1 The Lagrange dual function

Return to the standard-form problem (9.8) with optimal value $p^\ast$, and its Lagrangian $\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\mu}^\top \mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x})$. The *Lagrange dual function* is

$$q(\boldsymbol{\mu}, \boldsymbol{\lambda}) = \inf_{\mathbf{x} \in \mathbb{R}^n} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}), \tag{9.10}$$

which may take the value $-\infty$. As a pointwise infimum of functions that are affine in $(\boldsymbol{\mu}, \boldsymbol{\lambda})$, q is *always concave*, whether or not the original problem is convex. The *dual problem* is

$$d^\ast = \max_{\boldsymbol{\mu} \geq \mathbf{0},\, \boldsymbol{\lambda}} \; q(\boldsymbol{\mu}, \boldsymbol{\lambda}).$$

The economic reading is illuminating: think of μ_k as a price charged for using resource k. Instead of forbidding the decision maker from exceeding a constraint, we let her violate it and pay μ_k per unit (and receive μ_k per unit of slack). The dual function is her minimum cost at those prices, and the dual problem looks for the prices that make the penalty regime as demanding as the original constraints.

Theorem 9.12 (Weak duality). *For every $\boldsymbol{\mu} \geq \mathbf{0}$ and every $\boldsymbol{\lambda}$, $q(\boldsymbol{\mu}, \boldsymbol{\lambda}) \leq p^*$. Hence $d^* \leq p^*$.*

Proof. Let $\tilde{\mathbf{x}}$ be any feasible point. Since $\boldsymbol{\mu} \geq \mathbf{0}$, $\mathbf{g}(\tilde{\mathbf{x}}) \leq \mathbf{0}$ and $\mathbf{h}(\tilde{\mathbf{x}}) = \mathbf{0}$,

$$q(\boldsymbol{\mu}, \boldsymbol{\lambda}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) \leq \mathcal{L}(\tilde{\mathbf{x}}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\tilde{\mathbf{x}}) + \underbrace{\boldsymbol{\mu}^{\top} \mathbf{g}(\tilde{\mathbf{x}})}_{\leq 0} + \underbrace{\boldsymbol{\lambda}^{\top} \mathbf{h}(\tilde{\mathbf{x}})}_{=0} \leq f(\tilde{\mathbf{x}}).$$

Taking the infimum over feasible $\tilde{\mathbf{x}}$ gives $q(\boldsymbol{\mu}, \boldsymbol{\lambda}) \leq p^*$, and then the supremum over dual-feasible $(\boldsymbol{\mu}, \boldsymbol{\lambda})$ gives $d^* \leq p^*$. □

The difference $p^* - d^* \geq 0$ is the *duality gap*. Weak duality is remarkably useful even when the gap is positive: any dual-feasible point certifies a lower bound on what can be achieved, which is how branch-and-bound methods for integer programmes prune their search, and how one can stop an algorithm with a guaranteed accuracy.

9.4.2 Strong duality and Slater's condition

Theorem 9.13 (Strong duality). *If problem (9.8) is convex and Slater's condition holds, then $d^* = p^*$ and, if p^* is finite, the dual optimum is attained. Moreover $\mathbf{x}^*$ and $(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ are primal and dual optimal if and only if they satisfy the KKT conditions, which is equivalent to the* saddle-point *property*

$$\mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}, \boldsymbol{\lambda}) \leq \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \leq \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \quad \text{for all } \mathbf{x},\ \boldsymbol{\mu} \geq \mathbf{0},\ \boldsymbol{\lambda}.$$

Proof idea. Consider the set of achievable (constraint, objective) values $\mathcal{G} = \{(\mathbf{u}, t) : \exists \mathbf{x},\ \mathbf{g}(\mathbf{x}) \leq \mathbf{u},\ \mathbf{h}(\mathbf{x}) = \mathbf{0},\ f(\mathbf{x}) \leq t\}$. For a convex problem $\mathcal{G}$ is convex, and the point $(\mathbf{0}, p^*)$ lies on its boundary. A supporting hyperplane to $\mathcal{G}$ at that point has normal $(\boldsymbol{\mu}^*, 1)$ with $\boldsymbol{\mu}^* \geq \mathbf{0}$; the coefficient on t can be normalised to one because Slater's point $\tilde{\mathbf{x}}$ puts $(\mathbf{g}(\tilde{\mathbf{x}}), f(\tilde{\mathbf{x}}))$ strictly to the left of the hyperplane's vertical position. The supporting inequality $t + \boldsymbol{\mu}^{*\top} \mathbf{u} \geq p^*$ on $\mathcal{G}$ is exactly $q(\boldsymbol{\mu}^*) \geq p^*$, which with weak duality gives equality. (Affine equality constraints are handled the same way; see Boyd and Vandenberghe, 2004, Section 5.3.) □

Listing 9.5 contrasts two cases. For the convex quadratic programme $\min x_1^2 + x_2^2$ s.t. $x_1 + x_2 \geq 1$, minimising $\mathcal{L} = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$ gives $x_i = \lambda/2$ and $q(\lambda) = \lambda - \lambda^2/2$, maximised at $\lambda = 1$ with $d^* = 1/2 = p^*$. For a 0–1 *project selection* problem – choose a subset of projects with benefits v_i and costs c_i to maximise total benefit within a budget – the feasible set is discrete and non-convex, and a gap appears (Fig. 9.4).

Listing 9.5: Dual functions with and without a duality gap (plotting code omitted)
ch09/ch09_duality.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Lagrange dual functions: strong duality (convex QP) and a gap (knapsack).
# (c) 2026 Stephen E. Moore. MIT Licence.
import itertools
import numpy as np
import matplotlib.pyplot as plt

# ---- (1) convex QP: min x1^2 + x2^2  s.t.  x1 + x2 >= 1 ------------------
# L(x, lam) = x'x + lam (1 - x1 - x2); minimiser x = lam/2 (1, 1)
q = lambda lam: lam - lam**2 / 2                    # dual function
lams = np.linspace(0, 2.5, 300)
p_star, d_star = 0.5, q(1.0)
print("Convex QP:  p* = %.4f,  d* = max q(lam) = %.4f at lam = 1" %
      (p_star, d_star))
print("  duality gap = %.2e  (Slater holds: x = (1, 1) is strictly feasible)"
      % (p_star - d_star))

# ---- (2) 0-1 project selection (knapsack): duality gap -------------------
```

```python
v = np.array([10.0, 13.0, 7.0, 8.0, 5.0, 6.0])      # benefit (simulated)
c = np.array([5.0, 7.0, 4.0, 5.0, 3.0, 4.0])        # cost, GHS million
B = 13.0
best, best_x = -np.inf, None
for bits in itertools.product([0, 1], repeat=len(v)):
    xb = np.array(bits)
    if c @ xb <= B and v @ xb > best:
        best, best_x = v @ xb, xb


def knap_dual(lam):
    """q(lam) = max over x in {0,1}^n of v'x - lam (c'x - B)."""
    return lam * B + np.maximum(v - lam * c, 0).sum()


lam_k = np.linspace(0, 3, 600)
qk = np.array([knap_dual(L) for L in lam_k])
cands = np.r_[0, v / c]                              # kinks of q(lam)
qc = np.array([knap_dual(L) for L in cands])
d_k, lam_best = qc.min(), cands[np.argmin(qc)]
print("\nKnapsack: projects chosen", best_x, " cost = %.0f  p* = %.1f" %
      (c @ best_x, best))
print("  benefit-cost ratios:", np.round(v / c, 3))
print("  dual bound d* = min q(lam) = %.3f at lam = %.3f" % (d_k, lam_best))
print("  duality gap d* - p* = %.3f  (weak duality: d* >= p*)" % (d_k - best))
```

Output

```
Convex QP:  p* = 0.5000,  d* = max q(lam) = 0.5000 at lam = 1
  duality gap = 0.00e+00  (Slater holds: x = (1, 1) is strictly feasible)

Knapsack: projects chosen [1 0 0 1 1 0]  cost = 13  p* = 23.0
  benefit-cost ratios: [2.    1.857 1.75  1.6   1.667 1.5  ]
  dual bound d* = min q(lam) = 24.750 at lam = 1.750
  duality gap d* - p* = 1.750  (weak duality: d* >= p*)
```

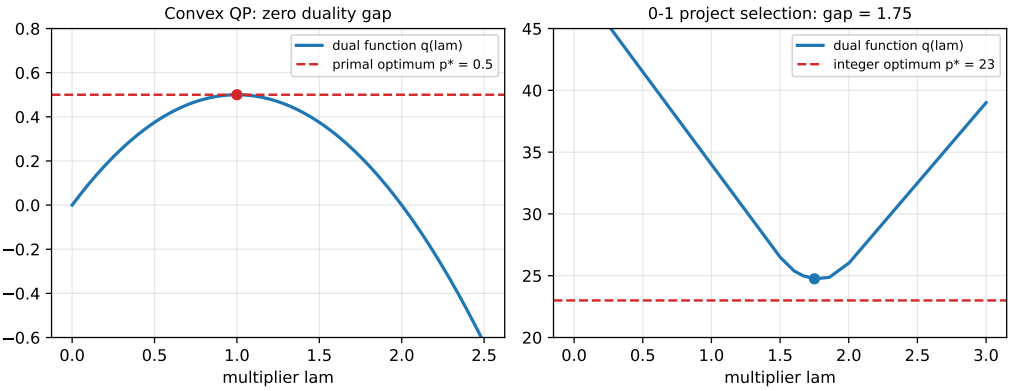


Figure 9.4: Left: the concave dual function of a convex QP touches the primal optimum (zero gap). Right: the dual function of a 0–1 project-selection problem (written as a maximisation, so the dual is minimised) is piecewise linear and convex, and its minimum lies strictly above the best integer value – a duality gap of 1.75.

For the knapsack, the dual bound 24.75 equals the value of the *LP relaxation* (allowing fractional projects): fund the projects in order of benefit–cost ratio and take a fraction of the marginal one. The optimal dual price $\lambda^* = 1.75$ is the ratio of that marginal project. This is a

general fact: Lagrangian duality "convexifies" a problem.

Graduate Perspective: Duality as convexification

For any problem, the Lagrange dual equals the dual of its *convex relaxation*: the bound d^* is the optimal value of the problem obtained by replacing the objective and constraints by their convex envelopes. Duality gaps therefore measure non-convexity. In large separable problems (many households, many projects) the relative gap shrinks as the number of components grows – the Shapley–Folkman lemma – which is why price-based decentralisation works well in large economies and Lagrangian relaxation works well in large integer programmes (Bertsekas, 1999, Chapter 5).

9.4.3 Linear programming and LP duality

A *linear programme* has a linear objective and linear constraints. In the form we will use for the diet problem,

$$\text{(P)}\quad \min_{\mathbf{x}} \ \mathbf{c}^\top \mathbf{x} \ \text{ s.t. } \ \mathbf{A}\mathbf{x} \geq \mathbf{b},\ \mathbf{x} \geq \mathbf{0}, \qquad \text{(D)}\quad \max_{\mathbf{y}} \ \mathbf{b}^\top \mathbf{y} \ \text{ s.t. } \ \mathbf{A}^\top \mathbf{y} \leq \mathbf{c},\ \mathbf{y} \geq \mathbf{0}. \tag{9.11}$$

To derive (D), attach $\mathbf{y} \geq \mathbf{0}$ to $\mathbf{b} - \mathbf{A}\mathbf{x} \leq \mathbf{0}$ and $\mathbf{s} \geq \mathbf{0}$ to $-\mathbf{x} \leq \mathbf{0}$: $\mathcal{L} = \mathbf{c}^\top \mathbf{x} + \mathbf{y}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) - \mathbf{s}^\top \mathbf{x} = \mathbf{b}^\top \mathbf{y} + (\mathbf{c} - \mathbf{A}^\top \mathbf{y} - \mathbf{s})^\top \mathbf{x}$. The infimum over $\mathbf{x}$ is $-\infty$ unless $\mathbf{c} - \mathbf{A}^\top \mathbf{y} - \mathbf{s} = \mathbf{0}$, in which case it is $\mathbf{b}^\top \mathbf{y}$; eliminating $\mathbf{s} \geq \mathbf{0}$ gives $\mathbf{A}^\top \mathbf{y} \leq \mathbf{c}$.

Theorem 9.14 (LP duality). *For the pair (9.11): (i) $\mathbf{b}^\top \mathbf{y} \leq \mathbf{c}^\top \mathbf{x}$ for all feasible $\mathbf{x}, \mathbf{y}$; (ii) if either problem has a finite optimum, so does the other and the optimal values are equal; (iii) if (P) is unbounded, (D) is infeasible, and vice versa; (iv) feasible $\mathbf{x}, \mathbf{y}$ are optimal if and only if they satisfy complementary slackness: $y_i(\mathbf{A}\mathbf{x} - \mathbf{b})_i = 0$ for all i and $x_j(\mathbf{c} - \mathbf{A}^\top \mathbf{y})_j = 0$ for all j.*

⋆ Key Concept: Duality

Every constrained problem has a dual problem in the multipliers (prices). The dual function is always concave and every dual-feasible price vector gives a bound on the primal optimum (weak duality). For convex problems satisfying Slater's condition, and for every LP with a finite optimum, the bounds are tight (strong duality), and optimal prices and quantities are linked by complementary slackness.

No constraint qualification is needed: linear constraints always admit multipliers, so strong duality holds whenever an optimum exists. In the diet interpretation, $\mathbf{x}$ are food quantities, $\mathbf{c}$ their prices, $\mathbf{A}$ the nutrient content and $\mathbf{b}$ the requirements. The dual imagines a "pill seller" who sets prices y_i for pure nutrients so as to maximise the value $\mathbf{b}^\top \mathbf{y}$ of the requirement bundle, subject to no food being cheaper than the nutrients it contains ($\mathbf{A}^\top \mathbf{y} \leq \mathbf{c}$). LP duality says the pill seller can earn exactly the least cost of a diet. Complementary slackness says: a nutrient supplied in excess has price zero, and a food whose cost exceeds the value of its nutrients (positive *reduced cost* $c_j - \mathbf{a}_j^\top \mathbf{y}$) is not bought.

Historical Note: The birth of linear programming

Leonid Kantorovich formulated production-planning problems as linear programmes in *Mathematical Methods of Organizing and Planning Production* (Leningrad, 1939), introducing "resolving multipliers" that act as prices. Tjalling Koopmans developed activity analysis and the economic interpretation of LP for transportation problems in the 1940s; Kantorovich and Koopmans shared the 1975 Nobel Memorial Prize "for their contributions to the theory of optimum allocation of resources". George Dantzig invented the simplex method in 1947 while working on planning problems for the US Air Force. In the same year John von Neumann, drawing on his theory of zero-sum games, described the

duality theorem to Dantzig; a proof was published by Gale, Kuhn and Tucker in 1951 (Dantzig, 1963).

9.4.4 The least-cost nutritious food basket

George Stigler (1945) asked for the cheapest diet that meets the recommended allowances of nine nutrients using 77 foods. Without an algorithm, he found a good diet by heuristics, costing \$39.93 per year at 1939 prices. When the simplex method became available the problem was solved exactly, and Stigler's heuristic answer turned out to be within a few cents of the optimum of about \$39.69 per year (Stigler, 1945; Dantzig, 1963).

Listing 9.6 solves a modern, stylised version with eleven foods common in West African markets and six nutrients. The nutrient contents are rounded teaching values and the prices (GHS per 100 g) are *simulated*; neither should be used for programme design. We add upper bounds on each food (no more than 500 g of gari a day, for instance) as a crude palatability constraint. `linprog` with the HiGHS solver returns the primal solution and, in `res.ineqlin.marginals`, the dual values.

Listing 9.6: Least-cost nutritious diet and its shadow prices (plotting code omitted)
ch09/ch09_diet_lp.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Least-cost nutritious food basket (Stigler diet LP) with simulated prices.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog

# Stylised nutrient content per 100 g (teaching values, NOT a food table)
foods = ["maize flour", "rice", "gari", "cowpea", "groundnut", "dried fish",
         "eggs", "leafy greens", "red palm oil", "milk powder", "plantain"]
nutr = ["energy kcal", "protein g", "fat g", "iron mg", "calcium mg",
        "vit A ug"]
N = np.array([
    # kcal  prot   fat  iron   calc   vitA
    [360,   9.0,  4.0,  3.5,     7,     10],   # maize flour
    [360,   7.0,  0.6,  0.8,    10,      0],   # rice
    [360,   1.2,  0.5,  1.5,    40,      0],   # gari
    [340,  23.0,  1.3,  8.0,   110,      2],   # cowpea
    [570,  25.0, 49.0,  4.6,    90,      0],   # groundnut
    [330,  60.0,  6.0,  5.0,  1500,     50],   # dried fish
    [145,  12.5, 10.0,  1.8,    55,    160],   # eggs
    [ 40,   3.5,  0.7,  2.5,   200,    300],   # leafy greens
    [880,   0.0, 99.0,  0.0,     0,   5000],   # red palm oil
    [500,  26.0, 27.0,  0.5,   900,    300],   # milk powder
    [120,   1.3,  0.4,  0.6,     3,     50]]).T  # plantain
price = np.array([0.9, 1.6, 0.8, 2.0, 2.6, 4.5, 3.5, 0.6, 2.2, 6.0, 0.7])
req = np.array([2300, 50, 45, 18, 1000, 600])     # daily requirement
upper = np.array([5, 4, 5, 3, 1.5, 1, 2, 3, 0.6, 1, 4])  # max 100 g units

# min price'x  s.t.  N x >= req  (written as -N x <= -req),  0 <= x <= upper
res = linprog(price, A_ub=-N, b_ub=-req, bounds=list(zip(0 * upper, upper)),
              method="highs")
x = res.x
print("Least-cost basket (simulated prices), solver status =", res.status)
for f, xi, pi in zip(foods, x, price):
    if xi > 1e-9:
        print("  %-13s %5.0f g   cost GHS %5.2f" % (f, 100 * xi, pi * xi))
print("  daily cost = GHS %.3f" % res.fun)

y = -res.ineqlin.marginals                 # shadow prices (>= 0)
```

```
print("\n  nutrient        required  supplied  shadow price (GHS/unit)")
for n, r, s, yi in zip(nutr, req, N @ x, y):
    print("  %-13s %9.0f %9.1f   %.5f" % (n, r, s, yi))
z = -res.upper.marginals                    # value of relaxing upper bounds
print("  binding upper bounds:", [f for f, zi in zip(foods, z) if zi > 1e-9])

# dual LP: max req'y - upper'z  s.t.  N'y - z <= price,  y, z >= 0
nf, nn = len(foods), len(nutr)
dual = linprog(np.r_[-req, upper], A_ub=np.hstack([N.T, -np.eye(nf)]),
               b_ub=price, bounds=[(0, None)] * (nn + nf), method="highs")
print("\nDual LP optimum = %.3f  (primal %.3f): strong duality" %
      (-dual.fun, res.fun))
rc = price - N.T @ y + z
print("reduced costs (GHS per 100 g) of foods not in the basket:")
print("  " + ", ".join("%s %.2f" % (f, r) for f, r in zip(foods, rc)
                       if r > 1e-6))

# parametric analysis: cost as a function of the protein requirement
prot = np.linspace(20, 140, 121)
cost = []
for pr in prot:
    r2 = req.copy()
    r2[1] = pr
    cost.append(linprog(price, A_ub=-N, b_ub=-r2,
                        bounds=list(zip(0 * upper, upper)),
                        method="highs").fun)
```

Output

```
Least-cost basket (simulated prices), solver status = 0
  maize flour     216 g   cost GHS  1.95
  gari            293 g   cost GHS  2.35
  dried fish       35 g   cost GHS  1.58
  leafy greens    171 g   cost GHS  1.03
  red palm oil     32 g   cost GHS  0.70
  daily cost = GHS 7.599

  nutrient        required  supplied  shadow price (GHS/unit)
  energy kcal         2300    2300.0   0.00182
  protein g             50      50.0   0.01083
  fat g                 45      45.0   0.00602
  iron mg               18      18.0   0.03088
  calcium mg          1000    1000.0   0.00204
  vit A ug             600    2146.9   0.00000
  binding upper bounds: []

Dual LP optimum = 7.599  (primal 7.599): strong duality
reduced costs (GHS per 100 g) of foods not in the basket:
  rice 0.82, cowpea 0.65, groundnut 0.67, eggs 2.87, milk powder 2.79, plantain 0.44
```

The optimal basket uses five foods: maize flour and gari for cheap energy, dried fish for protein and calcium, leafy greens for iron, and a little red palm oil for fat. Five of the six nutrient constraints bind. Vitamin A is supplied at more than three times the requirement – a by-product of palm oil and greens – so complementary slackness forces its shadow price to zero. The binding nutrients have positive prices: an extra gram of protein in the requirement raises the least cost by about GHS 0.011, an extra milligram of iron by about GHS 0.031. Solving the dual LP explicitly reproduces the optimal cost (strong duality), and the reduced costs say how far each excluded food's price would have to fall before it enters the basket; eggs would have to become about GHS 2.87 per 100 g cheaper. Figure 9.5 traces the least cost as a function of the protein requirement:

the curve is piecewise linear and convex, and its slope at the current requirement is the protein shadow price.

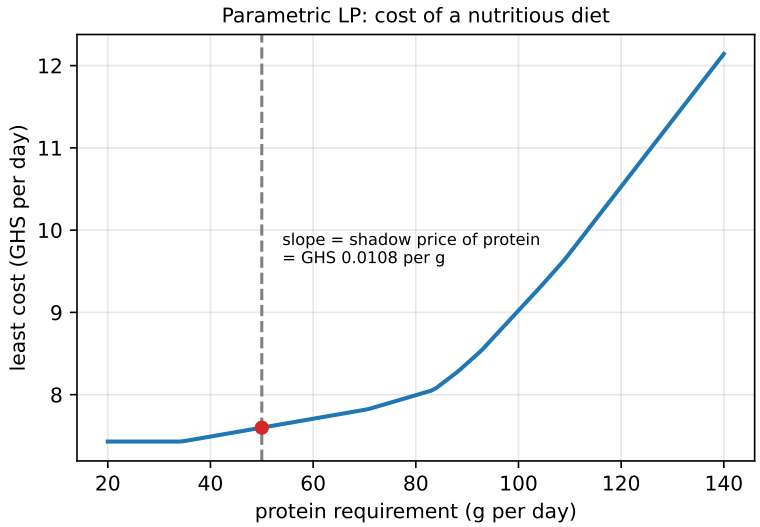


Figure 9.5: Parametric linear programming: least daily cost of the simulated diet as a function of the protein requirement. The function is convex and piecewise linear; each kink is a change of optimal basis, and the slope equals the shadow price of protein.

Policy Lens: The cost of a nutritious diet

International agencies now monitor the affordability of diets by solving exactly this kind of LP with local food prices and nutrient requirements: the least cost of a diet that meets energy needs, of one that meets all nutrient needs, and of a healthy diet meeting food-group recommendations (FAO et al., 2020). The ratio of the least cost to household food expenditure is a direct measure of whether a nutritious diet is within reach. The dual values add policy content: they identify the nutrients that drive the cost (here iron and protein) and thus the foods – or fortification and bio-fortification programmes – whose price reduction would make adequate diets cheaper. Shadow prices also show which requirements are "free" at current prices.

△ Caution: LP solutions are vertices

An LP optimum sits at a vertex of the feasible polytope, so the solution typically uses only as many foods as there are binding constraints, and it can jump discontinuously when a price changes. A diet of five foods that changes completely when the price of one food rises by 5% is not a credible recommendation. Report shadow prices and reduced costs alongside the basket, add realistic food-group and palatability constraints, and examine ranges of optimality. When the optimal basis is degenerate, the dual values are not unique and solvers may report any one of them.

Exercises

9.4.1 Theory Prove weak duality for the LP pair (9.11) directly from the constraints, without the Lagrangian, and use it to show part (iii) of Theorem 9.14.

9.4.2 Theory Derive the dual of the LP $\min \mathbf{c}^{\top}\mathbf{x}$ s.t. $\mathbf{A}\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, and show that the dual of the dual is the primal.

9.4.3 Theory Compute the Lagrange dual of $\min \tfrac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{c}^\top \mathbf{x}$ s.t. $\mathbf{A}\mathbf{x} \le \mathbf{b}$ with $\mathbf{Q}$ positive definite. Show that the dual is a quadratic programme in $\boldsymbol{\mu}$ with only the constraint $\boldsymbol{\mu} \ge \mathbf{0}$.

9.4.4 Python Modify Listing 9.6 to compute, for each food in the basket, the range of its price over which the optimal basket does not change (by re-solving on a grid). Which recommendation is most fragile?

9.4.5 Python Add food-group constraints (at least 100 g of vegetables, at most 60% of energy from starchy staples) to the diet LP. How much does the least cost rise, and how do the shadow prices change?

9.4.6 ★ Graduate For the 0–1 knapsack in Listing 9.5, prove that the Lagrangian dual bound equals the LP relaxation value. Then generate random knapsacks with $n = 10, 20, 50, 100$ items and plot the relative duality gap against n. Relate your findings to the Shapley–Folkman lemma.

9.4.7 Policy A school feeding programme must serve a lunch that provides at least one third of daily requirements at minimum cost per pupil. Discuss how you would obtain the price vector (market surveys, seasonality), which constraints you would add, and how the shadow prices could inform procurement and fortification decisions.

9.5 Constrained Estimation in Statistics and Machine Learning

Estimation is optimisation (Chapters 7 and 8), and much of statistics and machine learning is *constrained* estimation: economic theory restricts parameters, probabilities must sum to one, variances must be non-negative, and regularisation limits the size of coefficients.

9.5.1 Restricted least squares

Consider the linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ with $\mathbf{X}$ of full column rank k, and q linear restrictions $\mathbf{R}\boldsymbol{\beta} = \mathbf{r}$ with $\mathbf{R} \in \mathbb{R}^{q \times k}$ of full row rank. Restricted least squares solves

$$\min_{\boldsymbol{\beta}} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \quad \text{s.t.} \quad \mathbf{R}\boldsymbol{\beta} = \mathbf{r}.$$

With Lagrangian $\mathcal{L} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + \boldsymbol{\mu}^\top(\mathbf{R}\boldsymbol{\beta} - \mathbf{r})$, the first-order conditions are $-2\mathbf{X}^\top\mathbf{y} + 2\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} + \mathbf{R}^\top\boldsymbol{\mu} = \mathbf{0}$ and $\mathbf{R}\boldsymbol{\beta} = \mathbf{r}$. The first gives $\boldsymbol{\beta} = \hat{\boldsymbol{\beta}} - \tfrac{1}{2}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top\boldsymbol{\mu}$, where $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ is OLS; substituting into the constraint and solving for $\boldsymbol{\mu}$ yields

$$\hat{\boldsymbol{\beta}}_R = \hat{\boldsymbol{\beta}} - (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top\left[\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top\right]^{-1}(\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r}), \qquad \hat{\boldsymbol{\mu}} = 2\left[\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top\right]^{-1}(\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r}). \tag{9.12}$$

The correction is proportional to the amount $\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r}$ by which OLS violates the restriction. Because the objective is a convex quadratic and the constraints are linear, (9.12) is the global solution.

Proposition 9.15 (Properties of restricted least squares)**.** *Under* $\mathbb{E}[\boldsymbol{\varepsilon} \mid \mathbf{X}] = \mathbf{0}$ *and* $\operatorname{Var}(\boldsymbol{\varepsilon} \mid \mathbf{X}) = \sigma^2\mathbf{I}$*:*

1. *if the restrictions are true,* $\hat{\boldsymbol{\beta}}_R$ *is unbiased and* $\operatorname{Var}(\hat{\boldsymbol{\beta}}) - \operatorname{Var}(\hat{\boldsymbol{\beta}}_R) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top[\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top]^{-1}\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}$*, which is positive semi-definite;*
2. *if they are false,* $\hat{\boldsymbol{\beta}}_R$ *is biased, with bias* $-(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top[\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top]^{-1}(\mathbf{R}\boldsymbol{\beta} - \mathbf{r})$*;*
3. *the increase in the residual sum of squares is* $\mathrm{SSR}_R - \mathrm{SSR} = (\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r})^\top[\mathbf{R}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{R}^\top]^{-1}(\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r})$*, and under normal errors and a true null,*

$$F = \frac{(\mathrm{SSR}_R - \mathrm{SSR})/q}{\mathrm{SSR}/(n-k)} \sim F_{q,\,n-k}.$$

The multiplier has its usual meaning: by the envelope theorem $\partial \mathrm{SSR}_R / \partial \mathbf{r} = -\hat{\boldsymbol{\mu}}$, the rate at which the restricted fit would improve if the restriction were moved. A large multiplier relative to its sampling variability is evidence against the restriction; this is the idea behind the *Lagrange multiplier* (score) test. Listing 9.7 estimates a Cobb–Douglas production function on a simulated farm survey and tests constant returns to scale, $\beta_K + \beta_L + \beta_A = 1$.

Listing 9.7: Restricted least squares and the F test ch09/ch09_restricted_ls.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Restricted least squares: closed form, F test, check with SLSQP.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats
from scipy.optimize import minimize

rng = np.random.default_rng(803)
n = 200                                  # simulated farm survey
logK = rng.normal(1.0, 0.6, n)           # log capital
logL = 0.5 * logK + rng.normal(0.8, 0.5, n)  # log labour (correlated)
logA = rng.normal(0.0, 0.4, n)           # log land
X = np.column_stack([np.ones(n), logK, logL, logA])
beta_true = np.array([0.5, 0.35, 0.45, 0.20])  # constant returns: sum = 1
y = X @ beta_true + rng.normal(0, 0.3, n)

XtX_inv = np.linalg.inv(X.T @ X)
b = XtX_inv @ X.T @ y                    # unrestricted OLS


def restricted(R, r):
    """b_R = b - (X'X)^-1 R' [R (X'X)^-1 R']^-1 (R b - r)."""
    W = np.linalg.inv(R @ XtX_inv @ R.T)
    lam = W @ (R @ b - r)                # (scaled) Lagrange multipliers
    return b - XtX_inv @ R.T @ lam, lam


def f_test(R, r):
    bR, _ = restricted(R, r)
    ssr_u = np.sum((y - X @ b)**2)
    ssr_r = np.sum((y - X @ bR)**2)
    q, k = R.shape[0], X.shape[1]
    F = ((ssr_r - ssr_u) / q) / (ssr_u / (n - k))
    return F, stats.f.sf(F, q, n - k)


print("OLS (unrestricted):       ", np.round(b, 4), " sum of slopes = %.4f"
      % b[1:].sum())
R1, r1 = np.array([[0.0, 1, 1, 1]]), np.array([1.0])   # constant returns
bR, lam = restricted(R1, r1)
print("Restricted (CRS):         ", np.round(bR, 4), " sum of slopes = %.4f"
      % bR[1:].sum())

# numerical check: minimise SSR subject to R b = r with SLSQP
obj = lambda be: np.sum((y - X @ be)**2)
out = minimize(obj, x0=np.zeros(4), method="SLSQP", tol=1e-12,
               constraints=[{"type": "eq", "fun": lambda be: R1 @ be - r1}])
print("SLSQP check:              ", np.round(out.x, 4))
print("multiplier of the SSR problem (2*lam): %.4f" % (2 * lam[0]))
F, pv = f_test(R1, r1)
print("\nH0 constant returns (1 restriction):  F = %.3f   p = %.3f" % (F, pv))

R2 = np.array([[0.0, 1, 1, 1], [0.0, 1, -1, 0]])       # CRS and bK = bL
```

```python
r2 = np.array([1.0, 0.0])
F2, pv2 = f_test(R2, r2)
print("H0 CRS and bK = bL (2 restrictions): F = %.3f   p = %.4f" % (F2, pv2))
# variance reduction from imposing a true restriction
s2 = np.sum((y - X @ b)**2) / (n - 4)
V_u = s2 * XtX_inv
M = XtX_inv @ R1.T @ np.linalg.inv(R1 @ XtX_inv @ R1.T) @ R1
V_r = (np.eye(4) - M) @ V_u
print("\nstd errors unrestricted:", np.round(np.sqrt(np.diag(V_u)), 4))
print("std errors restricted:  ", np.round(np.sqrt(np.diag(V_r)), 4))
```

Output

```
OLS (unrestricted):        [0.5386 0.4005 0.403  0.255 ]  sum of slopes = 1.0585
Restricted (CRS):          [0.5615 0.3894 0.3924 0.2181]  sum of slopes = 1.0000
SLSQP check:               [0.5615 0.3894 0.3924 0.2181]
multiplier of the SSR problem (2*lam): 2.0008

H0 constant returns (1 restriction):  F = 0.709   p = 0.401
H0 CRS and bK = bL (2 restrictions): F = 0.355   p = 0.7014

std errors unrestricted: [0.0535 0.0368 0.0408 0.0548]
std errors restricted:   [0.046  0.0344 0.0388 0.033 ]
```

OLS gives returns to scale of 1.06; the restricted estimate imposes exactly 1 and agrees with SLSQP. The F test does not reject constant returns ($p = 0.40$), nor the joint restriction that capital and labour elasticities are equal. Imposing the true restriction reduces every standard error, most strongly for land, whose coefficient absorbs most of the adjustment.

9.5.2 Constrained maximum likelihood

More generally, maximise a log-likelihood $\ell(\boldsymbol{\theta})$ subject to $\mathbf{h}(\boldsymbol{\theta}) = \mathbf{0}$. The Lagrange conditions are $\nabla \ell(\tilde{\boldsymbol{\theta}}) = D\mathbf{h}(\tilde{\boldsymbol{\theta}})^{\top} \tilde{\boldsymbol{\lambda}}$: at the restricted estimate the score is not zero but is balanced by the multipliers. The *score* or *LM test* statistic $\nabla \ell(\tilde{\boldsymbol{\theta}})^{\top} \mathcal{I}(\tilde{\boldsymbol{\theta}})^{-1} \nabla \ell(\tilde{\boldsymbol{\theta}})$, asymptotically χ^2_q under the null, is a quadratic form in the multipliers (Casella and Berger, 2002, Section 10.3).

Example 9.16 (Multinomial probabilities)**.** With counts $n_1, \dots, n_K$, $\sum_k n_k = n$, maximise $\ell(\mathbf{p}) = \sum_k n_k \log p_k$ subject to $\sum_k p_k = 1$. The Lagrange conditions $n_k / p_k = \lambda$ give $p_k = n_k / \lambda$, and the constraint gives $\lambda = n$, so $\hat{p}_k = n_k / n$. The multiplier $\lambda = n$ is the marginal log-likelihood of relaxing $\sum p_k \le 1$.

△ Caution: Parameters on the boundary

Inequality constraints such as $\sigma^2 \ge 0$ or $\beta \ge 0$ create a problem for inference when the true parameter is on the boundary. The constrained MLE then has a non-normal limiting distribution (often a mixture with a point mass at the boundary), and the likelihood-ratio statistic is asymptotically a mixture of χ^2 distributions rather than χ^2_q. Testing for a zero variance component in a random-effects model is the standard example. Naive p-values are wrong in this setting; see Andrews (1999).

9.5.3 Ridge and lasso as constrained problems

Ridge regression and the lasso are usually written in *penalised* form,

$$\hat{\boldsymbol{\beta}}_{\alpha} = \arg\min_{\boldsymbol{\beta}} \; \tfrac{1}{2n} \lVert \mathbf{y} - \mathbf{X}\boldsymbol{\beta} \rVert^2 + \alpha \, P(\boldsymbol{\beta}), \qquad P(\boldsymbol{\beta}) = \tfrac{1}{2} \lVert \boldsymbol{\beta} \rVert_2^2 \ \text{(ridge)}, \ \ \lVert \boldsymbol{\beta} \rVert_1 \ \text{(lasso)},$$

but they were introduced in *constrained* form, $\min \frac{1}{2n}\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|^2$ s.t. $P(\boldsymbol{\beta}) \le t$ (Hoerl and Kennard, 1970; Tibshirani, 1996).

Theorem 9.17 (Penalised and constrained forms are equivalent)**.** *Let P be convex and $t > 0$ with $t < P(\hat{\boldsymbol{\beta}}_{\mathrm{OLS}})$ (so the constraint binds). If $\hat{\boldsymbol{\beta}}$ solves the constrained problem, there exists $\alpha \ge 0$ such that $\hat{\boldsymbol{\beta}}$ solves the penalised problem; conversely, if $\hat{\boldsymbol{\beta}}_\alpha$ solves the penalised problem with $\alpha > 0$, it solves the constrained problem with $t = P(\hat{\boldsymbol{\beta}}_\alpha)$.*

Proof. The constrained problem is convex and satisfies Slater's condition ($\boldsymbol{\beta} = \mathbf{0}$ is strictly feasible), so by Theorem 9.9 there is a KKT multiplier $\alpha \ge 0$ with $\mathbf{0} \in \nabla \frac{1}{2n}\left\|\mathbf{y}-\mathbf{X}\hat{\boldsymbol{\beta}}\right\|^2 + \alpha\, \partial P(\hat{\boldsymbol{\beta}})$ (a subgradient for the non-smooth ℓ_1 norm). This is precisely the optimality condition of the penalised problem, which is convex, so $\hat{\boldsymbol{\beta}}$ minimises it. Conversely, if $\hat{\boldsymbol{\beta}}_\alpha$ minimises the penalised objective and $\boldsymbol{\beta}$ satisfies $P(\boldsymbol{\beta}) \le P(\hat{\boldsymbol{\beta}}_\alpha)$, then $\frac{1}{2n}\left\|\mathbf{y}-\mathbf{X}\hat{\boldsymbol{\beta}}_\alpha\right\|^2 \le \frac{1}{2n}\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|^2 + \alpha[P(\boldsymbol{\beta}) - P(\hat{\boldsymbol{\beta}}_\alpha)] \le \frac{1}{2n}\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|^2$. □

The penalty parameter *is* the Lagrange multiplier of the size constraint. Its envelope interpretation is that α is the marginal reduction in training loss from enlarging the budget t for coefficient size. The geometry explains sparsity: the ℓ_1 ball has corners on the coordinate axes, so the first contact between a residual-sum-of-squares ellipse and the ball is often at a corner, where some coefficients are exactly zero; the ℓ_2 ball is smooth and gives shrinkage without selection (Hastie et al., 2009, Section 3.4). Listing 9.8 verifies the equivalence numerically on the diabetes data, solving the constrained lasso with SLSQP after splitting $\boldsymbol{\beta} = \mathbf{u} - \mathbf{v}$, $\mathbf{u}, \mathbf{v} \ge \mathbf{0}$, so that the ℓ_1 constraint becomes linear.

Listing 9.8: Penalised versus constrained ridge and lasso (plotting code omitted)
ch09/ch09_lasso_ridge.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Ridge and lasso: penalised form equals constrained form (diabetes data).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Lasso, lasso_path

X, y = load_diabetes(return_X_y=True)
X = (X - X.mean(0)) / X.std(0)
y = y - y.mean()
n, p = X.shape
rss = lambda b: np.sum((y - X @ b)**2) / (2 * n)
grad = lambda b: -X.T @ (y - X @ b) / n

# ---- ridge: penalised  min rss + (alpha/2)||b||^2 ------------------------
alpha = 5.0
b_pen = np.linalg.solve(X.T @ X / n + alpha * np.eye(p), X.T @ y / n)
t = b_pen @ b_pen                              # implied constraint level
con = {"type": "ineq", "fun": lambda b: t - b @ b, "jac": lambda b: -2 * b}
b_con = minimize(rss, np.zeros(p), jac=grad, method="SLSQP",
                 constraints=[con], tol=1e-14, options={"maxiter": 500}).x
mu = -(grad(b_con) @ b_con) / (2 * b_con @ b_con)  # from 0 = grad + 2 mu b
print("Ridge: alpha = %.1f  ->  t = ||b||^2 = %.3f" % (alpha, t))
print("  max |b_penalised - b_constrained| = %.2e" % np.abs(b_pen - b_con).max())
print("  recovered multiplier 2*mu = %.4f (equals alpha)" % (2 * mu))

# ---- lasso: penalised (sklearn) vs constrained ||b||_1 <= t -------------
alpha = 2.0
b_las = Lasso(alpha=alpha, fit_intercept=False, tol=1e-12,
              max_iter=100000).fit(X, y).coef_
t1 = np.abs(b_las).sum()
```

```
# split b = u - v with u, v >= 0 so that the l1 constraint is linear
f_uv = lambda z: rss(z[:p] - z[p:])
g_uv = lambda z: np.r_[grad(z[:p] - z[p:]), -grad(z[:p] - z[p:])]
con1 = {"type": "ineq", "fun": lambda z: t1 - z.sum(),
        "jac": lambda z: -np.ones(2 * p)}
z = minimize(f_uv, np.zeros(2 * p), jac=g_uv, method="SLSQP",
             bounds=[(0, None)] * (2 * p), constraints=[con1], tol=1e-14,
             options={"maxiter": 1000}).x
b_l1 = z[:p] - z[p:]
act = np.abs(b_l1) > 1e-6
mu1 = np.mean(-grad(b_l1)[act] / np.sign(b_l1[act]))
print("\nLasso: alpha = %.1f  ->  t = ||b||_1 = %.3f" % (alpha, t1))
print("  nonzero coefficients: sklearn %d, constrained %d" %
      ((np.abs(b_las) > 1e-6).sum(), act.sum()))
print("  max |b_sklearn - b_constrained| = %.2e" % np.abs(b_las - b_l1).max())
print("  multiplier from KKT on active set = %.4f (equals alpha)" % mu1)
print("  |grad_j| on zero coefficients <= alpha:",
      bool(np.all(np.abs(grad(b_l1)[~act]) <= alpha + 1e-4)))
```

Output

```
Ridge: alpha = 5.0  ->  t = ||b||^2 = 125.066
  max |b_penalised - b_constrained| = 3.56e-10
  recovered multiplier 2*mu = 5.0000 (equals alpha)

Lasso: alpha = 2.0  ->  t = ||b||_1 = 82.978
  nonzero coefficients: sklearn 7, constrained 7
  max |b_sklearn - b_constrained| = 2.05e-07
  multiplier from KKT on active set = 2.0000 (equals alpha)
  |grad_j| on zero coefficients <= alpha: True
```

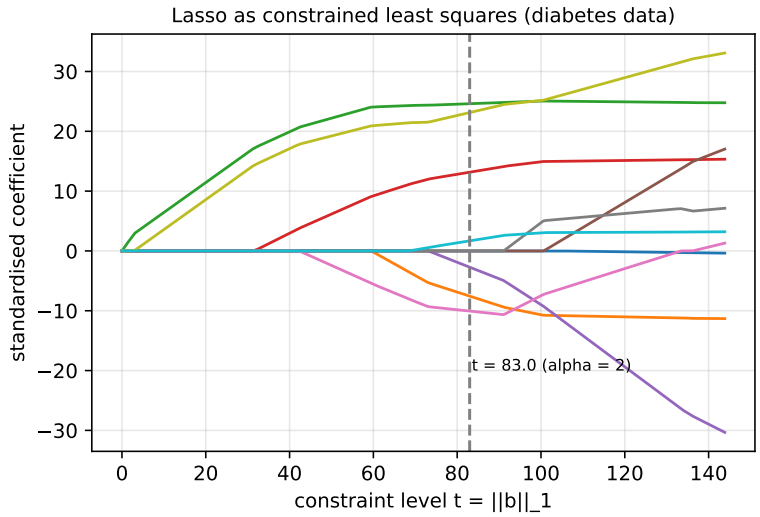


Figure 9.6: Lasso coefficient paths for the diabetes data plotted against the constraint level $t = \|\beta\|_1$. Coefficients enter one at a time as the constraint is relaxed; the dashed line marks the solution for $\alpha = 2$.

The recovered multipliers equal the penalty parameters, and on the inactive (zero) coefficients the KKT condition of the lasso, $|\nabla_j| \le \alpha$, holds – the subgradient form of complementary slackness.

⋆ Key Concept: Regularisation is constrained estimation

Every penalised estimator $\min L(\boldsymbol{\beta}) + \alpha P(\boldsymbol{\beta})$ with convex L, P is a constrained estimator $\min L(\boldsymbol{\beta})$ s.t. $P(\boldsymbol{\beta}) \le t$, and the penalty α is the Lagrange multiplier. Choosing α by cross-validation is choosing how much "coefficient budget" the model may spend.

9.5.4 The support vector machine

For labelled data $(\mathbf{x}_i, y_i)$, $y_i \in \{-1, 1\}$, the soft-margin SVM solves the quadratic programme

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1 - \xi_i, \;\; \xi_i \ge 0.$$

With multipliers $a_i \ge 0$ for the margin constraints and $r_i \ge 0$ for $\xi_i \ge 0$, stationarity gives $\mathbf{w} = \sum_i a_i y_i \mathbf{x}_i$, $\sum_i a_i y_i = 0$ and $a_i + r_i = C$. Substituting back gives the dual

$$\max_{\mathbf{a}} \sum_i a_i - \tfrac{1}{2}\sum_{i,j} a_i a_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \le a_i \le C, \;\; \sum_i a_i y_i = 0.$$

Complementary slackness classifies the observations: $a_i = 0$ for points strictly outside the margin, $0 < a_i < C$ for points exactly on the margin, and $a_i = C$ for points inside the margin or misclassified. Only the *support vectors* ($a_i > 0$) determine $\mathbf{w}$. The dual depends on the data only through inner products $\mathbf{x}_i^\top \mathbf{x}_j$, which is what allows the kernel trick (Cortes and Vapnik, 1995); see also Géron (2022, Chapter 5).

Listing 9.9: Soft-margin SVM through its dual (plotting code omitted) ch09/ch09_svm_dual.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Soft-margin linear SVM: solve the dual QP with SLSQP, compare with sklearn.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.svm import SVC

rng = np.random.default_rng(803)
n = 40                                          # simulated two-class data
X = np.r_[rng.normal([1.0, 1.0], 0.7, (n // 2, 2)),
          rng.normal([-1.0, -0.8], 0.7, (n // 2, 2))]
y = np.r_[np.ones(n // 2), -np.ones(n // 2)]
C = 1.0
Q = (y[:, None] * X) @ (y[:, None] * X).T       # Q_ij = y_i y_j x_i'x_j

# dual: max sum(a) - a'Qa/2  s.t.  0 <= a <= C,  y'a = 0
obj = lambda a: 0.5 * a @ Q @ a - a.sum()
jac = lambda a: Q @ a - 1
res = minimize(obj, np.zeros(n), jac=jac, method="SLSQP",
               bounds=[(0, C)] * n, tol=1e-12, options={"maxiter": 500},
               constraints=[{"type": "eq", "fun": lambda a: y @ a,
                             "jac": lambda a: y}])
a = np.where(res.x < 1e-7, 0.0, res.x)
w = (a * y) @ X                                 # primal from stationarity
free = (a > 1e-6) & (a < C - 1e-6)              # margin support vectors
b = np.mean(y[free] - X[free] @ w)

svc = SVC(kernel="linear", C=C, tol=1e-8).fit(X, y)
print("Dual SLSQP:  w = %s  b = %.4f" % (np.round(w, 4), b))
print("sklearn SVC: w = %s  b = %.4f" % (np.round(svc.coef_[0], 4),
                                         svc.intercept_[0]))
print("support vectors: dual %d (on margin %d, a = C %d), sklearn %d" %
```

```
      ((a > 1e-6).sum(), free.sum(), (a >= C - 1e-6).sum(),
       len(svc.support_)))
xi = np.maximum(0, 1 - y * (X @ w + b))       # slack variables
primal = 0.5 * w @ w + C * xi.sum()
print("primal value %.5f   dual value %.5f   gap %.1e" %
      (primal, -res.fun, primal + res.fun))
print("complementary slackness: max (C - a_i) xi_i = %.1e" %
      np.max((C - a) * xi))
```

Output

```
Dual SLSQP:  w = [1.3729 1.0222]  b = 0.0451
sklearn SVC: w = [1.3729 1.0222]  b = 0.0451
support vectors: dual 5 (on margin 3, a = C 2), sklearn 5
primal value 2.33409   dual value 2.33409   gap 4.2e-07
complementary slackness: max (C - a_i) xi_i = 0.0e+00
```

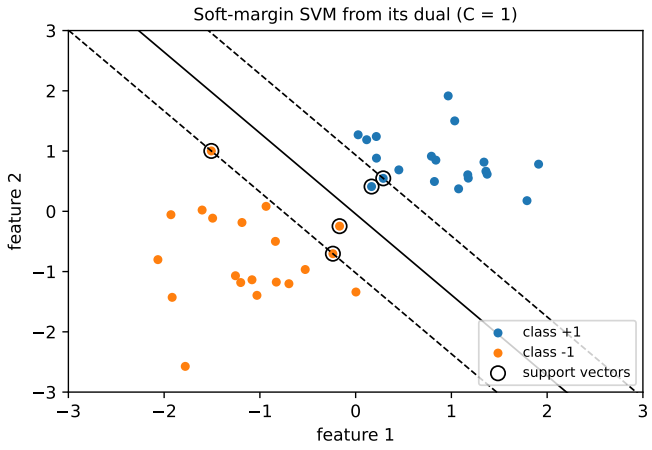


Figure 9.7: Simulated two-class data, the SVM decision boundary (solid) and margins (dashed) recovered from the dual solution. Circled points are support vectors.

9.5.5 Algorithms: projection, penalties and barriers

General-purpose solvers hide the mechanics. Three simple ideas underlie most constrained algorithms used in data science.

Projected gradient descent. For $\min_{\mathbf{x}\in C} f(\mathbf{x})$ with C closed and convex, iterate

$$\mathbf{x}_{k+1} = \Pi_C\big(\mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)\big), \qquad \Pi_C(\mathbf{z}) = \arg\min_{\mathbf{x}\in C} \|\mathbf{x} - \mathbf{z}\|,$$

the Euclidean projection. A point is optimal if and only if it is a fixed point of the map, which is the variational-inequality form of the KKT conditions. For convex f with L-Lipschitz gradient and $\eta = 1/L$, $f(\mathbf{x}_k) - f^* = O(1/k)$, and linear convergence holds under strong convexity – the same rates as unconstrained gradient descent (Chapter 8). The method is attractive when the projection is cheap: boxes, balls, the simplex (budget shares), and the set $\{\mathbf{x} \geq \mathbf{0},\ \mathbf{1}^\top \mathbf{x} \leq B\}$ used below, whose projection reduces to sorting (Duchi et al., 2008).

Penalty methods. Replace the constraints by a penalty, $\min f(\mathbf{x}) + \frac{\rho}{2}\big[\sum_k g_k(\mathbf{x})_+^2 + \sum_j h_j(\mathbf{x})^2\big]$, and let $\rho \to \infty$. The minimisers are infeasible for every finite ρ, with violation of order $1/\rho$,

and $\rho\, g_k(\mathbf{x}_\rho)_+$ converges to the KKT multiplier. Large ρ makes the problem ill-conditioned; augmented Lagrangian methods add a multiplier estimate to the penalty so that a moderate ρ suffices.

Barrier (interior-point) methods. For inequality constraints, minimise $t\, f(\mathbf{x}) - \sum_k \log(-g_k(\mathbf{x}))$ for an increasing sequence of t. The minimisers $\mathbf{x}^*(t)$ trace the *central path*; each is strictly feasible, and the quantities $\mu_k(t) = -1/(t\, g_k(\mathbf{x}^*(t)))$ are dual feasible, which yields the bound $f(\mathbf{x}^*(t)) - p^* \le p/t$ with p the number of inequalities (Boyd and Vandenberghe, 2004, Section 11.2). This is the idea behind modern interior-point solvers, including the one inside HiGHS.

Listing 9.10 implements all three for a non-negative, budget-constrained least-squares problem and compares them with SLSQP; Fig. 9.8 shows their convergence.

Listing 9.10: Projected gradient, penalty and barrier methods (plotting code omitted)
ch09/ch09_projected_gradient.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Projected gradient, quadratic penalty and log-barrier methods compared.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

rng = np.random.default_rng(803)
m, n = 30, 5
A = rng.normal(size=(m, n))
bvec = A @ np.array([0.6, -0.2, 0.5, 0.4, 0.1]) + rng.normal(0, 0.1, m)
B = 1.0                          # feasible set: x >= 0, sum(x) <= B
f = lambda x: 0.5 * np.sum((A @ x - bvec)**2)
df = lambda x: A.T @ (A @ x - bvec)
ref = minimize(f, np.full(n, 0.1), jac=df, method="SLSQP", tol=1e-14,
               bounds=[(0, None)] * n,
               constraints=[{"type": "ineq", "fun": lambda x: B - x.sum()}])
fstar = ref.fun


def project(v):
    """Euclidean projection onto {x >= 0, sum x <= B}."""
    w = np.maximum(v, 0)
    if w.sum() <= B:
        return w
    u = np.sort(v)[::-1]                      # simplex projection (sorting)
    css = np.cumsum(u) - B
    k = np.nonzero(u - css / np.arange(1, n + 1) > 0)[0][-1]
    return np.maximum(v - css[k] / (k + 1), 0)


L = np.linalg.eigvalsh(A.T @ A).max()        # Lipschitz constant of grad f
x = np.zeros(n)
gap_pg = []
for it in range(200):
    x = project(x - df(x) / L)
    gap_pg.append(f(x) - fstar)
print("reference (SLSQP) x* =", np.round(ref.x, 4), " f* = %.6f" % fstar)
print("projected gradient x =", np.round(x, 4), " gap after 200 it = %.1e"
      % gap_pg[-1])

# quadratic penalty: f + rho/2 [ sum(min(x,0)^2) + max(sum x - B, 0)^2 ]
print("\n   rho    penalty-method x                      violation")
for rho in [1e0, 1e1, 1e2, 1e3, 1e4]:
    pen = lambda x: f(x) + rho / 2 * (np.sum(np.minimum(x, 0)**2) +
```

```
                                    max(x.sum() - B, 0)**2)
    xp = minimize(pen, np.zeros(n), method="BFGS", tol=1e-12).x
    viol = max(xp.sum() - B, 0) + np.abs(np.minimum(xp, 0)).sum()
    print("  %6.0e  %s  %.2e" % (rho, np.round(xp, 4), viol))

# log barrier: t f(x) - sum log x_i - log(B - sum x), Newton with damping
H0 = A.T @ A
x = np.full(n, B / (n + 1))                  # strictly feasible start
print("\n      t     f(x_t) - f*    bound (n+1)/t")
gap_bar, ts = [], []
for t in [1, 10, 100, 1e3, 1e4, 1e5]:
    for _ in range(50):
        s = B - x.sum()
        g = t * df(x) - 1 / x + 1 / s
        H = t * H0 + np.diag(1 / x**2) + 1 / s**2
        d = -np.linalg.solve(H, g)
        step = 1.0
        while np.any(x + step * d <= 0) or (x + step * d).sum() >= B:
            step *= 0.5                      # stay strictly inside
        x = x + step * d
        if np.abs(g @ d) < 1e-12:
            break
    ts.append(t)
    gap_bar.append(f(x) - fstar)
    print("  %6.0e   %.3e      %.1e" % (t, gap_bar[-1], (n + 1) / t))
```

Output

```
reference (SLSQP) x* = [0.4115 0.     0.3143 0.2318 0.0424]  f* = 2.171258
projected gradient x = [0.4115 0.     0.3143 0.2318 0.0424]  gap after 200 it = 5.7e-13

   rho    penalty-method x                       violation
   1e+00  [ 0.5516 -0.2233  0.4822  0.4019  0.0873]  5.23e-01
   1e+01  [ 0.5176 -0.2104  0.4248  0.3558  0.0674]  3.66e-01
   1e+02  [ 0.4459 -0.0751  0.3473  0.2712  0.049 ]  1.14e-01
   1e+03  [ 0.4159 -0.0099  0.3185  0.2369  0.0433]  1.47e-02
   1e+04  [ 0.4119 -0.001   0.3147  0.2323  0.0425]  1.51e-03

      t     f(x_t) - f*    bound (n+1)/t
   1e+00   1.385e+00      6.0e+00
   1e+01   2.007e-01      6.0e-01
   1e+02   2.027e-02      6.0e-02
   1e+03   2.004e-03      6.0e-03
   1e+04   2.000e-04      6.0e-04
   1e+05   2.000e-05      6.0e-05
```

The penalty method approaches the solution from outside – the second coordinate is negative for every ρ – while the barrier method approaches from inside. In the barrier output the gap is almost exactly $2/t$: only the two active constraints ($x_2 \geq 0$ and $\mathbf{1}^\top \mathbf{x} \leq 1$) contribute, since the inactive constraints have multipliers tending to zero.

Graduate Perspective: ADMM and proximal methods

Projected gradient is the special case of the *proximal gradient* method in which the non-smooth term is the indicator function of C. Replacing the projection by the soft-thresholding operator gives ISTA for the lasso, and splitting a problem into pieces coordinated by multipliers gives the alternating direction method of multipliers (ADMM), widely used for large-scale and distributed statistical learning (Boyd et al., 2011). All of these are, at heart, algorithms for

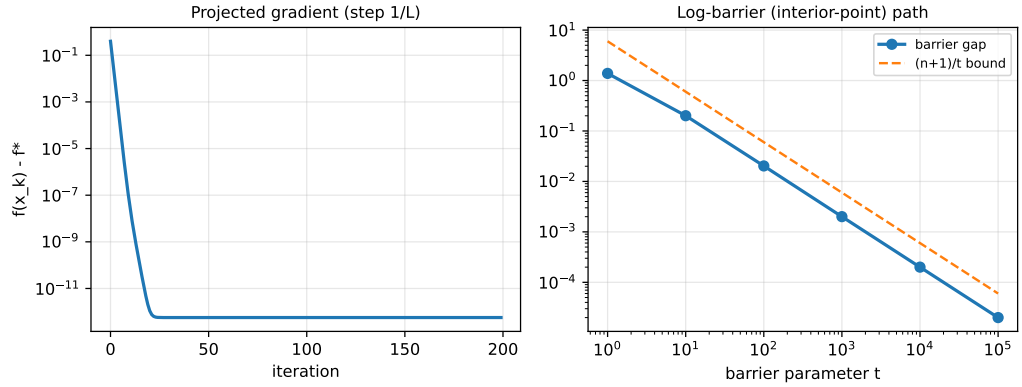


Figure 9.8: Left: projected gradient converges linearly on this strongly convex problem. Right: along the barrier path the optimality gap falls like $1/t$, below the theoretical bound $(n+1)/t$; the actual gap is close to $2/t$ because two constraints are active at the solution.

finding a saddle point of a Lagrangian.

Exercises

9.5.1 Theory Derive (9.12) in full and prove part 1 of Proposition 9.15. Show also that $\hat{\boldsymbol{\beta}}_R$ can be obtained by OLS on a transformed model after substituting the restrictions.

9.5.2 Theory Show that the projection onto $C = \{\mathbf{x} : \mathbf{x} \geq \mathbf{0}, \mathbf{1}^\top \mathbf{x} \leq B\}$ is $\Pi_C(\mathbf{z}) = (\mathbf{z} - \tau \mathbf{1})_+$ for a scalar $\tau \geq 0$, and characterise τ using KKT conditions.

9.5.3 Theory For ridge regression, find the multiplier α as a function of t using the singular value decomposition of $\mathbf{X}$, and show that $t \mapsto \alpha(t)$ is decreasing.

9.5.4 Python Extend Listing 9.7 with a Monte Carlo study: under a true restriction, compare the sampling variances of $\hat{\boldsymbol{\beta}}$ and $\hat{\boldsymbol{\beta}}_R$; under a false restriction ($\sum \beta = 1.1$) compare their mean squared errors as n grows. When is imposing a false restriction still worthwhile?

9.5.5 Python Implement ISTA for the lasso and check it against `sklearn`'s `Lasso` on the diabetes data. Then implement the augmented Lagrangian method for the problem in Listing 9.10 and compare the number of iterations needed with the pure penalty method.

9.5.6 ★ Graduate Prove the barrier bound $f(\mathbf{x}^*(t)) - p^* \leq p/t$ by showing that $\mu_k(t) = -1/(t\, g_k(\mathbf{x}^*(t)))$ is dual feasible and computing the corresponding duality gap.

9.5.7 Policy A ministry wants to estimate a demand system for food groups from a household survey and to use it for price-policy simulations. Which restrictions from consumer theory (adding up, homogeneity, symmetry) would you impose, how would you test them, and what would you do if the survey rejects them?

9.6 Policy Optimisation Applications

This section assembles the tools of the chapter into four stylised policy problems and ends with the question every policy model must face: how sensitive are its recommendations to the parameters fed into it? All data are simulated.

9.6.1 Allocating a fixed budget across programmes

A district has a development budget B to divide among n programmes. Programme i produces an outcome $f_i(x_i)$ (for instance, a weighted index of learning, health and yield gains) that is increasing and strictly concave in spending – returns diminish as the easiest gains are exhausted. The planner solves

$$\max_{\mathbf{x}} \sum_{i=1}^{n} f_i(x_i) \quad \text{s.t.} \quad \sum_i x_i \le B, \;\; x_i \ge 0. \tag{9.13}$$

This is a convex problem (maximising a concave function over a polytope) and Slater's condition holds for $B > 0$, so the KKT conditions are necessary and sufficient. They read: $f_i'(x_i) \le \lambda$ with equality if $x_i > 0$, $\lambda \ge 0$, and $\lambda = 0$ unless the budget is exhausted.

Proposition 9.18 (Equalise marginal returns)**.** *If each f_i is strictly concave and increasing, the solution of (9.13) uses the whole budget and is $x_i^* = \max\{0, (f_i')^{-1}(\lambda^*)\}$, where $\lambda^* > 0$ is the unique number with $\sum_i x_i^* = B$. Programme i is funded if and only if its marginal return at zero exceeds the shadow value of the budget, $f_i'(0) > \lambda^*$. As B grows, λ^* falls and programmes enter in decreasing order of $f_i'(0)$.*

For $f_i(x) = a_i \log(1 + x/b_i)$, $f_i'(x) = a_i/(b_i + x)$ and $x_i^* = \max\{0, a_i/\lambda^* - b_i\}$. The left-hand side $\sum_i x_i^*(\lambda)$ is continuous and strictly decreasing in λ wherever positive, so λ^* can be found by bisection – a "water-filling" algorithm: imagine pouring the budget into vessels whose floors are at heights b_i and whose widths are proportional to a_i. Listing 9.11 solves a five-programme example.

Listing 9.11: Budget allocation with diminishing returns (plotting code omitted)
ch09/ch09_budget_allocation.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# District budget allocation across programmes: KKT "water filling".
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq, minimize

# Simulated returns: outcome_i(x) = a_i log(1 + x_i / b_i), x in GHS million
names = ["education", "health", "agric. ext.", "water/sanit.", "feeder roads"]
a = np.array([3.0, 4.0, 2.0, 2.5, 1.5])
b = np.array([1.0, 2.0, 0.5, 1.5, 0.6])
first = a / b                                   # marginal return at x_i = 0


def allocate(B):
    """KKT: x_i = max(0, a_i/lam - b_i) with sum x_i = B (bisection)."""
    lam = brentq(lambda L: np.maximum(a / L - b, 0).sum() - B, 1e-9,
                 first.max(), xtol=1e-14)
    return np.maximum(a / lam - b, 0), lam


B0 = 1.8
x, lam = allocate(B0)
obj = lambda z: -np.sum(a * np.log1p(z / b))
res = minimize(obj, np.full(5, B0 / 5), method="SLSQP", tol=1e-12,
               bounds=[(0, None)] * 5,
               constraints=[{"type": "ineq", "fun": lambda z: B0 - z.sum()}])
print("Budget B = GHS %.1f million (simulated returns)" % B0)
print("  programme     a_i/b_i   x_i(KKT)  x_i(SLSQP)  marg. return")
for i in range(5):
    print("  %-13s %7.2f  %8.3f  %9.3f   %8.4f" %
```

```
          (names[i], first[i], x[i], res.x[i], a[i] / (b[i] + x[i])))
print("  lambda = %.4f: programmes funded iff a_i/b_i > lambda" % lam)
print("  outcome index V(B) = %.4f" % -obj(x))

# expansion path: who is funded as the budget grows
Bs = np.linspace(0.01, 15, 300)
path = np.array([allocate(B)[0] for B in Bs])
lams = np.array([allocate(B)[1] for B in Bs])
print("\nentry budgets (B at which each programme starts):")
for i in np.argsort(-first):
    Bi = np.maximum(a / first[i] - b, 0).sum()
    print("  %-13s B = %.3f" % (names[i], Bi))
```

Output

```
Budget B = GHS 1.8 million (simulated returns)
  programme     a_i/b_i   x_i(KKT)  x_i(SLSQP)  marg. return
  education        3.00      0.686       0.686     1.7797
  health           2.00      0.248       0.248     1.7797
  agric. ext.      4.00      0.624       0.624     1.7797
  water/sanit.     1.67      0.000       0.000     1.6667
  feeder roads     2.50      0.243       0.243     1.7797
  lambda = 1.7797: programmes funded iff a_i/b_i > lambda
  outcome index V(B) = 4.1630

entry budgets (B at which each programme starts):
  agric. ext.   B = 0.000
  education     B = 0.167
  feeder roads  B = 0.500
  health        B = 1.150
  water/sanit.  B = 2.200
```

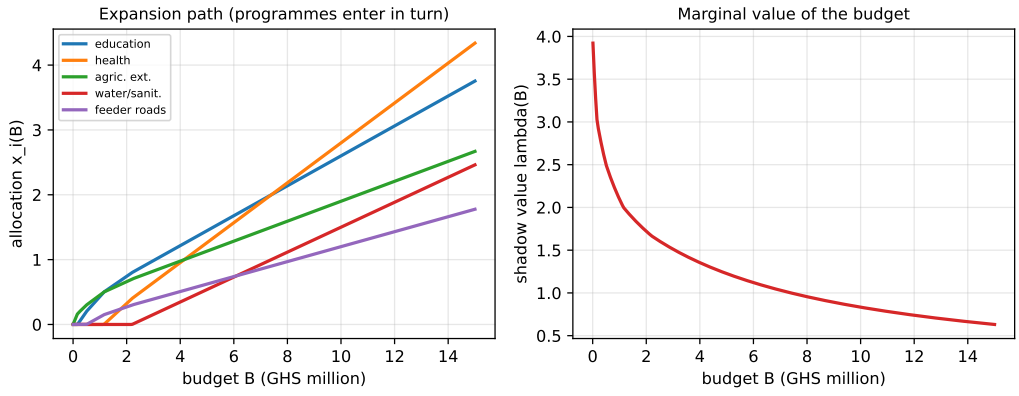


Figure 9.9: Left: optimal allocations as functions of the budget. Programmes enter in decreasing order of their marginal return at zero, a_i/b_i. Right: the shadow value of the budget $\lambda(B)$ falls as the budget grows; kinks occur where a new programme enters.

With $B = 1.8$, water and sanitation is not funded: its marginal return at zero, $a_4/b_4 = 1.67$, is below the shadow value $\lambda = 1.78$ achieved by the other four programmes. It enters only when the budget exceeds GHS 2.2 million. Note that the programme with the largest *total* potential (health, $a_2 = 4$) is not the first to be funded; what matters at the margin is the return on the next cedi.

⋆ Key Concept: Equalise the margin

In a convex allocation problem the KKT conditions say: every funded use of a scarce resource yields the same marginal return, equal to the resource's shadow price, and every unfunded use yields less at zero. The same logic, with different objective functions, gives water-filling budgets, bottom-up transfers, equal post-transfer poverty rates across funded regions and the Ramsey rule.

Policy Lens: Budget allocation in a District Assembly

District Assemblies allocate their share of the Common Fund and their internally generated funds across education, health, agriculture, water and roads. The KKT conditions translate into a discipline that is easy to state and hard to practise: fund a programme up to the point where its marginal outcome per cedi equals that of every other funded programme, and do not fund at all a programme whose *first* cedi is worth less than the common margin. Equal shares across departments, or allocations proportional to last year's, satisfy this only by accident. The analysis also shows what the district would need to know – marginal, not average, returns – and the multiplier λ gives a benchmark against which proposals for new spending (or the value of an extra transfer from the centre) can be judged.

9.6.2 The social planner with equity weights

Suppose now that the budget is spent on per-capita transfers $T_i \geq 0$ to n regional groups with populations N_i and pre-transfer incomes y_i. A planner with social welfare function

$$W(\mathbf{t}) = \sum_i N_i \, \omega_i \, u(y_i + T_i), \qquad u(c) = \frac{c^{1-\eta}}{1-\eta} \ (\eta \neq 1), \quad u(c) = \log c \ (\eta = 1),$$

chooses transfers subject to $\sum_i N_i T_i = B$. Here $\eta \geq 0$ is the planner's inequality aversion (as in the Atkinson index) and $\omega_i > 0$ are *equity weights*, which can reflect deprivation not captured by income, historical neglect or political priorities. The KKT conditions

$$\omega_i (y_i + T_i)^{-\eta} \leq \lambda, \quad \text{with equality if } T_i > 0,$$

say that weighted marginal utilities are equalised among recipients and that no group whose weighted marginal utility is below λ receives anything. Recipients end at consumption $c_i = (\omega_i/\lambda)^{1/\eta}$: transfers "fill from the bottom".

Listing 9.12: Equity-weighted transfers ch09/ch09_social_planner.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Social planner: transfers with equity weights and inequality aversion.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy.optimize import brentq, minimize

# Five simulated regional groups: pre-transfer income (GHS per capita/month)
groups = ["North", "Upper", "Middle", "Coast", "Capital"]
y = np.array([320.0, 380.0, 520.0, 650.0, 900.0])
pop = np.array([1.2, 0.9, 2.0, 1.5, 2.4])          # millions of people
Bud = 300.0                                        # GHS million per month


def uprime(c, eta):
    return c**(-eta)

```

```python

def plan(w, eta):
    """max sum pop_i w_i u(y_i + T_i)  s.t.  sum pop_i T_i = Bud, T >= 0.
    KKT: w_i u'(y_i + T_i) = lam if T_i > 0 (fill from the bottom)."""
    def spend(lam):
        c = (lam / w)**(-1 / eta)                  # inverse of w u'(c) = lam
        return np.sum(pop * np.maximum(c - y, 0)) - Bud
    lam = brentq(spend, 1e-12, np.max(w * uprime(y, eta)))
    T = np.maximum((lam / w)**(-1 / eta) - y, 0)
    return T, lam


print("Budget GHS %.0f million; transfers T_i in GHS per capita" % Bud)
print("  weights      eta    " + "  ".join("%7s" % g for g in groups))
for wname, w in [("utilitarian", np.ones(5)),
                 ("equity", np.array([1.5, 1.3, 1.0, 1.0, 0.8]))]:
    for eta in [0.5, 1.0, 2.0]:
        T, lam = plan(w, eta)
        print("  %-11s %4.1f    " % (wname, eta) +
              "  ".join("%7.1f" % t for t in T))

# numerical check of one case with SLSQP (log utility, equity weights)
w = np.array([1.5, 1.3, 1.0, 1.0, 0.8])
T, lam = plan(w, 1.0)
W = lambda t: -np.sum(pop * w * np.log(y + t))
r = minimize(W, np.full(5, Bud / pop.sum()), method="SLSQP", tol=1e-12,
             bounds=[(0, None)] * 5,
             constraints=[{"type": "eq", "fun": lambda t: pop @ t - Bud}])
print("\nSLSQP check (equity, eta=1):", np.round(r.x, 1))
print("multiplier lambda = %.6f = marginal social welfare of GHS 1 million"
      % lam)
print("weighted marginal utilities w_i/(y_i+T_i):",
      np.round(w / (y + T), 6))
```

Output

```
Budget GHS 300 million; transfers T_i in GHS per capita
  weights      eta     North    Upper   Middle    Coast  Capital
  utilitarian  0.5     168.6    108.6      0.0      0.0      0.0
  utilitarian  1.0     168.6    108.6      0.0      0.0      0.0
  utilitarian  2.0     168.6    108.6      0.0      0.0      0.0
  equity       0.5     226.9     30.8      0.0      0.0      0.0
  equity       1.0     198.2     69.1      0.0      0.0      0.0
  equity       2.0     183.5     88.7      0.0      0.0      0.0

SLSQP check (equity, eta=1): [198.2  69.1   0.    0.    0. ]
multiplier lambda = 0.002895 = marginal social welfare of GHS 1 million
weighted marginal utilities w_i/(y_i+T_i): [0.002895 0.002895 0.001923 0.001538 0.000889]
```

Two lessons emerge. With equal weights the solution does not depend on η at all: any strictly concave u implies that the poorest group is raised first until it reaches the next poorest, and so on. Inequality aversion matters only once the planner uses non-uniform weights, and then a higher η *reduces* the influence of the weights, because $c_i \propto \omega_i^{1/\eta}$. The multiplier λ is the marginal social welfare of the budget; its value depends on the normalisation of u and the weights, so it is comparable across budgets for the same welfare function but not across welfare functions.

9.6.3 Targeting transfers to minimise the poverty gap

The poverty gap index of Foster et al. (1984) is $P_1 = \frac{1}{n}\sum_i \max\{0, z - y_i\}/z$, where z is the poverty line. If a programme could observe every household's income, minimising P_1 with a budget B would be trivial: any allocation that gives money only to the poor and never lifts anyone above z reduces the total gap by exactly B. In practice incomes are not observed, and the agency can target only on observable characteristics such as region (Kanbur, 1987; Ravallion and Chao, 1989). With G regions and a uniform transfer t_g to every household in region g, the problem

$$\min_{\mathbf{t},\mathbf{s}} \sum_{i=1}^{n} s_i \quad \text{s.t.} \quad s_i \ge z - y_i - t_{g(i)}, \;\; s_i \ge 0, \;\; t_g \ge 0, \;\; \sum_{g} n_g t_g \le B$$

is a linear programme: the auxiliary variable s_i is household i's post-transfer poverty gap. It has $n + G$ variables and $n + 1$ inequality constraints, which HiGHS solves in a fraction of a second for a survey of a few thousand households.

Listing 9.13: Poverty-gap-minimising regional targeting (plotting code omitted)
ch09/ch09_poverty_targeting.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Poverty-gap-minimising regional transfers as a linear programme.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
from scipy.sparse import csr_matrix, hstack, identity

rng = np.random.default_rng(803)
G, n = 6, 3000                                  # regions, simulated households
region = rng.integers(0, G, n)
mu = np.log([180, 230, 300, 380, 470, 600])  # regional log-income means
y = np.exp(mu[region] + rng.normal(0, 0.55, n))   # GHS per adult per month
z = 300.0                                       # poverty line (simulated)


def pgap(inc):
    return np.mean(np.maximum(z - inc, 0)) / z      # FGT(1) index


def best_regional(Bud):
    """min sum_i s_i  s.t. s_i >= z - y_i - t_g(i), s >= 0, t >= 0,
    sum_i t_g(i) <= Bud.  Variables: (t_1..t_G, s_1..s_n)."""
    D = csr_matrix((np.ones(n), (np.arange(n), region)), shape=(n, G))
    A1 = hstack([-D, -identity(n)])          # -t_g - s_i <= y_i - z
    cnt = np.bincount(region, minlength=G).astype(float)
    A2 = csr_matrix(np.r_[cnt, np.zeros(n)][None, :])
    res = linprog(np.r_[np.zeros(G), np.ones(n)],
                  A_ub=hstack([A1.T, A2.T]).T.tocsr(),
                  b_ub=np.r_[y - z, Bud], bounds=[(0, None)] * (G + n),
                  method="highs")
    return res.x[:G], res, cnt


Bud = 60.0 * n                                  # GHS 60 per household on average
t, res, cnt = best_regional(Bud)
print("Simulated survey: n = %d households, poverty line z = %.0f" % (n, z))
print("  headcount = %.3f   poverty gap index = %.4f" %
      (np.mean(y < z), pgap(y)))
print("  budget = GHS %.0f (GHS %.0f per household)" % (Bud, Bud / n))
print("\n  region  mean income  poor share  optimal transfer t_g")
```

```
for g in range(G):
    sel = region == g
    print("  %4d   %10.0f   %9.3f   %10.1f" %
          (g + 1, y[sel].mean(), np.mean(y[sel] < z), t[g]))
print("\n  poverty gap index: universal %.4f  regional LP %.4f" %
      (pgap(y + Bud / n), res.fun / n / z))
lam = -res.ineqlin.marginals[-1]
print("  shadow price of budget: total gap falls by %.3f per GHS 1" % lam)
print("  (perfect targeting would give 1.000 until the gap is closed)")

budgets = np.linspace(0, 250, 26) * n
uni = [pgap(y + Bb / n) for Bb in budgets]
reg = [best_regional(Bb)[1].fun / n / z for Bb in budgets]
perfect = np.maximum(pgap(y) - budgets / n / z, 0)
```

Output

```
Simulated survey: n = 3000 households, poverty line z = 300
  headcount = 0.446   poverty gap index = 0.1636
  budget = GHS 180000 (GHS 60 per household)

  region  mean income  poor share  optimal transfer t_g
     1          203       0.840        166.0
     2          254       0.723        124.8
     3          350       0.505         71.5
     4          459       0.316          0.0
     5          580       0.177          0.0
     6          657       0.133          0.0

  poverty gap index: universal 0.0861  regional LP 0.0524
  shadow price of budget: total gap falls by 0.331 per GHS 1
  (perfect targeting would give 1.000 until the gap is closed)
```

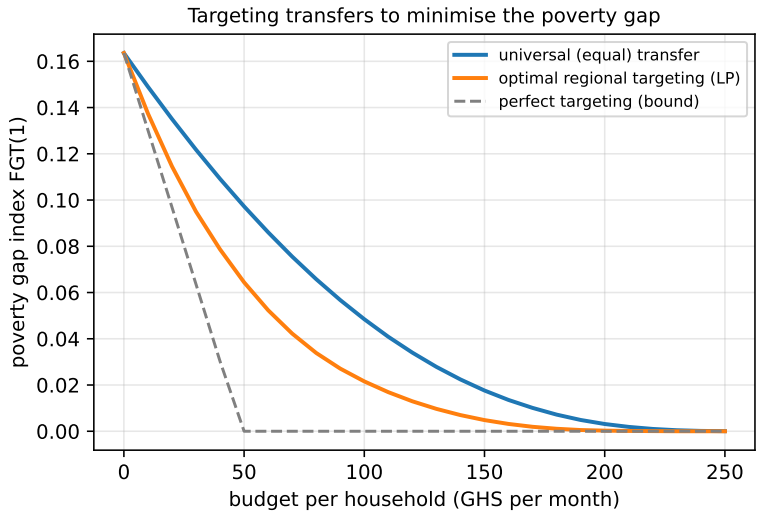


Figure 9.10: Poverty gap index of a simulated survey as a function of the budget per household, for a universal transfer, the LP-optimal regional transfers, and the (unattainable) perfect-information bound. Regional targeting captures a substantial part of the gain from perfect targeting.

The optimal policy concentrates the budget in the three poorest regions and gives the poorest region the largest transfer. The KKT conditions explain the pattern. Raising t_g by one cedi reduces the total gap by the number of households in g that are still poor after the transfer, at

a cost of n_g cedis. At the optimum, therefore, *the share of households still poor after the transfer is equal across all funded regions, and equal to the shadow price* λ; unfunded regions are those whose poverty rate is already below λ. In the simulation $\lambda = 0.33$: about one third of households in each funded region remain poor, and each extra cedi of budget reduces the aggregate gap by GHS 0.33, compared with GHS 1 under perfect targeting. That difference is the cost of imperfect information, and it puts an upper bound on what it is worth paying for better targeting data such as a proxy-means test.

Policy Lens: Poverty-gap-minimising transfers

LP formulations of targeting make the trade-offs of social protection design explicit. The shadow price of the budget measures the poverty reduction per cedi; the shadow prices of additional constraints – a minimum transfer in every region for political reasons, a cap on any one region's share – measure what those constraints cost in poverty terms. A ministry can then report, for example, that a regional floor costs a certain number of cedis of poverty-gap reduction per cedi of budget, and let the political process decide whether it is worth it. The squared poverty gap P_2, which weights the poorest more heavily, leads to a convex (quadratic) rather than a linear programme and pushes transfers further towards the poorest regions.

9.6.4 A Ramsey-style taxation problem

A government must raise revenue R through commodity taxes t_i on goods with producer prices normalised to one and independent constant-elasticity demands $x_i(q_i) = A_i q_i^{-e_i}$, $q_i = 1 + t_i$. The welfare loss to consumers is the loss of consumer surplus $\sum_i \int_1^{1+t_i} x_i(q)\,dq$. The Ramsey problem is

$$\min_{\mathbf{t}\geq\mathbf{0}} \sum_i \int_1^{1+t_i} x_i(q)\,dq \quad \text{s.t.} \quad \sum_i t_i x_i(1+t_i) \geq R.$$

With multiplier μ on the revenue constraint, the first-order condition for t_i is $x_i = \mu\,(x_i + t_i x_i')$: the marginal welfare cost of raising the tax equals μ times its marginal revenue. Using $x_i' = -e_i x_i / q_i$,

$$\frac{t_i}{1+t_i} = \frac{k}{e_i}, \qquad k = 1 - \frac{1}{\mu}, \tag{9.14}$$

the *inverse elasticity rule*: tax rates (as a share of the consumer price) are inversely proportional to demand elasticities. The multiplier $\mu = 1/(1-k) > 1$ is the *marginal cost of public funds*: by the envelope theorem, raising one more cedi of revenue costs consumers μ cedis.

Listing 9.14: Ramsey taxes and the marginal cost of public funds ch09/ch09_ramsey.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Ramsey toy problem: least-distortion commodity taxes for a revenue target.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy.optimize import brentq, minimize

goods = ["staple food", "fuel", "telecom", "beverages"]
A = np.array([100.0, 60.0, 50.0, 40.0])       # demand scale at price 1
e = np.array([0.3, 0.6, 1.2, 1.8])            # own-price elasticities
R0 = 30.0                                     # revenue target


def demand(t):                                # producer prices equal 1
    return A * (1 + t)**(-e)

```

```

def cs_loss(t):                                # loss of consumer surplus
    q = 1 + t
    return np.sum(A * (q**(1 - e) - 1) / (1 - e))


revenue = lambda t: np.sum(t * demand(t))
res = minimize(cs_loss, np.full(4, 0.1), method="SLSQP", tol=1e-13,
               bounds=[(0, None)] * 4,
               constraints=[{"type": "ineq", "fun": lambda t: revenue(t) - R0}])
t = res.x
print("Ramsey taxes for revenue R0 = %.0f (simulated demands)" % R0)
print("  good          elast.   tax rate   t/(1+t)   e * t/(1+t)")
for g, ei, ti in zip(goods, e, t):
    print("  %-12s %6.2f   %8.4f   %7.4f   %9.4f" %
          (g, ei, ti, ti / (1 + ti), ei * ti / (1 + ti)))
print("  inverse elasticity rule: e_i t_i/(1+t_i) is the same for all i")
eb_r = cs_loss(t) - revenue(t)

# uniform tax raising the same revenue
tu = brentq(lambda s: revenue(np.full(4, s)) - R0, 0, 2)
eb_u = cs_loss(np.full(4, tu)) - R0
print("\nexcess burden: Ramsey %.4f   uniform rate %.4f (t = %.4f)" %
      (eb_r, eb_u, tu))

# multiplier = marginal cost of public funds (MCPF)
mu = demand(t) / (demand(t) * (1 - e * t / (1 + t)))   # dLoss/dt / dRev/dt
print("ratio dLoss/dt_i / dRev/dt_i (all equal mu):", np.round(mu, 4))


def ramsey_exact(R):
    """t_i/(1+t_i) = k/e_i  =>  t_i = k/(e_i - k); choose k to hit R."""
    k = brentq(lambda k: revenue(k / (e - k)) - R, 0, 0.99 * e.min())
    return k / (e - k)


h = 1e-4
mcpf = (cs_loss(ramsey_exact(R0 + h)) - cs_loss(ramsey_exact(R0 - h))) / (2 * h)
print("envelope theorem: dLoss/dR = %.4f  (MCPF; excess burden per extra"
      " GHS = %.4f)" % (mcpf, mcpf - 1))
```

Output

```
Ramsey taxes for revenue R0 = 30 (simulated demands)
  good          elast.   tax rate   t/(1+t)   e * t/(1+t)
  staple food     0.30     0.2220    0.1817      0.0545
  fuel            0.60     0.0999    0.0908      0.0545
  telecom         1.20     0.0476    0.0454      0.0545
  beverages       1.80     0.0312    0.0303      0.0545
  inverse elasticity rule: e_i t_i/(1+t_i) is the same for all i

excess burden: Ramsey 0.8788   uniform rate 1.4438 (t = 0.1321)
ratio dLoss/dt_i / dRev/dt_i (all equal mu): [1.0576 1.0576 1.0576 1.0576]
envelope theorem: dLoss/dR = 1.0576  (MCPF; excess burden per extra GHS = 0.0576)
```

The SLSQP solution satisfies (9.14) to four decimals: $e_i t_i/(1+t_i)$ equals 0.0545 for every good. The Ramsey structure raises the same revenue as a uniform 13% tax with about 40% less excess burden, and the marginal cost of public funds is 1.058: each additional cedi of revenue costs about 5.8 pesewas of deadweight loss. That number connects back to the budget-allocation problem: a public programme is worth expanding only if its return per cedi exceeds the marginal

cost of public funds.

Historical Note: Ramsey and optimal taxation

Frank Ramsey (1903–1930), a Cambridge mathematician and philosopher, published "A contribution to the theory of taxation" in the *Economic Journal* in 1927 in response to a problem posed by A. C. Pigou: how should a given revenue be raised by proportionate taxes on commodities so as to minimise the decrease in utility? His answer, derived with Lagrange multipliers, is the origin of the inverse elasticity rule. The theory was generalised to many households and to production efficiency by Diamond and Mirrlees (1971) and became the foundation of modern optimal tax theory (Ramsey, 1927).

△ Caution: Efficiency is not the only criterion

The inverse elasticity rule taxes the staple food most heavily because its demand is least elastic. Since staples take a large share of poor households' budgets, the efficient tax structure can be strongly regressive. Many-person Ramsey rules, which attach equity weights to households as in the social-planner problem, reduce the tax on goods consumed disproportionately by the poor; in practice VAT exemptions or zero rates on basic foods reflect this concern. A single-consumer model should never be the only input to tax design.

9.6.5 Sensitivity analysis

Optimisation models are only as good as their parameters, and policy parameters – returns to programmes, elasticities, poverty lines – are estimated with error. Two complementary analyses should accompany any recommended solution.

Local sensitivity. If the active set does not change, the comparative statics formula (9.7) applied to the KKT system gives the derivatives of the solution and of the multipliers with respect to every parameter, at the cost of one linear solve. Elasticities computed from these derivatives show which parameters the solution is most sensitive to. When a parameter change alters the active set (a programme enters or leaves), the solution is continuous but not differentiable, and the derivatives change discontinuously.

Global sensitivity and regret. Draw parameters from a distribution that represents uncertainty, re-solve, and summarise the distribution of the optimal decisions and of the *regret* – the outcome lost by implementing the nominal plan instead of the plan that would be optimal under the drawn parameters.

Listing 9.15: Local (IFT) and Monte Carlo sensitivity analysis (plotting code omitted)
ch09/ch09_sensitivity.py

```
# DEP 803 -- Chapter 9: Constrained Optimisation and Lagrange Multipliers
# Sensitivity of an optimal allocation: implicit function theorem + Monte Carlo.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq

names = ["education", "health", "agric. ext.", "water/sanit.", "feeder roads"]
a = np.array([3.0, 4.0, 2.0, 2.5, 1.5])      # same simulated returns as
b = np.array([1.0, 2.0, 0.5, 1.5, 0.6])      # ch09_budget_allocation.py
B = 4.0


```

```
def allocate(a, B):
    lam = brentq(lambda L: np.maximum(a / L - b, 0).sum() - B, 1e-9,
                 (a / b).max(), xtol=1e-14)
    return np.maximum(a / lam - b, 0), lam


x, lam = allocate(a, B)
# KKT system F(x, lam; a, B) = [a/(b+x) - lam ; sum x - B] = 0 (all active)
n = len(a)
J = np.block([[np.diag(-a / (b + x)**2), -np.ones((n, 1))],
              [np.ones((1, n)), np.zeros((1, 1))]])
dF_da = np.vstack([np.diag(1 / (b + x)), np.zeros((1, n))])
dF_dB = np.r_[np.zeros(n), -1.0]
S = -np.linalg.solve(J, np.column_stack([dF_da, dF_dB]))  # d(x,lam)/d(a,B)
h = 1e-6
fd = (allocate(a + h * np.eye(n)[1], B)[0] -
      allocate(a - h * np.eye(n)[1], B)[0]) / (2 * h)
print("Allocation at B = %.1f:" % B, np.round(x, 3), " lambda = %.4f" % lam)
print("dx/da_health (IFT):        ", np.round(S[:n, 1], 4))
print("dx/da_health (finite diff):", np.round(fd, 4))
print("dx/dB (IFT):               ", np.round(S[:n, n], 4),
      " dlam/dB = %.4f" % S[n, n])
el = S[:n, 1] * a[1] / np.where(x > 0, x, np.nan)
print("elasticity of x_i w.r.t. a_health:", np.round(el, 3))

# Monte Carlo: returns uncertain, a_i * exp(N(0, 0.25^2))
rng = np.random.default_rng(803)
R = 2000
draws = a * np.exp(rng.normal(0, 0.25, (R, n)))
X = np.array([allocate(d, B)[0] for d in draws])
loss = np.array([np.sum(d * np.log1p(allocate(d, B)[0] / b)) -
                 np.sum(d * np.log1p(x / b)) for d in draws])
print("\nMonte Carlo (R = %d, 25%% lognormal uncertainty in returns):" % R)
print("  programme      x_nominal  mean   5%   95%   P(funded)")
for i in range(n):
    q5, q95 = np.percentile(X[:, i], [5, 95])
    print("  %-13s %8.3f  %5.2f %5.2f %5.2f   %5.2f" %
          (names[i], x[i], X[:, i].mean(), q5, q95, np.mean(X[:, i] > 0)))
print("  regret of nominal plan: mean %.4f, 95th pct %.4f (outcome units)" %
      (loss.mean(), np.percentile(loss, 95)))
```

Output

```
Allocation at B = 4.0: [1.215 0.954 0.977 0.346 0.508]  lambda = 1.3542
dx/da_health (IFT):         [-0.1704  0.5112 -0.1136 -0.142  -0.0852]
dx/da_health (finite diff): [-0.1704  0.5112 -0.1136 -0.142  -0.0852]
dx/dB (IFT):                [0.2308 0.3077 0.1538 0.1923 0.1154]  dlam/dB = -0.1411
elasticity of x_i w.r.t. a_health: [-0.561  2.144 -0.465 -1.641 -0.671]

Monte Carlo (R = 2000, 25% lognormal uncertainty in returns):
  programme      x_nominal  mean   5%   95%   P(funded)
  education          1.215   1.20  0.49  2.02    1.00
  health             0.954   0.94  0.07  1.89    0.96
  agric. ext.        0.977   0.97  0.46  1.62    1.00
  water/sanit.       0.346   0.39  0.00  1.05    0.80
  feeder roads       0.508   0.51  0.09  1.01    0.98
  regret of nominal plan: mean 0.3099, 95th pct 0.7506 (outcome units)
```

The implicit-function derivatives agree with finite differences. An increase in the health returns parameter draws funds from every other programme, most strongly (in elasticity terms)

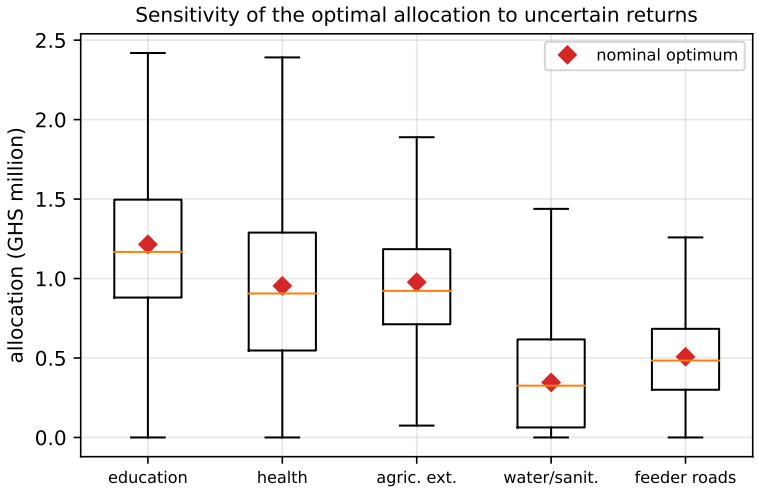


Figure 9.11: Distribution of optimal allocations under 25% lognormal uncertainty in the returns parameters (boxes show quartiles, whiskers 1.5 IQR), with the nominal optimum marked. Water and sanitation is unfunded in about one draw in five.

from water and sanitation, the marginal programme. The Monte Carlo analysis shows that a 25% uncertainty in returns translates into very wide ranges for the optimal allocations – the 90% interval for health runs from almost nothing to twice its nominal value – yet the expected regret of the nominal plan is modest (about 0.31 outcome units on average, against a nominal total outcome of about 7.6 units at $B = 4$). This is typical: near an optimum the objective is flat (the envelope theorem again), so errors in the decision cost less than errors in the parameters might suggest.

Think About It

If the outcome surface is flat near the optimum, the data cannot pin down the optimal allocation precisely, but it also does not matter much which nearby allocation is chosen. What, then, should a policy analyst report to a District Assembly: the point optimum, a range of near-optimal allocations, or the multipliers? How would your answer change if one of the programmes had a large fixed cost?

Exercises

9.6.1 Theory Prove Proposition 9.18. Show in particular that λ^* is unique and that $V'(B) = \lambda^*$ even at budgets where a programme enters.

9.6.2 Theory For the social planner with $\eta = 1$ and weights ω_i, derive the transfer schedule and show that the post-transfer consumption of all recipients is proportional to their weights. What happens as $\eta \to \infty$?

9.6.3 Theory Derive the inverse elasticity rule (9.14) and the expression $\mu = 1/(1-k)$. Show that μ increases with the revenue requirement.

9.6.4 Python Add to the targeting LP a constraint that every region receives at least GHS 20 per household. Report the increase in the poverty gap index and the shadow price of the floor. Then replace the objective by the squared poverty gap and solve the resulting convex problem with SLSQP or by projected gradient.

9.6.5 Python In Listing 9.11, add a minimum allocation $x_i \geq 0.2$ for every programme (a statutory floor). Compute the welfare cost of the floors using the multipliers, and verify by re-solving.

9.6.6 ★ Graduate Consider the budget-allocation problem under uncertainty, $\max_{\mathbf{x}} \mathbb{E}\left[\sum_i f_i(x_i;\boldsymbol{\theta})\right]$ subject to the budget. Show that for $f_i = a_i \log(1 + x_i/b_i)$ with random a_i the stochastic solution depends on $\mathbb{E}[a_i]$ only, and contrast this with the average of the Monte Carlo solutions in Listing 9.15. Why do the two differ, and which should be implemented?

9.6.7 Policy A government is considering a VAT exemption for staple foods. Using the Ramsey model with equity weights, explain the trade-off between efficiency and equity, and describe the data you would need to quantify it. How would the marginal cost of public funds enter a cost–benefit analysis of the exemption?

Chapter Summary

- At a regular equality-constrained optimum the objective gradient is a linear combination of the constraint gradients, $\nabla f = \sum_j \lambda_j \nabla h_j$; level sets are tangent to the feasible surface. The constraint qualification (full-rank Jacobian) is essential; the proof reduces the problem to an unconstrained one via the implicit function theorem.
- Marshallian demand follows from the Lagrange conditions: constant budget shares for Cobb–Douglas, share formula (9.5) for CES. Problems can be solved numerically either directly (SLSQP) or through the first-order system (`root`).
- By the envelope theorem, multipliers are shadow prices: $\lambda_j = \partial V / \partial c_j$. The marginal utility of income, the marginal value of a public budget and the marginal cost of public funds are all multipliers. Comparative statics follow from the implicit function theorem applied to the KKT system; second-order conditions use the Hessian of the Lagrangian on the tangent space, checked with the bordered Hessian.
- With inequality constraints the KKT conditions add dual feasibility ($\mu_k \geq 0$) and complementary slackness ($\mu_k g_k = 0$). They are necessary under LICQ, and necessary and sufficient for convex problems satisfying Slater's condition. Corner solutions arise naturally from non-negativity.
- The dual function is always concave and gives lower bounds (weak duality); strong duality holds for convex problems under Slater's condition and for all feasible bounded LPs. Dual variables of the diet LP are nutrient shadow prices; duality gaps measure non-convexity.
- Restricted least squares, constrained MLE, ridge, lasso and the SVM are constrained estimation problems; penalties are multipliers. Projected gradient, penalty and barrier methods are the basic algorithms for constrained problems.
- In policy problems the KKT conditions deliver clear rules: equalise marginal returns across funded programmes, fill transfers from the bottom, equalise the post-transfer poverty rate across funded regions, set taxes inversely to elasticities. Every recommendation should come with its multipliers and a sensitivity analysis.

Further Reading

Simon and Blume (1994) give the standard economics-oriented treatment of Lagrange multipliers, the bordered Hessian and the envelope theorem; Sundaram (1996) is a rigorous but accessible text on optimisation theory with full proofs, and Dixit (1990) is a short classic on optimisation in economic theory. For consumer theory and its comparative statics, see Mas-Colell et al. (1995). Boyd and Vandenberghe (2004) is the definitive reference for convex optimisation and duality (Chapters 5 and 11 cover duality and interior-point methods), and Nocedal and Wright (2006) covers the theory and algorithms of nonlinear programming, including KKT theory, SQP and

augmented Lagrangian methods. Bertsekas (1999) treats duality and Lagrangian relaxation in depth, and Dantzig (1963) remains a readable account of linear programming by its inventor.

On the statistical side, restricted least squares and the trinity of Wald, LR and LM tests are covered in Hansen (2022) and Wooldridge (2010); Murphy (2022, Chapter 11) and Hastie et al. (2009, Chapter 3) discuss ridge and lasso, including their constrained forms, and Géron (2022, Chapters 4–5) presents regularised linear models and SVMs from a practitioner's perspective. Goulet Coulombe et al. (2022) show how regularisation, as a constraint on model complexity, underlies the success of machine learning in macroeconomic forecasting. Matsuura (2022) discusses how constrained parameters (simplexes, positive variances) are handled by transformation in Bayesian modelling with Stan.

For policy applications, Atkinson and Stiglitz (1980) develop optimal taxation and the Ramsey problem, Foster et al. (1984) introduce the FGT poverty measures, and Kanbur (1987) and Ravallion and Chao (1989) analyse optimal targeting of poverty-alleviation budgets. FAO et al. (2020) describes how least-cost diets are used to monitor the affordability of nutritious diets.

Part IV

Computation and Uncertainty

Chapter 10

Numerical Integration and Monte Carlo Simulation

Learning Objectives

After working through this chapter you should be able to:

1. recognise expectations, probabilities, marginal likelihoods, posterior means, consumer surplus and present values as integrals, and check numerical answers against closed forms;
2. derive the error of the trapezoidal rule, state the error of Simpson's rule, and explain why Gaussian quadrature with n nodes is exact for polynomials of degree $2n - 1$;
3. compute expectations under a normal distribution by Gauss–Hermite quadrature and use `scipy.integrate.quad` and `dblquad` competently, including their failure modes;
4. explain the curse of dimensionality for tensor-product rules and why the Monte Carlo error rate $O(n^{-1/2})$ does not depend on the dimension;
5. prove that the Monte Carlo estimator is unbiased with variance σ^2/n, and report Monte Carlo standard errors and confidence intervals;
6. derive and implement antithetic variates, control variates (with the optimal coefficient), importance sampling (with the optimal proposal and the effective sample size), stratified sampling and quasi-Monte Carlo;
7. generate random variables by the inverse-CDF and acceptance–rejection methods and prove that both are correct;
8. prove detailed balance for Metropolis–Hastings, implement random-walk Metropolis and Gibbs samplers, and diagnose them with trace plots, autocorrelations, effective sample sizes and $\widehat{R}$;
9. design, run and report a simulation study using the ADEMP framework, and build a microsimulation of a cash-transfer programme that propagates parameter uncertainty into poverty outcomes.

Almost every quantity a policy analyst reports is an integral in disguise. A poverty headcount is the integral of an income density below the poverty line. The welfare gain from a fuel or fertiliser subsidy is the area under a demand curve. The expected net present value of a road or irrigation scheme is an integral over uncertain costs, delays and benefits. The probability that a household enrols in a programme, in a probit or mixed-logit model, is an integral over unobserved taste shocks. And in Bayesian policy evaluation, every posterior mean, credible

interval and "probability that the programme works" is a ratio of integrals over the parameter space.

A handful of these integrals have closed forms; most do not. This chapter is about the two great families of methods for computing them. *Deterministic quadrature* (trapezoidal, Simpson and Gaussian rules) places carefully chosen nodes and weights and is unbeatable in one or a few dimensions when the integrand is smooth. *Monte Carlo* methods replace the integral by an average over random draws; their error shrinks only like $n^{-1/2}$, but that rate is the same in one dimension as in one thousand, the error can be estimated from the same draws, and the method extends naturally to sampling from complicated distributions through Markov chain Monte Carlo (MCMC). Monte Carlo is also the engine of two everyday tools of evidence-based policy: the *simulation study*, in which we test an estimator on data generated from a known truth, and *microsimulation*, in which we push a simulated population through a tax or transfer rule.

Section 10.1 collects the integrals that recur in data science and economic policy, with closed forms we can use as test cases. Section 10.2 develops deterministic quadrature and explains the curse of dimensionality. Section 10.3 introduces Monte Carlo integration and its error analysis, building on the law of large numbers and the central limit theorem of Chapter 6. Section 10.4 shows how to make Monte Carlo more efficient by variance reduction and quasi-random points. Section 10.5 turns to sampling: from inverse CDFs and acceptance–rejection to Metropolis–Hastings, Gibbs sampling and Hamiltonian Monte Carlo, the algorithm behind Stan (Matsuura, 2022). Section 10.6 closes with the design and reporting of simulation studies and a policy microsimulation. The standard references are Robert and Casella (2004) for Monte Carlo statistical methods, Davis and Rabinowitz (1984) for numerical integration, Judd (1998) for economic applications and Murphy (2022) and Gelman et al. (2013) for the Bayesian side.

10.1 Why Integrals Are Everywhere

Before we learn how to compute integrals numerically it is worth seeing how many apparently different quantities are integrals of the form

$$I \;=\; \int_{\mathcal{X}} h(x)\, p(x)\, \mathrm{d}x \;=\; \mathbb{E}_{p}[h(X)], \tag{10.1}$$

where p is a density on $\mathcal{X} \subseteq \mathbb{R}^d$ and h is a function we care about. Writing an integral as an expectation is the key move of the whole chapter: once $I = \mathbb{E}_p[h(X)]$, we can approximate it either by a weighted sum over well-chosen points (quadrature) or by an average over random draws from p (Monte Carlo).

10.1.1 Expectations and probabilities

By the law of the unconscious statistician (Chapter 6), $\mathbb{E}[h(X)] = \int h(x)p(x)\,\mathrm{d}x$ whenever $\int |h(x)|\,p(x)\,\mathrm{d}x < \infty$. Probabilities are expectations of indicators:

$$\mathbb{P}(X \in A) \;=\; \int \mathbf{1}\{x \in A\}\, p(x)\, \mathrm{d}x \;=\; \mathbb{E}\big[\mathbf{1}\{X \in A\}\big]. \tag{10.2}$$

Hence a poverty headcount is $\mathbb{P}(Y < z) = \int_0^z f_Y(y)\,\mathrm{d}y$ for income Y with density f_Y and poverty line z, and the Foster–Greer–Thorbecke family (Foster et al., 1984)

$$P_\alpha \;=\; \int_0^{z} \left(1 - \frac{y}{z}\right)^{\alpha} f_Y(y)\, \mathrm{d}y, \qquad \alpha \geq 0, \tag{10.3}$$

interpolates between the headcount ($\alpha = 0$), the poverty gap ($\alpha = 1$) and the squared gap ($\alpha = 2$), which weights the poorest most heavily.

Example 10.1 (Probit and mixed probit)**.** In a probit model $\mathbb{P}(y=1 \mid \mathbf{x}) = \Phi(\mathbf{x}^\top \boldsymbol{\beta}) = \int_{-\infty}^{\mathbf{x}^\top \boldsymbol{\beta}} \phi(t)\,\mathrm{d}t$, an integral with a fast special-function implementation. If a coefficient varies across households, $b \sim \mathcal{N}(m, s_b^2)$, the choice probability becomes

$$\mathbb{P}(y=1 \mid z) \;=\; \int_{-\infty}^{\infty} \Phi(a+bz)\,\frac{1}{s_b}\phi\!\left(\frac{b-m}{s_b}\right)\mathrm{d}b \;=\; \Phi\!\left(\frac{a+mz}{\sqrt{1+s_b^2 z^2}}\right),$$

where the closed form follows by writing $\Phi(a+bz) = \mathbb{P}(\varepsilon \le a+bz)$ with $\varepsilon \sim \mathcal{N}(0,1)$ independent of b and noting that $\varepsilon - bz \sim \mathcal{N}(-mz, 1+s_b^2 z^2)$. With several random coefficients and a logit kernel there is no closed form, and the integral must be simulated: this is the *mixed logit* of Train (2009).

10.1.2 Bayesian integrals

For data $\mathbf{y}$, likelihood $p(\mathbf{y} \mid \boldsymbol{\theta})$ and prior $p(\boldsymbol{\theta})$, Bayes' theorem gives the posterior $p(\boldsymbol{\theta} \mid \mathbf{y}) = p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})/p(\mathbf{y})$, where the *marginal likelihood* (or evidence)

$$p(\mathbf{y}) \;=\; \int p(\mathbf{y} \mid \boldsymbol{\theta})\,p(\boldsymbol{\theta})\,\mathrm{d}\boldsymbol{\theta} \tag{10.4}$$

is an integral over the whole parameter space. Every posterior summary is a ratio of integrals,

$$\mathbb{E}[g(\boldsymbol{\theta}) \mid \mathbf{y}] \;=\; \frac{\int g(\boldsymbol{\theta})\,p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})\,\mathrm{d}\boldsymbol{\theta}}{\int p(\mathbf{y} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})\,\mathrm{d}\boldsymbol{\theta}}, \tag{10.5}$$

for example the posterior mean ($g(\boldsymbol{\theta}) = \boldsymbol{\theta}$), a posterior probability ($g = \mathbf{1}\{\theta_j > 0\}$) or a posterior predictive probability. With a parameter vector of dimension $d = 20$ or $d = 2000$ these integrals are far beyond deterministic quadrature; making them computable was the achievement of MCMC (Section 10.5).

Example 10.2 (Beta–binomial)**.** Suppose $k = 21$ of $n = 30$ sampled villages report that a new water point is functioning, and the functioning probability θ has a uniform prior. Then

$$p(k) \;=\; \binom{n}{k}\int_0^1 \theta^k(1-\theta)^{n-k}\,\mathrm{d}\theta \;=\; \binom{n}{k} B(k+1, n-k+1) \;=\; \frac{1}{n+1},$$

and the posterior is $\mathrm{Beta}(k+1, n-k+1)$ with mean $(k+1)/(n+2) = 22/32 = 0.6875$. Note the striking fact that under the uniform prior $p(k) = 1/31$ for every $k = 0, \dots, 30$: before seeing the data, all success counts are equally likely.

10.1.3 Welfare: consumer surplus and present values

Let $D(p)$ be a market demand curve. The change in (Marshallian) consumer surplus when the price falls from p_0 to $p_1 < p_0$, for example because of a subsidy, is the area to the left of the demand curve between the two prices:

$$\Delta CS \;=\; \int_{p_1}^{p_0} D(p)\,\mathrm{d}p. \tag{10.6}$$

For linear demand $D(p) = a - bp$, $\Delta CS = a(p_0 - p_1) - \tfrac{b}{2}(p_0^2 - p_1^2)$, a trapezium of area $(p_0 - p_1)\big(D(p_0) + D(p_1)\big)/2$. For constant-elasticity demand $D(p) = Ap^{-\varepsilon}$ with $\varepsilon \neq 1$, $\Delta CS = \frac{A}{1-\varepsilon}\big(p_0^{1-\varepsilon} - p_1^{1-\varepsilon}\big)$. For estimated demand systems (for example an almost-ideal demand system or a random-coefficients logit) the integral must be computed numerically, and exact welfare measures (compensating

and equivalent variation) require integrating along a Hicksian demand curve (Simon and Blume, 1994).

A stream of benefits $B(t)$ over a horizon T, discounted continuously at rate r, has present value $PV = \int_0^T B(t)e^{-rt}\,\mathrm{d}t$; for constant B, $PV = B(1 - e^{-rT})/r$. When benefits, costs and r are uncertain, the *expected* present value is an integral over the joint distribution of all uncertain inputs, and it is natural to compute it by simulation (Section 10.3).

Historical Note: Dupuit, Marshall and the area under the demand curve

The French engineer Jules Dupuit argued in 1844, in an essay on measuring the utility of public works such as bridges and canals, that the benefit of a project to its users exceeds what they pay, and he measured the excess by an area under the demand curve. Alfred Marshall gave the idea its modern name, *consumer's surplus*, in his *Principles of Economics* (1890). The debate about when this Marshallian area is an exact welfare measure was largely settled by John Hicks in the 1940s through compensating and equivalent variation. Dupuit's purpose, appraising public infrastructure, is exactly the cost–benefit analysis that ministries of finance and development banks still carry out, now usually by numerical integration and simulation.

10.1.4 A test bench of integrals with closed forms

Numerical methods should always be tested on problems whose answers are known. Listing 10.1 computes seven such integrals, from the consumer-surplus gain of a subsidy to a Bayesian posterior mean, both from their closed forms and with SciPy's adaptive quadrature routine `integrate.quad` (Section 10.2). The agreement is to machine precision or close to it; Fig. 10.1 shows the first integral as an area.

Listing 10.1: Policy integrals with closed forms, checked by quadrature
ch10/ch10_integrals_closed_form.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Policy-relevant integrals with closed forms, checked by numerical quadrature.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import integrate, special, stats

rows = []

# 1. Change in consumer surplus from a price subsidy (linear demand)
a, b, p0, p1 = 100.0, 4.0, 10.0, 8.0
demand = lambda p: a - b * p
exact = a * (p0 - p1) - 0.5 * b * (p0**2 - p1**2)
rows.append(("dCS, linear demand", exact, integrate.quad(demand, p1, p0)[0]))

# 2. Same subsidy, constant-elasticity demand D(p) = A p^(-eps)
A, eps = 100.0, 1.5
exact = A / (1 - eps) * (p0 ** (1 - eps) - p1 ** (1 - eps))
num = integrate.quad(lambda p: A * p ** (-eps), p1, p0)[0]
rows.append(("dCS, CES demand", exact, num))

# 3. Present value of a benefit stream B for T years at rate r
B, r, T = 50.0, 0.08, 20.0
exact = B * (1 - np.exp(-r * T)) / r
rows.append(("PV of benefits", exact,
             integrate.quad(lambda t: B * np.exp(-r * t), 0, T)[0]))

# 4. Expected income when log income ~ N(mu, s^2)
mu, s = 7.0, 0.8
```

```python
f = lambda y: y * stats.lognorm.pdf(y, s, scale=np.exp(mu))
rows.append(("E[income], lognormal", np.exp(mu + s**2 / 2),
             integrate.quad(f, 0, np.inf)[0]))

# 5. Probit probability P(y=1) = Phi(x'beta) as an integral of the density
xb = 0.4
rows.append(("probit P(y=1)", stats.norm.cdf(xb),
             integrate.quad(stats.norm.pdf, -np.inf, xb)[0]))

# 6. Beta-binomial marginal likelihood: k successes in n trials, flat prior
n, k = 30, 21
exact = special.comb(n, k) * special.beta(k + 1, n - k + 1)
lik = lambda th: special.comb(n, k) * th**k * (1 - th) ** (n - k)
rows.append(("marginal likelihood", exact, integrate.quad(lik, 0, 1)[0]))

# 7. Posterior mean of theta under the same model: (k+1)/(n+2)
num = integrate.quad(lambda th: th * lik(th), 0, 1)[0] / rows[-1][2]
rows.append(("posterior mean", (k + 1) / (n + 2), num))

print(f"{'quantity':24s}{'closed form':>16s}{'quad':>16s}{'abs error':>11s}")
for name, ex, nu in rows:
    print(f"{name:24s}{ex:16.8f}{nu:16.8f}{abs(ex - nu):11.1e}")

# Figure: consumer surplus gain under the linear demand curve
q = np.linspace(0, a, 200)
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(q, (a - q) / b, color="navy", lw=2, label="inverse demand P(Q)")
ax.fill_between([0, demand(p0)], p1, p0, color="seagreen", alpha=0.35,
                label=f"gain in CS = {rows[0][1]:.0f}")
qq = np.linspace(demand(p0), demand(p1), 50)
ax.fill_between(qq, p1, (a - qq) / b, color="seagreen", alpha=0.35)
for p in (p0, p1):
    ax.axhline(p, color="gray", ls=":", lw=1)
ax.text(2, p0 + 0.4, "P0 = 10")
ax.text(2, p1 - 1.2, "P1 = 8 (subsidised)")
ax.set_xlabel("quantity Q")
ax.set_ylabel("price P")
ax.set_xlim(0, a)
ax.set_ylim(0, a / b)
ax.legend()
plt.savefig("ch10_consumer_surplus.pdf", bbox_inches="tight")
```

Output

```
quantity                   closed form           quad  abs error
dCS, linear demand          128.00000000  128.00000000    0.0e+00
dCS, CES demand               7.46512492    7.46512492    3.6e-15
PV of benefits              498.81467625  498.81467625    5.7e-14
E[income], lognormal       1510.20396976 1510.20396976    7.3e-10
probit P(y=1)                 0.65542174    0.65542174    0.0e+00
marginal likelihood           0.03225806    0.03225806    0.0e+00
posterior mean                0.68750000    0.68750000    1.1e-16
```

★ Key Concept: Integrals as expectations

Expectations, probabilities, poverty indices, choice probabilities, marginal likelihoods, posterior means, consumer surplus and expected present values are all integrals. Writing $I = \int h(x)p(x)\,\mathrm{d}x = \mathbb{E}_p[h(X)]$ separates *what* we average (h) from *where* the mass is (p).

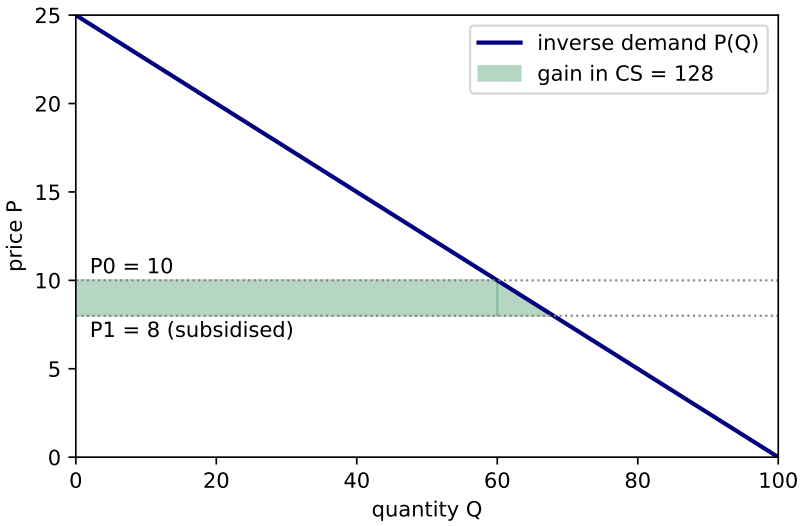


Figure 10.1: Gain in consumer surplus when a subsidy lowers the price from $P_0 = 10$ to $P_1 = 8$ under the linear demand $D(p) = 100 - 4p$: the shaded area $\int_8^{10} D(p)\,dp = 128$.

Quadrature approximates I by $\sum_i w_i h(x_i)p(x_i)$ with deterministic nodes; Monte Carlo approximates it by $\frac{1}{n}\sum_i h(X_i)$ with $X_i \overset{\text{iid}}{\sim} p$. Keep a library of closed-form test cases and check every numerical integrator against them.

Policy Lens: The welfare arithmetic of a price subsidy

In the linear example the government pays $s = 2$ per unit on the new quantity $D(8) = 68$, a fiscal cost of 136. Consumers gain $\Delta CS = 128$. The difference, $8 = \frac{1}{2} \times 2 \times (68 - 60)$, is the *deadweight loss*: the triangle between the demand curve and the subsidised price over the extra units consumed. With constant-elasticity or estimated demand the same accounting requires numerical integration, and when the demand parameters are uncertain (they are always estimated) the welfare gain is a random quantity whose distribution can be obtained by simulating parameter draws and integrating for each. Cost–benefit manuals for fuel, fertiliser and electricity subsidies rest on exactly this calculation; the integrals are simple, but getting the demand curve right is not.

△ Caution: Does the integral exist?

Numerical routines will happily return a number for an integral that does not exist. The expectation of a Cauchy random variable is undefined because $\int |x|/(\pi(1+x^2))\,dx = \infty$; the mean of a Pareto income distribution with tail index $\alpha \le 1$ is infinite; and the variance of a Pareto with $\alpha \le 2$ is infinite, so Monte Carlo standard errors are meaningless there (Section 10.3). Before integrating, check integrability analytically, at least for the tails.

Exercises

10.1.1 Theory Show that for the linear demand $D(p) = a - bp$ the change in consumer surplus (10.6) equals the area of a trapezium with parallel sides $D(p_0)$ and $D(p_1)$ and width $p_0 - p_1$. Derive the deadweight loss of a per-unit subsidy s and show that it is $\frac{1}{2}bs^2$.

10.1.2 Theory Prove the closed form of the mixed-probit probability in Example 10.1. Hence show that random coefficients "flatten" the response: for fixed $a + mz > 0$ the probability moves

towards $\frac{1}{2}$ as s_b grows.

10.1.3 Theory Let Y be lognormal, $\log Y \sim \mathcal{N}(\mu, s^2)$. Show that the poverty gap index is $P_1 = \Phi(d) - e^{\mu + s^2/2}\Phi(d - s)/z$ with $d = (\log z - \mu)/s$.

10.1.4 Python Extend Listing 10.1 to compute P_0, P_1 and P_2 of (10.3) for lognormal income with $\mu = 7$, $s = 0.8$ and $z = 900$ using `quad`, and check P_0 and P_1 against closed forms.

10.1.5 Python For constant-elasticity demand $D(p) = 100p^{-\varepsilon}$, plot the consumer-surplus gain of a price fall from 10 to 8 as a function of $\varepsilon \in (0, 3)$. What happens at $\varepsilon = 1$, and how should the code handle it?

10.1.6 Policy A ministry is choosing between a 20% price subsidy on a staple food and an equal-cost cash transfer. Using linear demand with an elasticity of -0.4 at the current price, compute the consumer-surplus gain and the deadweight loss of the subsidy per unit of fiscal cost. What does the calculation leave out (distribution, leakage, administrative costs)?

10.1.7 ★ Graduate Show that the Marshallian consumer surplus equals the exact compensating variation if and only if the income effect is zero (quasi-linear utility). Discuss how one would compute compensating variation numerically from an estimated Marshallian demand by solving an ordinary differential equation along the price path.

10.2 Deterministic Quadrature

A *quadrature rule* approximates $\int_a^b f(x)\,\mathrm{d}x$ by a weighted sum $Q_n(f) = \sum_{i=1}^{n} w_i f(x_i)$ with nodes x_i and weights w_i. The two classical design strategies are (i) to fix equally spaced nodes and choose the weights so that the rule integrates low-degree polynomials exactly (Newton–Cotes rules), and (ii) to choose both nodes and weights optimally (Gaussian rules). Both rest on the same idea: replace f by an interpolating polynomial and integrate the polynomial exactly.

10.2.1 The trapezoidal rule

On one interval $[a, b]$ with $h = b - a$, the linear interpolant of f gives the *trapezoidal rule* $T(f) = \frac{h}{2}\big(f(a) + f(b)\big)$. Splitting $[a, b]$ into n subintervals of width $h = (b - a)/n$ with nodes $x_j = a + jh$ and adding gives the *composite* rule

$$T_n(f) \;=\; h\Big[\tfrac{1}{2}f(x_0) + f(x_1) + \cdots + f(x_{n-1}) + \tfrac{1}{2}f(x_n)\Big]. \tag{10.7}$$

Theorem 10.3 (Error of the trapezoidal rule)**.** *If $f \in C^2[a, b]$ then for some $\eta \in (a, b)$*

$$\int_a^b f(x)\,\mathrm{d}x - \frac{b-a}{2}\big(f(a) + f(b)\big) \;=\; -\frac{(b-a)^3}{12}\,f''(\eta),$$

and for the composite rule with n subintervals, for some $\xi \in (a, b)$,

$$\int_a^b f(x)\,\mathrm{d}x - T_n(f) \;=\; -\frac{(b-a)\,h^2}{12}\,f''(\xi) \;=\; O(n^{-2}). \tag{10.8}$$

Proof. Let p be the linear interpolant of f at a and b. The interpolation error formula states that for each $x \in [a, b]$ there is $\zeta_x \in (a, b)$ with $f(x) - p(x) = \frac{1}{2}f''(\zeta_x)(x - a)(x - b)$. (To see this, fix x, put $g(t) = f(t) - p(t) - \lambda(t - a)(t - b)$ with λ chosen so that $g(x) = 0$; g has three zeros a, x, b, so

by Rolle's theorem g'' has a zero ζ_x, and $g''(\zeta_x) = f''(\zeta_x) - 2\lambda = 0$.) Since $\int_a^b p(x)\,\mathrm{d}x = T(f)$,

$$\int_a^b f - T(f) \;=\; \int_a^b \tfrac{1}{2} f''(\zeta_x)\,(x-a)(x-b)\,\mathrm{d}x.$$

The weight $(x-a)(x-b)$ does not change sign on $[a,b]$ and $x \mapsto f''(\zeta_x)$ is continuous (it equals $2(f(x)-p(x))/((x-a)(x-b))$ for $x \in (a,b)$), so the mean value theorem for integrals gives η with

$$\int_a^b f - T(f) \;=\; \tfrac{1}{2} f''(\eta) \int_a^b (x-a)(x-b)\,\mathrm{d}x \;=\; \tfrac{1}{2} f''(\eta)\Big(-\frac{(b-a)^3}{6}\Big) \;=\; -\frac{(b-a)^3}{12} f''(\eta).$$

For the composite rule, sum the n local errors $-\frac{h^3}{12} f''(\eta_j)$. Because f'' is continuous, the average $\frac{1}{n}\sum_j f''(\eta_j)$ lies between $\min f''$ and $\max f''$ and so equals $f''(\xi)$ for some ξ by the intermediate value theorem. Hence the total error is $-\frac{nh^3}{12} f''(\xi) = -\frac{(b-a)h^2}{12} f''(\xi)$. □

The theorem says that halving h divides the error by four, provided f'' is bounded. For periodic functions integrated over a full period, and for rapidly decaying integrands on $\mathbb{R}$, the trapezoidal rule is far better than (10.8) suggests: the Euler–Maclaurin formula shows that the $h^2, h^4, \ldots$ error terms involve differences of derivatives at the end-points, which then cancel, and convergence becomes geometric (Davis and Rabinowitz, 1984).

10.2.2 Simpson's rule

Interpolating by a quadratic through a, $m = (a+b)/2$ and b gives Simpson's rule $S(f) = \frac{b-a}{6}\big(f(a) + 4f(m) + f(b)\big)$. With an even number n of subintervals of width h, the composite rule is

$$S_n(f) \;=\; \frac{h}{3}\Big[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n)\Big]. \tag{10.9}$$

Theorem 10.4 (Error of Simpson's rule). *If $f \in C^4[a,b]$ and n is even, then for some $\xi \in (a,b)$*

$$\int_a^b f(x)\,\mathrm{d}x - S_n(f) \;=\; -\frac{(b-a)\,h^4}{180}\, f^{(4)}(\xi) \;=\; O(n^{-4}).$$

Although Simpson's rule uses a quadratic interpolant, it integrates cubics exactly: by symmetry the error of $(x-m)^3$ is zero. This "bonus" degree is why the error involves $f^{(4)}$ and is $O(h^4)$ rather than $O(h^3)$. A proof uses Hermite interpolation at a, m, m, b (Davis and Rabinowitz, 1984); Exercise **10.2.1** asks you to verify it. Note also that $S_{2n} = \frac{4T_{2n} - T_n}{3}$: Simpson's rule is one step of Richardson extrapolation applied to the trapezoidal rule, which removes its leading h^2 error term. Repeating the extrapolation gives Romberg integration.

Listing 10.2 measures the convergence on two integrands. For $\int_0^1 e^x\,\mathrm{d}x$ the error ratios on successive halvings are 4.00 and 16.0, exactly as the theorems predict. For $\int_0^1 \sqrt{x}\,\mathrm{d}x$ both rules converge only like $h^{1.5}$: the integrand's derivative is unbounded at 0, the theorems' smoothness assumptions fail, and Simpson's rule loses its advantage (Fig. 10.2).

Listing 10.2: Composite trapezoidal and Simpson rules ch10/ch10_newton_cotes.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Composite trapezoidal and Simpson rules: observed convergence rates.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt


```

```python
def trapezoid(f, a, b, n):
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    return h * (y.sum() - 0.5 * (y[0] + y[-1]))


def simpson(f, a, b, n):          # n must be even
    x = np.linspace(a, b, n + 1)
    y = f(x)
    h = (b - a) / n
    return h / 3 * (y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum())


cases = [("exp(x) on [0,1]", np.exp, np.e - 1),
         ("sqrt(x) on [0,1]", np.sqrt, 2.0 / 3.0)]
ns = 2 ** np.arange(1, 11)
fig, ax = plt.subplots(figsize=(6, 3.8))
for (label, f, exact), ls in zip(cases, ["-", "--"]):
    et = np.array([abs(trapezoid(f, 0, 1, n) - exact) for n in ns])
    es = np.array([abs(simpson(f, 0, 1, n) - exact) for n in ns])
    print(f"{label}")
    print("     n   trap error  ratio   simp error  ratio")
    for i, n in enumerate(ns[:6]):
        rt = et[i - 1] / et[i] if i else np.nan
        rs = es[i - 1] / es[i] if i else np.nan
        print(f"{n:6d}   {et[i]:9.2e}  {rt:5.2f}   {es[i]:9.2e}  {rs:5.2f}")
    slope_t = np.polyfit(np.log(ns), np.log(et), 1)[0]
    slope_s = np.polyfit(np.log(ns[:6]), np.log(es[:6]), 1)[0]
    print(f"   fitted log-log slopes: trapezoid {slope_t:.2f}, "
          f"Simpson {slope_s:.2f}")
    ax.loglog(ns, et, "o" + ls, color="navy", ms=4, label=f"trapezoid, {label}")
    ax.loglog(ns, np.maximum(es, 1e-17), "s" + ls, color="darkorange", ms=4,
              label=f"Simpson, {label}")
ax.set_xlabel("number of subintervals n")
ax.set_ylabel("absolute error")
ax.legend(fontsize=8)
plt.savefig("ch10_newton_cotes.pdf", bbox_inches="tight")
```

Output

```
exp(x) on [0,1]
     n   trap error  ratio   simp error  ratio
     2    3.56e-02    nan    5.79e-04    nan
     4    8.94e-03   3.99    3.70e-05  15.65
     8    2.24e-03   4.00    2.33e-06  15.91
    16    5.59e-04   4.00    1.46e-07  15.98
    32    1.40e-04   4.00    9.10e-09  15.99
    64    3.50e-05   4.00    5.69e-10  16.00
   fitted log-log slopes: trapezoid -2.00, Simpson -3.99
sqrt(x) on [0,1]
     n   trap error  ratio   simp error  ratio
     2    6.31e-02    nan    2.86e-02    nan
     4    2.34e-02   2.70    1.01e-02   2.82
     8    8.54e-03   2.74    3.59e-03   2.83
    16    3.09e-03   2.77    1.27e-03   2.83
    32    1.11e-03   2.79    4.48e-04   2.83
    64    3.96e-04   2.80    1.59e-04   2.83
   fitted log-log slopes: trapezoid -1.48, Simpson -1.50
```

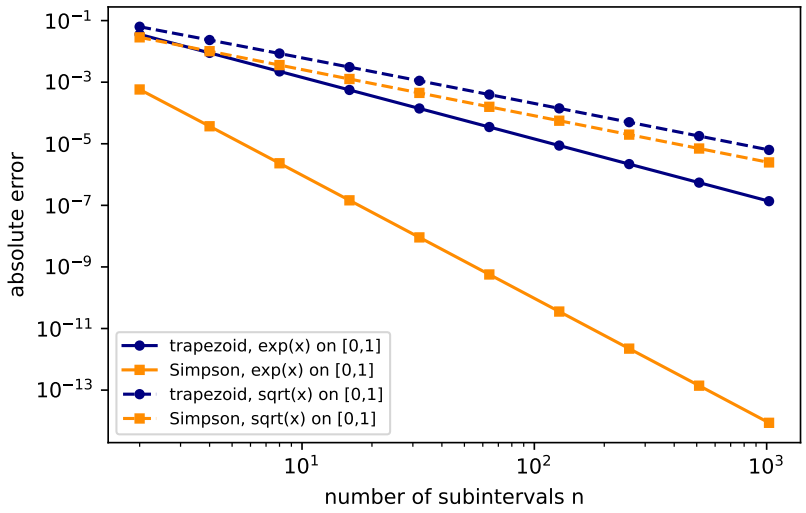


Figure 10.2: Absolute error of the composite trapezoidal and Simpson rules against the number of subintervals. For the smooth integrand e^x the slopes are -2 and -4; for $\sqrt{x}$, whose derivative is unbounded at 0, both rules converge at the rate $n^{-3/2}$.

△ Caution: Smoothness is an assumption, not a gift

Error bounds such as (10.8) are only as good as the derivative they contain. Integrands in economics are often kinked (tax schedules, $\max(z-y,0)$ in poverty gaps, option-like insurance payouts) or singular at an end-point. Place nodes at kinks (split the interval there), transform singularities away (e.g. $x = t^2$ in $\int_0^1 \sqrt{x}\, g(x)\,\mathrm{d}x$), or use an adaptive method that refines where the error is large.

10.2.3 Gaussian quadrature

Newton–Cotes rules with n nodes integrate polynomials of degree $n-1$ (or n) exactly. If we also free the nodes, the $2n$ parameters (x_i, w_i) suggest that degree $2n-1$ should be attainable. It is, and the optimal nodes are the zeros of orthogonal polynomials.

Let $w(x) > 0$ be a weight function on (a,b) and let $p_0, p_1, \dots$ be the polynomials orthogonal with respect to $\langle f, g\rangle_w = \int_a^b f(x)g(x)w(x)\,\mathrm{d}x$, with $\deg p_k = k$. Classical cases are Legendre ($w = 1$ on $[-1,1]$), Hermite ($w = e^{-x^2}$ on $\mathbb{R}$) and Laguerre ($w = e^{-x}$ on $[0,\infty)$).

Theorem 10.5 (Gaussian quadrature). *Let $x_1, \dots, x_n$ be the zeros of p_n, and let $w_i = \int_a^b \ell_i(x)w(x)\,\mathrm{d}x$, where ℓ_i is the Lagrange basis polynomial with $\ell_i(x_j) = \delta_{ij}$. Then*

(i) the zeros are real, simple and lie in (a,b);

(ii) $\int_a^b f(x)w(x)\,\mathrm{d}x = \sum_{i=1}^n w_i f(x_i)$ for every polynomial f of degree at most $2n-1$, and no n-point rule is exact for all polynomials of degree $2n$;

(iii) all weights are positive and $\sum_i w_i = \int_a^b w(x)\,\mathrm{d}x$.

Proof. (ii) Let $\deg f \le 2n-1$. Divide by p_n: $f = qp_n + r$ with $\deg q, \deg r \le n-1$. Orthogonality gives $\int qp_n w = 0$ because q is a combination of $p_0, \dots, p_{n-1}$. The rule is interpolatory, so it integrates r exactly: $\int rw = \sum_i w_i r(x_i)$. Finally $f(x_i) = r(x_i)$ because $p_n(x_i) = 0$. Hence $\int fw = \int rw = \sum_i w_i r(x_i) = \sum_i w_i f(x_i)$. For degree $2n$, the polynomial $\prod_i (x-x_i)^2$ has positive integral but vanishes at every node. (iii) Apply (ii) to ℓ_i^2, of degree $2n-2$: $0 < \int \ell_i^2 w = \sum_j w_j \ell_i(x_j)^2 = w_i$. Apply it to $f \equiv 1$ for the sum. (i) If p_n changed sign at only $k < n$ points $y_1, \dots, y_k$ in (a,b), then

$p_n(x)\prod_j(x-y_j)$ would not change sign, so its integral against w would be non-zero, contradicting orthogonality of p_n to polynomials of degree $k<n$. □

Positivity of the weights matters numerically: the rule is a convex-like combination of function values, so rounding errors are not amplified, unlike high-order Newton–Cotes rules whose weights become negative and large. For smooth (analytic) integrands the error of Gaussian quadrature falls geometrically in n. Nodes and weights are computed stably from the eigen-decomposition of a symmetric tridiagonal (Jacobi) matrix built from the three-term recurrence of the p_k: the Golub–Welsch algorithm, a nice application of Chapter 3.

10.2.4 Gauss–Hermite quadrature for normal expectations

Economists integrate against the normal density constantly: expected utility under normal log-income shocks, probit and random-coefficient choice probabilities, expectations in dynamic models with normal productivity shocks. With $X\sim\mathcal{N}(\mu,\sigma^2)$ and the substitution $x=\mu+\sqrt{2}\sigma t$,

$$\mathbb{E}[g(X)] \;=\; \frac{1}{\sqrt{\pi}}\int_{-\infty}^{\infty} g\big(\mu+\sqrt{2}\sigma t\big)e^{-t^2}\,\mathrm{d}t \;\approx\; \frac{1}{\sqrt{\pi}}\sum_{i=1}^{n} w_i\, g\big(\mu+\sqrt{2}\sigma t_i\big), \tag{10.10}$$

where (t_i,w_i) are the Gauss–Hermite nodes and weights (`np.polynomial.hermite.hermgauss`). Listing 10.3 first confirms the degree of exactness of a three-point Gauss–Legendre rule (x^0 to x^5 exact, x^6 not), then computes the expected CRRA utility $\mathbb{E}[Y^{1-\gamma}/(1-\gamma)]$ with $\log Y\sim\mathcal{N}(7,0.8^2)$, $\gamma=2$, and the mixed-probit probability of Example 10.1. Ten nodes give relative errors of 10^{-14} and 10^{-10}.

Listing 10.3: Gauss–Legendre and Gauss–Hermite quadrature `ch10/ch10_gauss_quadrature.py`

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Gauss-Legendre exactness and Gauss-Hermite expectations under the normal.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

# Gauss-Legendre on [-1,1]: n nodes integrate polynomials of degree 2n-1
n = 3
x, w = np.polynomial.legendre.leggauss(n)
print(f"Gauss-Legendre n={n}: nodes {np.round(x, 6)}")
print(f"                    weights {np.round(w, 6)}")
for deg in range(2 * n + 2):
    exact = 2.0 / (deg + 1) if deg % 2 == 0 else 0.0
    err = abs(w @ x**deg - exact)
    print(f"  x^{deg}: error {err:8.1e}", "(exact)" if err < 1e-14 else "")


def gh_expect(g, mu, sigma, n):
    """E[g(X)], X ~ N(mu, sigma^2), by n-point Gauss-Hermite quadrature."""
    t, w = np.polynomial.hermite.hermgauss(n)
    return (w @ g(mu + np.sqrt(2.0) * sigma * t)) / np.sqrt(np.pi)


# Expected CRRA utility of income Y = exp(X), X ~ N(mu, s^2), gamma = 2
mu, s, gam = 7.0, 0.8, 2.0
u = lambda xx: np.exp((1 - gam) * xx) / (1 - gam)
eu = np.exp((1 - gam) * mu + (1 - gam) ** 2 * s**2 / 2) / (1 - gam)
# Random-coefficient probit: P = E_b[Phi(a + b z)], b ~ N(m, sb^2)
a0, m, sb, z = -0.5, 1.0, 0.7, 0.8
pr = stats.norm.cdf((a0 + m * z) / np.sqrt(1 + sb**2 * z**2))
g = lambda bb: stats.norm.cdf(a0 + bb * z)
print("\nGauss-Hermite relative errors")
```

```
print(" nodes   E[u(Y)] CRRA   mixed-probit P")
for nn in (1, 2, 3, 5, 10, 20):
    e1 = abs(gh_expect(u, mu, s, nn) / eu - 1)
    e2 = abs(gh_expect(g, m, sb, nn) / pr - 1)
    print(f"{nn:6d}   {e1:12.2e}   {e2:12.2e}")
print(f"exact: E[u(Y)] = {eu:.6e},  P(y=1) = {pr:.6f}")
```

Output

```
Gauss-Legendre n=3: nodes [-0.774597  0.        0.774597]
                   weights [0.555556 0.888889 0.555556]
  x^0: error  4.4e-16 (exact)
  x^1: error  1.8e-17 (exact)
  x^2: error  3.3e-16 (exact)
  x^3: error  4.8e-18 (exact)
  x^4: error  1.1e-16 (exact)
  x^5: error  2.3e-18 (exact)
  x^6: error  4.6e-02
  x^7: error  2.7e-18 (exact)

Gauss-Hermite relative errors
 nodes   E[u(Y)] CRRA   mixed-probit P
     1       2.74e-01       2.43e-02
     2       2.88e-02       3.27e-03
     3       1.84e-03       4.36e-04
     5       3.00e-06       7.65e-06
    10       1.45e-14       3.06e-10
    20       0.00e+00       2.22e-16
exact: E[u(Y)] = -1.255778e-03,  P(y=1) = 0.603244
```

Historical Note: Simpson, Gauss and the art of quadrature

The rule that bears Thomas Simpson's name appears in his *Mathematical Dissertations* of 1743, but it was known earlier: Bonaventura Cavalieri and James Gregory used it in the seventeenth century, and Kepler's 1615 rule for the volume of wine barrels is closely related. Carl Friedrich Gauss presented his method of choosing nodes optimally to the Göttingen Society of Sciences in 1814 (published 1816), working with continued fractions; the interpretation through orthogonal polynomials is due to Carl Jacobi (1826). The word *quadrature* itself recalls the Greek problem of constructing a square with the same area as a given figure.

10.2.5 Adaptive quadrature in SciPy

In practice one rarely codes a fixed rule. `scipy.integrate.quad` wraps the Fortran library QUADPACK: it applies a 21-point Gauss–Kronrod pair on an interval, uses the difference between the two embedded rules as an error estimate, and bisects the sub-interval with the largest estimated error until a tolerance is met. Infinite ranges are mapped to finite ones by a change of variable. `dblquad` and `tplquad` nest `quad` calls; `nquad` handles general dimension. Listing 10.4 computes an expected shortfall below a poverty line, a bivariate normal probability, and an oscillatory integral for which a too-small subdivision limit produces a warning and a wrong answer with the wrong sign.

Listing 10.4: Adaptive quadrature with `quad` and `dblquad` ch10/ch10_scipy_quad.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Adaptive quadrature with scipy.integrate.quad and dblquad.
# (c) 2026 Stephen E. Moore. MIT Licence.
```

```python
import warnings
import numpy as np
from scipy import integrate, stats

# Expected shortfall E[max(c - Y, 0)] of income Y below a line c,
# Y lognormal: infinite range, kinked integrand
mu, s, c = 7.0, 0.8, 900.0
pdf = lambda y: stats.lognorm.pdf(y, s, scale=np.exp(mu))
out = integrate.quad(lambda y: max(c - y, 0.0) * pdf(y), 0, np.inf,
                     full_output=True)
print(f"shortfall, quad on [0,inf): {out[0]:.6f}  est.err {out[1]:.1e}"
      f"  evals {out[2]['neval']}")
val2, err2 = integrate.quad(lambda y: (c - y) * pdf(y), 0, c)
d1 = (np.log(c) - mu) / s
closed = c * stats.norm.cdf(d1) - np.exp(mu + s**2 / 2) * stats.norm.cdf(d1 - s)
print(f"shortfall, quad on [0,c]  : {val2:.6f}  closed form {closed:.6f}")

# Bivariate normal probability with dblquad versus scipy's mvn cdf
rho, a1, a2 = 0.6, 0.5, -0.2
mvn = stats.multivariate_normal(mean=[0, 0], cov=[[1.0, rho], [rho, 1.0]])
p_dbl, e_dbl = integrate.dblquad(lambda y, x: mvn.pdf([x, y]),
                                 -8, a1, lambda x: -8, lambda x: a2)
print(f"P(X1<{a1}, X2<{a2}), rho={rho}:")
print(f"   dblquad {p_dbl:.8f} (est.err {e_dbl:.1e}),"
      f" mvn.cdf {mvn.cdf([a1, a2]):.8f}")

# Oscillatory integrand: too few subintervals triggers a warning
f = lambda x: np.cos(200 * x) * np.exp(-x)
exact = (1 + 200 * np.exp(-10) * np.sin(2000) - np.exp(-10) * np.cos(2000))
exact /= 1 + 200**2
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    v1 = integrate.quad(f, 0, 10, limit=20)[0]
v2 = integrate.quad(f, 0, 10, limit=500)[0]
print(f"oscillatory, limit=20 : {v1: .10e}  warnings: {len(caught)}")
print(f"oscillatory, limit=500: {v2: .10e}")
print(f"exact                 : {exact: .10e}")
```

Output

```
shortfall, quad on [0,inf): 139.378424  est.err 3.9e-07  evals 645
shortfall, quad on [0,c]  : 139.378424  closed form 139.378424
P(X1<0.5, X2<-0.2), rho=0.6:
   dblquad 0.37422109 (est.err 5.4e-09), mvn.cdf 0.37422109
oscillatory, limit=20 : -2.9060023914e-02  warnings: 1
oscillatory, limit=500:  2.5210905434e-05
exact                 :  2.5210905434e-05
```

△ Caution: Read the warnings and the error estimate

`quad` returns a pair `(value, abserr)`. The error estimate is heuristic: it can be badly optimistic for integrands with narrow spikes the initial nodes miss (a density concentrated far from where `quad` looks, for example a posterior with n large), and it is meaningless once an `IntegrationWarning` has been issued. Supply `points=` for known kinks, rescale integrands so the mass is near the origin, and always check against a second method.

10.2.6 The curse of dimensionality

Tensor-product rules extend quadrature to $[0,1]^d$: take an m-point rule in each coordinate and use all m^d combinations of nodes with product weights. If the one-dimensional rule has error $O(m^{-r})$, the product rule with $N = m^d$ points has error $O(N^{-r/d})$. For fixed accuracy the cost grows *exponentially* in d: this is Richard Bellman's *curse of dimensionality*. Listing 10.5 shows that a 4-point Gauss rule per coordinate remains accurate on a smooth product integrand, but the node count 4^d explodes; a 10-point rule in $d = 20$ dimensions would take thousands of years at a billion evaluations per second.

Listing 10.5: Tensor-product quadrature as dimension grows ch10/ch10_curse_dimensionality.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Tensor-product Gauss rules versus Monte Carlo as the dimension grows.
# (c) 2026 Stephen E. Moore. MIT Licence.
import itertools
import numpy as np

rng = np.random.default_rng(803)
g = lambda u: 0.5 * np.pi * np.sin(np.pi * u)   # integrates to 1 on [0,1]
m = 4                                           # Gauss nodes per dimension
t, w = np.polynomial.legendre.leggauss(m)
t, w = 0.5 * (t + 1), 0.5 * w                   # map nodes to [0,1]

print(" d   tensor nodes   tensor error   MC error (same n)")
for d in range(1, 9):
    nodes = np.array(list(itertools.product(t, repeat=d)))
    wts = np.prod(np.array(list(itertools.product(w, repeat=d))), axis=1)
    est = wts @ np.prod(g(nodes), axis=1)
    n = m**d
    mc = np.prod(g(rng.random((n, d))), axis=1).mean()
    print(f"{d:2d} {n:13d}   {abs(est - 1):12.2e}   {abs(mc - 1):12.2e}")

print("\n10-point tensor rule: nodes and time at 1e9 evaluations/second")
for d in (1, 2, 5, 10, 20, 50):
    secs = 10.0**d / 1e9
    yrs = secs / (3600 * 24 * 365.25)
    print(f" d = {d:2d}: n = 1e{d:<3d} time = {secs:8.1e} s = {yrs:8.1e} years")
```

Output

```
 d   tensor nodes   tensor error   MC error (same n)
 1             4       7.89e-06       4.74e-01
 2            16       1.58e-05       4.59e-02
 3            64       2.37e-05       1.29e-01
 4           256       3.15e-05       5.97e-02
 5          1024       3.94e-05       6.76e-02
 6          4096       4.73e-05       1.01e-02
 7         16384       5.52e-05       1.74e-04
 8         65536       6.31e-05       1.34e-02

10-point tensor rule: nodes and time at 1e9 evaluations/second
 d =  1: n = 1e1   time =  1.0e-08 s =  3.2e-16 years
 d =  2: n = 1e2   time =  1.0e-07 s =  3.2e-15 years
 d =  5: n = 1e5   time =  1.0e-04 s =  3.2e-12 years
 d = 10: n = 1e10  time =  1.0e+01 s =  3.2e-07 years
 d = 20: n = 1e20  time =  1.0e+11 s =  3.2e+03 years
 d = 50: n = 1e50  time =  1.0e+41 s =  3.2e+33 years
```

⋆ Key Concept: Quadrature: accuracy versus dimension

In one dimension, the composite trapezoidal rule has error $O(n^{-2})$, Simpson's rule $O(n^{-4})$ and n-point Gaussian quadrature is exact for polynomials of degree $2n-1$ and converges geometrically for analytic integrands. Gauss–Hermite quadrature computes normal expectations with a handful of nodes. But tensor-product rules need m^d points, so their error is $O(N^{-r/d})$: in more than a few dimensions deterministic quadrature is hopeless, and we need a method whose error rate does not depend on d.

Graduate Perspective: Sparse grids and their limits

Smolyak's construction (1963) combines low-order tensor rules so that only $O(m(\log m)^{d-1})$ points are needed for functions with bounded mixed derivatives, and sparse grids are used in economics to solve dynamic stochastic models in moderate dimension (Judd, 1998). Heiss and Winschel (2008) show that sparse-grid integration can beat simulation for mixed-logit likelihoods in dimensions up to about ten. In higher dimension, or for non-smooth integrands, Monte Carlo and quasi-Monte Carlo (Sections 10.3 and 10.4) take over.

Exercises

10.2.1 Theory Show that Simpson's rule on $[-1,1]$ integrates $1, x, x^2, x^3$ exactly but not x^4, and use the error $\int_{-1}^{1} x^4\,dx - S(x^4)$ to identify the constant $1/180$ in Theorem 10.4.

10.2.2 Theory Prove that $S_{2n} = (4T_{2n} - T_n)/3$, and explain why this is Richardson extrapolation.

10.2.3 Theory Derive the two-point Gauss–Legendre rule on $[-1,1]$ from scratch by imposing exactness for $1, x, x^2, x^3$. Verify that the nodes are the zeros of $P_2(x) = (3x^2-1)/2$.

10.2.4 Python Use `np.polynomial.hermite.hermgauss` to compute the certainty equivalent of income Y with $\log Y \sim \mathcal{N}(7, 0.8^2)$ for CRRA coefficients $\gamma \in \{0.5, 1, 2, 4\}$. Compare with the closed form $e^{\mu + (1-\gamma)s^2/2}$ and interpret the risk premium.

10.2.5 Python The trapezoidal rule on $\mathbb{R}$ with spacing h is $h\sum_k f(kh)$. Apply it to $\int e^{-x^2/2}\,dx = \sqrt{2\pi}$ for $h = 1, 0.5, 0.25$ and explain why the convergence is so much faster than (10.8).

10.2.6 Python Compute the two-dimensional random-coefficients probit probability $\mathbb{E}[\Phi(a + b_1 z_1 + b_2 z_2)]$ with independent normal b_1, b_2 by a tensor Gauss–Hermite rule, and check it against the closed form generalising Example 10.1.

10.2.7 Policy A pension fund's actuary integrates expected discounted benefits over normal shocks to wages, returns and inflation for each of 40 future years. Explain why a tensor-product rule is infeasible, and propose an alternative.

10.2.8 ★ Graduate Implement the Golub–Welsch algorithm for Gauss–Hermite nodes using the recurrence $H_{k+1} = 2xH_k - 2kH_{k-1}$ and `np.linalg.eigh`; compare your nodes and weights with `hermgauss` for $n = 20$.

10.3 Monte Carlo Integration

Monte Carlo integration rests on a single observation: if $I = \mathbb{E}_p[h(X)]$ and we can draw $X_1, X_2, \ldots \overset{\text{iid}}{\sim} p$, then the law of large numbers tells us that the sample average of $h(X_i)$ converges to I, and the central limit theorem tells us how fast.

10.3.1 The estimator and its error

Definition 10.6 (Monte Carlo estimator)**.** Let $X_1, \dots, X_n \overset{\text{iid}}{\sim} p$ and suppose $\mathbb{E}_p\,|h(X)| < \infty$. The (crude or plain) Monte Carlo estimator of $I = \mathbb{E}_p[h(X)]$ is

$$\hat{I}_n \;=\; \frac{1}{n}\sum_{i=1}^{n} h(X_i). \tag{10.11}$$

Any integral can be put in this form. For $\int_{[0,1]^d} f(\mathbf{u})\,\mathrm{d}\mathbf{u}$ take p uniform on the cube and $h = f$. For $\int_{\mathbb{R}} f(x)\,\mathrm{d}x$ choose any density p that is positive where $f \neq 0$ and take $h = f/p$; this is the starting point of importance sampling (Section 10.4).

Theorem 10.7 (Properties of the Monte Carlo estimator)**.** *Let $\sigma^2 = \mathrm{Var}_p[h(X)]$.*

(i) $\hat{I}_n$ is unbiased: $\mathbb{E}[\hat{I}_n] = I$.

(ii) If $\sigma^2 < \infty$, $\mathrm{Var}(\hat{I}_n) = \sigma^2/n$, so the root-mean-square error is $\sigma/\sqrt{n}$.

(iii) $\hat{I}_n \xrightarrow{\text{a.s.}} I$ as $n \to \infty$ (strong consistency).

(iv) If $0 < \sigma^2 < \infty$, $\sqrt{n}(\hat{I}_n - I) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$, and with $\hat{\sigma}_n^2 = \frac{1}{n-1}\sum_i \big(h(X_i) - \hat{I}_n\big)^2$, $\sqrt{n}(\hat{I}_n - I)/\hat{\sigma}_n \xrightarrow{d} \mathcal{N}(0,1)$.

Proof. (i) By linearity, $\mathbb{E}[\hat{I}_n] = \frac{1}{n}\sum_i \mathbb{E}[h(X_i)] = \frac{1}{n}\cdot nI = I$. (ii) The $h(X_i)$ are independent, so their variances add: $\mathrm{Var}(\hat{I}_n) = \frac{1}{n^2}\sum_i \mathrm{Var}(h(X_i)) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. Since the estimator is unbiased, its MSE equals its variance. (iii) is Kolmogorov's strong law of large numbers applied to the iid integrable variables $h(X_i)$. (iv) The first statement is the Lindeberg–Lévy CLT. The sample variance $\hat{\sigma}_n^2 \xrightarrow{\text{a.s.}} \sigma^2$ by the strong law applied to $h(X_i)$ and $h(X_i)^2$, so the second statement follows from Slutsky's theorem (Chapter 6). □

Three features of this theorem explain the dominance of Monte Carlo in high-dimensional problems. First, the rate $n^{-1/2}$ *does not depend on the dimension d*; the dimension enters only through the constant σ. Compare $O(N^{-r/d})$ for tensor quadrature: for $d > 2r$, Monte Carlo wins asymptotically. Second, the rate does not require smoothness of h: indicators, kinks and discontinuities are fine, provided $\sigma^2 < \infty$. Third, the *same draws* that produce the estimate also produce an estimate of its error.

Definition 10.8 (Monte Carlo standard error and confidence interval)**.** The Monte Carlo standard error (MCSE) of $\hat{I}_n$ is $\widehat{\mathrm{se}}(\hat{I}_n) = \hat{\sigma}_n/\sqrt{n}$, and an asymptotic $(1-\alpha)$ confidence interval for I is $\hat{I}_n \pm z_{1-\alpha/2}\,\hat{\sigma}_n/\sqrt{n}$.

The price of dimension-free convergence is that the rate is slow: to gain one more correct decimal digit we need 100 times as many draws. This is the motivation for variance reduction (Section 10.4): halving σ is worth as much as quadrupling n. It also gives a simple rule for planning: to achieve a standard error δ we need $n \approx \sigma^2/\delta^2$ draws, with σ estimated from a pilot run.

10.3.2 Estimating π

The classic example takes $U = (U_1, U_2)$ uniform on $[0,1]^2$ and $h(U) = 4 \cdot \mathbf{1}\{U_1^2 + U_2^2 \le 1\}$. Then $\mathbb{E}[h(U)] = 4 \cdot \pi/4 = \pi$ and $\sigma^2 = 16 \cdot \frac{\pi}{4}(1 - \frac{\pi}{4}) = \pi(4-\pi)$, so $\sigma \approx 1.642$. Listing 10.6 reports the estimate, its MCSE and a 95% interval for $n = 10^2, \dots, 10^7$. The error falls by about $\sqrt{10} \approx 3.16$

per row. It then checks the *frequentist* claim behind the interval: across 2000 independent replications with $n = 1000$, the nominal 95% interval covers π in 94.4% of cases, within Monte Carlo error of 95%. Figure 10.3 shows a running estimate inside the band $\pi \pm 1.96\sigma/\sqrt{n}$.

Listing 10.6: Monte Carlo estimation of π with standard errors ch10/ch10_mc_pi.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Monte Carlo estimate of pi with standard errors, CIs and a coverage check.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)


def mc_pi(n, rng):
    u = rng.random((n, 2))
    h = 4.0 * (u[:, 0] ** 2 + u[:, 1] ** 2 <= 1.0)  # h(U) = 4 * indicator
    return h.mean(), h.std(ddof=1) / np.sqrt(n)


print("        n    estimate     MC s.e.   95% CI               |error|")
for n in 10 ** np.arange(2, 8):
    est, se = mc_pi(n, rng)
    print(f"{n:9d}  {est:10.6f}  {se:10.6f}   [{est - 1.96 * se:8.5f},"
          f" {est + 1.96 * se:8.5f}]  {abs(est - np.pi):.2e}")

# Do nominal 95% intervals cover pi 95% of the time? (R = 2000 replications)
R, n = 2000, 1000
u = rng.random((R, n, 2))
h = 4.0 * (u[..., 0] ** 2 + u[..., 1] ** 2 <= 1.0)
est, se = h.mean(axis=1), h.std(axis=1, ddof=1) / np.sqrt(n)
cover = np.mean(np.abs(est - np.pi) <= 1.96 * se)
print(f"\ncoverage of 95% CI, n={n}: {cover:.3f}"
      f"  (MC s.e. of coverage {np.sqrt(cover * (1 - cover) / R):.3f})")
print(f"theoretical sd of h(U): {4 * np.sqrt(np.pi / 4 * (1 - np.pi / 4)):.4f};"
      f" mean estimated sd: {np.mean(se) * np.sqrt(n):.4f}")

# Figure: running estimate with a 95% band
n = 20000
u = rng.random((n, 2))
h = 4.0 * (u[:, 0] ** 2 + u[:, 1] ** 2 <= 1.0)
k = np.arange(1, n + 1)
run = np.cumsum(h) / k
sd = np.sqrt(np.pi * (4 - np.pi))
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(k, run, color="navy", lw=1, label="running MC estimate")
ax.fill_between(k, np.pi - 1.96 * sd / np.sqrt(k), np.pi + 1.96 * sd / np.sqrt(k),
                color="darkorange", alpha=0.25, label="pi +/- 1.96 sd/sqrt(n)")
ax.axhline(np.pi, color="black", lw=0.8)
ax.set_xscale("log")
ax.set_ylim(2.6, 3.7)
ax.set_xlabel("number of draws n")
ax.set_ylabel("estimate of pi")
ax.legend()
plt.savefig("ch10_mc_pi.pdf", bbox_inches="tight")
```

Output

```
        n    estimate     MC s.e.   95% CI                |error|
      100    3.000000    0.174078   [ 2.65881,  3.34119]  1.42e-01
     1000    3.120000    0.052425   [ 3.01725,  3.22275]  2.16e-02
    10000    3.156800    0.016316   [ 3.12482,  3.18878]  1.52e-02
   100000    3.144240    0.005187   [ 3.13407,  3.15441]  2.65e-03
  1000000    3.140088    0.001643   [ 3.13687,  3.14331]  1.50e-03
 10000000    3.141807    0.000519   [ 3.14079,  3.14282]  2.14e-04

coverage of 95% CI, n=1000: 0.944  (MC s.e. of coverage 0.005)
theoretical sd of h(U): 1.6422; mean estimated sd: 1.6413
```

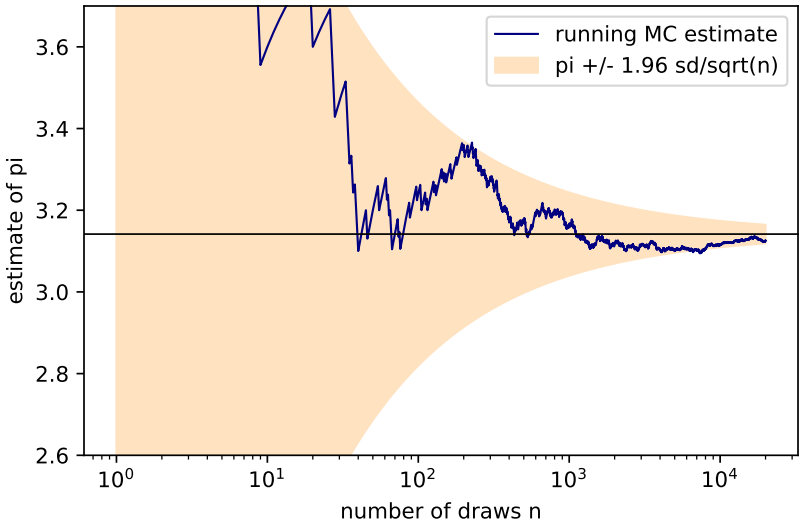


Figure 10.3: Running Monte Carlo estimate of π (log scale for n) with the band $\pi \pm 1.96\,\sigma/\sqrt{n}$. The band narrows like $n^{-1/2}$, whatever the dimension.

Historical Note: From Buffon's needle to Los Alamos

Georges-Louis Leclerc, Comte de Buffon, posed in 1733 and published in 1777 the problem of a needle of length ℓ dropped on a floor ruled with parallel lines a distance $d \geq \ell$ apart; it crosses a line with probability $2\ell/(\pi d)$, so repeated throws estimate π. In 1908 W. S. Gosset ("Student") checked his derivation of the t distribution partly by a sampling experiment: he wrote physical measurements of 3000 criminals on cards, shuffled them and drew 750 samples of size four. The modern method was born at Los Alamos after the Second World War. Stanislaw Ulam, convalescing in 1946 and wondering about the chance of winning a game of solitaire, realised that playing many random games might be more practical than combinatorics; with John von Neumann he developed the idea for neutron diffusion problems on the ENIAC computer. Nicholas Metropolis suggested the code name "Monte Carlo", after the casino where Ulam's uncle liked to gamble, and Metropolis and Ulam published "The Monte Carlo method" in 1949 (Metropolis and Ulam, 1949).

10.3.3 High-dimensional integrals: the rate is free, the constant is not

Listing 10.7 integrates two functions over $[0,1]^d$, both with integral 1. For the average $f_1(\mathbf{u}) = \frac{3}{d}\sum_j u_j^2$ the variance *falls* with d (it is $4/(5d)$), so Monte Carlo becomes easier in high dimension. For the product $f_2(\mathbf{u}) = \prod_j \frac{\pi}{2}\sin(\pi u_j)$ the variance is $(\pi^2/8)^d - 1$, which grows exponentially: with $d = 50$ it is about 3.6×10^4, and $n = 10^5$ draws give an estimate of 0.94 with a reported

standard error that is itself far too small. The rate is dimension-free; the constant σ is not, and it depends on how the integrand's mass is spread.

△ Caution: Heavy tails and the reported standard error

The MCSE $\hat{\sigma}_n/\sqrt{n}$ is itself a Monte Carlo estimate. When $h(X)$ is heavy-tailed or its mass is concentrated on a small, rarely sampled region, $\hat{\sigma}_n$ systematically underestimates σ (the rare large values that dominate the variance have not been seen), and the estimate and its interval are both wrong while looking precise. In the $d = 50$ row the reported s.e. is 0.17, whereas the true s.d. of the estimate is $\sqrt{3.6 \times 10^4/10^5} \approx 0.6$. If $\operatorname{Var} h(X) = \infty$ (e.g. Pareto losses with tail index below 2) the CLT fails altogether. Diagnose with plots of the running mean and of the largest values of $h(X_i)$.

10.3.4 Simulated probabilities

By (10.2), a probability is an expectation of an indicator and is estimated by a frequency, $\hat{p} = \frac{1}{R}\sum_r \mathbf{1}\{X_r \in A\}$, with $\operatorname{se}(\hat{p}) = \sqrt{p(1-p)/R}$. In the multinomial probit model of discrete choice, household i chooses alternative j with probability $\mathbb{P}(U_{ij} > U_{ik}\ \forall k \neq j)$, where utilities $U = V + \varepsilon$ have correlated normal errors $\varepsilon \sim \mathcal{N}(\mathbf{0}, \Omega)$. With J alternatives this is a $(J-1)$-dimensional normal orthant probability. Listing 10.7 computes it for $J = 3$ in two ways: exactly, by differencing utilities and calling a bivariate normal cdf, and with the *frequency simulator* that draws ε via the Cholesky factor of Ω and records which alternative wins.

Listing 10.7: Monte Carlo in high dimension and simulated choice probabilities
ch10/ch10_mc_highdim.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Monte Carlo in high dimension and simulated choice probabilities.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
n = 100_000

# Two integrands on [0,1]^d with known integral 1:
#   f1 = (3/d) sum u_j^2            (variance falls with d)
#   f2 = prod (pi/2) sin(pi u_j)    (variance grows like (pi^2/8)^d)
print("  d   f1 estimate  s.e.      f2 estimate  s.e.      Var f2 (theory)")
for d in (1, 5, 20, 50):
    u = rng.random((n, d))
    f1 = 3.0 * (u**2).mean(axis=1)
    f2 = np.prod(0.5 * np.pi * np.sin(np.pi * u), axis=1)
    v2 = (np.pi**2 / 8) ** d - 1
    print(f"{d:3d}   {f1.mean():9.5f}  {f1.std() / np.sqrt(n):.1e}"
          f"   {f2.mean():9.5f}  {f2.std() / np.sqrt(n):.1e}   {v2:10.3e}")

# Simulated choice probability: multinomial probit with 3 alternatives.
# U_j = V_j + e_j, e ~ N(0, Omega); P(choose 1) = P(U1 > U2, U1 > U3).
V = np.array([0.5, 0.2, 0.0])
Om = np.array([[1.0, 0.5, 0.2], [0.5, 1.0, 0.3], [0.2, 0.3, 1.0]])
D = np.array([[1.0, -1.0, 0.0], [1.0, 0.0, -1.0]])   # differences U1-U2, U1-U3
mD, SD = D @ V, D @ Om @ D.T
# P(D e > -D V) = P(W < D V) with W = -D e ~ N(0, SD)
exact = stats.multivariate_normal(mean=[0, 0], cov=SD).cdf(mD)
L = np.linalg.cholesky(Om)
print("\nmultinomial probit P(choose alternative 1)")
print(f"   bivariate normal cdf: {exact:.5f}")
```

```
for R in (100, 1000, 10000, 100000):
    e = rng.standard_normal((R, 3)) @ L.T
    U = V + e
    ind = (U.argmax(axis=1) == 0).astype(float)
    p, se = ind.mean(), ind.std(ddof=1) / np.sqrt(R)
    print(f"   frequency simulator R={R:6d}: {p:.5f}  (s.e. {se:.5f})")
```

Output

```
  d   f1 estimate  s.e.      f2 estimate  s.e.      Var f2 (theory)
  1     0.99664  2.8e-03     1.00061  1.5e-03    2.337e-01
  5     0.99959  1.3e-03     0.99842  4.3e-03    1.858e+00
 20     1.00064  6.3e-04     1.00679  2.8e-02    6.571e+01
 50     1.00026  4.0e-04     0.93697  1.7e-01    3.635e+04

multinomial probit P(choose alternative 1)
   bivariate normal cdf: 0.47498
   frequency simulator R=   100: 0.53000  (s.e. 0.05016)
   frequency simulator R=  1000: 0.48600  (s.e. 0.01581)
   frequency simulator R= 10000: 0.47600  (s.e. 0.00499)
   frequency simulator R=100000: 0.47502  (s.e. 0.00158)
```

With $R = 100$ the simulator is off by 0.055, about one standard error; with $R = 10^5$ the error is 4×10^{-5}. The frequency simulator is unbiased but a step function of the parameters, which makes it awkward inside likelihood maximisation; smooth simulators (logit-kernel smoothing, the GHK simulator) and the method of simulated moments (McFadden, 1989) were developed to handle this. Note also that $\log \hat{p}$ is a *biased* estimator of $\log p$ (Jensen), so simulated maximum likelihood is consistent only if $R \to \infty$ with the sample size (Train, 2009).

10.3.5 Risk assessment of a public investment

Monte Carlo is the standard way to turn uncertainty about the inputs of a cost–benefit analysis into a distribution of its outputs. Listing 10.8 simulates a hypothetical irrigation project with uncertain capital cost (triangular, with an overrun tail), a delay of 0–2 years, lognormal first-year benefits, uncertain growth and a 25-year life discounted at 6%. Each of the 10^5 replications computes one NPV; the collection approximates its distribution (Fig. 10.4).

Listing 10.8: Monte Carlo risk assessment of a public investment ch10/ch10_npv_risk.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Monte Carlo risk assessment of a (hypothetical) public irrigation investment.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
R, T = 100_000, 25                      # replications, project life (years)
# Uncertain inputs (illustrative assumptions, monetary units = millions)
cost = rng.triangular(80, 100, 140, R)          # capital cost, overrun risk
delay = rng.integers(0, 3, R)                   # years before benefits start
b0 = rng.lognormal(np.log(9.0), 0.25, R)        # first-year net benefit
growth = rng.normal(0.02, 0.01, R)              # annual benefit growth
rate = 0.06                                     # social discount rate

t = np.arange(1, T + 1)
active = t[None, :] > delay[:, None]
flows = b0[:, None] * (1 + growth[:, None]) ** (t[None, :] - 1) * active
```

```
npv = (flows / (1 + rate) ** t).sum(axis=1) - cost

m, se = npv.mean(), npv.std(ddof=1) / np.sqrt(R)
p_loss = np.mean(npv < 0)
se_p = np.sqrt(p_loss * (1 - p_loss) / R)
q05, q50, q95 = np.percentile(npv, [5, 50, 95])
print(f"expected NPV        : {m:8.2f}  (MC s.e. {se:.2f})")
print(f"P(NPV < 0)          : {p_loss:8.4f}  (MC s.e. {se_p:.4f})")
print(f"5%, 50%, 95% NPV    : {q05:8.2f} {q50:8.2f} {q95:8.2f}")
# Which input matters most? rank correlation of each input with NPV
ranks = lambda z: np.argsort(np.argsort(z))
for name, z in [("cost", cost), ("delay", delay + rng.random(R) * 1e-6),
                ("benefit", b0), ("growth", growth)]:
    rho = np.corrcoef(ranks(z), ranks(npv))[0, 1]
    print(f"   Spearman corr(NPV, {name:8s}) = {rho: .3f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.hist(npv, bins=120, density=True, color="steelblue", alpha=0.8)
ax.axvline(0, color="firebrick", lw=1.5, label=f"P(NPV<0) = {p_loss:.3f}")
ax.axvline(m, color="black", ls="--", lw=1, label=f"E[NPV] = {m:.1f}")
ax.set_xlabel("net present value (millions, simulated)")
ax.set_ylabel("density")
ax.legend()
plt.savefig("ch10_npv_risk.pdf", bbox_inches="tight")
```

Output

```
expected NPV        :    29.13  (MC s.e. 0.13)
P(NPV < 0)          :   0.2391  (MC s.e. 0.0013)
5%, 50%, 95% NPV    :   -28.04    24.77   101.01
   Spearman corr(NPV, cost    ) = -0.318
   Spearman corr(NPV, delay   ) = -0.170
   Spearman corr(NPV, benefit ) =  0.841
   Spearman corr(NPV, growth  ) =  0.343
```

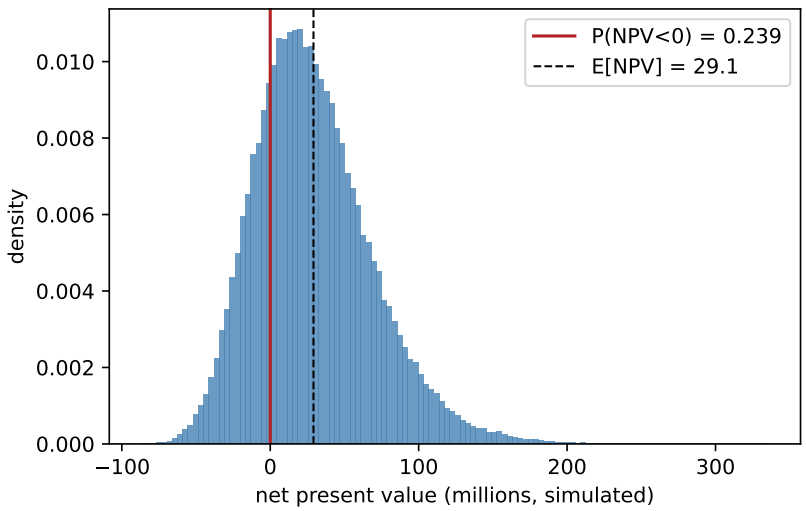


Figure 10.4: Simulated distribution of the net present value of a hypothetical irrigation project (illustrative inputs). The expected NPV is positive, but the project loses money in about one replication in four.

Policy Lens: Expected NPV is not the whole story

The simulated project has $\mathbb{E}[NPV] \approx 29$ (MCSE 0.13) yet $\mathbb{P}(NPV < 0) \approx 0.24$, and its 5th percentile is -28. A decision maker who is risk-neutral and can pool many projects cares about the mean; a finance ministry facing a binding budget or a donor with a reputational stake may care about the probability of loss or the lower tail. Rank correlations (Spearman) between each input and the NPV, a simple global sensitivity analysis, show that the first-year benefit dominates the risk, so better demand studies would reduce uncertainty more than tighter cost control. Report the Monte Carlo error of every simulated figure: here 0.0013 on the loss probability, which is negligible relative to the uncertainty in the inputs themselves.

★ Key Concept: Monte Carlo integration

$\hat{I}_n = \frac{1}{n}\sum_i h(X_i)$ with $X_i \overset{\text{iid}}{\sim} p$ is unbiased for $\mathbb{E}_p[h(X)]$, has variance σ^2/n, and is asymptotically normal. Report it with its MCSE $\hat{\sigma}_n/\sqrt{n}$ and a CLT-based interval. The $n^{-1/2}$ rate is independent of dimension and smoothness, which is why Monte Carlo is the method of choice beyond a few dimensions; but the constant σ can grow with dimension, and heavy tails can make both the estimate and its standard error unreliable.

Exercises

10.3.1 Theory Show that for the estimator of π, $\sigma^2 = \pi(4-\pi)$. How many draws are needed for a 95% interval of half-width 10^{-4}?

10.3.2 Theory For $f_2(\mathbf{u}) = \prod_{j=1}^{d} \frac{\pi}{2}\sin(\pi u_j)$ prove that $\text{Var}\, f_2(\mathbf{U}) = (\pi^2/8)^d - 1$ when $\mathbf{U}$ is uniform on $[0,1]^d$. For what d does the relative standard error with $n = 10^6$ exceed 10%?

10.3.3 Theory Prove that the sample variance $\hat{\sigma}_n^2$ is unbiased for σ^2, and use the delta method to find the asymptotic distribution of $\hat{\sigma}_n$ when $\mathbb{E}h(X)^4 < \infty$.

10.3.4 Python Simulate Buffon's needle with $\ell = d = 1$ (draw the distance of the centre to the nearest line and the angle) and estimate π by $2n/(\text{number of crossings})$. Why is this estimator biased, and how large is its standard error compared with Listing 10.6?

10.3.5 Python Estimate the expected poverty gap of lognormal income ($\mu = 7$, $s = 0.8$, $z = 900$) by Monte Carlo with $n = 10^4$, report the MCSE and a 95% interval, and compare with the closed form from Exercise 10.1.3.

10.3.6 Policy In Listing 10.8, replace the 6% discount rate by a random rate uniform on [3%, 9%]. How do $\mathbb{E}[NPV]$ and $\mathbb{P}(NPV < 0)$ change? Discuss whether discount-rate uncertainty belongs inside the simulation or in a separate sensitivity table.

10.3.7 ★ Graduate (Simulated maximum likelihood bias.) Let $\hat{p}_R$ be the frequency simulator of a choice probability p with R draws. Use a second-order Taylor expansion to show that $\mathbb{E}[\log \hat{p}_R] - \log p \approx -(1-p)/(2Rp)$, and explain why simulated ML with N observations is $\sqrt{N}$-consistent only if $\sqrt{N}/R \to 0$.

10.4 Variance Reduction and Quasi-Monte Carlo

Since the Monte Carlo error is $\sigma/\sqrt{n}$, there are only two ways to reduce it: increase n or decrease σ. Variance-reduction techniques construct a different estimator of the same I whose variance per unit of computing cost is smaller. We compare methods by their *efficiency*: if estimator A has

variance v_A/n at cost c_A per draw, the relative efficiency of A to B is $(v_B c_B)/(v_A c_A)$, the factor by which B would have to increase its sample size to match A.

10.4.1 Antithetic variates

If $U \sim \mathrm{U}(0,1)$ then so is $1-U$; if $Z \sim \mathcal{N}(0,1)$ so is $-Z$. The *antithetic* estimator averages h over such pairs:

$$\hat{I}_n^{\mathrm{AV}} \;=\; \frac{1}{n/2}\sum_{i=1}^{n/2} \frac{h(U_i)+h(1-U_i)}{2}, \tag{10.12}$$

which uses n evaluations of h. It is unbiased, and with $\rho = \mathrm{Corr}\big(h(U), h(1-U)\big)$,

$$\mathrm{Var}\left(\hat{I}_n^{\mathrm{AV}}\right) \;=\; \frac{2}{n}\cdot\frac{2\sigma^2+2\rho\sigma^2}{4} \;=\; \frac{\sigma^2}{n}(1+\rho).$$

So antithetic sampling helps exactly when $\rho < 0$, and can at worst double the variance ($\rho = 1$).

Proposition 10.9 (Antithetic variates help for monotone integrands)**.** *If h is monotone on $[0,1]$ then $\mathrm{Cov}\big(h(U), h(1-U)\big) \le 0$, so the antithetic estimator is at least as efficient as plain Monte Carlo.*

Proof. Let h be non-decreasing (otherwise replace h by $-h$) and $g(u) = -h(1-u)$, also non-decreasing. Let U, V be iid uniform. Since h and g are both non-decreasing, $\big(h(U)-h(V)\big)\big(g(U)-g(V)\big) \ge 0$ pointwise. Taking expectations and expanding, $2\mathbb{E}[h(U)g(U)] - 2\mathbb{E}[h(U)]\mathbb{E}[g(U)] \ge 0$, i.e. $\mathrm{Cov}(h(U), g(U)) \ge 0$, which is $\mathrm{Cov}\big(h(U), h(1-U)\big) \le 0$. □

10.4.2 Control variates

Suppose we know the mean $\mu_g = \mathbb{E}[g(X)]$ of some other function g that is correlated with h. For any constant c,

$$\hat{I}_n^{\mathrm{CV}}(c) \;=\; \frac{1}{n}\sum_{i=1}^{n}\Big[h(X_i) + c\big(g(X_i)-\mu_g\big)\Big] \tag{10.13}$$

is unbiased for I, because the added term has mean zero. Its variance is $\frac{1}{n}\big[\sigma_h^2 + c^2\sigma_g^2 + 2c\ \mathrm{Cov}(h,g)\big]$, a quadratic in c.

Theorem 10.10 (Optimal control-variate coefficient)**.** *The variance of (10.13) is minimised by*

$$c^\star \;=\; -\frac{\mathrm{Cov}\big(h(X), g(X)\big)}{\mathrm{Var}\big(g(X)\big)}, \qquad \textit{giving} \qquad \mathrm{Var}\left(\hat{I}_n^{\mathrm{CV}}(c^\star)\right) \;=\; \frac{\sigma_h^2}{n}\big(1-\rho_{hg}^2\big),$$

where $\rho_{hg} = \mathrm{Corr}(h(X), g(X))$. The variance-reduction factor is $1/(1-\rho_{hg}^2)$.

Proof. Differentiate $v(c) = \sigma_h^2 + c^2\sigma_g^2 + 2c\sigma_{hg}$: $v'(c) = 2c\sigma_g^2 + 2\sigma_{hg} = 0$ gives $c^\star = -\sigma_{hg}/\sigma_g^2$, a minimum since $v'' = 2\sigma_g^2 > 0$. Substituting, $v(c^\star) = \sigma_h^2 - \sigma_{hg}^2/\sigma_g^2 = \sigma_h^2(1-\rho_{hg}^2)$. □

In practice $c^\star$ is estimated from the same draws, which is exactly the OLS slope of $h(X_i)$ on $g(X_i)$ with a minus sign: the control-variate estimator is the intercept of a regression of h on $g - \mu_g$. Estimating c introduces a bias of order $1/n$, negligible relative to the standard error. With several controls, $c^\star$ becomes a vector of regression coefficients and $1-\rho^2$ becomes $1-R^2$ of that regression.

Listing 10.9 first treats the textbook integrand $\mathbb{E}[e^U] = e - 1$: antithetic sampling reduces the variance per evaluation from 0.244 to 0.002 (a factor of over 100, as e^u is monotone and nearly linear) and the control $g(U) = U$ by a factor of about 60. It then prices a stylised *rainfall index-insurance* contract that pays $\max(K-R, 0)$ when seasonal rainfall R (lognormal, median

600 mm) falls below a trigger $K = 500$ mm. The payout is a monotone function of the normal shock, so antithetic draws help (gain ≈ 5); rainfall itself, whose mean is known, is a control variate with correlation -0.60 and gain $1/(1 - 0.36) \approx 1.55$, matching the theory.

Listing 10.9: Antithetic and control variates ch10/ch10_variance_reduction.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Antithetic variates and control variates for a rainfall index-insurance payout.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)

# Warm-up: theta = E[exp(U)], U ~ U(0,1), theta = e - 1
n = 10_000
u = rng.random(n)
plain = np.exp(u)
anti = 0.5 * (np.exp(u[: n // 2]) + np.exp(1 - u[: n // 2]))   # n/2 pairs
c = -np.cov(plain, u)[0, 1] / np.var(u, ddof=1)
cv = plain + c * (u - 0.5)
print("E[exp(U)] = e - 1 =", f"{np.e - 1:.6f}")
print(f"  plain      {plain.mean():.6f}  var/eval {plain.var():.5f}")
print(f"  antithetic {anti.mean():.6f}  var/eval {anti.var() / 2:.5f}")
print(f"  control    {cv.mean():.6f}  var/eval {cv.var():.5f}  c* = {c:.4f}")

# Index insurance: seasonal rainfall R = exp(m + s Z); payout max(K - R, 0)
m, s, K = np.log(600.0), 0.3, 500.0
d = (np.log(K) - m) / s
exact = K * stats.norm.cdf(d) - np.exp(m + s**2 / 2) * stats.norm.cdf(d - s)
z = rng.standard_normal(n)
pay = lambda zz: np.maximum(K - np.exp(m + s * zz), 0.0)
y = pay(z)
ya = 0.5 * (pay(z[: n // 2]) + pay(-z[: n // 2]))
# control variate: rainfall R itself, whose mean is known exactly
r_ = np.exp(m + s * z)
c = -np.cov(y, r_)[0, 1] / np.var(r_, ddof=1)
ycv = y + c * (r_ - np.exp(m + s**2 / 2))
rho = np.corrcoef(y, r_)[0, 1]
print(f"\nexpected payout, exact: {exact:.4f}")
print("method        estimate    s.e.     var/eval   efficiency gain")
v0 = y.var()
for name, est, se, v in [
        ("plain", y.mean(), y.std() / np.sqrt(n), v0),
        ("antithetic", ya.mean(), ya.std() / np.sqrt(n // 2), ya.var() / 2),
        ("control var", ycv.mean(), ycv.std() / np.sqrt(n), ycv.var())]:
    print(f"{name:12s}  {est:9.4f}  {se:7.4f}  {v:10.2f}   {v0 / v:6.2f}")
print(f"corr(payout, rainfall) = {rho:.3f}; theory gain 1/(1-rho^2) ="
      f" {1 / (1 - rho**2):.2f}")
```

Output

```
E[exp(U)] = e - 1 = 1.718282
  plain      1.712713  var/eval 0.24394
  antithetic 1.719528  var/eval 0.00197
  control    1.719111  var/eval 0.00389  c* = -1.6869

expected payout, exact: 21.6088
method        estimate    s.e.     var/eval   efficiency gain
plain           21.0828   0.4514     2037.88     1.00
```

```
antithetic      21.0105   0.4056      411.20     4.96
control var     20.8882   0.3623     1312.76     1.55
corr(payout, rainfall) = -0.597; theory gain 1/(1-rho^2) = 1.55
```

△ Caution: Compare at equal cost

An antithetic pair costs two evaluations of h. Comparing the variance of the mean of n pairs with that of n plain draws flatters antithetic sampling by a factor of two. The script therefore reports "variance per evaluation". Similarly, a control variate whose mean must itself be computed numerically, or a stratified design that needs expensive bookkeeping, should be charged for that cost. The honest comparison is variance × computing time.

10.4.3 Importance sampling

Plain Monte Carlo samples where p puts its mass, which may be far from where $h\,p$ puts its mass. To estimate the probability of a one-in-30,000 drought, $n = 10^4$ draws from p will usually see no event at all. *Importance sampling* draws instead from a proposal q and corrects by the likelihood ratio, or *importance weight*, $w(x) = p(x)/q(x)$:

$$I = \int h(x)\frac{p(x)}{q(x)}q(x)\,\mathrm{d}x = \mathbb{E}_q\big[h(X)w(X)\big], \qquad \hat{I}_n^{\mathrm{IS}} = \frac{1}{n}\sum_{i=1}^{n} h(X_i)\,w(X_i), \quad X_i \overset{\text{iid}}{\sim} q. \tag{10.14}$$

Theorem 10.11 (Importance sampling). *Suppose $q(x) > 0$ whenever $h(x)p(x) \neq 0$. Then*

(i) $\hat{I}_n^{\mathrm{IS}}$ is unbiased for I, with variance $\frac{1}{n}\Big(\int \frac{h(x)^2p(x)^2}{q(x)}\,\mathrm{d}x - I^2\Big)$;

(ii) the variance is minimised over densities q by $q^\star(x) = |h(x)|\,p(x)/\int |h(y)|\,p(y)\,\mathrm{d}y$;

(iii) if $h \geq 0$ then $q^\star = hp/I$ and $\hat{I}_n^{\mathrm{IS}} = I$ exactly (zero variance).

Proof. (i) $\mathbb{E}_q[hw] = \int h\frac{p}{q}q = \int hp = I$ (the support condition ensures nothing is lost where $q = 0$), and $\mathbb{E}_q[(hw)^2] = \int h^2p^2/q$. (ii) By the Cauchy–Schwarz inequality,

$$\Big(\int |h|\,p\Big)^2 = \Big(\int \frac{|h|\,p}{\sqrt{q}}\sqrt{q}\Big)^2 \leq \int \frac{h^2p^2}{q}\int q = \int \frac{h^2p^2}{q},$$

so the second moment is at least $(\int |h|\,p)^2$, with equality iff $|h|\,p/\sqrt{q} \propto \sqrt{q}$, i.e. $q \propto |h|\,p$. (iii) If $h \geq 0$, $h(X)w(X) = h(X)p(X)I/(h(X)p(X)) = I$ for every draw. □

The optimal proposal requires I itself, so it is never available; but the theorem says what to aim for: q should be roughly proportional to $|h|\,p$, and, crucially, q must have *heavier tails* than $|h|\,p$, otherwise $\int h^2p^2/q$ can be infinite. When p is known only up to a constant (a posterior), one uses the *self-normalised* estimator $\sum_i h(X_i)w(X_i)/\sum_i w(X_i)$, which is biased but consistent.

Definition 10.12 (Effective sample size). For importance weights $w_1, \dots, w_n$, the (Kong) effective sample size is $\mathrm{ESS} = \big(\sum_i w_i\big)^2 \big/ \sum_i w_i^2 \in [1, n]$. It equals n when all weights are equal and 1 when one weight dominates. Approximately, the self-normalised IS estimate of a generic expectation is as precise as plain Monte Carlo with ESS draws from p (Liu, 2001).

Listing 10.10 estimates $\mathbb{P}(Z > 4) = 3.17 \times 10^{-5}$. Plain Monte Carlo with $n = 10^4$ sees no exceedance. A proposal $\mathcal{N}(4, 1)$ centred on the rare region achieves a relative error of 1.5%; a

proposal that is too narrow and too far out, $\mathcal{N}(6, 0.5^2)$, has light tails relative to the target near $x = 4$ and gives an estimate off by half, with a reported standard error that badly understates the error. The output shows that the generic ESS of the weights is a diagnostic for how well q represents p as a whole, *not* for a particular integrand: the excellent $\mathcal{N}(4, 1)$ proposal has $\mathrm{ESS}(w) = 36$ because it ignores the bulk of p, which this integrand does not need. The integrand-specific version $(\sum h_i w_i)^2 / \sum (h_i w_i)^2$ is the more relevant measure here.

Listing 10.10: Importance sampling for a rare event ch10/ch10_importance_sampling.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Importance sampling for a rare-event probability; effective sample size.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n, a = 10_000, 4.0
exact = stats.norm.sf(a)
print(f"target P(Z > {a}) = {exact:.4e}, n = {n}")

z = rng.standard_normal(n)
h = (z > a).astype(float)
print(f"plain MC           : {h.mean():.4e}  hits {int(h.sum())}")


def importance(mu, sd):
    x = rng.normal(mu, sd, n)
    w = stats.norm.pdf(x) / stats.norm.pdf(x, mu, sd)      # p / q
    hw = (x > a) * w
    est, se = hw.mean(), hw.std(ddof=1) / np.sqrt(n)
    ess = w.sum() ** 2 / (w**2).sum()              # Kong ESS of the weights
    ess_h = hw.sum() ** 2 / (hw**2).sum()          # ESS for this integrand
    return est, se, ess, ess_h


print("proposal          estimate     s.e.      rel.err  ESS(w)  ESS(h*w)")
for mu, sd in [(4.0, 1.0), (4.5, 1.0), (3.0, 2.0), (0.0, 2.0), (6.0, 0.5)]:
    est, se, ess, ess_h = importance(mu, sd)
    print(f"N({mu:.1f}, {sd:.1f}^2)       {est:.4e}  {se:.2e}  "
          f"{abs(est / exact - 1):7.4f}  {ess:6.0f}  {ess_h:8.0f}")

# Optimal proposal q* ~ |h| p: here the normal truncated to (a, inf)
x = stats.truncnorm.rvs(a, np.inf, size=n, random_state=rng)
w = stats.norm.pdf(x) / stats.truncnorm.pdf(x, a, np.inf)
print(f"optimal q* (trunc) : {w.mean():.4e}  s.e. {w.std():.1e} (zero variance)")

# Figure: integrand p(x) 1{x>a}, and two proposals
xx = np.linspace(-3, 8, 500)
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.plot(xx, stats.norm.pdf(xx), color="navy", label="p = N(0,1)")
ax.fill_between(xx, 0, stats.norm.pdf(xx) * (xx > a) * 1e3, color="firebrick",
                alpha=0.4, label="p(x) 1{x>4}  (x 1000)")
ax.plot(xx, stats.norm.pdf(xx, 4, 1), color="darkorange", ls="--",
        label="good q = N(4,1)")
ax.plot(xx, stats.norm.pdf(xx, 6, 0.5), color="gray", ls=":",
        label="poor q = N(6,0.5^2)")
ax.set_xlabel("x")
ax.set_ylabel("density")
ax.legend(fontsize=8)
plt.savefig("ch10_importance_sampling.pdf", bbox_inches="tight")
```

Output

```
target P(Z > 4.0) = 3.1671e-05, n = 10000
plain MC           : 0.0000e+00  hits 0
proposal          estimate     s.e.      rel.err  ESS(w)  ESS(h*w)
N(4.0, 1.0^2)       3.2156e-05  6.74e-07   0.0153      36      1852
N(4.5, 1.0^2)       3.1082e-05  6.87e-07   0.0186       8      1698
N(3.0, 2.0^2)       3.0617e-05  1.06e-06   0.0333    1815       772
N(0.0, 2.0^2)       3.6168e-05  3.28e-06   0.1420    6628       120
N(6.0, 0.5^2)       1.6505e-05  4.31e-06   0.4789      15        15
optimal q* (trunc) : 3.1671e-05  s.e. 3.6e-21 (zero variance)
```

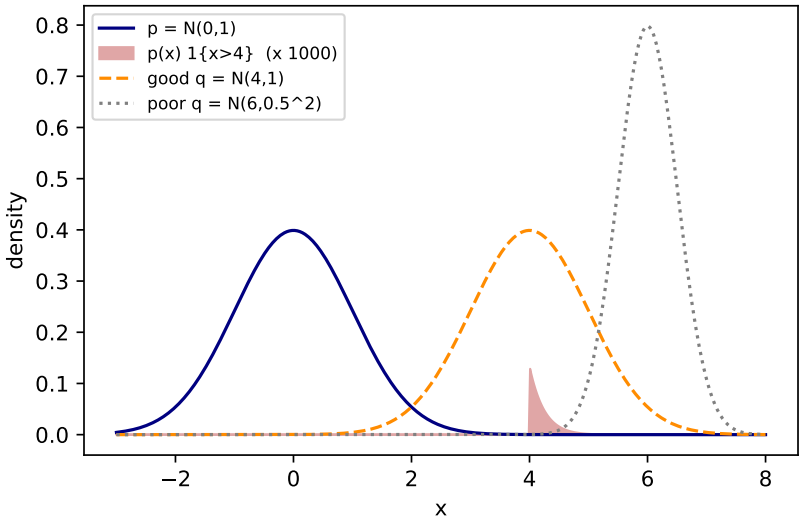


Figure 10.5: The rare-event integrand $p(x)\mathbf{1}\{x > 4\}$ (magnified 1000×) lives far in the tail of $p = \mathcal{N}(0, 1)$. A good proposal covers it with heavier tails; a narrow proposal centred too far out misses the region just above 4 where most of the mass lies.

△ Caution: Importance sampling can fail silently

If the proposal's tails are lighter than those of $|h|\,p$, the variance of the weights is infinite. The estimator is still unbiased and consistent, but convergence is erratic, occasional huge weights cause jumps, and the sample standard error is usually far too small. Always plot the weights, report the largest normalised weight $\max_i w_i / \sum_j w_j$, and prefer proposals with heavier tails (Student-t rather than normal). In high dimension, importance sampling with a fixed proposal degenerates quickly; sequential Monte Carlo and MCMC are the remedies.

10.4.4 Stratified sampling

Partition the domain into strata $A_1, \dots, A_H$ with known probabilities $W_h = \mathbb{P}_p(X \in A_h)$, draw n_h points from p restricted to each stratum, and combine the stratum means:

$$\hat{I}^{\mathrm{ST}} = \sum_{h=1}^{H} W_h \, \bar{Y}_h, \qquad \operatorname{Var}\left(\hat{I}^{\mathrm{ST}}\right) = \sum_{h=1}^{H} \frac{W_h^2 \sigma_h^2}{n_h}, \tag{10.15}$$

where $\bar{Y}_h$ and σ_h^2 are the mean and variance of $h(X)$ within stratum h.

Proposition 10.13 (Stratification never hurts with proportional allocation). *With proportional allocation $n_h = nW_h$, $\mathrm{Var}(\hat{I}^{\mathrm{ST}}) = \frac{1}{n}\sum_h W_h\sigma_h^2 \le \frac{\sigma^2}{n}$. The variance is minimised for fixed n by Neyman allocation $n_h \propto W_h\sigma_h$, giving $\frac{1}{n}\big(\sum_h W_h\sigma_h\big)^2$.*

Proof. By the law of total variance with stratum label S, $\sigma^2 = \mathbb{E}[\mathrm{Var}(h \mid S)] + \mathrm{Var}(\mathbb{E}[h \mid S]) = \sum_h W_h\sigma_h^2 + \sum_h W_h(\mu_h - I)^2 \ge \sum_h W_h\sigma_h^2$. Stratification removes the between-stratum component. For Neyman allocation minimise $\sum_h W_h^2\sigma_h^2/n_h$ subject to $\sum_h n_h = n$: the Lagrangian condition $-W_h^2\sigma_h^2/n_h^2 + \lambda = 0$ gives $n_h \propto W_h\sigma_h$, and substitution gives the minimum (equivalently, apply Cauchy–Schwarz to $\sum_h (W_h\sigma_h/\sqrt{n_h})\sqrt{n_h}$). □

This is precisely the theory of stratified survey sampling: national household surveys stratify by region and urban/rural status for the same reason, and Neyman's allocation (Neyman, 1934) guides how many households to interview in each stratum. Listing 10.11 shows both faces. For $\int_0^1 e^u\,du$ with one uniform draw in each of $n = 1000$ equal strata, the variance falls by a factor of nearly 10^6 (stratification with one draw per cell is close to the midpoint rule). For the poverty headcount of a *simulated* population in three regions with headcounts 4%, 28% and 53%, proportional stratification has a design effect of 0.83 and Neyman allocation, which over-samples the more heterogeneous poorer regions, of 0.72.

Listing 10.11: Stratified sampling: an integral and a simulated survey ch10/ch10_stratified.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Stratified sampling: an integral, and a simulated three-region survey.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# (a) theta = int_0^1 exp(u) du with K equal-width strata, 1 draw per stratum
n, R = 1000, 2000
plain = np.exp(rng.random((R, n))).mean(axis=1)
k = np.arange(n)
strat = np.exp((k + rng.random((R, n))) / n).mean(axis=1)
print(f"integral of exp on [0,1], n = {n}, {R} replications")
print(f"   plain      sd {plain.std():.2e}")
print(f"   stratified sd {strat.std():.2e}   (variance ratio "
      f"{plain.var() / strat.var():.0f})")

# (b) A simulated population of 60,000 households in three regions
N_h = np.array([30_000, 20_000, 10_000])              # stratum sizes
mu_h, sd_h = np.array([7.4, 6.9, 6.5]), np.array([0.5, 0.6, 0.9])
pop = [np.exp(rng.normal(mu_h[h], sd_h[h], N_h[h])) for h in range(3)]
W = N_h / N_h.sum()
zline = 700.0                                          # poverty line
poor = [(p < zline).astype(float) for p in pop]
true = sum(W[h] * poor[h].mean() for h in range(3))
S_h = np.array([poor[h].std() for h in range(3)])
allpop = np.concatenate(poor)

n = 600
n_prop = np.round(n * W).astype(int)
n_ney = np.round(n * W * S_h / (W * S_h).sum()).astype(int)
est = {"SRS": [], "proportional": [], "Neyman": []}
for _ in range(R):
    est["SRS"].append(rng.choice(allpop, n, replace=False).mean())
    for lab, nh in (("proportional", n_prop), ("Neyman", n_ney)):
        e = sum(W[h] * rng.choice(poor[h], nh[h], replace=False).mean()
                for h in range(3))
```

```
        est[lab].append(e)
print(f"\npoverty headcount, true = {true:.4f}; n = {n}; {R} surveys")
print(f"   stratum headcounts {[round(p.mean(), 3) for p in poor]}")
print(f"   Neyman allocation  {n_ney.tolist()}  (proportional "
      f"{n_prop.tolist()})")
v0 = np.var(est["SRS"])
for lab, e in est.items():
    e = np.asarray(e)
    print(f"   {lab:13s} mean {e.mean():.4f}  sd {e.std():.4f}  "
          f"design effect {e.var() / v0:.3f}")
```

Output

```
integral of exp on [0,1], n = 1000, 2000 replications
   plain      sd 1.57e-02
   stratified sd 1.62e-05   (variance ratio 937317)

poverty headcount, true = 0.2026; n = 600; 2000 surveys
   stratum headcounts [0.043, 0.279, 0.528]
   Neyman allocation  [182, 269, 149]  (proportional [300, 200, 100])
   SRS           mean 0.2025  sd 0.0162  design effect 1.000
   proportional  mean 0.2025  sd 0.0148  design effect 0.833
   Neyman        mean 0.2028  sd 0.0137  design effect 0.719
```

10.4.5 Quasi-Monte Carlo

Random points clump and leave gaps (Fig. 10.6, left). *Quasi-Monte Carlo* (QMC) replaces them by deterministic *low-discrepancy* sequences that fill $[0,1]^d$ evenly. The star discrepancy of points $\mathbf{u}_1, \dots, \mathbf{u}_n$,

$$D_n^\star \;=\; \sup_{\boldsymbol{a} \in [0,1]^d} \left| \frac{\#\{i : \mathbf{u}_i \in [\mathbf{0}, \boldsymbol{a})\}}{n} - \prod_{j=1}^{d} a_j \right|,$$

measures the worst discrepancy between the fraction of points in an anchored box and the box's volume. The Koksma–Hlawka inequality bounds the integration error by a product of a property of the integrand and a property of the points:

$$\left| \frac{1}{n} \sum_{i=1}^{n} f(\mathbf{u}_i) - \int_{[0,1]^d} f(\mathbf{u}) \, \mathrm{d}\mathbf{u} \right| \;\le\; V_{\mathrm{HK}}(f)\, D_n^\star, \tag{10.16}$$

where V_{HK} is the variation of f in the sense of Hardy and Krause (Niederreiter, 1992). Random points have $D_n^\star$ of order $n^{-1/2}$ (up to logarithms), whereas Halton and Sobol sequences achieve $D_n^\star = O\big((\log n)^d / n\big)$, so for integrands of bounded variation the QMC error is nearly $O(n^{-1})$.

The *Halton* sequence uses, in coordinate j, the radical inverse in the j-th prime base: write i in base b and reflect the digits about the decimal point (in base 2, $1, 2, 3, 4 \mapsto 0.1, 0.01, 0.11, 0.001$, i.e. $\tfrac{1}{2}, \tfrac{1}{4}, \tfrac{3}{4}, \tfrac{1}{8}$). *Sobol* sequences use base 2 in every coordinate with carefully chosen "direction numbers" and have their best properties when n is a power of 2. Because the points are deterministic, the error bound (10.16) is impossible to compute in practice. *Randomised* QMC fixes this: a random scrambling or shift keeps each point uniformly distributed (so the estimator is unbiased) while preserving low discrepancy, and independent replicates give a standard error. In `scipy.stats.qmc`, `qmc.Sobol(d, scramble=True)` and `qmc.Halton(d, scramble=True)` provide this.

Listing 10.12 estimates a smooth five-dimensional integral and computes the root-mean-square error over 30 independent scramblings. Plain Monte Carlo converges at the expected rate $n^{-0.5}$; scrambled Halton at about n^{-1} and scrambled Sobol at about $n^{-1.3}$ (scrambled nets can

beat n^{-1} for smooth integrands). At $n = 16{,}384$ the Sobol RMSE is 650 times smaller than that of Monte Carlo, an efficiency gain of more than 10^5 (Fig. 10.7).

Listing 10.12: Quasi-Monte Carlo with Sobol and Halton sequences ch10/ch10_qmc.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Quasi-Monte Carlo (scrambled Sobol and Halton) versus plain Monte Carlo.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import qmc

rng = np.random.default_rng(803)
d, reps = 5, 30
# smooth test integrand on [0,1]^d with a known integral
f = lambda u: np.prod(1 + (u - 0.5) * np.arange(1, d + 1) / d, axis=1) ** 2
exact = np.prod(1 + (np.arange(1, d + 1) / d) ** 2 / 12)

ms = np.arange(6, 15)
rmse = {"MC": [], "Sobol": [], "Halton": []}
for mm in ms:
    n = 2**mm
    err = {k: [] for k in rmse}
    for r in range(reps):
        err["MC"].append(f(rng.random((n, d))).mean() - exact)
        sob = qmc.Sobol(d, scramble=True, seed=rng).random_base2(mm)
        err["Sobol"].append(f(sob).mean() - exact)
        hal = qmc.Halton(d, scramble=True, seed=rng).random(n)
        err["Halton"].append(f(hal).mean() - exact)
    for k in rmse:
        rmse[k].append(np.sqrt(np.mean(np.square(err[k]))))

print(f"d = {d}, RMSE over {reps} randomisations")
print("      n        MC       Sobol      Halton")
for i, mm in enumerate(ms):
    print(f"{2**mm:7d}  {rmse['MC'][i]:.2e}  {rmse['Sobol'][i]:.2e}"
          f"  {rmse['Halton'][i]:.2e}")
for k in rmse:
    sl = np.polyfit(ms * np.log(2), np.log(rmse[k]), 1)[0]
    print(f"fitted rate n^({sl:.2f}) for {k}")
pts = qmc.Sobol(2, scramble=False).random_base2(8)
dis_s, dis_u = qmc.discrepancy(pts), qmc.discrepancy(rng.random((256, 2)))
print(f"centred L2 discrepancy, 256 points in 2-d: Sobol {dis_s:.2e},"
      f" pseudo-random {dis_u:.2e}")

fig, axs = plt.subplots(1, 2, figsize=(7.5, 3.6))
axs[0].scatter(*rng.random((256, 2)).T, s=6, color="navy")
axs[0].set_title("256 pseudo-random points")
axs[1].scatter(*pts.T, s=6, color="darkorange")
axs[1].set_title("256 Sobol points")
for ax in axs:
    ax.set_aspect("equal")
    ax.set_xlabel("u1")
    ax.set_ylabel("u2")
plt.tight_layout()
plt.savefig("ch10_qmc_points.pdf", bbox_inches="tight")

fig, ax = plt.subplots(figsize=(6, 3.8))
for k, mk in zip(rmse, ["o", "s", "^"]):
    ax.loglog(2.0**ms, rmse[k], mk + "-", ms=4, label=k)
ax.loglog(2.0**ms, rmse["MC"][0] * (2.0 ** (ms - ms[0])) ** -0.5, "k:",
          label="slope -1/2")
```

```
ax.set_xlabel("number of points n")
ax.set_ylabel("RMSE")
ax.legend()
plt.savefig("ch10_qmc_rmse.pdf", bbox_inches="tight")
```

Output

```
d = 5, RMSE over 30 randomisations
      n        MC       Sobol      Halton
     64  1.53e-01  2.55e-02  4.94e-02
    128  9.06e-02  5.76e-03  2.43e-02
    256  5.13e-02  3.45e-03  1.23e-02
    512  5.12e-02  1.53e-03  8.07e-03
   1024  3.35e-02  6.90e-04  3.84e-03
   2048  2.43e-02  2.36e-04  1.63e-03
   4096  1.57e-02  8.50e-05  8.59e-04
   8192  1.04e-02  4.28e-05  4.13e-04
  16384  8.08e-03  1.24e-05  2.69e-04
fitted rate n^(-0.51) for MC
fitted rate n^(-1.31) for Sobol
fitted rate n^(-0.96) for Halton
centred L2 discrepancy, 256 points in 2-d: Sobol 1.66e-05, pseudo-random 3.01e-03
```

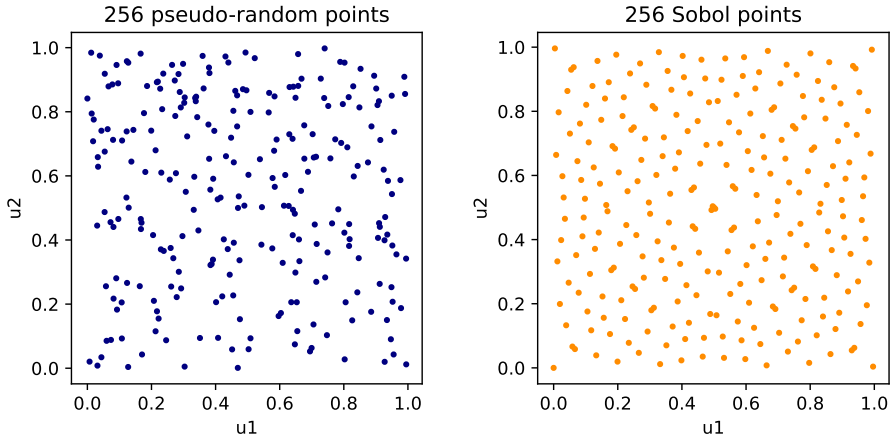


Figure 10.6: 256 pseudo-random points (left) and the first 256 points of the two-dimensional Sobol sequence (right). Low-discrepancy points fill the square without clusters or holes.

Historical Note: Importance sampling and low-discrepancy sequences

Importance sampling was one of the first variance-reduction ideas of the Los Alamos era; Herman Kahn and Theodore Harris described it in 1951 for estimating particle transmission through shields, in the same National Bureau of Standards volume in which John von Neumann described the acceptance–rejection method (Kahn and Harris, 1951; von Neumann, 1951). Low-discrepancy sequences came from number theory: John Halton proposed his sequence in 1960 (Halton, 1960) and Ilya Sobol' his in 1967 (Sobol', 1967). QMC found a major application in the 1990s when Sobol points were shown to price high-dimensional financial derivatives much faster than random sampling; Halton draws were popularised in econometrics for mixed-logit estimation (Train, 2009).

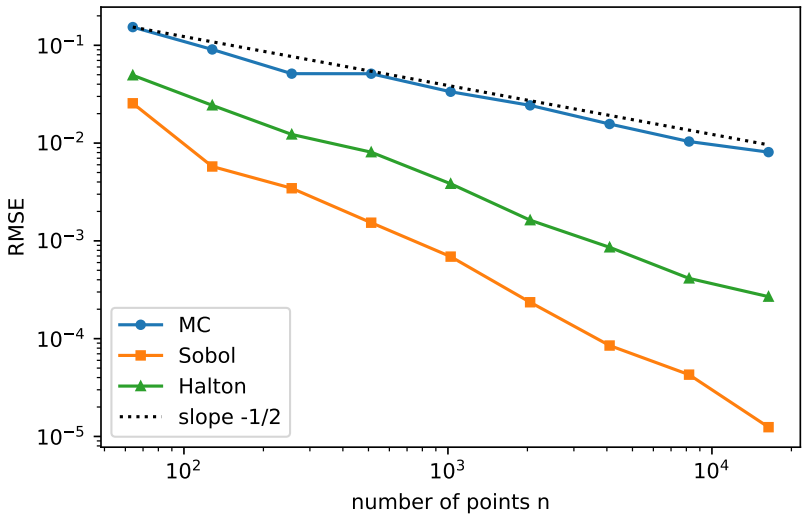


Figure 10.7: Root-mean-square error against n for a smooth integrand in $d = 5$: Monte Carlo follows the $n^{-1/2}$ line; scrambled Halton and Sobol converge much faster.

⋆ Key Concept: Variance reduction

- Antithetic variates: average h over negatively correlated pairs; variance $(1+\rho)\sigma^2/n$; guaranteed gain for monotone h.
- Control variates: add $c(g - \mu_g)$ with known μ_g; optimal $c^\star = -\mathrm{Cov}(h,g)/\mathrm{Var}(g)$ (a regression slope); variance reduced by $1 - \rho_{hg}^2$.
- Importance sampling: draw from q, weight by p/q; optimal $q^\star \propto |h|\,p$; q must have heavier tails than $|h|\,p$; monitor the effective sample size.
- Stratification: remove between-stratum variance; Neyman allocation $n_h \propto W_h\sigma_h$.
- Randomised QMC: low-discrepancy points with error close to $O(n^{-1})$ for smooth integrands, with standard errors from independent scramblings.

Graduate Perspective: Effective dimension and why QMC works in finance

The Koksma–Hlawka bound grows like $(\log n)^d$ and looks useless for $d = 360$, yet QMC works well for 360-dimensional mortgage-backed security valuations. The explanation is *effective dimension*: the integrand's ANOVA decomposition is dominated by a few variables or by low-order interactions, so QMC's uniformity in low-dimensional projections is what matters. Ordering the inputs so that the first coordinates carry most variance (e.g. a Brownian bridge or principal-components construction of a random path) increases QMC's advantage. See Glasserman (2003) for financial applications.

Exercises

10.4.1 Theory Prove that the control-variate estimator with estimated $\hat{c}$ is the intercept $\hat{\alpha}$ of the OLS regression $h(X_i) = \alpha + \beta\big(g(X_i) - \mu_g\big) + e_i$, and that $\hat{c} = -\hat{\beta}$.

10.4.2 Theory Show that the variance of $h(X)w(X)$ under $q = \mathcal{N}(\mu, 1)$ for $h = \mathbf{1}\{x > a\}$ and $p = \mathcal{N}(0,1)$ is $e^{\mu^2}\Phi(-a-\mu) - \Phi(-a)^2$, and find the μ that minimises it for $a = 4$. Compare with the choice $\mu = a$.

10.4.3 Theory Show that a proposal $q = \mathcal{N}(\mu, s^2)$ for the target $p = \mathcal{N}(0,1)$ gives importance weights with finite variance if and only if $s^2 > 1/2$.

10.4.4 Python Combine antithetic variates *and* a control variate for the index-insurance payout in Listing 10.9. Is the gain multiplicative?

10.4.5 Python Estimate $\mathbb{E}[\max(\bar{S} - K, 0)]$, where $\bar{S}$ is the average of a lognormal price path over 64 dates (an "Asian" payout, relevant to price-stabilisation funds), using plain Monte Carlo, Sobol points with the standard construction, and Sobol points with a Brownian-bridge construction. Report RMSE over 20 scramblings.

10.4.6 Policy A statistics office plans a poverty survey of $n = 6000$ households in four regions. Given stratum sizes and pilot standard deviations of the poverty indicator, compute the proportional and Neyman allocations and their design effects. Why might the office still choose a compromise allocation (hint: regional estimates must also be reported)?

10.4.7 ★ Graduate Prove that stratified sampling with n equal strata of $[0,1]$ and one uniform draw per stratum has variance $O(n^{-3})$ for $f \in C^1[0,1]$, so its RMSE is $O(n^{-3/2})$.

10.4.8 ★ Graduate Read Owen (1997) and explain why scrambled (t, m, s)-nets achieve an RMSE of $O(n^{-3/2}(\log n)^{(s-1)/2})$ for smooth integrands. Is the fitted Sobol rate in Listing 10.12 consistent with this?

10.5 Sampling from Distributions and Markov Chain Monte Carlo

Monte Carlo integration needs draws from p. NumPy supplies uniform and standard distributions, but for a Pareto income tail, a truncated wage distribution or, above all, a Bayesian posterior known only up to its normalising constant, we must build the sampler ourselves. This section moves from exact independent samplers to Markov chain methods, which produce dependent draws from almost any distribution.

10.5.1 The inverse-CDF method

Theorem 10.14 (Inverse transform)**.** *Let F be a distribution function and $F^{-}(u) = \inf\{x : F(x) \geq u\}$ its generalised inverse. If $U \sim \mathrm{U}(0,1)$ then $X = F^{-}(U)$ has distribution function F.*

Proof. Because F is non-decreasing and right-continuous, $F^{-}(u) \leq x \iff u \leq F(x)$. Hence $\mathbb{P}(X \leq x) = \mathbb{P}(U \leq F(x)) = F(x)$. □

For the Pareto distribution of top incomes, $F(x) = 1 - (x_m/x)^{\alpha}$ for $x \geq x_m$, so $X = x_m(1-U)^{-1/\alpha}$. For a discrete distribution on $1, \dots, K$ with cumulative probabilities c_k, $X = \min\{k : U \leq c_k\}$, which `np.searchsorted` computes for a whole vector of uniforms. The inverse-CDF method is exact, needs one uniform per draw, preserves monotone dependence (useful for antithetic and QMC sampling) and is the basis of copula simulation (Chapter 5). Its limitation is that F^{-} must be computable.

10.5.2 Acceptance–rejection

Suppose we can sample from a proposal density g and there is a constant M with $f(x) \leq M g(x)$ for all x. Von Neumann's acceptance–rejection algorithm repeats: draw $Y \sim g$ and $U \sim \mathrm{U}(0,1)$ independently; if $U \leq f(Y)/(Mg(Y))$, accept $X = Y$.

Theorem 10.15 (Acceptance–rejection is exact)**.** *The accepted value has density f, and the number of proposals per accepted draw is geometric with mean M (acceptance probability $1/M$). The result also holds if f is known only up to a constant, $f = \tilde{f}/Z$, provided $\tilde{f} \leq Mg$; the acceptance rate is then*

Z/M.

Proof. For one proposal, $\mathbb{P}(\text{accept}) = \int g(y)\,\mathbb{P}\big(U \le \frac{f(y)}{Mg(y)}\big)\,dy = \int g(y)\frac{f(y)}{Mg(y)}\,dy = \frac{1}{M}$, and for any set A,

$$\mathbb{P}(Y \in A,\ \text{accept}) \;=\; \int_A g(y)\frac{f(y)}{Mg(y)}\,dy \;=\; \frac{1}{M}\int_A f(y)\,dy.$$

Hence $\mathbb{P}(Y \in A \mid \text{accept}) = \int_A f$. Successive proposals are independent, so the number of trials until acceptance is geometric with success probability $1/M$. With $\tilde{f}$ in place of f, the same computation gives acceptance probability Z/M and conditional density $\tilde{f}/Z = f$. □

Geometrically, $(Y, UMg(Y))$ is uniform under the curve Mg, and accepting the points that fall under f leaves a point uniform under f, whose first coordinate has density f. Listing 10.13 samples Pareto incomes and household sizes by inversion and a Beta(2.7, 6.3) distribution by acceptance–rejection from a uniform envelope with $M = \max f = 2.67$; the observed acceptance rate 0.372 matches $1/M = 0.375$ (Fig. 10.8). The last lines show the weakness of naive rejection: to sample wages above a minimum wage three standard deviations above the mean, only 0.14% of normal draws are kept, and a better envelope (an exponential proposal on $[c, \infty)$) is needed.

Listing 10.13: Inverse-CDF and acceptance–rejection sampling ch10/ch10_sampling.py

```python
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Inverse-CDF sampling and acceptance-rejection sampling.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)
n = 100_000
u = rng.random(n)

# Inverse CDF: Pareto top incomes, F(x) = 1 - (xm/x)^alpha  ->  x = xm u^(-1/a)
xm, alpha = 5000.0, 2.5
x = xm * (1 - u) ** (-1 / alpha)
ks = stats.kstest(x, stats.pareto(alpha, scale=xm).cdf)
print(f"Pareto by inverse CDF: mean {x.mean():.1f} (theory "
      f"{alpha * xm / (alpha - 1):.1f}), KS p-value {ks.pvalue:.3f}")

# Inverse CDF for a discrete distribution: household size 1..6
probs = np.array([0.10, 0.18, 0.22, 0.20, 0.17, 0.13])
size = 1 + np.searchsorted(np.cumsum(probs), u)
print("household size freq :", np.round(np.bincount(size)[1:] / n, 3))
print("target probabilities:", probs)

# Acceptance-rejection: Beta(2.7, 6.3) with a U(0,1) envelope, M = max density
a, b = 2.7, 6.3
target = stats.beta(a, b)
mode = (a - 1) / (a + b - 2)
M = target.pdf(mode)
prop = rng.random(n)
acc = rng.random(n) <= target.pdf(prop) / M
draws = prop[acc]
print(f"\nAR for Beta({a},{b}): M = {M:.4f}, acceptance rate "
      f"{acc.mean():.4f} (theory 1/M = {1 / M:.4f})")
print(f"   sample mean {draws.mean():.4f} (theory {a / (a + b):.4f}), "
      f"KS p-value {stats.kstest(draws, target.cdf).pvalue:.3f}")

# AR for a truncated normal: wages above a minimum wage c (standardised)
for c in (0.0, 1.0, 2.0, 3.0):
```

```
    z = rng.standard_normal(n)
    print(f"   N(0,1) truncated at c = {c:.0f}: naive acceptance "
          f"{np.mean(z > c):.4f}")

fig, ax = plt.subplots(figsize=(6, 3.8))
grid = np.linspace(0, 1, 300)
ax.hist(draws, bins=60, density=True, color="steelblue", alpha=0.7,
        label=f"accepted draws ({acc.sum()})")
ax.plot(grid, target.pdf(grid), color="navy", lw=2, label="target Beta(2.7,6.3)")
ax.axhline(M, color="darkorange", ls="--", label="envelope M x U(0,1) density")
ax.set_xlabel("x")
ax.set_ylabel("density")
ax.legend()
plt.savefig("ch10_accept_reject.pdf", bbox_inches="tight")
```

Output

```
Pareto by inverse CDF: mean 8306.4 (theory 8333.3), KS p-value 0.181
household size freq : [0.099 0.181 0.223 0.198 0.169 0.13 ]
target probabilities: [0.1  0.18 0.22 0.2  0.17 0.13]

AR for Beta(2.7,6.3): M = 2.6697, acceptance rate 0.3720 (theory 1/M = 0.3746)
   sample mean 0.2985 (theory 0.3000), KS p-value 0.072
   N(0,1) truncated at c = 0: naive acceptance 0.4981
   N(0,1) truncated at c = 1: naive acceptance 0.1570
   N(0,1) truncated at c = 2: naive acceptance 0.0228
   N(0,1) truncated at c = 3: naive acceptance 0.0014
```

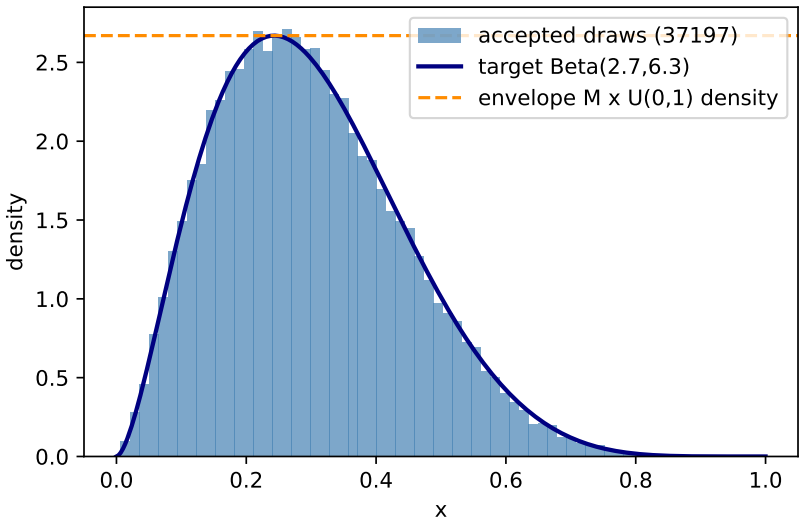


Figure 10.8: Acceptance–rejection for Beta(2.7, 6.3) with a uniform envelope $M \cdot \mathrm{U}(0,1)$: the histogram of accepted draws matches the target density. The area between the envelope and the density is the wasted fraction $1 - 1/M$.

10.5.3 Markov chains with a prescribed stationary distribution

In dimension $d = 10$ or 1000, good envelopes are unavailable: M grows exponentially with d. MCMC abandons independence. It constructs a Markov chain $\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \ldots$ with transition

kernel $K(\boldsymbol{\theta}, \boldsymbol{\theta}')$ whose *stationary distribution* is the target π:

$$\int \pi(\boldsymbol{\theta}) K(\boldsymbol{\theta}, \boldsymbol{\theta}') \, \mathrm{d}\boldsymbol{\theta} \;=\; \pi(\boldsymbol{\theta}').$$

If the chain is also irreducible and aperiodic (more precisely, Harris recurrent), then the ergodic theorem gives, for any π-integrable h and any starting point,

$$\frac{1}{T} \sum_{t=1}^{T} h\big(\boldsymbol{\theta}^{(t)}\big) \;\xrightarrow{\text{a.s.}}\; \mathbb{E}_{\pi}[h(\boldsymbol{\theta})], \tag{10.17}$$

so time averages along the chain replace averages over independent draws (Robert and Casella, 2004). Markov chains on finite state spaces and their stationary distributions were met in Chapter 3, as the leading left eigenvector of a stochastic matrix.

A simple sufficient condition for stationarity is *detailed balance* (reversibility):

$$\pi(\boldsymbol{\theta}) K(\boldsymbol{\theta}, \boldsymbol{\theta}') \;=\; \pi(\boldsymbol{\theta}') K(\boldsymbol{\theta}', \boldsymbol{\theta}) \qquad \text{for all } \boldsymbol{\theta}, \boldsymbol{\theta}'. \tag{10.18}$$

Integrating both sides over $\boldsymbol{\theta}$ gives $\int \pi(\boldsymbol{\theta}) K(\boldsymbol{\theta}, \boldsymbol{\theta}') \, \mathrm{d}\boldsymbol{\theta} = \pi(\boldsymbol{\theta}') \int K(\boldsymbol{\theta}', \boldsymbol{\theta}) \, \mathrm{d}\boldsymbol{\theta} = \pi(\boldsymbol{\theta}')$. In words: in equilibrium the flow of probability from $\boldsymbol{\theta}$ to $\boldsymbol{\theta}'$ equals the flow back.

10.5.4 The Metropolis–Hastings algorithm

Let the target be known up to a constant, $\pi(\boldsymbol{\theta}) = \tilde{\pi}(\boldsymbol{\theta}) / Z$; for a posterior, $\tilde{\pi}(\boldsymbol{\theta}) = p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})$ and $Z = p(\mathbf{y})$ is the intractable marginal likelihood (10.4). Choose a proposal density $q(\boldsymbol{\theta}' \mid \boldsymbol{\theta})$.

Algorithm 5 Metropolis–Hastings

1: choose $\boldsymbol{\theta}^{(0)}$
2: **for** $t = 0, 1, \dots, T-1$ **do**
3: draw a proposal $\boldsymbol{\theta}' \sim q(\cdot \mid \boldsymbol{\theta}^{(t)})$ and $U \sim \mathrm{U}(0,1)$
4: $\alpha \leftarrow \min\left\{1, \; \dfrac{\tilde{\pi}(\boldsymbol{\theta}') \, q(\boldsymbol{\theta}^{(t)} \mid \boldsymbol{\theta}')}{\tilde{\pi}(\boldsymbol{\theta}^{(t)}) \, q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}^{(t)})}\right\}$
5: **if** $U \le \alpha$ **then** $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}'$ **else** $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)}$
6: **end for**

The constant Z cancels in the ratio: only *ratios* of the unnormalised posterior are needed. In code one compares logarithms, $\log U < \log \tilde{\pi}(\boldsymbol{\theta}') - \log \tilde{\pi}(\boldsymbol{\theta}^{(t)}) + \dots$, to avoid underflow.

Theorem 10.16 (Metropolis–Hastings satisfies detailed balance)**.** *The Metropolis–Hastings kernel satisfies* (10.18) *with respect to π, so π is a stationary distribution of the chain.*

Proof. For $\boldsymbol{\theta}' \neq \boldsymbol{\theta}$ the kernel has density $K(\boldsymbol{\theta}, \boldsymbol{\theta}') = q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) \alpha(\boldsymbol{\theta}, \boldsymbol{\theta}')$; the remaining mass is a point mass at $\boldsymbol{\theta}$ (rejection), for which (10.18) is trivial. Then

$$\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) \alpha(\boldsymbol{\theta}, \boldsymbol{\theta}') \;=\; \min\big\{\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}), \; \pi(\boldsymbol{\theta}') q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')\big\},$$

because multiplying $\min\{1, r\}$ by $\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' \mid \boldsymbol{\theta})$, with $r = \pi(\boldsymbol{\theta}') q(\boldsymbol{\theta} \mid \boldsymbol{\theta}') / \big(\pi(\boldsymbol{\theta}) q(\boldsymbol{\theta}' \mid \boldsymbol{\theta})\big)$, gives the minimum of the two products. This expression is symmetric in $(\boldsymbol{\theta}, \boldsymbol{\theta}')$, so it also equals $\pi(\boldsymbol{\theta}') q(\boldsymbol{\theta} \mid \boldsymbol{\theta}') \alpha(\boldsymbol{\theta}', \boldsymbol{\theta})$, which is detailed balance. □

Stationarity is not convergence: we also need the chain to reach every region of positive probability (irreducibility), which holds for example if $q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) > 0$ everywhere. Two important special cases are the *random-walk Metropolis* (RWM) algorithm, $\boldsymbol{\theta}' = \boldsymbol{\theta} + \varepsilon$ with symmetric ε, for which the q terms cancel and $\alpha = \min\{1, \tilde{\pi}(\boldsymbol{\theta}') / \tilde{\pi}(\boldsymbol{\theta})\}$ (Metropolis et al., 1953); and the *independence sampler*, $q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) = q(\boldsymbol{\theta}')$, which resembles importance sampling.

The step size of RWM is a trade-off: small steps are almost always accepted but move slowly; large steps are mostly rejected. For targets close to a d-dimensional Gaussian, Roberts et al. (1997) showed that the optimal proposal is $\mathcal{N}(\boldsymbol{\theta}, (2.38^2/d)\boldsymbol{\Sigma})$, with $\boldsymbol{\Sigma}$ the target covariance, and that the resulting acceptance rate is about 0.234 for large d (about 0.44 for $d = 1$).

10.5.5 Random-walk Metropolis for a logistic regression posterior

Listing 10.14 analyses a *simulated* programme-enrolment dataset with $n = 300$ households: enrolment y_i depends on standardised log income and a randomly assigned information treatment through a logistic regression, $\mathbb{P}(y_i = 1) = \left(1 + e^{-\mathbf{x}_i^\top \boldsymbol{\beta}}\right)^{-1}$, with independent $\mathcal{N}(0, 10^2)$ priors. The log posterior is

$$\log \tilde{\pi}(\boldsymbol{\beta}) \;=\; \sum_{i=1}^{n} \left[y_i \mathbf{x}_i^\top \boldsymbol{\beta} - \log(1 + e^{\mathbf{x}_i^\top \boldsymbol{\beta}}) \right] - \frac{\|\boldsymbol{\beta}\|^2}{2 \cdot 10^2}.$$

The program finds the posterior mode and the inverse Hessian with BFGS (Chapter 8), uses them to scale a random-walk proposal by the Roberts–Gelman–Gilks rule, and runs four chains of 6000 iterations from over-dispersed starting points, discarding the first 1000 as burn-in.

Listing 10.14: Random-walk Metropolis for a Bayesian logistic regression ch10/ch10_mh_logit.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Random-walk Metropolis-Hastings for a Bayesian logistic regression.
# (c) 2026 Stephen E. Moore. MIT Licence.
import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")  # tiny matrices: one thread
import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize

rng = np.random.default_rng(803)
# Simulated programme data: enrolment y on intercept, log income, treatment
n = 300
loginc = rng.normal(0, 1, n)
treat = rng.integers(0, 2, n)
X = np.column_stack([np.ones(n), loginc, treat])
beta_true = np.array([-0.5, 0.8, 0.7])
y = (rng.random(n) < 1 / (1 + np.exp(-X @ beta_true))).astype(float)


def log_post(b, tau=10.0):              # N(0, tau^2) prior on each coefficient
    eta = X @ b
    return y @ eta - np.logaddexp(0, eta).sum() - 0.5 * b @ b / tau**2


def ess(x):                             # Geyer initial-positive-sequence ESS
    x = x - x.mean()
    m = len(x)
    f = np.fft.rfft(x, 2 * m)
    acf = np.fft.irfft(f * np.conj(f))[:m] / (x @ x)
    s = 0.0
    for k in range(0, m - 1, 2):        # Gamma_k = rho_2k + rho_2k+1 > 0
        pair = acf[k] + acf[k + 1]
        if pair < 0:
            break
        s += pair
    return m / (2 * s - 1)              # tau = -1 + 2 sum Gamma_k


def rhat(ch):                           # split-R-hat, ch has shape (chains, it)
```

```
    h = ch.shape[1] // 2
    s = np.vstack([ch[:, :h], ch[:, h:2 * h]])
    w = s.var(axis=1, ddof=1).mean()
    bvar = h * s.mean(axis=1).var(ddof=1)
    return np.sqrt(((h - 1) / h * w + bvar / h) / w)


# Laplace approximation gives the mode and a proposal covariance
opt = optimize.minimize(lambda b: -log_post(b), np.zeros(3), method="BFGS")
L = np.linalg.cholesky(opt.hess_inv)
scale = 2.38 / np.sqrt(3)
n_iter, burn, chains = 6000, 1000, 4
draws = np.empty((chains, n_iter, 3))
acc = np.zeros(chains)
for c in range(chains):
    b = opt.x + 2 * rng.standard_normal(3)          # over-dispersed start
    lp = log_post(b)
    for t in range(n_iter):
        prop = b + scale * L @ rng.standard_normal(3)
        lp_prop = log_post(prop)
        if np.log(rng.random()) < lp_prop - lp:      # symmetric proposal
            b, lp = prop, lp_prop
            acc[c] += t >= burn
        draws[c, t] = b
post = draws[:, burn:]
print(f"MLE/mode: {np.round(opt.x, 3)}; acceptance rate "
      f"{acc.mean() / (n_iter - burn):.3f}")
print("param     true   post.mean  post.sd   95% cred. int.     ESS   R-hat")
for j, name in enumerate(["const", "loginc", "treat"]):
    flat = post[..., j].ravel()
    lo, hi = np.percentile(flat, [2.5, 97.5])
    e = sum(ess(post[c, :, j]) for c in range(chains))
    print(f"{name:7s} {beta_true[j]:6.2f}  {flat.mean():9.3f} {flat.std():8.3f}"
          f"  [{lo:6.3f}, {hi:6.3f}]  {e:6.0f}  {rhat(post[..., j]):.3f}")
# Policy quantities: P(effect > 0) and average change in enrolment probability
bt = post.reshape(-1, 3)
p1 = 1 / (1 + np.exp(-(bt[:, :2] @ X[:, :2].T + bt[:, [2]])))
p0 = 1 / (1 + np.exp(-(bt[:, :2] @ X[:, :2].T)))
ame = (p1 - p0).mean(axis=1)
print(f"P(treatment coefficient > 0 | data) = {np.mean(bt[:, 2] > 0):.4f}")
print(f"average marginal effect of treatment: {ame.mean():.3f} "
      f"[{np.percentile(ame, 2.5):.3f}, {np.percentile(ame, 97.5):.3f}]")

fig, axs = plt.subplots(1, 2, figsize=(7.5, 3.4),
                        gridspec_kw={"width_ratios": [2, 1]})
for c in range(chains):
    axs[0].plot(draws[c, :, 2], lw=0.4)
axs[0].axvline(burn, color="black", ls="--", lw=1)
axs[0].set_xlabel("iteration")
axs[0].set_ylabel("treat coefficient")
axs[0].set_title("trace, 4 chains (dashed: burn-in)", fontsize=10)
xc = post[0, :, 2] - post[0, :, 2].mean()
lags = np.arange(31)
acf = [1.0] + [np.corrcoef(xc[:-k], xc[k:])[0, 1] for k in lags[1:]]
axs[1].bar(lags, acf, color="steelblue")
axs[1].set_xlabel("lag")
axs[1].set_ylabel("autocorrelation")
axs[1].set_title("ACF, chain 1", fontsize=10)
plt.tight_layout()
plt.savefig("ch10_mh_trace.pdf", bbox_inches="tight")
```

Output

```
MLE/mode: [-0.551  0.661  0.625]; acceptance rate 0.314
param     true   post.mean  post.sd   95% cred. int.      ESS   R-hat
const    -0.50     -0.549    0.177   [-0.900, -0.202]    2029  1.000
loginc    0.80      0.669    0.140   [ 0.389,  0.939]    1710  1.001
treat     0.70      0.619    0.249   [ 0.140,  1.109]    1980  1.001
P(treatment coefficient > 0 | data) = 0.9932
average marginal effect of treatment: 0.137 [0.031, 0.242]
```

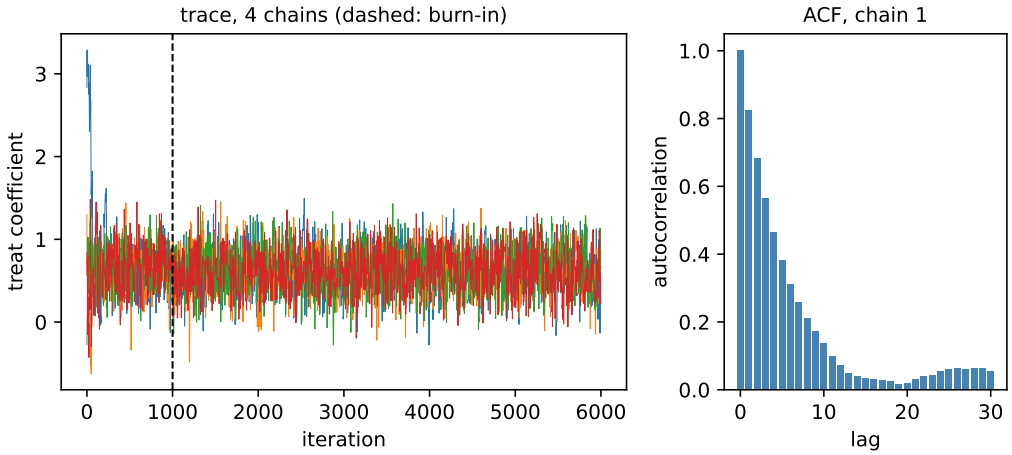


Figure 10.9: Left: trace plots of the treatment coefficient for four random-walk Metropolis chains started from dispersed points; after a short transient (dashed line: end of burn-in) they overlap. Right: autocorrelation function of one chain after burn-in.

The acceptance rate is 0.31, close to the theoretical optimum for $d = 3$. The posterior means are close to the mode, as expected with $n = 300$ and a vague prior, and the true coefficients lie within the 95% credible intervals. The four chains together yield about 2000 effective draws per parameter out of 20,000 retained, an efficiency of around 10% typical for RWM.

Policy Lens: Bayesian policy evaluation

Bayesian output answers the questions decision makers ask directly. From the same posterior draws, the program computes the probability that the information treatment raises enrolment, $\mathbb{P}(\beta_{\text{treat}} > 0 \mid \text{data}) \approx 0.99$, and the posterior distribution of the *average marginal effect* on the enrolment probability, 0.14 with 95% credible interval $[0.03, 0.24]$. Any function of the parameters, such as the cost per additional enrolee or the probability that the effect exceeds a cost-effectiveness threshold, is obtained by evaluating it at each draw: Monte Carlo integration over the posterior. Matsuura (2022) works through many such models in Stan; the policy literature uses Bayesian hierarchical models to pool evidence from multiple trials of the same intervention.

10.5.6 Gibbs sampling

When $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_d)$ and every *full conditional* $\pi(\theta_j \mid \boldsymbol{\theta}_{-j})$ is easy to sample, the *Gibbs sampler* updates the coordinates in turn, each from its full conditional given the current values of the others. A Gibbs update is a Metropolis–Hastings step with proposal $q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) = \pi(\theta_j' \mid \boldsymbol{\theta}_{-j})$ (with

$\boldsymbol{\theta}'_{-j} = \boldsymbol{\theta}_{-j}$), and the MH ratio is then identically one:

$$\frac{\pi(\boldsymbol{\theta}')\,\pi(\theta_j \mid \boldsymbol{\theta}_{-j})}{\pi(\boldsymbol{\theta})\,\pi(\theta'_j \mid \boldsymbol{\theta}_{-j})} = \frac{\pi(\theta'_j \mid \boldsymbol{\theta}_{-j})\pi(\boldsymbol{\theta}_{-j})\,\pi(\theta_j \mid \boldsymbol{\theta}_{-j})}{\pi(\theta_j \mid \boldsymbol{\theta}_{-j})\pi(\boldsymbol{\theta}_{-j})\,\pi(\theta'_j \mid \boldsymbol{\theta}_{-j})} = 1.$$

So every proposal is accepted, and each coordinate update leaves π invariant. Conjugate hierarchical models, probit models with latent utilities (data augmentation) and mixture models all have convenient full conditionals.

Example 10.17 (Gibbs sampling for a bivariate normal)**.** For (X_1, X_2) standard bivariate normal with correlation ρ, the full conditionals are $X_1 \mid X_2 = x_2 \sim \mathcal{N}(\rho x_2, 1-\rho^2)$ and $X_2 \mid X_1 = x_1 \sim \mathcal{N}(\rho x_1, 1-\rho^2)$. Substituting one update into the other, the X_1-chain satisfies $X_1^{(t+1)} = \rho^2 X_1^{(t)} + \eta_t$ with $\eta_t \sim \mathcal{N}(0, 1-\rho^4)$ independent: an AR(1) process with coefficient ρ^2. Its lag-k autocorrelation is ρ^{2k}, and the effective sample size of T draws is $T(1-\rho^2)/(1+\rho^2)$. With $\rho = 0.99$ that is under 1% of T: Gibbs moves in axis-parallel steps and crawls along a narrow ridge.

Listing 10.15 confirms these formulas: the lag-1 autocorrelations $(0.24, 0.81, 0.98)$ match ρ^2 and the estimated ESS (12,207, 2068, 184 out of 19,500) matches the AR(1) theory. Figure 10.10 shows the staircase path and the slow mixing for $\rho = 0.99$.

Listing 10.15: Gibbs sampling for a bivariate normal ch10/ch10_gibbs.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Gibbs sampling for a bivariate normal; autocorrelation and effective size.
# (c) 2026 Stephen E. Moore. MIT Licence.
import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")  # tiny matrices: one thread
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)


def gibbs(rho, n_iter, start=(-4.0, 4.0)):
    """Alternate x1 | x2 ~ N(rho x2, 1-rho^2) and x2 | x1 ~ N(rho x1, 1-rho^2)."""
    x = np.empty((n_iter, 2))
    x1, x2 = start
    sd = np.sqrt(1 - rho**2)
    z = rng.standard_normal((n_iter, 2))
    for t in range(n_iter):
        x1 = rho * x2 + sd * z[t, 0]
        x2 = rho * x1 + sd * z[t, 1]
        x[t] = x1, x2
    return x


def ess(x):                             # Geyer initial-positive-sequence ESS
    x = x - x.mean()
    m = len(x)
    f = np.fft.rfft(x, 2 * m)
    acf = np.fft.irfft(f * np.conj(f))[:m] / (x @ x)
    s = 0.0
    for k in range(0, m - 1, 2):
        if acf[k] + acf[k + 1] < 0:
            break
        s += acf[k] + acf[k + 1]
    return m / (2 * s - 1)

```

```

n_iter, burn = 20_000, 500
print(" rho   mean(x1)  var(x1)  corr   lag-1 acf  theory rho^2"
      "   ESS    theory ESS")
chains = {}
for rho in (0.5, 0.9, 0.99):
    x = gibbs(rho, n_iter)
    chains[rho] = x
    x1 = x[burn:, 0]
    a1 = np.corrcoef(x1[:-1], x1[1:])[0, 1]
    r2 = rho**2
    th = len(x1) * (1 - r2) / (1 + r2)        # AR(1) with coefficient rho^2
    print(f"{rho:5.2f}  {x1.mean():8.3f}  {x1.var():7.3f}  "
          f"{np.corrcoef(x[burn:].T)[0, 1]:5.3f}  {a1:9.3f}  {r2:12.3f}"
          f"  {ess(x1):6.0f}  {th:9.0f}")

fig, axs = plt.subplots(1, 2, figsize=(7.5, 3.6))
x = chains[0.9]
axs[0].plot(*x[:40].T, "-o", ms=2.5, lw=0.7, color="navy")
axs[0].scatter(*x[burn:burn + 2000].T, s=2, color="gray", alpha=0.4)
axs[0].set_title("first 40 Gibbs steps, rho = 0.9", fontsize=10)
axs[0].set_xlabel("x1")
axs[0].set_ylabel("x2")
for rho, col in zip((0.5, 0.99), ("darkorange", "firebrick")):
    axs[1].plot(chains[rho][:600, 0], lw=0.7, color=col, label=f"rho = {rho}")
axs[1].set_xlabel("iteration")
axs[1].set_ylabel("x1")
axs[1].set_title("trace plots", fontsize=10)
axs[1].legend(fontsize=8)
plt.tight_layout()
plt.savefig("ch10_gibbs.pdf", bbox_inches="tight")
```

Output

```
rho   mean(x1)  var(x1)  corr   lag-1 acf  theory rho^2   ESS    theory ESS
0.50     0.006    0.981  0.489      0.239         0.250   12207      11700
0.90     0.003    0.968  0.896      0.806         0.810    2068       2047
0.99     0.010    0.998  0.990      0.980         0.980     184        196
```

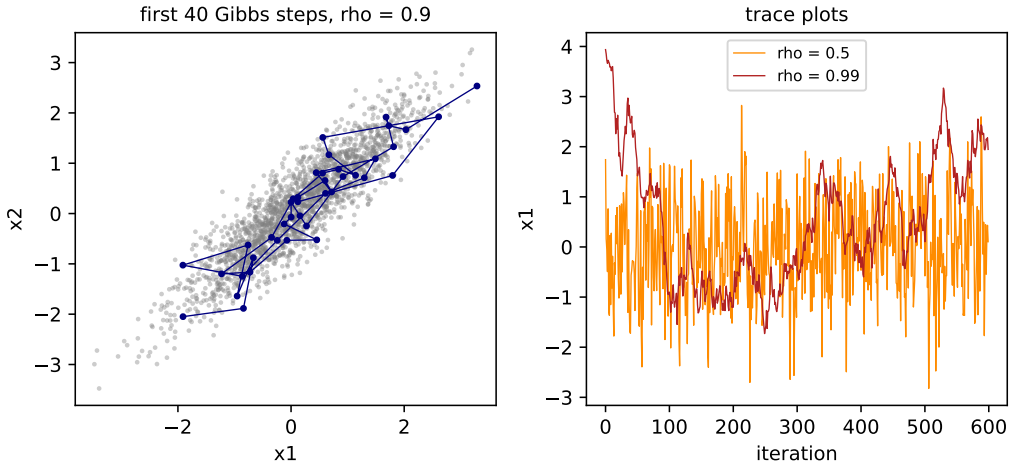


Figure 10.10: Left: the first 40 Gibbs steps from $(-4, 4)$ for $\rho = 0.9$, moving parallel to the axes, over 2000 later draws. Right: trace plots for $\rho = 0.5$ (fast mixing) and $\rho = 0.99$ (slow mixing).

10.5.7 Convergence diagnostics

The ergodic theorem is asymptotic and says nothing about how long a chain must run. Diagnostics cannot prove convergence, but they detect many failures.

Trace plots and burn-in. Plot each parameter against iteration for several chains started from over-dispersed points (Fig. 10.9). The initial transient, during which the chain travels from its starting point to the bulk of π, is discarded as *burn-in* (or warm-up). Chains that have not yet overlapped, or that drift, signal non-convergence.

Autocorrelation and effective sample size. For a stationary chain with lag-k autocorrelations ρ_k of $h(\boldsymbol{\theta}^{(t)})$, the Markov chain CLT gives

$$\sqrt{T}\left(\bar{h}_T - \mathbb{E}_\pi h\right) \xrightarrow{d} \mathcal{N}\left(0,\ \sigma^2\tau\right), \qquad \tau \;=\; 1 + 2\sum_{k=1}^{\infty} \rho_k, \tag{10.19}$$

when the sum converges. The *integrated autocorrelation time* τ inflates the variance relative to independent sampling, and the *effective sample size* is $\mathrm{ESS} = T/\tau$: T correlated draws carry as much information about $\mathbb{E}_\pi h$ as ESS independent ones. The MCMC standard error is $\hat{\sigma}/\sqrt{\mathrm{ESS}}$. Estimating τ requires truncating the sum; Geyer's initial positive sequence estimator (Geyer, 1992), used in our scripts, sums the pairs $\rho_{2k} + \rho_{2k+1}$ while they remain positive.

$\widehat{R}$. With m chains of length T, let W be the average within-chain variance and B/T the variance of the chain means. The potential scale reduction factor (Gelman and Rubin, 1992) is

$$\widehat{R} \;=\; \sqrt{\frac{\frac{T-1}{T}W + \frac{B}{T}}{W}}, \tag{10.20}$$

the ratio of an estimate of $\mathrm{Var}_\pi(h)$ that is inflated if the chains have not mixed to one that is deflated. It approaches 1 from above as the chains converge. Modern practice splits each chain in half (so that within-chain drift is detected), rank-normalises the draws, and requires $\widehat{R} < 1.01$ and a bulk and tail ESS of at least a few hundred (Vehtari et al., 2021). All our $\widehat{R}$ values are at most 1.001.

△ Caution: Diagnostics can only detect failure

A chain trapped in one mode of a multimodal posterior can look perfectly converged, with flat trace plots, small autocorrelations and $\widehat{R} \approx 1$ if all chains are trapped in the same mode. Use several over-dispersed starting points, compare with an approximation (Laplace, variational) or a simpler model, simulate fake data from known parameters and check that the sampler recovers them (simulation-based calibration), and never thin a chain in the belief that this improves precision: thinning discards information and is only a storage device.

10.5.8 Hamiltonian Monte Carlo and Stan

Random-walk proposals explore a d-dimensional posterior by diffusion, needing $O(d)$ steps per effective draw, and Gibbs sampling is crippled by strong correlation. *Hamiltonian Monte Carlo* (HMC) uses the gradient of $\log \pi$ to propose distant moves that are still likely to be accepted. It augments $\boldsymbol{\theta}$ with a momentum $\boldsymbol{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and simulates the Hamiltonian dynamics of the energy $H(\boldsymbol{\theta}, \boldsymbol{r}) = -\log \pi(\boldsymbol{\theta}) + \frac{1}{2}\|\boldsymbol{r}\|^2$ with the *leapfrog* integrator,

$$\boldsymbol{r} \leftarrow \boldsymbol{r} + \tfrac{\epsilon}{2}\nabla \log \pi(\boldsymbol{\theta}), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon\, \boldsymbol{r}, \qquad \boldsymbol{r} \leftarrow \boldsymbol{r} + \tfrac{\epsilon}{2}\nabla \log \pi(\boldsymbol{\theta}),$$

repeated L times, followed by a Metropolis accept/reject step on $\exp(-\Delta H)$. Exact dynamics conserve H, so the acceptance probability is close to one when ϵ is small; the leapfrog map is reversible and volume-preserving, which is what makes the MH correction valid (Neal, 2011). Listing 10.16 compares a 30-line HMC sampler with RWM on a correlated ten-dimensional Gaussian at equal cost in gradient/density evaluations: HMC delivers about four times as many effective draws per evaluation, and its variance estimate is accurate where RWM's is not yet.

Listing 10.16: A minimal Hamiltonian Monte Carlo sampler ch10/ch10_hmc.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# A minimal Hamiltonian Monte Carlo sampler versus random-walk Metropolis.
# (c) 2026 Stephen E. Moore. MIT Licence.
import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")  # tiny matrices: one thread
import numpy as np

rng = np.random.default_rng(803)
d, rho = 10, 0.9                          # correlated 10-dimensional Gaussian
S = rho ** np.abs(np.subtract.outer(np.arange(d), np.arange(d)))   # AR(1) cov
P = np.linalg.inv(S)
logp = lambda x: -0.5 * x @ P @ x
grad = lambda x: -P @ x


def ess(x):
    x = x - x.mean()
    m = len(x)
    f = np.fft.rfft(x, 2 * m)
    acf = np.fft.irfft(f * np.conj(f))[:m] / (x @ x)
    s = 0.0
    for k in range(0, m - 1, 2):
        if acf[k] + acf[k + 1] < 0:
            break
        s += acf[k] + acf[k + 1]
    return m / (2 * s - 1)


def hmc(n, eps=0.15, L=20):
    x, out, acc = np.zeros(d), np.empty((n, d)), 0
    for t in range(n):
        p = rng.standard_normal(d)
        xn, pn = x.copy(), p + 0.5 * eps * grad(x)      # leapfrog integrator
        for i in range(L):
            xn = xn + eps * pn
            if i < L - 1:
                pn = pn + eps * grad(xn)
        pn = pn + 0.5 * eps * grad(xn)
        dH = logp(xn) - 0.5 * pn @ pn - logp(x) + 0.5 * p @ p
        if np.log(rng.random()) < dH:
            x, acc = xn, acc + 1
        out[t] = x
    return out, acc / n


def rwm(n, scale):
    x, out, acc = np.zeros(d), np.empty((n, d)), 0
    lp = logp(x)
    for t in range(n):
        prop = x + scale * rng.standard_normal(d)
        lq = logp(prop)
        if np.log(rng.random()) < lq - lp:
```

```
            x, lp, acc = prop, lq, acc + 1
        out[t] = x
    return out, acc / n


n_h, L = 2000, 20
xh, ah = hmc(n_h, L=L)
xr, ar = rwm(n_h * L, 2.38 / np.sqrt(d) * 0.45)    # same gradient budget
for name, x, a in [("HMC", xh, ah), ("RW-Metropolis", xr, ar)]:
    e = np.array([ess(x[:, j]) for j in range(d)])
    print(f"{name:14s} iterations {len(x):6d}  acceptance {a:.3f}  "
          f"min/median ESS {e.min():6.0f} / {np.median(e):6.0f}")
    print(f"{'':14s} var(x1) {x[:, 0].var():.3f} (target 1),  "
          f"ESS per 1000 density evaluations {1000 * e.min() / (n_h * L):.1f}")
```

Output

```
HMC            iterations   2000  acceptance 0.960  min/median ESS    260 /    704
               var(x1) 0.959 (target 1),  ESS per 1000 density evaluations 6.5
RW-Metropolis  iterations  40000  acceptance 0.161  min/median ESS     59 /     65
               var(x1) 0.790 (target 1),  ESS per 1000 density evaluations 1.5
```

The step size ϵ, the number of steps L and a mass matrix (a preconditioner, playing the role of $\boldsymbol{\Sigma}$ in RWM) must be tuned. The No-U-Turn Sampler (Hoffman and Gelman, 2014) chooses L adaptively, and adapts ϵ and the mass matrix during warm-up. It is the default algorithm of Stan (Carpenter et al., 2017), in which the user writes only the log density in a modelling language and gradients are computed by automatic differentiation. Matsuura (2022) is a practical guide to Bayesian modelling in Stan from R and Python, including the diagnostics above and the divergent-transition warnings that are specific to HMC.

Historical Note: From MANIAC to Stan

Metropolis et al. (1953), by Nicholas Metropolis, Arianna and Marshall Rosenbluth, and Augusta and Edward Teller, introduced the random-walk algorithm to compute equilibrium properties of a system of interacting particles on the MANIAC computer at Los Alamos. W. Keith Hastings generalised it to asymmetric proposals and presented it to statisticians in 1970 (Hastings, 1970). Stuart and Donald Geman named and used the Gibbs sampler for Bayesian image restoration in 1984 (Geman and Geman, 1984), and Alan Gelfand and Adrian Smith's 1990 paper (Gelfand and Smith, 1990) showed statisticians how sampling could compute the marginal posteriors of ordinary Bayesian models, triggering the MCMC revolution of the 1990s. Hamiltonian (originally "hybrid") Monte Carlo was devised by Simon Duane, Anthony Kennedy, Brian Pendleton and Duncan Roweth in 1987 for lattice field theory (Duane et al., 1987); Radford Neal brought it to statistics, and Stan (first released in 2012) made it routine.

⋆ Key Concept: Sampling and MCMC

Inverse-CDF sampling sets $X = F^{-}(U)$; acceptance–rejection accepts $Y \sim g$ with probability $f(Y)/(Mg(Y))$ and needs M proposals per draw. MCMC builds a Markov chain with stationary distribution π: Metropolis–Hastings accepts with probability $\min\{1, \tilde{\pi}(\boldsymbol{\theta}')q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')/(\tilde{\pi}(\boldsymbol{\theta})q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}))\}$, which satisfies detailed balance and needs π only up to a constant; Gibbs sampling is MH with full-conditional proposals and acceptance one. Draws are dependent: report $\mathrm{ESS} = T/\tau$ and MCSE $\hat{\sigma}/\sqrt{\mathrm{ESS}}$, run several chains, discard warm-up, and check trace plots and $\widehat{R} < 1.01$. HMC uses gradients to move far at high acceptance and is the engine of Stan.

Graduate Perspective: Beyond the basics

Research-level MCMC includes adaptive MCMC (proposals tuned on the fly, with conditions that preserve ergodicity), pseudo-marginal methods that replace an intractable likelihood by an unbiased Monte Carlo estimate (used for state-space macro models with particle filters), sequential Monte Carlo, and stochastic-gradient MCMC for large data. On the theory side, geometric ergodicity guarantees a Markov chain CLT and is studied through drift and minorisation conditions; Robert and Casella (2004) is the standard reference.

Exercises

10.5.1 Theory Derive the inverse-CDF sampler for the logistic, Weibull and Gumbel distributions. Show that if E_1, E_2 are iid standard Gumbel then $E_1 - E_2$ is logistic, and relate this to the logit model of discrete choice.

10.5.2 Theory In acceptance–rejection for $f = \text{Beta}(a, b)$ with a uniform proposal, show that $M = f(\text{mode})$ is optimal. For a target $\mathcal{N}(0,1)$ truncated to $[c, \infty)$ with $c > 0$ and proposal $c + \text{Exp}(\lambda)$, find the optimal λ.

10.5.3 Theory Show that the Gibbs sampler that updates coordinates in a fixed order is *not* reversible in general, although each coordinate update is. Why is π nonetheless stationary?

10.5.4 Theory Verify the AR(1) representation of Example 10.17, and show that $\tau = (1+\rho^2)/(1-\rho^2)$.

10.5.5 Python Modify Listing 10.14 to use the proposal scale $0.1\times$ and $10\times$ the optimal value. Report acceptance rates and ESS, and plot trace plots for both. Relate your results to the 0.234 rule.

10.5.6 Python Implement an independence Metropolis–Hastings sampler for the same logistic posterior with a multivariate Student-t_4 proposal centred at the mode with the Laplace covariance. Compare its ESS per second with RWM, and its estimates with self-normalised importance sampling from the same proposal.

10.5.7 Policy Using the posterior draws of Listing 10.14, suppose the information treatment costs 5 units per household and an additional enrolee is worth 40 units. Compute the posterior distribution of net benefit per household and the probability that the programme is cost-effective. How would you present this to a ministry?

10.5.8 ★ Graduate Write a Gibbs sampler for the Bayesian probit model using the data augmentation of Albert and Chib (1993): latent $z_i \sim \mathcal{N}(\mathbf{x}_i^\top \boldsymbol{\beta}, 1)$ truncated to be positive if $y_i = 1$ and negative otherwise, and $\boldsymbol{\beta} \mid \mathbf{z}$ normal. Compare its mixing with RWM on the same data.

10.6 Designing Simulation Studies for Policy

Monte Carlo is not only a way of computing integrals. It is also the laboratory of the methodologist and the policy analyst: a *simulation study* generates many datasets from a known truth to see how an estimator, test or interval behaves, and a *microsimulation* applies a policy rule to a (real or simulated) population to see what it does. Both are experiments, and both deserve the discipline we expect of experiments: a stated aim, a design, a pre-specified analysis, a quantified error and reproducibility.

10.6.1 The ADEMP framework

Morris et al. (2019) organise the planning of a simulation study around five elements.

A – Aims.
What question is the study answering? For example, "do heteroskedasticity-robust confidence intervals for an OLS slope achieve nominal coverage in samples of 50 households?"

D – Data-generating mechanisms.
How are datasets simulated, and which factors vary (sample size, degree of heteroskedasticity, effect size, distribution of errors)? A factorial design over these factors is common; parametric DGMs should be realistic for the application.

E – Estimands.
Which quantity is being estimated or tested (here the slope β_1)?

M – Methods.
Which estimators, intervals or tests are compared (classical and HC1 standard errors)?

P – Performance measures.
How is success judged: bias, empirical standard error, MSE, coverage, power, type I error, interval width, computing time?

10.6.2 Performance measures and their Monte Carlo error

Let $\hat{\theta}_1, \ldots, \hat{\theta}_{n_{\text{sim}}}$ be the estimates from n_{sim} independent simulated datasets with true value θ, $\widehat{\text{se}}_r$ the model-based standard error in replication r, and $\bar{\theta}$ and $s_{\hat{\theta}}$ the mean and standard deviation of the estimates. Each performance measure is itself a Monte Carlo estimate, with its own MCSE (Morris et al., 2019):

Measure	Estimate	Monte Carlo SE
Bias	$\bar{\theta} - \theta$	$s_{\hat{\theta}}/\sqrt{n_{\text{sim}}}$
Empirical SE	$s_{\hat{\theta}}$	$s_{\hat{\theta}}/\sqrt{2(n_{\text{sim}}-1)}$
MSE	$\frac{1}{n_{\text{sim}}}\sum_r(\hat{\theta}_r - \theta)^2$	$\text{sd}\big((\hat{\theta}_r - \theta)^2\big)/\sqrt{n_{\text{sim}}}$
Coverage	$\hat{c} = \frac{1}{n_{\text{sim}}}\sum_r \mathbf{1}\{\lvert\hat{\theta}_r - \theta\rvert \le z\,\widehat{\text{se}}_r\}$	$\sqrt{\hat{c}(1-\hat{c})/n_{\text{sim}}}$
Rejection rate	$\hat{\pi} = \frac{1}{n_{\text{sim}}}\sum_r \mathbf{1}\{p_r \le \alpha\}$	$\sqrt{\hat{\pi}(1-\hat{\pi})/n_{\text{sim}}}$

(The empirical-SE formula assumes approximately normal estimates.) The table also tells us how many replications to run. To estimate a coverage near 0.95 with MCSE at most 0.005 we need $n_{\text{sim}} \ge 0.95 \times 0.05/0.005^2 = 1900$; to detect a bias of size δ we need $s_{\hat{\theta}}/\sqrt{n_{\text{sim}}}$ well below δ.

10.6.3 Example: confidence intervals under heteroskedasticity

Listing 10.17 implements a small ADEMP study. The DGM is $y = 1 + 0.5x + e$ with $x \sim \text{Exp}(1)$ and $\text{sd}(e \mid x) = x^{\gamma}$, with $\gamma \in \{0, 1\}$ (homoskedastic or strongly heteroskedastic) and $n \in \{50, 400\}$. The methods are OLS with classical and with HC1 (White) standard errors (White, 1980; Hansen, 2022), and the performance measures are bias, empirical SE and coverage of nominal 95% intervals, each with its MCSE. All 4000 replications of a scenario are generated as one array, so the study runs in about a second.

Listing 10.17: An ADEMP simulation study of OLS confidence intervals ch10/ch10_sim_study.py

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# ADEMP simulation study: OLS confidence intervals under heteroskedasticity.
# (c) 2026 Stephen E. Moore. MIT Licence.
import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")  # tiny matrices: one thread
import numpy as np
```

```

# Aims: do classical and HC1 (White) 95% CIs for the slope achieve coverage?
# Data-generating mechanisms: y = 1 + 0.5 x + e, x ~ Exp(1), sd(e) = x^gamma
# Estimand: slope beta1 = 0.5.  Methods: OLS + classical / HC1 s.e.
# Performance: bias, empirical s.e., coverage, each with its MC s.e.
rng = np.random.default_rng(803)
n_sim, beta1 = 4000, 0.5
print(f"n_sim = {n_sim} per scenario; Monte Carlo s.e. in parentheses")
print("  n  gamma  bias            emp.se   cover(classical)  cover(HC1)")
for n in (50, 400):
    for gam in (0.0, 1.0):
        x = rng.exponential(1.0, (n_sim, n))
        e = rng.standard_normal((n_sim, n)) * x**gam
        y = 1 + beta1 * x + e
        xc = x - x.mean(axis=1, keepdims=True)
        sxx = (xc**2).sum(axis=1)
        b1 = (xc * y).sum(axis=1) / sxx
        b0 = y.mean(axis=1) - b1 * x.mean(axis=1)
        res = y - b0[:, None] - b1[:, None] * x
        se_cl = np.sqrt((res**2).sum(axis=1) / (n - 2) / sxx)
        se_hc = np.sqrt(n / (n - 2) * (xc**2 * res**2).sum(axis=1)) / sxx
        bias = b1.mean() - beta1
        emp = b1.std(ddof=1)
        out = f"{n:3d}  {gam:4.1f}  {bias: .4f} ({emp / np.sqrt(n_sim):.4f})"
        out += f"  {emp:.4f}"
        for se in (se_cl, se_hc):
            cov = np.mean(np.abs(b1 - beta1) <= 1.96 * se)
            mcse = np.sqrt(cov * (1 - cov) / n_sim)
            out += f"   {cov:.3f} ({mcse:.4f})"
        print(out)
# How many replications for a coverage MC s.e. of 0.005 at 95%?
print(f"n_sim needed for MC s.e. 0.005 at coverage 0.95: "
      f"{int(np.ceil(0.95 * 0.05 / 0.005**2))}")
```

Output

```
n_sim = 4000 per scenario; Monte Carlo s.e. in parentheses
 n  gamma  bias            emp.se   cover(classical)  cover(HC1)
 50   0.0   0.0018 (0.0024)  0.1525   0.947 (0.0036)   0.904 (0.0047)
 50   1.0   0.0179 (0.0072)  0.4544   0.590 (0.0078)   0.837 (0.0058)
400   0.0   0.0001 (0.0008)  0.0497   0.955 (0.0033)   0.947 (0.0035)
400   1.0   0.0028 (0.0029)  0.1829   0.564 (0.0078)   0.928 (0.0041)
n_sim needed for MC s.e. 0.005 at coverage 0.95: 1900
```

Under homoskedasticity both intervals are close to 95% at $n = 400$, but the HC1 interval undercovers at $n = 50$ (0.904, MCSE 0.005): robust standard errors are biased downwards in small samples. Under heteroskedasticity, the classical interval is disastrous (56–59% coverage at both sample sizes: more data do not help, because the standard error is inconsistent), while HC1 approaches nominal coverage as n grows (0.84 at $n = 50$, 0.93 at $n = 400$). OLS remains unbiased throughout; the problem is entirely in the uncertainty statement. Every number is reported with an MCSE, so the reader can tell that 0.947 and 0.955 are both consistent with 0.95, while 0.904 is not.

△ Caution: Common flaws in simulation studies

- No Monte Carlo standard errors, so differences between methods cannot be judged.
- DGMs chosen to favour the proposed method, or so simple that every method works.

- Failed replications (non-convergence) silently dropped, biasing the comparison; report how many failed and why.
- Using the same random numbers for different scenarios without saying so (common random numbers are a valid variance-reduction device for *comparisons*, but induce correlation between reported results).
- No seed, no code, no record of software versions: the study cannot be reproduced.

10.6.4 Reproducibility and random number generation

Pseudo-random number generators are deterministic algorithms, and that is a virtue: a fixed seed makes a simulation exactly reproducible. NumPy's `default_rng` uses the PCG64 generator; every script in this book starts from `np.random.default_rng(803)`. Good practice is to (i) set one seed per study and record it; (ii) pass the generator object explicitly to functions rather than relying on global state; (iii) for parallel replications, spawn independent streams with `np.random.SeedSequence(803).spawn(k)` rather than using seeds $1, 2, \ldots, k$; (iv) store the raw replication-level results, not just summaries, so that new performance measures can be computed later; and (v) record package versions. A simulation study is a piece of empirical research, and these are its data-management standards.

10.6.5 Microsimulation of a cash-transfer programme

A *static microsimulation* model applies a tax or benefit rule to each unit of a household survey, recomputes incomes, and aggregates the result into poverty, inequality and fiscal indicators (Bourguignon and Spadaro, 2006). Each indicator, such as the FGT index (10.3), is an integral over the income distribution, estimated by a weighted average over the sample. Monte Carlo enters in two further ways: to represent *behaviour and implementation* that are random at the household level (who is enrolled, who is mis-targeted), and to propagate *parameter uncertainty* (how much of the transfer is consumed, how accurate the targeting formula is) into the results.

Listing 10.18 builds such a model on a *simulated* household survey of 8000 households (these are not data from any country). Per-capita consumption is lognormal and lower for rural and larger households; the poverty line is 900; individuals are counted by weighting households by size. A transfer with a budget of 2% of total consumption is targeted by a proxy-means test (PMT): the targeting score is log consumption plus noise, and the households with the lowest 25% of scores are eligible. Three parameters are uncertain and drawn afresh in each of $S = 1000$ outer replications:

- PMT error standard deviation, U(0.2, 0.6), governing inclusion and exclusion errors;
- marginal propensity to consume the transfer, U(0.7, 1.0);
- take-up among eligible households, Beta(16, 4) (mean 0.8).

Within each replication, household-level targeting errors and take-up decisions are also random. The budget is divided equally among enrolled persons, so lower take-up raises the per-person transfer.

Listing 10.18: Microsimulation of a targeted cash transfer under uncertainty `ch10/ch10_microsim.py`

```
# DEP 803 -- Chapter 10: Numerical Integration and Monte Carlo Simulation
# Microsimulation of a targeted cash transfer on poverty, with uncertainty.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

```

```
rng = np.random.default_rng(803)
# A simulated household survey (NOT real data): 8,000 households
H = 8000
size = rng.integers(1, 9, H)
rural = rng.random(H) < 0.55
logc = 7.55 - 0.35 * rural - 0.06 * size + rng.normal(0, 0.55, H)
cons = np.exp(logc)                                # per-capita consumption
zline = 900.0                                      # poverty line (per capita)


def fgt(c, alpha, w):
    g = np.maximum(1 - c / zline, 0.0)
    return np.sum(w * g**alpha * (c < zline)) / w.sum()


w = size.astype(float)                             # person weights
P0, P1 = fgt(cons, 0, w), fgt(cons, 1, w)
budget = 0.02 * np.sum(w * cons)                   # 2% of total consumption


def simulate(pmt_noise, mpc, take_up, rng):
    """One draw of the policy outcome given uncertain parameters."""
    score = logc + rng.normal(0, pmt_noise, H)     # proxy-means test score
    elig = score < np.quantile(score, 0.25)        # bottom 25% by PMT score
    enrol = elig & (rng.random(H) < take_up)
    per_cap = budget / np.sum(w * enrol)           # budget split per person
    new = cons + mpc * per_cap * enrol
    return fgt(new, 0, w), fgt(new, 1, w)


# Outer loop: parameter uncertainty; inner noise: targeting and take-up
S = 1000
draws = np.empty((S, 2))
for s in range(S):
    pmt_noise = rng.uniform(0.2, 0.6)              # targeting error
    mpc = rng.uniform(0.7, 1.0)                    # share consumed
    take_up = rng.beta(16, 4)                      # mean 0.8
    draws[s] = simulate(pmt_noise, mpc, take_up, rng)
d0, d1 = 100 * (P0 - draws[:, 0]), 100 * (P1 - draws[:, 1])
print(f"baseline headcount P0 = {100 * P0:.2f}%, poverty gap P1 = {100 * P1:.2f}%")
print(f"budget per poor person (if perfectly targeted): "
      f"{budget / np.sum(w * (cons < zline)):.1f}")
for lab, d in (("headcount", d0), ("poverty gap", d1)):
    lo, hi = np.percentile(d, [5, 95])
    print(f"reduction in {lab:11s}: mean {d.mean():.3f} pp (MC s.e. "
          f"{d.std(ddof=1) / np.sqrt(S):.3f}), 90% band [{lo:.3f}, {hi:.3f}]")
# Benchmark: perfect targeting of the poorest 25%, full take-up, mpc = 1
best = simulate(0.0, 1.0, 1.0, rng)
print(f"benchmark (no PMT error, full take-up, mpc 1): P0 {100 * best[0]:.2f}%"
      f", P1 {100 * best[1]:.2f}%")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.hist(d0, bins=40, color="steelblue", alpha=0.75, label="headcount (P0)")
ax.hist(d1, bins=40, color="darkorange", alpha=0.75, label="poverty gap (P1)")
ax.set_xlabel("reduction in poverty index (percentage points)")
ax.set_ylabel("number of simulation draws")
ax.legend()
plt.savefig("ch10_microsim.pdf", bbox_inches="tight")
```

Output

```
baseline headcount P0 = 36.00%, poverty gap P1 = 10.40%
budget per poor person (if perfectly targeted): 73.0
reduction in headcount  : mean 2.207 pp (MC s.e. 0.010), 90% band [1.714, 2.684]
reduction in poverty gap: mean 1.907 pp (MC s.e. 0.008), 90% band [1.512, 2.339]
benchmark (no PMT error, full take-up, mpc 1): P0 36.00%, P1 7.48%
```

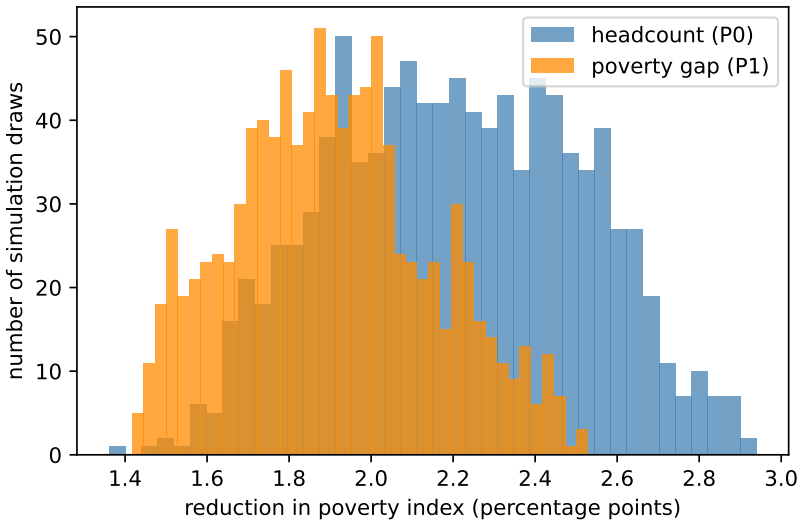


Figure 10.11: Distribution over 1000 simulation draws of the reduction (percentage points) in the poverty headcount and the poverty gap produced by a PMT-targeted transfer in a simulated population. The spread reflects uncertainty in targeting error, consumption response and take-up.

The transfer reduces the headcount by 2.2 percentage points on average (90% simulation band 1.7–2.7) and the poverty gap by 1.9 points (1.5–2.3). The Monte Carlo standard errors of these means, 0.010 and 0.008, show that $S = 1000$ replications are ample: the reported uncertainty is almost entirely due to the parameters, not to the simulation. A striking benchmark is the last line. With *perfect* targeting of the poorest quarter, full take-up and full consumption, the poverty gap falls by 2.9 points, much more than under the noisy PMT, but the headcount does not fall at all: the transfer is too small to lift any of the poorest households over the line. Targeting errors that leak benefits to the near-poor reduce the headcount but are less effective against the gap.

Policy Lens: Which poverty measure should a programme be judged by?

The microsimulation illustrates a classic point: a budget-constrained planner who minimises the headcount should give transfers to those *closest* to the poverty line, which is rarely what an anti-poverty programme intends. The poverty gap P_1 and squared gap P_2 reward reaching the poorest. Simulation lets the analyst show decision makers the full trade-off, with honest uncertainty bands, before a programme is launched: how outcomes vary with the targeting threshold, with the accuracy of the PMT, with take-up, and with the budget. Results should always be presented with their ranges and with a statement that the household data and parameters are simulated or estimated, not observed outcomes.

⋆ Key Concept: Simulation studies and microsimulation

Plan simulation studies with ADEMP: aims, data-generating mechanisms, estimands, methods and performance measures. Every performance measure (bias, empirical SE, coverage, power) is a Monte Carlo estimate and must be reported with its MCSE; choose n_{sim} from the MCSE you need. Microsimulation applies policy rules to unit-level data; nesting it inside an outer Monte Carlo loop over uncertain parameters turns a point projection into a distribution of policy outcomes. Fix and record seeds, store replication-level results, and keep parameter uncertainty and simulation error conceptually separate.

Graduate Perspective: Uncertainty quantification and Bayesian decision analysis

The outer loop of the microsimulation draws parameters from ad hoc distributions. A more principled approach takes them from a posterior (for example from a Bayesian model of take-up estimated on pilot data, using MCMC as in Section 10.5), so that the policy-outcome distribution is a posterior predictive distribution. Decision theory then chooses the policy that maximises posterior expected social welfare, and the *expected value of perfect information*, also a Monte Carlo integral, tells the planner how much it would be worth to resolve the uncertainty by further data collection before deciding. Global sensitivity analysis (Sobol' indices) decomposes the output variance into contributions from each uncertain input.

Exercises

10.6.1 Theory Derive the MCSE of the empirical standard error, $s_{\hat{\theta}}/\sqrt{2(n_{\text{sim}}-1)}$, for normally distributed estimates using the delta method and $\text{Var}(s^2) = 2\sigma^4/(n-1)$.

10.6.2 Theory A simulation study compares two estimators on the same simulated datasets. Show that the MCSE of the difference in their MSEs is smaller than if independent datasets were used whenever the squared errors are positively correlated. What is the analogue for coverage?

10.6.3 Python Extend Listing 10.17 with the HC3 standard error and a wild bootstrap interval, and add $n = 100$ and $\gamma = 0.5$ to the design. Present the results as a table with MCSEs and as a "nested loop" plot.

10.6.4 Python Add a fourth performance measure to Listing 10.17: the average width of the HC1 interval relative to the classical one, with its MCSE. Interpret the coverage–width trade-off.

10.6.5 Policy Using Listing 10.18, vary the eligibility threshold from the bottom 10% to the bottom 50% of PMT scores for a fixed budget. Plot the mean reduction in P_0, P_1 and P_2 with 90% bands against the threshold and advise on the choice.

10.6.6 Policy Replace the equal per-person transfer with a transfer proportional to the predicted poverty gap $\max(1 - \hat{c}/z, 0)$ based on the PMT score. Compare outcomes and discuss the incentives such a design creates.

10.6.7 ★ Graduate Separate the two sources of uncertainty in Listing 10.18: for each of $S = 200$ parameter draws, run $R = 20$ inner replications, and use a one-way ANOVA (law of total variance) to estimate the share of the variance in the headcount reduction due to parameters versus household-level randomness. How should S and R be chosen for a fixed budget of $S \times R$ runs?

Chapter Summary

- Expectations, probabilities, poverty indices, choice probabilities, marginal likelihoods, posterior means, consumer surplus and present values are integrals $I = \mathbb{E}_p[h(X)]$.
- The composite trapezoidal rule has error $-(b-a)h^2 f''(\xi)/12$, Simpson's rule $-(b-a)h^4 f^{(4)}(\xi)/180$; n-point Gaussian quadrature is exact to degree $2n-1$, has positive weights, and Gauss–Hermite computes normal expectations with a few nodes. Tensor rules cost m^d points: the curse of dimensionality.
- Plain Monte Carlo $\hat{I}_n = \frac{1}{n}\sum h(X_i)$ is unbiased with variance σ^2/n; it is asymptotically normal, its MCSE is $\hat{\sigma}/\sqrt{n}$, and its $n^{-1/2}$ rate is independent of dimension and smoothness, though σ may not be.
- Antithetic variates, control variates ($c^\star = -\operatorname{Cov}(h,g)/\operatorname{Var}(g)$, gain $1/(1-\rho^2)$), importance sampling ($q^\star \propto |h|\,p$, heavy-tailed proposals, ESS) and stratification (Neyman allocation) reduce σ; randomised quasi-Monte Carlo improves the rate towards n^{-1}.
- Inverse-CDF and acceptance–rejection sampling are exact; MCMC constructs a Markov chain whose stationary distribution is the target. Metropolis–Hastings satisfies detailed balance and needs the target only up to a constant; Gibbs sampling is MH with acceptance one; HMC uses gradients and powers Stan.
- MCMC output must be diagnosed: several chains, warm-up, trace plots, autocorrelation, ESS $= T/\tau$, and $\widehat{R} < 1.01$.
- Simulation studies should follow ADEMP and report Monte Carlo standard errors for every performance measure; microsimulation with an outer loop over uncertain parameters gives policy-outcome distributions rather than point projections. Seeds, code and replication-level results make the work reproducible.

Further Reading

Robert and Casella (2004) is the standard graduate text on Monte Carlo statistical methods, covering random variable generation, variance reduction and the theory of MCMC; Liu (2001) is a strong complement, and Glasserman (2003) develops variance reduction and quasi-Monte Carlo in depth for financial applications. For deterministic quadrature, Davis and Rabinowitz (1984) remains the classic, and Judd (1998) shows how quadrature, Monte Carlo and quasi-Monte Carlo are used to solve economic models. Train (2009) is the reference on simulation-based estimation of discrete choice models, including Halton draws for mixed logit, and Cameron and Trivedi (2005) covers simulation-based econometrics more broadly. On the Bayesian side, Gelman et al. (2013) explains MCMC and its diagnostics in applied modelling, Matsuura (2022) is a practical guide to Stan with R and Python, and Murphy (2022) places Monte Carlo inference in the context of probabilistic machine learning; the HMC review of Neal (2011) and the $\widehat{R}$ paper of Vehtari et al. (2021) are essential reading for anyone using Stan. Géron (2022) uses sampling and stochastic methods throughout the training of machine-learning models, and Goulet Coulombe et al. (2022) illustrates the out-of-sample, simulation- and resampling-based evaluation of forecasting methods in macroeconomics. For the design of simulation studies read Morris et al. (2019); for microsimulation in policy analysis, Bourguignon and Spadaro (2006). The history of the method is told by Metropolis (1987) and Eckhardt (1987).

Chapter 11

Resampling: the Bootstrap, Permutation Tests and Uncertainty Intervals

Learning Objectives

After working through this chapter you should be able to:

(i) explain what a sampling distribution, a standard error and a confidence interval are, and check the frequentist meaning of "95% coverage" by simulation;

(ii) write a statistic as a functional $T(F)$ of the distribution function and apply the plug-in principle, including to medians, Gini coefficients, ratio estimators and poverty indices;

(iii) implement the nonparametric and parametric bootstrap, choose the number of replications, and recognise the situations in which the bootstrap fails;

(iv) construct and compare normal, percentile, basic, studentised and BCa bootstrap intervals, by hand and with `scipy.stats.bootstrap`;

(v) choose an appropriate bootstrap for regression and dependent data: pairs, residual, wild, cluster, wild cluster and moving block bootstraps;

(vi) carry out exact and Monte Carlo randomisation tests for experiments, invert them into confidence sets, simulate power, and control the family-wise error rate and the false discovery rate;

(vii) use the jackknife, cross-validation, split conformal prediction and pseudo-out-of-sample evaluation to quantify predictive uncertainty.

Every number that a statistical office, a central bank or a research team publishes is an estimate computed from a finite, noisy sample. The national poverty headcount comes from a household survey of a few thousand households; the effect of a school-feeding programme comes from a trial in a few dozen schools; next year's inflation forecast comes from a model fitted to a few hundred monthly observations. A number without a statement of its uncertainty invites overconfident decisions: a fall in measured poverty from 24.2% to 23.5% may be a genuine improvement or pure sampling noise, and only a standard error or a confidence interval can tell the two apart.

For the textbook statistics of Chapter 7, the sample mean and the least squares coefficients, the uncertainty has a closed form. But the quantities that matter most in economic policy are rarely so convenient. The *median* income, the *Gini coefficient*, the *poverty gap*, a *ratio* such as the food share of household expenditure, the treatment effect in a *clustered* survey or a forecast *several steps ahead*: for each, an analytic standard error requires a delicate asymptotic derivation, if one exists at all. *Resampling* replaces this derivation by computation. We treat the observed sample

as a stand-in for the population, generate many artificial samples from it, recompute the statistic on each, and read the uncertainty directly from the variation we observe. The idea builds on the Monte Carlo methods of Chapter 10: simulation, but from the data themselves.

This chapter covers the main resampling tools used in modern data science and econometrics. The *bootstrap* (Sections 11.2 to 11.4) estimates sampling distributions and builds confidence intervals; *permutation and randomisation tests* (Section 11.5) give exact tests for experiments under minimal assumptions; the *jackknife*, *cross-validation* and *conformal prediction* (Section 11.6) quantify the uncertainty of predictions. Throughout, the guiding question is the same: *if we could repeat the data collection, how different might our answer be?*

Historical Note: From the jackknife to the bootstrap

Maurice Quenouille introduced the idea of recomputing an estimator on subsets of the data to reduce its bias, first for serial correlation in time series (Quenouille, 1949) and then in general form (Quenouille, 1956). John Tukey (Tukey, 1958) saw that the same leave-out replicates also give a variance estimate, and named the method the *jackknife*: a rough-and-ready tool that is useful in many situations. Bradley Efron's 1979 paper, "Bootstrap methods: another look at the jackknife" (Efron, 1979), recast the problem: sampling *with replacement* from the data imitates sampling from the population. The name alludes to the impossible feat of pulling oneself up by one's own bootstraps. Cheap computing turned the bootstrap into one of the most widely used statistical tools of the late twentieth century; see Efron and Tibshirani (1993) and Davison and Hinkley (1997).

11.1 Sampling Distributions and the Plug-in Principle

11.1.1 Sampling distributions, standard errors and confidence intervals

Let $X_1, \dots, X_n \overset{\text{iid}}{\sim} F$ be a sample from an unknown distribution F and let $\widehat{\theta} = s(X_1, \dots, X_n)$ be a statistic that estimates a parameter $\theta = \theta(F)$. Because the sample is random, so is $\widehat{\theta}$.

Definition 11.1 (Sampling distribution, standard error)**.** The *sampling distribution* of $\widehat{\theta}$ is its probability distribution under repeated sampling from F:

$$G_n(t) = \mathbb{P}_F\big(\widehat{\theta} \le t\big), \qquad t \in \mathbb{R}.$$

The *standard error* of $\widehat{\theta}$ is the standard deviation of its sampling distribution, $\operatorname{se}_F(\widehat{\theta}) = \sqrt{\operatorname{Var}_F(\widehat{\theta})}$, and its *bias* is $\operatorname{Bias}_F(\widehat{\theta}) = \mathbb{E}_F\widehat{\theta} - \theta(F)$.

Both G_n and $\operatorname{se}_F(\widehat{\theta})$ depend on the unknown F; this is the central difficulty of inference. For the sample mean $\bar{X}$ we know that $\operatorname{se}_F(\bar{X}) = \sigma/\sqrt{n}$, so a natural estimate is $s/\sqrt{n}$. For most other statistics no such formula is available.

Definition 11.2 (Confidence interval and coverage)**.** A (two-sided) $1-\alpha$ *confidence interval* for θ is a pair of statistics $L = L(X_{1:n})$, $U = U(X_{1:n})$ such that

$$\mathbb{P}_F\big(L \le \theta(F) \le U\big) \;\ge\; 1-\alpha \qquad \text{for all } F \text{ in the model.}$$

The left-hand side is the *coverage probability*. An interval is *asymptotically valid* if the coverage tends to $1-\alpha$ as $n \to \infty$, and the *coverage error* is $\mathbb{P}_F(L \le \theta \le U) - (1-\alpha)$.

The probability statement is about the random endpoints L and U, not about θ, which is fixed. "We are 95% confident that $\theta \in [L, U]$" means: the procedure that produced $[L, U]$ captures the

true value in 95% of hypothetical repetitions of the survey. Once the data are in, the realised interval either contains θ or it does not. The cleanest way to understand this is to simulate it.

Example 11.3 (Coverage of the t-interval for skewed incomes)**.** Incomes are right-skewed. Suppose household income is lognormal, $\log X \sim \mathcal{N}(7, 1)$, so that the population mean is $e^{7.5} \approx 1808$ (arbitrary units). For each sample size n we draw 20 000 independent "surveys", compute the textbook interval $\bar{X} \pm t_{n-1,0.975}\, s/\sqrt{n}$ on each, and record how often it contains 1808.

Listing 11.1: Coverage of the t-interval for a skewed mean (plotting code omitted)
ch11/ch11_coverage_sim.py

```python
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Frequentist coverage of the 95% t-interval for a mean of skewed incomes.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)

# Simulated "population" of monthly incomes: lognormal (arbitrary units)
mu, sigma = 7.0, 1.0
true_mean = np.exp(mu + sigma**2 / 2)
R = 20000                       # Monte Carlo replications per sample size

print("Coverage of the nominal 95% t-interval for the mean income")
print(f"true mean = {true_mean:.1f}, replications = {R}")
print(f"{'n':>6} {'coverage':>9} {'miss low':>9} {'miss high':>10} {'avg len':>9}")
for n in (10, 30, 100, 500):
    x = rng.lognormal(mu, sigma, size=(R, n))
    xbar, s = x.mean(axis=1), x.std(axis=1, ddof=1)
    half = stats.t.ppf(0.975, n - 1) * s / np.sqrt(n)
    lo, hi = xbar - half, xbar + half
    cover = np.mean((lo <= true_mean) & (true_mean <= hi))
    below = np.mean(hi < true_mean)          # interval entirely below truth
    above = np.mean(lo > true_mean)          # interval entirely above truth
    print(f"{n:6d} {cover:9.3f} {below:9.3f} {above:10.3f} {np.mean(2*half):9.1f}")

# Figure: 60 realised intervals for n = 30
n, K = 30, 60
x = rng.lognormal(mu, sigma, size=(K, n))
xbar, s = x.mean(axis=1), x.std(axis=1, ddof=1)
half = stats.t.ppf(0.975, n - 1) * s / np.sqrt(n)
miss = (xbar - half > true_mean) | (xbar + half < true_mean)
```

Output

```
Coverage of the nominal 95% t-interval for the mean income
true mean = 1808.0, replications = 20000
     n  coverage  miss low  miss high   avg len
    10     0.839     0.159      0.001    2693.1
    30     0.885     0.113      0.002    1566.7
   100     0.916     0.079      0.005     890.7
   500     0.940     0.050      0.010     409.1
figure: 7 of 60 intervals miss the true mean
```

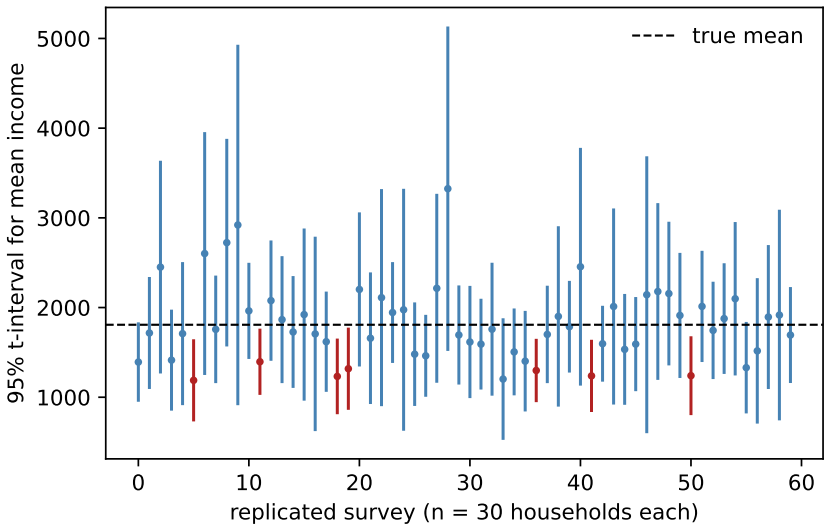


Figure 11.1: Sixty realised 95% t-intervals for the mean of a lognormal income distribution, each from a different simulated survey of $n = 30$ households. Intervals that miss the true mean (dashed) are drawn in red; nearly all of them lie below it.

The output is sobering. With $n = 10$ households the nominal 95% interval covers the truth only 84% of the time, and almost all of the failures are on one side: the interval lies entirely *below* the true mean. The reason is that a skewed sample usually misses the rare very large incomes, so $\bar{X}$ is typically too small *and* s is typically too small at the same time. The central limit theorem eventually rescues the interval, but slowly: even with $n = 100$ the coverage is only about 92%. Figure 11.1 shows 60 realised intervals for $n = 30$.

⋆ Key Concept: Coverage is a property of a procedure

A confidence interval is a *procedure* for turning data into a range. Its coverage probability $\mathbb{P}_F(L \le \theta \le U)$ is computed over repeated samples from F and can be checked by Monte Carlo simulation: simulate many samples from a known F, apply the procedure, and count how often the truth is captured. Nominal coverage (the "95%" in the name) and actual coverage can differ substantially in small or skewed samples; the gap is the coverage error, and reducing it is the main goal of the more refined bootstrap intervals in Section 11.3.

11.1.2 The empirical distribution function

The key object of this chapter is the empirical distribution function.

Definition 11.4 (Empirical distribution function)**.** The *empirical distribution function* (ECDF) of $X_1, \dots, X_n$ is

$$\widehat{F}_n(x) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{X_i \le x\}, \qquad x \in \mathbb{R}.$$

Equivalently, $\widehat{F}_n$ is the distribution that puts probability mass $1/n$ on each observed data point.

For each fixed x, $n\widehat{F}_n(x) \sim \text{Binomial}(n, F(x))$, so $\mathbb{E}\widehat{F}_n(x) = F(x)$ and $\text{Var}\,\widehat{F}_n(x) = F(x)(1 - F(x))/n$. Much more is true: the ECDF converges to F *uniformly*.

Theorem 11.5 (Glivenko–Cantelli). *If* $X_1, X_2, \ldots \overset{iid}{\sim} F$, *then* $\sup_{x\in\mathbb{R}} \left|\widehat{F}_n(x) - F(x)\right| \xrightarrow{a.s.} 0$.

Proof sketch. Fix $\varepsilon > 0$ and choose points $-\infty = x_0 < x_1 < \cdots < x_k = \infty$ with $F(x_j^-) - F(x_{j-1}) \le \varepsilon$. By the strong law of large numbers, $\widehat{F}_n(x_j) \to F(x_j)$ and $\widehat{F}_n(x_j^-) \to F(x_j^-)$ almost surely for each of the finitely many j. For $x \in [x_{j-1}, x_j)$ monotonicity gives $\widehat{F}_n(x) - F(x) \le \widehat{F}_n(x_j^-) - F(x_{j-1}) \le \widehat{F}_n(x_j^-) - F(x_j^-) + \varepsilon$, and similarly from below. Hence $\limsup_n \sup_x \left|\widehat{F}_n - F\right| \le \varepsilon$ almost surely; let $\varepsilon \downarrow 0$. □

A quantitative, finite-sample version is the Dvoretzky–Kiefer–Wolfowitz inequality, with the sharp constant due to Massart (1990):

$$\mathbb{P}\Big(\sup_x \left|\widehat{F}_n(x) - F(x)\right| > \varepsilon\Big) \;\le\; 2e^{-2n\varepsilon^2} \qquad \text{for all } \varepsilon > 0. \tag{11.1}$$

Setting the right-hand side equal to α gives a distribution-free $1-\alpha$ confidence *band* $\widehat{F}_n(x) \pm \varepsilon_n$ with $\varepsilon_n = \sqrt{\log(2/\alpha)/(2n)}$, valid simultaneously for all x.

11.1.3 Statistical functionals and the plug-in principle

Many parameters are functions of the whole distribution F.

Definition 11.6 (Statistical functional, plug-in estimator). A *statistical functional* is a map $T : F \mapsto T(F) \in \mathbb{R}$ from a class of distributions to the real line. The *plug-in estimator* of $\theta = T(F)$ is $\widehat{\theta} = T(\widehat{F}_n)$: the same functional evaluated at the empirical distribution.

Example 11.7 (Plug-in estimators of common policy parameters).

(a) Mean: $T(F) = \int x\, dF(x)$, so $T(\widehat{F}_n) = \bar{X}$.

(b) Variance: $T(F) = \int (x - \mu_F)^2 dF$, so $T(\widehat{F}_n) = \frac{1}{n}\sum (X_i - \bar{X})^2$, which is biased by the factor $(n-1)/n$.

(c) Quantiles: $T(F) = F^{-1}(p) = \inf\{x : F(x) \ge p\}$, so $T(\widehat{F}_n)$ is the sample p-quantile; $p = 1/2$ gives the median.

(d) Gini coefficient: with $\mu = \int x\, dF$, $T(F) = \frac{1}{2\mu} \iint \left|x - y\right| dF(x)\, dF(y)$. Plugging in $\widehat{F}_n$ and simplifying with the order statistics $X_{(1)} \le \cdots \le X_{(n)}$ gives

$$\widehat{G} = T(\widehat{F}_n) = \frac{\sum_{i=1}^n (2i - n - 1)\, X_{(i)}}{n \sum_{i=1}^n X_i}. \tag{11.2}$$

(e) Foster–Greer–Thorbecke poverty indices (Foster et al., 1984) with poverty line z: $P_\alpha(F) = \int \left(\frac{z-x}{z}\right)^\alpha \mathbf{1}\{x < z\}\, dF(x)$, so the plug-in estimator is the sample average of $((z - X_i)/z)^\alpha \mathbf{1}\{X_i < z\}$. Here $\alpha = 0$ is the headcount ratio, $\alpha = 1$ the poverty gap and $\alpha = 2$ the severity index.

(f) Ratio: for pairs (Y_i, X_i), e.g. food expenditure and total income, $R(F) = \mathbb{E}Y/\mathbb{E}X$ and $\widehat{R} = \bar{Y}/\bar{X}$.

For the lognormal model the Gini coefficient has the closed form $G = 2\Phi(\sigma/\sqrt{2}) - 1$, which lets us check the plug-in estimates against the truth.

Listing 11.2: The ECDF, the DKW band and plug-in estimators (plotting code omitted)
ch11/ch11_ecdf_plugin.py

```python
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# The empirical distribution function, the DKW band and plug-in estimators.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)


def gini(x):
    """Plug-in Gini coefficient of the empirical distribution of x."""
    x = np.sort(x)
    n = x.size
    i = np.arange(1, n + 1)
    return np.sum((2 * i - n - 1) * x) / (n * np.sum(x))


def fgt(x, z, alpha):
    """Foster-Greer-Thorbecke poverty index P_alpha with poverty line z."""
    gap = np.clip((z - x) / z, 0, None)
    return np.mean(gap**alpha * (x < z))


mu, sigma, n = 7.0, 0.9, 200
x = rng.lognormal(mu, sigma, size=n)     # simulated household incomes
z = 600.0                                # illustrative poverty line
pop = stats.lognorm(s=sigma, scale=np.exp(mu))
big = pop.rvs(size=2_000_000, random_state=rng)   # for "true" FGT values

rows = [("mean", x.mean(), pop.mean()),
        ("median", np.median(x), pop.median()),
        ("Gini", gini(x), 2 * stats.norm.cdf(sigma / np.sqrt(2)) - 1),
        ("P0 headcount", fgt(x, z, 0), pop.cdf(z)),
        ("P1 poverty gap", fgt(x, z, 1), fgt(big, z, 1)),
        ("P2 severity", fgt(x, z, 2), fgt(big, z, 2))]
print(f"Plug-in estimates from one simulated survey (n = {n})")
print(f"{'functional':<16}{'plug-in T(F_n)':>16}{'true T(F)':>12}")
for name, est, tru in rows:
    print(f"{name:<16}{est:16.4f}{tru:12.4f}")

# Kolmogorov distance and the DKW 95% band
xs = np.sort(x)
Fn = np.arange(1, n + 1) / n
F = pop.cdf(xs)
D = np.max(np.maximum(np.abs(Fn - F), np.abs(Fn - 1 / n - F)))
eps = np.sqrt(np.log(2 / 0.05) / (2 * n))
print(f"sup|F_n - F| = {D:.4f};  DKW 95% half-width = {eps:.4f}")
```

Output

```
Plug-in estimates from one simulated survey (n = 200)
functional        plug-in T(F_n)    true T(F)
```

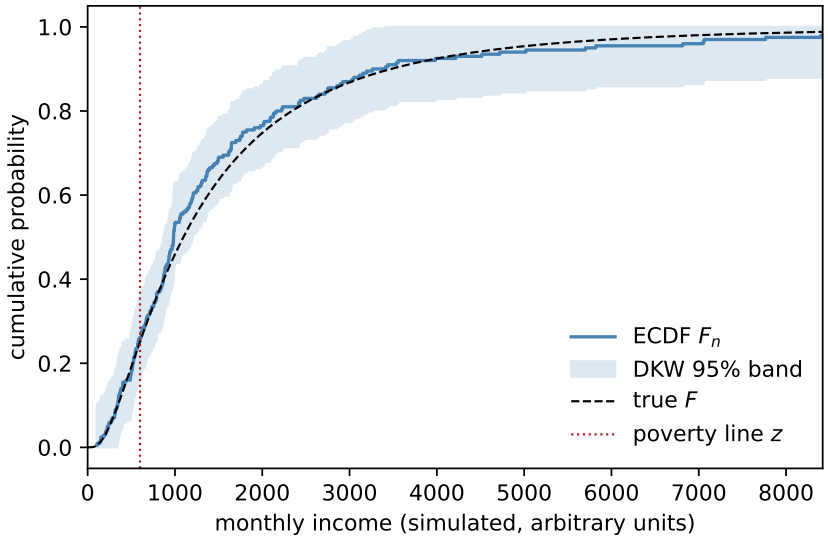


Figure 11.2: Empirical distribution function of a simulated survey of $n = 200$ household incomes, with the 95% Dvoretzky–Kiefer–Wolfowitz band and the true lognormal distribution function. The poverty headcount is the height of $\widehat{F}_n$ at the poverty line.

```
mean                  1647.2545   1644.1848
median                 983.7885   1096.6332
Gini                     0.5011      0.4755
P0 headcount             0.2600      0.2514
P1 poverty gap           0.0944      0.0919
P2 severity              0.0487      0.0464
sup|F_n - F| = 0.0751;  DKW 95% half-width = 0.0960
```

The plug-in estimates are close to the truth, but by how much might they be off? For the mean we have $s/\sqrt{n}$. For the others we need a general tool. One such tool, which motivates much of what follows, is the linear approximation of a functional.

11.1.4 When analytic standard errors are hard: the influence function

Definition 11.8 (Influence function)**.** The *influence function* of T at F is the derivative of T in the direction of a point mass δ_x:

$$\mathrm{IF}(x; T, F) = \lim_{\varepsilon \downarrow 0} \frac{T\big((1-\varepsilon)F + \varepsilon \delta_x\big) - T(F)}{\varepsilon}.$$

For a smooth functional, a first-order (von Mises) expansion gives

$$T(\widehat{F}_n) - T(F) \;\approx\; \frac{1}{n}\sum_{i=1}^{n} \mathrm{IF}(X_i; T, F), \qquad \mathbb{E}_F\,\mathrm{IF}(X; T, F) = 0, \tag{11.3}$$

so that $\sqrt{n}\,(T(\widehat{F}_n) - T(F)) \xrightarrow{d} \mathcal{N}(0, \tau^2)$ with $\tau^2 = \mathbb{E}_F\,\mathrm{IF}(X;T,F)^2$. This is the delta method of Chapter 7 in function space. Three examples show both its power and its inconvenience:

- Mean: $\mathrm{IF}(x) = x - \mu$, so $\tau^2 = \sigma^2$, as expected.

- Ratio: $\mathrm{IF}(y, x) = (y - Rx)/\mu_X$, so $\mathrm{se}(\widehat{R}) \approx \sqrt{\mathrm{Var}(Y - RX)/n}\,/\mu_X$ (the linearisation formula used by survey statisticians).
- Median: $\mathrm{IF}(x) = \mathrm{sign}(x - m)/(2f(m))$, so $\mathrm{se}(\hat{m}) \approx 1/(2f(m)\sqrt{n})$, which requires an estimate of the *density* f at the median, a nonparametric smoothing problem in its own right.

For the Gini coefficient the influence function exists but is messy, and for poverty measures with an *estimated* poverty line (e.g. 60% of median income) one must also propagate the uncertainty of the line. Each new statistic demands a new derivation. The bootstrap automates precisely this step.

△ Caution: Plug-in estimates can be biased

Plug-in estimators are consistent for smooth functionals, but they are often biased in finite samples: the plug-in variance divides by n; the plug-in Gini is biased downwards because small samples rarely contain the extreme top incomes; the ratio $\bar{Y}/\bar{X}$ has bias of order $1/n$. When the sample is small and the policy decision hinges on a threshold (e.g. eligibility rules defined by a poverty rate), estimate and report the bias as well as the standard error; Sections 11.2 and 11.6 show how.

Policy Lens: Uncertainty in official poverty and inequality statistics

National poverty and inequality figures are estimated from household surveys with complex designs: stratification by region and urban/rural status, clustering of households within enumeration areas, and sampling weights. Analysts report standard errors and confidence intervals for headcount ratios and Gini coefficients so that users can judge whether a change between survey rounds, or a difference between regions, exceeds sampling noise. Because the analytic variance of a Gini coefficient under a clustered design is cumbersome, replication methods (the jackknife and the bootstrap of primary sampling units, Section 11.4) are a standard way of computing it; see Deaton (1997) for the econometrics of household surveys. A useful discipline for any analyst is never to publish a poverty rate without its interval.

Exercises

11.1.1 Theory Show that for each fixed x, $\widehat{F}_n(x)$ is unbiased for $F(x)$ with variance $F(x)(1-F(x))/n$, and that $\mathrm{Cov}(\widehat{F}_n(x), \widehat{F}_n(y)) = \{F(\min(x,y)) - F(x)F(y)\}/n$.

11.1.2 Theory Derive the order-statistic formula (11.2) for the plug-in Gini coefficient from the definition $T(F) = \frac{1}{2\mu}\iint |x - y|\, dF\, dF$. Show also that the plug-in Gini satisfies $0 \le \widehat{G} \le 1 - 1/n$ for non-negative data.

11.1.3 Theory Compute the influence function of the ratio functional $R(F) = \mathbb{E}Y/\mathbb{E}X$ and use (11.3) to derive its asymptotic standard error.

11.1.4 Python Modify `ch11_coverage_sim.py` to compute the coverage of the t-interval for the mean when $\log X \sim \mathcal{N}(7, \sigma^2)$ for $\sigma \in \{0.25, 0.5, 1, 1.5\}$ and $n \in \{10, 30, 100\}$. Plot coverage against σ. At what skewness does $n = 100$ stop being "large"?

11.1.5 Python Using the DKW inequality, compute the sample size needed so that the 95% band for F has half-width 0.01. What does this imply for the precision of a poverty headcount estimated from a survey of that size, and why is the pointwise interval much narrower?

11.1.6 ★ Graduate Show that the influence function of the p-quantile is $\mathrm{IF}(x) = (p - \mathbf{1}\{x \le F^{-1}(p)\})/f(F^{-1}(p))$ and deduce the asymptotic variance $p(1-p)/(n f(F^{-1}(p))^2)$. Evaluate

Algorithm 6 Nonparametric bootstrap standard error, bias and distribution

Require: data $X_1, \dots, X_n$; statistic $s(\cdot)$; number of replications B

1: $\widehat{\theta} \leftarrow s(X_1, \dots, X_n)$
2: **for** $b = 1, \dots, B$ **do**
3: draw indices $I_1, \dots, I_n$ independently and uniformly from $\{1, \dots, n\}$
4: $\widehat{\theta}^*_b \leftarrow s(X_{I_1}, \dots, X_{I_n})$
5: **end for**
6: $\bar{\theta}^* \leftarrow \frac{1}{B}\sum_b \widehat{\theta}^*_b$, $\widehat{\mathrm{se}}_B \leftarrow \left\{\frac{1}{B-1}\sum_b(\widehat{\theta}^*_b - \bar{\theta}^*)^2\right\}^{1/2}$, $\widehat{\mathrm{Bias}}_B \leftarrow \bar{\theta}^* - \widehat{\theta}$
7: **return** $\widehat{\mathrm{se}}_B$, $\widehat{\mathrm{Bias}}_B$ and the empirical distribution of $\{\widehat{\theta}^*_b\}$

it for the median of the lognormal model in `ch11_ecdf_plugin.py` and compare with the bootstrap standard error in Section 11.2.

11.1.7 Policy A statistical office reports that the national poverty headcount fell from 25.1% to 23.9% between two independent survey rounds of 8 000 households each. Assuming simple random sampling, is the change statistically significant at the 5% level? How does your answer change if the design effect due to clustering is 2.5? Explain to a non-specialist what the "design effect" means.

11.2 The Nonparametric Bootstrap

11.2.1 The bootstrap principle

The plug-in principle estimates a *parameter* $T(F)$ by $T(\widehat{F}_n)$. The bootstrap applies the same principle one level up: it estimates the *sampling distribution* of $\widehat{\theta}$ under F by the sampling distribution of the same statistic under $\widehat{F}_n$.

Definition 11.9 (Bootstrap sample and bootstrap distribution)**.** A *bootstrap sample* $X^*_1, \dots, X^*_n$ is an iid sample of size n from $\widehat{F}_n$, i.e. a sample drawn *with replacement* from $\{X_1, \dots, X_n\}$. Writing $\mathbb{P}^*$, $\mathbb{E}^*$ and Var^* for probability, expectation and variance conditional on the data, the *bootstrap distribution* of $\widehat{\theta}^* = s(X^*_1, \dots, X^*_n)$ is $\widehat{G}_n(t) = \mathbb{P}^*(\widehat{\theta}^* \le t)$, and the *ideal bootstrap estimates* of standard error and bias are

$$\mathrm{se}_{\widehat{F}_n}(\widehat{\theta}) = \sqrt{\mathrm{Var}^*(\widehat{\theta}^*)}, \qquad \widehat{\mathrm{Bias}} = \mathbb{E}^*(\widehat{\theta}^*) - T(\widehat{F}_n).$$

The analogy is exact: the real world has population F, sample $X_{1:n}$ and estimate $\widehat{\theta}$; the bootstrap world has "population" $\widehat{F}_n$, sample $X^*_{1:n}$ and estimate $\widehat{\theta}^*$. In the bootstrap world everything is known, so any feature of the sampling distribution can be computed, at least by simulation. Except in simple cases, the ideal bootstrap quantities have no closed form (there are $\binom{2n-1}{n}$ distinct resamples), so we approximate them by Monte Carlo.

In NumPy the loop disappears: one call `rng.integers(0, n, size=(B, n))` produces all B index vectors at once, and a statistic written to act along the last axis is evaluated on all resamples simultaneously. The next listing uses this to bootstrap four statistics of a simulated household survey, and compares the bootstrap standard errors with the "true" standard errors obtained from 20 000 fresh surveys drawn from the population.

Listing 11.3: Bootstrap standard errors and bias (plotting code omitted)
ch11/ch11_bootstrap_basic.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Nonparametric bootstrap standard errors and bias for four statistics.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)


def gini(x, axis=-1):
    """Plug-in Gini coefficient along the last axis (`axis` is for scipy)."""
    x = np.sort(x, axis=-1)
    n = x.shape[-1]
    w = 2 * np.arange(1, n + 1) - n - 1
    return np.sum(w * x, axis=-1) / (n * np.sum(x, axis=-1))


def draw(size):
    """Simulated household survey: income and food expenditure."""
    inc = rng.lognormal(7.0, 0.9, size=size)
    food = 0.45 * inc**0.85 * rng.lognormal(0, 0.3, size=size)
    return inc, food


stats_fns = {
    "mean income": lambda inc, food: inc.mean(axis=-1),
    "median income": lambda inc, food: np.median(inc, axis=-1),
    "Gini": lambda inc, food: gini(inc),
    "food share (ratio)": lambda inc, food: food.mean(-1) / inc.mean(-1),
}

n, B = 150, 4000
inc, food = draw(n)                          # the one observed sample
idx = rng.integers(0, n, size=(B, n))        # B resamples of row indices
inc_b, food_b = inc[idx], food[idx]

# "Truth" for comparison: sampling distribution from 20000 fresh surveys
inc_mc, food_mc = draw((20000, n))

print(f"n = {n}, B = {B} bootstrap replications")
print(f"{'statistic':<20}{'estimate':>10}{'boot se':>10}{'true se':>10}"
      f"{'boot bias':>11}")
for name, f in stats_fns.items():
    t_hat = f(inc, food)
    t_star = f(inc_b, food_b)
    se_boot = t_star.std(ddof=1)
    bias_boot = t_star.mean() - t_hat
    se_true = f(inc_mc, food_mc).std(ddof=1)
    print(f"{name:<20}{t_hat:10.4f}{se_boot:10.4f}{se_true:10.4f}"
          f"{bias_boot:11.4f}")
print(f"analytic se of mean, s/sqrt(n) = {inc.std(ddof=1) / np.sqrt(n):.4f}")

g_hat, g_star = gini(inc), gini(inc_b)
```

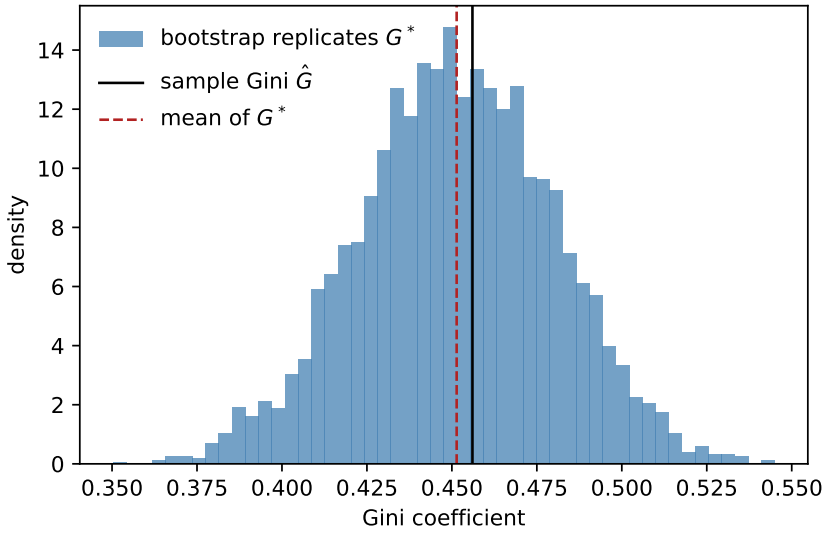


Figure 11.3: Bootstrap distribution of the Gini coefficient for a simulated survey of $n = 150$ households ($B = 4000$). The distribution is slightly skewed and centred a little below $\widehat{G}$: the bootstrap detects the downward bias of the plug-in Gini.

Output

```
n = 150, B = 4000 bootstrap replications
statistic             estimate   boot se   true se  boot bias
mean income          1502.5024  126.8523  150.1459    -1.1686
median income         985.9282   58.8148  101.3965     8.0945
Gini                    0.4560    0.0285    0.0301    -0.0046
food share (ratio)      0.1480    0.0060    0.0057     0.0002
analytic se of mean, s/sqrt(n) = 128.4887
```

Several lessons emerge. For the Gini coefficient and the ratio, the bootstrap standard errors are within about 5% of the truth, with no derivation at all. For the mean, the bootstrap reproduces the analytic $s/\sqrt{n}$ (up to the factor $\sqrt{(n-1)/n}$), and both are below the truth because *this particular sample* happens to under-represent the top tail: the bootstrap can only know what the sample shows it. For the median, the bootstrap standard error is far from the truth; the bootstrap distribution of a sample quantile is discrete and its standard error is itself very noisy from sample to sample (Exercise **11.2.5**). The bootstrap bias of the Gini, -0.005, is a sizeable fraction of its standard error.

⋆ Key Concept: The bootstrap principle

To learn how $\widehat{\theta}$ varies around $\theta = T(F)$, study how $\widehat{\theta}^*$ varies around $\widehat{\theta} = T(\widehat{F}_n)$:

$$\text{real world: } F \longrightarrow X_{1:n} \longrightarrow \widehat{\theta} \qquad \text{bootstrap world: } \widehat{F}_n \longrightarrow X^*_{1:n} \longrightarrow \widehat{\theta}^*.$$

The bootstrap introduces two approximations: the *statistical* error of replacing F by $\widehat{F}_n$ (which does not vanish as $B \to \infty$) and the *Monte Carlo* error of using finite B (which does). Only the second is under the analyst's control.

11.2.2 Bias correction

If $\widehat{\text{Bias}}$ estimates $\mathbb{E}\hat{\theta} - \theta$, the *bias-corrected estimator* is

$$\tilde{\theta} = \hat{\theta} - \widehat{\text{Bias}} = 2\hat{\theta} - \bar{\theta}^*.$$

Bias correction removes the leading $O(n^{-1})$ term of the bias of smooth statistics, but it also increases variance. Efron and Tibshirani (1993) suggest a rule of thumb: if $\left|\widehat{\text{Bias}}\right| / \widehat{\text{se}} < 0.25$ the bias is negligible relative to sampling uncertainty and correction is not worth its cost. For the Gini above the ratio is about 0.16.

11.2.3 How many bootstrap replications?

The bootstrap standard error $\widehat{\text{se}}_B$ is a Monte Carlo estimate of the ideal $\widehat{\text{se}}_\infty$. Conditional on the data, a delta-method argument gives

$$\text{CV}^*(\widehat{\text{se}}_B) = \frac{\sqrt{\text{Var}^*(\widehat{\text{se}}_B)}}{\widehat{\text{se}}_\infty} \approx \sqrt{\frac{\hat{\kappa} + 2}{4B}}, \tag{11.4}$$

where $\hat{\kappa}$ is the excess kurtosis of the bootstrap distribution. For a roughly normal bootstrap distribution, CV $\approx 1/\sqrt{2B}$: $B = 200$ gives 5% Monte Carlo error in the standard error, which is small relative to the statistical error. Quantiles in the tails are much harder to estimate, so confidence intervals need B of the order of 1000–10 000.

Monte Carlo error of the bootstrap as a function of B: output of `ch11_boot_reps.py` (the full program is in `code/ch11`).

Output

```
median = 926.1; ideal bootstrap se = 76.35, 97.5% quantile = 1126.5
Variability of the bootstrap output over 300 repeated bootstraps
     B  mean se_B  sd se_B  CV(se_B)  sd q975
    25      74.34    16.35     0.220    58.85
    50      75.80    11.34     0.150    47.64
   100      76.30     7.97     0.104    34.53
   200      76.05     5.61     0.074    25.18
   500      76.32     3.40     0.045    15.46
  1000      76.27     2.54     0.033    10.85
  2000      76.32     1.82     0.024     7.90
```

The coefficient of variation falls like $1/\sqrt{B}$, as (11.4) predicts, while the standard deviation of the estimated 97.5% quantile at $B = 200$ is about a third of a standard error. Because computation is cheap, a sensible default today is $B \geq 1000$ for standard errors and $B \geq 5000$ for interval endpoints, with the Monte Carlo error checked by repeating the bootstrap with a different seed.

△ Caution: Common implementation mistakes

(i) Resample *units*, not variables: for paired data (Y_i, X_i) the same index must select both components (otherwise the dependence between them is destroyed).

(ii) Recompute the *whole* estimator on each resample, including data-driven choices such as bandwidths, poverty lines defined relative to the median, or selected tuning parameters.

(iii) Do not report $\bar{\theta}^*$ as "the bootstrap estimate"; the estimate is $\hat{\theta}$.

(iv) Fix and report the random seed so that results are reproducible.

11.2.4 Why the bootstrap works

The bootstrap is justified by showing that the conditional distribution of a suitably normalised $\widehat{\theta}^*$ approaches the distribution of the normalised $\widehat{\theta}$. The basic result is for the mean (Bickel and Freedman, 1981).

Theorem 11.10 (Bootstrap consistency for the mean)**.** *Let $X_1, X_2, \ldots \overset{iid}{\sim} F$ with $\mathbb{E}X^2 < \infty$, mean μ and variance $\sigma^2 > 0$. Then, for almost every sequence $X_1, X_2, \ldots$,*

$$\sup_{x \in \mathbb{R}} \left| \mathbb{P}^*\big(\sqrt{n}(\bar{X}^* - \bar{X}) \le x\big) - \mathbb{P}\big(\sqrt{n}(\bar{X} - \mu) \le x\big) \right| \longrightarrow 0.$$

Proof sketch. Conditional on the data, $\sqrt{n}(\bar{X}^* - \bar{X}) = n^{-1/2} \sum_{i=1}^n (X_i^* - \bar{X})$ is a normalised sum of iid variables with mean 0 and variance $\hat{\sigma}_n^2 = \frac{1}{n}\sum (X_i - \bar{X})^2$, which converges almost surely to σ^2 by the strong law. The Lindeberg condition for this triangular array, $\mathbb{E}^*[(X^* - \bar{X})^2 \mathbf{1}\{|X^* - \bar{X}| > \varepsilon\sqrt{n}\}] \to 0$, follows from $\frac{1}{n}\sum X_i^2 \mathbf{1}\{|X_i| > c\} \to \mathbb{E}[X^2 \mathbf{1}\{|X| > c\}]$ and the finiteness of $\mathbb{E}X^2$. Hence $\sqrt{n}(\bar{X}^* - \bar{X}) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$ conditionally, almost surely. The CLT gives the same limit for $\sqrt{n}(\bar{X} - \mu)$, and since the limit is continuous, Pólya's theorem upgrades pointwise to uniform convergence. □

The argument extends to smooth functionals through the expansion (11.3): if T is Hadamard differentiable, the functional delta method applies equally in the real and the bootstrap world, so the bootstrap is consistent for $\sqrt{n}(T(\widehat{F}_n) - T(F))$; see van der Vaart (1998, Ch. 23). This covers means, variances, ratios, smooth functions of moments, quantiles of distributions with positive densities, the Gini coefficient and the FGT indices with a fixed poverty line.

Graduate Perspective: Consistency is not the end of the story

Consistency says the bootstrap gets the first-order (normal) approximation right. Its real advantage appears at the next order. Edgeworth expansions show that for an asymptotically *pivotal* statistic, such as the studentised mean $(\bar{X} - \mu)/(s/\sqrt{n})$, the bootstrap approximation has error $O_p(n^{-1})$, whereas the normal approximation has error $O(n^{-1/2})$: the bootstrap automatically corrects for skewness. This *second-order accuracy* (Hall, 1992) is why studentised and BCa intervals (Section 11.3) outperform the simple ones in small samples.

11.2.5 When the bootstrap fails

The bootstrap is not a universal remedy. It can fail when the statistic is not a smooth functional of F, when the parameter is on the boundary of the parameter space, when tails are too heavy, or when n is too small for $\widehat{F}_n$ to resemble F.

Example 11.11 (The sample maximum)**.** Let $X_i \overset{\text{iid}}{\sim} \text{Uniform}(0, \theta)$ and $\widehat{\theta} = X_{(n)}$. Then $n(\theta - \widehat{\theta})/\theta \xrightarrow{d} \text{Exp}(1)$, a continuous law. But the bootstrap maximum equals $\widehat{\theta}$ whenever the largest observation is drawn at least once, so

$$\mathbb{P}^*\big(\widehat{\theta}^* = \widehat{\theta}\big) = 1 - \left(1 - \frac{1}{n}\right)^n \longrightarrow 1 - e^{-1} \approx 0.632.$$

The bootstrap distribution has an atom of mass 0.632 at zero that never disappears, so it cannot converge to the continuous limit. The *parametric* bootstrap, which resamples from $\text{Uniform}(0, \widehat{\theta})$, does converge.

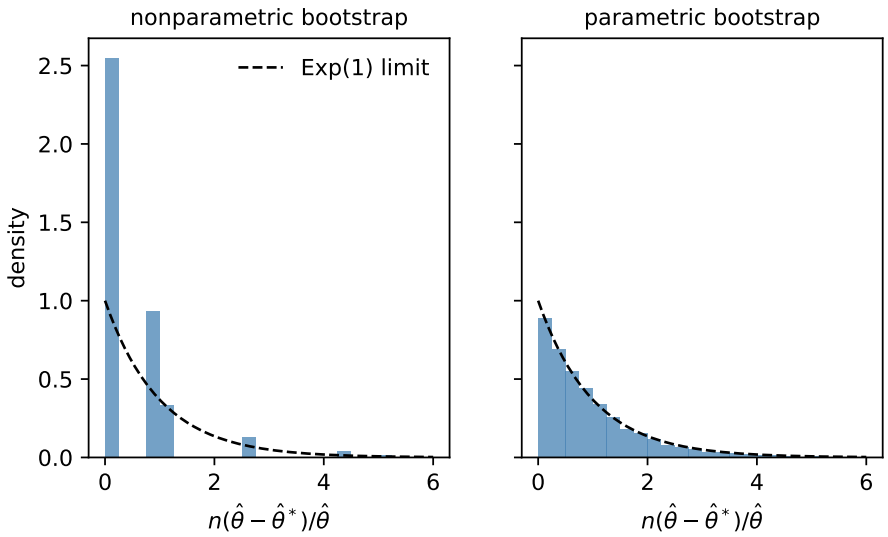


Figure 11.4: Bootstrap distributions of $n(\hat\theta - \hat\theta^*)/\hat\theta$ for the maximum of a Uniform$(0, \theta)$ sample with $n = 50$. The nonparametric bootstrap (left) puts 63% of its mass at zero (the first bar, far off the scale of the limiting density); the parametric bootstrap (right) matches the Exp(1) limit.

Definition 11.12 (Parametric bootstrap)**.** If the model is $\{F_\theta\}$ and $\widehat\theta$ is an estimate (e.g. the MLE), the *parametric bootstrap* draws $X_1^*, \dots, X_n^* \overset{\text{iid}}{\sim} F_{\widehat\theta}$ instead of $\widehat F_n$, and otherwise proceeds as in Algorithm 6.

The parametric bootstrap is more efficient when the model is right and can be badly wrong when it is not. It is the natural tool for models that are fully specified anyway (e.g. computing the distribution of a likelihood-ratio statistic, or simulating from a fitted time-series model in Section 11.6).

A second failure involves heavy tails. If $\mathbb{E}X^2 = \infty$, the theorem above fails: for a distribution in the domain of attraction of a stable law with index $\alpha \in (1, 2)$, the bootstrap distribution of the mean converges to a *random* limit (Athreya, 1987). Income and wealth distributions often have Pareto upper tails with α between 1.5 and 3, so this is not merely a theoretical curiosity.

Two failures of the nonparametric bootstrap: output of `ch11_boot_failure.py` (the full program is in `code/ch11`).

Output

```
(a) maximum of a Uniform(0, theta) sample, n = 50
  P*(theta* = theta_hat): nonparametric = 0.636, theory 1-(1-1/n)^n = 0.636
  P*(theta* = theta_hat): parametric    = 0.000 (true limit law is continuous)
(b) coverage of the 95% percentile bootstrap CI for a mean, n = 100
  Pareto, alpha = 4.0 (finite variance)    coverage = 0.922
  Pareto, alpha = 1.5 (infinite variance)  coverage = 0.736
```

△ Caution: Very small samples

With $n = 10$ the empirical distribution has only ten support points; the bootstrap cannot invent information about tails it has not seen. For the skewed income data of Example 11.3, the normal, percentile, basic and BCa bootstrap intervals all cover the mean less than 83%

of the time when $n = 10$ (Section 11.3). The bootstrap is a large-sample method that often works well in moderate samples, not a cure for small ones.

Think About It

A colleague bootstraps the *maximum* daily rainfall over 30 years to obtain a confidence interval for a 100-year flood level used in an insurance scheme for farmers. Using Example 11.11, explain what will go wrong. What alternatives (parametric extreme-value models, m-out-of-n bootstrap) might be used instead?

Exercises

11.2.1 Theory Show that the number of distinct bootstrap resamples (as multisets) of n distinct points is $\binom{2n-1}{n}$, and evaluate it for $n = 10$ and $n = 20$.

11.2.2 Theory For the sample mean, show that $\mathbb{E}^* \bar{X}^* = \bar{X}$ and $\mathrm{Var}^* \bar{X}^* = \hat{\sigma}^2/n$ with $\hat{\sigma}^2 = \frac{1}{n}\sum (X_i - \bar{X})^2$. Conclude that the ideal bootstrap standard error of the mean is $\sqrt{(n-1)/n}\, s/\sqrt{n}$.

11.2.3 Theory Prove that $\mathbb{P}^*(X_{(n)} \notin \{X_1^*, \dots, X_n^*\}) = (1 - 1/n)^n$ and that the expected number of distinct original observations in a bootstrap sample is $n\{1 - (1 - 1/n)^n\} \approx 0.632\, n$.

11.2.4 Python Bootstrap the FGT poverty gap P_1 for the sample in `ch11_ecdf_plugin.py` twice: with the poverty line fixed at $z = 600$, and with the line set at 60% of the sample median and recomputed on every resample. By how much does the standard error increase when the line is estimated?

11.2.5 Python Repeat `ch11_bootstrap_basic.py` on 500 independent samples and plot the distribution of the bootstrap standard error of the median relative to its true value. Compare with the same plot for the Gini coefficient. Why is the median so much worse?

11.2.6 Python Implement the m-out-of-n bootstrap for the Uniform maximum (resample $m = \lfloor \sqrt{n} \rfloor$ points and normalise by m instead of n) and show by simulation that it fixes the failure in Example 11.11.

11.2.7 ★ Graduate Derive (11.4): if $\widehat{\theta}_1^*, \dots, \widehat{\theta}_B^*$ are iid with variance τ^2 and excess kurtosis κ, show that $\mathrm{Var}(\hat{\tau}_B^2) \approx \tau^4(\kappa + 2)/B$ and apply the delta method to $\hat{\tau}_B = \sqrt{\hat{\tau}_B^2}$.

11.2.8 Policy A ministry estimates that average monthly household consumption in a district rose by 4% between two survey rounds, with a bootstrap standard error of 2.5% computed by resampling households. Write a short memo explaining what assumptions the bootstrap standard error rests on, and which features of the survey design (clustering, stratification, weights) it may have ignored.

11.3 Bootstrap Confidence Intervals

A standard error summarises the spread of the bootstrap distribution in one number. A confidence interval can use its whole shape. Throughout this section let $\widehat{\theta}^*_{(\beta)}$ denote the β-quantile of the bootstrap replicates $\widehat{\theta}_1^*, \dots, \widehat{\theta}_B^*$, let $\widehat{\mathrm{se}}$ be the bootstrap standard error and $z_\beta = \Phi^{-1}(\beta)$.

11.3.1 Five intervals

Definition 11.13 (Bootstrap confidence intervals)**.** The following are approximate $1-\alpha$ intervals for θ.

(a) *Normal*: $\widehat{\theta} \pm z_{1-\alpha/2}\,\widehat{\mathrm{se}}$ (optionally centred at the bias-corrected $\widehat{\theta} - \widehat{\mathrm{Bias}}$).

(b) *Percentile*: $\big[\widehat{\theta}^*_{(\alpha/2)},\ \widehat{\theta}^*_{(1-\alpha/2)}\big]$.

(c) *Basic* (reverse percentile): $\big[2\widehat{\theta} - \widehat{\theta}^*_{(1-\alpha/2)},\ 2\widehat{\theta} - \widehat{\theta}^*_{(\alpha/2)}\big]$.

(d) *Studentised* (bootstrap-t): with $t^*_b = (\widehat{\theta}^*_b - \widehat{\theta})/\widehat{\mathrm{se}}^*_b$, where $\widehat{\mathrm{se}}^*_b$ is a standard error computed *within* the b-th resample, $\big[\widehat{\theta} - t^*_{(1-\alpha/2)}\widehat{\mathrm{se}},\ \widehat{\theta} - t^*_{(\alpha/2)}\widehat{\mathrm{se}}\big]$.

(e) *BCa* (bias-corrected and accelerated): $\big[\widehat{\theta}^*_{(\beta_1)},\ \widehat{\theta}^*_{(\beta_2)}\big]$ with

$$\beta_j = \Phi\Big(\hat{z}_0 + \frac{\hat{z}_0 + z_{\alpha_j}}{1 - \hat{a}(\hat{z}_0 + z_{\alpha_j})}\Big), \quad \alpha_1 = \tfrac{\alpha}{2},\ \alpha_2 = 1 - \tfrac{\alpha}{2}, \tag{11.5}$$

bias-correction $\hat{z}_0 = \Phi^{-1}\big(\#\{\widehat{\theta}^*_b < \widehat{\theta}\}/B\big)$ and acceleration

$$\hat{a} = \frac{\sum_{i=1}^n (\bar{\theta}_{(\cdot)} - \widehat{\theta}_{(i)})^3}{6\big\{\sum_{i=1}^n (\bar{\theta}_{(\cdot)} - \widehat{\theta}_{(i)})^2\big\}^{3/2}}, \tag{11.6}$$

where $\widehat{\theta}_{(i)}$ is the estimate with observation i deleted (the jackknife replicate) and $\bar{\theta}_{(\cdot)}$ their mean.

11.3.2 Where the intervals come from

All five intervals approximate the distribution of a *root* $R = \widehat{\theta} - \theta$ (or a standardised version of it) by the bootstrap distribution of $R^* = \widehat{\theta}^* - \widehat{\theta}$.

Basic interval. If R and R^* have approximately the same distribution, then $\mathbb{P}(r_{\alpha/2} \le \widehat{\theta} - \theta \le r_{1-\alpha/2}) \approx 1-\alpha$ with r_β the quantiles of R^*, i.e. $r_\beta = \widehat{\theta}^*_{(\beta)} - \widehat{\theta}$. Solving for θ gives $[\widehat{\theta} - r_{1-\alpha/2},\ \widehat{\theta} - r_{\alpha/2}]$, which is the basic interval. Note the *reversal*: a long right tail in the bootstrap distribution moves the interval to the *left*.

Percentile interval. The percentile interval is justified differently. Suppose there is an unknown increasing transformation ϕ such that $\phi(\widehat{\theta}) - \phi(\theta) \sim \mathcal{N}(0, c^2)$ exactly, a symmetric pivot. Then the percentile interval in the ϕ scale is exact, and since quantiles commute with increasing transformations, mapping back gives exactly $[\widehat{\theta}^*_{(\alpha/2)}, \widehat{\theta}^*_{(1-\alpha/2)}]$; we never need to know ϕ. The percentile interval is therefore *transformation-respecting*: the interval for $\log\theta$ is the log of the interval for θ. It also always lies within the range of possible values (e.g. $[0,1]$ for a Gini coefficient), whereas the normal and basic intervals may not.

Studentised interval. The studentised interval approximates the distribution of the *pivot* $(\widehat{\theta} - \theta)/\widehat{\mathrm{se}}$, which for the mean is the t-statistic. Because its asymptotic distribution does not depend on unknown parameters, the bootstrap captures its skewness correctly, which gives second-order accuracy. The price is that we need a standard error in every resample: an analytic formula, a jackknife, or a nested bootstrap.

BCa interval. Efron (1987) generalised the percentile argument to allow (i) a median bias, $\phi(\widehat{\theta}) - \phi(\theta) \sim \mathcal{N}(-z_0 c, c^2)$, and (ii) a standard deviation that changes with the parameter, $c = 1 + a\phi(\theta)$. Under this model the exact interval is the percentile interval with adjusted levels (11.5). The constant z_0 measures the median bias of $\widehat{\theta}^*$ relative to $\widehat{\theta}$; the *acceleration* a measures how fast the standard error changes with θ and is related to the skewness of the influence function, which is why it can be estimated from jackknife replicates. When $z_0 = a = 0$, BCa reduces to the percentile interval.

Theorem 11.14 (Order of coverage error)**.** *Under smoothness conditions (a smooth function of means, with a non-lattice distribution and enough moments), the one-sided coverage errors of nominal $1-\alpha$ bootstrap intervals satisfy*

$$\mathbb{P}\big(\theta \le U\big) = 1 - \alpha + \begin{cases} O(n^{-1/2}) & \text{normal, percentile, basic,} \\ O(n^{-1}) & \text{studentised, BCa.} \end{cases}$$

For two-sided equal-tailed intervals the $n^{-1/2}$ terms cancel, and all five have coverage error $O(n^{-1})$, but the constants for the first three are typically much larger in skewed problems.

Idea of proof. Write the distribution of the studentised statistic as an Edgeworth expansion $\mathbb{P}(\sqrt{n}(\widehat{\theta} - \theta)/\hat{\sigma} \le x) = \Phi(x) + n^{-1/2} q(x)\phi(x) + O(n^{-1})$, where the polynomial q depends on skewness-type moments of F. The bootstrap distribution has the same expansion with F replaced by $\widehat{F}_n$, i.e. with $\hat{q}$ in place of q, and $\hat{q} - q = O_p(n^{-1/2})$. Hence the bootstrap quantiles of the pivot are correct to $O_p(n^{-1})$. For a non-pivotal root the leading term involves the unknown variance, which the bootstrap estimates only to $O_p(n^{-1/2})$, leaving an error of that order. The full argument is in Hall (1992). □

11.3.3 A confidence interval for the Gini coefficient

Listing 11.4 computes all five intervals for the Gini coefficient of a simulated sample of $n = 150$ incomes, using the jackknife within each resample for the studentised interval and for the BCa acceleration, and then compares with `scipy.stats.bootstrap`, whose `method` argument accepts `"percentile"`, `"basic"` and `"BCa"` (the default).

Listing 11.4: Five bootstrap intervals for a Gini coefficient ch11/ch11_boot_ci.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Five bootstrap confidence intervals for a Gini coefficient, plus scipy.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)


def gini(x, axis=-1):
    """Plug-in Gini coefficient along the last axis (`axis` is for scipy)."""
    x = np.sort(x, axis=-1)
    n = x.shape[-1]
    w = 2 * np.arange(1, n + 1) - n - 1
    return np.sum(w * x, axis=-1) / (n * np.sum(x, axis=-1))


def jackknife(x, stat):
    """Leave-one-out replicates theta_(i), i = 1..n."""
```

```
    n = x.size
    keep = ~np.eye(n, dtype=bool)
    return stat(np.broadcast_to(x, (n, n))[keep].reshape(n, n - 1))


def jack_se(x, stat):
    th = jackknife(x, stat)
    n = x.size
    return np.sqrt((n - 1) / n * np.sum((th - th.mean())**2))


sigma, n, B, alpha = 0.9, 150, 4999, 0.05
x = rng.lognormal(7.0, sigma, size=n)          # simulated incomes
g_true = 2 * stats.norm.cdf(sigma / np.sqrt(2)) - 1
g_hat = gini(x)
idx = rng.integers(0, n, size=(B, n))
g_star = gini(x[idx])
se_b = g_star.std(ddof=1)
z = stats.norm.ppf(1 - alpha / 2)
qlo, qhi = np.quantile(g_star, [alpha / 2, 1 - alpha / 2])

ci = {}
ci["normal"] = (g_hat - z * se_b, g_hat + z * se_b)
ci["percentile"] = (qlo, qhi)
ci["basic"] = (2 * g_hat - qhi, 2 * g_hat - qlo)

# studentised (bootstrap-t): jackknife se inside every bootstrap sample
se_hat = jack_se(x, gini)
t_star = np.array([(g_star[b] - g_hat) / jack_se(x[idx[b]], gini)
                   for b in range(B)])
tlo, thi = np.quantile(t_star, [alpha / 2, 1 - alpha / 2])
ci["studentised"] = (g_hat - thi * se_hat, g_hat - tlo * se_hat)

# BCa: bias correction z0 and jackknife acceleration a
z0 = stats.norm.ppf(np.mean(g_star < g_hat))
th = jackknife(x, gini)
d = th.mean() - th
a = np.sum(d**3) / (6 * np.sum(d**2)**1.5)
zq = stats.norm.ppf([alpha / 2, 1 - alpha / 2])
adj = stats.norm.cdf(z0 + (z0 + zq) / (1 - a * (z0 + zq)))
ci["BCa"] = tuple(np.quantile(g_star, adj))

print(f"n = {n}, B = {B}; sample Gini = {g_hat:.4f} (population {g_true:.4f})")
print(f"bootstrap se = {se_b:.4f}, jackknife se = {se_hat:.4f}")
print(f"BCa constants: z0 = {z0:.4f}, a = {a:.4f}; "
      f"quantile levels = {adj[0]:.4f}, {adj[1]:.4f}")
print(f"{'method':<14}{'lower':>9}{'upper':>9}{'length':>9}{'skew ratio':>12}")
for k, (lo, hi) in ci.items():
    print(f"{k:<14}{lo:9.4f}{hi:9.4f}{hi - lo:9.4f}"
          f"{(hi - g_hat) / (g_hat - lo):12.3f}")

print("scipy.stats.bootstrap (n_resamples = 4999):")
for m in ("percentile", "basic", "BCa"):
    res = stats.bootstrap((x,), gini, n_resamples=B, method=m,
                          random_state=np.random.default_rng(11))
    lo, hi = res.confidence_interval
    print(f"  {m:<12}{lo:9.4f}{hi:9.4f}   se = {res.standard_error:.4f}")
```

Output

```
n = 150, B = 4999; sample Gini = 0.4560 (population 0.4755)
bootstrap se = 0.0284, jackknife se = 0.0299
BCa constants: z0 = 0.1467, a = 0.0609; quantile levels = 0.0686, 0.9948
method            lower    upper   length  skew ratio
normal           0.4004   0.5117   0.1113       1.000
percentile       0.3949   0.5066   0.1117       0.826
basic            0.4055   0.5172   0.1117       1.210
studentised      0.4044   0.5399   0.1356       1.623
BCa              0.4093   0.5232   0.1139       1.437
scipy.stats.bootstrap (n_resamples = 4999):
  percentile     0.3938   0.5063   se = 0.0288
  basic          0.4058   0.5183   se = 0.0288
  BCa            0.4094   0.5232   se = 0.0288
```

The last column, $(U - \widehat{\theta})/(\widehat{\theta} - L)$, shows the asymmetry of each interval. The normal interval is symmetric by construction; the percentile interval is longer on the *left*, mirroring the bootstrap distribution; the basic, studentised and BCa intervals are longer on the *right*. Because the plug-in Gini is biased downward and its variability increases with inequality, the right-skewed intervals are the more plausible ones; indeed BCa has $\hat{z}_0 > 0$ (more than half of the replicates are below $\widehat{\theta}$) and $\hat{a} > 0$, and it shifts both quantile levels upwards, to 0.069 and 0.995. Our hand-coded BCa agrees with SciPy's to three decimal places; the small differences in the other endpoints are Monte Carlo error from different random draws.

△ Caution: Using `scipy.stats.bootstrap`

- `data` must be a *sequence* of samples: write `(x,)`, not `x`.
- With `vectorized=True` (the default in SciPy 1.9 when the statistic accepts an `axis` argument) the statistic must operate along `axis`; a statistic that silently reduces over the wrong axis gives nonsense.
- For paired data (e.g. a ratio $\bar{Y}/\bar{X}$) pass both arrays with `paired=True`, so that the same indices are used for both.
- BCa can be unstable when the statistic takes few distinct values (e.g. the median of a small sample), and will fail if all replicates are equal.

11.3.4 Which interval should we use? A coverage study

Theory tells us the order of the errors; simulation tells us their size. The program `ch11_ci_coverage.py` repeats Example 11.3 for all six intervals (the five bootstrap intervals and the classical t-interval), for the mean of a lognormal$(0, 1)$ distribution, a deliberately difficult skewed case.

Coverage study of bootstrap intervals: output of `ch11_ci_coverage.py` (the full program is in `code/ch11`).

Output

```
Coverage of nominal 95% CIs for a lognormal mean (2000 reps, B = 499)
method            n=10     n=25    n=100   avg length (n=25)
t-interval       0.839    0.871    0.918       1.548
normal           0.796    0.858    0.910       1.441
percentile       0.804    0.864    0.914       1.402
basic            0.772    0.841    0.897       1.402
studentised      0.900    0.916    0.932       2.241
```

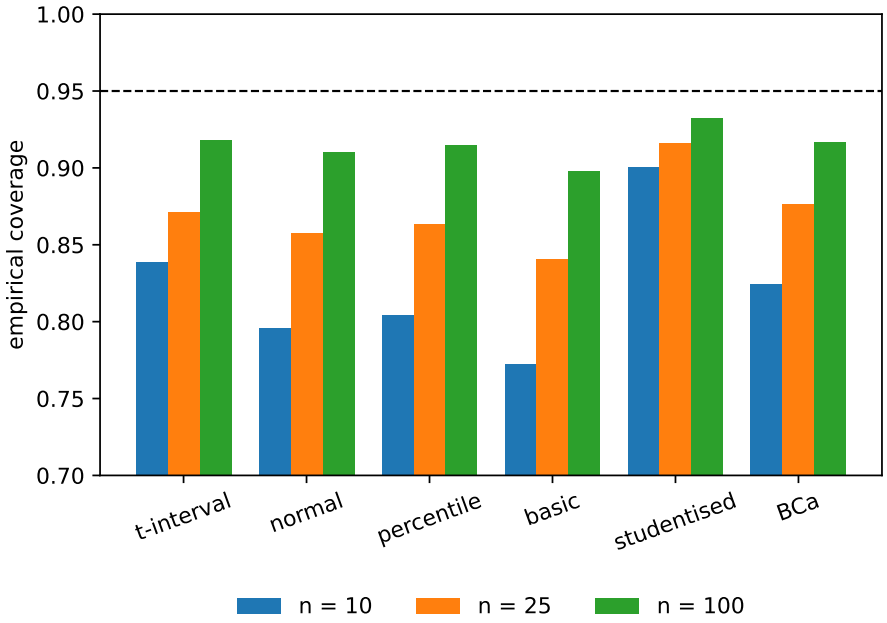


Figure 11.5: Empirical coverage of nominal 95% intervals for the mean of a lognormal$(0, 1)$ distribution (2000 Monte Carlo replications, $B = 499$). The dashed line is the nominal level.

```
BCa              0.825    0.876    0.916       1.574
```

The results, in Fig. 11.5, are typical. The normal, percentile and basic intervals all under-cover, the basic interval most of all; BCa is better than the percentile interval, but only modestly so at this level of skewness. The studentised interval comes closest to 95%, at the price of being considerably longer and occasionally very long, because a resample with a small s^* produces an extreme t^*. No interval achieves nominal coverage at $n = 10$: there the information about the right tail is simply not in the data.

⋆ Key Concept: Choosing a bootstrap interval

- Use the **percentile** interval for quick, range-respecting intervals for roughly symmetric, unbiased statistics.
- Use **BCa** as the default for smooth statistics of iid data: it is transformation respecting, corrects for bias and skewness, and is second-order accurate.
- Use the **studentised** interval when a reliable standard error is available inside each resample (e.g. means, regression coefficients with robust standard errors); it has the best coverage in skewed problems but can be long and unstable.
- Treat **normal** and **basic** intervals as diagnostics: if they disagree markedly with the percentile interval, the sampling distribution is skewed or biased.
- Check coverage by simulation from a model resembling your data whenever the decision matters.

Historical Note: Fisher, Efron and better bootstrap intervals

The percentile interval appeared in Efron (1979). Its weaknesses in skewed problems led Efron (1987) to propose the bias-corrected and accelerated (BCa) interval, published in the *Journal of the American Statistical Association* with an extended discussion. Peter Hall's monograph (Hall, 1992) used Edgeworth expansions to explain why studentised and BCa intervals are second-order accurate, and settled much of the early debate about which intervals to prefer.

Exercises

11.3.1 Theory Show that the percentile interval is transformation-respecting: if ϕ is strictly increasing, the percentile interval for $\phi(\theta)$ computed from $\phi(\widehat{\theta}^*_b)$ equals $[\phi(L), \phi(U)]$. Show by example that the basic and normal intervals do not have this property.

11.3.2 Theory Show that when $\hat{z}_0 = \hat{a} = 0$, the BCa levels (11.5) equal $\alpha/2$ and $1-\alpha/2$. Compute β_1, β_2 for $\hat{z}_0 = 0.15$, $\hat{a} = 0.06$ and $\alpha = 0.05$ and compare with the output of `ch11_boot_ci.py`.

11.3.3 Theory For the sample mean, show that the jackknife replicates satisfy $\bar{\theta}_{(\cdot)} - \widehat{\theta}_{(i)} = (X_i - \bar{X})/(n-1)$, so that the acceleration (11.6) is $\hat{a} = \frac{1}{6}\hat{\gamma}/\sqrt{n}$, where $\hat{\gamma}$ is the sample skewness (with divisor n).

11.3.4 Python Extend `ch11_ci_coverage.py` to the Gini coefficient of lognormal incomes with $\sigma = 0.9$, using the closed-form $G = 2\Phi(\sigma/\sqrt{2}) - 1$ as the truth, for $n \in \{50, 200\}$ (omit the studentised interval if it is too slow). Which interval do you recommend for a survey of 200 households?

11.3.5 Python Compute the coverage of the one-sided upper 95% percentile and studentised bounds for the lognormal mean with $n = 25$. Verify that the one-sided errors are larger than the two-sided ones, as Theorem 11.14 predicts.

11.3.6 ★ Graduate Implement the *double* (iterated) bootstrap for calibrating the percentile interval: for each outer resample run an inner bootstrap and record whether the inner percentile interval at level $1-\alpha'$ covers $\widehat{\theta}$; choose α' so that the estimated coverage is $1-\alpha$. Apply it to the lognormal mean with $n = 25$ and discuss the computational cost.

11.3.7 Policy A think tank reports a 95% bootstrap interval of $[0.41, 0.52]$ for a regional Gini coefficient and $[0.43, 0.50]$ for the national one, and concludes that "regional inequality is not significantly different from national inequality because the intervals overlap". Explain why overlapping intervals do not imply an insignificant difference, and how you would bootstrap the difference directly (taking care that the region is part of the nation).

11.4 The Bootstrap for Regression and Dependent Data

The bootstrap of Section 11.2 assumes that the observations are iid, so that $\widehat{F}_n$ is a good estimate of the data-generating process. Regression models, clustered surveys and time series violate this in different ways, and each calls for a resampling scheme that reproduces the relevant dependence structure. The guiding rule is: *resample whatever is independent*.

11.4.1 Pairs and residual bootstrap

Consider the linear model $y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + u_i$, $i = 1, \dots, n$, with OLS estimate $\widehat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}$ and residuals $\hat{u}_i = y_i - \mathbf{x}_i^\top \widehat{\boldsymbol{\beta}}$.

Definition 11.15 (Pairs and residual bootstrap). (a) The *pairs* (case) bootstrap resamples the rows $(y_i, \mathbf{x}_i)$ with replacement and refits OLS on each resample.

(b) The *residual* bootstrap keeps $\mathbf{X}$ fixed, draws $u_1^*, \dots, u_n^*$ with replacement from the (centred) residuals, sets $y_i^* = \mathbf{x}_i^\top \widehat{\boldsymbol{\beta}} + u_i^*$ and refits.

The pairs bootstrap treats $(y_i, \mathbf{x}_i)$ as an iid draw from a joint distribution, which is the natural model for survey data; it is valid under heteroskedasticity of unknown form. The residual bootstrap treats the regressors as fixed and the errors as iid, which is the classical textbook model: it is more efficient when errors really are homoskedastic, but it imposes homoskedasticity, because every u_i^* is drawn from the same pool regardless of $\mathbf{x}_i$.

11.4.2 The wild bootstrap

The wild bootstrap keeps $\mathbf{X}$ fixed but lets each error keep its own scale.

Definition 11.16 (Wild bootstrap). Draw auxiliary variables $v_1, \dots, v_n$ iid with $\mathbb{E}v = 0$, $\mathbb{E}v^2 = 1$, independently of the data, and set $y_i^* = \mathbf{x}_i^\top \widehat{\boldsymbol{\beta}} + \hat{u}_i v_i$. The *Rademacher* choice is $v_i = \pm 1$ with probability $1/2$ each; Mammen (1993) proposed a two-point distribution with $\mathbb{E}v^3 = 1$ that also matches third moments.

Proposition 11.17 (The wild bootstrap reproduces the sandwich). *For the wild bootstrap with fixed* $\mathbf{X}$,

$$\operatorname{Var}^*(\widehat{\boldsymbol{\beta}}^*) = (\mathbf{X}^\top\mathbf{X})^{-1}\Big(\sum_{i=1}^n \hat{u}_i^2 \mathbf{x}_i\mathbf{x}_i^\top\Big)(\mathbf{X}^\top\mathbf{X})^{-1},$$

the HC0 heteroskedasticity-robust (Eicker–Huber–White) variance. For the residual bootstrap, $\operatorname{Var}^*(\widehat{\boldsymbol{\beta}}^*) = \hat{\sigma}^2(\mathbf{X}^\top\mathbf{X})^{-1}$ *with* $\hat{\sigma}^2 = \frac{1}{n}\sum \hat{u}_i^2$, *the classical homoskedastic formula.*

Proof. In the wild bootstrap, $\widehat{\boldsymbol{\beta}}^* - \widehat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{u}^*$ with $u_i^* = \hat{u}_i v_i$. Conditionally on the data the u_i^* are independent with mean zero and variance $\hat{u}_i^2$, so $\operatorname{Var}^*(\mathbf{X}^\top\mathbf{u}^*) = \sum_i \hat{u}_i^2 \mathbf{x}_i\mathbf{x}_i^\top$, and the result follows. In the residual bootstrap the u_i^* are iid with variance $\hat{\sigma}^2$ (after centring), so $\operatorname{Var}^*(\mathbf{X}^\top\mathbf{u}^*) = \hat{\sigma}^2\mathbf{X}^\top\mathbf{X}$. □

Proposition 11.17 says the wild bootstrap is the resampling counterpart of robust standard errors; its advantage over the analytic sandwich is in the *shape* of the distribution, which matters for bootstrap-t tests and intervals. The next listing compares the four standard errors with the classical one in a simulated earnings regression in which the error standard deviation rises with schooling.

Listing 11.5: Pairs, residual and wild bootstrap for regression ch11/ch11_regression_boot.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Pairs, residual and wild bootstrap for a heteroskedastic regression slope.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n, B, R = 100, 499, 1000
b0, b1 = 1.0, 0.08            # log earnings on years of schooling (simulated)


```

```
def simulate(size):
    s = rng.uniform(0, 16, size=size)                    # years of schooling
    e = rng.normal(0, 0.02 * s**1.5, size=size)          # sd rises with s
    return s, b0 + b1 * s + e


def slope(x, y):
    """OLS slope along the last axis (works for stacked resamples)."""
    xc = x - x.mean(axis=-1, keepdims=True)
    return np.sum(xc * y, axis=-1) / np.sum(xc**2, axis=-1)


def all_se(x, y):
    """Classical, HC1, pairs, residual and wild bootstrap se of the slope."""
    b = slope(x, y)
    a = y.mean() - b * x.mean()
    u = y - a - b * x
    xc = x - x.mean()
    sxx = np.sum(xc**2)
    se_ols = np.sqrt(np.sum(u**2) / (n - 2) / sxx)
    se_hc1 = np.sqrt(n / (n - 2) * np.sum(xc**2 * u**2)) / sxx
    idx = rng.integers(0, n, size=(B, n))
    se_pairs = slope(x[idx], y[idx]).std(ddof=1)
    fit = a + b * x
    se_res = slope(x, fit + u[idx]).std(ddof=1)          # iid residual draws
    v = rng.choice([-1.0, 1.0], size=(B, n))             # Rademacher weights
    se_wild = slope(x, fit + u * v).std(ddof=1)
    return b, np.array([se_ols, se_hc1, se_pairs, se_res, se_wild])


names = ["classical OLS", "HC1 sandwich", "pairs bootstrap",
         "residual bootstrap", "wild (Rademacher)"]
x, y = simulate(n)
b, se = all_se(x, y)
print(f"one sample: slope = {b:.4f}")
xs, ys = simulate((20000, n))
se_true = slope(xs, ys).std(ddof=1)
print(f"Monte Carlo (true) sd of the slope = {se_true:.5f}")

# Coverage of b +/- 1.96 se over R simulated samples
hits, ses = np.zeros(5), np.zeros(5)
for r in range(R):
    x, y = simulate(n)
    b, se = all_se(x, y)
    hits += np.abs(b - b1) <= 1.96 * se
    ses += se
print(f"{'standard error':<20}{'mean se':>10}{'ratio to true':>15}"
      f"{'95% coverage':>14}")
for k in range(5):
    print(f"{names[k]:<20}{ses[k] / R:10.5f}{ses[k] / R / se_true:15.3f}"
          f"{hits[k] / R:14.3f}")
```

Output

```
one sample: slope = 0.0913
Monte Carlo (true) sd of the slope = 0.01652
```

```
standard error          mean se  ratio to true  95% coverage
classical OLS           0.01379          0.835         0.906
HC1 sandwich            0.01615          0.978         0.944
pairs bootstrap         0.01608          0.973         0.945
residual bootstrap      0.01363          0.825         0.905
wild (Rademacher)       0.01599          0.968         0.943
```

As Proposition 11.17 predicts, the residual bootstrap reproduces the classical standard error and inherits its downward bias (about 17% too small), so that its nominal 95% interval covers only 90% of the time. The pairs bootstrap, the wild bootstrap and the HC1 sandwich are all close to the truth and give close to nominal coverage.

11.4.3 Clustered data: the cluster bootstrap

Household surveys are rarely simple random samples. A typical design first selects *enumeration areas* (EAs, villages or urban blocks) and then a fixed number of households within each selected EA. Households in the same EA share prices, infrastructure, weather and local institutions, so their outcomes are correlated. If a programme is also assigned at the EA level (e.g. a community-driven development project), the regressor of interest is constant within clusters.

Suppose G clusters of m households each, a regressor that is constant within clusters and an error with intra-cluster correlation ρ. Then the true variance of the OLS coefficient is approximately

$$\operatorname{Var}(\hat{\beta}) \approx \operatorname{Var}_{\text{iid}}(\hat{\beta}) \times \big\{1 + (m-1)\rho\big\}, \tag{11.7}$$

the *Moulton factor* (Moulton, 1990), known to survey statisticians as the *design effect*. With $m = 15$ and a modest $\rho = 0.15$, the factor is 3.1: naive standard errors are too small by a factor of $\sqrt{3.1} \approx 1.8$.

Definition 11.18 (Pairs cluster bootstrap)**.** Resample G clusters with replacement from the G observed clusters, keep all observations of each selected cluster together, and recompute the estimator. More generally, for stratified multistage designs, resample primary sampling units independently within each stratum.

Listing 11.6: Cluster bootstrap for a survey clustered by EA ch11/ch11_cluster_boot.py

```python
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Cluster bootstrap for a simulated survey clustered by enumeration area.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
G, m, B = 40, 15, 1999          # enumeration areas, households per EA
tau, s_ea, s_hh = 0.10, 0.25, 0.50


def survey():
    """Log consumption; a programme is assigned to half the EAs."""
    d = np.repeat(rng.permutation(np.r_[np.ones(G // 2), np.zeros(G // 2)]), m)
    ea = np.repeat(rng.normal(0, s_ea, G), m)            # shared EA shock
    return d, 5.0 + tau * d + ea + rng.normal(0, s_hh, G * m)


def effect(d, y):
```

```python
    """Difference in means = OLS coefficient on d (last axis = households)."""
    return (np.sum(d * y, -1) / np.sum(d, -1)
            - np.sum((1 - d) * y, -1) / np.sum(1 - d, -1))


d, y = survey()
n = G * m
b = effect(d, y)
u = y - np.where(d == 1, y[d == 1].mean(), y[d == 0].mean())
# naive iid standard error (two-sample formula)
se_iid = np.sqrt(y[d == 1].var(ddof=1) / np.sum(d)
                 + y[d == 0].var(ddof=1) / np.sum(1 - d))
# cluster-robust (CR1) sandwich: sum scores within each EA
X = np.column_stack([np.ones(n), d])
XtXi = np.linalg.inv(X.T @ X)
scores = (X * u[:, None]).reshape(G, m, 2).sum(axis=1)
V = XtXi @ (scores.T @ scores) @ XtXi * G / (G - 1) * (n - 1) / (n - 2)
se_cr1 = np.sqrt(V[1, 1])
# pairs cluster bootstrap: resample whole EAs with replacement
gidx = rng.integers(0, G, size=(B, G))
rows = (gidx[:, :, None] * m + np.arange(m)).reshape(B, n)
with np.errstate(invalid="ignore", divide="ignore"):
    bb = effect(d[rows], y[rows])
ok = np.isfinite(bb)              # drop rare draws with no treated EA
se_cboot = bb[ok].std(ddof=1)
idx = rng.integers(0, n, size=(B, n))   # ignores clustering: wrong
se_hhboot = effect(d[idx], y[idx]).std(ddof=1)

# intra-cluster correlation (ANOVA estimator) and Monte Carlo truth
ybar_g = u.reshape(G, m).mean(1)
msb = m * np.sum(ybar_g**2) / (G - 2)
msw = np.sum((u.reshape(G, m) - ybar_g[:, None])**2) / (G * (m - 1))
icc = (msb - msw) / (msb + (m - 1) * msw)
sims = np.array([effect(*survey()) for _ in range(4000)])
print(f"G = {G} EAs x m = {m} households; estimated effect = {b:.4f}")
print(f"estimated ICC = {icc:.3f}; Moulton factor 1+(m-1)ICC = "
      f"{1 + (m - 1) * icc:.2f}")
print(f"{'standard error':<30}{'value':>8}")
for k, v in [("naive iid formula", se_iid),
             ("household bootstrap (wrong)", se_hhboot),
             ("cluster-robust CR1", se_cr1),
             ("cluster (EA) bootstrap", se_cboot),
             ("Monte Carlo truth", sims.std(ddof=1))]:
    print(f"{k:<30}{v:8.4f}")
print(f"cluster draws dropped (no treated EA): {np.sum(~ok)}")
```

Output

```
G = 40 EAs x m = 15 households; estimated effect = 0.1402
estimated ICC = 0.163; Moulton factor 1+(m-1)ICC = 3.28
standard error                   value
naive iid formula               0.0449
household bootstrap (wrong)     0.0446
cluster-robust CR1              0.0806
cluster (EA) bootstrap          0.0827
Monte Carlo truth               0.0873
```

Algorithm 7 Wild cluster restricted (WCR) bootstrap test of $H_0 : \beta_k = 0$

1: Estimate the full model; compute the CR1 t-statistic $t = \hat{\beta}_k / \mathrm{se}_{\mathrm{CR}}$
2: Re-estimate imposing $\beta_k = 0$; keep restricted fitted values $\tilde{y}_i$ and residuals $\tilde{u}_i$
3: **for** $b = 1, \ldots, B$ **do**
4: draw Rademacher weights $w_1, \ldots, w_G$, one per cluster
5: $y^*_{ig} \leftarrow \tilde{y}_{ig} + w_g \tilde{u}_{ig}$ for every observation i in cluster g
6: re-estimate the full model on $(y^*, \mathbf{X})$ and compute t^*_b with CR1 standard errors
7: **end for**
8: **return** $p = \frac{1}{B} \#\{|t^*_b| \geq |t|\}$

```
cluster draws dropped (no treated EA): 0
```

The household-level bootstrap, which treats the 600 households as independent, reproduces the naive iid standard error and is wrong by a factor of almost two. The cluster bootstrap and the cluster-robust (CR1) sandwich, both of which treat the 40 EAs as the independent units, are close to the Monte Carlo truth.

⋆ Key Concept: Resample the independent units

The bootstrap is valid only if the resampled units are (approximately) independent draws from a common distribution. In a clustered survey these are the clusters (primary sampling units), not the households; in a panel they are the individuals or firms, not the person-years; in a time series they are blocks of consecutive observations, not single periods. Resampling at too fine a level reproduces exactly the error that cluster-robust inference is designed to avoid.

11.4.4 Few clusters: the wild cluster bootstrap

Cluster-robust standard errors are justified as $G \to \infty$. With few clusters, or few *treated* clusters, the sandwich is biased downward and very variable, and t-tests over-reject (Cameron et al., 2008). Resampling clusters does not help much either: with $G = 8$, many cluster resamples contain only one or two treated clusters, or none at all. Cameron et al. (2008) recommended the *wild cluster bootstrap*, which applies the wild weights at the cluster level, and showed that imposing the null hypothesis on the bootstrap data-generating process (the "restricted" version, WCR) and bootstrapping the t-statistic gives much more accurate tests.

The following simulation draws data under the null with unbalanced EA sizes, heavy-tailed cluster shocks whose variance differs between treated and control EAs, and a programme in a third of the EAs, and compares the rejection rates of three 5% tests.

Wild cluster bootstrap with few clusters: output of `ch11_wild_cluster.py` (the full program is in `code/ch11`).

Output

```
Rejection rates of a true null at the 5% level (400 reps, B = 399)
   G  CR1 + N(0,1)  CR1 + t(G-1)  wild cluster
   5         0.618         0.522         0.080
   8         0.295         0.200         0.085
  12         0.158         0.105         0.065
  20         0.113         0.095         0.072
  40         0.070         0.065         0.052
```

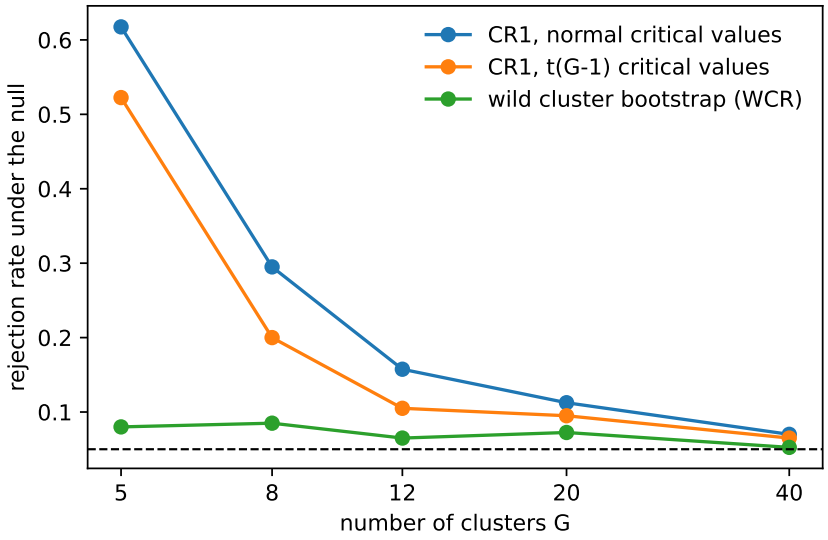


Figure 11.6: Rejection rates of a true null hypothesis at the 5% level as a function of the number of clusters. CR1 t-tests over-reject badly with few clusters, even with $t(G-1)$ critical values; the wild cluster restricted bootstrap stays much closer to 5%.

With $G = 5$ (one treated cluster) the conventional test rejects a true null more than half the time; even at $G = 20$ it rejects 10% of the time. The wild cluster bootstrap is far more reliable, although not perfect with the fewest clusters. Note that with Rademacher weights and G clusters there are only 2^G distinct bootstrap samples (32 when $G = 5$), so the bootstrap p-value is coarse; MacKinnon and Webb (2017) analyse the case of very few treated clusters, where even the wild cluster bootstrap can fail.

Policy Lens: Clustered designs in programme evaluation

Most development RCTs randomise at the level of the village, school or clinic, and most national surveys are clustered by enumeration area. Two practical implications follow. First, the *effective* sample size is closer to the number of clusters than to the number of households: a trial with 20 villages and 1000 households per village is a small trial. Power calculations must use the design effect (11.7). Second, inference with fewer than about 30–50 clusters should not rely on conventional cluster-robust t-tests; report wild cluster bootstrap or randomisation-inference p-values (Section 11.5) as well. Reviewers of evaluations in education, health and agriculture now routinely ask for both.

11.4.5 Time series: the moving block bootstrap

For a stationary time series $X_1, \dots, X_T$, iid resampling destroys the serial dependence and, for positively autocorrelated series, underestimates the variance of the mean, whose correct asymptotic form involves the *long-run variance*

$$T \operatorname{Var}(\bar{X}) \to \sigma^2_{\mathrm{LR}} = \sum_{h=-\infty}^{\infty} \gamma(h), \qquad \text{for AR(1): } \sigma^2_{\mathrm{LR}} = \frac{\sigma^2_\varepsilon}{(1-\phi)^2}.$$

Definition 11.19 (Moving block bootstrap)**.** Fix a block length ℓ. Form the $T - \ell + 1$ overlapping blocks $\mathcal{B}_j = (X_j, \dots, X_{j+\ell-1})$. Draw $k = \lceil T/\ell \rceil$ blocks with replacement, concatenate them and truncate to length T.

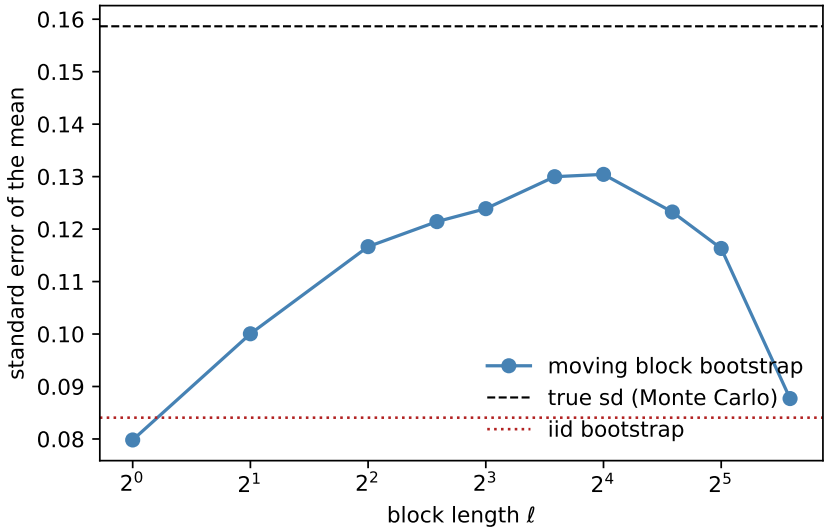


Figure 11.7: Moving block bootstrap standard error of the mean of a simulated AR(1) series ($\phi = 0.6$, $T = 240$) as a function of block length. Short blocks reproduce the iid bootstrap; long blocks leave too few distinct blocks.

Within each block the original dependence is preserved; between blocks it is broken. If $\ell \to \infty$ and $\ell/T \to 0$, the moving block bootstrap (MBB) is consistent for the distribution of $\sqrt{T}(\bar{X} - \mu)$ under weak dependence (Künsch, 1989). For variance estimation the mean-squared-error optimal block length grows like $T^{1/3}$.

Moving block bootstrap for an AR(1) mean: output of `ch11_block_boot.py` (the full program is in `code/ch11`).

Output

```
T = 240, phi = 0.6: sample mean = -0.027
true sd of mean (MC) = 0.1587; asymptotic = 0.1614
iid bootstrap se = 0.0841  (ignores dependence)
block length   MBB se
           1   0.0798
           2   0.1000
           4   0.1167
           6   0.1214
           8   0.1239
          12   0.1300
          16   0.1304
          24   0.1233
          32   0.1163
          48   0.0877
rule of thumb ell ~ T^(1/3) = 6.2
```

The iid bootstrap standard error (0.084) is about half of the truth (0.159). As ℓ increases, the MBB standard error rises, peaks for blocks of 12–16 months, and then falls again as the number of blocks becomes too small to estimate a variance reliably. Even the best block length underestimates the truth in this sample, which illustrates that block bootstraps have a downward bias of order ℓ/T and that choosing ℓ is genuinely difficult.

△ Caution: Block length and stationarity

The MBB requires stationarity and a block length that captures most of the dependence. Results can be sensitive to ℓ; report several choices or use data-driven rules. Variants include the *stationary bootstrap* with random geometric block lengths (Politis and Romano, 1994), the *circular* block bootstrap, and model-based (sieve) bootstraps that fit an autoregression and resample its residuals, as in the fan chart of Section 11.6.

Historical Note: The wild and the block bootstrap

The wild bootstrap was proposed by Wu (1986) in a paper on resampling methods for heteroskedastic regression and developed by Liu (1988); the name and the asymmetric two-point weights are due to Mammen (1993). Künsch (1989) introduced the moving block bootstrap and the related blockwise jackknife for stationary observations. In econometrics, Cameron et al. (2008) made the wild cluster bootstrap a standard tool for inference with few clusters.

Exercises

11.4.1 Theory Complete the proof of Proposition 11.17 for the residual bootstrap, showing that if the residuals are not centred (possible when there is no intercept) the bootstrap estimator is biased: $\mathbb{E}^*\widehat{\boldsymbol{\beta}}^* \neq \widehat{\boldsymbol{\beta}}$.

11.4.2 Theory Derive the Moulton factor (11.7) for the difference in means between treated and control clusters, assuming equal cluster sizes m and an equicorrelated error with variance σ^2 and intra-cluster correlation ρ.

11.4.3 Theory Show that with Rademacher weights the wild cluster bootstrap with G clusters has at most 2^G distinct bootstrap samples, and that w and $-w$ give the same $|t^*|$ in the restricted bootstrap. What is the smallest attainable two-sided p-value when $G = 6$?

11.4.4 Python Add the *wild* bootstrap-t (studentising each $\hat{\beta}^*$ with its own HC1 standard error) to `ch11_regression_boot.py` and compare the coverage of its interval with that of the HC1 normal interval for $n = 30$.

11.4.5 Python Modify `ch11_cluster_boot.py` to vary the intra-cluster correlation from 0 to 0.4 and plot the ratio of the naive to the cluster bootstrap standard error against $\sqrt{1 + (m-1)\rho}$.

11.4.6 Python Implement the stationary bootstrap of Politis and Romano (1994) (blocks of geometric length with mean ℓ, wrapping around the end of the series) and compare it with the MBB in `ch11_block_boot.py` over 200 independent series.

11.4.7 ★ Graduate Show that for the sample mean the MBB variance estimator, ignoring end effects, equals a Bartlett-kernel (Newey–West type) estimator of the long-run variance with bandwidth ℓ, divided by T.

11.4.8 Policy An evaluation of a teacher-training programme in 14 districts (7 treated) reports a cluster-robust t-statistic of 2.3. Explain to the programme's funders why this may overstate the evidence, and what additional analysis you would request before scaling up the programme nationally.

11.5 Permutation and Randomisation Tests

The bootstrap approximates sampling distributions by resampling *with* replacement. Permutation tests rearrange the data *without* replacement. When the rearrangement mirrors how the data

were actually generated, as in a randomised experiment, the resulting test is *exact*: its size is controlled in finite samples, with no appeal to large-sample theory or distributional assumptions.

11.5.1 Randomisation inference in an experiment

Consider an experiment with N units, of which N_1 are assigned to treatment by complete randomisation: the assignment vector $\mathbf{z} = (z_1, \dots, z_N) \in \{0,1\}^N$ is drawn uniformly from the set Ω of all $\binom{N}{N_1}$ vectors with N_1 ones. Each unit has two *potential outcomes*, $y_i(1)$ if treated and $y_i(0)$ if not, and we observe $y_i = z_i y_i(1) + (1 - z_i) y_i(0)$.

Definition 11.20 (Fisher's sharp null hypothesis)**.** The *sharp null* of no effect states $H_0 : \; y_i(1) = y_i(0)$ for every unit i. More generally, $H_0^{\tau} : \; y_i(1) = y_i(0) + \tau_i$ for a fully specified vector of effects, e.g. a constant effect $\tau_i \equiv \tau_0$.

A sharp null specifies *both* potential outcomes for every unit: under H_0 the observed $\mathbf{y}$ would have been observed whatever the assignment. Hence for any test statistic $T(\mathbf{z}, \mathbf{y})$, such as the difference in means $T = \bar{y}_{\text{treated}} - \bar{y}_{\text{control}}$, we can compute the value it would have taken under every possible assignment $\boldsymbol{\omega} \in \Omega$. The randomness in the test comes only from the randomisation, which the experimenter controls.

Definition 11.21 (Randomisation p-value)**.** Under a sharp null, the (one-sided) randomisation p-value is

$$p(\mathbf{z}) = \frac{1}{|\Omega|} \#\big\{\boldsymbol{\omega} \in \Omega : \; T(\boldsymbol{\omega}, \mathbf{y}) \geq T(\mathbf{z}, \mathbf{y})\big\};$$

for a two-sided test replace T by $|T|$.

Theorem 11.22 (Exactness of randomisation tests)**.** *If $\mathbf{z}$ is uniform on Ω and the sharp null holds, then $\mathbb{P}(p(\mathbf{z}) \leq \alpha) \leq \alpha$ for every $\alpha \in [0,1]$.*

Proof. Under H_0, $\mathbf{y}$ does not depend on $\mathbf{z}$, so $t(\boldsymbol{\omega}) = T(\boldsymbol{\omega}, \mathbf{y})$ is a fixed function on Ω and $p(\boldsymbol{\omega}) = |\Omega|^{-1} \#\{\boldsymbol{\omega}' : t(\boldsymbol{\omega}') \geq t(\boldsymbol{\omega})\}$ is the upper-tail proportion of $t(\boldsymbol{\omega})$ among all values. Let $A = \{\boldsymbol{\omega} : p(\boldsymbol{\omega}) \leq \alpha\}$. If A is non-empty, let $\boldsymbol{\omega}_0 \in A$ have the smallest value of t in A; every element of A has $t \geq t(\boldsymbol{\omega}_0)$, so $|A| \leq \#\{\boldsymbol{\omega}' : t(\boldsymbol{\omega}') \geq t(\boldsymbol{\omega}_0)\} = |\Omega| \; p(\boldsymbol{\omega}_0) \leq \alpha \, |\Omega|$. Since $\mathbf{z}$ is uniform on Ω, $\mathbb{P}(p(\mathbf{z}) \leq \alpha) = |A| \, / |\Omega| \leq \alpha$. □

Nothing was assumed about the distribution of the outcomes, about independence between units, or about the sample size: the proof uses only the known randomisation. When $|\Omega|$ is too large to enumerate, we draw M assignments at random. Since the observed assignment and the M random ones are exchangeable under H_0, the Monte Carlo p-value

$$\hat{p} = \frac{1 + \#\{m : \; T(\boldsymbol{\omega}_m, \mathbf{y}) \geq T(\mathbf{z}, \mathbf{y})\}}{M + 1} \tag{11.8}$$

is also exactly valid, for any M. (Without the "+1" it is not, and it can be zero.)

Example 11.23 (A school-feeding trial)**.** A pilot of a school-feeding programme randomly assigns 6 of 12 schools to provide a daily meal, and records the attendance rate of each school during the following term. With $N = 12$ the $\binom{12}{6} = 924$ possible assignments can be enumerated exactly. In a larger follow-up, 160 pupils are randomised individually and their test-score gains recorded; there we use Monte Carlo draws and `scipy.stats.permutation_test`. All data are simulated.

Listing 11.7: Randomisation inference for a simulated school-feeding trial (plotting code omitted)
ch11/ch11_randomisation_rct.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Fisher randomisation inference for a simulated school-feeding trial.
# (c) 2026 Stephen E. Moore. MIT Licence.
from itertools import combinations
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

rng = np.random.default_rng(803)

# (a) Pilot: 12 schools, 6 randomly assigned to school feeding.
N, N1 = 12, 6
y0 = rng.normal(80, 6, N)                       # attendance rate (%) if control
z = rng.permutation(np.r_[np.ones(N1), np.zeros(N - N1)])
y = y0 + 5.0 * z                                # true effect: +5 points
t_obs = y[z == 1].mean() - y[z == 0].mean()
# Exact randomisation distribution under the sharp null y_i(1) = y_i(0)
assign = np.array([np.isin(np.arange(N), c) for c in combinations(range(N), N1)])
t_all = (assign * y).sum(1) / N1 - (~assign * y).sum(1) / (N - N1)
p_exact = np.mean(np.abs(t_all) >= abs(t_obs) - 1e-12)
p_welch = stats.ttest_ind(y[z == 1], y[z == 0], equal_var=False).pvalue
print("(a) pilot with 12 schools, 6 treated")
print(f"  observed difference = {t_obs:.3f} points; "
      f"{len(t_all)} possible assignments")
print(f"  exact randomisation p = {p_exact:.4f}; Welch t-test p = {p_welch:.4f}")
print(f"  smallest attainable p-value = {2 / len(t_all):.4f}")

# (b) Pupil-level trial: 160 pupils, skewed test-score gains, effect +3
n = 160
d = rng.permutation(np.r_[np.ones(n // 2), np.zeros(n // 2)]).astype(bool)
gain = rng.gamma(2.0, 6.0, n) + 3.0 * d
diff = lambda a, b, axis=-1: a.mean(axis) - b.mean(axis)
res = stats.permutation_test((gain[d], gain[~d]), diff, vectorized=True,
                             n_resamples=9999, random_state=rng)
p_t = stats.ttest_ind(gain[d], gain[~d], equal_var=False).pvalue
print("(b) pupil-level trial, n = 160")
print(f"  difference in means = {res.statistic:.3f}")
print(f"  Monte Carlo randomisation p (9999 draws) = {res.pvalue:.4f}; "
      f"Welch p = {p_t:.4f}")

# Fisher interval: invert sharp nulls y_i(1) - y_i(0) = tau0 over a grid
perm = np.array([rng.permutation(n) for _ in range(4000)])
taus, keep = np.round(np.arange(-2, 9.01, 0.05), 2), []
for tau0 in taus:
    y0_imp = gain - tau0 * d                    # imputed control outcomes
    dp = d[perm]
    tp = (y0_imp * dp).sum(1) / dp.sum(1) - (y0_imp * ~dp).sum(1) / (~dp).sum(1)
    t0 = y0_imp[d].mean() - y0_imp[~d].mean()
    keep.append(np.mean(np.abs(tp) >= abs(t0)) > 0.05)
acc = taus[np.array(keep)]
print(f"  95% Fisher interval for a constant effect: [{acc.min():.2f}, "
      f"{acc.max():.2f}]")
```

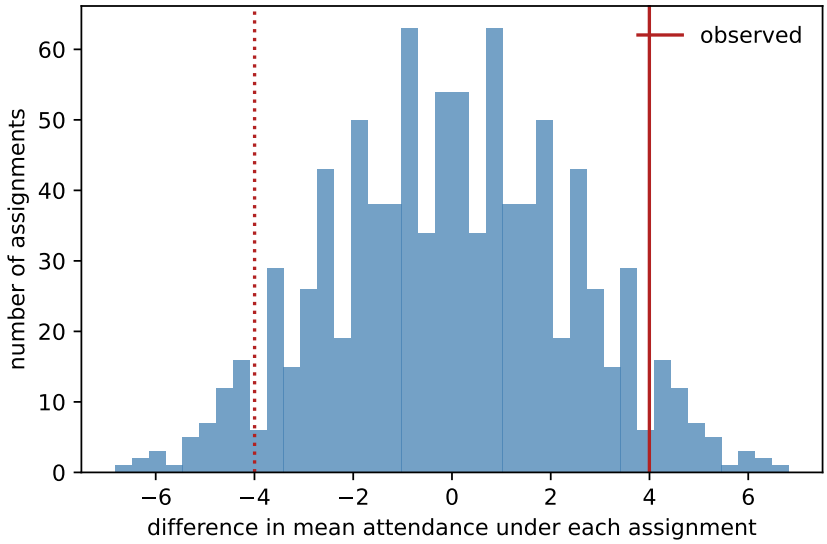


Figure 11.8: The exact randomisation distribution of the difference in mean attendance over all 924 ways of assigning 6 of 12 schools to treatment, under the sharp null of no effect. The p-value is the proportion of assignments at least as extreme as the observed one (solid line) in absolute value.

Output

```
(a) pilot with 12 schools, 6 treated
  observed difference = 3.998 points; 924 possible assignments
  exact randomisation p = 0.1061; Welch t-test p = 0.1136
  smallest attainable p-value = 0.0022
(b) pupil-level trial, n = 160
  difference in means = 5.043
  Monte Carlo randomisation p (9999 draws) = 0.0010; Welch p = 0.0009
  95% Fisher interval for a constant effect: [2.15, 8.00]
```

In the pilot, an observed difference of 4.0 percentage points is not significant ($p = 0.106$), even though the true effect in the simulation is 5 points: twelve schools give a test with little power. The Welch t-test gives a similar p-value here, but its justification is asymptotic, whereas the randomisation p-value is exact. In the pupil-level trial the effect is clearly detected.

11.5.2 Confidence sets by test inversion

A randomisation test of $H_0^{\tau_0}$: "the effect is τ_0 for every unit" is carried out by imputing $y_i(0) = y_i - \tau_0 z_i$ for all units, which makes the null sharp again, and testing for no effect on the imputed outcomes. The set of τ_0 not rejected at level α,

$$C_{1-\alpha} = \{\tau_0 : \; p(\tau_0) > \alpha\},$$

is a $1-\alpha$ confidence set for a constant additive effect (a *Fisher interval*), again exact in finite samples. For the pupil-level trial it is $[2.15, 8.00]$ test-score points.

⋆ Key Concept: Randomisation inference

In a randomised experiment, the randomisation itself is the basis for inference. Under a sharp null every potential outcome is known, so the exact null distribution of any statistic

can be computed by re-randomising. The resulting tests are exact for any sample size and any outcome distribution, permit clustered and stratified designs (re-randomise exactly as the design did), and can be inverted into confidence intervals. The price is that the null is sharp: it concerns every unit, not just the average effect.

△ Caution: Re-randomise as you randomised

The reference set Ω must be the set of assignments the actual design could have produced. If the trial was stratified by district, permute within districts; if it was randomised by school, permute school-level assignments, keeping pupils in the same school together. A pupil-level permutation of a school-level trial is the randomisation-inference version of ignoring clustering, and it produces p-values that are far too small.

11.5.3 Permutation tests without an experiment

Permutation tests are also used outside experiments, where their justification is *exchangeability* under the null rather than physical randomisation.

- *Two samples.* If $X_1, \dots, X_{n_1} \overset{\text{iid}}{\sim} F$ and $Y_1, \dots, Y_{n_0} \overset{\text{iid}}{\sim} G$, under $H_0 : F = G$ the pooled sample is exchangeable, so all $\binom{n_1+n_0}{n_1}$ splits are equally likely and the two-sample permutation test is exact. It tests equality of distributions; it is only approximately valid for the weaker hypothesis of equal means when variances differ, unless a studentised statistic is used.
- *Independence.* For pairs (X_i, Y_i), under H_0: X and Y are independent, the pairing is arbitrary, so permuting the Y's against fixed X's generates the null distribution of any measure of association, e.g. the correlation.

Permutation test for a correlation: output of `ch11_permutation_corr.py` (the full program is in `code/ch11`).

Output

```
n = 24 districts; Pearson r = 0.2609, Spearman rho = 0.0539
permutation p (by hand, M = 20000)     = 0.2128
permutation p (scipy, pairings)      = 0.2418
normal-theory t-test p (pearsonr)    = 0.2182
Spearman rank-correlation p          = 0.8024
sd of permutation r = 0.2080; 1/sqrt(n-1) = 0.2085
```

In SciPy, `permutation_type="independent"` permutes observations between samples (the two-sample test), `"pairings"` permutes the pairing of observations (association tests), and `"samples"` swaps observations within pairs (paired designs). For this sample the hand-coded permutation p-value and the normal-theory p-value for Pearson's r are close. SciPy's value is a little larger because, for `alternative="two-sided"`, it doubles the smaller one-sided p-value instead of counting $|r^*| \geq |r|$; the two definitions differ when the permutation distribution is asymmetric. The standard deviation of the permutation distribution of r is almost exactly $1/\sqrt{n-1}$, its theoretical value under independence. The rank-based Spearman correlation is close to zero: the Pearson correlation is driven by a few districts with very high spending, a warning that a permutation test controls size but does not make a statistic robust.

11.5.4 Power

Exactness concerns size; we also want power. Power depends on the test statistic, the design and the outcome distribution, and is most easily studied by simulation.

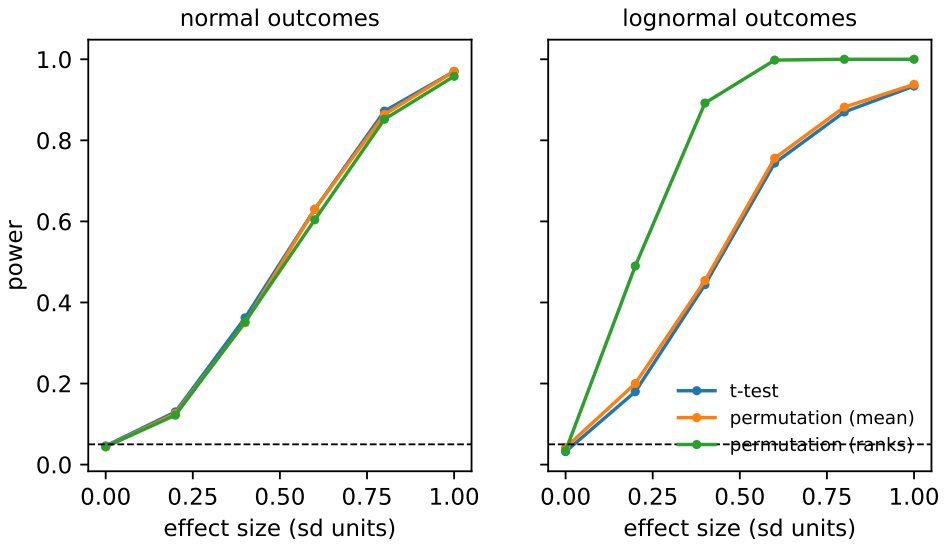


Figure 11.9: Power curves of three 5% tests for a two-arm trial with 30 units per arm. With normal outcomes the three tests are equivalent; with skewed (lognormal) outcomes, such as consumption, the permutation test on ranks is far more powerful.

Power of permutation tests for a simulated cash-transfer trial: output of `ch11_power_sim.py` (the full program is in `code/ch11`).

Output

```
Rejection rates at 5% (n = 30+30, 500 reps, 499 permutations)
normal outcomes:  effect   t-test  perm(mean)  perm(rank)
                     0.0    0.046       0.044       0.044
                     0.2    0.130       0.126       0.122
                     0.4    0.362       0.350       0.352
                     0.6    0.630       0.630       0.604
                     0.8    0.872       0.864       0.852
                     1.0    0.970       0.970       0.958
lognormal outcomes:  effect   t-test  perm(mean)  perm(rank)
                     0.0    0.032       0.042       0.034
                     0.2    0.180       0.200       0.490
                     0.4    0.444       0.454       0.892
                     0.6    0.744       0.756       0.998
                     0.8    0.870       0.882       1.000
                     1.0    0.934       0.938       1.000
```

With normal outcomes, the t-test and the permutation test on the difference in means have essentially the same power: little is lost by using the exact test. With skewed outcomes the permutation test of means still holds its size, while the t-test is somewhat conservative; and a permutation test on *ranks* (equivalent to the Wilcoxon–Mann–Whitney test) is far more powerful, because it is not dominated by a few large values. The choice of statistic, not the choice between permutation and t reference distributions, is what matters for power.

Policy Lens: Randomisation inference in development RCTs

Randomisation inference has become a standard robustness check for field experiments. Young (2019) re-analysed a large set of published experiments in economics with randomisa-

tion tests and found that a substantial share of results reported as significant on the basis of conventional cluster-robust t-tests were not significant under randomisation inference, mainly because of few clusters, high-leverage observations and multiple treatments. For evaluations of cash transfers, school-feeding programmes or health interventions, pre-registering the randomisation-inference procedure (statistic, permutation scheme, number of draws) alongside the primary analysis protects against both false positives and specification search. Imbens and Rubin (2015) give a thorough textbook treatment.

11.5.5 Multiple testing

An evaluation rarely has a single outcome. A cash-transfer programme may be evaluated on consumption, food security, school enrolment, child health, labour supply, assets and indicators of empowerment. With m independent tests at level α and no true effects, the probability of at least one false positive is $1-(1-\alpha)^m$, which is 0.64 for $m=20$ and $\alpha=0.05$.

Definition 11.24 (Error rates for multiple testing)**.** Of m hypotheses, m_0 are true. A procedure makes R rejections, of which V are false. The *family-wise error rate* is FWER $=\mathbb{P}(V\geq 1)$; the *false discovery rate* is FDR $=\mathbb{E}\big[V/\max(R,1)\big]$, the expected proportion of false rejections among all rejections.

Let $p_{(1)}\leq\cdots\leq p_{(m)}$ be the ordered p-values with hypotheses $H_{(1)},\dots,H_{(m)}$.

- **Bonferroni**: reject H_i if $p_i\leq\alpha/m$.
- **Holm** (Holm, 1979): find the first j with $p_{(j)}>\alpha/(m-j+1)$ and reject $H_{(1)},\dots,H_{(j-1)}$.
- **Benjamini–Hochberg** (Benjamini and Hochberg, 1995): let $k=\max\{j:\ p_{(j)}\leq jq/m\}$ and reject $H_{(1)},\dots,H_{(k)}$.

Theorem 11.25 (Error control)**.** *(a) Bonferroni and Holm control the FWER at level α under arbitrary dependence among the p-values.*

(b) If the p-values of true nulls are independent of each other and of the false-null p-values, and uniform on $[0,1]$, the Benjamini–Hochberg procedure has FDR $=\frac{m_0}{m}q\leq q$.

Proof. (a) Bonferroni: by the union bound, $\mathbb{P}(V\geq 1)\leq\sum_{i\text{ true}}\mathbb{P}(p_i\leq\alpha/m)\leq m_0\alpha/m\leq\alpha$. Holm: suppose Holm makes a false rejection, and let $H_{(j)}$ be the first true null rejected. The $j-1$ hypotheses rejected before it are all false, so $j-1\leq m-m_0$, i.e. $m-j+1\geq m_0$, and $p_{(j)}\leq\alpha/(m-j+1)\leq\alpha/m_0$. Hence some true null has $p_i\leq\alpha/m_0$, an event of probability at most $m_0\cdot\alpha/m_0=\alpha$ by the union bound.

(b) (Sketch.) Write $V=\sum_{i\in\mathcal{H}_0}\mathbf{1}\{H_i\text{ rejected}\}$. One shows that $\mathbb{E}\big[\mathbf{1}\{H_i\text{ rejected}\}/\max(R,1)\big]=\sum_{k=1}^{m}\frac{1}{k}\mathbb{P}(p_i\leq kq/m,\ R=k)$, and, by conditioning on the other p-values (which fixes the number of rejections $R_k^{(i)}$ that would result if $p_i\leq kq/m$) and using the uniformity and independence of p_i, that this sum equals q/m for every true null. Summing over the m_0 true nulls gives FDR $=m_0q/m$. See Benjamini and Hochberg (1995) for the original proof. □

FWER and FDR for 20 outcomes of a simulated trial: output of `ch11_multiple_testing.py` (the full program is in `code/ch11`).

Output

```
m = 20 outcomes (5 truly affected), 4000 simulated trials
procedure             FWER    FDR   power  avg rej
no correction        0.533  0.137   0.807     4.79
```

```
Bonferroni            0.038  0.015   0.413     2.11
Holm                  0.043  0.016   0.423     2.16
Benjamini-Hochberg    0.139  0.036   0.562     2.97
trial 1, five smallest p-values: 0.0042 0.0063 0.0065 0.0207 0.0485
Bonferroni cut-off q/m = 0.0025
BH cut-offs k*q/m, k = 1..5:      0.0025 0.0050 0.0075 0.0100 0.0125
```

Without correction, more than half of the simulated trials report at least one spurious effect. Bonferroni and Holm hold the FWER below 5% (Holm, being uniformly more powerful, rejects slightly more); Benjamini–Hochberg allows some false positives (FWER 0.14) but keeps their *proportion* at $0.036 \approx \frac{15}{20} \times 0.05$, exactly as Theorem 11.25 predicts, while detecting a third more true effects than Holm.

Graduate Perspective: Resampling-based multiple testing

Bonferroni and Holm ignore the correlation between outcomes, which in an evaluation is usually substantial (consumption and food security move together). Resampling methods exploit it: the Westfall–Young step-down procedure computes the distribution of the *minimum* p-value across outcomes by re-randomising the treatment vector *jointly* for all outcomes, giving exact FWER control that is less conservative when outcomes are correlated. An alternative is to pre-specify a small number of outcome families and aggregate within each family into a standardised index (Anderson, 2008).

Historical Note: The lady tasting tea and the permutation test

In *The Design of Experiments* Fisher (1935) described a lady who claimed to tell whether milk or tea had been poured into the cup first. Eight cups, four of each kind, are presented in random order; there are $\binom{8}{4} = 70$ ways to choose four, so identifying all four correctly by chance has probability $1/70$. The example contains the whole logic of randomisation inference: the null distribution comes from the physical act of randomisation. Pitman (1937) developed permutation tests for samples from arbitrary populations. The Bonferroni inequality is named after Carlo Emilio Bonferroni (Bonferroni, 1936); Sture Holm's sequentially rejective version appeared in 1979, and Yoav Benjamini and Yosef Hochberg introduced the false discovery rate in 1995.

Exercises

11.5.1 Theory Show that when $N_1 = N_0$, the randomisation distribution of the difference in means under the sharp null is symmetric about zero. Deduce that the smallest attainable two-sided p-value is $2/\binom{N}{N_1}$.

11.5.2 Theory Prove that the Monte Carlo p-value (11.8) satisfies $\mathbb{P}(\hat{p} \le \alpha) \le \alpha$ under the null. (Hint: the observed statistic and the M simulated ones are exchangeable, so the rank of the observed one is uniform.)

11.5.3 Theory Show that Holm's procedure rejects every hypothesis rejected by Bonferroni, and give an example with $m = 3$ where it rejects strictly more.

11.5.4 Python Redo the pilot of `ch11_randomisation_rct.py` as a *stratified* design: the 12 schools are in 3 districts of 4 schools each and 2 schools per district are treated. Enumerate the $\binom{4}{2}^3 = 216$ assignments, compute the exact p-value, and compare with an (incorrect) unstratified permutation test.

11.5.5 Python Compute, for the pupil-level trial, the randomisation-based 95% Fisher interval using the rank statistic instead of the difference in means, and compare its length with the

interval in the text.

11.5.6 Python Repeat `ch11_multiple_testing.py` with correlated outcomes (equicorrelation 0.5 among the 20 outcomes). Do Bonferroni, Holm and Benjamini–Hochberg still control their error rates? Which becomes most conservative?

11.5.7 ★ Graduate The permutation test of equal means is not exact when the two populations have different variances. Show by simulation (unequal variances and unequal sample sizes) that the unstudentised permutation test can have size far from α, and that permuting the Welch t-statistic restores approximately correct size.

11.5.8 Policy A cash-transfer evaluation reports effects on 24 outcomes, of which 4 are significant at 5% without correction. Write a short note for the ministry explaining what can and cannot be concluded, which correction you would apply, and how a pre-analysis plan would have helped.

11.6 The Jackknife, Cross-Validation and Predictive Uncertainty

The final group of resampling methods works by *leaving data out*. Removing one observation at a time gives the jackknife, a cheap linear approximation to the bootstrap. Removing a fold of observations, fitting on the rest and predicting the fold gives cross-validation, the standard way to estimate prediction error in machine learning. Calibrating prediction errors on held-out data gives conformal prediction intervals, and holding out the *future* gives the pseudo-out-of-sample evaluation used in macroeconomic forecasting.

11.6.1 The jackknife

Let $\widehat{\theta}_{(i)}$ be the estimate computed with observation i deleted, and $\bar{\theta}_{(\cdot)} = \frac{1}{n}\sum_i \widehat{\theta}_{(i)}$.

Definition 11.26 (Jackknife bias and variance)**.** The jackknife estimates of bias and variance are

$$\widehat{\mathrm{Bias}}_{\mathrm{J}} = (n-1)\big(\bar{\theta}_{(\cdot)} - \widehat{\theta}\big), \qquad \widehat{\mathrm{Var}}_{\mathrm{J}} = \frac{n-1}{n}\sum_{i=1}^{n}\big(\widehat{\theta}_{(i)} - \bar{\theta}_{(\cdot)}\big)^2,$$

and the bias-corrected jackknife estimator is $\tilde{\theta}_{\mathrm{J}} = \widehat{\theta} - \widehat{\mathrm{Bias}}_{\mathrm{J}} = n\widehat{\theta} - (n-1)\bar{\theta}_{(\cdot)}$.

The factors $(n-1)$ look strange but are exactly right: the deleted-one replicates are much closer to each other than independent estimates would be, because any two of them share $n-2$ observations.

Proposition 11.27 (Jackknife bias reduction)**.** *Suppose $\mathbb{E}\widehat{\theta}_n = \theta + \frac{a}{n} + \frac{b}{n^2} + O(n^{-3})$ for constants a, b. Then $\mathbb{E}\tilde{\theta}_{\mathrm{J}} = \theta - \frac{b}{n(n-1)} + O(n^{-2})$, so the jackknife removes the $O(n^{-1})$ term of the bias.*

Proof. Each $\widehat{\theta}_{(i)}$ is the estimator applied to $n-1$ observations, so $\mathbb{E}\bar{\theta}_{(\cdot)} = \theta + \frac{a}{n-1} + \frac{b}{(n-1)^2} + O(n^{-3})$. Hence

$$\mathbb{E}\tilde{\theta}_{\mathrm{J}} = n\Big(\theta + \frac{a}{n} + \frac{b}{n^2}\Big) - (n-1)\Big(\theta + \frac{a}{n-1} + \frac{b}{(n-1)^2}\Big) + O(n^{-2}) = \theta + \frac{b}{n} - \frac{b}{n-1} + O(n^{-2}),$$

and $\frac{b}{n} - \frac{b}{n-1} = -\frac{b}{n(n-1)}$. □

Example 11.28 (Jackknifing the plug-in variance)**.** For $\widehat{\theta} = \frac{1}{n}\sum (X_i - \bar{X})^2$ we have $\mathbb{E}\widehat{\theta} = \sigma^2 - \sigma^2/n$ exactly ($a = -\sigma^2$, $b = 0$), so the jackknife removes the bias completely: one can check

algebraically that $\tilde{\theta}_{\mathrm{J}} = s^2 = \frac{1}{n-1}\sum(X_i - \bar{X})^2$.

For smooth statistics, the jackknife variance is a consistent estimate of $\mathrm{Var}(\widehat{\theta})$ and is close to the bootstrap variance; it can be viewed as the bootstrap applied to a linear approximation of the statistic (Efron and Tibshirani, 1993). A classical result of Efron and Stein (1981) shows that the jackknife variance estimate is, on average, conservative. But the jackknife needs smoothness much more than the bootstrap does: for the median, deleting one observation moves the estimate between just two neighbouring order statistics, and the jackknife variance estimate is inconsistent.

The jackknife versus the bootstrap: output of `ch11_jackknife.py` (the full program is in `code/ch11`).

Output

```
n = 60; bootstrap with B = 4000
statistic             estimate  jack bias  boot bias   jack se   boot se
plug-in variance        2.3860    -0.0404    -0.0316    1.6130    1.5459
Gini                    0.4368    -0.0156    -0.0137    0.0667    0.0547
ratio food/income       0.4088     0.0022     0.0020    0.0272    0.0258
median                  0.8796    -0.0000     0.0031    0.0507    0.0851
check: jackknife-corrected plug-in variance = 2.426447, s^2 = 2.426447
median: true sd = 0.1569; mean jackknife se = 0.1445, CV across samples = 0.98
```

The jackknife and the bootstrap agree closely for the variance and the ratio, and the jackknife-corrected plug-in variance reproduces s^2 to every printed digit. For the Gini coefficient the jackknife standard error is larger than the bootstrap one, consistent with its conservative tendency. For the median, the jackknife bias is exactly zero (with even n the deleted-one medians take two values, symmetrically placed) and the jackknife standard error is a scaled spacing between the two middle order statistics: across samples its coefficient of variation is about one, so in any single sample it is almost uninformative. In the limit, $\sqrt{n}$ times the jackknife standard error of the median, divided by its true asymptotic value, converges to an Exp(1) random variable rather than to 1, consistent with the coefficient of variation of about one in the output.

△ Caution: The jackknife needs smooth statistics

Use the jackknife for smooth statistics (means, ratios, regression coefficients, Gini coefficients, poverty indices with a fixed line), where it is fast, deterministic (no random seed) and widely implemented in survey software as the "JK1" or "JKn" replication methods. Do not use it for quantiles, maxima, or other non-smooth statistics; use the bootstrap, or the delete-d jackknife with d growing with n.

11.6.2 Cross-validation as resampling for prediction error

In prediction we care not about the sampling variability of a parameter but about the expected error of a fitted model $\hat{f}$ on *new* data, $\mathrm{Err} = \mathbb{E}\,L(Y_{\mathrm{new}}, \hat{f}(\mathbf{x}_{\mathrm{new}}))$. The training error $\frac{1}{n}\sum L(y_i, \hat{f}(\mathbf{x}_i))$ is optimistic, because $\hat{f}$ was chosen to fit the same data (Chapter 12).

Definition 11.29 (K-fold and leave-one-out cross-validation)**.** Partition $\{1, \dots, n\}$ at random into K folds $\mathcal{F}_1, \dots, \mathcal{F}_K$ of nearly equal size. Let $\hat{f}^{(-k)}$ be the model fitted without fold k. The K-fold cross-validation estimate of prediction error is

$$\mathrm{CV}_K = \frac{1}{n}\sum_{k=1}^{K}\sum_{i\in\mathcal{F}_k} L\big(y_i, \hat{f}^{(-k)}(\mathbf{x}_i)\big).$$

Leave-one-out cross-validation (LOO) is the case $K = n$.

LOO uses training sets of size $n - 1$ and so has almost no bias for Err at sample size n; but its n fitted models are nearly identical, so their errors are highly correlated and the estimate can be variable. Five- or ten-fold CV is biased slightly upward (training sets are smaller) but usually has lower variance, and costs only K fits. For linear smoothers, LOO costs a single fit.

Theorem 11.30 (LOO shortcut for linear smoothers)**.** *For ridge regression* $\hat{\boldsymbol{\beta}}_\lambda = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$ *with hat matrix* $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top$ *and residuals* $e_i = y_i - \hat{y}_i$*,*

$$y_i - \hat{y}_i^{(-i)} = \frac{e_i}{1 - h_{ii}}, \qquad \text{so} \qquad \mathrm{CV}_n = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{e_i}{1 - h_{ii}}\right)^2.$$

For OLS, $\lambda = 0$*.*

Proof. Let $\mathbf{A} = \mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$, so that deleting row i gives $\mathbf{A} - \mathbf{x}_i\mathbf{x}_i^\top$ and $\mathbf{X}^\top\mathbf{y} - \mathbf{x}_i y_i$. By the Sherman–Morrison formula, $(\mathbf{A} - \mathbf{x}_i\mathbf{x}_i^\top)^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1}\mathbf{x}_i\mathbf{x}_i^\top\mathbf{A}^{-1}}{1 - h_{ii}}$ with $h_{ii} = \mathbf{x}_i^\top\mathbf{A}^{-1}\mathbf{x}_i$. Multiplying out, $\mathbf{x}_i^\top\hat{\boldsymbol{\beta}}^{(-i)} = \hat{y}_i + \frac{h_{ii}\hat{y}_i}{1-h_{ii}} - h_{ii}y_i - \frac{h_{ii}^2 y_i}{1-h_{ii}} = \frac{\hat{y}_i - h_{ii}y_i}{1-h_{ii}}$, and therefore $y_i - \mathbf{x}_i^\top\hat{\boldsymbol{\beta}}^{(-i)} = \frac{y_i - \hat{y}_i}{1-h_{ii}}$. □

We apply this to ridge regression with all 65 quadratic features of the `load_diabetes` data (bundled with scikit-learn). Features are standardised and the response centred once on the full data, so that no intercept is refitted; the shortcut and brute-force LOO then agree exactly. In practice standardisation should be repeated inside each training fold.

Listing 11.8: LOO and K-fold cross-validation for ridge regression (plotting code omitted)
ch11/ch11_cv.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# LOO and K-fold cross-validation for ridge regression on load_diabetes.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

rng = np.random.default_rng(803)
X, y = load_diabetes(return_X_y=True)
X = PolynomialFeatures(2, include_bias=False).fit_transform(X)   # 65 features
X = StandardScaler().fit_transform(X)
y = y - y.mean()
n, p = X.shape
lams = np.logspace(-1, 4, 26)


def ridge_fit(X, y, lam):
    return np.linalg.solve(X.T @ X + lam * np.eye(X.shape[1]), X.T @ y)


def loo_shortcut(lam):
    """Exact LOO MSE for ridge: mean of (e_i / (1 - h_ii))^2."""
    H = X @ np.linalg.solve(X.T @ X + lam * np.eye(p), X.T)
    e = y - H @ y
    return np.mean((e / (1 - np.diag(H)))**2), np.mean(e**2)

```

```

def kfold(lam, K, perm):
    folds = np.array_split(perm, K)
    err = 0.0
    for f in folds:
        tr = np.setdiff1d(perm, f)
        mu = X[tr].mean(0)
        b = ridge_fit(X[tr] - mu, y[tr] - y[tr].mean(), lam)
        err += np.sum((y[f] - y[tr].mean() - (X[f] - mu) @ b)**2)
    return err / n


# Check the shortcut against brute-force LOO at one value of lambda
lam0 = 100.0
bf = np.mean([(y[i] - X[i] @ ridge_fit(np.delete(X, i, 0), np.delete(y, i),
                                       lam0))**2 for i in range(n)])
print(f"n = {n}, p = {p}; LOO at lambda = {lam0:g}: shortcut = "
      f"{loo_shortcut(lam0)[0]:.2f}, brute force = {bf:.2f}")

train, loo = np.array([loo_shortcut(l)[::-1] for l in lams]).T
cv5 = np.array([[kfold(l, 5, rng.permutation(n)) for l in lams]
                for _ in range(10)])                    # 10 random splits
cv10 = np.array([kfold(l, 10, rng.permutation(n)) for l in lams])
print(f"{'lambda':>9}{'train MSE':>11}{'LOO':>9}{'10-fold':>9}"
      f"{'5-fold':>9}{'sd(5-fold)':>12}")
for i in range(0, len(lams), 5):
    print(f"{lams[i]:9.1f}{train[i]:11.1f}{loo[i]:9.1f}{cv10[i]:9.1f}"
          f"{cv5[:, i].mean():9.1f}{cv5[:, i].std():12.1f}")
k = np.argmin(loo)
print(f"lambda chosen by LOO = {lams[k]:.1f} (LOO MSE {loo[k]:.1f}); "
      f"by 10-fold = {lams[np.argmin(cv10)]:.1f}")
```

Output

```
n = 442, p = 65; LOO at lambda = 100: shortcut = 3067.22, brute force = 3067.22
   lambda  train MSE      LOO  10-fold   5-fold  sd(5-fold)
      0.1     2428.8   3371.6   3608.1   3607.5       126.0
      1.0     2436.1   3284.9   3296.8   3462.5       114.6
     10.0     2469.9   3165.4   3251.3   3241.4        71.8
    100.0     2596.8   3067.2   3060.5   3147.9        33.6
   1000.0     3410.8   3634.8   3696.1   3804.1        31.8
  10000.0     5177.7   5244.1   5320.4   5390.8        22.9
lambda chosen by LOO = 100.0 (LOO MSE 3067.2); by 10-fold = 100.0
```

The training error decreases as the penalty shrinks and is badly optimistic for small λ, while all the resampling estimates select $\lambda \approx 100$. The 5-fold estimate depends on the random split: its standard deviation across splits is largest exactly where the model overfits. Repeating K-fold CV over several random splits and averaging is cheap insurance.

Graduate Perspective: Cross-validation estimates what?

A subtle point is *which* error cross-validation estimates. One might hope it estimates the error of the model fitted to *this* training set, $\mathrm{Err}_{\mathcal{T}}$; in fact it behaves more like an estimate of the *average* error over training sets of the same size, and its correlation with $\mathrm{Err}_{\mathcal{T}}$ is weak (Hastie et al., 2009, Sec. 7.12). Standard errors of CV estimates computed as if the n held-out

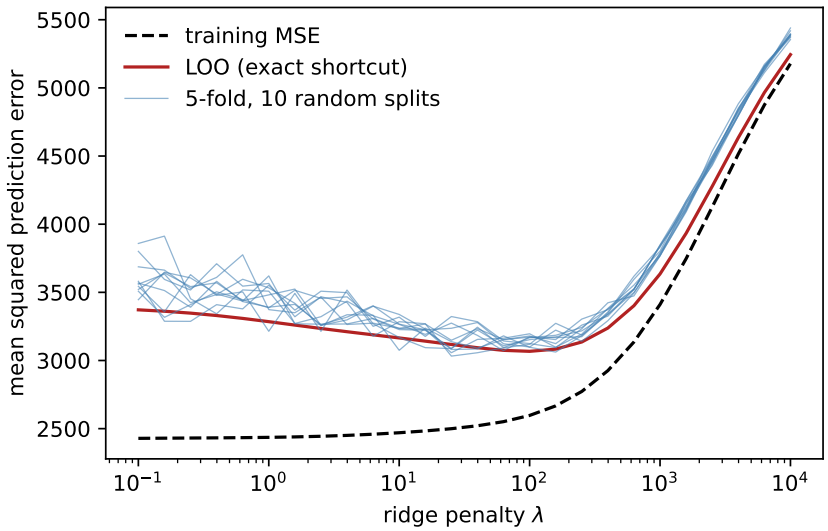


Figure 11.10: Training error, exact LOO error and 5-fold cross-validation error (10 random fold assignments) for ridge regression on the diabetes data with 65 quadratic features. The training error falls monotonically as $\lambda \to 0$; the cross-validated errors have a minimum near $\lambda = 100$.

losses were independent are too small, because the losses share training data. Bootstrap alternatives such as the .632 estimator (Efron, 1983) trade bias against variance differently.

11.6.3 Prediction intervals

A confidence interval covers a fixed parameter; a *prediction interval* covers a future random outcome Y_{n+1} at $\mathbf{x}_{n+1}$: $\mathbb{P}\big(Y_{n+1} \in [L(\mathbf{x}_{n+1}), U(\mathbf{x}_{n+1})]\big) \geq 1-\alpha$. It must account for both the estimation error in $\hat{f}$ and the irreducible noise in Y_{n+1}, and the second usually dominates. The classical normal-theory interval for linear regression, $\hat{y}_{n+1} \pm t_{n-p,1-\alpha/2}\,\hat{\sigma}\sqrt{1+\mathbf{x}_{n+1}^\top(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{x}_{n+1}}$, assumes normal, homoskedastic errors. The *residual bootstrap* prediction interval refits the model on bootstrap data, adds a freshly resampled residual to each bootstrap prediction, and takes quantiles; it relaxes normality but still assumes homoskedasticity.

11.6.4 Split conformal prediction

Conformal prediction gives distribution-free, finite-sample guarantees under one assumption: the data are *exchangeable*.

Definition 11.31 (Split conformal prediction interval)**.** Split the data at random into a proper training set $\mathcal{I}_1$ and a calibration set $\mathcal{I}_2$ of size n_2. Fit $\hat{f}$ on $\mathcal{I}_1$. Compute the conformity scores $S_i = \big|y_i - \hat{f}(\mathbf{x}_i)\big|$ for $i \in \mathcal{I}_2$, let $k = \lceil (n_2+1)(1-\alpha) \rceil$ and let $\hat{q}$ be the k-th smallest score. The split conformal interval is

$$C(\mathbf{x}) = \big[\hat{f}(\mathbf{x}) - \hat{q},\ \hat{f}(\mathbf{x}) + \hat{q}\big].$$

Theorem 11.32 (Marginal coverage of split conformal)**.** *If $(\mathbf{x}_i, y_i)$, $i = 1, \dots, n+1$, are exchangeable, then $\mathbb{P}\big(y_{n+1} \in C(\mathbf{x}_{n+1})\big) \geq 1-\alpha$. If in addition the scores are almost surely distinct, $\mathbb{P}(y_{n+1} \in C(\mathbf{x}_{n+1})) \leq 1-\alpha+\frac{1}{n_2+1}$.*

Proof. Condition on the training set $\mathcal{I}_1$, so that $\hat{f}$ is fixed. The calibration scores S_i, $i \in \mathcal{I}_2$, and

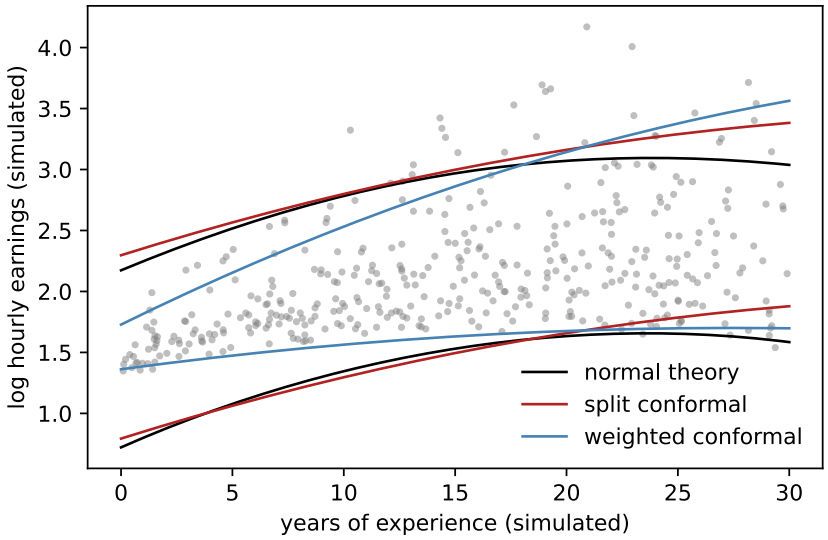


Figure 11.11: Nominal 90% prediction bands for simulated log earnings with skewed errors whose spread increases with experience. The normal-theory and split conformal bands have (nearly) constant width; the locally weighted conformal band widens with the noise.

the test score $S_{n+1} = \left|y_{n+1} - \hat{f}(\mathbf{x}_{n+1})\right|$ are then exchangeable. Hence the rank of S_{n+1} among these $n_2 + 1$ scores is uniformly distributed on $\{1, \ldots, n_2 + 1\}$ (with ties broken at random). Now $y_{n+1} \in C(\mathbf{x}_{n+1})$ iff $S_{n+1} \le \hat{q}$, which holds whenever the rank of S_{n+1} is at most k; this has probability at least $k/(n_2 + 1) \ge 1 - \alpha$. With distinct scores the event is exactly "rank $\le k$", of probability $k/(n_2 + 1) < 1 - \alpha + 1/(n_2 + 1)$. □

The guarantee is *marginal*: averaged over $\mathbf{x}_{n+1}$. With heteroskedastic errors a constant-width interval over-covers where the noise is small and under-covers where it is large. *Locally weighted* conformal prediction divides the scores by an estimate $\hat{\sigma}(\mathbf{x})$ of the local spread, fitted on $\mathcal{I}_1$, and uses $\hat{f}(\mathbf{x}) \pm \hat{q}\,\hat{\sigma}(\mathbf{x})$; the marginal guarantee is unchanged, and conditional coverage improves when $\hat{\sigma}$ is good.

Normal-theory, bootstrap and conformal prediction intervals: output of `ch11_conformal.py` (the full program is in `code/ch11`).

Output

```
Nominal 90% prediction intervals; n = 400, 300 replications
method               overall   x<10  10-20   x>20  length  sd cov
normal theory          0.924  0.984  0.938  0.850   1.385   0.018
residual bootstrap     0.897  0.989  0.932  0.769   1.322   0.020
split conformal        0.901  0.980  0.922  0.800   1.270   0.024
weighted conformal     0.901  0.899  0.903  0.901   1.158   0.024
```

Both conformal intervals deliver their promised 90% marginal coverage, while the normal-theory interval over-covers overall and the residual bootstrap is close. But the conditional picture differs sharply: the constant-width intervals cover 98% of young workers and only 77–85% of experienced workers. The weighted conformal interval covers about 90% in every experience band, and is also the shortest on average.

⋆ Key Concept: Conformal prediction

Split conformal prediction wraps *any* predictive model, from OLS to gradient boosting, in a prediction interval with a finite-sample marginal coverage guarantee, using nothing but exchangeability and a held-out calibration set. The guarantee is marginal, not conditional: a good choice of conformity score (e.g. residuals scaled by a local spread estimate) is needed for intervals that are honest for every subgroup.

11.6.5 Forecast uncertainty and pseudo-out-of-sample evaluation

Time series are not exchangeable: the future may differ from the past, and random K-fold splits leak information from the future into the training set. Forecasters therefore evaluate models by *pseudo-out-of-sample* (POOS) experiments that imitate real-time forecasting.

> **Definition 11.33** (Pseudo-out-of-sample evaluation)**.** Choose a first forecast origin T_0. For each $t = T_0, \dots, T-h$, estimate every model using only data up to time t (an *expanding* window, or a *rolling* window of fixed length), forecast y_{t+h}, and record the error $e_{t+h|t}$. Compare models by the root mean squared forecast error $\text{RMSE} = \{\frac{1}{T-T_0-h+1}\sum_t e^2_{t+h|t}\}^{1/2}$, usually relative to a benchmark.

To decide whether one model's RMSE is *significantly* lower, the Diebold–Mariano test (Diebold and Mariano, 1995) uses the loss differential $d_t = e^2_{1,t} - e^2_{2,t}$ and the statistic $\bar{d}/\widehat{\text{se}}(\bar{d})$, with a HAC standard error for $h > 1$.

Goulet Coulombe et al. (2022) carried out a very large POOS experiment on US macroeconomic data to ask how machine learning helps macroeconomic forecasting. They decompose the gains into nonlinearity, regularisation, hyperparameter selection and the loss function, and conclude that nonlinearity is the main source of improvement, that standard factor models remain a strong way to regularise, and that ordinary K-fold cross-validation is a good way to tune hyperparameters, even for time series. Their study is also a model of how POOS evaluation should be done: many targets, several horizons, a fixed benchmark, and reporting of relative RMSEs.

Uncertainty about a forecast is communicated by a *fan chart*: a set of nested prediction intervals at each horizon. A model-based (sieve) bootstrap produces one: fit an autoregression, generate bootstrap series recursively from resampled residuals, re-estimate the model on each series (to include parameter uncertainty), and simulate future paths from each re-estimated model.

Listing 11.9: Pseudo-out-of-sample evaluation and a bootstrap fan chart (plotting code omitted)
ch11/ch11_forecast.py

```
# DEP 803 -- Chapter 11: Resampling: Bootstrap, Permutation Tests, Intervals
# Pseudo-out-of-sample forecast evaluation and a bootstrap fan chart.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)
T, T0, B, H = 300, 150, 499, 12

# Simulated monthly inflation-like series driven by a persistent indicator
z = np.zeros(T)
infl = np.zeros(T)
for t in range(2, T):
```

```
    z[t] = 0.8 * z[t - 1] + rng.normal(0, 1)
    infl[t] = (0.8 + 0.5 * infl[t - 1] + 0.2 * infl[t - 2] + 0.15 * z[t - 1]
               + rng.normal(0, 0.5) * (1 + 0.5 * (t > 220)))  # volatility jump


def lagmat(y, x, t_end):
    """Regressors for y_t on (1, y_{t-1}, y_{t-2}[, x_{t-1}]), t < t_end."""
    rows = np.arange(2, t_end)
    Z = np.column_stack([np.ones(rows.size), y[rows - 1], y[rows - 2]])
    if x is not None:
        Z = np.column_stack([Z, x[rows - 1]])
    return Z, y[rows]


models = {"historical mean": None, "random walk": None, "AR(2)": None,
          "AR(2) + indicator": z}
err = {k: [] for k in models}
hits80 = []
for t in range(T0, T):                        # forecast infl[t] with data < t
    err["historical mean"].append(infl[t] - infl[:t].mean())
    err["random walk"].append(infl[t] - infl[t - 1])
    for k in ("AR(2)", "AR(2) + indicator"):
        Z, yy = lagmat(infl, models[k], t)
        b, *_ = np.linalg.lstsq(Z, yy, rcond=None)
        zt = np.r_[1, infl[t - 1], infl[t - 2]]
        if models[k] is not None:
            zt = np.r_[zt, z[t - 1]]
        err[k].append(infl[t] - zt @ b)
        if k == "AR(2)":                      # bootstrap 80% interval, h = 1
            u = yy - Z @ b
            q = np.quantile(zt @ b + u[rng.integers(0, u.size, B)], [0.1, 0.9])
            hits80.append(q[0] <= infl[t] <= q[1])
rmse_ar = np.sqrt(np.mean(np.square(err["AR(2)"])))
print(f"Pseudo-out-of-sample, expanding window: {T - T0} one-step forecasts")
print(f"{'model':<20}{'RMSE':>8}{'relative to AR(2)':>19}")
for k, e in err.items():
    rm = np.sqrt(np.mean(np.square(e)))
    print(f"{k:<20}{rm:8.3f}{rm / rmse_ar:19.3f}")
# Diebold-Mariano (h = 1, squared loss): AR(2)+indicator vs AR(2)
dl = np.square(err["AR(2) + indicator"]) - np.square(err["AR(2)"])
dm = dl.mean() / (dl.std(ddof=1) / np.sqrt(dl.size))
print(f"Diebold-Mariano statistic (indicator vs AR(2)) = {dm:.2f}")
hits80 = np.array(hits80)
print(f"bootstrap 80% interval coverage: before t=220 {hits80[:70].mean():.3f},"
      f" after {hits80[70:].mean():.3f}")

# Fan chart: residual bootstrap of AR(2) with parameter re-estimation
Z, yy = lagmat(infl, None, T)
b, *_ = np.linalg.lstsq(Z, yy, rcond=None)
u = yy - Z @ b
paths = np.empty((B, H))
for j in range(B):
    ys = infl.copy()                          # recursive bootstrap sample
    us = u[rng.integers(0, u.size, u.size)]
    for t in range(2, T):
        ys[t] = b[0] + b[1] * ys[t - 1] + b[2] * ys[t - 2] + us[t - 2]
    Zs, ys2 = lagmat(ys, None, T)
```

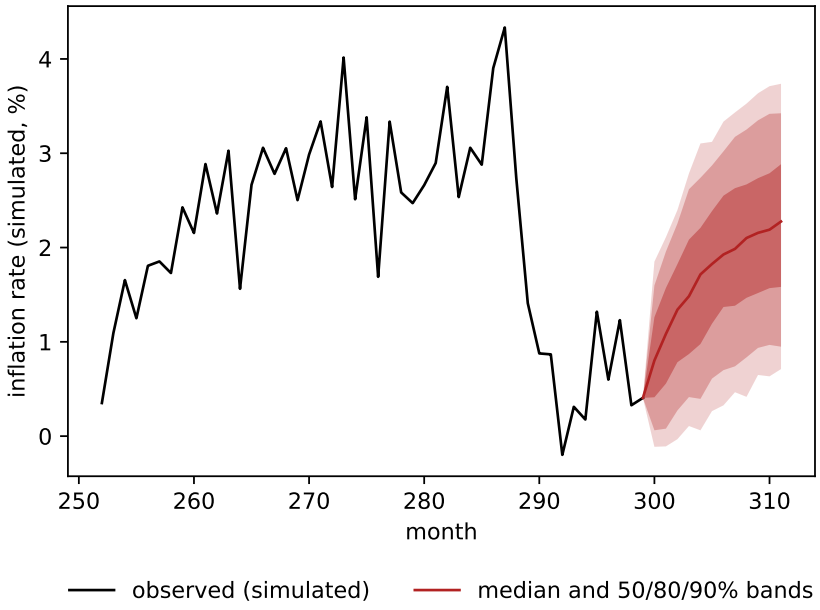


Figure 11.12: Bootstrap fan chart for a simulated monthly inflation series: median forecast and 50%, 80% and 90% prediction bands up to 12 months ahead from an AR(2) model with residual bootstrap and parameter re-estimation.

```
    bs, *_ = np.linalg.lstsq(Zs, ys2, rcond=None)
    hist = [infl[-2], infl[-1]]
    for h in range(H):
        hist.append(bs @ [1, hist[-1], hist[-2]] + rng.choice(u))
    paths[j] = hist[2:]
qs = np.quantile(paths, [0.05, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95], axis=0)
print("fan chart median path (h=1,6,12):",
      " ".join(f"{qs[3, h]:.2f}" for h in (0, 5, 11)),
      "| 90% band at h=12:", f"[{qs[0, -1]:.2f}, {qs[-1, -1]:.2f}]")
```

Output

```
Pseudo-out-of-sample, expanding window: 150 one-step forecasts
model                    RMSE  relative to AR(2)
historical mean         0.961              1.523
random walk             0.681              1.079
AR(2)                   0.631              1.000
AR(2) + indicator       0.596              0.943
Diebold-Mariano statistic (indicator vs AR(2)) = -2.13
bootstrap 80% interval coverage: before t=220 0.771, after 0.637
fan chart median path (h=1,6,12): 0.80 1.83 2.28 | 90% band at h=12: [0.71, 3.74]
```

In the POOS experiment, the AR(2) model beats the historical mean and the random walk, and adding the leading indicator lowers the RMSE by about 6%; the Diebold–Mariano statistic of -2.1 suggests that the gain is not just noise. The bootstrap 80% intervals cover 77% of outcomes before the simulated rise in volatility at $t = 220$, but only 64% afterwards: a bootstrap that resamples *past* residuals cannot anticipate a change in the volatility regime. The fan in Fig. 11.12 widens with the horizon as shocks accumulate, and its centre moves back towards the unconditional mean.

Policy Lens: Fan charts at central banks

The Bank of England began publishing fan charts for inflation projections in its *Inflation Report* in the mid-1990s (Britton et al., 1998), and many central banks, including several in sub-Saharan Africa, now publish similar charts for inflation and GDP growth. The fan conveys a simple but important message: a forecast is a distribution, not a number. Monetary policy committees use the probability that inflation will lie outside the target band at the policy horizon, not just the central projection. Pseudo-out-of-sample evaluation of the coverage of past fans, as in the listing above, is a valuable check on whether the published uncertainty is honest, especially after structural breaks such as the 2020 pandemic shock.

Historical Note: Cross-validation and conformal prediction

The idea of assessing a prediction rule on data not used to construct it is old, but its systematic use for model choice dates from Stone (1974) and Geisser (1975), who independently formalised cross-validation (Geisser called it predictive sample reuse). Conformal prediction was developed by Vladimir Vovk, Alexander Gammerman and Glenn Shafer (Vovk et al., 2005); the split version and its statistical theory for regression were popularised by Lei et al. (2018), and Angelopoulos and Bates (2023) give an accessible introduction.

Exercises

11.6.1 Theory Show algebraically that the jackknife bias-corrected plug-in variance equals s^2. (Hint: express $\widehat{\theta}_{(i)}$ in terms of $\bar{X}$, X_i and $\widehat{\theta}$.)

11.6.2 Theory Show that for the sample mean $\widehat{\mathrm{Var}}_{\mathrm{J}} = s^2/n$ exactly, and that for the ratio $\bar{Y}/\bar{X}$ the jackknife variance agrees with the linearisation variance of Section 11.1 to first order.

11.6.3 Theory Complete the proof of Theorem 11.30 by verifying the Sherman–Morrison step, and show that for OLS the generalised cross-validation approximation replaces h_{ii} by $\mathrm{tr}(\mathbf{H})/n$.

11.6.4 Python Replace OLS by `GradientBoostingRegressor` from scikit-learn in `ch11_conformal.py` and verify that split conformal coverage remains about 90%. Then implement conformalised quantile regression using the `loss="quantile"` option.

11.6.5 Python In `ch11_cv.py`, compute 10-fold CV with the standardisation done *inside* each training fold, and compare the selected λ and the CV error with the version that standardises once.

11.6.6 Python Extend `ch11_forecast.py` to horizons $h = 3$ and $h = 12$ (direct forecasts), and compute the Diebold–Mariano statistic with a Newey–West standard error with $h - 1$ lags. Does the indicator still help at longer horizons?

11.6.7 ★ Graduate Show that random K-fold cross-validation can be valid for a stationary autoregression when the model includes enough lags that the errors are uncorrelated (Bergmeir et al., 2018), and discuss why it fails when the model is underspecified. Relate this to the findings of Goulet Coulombe et al. (2022).

11.6.8 Policy A central bank's 80% fan chart for inflation was "hit" by the outcome in only 55% of the last 40 quarters. Draft a short paragraph for the monetary policy report that explains what this implies, what might have caused it, and how the fan could be recalibrated (e.g. by conformal or empirical-quantile methods based on past POOS errors).

Chapter Summary

- A confidence interval is a procedure with a coverage probability over repeated samples; nominal and actual coverage differ in small or skewed samples, and simulation reveals by how much.
- The plug-in principle estimates $T(F)$ by $T(\widehat{F}_n)$; the Glivenko–Cantelli theorem and the DKW inequality make $\widehat{F}_n$ a good stand-in for F. Analytic standard errors require influence-function calculations that the bootstrap automates.
- The nonparametric bootstrap estimates sampling distributions by resampling with replacement. It is consistent for smooth functionals, needs $B \geq 1000$ or more for intervals, and fails for extremes, infinite-variance data and very small samples; the parametric bootstrap is an alternative when the model is trusted.
- Percentile, basic and normal intervals are first-order accurate; studentised and BCa intervals are second-order accurate and usually cover better in skewed problems.
- In regression, the pairs and wild bootstraps are robust to heteroskedasticity, the residual bootstrap is not. In clustered surveys, resample clusters; with few clusters use the wild cluster restricted bootstrap. In time series, resample blocks.
- Randomisation tests are exact in experiments under the sharp null, can be inverted into confidence intervals, and must re-randomise as the design randomised. Permutation tests rest on exchangeability. Multiple outcomes call for Holm (FWER) or Benjamini–Hochberg (FDR) corrections.
- The jackknife is a cheap, smooth-statistic approximation to the bootstrap; cross-validation estimates prediction error; split conformal prediction provides distribution-free prediction intervals; and pseudo-out-of-sample evaluation and fan charts quantify forecast uncertainty.

Further Reading

The classic introductions to the bootstrap are Efron and Tibshirani (1993) and Davison and Hinkley (1997); Efron and Hastie (2016) place the bootstrap and the jackknife in the broader history of computer-age statistical inference, and Hesterberg (2015) is a very practical guide to what works. Hall (1992) develops the theory of bootstrap intervals, and van der Vaart (1998, Ch. 23) and Wasserman (2004, Ch. 8) give concise rigorous treatments. For econometric applications, see Cameron and Trivedi (2005) and Hansen (2022) on the bootstrap for regression, Cameron et al. (2008) and MacKinnon and Webb (2017) on clustered data, and Imbens and Rubin (2015) on randomisation inference. Benjamini and Hochberg (1995) remains the starting point for false discovery rates. On prediction, Hastie et al. (2009, Ch. 7), Murphy (2022) and Géron (2022, Ch. 2) discuss cross-validation from the machine-learning side, and Angelopoulos and Bates (2023) introduce conformal prediction. For forecasting, Goulet Coulombe et al. (2022) is a landmark pseudo-out-of-sample study, and Britton et al. (1998) explain the fan chart. Matsuura (2022) offers the Bayesian alternative of posterior predictive intervals, to which resampling-based intervals are a useful frequentist counterpart.

Chapter 12

Matrix Calculus for Machine Learning, Regularisation and the Bias-Variance Trade-off

Learning Objectives

After working through this chapter you should be able to:

1. state a layout convention and use it consistently to differentiate scalar, vector and matrix functions of vectors and matrices, including linear and quadratic forms, traces, inverses and log-determinants;
2. use the differential (Magnus–Neudecker) approach and the trace trick to derive gradients quickly, and verify any derivative numerically with finite differences;
3. derive the OLS estimator, the Gaussian MLE of a covariance matrix, the gradients and Hessians of logistic and multinomial logit models, and the GLS estimator;
4. explain forward- and reverse-mode automatic differentiation, and implement back-propagation for a two-layer neural network in NumPy, including a gradient check;
5. prove the bias–variance decomposition of expected prediction error, compute effective degrees of freedom, and recognise under- and overfitting (and double descent);
6. derive and implement ridge, lasso (by coordinate descent) and the elastic net, and explain their geometry, Bayesian interpretation and the role of standardisation;
7. choose tuning parameters by K-fold cross-validation, the one-standard-error rule and information criteria, and run a pseudo-out-of-sample forecasting comparison with many predictors.

Almost every modern data-science method is "minimise a loss over a parameter vector". The loss might be a residual sum of squares, a negative log-likelihood or the cross-entropy of a neural network; the parameters might be a handful of regression coefficients or millions of network weights. Two questions immediately arise. First, *how do we compute the derivatives* that every optimiser in Chapter 8 needs? For a function of a matrix, writing out partial derivatives entry by entry is error-prone and opaque; *matrix calculus* gives compact rules, and *automatic differentiation* turns those rules into software. Second, *what should we minimise*? Minimising the training loss alone rewards models that memorise noise. The *bias–variance trade-off* explains why, and *regularisation* — adding a penalty such as $\lambda \|\boldsymbol{\beta}\|_2^2$ or $\lambda \|\boldsymbol{\beta}\|_1$ — is the main remedy.

These questions are not academic for economic policy. A central bank that forecasts inflation from a hundred monthly indicators, a ministry that builds a proxy-means test from a long

household questionnaire, and a revenue authority that scores tax returns for audit all face the same problem: many candidate predictors, limited and noisy data, and a strong temptation to overfit. The tools of this chapter — ridge, lasso, elastic net, cross-validation and honest out-of-sample evaluation — are among those that Goulet Coulombe et al. (2022) find to matter for macroeconomic forecasting. The chapter builds on the linear algebra of Chapters 2 and 3 (SVD, projections), the estimation theory of Chapter 7 (MLE, Fisher information) and the optimisation methods of Chapters 8 and 9 (gradients, Newton's method, KKT conditions), and it prepares the ground for the resampling ideas of Chapter 11.

12.1 Matrix Calculus

12.1.1 Layout conventions

Let $f : \mathbb{R}^n \to \mathbb{R}^m$. Its mn partial derivatives can be arranged as an $m \times n$ or as an $n \times m$ matrix, and textbooks disagree. Mixing conventions within one derivation is the most common source of transposition errors, so we fix ours once and for all.

Definition 12.1 (Layout convention used in this book). (i) **Jacobian (numerator layout).** For $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian $Df(\mathbf{x}) = \partial f / \partial \mathbf{x}^\top$ is the $m \times n$ matrix with (i, j) entry $\partial f_i / \partial x_j$: rows index outputs, columns index inputs.

(ii) **Gradient (column).** For a scalar $f : \mathbb{R}^n \to \mathbb{R}$, the gradient $\nabla f(\mathbf{x}) = \partial f / \partial \mathbf{x} = Df(\mathbf{x})^\top$ is an $n \times 1$ column vector, the same shape as $\mathbf{x}$.

(iii) **Matrix argument.** For a scalar $f : \mathbb{R}^{m \times n} \to \mathbb{R}$, the gradient $\partial f / \partial \mathbf{X}$ is the $m \times n$ matrix with (i, j) entry $\partial f / \partial X_{ij}$, the same shape as $\mathbf{X}$.

(iv) **Hessian.** $\nabla^2 f(\mathbf{x}) = D(\nabla f)(\mathbf{x})$ is the $n \times n$ matrix with entries $\partial^2 f / \partial x_i \partial x_j$.

This is the convention of Magnus and Neudecker (2019) for Jacobians, of Deisenroth et al. (2020), and of ML software (a gradient returned by PyTorch or JAX has the shape of the parameter). With it, the first-order Taylor expansions read

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + Df(\mathbf{x})\,\mathbf{h} + o(\|\mathbf{h}\|), \qquad f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + o(\|\mathbf{h}\|) \ \ (m = 1), \tag{12.1}$$

and, for a matrix argument, $f(\mathbf{X} + \mathbf{E}) = f(\mathbf{X}) + \operatorname{tr}\big[(\partial f / \partial \mathbf{X})^\top \mathbf{E}\big] + o(\|\mathbf{E}\|)$. The "denominator layout" used in some econometrics texts stores the Jacobian as the $n \times m$ transpose; every formula below converts by transposition.

△ Caution: Check shapes before anything else

Before trusting a derived gradient, check that its dimensions match the argument. If β is $p \times 1$, then $\partial \|\mathbf{y} - \mathbf{X}\beta\|^2 / \partial \beta$ must be $p \times 1$; the expression $-2(\mathbf{y} - \mathbf{X}\beta)^\top \mathbf{X}$ is $1 \times p$ and is the numerator-layout Jacobian, not the gradient. Shape checking catches most errors; finite differences (Section 12.1.6) catch the rest.

12.1.2 Linear and quadratic forms

Theorem 12.2 (Derivatives of linear and quadratic forms). *Let $\mathbf{a} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{n \times n}$ and $\mathbf{B} \in \mathbb{R}^{m \times n}$ be constant. Then*

(a) $\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}$;

(b) $D_{\mathbf{x}}(\mathbf{B}\mathbf{x}) = \mathbf{B}$;

(c) $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$, *which equals* $2\mathbf{A}\mathbf{x}$ *if* $\mathbf{A}$ *is symmetric;*

(d) $\nabla^2_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = \mathbf{A} + \mathbf{A}^\top$;

(e) $\nabla_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$, *with Hessian* $2\mathbf{X}^\top\mathbf{X}$.

Proof. (a) $\mathbf{a}^\top\mathbf{x} = \sum_j a_j x_j$, so $\partial/\partial x_k = a_k$. (b) $(\mathbf{B}\mathbf{x})_i = \sum_j B_{ij}x_j$ gives $\partial(\mathbf{B}\mathbf{x})_i/\partial x_k = B_{ik}$. (c) Write $\mathbf{x}^\top\mathbf{A}\mathbf{x} = \sum_{i,j} A_{ij}x_i x_j$. By the product rule,

$$\frac{\partial}{\partial x_k}\sum_{i,j} A_{ij}x_i x_j = \sum_j A_{kj}x_j + \sum_i A_{ik}x_i = (\mathbf{A}\mathbf{x})_k + (\mathbf{A}^\top\mathbf{x})_k.$$

(d) Differentiate $(\mathbf{A}+\mathbf{A}^\top)\mathbf{x}$ using (b). (e) Expand $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 = \mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top\mathbf{X}\boldsymbol{\beta} + \boldsymbol{\beta}^\top\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta}$ and apply (a) with $\mathbf{a} = \mathbf{X}^\top\mathbf{y}$ and (c)–(d) with the symmetric matrix $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$. □

Only the symmetric part of $\mathbf{A}$ matters in a quadratic form, since $\mathbf{x}^\top\mathbf{A}\mathbf{x} = \mathbf{x}^\top\frac{1}{2}(\mathbf{A}+\mathbf{A}^\top)\mathbf{x}$; this is why the answer in (c) treats $\mathbf{A}$ and $\mathbf{A}^\top$ symmetrically.

12.1.3 The chain rule for vector functions

Theorem 12.3 (Chain rule)**.** *If $g : \mathbb{R}^n \to \mathbb{R}^k$ is differentiable at $\mathbf{x}$ and $f : \mathbb{R}^k \to \mathbb{R}^m$ is differentiable at $g(\mathbf{x})$, then $h = f \circ g$ is differentiable at $\mathbf{x}$ and*

$$\underbrace{Dh(\mathbf{x})}_{m\times n} = \underbrace{Df\big(g(\mathbf{x})\big)}_{m\times k}\,\underbrace{Dg(\mathbf{x})}_{k\times n}. \tag{12.2}$$

For scalar f ($m = 1$) this becomes $\nabla h(\mathbf{x}) = Dg(\mathbf{x})^\top \nabla f\big(g(\mathbf{x})\big)$.

Proof sketch. By (12.1), $g(\mathbf{x}+\mathbf{h}) = g(\mathbf{x}) + Dg\,\mathbf{h} + o(\|\mathbf{h}\|)$. Substituting into the expansion of f around $g(\mathbf{x})$ gives $f(g(\mathbf{x}+\mathbf{h})) = f(g(\mathbf{x})) + Df\,Dg\,\mathbf{h} + o(\|\mathbf{h}\|)$, because Df is a bounded linear map and $o\big(\|Dg\,\mathbf{h} + o(\|\mathbf{h}\|)\|\big) = o(\|\mathbf{h}\|)$. Uniqueness of the derivative gives (12.2). □

The numerator layout makes the chain rule read in the natural order: Jacobians multiply from the outermost function inwards. The *order* in which a long product $Df_L\,Df_{L-1}\cdots Df_1$ is evaluated — from the right or from the left — is exactly the difference between forward- and reverse-mode automatic differentiation (Section 12.3).

Example 12.4 (Jacobians of an elementwise map and of softmax)**.** If $\sigma : \mathbb{R} \to \mathbb{R}$ is applied elementwise, $D\sigma(\mathbf{z}) = \operatorname{diag}(\sigma'(z_1), \dots, \sigma'(z_k))$. Hence for $g(\mathbf{x}) = \boldsymbol{C}\tanh(\mathbf{D}\mathbf{x})$ the chain rule gives $Dg(\mathbf{x}) = \boldsymbol{C}\,\operatorname{diag}\big(1 - \tanh^2(\mathbf{D}\mathbf{x})\big)\,\mathbf{D}$. For the softmax $s_i(\mathbf{z}) = e^{z_i}/\sum_j e^{z_j}$, the quotient rule gives $\partial s_i/\partial z_j = s_i(\delta_{ij} - s_j)$, that is,

$$D\mathbf{s}(\mathbf{z}) = \operatorname{diag}(\mathbf{s}) - \mathbf{s}\mathbf{s}^\top. \tag{12.3}$$

This matrix is symmetric positive semidefinite with $D\mathbf{s}(\mathbf{z})\mathbf{1} = \mathbf{0}$ (adding a constant to every z_j leaves $\mathbf{s}$ unchanged). It is also the Hessian of the log-sum-exp function $\operatorname{lse}(\mathbf{z}) = \log\sum_j e^{z_j}$, whose gradient is $\mathbf{s}(\mathbf{z})$; hence log-sum-exp is convex, a fact used repeatedly in Section 12.2.

12.1.4 The differential approach of Magnus and Neudecker

Entrywise differentiation becomes painful as soon as matrices multiply or invert. The *differential* approach replaces derivatives by first-order changes. For a scalar function, the differential $\mathrm{d}f$ is the linear part of $f(\mathbf{x}+\mathrm{d}\mathbf{x}) - f(\mathbf{x})$; by (12.1),

$$\mathrm{d}f = \nabla f(\mathbf{x})^\top \mathrm{d}\mathbf{x} \iff \nabla f(\mathbf{x}) \text{ is the gradient}, \qquad \mathrm{d}f = \operatorname{tr}\big(\boldsymbol{G}^\top \mathrm{d}\mathbf{X}\big) \iff \frac{\partial f}{\partial \mathbf{X}} = \boldsymbol{G}. \tag{12.4}$$

This *identification theorem* (Magnus and Neudecker, 2019, Ch. 5) turns differentiation into algebra: compute $\mathrm{d}f$ using the rules below, manipulate it into the form $\operatorname{tr}(\boldsymbol{G}^\top \mathrm{d}\mathbf{X})$, and read off $\boldsymbol{G}$.

Proposition 12.5 (Rules for differentials)**.** *For conformable matrix functions* $\mathbf{U}, \mathbf{V}$ *and constant* $\mathbf{A}$*:*

(a) $\mathrm{d}\mathbf{A} = \mathbf{0}$, $\mathrm{d}(\mathbf{U}+\mathbf{V}) = \mathrm{d}\mathbf{U} + \mathrm{d}\mathbf{V}$, $\mathrm{d}(\mathbf{U}^\top) = (\mathrm{d}\mathbf{U})^\top$*;*

(b) $\mathrm{d}(\mathbf{UV}) = (\mathrm{d}\mathbf{U})\mathbf{V} + \mathbf{U}(\mathrm{d}\mathbf{V})$ *(product rule; order matters);*

(c) $\mathrm{d}\operatorname{tr}(\mathbf{U}) = \operatorname{tr}(\mathrm{d}\mathbf{U})$*;*

(d) $\mathrm{d}(\mathbf{X}^{-1}) = -\mathbf{X}^{-1}(\mathrm{d}\mathbf{X})\mathbf{X}^{-1}$*;*

(e) $\mathrm{d}\log|\det \mathbf{X}| = \operatorname{tr}(\mathbf{X}^{-1}\mathrm{d}\mathbf{X})$*;*

(f) $\mathrm{d}\,\sigma(\mathbf{U}) = \sigma'(\mathbf{U}) \odot \mathrm{d}\mathbf{U}$ *for an elementwise function* σ*.*

Proof. (a)–(c) and (f) follow from linearity and the scalar product rule applied entrywise. (d) Differentiate the identity $\mathbf{X}\mathbf{X}^{-1} = \mathbf{I}$ with (b): $(\mathrm{d}\mathbf{X})\mathbf{X}^{-1} + \mathbf{X}\,\mathrm{d}(\mathbf{X}^{-1}) = \mathbf{0}$, and solve for $\mathrm{d}(\mathbf{X}^{-1})$. (e) By cofactor expansion along row i, $\det \mathbf{X} = \sum_j X_{ij} C_{ij}$, where the cofactors C_{ij} do not involve X_{ij}; hence $\partial \det \mathbf{X}/\partial X_{ij} = C_{ij} = \det(\mathbf{X})\,(\mathbf{X}^{-1})_{ji}$ (Cramer's rule). Therefore $\mathrm{d}\det \mathbf{X} = \det(\mathbf{X})\operatorname{tr}(\mathbf{X}^{-1}\mathrm{d}\mathbf{X})$, and the chain rule for the scalar logarithm gives (e). □

Trace tricks. Three properties of the trace do most of the work:

$$a = \operatorname{tr}(a) \text{ for a scalar } a, \qquad \operatorname{tr}(\mathbf{ABC}) = \operatorname{tr}(\boldsymbol{CAB}) = \operatorname{tr}(\mathbf{B}\boldsymbol{CA}), \qquad \operatorname{tr}(\mathbf{A}^\top) = \operatorname{tr}(\mathbf{A}). \tag{12.5}$$

Cyclic permutation lets us move $\mathrm{d}\mathbf{X}$ to the end of any product, which is the form required by (12.4). A scalar such as $\mathbf{x}^\top\mathbf{A}\mathbf{x}$ can be written as $\operatorname{tr}(\mathbf{A}\mathbf{x}\mathbf{x}^\top)$, which is how sums of quadratic forms become traces of a matrix times an outer-product (scatter) matrix.

Theorem 12.6 (Gradients with respect to a matrix)**.** *For constant* $\mathbf{A}, \mathbf{B}$ *and* $\mathbf{a}$ *of conformable sizes, and invertible* $\mathbf{X}$*:*

$$\frac{\partial}{\partial \mathbf{X}} \operatorname{tr}(\mathbf{AX}) = \mathbf{A}^\top, \qquad \frac{\partial}{\partial \mathbf{X}} \operatorname{tr}(\mathbf{X}^\top\mathbf{BX}) = (\mathbf{B} + \mathbf{B}^\top)\mathbf{X}, \tag{12.6}$$

$$\frac{\partial}{\partial \mathbf{X}} \log|\det \mathbf{X}| = \mathbf{X}^{-\top}, \qquad \frac{\partial}{\partial \mathbf{X}} \operatorname{tr}(\mathbf{AX}^{-1}) = -\big(\mathbf{X}^{-1}\mathbf{A}\mathbf{X}^{-1}\big)^\top, \tag{12.7}$$

$$\frac{\partial}{\partial \mathbf{X}} \mathbf{a}^\top\mathbf{X}^{-1}\mathbf{a} = -\mathbf{X}^{-\top}\mathbf{a}\mathbf{a}^\top\mathbf{X}^{-\top}, \qquad \frac{\partial}{\partial \mathbf{X}} \mathbf{a}^\top\mathbf{X}\mathbf{b} = \mathbf{a}\mathbf{b}^\top. \tag{12.8}$$

Proof. Each follows the same three steps: differentiate, move $\mathrm{d}\mathbf{X}$ to the right by cyclicity, identify. For instance, $\mathrm{d}\operatorname{tr}(\mathbf{AX}) = \operatorname{tr}(\mathbf{A}\,\mathrm{d}\mathbf{X}) = \operatorname{tr}\big((\mathbf{A}^\top)^\top \mathrm{d}\mathbf{X}\big)$, so the gradient is $\mathbf{A}^\top$. Next, $\mathrm{d}\operatorname{tr}(\mathbf{X}^\top\mathbf{BX}) = \operatorname{tr}\big((\mathrm{d}\mathbf{X})^\top\mathbf{BX}\big) + \operatorname{tr}\big(\mathbf{X}^\top\mathbf{B}\,\mathrm{d}\mathbf{X}\big) = \operatorname{tr}\big(\mathbf{X}^\top\mathbf{B}^\top\mathrm{d}\mathbf{X}\big) + \operatorname{tr}\big(\mathbf{X}^\top\mathbf{B}\,\mathrm{d}\mathbf{X}\big)$, using $\operatorname{tr}(M^\top) = \operatorname{tr}(M)$; hence the gradient is $\big(\mathbf{X}^\top(\mathbf{B}+\mathbf{B}^\top)\big)^\top = (\mathbf{B}+\mathbf{B}^\top)\mathbf{X}$. The log-determinant is Proposition 12.5(e) with $\boldsymbol{G}^\top = \mathbf{X}^{-1}$. For the inverse, $\mathrm{d}\operatorname{tr}(\mathbf{AX}^{-1}) = -\operatorname{tr}(\mathbf{A}\mathbf{X}^{-1}\mathrm{d}\mathbf{X}\,\mathbf{X}^{-1}) = -\operatorname{tr}(\mathbf{X}^{-1}\mathbf{A}\mathbf{X}^{-1}\mathrm{d}\mathbf{X})$, so $\boldsymbol{G} = -(\mathbf{X}^{-1}\mathbf{A}\mathbf{X}^{-1})^\top$. The quadratic form in the inverse is the special case $\mathbf{A} = \mathbf{a}\mathbf{a}^\top$, and the last identity is $\mathrm{d}(\mathbf{a}^\top\mathbf{X}\mathbf{b}) = \operatorname{tr}(\mathbf{b}\mathbf{a}^\top\mathrm{d}\mathbf{X})$. □

△ Caution: Symmetric matrix arguments

The formulas in Theorem 12.6 treat all n^2 entries of $\mathbf{X}$ as free. If $\mathbf{X} = \boldsymbol{\Sigma}$ is constrained to be symmetric, the off-diagonal entries are not independent and some authors report $2\boldsymbol{\Sigma}^{-1} - \operatorname{diag}(\boldsymbol{\Sigma}^{-1})$ for $\partial \log\det\boldsymbol{\Sigma}/\partial\boldsymbol{\Sigma}$. Both are "correct" for their respective parametrisations. For finding stationary points the distinction is harmless — the unconstrained gradient is symmetric when evaluated at symmetric matrices and it vanishes exactly when the constrained one does — but it matters if you feed the gradient to an optimiser that updates only the lower triangle. In this book we use the unconstrained convention.

12.1.5 Jacobians and Hessians of vector functions

For a vector-valued map $f:\mathbb{R}^n\to\mathbb{R}^m$ the differential is $\mathrm{d}f = Df(\mathbf{x})\,\mathrm{d}\mathbf{x}$, so the Jacobian is identified by writing $\mathrm{d}f$ as "matrix times $\mathrm{d}\mathbf{x}$". For a scalar f the Hessian is obtained from the second differential: if $\mathrm{d}^2 f = \mathrm{d}\mathbf{x}^\top \boldsymbol{M}\,\mathrm{d}\mathbf{x}$, then $\nabla^2 f = \tfrac{1}{2}(\boldsymbol{M}+\boldsymbol{M}^\top)$. Matrix-valued functions of matrices are handled by vectorisation: $\mathrm{d}\operatorname{vec}\boldsymbol{F} = \boldsymbol{J}\,\mathrm{d}\operatorname{vec}\mathbf{X}$ defines the $mn\times pq$ Jacobian $\boldsymbol{J}$, and the identity $\operatorname{vec}(\mathbf{AXB}) = (\mathbf{B}^\top\otimes\mathbf{A})\operatorname{vec}\mathbf{X}$ converts products into Kronecker form. For instance, $\mathrm{d}\operatorname{vec}(\mathbf{X}^{-1}) = -(\mathbf{X}^{-\top}\otimes\mathbf{X}^{-1})\mathrm{d}\operatorname{vec}\mathbf{X}$.

⋆ Key Concept: The three-step recipe

To differentiate a scalar function of a vector or matrix: (1) take the differential using the product, inverse and log-det rules; (2) use $a=\operatorname{tr}(a)$ and the cyclic property to bring the expression to the form $\operatorname{tr}(\boldsymbol{G}^\top\mathrm{d}\mathbf{X})$ or $\mathbf{g}^\top\mathrm{d}\mathbf{x}$; (3) identify the gradient $\boldsymbol{G}$ or $\mathbf{g}$. Then (4) *always* check shape and verify numerically. The recipe scales to log-likelihoods, neural networks and penalised objectives alike.

12.1.6 Numerical verification with finite differences

A derived gradient should never be trusted until it has been compared with a numerical approximation. For a scalar f, the forward and central differences along coordinate $\mathbf{e}_j$ are

$$\frac{f(\mathbf{x}+h\mathbf{e}_j)-f(\mathbf{x})}{h} = \frac{\partial f}{\partial x_j}+O(h), \qquad \frac{f(\mathbf{x}+h\mathbf{e}_j)-f(\mathbf{x}-h\mathbf{e}_j)}{2h} = \frac{\partial f}{\partial x_j}+O(h^2). \tag{12.9}$$

Taylor expansion to second (respectively third) order proves the truncation errors. In floating point arithmetic each evaluation of f carries a rounding error of order $\varepsilon_{\mathrm{mach}}|f|$ (Chapter 1), which the division by h amplifies. Balancing truncation $\propto h$ against rounding $\propto \varepsilon_{\mathrm{mach}}/h$ gives an optimal step $h\approx\sqrt{\varepsilon_{\mathrm{mach}}}\approx 10^{-8}$ for forward differences, and balancing h^2 against $\varepsilon_{\mathrm{mach}}/h$ gives $h\approx\varepsilon_{\mathrm{mach}}^{1/3}\approx 10^{-5}$ for central differences.

Listing 12.1 verifies every identity of Theorems 12.2 and 12.6 by central differences on random inputs.

Listing 12.1: Finite-difference check of matrix-calculus identities `ch12/matcalc_fd_check.py`

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Verify matrix-calculus identities with central finite differences.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
n = 4


def num_grad(f, X, h=1e-6):
    """Central-difference gradient of a scalar function f at array X."""
    G = np.zeros_like(X)
    for idx in np.ndindex(X.shape):
        E = np.zeros_like(X)
        E[idx] = h
        G[idx] = (f(X + E) - f(X - E)) / (2 * h)
    return G


a = rng.normal(size=n)
A = rng.normal(size=(n, n))
```

```
B = rng.normal(size=(n, n))
x = rng.normal(size=n)
X = rng.normal(size=(n, n))
M = rng.normal(size=(n, n))
S = M @ M.T + n * np.eye(n)              # symmetric positive definite
Si = np.linalg.inv(S)
Z = rng.normal(size=(10, n))             # design matrix for least squares
y = rng.normal(size=10)

cases = [
    ("a'x", lambda v: a @ v, a, x),
    ("x'Ax", lambda v: v @ A @ v, (A + A.T) @ x, x),
    ("||y - Zb||^2", lambda b: np.sum((y - Z @ b) ** 2),
     -2 * Z.T @ (y - Z @ x), x),
    ("tr(BX)", lambda W: np.trace(B @ W), B.T, X),
    ("tr(X'BX)", lambda W: np.trace(W.T @ B @ W), (B + B.T) @ X, X),
    ("log det S", lambda W: np.linalg.slogdet(W)[1], Si.T, S),
    ("tr(B S^-1)", lambda W: np.trace(B @ np.linalg.inv(W)),
     -(Si @ B @ Si).T, S),
    ("a'S^-1 a", lambda W: a @ np.linalg.solve(W, a),
     -Si.T @ np.outer(a, a) @ Si.T, S),
]

print(f"{'function':<14}{'formula':>28}{'max abs error':>16}")
formulas = ["a", "(A + A')x", "-2Z'(y - Zb)", "B'", "(B + B')X",
            "S^-T", "-(S^-1 B S^-1)'", "-S^-T aa' S^-T"]
for (name, f, g_exact, at), form in zip(cases, formulas):
    err = np.max(np.abs(num_grad(f, at) - g_exact))
    print(f"{name:<14}{form:>28}{err:>16.2e}")

# Hessian of the quadratic form: differentiate the gradient numerically
H_num = np.column_stack([
    num_grad(lambda v: v @ A @ v, x + 1e-4 * e) for e in np.eye(n)])
H_num = (H_num - np.column_stack([
    num_grad(lambda v: v @ A @ v, x - 1e-4 * e) for e in np.eye(n)])) / 2e-4
print(f"\nHessian of x'Ax vs A + A': max abs error "
      f"{np.max(np.abs(H_num - (A + A.T))):.2e}")
```

Output

```
function                          formula   max abs error
a'x                                     a        2.55e-11
x'Ax                              (A + A')x        4.21e-11
||y - Zb||^2                   -2Z'(y - Zb)        2.63e-09
tr(BX)                                  B'        3.55e-10
tr(X'BX)                          (B + B')X        1.06e-09
log det S                             S^-T        3.29e-10
tr(B S^-1)                 -(S^-1 B S^-1)'        7.63e-11
a'S^-1 a                    -S^-T aa' S^-T        1.15e-11

Hessian of x'Ax vs A + A': max abs error 3.73e-07
```

All errors are of order 10^{-9} or smaller, the size expected from rounding when $h = 10^{-6}$. A wrong formula — for example forgetting the transpose in $\partial \operatorname{tr}(\mathbf{BX})/\partial \mathbf{X}$ — gives errors of order one. The Hessian check differentiates a numerical gradient a second time, so its error ($\approx 10^{-7}$) is

larger, as expected.

The companion script `code/ch12/fd_step_jacobian.py` uses the same `num_jac` idea (columns $(f(\mathbf{x}+h\mathbf{e}_j)-f(\mathbf{x}-h\mathbf{e}_j))/2h$) to check the Jacobians of Example 12.4, and studies the error of (12.9) as h varies. Its output confirms the softmax and chain-rule Jacobians and the optimal step sizes predicted above; Fig. 12.1 shows the characteristic "V" shapes.

Output

```
softmax Jacobian error   : 3.73e-11
eigenvalues of J (PSD)   : [0.     0.186  0.2623 0.2914]
chain-rule Jacobian shape: (2, 5)  error 1.01e-10
best h forward: 4.6e-08 (error 5.4e-09)
best h central: 1.0e-05 (error 4.4e-12)
```

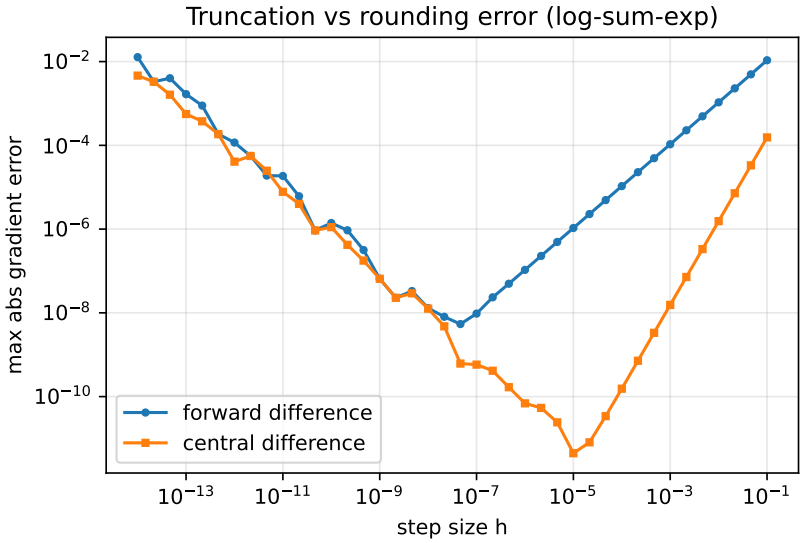


Figure 12.1: Error of forward and central finite-difference gradients of log-sum-exp as a function of the step size h. To the right of the minimum truncation error dominates (slopes 1 and 2); to the left, rounding error $\propto \varepsilon_{\text{mach}}/h$ dominates.

Historical Note: From Jacobi to Magnus and Neudecker

Carl Gustav Jacob Jacobi studied the functional determinants that now bear his name in an 1841 memoir on the subject. Otto Hesse introduced the determinant of second derivatives in his work on cubic and quartic curves in the 1840s, and Sylvester later named it the "Hessian". The systematic matrix calculus used in statistics is much younger: Jan Magnus and Heinz Neudecker developed the differential approach, with vectorisation and Kronecker products, in a series of papers in the 1980s culminating in their book, first published in 1988 and now in its third edition (Magnus and Neudecker, 2019). Their insistence that one should work with differentials, not with entrywise partial derivatives, is the approach taken in this chapter.

Graduate Perspective: Complex-step and automatic differentiation

Finite differences cost $n+1$ (forward) or $2n$ (central) function evaluations for a gradient in $\mathbb{R}^n$ and lose about half of the available digits. For analytic functions, the *complex-step* approximation $\partial f/\partial x_j \approx \operatorname{Im} f(\mathbf{x}+ih\mathbf{e}_j)/h$ has no subtractive cancellation and is accurate to machine precision for tiny h such as 10^{-20}. In practice, however, gradients in machine

learning are computed exactly (up to rounding) by automatic differentiation (Section 12.3); finite differences remain indispensable as an independent *check* of hand-written or AD gradients.

Exercises

12.1.1 Theory Prove from the definition that $\nabla_{\mathbf{x}} \|\mathbf{x}\|_2 = \mathbf{x}/\|\mathbf{x}\|_2$ for $\mathbf{x} \neq \mathbf{0}$, and that $\nabla_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 = 2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$.

12.1.2 Theory Using differentials, show that $\partial \operatorname{tr}(\mathbf{A}\mathbf{X}\mathbf{B}\mathbf{X}^\top)/\partial \mathbf{X}$ equals $\mathbf{A}^\top\mathbf{X}\mathbf{B}^\top + \mathbf{A}\mathbf{X}\mathbf{B}$, and that $\partial \det(\mathbf{X})/\partial \mathbf{X} = \det(\mathbf{X})\mathbf{X}^{-\top}$.

12.1.3 Theory Derive the Hessian of $f(\mathbf{x}) = \operatorname{lse}(\mathbf{x}) = \log \sum_j e^{x_j}$ and prove that it is positive semidefinite with a zero eigenvalue along $\mathbf{1}$. Conclude that lse is convex but not strictly convex.

12.1.4 Theory Show that $\mathrm{d}\operatorname{vec}(\mathbf{X}^{-1}) = -(\mathbf{X}^{-\top} \otimes \mathbf{X}^{-1})\,\mathrm{d}\operatorname{vec}\mathbf{X}$ and use it to write down the $n^2 \times n^2$ Jacobian of $\mathbf{X} \mapsto \mathbf{X}^{-1}$. Check your answer numerically for a random 3×3 matrix.

12.1.5 Python Extend `matcalc_fd_check.py` to verify (a) the derivative of $\log\det(\mathbf{X}^\top\mathbf{X})$ with respect to a tall $\mathbf{X}$ (answer: $2\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}$) and (b) the derivative of $\operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}\big)$. Introduce a deliberate transpose error and report the size of the discrepancy.

12.1.6 Python Implement the complex-step derivative for log-sum-exp and add it to Fig. 12.1. For which range of h is it accurate to 10^{-15}? Why can the complex step not be used for $|x|$ or $\max(x, 0)$?

12.1.7 ★ Graduate Let $\boldsymbol{\Sigma}(\boldsymbol{\theta})$ be a symmetric positive definite matrix depending smoothly on a scalar θ. Show that $\frac{\mathrm{d}}{\mathrm{d}\theta}\log\det\boldsymbol{\Sigma} = \operatorname{tr}(\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}})$ and $\frac{\mathrm{d}}{\mathrm{d}\theta}\boldsymbol{\Sigma}^{-1} = -\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}}\boldsymbol{\Sigma}^{-1}$, and derive the score of a Gaussian likelihood $\mathbf{y} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}(\theta))$, as used in Gaussian process and spatial econometric models.

12.2 Applications of Matrix Calculus

We now use the recipe to derive, in a few lines each, estimators that occupy whole chapters of econometrics texts.

12.2.1 Ordinary least squares

Let $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{X} \in \mathbb{R}^{n\times p}$ with full column rank, and $S(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$. By Theorem 12.2(e),

$$\nabla S(\boldsymbol{\beta}) = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}), \qquad \nabla^2 S(\boldsymbol{\beta}) = 2\mathbf{X}^\top\mathbf{X}.$$

Setting the gradient to zero gives the *normal equations* $\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\top\mathbf{y}$ and $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$. Because $\boldsymbol{v}^\top\mathbf{X}^\top\mathbf{X}\boldsymbol{v} = \|\mathbf{X}\boldsymbol{v}\|^2 > 0$ for $\boldsymbol{v} \neq \mathbf{0}$ when $\mathbf{X}$ has full column rank, the Hessian is positive definite, S is strictly convex, and $\hat{\boldsymbol{\beta}}$ is the unique global minimiser. The Hessian also governs precision: under homoskedastic errors $\operatorname{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1} = 2\sigma^2[\nabla^2 S]^{-1}$, so a flat objective (small Hessian eigenvalues, i.e. collinearity) means imprecise estimates — the observation that motivates ridge regression in Section 12.5.

12.2.2 MLE of the multivariate normal mean and covariance

Let $\mathbf{y}_1, \dots, \mathbf{y}_m \overset{\text{iid}}{\sim} \mathcal{N}_d(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Up to a constant, the log-likelihood is

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{m}{2}\log\det\boldsymbol{\Sigma} - \frac{1}{2}\sum_{i=1}^{m}(\mathbf{y}_i - \boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{y}_i - \boldsymbol{\mu}) = -\frac{m}{2}\log\det\boldsymbol{\Sigma} - \frac{1}{2}\operatorname{tr}\big(\boldsymbol{\Sigma}^{-1}\mathbf{W}(\boldsymbol{\mu})\big), \quad (12.10)$$

where $\mathbf{W}(\boldsymbol{\mu}) = \sum_i(\mathbf{y}_i - \boldsymbol{\mu})(\mathbf{y}_i - \boldsymbol{\mu})^\top$ is the scatter matrix (the trace trick $a = \operatorname{tr}(a)$ and cyclicity were used to collect the quadratic forms).

Theorem 12.7 (Gaussian MLE)**.** *If $m > d$ and the sample scatter matrix is nonsingular, the MLE is $\hat{\boldsymbol{\mu}} = \bar{\mathbf{y}} = \frac{1}{m}\sum_i \mathbf{y}_i$ and $\hat{\boldsymbol{\Sigma}} = \frac{1}{m}\sum_i(\mathbf{y}_i - \bar{\mathbf{y}})(\mathbf{y}_i - \bar{\mathbf{y}})^\top$.*

Proof. Mean. $\nabla_{\boldsymbol{\mu}}\ell = \boldsymbol{\Sigma}^{-1}\sum_i(\mathbf{y}_i - \boldsymbol{\mu})$ by Theorem 12.2, which vanishes iff $\boldsymbol{\mu} = \bar{\mathbf{y}}$ (for any $\boldsymbol{\Sigma}$), and $\nabla^2_{\boldsymbol{\mu}}\ell = -m\boldsymbol{\Sigma}^{-1} \prec 0$. *Covariance.* With $\mathbf{W} = \mathbf{W}(\bar{\mathbf{y}})$, Proposition 12.5 gives

$$\mathrm{d}\ell = -\frac{m}{2}\operatorname{tr}(\boldsymbol{\Sigma}^{-1}\mathrm{d}\boldsymbol{\Sigma}) + \frac{1}{2}\operatorname{tr}(\boldsymbol{\Sigma}^{-1}\mathrm{d}\boldsymbol{\Sigma}\,\boldsymbol{\Sigma}^{-1}\mathbf{W}) = \frac{1}{2}\operatorname{tr}\Big[\big(\boldsymbol{\Sigma}^{-1}\mathbf{W}\boldsymbol{\Sigma}^{-1} - m\boldsymbol{\Sigma}^{-1}\big)\mathrm{d}\boldsymbol{\Sigma}\Big],$$

so $\partial\ell/\partial\boldsymbol{\Sigma} = \frac{1}{2}(\boldsymbol{\Sigma}^{-1}\mathbf{W}\boldsymbol{\Sigma}^{-1} - m\boldsymbol{\Sigma}^{-1})$ (a symmetric matrix). It vanishes iff $\mathbf{W} = m\boldsymbol{\Sigma}$. To confirm a maximum, reparametrise by the precision $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$: $\ell = \frac{m}{2}\log\det\boldsymbol{K} - \frac{1}{2}\operatorname{tr}(\boldsymbol{K}\mathbf{W})$ is strictly concave in $\boldsymbol{K}$ (log-det is strictly concave on positive definite matrices, the trace term is linear), so the stationary point is the unique global maximiser. □

The MLE divides by m, not $m - 1$: it is biased, with $\mathbb{E}\hat{\boldsymbol{\Sigma}} = \frac{m-1}{m}\boldsymbol{\Sigma}$ (Chapter 7). Listing 12.2 confirms both derivations numerically.

Listing 12.2: OLS from its gradient; Gaussian MLE and its score ch12/ols_mvn_mle.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# OLS from the gradient; Gaussian MLE of mean and covariance, checked numerically.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# ---- OLS: simulated wage equation (log wage on schooling, experience, ...)
n, p = 500, 4
X = np.column_stack([np.ones(n), rng.normal(size=(n, p - 1))])
beta = np.array([1.5, 0.08, 0.03, -0.2])
y = X @ beta + 0.4 * rng.normal(size=n)

b_hat = np.linalg.solve(X.T @ X, X.T @ y)          # normal equations
grad = -2 * X.T @ (y - X @ b_hat)                   # gradient of RSS
H = 2 * X.T @ X                                     # Hessian of RSS
print("OLS beta_hat         :", np.round(b_hat, 4))
print("||gradient|| at b_hat:", f"{np.linalg.norm(grad):.2e}")
print("min eigenvalue of H  :", f"{np.linalg.eigvalsh(H).min():.1f} (> 0)")

# ---- Multivariate normal MLE: simulated (income, consumption, savings)
mu = np.array([2.0, 1.5, 0.5])
L = np.array([[1.0, 0, 0], [0.8, 0.5, 0], [0.2, -0.3, 0.4]])
Sigma = L @ L.T
m = 2000
Y = rng.multivariate_normal(mu, Sigma, size=m)


```

```python
def loglik(mu_, Sig_):
    R = Y - mu_
    _, logdet = np.linalg.slogdet(Sig_)
    quad = np.einsum("ij,jk,ik->", R, np.linalg.inv(Sig_), R)
    return -0.5 * (m * logdet + quad + m * 3 * np.log(2 * np.pi))


mu_hat = Y.mean(axis=0)
S_hat = (Y - mu_hat).T @ (Y - mu_hat) / m            # divide by m, not m-1
print("\nmu_hat               :", np.round(mu_hat, 4))
print("Sigma_hat equals np.cov(bias=True):",
      np.allclose(S_hat, np.cov(Y.T, bias=True)))

# analytic score w.r.t. Sigma: 0.5 * (S^-1 W S^-1 - m S^-1), W = sum r r'
Si = np.linalg.inv(S_hat)
W = (Y - mu_hat).T @ (Y - mu_hat)
score = 0.5 * (Si @ W @ Si - m * Si)
num = np.zeros((3, 3))
h = 1e-6
for i in range(3):
    for j in range(3):
        E = np.zeros((3, 3))
        E[i, j] = h
        num[i, j] = (loglik(mu_hat, S_hat + E) - loglik(mu_hat, S_hat - E)) / (2 * h)
print("max |score| at MLE (analytic):", f"{np.abs(score).max():.2e}")
print("max |score| at MLE (numeric) :", f"{np.abs(num).max():.2e}")
print("loglik at MLE vs at true    :",
      f"{loglik(mu_hat, S_hat):.2f} >= {loglik(mu, Sigma):.2f}")
```

Output

```
OLS beta_hat         : [ 1.4994  0.0473  0.0391 -0.1837]
||gradient|| at b_hat: 3.96e-13
min eigenvalue of H  : 891.4 (> 0)

mu_hat               : [2.0267 1.5244 0.5029]
Sigma_hat equals np.cov(bias=True): True
max |score| at MLE (analytic): 4.55e-12
max |score| at MLE (numeric) : 7.28e-06
loglik at MLE vs at true    : -5258.86 >= -5263.07
```

The analytic score at the MLE is zero to rounding error (10^{-12}), and the finite-difference score is zero to the accuracy of the difference quotient on a log-likelihood of size 5×10^3 (10^{-5}). As it must, the log-likelihood at the MLE exceeds that at the true parameters.

12.2.3 Logistic regression: gradient, Hessian and Newton's method

For binary $y_i \in \{0,1\}$ with $\mathbb{P}(y_i = 1 \mid \mathbf{x}_i) = \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})$ and $\sigma(t) = 1/(1+e^{-t})$, the negative log-likelihood is

$$L(\boldsymbol{\beta}) = \sum_{i=1}^{n} \Big[\log\!\big(1 + e^{\eta_i}\big) - y_i \eta_i \Big], \qquad \eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}. \tag{12.11}$$

Proposition 12.8 (Gradient and Hessian of the logit loss). *Let* $\mathbf{p} = \sigma(\mathbf{X}\boldsymbol{\beta})$ *and* $\mathbf{W} = \operatorname{diag}\big(p_i(1-p_i)\big)$.

Then

$$\nabla L(\boldsymbol{\beta}) = \mathbf{X}^\top(\boldsymbol{p} - \mathbf{y}), \qquad \nabla^2 L(\boldsymbol{\beta}) = \mathbf{X}^\top \mathbf{W} \mathbf{X} \succeq 0.$$

Proof. Since $\frac{\mathrm{d}}{\mathrm{d}\eta}\log(1+e^\eta) = \sigma(\eta)$ and $\sigma' = \sigma(1-\sigma)$, we have $\mathrm{d}L = \sum_i (p_i - y_i)\,\mathrm{d}\eta_i = (\boldsymbol{p}-\mathbf{y})^\top \mathbf{X}\,\mathrm{d}\boldsymbol{\beta}$, giving the gradient. Differentiating again, $\mathrm{d}(\boldsymbol{p}-\mathbf{y}) = \mathbf{W}\mathbf{X}\,\mathrm{d}\boldsymbol{\beta}$, so $\mathrm{d}\nabla L = \mathbf{X}^\top\mathbf{W}\mathbf{X}\,\mathrm{d}\boldsymbol{\beta}$. Positive semidefiniteness follows from $\boldsymbol{v}^\top\mathbf{X}^\top\mathbf{W}\mathbf{X}\boldsymbol{v} = \sum_i w_i(\mathbf{x}_i^\top \boldsymbol{v})^2 \geq 0$. □

The gradient has the same "design-transpose times residual" form as OLS, with $\boldsymbol{p} - \mathbf{y}$ in place of $\mathbf{X}\boldsymbol{\beta} - \mathbf{y}$. Newton's method (Chapter 8), $\boldsymbol{\beta}^{(k+1)} = \boldsymbol{\beta}^{(k)} - (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top(\boldsymbol{p}-\mathbf{y})$, can be rewritten as a weighted least-squares regression on a "working response" — hence the name *iteratively reweighted least squares* (IRLS). At the optimum, $(\mathbf{X}^\top\hat{\mathbf{W}}\mathbf{X})^{-1}$ is the inverse observed Fisher information and provides the usual standard errors.

Listing 12.3: Logistic regression by Newton–Raphson ch12/logit_newton.py

```python
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Logistic regression: analytic gradient/Hessian, Newton-Raphson, sklearn check.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(803)

# simulated loan applications: default (1) vs repay (0)
n = 2000
Z = rng.normal(size=(n, 3))          # debt ratio, income, credit history
X = np.column_stack([np.ones(n), Z])
beta_true = np.array([-1.0, 1.2, -0.8, -0.5])
prob = 1 / (1 + np.exp(-X @ beta_true))
y = rng.binomial(1, prob)


def negloglik(b):
    eta = X @ b
    return np.sum(np.logaddexp(0, eta) - y * eta)


def grad_hess(b):
    p = 1 / (1 + np.exp(-X @ b))
    g = X.T @ (p - y)                                # gradient
    H = X.T @ (X * (p * (1 - p))[:, None])           # Hessian X'WX
    return g, H


# gradient check at a random point
b0 = rng.normal(size=4) * 0.1
g0, _ = grad_hess(b0)
h = 1e-6
g_num = np.array([(negloglik(b0 + h * e) - negloglik(b0 - h * e)) / (2 * h)
                  for e in np.eye(4)])
print(f"gradient check: max abs diff = {np.abs(g0 - g_num).max():.2e}")

# Newton-Raphson (= iteratively reweighted least squares)
b = np.zeros(4)
print(f"{'iter':>4} {'neg loglik':>12} {'||grad||':>11}")
for it in range(8):
    g, H = grad_hess(b)
```

```
    print(f"{it:>4} {negloglik(b):>12.5f} {np.linalg.norm(g):>11.3e}")
    if np.linalg.norm(g) < 1e-10:
        break
    b = b - np.linalg.solve(H, g)

se = np.sqrt(np.diag(np.linalg.inv(grad_hess(b)[1])))
skl = LogisticRegression(penalty="none", tol=1e-10, max_iter=1000)
skl.fit(Z, y)
print("Newton beta :", np.round(b, 4))
print("std errors  :", np.round(se, 4))
print("sklearn     :", np.round(np.r_[skl.intercept_, skl.coef_.ravel()], 4))
```

Output

```
gradient check: max abs diff = 1.46e-07
iter   neg loglik    ||grad||
   0   1386.29436   6.104e+02
   1    979.11156   1.309e+02
   2    940.78768   2.592e+01
   3    938.60184   1.750e+00
   4    938.59081   9.486e-03
   5    938.59081   2.827e-07
   6    938.59081   4.454e-14
Newton beta : [-1.0651  1.2192 -0.7833 -0.5417]
std errors  : [0.0629 0.0698 0.0631 0.0609]
sklearn     : [-1.0651  1.2192 -0.7833 -0.5417]
```

The gradient norm falls from 10^{-2} to 10^{-7} to 10^{-14} in successive iterations: the number of correct digits roughly doubles, the signature of quadratic convergence. The estimates agree with scikit-learn's unpenalised logistic regression to four decimals.

△ Caution: Separation and the need for a penalty

If some linear combination of predictors perfectly separates the classes, the logit log-likelihood has no maximiser: $\|\hat{\boldsymbol{\beta}}\| \to \infty$ while $L \to 0$, and $\mathbf{W} \to \mathbf{0}$ makes the Hessian singular. This happens easily when p is large relative to n (and with many dummy variables in policy data). A ridge penalty $\frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ adds $\lambda\mathbf{I}$ to the Hessian, restores strict convexity and guarantees a unique finite solution — one reason why scikit-learn's `LogisticRegression` is penalised *by default*.

12.2.4 Softmax regression (multinomial logit)

With K classes, one-hot labels $\mathbf{Y} \in \{0,1\}^{n\times K}$ and coefficients $\mathbf{B} \in \mathbb{R}^{p\times K}$, let $\boldsymbol{H} = \mathbf{XB}$ and $\mathbf{P} = \operatorname{softmax}(\boldsymbol{H})$ row-wise. The cross-entropy loss is $L(\mathbf{B}) = \sum_i \left[\operatorname{lse}(\boldsymbol{\eta}_i) - \mathbf{y}_i^\top \boldsymbol{\eta}_i\right]$ where $\boldsymbol{\eta}_i^\top$ is row i of $\boldsymbol{H}$. Since $\nabla \operatorname{lse} = \mathbf{s}$,

$$\mathrm{d}L = \sum_i (\boldsymbol{p}_i - \mathbf{y}_i)^\top \mathrm{d}\boldsymbol{\eta}_i = \operatorname{tr}\big((\mathbf{P}-\mathbf{Y})^\top \mathrm{d}\boldsymbol{H}\big) = \operatorname{tr}\big((\mathbf{P}-\mathbf{Y})^\top \mathbf{X}\,\mathrm{d}\mathbf{B}\big) \implies \frac{\partial L}{\partial \mathbf{B}} = \mathbf{X}^\top(\mathbf{P}-\mathbf{Y}).$$

This is the multinomial logit of McFadden's random-utility model when utilities have i.i.d. Gumbel errors, and it is exactly the output layer of the neural network in Section 12.3. The Hessian is $\sum_i \big(\operatorname{diag}(\boldsymbol{p}_i) - \boldsymbol{p}_i\boldsymbol{p}_i^\top\big) \otimes \mathbf{x}_i\mathbf{x}_i^\top$ (in vec form), which is positive semidefinite but singular, because adding the same vector to every column of $\mathbf{B}$ leaves $\mathbf{P}$ unchanged; one normalises a reference category to zero, or adds a penalty.

12.2.5 Weighted and generalised least squares

If $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$ with $\operatorname{Var}(\boldsymbol{\varepsilon}) = \sigma^2\boldsymbol{\Omega}$ and known positive definite $\boldsymbol{\Omega}$, the GLS criterion is $(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top\boldsymbol{\Omega}^{-1}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$. Its gradient is $-2\mathbf{X}^\top\boldsymbol{\Omega}^{-1}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$ (Theorem 12.2 with the symmetric weight $\boldsymbol{\Omega}^{-1}$), so

$$\hat{\boldsymbol{\beta}}_{\mathrm{GLS}} = (\mathbf{X}^\top\boldsymbol{\Omega}^{-1}\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{\Omega}^{-1}\mathbf{y}, \qquad \operatorname{Var}(\hat{\boldsymbol{\beta}}_{\mathrm{GLS}}) = \sigma^2(\mathbf{X}^\top\boldsymbol{\Omega}^{-1}\mathbf{X})^{-1}. \tag{12.12}$$

Equivalently, with a Cholesky factor $\boldsymbol{\Omega} = \boldsymbol{L}\boldsymbol{L}^\top$, GLS is OLS on the whitened data $\boldsymbol{L}^{-1}\mathbf{y}$, $\boldsymbol{L}^{-1}\mathbf{X}$. With diagonal $\boldsymbol{\Omega}$ it is *weighted* least squares with weights $1/\omega_i$. By the Gauss–Markov theorem applied to the whitened model, GLS is the best linear unbiased estimator; OLS remains unbiased but has the larger "sandwich" variance $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{\Omega}\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\sigma^2$.

Listing 12.4: Softmax gradient check; GLS versus OLS ch12/softmax_gls.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Multinomial-logit gradient check, and GLS vs OLS under heteroskedasticity.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# ---- softmax regression: loss = -sum_i log p_{i, y_i},  grad = X'(P - Y)
n, p, K = 300, 4, 3                  # simulated choice of transport mode
X = rng.normal(size=(n, p))
W_true = rng.normal(size=(p, K))
U = X @ W_true + rng.gumbel(size=(n, K))
yc = U.argmax(axis=1)
Y = np.eye(K)[yc]                    # one-hot labels


def loss(W):
    Eta = X @ W
    lse = np.log(np.exp(Eta - Eta.max(1, keepdims=True)).sum(1)) + Eta.max(1)
    return np.sum(lse - np.sum(Y * Eta, axis=1))


def grad(W):
    Eta = X @ W
    P = np.exp(Eta - Eta.max(1, keepdims=True))
    P /= P.sum(1, keepdims=True)
    return X.T @ (P - Y)


W0 = rng.normal(size=(p, K))
G_num = np.zeros_like(W0)
for idx in np.ndindex(W0.shape):
    E = np.zeros_like(W0)
    E[idx] = 1e-6
    G_num[idx] = (loss(W0 + E) - loss(W0 - E)) / 2e-6
err = np.abs(grad(W0) - G_num).max()
print(f"softmax gradient X'(P - Y): max abs error {err:.2e}")

# ---- GLS: Var(e_i) = sigma_i^2 known; compare sampling variance with OLS
n, R = 200, 2000
x = rng.uniform(1, 10, size=n)       # e.g. firm size
Xd = np.column_stack([np.ones(n), x])
sig = 0.3 * x                        # error s.d. grows with size
```

```
w = 1 / sig**2                       # Omega^{-1} is diagonal
b_ols, b_gls = np.zeros((R, 2)), np.zeros((R, 2))
for r in range(R):
    y = Xd @ np.array([1.0, 0.5]) + sig * rng.normal(size=n)
    b_ols[r] = np.linalg.lstsq(Xd, y, rcond=None)[0]
    b_gls[r] = np.linalg.solve(Xd.T @ (w[:, None] * Xd), Xd.T @ (w * y))

V_gls = np.linalg.inv(Xd.T @ (w[:, None] * Xd))
XtXi = np.linalg.inv(Xd.T @ Xd)
V_ols = XtXi @ Xd.T @ (sig[:, None] ** 2 * Xd) @ XtXi    # sandwich
print(f"\n{'':18}{'MC var(slope)':>14}{'theory':>12}")
print(f"{'OLS':18}{b_ols[:, 1].var():>14.3e}{V_ols[1, 1]:>12.3e}")
print(f"{'GLS (= WLS)':18}{b_gls[:, 1].var():>14.3e}{V_gls[1, 1]:>12.3e}")
print(f"efficiency gain of GLS: {V_ols[1, 1] / V_gls[1, 1]:.2f}x smaller variance")
```

Output

```
softmax gradient X'(P - Y): max abs error 1.23e-07

                   MC var(slope)      theory
OLS                    2.677e-03   2.572e-03
GLS (= WLS)            1.320e-03   1.318e-03
efficiency gain of GLS: 1.95x smaller variance
```

In the simulated heteroskedastic design (error standard deviation proportional to firm size), GLS halves the sampling variance of the slope, and both Monte Carlo variances match the matrix formulas.

⋆ Key Concept: One template, many estimators

OLS, GLS, logit, multinomial logit and the output layer of a neural network share the gradient template $\nabla L = \mathbf{X}^\top(\text{fitted} - \text{observed})$, possibly weighted, and Hessians of the form $\mathbf{X}^\top\mathbf{W}\mathbf{X}$. Positive semidefiniteness of $\mathbf{W}$ makes the losses convex; the conditioning of $\mathbf{X}^\top\mathbf{W}\mathbf{X}$ determines both the speed of Newton-type algorithms and the precision of the estimates.

Policy Lens: Credit scoring, targeting and standard errors

The logit model of Listing 12.3 is the workhorse behind credit scoring by development banks and microfinance institutions, and behind the propensity scores used to evaluate programmes such as cash transfers. The Hessian $\mathbf{X}^\top\hat{\mathbf{W}}\mathbf{X}$ delivers standard errors that policy reports quote as evidence that, say, a debt ratio "significantly" predicts default. Those standard errors are valid only for the *pre-specified* model; if the predictors were chosen by searching over many candidates (Sections 12.5 and 12.6), naive standard errors overstate precision.

Exercises

12.2.1 Theory Show that the Gaussian log-likelihood (12.10), viewed as a function of the precision matrix $\boldsymbol{K} = \boldsymbol{\Sigma}^{-1}$, is strictly concave. (Hint: compute $\mathrm{d}^2 \log\det \boldsymbol{K} = -\operatorname{tr}(\boldsymbol{K}^{-1}\mathrm{d}\boldsymbol{K}\,\boldsymbol{K}^{-1}\mathrm{d}\boldsymbol{K})$.)

12.2.2 Theory Prove that one Newton step for the logit model equals the weighted least-squares regression of the working response $\boldsymbol{z} = \mathbf{X}\boldsymbol{\beta} + \mathbf{W}^{-1}(\mathbf{y} - \boldsymbol{p})$ on $\mathbf{X}$ with weights $\mathbf{W}$.

12.2.3 Theory Derive the gradient and Hessian of the Poisson regression log-likelihood $\sum_i [y_i \mathbf{x}_i^\top\boldsymbol{\beta} -$

$e^{x_i^\top \beta}]$ and show that it is concave.

12.2.4 Theory Derive (12.12) by whitening, and prove that $\mathrm{Var}(\hat\beta_{\mathrm{OLS}}) - \mathrm{Var}(\hat\beta_{\mathrm{GLS}}) \succeq 0$.

12.2.5 Python Generate data that are perfectly separated by the first predictor and run the Newton iteration of Listing 12.3. Describe what happens to $\|\beta\|$, the loss and the condition number of the Hessian. Then add a ridge penalty $\frac{\lambda}{2}\|\beta\|^2$ to both the gradient and Hessian and repeat.

12.2.6 ★ Graduate (Feasible GLS.) When Ω is unknown but $\omega_i = \exp(z_i^\top \gamma)$, propose a two-step estimator, simulate its sampling distribution, and compare its efficiency with infeasible GLS and with OLS plus heteroskedasticity-robust standard errors.

12.2.7 Policy A ministry estimates a logit model of school dropout on 40 household characteristics using 600 observations. Explain, using the Hessian, why some coefficients may be extremely large with huge standard errors, and propose two remedies.

12.3 Automatic Differentiation and Backpropagation

Hand derivation is feasible for the models of Section 12.2, but not for a network with fifty layers or a structural model solved by a fixed-point iteration. *Automatic differentiation* (AD) computes exact derivatives of any function given as a computer program, at a cost that is a small constant multiple of evaluating the function itself. It is neither symbolic differentiation (which manipulates formulas and can produce exponentially large expressions) nor numerical differentiation (which suffers the errors of Section 12.1.6).

12.3.1 Computational graphs

Any program that computes $f : \mathbb{R}^n \to \mathbb{R}^m$ from elementary operations $(+, \times, \exp, \log, \sin, \dots)$ defines a *computational graph*: a directed acyclic graph whose nodes are intermediate variables v_k and whose edges record which inputs each operation uses. For

$$f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2, \tag{12.13}$$

the graph has inputs $v_1 = x_1$, $v_2 = x_2$, intermediates $v_3 = \log v_1$, $v_4 = v_1 v_2$, $v_5 = \sin v_2$, $v_6 = v_3 + v_4$, and output $v_7 = v_6 - v_5$ (Fig. 12.2). Each elementary operation has a known local derivative, and the chain rule composes them.

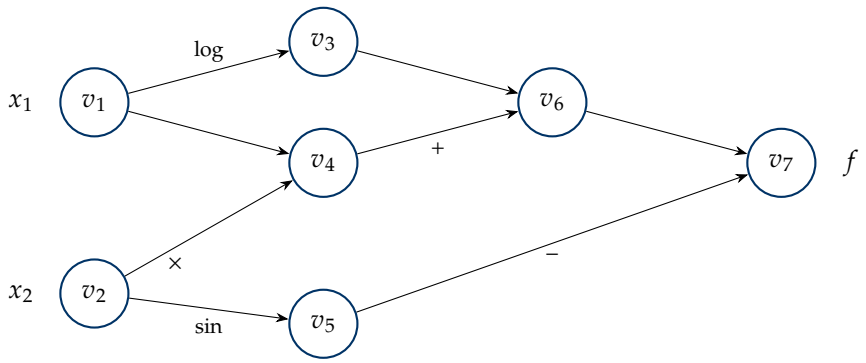


Figure 12.2: Computational graph of $f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2$. Forward mode propagates tangents $\dot v_k$ from left to right; reverse mode propagates adjoints $\bar v_k = \partial f / \partial v_k$ from right to left.

12.3.2 Forward and reverse mode

Forward mode attaches to each variable a *tangent* $\dot{v}_k = \partial v_k / \partial x_j$ for one chosen input direction and propagates it alongside the value: $\dot{v}_4 = \dot{v}_1 v_2 + v_1 \dot{v}_2$, $\dot{v}_3 = \dot{v}_1 / v_1$, and so on. Seeding $(\dot{x}_1, \dot{x}_2) = (1, 0)$ yields $\partial f / \partial x_1$ at the output. Algebraically, forward mode evaluates the Jacobian–vector product $D f(\mathbf{x})\boldsymbol{u}$ by multiplying the chain $D f_L \cdots D f_1 \boldsymbol{u}$ from the *right*. A full gradient of a scalar function of n inputs needs n forward passes.

Reverse mode first runs the program forward, storing the values v_k (the "tape"), then propagates *adjoints* $\bar{v}_k = \partial f / \partial v_k$ backwards from the seed $\bar{v}_{\text{out}} = 1$:

$$\bar{v}_k = \sum_{j \,:\, k \to j} \bar{v}_j \, \frac{\partial v_j}{\partial v_k}. \tag{12.14}$$

For (12.13): $\bar{v}_7 = 1$, $\bar{v}_6 = 1$, $\bar{v}_5 = -1$, $\bar{v}_3 = \bar{v}_4 = 1$, $\bar{v}_2 = \bar{v}_4 v_1 + \bar{v}_5 \cos v_2 = x_1 - \cos x_2$ and $\bar{v}_1 = \bar{v}_3 / v_1 + \bar{v}_4 v_2 = 1/x_1 + x_2$. Reverse mode evaluates the vector–Jacobian product $\boldsymbol{w}^\top D f(\mathbf{x})$, multiplying the chain from the *left*; one sweep gives the gradient with respect to *all* inputs.

Theorem 12.9 (Cheap gradient principle)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be evaluated by a program with cost $\operatorname{cost}(f)$ consisting of elementary operations with bounded arity. Then reverse-mode AD computes $\nabla f(\mathbf{x})$ exactly (in exact arithmetic) at cost at most $c \cdot \operatorname{cost}(f)$, where c is a small constant independent of n (about 4–5 in the analysis of Griewank and Walther, 2008). Forward mode requires cost $O(n \cdot \operatorname{cost}(f))$.*

Proof sketch. Each elementary operation $v_j = \phi(v_{k_1}, \dots, v_{k_a})$ is visited once in the forward sweep and once in the reverse sweep, where it contributes a multiply–adds $\bar{v}_{k_i} \mathrel{+}= \bar{v}_j \, \partial \phi / \partial v_{k_i}$. The local partials cost a bounded multiple of ϕ itself. Summing over operations gives a bound proportional to $\operatorname{cost}(f)$. Correctness follows by induction on the reverse topological order using the chain rule (12.14). □

Listing 12.5 implements both modes in under sixty lines of plain Python: dual numbers for forward mode, and a minimal tape with topological sorting for reverse mode.

Listing 12.5: Forward and reverse mode from scratch ch12/autodiff_tiny.py

```python
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Forward mode (dual numbers) and reverse mode (tape) automatic differentiation.
# (c) 2026 Stephen E. Moore. MIT Licence.
import math


class Dual:
    """Forward mode: carry (value, directional derivative)."""
    def __init__(self, v, d=0.0):
        self.v, self.d = v, d

    def __add__(self, o):
        return Dual(self.v + o.v, self.d + o.d)

    def __sub__(self, o):
        return Dual(self.v - o.v, self.d - o.d)

    def __mul__(self, o):
        return Dual(self.v * o.v, self.d * o.v + self.v * o.d)


def dlog(a):
```

```
    return Dual(math.log(a.v), a.d / a.v)


def dsin(a):
    return Dual(math.sin(a.v), math.cos(a.v) * a.d)


class Var:
    """Reverse mode: record parents and local partials, then sweep back."""
    def __init__(self, v, parents=()):
        self.v, self.parents, self.grad = v, parents, 0.0

    def __add__(self, o):
        return Var(self.v + o.v, ((self, 1.0), (o, 1.0)))

    def __sub__(self, o):
        return Var(self.v - o.v, ((self, 1.0), (o, -1.0)))

    def __mul__(self, o):
        return Var(self.v * o.v, ((self, o.v), (o, self.v)))

    def backward(self):
        order, seen = [], set()

        def topo(u):                        # topological sort of the graph
            if id(u) not in seen:
                seen.add(id(u))
                for par, _ in u.parents:
                    topo(par)
                order.append(u)
        topo(self)
        self.grad = 1.0                     # seed: d f / d f = 1
        for u in reversed(order):
            for par, local in u.parents:
                par.grad += local * u.grad  # adjoint accumulation


def rlog(a):
    return Var(math.log(a.v), ((a, 1.0 / a.v),))


def rsin(a):
    return Var(math.sin(a.v), ((a, math.cos(a.v)),))


# f(x1, x2) = log(x1) + x1 * x2 - sin(x2)  at (2, 5)
f_fwd = lambda a, b: dlog(a) + a * b - dsin(b)
f_rev = lambda a, b: rlog(a) + a * b - rsin(b)
x1, x2 = 2.0, 5.0

g1 = f_fwd(Dual(x1, 1.0), Dual(x2, 0.0)).d   # pass 1: seed e1
g2 = f_fwd(Dual(x1, 0.0), Dual(x2, 1.0)).d   # pass 2: seed e2
print(f"forward mode (2 passes): grad = ({g1:.6f}, {g2:.6f})")

a, b = Var(x1), Var(x2)
out = f_rev(a, b)
out.backward()                               # one reverse sweep
```

```
print(f"reverse mode (1 sweep) : grad = ({a.grad:.6f}, {b.grad:.6f})")
print(f"analytic               : grad = ({1 / x1 + x2:.6f}, "
      f"{x1 - math.cos(x2):.6f})")
print(f"function value         : f = {out.v:.6f}")
```

Output

```
forward mode (2 passes): grad = (5.500000, 1.716338)
reverse mode (1 sweep) : grad = (5.500000, 1.716338)
analytic               : grad = (5.500000, 1.716338)
function value         : f = 11.652071
```

⋆ Key Concept: Why reverse mode suits machine learning

Training a model means differentiating *one* scalar loss with respect to *many* parameters ($m = 1$, n large). Forward mode would need n passes; reverse mode needs one forward and one backward pass, whatever n is. The price is memory: the intermediate values must be stored for the backward sweep. Conversely, forward mode is preferable for functions with few inputs and many outputs, such as the sensitivity of a whole forecast path to one policy parameter.

12.3.3 Backpropagation for a two-layer network

Backpropagation is reverse-mode AD applied to a neural network. Consider a network with one hidden layer of width H for K-class classification:

$$\boldsymbol{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^{\top}, \quad \boldsymbol{A}_1 = \tanh(\boldsymbol{Z}_1), \quad \boldsymbol{Z}_2 = \boldsymbol{A}_1\mathbf{W}_2 + \mathbf{1}\mathbf{b}_2^{\top}, \quad \mathbf{Q} = \operatorname{softmax}(\boldsymbol{Z}_2), \tag{12.15}$$

with loss $L = -\frac{1}{n}\sum_{i,k} Y_{ik}\log Q_{ik} + \frac{\lambda}{2}(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$. Using the softmax result of Section 12.2 and differentials, the backward pass is

$$\begin{aligned} &\bar{\boldsymbol{Z}}_2 = \tfrac{1}{n}(\mathbf{Q} - \mathbf{Y}), \quad && \bar{\mathbf{W}}_2 = \boldsymbol{A}_1^{\top}\bar{\boldsymbol{Z}}_2 + \lambda\mathbf{W}_2, && \bar{\mathbf{b}}_2 = \bar{\boldsymbol{Z}}_2^{\top}\mathbf{1}, \\ &\bar{\boldsymbol{A}}_1 = \bar{\boldsymbol{Z}}_2\mathbf{W}_2^{\top}, && \bar{\boldsymbol{Z}}_1 = \bar{\boldsymbol{A}}_1 \odot (1 - \boldsymbol{A}_1 \odot \boldsymbol{A}_1), && \bar{\mathbf{W}}_1 = \mathbf{X}^{\top}\bar{\boldsymbol{Z}}_1 + \lambda\mathbf{W}_1, \quad \bar{\mathbf{b}}_1 = \bar{\boldsymbol{Z}}_1^{\top}\mathbf{1}. \end{aligned} \tag{12.16}$$

Each line is an application of Theorem 12.6: for $\boldsymbol{Z} = \boldsymbol{A}\mathbf{W}$, the identity $\mathrm{d}L = \operatorname{tr}(\bar{\boldsymbol{Z}}^{\top}\mathrm{d}\boldsymbol{Z}) = \operatorname{tr}(\bar{\boldsymbol{Z}}^{\top}\boldsymbol{A}\,\mathrm{d}\mathbf{W}) + \operatorname{tr}(\bar{\boldsymbol{Z}}^{\top}\mathrm{d}\boldsymbol{A}\,\mathbf{W})$ yields $\bar{\mathbf{W}} = \boldsymbol{A}^{\top}\bar{\boldsymbol{Z}}$ and $\bar{\boldsymbol{A}} = \bar{\boldsymbol{Z}}\mathbf{W}^{\top}$.

Listing 12.6 implements (12.15)–(12.16) on the handwritten-digits data bundled with scikit-learn (8×8 images, 1797 samples, 10 classes), checks the gradient by central differences, and trains by mini-batch stochastic gradient descent with momentum (Chapter 8).

Listing 12.6: A two-layer neural network with hand-coded backpropagation (plotting code omitted) ch12/backprop_mlp.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Two-layer neural network with hand-written backpropagation on sklearn digits.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(803)
X, y = load_digits(return_X_y=True)
```

```
X = X / 16.0                                  # pixel intensities in [0, 1]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=803)
Ytr = np.eye(10)[ytr]
d, H, K = X.shape[1], 32, 10


def init():
    return {"W1": rng.normal(0, 1 / np.sqrt(d), (d, H)), "b1": np.zeros(H),
            "W2": rng.normal(0, 1 / np.sqrt(H), (H, K)), "b2": np.zeros(K)}


def forward_backward(P, X, Y, lam=1e-4):
    n = X.shape[0]
    A1 = np.tanh(X @ P["W1"] + P["b1"])                # hidden layer
    Z2 = A1 @ P["W2"] + P["b2"]
    Z2 = Z2 - Z2.max(axis=1, keepdims=True)
    Q = np.exp(Z2)
    Q /= Q.sum(axis=1, keepdims=True)                  # softmax output
    loss = -np.sum(Y * np.log(Q)) / n
    loss += 0.5 * lam * (np.sum(P["W1"]**2) + np.sum(P["W2"]**2))
    # ---- backward pass: adjoints flow from the loss to every parameter
    dZ2 = (Q - Y) / n                                  # d loss / d Z2
    g = {"W2": A1.T @ dZ2 + lam * P["W2"], "b2": dZ2.sum(axis=0)}
    dA1 = dZ2 @ P["W2"].T
    dZ1 = dA1 * (1 - A1**2)                            # tanh' = 1 - tanh^2
    g["W1"] = X.T @ dZ1 + lam * P["W1"]
    g["b1"] = dZ1.sum(axis=0)
    return loss, g, Q


# ---- gradient check on a small batch
P = init()
Xb, Yb = Xtr[:20], Ytr[:20]
_, g, _ = forward_backward(P, Xb, Yb)
num_v, ana_v = [], []
for name in P:
    for idx in [tuple(rng.integers(0, s) for s in P[name].shape)
                for _ in range(5)]:
        old = P[name][idx]
        P[name][idx] = old + 1e-6
        lp = forward_backward(P, Xb, Yb)[0]
        P[name][idx] = old - 1e-6
        lm = forward_backward(P, Xb, Yb)[0]
        P[name][idx] = old
        num_v.append((lp - lm) / 2e-6)
        ana_v.append(g[name][idx])
num_v, ana_v = np.array(num_v), np.array(ana_v)
worst = np.linalg.norm(num_v - ana_v) / np.linalg.norm(num_v + ana_v)
print(f"gradient check (20 entries): relative error = {worst:.1e}")

# ---- mini-batch SGD with momentum
P = init()
V = {k: np.zeros_like(v) for k, v in P.items()}
lr, mom, epochs, bs = 0.5, 0.9, 60, 64
hist_tr, hist_te = [], []
for ep in range(epochs):
    perm = rng.permutation(len(Xtr))
```

```python
    for i in range(0, len(Xtr), bs):
        idx = perm[i:i + bs]
        _, g, _ = forward_backward(P, Xtr[idx], Ytr[idx])
        for k in P:
            V[k] = mom * V[k] - lr * g[k]
            P[k] += V[k]
    hist_tr.append(forward_backward(P, Xtr, Ytr)[0])
    hist_te.append(forward_backward(P, Xte, np.eye(10)[yte])[0])
    if ep % 10 == 0 or ep == epochs - 1:
        print(f"epoch {ep:>3}: train loss {hist_tr[-1]:.4f}  "
              f"test loss {hist_te[-1]:.4f}")

acc = lambda Xs, ys: np.mean(forward_backward(P, Xs, np.eye(10)[ys])[2]
                             .argmax(1) == ys)
print(f"accuracy: train {acc(Xtr, ytr):.3f}, test {acc(Xte, yte):.3f}")
n_par = sum(v.size for v in P.values())
print(f"parameters: {n_par}; one backward pass gives all {n_par} partials")
```

Output

```
gradient check (20 entries): relative error = 2.1e-09
epoch   0: train loss 0.3016  test loss 0.3710
epoch  10: train loss 0.0443  test loss 0.1559
epoch  20: train loss 0.0331  test loss 0.1447
epoch  30: train loss 0.0295  test loss 0.1345
epoch  40: train loss 0.0269  test loss 0.1303
epoch  50: train loss 0.0254  test loss 0.1281
epoch  59: train loss 0.0243  test loss 0.1211
accuracy: train 1.000, test 0.969
parameters: 2410; one backward pass gives all 2410 partials
```

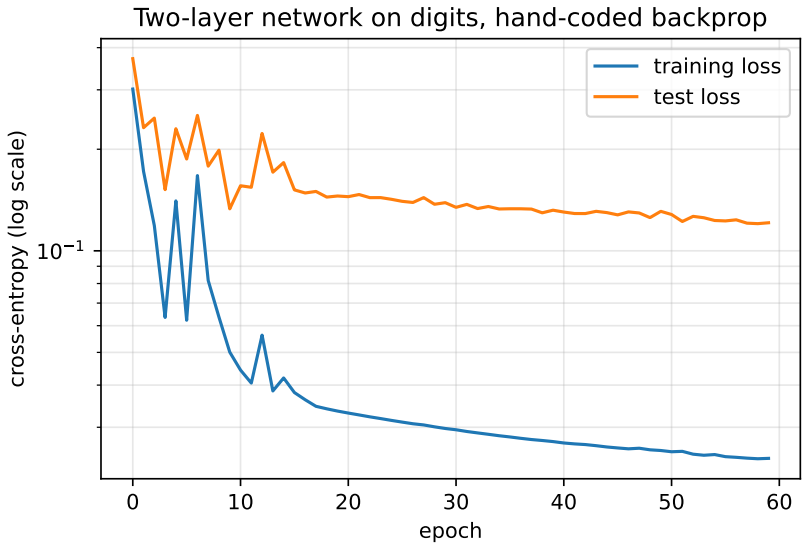


Figure 12.3: Training and test cross-entropy of the two-layer network. The widening gap between the two curves is the bias–variance trade-off of Section 12.4 in action: training loss keeps falling, test loss falls more slowly.

The relative gradient-check error of 2×10^{-9} certifies that (12.16) is correct; a single sign or

transpose error typically gives an error of order 10^{-1} or larger. The network reaches about 97% test accuracy with 2410 parameters, and its training accuracy of 100% shows the capacity to memorise — a theme of Section 12.4.

△ Caution: Gradient checking pitfalls

(i) Use double precision; in single precision central differences have only three or four reliable digits. (ii) Use the *relative* error $\|\mathbf{g}_{\text{num}} - \mathbf{g}\| / \|\mathbf{g}_{\text{num}} + \mathbf{g}\|$; absolute errors are meaningless for very small or very large gradients. (iii) Kinks (ReLU, $|\cdot|$, max-pooling) can make a correct gradient disagree with finite differences when a perturbation crosses the kink; check with smooth activations or away from kinks. (iv) Turn off randomness (dropout, data shuffling) during the check.

Historical Note: Reverse-mode differentiation and backpropagation

Forward-mode automatic differentiation was described by R. E. Wengert in 1964 (Wengert, 1964). Seppo Linnainmaa's 1970 master's thesis at the University of Helsinki described what is now called reverse-mode AD, as a way of computing the cumulative rounding error of an algorithm; the work appeared in English in Linnainmaa (1976). Related ideas arose in optimal control (adjoint methods) in the 1960s and were proposed for neural networks by Paul Werbos in the 1970s. The method became widely known in machine learning after Rumelhart et al. (1986) showed that "back-propagating errors" could train multi-layer networks to learn useful internal representations. Today, PyTorch, TensorFlow and JAX are, at their core, reverse-mode AD systems (Baydin et al., 2018).

Graduate Perspective: AD beyond neural networks

AD is now standard in econometrics and economics: Hamiltonian Monte Carlo in Stan relies on reverse-mode AD of the log posterior (Matsuura, 2022); structural models (dynamic discrete choice, DSGE) are estimated with exact gradients of likelihoods that involve fixed-point iterations, using the implicit function theorem to differentiate through the solution; and forward-over-reverse AD delivers Hessian–vector products for Newton-type optimisers at the cost of a few gradient evaluations.

Exercises

12.3.1 Theory Draw the computational graph of $f(\mathbf{x}) = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$ and write out the forward and reverse sweeps. Verify that reverse mode returns $2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$.

12.3.2 Theory For $L = f_L \circ \cdots \circ f_1$ with $f_\ell : \mathbb{R}^d \to \mathbb{R}^d$ for $\ell < L$ and $f_L : \mathbb{R}^d \to \mathbb{R}$, count the multiplications needed to form ∇L by multiplying the Jacobian chain right-to-left versus left-to-right. Relate your answer to Theorem 12.9.

12.3.3 Theory Derive (12.16) carefully with differentials, including the bias terms. Then derive the corresponding equations when tanh is replaced by $\mathrm{ReLU}(z) = \max(z, 0)$.

12.3.4 Python Add a `exp` and a `division` operation to both the `Dual` and `Var` classes of Listing 12.5 and use them to differentiate the logistic loss $\log(1 + e^{-yz})$.

12.3.5 Python Remove the weight-decay term ($\lambda = 0$) and train for 300 epochs. Plot training and test loss. At which epoch would *early stopping* on a validation set stop?

12.3.6 ★ Graduate Show that for a scalar loss, the Hessian–vector product $\nabla^2 L(\boldsymbol{\theta})\boldsymbol{v}$ equals the directional derivative of ∇L along $\boldsymbol{v}$, and explain how forward-over-reverse AD computes it at a cost independent of the number of parameters. Why is this the basis of Hessian-free and conjugate-gradient Newton methods?

12.3.7 Policy A central bank wants the sensitivity of a 24-month inflation forecast path (24 outputs) to 300 model parameters. Which AD mode would you use, and why? What if it only needed the sensitivity of one loss function (the forecast RMSE)?

12.4 The Bias–Variance Trade-off

Matrix calculus tells us how to minimise a loss; it does not tell us whether the minimiser will predict well on *new* data. This section makes precise why fitting the training data too well is harmful.

12.4.1 Decomposition of expected prediction error

Suppose $Y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\operatorname{Var}(\varepsilon) = \sigma^2$, and ε independent of the training sample $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$. A learning procedure maps $\mathcal{D}$ to a prediction function $\hat{f}_{\mathcal{D}}$. Fix a test input $\mathbf{x}_0$ and let $Y_0 = f(\mathbf{x}_0) + \varepsilon_0$ be a new response, independent of $\mathcal{D}$.

Theorem 12.10 (Bias–variance decomposition)**.** *The expected squared prediction error at $\mathbf{x}_0$, averaging over training samples and the new noise, is*

$$\mathbb{E}\big[(Y_0 - \hat{f}_{\mathcal{D}}(\mathbf{x}_0))^2\big] = \underbrace{\sigma^2}_{\text{irreducible}} + \underbrace{\big(\mathbb{E}\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - f(\mathbf{x}_0)\big)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}_{\mathcal{D}}(\mathbf{x}_0) - \mathbb{E}\hat{f}_{\mathcal{D}}(\mathbf{x}_0))^2\big]}_{\text{variance}}. \tag{12.17}$$

Proof. Write $\hat{f} = \hat{f}_{\mathcal{D}}(\mathbf{x}_0)$, $\bar{f} = \mathbb{E}\hat{f}$ and $f_0 = f(\mathbf{x}_0)$. Then $Y_0 - \hat{f} = \varepsilon_0 + (f_0 - \bar{f}) + (\bar{f} - \hat{f})$. Squaring and taking expectations, the three cross terms vanish: $\mathbb{E}[\varepsilon_0(f_0 - \bar{f})] = 0$ because $\mathbb{E}\varepsilon_0 = 0$; $\mathbb{E}[\varepsilon_0(\bar{f} - \hat{f})] = \mathbb{E}[\varepsilon_0]\mathbb{E}[\bar{f} - \hat{f}] = 0$ by independence of ε_0 and $\mathcal{D}$; and $\mathbb{E}[(f_0 - \bar{f})(\bar{f} - \hat{f})] = (f_0 - \bar{f})\mathbb{E}[\bar{f} - \hat{f}] = 0$ because $f_0 - \bar{f}$ is a constant. The three remaining squared terms are those in (12.17). □

Averaging (12.17) over the distribution of $\mathbf{x}_0$ gives the decomposition of the expected test MSE. The irreducible error σ^2 cannot be reduced by any method. Flexible methods have small bias but large variance; rigid methods the reverse. The best predictor balances the two, and it is generally *not* unbiased — a point at odds with the econometric tradition of preferring unbiased estimators, and the conceptual root of regularisation.

Example 12.11 (Linear smoothers)**.** Many estimators are *linear smoothers*: $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ for an $n \times n$ matrix $\mathbf{S}$ depending on $\mathbf{X}$ only (OLS: $\mathbf{S} = \mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$; ridge: $\mathbf{S}_\lambda$ below). With fixed design and $\operatorname{Var}(\mathbf{y}) = \sigma^2\mathbf{I}$, the in-sample average bias and variance are $\frac{1}{n}\,\|(\mathbf{S} - \mathbf{I})\boldsymbol{f}\|^2$ and $\frac{\sigma^2}{n}\operatorname{tr}(\mathbf{S}\mathbf{S}^\top)$; for OLS with p columns the variance term is exactly $\sigma^2 p/n$, growing linearly in the number of parameters.

12.4.2 A simulation: polynomial regression of increasing degree

Listing 12.7 fits polynomials of degree 0 to 10 (in a Legendre basis, for numerical stability) to 2000 independent training samples of size $n = 50$ from $f(x) = \sin(1.5\pi x) + 0.5x^2$ with $\sigma = 0.35$, and estimates each term in (12.17) by Monte Carlo (Chapter 10).

Listing 12.7: Monte Carlo bias–variance decomposition (plotting code omitted)
ch12/bias_variance_poly.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Monte Carlo bias-variance decomposition for polynomial regression.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```
import matplotlib.pyplot as plt
from numpy.polynomial import legendre

rng = np.random.default_rng(803)
f = lambda x: np.sin(1.5 * np.pi * x) + 0.5 * x**2    # true regression function
sigma, n, R = 0.35, 50, 2000
x0 = np.linspace(-0.95, 0.95, 60)                      # test inputs
degrees = np.arange(0, 11)


def design(x, deg):                                    # well-conditioned basis
    return legendre.legvander(x, deg)


xs = rng.uniform(-1, 1, (R, n))                       # R training samples
ys = f(xs) + sigma * rng.normal(size=(R, n))
y0 = f(x0) + sigma * rng.normal(size=(R, x0.size))  # fresh test responses
bias2, var, mse = [], [], []
for deg in degrees:
    Xs = design(xs, deg)                               # (R, n, deg + 1)
    coef = np.linalg.pinv(Xs) @ ys[:, :, None]         # batched least squares
    preds = (design(x0, deg) @ coef)[:, :, 0]          # (R, 60)
    bias2.append(np.mean((preds.mean(0) - f(x0)) ** 2))
    var.append(np.mean(preds.var(0)))
    mse.append(np.mean((y0 - preds) ** 2))
bias2, var, mse = map(np.array, (bias2, var, mse))

print(f"{'deg':>3}{'bias^2':>9}{'variance':>10}{'noise':>8}"
      f"{'sum':>9}{'MC test MSE':>13}")
for k in range(len(degrees)):
    print(f"{degrees[k]:>3}{bias2[k]:>9.4f}{var[k]:>10.4f}{sigma**2:>8.4f}"
          f"{bias2[k] + var[k] + sigma**2:>9.4f}{mse[k]:>13.4f}")
print("degree minimising test MSE:", degrees[np.argmin(mse)])
```

Output

```
deg   bias^2  variance   noise      sum  MC test MSE
  0   0.5021    0.0128  0.1225   0.6374       0.6356
  1   0.5033    0.0251  0.1225   0.6509       0.6488
  2   0.4829    0.0439  0.1225   0.6493       0.6471
  3   0.0763    0.0213  0.1225   0.2201       0.2202
  4   0.0743    0.0392  0.1225   0.2361       0.2361
  5   0.0018    0.0182  0.1225   0.1426       0.1421
  6   0.0017    0.0254  0.1225   0.1496       0.1490
  7   0.0000    0.0347  0.1225   0.1572       0.1570
  8   0.0001    0.0872  0.1225   0.2098       0.2102
  9   0.0003    0.5554  0.1225   0.6781       0.6788
 10   0.0009    1.7203  0.1225   1.8438       1.8392
degree minimising test MSE: 5
```

The columns “sum” and “MC test MSE” agree to Monte Carlo error, verifying the theorem. The squared bias drops sharply at degrees 3 and 5, when the polynomial becomes flexible enough to follow the sine wave, and is negligible thereafter; the variance stays small (and fluctuates, because odd and even degrees fit this function differently) up to degree 7 and then explodes (Fig. 12.4). The best degree is 5.

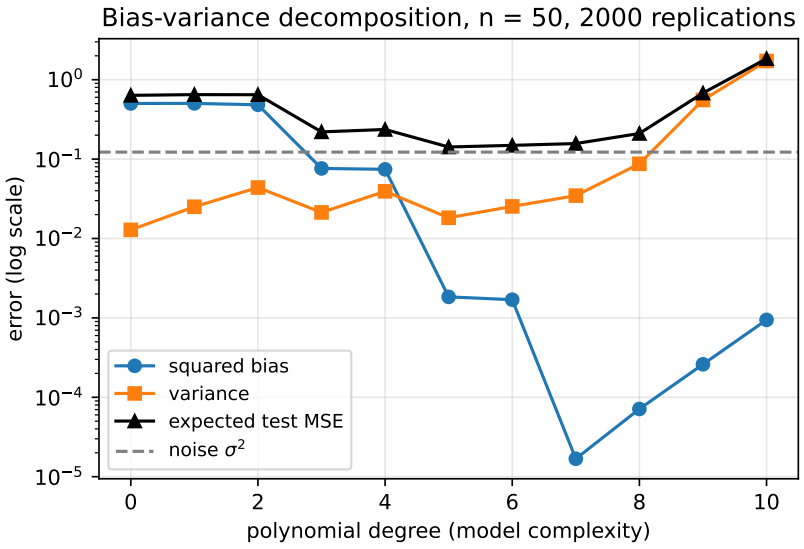


Figure 12.4: Squared bias, variance and expected test MSE (log scale) for polynomial regression, estimated from 2000 simulated training samples of size $n = 50$. The sum of the components reproduces the Monte Carlo test MSE, as Theorem 12.10 requires.

12.4.3 Under- and overfitting: training versus test error

In practice we observe one sample. The training error $\overline{\mathrm{err}} = \frac{1}{n}\sum_i (y_i - \hat f(\mathbf{x}_i))^2$ always decreases as the model grows, because the model classes are nested; the test error is U-shaped.

Proposition 12.12 (Optimism of the training error)**.** *For a linear smoother $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$ with fixed design and $\mathrm{Var}(\mathbf{y}) = \sigma^2\mathbf{I}$, let $\mathrm{Err}_{\mathrm{in}} = \frac{1}{n}\sum_i \mathbb{E}(Y_i^{\mathrm{new}} - \hat y_i)^2$ be the in-sample prediction error for new responses at the same inputs. Then*

$$\mathrm{Err}_{\mathrm{in}} = \mathbb{E}\,\overline{\mathrm{err}} + \frac{2\sigma^2}{n}\operatorname{tr}(\mathbf{S}). \tag{12.18}$$

Proof. Expanding $(Y_i^{\mathrm{new}} - \hat y_i)^2$ and $(y_i - \hat y_i)^2$ about f_i shows that the difference of their expectations is $2\,\mathrm{Cov}(y_i, \hat y_i)$. Summing, $\sum_i \mathrm{Cov}(y_i, \hat y_i) = \operatorname{tr}\mathrm{Cov}(\mathbf{y}, \mathbf{S}\mathbf{y}) = \sigma^2 \operatorname{tr}(\mathbf{S})$. □

Definition 12.13 (Effective degrees of freedom)**.** The effective degrees of freedom of a linear smoother is $\mathrm{df}(\mathbf{S}) = \operatorname{tr}(\mathbf{S})$, and more generally $\mathrm{df}(\hat f) = \frac{1}{\sigma^2}\sum_i \mathrm{Cov}(y_i, \hat y_i)$.

For OLS, $\operatorname{tr}(\mathbf{H}) = \operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{X}\big) = p$, the number of parameters. For ridge it is a continuous quantity between 0 and p (Section 12.5), which lets us compare models with different penalties on one complexity scale. Equation (12.18) with $\operatorname{tr}(\mathbf{S}) = p$ is the C_p statistic of Mallows, closely related to AIC (Section 12.6).

Listing 12.8: Training and test error; effective degrees of freedom (plotting code omitted)
ch12/train_test_curves.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Training vs test error, and effective degrees of freedom tr(H).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import legendre

```

```python
rng = np.random.default_rng(12)
f = lambda x: np.sin(1.5 * np.pi * x) + 0.5 * x**2
sigma, n = 0.35, 30
x = rng.uniform(-1, 1, n)
y = f(x) + sigma * rng.normal(size=n)
xt = rng.uniform(-1, 1, 5000)                 # large test sample
yt = f(xt) + sigma * rng.normal(size=xt.size)

degrees = np.arange(0, 16)
tr_err, te_err, edf = [], [], []
for deg in degrees:
    Xd = legendre.legvander(x, deg)
    Hat = Xd @ np.linalg.pinv(Xd)              # hat (projection) matrix
    coef = np.linalg.pinv(Xd) @ y
    tr_err.append(np.mean((y - Hat @ y) ** 2))
    te_err.append(np.mean((yt - legendre.legvander(xt, deg) @ coef) ** 2))
    edf.append(np.trace(Hat))
print(f"{'deg':>3}{'tr(H)':>7}{'train MSE':>11}{'test MSE':>10}")
for k in range(0, 16, 3):
    print(f"{degrees[k]:>3}{edf[k]:>7.2f}{tr_err[k]:>11.4f}{te_err[k]:>10.4f}")

# ridge on a degree-15 basis: edf = sum d_j^2 / (d_j^2 + lambda)
Xd = legendre.legvander(x, 15)[:, 1:]
Xc = Xd - Xd.mean(0)
d = np.linalg.svd(Xc, compute_uv=False)
print("\nridge on degree-15 basis (centred, intercept unpenalised):")
for lam in [1e-6, 1e-3, 1e-1, 10.0]:
    Hl = Xc @ np.linalg.solve(Xc.T @ Xc + lam * np.eye(15), Xc.T)
    print(f"lambda = {lam:<7g} edf = 1 + tr(H) = {1 + np.trace(Hl):6.2f}"
          f"   via SVD = {1 + np.sum(d**2 / (d**2 + lam)):6.2f}")
```

Output

```
deg  tr(H)  train MSE  test MSE
  0   1.00     0.5643    0.6536
  3   4.00     0.1577    0.2244
  6   7.00     0.0960    0.1462
  9  10.00     0.0901    0.3498
 12  13.00     0.0864    1.4599
 15  16.00     0.0547   62.1466

ridge on degree-15 basis (centred, intercept unpenalised):
lambda = 1e-06   edf = 1 + tr(H) =  15.99   via SVD =  15.99
lambda = 0.001   edf = 1 + tr(H) =  14.86   via SVD =  14.86
lambda = 0.1     edf = 1 + tr(H) =  12.08   via SVD =  12.08
lambda = 10      edf = 1 + tr(H) =   3.61   via SVD =   3.61
```

⋆ Key Concept: Bias–variance trade-off

Expected test error = noise + bias^2 + variance. Increasing flexibility lowers bias and raises variance, so test error is typically U-shaped in complexity while training error falls monotonically. Training error is an optimistic estimate of test error, by $2\sigma^2\,\mathrm{df}/n$ for linear smoothers. Model selection must therefore rest on held-out data, cross-validation or a complexity-penalised criterion — never on training fit alone.

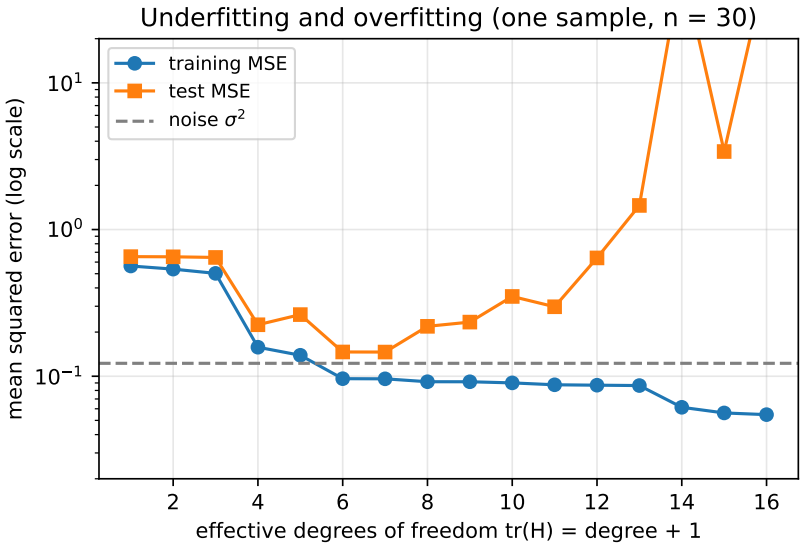


Figure 12.5: Training and test MSE against effective degrees of freedom for one simulated sample of $n = 30$. Training error falls monotonically; test error is U-shaped. Left of the minimum the model underfits (high bias); to the right it overfits (high variance).

12.4.4 A note on double descent

Modern neural networks have more parameters than observations and fit the training data exactly, yet they generalise well. Belkin et al. (2019) documented a *double-descent* curve: as capacity increases past the *interpolation threshold* (where the model can just fit the training data exactly), the test error first peaks and then *falls again*. For least squares with $p > n$ there are infinitely many interpolating solutions; the minimum-norm one, $\hat{\boldsymbol{\beta}} = \mathbf{X}^{+}\mathbf{y}$, is the limit of ridge as $\lambda \downarrow 0$ (Section 12.5) and is also what gradient descent started at zero converges to. Its norm — and therefore its variance — is largest near $p = n$, where $\mathbf{X}$ has tiny singular values, and decreases as p grows.

The script `code/ch12/double_descent.py` fits minimum-norm least squares on p random ReLU features $\max(\mathbf{x}^{\top}\mathbf{w}_j, 0)$ for $n = 40$ observations and p from 2 to 1200.

Output

```
features p =    10  (p/n =  0.25)  median test MSE =    0.3485
features p =    20  (p/n =  0.50)  median test MSE =    0.2924
features p =    30  (p/n =  0.75)  median test MSE =    0.4592
features p =    40  (p/n =  1.00)  median test MSE =   13.1010
features p =    50  (p/n =  1.25)  median test MSE =    0.9321
features p =   101  (p/n =  2.52)  median test MSE =    0.2985
features p =   416  (p/n = 10.40)  median test MSE =    0.2237
features p =  1200  (p/n = 30.00)  median test MSE =    0.2172
peak at p = 40 (interpolation threshold p = n = 40)
```

Double descent does not overturn Theorem 12.10, which is an identity; it shows that "number of parameters" is a poor measure of complexity. The minimum-norm interpolator is implicitly regularised, and Hastie et al. (2022) give a precise high-dimensional analysis for ridgeless least squares. For the moderate sample sizes typical of macroeconomics, the classical U-shape and explicit regularisation remain the relevant guide.

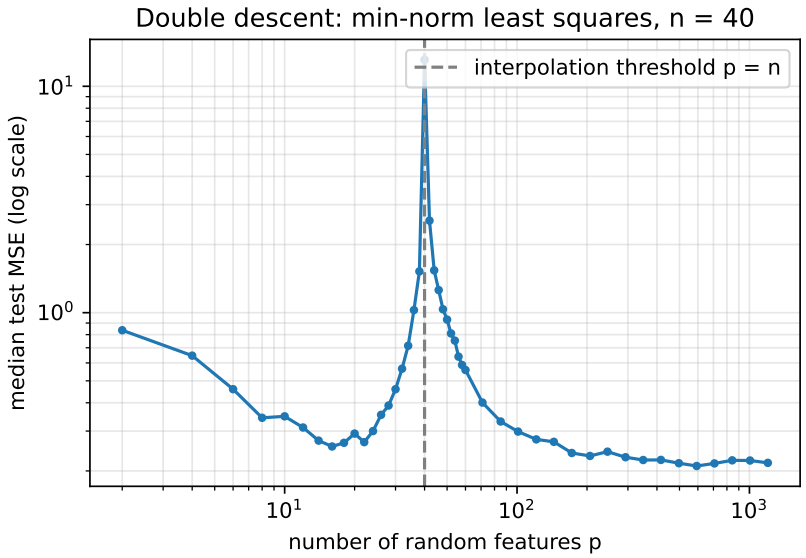


Figure 12.6: Median test MSE of minimum-norm least squares on p random ReLU features, $n = 40$ (30 replications). The error peaks at the interpolation threshold $p = n$ and descends again in the over-parametrised regime.

Historical Note: The bias–variance dilemma

The decomposition of mean squared error into squared bias plus variance is classical statistics (Chapter 7). Its central place in machine learning is due largely to Geman et al. (1992), who analysed neural networks and nonparametric estimators as a "bias/variance dilemma": reducing one component typically increases the other, and consistent nonparametric estimation requires letting model complexity grow slowly with the sample size. Belkin et al. (2019) later revisited the classical U-shaped curve in the light of over-parametrised models.

Policy Lens: Overfitting in policy scorecards

Suppose a ministry builds a proxy-means test (PMT) for targeting cash transfers by regressing household consumption on 80 asset and demographic indicators in a survey of 1500 households, and keeps the specification with the highest in-sample R^2. By (12.18), the in-sample error understates out-of-sample error by about $2\sigma^2 \cdot 80/1500$ — roughly a tenth of the noise variance — and the overfitted coefficients will misclassify more households when the scorecard is applied nationally. Exclusion and inclusion errors in targeting are bias–variance phenomena: they should be evaluated on held-out households, ideally from a later survey round, before the scorecard determines who receives benefits.

Exercises

12.4.1 Theory Extend Theorem 12.10 to the expected test error averaged over $\mathbf{x}_0 \sim P_X$ and show that the decomposition becomes $\sigma^2 + \mathbb{E}_{\mathbf{x}_0}[\mathrm{Bias}^2(\mathbf{x}_0)] + \mathbb{E}_{\mathbf{x}_0}[\mathrm{Var}(\mathbf{x}_0)]$.

12.4.2 Theory Show that $\mathrm{tr}(\mathbf{H}) = p$ for the OLS hat matrix and that $\mathrm{tr}(\mathbf{S}_k) = n/k$ for k-nearest neighbour regression with fixed design (when each point is its own neighbour).

12.4.3 Theory Prove Proposition 12.12 in detail, and deduce that $\hat{\sigma}^2 = \|\mathbf{y} - \mathbf{H}\mathbf{y}\|^2 / (n - p)$ is unbiased for OLS.

12.4.4 Theory (Bias–variance for classification.) Show by a counterexample that 0–1 loss does not admit an additive bias–variance decomposition like (12.17).

12.4.5 Python Modify Listing 12.7 to use $n = 25$ and $n = 200$. How does the optimal degree change with the sample size? Relate your answer to the history note.

12.4.6 Python Repeat `double_descent.py` replacing the minimum-norm solution with ridge, $\lambda = 10^{-2}$. What happens to the peak at $p = n$? Explain using the singular values of the feature matrix.

12.4.7 ★ Graduate For the minimum-norm interpolator $\hat{\boldsymbol{\beta}} = \mathbf{X}^{+}\mathbf{y}$ with $p > n$, Gaussian $\mathbf{X}$ and $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, derive $\mathbb{E}\left\|\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}\right\|^2$ conditional on $\mathbf{X}$ in terms of the singular values of $\mathbf{X}$. Explain why the variance term diverges as $p/n \to 1$.

12.4.8 Policy A regional office proposes to rank districts for a nutrition programme using a model with 60 indicators fitted on 70 districts, citing $R^2 = 0.97$. Write a short memo explaining why this R^2 is not evidence of predictive accuracy and what evidence you would require.

12.5 Regularisation

Regularisation deliberately accepts some bias in exchange for a larger reduction in variance. In penalised least squares we minimise

$$\frac{1}{2n}\,\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\, P(\boldsymbol{\beta}), \qquad \lambda \geq 0, \tag{12.19}$$

where the penalty P measures complexity. Throughout, we assume that the columns of $\mathbf{X}$ are *centred and standardised* and that $\mathbf{y}$ is centred, so that the intercept is estimated by $\bar{y}$ and is not penalised (see Section 12.5.6). For ridge we use the unnormalised form $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\,\|\boldsymbol{\beta}\|^2$, which is standard in the literature; the two scalings differ only by a reparametrisation of λ.

12.5.1 Ridge regression

Theorem 12.14 (Ridge regression)**.** *For any $\lambda > 0$ and any $\mathbf{X} \in \mathbb{R}^{n\times p}$ (including $p > n$), the ridge objective $R(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\,\|\boldsymbol{\beta}\|^2$ has the unique minimiser*

$$\hat{\boldsymbol{\beta}}_\lambda = (\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_p)^{-1}\mathbf{X}^\top\mathbf{y} = \mathbf{V}\,\mathrm{diag}\Big(\frac{d_j}{d_j^2 + \lambda}\Big)\mathbf{U}^\top\mathbf{y}, \tag{12.20}$$

where $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ is the thin SVD with singular values $d_1 \geq \cdots \geq d_r > 0$. The fitted values are

$$\hat{\mathbf{y}}_\lambda = \mathbf{X}\hat{\boldsymbol{\beta}}_\lambda = \sum_{j=1}^{r} \mathbf{u}_j\,\frac{d_j^2}{d_j^2 + \lambda}\,\mathbf{u}_j^\top\mathbf{y}. \tag{12.21}$$

Proof. $\nabla R = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + 2\lambda\boldsymbol{\beta}$ and $\nabla^2 R = 2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})$. For $\boldsymbol{v} \neq \mathbf{0}$, $\boldsymbol{v}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\boldsymbol{v} = \|\mathbf{X}\boldsymbol{v}\|^2 + \lambda\,\|\boldsymbol{v}\|^2 > 0$, so the Hessian is positive definite *whatever the rank of* $\mathbf{X}$; R is strictly convex, the matrix is invertible, and the stationary point is the unique minimiser. For the SVD form, complete $\mathbf{V}$ to an orthogonal $\tilde{\mathbf{V}}$; then $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I} = \tilde{\mathbf{V}}\,\mathrm{diag}(d_j^2 + \lambda, \lambda, \ldots)\tilde{\mathbf{V}}^\top$, and $\mathbf{X}^\top\mathbf{y} = \mathbf{V}\mathbf{D}\mathbf{U}^\top\mathbf{y}$ lies in the span of $\mathbf{V}$, giving (12.20). Multiplying by $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ gives (12.21). □

Equation (12.21) is the heart of ridge. OLS projects $\mathbf{y}$ onto the left singular vectors $\mathbf{u}_j$ with weight one; ridge applies the *shrinkage factor* $d_j^2/(d_j^2 + \lambda) \in (0, 1)$, shrinking most strongly along the directions of smallest variance in the predictors — precisely the directions in which OLS is

most unstable under collinearity (Chapter 3). Consequently

$$\mathrm{df}(\lambda) = \operatorname{tr}\!\big(\mathbf{X}(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^\top\big) = \sum_{j=1}^{r} \frac{d_j^2}{d_j^2+\lambda}, \tag{12.22}$$

which decreases continuously from $r = \operatorname{rank}\mathbf{X}$ at $\lambda = 0$ to 0 as $\lambda \to \infty$; the output of Listing 12.8 checks this formula. As $\lambda \downarrow 0$, $\hat{\boldsymbol{\beta}}_\lambda \to \sum_j d_j^{-1}\boldsymbol{v}_j\mathbf{u}_j^\top\mathbf{y} = \mathbf{X}^+\mathbf{y}$, the minimum-norm least-squares solution. Ridge is also OLS on augmented data: $\hat{\boldsymbol{\beta}}_\lambda = \arg\min \left\| \binom{\mathbf{y}}{\mathbf{0}} - \binom{\mathbf{X}}{\sqrt{\lambda}\mathbf{I}}\boldsymbol{\beta} \right\|^2$, which is how it is computed stably via QR.

Theorem 12.15 (Bias and variance of ridge). *With fixed* $\mathbf{X}$, $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, $\mathbb{E}\boldsymbol{\varepsilon} = \mathbf{0}$, $\operatorname{Var}\boldsymbol{\varepsilon} = \sigma^2\mathbf{I}$, *and* $\boldsymbol{A}_\lambda = (\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})^{-1}$,

$$\mathbb{E}\hat{\boldsymbol{\beta}}_\lambda - \boldsymbol{\beta} = -\lambda\boldsymbol{A}_\lambda\boldsymbol{\beta}, \qquad \operatorname{Var}(\hat{\boldsymbol{\beta}}_\lambda) = \sigma^2\boldsymbol{A}_\lambda\mathbf{X}^\top\mathbf{X}\boldsymbol{A}_\lambda.$$

If $\mathbf{X}^\top\mathbf{X}$ *is invertible,* $\operatorname{Var}(\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}) - \operatorname{Var}(\hat{\boldsymbol{\beta}}_\lambda)$ *is positive definite for every* $\lambda > 0$, *and there exists* $\lambda > 0$ *with* $\mathrm{MSE}(\hat{\boldsymbol{\beta}}_\lambda) < \mathrm{MSE}(\hat{\boldsymbol{\beta}}_{\mathrm{OLS}})$, *where* $\mathrm{MSE}(\hat{\boldsymbol{\beta}}) = \mathbb{E}\left\|\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}\right\|^2$.

Proof. $\mathbb{E}\hat{\boldsymbol{\beta}}_\lambda = \boldsymbol{A}_\lambda\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = \boldsymbol{A}_\lambda(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I}-\lambda\mathbf{I})\boldsymbol{\beta} = \boldsymbol{\beta} - \lambda\boldsymbol{A}_\lambda\boldsymbol{\beta}$, and the variance follows from $\operatorname{Var}(\boldsymbol{M}\mathbf{y}) = \sigma^2\boldsymbol{M}\boldsymbol{M}^\top$. In the eigenbasis of $\mathbf{X}^\top\mathbf{X}$ (eigenvalues d_j^2, coordinates $\alpha_j = \boldsymbol{v}_j^\top\boldsymbol{\beta}$),

$$\mathrm{MSE}(\lambda) = \sum_{j=1}^{p} \frac{\sigma^2 d_j^2 + \lambda^2\alpha_j^2}{(d_j^2+\lambda)^2}, \qquad \mathrm{MSE}'(\lambda) = \sum_{j=1}^{p} \frac{2d_j^2(\lambda\alpha_j^2 - \sigma^2)}{(d_j^2+\lambda)^3}.$$

At $\lambda = 0$ the derivative equals $-2\sigma^2\sum_j d_j^{-4} < 0$, so the MSE decreases for small $\lambda > 0$. The variance comparison follows because each eigenvalue $\sigma^2 d_j^2/(d_j^2+\lambda)^2$ is smaller than σ^2/d_j^2. □

This existence result is due to Hoerl and Kennard (1970). Listing 12.9 verifies the three equivalent formulas for $p > n$ and the bias–variance expressions by Monte Carlo with a collinear design (autocorrelation 0.9 between neighbouring predictors).

Listing 12.9: Ridge: closed form, SVD, augmented data, bias and variance ch12/ridge_svd.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Ridge: three equivalent solutions, p > n, SVD shrinkage, bias and variance.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)

# ---- p > n: X'X is singular but X'X + lambda I is not
n, p, lam = 30, 60, 2.0
X = rng.normal(size=(n, p))
y = X[:, :3] @ np.array([2.0, -1.0, 1.0]) + rng.normal(size=n)
print(f"n = {n}, p = {p}: rank(X'X) = {np.linalg.matrix_rank(X.T @ X)}")
b1 = np.linalg.solve(X.T @ X + lam * np.eye(p), X.T @ y)        # normal eq.
U, d, Vt = np.linalg.svd(X, full_matrices=False)
b2 = Vt.T @ (d / (d**2 + lam) * (U.T @ y))                      # SVD form
Xa = np.vstack([X, np.sqrt(lam) * np.eye(p)])                   # augmented
b3 = np.linalg.lstsq(Xa, np.r_[y, np.zeros(p)], rcond=None)[0]
print(f"max |normal - SVD| = {np.abs(b1 - b2).max():.1e}, "
      f"max |normal - augmented| = {np.abs(b1 - b3).max():.1e}")
b_small = Vt.T @ (d / (d**2 + 1e-10) * (U.T @ y))
```

```
print("lambda -> 0 gives the min-norm solution pinv(X)y:",
      np.allclose(b_small, np.linalg.pinv(X) @ y, atol=1e-6))
print("shrinkage factors d^2/(d^2+lambda), largest and smallest d:",
      np.round((d**2 / (d**2 + lam))[[0, -1]], 4))

# ---- bias and variance of ridge with a fixed, collinear design
n, p, sigma = 100, 10, 1.0
C = 0.9 ** np.abs(np.subtract.outer(np.arange(p), np.arange(p)))
X = rng.multivariate_normal(np.zeros(p), C, size=n)
beta = rng.normal(size=p) * 0.5
G = X.T @ X
R = 4000
Y = X @ beta + sigma * rng.normal(size=(R, n))                  # R samples
print(f"\n{'lambda':>7}{'bias^2':>9}{'trace Var':>11}{'MSE theory':>12}"
      f"{'MSE MC':>9}")
for lam in [0.0, 1.0, 5.0, 10.0, 25.0, 50.0, 100.0]:
    A = np.linalg.inv(G + lam * np.eye(p))
    bias = -lam * A @ beta                                      # E b - beta
    V = sigma**2 * A @ G @ A
    B = Y @ X @ A                                               # rows = b_hat
    mse_mc = np.mean(np.sum((B - beta) ** 2, axis=1))
    print(f"{lam:>7.1f}{bias @ bias:>9.4f}{np.trace(V):>11.4f}"
          f"{bias @ bias + np.trace(V):>12.4f}{mse_mc:>9.4f}")
```

Output

```
n = 30, p = 60: rank(X'X) = 30
max |normal - SVD| = 2.4e-15, max |normal - augmented| = 2.5e-15
lambda -> 0 gives the min-norm solution pinv(X)y: True
shrinkage factors d^2/(d^2+lambda), largest and smallest d: [0.9867 0.7293]

 lambda   bias^2  trace Var  MSE theory   MSE MC
    0.0   0.0000     0.9485      0.9485   0.9467
    1.0   0.0176     0.7185      0.7361   0.7364
    5.0   0.1857     0.3349      0.5206   0.5236
   10.0   0.3616     0.1842      0.5459   0.5491
   25.0   0.6312     0.0673      0.6985   0.7010
   50.0   0.8087     0.0282      0.8369   0.8388
  100.0   0.9562     0.0113      0.9676   0.9689
```

With $p = 60 > n = 30$, $\mathbf{X}^\top\mathbf{X}$ has rank 30, yet all three ridge formulas agree to 10^{-15}. In the collinear design, the theoretical MSE falls from 0.95 at $\lambda = 0$ (OLS) to about 0.52 at $\lambda = 5$ and then rises again as bias takes over — a bias–variance trade-off in parameter space.

Proposition 12.16 (Bayesian interpretation). *If $\mathbf{y} \mid \boldsymbol{\beta} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \sigma^2\mathbf{I})$ and $\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \tau^2\mathbf{I})$, then the posterior is Gaussian with mean (and mode) $\hat{\boldsymbol{\beta}}_\lambda$ for $\lambda = \sigma^2/\tau^2$. If instead the β_j are i.i.d. Laplace with density $\propto e^{-|\beta_j|/b}$, the posterior mode is the lasso estimate with penalty $\lambda = \sigma^2/b$ in the scaling $\frac{1}{2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$.*

Proof. The negative log posterior is, up to constants, $\frac{1}{2\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2$; multiplying by $2\sigma^2$ gives the ridge objective. Since it is a quadratic in $\boldsymbol{\beta}$, the posterior is Gaussian and its mean equals its mode. The Laplace case is identical with $\frac{1}{b}\|\boldsymbol{\beta}\|_1$. □

The penalty is therefore a prior belief that coefficients are small; λ is the ratio of noise to prior variance. The full posterior, not only the mode, is available in the Bayesian treatment of

Matsuura (2022) and Gelman et al. (2013); the Laplace-prior posterior *mean* is not sparse, which is one reason Bayesian sparsity uses other priors (spike-and-slab, horseshoe).

Historical Note: Tikhonov, Hoerl and Kennard

Regularisation was invented twice. In mathematical physics, Andrey Tikhonov studied the stability of inverse problems (Tikhonov, 1943) and in 1963 proposed stabilising ill-posed equations $\mathbf{A}\mathbf{x} = \mathbf{b}$ by minimising $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 + \lambda \|\mathbf{x}\|^2$ (Tikhonov, 1963); in numerical analysis this is still called Tikhonov regularisation. Independently, in statistics, Arthur Hoerl and Robert Kennard introduced *ridge regression* for nonorthogonal (collinear) problems in two 1970 papers in *Technometrics* (Hoerl and Kennard, 1970), showing that a positive ridge parameter always exists that lowers the mean squared error of OLS. The name echoes Hoerl's earlier "ridge analysis" of response surfaces.

12.5.2 The lasso

The *lasso* (Tibshirani, 1996) replaces the squared ℓ_2 penalty with the ℓ_1 norm:

$$\hat{\boldsymbol{\beta}}_{\lambda}^{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \; \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_1 . \tag{12.23}$$

(This is scikit-learn's scaling, with λ called `alpha`.) The objective is convex but not differentiable where some $\beta_j = 0$, so we need the *subdifferential* (Chapter 9): $\partial |b| = \{\operatorname{sign} b\}$ for $b \neq 0$ and $\partial |0| = [-1, 1]$.

Lemma 12.17 (Soft-thresholding). *For $z \in \mathbb{R}$ and $t \geq 0$, $\arg\min_b \; \tfrac{1}{2}(b - z)^2 + t\,|b| = S(z, t)$, where $S(z, t) = \operatorname{sign}(z)\max(|z| - t, 0)$.*

Proof. b^* is optimal iff $0 \in b^* - z + t\,\partial|b^*|$. If $b^* > 0$ this reads $b^* = z - t$, which requires $z > t$; similarly $b^* = z + t < 0$ requires $z < -t$. Otherwise $b^* = 0$, which is optimal iff $z \in t[-1, 1]$, i.e. $|z| \leq t$. □

Theorem 12.18 (Lasso optimality (KKT) conditions). *$\hat{\boldsymbol{\beta}}$ solves (12.23) if and only if, with $c_j = \frac{1}{n}\mathbf{x}_j^\top(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$ the correlation of predictor j with the residual,*

$$c_j = \lambda \, \operatorname{sign}(\hat{\beta}_j) \;\; \text{if } \hat{\beta}_j \neq 0, \qquad |c_j| \leq \lambda \;\; \text{if } \hat{\beta}_j = 0. \tag{12.24}$$

In particular $\hat{\boldsymbol{\beta}} = \mathbf{0}$ iff $\lambda \geq \lambda_{\max} = \max_j \left|\mathbf{x}_j^\top \mathbf{y}\right| / n$.

Proof. For a convex function, $\hat{\boldsymbol{\beta}}$ is a minimiser iff $\mathbf{0}$ belongs to the subdifferential: $\mathbf{0} \in -\frac{1}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) + \lambda\,\partial\left\|\hat{\boldsymbol{\beta}}\right\|_1$, and the subdifferential of the ℓ_1 norm is the product of the coordinatewise sets. At $\hat{\boldsymbol{\beta}} = \mathbf{0}$ the condition is $\left|\mathbf{x}_j^\top \mathbf{y}\right| / n \leq \lambda$ for all j. □

The conditions say that all active predictors have the *same* absolute correlation λ with the residual, and inactive predictors have less. This is why the lasso sets coefficients exactly to zero: a predictor stays out until its correlation with the residual reaches λ.

Coordinate descent. Minimising (12.23) over β_j with the others held fixed is a one-dimensional problem of the form in Lemma 12.17. With the partial residual $\mathbf{r}^{(j)} = \mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \beta_k$,

$$\beta_j \leftarrow \frac{S\left(\frac{1}{n}\mathbf{x}_j^\top \mathbf{r}^{(j)},\; \lambda\right)}{\frac{1}{n}\mathbf{x}_j^\top \mathbf{x}_j}. \tag{12.25}$$

Cycling through $j = 1, \ldots, p$ until convergence is *cyclic coordinate descent*. Because the non-smooth part of the objective is separable across coordinates, coordinate descent converges to a global

minimiser (Friedman et al., 2010). It is extremely fast in practice: each update costs $O(n)$, the residual can be updated in place, and along a decreasing grid of λ values each solution is a "warm start" for the next.

Listing 12.10: The lasso by coordinate descent, with a KKT check ch12/lasso_cd.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Lasso by cyclic coordinate descent with soft-thresholding; KKT check.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from sklearn.linear_model import Lasso

rng = np.random.default_rng(803)


def soft(z, t):
    return np.sign(z) * np.maximum(np.abs(z) - t, 0.0)


def lasso_cd(X, y, alpha, b=None, tol=1e-10, max_sweeps=5000):
    """Minimise (1/2n)||y - Xb||^2 + alpha ||b||_1 (sklearn's scaling)."""
    n, p = X.shape
    b = np.zeros(p) if b is None else b.copy()
    col_ss = np.sum(X**2, axis=0) / n
    r = y - X @ b                                  # current residual
    for sweep in range(max_sweeps):
        max_step = 0.0
        for j in range(p):
            old = b[j]
            rho = X[:, j] @ r / n + col_ss[j] * old  # partial-residual corr.
            b[j] = soft(rho, alpha) / col_ss[j]
            if b[j] != old:
                r -= X[:, j] * (b[j] - old)
                max_step = max(max_step, abs(b[j] - old))
        if max_step < tol:
            break
    return b, sweep + 1


# simulated targeting problem: 200 candidate indicators, 5 matter
n, p = 100, 200
X = rng.normal(size=(n, p))
X = (X - X.mean(0)) / X.std(0)
beta = np.zeros(p)
beta[[3, 17, 42, 99, 150]] = [2.0, -1.5, 1.0, 0.8, -0.6]
y = X @ beta + 0.5 * rng.normal(size=n)
y = y - y.mean()

alpha = 0.1
b, sweeps = lasso_cd(X, y, alpha)
skl = Lasso(alpha=alpha, fit_intercept=False, tol=1e-12, max_iter=100000)
skl.fit(X, y)
print(f"coordinate descent converged in {sweeps} sweeps")
print(f"max |ours - sklearn| = {np.abs(b - skl.coef_).max():.1e}")
sel = np.flatnonzero(b)
print(f"{len(sel)} selected; {np.sum(beta[sel] != 0)} of 5 true signals among them")
print(f"{'j':>5}{'b_j':>9}{'true':>7}")
```

```
for j in sel:
    print(f"{j:>5}{b[j]:>9.4f}{beta[j]:>7.1f}")

# KKT (subgradient) conditions: x_j'(y - Xb)/n = alpha*sign(b_j) if b_j != 0,
# and |x_j'(y - Xb)/n| <= alpha if b_j = 0
c = X.T @ (y - X @ b) / n
act = b != 0
print(f"active set : max |c_j - alpha sign(b_j)| = "
      f"{np.abs(c[act] - alpha * np.sign(b[act])).max():.1e}")
print(f"inactive   : max |c_j| = {np.abs(c[~act]).max():.4f} <= alpha = {alpha}")
print(f"alpha_max = max|x_j'y|/n = {np.abs(X.T @ y).max() / n:.4f} "
      "(all coefficients zero above this)")
```

Output

```
coordinate descent converged in 17 sweeps
max |ours - sklearn| = 3.5e-12
11 selected; 5 of 5 true signals among them
    j      b_j   true
    3   1.8651    2.0
   17  -1.4025   -1.5
   42   0.8707    1.0
   86   0.0002    0.0
   88   0.0176    0.0
   99   0.6340    0.8
  105   0.0297    0.0
  127   0.0068    0.0
  136  -0.0112    0.0
  150  -0.5216   -0.6
  161  -0.0083    0.0
active set : max |c_j - alpha sign(b_j)| = 3.0e-12
inactive   : max |c_j| = 0.0990 <= alpha = 0.1
alpha_max = max|x_j'y|/n = 2.1465 (all coefficients zero above this)
```

Our twenty-line implementation agrees with scikit-learn to 10^{-12} and satisfies the KKT conditions (12.24) to rounding error. With $p = 200 > n = 100$ the lasso finds all five true signals, shrinks them towards zero by roughly λ, and admits six small false positives — a typical behaviour at the prediction-optimal penalty (Section 12.6).

12.5.3 Sparsity and the geometry of the ℓ_1 ball

By Lagrangian duality (Chapter 9), the penalised problems are equivalent to constrained ones: minimise $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$ subject to $\|\boldsymbol{\beta}\|_1 \le t$ (lasso) or $\|\boldsymbol{\beta}\|_2 \le t$ (ridge), with a monotone correspondence between λ and t. The solution is where the smallest RSS ellipse touches the constraint set. The ℓ_1 ball is a cross-polytope with corners on the axes; an ellipse in general position first touches it at a corner, where some coordinates are exactly zero. The ℓ_2 ball is round, so the touching point generically has all coordinates non-zero (Fig. 12.7).

The script `code/ch12/penalty_geometry.py` solves both constrained problems for a two-coefficient example with SciPy's SLSQP.

Output

```
OLS            : [1.6 0.6]
l1-constrained : [1. 0.] <- second coefficient exactly 0
```

```
l2-constrained : [0.8399 0.5427] <- both shrunk, none zero
```

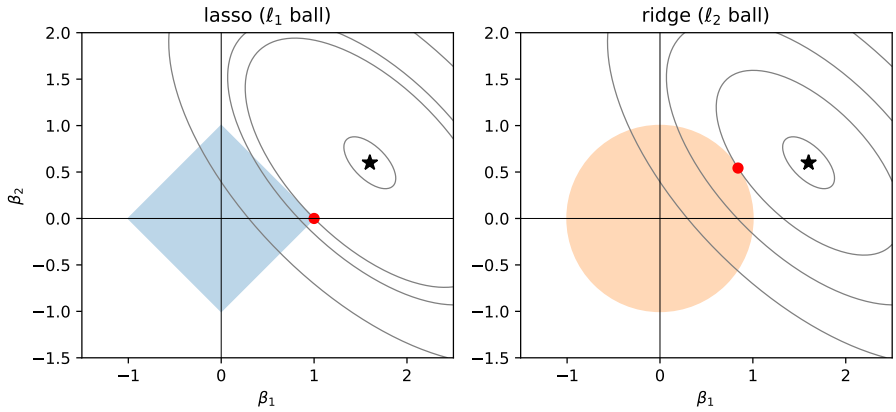


Figure 12.7: Contours of the residual sum of squares around the OLS estimate (star) and the constraint sets $\|\boldsymbol{\beta}\|_1 \le 1$ (left) and $\|\boldsymbol{\beta}\|_2 \le 1$ (right). The lasso solution (dot) sits on a corner with $\beta_2 = 0$; the ridge solution shrinks both coefficients but sets neither to zero.

Historical Note: The lasso, LARS and coordinate descent

Robert Tibshirani introduced the lasso ("least absolute shrinkage and selection operator") in 1996 (Tibshirani, 1996), building on Breiman's non-negative garrotte; the same ℓ_1 idea appeared in signal processing as *basis pursuit*. Computing it was initially slow. Efron et al. (2004) showed with least angle regression (LARS) that the whole piecewise-linear lasso path can be computed at the cost of one least-squares fit. Coordinate descent, long known in optimisation, was revived for the lasso by Friedman, Hastie, Höfling and Tibshirani in 2007 and became the engine of the `glmnet` package (Friedman et al., 2010), which made penalised GLMs routine for problems with hundreds of thousands of predictors.

12.5.4 The elastic net

When predictors are highly correlated, the lasso tends to pick one from a group somewhat arbitrarily, and with $p > n$ it can select at most n variables. Zou and Hastie (2005) proposed the *elastic net*

$$\frac{1}{2n}\,\|\mathbf{y}-\mathbf{X}\boldsymbol{\beta}\|^2 + \lambda\Big(\alpha\,\|\boldsymbol{\beta}\|_1 + \frac{1-\alpha}{2}\,\|\boldsymbol{\beta}\|_2^2\Big), \qquad 0 \le \alpha \le 1, \tag{12.26}$$

interpolating between ridge ($\alpha = 0$) and lasso ($\alpha = 1$) (in scikit-learn, λ is `alpha` and α is `l1_ratio`). The quadratic part makes the objective strictly convex and yields a *grouping effect*: if $\mathbf{x}_j = \mathbf{x}_k$, then $\hat\beta_j = \hat\beta_k$. The coordinate update is (12.25) with denominator $\frac{1}{n}\mathbf{x}_j^\top\mathbf{x}_j + \lambda(1-\alpha)$.

In `code/ch12/enet_correlated.py`, two latent drivers each have three noisy proxies, and there are four irrelevant predictors ($n = 100$); we fit `Lasso` and `ElasticNet`.

Output

```
corr within group 1: [0.989 0.993]
               x1     x2     x3     x4     x5     x6     x7     x8     x9    x10
lasso        0.00   0.06   2.62  -0.98  -0.78  -0.00  -0.00   0.00  -0.00  -0.00
enet         0.86   0.87   0.91  -0.63  -0.63  -0.62  -0.00   0.01  -0.00  -0.00
lasso : nonzero in group 1 = 2, group 2 = 2, noise = 0; group sums 2.68, -1.76
enet  : nonzero in group 1 = 3, group 2 = 3, noise = 1; group sums 2.64, -1.88
```

Three noisy proxies for each latent driver are correlated above 0.98. The lasso concentrates the first group's effect almost entirely on x_3; the elastic net spreads it evenly across x_1, x_2, x_3, which is more stable across samples and easier to interpret as "the labour-market block matters". (Printed values ±0.00 are exact zeros or coefficients below 0.005.)

12.5.5 Regularisation paths

The *regularisation path* $\lambda \mapsto \hat{\beta}_\lambda$ summarises a penalised model. For ridge the path is smooth and follows from one SVD via (12.20); for the lasso it is piecewise linear, with "knots" where variables enter or leave the active set.

Listing 12.11: Ridge and lasso paths on the diabetes data (plotting code omitted) ch12/reg_paths.py

```python
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Ridge and lasso regularisation paths on the (standardised) diabetes data.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.linear_model import lasso_path

X, y = load_diabetes(return_X_y=True)
names = load_diabetes().feature_names
X = (X - X.mean(0)) / X.std(0)            # standardise: penalty is scale-aware
y = y - y.mean()
n, p = X.shape

# ridge path via one SVD:  b(lambda) = V diag(d/(d^2+lambda)) U'y
U, d, Vt = np.linalg.svd(X, full_matrices=False)
lams = np.logspace(-2, 5, 100)
ridge = np.array([Vt.T @ (d / (d**2 + l) * (U.T @ y)) for l in lams])
edf = np.array([np.sum(d**2 / (d**2 + l)) for l in lams])

# lasso path (sklearn coordinate descent with warm starts)
alphas, coefs, _ = lasso_path(X, y, n_alphas=200, eps=1e-4)
coefs = coefs.T                            # (n_alphas, p), alphas decreasing
entry = [np.argmax(np.abs(coefs[:, j]) > 0) for j in range(p)]
order = np.argsort(entry)
print("order in which variables enter the lasso path:")
for j in order:
    print(f"  {names[j]:>4} enters at alpha = {alphas[entry[j]]:8.3f}")
print(f"ridge edf at lambda = 1e-2: {edf[0]:.2f};  at 1e5: {edf[-1]:.3f}")
print("ridge: any exact zeros along the path?", bool(np.any(ridge == 0)))
```

Output

```
order in which variables enter the lasso path:
   bmi enters at alpha =   43.118
    s5 enters at alpha =   41.167
    bp enters at alpha =   21.535
    s3 enters at alpha =   14.871
   sex enters at alpha =    6.172
    s6 enters at alpha =    4.069
    s1 enters at alpha =    3.229
    s4 enters at alpha =    0.925
    s2 enters at alpha =    0.253
```

```
   age enters at alpha =    0.242
ridge edf at lambda = 1e-2: 10.00;  at 1e5: 0.044
ridge: any exact zeros along the path? False
```

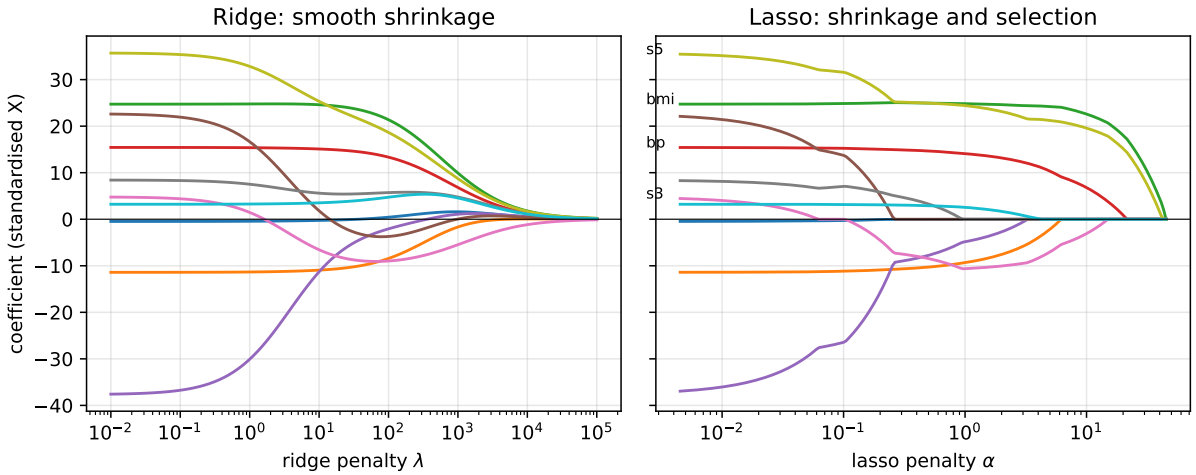


Figure 12.8: Regularisation paths for the ten standardised predictors of the diabetes data bundled with scikit-learn ($n = 442$). Ridge shrinks all coefficients smoothly towards zero; the lasso sets them to zero one by one, so reading the right panel from left to right gives an ordering of variable importance.

12.5.6 Standardisation of features

Penalties are not scale invariant. If a predictor measured in cedis is re-expressed in thousands of cedis, its OLS coefficient is multiplied by 1000 and the fit is unchanged; but its ridge or lasso penalty $\lambda\beta_j^2$ or $\lambda\left|\beta_j\right|$ changes dramatically, so the variable is effectively penalised less. Standard practice is therefore: (i) centre $\mathbf{y}$ and every column of $\mathbf{X}$ (so the intercept is $\bar{y}$ and unpenalised); (ii) scale each column to unit standard deviation; (iii) fit; (iv) transform the coefficients back to original units if required. Dummy variables are usually standardised too, although this is a modelling choice.

△ Caution: Standardise inside the resampling loop

The means and standard deviations used for standardisation are themselves estimated from data. In cross-validation or forecasting they must be computed on the training part only and then applied to the validation or test part. Standardising the full data set first leaks information from the test set into training — a mild leak for scaling, but the same mistake with variable screening can produce spectacularly optimistic results. scikit-learn's `Pipeline` enforces the correct order automatically (Section 12.6).

⋆ Key Concept: Ridge versus lasso

Ridge (ℓ_2) shrinks along the principal directions of $\mathbf{X}$ by factors $d_j^2/(d_j^2+\lambda)$, keeps every predictor, has a closed form, and handles collinearity gracefully. Lasso (ℓ_1) soft-thresholds, producing sparse models via the KKT condition $\left|c_j\right| \le \lambda$, and requires iterative algorithms such as coordinate descent. The elastic net combines sparsity with grouping of correlated

predictors. All three require standardised predictors and a tuning parameter chosen out of sample.

Policy Lens: Variable selection for targeting models

Proxy-means tests and early-warning systems often start from hundreds of candidate indicators. The lasso offers a transparent way to shortlist a few variables that are cheap to verify (roof material, livestock, household size), and the selected model is easy to publish as a scorecard. Two cautions apply. First, when indicators are strongly correlated, *which* one the lasso picks can change between survey rounds; the elastic net, or reporting groups of substitutable indicators, gives more stable and defensible scorecards. Second, lasso coefficients are shrunk and selected for prediction, not for causal interpretation; a zero coefficient does not mean that a variable does not matter for welfare (Mullainathan and Spiess, 2017).

Graduate Perspective: Inference after selection and high-dimensional theory

Under sparsity and "restricted eigenvalue" conditions on $\mathbf{X}$, the lasso with $\lambda \asymp \sigma\sqrt{\log p/n}$ achieves prediction error of order $s\,\sigma^2 \log p/n$, where s is the number of non-zero coefficients, and recovers the support when signals are strong enough (Hastie et al., 2015; Vershynin, 2018). Naive confidence intervals after lasso selection are invalid, however. For causal effects in the presence of many controls, *post-double-selection* (Belloni et al., 2014) and double/debiased machine learning restore valid inference. In macroeconomic data, sparsity itself is questionable: Giannone et al. (2021) find that the posterior often favours dense models, an "illusion of sparsity".

Exercises

12.5.1 Theory Prove that $\left\|\hat{\boldsymbol{\beta}}_\lambda\right\|_2$ is a non-increasing function of λ for ridge, using the SVD form (12.20), and show that $\hat{\boldsymbol{\beta}}_\lambda = \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I}_n)^{-1}\mathbf{y}$. Why is the second form cheaper when $p \gg n$?

12.5.2 Theory For an orthonormal design ($\mathbf{X}^\top\mathbf{X} = n\mathbf{I}$), show that the lasso solution is $\hat{\beta}_j = S(\hat{\beta}_j^{\mathrm{OLS}}, \lambda)$, ridge (in the scaling $\frac{1}{2n}\|\cdot\|^2 + \frac{\lambda}{2}\|\cdot\|^2$) is $\hat{\beta}_j^{\mathrm{OLS}}/(1+\lambda)$, and best subset selection is hard thresholding. Sketch the three functions.

12.5.3 Theory Prove the grouping effect of the elastic net: if $\mathbf{x}_j = \mathbf{x}_k$ and $\alpha < 1$, then $\hat{\beta}_j = \hat{\beta}_k$. Show by example that this fails for the lasso.

12.5.4 Theory Show that the lasso path is piecewise linear in λ. (Hint: on an interval where the active set $\mathcal{A}$ and signs $\boldsymbol{s}$ are fixed, solve (12.24) for $\hat{\boldsymbol{\beta}}_{\mathcal{A}}$.)

12.5.5 Python Extend `lasso_cd` in Listing 12.10 to the elastic net and to a decreasing grid of 50 penalties with warm starts. Compare the number of sweeps with and without warm starts.

12.5.6 Python Re-run Listing 12.11 *without* standardising $\mathbf{X}$, after multiplying the `bmi` column by 100. How does the order in which variables enter change? Explain.

12.5.7 ★ Graduate Derive the posterior distribution of $\boldsymbol{\beta}$ in Proposition 12.16 and show that its covariance is $\sigma^2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}$. Compare with the frequentist variance in Theorem 12.15 and explain the difference.

12.6 Model Selection and Forecasting with Many Predictors

Every regularised method has a tuning parameter. This section shows how to choose it honestly, and then compares methods in a forecasting exercise designed to mimic a data-rich policy environment.

12.6.1 K-fold cross-validation

Definition 12.19 (K-fold cross-validation)**.** Partition $\{1,\dots,n\}$ into K folds $F_1,\dots,F_K$ of roughly equal size. For each λ on a grid and each k, fit the model on all data except F_k, obtaining $\hat f^{(-k)}_\lambda$, and compute the fold error $E_k(\lambda)=\frac{1}{|F_k|}\sum_{i\in F_k}(y_i-\hat f^{(-k)}_\lambda(\mathbf{x}_i))^2$. The CV estimate is $\mathrm{CV}(\lambda)=\frac{1}{K}\sum_k E_k(\lambda)$, with standard error $\mathrm{se}(\lambda)=\mathrm{sd}_k\{E_k(\lambda)\}/\sqrt{K}$.

Choosing $\hat\lambda_{\min}=\arg\min \mathrm{CV}(\lambda)$ targets prediction error. The *one-standard-error rule* (Hastie et al., 2009) chooses instead the most heavily penalised model whose CV error is within one standard error of the minimum, $\hat\lambda_{1\mathrm{se}}=\max\{\lambda:\mathrm{CV}(\lambda)\le \mathrm{CV}(\hat\lambda_{\min})+\mathrm{se}(\hat\lambda_{\min})\}$. Since the CV curve is itself noisy and often flat near its minimum, this yields a simpler model whose predictive performance is statistically indistinguishable from the best. Typical choices are $K=5$ or 10; leave-one-out ($K=n$) is nearly unbiased but has high variance and is expensive except for linear smoothers, where $\mathrm{CV}_{\mathrm{LOO}}=\frac{1}{n}\sum_i\big(\frac{y_i-\hat y_i}{1-S_{ii}}\big)^2$ exactly — the trick that makes `RidgeCV` so fast.

Listing 12.12: 10-fold cross-validation for the lasso, with the one-SE rule (plotting code omitted)
ch12/cv_lambda.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# K-fold cross-validation for the lasso penalty, with the one-standard-error rule.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso, LassoCV
from sklearn.model_selection import KFold

rng = np.random.default_rng(803)
n, p = 150, 60                              # simulated district-level panel
X = rng.normal(size=(n, p))
beta = np.zeros(p)
beta[:8] = [1.5, -1.2, 1.0, 0.8, -0.6, 0.5, 0.4, -0.3]
y = X @ beta + 1.0 * rng.normal(size=n)

alphas = np.logspace(0, -3, 40)
K = 10
folds = KFold(n_splits=K, shuffle=True, random_state=803)
cv_err = np.zeros((K, alphas.size))
for k, (tr, va) in enumerate(folds.split(X)):
    mu, sd = X[tr].mean(0), X[tr].std(0)   # standardise using training fold
    Xtr, Xva = (X[tr] - mu) / sd, (X[va] - mu) / sd
    for a, alpha in enumerate(alphas):
        m = Lasso(alpha=alpha, max_iter=20000).fit(Xtr, y[tr])
        cv_err[k, a] = np.mean((y[va] - m.predict(Xva)) ** 2)
cv_mean = cv_err.mean(0)
cv_se = cv_err.std(0, ddof=1) / np.sqrt(K)
i_min = np.argmin(cv_mean)
ok = cv_mean <= cv_mean[i_min] + cv_se[i_min]
```

```
i_1se = np.flatnonzero(ok)[0]              # largest alpha within one SE
Xs = (X - X.mean(0)) / X.std(0)
for lab, i in [("alpha_min", i_min), ("alpha_1se", i_1se)]:
    m = Lasso(alpha=alphas[i], max_iter=20000).fit(Xs, y)
    print(f"{lab}: {alphas[i]:.4f}  CV MSE = {cv_mean[i]:.3f} "
          f"(se {cv_se[i]:.3f})  nonzero = {np.sum(m.coef_ != 0)}, "
          f"true found = {np.sum(m.coef_[:8] != 0)}/8")
auto = LassoCV(alphas=alphas, cv=folds, max_iter=20000).fit(Xs, y)
print(f"LassoCV alpha = {auto.alpha_:.4f} (same folds; standardised once)")
```

Output

```
alpha_min: 0.0702  CV MSE = 1.088 (se 0.098)  nonzero = 28, true found = 8/8
alpha_1se: 0.1425  CV MSE = 1.150 (se 0.118)  nonzero = 11, true found = 8/8
LassoCV alpha = 0.0702 (same folds; standardised once)
```

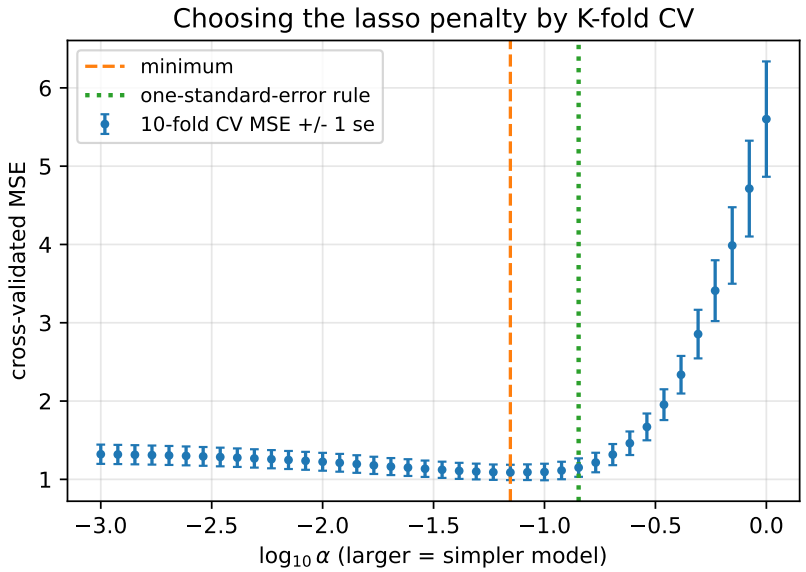


Figure 12.9: Ten-fold CV error (± one standard error) along the lasso path for a simulated problem with $n = 150$, $p = 60$ and eight true signals. The one-standard-error rule picks a sparser model (11 rather than 28 non-zero coefficients) with nearly the same CV error.

Both choices find all eight true signals; the one-SE model drops most of the twenty false positives of $\hat{\lambda}_{\min}$, which is valuable when the selected variables will be reported or collected.

12.6.2 Information criteria: AIC and BIC

For a model with k estimated parameters and maximised log-likelihood $\hat{\ell}$,

$$\mathrm{AIC} = -2\hat{\ell} + 2k, \qquad \mathrm{BIC} = -2\hat{\ell} + k \log n. \tag{12.27}$$

AIC estimates the expected Kullback–Leibler divergence between the fitted and true models (it is asymptotically equivalent to leave-one-out CV and, for Gaussian regression, to Mallows' C_p of (12.18)); it targets prediction. BIC approximates $-2\log$ of the marginal likelihood (Chapter 7); its heavier penalty is consistent for selecting the true model when it is in the candidate set and fixed in dimension. For the lasso, the number of non-zero coefficients is an unbiased estimate of

the degrees of freedom (Zou et al., 2007), so (12.27) can be evaluated along the path at no extra cost (`LassoLarsIC`).

Historical Note: Akaike and Schwarz

Hirotugu Akaike presented his information criterion at the Second International Symposium on Information Theory in Tsahkadsor (Armenia), published in 1973 (Akaike, 1973), and in the widely cited paper Akaike (1974). He originally called it "an information criterion"; the "A" was later read as "Akaike". Gideon Schwarz derived the Bayesian criterion in a four-page paper in the *Annals of Statistics* (Schwarz, 1978). Cross-validation as a principle for choosing and assessing predictors was formalised by Stone (1974).

12.6.3 Pipelines in scikit-learn

A `Pipeline` chains preprocessing and estimation so that every fit — including each fit inside cross-validation — learns the scaling from training data only. Listing 12.13 expands the ten diabetes predictors to 65 by adding squares and interactions and compares OLS, `RidgeCV` (efficient leave-one-out), `LassoCV` (10-fold) and the lasso tuned by AIC and BIC.

Listing 12.13: Pipelines: StandardScaler with RidgeCV, LassoCV and AIC/BIC `ch12/pipelines_ic.py`

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# sklearn Pipelines (StandardScaler + RidgeCV / LassoCV) and AIC/BIC selection.
# (c) 2026 Stephen E. Moore. MIT Licence.
import warnings
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LassoCV, LassoLarsIC, LinearRegression, RidgeCV
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

warnings.filterwarnings("ignore")        # LARS early-stopping notices
X, y = load_diabetes(return_X_y=True)
# add squares and interactions: 10 -> 65 predictors, a "many predictors" setting
X = PolynomialFeatures(degree=2, include_bias=False).fit_transform(X)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=803)
print(f"training n = {len(ytr)}, predictors p = {X.shape[1]}")

models = {
    "OLS": make_pipeline(StandardScaler(), LinearRegression()),
    "RidgeCV": make_pipeline(StandardScaler(),
                             RidgeCV(alphas=np.logspace(-2, 4, 60))),
    "LassoCV": make_pipeline(StandardScaler(),
                             LassoCV(cv=10, random_state=803, max_iter=50000)),
    "Lasso-AIC": make_pipeline(StandardScaler(), LassoLarsIC(criterion="aic",
                                                             normalize=False)),
    "Lasso-BIC": make_pipeline(StandardScaler(), LassoLarsIC(criterion="bic",
                                                             normalize=False)),
}
print(f"{'model':<10}{'penalty':>9}{'nonzero':>9}{'CV RMSE':>9}{'test RMSE':>11}")
for name, pipe in models.items():
    cv = -cross_val_score(pipe, Xtr, ytr, cv=5,
                          scoring="neg_root_mean_squared_error").mean()
    pipe.fit(Xtr, ytr)
    est = pipe[-1]
    pen = getattr(est, "alpha_", np.nan)
```

```
    nz = np.sum(np.abs(est.coef_) > 1e-10)
    rmse = np.sqrt(np.mean((yte - pipe.predict(Xte)) ** 2))
    print(f"{name:<10}{pen:>9.3f}{nz:>9d}{cv:>9.2f}{rmse:>11.2f}")
```

Output

```
training n = 309, predictors p = 65
model       penalty  nonzero  CV RMSE  test RMSE
OLS             nan       65    65.82      60.14
RidgeCV     116.895       65    56.63      56.15
LassoCV       2.906       19    55.11      54.08
Lasso-AIC     2.961       16    54.98      54.06
Lasso-BIC     5.243       10    56.79      54.29
```

OLS on 65 predictors with $n = 309$ overfits (test RMSE 60.0). All regularised methods improve on it by 6–10%; the lasso variants use 10–19 predictors, and BIC, as expected, picks the sparsest model. Note that the AIC and CV choices are close, consistent with their asymptotic equivalence.

△ Caution: Time series are not exchangeable

Random K-fold CV assumes the observations are exchangeable. With serially dependent data it can be optimistic, and it uses "future" data to predict the "past". For forecasting, the gold standard is *pseudo-out-of-sample* evaluation: re-estimate the model using only data available at each forecast origin, forecast, and record the error. Interestingly, for tuning the hyperparameters of predictive models (as opposed to evaluating them), Goulet Coulombe et al. (2022) find that standard K-fold CV performs well in macroeconomic applications; the final evaluation should nevertheless always be out of sample.

12.6.4 Forecasting with many predictors: a pseudo-out-of-sample exercise

Central banks and finance ministries monitor hundreds of monthly series — prices, money and credit, exchange rates, commodity prices, surveys, trade — that co-move strongly. A convenient model of such a panel is the *approximate factor model*

$$\mathbf{x}_t = \boldsymbol{\Lambda} \boldsymbol{F}_t + \mathbf{e}_t, \qquad y_{t+h} = \boldsymbol{\gamma}^\top \boldsymbol{F}_t + \boldsymbol{\delta}^\top \mathbf{x}_t + u_{t+h}, \tag{12.28}$$

with a few latent factors $\boldsymbol{F}_t \in \mathbb{R}^r$ ($r \ll N$) driving N indicators. Under (12.28), principal components of the standardised panel consistently estimate the factor space as $N, T \to \infty$ (Stock and Watson, 2002a; Bai and Ng, 2002), and *principal components regression* (PCR, the "diffusion index" forecast) regresses y_{t+h} on the first k components. Ridge regression is closely related: by (12.21) it shrinks the low-variance principal directions smoothly instead of truncating them, and De Mol et al. (2008) show that ridge (Bayesian shrinkage) and PCR forecasts are highly correlated in large macro panels. The lasso instead bets on a few individual indicators.

Listing 12.14 simulates a monthly panel of $N = 100$ indicators driven by $r = 3$ persistent factors, and a target ("inflation") that depends one month ahead on the factors and on two specific indicators. All numbers are simulated; they illustrate the methodology and are not statistics of any economy. Starting with 120 months of data, it repeatedly (i) estimates five models on the expanding window of data available at origin t, (ii) forecasts y_{t+1}, and (iii) records the error: an AR(1) benchmark; OLS on all 101 predictors; ridge and lasso with penalties re-tuned every 12 months by cross-validation; and PCR with three components. Standardisation is done inside each pipeline.

Listing 12.14: Expanding-window forecasting: OLS, ridge, lasso, PCR (plotting code omitted)
ch12/forecast_factor.py

```
# DEP 803 -- Chapter 12: Matrix Calculus, Regularisation and Bias-Variance
# Pseudo-out-of-sample forecasting with many predictors: OLS, ridge, lasso, PCR.
# (c) 2026 Stephen E. Moore. MIT Licence.
import warnings
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.linear_model import (Lasso, LassoCV, LinearRegression, Ridge,
                                  RidgeCV)
from sklearn.model_selection import KFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
rng = np.random.default_rng(803)
# ---- simulated monthly macro panel: 3 AR(1) factors drive 100 indicators
T, N, r = 300, 100, 3
F = np.zeros((T, r))
for t in range(1, T):
    F[t] = np.array([0.8, 0.6, 0.4]) * F[t - 1] + rng.normal(size=r)
Lam = rng.normal(size=(N, r))
Xall = F @ Lam.T + 1.0 * rng.normal(size=(T, N))      # indicators
# target ("inflation", h = 1 month ahead) depends on factors + 2 indicators
y = np.zeros(T)
y[1:] = (F[:-1] @ np.array([0.6, -0.4, 0.3]) + 0.3 * Xall[:-1, 0]
         + 0.2 * Xall[:-1, 1] + 0.8 * rng.normal(size=T - 1))
# predictors dated t: indicators and own lag; target dated t + 1
Z = np.column_stack([Xall[:-1], y[:-1]])
target = y[1:]

cv = KFold(5, shuffle=True, random_state=803)
ols = lambda: LinearRegression()
pcr = lambda: make_pipeline(StandardScaler(), PCA(3), LinearRegression())
ridge_cv = lambda: make_pipeline(StandardScaler(),
                                 RidgeCV(alphas=np.logspace(-1, 4, 30)))
lasso_cv = lambda: make_pipeline(StandardScaler(), LassoCV(
    n_alphas=25, cv=cv, max_iter=5000, tol=1e-3))
start, retune = 120, 12                 # first origin; re-tune yearly
err = {m: [] for m in ["AR(1)", "OLS-all", "Ridge", "Lasso", "PCR(3)"]}
for t in range(start, len(target)):     # expanding window: data up to t
    Zt, yt, znew = Z[:t], target[:t], Z[t:t + 1]
    if (t - start) % retune == 0:       # cross-validate the penalties
        lam_r = ridge_cv().fit(Zt, yt)[-1].alpha_
        lam_l = lasso_cv().fit(Zt, yt)[-1].alpha_
    fits = {"AR(1)": ols().fit(Zt[:, [-1]], yt).predict(znew[:, [-1]]),
            "OLS-all": ols().fit(Zt, yt).predict(znew),
            "Ridge": make_pipeline(StandardScaler(), Ridge(lam_r))
            .fit(Zt, yt).predict(znew),
            "Lasso": make_pipeline(StandardScaler(), Lasso(lam_l))
            .fit(Zt, yt).predict(znew),
            "PCR(3)": pcr().fit(Zt, yt).predict(znew)}
    for m, f_hat in fits.items():
        err[m].append(target[t] - f_hat[0])
err = {m: np.array(e) for m, e in err.items()}
```

```
rmse_ar = np.sqrt(np.mean(err["AR(1)"] ** 2))
print(f"{len(err['AR(1)'])} one-step forecasts, origins t = {start}..."
      f"{len(target) - 1}, p = {Z.shape[1]} predictors")
print(f"last tuned penalties: ridge {lam_r:.2f}, lasso {lam_l:.4f}")
print(f"{'model':<9}{'RMSE':>8}{'rel. to AR':>12}")
for m, e in err.items():
    rm = np.sqrt(np.mean(e ** 2))
    print(f"{m:<9}{rm:>8.3f}{rm / rmse_ar:>12.3f}")
```

Output

```
179 one-step forecasts, origins t = 120...298, p = 101 predictors
last tuned penalties: ridge 280.72, lasso 0.0418
model        RMSE  rel. to AR
AR(1)       1.239       1.000
OLS-all     1.212       0.978
Ridge       0.842       0.679
Lasso       0.825       0.666
PCR(3)      0.840       0.678
```

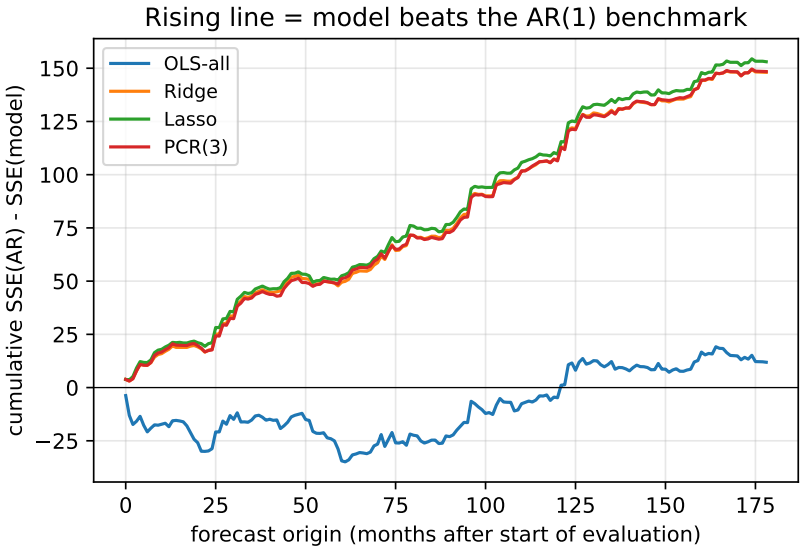


Figure 12.10: Cumulative difference in squared forecast errors between the AR(1) benchmark and each model (simulated data). An upward-sloping line means the model forecasts better than the benchmark over that period. OLS with 101 predictors is erratic, especially early on when the estimation window is short; ridge, lasso and PCR dominate throughout.

Three lessons emerge. First, unregularised OLS barely beats the AR benchmark: with $p = 101$ and an initial window of 120 observations, its variance swamps the information in the panel, even though the panel is highly informative. Second, all three regularised methods reduce the RMSE by about a third, and they are close to each other: the factor structure means that ridge, PCR and the lasso all capture the common component. Third, the gains are steady over time (Fig. 12.10), the hallmark of a genuine rather than a lucky improvement. In real applications one would also test whether differences in accuracy are significant, for instance with the Diebold–Mariano test, and examine several horizons.

The evidence of Goulet Coulombe et al. (2022). In a large pseudo-out-of-sample study of US macroeconomic forecasting, Goulet Coulombe, Leroux, Stevanovic and Surprenant decompose the contribution of machine learning into four features: nonlinearity, regularisation, cross-validation and the loss function. Their main findings are that (i) *nonlinearity* is the most important source of gains, especially at longer horizons and in periods of high macroeconomic uncertainty; (ii) among regularisation schemes, the classic *factor* (data-rich, PCR-type) approach is hard to beat, while sparsity-inducing penalties such as the lasso help less; (iii) standard *K-fold cross-validation* is a good way to tune hyperparameters; and (iv) the standard quadratic loss is preferable to the ϵ-insensitive loss of support vector regression. Our linear simulation cannot speak to (i), but it reproduces the spirit of (ii) and (iii): with a factor structure, dense shrinkage and principal components are natural, and cross-validated tuning works.

Policy Lens: Forecasting inflation and GDP at a central bank

An inflation-targeting central bank, such as the Bank of Ghana, must forecast inflation and output several quarters ahead to set its policy rate, using many indicators released at different times. The methods of this section — factor models, ridge, lasso, and their nonlinear extensions — are the standard toolkit of "nowcasting" and short-term forecasting units. Three practical lessons follow. (1) Evaluate every model pseudo-out-of-sample against simple benchmarks (random walk, AR); in-sample fit is irrelevant. (2) Prefer methods that remain stable as new data arrive; a scorecard whose selected indicators change every month is hard to communicate to a Monetary Policy Committee. (3) Interpretability matters: decompositions of a forecast into the contributions of indicator blocks (prices, money, external sector) help policymakers understand *why* a forecast changed, which is part of the bank's accountability.

Policy Lens: Data-rich policy environments and interpretability

Tax administrations, social-protection agencies and health ministries increasingly hold administrative data with thousands of variables per person or firm. Regularised models make prediction feasible, but prediction is only one input into a policy decision (Kleinberg et al., 2015). A lasso audit-selection model may predict evasion well and still be unacceptable if it relies on variables that proxy protected characteristics, or if it cannot be explained to those affected. Choosing λ by the one-standard-error rule, reporting groups of correlated predictors, and documenting the out-of-sample evaluation are simple steps that make such models more transparent and accountable.

⋆ Key Concept: Honest model selection

Tune by cross-validation (or AIC/BIC) using only training data, with all preprocessing inside the loop; prefer the one-standard-error model when simplicity or stability matters; and assess the final model on data that played no role in fitting or tuning — for time series, by an expanding- or rolling-window pseudo-out-of-sample exercise against simple benchmarks.

Exercises

12.6.1 Theory Prove the leave-one-out identity $y_i - \hat{f}^{(-i)}(\mathbf{x}_i) = (y_i - \hat{y}_i)/(1 - H_{ii})$ for OLS, using the Sherman–Morrison formula. Extend it to ridge with $\mathbf{S}_\lambda$ in place of $\mathbf{H}$.

12.6.2 Theory Explain why PCR with k components is a "hard" version of ridge: express both fitted values in the basis $\{\mathbf{u}_j\}$ and compare the weights applied to $\mathbf{u}_j^\top \mathbf{y}$.

12.6.3 Python In Listing 12.12, repeat the CV with 20 different random fold assignments. How variable are $\hat{\lambda}_{\min}$ and $\hat{\lambda}_{1\text{se}}$? What does this suggest about reporting a single selected model?

12.6.4 Python Modify Listing 12.14 to (a) forecast $h = 6$ months ahead, (b) add an elastic net and PCR with k chosen by BIC at each origin, and (c) compute a Diebold–Mariano statistic for each model relative to the AR benchmark. Which conclusions change?

12.6.5 Python Make the target in Listing 12.14 depend nonlinearly on the first factor (e.g. through $\max(F_{1t}, 0)$). Add a random-forest or a small neural-network forecast and relate your findings to the first conclusion of Goulet Coulombe et al. (2022).

12.6.6 ★ Graduate Replace random K-fold by blocked time-series cross-validation (`TimeSeriesSplit`) for tuning ridge and lasso in the forecasting exercise. Does the choice of CV scheme matter for the out-of-sample RMSE? Discuss in light of Goulet Coulombe et al. (2022).

12.6.7 Policy You are asked to advise a ministry of finance on building a GDP nowcasting model from 150 indicators with 15 years of quarterly data. Write a one-page design note covering the choice of methods, the evaluation scheme, benchmarks, and how the results would be communicated to policymakers.

Chapter Summary

- **Layout.** We use numerator-layout Jacobians ($m \times n$) and gradients with the shape of the argument. Key results: $\nabla \mathbf{a}^\top \mathbf{x} = \mathbf{a}$, $\nabla \mathbf{x}^\top \mathbf{A}\mathbf{x} = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$, $\partial \log\det \mathbf{X}/\partial \mathbf{X} = \mathbf{X}^{-\top}$, $\mathrm{d}\mathbf{X}^{-1} = -\mathbf{X}^{-1}\mathrm{d}\mathbf{X}\,\mathbf{X}^{-1}$.
- **Differentials and traces.** Differentiate, cycle $\mathrm{d}\mathbf{X}$ to the end of a trace, identify; then verify with central differences ($h \approx 10^{-5}$).
- **Applications.** OLS, GLS, the Gaussian MLE, logit and softmax regression all follow in a few lines; their gradients have the form $\mathbf{X}^\top(\text{fitted} - \text{observed})$ and their Hessians $\mathbf{X}^\top\mathbf{W}\mathbf{X}$.
- **AD.** Reverse mode (backpropagation) computes the gradient of one scalar loss with respect to all parameters at a small multiple of the cost of the loss; always gradient-check.
- **Bias–variance.** Test error $= \sigma^2 + \text{Bias}^2 + \text{Var}$; training error is optimistic by $2\sigma^2\,\mathrm{df}/n$; $\mathrm{df} = \mathrm{tr}(\mathbf{S})$; double descent appears beyond the interpolation threshold.
- **Regularisation.** Ridge shrinks by $d_j^2/(d_j^2 + \lambda)$, exists for $p > n$ and is a Gaussian-prior MAP; the lasso soft-thresholds and selects; the elastic net groups. Standardise first.
- **Selection and forecasting.** Tune by K-fold CV (one-SE rule) or AIC/BIC inside pipelines; evaluate forecasts pseudo-out-of-sample against simple benchmarks.

Further Reading

For matrix calculus, Magnus and Neudecker (2019) is the definitive treatment of the differential approach, Petersen and Pedersen (2012) is a handy table of identities, and Deisenroth et al. (2020, Ch. 5) gives an ML-oriented introduction. Murphy (2022, Ch. 7 and 13) covers matrix calculus, automatic differentiation and neural networks; Baydin et al. (2018) surveys AD and Griewank and Walther (2008) is the standard monograph; Goodfellow et al. (2016) treats backpropagation in depth. For the bias–variance trade-off, regularisation and model selection, read Hastie et al. (2009, Ch. 3 and 7), James et al. (2021, Ch. 5–6) and Hastie et al. (2015) on sparsity; Géron (2022, Ch. 4) shows the scikit-learn implementations of ridge, lasso and elastic net, and Matsuura (2022) gives the Bayesian view of shrinkage priors. For forecasting with many predictors, start with Stock and Watson (2002a), De Mol et al. (2008) and Goulet Coulombe et al. (2022), and see Giannone et al. (2021) on sparse versus dense models. Mullainathan and Spiess (2017) and Kleinberg et al. (2015) discuss when prediction is the right goal in economic policy.

Part V

Synthesis

Chapter 13

Problem Set Clinic and Applied Foundations Review

Learning Objectives

By the end of this chapter you should be able to:

1. apply a disciplined workflow to a new problem: understand it, fix notation, check dimensions, try small cases, prove, verify by simulation and interpret;
2. choose among direct proof, contradiction, induction and chains of inequalities, and check an analytical result (a variance formula, a gradient) numerically;
3. organise an analysis so that it is reproducible: seeds, environments, version control and a sensible project structure;
4. carry out four complete applied analyses that combine the tools of Chapter 1–Chapter 12: a Mincer wage regression with robust and bootstrap inference, a proxy-means test for poverty targeting, a many-predictor macroeconomic forecasting exercise and a budget allocation under uncertainty;
5. state the key results of the course, recognise the most common pitfalls and solve exam-style problems drawn from every chapter.

The previous twelve chapters built a toolkit piece by piece: vectors and floating-point arithmetic, projections and decompositions, eigenvalues and the singular value decomposition, probability and distributions, expectation and limit theorems, estimation, optimisation with and without constraints, Monte Carlo, resampling and regularisation. Real problems in data science and economic policy never arrive labelled by chapter. A ministry asking "what is the return to an extra year of schooling?" needs least squares (Chapter 2), asymptotic and bootstrap inference (Chapters 6 and 11) and an honest discussion of identification. A social-protection agency choosing whom to enrol in a cash-transfer programme needs principal components (Chapter 3), maximum likelihood (Chapter 7), Newton's method (Chapter 8) and regularisation (Chapter 12). A central bank forecasting inflation from hundreds of indicators needs all of these plus careful out-of-sample evaluation, and a treasury dividing a fixed budget across programmes needs constrained optimisation (Chapter 9) and simulation (Chapter 10).

This chapter is therefore a *clinic*. Section 13.1 describes how to attack problems and how to check your own answers. Sections 13.2 to 13.5 are four integrated case studies, each implemented from first principles on clearly *simulated* data so that the true answer is known and every number can be reproduced. Nothing in them should be read as a statistic about Ghana or any other country. Section 13.6 closes the course with a concept map, a table of key results, a list of pitfalls, ten fully worked exam-style problems and a comprehensive problem set.

13.1 How to Solve Problems in This Course

13.1.1 A seven-step workflow

Most difficulties students have with problem sets are not gaps in knowledge but gaps in *process*: starting to compute before the question is understood, losing track of dimensions, or proving a statement that is false for $n = 1$. The following workflow is a discipline worth practising until it is automatic.

Definition 13.1 (Problem-solving workflow). **S1. Understand.** Restate the problem in your own words. What is given, what is unknown, what are the conditions? Is the claim plausible?

S2. Set up notation. Name every object, state its type (scalar, vector in $\mathbb{R}^n$, matrix in $\mathbb{R}^{n\times k}$, random variable, function) and its assumptions (independence, full rank, convexity).

S3. Check dimensions. Every product, sum and equality must be conformable; every quantity should have sensible units (cedis, log points, probability).

S4. Try small and extreme cases. $n = 1$, $k = 1$, a 2×2 matrix, $\sigma = 0$, $\lambda \to 0$ or $\lambda \to \infty$. Small cases suggest the pattern and expose false claims.

S5. Prove. Choose a technique (direct, contradiction, induction, inequality chain) and write a complete argument in which each step is justified.

S6. Simulate to verify. Implement the result and check it against a brute-force or Monte Carlo computation, reporting the simulation error.

S7. Interpret. What does the result mean for the data-science or policy question? When does it fail?

Historical Note: Pólya and *How to Solve It*

George Pólya (1887–1985), a Hungarian mathematician who worked at ETH Zürich and later at Stanford, published *How to Solve It* in 1945 (Pólya, 1945). The book organises problem solving into four phases — understanding the problem, devising a plan, carrying out the plan and looking back — and catalogues heuristics such as solving a related or more special problem, working backwards and introducing auxiliary elements. Our steps S1–S4 are a version of "understanding" and "devising a plan"; S6 and S7 are a computational form of "looking back" that Pólya could not have used but would surely have approved.

Example 13.2 (Dimension checking catches an error). A student writes the OLS variance as $\operatorname{Var}(\hat{\boldsymbol{\beta}}) = \sigma^2 \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$. With $\mathbf{X} \in \mathbb{R}^{n\times k}$ the right side is $n \times n$, while $\hat{\boldsymbol{\beta}} \in \mathbb{R}^k$ needs a $k \times k$ covariance. The $n \times n$ matrix is in fact σ^2 times the hat matrix, the covariance of the fitted values $\hat{\mathbf{y}}$. The correct expression is $\sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$.

13.1.2 Proof techniques used throughout the course

Direct proof. Start from the definitions and manipulate until the claim appears.

Proposition 13.3. *If $\mathbf{X} \in \mathbb{R}^{n\times k}$ has full column rank, $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ is symmetric and idempotent, and $\operatorname{tr}\mathbf{H} = k$.*

Proof. $\mathbf{H}^\top = \mathbf{X}((\mathbf{X}^\top\mathbf{X})^{-1})^\top\mathbf{X}^\top = \mathbf{H}$ because $\mathbf{X}^\top\mathbf{X}$ is symmetric. $\mathbf{H}^2 = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}(\mathbf{X}^\top\mathbf{X})(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top = \mathbf{H}$. Finally $\operatorname{tr}\mathbf{H} = \operatorname{tr}\left((\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{X}\right) = \operatorname{tr}\mathbf{I}_k = k$ by the cyclic property of the trace. □

Proof by contradiction. Assume the negation and derive an impossibility.

Proposition 13.4. *A strictly convex function $f : \mathbb{R}^d \to \mathbb{R}$ has at most one minimiser.*

Proof. Suppose $\mathbf{x}_1 \neq \mathbf{x}_2$ both minimise f, with common value f^*. By strict convexity $f(\frac{1}{2}\mathbf{x}_1 + \frac{1}{2}\mathbf{x}_2) < \frac{1}{2}f(\mathbf{x}_1) + \frac{1}{2}f(\mathbf{x}_2) = f^*$, contradicting the minimality of f^*. □

This is why ridge regression (strictly convex for $\lambda > 0$) always has a unique solution, whereas the lasso and unpenalised least squares with collinear regressors may not.

Induction. Prove a base case and the step from n to $n+1$.

Proposition 13.5. *If $X_1, \dots, X_n$ are pairwise uncorrelated with finite variances, then* $\operatorname{Var}\left(\sum_{i=1}^n X_i\right) = \sum_{i=1}^n \operatorname{Var}(X_i)$.

Proof. The case $n = 1$ is trivial. Suppose the claim holds for n and let $S_n = \sum_{i \le n} X_i$. Then $\operatorname{Var}(S_n + X_{n+1}) = \operatorname{Var}(S_n) + \operatorname{Var}(X_{n+1}) + 2\operatorname{Cov}(S_n, X_{n+1})$ and $\operatorname{Cov}(S_n, X_{n+1}) = \sum_{i \le n} \operatorname{Cov}(X_i, X_{n+1}) = 0$. The induction hypothesis completes the proof. □

Chains of inequalities. Many results in Chapters 6 and 7 are a sequence of known inequalities: Cauchy–Schwarz, Jensen, Markov, triangle. For example, Cauchy–Schwarz applied to the centred variables gives $|\operatorname{Cov}(X, Y)| \le \sqrt{\operatorname{Var} X \operatorname{Var} Y}$, hence $|\operatorname{Corr}(X, Y)| \le 1$; Jensen's inequality for the concave logarithm gives $\mathbb{E}\log X \le \log \mathbb{E}X$, which is why the mean of log incomes understates the log of mean income. When writing a chain, label each step with the result it uses and check that its hypotheses (finiteness, convexity, independence) hold.

⋆ Key Concept: A proof is a sequence of justified steps

Every line of a proof must follow from definitions, assumptions or previously established results, and you should be able to name which. "It is obvious" is not a justification; "by the cyclic property of the trace" is. Before proving, test the claim on a small case: if it fails for $n = 1$ or a 2×2 matrix, no proof will save it.

13.1.3 Checking analytical results by simulation

Simulation cannot prove a theorem, but it is the fastest way to catch a wrong one. The key is to check against an *independent* computation (brute force, Monte Carlo, finite differences) and to know how large the discrepancy should be if the result is correct.

Theorem 13.6 (Variance of the sample variance)**.** *Let $X_1, \dots, X_n$ be iid with variance σ^2 and fourth central moment $\mu_4 = \mathbb{E}(X - \mu)^4 < \infty$, and $s^2 = \frac{1}{n-1}\sum_i (X_i - \bar{X})^2$. Then*

$$\operatorname{Var}(s^2) = \frac{\mu_4}{n} - \frac{\sigma^4(n-3)}{n(n-1)}. \tag{13.1}$$

For normal data $\mu_4 = 3\sigma^4$ and (13.1) reduces to $2\sigma^4/(n-1)$.

Proof sketch. Without loss of generality $\mu = 0$. Write $(n-1)s^2 = \sum_i X_i^2 - n\bar{X}^2 = \frac{n-1}{n}\sum_i X_i^2 - \frac{1}{n}\sum_{i \neq j} X_i X_j$. The two sums are uncorrelated (every cross-moment $\mathbb{E}[X_i^2 X_j X_l]$ with $j \neq l$ contains an isolated factor of mean zero), so $\operatorname{Var}((n-1)s^2) = \frac{(n-1)^2}{n^2} n(\mu_4 - \sigma^4) + \frac{1}{n^2} 2n(n-1)\sigma^4$. Dividing by $(n-1)^2$ and simplifying gives (13.1). □

The textbook formula $2\sigma^4/(n-1)$ is often applied without checking normality. For skewed, heavy-tailed data such as incomes or consumption it can be badly wrong. A

Monte Carlo check with R replications estimates $\mathrm{Var}(s^2)$ with standard error approximately $\sqrt{(\mathbb{E}[(s^2 - \mathbb{E}s^2)^4] - \mathrm{Var}(s^2)^2)/R}$, which tells us how close "close" must be.

For derivatives, the standard check compares an analytical gradient with central differences. The central difference $\{f(x+h) - f(x-h)\}/(2h)$ has truncation error $O(h^2)$ but rounding error of order $\varepsilon_{\text{mach}} \left|f\right| / h$ (Chapter 1); balancing the two gives an optimal $h \approx \varepsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6}$. The forward difference has truncation error $O(h)$ and optimal $h \approx \varepsilon_{\text{mach}}^{1/2} \approx 1.5 \times 10^{-8}$.

Listing 13.1: Simulation checks of a variance formula and a gradient ch13/ch13_verify_formulas.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Simulation checks of an analytical variance formula and of a derivative.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(803)

# ---- Part 1: Var(s^2) for iid data with variance sigma2, 4th central moment mu4
#      Var(s^2) = mu4/n - sigma2^2 (n - 3) / (n (n - 1))
def var_s2(n, sigma2, mu4):
    return mu4 / n - sigma2**2 * (n - 3) / (n * (n - 1))

n, reps = 20, 200_000
cases = {"Normal(0,1)": (lambda size: rng.standard_normal(size), 1.0, 3.0),
         "Exponential(1)": (lambda size: rng.exponential(1.0, size), 1.0, 9.0)}
print("Part 1: Var(s^2), n = %d, %d replications" % (n, reps))
print(f"{'population':<16}{'formula':>10}{'normal-only':>13}{'simulated':>11}"
      f"{'MC s.e.':>9}")
for name, (draw, s2, mu4) in cases.items():
    x = draw((reps, n))
    s2_hat = x.var(axis=1, ddof=1)
    sim = s2_hat.var()
    # Monte Carlo s.e. of a variance estimate: sqrt((m4 - v^2) / reps)
    dev2 = (s2_hat - s2_hat.mean())**2
    mcse = np.sqrt(dev2.var() / reps)
    print(f"{name:<16}{var_s2(n, s2, mu4):10.4f}{2 * s2**2 / (n - 1):13.4f}"
          f"{sim:11.4f}{mcse:9.4f}")

# ---- Part 2: gradient of the logistic log-likelihood, analytic vs numerical
m, k = 500, 3
X = np.column_stack([np.ones(m), rng.standard_normal((m, k - 1))])
y = (rng.random(m) < 1 / (1 + np.exp(-X @ np.array([-0.5, 1.0, -2.0])))) * 1.0

def loglik(b):
    eta = X @ b
    return np.sum(y * eta - np.logaddexp(0.0, eta))

def grad(b):
    return X.T @ (y - 1 / (1 + np.exp(-X @ b)))

b0 = np.array([0.2, 0.3, -0.4])
e1 = np.eye(k)
num = np.array([(loglik(b0 + 1e-5 * e) - loglik(b0 - 1e-5 * e)) / 2e-5
                for e in e1])
print("\nPart 2: gradient of logistic log-likelihood at b0")
print("analytic :", np.array2string(grad(b0), precision=6))
```

```
print("central  :", np.array2string(num, precision=6))
rel = np.linalg.norm(num - grad(b0)) / np.linalg.norm(grad(b0))
print(f"relative error = {rel:.2e}")

# error of forward / central differences as a function of h (first coordinate)
hs = np.logspace(-12, 0, 49)
g1 = grad(b0)[0]
fwd = [abs((loglik(b0 + h * e1[0]) - loglik(b0)) / h - g1) for h in hs]
cen = [abs((loglik(b0 + h * e1[0]) - loglik(b0 - h * e1[0])) / (2 * h) - g1)
       for h in hs]
print(f"best h: forward {hs[np.argmin(fwd)]:.1e}, central {hs[np.argmin(cen)]:.1e}")

plt.rcParams.update({"font.size": 10})
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.loglog(hs, fwd, "o-", ms=3, label="forward difference")
ax.loglog(hs, cen, "s-", ms=3, label="central difference")
ax.set_xlabel("step size h")
ax.set_ylabel("absolute error in derivative")
ax.set_title("Truncation error (right) versus rounding error (left)")
ax.legend()
ax.grid(True, which="major", alpha=0.3)
plt.savefig("ch13_fd_error.pdf", bbox_inches="tight")
```

Output

```
Part 1: Var(s^2), n = 20, 200000 replications
population         formula  normal-only  simulated  MC s.e.
Normal(0,1)         0.1053       0.1053     0.1055   0.0004
Exponential(1)      0.4053       0.1053     0.4038   0.0030

Part 2: gradient of logistic log-likelihood at b0
analytic : [ -80.367936   27.506917 -104.357885]
central  : [ -80.367936   27.506917 -104.357885]
relative error = 1.29e-11
best h: forward 1.8e-08, central 1.0e-05
```

The output confirms (13.1) within one or two Monte Carlo standard errors for both populations, and shows that the "normal-only" formula understates the variance of s^2 by a factor of almost four for exponential data. The analytical logistic gradient $\mathbf{X}^\top(\mathbf{y} - \mathbf{p})$ (Chapter 7) agrees with central differences to a relative error of about 10^{-11}. Figure 13.1 shows the classic "V" shape of finite-difference error: on the right, truncation error dominates; on the left, rounding error explodes. The empirical optimal step sizes match the theory above.

△ Caution: A simulation that agrees is not a proof — and one that disagrees may be buggy

A Monte Carlo check can only detect discrepancies larger than its own standard error, and it only tests the parameter values you tried. Conversely, a disagreement may come from the simulation (wrong seed handling, `ddof` confusion between n and $n-1$, a different parameterisation of the exponential distribution) rather than from the theory. Always report the Monte Carlo standard error and test several parameter values, including extreme ones.

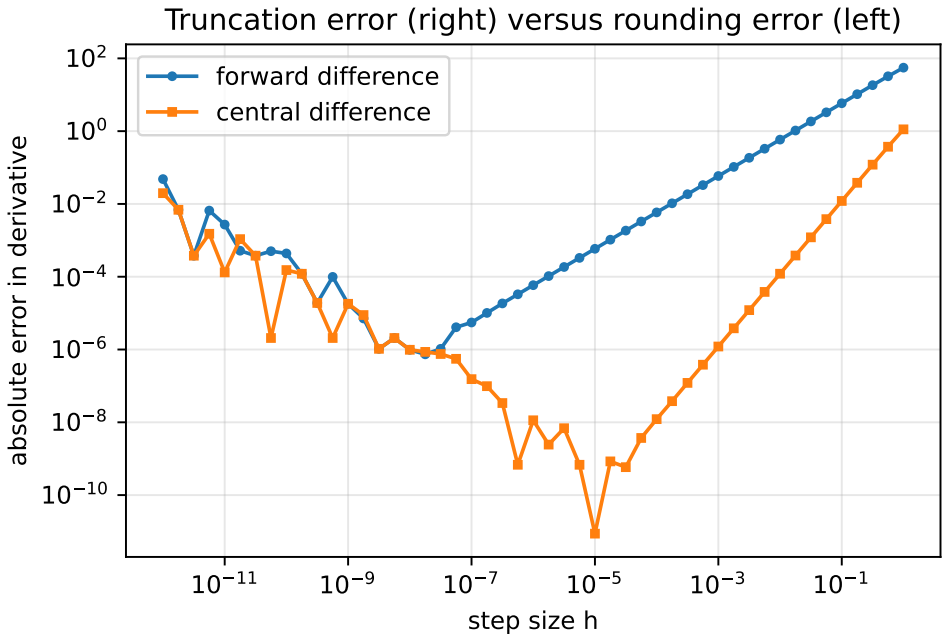


Figure 13.1: Error of forward and central differences for one component of the logistic log-likelihood gradient as a function of the step h (log–log scale).

13.1.4 A reproducible research workflow

An analysis is *reproducible* if someone else (including you, six months later) can regenerate every number, table and figure from the raw data and code. It is *replicable* if an independent study reaches the same conclusion with new data. Policy analysis that cannot be reproduced cannot be audited, and analysis that cannot be audited should not guide public spending.

Seeds and random streams. Every stochastic computation in this book begins with `rng = np.random.default_rng(803)`. A seed makes a pseudo-random sequence deterministic. For parallel work (one stream per bootstrap worker or per Monte Carlo cell), `np.random.SeedSequence(803).spawn(k)` creates k statistically independent streams that are themselves reproducible. Avoid the legacy global state `np.random.seed`, which any imported library can silently alter.

Environments. Results can change across library versions (default solvers, tolerances, random-number algorithms). Record the versions with each result and pin them in a `requirements.txt` or `environment.yml` file.

Version control. Use `git`: `git init` once; then repeatedly `git add` the changed files and `git commit -m "message"` with a meaningful message; `git log` and `git diff` show the history; a tag such as `git tag submitted-v1` marks the exact code behind a submitted report. Never commit confidential microdata; keep raw data read-only and outside the repository if necessary.

Project structure. A simple layout that scales from a problem set to a thesis:

`data/raw/`	original data, never edited by hand
`data/derived/`	cleaned data produced by scripts
`code/`	numbered scripts: `01_clean.py`, `02_estimate.py`, ...
`output/`	tables, figures and logs, all generated by code
`README.md`	how to run everything, in what order, with which environment
`requirements.txt`	pinned package versions

A single driver script (like this book's `run_all.py`) that rebuilds all outputs from scratch is the strongest test of reproducibility.

Listing 13.2: Seeds, spawned streams, environment and result fingerprints
`ch13/ch13_reproducible.py`

```python
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Reproducibility toolkit: seeds, independent streams, environment, fingerprints.
# (c) 2026 Stephen E. Moore. MIT Licence.
import hashlib
import platform
import numpy as np
import scipy
import sklearn


def experiment(rng, n=1000):
    """A toy 'analysis': bootstrap s.e. of a mean of simulated incomes."""
    income = rng.lognormal(mean=6.0, sigma=0.8, size=n)
    idx = rng.integers(0, n, size=(500, n))
    return income.mean(), income[idx].mean(axis=1).std(ddof=1)


def fingerprint(*arrays):
    """Short SHA-256 hash of results: equal hashes => bitwise equal results."""
    h = hashlib.sha256()
    for a in arrays:
        h.update(np.ascontiguousarray(a, dtype=float).tobytes())
    return h.hexdigest()[:16]


# 1. Same seed => identical results; no seed => different results
r1 = experiment(np.random.default_rng(803))
r2 = experiment(np.random.default_rng(803))
r3 = experiment(np.random.default_rng())
print("seed 803, run 1 : mean %.3f  boot se %.4f  hash %s"
      % (*r1, fingerprint(r1)))
print("seed 803, run 2 : mean %.3f  boot se %.4f  hash %s"
      % (*r2, fingerprint(r2)))
print("no seed         : identical to run 1?", fingerprint(r3) == fingerprint(r1))

# 2. Independent streams for parallel workers via SeedSequence.spawn
root = np.random.SeedSequence(803)
streams = [np.random.default_rng(s) for s in root.spawn(4)]
first = np.array([g.standard_normal() for g in streams])
print("\nfirst draw of 4 spawned streams:", np.array2string(first, precision=4))
print("spawned stream 0 reproducible?  ",
      np.random.default_rng(np.random.SeedSequence(803).spawn(4)[0])
      .standard_normal() == first[0])

```

```python
# 3. Record the computational environment alongside every result
print("\nenvironment:")
print(f"  python {platform.python_version()}, numpy {np.__version__}, "
      f"scipy {scipy.__version__}, sklearn {sklearn.__version__}")
```

Output

```
seed 803, run 1 : mean 544.066  boot se 14.9441  hash ef49432bfaa683f8
seed 803, run 2 : mean 544.066  boot se 14.9441  hash ef49432bfaa683f8
no seed         : identical to run 1? False

first draw of 4 spawned streams: [ 0.4086 -1.0578  1.2185 -0.1658]
spawned stream 0 reproducible?   True

environment:
  python 3.9.13, numpy 1.23.5, scipy 1.9.3, sklearn 1.1.3
```

The fingerprint (a truncated SHA-256 hash of the result bytes) lets you confirm that two runs are *bitwise* identical without comparing numbers by eye; spawned streams give independent but reproducible draws for parallel workers.

Historical Note: The replication movement in economics

Dewald et al. (1986) tried to reproduce empirical articles submitted to the *Journal of Money, Credit and Banking* and found that inadvertent errors were common and that many authors could not supply usable data and code. Their study is widely credited with prompting economics journals, the *American Economic Review* among them, eventually to adopt data-availability policies. A widely discussed case was the critique by Herndon et al. (2014) of an influential study of public debt and growth, which identified a spreadsheet coding error among other issues. Large replication projects in experimental economics (Camerer et al., 2016) and the review by Christensen and Miguel (2018) have since made transparency — pre-registration, shared code, data editors who re-run submitted packages — a mainstream norm.

⋆ Key Concept: Reproducibility checklist

(1) fixed seeds and explicit random generators; (2) recorded and pinned software versions; (3) code under version control, with the commit that produced each report tagged; (4) raw data never edited by hand; (5) one command regenerates every number in the report.

△ Caution: Seed shopping

A seed exists to make results reproducible, not to make them look good. Re-running an analysis with different seeds until a bootstrap interval excludes zero is a form of specification search. If results are sensitive to the seed, the Monte Carlo or bootstrap sample is too small: increase B or R and report the simulation error.

Exercises

13.1.1 Theory Prove Proposition 13.3 for the residual-maker $\mathbf{M} = \mathbf{I} - \mathbf{H}$: show $\mathbf{M}$ is symmetric, idempotent, has trace $n - k$, and $\mathbf{MX} = \mathbf{0}$. What are the eigenvalues of $\mathbf{M}$?

13.1.2 Theory Use induction to prove Bernoulli's inequality $(1 + x)^n \geq 1 + nx$ for $x \geq -1$ and integers $n \geq 1$. Where exactly is $x \geq -1$ needed?

13.1.3 Theory Fill in the details of the proof of Theorem 13.6. Then show that $\operatorname{Var}(s^2) \geq 0$ requires $\mu_4 \geq \sigma^4(n-3)/(n-1)$ and explain why this is implied by Jensen's inequality.

13.1.4 Python Extend `ch13_verify_formulas.py` to check the Hessian of the logistic log-likelihood, $-\mathbf{X}^\top \mathbf{W}\mathbf{X}$ with $\mathbf{W} = \operatorname{diag}\{p_i(1-p_i)\}$, by central differences of the analytical gradient. Report the relative error.

13.1.5 Python Check (13.1) for Student-t data with $\nu = 5$ degrees of freedom ($\sigma^2 = 5/3$, $\mu_4 = 25$). Repeat with $\nu = 4.5$ and explain why the Monte Carlo becomes unreliable as $\nu \downarrow 4$.

13.1.6 ★ Graduate Derive the optimal step size for central differences by minimising the bound $\frac{h^2}{6}\left|f'''\right| + \varepsilon_{\text{mach}}\left|f\right|/h$. How does the answer change for complex-step differentiation $f'(x) \approx \operatorname{Im} f(x+ih)/h$, and why is that method free of cancellation?

13.1.7 Policy A colleague's poverty report cannot be reproduced: it was produced by point-and-click in a spreadsheet with manual edits to the data. Write a one-page plan, using the checklist above, to convert the analysis into a reproducible project, and explain to a non-technical director why this matters for public accountability.

13.2 Case Study 1: Returns to Education from a Simulated Labour Force Survey

How much does an additional year of schooling raise earnings? The answer feeds directly into education budgets, student-loan design and the evaluation of free secondary schooling policies. The workhorse model is the Mincer earnings function

$$\log w_i = \beta_0 + \rho\, S_i + \beta_1 E_i + \beta_2 E_i^2 + \mathbf{z}_i^\top \boldsymbol{\gamma} + u_i, \tag{13.2}$$

where w_i is the hourly wage, S_i years of schooling, E_i potential experience (age − schooling − 6) and $\mathbf{z}_i$ other controls (sex, region). Under the assumptions behind the model, ρ is the proportional wage gain from one more year of schooling.

Historical Note: Jacob Mincer and the earnings function

Jacob Mincer (1922–2006) of Columbia University set out the human-capital earnings function in *Schooling, Experience, and Earnings* (Mincer, 1974). Its log-linear form with a quadratic in experience became perhaps the most estimated equation in labour economics. Reviews of hundreds of estimates across countries (Psacharopoulos and Patrinos, 2018) typically find private returns of the order of several to ten percent per year of schooling, while Heckman et al. (2006) show the conditions under which the Mincer coefficient can be read as an internal rate of return — conditions that often fail.

13.2.1 The simulated survey

Our simulated labour force survey has $n = 4000$ workers in ten regions. Regions differ in both average wages and average schooling, so region is a *confounder*: omitting it biases the schooling coefficient. Schooling ranges from 0 to 20 years, and the true data-generating process is (13.2) with $\rho = 0.10$, $\beta_1 = 0.035$, $\beta_2 = -0.0006$, a female wage gap of -0.12 log points, regional effects between -0.3 and 0.3, and *heteroskedastic* errors whose standard deviation $0.10 + 0.045 S_i$ rises with schooling (wages of the highly educated are more dispersed). In this first simulation schooling is exogenous; we relax that in Section 13.2.6.

13.2.2 Least squares via QR

Stack the regressors into $\mathbf{X} \in \mathbb{R}^{n \times k}$ ($k = 14$: intercept, schooling, experience, its square, female and nine region dummies). As in Chapter 2, compute OLS from the thin QR factorisation

$\mathbf{X} = \mathbf{QR}$, $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_k$:

$$\hat{\boldsymbol{\beta}} = \mathbf{R}^{-1}\mathbf{Q}^\top\mathbf{y}, \qquad (\mathbf{X}^\top\mathbf{X})^{-1} = \mathbf{R}^{-1}\mathbf{R}^{-\top}. \tag{13.3}$$

The normal equations $\mathbf{X}^\top\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\top\mathbf{y}$ give the same answer in exact arithmetic, but their condition number is $\operatorname{cond}(\mathbf{X})^2$ (Chapter 3). With experience and its square, $\operatorname{cond}(\mathbf{X}) \approx 10^4$ here, so the normal equations would lose about eight digits instead of four.

13.2.3 Fixed effects and the Frisch–Waugh–Lovell theorem

Theorem 13.7 (Frisch–Waugh–Lovell). *Partition $\mathbf{X} = [\mathbf{x}_1, \mathbf{Z}]$ with $\mathbf{x}_1 \in \mathbb{R}^n$ and $\mathbf{Z} \in \mathbb{R}^{n\times(k-1)}$ of full column rank, and let $\mathbf{M}_Z = \mathbf{I} - \mathbf{Z}(\mathbf{Z}^\top\mathbf{Z})^{-1}\mathbf{Z}^\top$. The OLS coefficient on $\mathbf{x}_1$ in the regression of $\mathbf{y}$ on $\mathbf{X}$ equals*

$$\hat{\beta}_1 = \frac{(\mathbf{M}_Z\mathbf{x}_1)^\top(\mathbf{M}_Z\mathbf{y})}{(\mathbf{M}_Z\mathbf{x}_1)^\top(\mathbf{M}_Z\mathbf{x}_1)}, \tag{13.4}$$

the slope from regressing the residualised outcome on the residualised regressor; the residuals of the two regressions are identical.

Proof. Write $\mathbf{y} = \mathbf{x}_1\hat{\beta}_1 + \mathbf{Z}\hat{\gamma} + \hat{\mathbf{e}}$ with $\mathbf{X}^\top\hat{\mathbf{e}} = \mathbf{0}$. Premultiply by $\mathbf{M}_Z$: since $\mathbf{M}_Z\mathbf{Z} = \mathbf{0}$ and $\mathbf{M}_Z\hat{\mathbf{e}} = \hat{\mathbf{e}}$ (because $\mathbf{Z}^\top\hat{\mathbf{e}} = \mathbf{0}$), $\mathbf{M}_Z\mathbf{y} = \mathbf{M}_Z\mathbf{x}_1\hat{\beta}_1 + \hat{\mathbf{e}}$. Taking the inner product with $\mathbf{M}_Z\mathbf{x}_1$ and using $(\mathbf{M}_Z\mathbf{x}_1)^\top\hat{\mathbf{e}} = \mathbf{x}_1^\top\hat{\mathbf{e}} = 0$ gives (13.4). □

When $\mathbf{Z}$ contains a full set of region dummies, $\mathbf{M}_Z$ subtracts region means: the *within* or fixed-effects estimator is an application of FWL. The theorem also explains omitted-variable bias: dropping the region dummies replaces $\mathbf{M}_Z$ by a projection that no longer removes regional differences in schooling, which are correlated with regional wage levels.

13.2.4 Heteroskedasticity-robust standard errors

If $\mathbb{E}(\mathbf{u} \mid \mathbf{X}) = \mathbf{0}$ and $\operatorname{Var}(\mathbf{u} \mid \mathbf{X}) = \boldsymbol{\Omega} = \operatorname{diag}(\sigma_1^2, \dots, \sigma_n^2)$, then $\hat{\boldsymbol{\beta}} - \boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{u}$ and

$$\operatorname{Var}(\hat{\boldsymbol{\beta}} \mid \mathbf{X}) = (\mathbf{X}^\top\mathbf{X})^{-1}\Big(\sum_{i=1}^{n}\sigma_i^2\mathbf{x}_i\mathbf{x}_i^\top\Big)(\mathbf{X}^\top\mathbf{X})^{-1}. \tag{13.5}$$

The classical formula $\sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$ is the special case $\sigma_i^2 = \sigma^2$. White (1980) showed that replacing σ_i^2 by $\hat{e}_i^2$ yields a consistent estimator (HC0) of the asymptotic variance; HC1 multiplies by $n/(n-k)$ as a degrees-of-freedom correction, and MacKinnon and White (1985) proposed further refinements (HC2, HC3) using the leverages h_{ii}.

Definition 13.8 (HC1 covariance estimator). $\hat{\mathbf{V}}_{\mathrm{HC1}} = \dfrac{n}{n-k}(\mathbf{X}^\top\mathbf{X})^{-1}\Big(\sum_{i=1}^{n}\hat{e}_i^2\mathbf{x}_i\mathbf{x}_i^\top\Big)(\mathbf{X}^\top\mathbf{X})^{-1}$,

and $\operatorname{se}_{\mathrm{HC1}}(\hat{\beta}_j) = \sqrt{[\hat{\mathbf{V}}_{\mathrm{HC1}}]_{jj}}$.

In code, the "meat" $\sum_i \hat{e}_i^2\mathbf{x}_i\mathbf{x}_i^\top$ is `(X * e[:, None]**2).T @ X`: scaling the rows of $\mathbf{X}$ avoids ever forming the $n \times n$ matrix $\operatorname{diag}(\hat{e}_i^2)$.

Listing 13.3: Mincer regression: QR, FWL, within estimator and HC1 `ch13/ch13_mincer_ols.py`

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Mincer regression on a simulated labour force survey: QR, FWL, HC1 by hand.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
```

```

def simulate_lfs(rng, n=4000, n_reg=10):
    """Simulated labour force survey (NOT real data for any country)."""
    region = rng.integers(0, n_reg, n)
    reg_fe = np.linspace(-0.3, 0.3, n_reg)           # regional wage levels
    reg_edu = np.linspace(-2.0, 2.0, n_reg)          # regional schooling gaps
    female = (rng.random(n) < 0.5).astype(float)
    age = rng.integers(18, 61, n)
    educ = np.clip(np.round(9 + reg_edu[region] + 3.5 * rng.standard_normal(n)),
                   0, 20)
    exper = np.maximum(age - educ - 6, 0)
    sd = 0.10 + 0.045 * educ                         # heteroskedastic errors
    logw = (1.0 + reg_fe[region] + 0.10 * educ + 0.035 * exper
            - 0.0006 * exper**2 - 0.12 * female + sd * rng.standard_normal(n))
    return logw, educ, exper, female, region


def ols_qr(X, y):
    """OLS via thin QR: returns beta, residuals and R."""
    Q, R = np.linalg.qr(X)
    beta = np.linalg.solve(R, Q.T @ y)
    return beta, y - X @ beta, R


rng = np.random.default_rng(803)
logw, educ, exper, female, region = simulate_lfs(rng)
n = logw.size
D = (region[:, None] == np.arange(1, 10)).astype(float)   # 9 region dummies
X = np.column_stack([np.ones(n), educ, exper, exper**2, female, D])
k = X.shape[1]
beta, e, R = ols_qr(X, logw)

# Classical and HC1 standard errors, using (X'X)^{-1} = R^{-1} R^{-T}
Rinv = np.linalg.solve(R, np.eye(k))
XtX_inv = Rinv @ Rinv.T
s2 = e @ e / (n - k)
se_cl = np.sqrt(np.diag(s2 * XtX_inv))
meat = (X * e[:, None]**2).T @ X
V_hc1 = n / (n - k) * XtX_inv @ meat @ XtX_inv
se_hc1 = np.sqrt(np.diag(V_hc1))

names = ["const", "educ", "exper", "exper^2", "female"]
print(f"n = {n}, k = {k} (incl. 9 region dummies), cond(X) = "
      f"{np.linalg.cond(X):.2e}")
print(f"{'':<9}{'coef':>10}{'se(class)':>11}{'se(HC1)':>10}{'ratio':>7}")
for j, nm in enumerate(names):
    print(f"{nm:<9}{beta[j]:10.5f}{se_cl[j]:11.5f}{se_hc1[j]:10.5f}"
          f"{se_hc1[j] / se_cl[j]:7.2f}")
b_ls = np.linalg.lstsq(X, logw, rcond=None)[0]
print("agrees with lstsq:", np.allclose(beta, b_ls))

# Frisch-Waugh-Lovell: partial out controls Z (all columns except educ)
Z = np.delete(X, 1, axis=1)
_, e_educ, _ = ols_qr(Z, educ)
_, e_logw, _ = ols_qr(Z, logw)
b_fwl = (e_educ @ e_logw) / (e_educ @ e_educ)
```

```python
# Within (demeaning by region) transformation = FWL for the region dummies
def demean(v):
    means = np.bincount(region, weights=v) / np.bincount(region)
    return v - means[region]
W = np.column_stack([demean(c) for c in (educ, exper, exper**2, female)])
b_within = np.linalg.lstsq(W, demean(logw), rcond=None)[0]
print(f"\nreturn to schooling: full OLS {beta[1]:.6f}, FWL {b_fwl:.6f}, "
      f"within {b_within[0]:.6f}")
print(f"no region FE        : {ols_qr(X[:, :5], logw)[0][1]:.6f}")
pct = 100 * (np.exp(beta[1]) - 1)
lo, hi = beta[1] - 1.96 * se_hc1[1], beta[1] + 1.96 * se_hc1[1]
print(f"one more year of school: +{pct:.2f}% wage; 95% HC1 CI for beta "
      f"[{lo:.4f}, {hi:.4f}]")
```

Output

```
n = 4000, k = 14 (incl. 9 region dummies), cond(X) = 1.04e+04
               coef  se(class)   se(HC1)  ratio
const       0.71909    0.04470   0.04378   0.98
educ        0.10107    0.00248   0.00265   1.07
exper       0.03201    0.00258   0.00281   1.09
exper^2    -0.00055    0.00005   0.00005   1.01
female     -0.11457    0.01666   0.01663   1.00
agrees with lstsq: True

return to schooling: full OLS 0.101074, FWL 0.101074, within 0.101074
no region FE        : 0.117833
one more year of school: +10.64% wage; 95% HC1 CI for beta [0.0959, 0.1063]
```

The estimated return $\hat{\rho} = 0.1011$ is close to the truth 0.10; full OLS, FWL and the within estimator agree to all printed digits, as Theorem 13.7 guarantees. Omitting regional effects inflates the estimate to 0.118 because high-schooling regions are also high-wage regions. The HC1 standard error of $\hat{\rho}$ is 7% larger than the classical one, reflecting the error variance that rises with schooling. Since $\log w$ is in log points, the exact proportional effect is $e^{\hat{\rho}} - 1 \approx 10.6\%$.

⋆ Key Concept: Robust inference is the default

In cross-sectional microdata, report heteroskedasticity-robust (HC1 or HC3) standard errors unless there is a strong reason not to. Homoskedasticity is an assumption about the data that is rarely plausible for wages, consumption or firm outcomes. With clustered sampling designs (households within enumeration areas), cluster-robust errors are needed instead.

13.2.5 Bootstrap confidence intervals

The bootstrap (Chapter 11) offers a check on the asymptotic normal interval $\hat{\rho} \pm 1.96\ \mathrm{se}_{\mathrm{HC1}}$. Two schemes respect heteroskedasticity:

- the **pairs bootstrap** resamples whole observations $(y_i, \mathbf{x}_i)$ with replacement and re-estimates; it treats the regressors as random;
- the **wild bootstrap** (Wu, 1986; Mammen, 1993) keeps $\mathbf{X}$ fixed and sets $y_i^* = \mathbf{x}_i^\top \hat{\boldsymbol{\beta}} + \hat{e}_i v_i$ with independent weights v_i of mean zero and variance one (here Rademacher, ± 1 with probability $\tfrac{1}{2}$), so each bootstrap error keeps the scale of its own residual.

Because $\hat{\rho}^* = \hat{\rho} + \mathbf{a}_1^\top(\hat{\mathbf{e}} \odot \mathbf{v})$ with $\mathbf{a}_1^\top$ the schooling row of $(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$, the wild bootstrap needs no

re-fitting at all — a useful trick when B is large.

Listing 13.4: Pairs and wild bootstrap intervals for the return to schooling
ch13/ch13_mincer_bootstrap.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Pairs and wild bootstrap confidence intervals for the return to schooling.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt


def simulate_lfs(rng, n=4000, n_reg=10):
    """Same simulated labour force survey as ch13_mincer_ols.py."""
    region = rng.integers(0, n_reg, n)
    reg_fe = np.linspace(-0.3, 0.3, n_reg)
    reg_edu = np.linspace(-2.0, 2.0, n_reg)
    female = (rng.random(n) < 0.5).astype(float)
    age = rng.integers(18, 61, n)
    educ = np.clip(np.round(9 + reg_edu[region] + 3.5 * rng.standard_normal(n)),
                   0, 20)
    exper = np.maximum(age - educ - 6, 0)
    sd = 0.10 + 0.045 * educ
    logw = (1.0 + reg_fe[region] + 0.10 * educ + 0.035 * exper
            - 0.0006 * exper**2 - 0.12 * female + sd * rng.standard_normal(n))
    return logw, educ, exper, female, region


rng = np.random.default_rng(803)
logw, educ, exper, female, region = simulate_lfs(rng)
n = logw.size
D = (region[:, None] == np.arange(1, 10)).astype(float)
X = np.column_stack([np.ones(n), educ, exper, exper**2, female, D])
k = X.shape[1]
Q, R = np.linalg.qr(X)
beta = np.linalg.solve(R, Q.T @ logw)
e = logw - X @ beta
Rinv = np.linalg.inv(R)
XtX_inv = Rinv @ Rinv.T
V = n / (n - k) * XtX_inv @ ((X * e[:, None]**2).T @ X) @ XtX_inv
se_hc1 = np.sqrt(V[1, 1])
# row 1 of (X'X)^{-1} X' maps any outcome vector to its educ coefficient
a1 = XtX_inv[1] @ X.T

B = 1999
b_pairs = np.empty(B)
b_wild = np.empty(B)
for b in range(B):
    idx = rng.integers(0, n, n)                       # resample workers
    Xb, yb = X[idx], logw[idx]
    b_pairs[b] = np.linalg.lstsq(Xb, yb, rcond=None)[0][1]
    v = rng.choice([-1.0, 1.0], n)                    # Rademacher weights
    b_wild[b] = beta[1] + a1 @ (e * v)                # fixed design, y* = Xb + e v

z = 1.959964
print(f"beta_educ = {beta[1]:.5f}, HC1 se = {se_hc1:.5f}")
print(f"{'method':<22}{'se':>9}{'95% lower':>11}{'95% upper':>11}")
```

```
print(f"{'normal, HC1':<22}{se_hc1:9.5f}{beta[1] - z * se_hc1:11.5f}"
      f"{beta[1] + z * se_hc1:11.5f}")
for name, bs in [("pairs, percentile", b_pairs), ("wild, percentile", b_wild)]:
    lo, hi = np.quantile(bs, [0.025, 0.975])
    print(f"{name:<22}{bs.std(ddof=1):9.5f}{lo:11.5f}{hi:11.5f}")
lo, hi = np.quantile(b_pairs, [0.025, 0.975])
print(f"{'pairs, basic':<22}{'':9}{2 * beta[1] - hi:11.5f}{2 * beta[1] - lo:11.5f}")
pct = 100 * (np.exp(np.quantile(b_pairs, [0.025, 0.975])) - 1)
print(f"implied % wage gain per year (pairs): [{pct[0]:.2f}%, {pct[1]:.2f}%]")

fig, ax = plt.subplots(figsize=(6, 3.8))
ax.hist(b_pairs, bins=50, density=True, alpha=0.55, label="pairs bootstrap")
ax.hist(b_wild, bins=50, density=True, alpha=0.55, label="wild bootstrap")
grid = np.linspace(b_pairs.min(), b_pairs.max(), 200)
ax.plot(grid, np.exp(-0.5 * ((grid - beta[1]) / se_hc1)**2)
        / (se_hc1 * np.sqrt(2 * np.pi)), "k-", lw=1.5, label="Normal(b, HC1 se)")
ax.axvline(beta[1], color="k", ls="--", lw=1)
ax.xaxis.set_major_locator(plt.MaxNLocator(6))
ax.set_xlabel("estimated return to one year of schooling")
ax.set_ylabel("density")
ax.legend(fontsize=9)
plt.savefig("ch13_mincer_boot.pdf", bbox_inches="tight")
```

Output

```
beta_educ = 0.10107, HC1 se = 0.00265
method                       se  95% lower  95% upper
normal, HC1             0.00265    0.09589    0.10626
pairs, percentile       0.00268    0.09565    0.10631
wild, percentile        0.00259    0.09606    0.10602
pairs, basic                       0.09584    0.10649
implied % wage gain per year (pairs): [10.04%, 11.22%]
```

All intervals agree closely (Fig. 13.2): with $n = 4000$ and a well-behaved design, the asymptotic theory of Chapter 6 is accurate. The bootstrap earns its keep when n is small, when the statistic is a nonlinear function of $\hat{\beta}$ (such as $e^{\hat{\rho}} - 1$ or a ratio of coefficients) or when the sampling design is complex.

13.2.6 Interpretation and the endogeneity problem

The simulation above was built so that schooling is uncorrelated with the error. In real data it almost certainly is not. Two mechanisms pull in opposite directions.

Proposition 13.9 (Ability bias and attenuation)**.** *Suppose* $\log w = \beta_0 + \rho S + \gamma A + u$ *with unobserved ability* A, $\operatorname{Cov}(S, u) = \operatorname{Cov}(A, u) = 0$, *and schooling is observed with classical error,* $\tilde{S} = S + \eta$, η *independent of everything else. The probability limit of the simple OLS slope of* $\log w$ *on* $\tilde{S}$ *is*

$$\operatorname{plim} \hat{\rho} = \frac{\rho \operatorname{Var}(S) + \gamma \operatorname{Cov}(S, A)}{\operatorname{Var}(S) + \operatorname{Var}(\eta)}. \tag{13.6}$$

With $\operatorname{Var}(\eta) = 0$ *this is* $\rho + \gamma \operatorname{Cov}(S, A)/\operatorname{Var}(S)$ *(upward ability bias when* $\gamma \operatorname{Cov}(S, A) > 0$*); with* $\gamma = 0$ *it is* $\rho \operatorname{Var}(S)/\{\operatorname{Var}(S) + \operatorname{Var}(\eta)\}$ *(attenuation towards zero).*

Proof. $\operatorname{plim} \hat{\rho} = \operatorname{Cov}(\tilde{S}, \log w)/\operatorname{Var}(\tilde{S})$ by the law of large numbers and Slutsky's theorem (Chapter 6). Expand $\operatorname{Cov}(S+\eta, \rho S+\gamma A+u) = \rho \operatorname{Var}(S)+\gamma \operatorname{Cov}(S, A)$ and $\operatorname{Var}(S+\eta) = \operatorname{Var}(S)+\operatorname{Var}(\eta)$. □

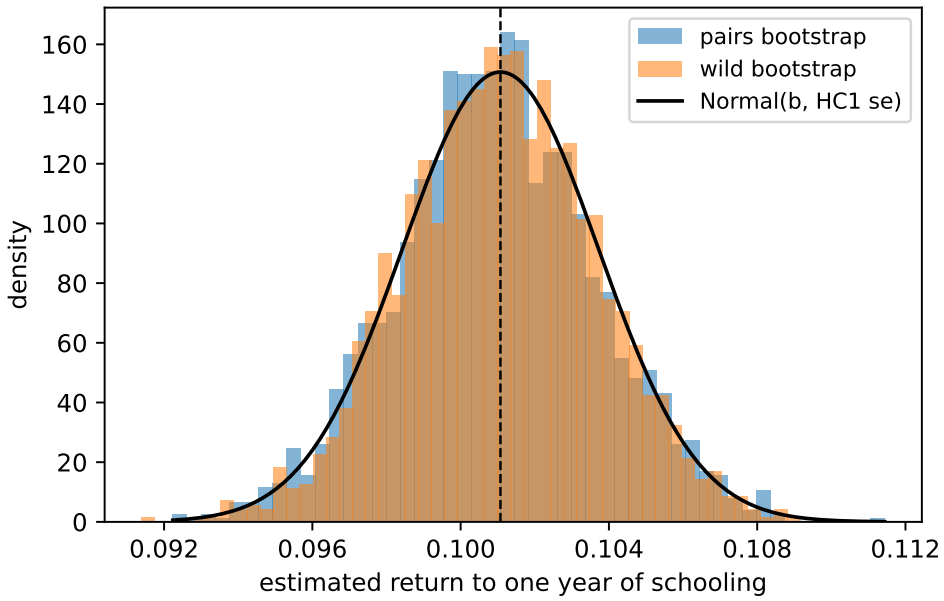


Figure 13.2: Bootstrap distributions of the return to schooling ($B = 1999$) compared with the normal approximation based on the HC1 standard error. Simulated data.

An *instrumental variable* Z that shifts schooling ($\mathrm{Cov}(Z, S) \neq 0$, relevance) but affects wages only through schooling ($\mathrm{Cov}(Z, A) = \mathrm{Cov}(Z, u) = \mathrm{Cov}(Z, \eta) = 0$, exclusion) solves both problems: $\hat{\rho}_{IV} = \widehat{\mathrm{Cov}}(Z, \log w)/\widehat{\mathrm{Cov}}(Z, \tilde{S}) \xrightarrow{p} \rho$. Classic instruments include quarter of birth interacted with compulsory schooling laws (Angrist and Krueger, 1991) and exposure to a large school-construction programme (Duflo, 2001); Card (2001) surveys the evidence. The next listing simulates a hypothetical reform that raises schooling by one year for exposed cohorts.

Listing 13.5: Monte Carlo of ability bias, measurement error and IV ch13/ch13_mincer_iv.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Monte Carlo: ability bias and measurement error in OLS; an IV preview.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)
R, n, beta_true = 1000, 2000, 0.08
gamma_ability, me_sd = 0.10, 1.5          # ability effect, measurement-error sd


def slope(x, y):
    xc = x - x.mean()
    return xc @ (y - y.mean()) / (xc @ xc)


def iv(z, x, y):
    zc = z - z.mean()
    return zc @ (y - y.mean()) / (zc @ (x - x.mean()))


est = np.empty((R, 4))
for r in range(R):
    ability = rng.standard_normal(n)
```

```
    z = (rng.random(n) < 0.5).astype(float)            # exposure to a reform
    educ = 9 + 1.0 * z + 1.5 * ability + 2.5 * rng.standard_normal(n)
    logw = (1 + beta_true * educ + gamma_ability * ability
            + 0.4 * rng.standard_normal(n))
    educ_obs = educ + me_sd * rng.standard_normal(n)  # survey recall error
    fs = slope(z, educ_obs)                            # first stage
    resid = educ_obs - educ_obs.mean() - fs * (z - z.mean())
    F = fs**2 / (resid @ resid / (n - 2) / ((z - z.mean()) @ (z - z.mean())))
    est[r] = [slope(educ, logw), slope(educ_obs, logw), iv(z, educ_obs, logw), F]

var_e = 1.5**2 + 2.5**2 + 0.25
plim_ols = beta_true + gamma_ability * 1.5 / var_e
plim_me = (beta_true * var_e + gamma_ability * 1.5) / (var_e + me_sd**2)
print(f"true return = {beta_true}, {R} samples of n = {n}")
print(f"{'estimator':<32}{'mean':>8}{'sd':>8}{'plim':>8}")
print(f"{'OLS, true schooling':<32}{est[:, 0].mean():8.4f}{est[:, 0].std():8.4f}"
      f"{plim_ols:8.4f}")
print(f"{'OLS, mismeasured schooling':<32}{est[:, 1].mean():8.4f}"
      f"{est[:, 1].std():8.4f}{plim_me:8.4f}")
print(f"{'IV (reform exposure)':<32}{np.median(est[:, 2]):8.4f}"
      f"{est[:, 2].std():8.4f}{beta_true:8.4f}")
print(f"median first-stage F = {np.median(est[:, 3]):.1f}")
cover = np.mean(np.abs(est[:, 2] - beta_true) < 1.96 * est[:, 2].std())
print(f"share of IV estimates within 1.96 sd of truth: {cover:.3f}")
print("(IV row reports the median: IV has heavy tails in finite samples)")
```

Output

```
true return = 0.08, 1000 samples of n = 2000
estimator                          mean      sd    plim
OLS, true schooling              0.0973  0.0031  0.0971
OLS, mismeasured schooling       0.0774  0.0029  0.0773
IV (reform exposure)             0.0807  0.0204  0.0800
median first-stage F = 45.8
share of IV estimates within 1.96 sd of truth: 0.946
(IV row reports the median: IV has heavy tails in finite samples)
```

Ability bias raises OLS from 0.080 to 0.097; adding recall error in schooling pulls it back down to 0.077. The two biases almost cancel *by coincidence*, which is why an OLS estimate close to a "plausible" value is no evidence of its validity. IV is centred on the truth but its standard deviation is about seven times that of OLS: identification is bought with precision. The probability limits in the last column are those of (13.6).

△ Caution: A Mincer coefficient is not automatically a causal return

Without a credible identification strategy, $\hat{\rho}$ is a *conditional association*: it describes how wages differ between otherwise similar workers with different schooling. Reading it as "the wage gain if the government provides one more year of schooling" requires exogeneity, no general-equilibrium effects (if everyone gets more schooling, the skill premium may fall) and homogeneous returns. IV estimates, in turn, measure the return for those whose schooling responds to the instrument (a local average treatment effect; Angrist and Pischke, 2009).

Policy Lens: Returns to education: data, ethics and communication

Data quality. Labour force surveys often record earnings only for wage employees, omitting the self-employed and informal workers, who are the majority in many low- and middle-income labour markets; estimates may therefore describe a selected group. Schooling is self-reported and earnings are top-coded or rounded. **Ethics.** Microdata must be anonymised; small regions combined with rare occupations can identify individuals. **Communicating uncertainty.** Report an interval, not a point ("between 10.0% and 11.2% in this sample"), say which population it describes, and state the identification assumption in plain words. A minister should hear "this is what workers with more schooling earn today, not necessarily what extra schooling would add" unless there is a design that justifies the stronger claim (Manski, 2013).

Graduate Perspective: Heterogeneous returns and the limits of a single number

When returns vary across individuals, ρ_i, OLS and different IVs estimate different weighted averages of ρ_i. Card (2001) shows that IV estimates often exceed OLS, consistent with high returns for people at the margin of schooling choices, who may be credit-constrained. Heckman et al. (2006) develop the treatment-effect interpretation and show how to estimate internal rates of return non-parametrically. Machine-learning methods for heterogeneous effects (causal forests, double machine learning) generalise the FWL idea of partialling out controls to flexible nuisance functions.

Exercises

13.2.1 Theory Show that (13.3) solves the normal equations, and that $\mathrm{cond}(\mathbf{X}^\top\mathbf{X}) = \mathrm{cond}(\mathbf{X})^2$ using the SVD of $\mathbf{X}$.

13.2.2 Theory Prove that the within estimator (demeaning by region) gives the same slope as including region dummies, by showing that $\mathbf{M}_Z$ for a full set of group dummies subtracts group means.

13.2.3 Theory Derive the HC0 estimator as a plug-in estimator of the asymptotic variance $\mathbf{Q}^{-1}\mathbb{E}(u_i^2\mathbf{x}_i\mathbf{x}_i^\top)\mathbf{Q}^{-1}$, where $\mathbf{Q} = \mathbb{E}(\mathbf{x}_i\mathbf{x}_i^\top)$, and state the conditions under which it is consistent.

13.2.4 Python Add HC3 standard errors, $\hat{e}_i^2/(1-h_{ii})^2$ in the meat, to `ch13_mincer_ols.py`, computing the leverages h_{ii} as the squared row norms of the thin $\mathbf{Q}$ factor, without forming $\mathbf{H}$. Compare HC0, HC1 and HC3 for $n = 200$ and $n = 4000$.

13.2.5 Python Use the bootstrap to obtain a 95% interval for the experience level at which log wages peak, $E^* = -\beta_1/(2\beta_2)$. Compare with the delta method (Chapter 7).

13.2.6 ★ Graduate In `ch13_mincer_iv.py`, reduce the first-stage effect of the reform from 1.0 to 0.2 years. Report the median first-stage F, the bias and the dispersion of IV. Relate your findings to the weak-instrument literature.

13.2.7 Policy A ministry proposes to justify free senior high schooling with the OLS Mincer return. Write a 300-word brief explaining which biases might affect the estimate, in which direction, what evidence would be more convincing, and how you would present the uncertainty.

13.3 Case Study 2: Poverty Targeting with Logistic Regression

Cash-transfer programmes must decide who is eligible. Measuring consumption for every household is expensive, so many programmes use a *proxy-means test* (PMT): a score computed

from easily verified characteristics (assets, dwelling quality, household size, location) that predicts poverty. Building a PMT is a prediction problem in the sense of Kleinberg et al. (2015): the policy question ("is this household poor?") does not need a causal effect, but it does need honest out-of-sample accuracy and attention to who is misclassified.

13.3.1 The simulated household survey

We simulate $n = 4000$ households. A latent living standard $W_i \sim \mathcal{N}(-0.6R_i, 1)$, where R_i indicates rural residence, drives ownership of ten assets through a latent-index model, $\text{asset}_{ij} = \mathbf{1}\{a_j + b_j W_i + \varepsilon_{ij} > 0\}$, and drives log consumption per head, $\log c_i = 7 + 0.55 W_i - 0.07\,\text{hhsize}_i + 0.35\,\xi_i$. A household is *poor* if $\log c_i$ is below the 30th percentile. Assets are thus noisy proxies of W_i, and W_i is an imperfect proxy of poverty — exactly the situation in real PMTs.

13.3.2 Two scores: a PCA asset index and a logit PMT

PCA asset index. Filmer and Pritchett (2001) proposed using the first principal component of standardised asset indicators as a wealth index when consumption is not measured. If $\mathbf{Z}$ is the $n \times p$ standardised asset matrix with SVD $\mathbf{Z} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, the index is $\mathbf{Z}\mathbf{v}_1$, the linear combination with maximal variance subject to $\|\mathbf{v}_1\| = 1$ (Chapter 3). It is *unsupervised*: it never looks at poverty status.

Logit PMT. When a survey measures consumption for a sample, we can instead fit $\mathbb{P}(\text{poor}_i = 1 \mid \mathbf{x}_i) = \sigma(\mathbf{x}_i^\top \boldsymbol{\beta})$ with $\sigma(t) = 1/(1 + e^{-t})$ by maximum likelihood (Chapters 5 and 7) and apply the fitted model to the whole population. The log-likelihood, gradient and Hessian are

$$\ell(\boldsymbol{\beta}) = \sum_i \{y_i \eta_i - \log(1 + e^{\eta_i})\}, \quad \nabla \ell = \mathbf{X}^\top(\mathbf{y} - \mathbf{p}), \quad \nabla^2 \ell = -\mathbf{X}^\top \mathbf{W} \mathbf{X}, \tag{13.7}$$

with $\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$, $p_i = \sigma(\eta_i)$ and $\mathbf{W} = \operatorname{diag}\{p_i(1 - p_i)\}$.

Theorem 13.10 (Newton's method for the logit is IRLS)**.** *The Newton step for maximising* (13.7) *can be written as the weighted least squares problem*

$$\boldsymbol{\beta}^{(t+1)} = (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \mathbf{z}, \qquad \mathbf{z} = \mathbf{X}\boldsymbol{\beta}^{(t)} + \mathbf{W}^{-1}(\mathbf{y} - \mathbf{p}), \tag{13.8}$$

with $\mathbf{W}$ *and* $\mathbf{p}$ *evaluated at* $\boldsymbol{\beta}^{(t)}$*. Because* ℓ *is concave, any stationary point is the global maximiser; if* $\mathbf{X}$ *has full column rank and the data are not separable, the maximiser exists, is unique, and Newton's method converges quadratically near it.*

Proof sketch. Newton gives $\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \mathbf{p}) = (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \{\mathbf{X}\boldsymbol{\beta}^{(t)} + \mathbf{W}^{-1}(\mathbf{y} - \mathbf{p})\}$. Concavity follows because $-\mathbf{X}^\top \mathbf{W} \mathbf{X}$ is negative semidefinite, and negative definite when $\mathbf{X}$ has full rank since $0 < p_i < 1$. Quadratic convergence is the standard local result for Newton's method with a Lipschitz Hessian (Chapter 8). □

Listing 13.6: PCA asset index and logit PMT by Newton/IRLS, checked against scikit-learn
ch13/ch13_pmt_logit.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Proxy-means test: PCA asset index and a logit fitted by Newton/IRLS by hand.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

ASSETS = ["radio", "tv", "fridge", "phone", "motorbike", "floor", "electric",
```

```
          "water", "toilet", "gas"]


def simulate_households(rng, n=4000):
    """Simulated household survey: assets, size, rural, poverty status."""
    rural = (rng.random(n) < 0.55).astype(float)
    hhsize = np.clip(rng.poisson(4.5 + 1.0 * rural), 1, 14).astype(float)
    welfare = rng.standard_normal(n) - 0.6 * rural   # latent living standard
    a = np.array([0.8, -0.2, -1.0, 1.2, -0.8, 0.0, 0.3, -0.4, -0.3, -1.3])
    b = np.array([0.5, 1.1, 1.2, 0.6, 0.6, 0.9, 1.0, 0.9, 1.0, 1.1])
    assets = (a + np.outer(welfare, b)
              + rng.standard_normal((n, a.size)) > 0).astype(float)
    logc = (7.0 + 0.55 * welfare - 0.07 * hhsize
            + 0.35 * rng.standard_normal(n))       # log consumption per head
    poor = (logc < np.quantile(logc, 0.30)).astype(float)
    return assets, hhsize, rural, poor


def irls_logit(X, y, tol=1e-10, max_iter=25, verbose=True):
    """Newton-Raphson (= IRLS) for the logistic MLE."""
    beta = np.zeros(X.shape[1])
    for it in range(max_iter):
        eta = X @ beta
        p = 1 / (1 + np.exp(-eta))
        w = p * (1 - p)
        grad = X.T @ (y - p)
        H = (X * w[:, None]).T @ X                  # minus the Hessian
        step = np.linalg.solve(H, grad)
        beta = beta + step
        ll = np.sum(y * eta - np.logaddexp(0, eta))
        if verbose:
            print(f"  iter {it}: loglik(before step) = {ll:10.4f}, "
                  f"|step| = {np.linalg.norm(step):.2e}")
        if np.linalg.norm(step) < tol:
            break
    return beta, np.linalg.inv(H)


rng = np.random.default_rng(803)
assets, hhsize, rural, poor = simulate_households(rng)
n = poor.size
train = rng.random(n) < 0.7
test = ~train

# (a) PCA asset index (Filmer-Pritchett style): first PC of standardised assets
mu, sd = assets[train].mean(0), assets[train].std(0)
Zs = (assets - mu) / sd
_, s, Vt = np.linalg.svd(Zs[train], full_matrices=False)
w1 = Vt[0] * np.sign(Vt[0].sum())                   # orient: more assets = richer
index = Zs @ w1
print(f"PC1 explains {s[0]**2 / np.sum(s**2):.1%} of standardised asset variance")
for row in (slice(0, 5), slice(5, 10)):
    print("  weights:", " ".join(f"{a}={w:.2f}"
                                 for a, w in zip(ASSETS[row], w1[row])))

# (b) logit proxy-means test by IRLS
X = np.column_stack([np.ones(n), assets, hhsize, rural])
```

```python
print("IRLS iterations:")
beta, cov = irls_logit(X[train], poor[train])
sk = LogisticRegression(penalty="none", tol=1e-12, max_iter=5000)
sk.fit(X[train, 1:], poor[train])
b_sk = np.r_[sk.intercept_, sk.coef_.ravel()]
print(f"max |beta_IRLS - beta_sklearn| = {np.max(np.abs(beta - b_sk)):.1e}")
se = np.sqrt(np.diag(cov))
for nm, j in [("tv", 2), ("fridge", 3), ("hhsize", 11), ("rural", 12)]:
    print(f"  {nm:<7} coef {beta[j]:7.3f}  se {se[j]:.3f}  "
          f"odds ratio {np.exp(beta[j]):.2f}")

p_test = 1 / (1 + np.exp(-X[test] @ beta))
print(f"test AUC: PCA index {roc_auc_score(poor[test], -index[test]):.3f}, "
      f"logit PMT {roc_auc_score(poor[test], p_test):.3f}")
```

Output

```
PC1 explains 33.0% of standardised asset variance
  weights: radio=0.22 tv=0.36 fridge=0.35 phone=0.24 motorbike=0.26
  weights: floor=0.33 electric=0.35 water=0.33 toilet=0.35 gas=0.32
IRLS iterations:
  iter 0: loglik(before step) = -1958.8339, |step| = 1.40e+00
  iter 1: loglik(before step) = -1236.6313, |step| = 7.65e-01
  iter 2: loglik(before step) = -1128.3275, |step| = 6.73e-01
  iter 3: loglik(before step) = -1104.9411, |step| = 3.68e-01
  iter 4: loglik(before step) = -1102.6694, |step| = 6.94e-02
  iter 5: loglik(before step) = -1102.6273, |step| = 1.85e-03
  iter 6: loglik(before step) = -1102.6273, |step| = 1.23e-06
  iter 7: loglik(before step) = -1102.6273, |step| = 5.56e-13
max |beta_IRLS - beta_sklearn| = 5.1e-05
  tv      coef  -0.879  se 0.149  odds ratio 0.42
  fridge  coef  -1.057  se 0.262  odds ratio 0.35
  hhsize  coef   0.258  se 0.025  odds ratio 1.29
  rural   coef   0.480  se 0.113  odds ratio 1.62
test AUC: PCA index 0.857, logit PMT 0.877
```

Three lessons. First, all PC1 weights are positive, so the index is a sensible "more assets, less poor" score; it explains only a third of asset variance, a typical figure. Second, the step norms $0.07 \to 2\times10^{-3} \to 10^{-6} \to 6\times10^{-13}$ show the quadratic convergence of Theorem 13.10: the number of correct digits roughly doubles each iteration. Our implementation matches scikit-learn's L-BFGS solution to that solver's tolerance, and also delivers standard errors from the inverse observed information $(\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}$. Third, the supervised logit beats the PCA index in test AUC (0.877 versus 0.857), because it uses household size and location and weights assets by their predictive, not their variance, content.

△ Caution: Separation and the non-existence of the MLE

If some combination of predictors perfectly separates poor from non-poor households (say, no household with a fridge is poor in a small sample), the logit likelihood increases towards its supremum as a coefficient diverges, and Newton's method "converges" to huge coefficients with enormous standard errors. Small samples, many binary indicators and rare categories make this common. Regularisation (below) or merging categories restores a finite solution.

13.3.3 Regularisation chosen by cross-validation

Real PMT datasets contain dozens of candidate indicators, many weakly related to poverty. We add 30 irrelevant binary "survey items" and shrink the training sample to about 1000 households. Penalised logistic regression (Chapter 12) maximises $\ell(\boldsymbol{\beta}) - \lambda \,\mathrm{pen}(\boldsymbol{\beta})$ with $\mathrm{pen} = \frac{1}{2}\|\boldsymbol{\beta}\|_2^2$ (L2, ridge) or $\|\boldsymbol{\beta}\|_1$ (L1, lasso), leaving the intercept unpenalised and standardising the predictors so that one λ treats them equally. scikit-learn parameterises the penalty by $C = 1/\lambda$, chosen here by 5-fold cross-validated log-loss.

13.3.4 Targeting under a budget: inclusion and exclusion errors

A budget that covers a share q of households turns any score s_i into a rule: enrol the qn households with the highest predicted poverty. Two error rates matter.

Definition 13.11 (Targeting errors)**.** For a selection rule $D_i \in \{0,1\}$ and true poverty status Y_i, the *inclusion error* (leakage) is $\mathbb{P}(Y = 0 \mid D = 1)$, the share of beneficiaries who are not poor; the *exclusion error* (under-coverage) is $\mathbb{P}(D = 0 \mid Y = 1)$, the share of the poor who are left out. In ROC terms, the exclusion error is one minus the true positive rate.

Proposition 13.12 (AUC as a probability)**.** *For a continuous score, the area under the ROC curve equals $\mathbb{P}(s_{poor} > s_{non\text{-}poor})$ for an independently drawn poor and non-poor household; its empirical version is the Mann–Whitney statistic.*

Proof sketch. The ROC curve plots $\mathrm{TPR}(t) = \mathbb{P}(s_1 > t)$ against $\mathrm{FPR}(t) = \mathbb{P}(s_0 > t)$. As t decreases, FPR increases with $d\,\mathrm{FPR} = f_0(t)\,dt$, so $\mathrm{AUC} = \int \mathbb{P}(s_1 > t)\, f_0(t)\, dt = \mathbb{P}(s_1 > s_0)$ for independent s_1, s_0. □

Listing 13.7: Regularised PMT, budget-constrained targeting errors, ROC and a fairness audit
ch13/ch13_pmt_targeting.py

```python
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Regularised PMT by CV; inclusion/exclusion errors at a budget; ROC; fairness.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.metrics import roc_auc_score, roc_curve, log_loss
from sklearn.preprocessing import StandardScaler


def simulate_households(rng, n=4000, n_noise=30):
    """Simulated survey (as in ch13_pmt_logit.py) plus irrelevant items."""
    rural = (rng.random(n) < 0.55).astype(float)
    hhsize = np.clip(rng.poisson(4.5 + 1.0 * rural), 1, 14).astype(float)
    welfare = rng.standard_normal(n) - 0.6 * rural
    a = np.array([0.8, -0.2, -1.0, 1.2, -0.8, 0.0, 0.3, -0.4, -0.3, -1.3])
    b = np.array([0.5, 1.1, 1.2, 0.6, 0.6, 0.9, 1.0, 0.9, 1.0, 1.1])
    assets = (a + np.outer(welfare, b)
              + rng.standard_normal((n, a.size)) > 0).astype(float)
    logc = 7.0 + 0.55 * welfare - 0.07 * hhsize + 0.35 * rng.standard_normal(n)
    poor = (logc < np.quantile(logc, 0.30)).astype(float)
    noise = (rng.random((n, n_noise)) < 0.4).astype(float)
    return assets, hhsize, rural, poor, noise


```

```
rng = np.random.default_rng(803)
assets, hhsize, rural, poor, noise = simulate_households(rng)
n = poor.size
train = rng.random(n) < 0.25                          # small training sample
test = ~train
X = np.column_stack([assets, hhsize, rural, noise])
sc = StandardScaler().fit(X[train])
Xs = sc.transform(X)
ytr, yte = poor[train], poor[test]

# PCA asset index fitted on the training sample
Za = (assets - assets[train].mean(0)) / assets[train].std(0)
w1 = np.linalg.svd(Za[train], full_matrices=False)[2][0]
pca_score = -Za @ (w1 * np.sign(w1.sum()))          # high = predicted poor

Cs = np.logspace(-3, 1, 25)
models = {
    "logit (no penalty)": LogisticRegression(penalty="none", max_iter=5000),
    "logit L2, CV": LogisticRegressionCV(Cs=Cs, cv=5, penalty="l2",
                                         scoring="neg_log_loss", max_iter=5000),
    "logit L1, CV": LogisticRegressionCV(Cs=Cs, cv=5, penalty="l1",
                                         solver="liblinear",
                                         scoring="neg_log_loss"),
}
scores = {"PCA asset index": pca_score[test]}
for name, m in models.items():
    m.fit(Xs[train], ytr)
    scores[name] = m.predict_proba(Xs[test])[:, 1]
l1 = models["logit L1, CV"]
print(f"CV-chosen C: L2 {models['logit L2, CV'].C_[0]:.3f}, L1 {l1.C_[0]:.3f}; "
      f"L1 keeps {np.sum(l1.coef_ != 0)} of {X.shape[1]} predictors "
      f"({np.sum(l1.coef_[0, 12:] != 0)} of 30 noise items)")

share = 0.25                                          # budget covers 25%
print(f"\nbudget covers {share:.0%} of households; poverty rate "
      f"{yte.mean():.1%} (test)")
print(f"{'method':<20}{'AUC':>7}{'logloss':>9}{'inclusion':>11}{'exclusion':>11}")
for name, s in scores.items():
    sel = s >= np.quantile(s, 1 - share)
    incl = np.mean(yte[sel] == 0)                     # benefits to non-poor
    excl = np.mean(sel[yte == 1] == 0)                # poor left out
    ll = "--" if name.startswith("PCA") else f"{log_loss(yte, s):.3f}"
    print(f"{name:<20}{roc_auc_score(yte, s):7.3f}{ll:>9}{incl:11.1%}"
          f"{excl:11.1%}")

# Fairness audit of the L2 model: error rates by area
s = scores["logit L2, CV"]
sel = s >= np.quantile(s, 1 - share)
print("\nL2 PMT by area      poverty  coverage  exclusion  inclusion")
for lab, g in [("urban", rural[test] == 0), ("rural", rural[test] == 1)]:
    pr, cov = yte[g].mean(), sel[g].mean()
    ex = np.mean(sel[g & (yte == 1)] == 0)
    inc = np.mean(yte[g & sel] == 0)
    print(f"  {lab:<17}{pr:9.1%}{cov:10.1%}{ex:11.1%}{inc:11.1%}")

fig, ax = plt.subplots(figsize=(6, 3.8))
for name in ["PCA asset index", "logit L2, CV"]:
```

```
    fpr, tpr, _ = roc_curve(yte, scores[name])
    ax.plot(fpr, tpr, label=f"{name} (AUC {roc_auc_score(yte, scores[name]):.3f})")
    sel = scores[name] >= np.quantile(scores[name], 1 - share)
    ax.plot(np.mean(sel[yte == 0]), np.mean(sel[yte == 1]), "ko", ms=5)
ax.plot([0, 1], [0, 1], "k:", lw=1)
ax.set_xlabel("false positive rate (non-poor selected)")
ax.set_ylabel("true positive rate (poor selected)")
ax.set_title("ROC curves; dots = 25% budget operating point")
ax.legend(loc="lower right", fontsize=9)
plt.savefig("ch13_pmt_roc.pdf", bbox_inches="tight")
```

Output

```
CV-chosen C: L2 0.032, L1 0.147; L1 keeps 26 of 42 predictors (14 of 30 noise items)

budget covers 25% of households; poverty rate 30.2% (test)
method                  AUC  logloss  inclusion  exclusion
PCA asset index       0.862       --      30.8%      41.0%
logit (no penalty)    0.867    0.414      28.6%      40.8%
logit L2, CV          0.870    0.408      28.0%      40.4%
logit L1, CV          0.873    0.403      27.1%      39.6%

L2 PMT by area      poverty  coverage  exclusion  inclusion
  urban               18.8%      9.9%      61.7%      27.1%
  rural               39.6%     37.4%      32.1%      28.2%
```

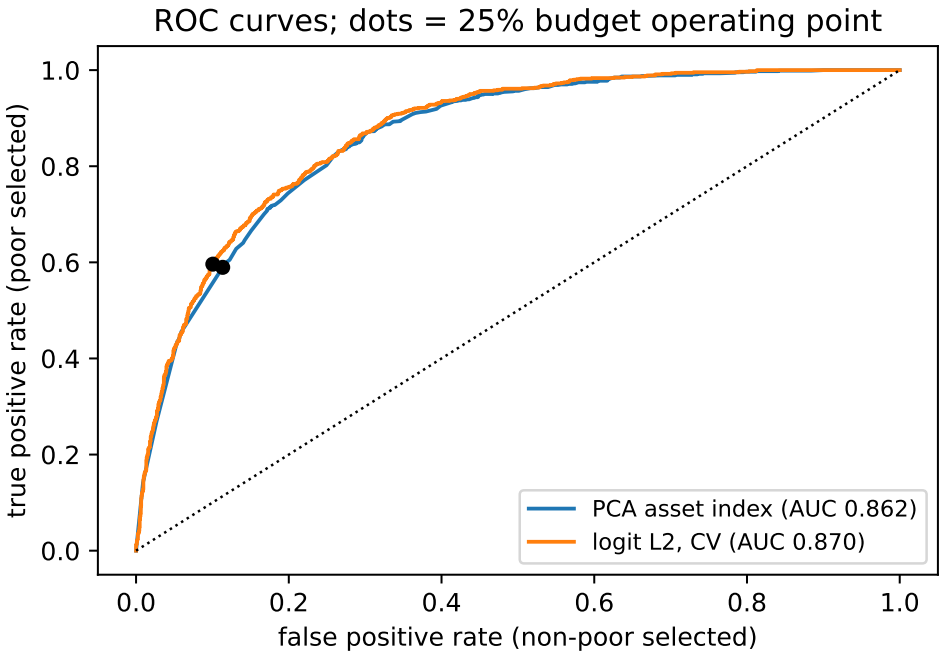


Figure 13.3: ROC curves of the PCA asset index and the L2-penalised logit PMT on the test sample. The dots mark the operating point implied by a budget covering 25% of households. Simulated data.

Regularisation helps modestly: the L1 model discards about half of the noise items and attains the best test log-loss and AUC. But the striking numbers are the error rates. Even the best

model, with AUC near 0.87, excludes about 40% of the poor and leaks about 27% of benefits to the non-poor. Part of this is arithmetic: with a poverty rate of 30% and a budget for 25%, at least one sixth of the poor must be excluded even by a perfect score. The rest is the noise linking assets to consumption. Figure 13.3 shows that the gap between the methods is small compared with the distance from the perfect corner.

The fairness audit is the most policy-relevant output. Because the model ranks urban households as less likely to be poor, the urban poor are excluded at almost twice the rate of the rural poor (62% versus 32%), even though the overall model is "accurate".

⋆ Key Concept: Accuracy metrics answer different questions

AUC measures ranking quality across all thresholds; log-loss measures calibration of probabilities; inclusion and exclusion errors measure the consequences at the *operating point the budget actually implies*. A policy evaluation of a targeting model must report the latter, overall and for the groups society cares about.

Policy Lens: Targeting: data quality, ethics and fairness

Evidence. Brown et al. (2018) find that standard PMTs in nine African countries help but leave large exclusion errors among the poorest; Aiken et al. (2022) show that machine learning on mobile-phone data can improve targeting when survey data are out of date, while raising questions of consent and privacy. **Data quality.** PMT weights estimated from an old survey may be wrong after a shock; asset questions are easy to game once households learn the formula. **Ethics and fairness.** Error rates that differ by region, gender of household head, disability or ethnicity are a fairness problem even if no protected attribute is used directly, because proxies can encode it (Obermeyer et al., 2019). Options include group-specific thresholds, community validation and an appeals process. **Communicating uncertainty.** Tell decision makers the expected number of poor households excluded per thousand enrolled, not just the AUC.

Exercises

13.3.1 Theory Derive the gradient and Hessian in (13.7) and prove that ℓ is strictly concave when $\mathbf{X}$ has full column rank.

13.3.2 Theory Show that if a vector β_0 satisfies $y_i = 1 \Leftrightarrow \mathbf{x}_i^\top \beta_0 > 0$ and $\mathbf{x}_i^\top \beta_0 \neq 0$ for all i (complete separation), then $\ell(c\beta_0) \to 0$ as $c \to \infty$, so the MLE does not exist. What happens with an L2 penalty?

13.3.3 Theory Prove Proposition 13.12 carefully, and show that the AUC is invariant to any strictly increasing transformation of the score. Why does this mean that AUC cannot assess calibration?

13.3.4 Python Modify `ch13_pmt_logit.py` to compute the robust ("sandwich") covariance of the logit MLE and compare it with $(\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1}$. When should they differ?

13.3.5 Python Plot inclusion and exclusion errors of the L2 PMT as the budget share q varies from 5% to 60%. At what q are the two error rates equal, and why?

13.3.6 ★ Graduate Implement a group-specific threshold that equalises exclusion errors between urban and rural households at the same total budget. Report the change in overall exclusion and inclusion errors and discuss the trade-off formally.

13.3.7 Policy A government wants to re-use PMT weights estimated from a survey conducted

before a drought. Write a short memo describing the risks, how you would test for them with a small new survey, and how you would communicate expected targeting errors.

13.4 Case Study 3: Macroeconomic Forecasting with Many Predictors

Central banks and finance ministries forecast inflation, output and revenue from large panels of monthly indicators: prices, production, credit, trade, surveys. With N indicators and T months, N is often comparable to or larger than T, so unrestricted regression on all predictors is infeasible or wildly overfitted. Three strategies from this course tame the problem: compress the predictors with PCA (Chapter 3), shrink coefficients with ridge or lasso (Chapter 12), and evaluate honestly with pseudo-out-of-sample forecasts and resampling (Chapter 11).

13.4.1 An approximate factor model

Definition 13.13 (Approximate factor model)**.** The panel $\mathbf{x}_t \in \mathbb{R}^N$ follows $\mathbf{x}_t = \boldsymbol{\Lambda}\mathbf{f}_t + \mathbf{e}_t$, $t = 1, \dots, T$, with $r \ll N$ latent factors $\mathbf{f}_t$, loadings $\boldsymbol{\Lambda} \in \mathbb{R}^{N \times r}$ and idiosyncratic errors $\mathbf{e}_t$ that may be weakly correlated across series and over time. The target obeys a forecasting equation $y_{t+1} = \alpha + \phi y_t + \boldsymbol{\beta}^\top \mathbf{f}_t + \varepsilon_{t+1}$.

Our simulation has $T = 300$ months, $N = 100$ indicators and $r = 3$ AR(1) factors with persistence 0.8, 0.5 and 0.3; idiosyncratic errors are AR(1) with coefficient 0.3 and standard deviation 1.5, so each single indicator is a noisy signal. The target depends on its own lag and on the first two factors.

Historical Note: Diffusion indexes

The idea of summarising many business-cycle indicators goes back to the NBER "diffusion indexes" of the mid-twentieth century. Stock and Watson (2002a,b) gave it a modern statistical foundation: estimate the factors of a large panel by principal components and use them as predictors, which they called *diffusion index* forecasting. Bai and Ng (2002) provided criteria for choosing the number of factors, and McCracken and Ng (2016) assembled FRED-MD, a public monthly panel of more than one hundred US series that has become a standard testbed.

13.4.2 Three ways to use 100 predictors

Diffusion index (PCA). At each forecast origin, standardise the panel, take the first k principal components $\hat{\mathbf{f}}_t$ and regress y_{t+1} on $(1, y_t, \hat{\mathbf{f}}_t)$ by OLS. Under the approximate factor model, as $N, T \to \infty$, $\hat{\mathbf{f}}_t$ consistently estimates $\mathbf{H}\mathbf{f}_t$ for an invertible rotation $\mathbf{H}$; since the forecast depends on the factors only through the span, the rotation is harmless.

Ridge. Regress y_{t+1} on $(y_t, \mathbf{x}_t)$ with penalty $\lambda \lVert\boldsymbol{\beta}\rVert_2^2$. In terms of the SVD $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, ridge shrinks the component of the fit along the j-th principal component by $d_j^2/(d_j^2 + \lambda)$; PCA regression keeps the first k components with weight one and discards the rest. Ridge is therefore a "soft" version of PCA regression, and De Mol et al. (2008) show that the two give very similar forecasts when the data have a factor structure.

Lasso. The penalty $\lambda \lVert\boldsymbol{\beta}\rVert_1$ selects a few individual indicators. With a factor structure, many indicators are near-substitutes and the lasso picks some of them, somewhat arbitrarily; it does best when a few series matter on their own.

13.4.3 Pseudo-out-of-sample evaluation

Definition 13.14 (Expanding-window pseudo-out-of-sample design)**.** For each origin $t = t_0, \dots, T-1$: (i) use only data up to t; (ii) estimate every model, including standardisation, PCA and tuning parameters, on that window; (iii) forecast y_{t+1} and record the error $e_{t+1}^{(m)}$ of each model m. Compare models by $\mathrm{RMSE}_m = \{P^{-1}\sum_t (e_{t+1}^{(m)})^2\}^{1/2}$ over the P forecasts.

Penalties are tuned on the last 36 months of each window and re-tuned once a year; the benchmark is an AR(1).

△ Caution: Look-ahead bias

Standardising the full panel, extracting principal components from the full sample or choosing λ by cross-validation on all T observations lets information from the evaluation period leak into the forecasts. The resulting RMSEs are optimistic, sometimes dramatically so. Every data-dependent step must sit inside the loop over forecast origins. Real-time data vintages raise a further issue: published macro series are revised, and a forecaster at time t saw only the first release.

13.4.4 Is the improvement significant? Diebold–Mariano and the block bootstrap

Definition 13.15 (Diebold–Mariano statistic)**.** Let $d_t = L(e_t^{(0)}) - L(e_t^{(1)})$ be the loss differential between a benchmark and a competitor, here with squared-error loss. Under $H_0 : \mathbb{E}d_t = 0$ and weak dependence,

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\hat{\omega}^2 / P}} \xrightarrow{d} \mathcal{N}(0,1), \tag{13.9}$$

where $\hat{\omega}^2$ is a heteroskedasticity- and autocorrelation-consistent (Newey–West) estimate of the long-run variance $\sum_{j=-\infty}^{\infty} \mathrm{Cov}(d_t, d_{t-j})$ (Diebold and Mariano, 1995).

Harvey et al. (1997) propose a small-sample correction and Student-t critical values. When the benchmark is nested in the competitor (the AR(1) is the DI model with zero factor coefficients), the DM statistic is not asymptotically normal under the null and tends to be undersized; Clark and West (2007) propose an adjusted statistic. We therefore treat DM as a descriptive measure of evidence and complement it with a *moving-block bootstrap* (Künsch, 1989): resample blocks of $L = 8$ consecutive forecast periods (keeping the pairs of errors together) to preserve serial dependence, and compute the interval of the RMSE ratio.

Listing 13.8: Expanding-window forecasts: AR, diffusion index, ridge and lasso
ch13/ch13_forecast.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Pseudo-out-of-sample forecasting with many predictors: AR, PCA, ridge, lasso.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso

rng = np.random.default_rng(803)
T, N, r = 300, 100, 3                          # months, indicators, factors

```

```

def simulate_panel(rng):
    """Simulated factor-structured panel of macro indicators and a target."""
    phi = np.array([0.8, 0.5, 0.3])
    F = np.zeros((T, r))
    for t in range(1, T):
        F[t] = phi * F[t - 1] + rng.standard_normal(r)
    Lam = rng.standard_normal((N, r))
    E = np.zeros((T, N))
    for t in range(1, T):
        E[t] = 0.3 * E[t - 1] + rng.standard_normal(N)
    X = F @ Lam.T + 1.5 * E
    y = np.zeros(T)
    for t in range(1, T):                    # target depends on lagged factors
        y[t] = (0.3 * y[t - 1] + 0.5 * F[t - 1, 0] - 0.3 * F[t - 1, 1]
                + 0.8 * rng.standard_normal())
    return X, y


def ols(Z, v):
    return np.linalg.lstsq(Z, v, rcond=None)[0]


def ridge(Z, v, lam):
    """Ridge with unpenalised intercept (Z is already standardised)."""
    zm, vm = Z.mean(0), v.mean()
    Zc = Z - zm
    b = np.linalg.solve(Zc.T @ Zc + lam * np.eye(Z.shape[1]), Zc.T @ (v - vm))
    return vm - zm @ b, b


def tune(fit_predict, grid, Z, v, n_val=36):
    """Choose a penalty on the last n_val points of the estimation window."""
    errs = [np.mean((v[-n_val:] - fit_predict(Z[:-n_val], v[:-n_val],
                                              Z[-n_val:], g))**2) for g in grid]
    return grid[int(np.argmin(errs))]


def ridge_fp(Z, v, Znew, lam):
    a, b = ridge(Z, v, lam)
    return a + Znew @ b


def lasso_fp(Z, v, Znew, alpha):
    return Lasso(alpha=alpha, max_iter=5000).fit(Z, v).predict(Znew)


X, y = simulate_panel(rng)
t0 = 150                                     # first forecast origin
lam_grid = np.logspace(0, 4, 13)
alpha_grid = np.logspace(-2.5, -0.5, 9)
fc = {m: [] for m in ["AR(1)", "PCA (DI)", "ridge", "lasso"]}
actual = []
for t in range(t0, T - 1):                   # origin t, forecast y[t+1]
    Xs = (X[:t + 1] - X[:t + 1].mean(0)) / X[:t + 1].std(0)
    ylag, ytar = y[:t], y[1:t + 1]           # regress y[s+1] on info at s
    one = np.ones(t)
```

```
    b = ols(np.column_stack([one, ylag]), ytar)
    fc["AR(1)"].append(b[0] + b[1] * y[t])
    U, S, Vt = np.linalg.svd(Xs, full_matrices=False)
    Fh = U[:, :3] * S[:3]                          # PCA diffusion indexes
    b = ols(np.column_stack([one, ylag, Fh[:t]]), ytar)
    fc["PCA (DI)"].append(b @ np.r_[1.0, y[t], Fh[t]])
    Z = np.column_stack([ylag, Xs[:t]])
    znew = np.r_[y[t], Xs[t]][None, :]
    if (t - t0) % 12 == 0:                         # re-tune once a year
        lam = tune(ridge_fp, lam_grid, Z, ytar)
        alpha = tune(lasso_fp, alpha_grid, Z, ytar)
    fc["ridge"].append(ridge_fp(Z, ytar, znew, lam)[0])
    fc["lasso"].append(lasso_fp(Z, ytar, znew, alpha)[0])
    actual.append(y[t + 1])

actual = np.array(actual)
err = {m: actual - np.array(f) for m, f in fc.items()}
P = actual.size
rmse_ar = np.sqrt(np.mean(err["AR(1)"]**2))


def dm_stat(d, lags=4):
    """Diebold-Mariano statistic with a Newey-West long-run variance."""
    dc = d - d.mean()
    lrv = dc @ dc / P
    for j in range(1, lags + 1):
        lrv += 2 * (1 - j / (lags + 1)) * (dc[j:] @ dc[:-j]) / P
    return d.mean() / np.sqrt(lrv / P)


def block_boot_ratio(e1, e0, B=2000, L=8):
    """Moving-block bootstrap interval for RMSE(model) / RMSE(AR)."""
    starts = rng.integers(0, P - L + 1, size=(B, int(np.ceil(P / L))))
    idx = (starts[:, :, None] + np.arange(L)).reshape(B, -1)[:, :P]
    ratio = np.sqrt(np.mean(e1[idx]**2, 1) / np.mean(e0[idx]**2, 1))
    return np.quantile(ratio, [0.05, 0.95])


print(f"T = {T}, N = {N} indicators, {P} expanding-window forecasts (h = 1)")
print(f"{'model':<10}{'RMSE':>8}{'rel. AR':>9}{'DM':>7}{'90% block-boot CI':>21}")
for m, e in err.items():
    rm = np.sqrt(np.mean(e**2))
    if m == "AR(1)":
        print(f"{m:<10}{rm:8.3f}{1:9.3f}")
        continue
    lo, hi = block_boot_ratio(e, err["AR(1)"])
    dm = dm_stat(err["AR(1)"]**2 - e**2)
    print(f"{m:<10}{rm:8.3f}{rm / rmse_ar:9.3f}{dm:7.2f}      [{lo:.3f}, {hi:.3f}]")
print(f"last tuned ridge lambda = {lam:.0f}, lasso alpha = {alpha:.4f}")
print("DM > 0 means the model beats AR(1); compare with N(0,1) quantiles")

fig, ax = plt.subplots(figsize=(6, 3.8))
for m in ["PCA (DI)", "ridge", "lasso"]:
    ax.plot(np.arange(t0 + 1, T), np.cumsum(err["AR(1)"]**2 - err[m]**2), label=m)
ax.axhline(0, color="k", lw=0.8)
ax.set_xlabel("forecast target month")
ax.set_ylabel("cumulative SE(AR) - SE(model)")
```

```
ax.set_title("Rising line = model outperforming the AR(1) benchmark")
ax.legend()
plt.savefig("ch13_forecast_csfe.pdf", bbox_inches="tight")
```

Output

```
T = 300, N = 100 indicators, 149 expanding-window forecasts (h = 1)
model         RMSE  rel. AR     DM    90% block-boot CI
AR(1)        0.995    1.000
PCA (DI)     0.766    0.769   5.46      [0.720, 0.823]
ridge        0.773    0.776   4.55      [0.720, 0.844]
lasso        0.770    0.773   5.21      [0.721, 0.828]
last tuned ridge lambda = 215, lasso alpha = 0.0316
DM > 0 means the model beats AR(1); compare with N(0,1) quantiles
```

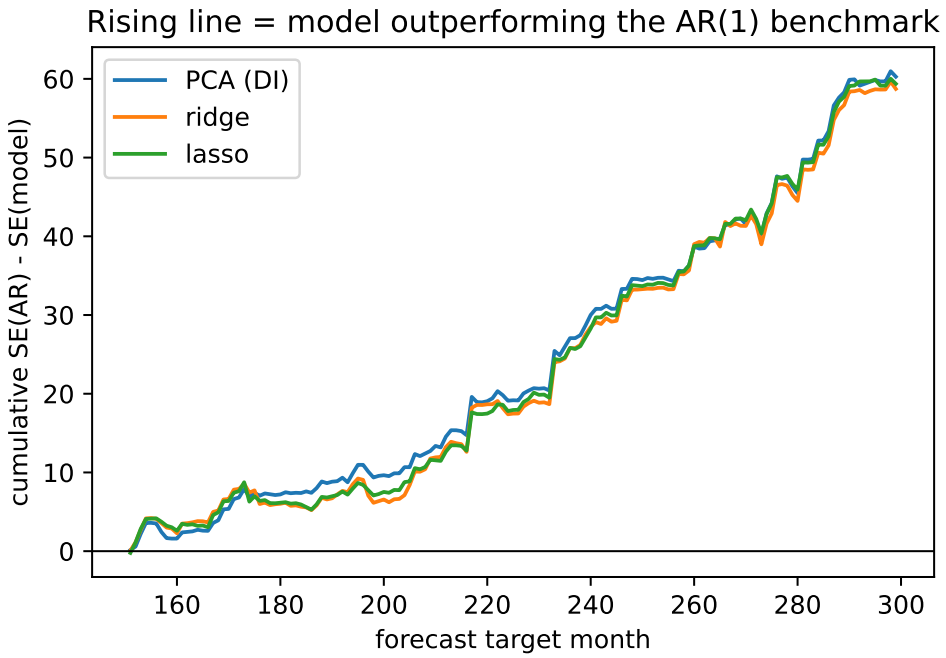


Figure 13.4: Cumulative squared-error difference between the AR(1) benchmark and each many-predictor model; an upward-sloping line means the model is forecasting better. Simulated data.

All three many-predictor methods reduce RMSE by about 23% relative to the AR(1), the DM statistics are far beyond normal critical values, and the 90% block-bootstrap intervals for the RMSE ratio exclude one. Ridge and PCA are almost indistinguishable, as the theory predicts for a factor-structured panel. Figure 13.4 shows that the gains accrue steadily rather than from a few episodes — an important robustness check, since in real data gains often come from recessions alone.

⋆ Key Concept: Forecast evaluation in three layers

(1) A *design* that mimics real time: expanding or rolling windows with every data-dependent step inside the loop. (2) A *benchmark* that is hard to beat (AR, random walk). (3) An *uncertainty statement* for the comparison that respects serial dependence (Diebold–Mariano

with HAC variance, block bootstrap). A lower RMSE without all three is an anecdote, not evidence.

Graduate Perspective: How is machine learning useful for macroeconomic forecasting?

Goulet Coulombe et al. (2022) decompose machine-learning methods into four features — nonlinearity, regularisation, cross-validation and the loss function — and evaluate each on a large US panel across many targets and horizons. Their headline conclusion is that nonlinearity is the feature that matters most, particularly at longer horizons and in periods of high macroeconomic uncertainty, while factor models remain a strong way to regularise linear models. Our simulation is linear by construction, so it can only show the regularisation part of the story; Exercise **13.4.6** adds a nonlinearity.

Policy Lens: Forecasts for budgets and monetary policy

Data quality. Many indicators in low-income economies are published with long lags, are revised substantially or have short histories; a forecast evaluation on revised data overstates real-time accuracy. **Communicating uncertainty.** A point forecast of revenue or inflation invites false precision. Present intervals or fan charts, report the historical RMSE of the method, and explain that the interval reflects model uncertainty only partially (Manski, 2019). **Ethics.** Forecasts used to justify spending cuts should be stress-tested and their track record published; an opaque model that cannot be explained to a parliamentary committee is a governance risk.

Exercises

13.4.1 Theory Show that the ridge fitted values are $\sum_j \mathbf{u}_j \frac{d_j^2}{d_j^2+\lambda} \mathbf{u}_j^\top \mathbf{y}$, and that principal-components regression with k components replaces the shrinkage factor by $\mathbf{1}\{j \le k\}$.

13.4.2 Theory Explain why the principal components of $\mathbf{X}$ identify the factors only up to an invertible rotation $\mathbf{H}$, and prove that the OLS forecast from $(1, y_t, \hat{\mathbf{f}}_t)$ is invariant to replacing $\hat{\mathbf{f}}_t$ by $\mathbf{H}\hat{\mathbf{f}}_t$.

13.4.3 Theory Show that for h-step-ahead forecasts the loss differential d_t is typically serially correlated up to order $h-1$ even under an optimal forecast, and deduce the truncation lag for the long-run variance in (13.9).

13.4.4 Python Introduce look-ahead bias deliberately: standardise and extract the PCs from the full sample once, outside the loop. By how much does the DI RMSE fall? Repeat with the lasso penalty chosen on the full sample.

13.4.5 Python Choose the number of factors $k \in \{1, \dots, 8\}$ in each window by BIC and report the distribution of the selected k and the resulting RMSE.

13.4.6 ★ Graduate Make the target nonlinear, e.g. $y_{t+1} = 0.3y_t + 0.5f_{1t} + 0.4\max(0, f_{2t})^2 + \varepsilon_{t+1}$, and add a random forest (`sklearn.ensemble.RandomForestRegressor`) on the PCA factors. Does nonlinearity help, as Goulet Coulombe et al. (2022) find?

13.4.7 Policy A finance ministry uses a single model's point forecast of tax revenue to set the budget. Draft guidance on how forecasts should be evaluated and presented (benchmark, real-time evaluation, intervals), and how the budget could incorporate forecast uncertainty.

13.5 Case Study 4: Budget Allocation under Uncertainty

A ministry has a fixed budget B to divide among J programmes — say cash transfers, school feeding, a health intervention and vocational training. Each programme has diminishing returns, and the evidence on returns is uncertain: some programmes have been evaluated repeatedly, others once or not at all. This case study combines constrained optimisation (Chapter 9), Monte Carlo (Chapter 10) and sensitivity analysis.

Historical Note: Randomised evaluations and evidence-based policy

The 2019 Sveriges Riksbank Prize in Economic Sciences in Memory of Alfred Nobel was awarded to Abhijit Banerjee, Esther Duflo and Michael Kremer "for their experimental approach to alleviating global poverty". From the mid-1990s onward, their field experiments on schooling, health and microfinance helped make randomised controlled trials a central tool of development policy (Banerjee and Duflo, 2011). A recurring theme of that literature is that evidence about programme effectiveness varies enormously in quality and quantity across programmes — precisely the uncertainty modelled below.

13.5.1 The model

Programme j receives $x_j \geq 0$ and yields benefit

$$b_j(x_j; a_j, c_j) = a_j c_j \log(1 + x_j/c_j), \qquad b_j'(x_j) = \frac{a_j}{1 + x_j/c_j}, \tag{13.10}$$

so a_j is the benefit per unit of the first unit spent and c_j is a saturation scale. Parameters are uncertain: a_j is log-normal with mean $\bar{a}_j$ and log-standard deviation s_j reflecting the strength of the evidence, and c_j is log-normal around $\bar{c}_j$. The planner solves

$$\max_{\mathbf{x} \geq \mathbf{0}} \; U_\kappa(\mathbf{x}) = \mathbb{E}\Big[\sum_j b_j(x_j)\Big] - \kappa \,\mathrm{sd}\Big[\sum_j b_j(x_j)\Big] \quad \text{s.t.} \quad \sum_j x_j = B, \tag{13.11}$$

with $\kappa = 0$ for a risk-neutral planner and $\kappa > 0$ for a planner who penalises the dispersion of outcomes, in the mean–standard-deviation tradition of Markowitz (1952).

Theorem 13.16 (KKT conditions and the shadow price)**.** *For $\kappa = 0$, U_0 is concave and (13.11) has a solution $\mathbf{x}^*$ with multiplier λ^* such that*

$$\mathbb{E}[b_j'(x_j^*)] = \lambda^* \;\text{ if } x_j^* > 0, \qquad \mathbb{E}[b_j'(0)] \leq \lambda^* \;\text{ if } x_j^* = 0. \tag{13.12}$$

Moreover, if $V(B)$ denotes the optimal value, $V'(B) = \lambda^*$: the multiplier is the marginal value of an extra unit of budget.

Proof sketch. Each b_j is concave in x_j for every parameter value, and expectation preserves concavity. The feasible set is compact and the constraints are linear, so a maximiser exists and the KKT conditions are necessary and sufficient (Chapter 9). Stationarity of the Lagrangian $U_0(\mathbf{x}) - \lambda(\sum_j x_j - B) + \sum_j \mu_j x_j$ with complementary slackness gives (13.12). The envelope theorem gives $V'(B) = \partial \mathcal{L}/\partial B = \lambda^*$. □

Proposition 13.17 (Water-filling with known parameters)**.** *If the parameters are known, (13.12) gives the closed form $x_j^* = c_j \max\{0,\, a_j/\lambda^* - 1\}$, where λ^* solves $\sum_j c_j \max\{0, a_j/\lambda - 1\} = B$. A programme is funded only if its initial marginal benefit a_j exceeds the shadow price.*

With the mean parameters of our example, $\bar{\mathbf{a}} = (1.6, 2.2, 3.0, 2.6)$ and $\bar{\mathbf{c}} = (60, 30, 15, 25)$, all four programmes are funded at $B = 100$ and $\lambda^* = \sum_j \bar{a}_j \bar{c}_j / (B + \sum_j \bar{c}_j) = 272/230 \approx 1.183$. This provides a benchmark for the numerical solution.

13.5.2 Monte Carlo: sample average approximation

The expectation in (13.11) has no closed form once c_j is random, so we replace it by an average over S draws $(\mathbf{a}^{(s)}, \mathbf{c}^{(s)})$: $\hat{U}_\kappa(\mathbf{x}) = \hat{\mu}(\mathbf{x}) - \kappa\hat{\sigma}(\mathbf{x})$. This *sample average approximation* (SAA) is solved by SLSQP using analytical gradients, $\partial\hat{\mu}/\partial x_j = S^{-1}\sum_s b_j'(x_j;\theta^{(s)})$ and $\partial\hat{\sigma}/\partial x_j = \widehat{\mathrm{Cov}}\{\sum_k b_k,\, b_j'\}/\hat{\sigma}$. Under mild conditions the SAA optimal value and solutions converge to their true counterparts as $S \to \infty$ (Shapiro et al., 2009).

⋆ Key Concept: Common random numbers

Draw the parameter sample *once* and evaluate every candidate allocation on the same draws. Then $\hat{U}(\mathbf{x})$ is a smooth deterministic function of $\mathbf{x}$ that an optimiser can handle, and differences $\hat{U}(\mathbf{x}) - \hat{U}(\mathbf{x}')$ have far smaller Monte Carlo error than the levels. Re-drawing inside the objective makes it noisy and breaks gradient-based solvers.

Listing 13.9: Budget allocation by SAA and SLSQP: expected and risk-adjusted objectives
ch13/ch13_budget.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Budget allocation under uncertain concave returns: Monte Carlo + SLSQP.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

rng = np.random.default_rng(803)
names = ["cash", "school", "health", "training"]
m_a = np.array([1.6, 2.2, 3.0, 2.6])        # mean initial marginal benefit
s_a = np.array([0.10, 0.25, 0.50, 0.70])    # log-sd: evidence quality differs
m_c = np.array([60.0, 30.0, 15.0, 25.0])    # saturation scales
S, B = 4000, 100.0                          # Monte Carlo draws, budget

# Parameter draws (common random numbers, reused for every allocation)
z = rng.standard_normal((S, 4))
a = m_a * np.exp(s_a * z - 0.5 * s_a**2)                  # E[a_j] = m_a[j]
c = m_c * np.exp(0.3 * rng.standard_normal((S, 4)) - 0.045)


def benefit_draws(x):
    """Total benefit in each draw: sum_j a_j c_j log(1 + x_j / c_j)."""
    return np.sum(a * c * np.log1p(x / c), axis=1)


def objective(x, kappa):
    """Negative of mean - kappa * sd, with its gradient."""
    tot = benefit_draws(x)
    marg = a / (1 + x / c)                                # d benefit / d x_j
    mean, sd = tot.mean(), tot.std()
    g_mean = marg.mean(0)
    g_sd = ((tot - mean)[:, None] * (marg - g_mean)).mean(0) / sd
    return -(mean - kappa * sd), -(g_mean - kappa * g_sd)


def solve(kappa, budget=B, params=None):
    cons = {"type": "eq", "fun": lambda x: x.sum() - budget,
            "jac": lambda x: np.ones(4)}
```

```
    res = minimize(objective, np.full(4, budget / 4), args=(kappa,), jac=True,
                   method="SLSQP", bounds=[(0, budget)] * 4, constraints=cons,
                   options={"ftol": 1e-12, "maxiter": 500})
    return res.x, -res.fun


def summary(x):
    tot = benefit_draws(x)
    return tot.mean(), tot.std(), np.quantile(tot, 0.05)


# Plug-in solution: ignore uncertainty, use mean parameters (closed form check)
def plug_in(budget=B):
    obj = lambda x: (-np.sum(m_a * m_c * np.log1p(x / m_c)),
                     -m_a / (1 + x / m_c))
    cons = {"type": "eq", "fun": lambda x: x.sum() - budget}
    return minimize(obj, np.full(4, budget / 4), jac=True, method="SLSQP",
                    bounds=[(0, budget)] * 4, constraints=cons).x


print(f"budget B = {B:.0f}, S = {S} Monte Carlo draws of (a, c)")
print(f"{'allocation':<22}" + "".join(f"{nm:>9}" for nm in names)
      + f"{'mean':>8}{'sd':>7}{'P5':>7}")
rows = [("plug-in (means)", plug_in()), ("max E[benefit]", solve(0.0)[0]),
        ("mean - 1.0 sd", solve(1.0)[0]), ("mean - 2.0 sd", solve(2.0)[0])]
for lab, x in rows:
    mn, sd, p5 = summary(x)
    print(f"{lab:<22}" + "".join(f"{v:9.1f}" for v in x)
          + f"{mn:8.1f}{sd:7.1f}{p5:7.1f}")

# Shadow price of the budget: KKT multiplier vs finite difference of V(B)
x0, V0 = solve(0.0)
marg = (a / (1 + x0 / c)).mean(0)
print("\nexpected marginal benefit per unit:", np.array2string(marg, precision=3))
print(f"KKT multiplier (funded programmes): {marg[x0 > 1e-6].mean():.4f}")
dB = 1.0
Vp, Vm = solve(0.0, B + dB)[1], solve(0.0, B - dB)[1]
print(f"finite difference dV/dB          : {(Vp - Vm) / (2 * dB):.4f}")

# Monte Carlo error of the optimal value: split draws into 10 batches
batches = benefit_draws(x0).reshape(10, -1).mean(1)
print(f"MC s.e. of expected benefit at optimum: {batches.std(ddof=1) / 10**0.5:.3f}")

# Sensitivity: optimal (risk-neutral) allocation as the budget varies
budgets = np.linspace(10, 250, 25)
alloc = np.array([solve(0.0, bb)[0] for bb in budgets])
fig, ax = plt.subplots(figsize=(6, 3.8))
ax.stackplot(budgets, alloc.T / budgets, labels=names, alpha=0.85)
ax.set_xlabel("total budget B")
ax.set_ylabel("share of budget")
ax.set_xlim(budgets[0], budgets[-1])
ax.set_ylim(0, 1)
ax.set_title("Risk-neutral optimal allocation shares")
ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1.0), fontsize=9)
plt.savefig("ch13_budget_shares.pdf", bbox_inches="tight")
```

Output

```
budget B = 100, S = 4000 Monte Carlo draws of (a, c)
allocation                cash   school   health training    mean     sd     P5
plug-in (means)           21.2     25.8     23.1     30.0   160.6   47.6  105.5
max E[benefit]            21.3     25.4     23.0     30.4   160.6   47.9  105.3
mean - 1.0 sd             40.0     32.3     17.4     10.3   151.8   28.6  112.9
mean - 2.0 sd             54.7     30.8     10.0      4.5   140.1   20.4  110.9

expected marginal benefit per unit: [1.159 1.159 1.159 1.159]
KKT multiplier (funded programmes): 1.1587
finite difference dV/dB          : 1.1587
MC s.e. of expected benefit at optimum: 0.698
```

Reading the output. (i) The risk-neutral solution is almost identical to the plug-in solution that ignores uncertainty. This is not an accident: benefits are *linear* in a_j, so uncertainty about a_j averages out of the expectation; only uncertainty about c_j enters, through Jensen's inequality, and it is modest. (ii) A risk-averse planner moves money from the poorly evidenced, high-variance training and health programmes ($s_j = 0.7$ and 0.5) towards well-evidenced cash transfers, giving up 5% of expected benefit to cut the standard deviation by 40% and *raise* the 5th percentile. (iii) At the risk-neutral optimum all four expected marginal benefits are equal to 1.159, the KKT multiplier, and a finite-difference derivative of the optimal value gives the same number, as Theorem 13.16 predicts. It is slightly below the plug-in value 1.183 because uncertainty in c_j lowers expected marginal returns.

Interpreting the shadow price. An extra unit of budget yields about 1.16 units of expected benefit. If raising a unit of public funds costs more than one unit of welfare (because taxation is distortionary), the budget should expand only if λ^* exceeds the marginal cost of public funds — a direct use of a Lagrange multiplier in fiscal policy.

13.5.3 Sensitivity analysis

Figure 13.5 varies B from 10 to 250. At the smallest budgets only the two programmes with the highest initial marginal benefit (health, then training) are funded; school feeding and cash transfers enter in order of $\bar{a}_j$ as the shadow price falls, and with large budgets the shares settle in proportion to $\bar{a}_j c_j$ (the large-budget limit of Proposition 13.17). Policy makers should see this kind of figure rather than one optimal vector: it shows which conclusions are robust and which hinge on the budget.

△ Caution: Optimisers amplify errors in their inputs

An optimiser pushes money towards whichever programme *appears* to have the highest return, so upward errors in estimated benefits are systematically selected — an "optimiser's curse". A programme with one spectacular but noisy evaluation will be over-funded by a risk-neutral plug-in rule. Shrinking noisy estimates towards a common mean (as in Chapter 12), penalising variance, or capping allocations are partial remedies.

Policy Lens: Allocating under uncertainty

Data quality. Benefit parameters come from evaluations in other places and times; external validity is itself uncertain and should widen s_j. **Ethics.** The objective aggregates benefits across people; it says nothing about who benefits. Distributional weights or minimum

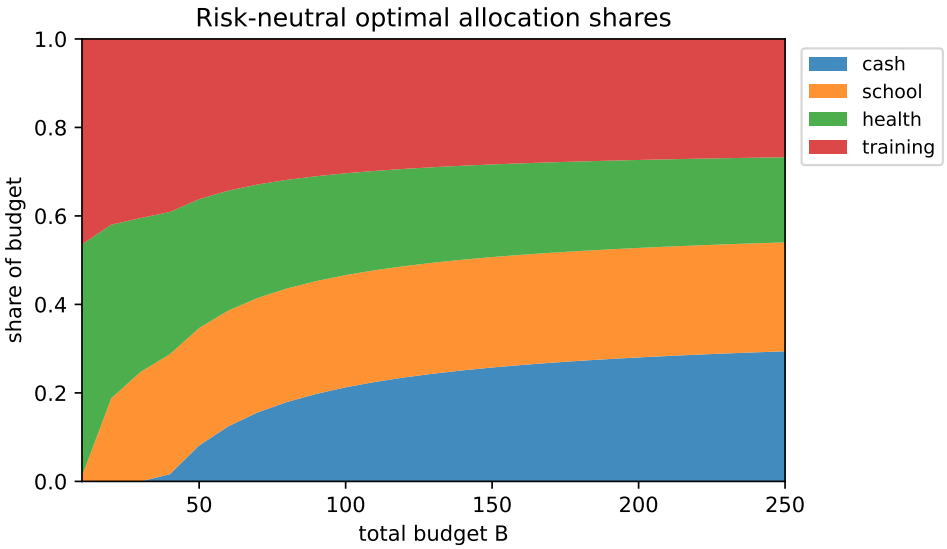


Figure 13.5: Risk-neutral optimal budget shares as the total budget varies. Programmes enter in order of their initial marginal benefit. Simulated parameters.

allocations for vulnerable groups can be added as constraints, and their shadow prices reveal what they cost. **Communicating uncertainty.** Present the allocation together with the spread of outcomes (mean, standard deviation, 5th percentile), the shadow price of the budget, and the sensitivity of the allocation to the budget and to risk aversion.

Exercises

13.5.1 Theory Prove Proposition 13.17, and show that $\lambda^*(B)$ is continuous and strictly decreasing in B. Derive λ^* in closed form when all programmes are funded.

13.5.2 Theory Show that $\mathbb{E}[b_j(x_j)] = \bar{a}_j c_j \log(1 + x_j/c_j)$ when only a_j is uncertain (independent of c_j), and hence that the risk-neutral solution coincides with the plug-in solution. Use Jensen's inequality to sign the effect of uncertainty in c_j on expected marginal benefit.

13.5.3 Theory Is U_κ concave for $\kappa > 0$? Show that $\mathrm{sd}[\sum_j b_j(x_j)]$ is not convex in general, and discuss what this implies for the KKT conditions and for SLSQP.

13.5.4 Python Replace the standard-deviation penalty with conditional value-at-risk: maximise the mean of the worst 10% of outcomes. (Hint: use the Rockafellar–Uryasev formulation with an auxiliary variable.) Compare the allocation with the mean–sd solutions.

13.5.5 Python Repeat the risk-neutral optimisation with ten independent sets of $S = 4000$ draws and report the Monte Carlo standard deviation of each x_j^* and of the optimal value. How large must S be for allocations accurate to ± 0.5?

13.5.6 ★ Graduate Implement the optimiser's curse: let the planner see noisy estimates $\hat{a}_j = a_j + \nu_j$ and allocate by plug-in. Show by simulation that the realised benefit is below the planner's predicted benefit on average, and that shrinking $\hat{a}_j$ towards their mean reduces the gap.

13.5.7 Policy Add a constraint that at least 30% of the budget goes to programmes reaching rural areas (say, school feeding and health). Report the new allocation and the shadow price of the constraint, and write two sentences explaining to a minister what that shadow price means.

13.6 Comprehensive Review

13.6.1 A concept map of the course

Figure 13.6 arranges the course as a pipeline. Linear algebra supplies the language and the algorithms of least squares; probability supplies the model of how data arise; estimation connects the two; optimisation computes estimators; simulation and resampling quantify uncertainty when formulas are unavailable; and regularisation controls the bias–variance trade-off when models are large. The dashed links are the shortcuts used repeatedly in the case studies.

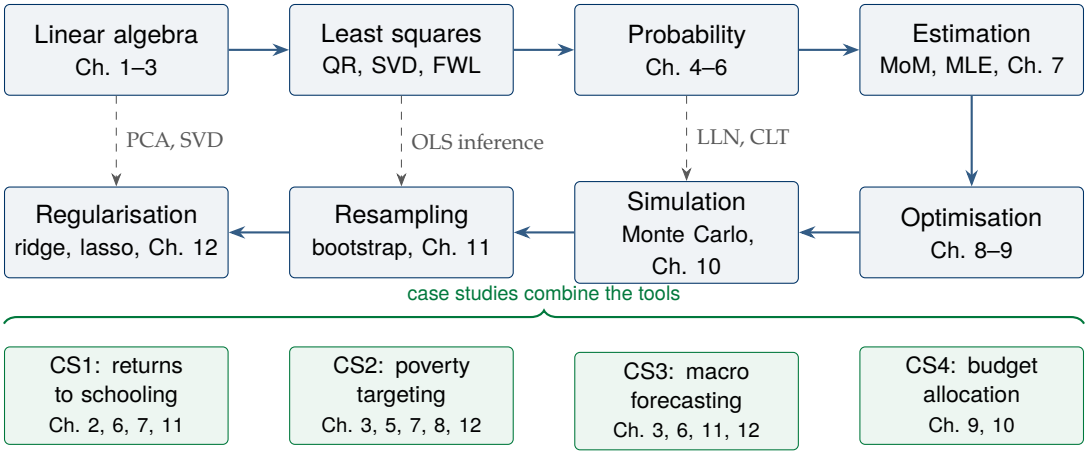


Figure 13.6: Concept map of DEP 803. Blue boxes are the mathematical building blocks in the order of the course; green boxes are the case studies of this chapter, with the chapters whose tools they use most heavily. Dashed arrows are frequently used shortcuts.

13.6.2 Key results and where they are used

Result	Statement (informal)	Used in
Floating-point error	relative rounding error $\le \varepsilon_{\text{mach}}/2$, $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$; cancellation destroys digits	Chapter 1; finite differences
QR least squares	$\hat{\boldsymbol{\beta}} = \mathbf{R}^{-1}\mathbf{Q}^\top \mathbf{y}$, error $\propto \text{cond}(\mathbf{X})$ not $\text{cond}(\mathbf{X})^2$	CS1, CS3
Frisch–Waugh–Lovell	partial out controls, then regress residual on residual	CS1 fixed effects
Spectral theorem, SVD	$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$; $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$; Eckart–Young	PCA in CS2, CS3; ridge
Change of variables	$f_Y(y) = f_X(g^{-1}(y))\left\lvert dg^{-1}/dy \right\rvert$	simulation, log-normal
LLN, CLT, Slutsky	$\bar{X} \xrightarrow{p} \mu$; $\sqrt{n}(\bar{X} - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$	every interval and test
Concentration	Hoeffding: $\mathbb{P}(\lvert\bar{X} - \mu\rvert \ge \varepsilon) \le 2e^{-2n\varepsilon^2}$ for $[0,1]$ data	sample-size planning
MLE asymptotics	$\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} \mathcal{N}(0, \mathcal{I}^{-1})$; delta method	CS2 standard errors
Convexity, Newton	convex $\Rightarrow$ local = global; Newton converges quadratically	CS2 IRLS
KKT, envelope	equal marginal benefits; $V'(B) = \lambda^*$	CS4 shadow price
Monte Carlo error	se $= \sigma/\sqrt{R}$; variance reduction	CS4 SAA, checks in §13.1
Bootstrap	resample to approximate the sampling distribution; blocks for dependence	CS1, CS3
Bias–variance, ridge/lasso	MSE = Bias^2 + Var; shrinkage $d_j^2/(d_j^2 + \lambda)$; L1 sparsity	CS2, CS3

13.6.3 The most common pitfalls

P1. Inverting matrices to solve systems. Use `np.linalg.solve`, QR or `lstsq`; never multiply by `inv(X.T @ X)` in production code.

P2. Forgetting dimensions and broadcasting. A $(n,)$ vector times a $(n,1)$ array broadcasts to (n,n) silently. Check `.shape` at every step.

P3. n versus $n-1$. NumPy's `var` uses `ddof=0` by default; pandas uses `ddof=1`.

P4. Confusing association and causation. Regression coefficients are causal only under an identification argument (CS1).

P5. Classical standard errors with heteroskedastic or clustered data.

P6. Using a normal-theory formula for non-normal data, as with $\mathrm{Var}(s^2)$ in Theorem 13.6.

P7. Look-ahead bias and data leakage. Standardisation, PCA, feature selection and tuning must be done within each training fold or window (CS3).

P8. Reporting in-sample fit as predictive accuracy.

P9. Ignoring Monte Carlo error. A simulation estimate without a standard error is incomplete.

P10. Penalising unstandardised predictors or the intercept in ridge and lasso.

P11. Using a bootstrap scheme that ignores dependence (time series, clusters).

P12. Trusting an optimiser's output without checking KKT conditions, gradients and multiple starting points (CS4).

P13. Reporting a single accuracy number for a targeting rule without the error rates at the operating point and by group (CS2).

⋆ Key Concept: Three questions to ask of every result

(1) *Is it computed correctly?* — check dimensions, conditioning, convergence and simulation error. (2) *Is it estimated well?* — bias, variance, robustness of the standard errors, out-of-sample validity. (3) *Does it answer the policy question?* — identification, population, data quality and how the uncertainty will be communicated.

13.6.4 Ten exam-style problems with worked solutions

Each problem below is of the length and level of an examination question. Attempt it before reading the solution. Listing 13.10 at the end of the subsection checks every numerical answer.

Problem R1 (floating point, Chapter 1). Evaluate $f(x) = 1 - \cos x$ at $x = 10^{-8}$ in double precision. Explain the result, compute the condition number of f at x, and give a stable formula.

Solution. Here $\cos x = 1 - x^2/2 + O(x^4)$ with $x^2/2 = 5 \times 10^{-17}$. Doubles just below 1 are spaced $\varepsilon_{\text{mach}}/2 \approx 1.1 \times 10^{-16}$ apart, so $1 - 5 \times 10^{-17}$ is closer to 1 than to the next double below it: $\mathrm{fl}(\cos x) = 1$ and the computed f is 0, a 100% relative error. The problem itself is well conditioned: $\kappa(x) = \left|x f'(x)/f(x)\right| = x \sin x/(1 - \cos x) \to 2$ as $x \to 0$. The algorithm is unstable because of *cancellation* in subtracting two nearly equal numbers. Using $1 - \cos x = 2\sin^2(x/2)$ involves no subtraction and returns 5×10^{-17} to full precision.

Problem R2 (projections and conditioning, Chapters 2 and 3). Let $\mathbf{X} \in \mathbb{R}^{n \times k}$ have full column rank and $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top$. (a) Show that every eigenvalue of $\mathbf{H}$ is 0 or 1 and that $\operatorname{rank} \mathbf{H} = \operatorname{tr} \mathbf{H} = k$. (b) Show that $\operatorname{cond}_2(\mathbf{X}^\top \mathbf{X}) = \operatorname{cond}_2(\mathbf{X})^2$.

Solution. (a) $\mathbf{H}$ is symmetric and idempotent (Proposition 13.3). If $\mathbf{H}v = \lambda v$ with $v \neq 0$, then $\lambda v = \mathbf{H}v = \mathbf{H}^2 v = \lambda^2 v$, so $\lambda \in \{0, 1\}$. Being symmetric, $\mathbf{H}$ is orthogonally diagonalisable, so

its rank equals the number of unit eigenvalues, which equals the trace; and $\operatorname{tr}\mathbf{H} = k$ by the cyclic property. (b) With the thin SVD $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}^\top$, whose singular values are d_j^2. Hence $\operatorname{cond}(\mathbf{X}^\top\mathbf{X}) = d_1^2/d_k^2 = \operatorname{cond}(\mathbf{X})^2$: solving the normal equations squares the sensitivity to rounding errors.

Problem R3 (eigenvalues and PCA, Chapter 3). Ten standardised asset indicators have equicorrelation matrix $\mathbf{C} = (1-\rho)\mathbf{I} + \rho\mathbf{1}\mathbf{1}^\top$ with $\rho = 0.3$. Find the eigenvalues of $\mathbf{C}$, the first principal component, and the share of variance it explains. For which ρ is $\mathbf{C}$ a valid correlation matrix?

Solution. $\mathbf{C}\mathbf{1} = (1-\rho)\mathbf{1} + \rho p\mathbf{1} = \{1 + (p-1)\rho\}\mathbf{1}$, so $\mathbf{1}/\sqrt{p}$ is an eigenvector with eigenvalue $1 + (p-1)\rho = 3.7$. Any $v \perp \mathbf{1}$ gives $\mathbf{C}v = (1-\rho)v$, so the remaining $p - 1 = 9$ eigenvalues equal 0.7. The first PC gives equal weight $1/\sqrt{10}$ to every asset and explains $3.7/\operatorname{tr}\mathbf{C} = 3.7/10 = 37\%$ of the variance. $\mathbf{C}$ is positive semidefinite iff all eigenvalues are non-negative: $-1/(p-1) \le \rho \le 1$. (Equal PCA weights are what a simple asset count would give.)

Problem R4 (transformations and likelihood, Chapters 4 and 5). Let $U \sim \mathrm{U}(0,1)$ and $X = -\lambda^{-1}\log U$. Find the distribution of X. Given an iid sample $x_1, \dots, x_n$ from this distribution, find the MLE of λ and its asymptotic standard error.

Solution. For $x \ge 0$, $\mathbb{P}(X \le x) = \mathbb{P}(\log U \ge -\lambda x) = \mathbb{P}(U \ge e^{-\lambda x}) = 1 - e^{-\lambda x}$, the exponential distribution with rate λ: this is the inverse-transform method. The log-likelihood is $\ell(\lambda) = n\log\lambda - \lambda\sum_i x_i$, with $\ell'(\lambda) = n/\lambda - \sum x_i = 0$ at $\hat\lambda = 1/\bar x$ and $\ell'' = -n/\lambda^2 < 0$, so this is the maximum. The Fisher information is $\mathcal{I}_n(\lambda) = n/\lambda^2$, giving $\operatorname{se}(\hat\lambda) \approx \hat\lambda/\sqrt{n}$.

Problem R5 (concentration, Chapter 6). A survey will estimate a poverty rate p by the sample proportion $\hat p$. How large must n be to guarantee $\mathbb{P}(|\hat p - p| \ge 0.05) \le 0.05$ for every p, using (a) Chebyshev's inequality, (b) Hoeffding's inequality? Compare with the CLT approximation.

Solution. (a) $\operatorname{Var}\hat p = p(1-p)/n \le 1/(4n)$, so Chebyshev gives $\mathbb{P}(|\hat p - p| \ge \varepsilon) \le 1/(4n\varepsilon^2)$; requiring this $\le \delta$ gives $n \ge 1/(4\varepsilon^2\delta) = 2000$. (b) Hoeffding gives $2e^{-2n\varepsilon^2} \le \delta$, i.e. $n \ge \log(2/\delta)/(2\varepsilon^2) = \log 40/0.005 = 737.8$, so $n = 738$. The CLT gives $n \approx 1.96^2/(4\varepsilon^2) = 385$, which is not a guarantee but an approximation. The exponential tail of Hoeffding is much sharper than Chebyshev's polynomial tail.

Problem R6 (estimation, Chapter 7). Let $X_1, \dots, X_n \overset{\text{iid}}{\sim} \mathrm{U}(0,\theta)$. Compare the mean squared errors of the method-of-moments estimator $\tilde\theta = 2\bar X$ and the MLE $\hat\theta = X_{(n)} = \max_i X_i$.

Solution. $\mathbb{E}X = \theta/2$ gives $\tilde\theta = 2\bar X$, unbiased with $\mathrm{MSE} = \operatorname{Var} = 4\cdot\theta^2/(12n) = \theta^2/(3n)$. The likelihood $\theta^{-n}\mathbf{1}\{\theta \ge X_{(n)}\}$ is decreasing in θ on its support, so $\hat\theta = X_{(n)}$. Since $\mathbb{P}(X_{(n)} \le t) = (t/\theta)^n$, $\mathbb{E}X_{(n)} = n\theta/(n+1)$ and $\mathbb{E}X_{(n)}^2 = n\theta^2/(n+2)$, hence

$$\mathrm{MSE}(\hat\theta) = \frac{n\theta^2}{n+2} - \frac{2n\theta^2}{n+1} + \theta^2 = \frac{2\theta^2}{(n+1)(n+2)}.$$

The MLE is biased but its MSE is $O(n^{-2})$, against $O(n^{-1})$ for $\tilde\theta$. This does not contradict the Cramér–Rao bound, whose regularity conditions fail because the support depends on θ. For $n = 10$: $0.0333\theta^2$ versus $0.0152\theta^2$.

Problem R7 (Newton's method, Chapter 8). Minimise $f(x) = e^x - 2x$. Show that f is strictly convex, find the minimiser, perform Newton iterations from $x_0 = 0$ and verify quadratic convergence.

Solution. $f''(x) = e^x > 0$, so f is strictly convex and the stationary point $f'(x) = e^x - 2 = 0$, $x^* = \log 2$, is the unique global minimiser. Newton's update is $x_{k+1} = x_k - f'(x_k)/f''(x_k) = x_k - 1 + 2e^{-x_k}$. From $x_0 = 0$: $x_1 = 1$, $x_2 = 2/e = 0.735759$, $x_3 = 0.694042$, $x_4 = 0.6931476$, with errors $e_k = x_k - \log 2$ of 0.307, 0.0426, 8.95×10^{-4}, 4.0×10^{-7}. Theory predicts $e_{k+1} \approx \frac{f'''(x^*)}{2f''(x^*)}e_k^2 = \frac12 e_k^2$: indeed $\frac12(0.0426)^2 = 9.1\times10^{-4}$ and $\frac12(8.95\times10^{-4})^2 = 4.0\times10^{-7}$.

Problem R8 (Lagrange multipliers, Chapter 9). A planner maximises $\sum_{j=1}^{3} a_j \log x_j$ subject to $\sum_j x_j = B$, with $\mathbf{a} = (1, 2, 5)$ and $B = 80$. Find the optimal allocation and the multiplier, and show that the multiplier is the marginal value of the budget.

Solution. The Lagrangian $\mathcal{L} = \sum_j a_j \log x_j - \lambda(\sum_j x_j - B)$ gives $a_j/x_j = \lambda$, so $x_j = a_j/\lambda$; the constraint gives $\lambda = \sum_j a_j / B = 8/80 = 0.1$ and $x_j = a_j B / \sum_k a_k$, i.e. $\mathbf{x}^* = (10, 20, 50)$. The objective is strictly concave and the constraint linear, so this is the unique global maximum (and $x_j > 0$ automatically, because $\log x_j \to -\infty$ at zero). The value function is $V(B) = \sum_j a_j \log(a_j B / \sum_k a_k)$, so $V'(B) = \sum_j a_j / B = \lambda^*$, as the envelope theorem asserts.

Problem R9 (Monte Carlo and the bootstrap, Chapters 10 and 11). (a) Estimate $\theta = \int_0^1 e^x \, dx$ by $\hat\theta = R^{-1} \sum_r e^{U_r}$. Find $\operatorname{Var}(e^U)$, and the variance of the antithetic estimator based on $\frac{1}{2}(e^U + e^{1-U})$. (b) Show that the probability that a given observation appears in a bootstrap sample of size n tends to $1 - e^{-1}$.

Solution. (a) $\mathbb{E}e^{2U} = (e^2 - 1)/2$, so $\operatorname{Var}(e^U) = (e^2 - 1)/2 - (e - 1)^2 = 0.2420$. For the antithetic pair, $\operatorname{Cov}(e^U, e^{1-U}) = \mathbb{E}e^1 - (e-1)^2 = e - (e-1)^2 = -0.2342$, so $\operatorname{Var}\{\frac{1}{2}(e^U + e^{1-U})\} = \frac{1}{2}(0.2420 - 0.2342) = 0.0039$. Each pair costs two function evaluations, so compare with $0.2420/2 = 0.121$: a variance reduction by a factor of about 31, because e^u is monotone and the pair is strongly negatively correlated. (b) Each of the n draws misses observation i with probability $1 - 1/n$, independently, so $\mathbb{P}(i \notin \text{sample}) = (1 - 1/n)^n \to e^{-1}$; the inclusion probability tends to $1 - e^{-1} \approx 0.632$, the origin of the ".632" bootstrap.

Problem R10 (ridge and bias–variance, Chapter 12). (a) Using the SVD $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, show that the ridge hat matrix has trace $\sum_j d_j^2/(d_j^2 + \lambda)$, and evaluate it for $d = (3, 1)$, $\lambda = 1$. (b) In the one-predictor model $y_i = \theta x_i + \varepsilon_i$ with $\sum_i x_i^2 = D$ and $\operatorname{Var} \varepsilon_i = \sigma^2$, find the λ minimising the MSE of $\hat\theta_\lambda = \sum_i x_i y_i / (D + \lambda)$.

Solution. (a) $\mathbf{H}_\lambda = \mathbf{X}(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top = \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^\top$, so $\operatorname{tr}\mathbf{H}_\lambda = \sum_j d_j^2/(d_j^2 + \lambda) = 9/10 + 1/2 = 1.4$ effective degrees of freedom, less than the 2 of OLS. (b) $\mathbb{E}\hat\theta_\lambda = D\theta/(D + \lambda)$ and $\operatorname{Var}\hat\theta_\lambda = \sigma^2 D/(D + \lambda)^2$, so $\operatorname{MSE}(\lambda) = (\lambda^2\theta^2 + \sigma^2 D)/(D + \lambda)^2$. Differentiating, $\operatorname{MSE}'(\lambda) = 2D(\lambda\theta^2 - \sigma^2)/(D + \lambda)^3$, which is negative for $\lambda < \sigma^2/\theta^2$ and positive afterwards: $\lambda^* = \sigma^2/\theta^2$. Some shrinkage always helps ($\operatorname{MSE}'(0) < 0$), and more is optimal when the signal-to-noise ratio θ^2/σ^2 is low — the essence of the bias–variance trade-off.

Listing 13.10: Numerical checks of the ten review problems ch13/ch13_review_checks.py

```
# DEP 803 -- Chapter 13: Problem Set Clinic and Applied Foundations Review
# Numerical checks of the ten worked review problems (Chapters 1-12).
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats

rng = np.random.default_rng(803)
rows = []

# R1 cancellation: 1 - cos(x) versus 2 sin^2(x/2) at x = 1e-8
x = 1e-8
rows.append(("R1 1-cos(1e-8)", 0.5 * x**2, 1 - np.cos(x), 2 * np.sin(x / 2)**2))

# R2 hat matrix: trace = k, cond(X'X) = cond(X)^2
X = np.column_stack([np.ones(50), rng.standard_normal((50, 3))])
H = X @ np.linalg.solve(X.T @ X, X.T)
rows.append(("R2 trace(H)", 4, np.trace(H), np.allclose(H @ H, H)))
rows.append(("R2 cond(X'X)/cond^2", 1.0,
             np.linalg.cond(X.T @ X) / np.linalg.cond(X)**2, np.nan))
```

```
20
21 # R3 equicorrelation p = 10, rho = 0.3: largest eigenvalue 1 + 9 rho
22 p, rho = 10, 0.3
23 C = (1 - rho) * np.eye(p) + rho * np.ones((p, p))
24 lam = np.linalg.eigvalsh(C)
25 rows.append(("R3 PC1 share", (1 + (p - 1) * rho) / p, lam[-1] / p, lam[0]))
26
27 # R4 inverse transform -log(U)/2 ~ Exp(rate 2); MLE of the rate = 1/mean
28 u = rng.random(100_000)
29 e = -np.log(u) / 2
30 ks = stats.kstest(e, "expon", args=(0, 0.5)).pvalue
31 rows.append(("R4 rate MLE", 2.0, 1 / e.mean(), ks))
32
33 # R5 Hoeffding / Chebyshev sample sizes for eps = 0.05, delta = 0.05
34 rows.append(("R5 n Hoeffding", 738, np.ceil(np.log(2 / 0.05) / (2 * 0.05**2)),
35              np.nan))
36 rows.append(("R5 n Chebyshev", 2000, np.ceil(0.25 / (0.05**2 * 0.05)), np.nan))
37
38 # R6 Uniform(0, theta): MSE of 2*mean and of the max, n = 10, theta = 1
39 n, R = 10, 200_000
40 U = rng.random((R, n))
41 rows.append(("R6 MSE(2 xbar)", 1 / (3 * n), np.mean((2 * U.mean(1) - 1)**2),
42              np.nan))
43 rows.append(("R6 MSE(max)", 2 / ((n + 1) * (n + 2)), np.mean((U.max(1) - 1)**2),
44              np.nan))
45
46 # R7 Newton on f(x) = exp(x) - 2x from x0 = 0
47 xk = 0.0
48 for _ in range(5):
49     xk -= (np.exp(xk) - 2) / np.exp(xk)
50 rows.append(("R7 Newton argmin", np.log(2), xk, abs(xk - np.log(2))))
51
52 # R8 max sum a_j log x_j s.t. sum x_j = B: x_j = a_j B / sum(a)
53 a, B = np.array([1.0, 2.0, 5.0]), 80.0
54 xs = a * B / a.sum()
55 rows.append(("R8 x_3 and lambda", 50.0, xs[2], a.sum() / B))
56
57 # R9 plain vs antithetic MC for int_0^1 e^x dx; bootstrap inclusion prob.
58 m = 100_000
59 u = rng.random(m)
60 plain = np.exp(u)
61 anti = 0.5 * (np.exp(u[: m // 2]) + np.exp(1 - u[: m // 2]))
62 rows.append(("R9 Var(e^U)", (np.e**2 - 1) / 2 - (np.e - 1)**2, plain.var(),
63              anti.var()))
64 nn = 500
65 inc = np.mean([np.unique(rng.integers(0, nn, nn)).size / nn for _ in range(2000)])
66 rows.append(("R9 P(in boot sample)", 1 - (1 - 1 / nn)**nn, inc, 1 - np.exp(-1)))
67
68 # R10 ridge shrinkage factors d^2/(d^2+lambda), singular values 3 and 1, lam = 1
69 d = np.array([3.0, 1.0])
70 Uo, _, Vo = np.linalg.svd(rng.standard_normal((20, 2)), full_matrices=False)
71 Xr = Uo * d @ Vo
72 Hr = Xr @ np.linalg.solve(Xr.T @ Xr + np.eye(2), Xr.T)
73 rows.append(("R10 ridge df", 0.9 + 0.5, np.trace(Hr), np.nan))
74
75 print(f"{'check':<22}{'analytical':>14}{'numerical':>14}{'extra':>14}")
76 for name, an, nu, ex in rows:
```

```
    if isinstance(ex, (bool, np.bool_)):
        exs = f"{str(ex):>14}"
    else:
        exs = f"{'--':>14}" if np.isnan(ex) else f"{ex:14.4g}"
    print(f"{name:<22}{an:14.6g}{nu:14.6g}{exs}")
```

Output

```
check                   analytical     numerical         extra
R1 1-cos(1e-8)               5e-17             0         5e-17
R2 trace(H)                      4             4          True
R2 cond(X'X)/cond^2              1             1            --
R3 PC1 share                  0.37          0.37           0.7
R4 rate MLE                      2       1.99702        0.1784
R5 n Hoeffding                 738           738            --
R5 n Chebyshev                2000          2000            --
R6 MSE(2 xbar)           0.0333333     0.0333526            --
R6 MSE(max)              0.0151515     0.0152032            --
R7 Newton argmin          0.693147      0.693147     8.016e-14
R8 x_3 and lambda               50            50           0.1
R9 Var(e^U)               0.242036      0.242242      0.003928
R9 P(in boot sample)      0.632489      0.632225        0.6321
R10 ridge df                   1.4           1.4            --
```

Think About It

Which of the ten problems could you have solved *only* numerically? Which numerical checks would have failed to detect an error in the corresponding derivation? (Hint: consider R5, where the "numerical" column merely re-evaluates the formula.)

Graduate Perspective: From problem sets to research

Research problems rarely come with a theorem to prove. The workflow of Definition 13.1 still applies, but steps S4 and S6 grow in importance: build the smallest simulated world in which your method should work, show that it does, then find the simplest world in which it fails. A thesis chapter that contains both simulations is far more convincing than one that only applies a method to real data.

Comprehensive Problem Set

13.6.1 Theory (Ch. 1–2) Let $\mathbf{X} = \mathbf{QR}$ be a thin QR factorisation of a full-column-rank matrix. Show that the diagonal of $\mathbf{R}$ has no zero entries, that the least-squares residual is $(\mathbf{I} - \mathbf{Q}\mathbf{Q}^\top)\mathbf{y}$, and that $\|\hat{\mathbf{e}}\|^2 = \|\mathbf{y}\|^2 - \|\mathbf{Q}^\top\mathbf{y}\|^2$.

13.6.2 Theory (Ch. 3) Prove that the first principal direction of a centred data matrix maximises the Rayleigh quotient $\mathbf{v}^\top\mathbf{S}\mathbf{v}/\mathbf{v}^\top\mathbf{v}$, where $\mathbf{S}$ is the sample covariance, and that the first k components explain the fraction $\sum_{j\le k}\lambda_j/\operatorname{tr}\mathbf{S}$ of total variance.

13.6.3 Theory (Ch. 4–5) If $X \sim \mathcal{N}(\mu, \sigma^2)$ and $Y = e^X$, derive the density, mean, median and variance of Y. Explain why mean income exceeds median income when log income is normal, and by how much when $\sigma = 0.8$.

13.6.4 Theory (Ch. 5, 7–8) For Poisson regression, $y_i \sim \text{Poisson}(e^{\mathbf{x}_i^\top\boldsymbol{\beta}})$, derive the log-likelihood, score and Hessian; show the log-likelihood is concave; and write Newton's method as

IRLS, identifying the weights and working response.

13.6.5 Theory (Ch. 6) Prove the weak law of large numbers for iid variables with finite variance using Chebyshev's inequality. Then show by simulation and by argument that the sample mean of Cauchy variables does not converge.

13.6.6 Theory (Ch. 7) Using the output of Listing 13.3, apply the delta method to compute a standard error and a 95% interval for the percentage effect $100(e^{\rho}-1)$. Compare with the bootstrap interval of Listing 13.4.

13.6.7 Theory (Ch. 8) For $f(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^{\top}\mathbf{A}\mathbf{x} - \mathbf{b}^{\top}\mathbf{x}$ with $\mu\mathbf{I} \preceq \mathbf{A} \preceq L\mathbf{I}$, $\mu > 0$, show that gradient descent with step $1/L$ satisfies $\|\mathbf{x}_k - \mathbf{x}^{\ast}\| \le (1-\mu/L)^{k}\,\|\mathbf{x}_0 - \mathbf{x}^{\ast}\|$. Relate the rate to $\operatorname{cond}(\mathbf{A})$ and to the conditioning of least-squares problems.

13.6.8 Theory (Ch. 9) Solve $\max \sum_j a_j c_j \log(1 + x_j/c_j)$ subject to $\sum_j x_j = B$, $x_j \ge 0$, for $\mathbf{a} = (1.6, 2.2, 3.0, 2.6)$, $\mathbf{c} = (60, 30, 15, 25)$ and $B = 20$. Which programmes are funded? Find $\lambda^{\ast}$ and interpret it.

13.6.9 Python (Ch. 10) Estimate $\mathbb{P}(Z > 4)$ for $Z \sim \mathcal{N}(0,1)$ by plain Monte Carlo and by importance sampling from $\mathcal{N}(4,1)$, with $R = 10^5$ each. Report estimates, standard errors and relative errors against the exact value.

13.6.10 Python (Ch. 11) Simulate a randomised evaluation of a cash transfer with 300 treated and 300 control households and log-normal consumption. Test for a treatment effect with a permutation test and a Welch t-test, and give a bootstrap 95% interval for the effect on mean and on median consumption.

13.6.11 Python (Ch. 12) Implement 10-fold cross-validation for ridge regression from scratch on `load_diabetes`, using the SVD to compute the whole regularisation path cheaply. Plot the CV curve, compare the chosen λ with `RidgeCV`, and plot the effective degrees of freedom against λ.

13.6.12 ★ Graduate (Ch. 12) Show that as $\lambda \downarrow 0$ the ridge estimator converges to the minimum-norm least-squares solution $\mathbf{X}^{+}\mathbf{y}$, even when $\mathbf{X}$ is rank deficient. What does this imply for "interpolating" regressions with $k > n$?

13.6.13 ★ Graduate (integrated) Extend Case Study 1 to a two-stage least squares estimator with two instruments, computed by QR. Derive and implement HC1 standard errors for 2SLS, and verify their coverage in a Monte Carlo with 1000 replications.

13.6.14 Policy (integrated) A social-protection agency plans to roll out the PMT of Case Study 2 nationally with a budget for 20% of households, and asks you to design an evaluation of its targeting performance and of the transfer's effect on consumption. Using tools from at least five chapters, write a two-page plan covering sampling, estimation, uncertainty quantification, fairness audits and how results will be communicated to the agency's board.

Chapter Summary

- A disciplined workflow — understand, notation, dimensions, small cases, proof, simulation, interpretation — prevents most errors. Simulation checks must report their own Monte Carlo error.
- Reproducibility requires fixed seeds, recorded environments, version control and a single command that regenerates every result.
- **Case Study 1:** OLS via QR, FWL for fixed effects and HC1 standard errors give a precise estimate of the Mincer coefficient, but interpreting it causally requires identification;

ability bias and measurement error can offset each other, and IV trades precision for credibility.

- **Case Study 2:** a logit PMT fitted by Newton/IRLS outperforms a PCA asset index modestly; with a realistic budget, exclusion and inclusion errors remain large and differ across groups, which must be audited and communicated.
- **Case Study 3:** with a factor-structured panel, PCA diffusion indexes, ridge and lasso all beat an AR benchmark; honest evaluation needs an expanding window without look-ahead, Diebold–Mariano comparisons and block-bootstrap uncertainty.
- **Case Study 4:** budget allocation under uncertainty is a concave programme solved by SAA and SLSQP; the KKT multiplier is the shadow price of the budget, and risk aversion shifts funds towards better-evidenced programmes.
- The course forms a pipeline from linear algebra to regularisation; the key results, pitfalls and worked problems of Section 13.6 summarise what every graduate student should be able to do unaided.

Further Reading

On problem solving and proof, Pólya (1945) remains a classic and Velleman (2019) teaches proof writing systematically; Wilson et al. (2014) and Christensen and Miguel (2018) cover reproducible computational practice. For the econometrics of Case Study 1 see Hansen (2022), Wooldridge (2010) and Angrist and Pischke (2009), and on returns to schooling Card (2001) and Heckman et al. (2006). Logistic regression, regularisation and evaluation metrics are treated in Murphy (2022) (Chapters 4, 5, 10 and 11) and, with practical scikit-learn code, in Géron (2022); Hastie et al. (2009) and James et al. (2021) cover cross-validation and the lasso, and Brown et al. (2018) is essential on PMT performance. For many-predictor forecasting read Stock and Watson (2002a) and Goulet Coulombe et al. (2022); Diebold and Mariano (1995) and Künsch (1989) give the evaluation and resampling tools. Bayesian approaches to the uncertain programme returns of Case Study 4 — posterior distributions in place of our assumed log-normals, and hierarchical shrinkage against the optimiser's curse — are developed with Stan and Python in Matsuura (2022) and Gelman et al. (2013); Boyd and Vandenberghe (2004) and Shapiro et al. (2009) cover the optimisation, and Manski (2013, 2019) the communication of uncertainty to policy makers.

Chapter A

A Python Toolkit for the Course

This appendix collects, in one place, the Python tools used throughout the book. It is a reference, not a tutorial: for a gentle introduction to Python for data analysis see Géron (2022), and for the scientific libraries see Harris et al. (2020), Virtanen et al. (2020) and Pedregosa et al. (2011).

A.1 Setting up

All programs were run with Python 3.9, NumPy 1.23, SciPy 1.9, pandas 1.5, Matplotlib 3.6 and scikit-learn 1.1; any later version should work. The simplest installation is the Anaconda distribution. Alternatively, with an existing Python installation,

```
git clone https://github.com/moorekwesi/math-stat-foundations-data-science.git
cd math-stat-foundations-data-science
pip install -r requirements.txt
python code/ch02/fwl_panel.py        # run one program
python run_all.py ch05               # run every program of Chapter 5
```

The programs can also be run without installing anything in Google Colab: upload a `.py` file or paste it into a notebook cell.

⋆ Key Concept: reproducibility

Every random number in this book comes from a seeded generator, `rng = np.random.default_rng(803)`. Re-running a program reproduces its output exactly. In your own research, fix the seed, record the package versions (`pip freeze > requirements.txt`) and keep the code under version control (git). A result that cannot be reproduced cannot be checked, and a policy recommendation that cannot be checked should not be trusted.

A.2 NumPy essentials

Listing A.1: NumPy essentials appA/numpy_essentials.py

```
# DEP 803 -- Appendix A: Python Toolkit
# NumPy essentials: arrays, broadcasting, linear algebra and random numbers.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np

rng = np.random.default_rng(803)            # reproducible random numbers

# Arrays, shapes and broadcasting
X = rng.normal(size=(5, 3))                 # 5 observations, 3 variables
```

```
mu = X.mean(axis=0)                          # column means, shape (3,)
Xc = X - mu                                  # broadcasting subtracts mu from each row
print("shape of X:", X.shape, " column means after centring:",
      np.abs(np.round(Xc.mean(axis=0), 12)))

# Matrix products and solving linear systems
S = Xc.T @ Xc / (X.shape[0] - 1)             # sample covariance matrix
print("covariance matrix equals np.cov:", np.allclose(S, np.cov(X, rowvar=False)))
b = np.array([1.0, 2.0, 3.0])
x = np.linalg.solve(S, b)                    # never form inv(S) @ b
print("residual of S x = b:", np.linalg.norm(S @ x - b))

# Decompositions
lam, V = np.linalg.eigh(S)                   # symmetric eigen-decomposition
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
print("eigenvalues of S:      ", np.round(lam[::-1], 4))
print("s**2/(n-1) from SVD:   ", np.round(s**2 / (X.shape[0] - 1), 4))

# Least squares with an intercept
n = 200
x1 = rng.uniform(0, 10, n)
y = 2.0 + 0.5 * x1 + rng.normal(0, 1, n)
A = np.column_stack([np.ones(n), x1])
beta, *_ = np.linalg.lstsq(A, y, rcond=None)
print("OLS coefficients (true 2.0, 0.5):", np.round(beta, 3))

# Boolean indexing and vectorised summaries
income = rng.lognormal(mean=8.0, sigma=0.8, size=10_000)
z = 1500.0                                   # an illustrative poverty line
print(f"headcount ratio below z: {np.mean(income < z):.3f}")
print(f"mean poverty gap ratio:  {np.mean(np.maximum(z - income, 0) / z):.3f}")
```

Output

```
shape of X: (5, 3)  column means after centring: [0. 0. 0.]
covariance matrix equals np.cov: True
residual of S x = b: 4.440892098500626e-16
eigenvalues of S:       [0.5149 0.3445 0.1612]
s**2/(n-1) from SVD:    [0.5149 0.3445 0.1612]
OLS coefficients (true 2.0, 0.5): [1.912 0.517]
headcount ratio below z: 0.195
mean poverty gap ratio:  0.062
```

△ Caution: three habits that prevent most errors

(i) Check shapes: `X.shape`, and remember that a NumPy vector of shape `(n,)` is neither a row nor a column. (ii) Never compute `np.linalg.inv(A) @ b`; use `np.linalg.solve(A, b)` or `np.linalg.lstsq`. (iii) Compare floating-point numbers with `np.isclose` or `np.allclose`, never with ==.

A.3 SciPy and scikit-learn essentials

Listing A.2: SciPy and scikit-learn essentials appA/scipy_toolkit.py

```
# DEP 803 -- Appendix A: Python Toolkit
# SciPy and scikit-learn essentials: distributions, optimisation, integration.
# (c) 2026 Stephen E. Moore. MIT Licence.
import numpy as np
from scipy import stats, optimize, integrate
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(803)

# Distributions: pdf, cdf, quantiles and random variates
Z = stats.norm(loc=0, scale=1)
print(f"P(Z <= 1.96) = {Z.cdf(1.96):.4f},  97.5% quantile = {Z.ppf(0.975):.4f}")
t5 = stats.t(df=5)
print(f"97.5% quantile of t(5) = {t5.ppf(0.975):.4f}")
sample = stats.gamma(a=2.0, scale=1.5).rvs(size=2000, random_state=rng)
a_hat, loc_hat, scale_hat = stats.gamma.fit(sample, floc=0)
print(f"gamma MLE: shape = {a_hat:.3f}, scale = {scale_hat:.3f}  (true 2, 1.5)")

# Optimisation: minimise the Rosenbrock function with BFGS
res = optimize.minimize(optimize.rosen, x0=[-1.2, 1.0], method="BFGS",
                        jac=optimize.rosen_der)
print("BFGS minimiser:", np.round(res.x, 6), " iterations:", res.nit)

# Constrained optimisation: maximise sqrt(x1) + sqrt(x2) s.t. x1 + 2 x2 = 10
cons = {"type": "eq", "fun": lambda v: v[0] + 2 * v[1] - 10}
res = optimize.minimize(lambda v: -(np.sqrt(v[0]) + np.sqrt(v[1])),
                        x0=[1.0, 1.0], method="SLSQP", constraints=[cons],
                        bounds=[(1e-9, None), (1e-9, None)],
                        options={"ftol": 1e-12})
print("SLSQP solution:", np.round(res.x, 4), " (exact 6.6667, 1.6667)")

# Integration: E[exp(Z)] for Z ~ N(0,1) equals exp(1/2)
integrand = lambda z: np.exp(z - z**2 / 2) / np.sqrt(2 * np.pi)   # exp(z) phi(z)
val, err = integrate.quad(integrand, -np.inf, np.inf)
print(f"quad: {val:.10f}   exact: {np.exp(0.5):.10f}")

# scikit-learn: the fit/predict interface
X = rng.normal(size=(500, 3))
y = X @ np.array([1.0, -2.0, 0.5]) + rng.normal(0, 0.5, 500)
model = LinearRegression().fit(X, y)
print("sklearn coefficients:", np.round(model.coef_, 3),
      " R^2 =", round(model.score(X, y), 3))
```

Output

```
P(Z <= 1.96) = 0.9750,  97.5% quantile = 1.9600
97.5% quantile of t(5) = 2.5706
gamma MLE: shape = 1.970, scale = 1.478  (true 2, 1.5)
BFGS minimiser: [1. 1.]  iterations: 32
SLSQP solution: [6.6667 1.6667]  (exact 6.6667, 1.6667)
quad: 1.6487212707   exact: 1.6487212707
sklearn coefficients: [ 0.986 -1.991  0.486]  R^2 = 0.952
```

A.4 From mathematics to code

Mathematics	Python (NumPy / SciPy / scikit-learn)
$\mathbf{AB}$, $\mathbf{A}^\top$, $\mathbf{x}^\top\mathbf{y}$	`A @ B`, `A.T`, `x @ y`
$\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_1$, $\|\mathbf{A}\|_F$	`np.linalg.norm(x)`, `np.linalg.norm(x, 1)`, `np.linalg.norm(A)`
solve $\mathbf{Ax} = \mathbf{b}$	`np.linalg.solve(A, b)`; `scipy.linalg.cho_solve` for SPD $\mathbf{A}$
least squares $\min \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|$	`np.linalg.lstsq(X, y, rcond=None)`
$\operatorname{rank}(\mathbf{A})$, $\det(\mathbf{A})$, $\kappa(\mathbf{A})$	`np.linalg.matrix_rank`, `np.linalg.slogdet`, `np.linalg.cond`
$\mathbf{A} = \mathbf{QR}$, $\mathbf{A} = \mathbf{LL}^\top$	`np.linalg.qr(A)`, `np.linalg.cholesky(A)`
$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top$, $\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$	`np.linalg.eigh(A)`, `np.linalg.svd(X, full_matrices=False)`
$\mathbf{A}^+$	`np.linalg.pinv(A)`
$X_1, \ldots, X_n \overset{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2)$	`rng.normal(mu, sigma, size=n)`
$F(x)$, $f(x)$, $F^{-1}(u)$	`stats.norm.cdf(x)`, `stats.norm.pdf(x)`, `stats.norm.ppf(u)`
$\log \sum_i e^{a_i}$	`scipy.special.logsumexp(a)`
$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$	`scipy.optimize.minimize(f, x0, method="BFGS")`
$\min c^\top \mathbf{x}$ s.t. $\mathbf{Ax} \le \mathbf{b}$	`scipy.optimize.linprog(c, A_ub=A, b_ub=b)`
$\int_a^b f(x)\,dx$	`scipy.integrate.quad(f, a, b)`
bootstrap CI	`scipy.stats.bootstrap((x,), np.mean)`
permutation test	`scipy.stats.permutation_test`
ridge / lasso with CV	`sklearn.linear_model.RidgeCV`, `LassoCV`
K-fold cross-validation	`sklearn.model_selection.cross_val_score`

A.5 pandas and Matplotlib in one paragraph each

pandas stores a data set as a `DataFrame`: columns are variables, rows are observations. The most useful operations are `df.describe()`, selection (`df.loc[df["region"] == "Central", "income"]`), group summaries (`df.groupby("region")["income"].mean()`), and conversion to NumPy (`df.to_numpy()`). Model-fitting code in this book works with NumPy arrays; pandas is used to organise and summarise data around it.

Matplotlib draws figures with the pattern `fig, ax = plt.subplots()`, `ax.plot(x, y)`, `ax.set_xlabel(...)`, `fig.savefig("name.pdf", bbox_inches="tight")`. Always label axes with units, keep a figure to one message, and save vector graphics (PDF) for documents.

Chapter B

Answers and Hints to Selected Exercises

Answers, hints and solution sketches are given for selected exercises. Exercises tagged Python have reference solutions that can be checked against the programs in the `code/` folder of the companion repository; for those we record the key numbers you should obtain. Graduate exercises are given as solution outlines: filling in the details is part of the exercise.

Chapter 1: Mathematical Notation, Vectors, Matrices and Computational Precision

Exercise 1.1.1 Expand $(x_i - x_j)^2 = ((x_i - \bar{x}) - (x_j - \bar{x}))^2$. Summing over i, j the cross term $-2\sum_i (x_i - \bar{x})\sum_j (x_j - \bar{x})$ vanishes by Proposition 1.2, and each square term contributes $n\sum_i (x_i - \bar{x})^2$. Hence $s^2 = \frac{1}{2n(n-1)}\sum_i \sum_j (x_i - x_j)^2$, the form behind U-statistics.

Exercise 1.1.2 $n^k / c^n = \exp(k \log n - n \log c) \to 0$ since $\log n = o(n)$. For $\log n!$: the upper bound is $\sum_{i \le n} \log i \le n \log n$; the lower bound is $\sum_{i > n/2} \log i \ge \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} \log 2$, which is $\ge \frac{n}{4} \log n$ for $n \ge 4$.

Exercise 1.1.3 $\sqrt{1+h} = 1 + \frac{h}{2} - \frac{h^2}{8} + O(h^3)$, so $f(h) = \frac{1}{2} - \frac{h}{8} + O(h^2)$. Then $f(h) - \frac{1}{2} = o(1)$ and $= O(h)$, but it is not $o(h)$ because $(f(h) - \frac{1}{2})/h \to -\frac{1}{8} \neq 0$.

Exercise 1.1.4 One vectorised form is `np.clip(y[:, None] - tau[:-1], 0, np.diff(tau)) @ rates`, with the last threshold set to `np.inf`. Key values: $T(10) = 0.5$, $T(50) = 1.5 + 6 = 7.5$, $T(100) = 22.5$ (average rate 22.5%, marginal rate 30%). The average rate is a weighted average of the marginal rates below y, hence never exceeds the current marginal rate.

Exercise 1.1.6 $\hat{P}_0 = n^{-1}\mathbf{1}^\top \mathbf{d}$ with $d_i = \mathbb{1}\{y_i < z\}$; $\hat{P}_1 = n^{-1}\mathbf{1}^\top(\mathbf{d} \odot (\mathbf{1} - \mathbf{y}/z))$. A small transfer that keeps a household below z leaves $\mathbf{d}$ unchanged but reduces $1 - y_i/z$, so only $\hat{P}_1$ falls.

Exercise 1.1.7 Minimise $cn + \lambda\sigma^2/n$: the first-order condition gives $n^\ast = \sigma\sqrt{\lambda/c} = \Theta(\lambda^{1/2})$. Halving the standard error requires four times the sample, so doubling precision of a labour-force survey roughly quadruples fieldwork cost.

Exercise 1.2.1 Absolute homogeneity and positivity are immediate; the triangle inequality follows from $|x_i + y_i| \le |x_i| + |y_i|$ summed (for ℓ_1) or maximised (for ℓ_∞). For $\mathbf{x} = \mathbf{e}_1, \mathbf{y} = \mathbf{e}_2$ in ℓ_1: LHS $= 4 + 4 = 8$, RHS $= 2 + 2 = 4$; in ℓ_∞: LHS $= 1 + 1 = 2$, RHS $= 4$. The law fails, so neither norm comes from an inner product.

Exercise 1.2.2 Symmetry and bilinearity follow from $\mathbf{W} = \mathbf{W}^\top$; positivity is the definition of positive definiteness. Cauchy–Schwarz becomes $(\sum_i w_i x_i y_i)^2 \le (\sum_i w_i x_i^2)(\sum_i w_i y_i^2)$: a weighted correlation lies in $[-1, 1]$.

Exercise 1.2.3 $\left|\sum_i x_i y_i\right| \le \sum_i |x_i| \left|y_i\right| \le \max_i \left|y_i\right| \sum_i |x_i|$. The bound is attained by $\mathbf{x} = \operatorname{sign}(y_k)\mathbf{e}_k$ with $k = \arg\max_i \left|y_i\right|$.

Exercise 1.2.4 The ratio of largest to smallest distance falls quickly towards 1 (with seed 803: about 3000 at $p = 2$, 9.6 at $p = 10$, 1.8 at $p = 100$ and 1.19 at $p = 1000$): distances concentrate around $\sqrt{2p/3}$ with spread $O(1)$, so "nearest" neighbours are hardly nearer than any other point unless the data have low intrinsic dimension.

Exercise 1.2.6 With the stated shares the poor household's weights are

$$(0.45,\ 0.013,\ 0.076,\ 0.135,\ 0.058,\ 0.054,\ 0.058,\ 0.032,\ 0.025,\ 0.10)$$

and its Laspeyres index is 121.48, against the headline 120.17: about 1.3 points more inflation, driven by the food weight. Indexing transfers to the headline CPI would erode their real value for this group.

Exercise 1.2.7 Write $\mathbf{s}$ for quantity relatives $s_k = q_{1k}/q_{0k}$. Then

$$\frac{P}{L} = \frac{\sum_k w_k r_k s_k}{\sum_k w_k r_k \ \sum_k w_k s_k} = 1 + \frac{\operatorname{Cov}_{\mathbf{w}}(\mathbf{r}, \mathbf{s})}{\bar{r}_{\mathbf{w}}\, \bar{s}_{\mathbf{w}}}.$$

Hence $L > P$ iff the weighted covariance between price and quantity relatives is negative, the normal case with substitution.

Exercise 1.3.1 $(\mathbf{UV})_{ij} = \sum_k u_{ik} v_{kj}$ with $u_{ik} = 0$ for $k < i$ and $v_{kj} = 0$ for $k > j$; if $i > j$ every term vanishes. For the inverse, solve $\mathbf{Ux} = \mathbf{e}_j$ by back substitution: $x_i = 0$ for $i > j$. The determinant follows by cofactor expansion along the first column.

Exercise 1.3.2 Multiply $(\mathbf{A} + \mathbf{uv}^\top)$ by the right-hand side and simplify, using that $\mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u}$ is a scalar. With $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$, $\mathbf{u} = \mathbf{v} = \mathbf{x}_0$, the new estimator is $\hat{\boldsymbol{\beta}}_{\text{new}} = \hat{\boldsymbol{\beta}} + \frac{\mathbf{A}^{-1}\mathbf{x}_0}{1 + \mathbf{x}_0^\top \mathbf{A}^{-1} \mathbf{x}_0} (y_0 - \mathbf{x}_0^\top \hat{\boldsymbol{\beta}})$, a correction proportional to the prediction error (the recursive least-squares update).

Exercise 1.3.3 $\mathbf{P} = \mathbf{UU}^\top$ with $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_r$, so $\operatorname{tr} \mathbf{P} = \operatorname{tr}(\mathbf{U}^\top \mathbf{U}) = r = \operatorname{rank} \mathbf{P}$. Equivalently the eigenvalues are r ones and $n - r$ zeros.

Exercise 1.3.6 Column sums (backward linkages) of $\mathbf{L}$: $(1.931, 2.251, 1.912)$; row sums (forward linkages): $(1.580, 2.345, 2.169)$. Industry maximises gross output per unit of demand. Employment multipliers $\boldsymbol{\ell}^\top \mathbf{L} = (18.5, 12.1, 13.0)$ jobs per GHS million, so agriculture is best for jobs: labour intensity, not just linkages, drives employment rankings.

Exercise 1.3.7 For any ϵ with $\rho(\mathbf{A}) + \epsilon < 1$ choose an induced norm with $\|\mathbf{A}\| \le \rho + \epsilon$; the proof of Theorem 1.15 then goes through verbatim. For $\mathbf{A} \ge 0$, $\rho(\mathbf{A}) < 1$ is equivalent to the Hawkins–Simon condition that all leading principal minors of $\mathbf{I} - \mathbf{A}$ are positive.

Exercise 1.4.1 $\sum_i \|\mathbf{x}_i - \mathbf{c}\|^2 = \sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 + n \|\bar{\mathbf{x}} - \mathbf{c}\|^2$ (the cross term vanishes), minimised at $\mathbf{c} = \bar{\mathbf{x}}$, with minimum $\operatorname{tr}(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}) = (n-1) \operatorname{tr} \mathbf{S}$.

Exercise 1.4.2 $\mathbf{HY} = \mathbf{HXB}$ because $\mathbf{H1} = \mathbf{0}$, so $\mathbf{S}_Y = \mathbf{B}^\top\mathbf{X}^\top\mathbf{HXB}/(n-1) = \mathbf{B}^\top\mathbf{S}_X\mathbf{B}$. With $\mathbf{B} = \operatorname{diag}(b_j)$, $b_j > 0$, the scale factors cancel in r_{jk}; a general $\mathbf{B}$ mixes variables and changes correlations.

Exercise 1.4.4 The diabetes features are centred and scaled so that each column has unit sum of squares; hence every diagonal entry of $\mathbf{S}$ equals $1/(n-1) = 1/441 \approx 0.00227$ and $\mathbf{S} = \mathbf{R}/441$.

Exercise 1.4.5 The one-pass covariance $(\sum_i \mathbf{x}_i\mathbf{x}_i^\top - n\bar{\mathbf{x}}\bar{\mathbf{x}}^\top)/(n-1)$ agrees with `np.cov` to about 10^{-14} without offset; with an offset of 10^8 it has essentially no correct digits (errors of order $\varepsilon_{\text{mach}} \times 10^{16}$), exactly as (1.16) predicts.

Exercise 1.4.7 For $\gamma = 0.25$ the eigenvalues fill $[(1-0.5)^2, (1+0.5)^2] = [0.25, 2.25]$ rather than clustering at 1. Sample covariances of large panels are therefore badly conditioned; shrinkage or factor structure is required for forecasting with many predictors.

Exercise 1.5.1 Example: $a = 1$, $b = c = 2^{-53}$. Then $\operatorname{fl}(\operatorname{fl}(a+b)+c) = 1$ (each addition is a tie rounded to even), whereas $\operatorname{fl}(a + \operatorname{fl}(b+c)) = 1 + 2^{-52}$. The product bound follows by induction: $\prod_{k=1}^{n}(1+\delta_k) = 1 + \theta_n$ with $|\theta_n| \le (1+u)^n - 1 \le \gamma_n$.

Exercise 1.5.2 $\kappa_{\log}(x) = 1/\left|\log x\right|$ (ill-conditioned near $x = 1$); $\kappa_{\exp}(x) = |x|$ (ill-conditioned for large $|x|$); $\kappa(x) = |x/(x-1)|$ for $x - 1$ (ill-conditioned near 1). For $g(x) = \log(1+x)$, $\kappa_g(x) = \left|x/((1+x)\log(1+x))\right| \to 1$: well conditioned. The naive algorithm first rounds $1 + x$, an error of up to u in absolute terms, i.e. a relative error of u/x in the answer; `np.log1p` avoids forming $1 + x$.

Exercise 1.5.3 With $d = x_k - \bar{x}_{k-1}$: $\bar{x}_k = \bar{x}_{k-1} + d/k$, and $SS_k = \sum_{i\le k}(x_i - \bar{x}_k)^2 = SS_{k-1} + (k-1)(d/k)^2 + (d(k-1)/k)^2 = SS_{k-1} + d^2(k-1)/k$. For two sub-samples $(n_a, \bar{x}_a, M_a)$ and $(n_b, \bar{x}_b, M_b)$ with $\Delta = \bar{x}_b - \bar{x}_a$: $M = M_a + M_b + \Delta^2 n_a n_b/(n_a+n_b)$.

Exercise 1.5.4 The direct formula loses accuracy steadily (relative error about ux^2) and returns exactly 0 from $x = 10^8$ on, since $x^2 + 1$ rounds to x^2; the rewritten form $1/(\sqrt{x^2+1} + x)$ is accurate to about 10^{-16} for all x (true value $\approx 1/(2x)$).

Exercise 1.5.6 Half-up rounding of 12.5% of integer pesewa amounts is biased upwards (exact halves occur for one amount in eight and are always rounded up), with mean rounding error $+1/16$ pesewa per payment, so it pays roughly $1.5 \times 10^6/16 \approx 9.4 \times 10^4$ pesewas (about GHS 940) more per month than exact computation; half-even is unbiased; truncation underpays by about 0.44 pesewa per beneficiary per month. The differences are small but systematic; half-even or exact-cedi rules are easiest to defend.

Exercise 1.6.1 $\mathbf{x}^\top(\mathbf{Ay})$ and $(\mathbf{x}^\top\mathbf{A})\mathbf{y}$ both cost $\approx 2n^2$. For the chain: $(\mathbf{A}_1\mathbf{A}_2)\mathbf{A}_3$ costs $2\cdot 10\cdot 1000\cdot 10 + 2\cdot 10\cdot 10\cdot 1000 = 4\times 10^5$ flops, whereas $\mathbf{A}_1(\mathbf{A}_2\mathbf{A}_3)$ costs $2\times 10^7 + 2\times 10^7 = 4\times 10^7$: a factor of 100.

Exercise 1.6.2 Step k updates an $(n-k)\times(n-k)$ block with one multiply and one subtract per entry: $\sum_{k=1}^{n-1} 2(n-k)^2 = 2\sum_{j=1}^{n-1} j^2 = \frac{(n-1)n(2n-1)}{3} \sim \frac{2}{3}n^3$.

Exercise 1.6.3 `inv` followed by a product is typically two to three times slower than `solve`, and for $\operatorname{cond}(\mathbf{A}) \approx 10^{10}$ its residual $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|$ is usually several orders of magnitude larger; `solve` is backward stable, `inv(A) @ b` is not.

Exercise 1.6.6 Dense contributor-by-district: $8 \times 10^6 \times 260 \times 8\,\mathrm{B} \approx 16.6\,\mathrm{GB}$; contributor-by-employer: $8 \times 10^6 \times 4 \times 10^4 \times 8\,\mathrm{B} \approx 2.6\,\mathrm{TB}$. In CSR each has 8×10^6 non-zeros: about $8 \times 10^6 \times (8 + 4) + 4 \times (8 \times 10^6 + 1) \approx 130\,\mathrm{MB}$, and all district rates need one sparse product ($\approx 1.6 \times 10^7$ flops, well under a second). Document data extraction dates, linkage rules, code version, package versions and seeds.

Exercise 1.6.7 Adding a constant to all α_i and subtracting it from all ψ_j leaves fitted values unchanged, so each connected component of the worker–firm bipartite graph contributes one dimension to the null space; connected components are found by graph search (`scipy.sparse.csgraph.connected_components`) on the incidence matrix. The normal equations have millions of unknowns but are sparse, and conjugate-gradient iterations cost one sparse product each, whereas factorisation creates fill-in.

Chapter 2: Linear Algebra for Modelling: Rank, Projections and Decompositions

Exercise 2.1.1 For $\mathrm{col}(A)$: $A\mathbf{x} + A\mathbf{x}' = A(\mathbf{x} + \mathbf{x}')$ and $cA\mathbf{x} = A(c\mathbf{x})$; for $\mathrm{null}(A)$: if $A\mathbf{x} = A\mathbf{x}' = \mathbf{0}$ then $A(\mathbf{x} + c\mathbf{x}') = \mathbf{0}$. The union of the two coordinate axes in $\mathbb{R}^2$ contains $\mathbf{e}_1$ and $\mathbf{e}_2$ but not $\mathbf{e}_1 + \mathbf{e}_2$. Intersections are closed because each set is; $S + T$ is closed because $(\mathbf{s}+\mathbf{t})+(\mathbf{s}'+\mathbf{t}') = (\mathbf{s}+\mathbf{s}')+(\mathbf{t}+\mathbf{t}')$.

Exercise 2.1.2 $AB\mathbf{z} = A(B\mathbf{z}) \in \mathrm{col}(A)$, and $B\mathbf{z} = \mathbf{0} \Rightarrow AB\mathbf{z} = \mathbf{0}$. The first inclusion gives $\mathrm{rank}(AB) \le \mathrm{rank}(A)$; the second, with rank–nullity for B and AB (both have p columns), gives $p - \mathrm{rank}(AB) \ge p - \mathrm{rank}(B)$.

Exercise 2.1.3 Row 3 = row 1 + row 2, so $r = 2$. Bases: $\mathrm{col}(A)$: columns 1 and 2, $(1,0,1)^\top, (0,1,1)^\top$; $\mathrm{col}(A^\top)$: rows 1 and 2; $\mathrm{null}(A)$: $(-2,-1,1,0)^\top$, $(-1,-1,0,1)^\top$ (dimension $4 - 2 = 2$); $\mathrm{null}(A^\top)$: $\mathbf{y} = (1,1,-1)^\top$ (dimension $3 - 2 = 1$). Any $\mathbf{b}$ with $b_1 + b_2 \ne b_3$, e.g. $\mathbf{b} = (0,0,1)^\top$, is inconsistent, certified by $\mathbf{y}^\top\mathbf{b} = -1 \ne 0$.

Exercise 2.1.5 With Gaussian factors, $\mathrm{rank}(BC) = k$ with probability one; the experiment returns k in all 1200 cases and $\dim \mathrm{null}(A) = 8 - k$, computed from `scipy.linalg.null_space`.

Exercise 2.1.6 (i) Inconsistent: the budgets $\mathbf{b}$ have a component in $\mathrm{null}(A^\top)$, i.e. no set of three unit costs reproduces all budgets (measurement error, district-specific costs, omitted services). (ii) Infinitely many solutions: $\mathrm{null}(A) \ne \{\mathbf{0}\}$, i.e. service volumes are proportional across districts, so costs are not separately identified. With $m = 30 \gg 3$ the typical case is (i); the analyst should fit by least squares (projection of $\mathbf{b}$ onto $\mathrm{col}(A)$), report the residuals, and check $\mathrm{rank}(A) = 3$.

Exercise 2.1.7 Take a basis $\mathbf{s}_1, \dots, \mathbf{s}_k$; $B = [\mathbf{s}_1, \dots, \mathbf{s}_k] \in \mathbb{R}^{n \times k}$ gives $S = \mathrm{col}(B)$. A basis $\mathbf{a}_1, \dots, \mathbf{a}_{n-k}$ of $S^\perp$ as rows of $A \in \mathbb{R}^{(n-k)\times n}$ gives $\mathrm{null}(A) = (S^\perp)^\perp = S$. Fewer rows or columns are impossible by rank–nullity.

Exercise 2.2.1 $\mathrm{col}(A+B) \subseteq \mathrm{col}(A) + \mathrm{col}(B)$ and $\dim(S+T) \le \dim S + \dim T$. For Sylvester, apply rank–nullity to the map A restricted to $\mathrm{col}(B)$: $\mathrm{rank}(AB) = \mathrm{rank}(B) - \dim(\mathrm{null}(A) \cap \mathrm{col}(B)) \ge \mathrm{rank}(B) - (n - \mathrm{rank}\,A)$.

Exercise 2.2.2 $X\mathbf{c} = \mu\mathbf{1} + \gamma\,\mathbf{educ} + D\boldsymbol{\alpha} = \mathbf{0}$. Multiplying by the indicator of each region shows $\mu + \gamma\,\mathrm{educ}_i + \alpha_g = 0$ for all i in region g; since educ varies (rank of $[\mathbf{1}, \mathbf{educ}]$ is two, assuming it varies within at least one region), $\gamma = 0$ and then $\alpha_g = -\mu$ for all g, so $\mathbf{c} \propto (1, 0, -\mathbf{1}^\top)^\top$. If region g is empty its column is zero, and $\mathbf{e}_{2+g}$ is an additional null vector (nullity two).

Exercise 2.2.3 The row space is spanned by $(1, \text{educ}_i, \mathbf{e}_{g(i)}^\top)$. Estimable: $\alpha_2 - \alpha_1$, $\mu + \alpha_3$, γ. Not estimable: μ and $\sum_g \alpha_g$ (both have non-zero inner product with the null vector $\mathbf{c}$: $\boldsymbol{\ell}^\top \mathbf{c} = 1$ and $-G$ respectively, whereas $\boldsymbol{\ell} \in \text{col}(X^\top) = \text{null}(X)^\perp$ requires $\boldsymbol{\ell}^\top \mathbf{c} = 0$).

Exercise 2.2.4 The design has $2 + 16 + 5 = 23$ columns but rank 17 (educ and the 16 region dummies span everything): nullity 6. Five null vectors express each zone dummy as the sum of its regions' dummies, the sixth is the intercept trap.

Exercise 2.2.5 The standard deviation of $\hat{\beta}_3$ is approximately 0.37, 3.6, 36, 354 for $\delta = 10^{-1}, \ldots, 10^{-4}$: slope -1 on the log–log scale, because the variance is $\sigma^2 / \|M_{-3}\mathbf{x}_3\|^2$ and $\|M_{-3}\mathbf{x}_3\| \approx \delta\sqrt{n}$.

Exercise 2.2.6 Two-way fixed effects $y_{dt} = \tau(T_d \cdot \text{post}_t) + \alpha_d + \lambda_t + \varepsilon_{dt}$. The regressor $T_d \, \text{post}_t$ is not in the span of district and year dummies, so the rank is full (verify with `matrix_rank`). Causal interpretation additionally needs parallel trends: absent the programme, treated and untreated districts would have evolved alike.

Exercise 2.2.7 $X_1\mathbf{a} + X_2\mathbf{b} = \mathbf{0}$ with $\mathbf{b} \neq \mathbf{0}$ is possible iff either $X_2\mathbf{b} = \mathbf{0}$ (rank deficiency of X_2) or $X_2\mathbf{b} = -X_1\mathbf{a} \neq \mathbf{0}$ is in both column spaces. A time-invariant regressor lies in $\text{col}(D)$, so the intersection condition fails for it.

Exercise 2.3.1 $P\mathbf{x} = \mathbf{x}$ for $\mathbf{x} \in \text{col}(X_1) \subseteq \text{col}(X)$, so $PP_1 = P_1$, and transposing gives $P_1P = P_1$. Then $(P - P_1)^2 = P - P_1 - P_1 + P_1 = P - P_1$ and it is symmetric, so it is the orthogonal projector onto $\text{col}(M_1X_2)$ (by FWL). Hence $\|M\mathbf{y}\|^2 = \|M_1\mathbf{y}\|^2 - \|(P - P_1)\mathbf{y}\|^2 \leq \|M_1\mathbf{y}\|^2$, so RSS cannot increase and R^2 cannot decrease.

Exercise 2.3.3 With an intercept, $\sum_i (\hat{y}_i - \bar{y})(y_i - \bar{y}) = \sum_i (\hat{y}_i - \bar{y})^2$ because $\mathbf{e} \perp \hat{\mathbf{y}} - \bar{y}\mathbf{1}$. Hence $\text{Corr}(\mathbf{y}, \hat{\mathbf{y}})^2 = \text{ESS}^2/(\text{ESS} \cdot \text{TSS}) = R^2$. Without an intercept, regress $y = (1, 2)$ on $x = (1, 0)$: $\hat{\mathbf{y}} = (1, 0)$, RSS = 4, TSS = 0.5, so $1 - \text{RSS}/\text{TSS} = -7$ while the squared correlation is 1.

Exercise 2.3.5 $\sum_i h_{ii} = 11$ (10 features plus constant), average $11/442 = 0.025$; the maximum leverage is 0.128 at observation index 322 (zero-based). All properties of Theorem 2.17 hold to about 10^{-15}.

Exercise 2.3.6 $P_W = X(X^\top W X)^{-1} X^\top W$ satisfies $P_W^2 = P_W$ and $(WP_W)^\top = WP_W$ (self-adjoint in the W-inner product) but $P_W \neq P_W^\top$ in general. The GLS estimator with $\text{Var}(\varepsilon) = \Omega$ is the $W = \Omega^{-1}$ projection.

Exercise 2.4.1 $\text{col}(XA) = \text{col}(X)$, so P, $\hat{\mathbf{y}}$ and $\mathbf{e}$ are unchanged, and $\hat{\boldsymbol{\beta}}_{XA} = A^{-1}\hat{\boldsymbol{\beta}}_X$. Changing the base category of a set of dummies is such an invertible re-parametrisation: fitted values, residuals and the coefficients on all other regressors are unchanged; only the dummy coefficients (differences from the base) are re-expressed.

Exercise 2.4.2 $\tilde{X}_2^\top M_1 \mathbf{y} = X_2^\top M_1 M_1 \mathbf{y} = \tilde{X}_2^\top \mathbf{y}$. Counterexample: $X_1 = (1, 1)^\top$, $X_2 = (1, 0)^\top$, $\mathbf{y} = (1, 0)^\top$. Then $M_1\mathbf{y} = (\frac{1}{2}, -\frac{1}{2})^\top$; regressing it on X_2 gives $\frac{1}{2}$, whereas the full regression (exact fit $\mathbf{y} = 0 \cdot X_1 + 1 \cdot X_2$) gives 1, as does regressing $M_1\mathbf{y}$ on $\tilde{X}_2 = (\frac{1}{2}, -\frac{1}{2})^\top$.

Exercise 2.4.3 With $X_1 = \mathbf{1}$, M_1 centres: $\hat{b} = \tilde{\mathbf{x}}^\top \tilde{\mathbf{y}} / \tilde{\mathbf{x}}^\top \tilde{\mathbf{x}}$. With $X_1 = [\mathbf{1}, \mathbf{t}]$, M_1 detrends, and FWL says the slope in the regression with a trend equals the slope between detrended series.

Exercise 2.4.4 Without year effects, M_D is still demeaning by worker (with group sizes T_i), so LSDV = one-way within. With year effects, FWL with $X_1 = D$ requires demeaning the year dummies too and regressing $M_D\mathbf{y}$ on $[M_D\mathbf{x}, M_D D_t]$; two-way demeaning of y and x alone is exact only for balanced panels.

Exercise 2.4.6 Distance is time-invariant: $M_D\mathbf{x} = \mathbf{0}$, so its coefficient is not identified and software drops it. Its interaction with a trend, $x_d \cdot t$, varies within district and is identified (it allows districts far from the capital to follow a different trend).

Exercise 2.4.7 In a balanced panel the subspaces $\mathrm{col}(D_{\text{worker}}) \ominus \mathrm{col}(\mathbf{1})$ and $\mathrm{col}(D_{\text{year}}) \ominus \mathrm{col}(\mathbf{1})$ are orthogonal, so the projector onto the sum is $P_{\text{worker}} + P_{\text{year}} - P_{\mathbf{1}}$, which gives the stated formula. In unbalanced panels the two subspaces are not orthogonal and the sum of projectors is not a projector; alternating projections (repeated worker and year demeaning) converge to the projection onto the complement of the sum (von Neumann's theorem), which is how high-dimensional fixed-effects routines work.

Exercise 2.5.1 $H^\top = H$ is clear; $H^2 = \mathbf{I} - 4\mathbf{v}\mathbf{v}^\top/\mathbf{v}^\top\mathbf{v} + 4\mathbf{v}(\mathbf{v}^\top\mathbf{v})\mathbf{v}^\top/(\mathbf{v}^\top\mathbf{v})^2 = \mathbf{I}$. With $\alpha = -\operatorname{sign}(x_1)\,\|\mathbf{x}\|$ and $\mathbf{v} = \mathbf{x} - \alpha\mathbf{e}_1$: $\mathbf{v}^\top\mathbf{x} = \|\mathbf{x}\|^2 - \alpha x_1$ and $\mathbf{v}^\top\mathbf{v} = 2(\|\mathbf{x}\|^2 - \alpha x_1)$, so $H\mathbf{x} = \mathbf{x} - \mathbf{v} = \alpha\mathbf{e}_1$.

Exercise 2.5.2 $\det P = \pm 1$, $\det L = 1$, $\det U = \prod u_{ii}$; $\det\boldsymbol{\Sigma} = (\det L)^2 = \prod \ell_{ii}^2$. Summing logarithms avoids overflow or underflow of the determinant itself (for $n = 1000$ it can easily be $10^{\pm 400}$), and the factor L is needed anyway for the quadratic form $\left\|L^{-1}(\mathbf{x} - \boldsymbol{\mu})\right\|^2$.

Exercise 2.5.3 $X^\top X = R^\top Q^\top Q R = R^\top R$, and $R^\top$ is lower triangular; by uniqueness of the Cholesky factor with positive diagonal, $R^\top = LS$ where S is a diagonal matrix of signs ($S = \mathbf{I}$ if the diagonal of R is made positive).

Exercise 2.5.5 At degree 13 (relative coefficient errors: normal equations / QR / SVD): monomials on $[0, 1]$, $\kappa = 4.0 \times 10^9$: 1.8 / 7×10^{-8} / 7×10^{-8}; monomials on $[-1, 1]$, $\kappa = 3.9 \times 10^4$: 3×10^{-8} / 4×10^{-13} / 5×10^{-13}; Legendre polynomials, $\kappa = 5.1$: all three at about 10^{-15}. Re-parametrising the design is more effective than any solver.

Exercise 2.5.6 $\mathbf{x} = L^\top\mathbf{z}$ has covariance $L^\top L$, which differs from $LL^\top = \boldsymbol{\Sigma}$ unless L is diagonal. For unit variances and correlation ρ, $L = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix}$ and $L^\top L = \begin{pmatrix} 1+\rho^2 & \rho\sqrt{1-\rho^2} \\ \rho\sqrt{1-\rho^2} & 1-\rho^2 \end{pmatrix}$: with $\rho = 0.8$ the simulated variances are 1.64 and 0.36 and the correlation is about 0.63, not 0.8.

Exercise 2.5.7

$$(\mathbf{I} - A)^{-1} \approx \begin{pmatrix} 1.241 & 0.213 & 0.125 \\ 0.414 & 1.586 & 0.345 \\ 0.276 & 0.451 & 1.442 \end{pmatrix},$$

with column sums (total output multipliers) 1.93, 2.25, 1.91: one extra unit of final demand for industry raises total gross output by 2.25 units. A large multiplier measures linkages, not welfare: it ignores import leakages, capacity constraints, prices and the value of the output, so it should inform, not determine, investment priorities.

Exercise 2.6.1 For full column rank, $AA^+ = A(A^\top A)^{-1}A^\top$ is the symmetric projector P, and $A^+A = \mathbf{I}$; the four conditions follow immediately; similarly for full row rank. For the single equation, $A = (1, 2, 2)$, $AA^\top = 9$, so $\mathbf{x}^+ = A^\top \cdot 9/9 = (1, 2, 2)^\top$: the point of the plane $x_1 + 2x_2 + 2x_3 = 9$ closest to the origin, in the direction of the normal vector.

Exercise 2.6.2 (3) AA^+ is symmetric idempotent by (1) and (3), and $\operatorname{col}(AA^+) = \operatorname{col}(A)$ because $A = AA^+A$. Similarly for A^+A and $\operatorname{col}(A^\top)$. A counterexample to the reverse-order law: $A = \left(\begin{smallmatrix}1&1\\0&0\end{smallmatrix}\right)$, $B = \left(\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right)$. Then $AB = B$ and $(AB)^+ = B$, but $A^+ = \frac{1}{2}\left(\begin{smallmatrix}1&0\\1&0\end{smallmatrix}\right)$ and $B^+A^+ = \frac{1}{2}\left(\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right) \neq (AB)^+$.

Exercise 2.6.3 If $G\mathbf{b} = A^+\mathbf{b}$ for all $\mathbf{b}$, then $G = A^+$ (apply to the unit vectors). Uniqueness of the minimum-norm least-squares solution (Theorem 2.29) shows that any procedure returning it defines the same linear map.

Exercise 2.6.5 Ridge removes the explosion at $p = n$: the factor $\sigma_i/(\sigma_i^2 + \lambda)$ is bounded by $1/(2\sqrt{\lambda})$, so small singular values are not inverted. The larger λ, the lower and flatter the peak near $p/n = 1$; the leave-one-out choice of λ (computable from the hat-matrix diagonal of ridge) typically selects a clearly positive penalty near the threshold.

Exercise 2.6.7 In the SVD basis, $\mathbf{c}_t = V^\top\mathbf{b}_t$ satisfies $c_{t+1,i} = (1 - \eta\sigma_i^2)c_{t,i} + \eta\sigma_i\,(\mathbf{u}_i^\top\mathbf{y})$, so $c_{t,i} = \left[1 - (1 - \eta\sigma_i^2)^t\right]\mathbf{u}_i^\top\mathbf{y}/\sigma_i$ for $\sigma_i > 0$ and $c_{t,i} = 0$ for $\sigma_i = 0$. With $0 < \eta < 2/\sigma_1^2$, $\left|1 - \eta\sigma_i^2\right| < 1$ and $\mathbf{b}_t \to V\Sigma^+U^\top\mathbf{y} = X^+\mathbf{y}$. Stopping at finite t damps directions with small σ_i, like ridge.

Chapter 3: Eigenvalues, the SVD, PCA and Conditioning

Exercise 3.1.1 $A^k\mathbf{v} = A^{k-1}(\lambda\mathbf{v}) = \cdots = \lambda^k\mathbf{v}$ by induction; linearity gives $p(A)\mathbf{v} = \sum_k c_k A^k\mathbf{v} = p(\lambda)\mathbf{v}$. If A is invertible, $\mathbf{v} = A^{-1}A\mathbf{v} = \lambda A^{-1}\mathbf{v}$, so $\lambda \neq 0$ and $A^{-1}\mathbf{v} = \lambda^{-1}\mathbf{v}$. Finally A is singular iff $A\mathbf{v} = \mathbf{0} = 0\cdot\mathbf{v}$ for some $\mathbf{v} \neq \mathbf{0}$, i.e. iff $0 \in \sigma(A)$.

Exercise 3.1.2 $\det(\lambda\mathbf{I} - A^\top) = \det((\lambda\mathbf{I} - A)^\top) = \det(\lambda\mathbf{I} - A)$. For $A = \left(\begin{smallmatrix}1&1\\0&2\end{smallmatrix}\right)$ the right eigenvector for 2 is $(1,1)^\top$ but that of $A^\top$ is $(0,1)^\top$. For triangular A, $\lambda\mathbf{I} - A$ is triangular and its determinant is $\prod_i(\lambda - a_{ii})$. The rotation $\left(\begin{smallmatrix}0&-1\\1&0\end{smallmatrix}\right)$ has $p(\lambda) = \lambda^2 + 1$ and eigenvalues $\pm i$: no real direction is left unrotated.

Exercise 3.1.3 $\operatorname{tr} P = 2 - s - f$ and 1 is an eigenvalue, so $\lambda_2 = 1 - s - f$. Solving $\boldsymbol{\pi}^\top P = \boldsymbol{\pi}^\top$: $\pi_e s = \pi_u f$, so $u^* = \pi_u = s/(s+f)$. Since $u_{t+1} = u_t(1-f) + (1-u_t)s$, we get $u_{t+1} - u^* = (1-s-f)(u_t - u^*)$, hence $u_t - u^* = (1-s-f)^t(u_0 - u^*)$.

Exercise 3.1.5 With $p_{21} = 0.10$ and $p_{22} = 0.80$, the stationary shares (formal, informal, unemployed) become 0.612, 0.286 and 0.102, instead of 0.488, 0.390 and 0.122; $|\lambda_2|$ falls from 0.884 to 0.845, so the half-life falls from 5.6 to about 4.1 quarters. Starting from the old steady state, the formal share is within one percentage point of its new value after 15 quarters. (All numbers refer to the simulated chain.)

Exercise 3.1.6 Eigenvalues ± 1; $\mathbf{x}_0^\top P^t$ alternates between $\mathbf{x}_0^\top$ and $\mathbf{x}_0^\top P$ unless $\mathbf{x}_0 = (\frac{1}{2}, \frac{1}{2})$. The Cesàro average converges to $(\frac{1}{2}, \frac{1}{2})$ because the $\lambda = -1$ term averages to zero. Irreducibility holds but primitivity (aperiodicity) fails. $\frac{1}{2}(\mathbf{I} + P)$ has all entries $\frac{1}{2} > 0$, hence is primitive; its eigenvalues are 1 and 0.

Exercise 3.1.7 If $DP = P^\top D$ then

$$D^{1/2}PD^{-1/2} = D^{-1/2}(DP)D^{-1/2} = D^{-1/2}P^\top DD^{-1/2} = (D^{1/2}PD^{-1/2})^\top.$$

A matrix similar to a symmetric matrix has real eigenvalues. The simulated chain *is* reversible: $\pi_i p_{ij} = \pi_j p_{ji}$ holds for all pairs (e.g. $0.4878 \times 0.04 = 0.3902 \times 0.05$), consistent with its real spectrum. Economically, reversibility means gross flows between any two states balance in

steady state (as many workers move formal→informal as informal→formal each quarter); real labour markets need not satisfy it, for example along a job ladder.

Exercise 3.2.1 $\operatorname{tr}(A) = \operatorname{tr}(Q\Lambda Q^\top) = \operatorname{tr}(\Lambda Q^\top Q) = \sum \lambda_i$; $\|A\|_F$ is invariant under orthogonal transformations, so $\|A\|_F^2 = \|\Lambda\|_F^2$; and $\|A\mathbf{x}\| = \|\Lambda Q^\top \mathbf{x}\|$ is maximised over unit $\mathbf{x}$ by the eigenvector of the eigenvalue of largest modulus.

Exercise 3.2.2 $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 \geq 0$, with equality iff $\mathbf{X}\mathbf{v} = \mathbf{0}$; this has only the trivial solution iff $\operatorname{rank} \mathbf{X} = p$. If $\mathbf{X}^\top \mathbf{X}\mathbf{v} = \lambda \mathbf{v}$ with $\lambda \neq 0$ then $\mathbf{X}\mathbf{v} \neq \mathbf{0}$ and $\mathbf{X}\mathbf{X}^\top(\mathbf{X}\mathbf{v}) = \lambda(\mathbf{X}\mathbf{v})$, and conversely.

Exercise 3.2.3 $A = \left(\begin{smallmatrix} 0 & 0 \\ 0 & -1 \end{smallmatrix}\right)$: leading minors 0 and 0, but $\mathbf{e}_2^\top A \mathbf{e}_2 = -1$. The principal minor $a_{22} = -1$ (not leading) detects it.

Exercise 3.2.4 Let $\mathbf{z} = \boldsymbol{\Sigma}^{-1/2}(\mathbf{x} - \boldsymbol{\mu}) \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_p)$. Then $d_M^2 = \mathbf{z}^\top \mathbf{z} = \sum_i z_i^2 \sim \chi_p^2$. For $p = 2$, $\mathbb{P}(\chi_2^2 > c) = e^{-c/2}$, so $c = -2 \ln 0.05 = 5.991$.

Exercise 3.2.6 Courant–Fischer: for $\dim S = k$, S meets $\operatorname{span}\{\mathbf{q}_k, \dots, \mathbf{q}_n\}$ nontrivially, on which $R_A \leq \lambda_k$, so $\min_S R_A \leq \lambda_k$; equality for $S = \operatorname{span}\{\mathbf{q}_1, \dots, \mathbf{q}_k\}$. Since $R_{A+E}(\mathbf{x}) \leq R_A(\mathbf{x}) + \|E\|_2$ for every $\mathbf{x}$, the max–min gives $\lambda_k(A + E) \leq \lambda_k(A) + \|E\|_2$, and symmetrically. Eigenvalues move by at most $\|E\|_2$ (absolute condition number 1). Eigenvectors can rotate by an angle of order $\|E\|_2$ /gap (Davis–Kahan), which is large when eigenvalues are close.

Exercise 3.3.1 Then $A^k \mathbf{x}_0$ stays in $\operatorname{span}\{\mathbf{v}_2, \dots, \mathbf{v}_n\}$. In floating point, rounding errors introduce a component of size about $\varepsilon_{\text{mach}}$ along $\mathbf{v}_1$, which is then amplified at rate $|\lambda_1/\lambda_2|$ per step and eventually dominates.

Exercise 3.3.3 $\mathbf{1}^\top G = \alpha \mathbf{1}^\top M + (1 - \alpha)\mathbf{1}^\top = \mathbf{1}^\top$. If $G\mathbf{x} = \lambda \mathbf{x}$ with $\lambda \neq 1$ then $\lambda \mathbf{1}^\top \mathbf{x} = \mathbf{1}^\top G\mathbf{x} = \mathbf{1}^\top \mathbf{x}$, so $\mathbf{1}^\top \mathbf{x} = 0$ and $G\mathbf{x} = \alpha M \mathbf{x}$. Thus λ/α is an eigenvalue of the column-stochastic M, whose spectral radius is 1; hence $|\lambda| \leq \alpha$.

Exercise 3.3.4 Hint: `for k in range(K): Q, R = np.linalg.qr(A); A = R @ Q`. For symmetric A with distinct $|\lambda_i|$, the $(i + 1, i)$ entry decays like $|\lambda_{i+1}/\lambda_i|^k$; the slowest subdiagonal entry corresponds to the closest pair of eigenvalue moduli.

Exercise 3.3.6 $(A - \lambda_1 \mathbf{q}_1 \mathbf{q}_1^\top)\mathbf{q}_1 = \mathbf{0}$ and, for $i \geq 2$, $(A - \lambda_1 \mathbf{q}_1 \mathbf{q}_1^\top)\mathbf{q}_i = \lambda_i \mathbf{q}_i$ by orthogonality. An error δ in $\mathbf{q}_1$ leaves a residual component of size $O(\lambda_1 \|\delta\|)$ in the deflated matrix, contaminating later eigenpairs; errors accumulate with each deflation, which is why block methods (subspace iteration, QR) are preferred.

Exercise 3.4.1 $\|\mathbf{X}\mathbf{v}\|^2 = \sum_j \sigma_j^2 (\mathbf{v}_j^\top \mathbf{v})^2 \leq \sigma_1^2 \|\mathbf{v}\|^2$, with equality at $\mathbf{v}_1$; and $\|\mathbf{X}\|_F^2 = \operatorname{tr}(\mathbf{X}^\top \mathbf{X}) = \sum_j \sigma_j^2$. Then $\sigma_1^2 \leq \sum_{j \leq r} \sigma_j^2 \leq r\sigma_1^2$.

Exercise 3.4.3 $|\det \mathbf{X}| = |\det U| \det \Sigma |\det V| = \prod \sigma_j$. For symmetric $A = Q\Lambda Q^\top$ write $\Lambda = |\Lambda| \operatorname{diag}(\operatorname{sign} \lambda_i)$ and absorb the signs into the left factor. Example: $\left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$ has eigenvalues $1, 1$ but singular values $(\sqrt{5} \pm 1)/2 \approx 1.618,\ 0.618$.

Exercise 3.4.4 The suggested smooth image is a sum of two *separable* terms ($\sin(i/10)\cos(j/15)$ and a product of two one-dimensional Gaussians), hence has rank exactly 2: $\sigma_2/\sigma_1 \approx 0.127$ and $\sigma_3 \approx 0$, so rank 1 already keeps 98.4% of the squared Frobenius norm and rank 2 is exact. The digit mosaic needs rank 13 for 90%.

Exercise 3.5.2 For orthonormal W, $\sum_i \|(\mathbf{I} - WW^\top)\tilde{\mathbf{x}}_i\|^2 = \operatorname{tr}(\mathbf{X}_c^\top \mathbf{X}_c) - \operatorname{tr}(W^\top \mathbf{X}_c^\top \mathbf{X}_c W)$; the trace term is maximised by the top k eigenvectors (Ky Fan), with value $(n-1)\sum_{j\le k}\lambda_j$. The minimum is therefore $(n-1)\sum_{j>k}\lambda_j$.

Exercise 3.5.3 $R = (1-\rho)\mathbf{I} + \rho\mathbf{1}\mathbf{1}^\top$. Eigenvalues: $1 + (p-1)\rho$ with eigenvector $\mathbf{1}/\sqrt{p}$, and $1-\rho$ (multiplicity $p-1$) on $\mathbf{1}^\perp$. PC1 is the simple average of the standardised variables and explains $[1 + (p-1)\rho]/p \to \rho$ of the variance; this is the correlation matrix of a one-factor model with equal loadings $\sqrt{\rho}$.

Exercise 3.5.4 For the simulated panel of `pca_macro.py`, the criterion takes the values -0.004, -0.645, -1.060, -1.024 and -0.986 for $k = 0, \dots, 4$: the minimum is at $k = 2$, the true number of factors.

Exercise 3.5.6 Hint: the correlation is near zero for small m and rises sharply once $m \cdot \bar{\lambda}^2/\sigma_e^2$ is of order $\sqrt{N/T}$; the threshold moves left as T grows. Weak factors are not consistently estimated by PCA.

Exercise 3.6.1 $\|A\|_2 = \sigma_1$ and $\left\|A^{-1}\right\|_2 = 1/\sigma_n$ (the singular values of A^{-1} are $1/\sigma_j$). $\|Q\mathbf{x}\| = \|\mathbf{x}\|$ gives $\kappa_2(Q) = 1$. Submultiplicativity gives $\kappa(AB) \le \kappa(A)\kappa(B)$. $\mathbf{X}^\top\mathbf{X} = V\Sigma^2 V^\top$ has extreme eigenvalues σ_1^2, σ_p^2.

Exercise 3.6.2 $E = -\sigma_n \mathbf{u}_n \mathbf{v}_n^\top$ makes $A + E$ singular with $\|E\|_2 = \sigma_n$. Conversely if $A + E$ is singular, $(A+E)\mathbf{x} = \mathbf{0}$ for a unit $\mathbf{x}$, so $\sigma_n \le \|A\mathbf{x}\| = \|E\mathbf{x}\| \le \|E\|_2$.

Exercise 3.6.3 With two regressors $R = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$, $R^{-1} = \frac{1}{1-\rho^2}\begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}$, so $\mathrm{VIF}_j = 1/(1-\rho^2)$; eigenvalues $1 \pm \rho$ with eigenvectors $(1, \pm 1)/\sqrt{2}$.

Exercise 3.6.4 On 100 points in $[0,1]$: for $n = 4, 8, 12$ columns, κ is about 1.2×10^2, 1.2×10^5, 1.2×10^8 for raw monomials, 8, 2.3×10^2, 6.9×10^3 for monomials in $2t-1$, and only 2.6, 3.7, 4.6 for Legendre polynomials.

Exercise 3.6.5 Monte Carlo with 1000 replications at $\rho = 0.95$: the MSE of $\hat{\beta}_1$ is 0.209 (OLS), 0.179 ($k=1$), 0.127 ($k=3$), 0.060 ($k=10$), 0.020 ($k=30$), 0.040 ($k=100$), so $k \approx 30$ is best. For $\hat{\beta}_1 + \hat{\beta}_2$ the MSE is ≈ 0.023 for $k \le 10$ and rises to 0.035 and 0.154 at $k = 30, 100$: the well-determined combination gains nothing from shrinkage. (Ridge does particularly well for β_1 here because the true $\beta_1 = \beta_2$, and ridge shrinks the poorly determined *difference* towards zero.)

Exercise 3.6.6 In the SVD basis, $\hat{\boldsymbol{\beta}}_{\text{ridge}} = \sum_j \frac{\sigma_j^2}{\sigma_j^2+k}\gamma_j \mathbf{v}_j$ + noise with $\gamma_j = \mathbf{v}_j^\top\boldsymbol{\beta}$; squared bias $\sum_j \frac{k^2\gamma_j^2}{(\sigma_j^2+k)^2}$, variance $\sigma^2\sum_j \frac{\sigma_j^2}{(\sigma_j^2+k)^2}$. At $k=0$ the derivative of the MSE is $-2\sigma^2\sum_j \sigma_j^{-4} < 0$, so a small $k > 0$ improves MSE. Gauss–Markov concerns only *unbiased* linear estimators; ridge is biased.

Chapter 4: Probability Foundations, Random Variables and Transformations

Exercise 4.1.1 $\varnothing = \Omega^c \in \mathcal{F}$. For $A_i \in \mathcal{F}$, $\bigcap_i A_i = (\bigcup_i A_i^c)^c \in \mathcal{F}$ by (ii), (iii) and De Morgan. Finally $A \setminus B = A \cap B^c$ is an intersection of two members of $\mathcal{F}$.

Exercise 4.1.2 The generated σ-algebra is that of the partition $\{\{1\},\{2\},\{3,4\}\}$; its events are all unions of cells: $\varnothing$, $\{1\}$, $\{2\}$, $\{3,4\}$, $\{1,2\}$, $\{1,3,4\}$, $\{2,3,4\}$, Ω, i.e. $8 = 2^3$ events. A partition with k cells generates 2^k events.

Exercise 4.1.3 By De Morgan and Boole, $\mathbb{P}(\bigcap A_i) = 1 - \mathbb{P}(\bigcup A_i^c) \geq 1 - \sum \mathbb{P}(A_i^c)$. If A_i is "test i does not falsely reject" and each test has level α/m, then $\mathbb{P}(\text{no false rejection}) \geq 1 - m(\alpha/m) = 1 - \alpha$: the Bonferroni correction controls the family-wise error rate.

Exercise 4.1.4 $\mathbb{P}(\text{exactly } k \text{ matches}) = \frac{1}{k!}\sum_{j=0}^{n-k} \frac{(-1)^j}{j!} \to e^{-1}/k!$. With $n = 10$ and 200,000 simulated permutations (seed 803) the frequencies for $k = 0, \ldots, 4$ are $0.368, 0.369, 0.183, 0.061, 0.016$, against Poisson(1) values $0.368, 0.368, 0.184, 0.061, 0.015$. The match indicators are nearly independent rare events (Poisson approximation).

Exercise 4.1.5 The prediction is $n \approx 1.96^2 \cdot \frac{1}{6} \cdot \frac{5}{6} / 0.005^2 \approx 21{,}300$ rolls, by the normal approximation to the binomial; a simulation with 1,000 paths should find the 95% criterion first met near this value.

Exercise 4.1.6 If $\mathbb{P}$ is countably additive, continuity follows as in Theorem 4.7. Conversely, for disjoint A_i put $B_n = \bigcup_{i \leq n} A_i \uparrow \bigcup_i A_i$; finite additivity gives $\mathbb{P}(B_n) = \sum_{i \leq n} \mathbb{P}(A_i)$, and continuity gives the limit. Countable additivity is what makes limits (laws of large numbers, CDF limits, convergence theorems for integrals) legitimate.

Exercise 4.1.7 $\hat{p} = 4/30 \approx 0.133$ with standard error $\sqrt{0.133 \times 0.867/30} \approx 0.062$. The past may mislead because of climate trends (non-stationarity) and changes in the station or in farming practice; a subjective prior can incorporate climate projections and expert knowledge.

Exercise 4.2.2 $\mathbb{P}(A) = \mathbb{P}(B) = \mathbb{P}(C) = \frac{1}{2}$ and each pairwise intersection has probability $\frac{1}{4}$, but $A \cap B \cap C = \varnothing$, so $\mathbb{P}(ABC) = 0 \neq \frac{1}{8}$.

Exercise 4.2.3 $\mathbb{P}(AB^c) = \mathbb{P}(A) - \mathbb{P}(AB) = \mathbb{P}(A)(1 - \mathbb{P}(B))$. For $A \perp B \mid C$ but not independent: two tests that are conditionally independent given disease status (malaria example). For $A \perp B$ but dependent given C: two independent fair coins and $C = \{\text{at least one head}\}$; given C, learning the first is tails forces the second to be heads.

Exercise 4.2.4 Setting PPV $= \frac{1}{2}$ gives $\text{sens} \cdot p = (1 - \text{spec})(1 - p)$, so $p^* = (1 - \text{spec})/(\text{sens} + 1 - \text{spec})$. With spec $= 0.95$, $p^* = 0.05/1.00 = 0.05$, in agreement with the output of `ch04_malaria.py`.

Exercise 4.2.5 At 25% coverage (equal to the poverty rate) both errors coincide: for noise s.d. $0, 0.2, 0.35, 0.5, 0.8$ they are approximately $0, 0.16, 0.27, 0.35, 0.46$. With zero noise the PMT ranks households exactly, so the selected 25% are precisely the poor.

Exercise 4.2.6 $\text{PPV} = 0.7p/(0.7p + 0.1(1 - p))$. At $p = 0.5$, PPV $= 0.875$ (inclusion error 12.5%); at $p = 0.1$, PPV $= 0.4375$ (inclusion error 56%). A uniform cut-off directs a much larger share of benefits to non-poor households in low-poverty districts.

Exercise 4.2.7 $\log \frac{\mathbb{P}(H|A_1 \ldots A_n)}{\mathbb{P}(H^c|A_1 \ldots A_n)} = \log \frac{\mathbb{P}(H)}{\mathbb{P}(H^c)} + \sum_i \log \frac{\mathbb{P}(A_i|H)}{\mathbb{P}(A_i|H^c)}$. With binary features each log likelihood ratio is affine in x_i, so the posterior log-odds is linear in $\mathbf{x}$: the logistic-regression form. Naive Bayes estimates the coefficients generatively (from class-conditional frequencies); logistic regression estimates them discriminatively by maximising the conditional likelihood.

Exercise 4.3.2 For each k, at most k jumps can exceed $1/k$ (otherwise the total increase would exceed 1). The set of jumps is a countable union of finite sets.

Exercise 4.3.4 With 2,000 replications the empirical coverage of the 95% DKW band is about 0.957 for $n = 50$ and 0.960 for $n = 500$: the band is slightly conservative, as expected from an inequality.

Exercise 4.3.5 $\mathbb{E}X^2 = (1-\pi_0)\mathbb{E}[Y^2]$ with $Y \sim G$, so $\operatorname{Var} X = (1-\pi_0)\mathbb{E}Y^2 - (1-\pi_0)^2(\mathbb{E}Y)^2$. For the log-normal with $\mu = 3$, $\sigma = 1$, $\mathbb{E}Y^2 = e^8 \approx 2981$ and $\mathbb{E}Y \approx 33.1$, giving $\operatorname{Var} X \approx 1394$; a simulation of 100,000 households gives about 1384.

Exercise 4.3.6 $nF_n(x) \sim \operatorname{Bin}(n, F(x))$ and the CLT gives the marginal limit. For $x < y$, $\operatorname{Cov}(\mathbf{1}\{X \le x\}, \mathbf{1}\{X \le y\}) = F(x) - F(x)F(y) = F(x)(1 - F(y))$, which is the covariance of a Brownian bridge B at times $F(x) \le F(y)$; hence the limit process is $B \circ F$ (Donsker's theorem).

Exercise 4.4.1 $f_X(x) = 2(1-x)$ on $(0,1)$, $f_Y(y) = 2y$ on $(0,1)$, and $f_{Y|X}(y \mid x) = 1/(1-x)$ on $(x,1)$: uniform. X, Y are dependent because the support is not a product set.

Exercise 4.4.3 M is the CDF of (U, U) (comonotone), W that of $(U, 1-U)$ (countermonotone), and Π that of independent uniforms. Every copula satisfies $W \le C \le M$.

Exercise 4.4.4 The person-level poverty rate is $\sum_x m_x p(x, \text{poor}) / \sum_x m_x p_X(x) \approx 0.369$, against 0.300 for households, because poor households are larger: a random person is more likely than a random household to live in a large household.

Exercise 4.4.5 With 2 million simulated pairs, $\mathbb{P}(U_2 > 0.99 \mid U_1 > 0.99) \approx 0.26$ for the Gaussian copula and ≈ 0.47 for the t_3 copula with $r = 0.7$. The t copula has positive tail dependence; the Gaussian copula's conditional tail probability tends to zero as the threshold tends to 1.

Exercise 4.5.1 $-\log X \sim \text{Exp}(1)$: $f(y) = e^{-y}$, $y > 0$. X^2 has density $f(y) = \frac{1}{2\sqrt{y}}$ on $(0,1)$.

Exercise 4.5.3 $\mathbb{P}(X_{(k)} \in [x, x+dx])$ requires $k-1$ observations below x, one in $[x, x+dx]$ and $n-k$ above; counting arrangements gives the multinomial coefficient. For uniforms $F(x) = x$, giving the $\text{Beta}(k, n-k+1)$ density.

Exercise 4.5.5 In a simulation with 100,000 observations the mean of Y is about 4.23; the mean of the naively exponentiated fits is about 3.08 (27% too low); multiplying by $e^{\hat{\sigma}^2/2}$ gives about 4.23.

Exercise 4.5.6 If $\mathbf{x} = \mathbf{g}(\mathbf{z})$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then $\log f_{\mathbf{X}}(\mathbf{x}) = \log \phi_d(\mathbf{h}(\mathbf{x})) + \log |\det J_{\mathbf{h}}(\mathbf{x})|$, and for a composition the log-determinants add. Triangular Jacobians make the determinant the product of the diagonal, costing $O(d)$ instead of $O(d^3)$.

Exercise 4.5.7 (a) The total is Gamma(40, 1): mean 40, s.d. 6.3, 99th percentile about 56.2. (b) The maximum has CDF $(1-e^{-x})^{40}$: mean $\sum_{k=1}^{40} 1/k \approx 4.28$ and 99th percentile $-\log(1-0.99^{1/40}) \approx 8.29$. Reserves are driven by the total; positive dependence fattens the tail of the total substantially, so independence-based reserves would be too low.

Exercise 4.6.1 With $c = 0$ and $m = 2^e$: if a is even the iterates eventually become 0 and stay there; if a is odd the power of 2 dividing x_k never changes, so only residues with the same 2-adic valuation as the seed occur (and 0 never occurs). Either way the period is less than m. $65539^2 = 2^{32} + 6 \cdot 2^{16} + 9 \equiv 6 \cdot 2^{16} + 9 = 6 \cdot 65539 - 9 \pmod{2^{31}}$, so $x_{k+2} = 65539^2 x_k = (6 \cdot 65539 - 9)x_k = 6x_{k+1} - 9x_k \pmod{2^{31}}$.

Exercise 4.6.2 Weibull: $F^{-1}(u) = \beta(-\log(1-u))^{1/k}$. Logistic: $F^{-1}(u) = \log(u/(1-u))$. Geometric: $\mathbb{P}(\lceil \log U / \log(1-p) \rceil \geq k) = \mathbb{P}(U \leq (1-p)^{k-1}) = (1-p)^{k-1}$, the geometric survivor function.

Exercise 4.6.3 $M = \sup f/g$; for the Beta with uniform proposal this is the density at the mode $(\alpha-1)/(\alpha+\beta-2)$. For the normal target with a Cauchy proposal, $f/g = \sqrt{\pi/2}\,(1+x^2)e^{-x^2/2}$ is maximised at $x = \pm 1$, giving $M = \sqrt{2\pi/e} \approx 1.52$.

Exercise 4.6.4 Over all 10,000 four-digit seeds, the number of distinct values before the first repeat is at most 111 and has median 45: the method is useless for serious simulation.

Exercise 4.6.5 $f/g = \sqrt{2/\pi}\, e^{x - x^2/2}$ is maximised at $x = 1$, so $M = \sqrt{2e/\pi} \approx 1.315$ and the acceptance rate is $1/M \approx 0.760$.

Exercise 4.6.7 Use one seed per scenario component and the *same* uniforms for take-up in both scenarios (a household takes up if $U_i < p_i$ under each scenario's probability). Because the two costs are then strongly positively correlated, $\operatorname{Var}(C_1 - C_0) = \operatorname{Var} C_1 + \operatorname{Var} C_0 - 2\operatorname{Cov}(C_1, C_0)$ is much smaller than with independent streams.

Chapter 5: Common Distributions in Data Science and Econometrics; Likelihood

Exercise 5.1.1 $\mathbb{E}[X(X-1)] = \sum_{k\geq 2} k(k-1)e^{-\lambda}\lambda^k/k! = \lambda^2 \sum_{j\geq 0} e^{-\lambda}\lambda^j/j! = \lambda^2$, so $\operatorname{Var} X = \lambda^2 + \lambda - \lambda^2 = \lambda$. $G(s) = \sum_k e^{-\lambda}(\lambda s)^k/k! = e^{\lambda(s-1)}$. For independent variables PGFs multiply: $e^{\lambda_1(s-1)}e^{\lambda_2(s-1)} = e^{(\lambda_1+\lambda_2)(s-1)}$, the PGF of Pois$(\lambda_1 + \lambda_2)$; a PGF determines the distribution.

Exercise 5.1.2 $\mathbb{P}(T > k) = (1-p)^k$, so $\mathbb{P}(T > s+t \mid T > s) = (1-p)^t$. Conversely, memorylessness gives $S(s+t) = S(s)S(t)$ on the integers, so $S(k) = S(1)^k = q^k$ and $\mathbb{P}(T = k) = q^{k-1}(1-q)$: geometric with $p = 1 - q$.

Exercise 5.1.3 $\mathbb{P}(N_F = a, N_D = b) = \mathbb{P}(N = a+b)\binom{a+b}{a} q^a (1-q)^b = e^{-\lambda q}\frac{(\lambda q)^a}{a!} \cdot e^{-\lambda(1-q)}\frac{(\lambda(1-q))^b}{b!}$, a product of Poisson PMFs.

Exercise 5.1.4 With $\lambda = 2$ the exact distribution of S (by repeated convolution) gives $d_{TV} = 0.070,\ 0.0061,\ 0.0006$ for $n = 10, 100, 1000$, against Le Cam bounds $\sum p_i^2 = 0.509,\ 0.053,\ 0.0053$. Heterogeneity barely affects the quality of the approximation; the bound is conservative by a factor of about 8.

Exercise 5.1.6 $\hat{r} = \bar{y}^2/(s^2 - \bar{y})$ requires $s^2 > \bar{y}$. With $n = 50$ draws from NB$(5, \mu = 2)$ (true variance 2.8), about 8% of samples (84 of 1000 with seed 803) have $s^2 \le \bar{y}$. In small samples the sampling variability of s^2 is large, so moment-based tests and estimates of overdispersion are unreliable; likelihood-based estimates constrained to $\alpha \ge 0$ (with a boundary at the Poisson) are preferable.

Exercise 5.1.7 (a) $\hat{\alpha} = (s^2 - \bar{y})/\bar{y}^2 = 7.2/16.81 = 0.428$ ($\hat{r} = 2.34$), $\hat{\mu} = 4.1$. (b) $\mathbb{P}(Y \ge 10) = 0.0095$ under Pois(4.1) and 0.073 under the negative binomial. (c) The rule flags about one week in 105 under the Poisson but about one week in 14 (roughly 4 weeks a year) under the negative binomial; with overdispersed data most "crime waves" would be ordinary fluctuations.

Exercise 5.2.1 $\mathbb{P}(-\log U/\lambda > t) = \mathbb{P}(U < e^{-\lambda t}) = e^{-\lambda t}$. Similarly $\mathbb{P}(\lambda(-\log U)^{1/k} > t) = \mathbb{P}(U < e^{-(t/\lambda)^k}) = e^{-(t/\lambda)^k}$. The mean is $\int_0^\infty S(t)\mathrm{d}t = \lambda \int_0^\infty e^{-u^k}\mathrm{d}u = \lambda\Gamma(1 + 1/k)$.

Exercise 5.2.2 $\mathbb{P}(\min_j T_j > t) = \prod_j e^{-\lambda_j t} = e^{-(\sum \lambda_j)t}$, and

$$\mathbb{P}\big(T_1 < \min_{j\neq 1} T_j\big) = \int_0^\infty \lambda_1 e^{-\lambda_1 t} e^{-\sum_{j\neq 1}\lambda_j t}\mathrm{d}t = \lambda_1 \Big/ \sum_j \lambda_j.$$

In a competing-risks model of unemployment the overall exit hazard is the sum of the destination-specific hazards, and each destination's share of exits is its share of the total hazard.

Exercise 5.2.4 $S(t) = \pi e^{-\lambda_H t} + (1 - \pi)e^{-\lambda_L t}$ and $h(t) = \sum_j w_j(t)\lambda_j$ with weights $w_H(t) = \pi e^{-\lambda_H t}/S(t)$ decreasing to 0. Hence $h'(t) = -\operatorname{Var}_{w(t)}(\lambda) < 0$ (differentiate) and $h(t) \to \lambda_L$.

Exercise 5.2.6 By simulation from the log-normal model ($z = 6000$): baseline $(H, \mathrm{FGT}_1, \mathrm{FGT}_2) = (0.193, 0.061, 0.028)$; uniform 10% growth gives $(0.162, 0.050, 0.022)$; reducing σ to 0.7 at constant mean (μ rises by $(0.8^2 - 0.7^2)/2 = 0.075$) gives $(0.136, 0.037, 0.015)$. The inequality reduction lowers FGT_2 much more, because it concentrates gains at the bottom.

Exercise 5.2.8 Solve $\Phi(\Phi^{-1}(0.4) - \sigma) = 0.15$: $\hat{\sigma} = 0.783$, implying $G = 2\Phi(\sigma/\sqrt{2}) - 1 = 0.420$ and a top-10% share $1 - \Phi(\Phi^{-1}(0.9) - \sigma) = 0.309$. One Lorenz point identifies the single shape parameter; everything else, notably the top tail, is imposed by the functional form.

Exercise 5.3.2 $T^2 = Z^2/(Q/\nu) = (\chi_1^2/1)/(\chi_\nu^2/\nu)$ with independent numerator and denominator. For the second part write $d_1F/(d_1F + d_2) = Q_1/(Q_1 + Q_2)$ with $Q_j \sim \mathrm{Gamma}(d_j/2, 1/2)$ independent; the ratio $Q_1/(Q_1 + Q_2)$ of independent gammas with a common rate is $\mathrm{Beta}(d_1/2, d_2/2)$.

Exercise 5.3.4 With $n = 8$ exponential observations the variance of $(n-1)S^2/\sigma^2$ is about 50 rather than $2(n-1) = 14$, because $\operatorname{Var}(S^2)$ depends on the fourth moment (excess kurtosis 6 for the exponential), and $\operatorname{Corr}(\bar{X}, S^2) \approx 0.69$: in a right-skewed sample a large observation raises both the mean and the variance. Independence of $\bar{X}$ and S^2 characterises the normal.

Exercise 5.3.5 With 20,000 replications the rejection rate is about 0.055 with t_3 errors and 0.051 with normal errors: the t-test of a slope is quite robust to heavy-tailed (symmetric) errors, although its power is lower than that of robust alternatives.

Exercise 5.3.6 Write $\boldsymbol{A} = \boldsymbol{U}\,\mathrm{diag}(\lambda_i)\boldsymbol{U}^\top$; $\mathbf{z}^\top\boldsymbol{A}\mathbf{z} = \sum_i \lambda_i w_i^2$ with w_i iid $\mathcal{N}(0,1)$, with MGF $\prod_i(1 - 2t\lambda_i)^{-1/2}$. This equals $(1-2t)^{-r/2}$ in a neighbourhood of zero iff exactly r eigenvalues equal 1 and the rest 0, i.e. $\boldsymbol{A}$ is idempotent of rank r.

Exercise 5.4.1 $\mathbb{P}(W \le w) = \frac{1}{2}\mathbb{P}(Z \le w) + \frac{1}{2}\mathbb{P}(-Z \le w) = \Phi(w)$; $\operatorname{Cov}(Z, W) = \mathbb{E}[S]\mathbb{E}[Z^2] = 0$; but $|W| = |Z|$, so they are dependent. $Z + W = 0$ with probability $1/2$, so $Z + W$ is not normal and (Z, W) is not jointly normal.

Exercise 5.4.2 $\Sigma_{12}\Sigma_{22}^{-1} = \rho\sigma_1\sigma_2/\sigma_2^2 = \rho\sigma_1/\sigma_2$ and $\Sigma_{11} - \Sigma_{12}^2/\Sigma_{22} = \sigma_1^2(1-\rho^2)$. In standard units the predicted X_1 is $\rho < 1$ times the standardised X_2: extreme values of X_2 predict less extreme X_1.

Exercise 5.4.3 From the block factorisation in the proof of Theorem 5.22, invert each factor: $\boldsymbol{\Sigma}^{-1} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ -\boldsymbol{W}^\top & \mathbf{I} \end{pmatrix} \operatorname{diag}(\boldsymbol{\Sigma}_{1|2}^{-1}, \boldsymbol{\Sigma}_{22}^{-1}) \begin{pmatrix} \mathbf{I} & -\boldsymbol{W} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}$, whose upper-left block is $\boldsymbol{\Sigma}_{1|2}^{-1}$.

Exercise 5.5.1 Binomial: $\eta = \log\frac{p}{1-p}$, $T = y$, $A = n\log(1+e^\eta)$, $h = \binom{n}{y}$; $A' = np$, $A'' = np(1-p)$. Negative binomial: $\eta = \log(1-p)$, $T = y$, $A = -r\log(1-e^\eta)$, $h = \binom{y+r-1}{y}$; $A' = r(1-p)/p$, $A'' = r(1-p)/p^2$. Gamma: $\boldsymbol{T} = (\log y, y)$, $\boldsymbol{\eta} = (\alpha - 1, -\beta)$, $A = \log\Gamma(\eta_1 + 1) - (\eta_1 + 1)\log(-\eta_2)$, $h = 1$; $\partial A/\partial\eta_2 = (\eta_1 + 1)/(-\eta_2) = \alpha/\beta$.

Exercise 5.5.3 Newton: $\boldsymbol{\beta}^+ = \boldsymbol{\beta} + (\mathbf{X}^\top\boldsymbol{W}\mathbf{X})^{-1}\mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu}) = (\mathbf{X}^\top\boldsymbol{W}\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{W}\{\mathbf{X}\boldsymbol{\beta} + \boldsymbol{W}^{-1}(\mathbf{y} - \boldsymbol{\mu})\}$, which is (5.19). For the normal with identity link, $\boldsymbol{W} = \mathbf{I}$ and $\mathbf{z} = \mathbf{y}$, so one step gives OLS.

Exercise 5.5.4 On the simulated data of Listing 5.12 the ratios $\hat\beta_{\text{logit}}/\hat\beta_{\text{probit}}$ are about 1.67–1.72, close to the folklore 1.6–1.8. Average marginal effects of a year of schooling are nearly identical (about 0.045 for both models): the models differ in scale, not in substance.

Exercise 5.5.6 The score equation for the intercept is $\sum_i (y_i - \hat\mu_i)\cdot 1 = 0$. A canonical-link model with an intercept therefore predicts the correct total, so an allocation formula based on $\hat\mu_i$ neither over- nor under-spends the budget in aggregate (though it may misallocate across districts).

Exercise 5.6.1 (a) $\theta^{-n}\mathbf{1}\{y_{(n)} \le \theta\}$: $T = \max_i y_i$. (b) $(\sum y_i, \sum y_i^2)$. (c) $\prod_i y_i$ (equivalently $\sum \log y_i$). (d) $\sum_i y_i^k$.

Exercise 5.6.2 Completed spells contribute $\lambda e^{-\lambda t_i}$, censored spells $e^{-\lambda c_j}$, so $\ell = d\log\lambda - \lambda\sum(\text{all observed times})$ and $\hat\lambda = d/\sum\text{time}$. Dropping censored spells discards the longest spells, so the estimated mean duration is biased downward; treating censoring times as completed spells also biases it downward.

Exercise 5.6.3 Binomial: $\mathbb{P}(Y \le 3 \mid n = 12, p = \frac{1}{2}) = 0.073$. Negative binomial: $\mathbb{P}(N \ge 12) = \mathbb{P}(\text{at most two successes in 11 trials}) = 0.033$. The same data are "significant" at 5% under one stopping rule but not under the other, although the likelihoods are proportional.

Exercise 5.6.4 For the sample in Listing 5.14, $\ell_p(\mu) = -\frac{n}{2}\log\{n^{-1}\sum(y_i - \mu)^2\}+\text{const}$; the LR interval is $[1.328, 1.988]$ versus the t interval $[1.321, 1.995]$ — nearly identical for $n = 40$.

Exercise 5.6.6 If $\psi = g(\theta)$, $\mathcal{L}^*(\psi) = \mathcal{L}(g^{-1}(\psi))$ is maximised at $g(\hat\theta)$. A posterior density transforms with a Jacobian, $p_\psi(\psi) = p_\theta(g^{-1}(\psi))\left|\mathrm{d}g^{-1}/\mathrm{d}\psi\right|$, so its mode moves. For example, a Beta$(3, 2)$ posterior for θ has mode $2/3$, but the implied density of $\psi = \log\{\theta/(1-\theta)\}$ is proportional to $\theta^3(1-\theta)^2$ (the Jacobian is $\theta(1-\theta)$), which is maximised at $\theta = 3/5$, not at $2/3$.

Exercise 5.6.7 $p(\lambda \mid \mathbf{y}) \propto \lambda^{\sum y_i} e^{-n\lambda} \lambda^{a-1} e^{-b\lambda}$, so $\lambda \mid \mathbf{y} \sim \text{Gamma}(a + \sum y_i, b + n)$. The predictive of a new count is a gamma mixture of Poissons and hence, by Proposition 5.5, $\text{NB}(r = a + \sum y_i,\ p = (b + n)/(b + n + 1))$.

Exercise 5.6.8 Posterior $\text{Beta}(9 + 61, 21 + 89) = \text{Beta}(70, 110)$, mean 0.389 (MLE 0.407, prior mean 0.30), 90% credible interval $[0.330, 0.449]$, and $\mathbb{P}(\theta > 0.35 \mid \text{data}) = 0.86$. With clustering, pupils in the same school are positively correlated, the effective sample size is smaller than 150, and the binomial likelihood overstates the information; a hierarchical (beta-binomial by school) model would widen the interval.

Chapter 6: Expectation, Covariance, Limit Theorems and Concentration

Exercise 6.1.1 $X = \sum_{k \geq 1} \mathbf{1}\{X \geq k\}$ pointwise; take expectations and exchange sum and expectation (monotone convergence): $\mathbb{E}X = \sum_{k \geq 1} \mathbb{P}(X \geq k)$. For the geometric law $\mathbb{P}(X \geq k) = (1 - p)^{k-1}$, so $\mathbb{E}X = \sum_{k \geq 1} (1 - p)^{k-1} = 1/p$.

Exercise 6.1.2 $\sqrt{x}$ is concave: $\mathbb{E}\sqrt{X} \leq \sqrt{\mu}$. x^2 is convex: $\mathbb{E}X^2 \geq \mu^2$. e^{-x} is convex: $\mathbb{E}e^{-X} \geq e^{-\mu}$. All three hold for any integrable $X > 0$ (for $\mathbb{E}X^2$ the inequality is trivially true if $\mathbb{E}X^2 = \infty$); no further assumptions are needed beyond integrability, and equality holds only for degenerate X.

Exercise 6.1.3 $u(\mu - \pi) \approx u(\mu) - \pi u'(\mu)$ and $\mathbb{E}u(W) \approx u(\mu) + \frac{1}{2}u''(\mu) \operatorname{Var} W$; equate to get $\pi \approx -\frac{1}{2}u''(\mu) \operatorname{Var} W / u'(\mu)$. For CRRA with $\log C \sim \mathcal{N}(\mu, s^2)$: $\mathbb{E}C^{1-\gamma} = \exp\{(1-\gamma)\mu + (1-\gamma)^2 s^2/2\}$ by the normal MGF, so $\text{CE} = (\mathbb{E}C^{1-\gamma})^{1/(1-\gamma)} = \exp\{\mu + (1 - \gamma)s^2/2\}$ (and the same formula for $\gamma = 1$ from $\mathbb{E}\log C = \mu$).

Exercise 6.1.5 By quadrature the poverty gap index is $P_1 \approx 0.1274$; Monte Carlo with 10^6 draws gives 0.1276 (s.e. 0.0002). The headcount is $\mathbb{P}(Y < 800) \approx 0.348$.

Exercise 6.1.6 If $M(\pm h) < \infty$, Markov gives $\mathbb{P}(X > x) \leq M(h)e^{-hx}$ and similarly for the left tail. The lognormal tail $\mathbb{P}(Y > y)$ decays like $\exp\{-(\log y)^2/(2s^2)\}$, slower than any exponential, so $\mathbb{E}e^{tY} = \infty$ for every $t > 0$. The lognormal is *not* determined by its moments: Heyde (1963) showed that the densities $f(y)\{1 + \epsilon \sin(2\pi \log y)\}$, $|\epsilon| \leq 1$, have the same moments.

Exercise 6.1.7 Both schemes cost 60 per household per year. With $u(c) = -1/c$: fixed transfer, incomes $(1060, 460)$, CE $= 840.7$; drought payment, incomes $(1000, 700)$, CE $= 921.1$. The drought-contingent scheme is far better because it transfers income to the state where marginal utility is high (it is insurance). The comparison assumes households cannot smooth consumption themselves through credit or savings; with perfect smoothing the two schemes would be close to equivalent.

Exercise 6.2.1 $\operatorname{Var} \bar{X}_n = n^{-2} \sum_{i,j} \operatorname{Cov}(X_i, X_j) = n^{-2}\{n\sigma^2 + n(n-1)\rho\sigma^2\}$. As $n \to \infty$ it tends to $\rho\sigma^2 > 0$: averaging cannot remove a common component. For clusters of size b with intra-cluster correlation ρ the variance is inflated by the design effect $1 + (b - 1)\rho$.

Exercise 6.2.2 $\mathbf{R} = \mathbf{D}^{-1/2}\boldsymbol{\Sigma}\mathbf{D}^{-1/2}$ is a congruence of a PSD matrix, so $\mathbf{w}^\top \mathbf{R} \mathbf{w} = \mathbf{v}^\top \boldsymbol{\Sigma} \mathbf{v} \geq 0$ with $\mathbf{v} = \mathbf{D}^{-1/2}\mathbf{w}$. The 3×3 equicorrelation matrix has eigenvalues $1 + 2\rho$ and $1 - \rho$ (twice), so it is valid iff $-\frac{1}{2} \leq \rho \leq 1$.

Exercise 6.2.3 $\mathbb{P}(Y \leq y) = \frac{1}{2}\mathbb{P}(X \leq y) + \frac{1}{2}\mathbb{P}(-X \leq y) = \Phi(y)$. $\mathbb{E}[XY] = \mathbb{E}[S]\mathbb{E}[X^2] = 0$. If (X, Y) were bivariate normal then $X + Y$ would be normal, but $X + Y = 0$ with probability $\frac{1}{2}$.

Exercise 6.2.6 For the Simpson simulation the between-group term is strongly negative (the region with high mean training has low mean earnings) and dominates the positive within-group term. For the collider: with independent scores A, B and admission $A + B > c$, the correlation of A and B among admitted applicants is clearly negative (about -0.47 when c is the median of $A + B$ and -0.61 when it is the upper quartile).

Exercise 6.3.1 $\mathbb{E}[S \mid N] = N\mu$ and $\operatorname{Var}(S \mid N) = N\sigma^2$. Hence $\mathbb{E}S = \nu\mu$ and $\operatorname{Var} S = \mathbb{E}[N\sigma^2] + \operatorname{Var}(N\mu) = \nu\sigma^2 + \mu^2\tau^2$.

Exercise 6.3.2 From (6.9), $\mathbb{E}\operatorname{Var}(Y \mid X) = \operatorname{Var} Y - \operatorname{Var}\mathbb{E}(Y \mid X) \le \operatorname{Var} Y$. Example: $X \sim$ Bernoulli(0.1), $Y \mid X = 0 \sim \mathcal{N}(0, 1)$, $Y \mid X = 1 \sim \mathcal{N}(0, 25)$; then $\operatorname{Var} Y = 3.4 < 25 = \operatorname{Var}(Y \mid X = 1)$.

Exercise 6.3.6 $\mathbb{E}[Y \mid X] = \mu_Y + \rho(\sigma_Y/\sigma_X)(X - \mu_X)$ and $\operatorname{Var}(Y \mid X) = \sigma_Y^2(1 - \rho^2)$. In standard units the slope is ρ, $|\rho| < 1$: children of tall parents are, on average, less extreme. Districts selected for unusually bad outcomes will on average improve even without a programme, so before–after comparisons overstate impact; a control group of similarly selected districts is needed.

Exercise 6.3.7 Adding ability as a control removes the selection bias. Because the gain $6+2\cdot$ability is linear in ability and the model without an interaction is misspecified, the coefficient on D is a variance-weighted average of stratum effects, which puts most weight on strata where the propensity to enrol is near $\frac{1}{2}$; it is neither the ATE (6.0) nor the ATT (7.4) in general. Adding $D\times$(ability $-$ mean ability) recovers the ATE.

Exercise 6.4.2 For $u \ge 0$, $\mathbb{P}(X - \mu \ge k) \le \mathbb{P}((X - \mu + u)^2 \ge (k + u)^2) \le (\sigma^2 + u^2)/(k + u)^2$. Minimising over u gives $u = \sigma^2/k$ and the bound $\sigma^2/(\sigma^2 + k^2)$.

Exercise 6.4.3 With $Y_i = X_i - \mu$, $\mathbb{E}(\sum Y_i)^4 = n\mathbb{E}Y^4 + 3n(n-1)\sigma^4 \le Cn^2$ (all terms with an unpaired index vanish). Markov with the fourth power gives $\mathbb{P}(|\bar{X}_n - \mu| > \varepsilon) \le C/(n^2\varepsilon^4)$, which is summable; by Borel–Cantelli only finitely many of these events occur, almost surely, for every rational ε, which gives $\bar{X}_n \xrightarrow{\text{a.s.}} \mu$.

Exercise 6.4.6 $\operatorname{Var}\bar{X}_n \approx n^{-1}\sum_h \gamma(h) = \sigma_u^2/\{n(1-\phi)^2\}$ (the long-run variance). Since this tends to zero, Chebyshev again gives the WLLN. Relative to iid data with the same marginal variance $\sigma_u^2/(1-\phi^2)$ the effective sample size is $n(1-\phi)/(1+\phi)$: about 5% of n when $\phi = 0.9$.

Exercise 6.4.7 The standard deviation of the average loss is $150\sqrt{\{1 + (n-1)\rho\}/n}$: for $\rho = 0$, 47.4, 15.0, 1.5 ($n = 10, 100, 10^4$); for $\rho = 0.1$, 65.4, 49.5, 47.5; for $\rho = 0.3$, 91.2, 83.1, 82.2. With $\rho > 0$ it tends to $150\sqrt{\rho}$, not to zero.

Exercise 6.5.2 At $\theta = 0$ the gradient vanishes and (6.16) gives the degenerate limit 0. A second-order expansion gives $n\bar{X}_n^2 = (\sqrt{n}\bar{X}_n)^2 \xrightarrow{d} \sigma^2\chi_1^2$.

Exercise 6.5.3 $g(p) = 2\arcsin\sqrt{p}$ has $g'(p) = 1/\sqrt{p(1-p)}$, so the asymptotic variance is $g'(p)^2 p(1-p) = 1$, independent of p.

Exercise 6.5.5 With $\boldsymbol{\theta} = (\mathbb{E}X, \mathbb{E}X^2)$ and $g(a,b) = \sqrt{b-a^2}/a$ the delta method gives, for exponential data (CV = 1, $\mu_3 = 2$, $\mu_4 = 9$ for central moments with $\sigma = 1$), an asymptotic variance of $\mathrm{CV}^2(\mu_4/\sigma^4 - 1)/4 - \mathrm{CV}^3\mu_3/\sigma^3 + \mathrm{CV}^4 = 2 - 2 + 1 = 1$, i.e. se $\approx 1/\sqrt{n}$. Simulation gives $\sqrt{n}\,\mathrm{sd}(\widehat{\mathrm{CV}}) \approx 0.93$ at $n = 50$ (with a downward bias, mean 0.98) and 0.99 at $n = 500$, close to the delta-method value 1.

Exercise 6.6.1 $K(\lambda) = n\sigma^2\lambda^2/2$ for the sum; $\sup_\lambda\{\lambda n\varepsilon - n\sigma^2\lambda^2/2\} = n\varepsilon^2/(2\sigma^2)$. The exact tail is smaller by a factor of order $1/(\sqrt{n}\varepsilon/\sigma)$ (Mills ratio), so the bound is tight on the log scale.

Exercise 6.6.4 For $p = 0.1$, $\varepsilon = 0.05$, $n = 1000$: Bernstein (with $M = 0.9$) gives 6.8×10^{-6}, compared with Hoeffding's 6.7×10^{-3} and the exact 4.5×10^{-7}: an improvement of three orders of magnitude, because Bernstein uses the small variance $p(1-p) = 0.09$.

Exercise 6.6.6 For a fixed h, Hoeffding gives $\mathbb{P}(|\text{test} - \text{train}| > t) \le 2e^{-2nt^2}$; the union bound over m rules and $2me^{-2nt^2} = \delta$ give the result. With $m = 1000$, $n = 5000$, $\delta = 0.05$ the bound is 0.033: the chosen formula's targeting error could be up to about 3 percentage points worse out of sample than in sample.

Exercise 6.6.7 Each district at level $\delta/m = 0.002$: (a) $z_{0.999} = 3.09$, so $n = 1.5{\times}3.09^2{\times}0.25/0.05^2 \approx 1433$ children; (b) Hoeffding $n = 1.5\log(2m/\delta)/(2\varepsilon^2) \approx 2073$. The CLT figure is the standard, defensible choice for coverage rates away from 0 and 1; Hoeffding guarantees the precision without appeal to asymptotics, at about 45% more cost.

Exercise 6.6.8 $\mathrm{Var}(\hat{p}_A - \hat{p}_B) = \{p_A(1-p_A) + p_B(1-p_B) + 2p_Ap_B\}/n$, giving a standard error of 0.0305 for the lead of 0.03: $z \approx 0.98$, not significant. The margin of error for the lead (about ±6 points) is roughly twice the reported ±3.1.

Chapter 07: Estimation Principles: Moments, Maximum Likelihood and Information Geometry

Exercise 7.1.1 Write $\hat{\boldsymbol{\theta}} - \boldsymbol{\theta} = (\hat{\boldsymbol{\theta}} - \boldsymbol{m}) + \mathbf{b}$ with $\boldsymbol{m} = \mathbb{E}\hat{\boldsymbol{\theta}}$. Expanding the outer product, the cross terms $\mathbb{E}[(\hat{\boldsymbol{\theta}} - \boldsymbol{m})]\mathbf{b}^\top$ and its transpose vanish, leaving $\mathrm{Var}(\hat{\boldsymbol{\theta}}) + \mathbf{b}\mathbf{b}^\top$. Taking traces, $\operatorname{tr}\mathbb{E}[(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta})^\top] = \mathbb{E}\left\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}\right\|^2$.

Exercise 7.1.2 With $m = n - 1$, $\mathrm{MSE}(d) = \sigma^4\{2m + (m-d)^2\}/d^2$. Differentiating, the derivative in d is proportional to $-(m-d)d - 2m - (m-d)^2 = -m(m-d+2)$, which is negative for $d < m+2$ and positive afterwards, so the minimum is at $d = m + 2 = n + 1$. Admissibility among *all* estimators cannot be settled by this calculation: it only compares estimators of the form SS/d (in fact $SS/(n+1)$ is itself inadmissible when μ is unknown, by a result of Stein, because one can improve on it using $\bar{X}$).

Exercise 7.1.3 $\mathrm{Var}(2\bar{X}) = 4\theta^2/(12n) = \theta^2/(3n)$. The maximum has density nx^{n-1}/θ^n, so $\mathbb{E}\hat{\theta}_2 = n\theta/(n+1)$ and $\mathbb{E}\hat{\theta}_2^2 = n\theta^2/(n+2)$, giving $\mathrm{MSE} = 2\theta^2/\{(n+1)(n+2)\}$. The maximum converges at rate n rather than $\sqrt{n}$, and it is not asymptotically normal: $n(\theta - \hat{\theta}_2)/\theta \xrightarrow{d} \mathrm{Exp}(1)$.

Exercise 7.1.4 For t_5 data the excess kurtosis is $\kappa = 6$, so $\mathrm{Var}(SS)$ is much larger than in the normal case and the MSE-optimal divisor is $d^* = \mathbb{E}[SS^2]/(\sigma^2\mathbb{E}SS) = (n-1)\{1 + 2/(n-1) + \kappa/n\} \approx 26.7$ for $n = 20$. A simulation with 10^5 replications gives MSE 5.99 for $d = 19$, 5.04 for $d = 21$ and 4.34 for $d = 26$: heavy tails favour more shrinkage.

Exercise 7.1.5 With $R = 20\,000$ replications the coverage of $\bar{X} \pm 1.96 S/\sqrt{n}$ is about 0.848 ($n = 20$), 0.913 ($n = 100$) and 0.940 ($n = 500$) (Monte Carlo s.e. ≈ 0.002). Skewness makes the interval undercover badly in small samples; intervals for mean consumption should use larger samples, transformations, or the bootstrap (Chapter 11).

Exercise 7.2.1 Use the delta method on $g(m, v) = m^2/v$ with gradient $(2m/v, -m^2/v^2)$ and, for $\vartheta = 1$, $\operatorname{Var} X = k$, $\mu_3 = 2k$, $\mu_4 - \sigma^4 = 2k^2 + 6k$. The asymptotic variance of $\sqrt{n}(\tilde{k} - k)$ is $4k - 2 \cdot 2 \cdot 2k + 2k^2 + 6k = 2k(k+1)$, independent of ϑ. For $k = 2.5$, $n = 300$ this gives a standard deviation of 0.242 (simulation: 0.240).

Exercise 7.2.2 $Q(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{Z} W \mathbf{Z}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})/n^2$ is a convex quadratic; setting its gradient $-2\mathbf{X}^\top \mathbf{Z} W \mathbf{Z}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})/n^2$ to zero gives (7.3). If $L = p$, $\mathbf{X}^\top \mathbf{Z}$ and W are invertible and the expression reduces to $(\mathbf{Z}^\top \mathbf{X})^{-1} \mathbf{Z}^\top \mathbf{y}$. With $W = (\mathbf{Z}^\top \mathbf{Z})^{-1}$, $\mathbf{X}^\top \mathbf{Z} (\mathbf{Z}^\top \mathbf{Z})^{-1} \mathbf{Z}^\top = \hat{\mathbf{X}}^\top$ and $\hat{\mathbf{X}}^\top \mathbf{X} = \hat{\mathbf{X}}^\top \hat{\mathbf{X}}$ (the projection is idempotent), giving $(\hat{\mathbf{X}}^\top \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^\top \mathbf{y}$.

Exercise 7.2.3 $\operatorname{Var} s = \pi^2 + 2$ and $\operatorname{Cov}(s, \gamma a + u) = \gamma$, so $\operatorname{plim} \hat{\beta}_{OLS} = \beta + \gamma/(\pi^2 + 2)$, while $\operatorname{Cov}(z, \gamma a + u) = 0$ gives $\operatorname{plim} \hat{\beta}_{IV} = \beta$. With $\beta = 0.08$, $\gamma = 0.25$, $\pi = 1$ the OLS limit is 0.163, matching the simulated median 0.1635.

Exercise 7.2.6 Hint: the moment conditions are $\mathbb{E}[z_d u_{id}] = 0$ where z_d is the roll-out timing. Exclusion requires that timing affects outcomes *only* through attendance; it fails if early districts were chosen for their better administration, or if the meals improve nutrition and hence outcomes directly. With one instrument the model is just identified and no J-test is available; even with several instruments, the J-test cannot detect a violation that biases all of them in the same direction.

Exercise 7.3.1 See Example 7.16. For Uniform$(0, \theta)$, $L(\theta) = \theta^{-n} \mathbf{1}\{\theta \geq \max x_i\}$ is decreasing on $[\max x_i, \infty)$, so $\hat{\theta} = \max x_i$. The likelihood is not differentiable at its maximum (the support depends on θ), so the score equation has no solution.

Exercise 7.3.2 For the logit, $-\nabla^2 \ell = \mathbf{X}^\top \operatorname{diag}\{p_i(1 - p_i)\} \mathbf{X} \geq 0$; for the Poisson, $-\nabla^2 \ell = \mathbf{X}^\top \operatorname{diag}\{\mu_i\} \mathbf{X} \geq 0$ (offsets do not enter the Hessian). With an intercept, the first score equation is $\sum_i (y_i - \hat{p}_i) = 0$.

Exercise 7.3.4 With $R = 2000$, $n = 300$: Monte Carlo s.d. of $\tilde{k}$ (MoM) 0.240, of $\hat{k}$ (ML) 0.192; ARE of MoM relative to ML ≈ 0.64. Theory: $\operatorname{avar}(\hat{k}) = k/\{k\psi'(k) - 1\}$, giving s.d. 0.192, and $2k(k+1)$ for MoM (Exercise 7.2.1), giving 0.242.

Exercise 7.3.6 The fitted total equals the observed total, 1 573 775 visits (a property of Poisson ML with an intercept). Setting the fee to zero in every district and using the fitted coefficients gives about 2.19 million visits, an increase of about 39%. This requires that the fee coefficient is causal, that no supply constraints bind (clinics can absorb the extra demand) and that the log-linear form extrapolates to fee = 0.

Exercise 7.4.1 The scores are $(x - p)/\{p(1-p)\}$, $x/\lambda - 1$, $1/\lambda - x$, $\big((x-\mu)/\sigma^2,\ -1/\sigma + (x-\mu)^2/\sigma^3\big)$ and $2(x - \theta)/\{1 + (x - \theta)^2\}$; their variances give the stated informations. For (μ, σ^2) the information is $\operatorname{diag}(1/\sigma^2, 1/(2\sigma^4))$; with $\sigma^2 = g(\sigma)$, $\mathbf{J} = \operatorname{diag}(1, 2\sigma)$ and $\mathbf{J}^\top \operatorname{diag}(1/\sigma^2, 1/(2\sigma^4)) \mathbf{J} = \operatorname{diag}(1/\sigma^2, 2/\sigma^2)$.

Exercise 7.4.2 The score $-n/\theta$ is non-random with non-zero mean, so (R2) fails (the support depends on θ) and differentiation under the integral is not allowed. $\mathrm{Var}\{(n+1)\max X_i/n\} = \theta^2/\{n(n+2)\} = O(n^{-2})$.

Exercise 7.4.3 Equality in Cauchy–Schwarz requires $U = c(\theta)(T-\theta)$ almost surely; taking covariances with T gives $c(\theta) = nI(\theta)$. Integrating $\partial\ell_n/\partial\theta = nI(\theta)(T-\theta)$ in θ gives $\ell_n = A(\theta)T + B(\theta) + C(x)$, the exponential-family form.

Exercise 7.4.4 Exact rejection probabilities (computed from the binomial distribution, treating $\hat{p} = 0$ as a Wald rejection) for $H_0 : p = 0.1$: $n = 20$: Wald 0.124, LR 0.133, score 0.043; $n = 50$: 0.121, 0.058, 0.030; $n = 200$: 0.073, 0.059, 0.044. The rates jump irregularly with n because $\sum y_i$ takes few values near the critical region; the Wald test is worst because $\hat{p}(1-\hat{p})$ is very variable when p is small.

Exercise 7.4.6 If the conditional variance is roughly $\phi\mu$ with $\phi \approx 4$, the sandwich standard error is about $\sqrt{\phi} \approx 2$ times the model-based one, i.e. about 0.08 (the marginal variance overstates ϕ somewhat, because part of it is explained by covariates). A coefficient that was "significant" with $t = 2.5$ would have $t \approx 1.25$: the evidence for the subsidy effect is much weaker than reported.

Exercise 7.5.1 $\mathrm{KL}(\mathrm{Poi}(\lambda_1)\|\mathrm{Poi}(\lambda_2)) = \lambda_1\log(\lambda_1/\lambda_2) + \lambda_2 - \lambda_1$ and, for exponential rates, $\log(\lambda_1/\lambda_2) + \lambda_2/\lambda_1 - 1$. Second-order expansions in $\delta = \lambda_2 - \lambda_1$ give $\delta^2/(2\lambda_1)$ and $\delta^2/(2\lambda_1^2)$, i.e. $\frac{1}{2}\delta^2 I(\lambda_1)$.

Exercise 7.5.2 $s_\eta = \mathbf{J}^\top s_\theta$ by the chain rule, hence $I_\eta = \mathbf{J}^\top I_\theta \mathbf{J}$. For the log-odds, $\ell = x\eta - \log(1+e^\eta)$, so $I(\eta) = p(1-p)$; equivalently $(dp/d\eta)^2 I(p) = p^2(1-p)^2/\{p(1-p)\}$.

Exercise 7.5.3 With $p = \sin^2\phi$, $dp/d\phi = 2\sin\phi\cos\phi$ and $I_\phi = (dp/d\phi)^2/\{p(1-p)\} = 4$, constant. The geodesic distance is $\int \sqrt{I_\phi}\, d\phi = 2\left|\phi_1 - \phi_2\right|$. Consequently $\arcsin\sqrt{\hat{p}}$ has approximate variance $1/(4n)$ whatever p is: the classical variance-stabilising transform.

Exercise 7.5.5 $s = T - \nabla A$, so $\mathbb{E}T = \nabla A$ and $I = \mathrm{Var}\,T = \nabla^2 A$. $\mathrm{KL} = \mathbb{E}_{\theta_1}[(\theta_1-\theta_2)^\top T] - A(\theta_1) + A(\theta_2)$, which equals the Bregman divergence after substituting $\mathbb{E}_{\theta_1}T = \nabla A(\theta_1)$. The score equation $\sum_i\{T(x_i) - \nabla A(\theta)\} = 0$ gives $\nabla A(\hat{\theta}) = \bar{T}$.

Exercise 7.6.1 Posterior mean $\frac{a+b}{a+b+n}\cdot\frac{a}{a+b} + \frac{n}{a+b+n}\cdot\bar{y}$; data weight $n/(a+b+n)$. In the pilot, $40/50 = 0.8$, giving $0.2\cdot 0.2 + 0.8\cdot 0.35 = 0.32$, as in the output.

Exercise 7.6.2 The exponent is $-\frac{1}{2\sigma^2}\{\boldsymbol{\beta}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\boldsymbol{\beta} - 2\boldsymbol{\beta}^\top\mathbf{X}^\top\mathbf{y}\}$+const; completing the square gives mean $(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$ and covariance $\sigma^2(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}$. A Laplace (double-exponential) prior gives the lasso.

Exercise 7.6.3 The posterior is Gamma(173, rate 14) ($\sum y = 153$); the predictive is negative binomial with size 173 and success probability 14/15, with mean 12.36. $\mathbb{P}(Y > 15 \mid \text{data}) \approx 0.19$.

Exercise 7.6.5 $\mathbb{P}(p > 0.3 \mid y = 14, n = 40)$ is 0.61 under Beta(2, 8), 0.78 under Beta(1, 1), 0.76 under Beta($\frac{1}{2}, \frac{1}{2}$) and only 0.06 under the strong Beta(20, 80) prior. Report the results for a range of priors, state the prior's effective sample size, and explain which conclusions are robust.

Exercise 7.6.6 The Beta(2, 8) posterior is Beta(16, 34) and $\mathbb{P}(p > 0.3 \mid \text{data}) \approx 0.61$. The decision should also weigh the costs of the two possible errors, the expected net benefit (which uses the whole posterior, not just one tail probability) and the value of collecting more data before scaling up.

Chapter 8: Optimisation: Convexity, Gradients, Hessians, Newton and Stochastic Gradient Methods

Exercise 8.1.1 $\frac{d}{dm}\sum_i (y_i - m)^2 = -2\sum_i (y_i - m) = 0$ gives $m = \bar{y}$, and the second derivative $2n > 0$. For $g(m) = \sum_i |y_i - m|$, g is convex and piecewise linear with right derivative $\#\{i : y_i \le m\} - \#\{i : y_i > m\}$; it changes sign where half the observations lie on each side, i.e. at a median. For even n with ordered values $y_{(1)} \le \cdots \le y_{(n)}$ the set of minimisers is the whole interval $[y_{(n/2)}, y_{(n/2+1)}]$.

Exercise 8.1.2 For $|r| < c$, $\rho_c'(r) = r$; for $|r| > c$, $\rho_c'(r) = c\,\operatorname{sign}(r)$; at $r = \pm c$ both one-sided derivatives equal $\pm c$, so $\rho_c' = \psi_c$ is continuous. ψ_c' jumps from 1 to 0 at $|r| = c$, so $\rho_c \notin C^2$. Gaussian: $-\log f = \frac{(y - \mathbf{x}^\top \boldsymbol{\beta})^2}{2\sigma^2} + \text{const}$, Laplace: $-\log f = |y - \mathbf{x}^\top \boldsymbol{\beta}|/b + \text{const}$, so the minimisers coincide with OLS and LAD.

Exercise 8.1.3 $f(x) = e^x$ on $\mathbb{R}$ (bounded below by 0, no minimiser: the domain is not compact; f is not coercive). $f(x) = x$ on $(0, 1)$: bounded, but the set is not closed, so the infimum 0 is not attained.

Exercise 8.1.5 Running BFGS from a fine grid of 4001 starts on $[-6, 6]$, about 16% of starts reach the global minimiser $x^\star \approx -0.579$ (its basin of attraction has width about 1.9). With independent uniform starts, $1 - (1 - 0.162)^m \ge 0.95$ requires $m \ge 17$.

Exercise 8.1.6 Within villages with a health worker every outcome is 1, so the log-likelihood increases monotonically as the coefficient on the indicator tends to $+\infty$ (quasi-complete separation): the likelihood is not coercive and the MLE does not exist. Software reports a very large coefficient and standard error, or a convergence warning. Remedies: a ridge or Firth-type penalised likelihood (which restores coercivity), exact conditional logistic regression, or reporting the raw take-up rates for that group and estimating the model on the remaining variation.

Exercise 8.1.7 For each sample, $R_n(\boldsymbol{\theta})$ is a finite sum of continuous functions of $\boldsymbol{\theta}$, hence continuous on the compact set Θ, so by Weierstrass it attains its minimum. Compactness also makes uniform laws of large numbers ($\sup_\Theta |R_n - R| \xrightarrow{p} 0$) available and prevents the minimiser from escaping to infinity; combined with identification (a unique, well-separated minimiser of R) this yields consistency.

Exercise 8.2.1 With $z_i = \mathbf{x}_i^\top \boldsymbol{\beta}$, $\frac{\partial}{\partial \boldsymbol{\beta}} \log(1 + e^{z_i}) = \sigma(z_i)\mathbf{x}_i$ and $\frac{\partial}{\partial \boldsymbol{\beta}} y_i z_i = y_i \mathbf{x}_i$, giving $\nabla f = \frac{1}{n}\sum_i (p_i - y_i)\mathbf{x}_i$. Differentiating again with $\sigma' = \sigma(1 - \sigma)$ gives $\frac{1}{n}\sum_i p_i(1 - p_i)\mathbf{x}_i\mathbf{x}_i^\top$. For Poisson, $\nabla f = \frac{1}{n}\sum_i (e^{z_i} - y_i)\mathbf{x}_i$ and $\nabla^2 f = \frac{1}{n}\sum_i e^{z_i}\mathbf{x}_i\mathbf{x}_i^\top$.

Exercise 8.2.3 $f = y^2 - 3x^2 y + 2x^4$. Along $y = mx$: $f = m^2x^2 - 3mx^3 + 2x^4$, which has a strict local minimum at 0 for $m \ne 0$; along $x = 0$, $f = y^2$; along $y = 0$, $f = 2x^4$. But on the parabola $y = \frac{3}{2}x^2$, $f = -\frac{1}{4}x^4 < 0$. The Hessian at the origin is $\operatorname{diag}(0, 2)$, positive semidefinite but singular, so only the (inconclusive) necessary condition of Theorem 8.14(a) holds.

Exercise 8.2.4 Central differences of the analytic gradient give relative Hessian errors of about 8×10^{-8} ($h=10^{-3}$), 8×10^{-10} (10^{-4}), 7×10^{-11} (10^{-5}, best), 4×10^{-10} (10^{-6}) and 4×10^{-8} (10^{-8}). The optimum is again near $\varepsilon_{\text{mach}}^{1/3}$, as for gradients, because the same central formula is applied to a function (the gradient) computed with relative error $\approx \varepsilon_{\text{mach}}$.

Exercise 8.2.6 $\nabla f = (4x^3-4y,\, 4y^3-4x)$ gives $y=x^3$ and $x^9=x$: $x\in\{0,1,-1\}$. Stationary points $(0,0)$, $(1,1)$, $(-1,-1)$. $\nabla^2 f = \left(\begin{smallmatrix} 12x^2 & -4 \\ -4 & 12y^2 \end{smallmatrix}\right)$: at the origin eigenvalues ±4 (saddle); at $(\pm1,\pm1)$ eigenvalues 8 and 16 (strict minima, $f=-2$, also global since f is coercive).

Exercise 8.2.7 GD gives $\mathbf{x}_k = (\mathbf{I}-\alpha\mathbf{A})^k\mathbf{x}_0 = \sum_i (1-\alpha\lambda_i)^k c_i\mathbf{u}_i$. For a negative eigenvalue $\lambda_i<0$, $|1-\alpha\lambda_i|>1$, so $\mathbf{x}_k\to\mathbf{0}$ requires $c_i=0$ for all such i: $\mathbf{x}_0$ must lie in the span of the non-negative eigenvectors, a proper subspace, which has Lebesgue (and Gaussian) measure zero.

Exercise 8.3.2 (i) Add the inequalities (8.8) for each f_j with weights $w_j\ge0$. (ii) $\mathbf{A}(\theta\mathbf{x}+(1-\theta)\mathbf{y})+\mathbf{b} = \theta(\mathbf{A}\mathbf{x}+\mathbf{b})+(1-\theta)(\mathbf{A}\mathbf{y}+\mathbf{b})$. (iii) $f_\alpha(\theta\mathbf{x}+(1-\theta)\mathbf{y}) \le \theta f_\alpha(\mathbf{x})+(1-\theta)f_\alpha(\mathbf{y}) \le \theta\sup f+(1-\theta)\sup f$; take the sup on the left. (iv) $h(g(\theta\mathbf{x}+(1-\theta)\mathbf{y})) \le h(\theta g(\mathbf{x})+(1-\theta)g(\mathbf{y})) \le \theta h(g(\mathbf{x}))+(1-\theta)h(g(\mathbf{y}))$, using monotonicity then convexity of h. The hinge loss is a maximum of two affine functions; Huber is C^1 with non-decreasing derivative ψ_c; softplus has second derivative $\sigma(x)(1-\sigma(x))>0$.

Exercise 8.3.3 $g = f - \frac{\mu}{2}\|\cdot\|^2$ satisfies $g(\mathbf{y})-g(\mathbf{x})-\nabla g(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) = f(\mathbf{y})-f(\mathbf{x})-\nabla f(\mathbf{x})^\top(\mathbf{y}-\mathbf{x}) - \frac{\mu}{2}\|\mathbf{y}-\mathbf{x}\|^2$ (expand $\|\mathbf{y}\|^2-\|\mathbf{x}\|^2-2\mathbf{x}^\top(\mathbf{y}-\mathbf{x})$), so (8.10) for f is the first-order condition for g. Coercivity: with $\mathbf{x}=\mathbf{0}$, $f(\mathbf{y}) \ge f(\mathbf{0}) - \|\nabla f(\mathbf{0})\|\,\|\mathbf{y}\| + \frac{\mu}{2}\|\mathbf{y}\|^2 \to \infty$. Existence by Corollary 8.6, uniqueness by strict convexity.

Exercise 8.3.5 With the second covariate multiplied by s, $\kappa\approx1.2$ ($s=1$), 107 ($s=10$), 1.07×10^4 ($s=100$): κ grows like s^2, because the largest eigenvalue of $\mathbf{X}^\top\mathbf{X}/n$ is $\approx s^2$ while the smallest stays ≈1. A ridge penalty adds λ to both, so $\kappa=(L+\lambda)/(\mu+\lambda)$ falls quickly once λ is comparable to μ.

Exercise 8.3.7 Lagrangian $\sum_d a_d/(1+n_d)+\lambda(\sum_d n_d - N)$: first-order conditions $a_d/(1+n_d)^2=\lambda$, so $1+n_d=\sqrt{a_d/\lambda}\propto\sqrt{a_d}$, with λ fixed by the budget. The objective is convex, so this is the global optimum (ignoring integrality and $n_d\ge0$). A district with twice the need receives $\sqrt{2}\approx1.41$ times as many workers (plus one): diminishing returns imply that allocation should be less than proportional to need.

Exercise 8.3.8 With $\phi=\Phi'$, $(\log\Phi)'' = \frac{-z\phi\Phi-\phi^2}{\Phi^2} = -\frac{\phi(z)}{\Phi(z)^2}\big(z\Phi(z)+\phi(z)\big)$, and $z\Phi(z)+\phi(z) = \int_{-\infty}^{z}\Phi(t)\,dt>0$. Hence $\log\Phi$ is strictly concave; so is $\log(1-\Phi(z)) = \log\Phi(-z)$. The probit log-likelihood $\sum_i[y_i\log\Phi(\mathbf{x}_i^\top\boldsymbol{\beta})+(1-y_i)\log\Phi(-\mathbf{x}_i^\top\boldsymbol{\beta})]$ is a sum of concave functions of affine maps of $\boldsymbol{\beta}$, hence concave.

Exercise 8.4.1 From the proof of Theorem 8.31, $\|\mathbf{x}_{k+1}-\mathbf{x}^\star\|^2 = \|\mathbf{x}_k-\mathbf{x}^\star\|^2 - \frac{2}{L}\big[\mathbf{g}_k^\top(\mathbf{x}_k-\mathbf{x}^\star) - \frac{1}{2L}\|\mathbf{g}_k\|^2\big]$, and the bracket is $\ge f(\mathbf{x}_{k+1})-f^\star\ge0$. Hence the distance to $\mathbf{x}^\star$ is non-increasing and all iterates lie in the ball of radius $\|\mathbf{x}_0-\mathbf{x}^\star\|$ around $\mathbf{x}^\star$.

Exercise 8.4.2 By strong convexity at $\mathbf{x}^\star$ (where $\nabla f=\mathbf{0}$), $\frac{\mu}{2}\|\mathbf{x}_k-\mathbf{x}^\star\|^2 \le f(\mathbf{x}_k)-f^\star$; by the descent lemma $f(\mathbf{x}_0)-f^\star\le\frac{L}{2}\|\mathbf{x}_0-\mathbf{x}^\star\|^2$. Combine with (8.15).

Exercise 8.4.3 For a quadratic the exact step is $\alpha_k = \mathbf{g}_k^\top \mathbf{g}_k / \mathbf{g}_k^\top \mathbf{A}\mathbf{g}_k$. At $\mathbf{x}_0 = (\kappa, 1)$, $\mathbf{g}_0 = (\kappa, \kappa)$ and $\alpha_0 = 2\kappa^2/(\kappa^2 + \kappa^3) = 2/(1+\kappa)$, so $\mathbf{x}_1 = \frac{\kappa-1}{\kappa+1}(\kappa, -1)$. The point $(\kappa, -1)$ is the mirror image of $(\kappa, 1)$, so the same computation repeats with alternating sign. The error contracts by exactly $(\kappa-1)/(\kappa+1)$ per step: exact line search is no better than the optimal constant step from this (worst-case) start.

Exercise 8.4.4 For $\kappa = 50$ from $\mathbf{x}_0 = (50, 1)$ and tolerance 10^{-6}: optimal constant step 444 iterations; exact line search also 444 (see 8.4.3); heavy ball 73; Nesterov 136. The accelerated methods' counts scale like $\sqrt{\kappa} \approx 7$ times smaller.

Exercise 8.4.5 With $\alpha = 1.9/L$ GD still converges ($q = \max\{|1 - 1.9/\kappa|, 0.9\} < 1$) but the steep component oscillates in sign; with $2.05/L$, $|1 - \alpha L| = 1.05 > 1$ and the iterates diverge geometrically along the top eigenvector. Centring only gives $\kappa \approx 26$, scaling only gives $\kappa \approx 180$, both gives 1.1 (raw: 1755): centring removes the near-collinearity between the intercept and the positive-valued covariates and matters more here, but both help.

Exercise 8.4.6 Income in cedis makes $\lambda_{\max}(\mathbf{X}^\top\mathbf{X}/n) \approx 10^8$ while the household-size direction has curvature of order 1-25, so κ is astronomically large. A stable step $\alpha < 2/L$ is then so small that the household-size coefficient barely moves in 10 000 iterations: the algorithm has stalled, not converged. Remedies: standardise (or log-transform) covariates; use a closed-form/QR solution or Newton/L-BFGS; check the gradient norm rather than the change in loss.

Exercise 8.4.7 For each eigenvalue λ the error component obeys $e_{k+1} = (1 + \beta - \alpha\lambda)e_k - \beta e_{k-1}$, with characteristic polynomial $z^2 - (1 + \beta - \alpha\lambda)z + \beta$. Its roots are complex conjugates of modulus $\sqrt{\beta}$ whenever $(1 + \beta - \alpha\lambda)^2 \le 4\beta$, i.e. $(1 - \sqrt{\beta})^2 \le \alpha\lambda \le (1 + \sqrt{\beta})^2$. With the stated α, β one checks $\alpha\mu = (1 - \sqrt{\beta})^2$ and $\alpha L = (1 + \sqrt{\beta})^2$, so every $\lambda \in [\mu, L]$ gives spectral radius $\sqrt{\beta} = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$.

Exercise 8.5.1 $\nabla g(\mathbf{y}) = \mathbf{B}^\top \nabla f(\mathbf{B}\mathbf{y})$ and $\nabla^2 g(\mathbf{y}) = \mathbf{B}^\top \nabla^2 f(\mathbf{B}\mathbf{y})\mathbf{B}$, so the Newton step for g is $-\mathbf{B}^{-1}[\nabla^2 f]^{-1}\mathbf{B}^{-\top}\mathbf{B}^\top \nabla f = \mathbf{B}^{-1}\mathbf{d}_f$; by induction $\mathbf{x}_k = \mathbf{B}\mathbf{y}_k$. The GD step for g is $-\alpha\mathbf{B}^\top\nabla f$, which maps to $-\alpha\mathbf{B}\mathbf{B}^\top\nabla f$ in $\mathbf{x}$ coordinates, a different direction unless $\mathbf{B}\mathbf{B}^\top \propto \mathbf{I}$.

Exercise 8.5.2 $\boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k + (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top(\mathbf{y} - \mathbf{p}) = (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{W}[\mathbf{X}\boldsymbol{\beta}_k + \mathbf{W}^{-1}(\mathbf{y} - \mathbf{p})]$. For Poisson with log link, $\mu_i = e^{\mathbf{x}_i^\top\boldsymbol{\beta}}$, $\mathbf{W} = \operatorname{diag}(\mu_i)$ and $z_i = \mathbf{x}_i^\top\boldsymbol{\beta} + (y_i - \mu_i)/\mu_i$.

Exercise 8.5.3 $\mathbf{H}_{k+1}\mathbf{y}_k = (\mathbf{I} - \rho\mathbf{s}\mathbf{y}^\top)\mathbf{H}(\mathbf{y} - \rho\mathbf{y}\,\mathbf{s}^\top\mathbf{y}) + \rho\mathbf{s}(\mathbf{s}^\top\mathbf{y}) = \mathbf{0} + \mathbf{s}$, since $\rho\,\mathbf{s}^\top\mathbf{y} = 1$. For $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top\mathbf{H}_{k+1}\mathbf{v} = \mathbf{w}^\top\mathbf{H}_k\mathbf{w} + \rho(\mathbf{s}^\top\mathbf{v})^2$ with $\mathbf{w} = \mathbf{v} - \rho(\mathbf{s}^\top\mathbf{v})\mathbf{y}$; both terms are ≥ 0, and if the first is zero then $\mathbf{w} = \mathbf{0}$, so $\mathbf{v} \propto \mathbf{y}$ and $\mathbf{s}^\top\mathbf{v} \neq 0$ (as $\mathbf{s}^\top\mathbf{y} > 0$), making the second positive.

Exercise 8.5.7 $\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}^\star = \boldsymbol{\theta}_k - \boldsymbol{\theta}^\star - (\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{J}^\top\mathbf{r}$. Expanding $\mathbf{J}^\top\mathbf{r}$ around $\boldsymbol{\theta}^\star$ gives $\mathbf{J}^\top\mathbf{r} \approx (\mathbf{J}^\top\mathbf{J} + \mathbf{S})\mathbf{e}_k$ with $\mathbf{S} = \sum_i r_i\nabla^2 r_i$, hence $\mathbf{e}_{k+1} \approx -(\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{S}\mathbf{e}_k + O(\|\mathbf{e}_k\|^2)$. If $\mathbf{r}(\boldsymbol{\theta}^\star) = \mathbf{0}$ then $\mathbf{S} = \mathbf{0}$ at $\boldsymbol{\theta}^\star$ and $\mathbf{S} = O(\|\mathbf{e}_k\|)$ nearby, so convergence is quadratic; otherwise it is linear with rate $\approx \left\|(\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{S}\right\|$, and fails if this exceeds one.

Exercise 8.6.1 $\mathbb{E}[\nabla f_I(\boldsymbol{\theta})] = \frac{1}{n}\sum_i \nabla f_i = \nabla f$ for I uniform; with B independent draws the minibatch mean has covariance $\frac{1}{B}\operatorname{Cov}(\nabla f_I)$. Without replacement the covariance is multiplied by the finite-population correction $\frac{n-B}{n-1}$, which is zero when $B = n$.

Exercise 8.6.2 c: fails $\sum \alpha_k^2 < \infty$; $c/\sqrt{k+1}$: $\sum \alpha_k = \infty$ but $\sum \alpha_k^2 = \infty$ (fails the second); $c/(k+1)$: satisfies both; $c/(k+1)^2$: $\sum \alpha_k < \infty$ (fails the first). For $f = \frac{1}{2}x^2$ with exact gradients, $x_{k+1} = (1-\alpha_k)x_k$ and $x_k = x_0 \prod_j (1-\alpha_j)$, which converges to a non-zero limit when $\sum \alpha_j < \infty$ and $\alpha_j < 1$: the steps are too short to reach the optimum.

Exercise 8.6.3 With $\mathbf{m}_0 = \mathbf{0}$ and a constant gradient, $\mathbf{m}_k = (1-\beta_1)\sum_{j<k} \beta_1^j \mathbf{g} = (1-\beta_1^k)\mathbf{g}$, so $\hat{\mathbf{m}} = \mathbf{g}$; likewise $\hat{\mathbf{v}} = \mathbf{g}^2$. The step is $-\alpha \mathbf{g}/(|\mathbf{g}| + \epsilon) \approx -\alpha \operatorname{sign}(\mathbf{g})$: Adam's steps are invariant to rescaling the gradient coordinatewise, which explains its robustness in Listing 8.13.

Exercise 8.6.6 Key points: standardise covariates; a few epochs of minibatch SGD or Adam with a decaying schedule (or L-BFGS on a large subsample) to get close; warm-start from last month's coefficients; finish with Newton/L-BFGS on the full data in chunks (accumulating $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})$ and $\mathbf{X}^\top \mathbf{W} \mathbf{X}$, which needs only $O(p^2)$ memory); stop on the full-data gradient norm; report MLE standard errors from the full-data Hessian (or a bootstrap); monitor for separation and data drift between months.

Exercise 8.6.7 SVRG's estimate has variance $\mathbb{E}\left\|\nabla f_i(\boldsymbol{\theta}_k) - \nabla f_i(\tilde{\boldsymbol{\theta}})\right\|^2 \le L^2 \left\|\boldsymbol{\theta}_k - \tilde{\boldsymbol{\theta}}\right\|^2 \to 0$ as both approach $\boldsymbol{\theta}^\star$, so σ^2 in (8.26) is replaced by a quantity proportional to the current error and the noise floor disappears; empirically the log excess loss decreases linearly in the number of epochs with a constant step of order $1/(10L)$.

Chapter 9: Constrained Optimisation and Lagrange Multipliers in Policy Problems

Exercise 9.1.1 For feasible $\mathbf{x}$, $\mathbf{A}(\mathbf{x} - \mathbf{x}^*) = \mathbf{0}$. Concavity gives $f(\mathbf{x}) \le f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^\top (\mathbf{x} - \mathbf{x}^*) = f(\mathbf{x}^*) + \boldsymbol{\lambda}^{*\top} \mathbf{A}(\mathbf{x} - \mathbf{x}^*) = f(\mathbf{x}^*)$, using $\nabla f(\mathbf{x}^*) = \mathbf{A}^\top \boldsymbol{\lambda}^*$.

Exercise 9.1.2 The conditions $y = 2\lambda x$, $x = 2\lambda y$, $x^2 + y^2 = 2$ give $x^2 = y^2 = 1$. Critical points $(1,1), (-1,-1)$ with $\lambda = \frac{1}{2}$ and $f = 1$ (maxima), and $(1,-1), (-1,1)$ with $\lambda = -\frac{1}{2}$ and $f = -1$ (minima). The gradient $(2x, 2y)$ of the constraint vanishes only at the origin, which is not on the circle, so the constraint qualification holds everywhere; the circle is compact, so the best critical points are global.

Exercise 9.1.3 From $x_i/x_k = (a_i p_k/(a_k p_i))^\sigma$ and the budget, $x_i = m\, a_i^\sigma p_i^{-\sigma} / \sum_k a_k^\sigma p_k^{1-\sigma}$. Taking logs, $\partial \log x_i / \partial \log p_i = -\sigma - (1-\sigma) a_i^\sigma p_i^{1-\sigma} / \sum_k a_k^\sigma p_k^{1-\sigma} = -\sigma + (\sigma - 1) s_i$. As $\sigma \to 0$ the elasticity tends to $-s_i$ (Leontief: only the income effect remains); as $\sigma \to \infty$ it diverges (perfect substitutes).

Exercise 9.1.5 The Lagrangian $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} - \lambda(\mathbf{1}^\top \mathbf{w} - 1)$ gives $2\boldsymbol{\Sigma}\mathbf{w} = \lambda \mathbf{1}$, so $\mathbf{w} \propto \boldsymbol{\Sigma}^{-1}\mathbf{1}$, normalised to sum to one. The multiplier is $\lambda = 2/(\mathbf{1}^\top \boldsymbol{\Sigma}^{-1} \mathbf{1}) = 2\sigma_{\min}^2$, twice the minimum variance, consistent with $dV/dc = \lambda$ for $V(c) = c^2 \sigma_{\min}^2$ at $c = 1$.

Exercise 9.1.6 The Lagrange condition $2\mathbf{A}\mathbf{x} = 2\lambda \mathbf{x}$ says $\mathbf{x}$ is an eigenvector with eigenvalue λ, and $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x} = \lambda$ on the sphere. The maximum of f on the (compact) sphere is therefore the largest eigenvalue: $\lambda_{\max} = \max_{\|\mathbf{x}\|=1} \mathbf{x}^\top \mathbf{A} \mathbf{x}$.

Exercise 9.1.7 With Cobb–Douglas shares the expenditure on each group is fixed: the quantity of group 1 falls by $1 - 1/1.2 \approx 16.7\%$ and the other two quantities do not change (zero cross-price effects). For a poor household, constant shares and unit price elasticity are questionable; with CES and $\sigma < 1$ the quantity of group 1 falls by less and spending is squeezed from the other groups.

Exercise 9.2.2 With all programmes funded, $x_i = a_i/\lambda - b_i$ and $\sum_i x_i = B$ give $\lambda = \sum a_i/(B + \sum b_i)$, and $V(B) = \sum_i a_i \log\big(a_i(B + \sum_k b_k)/(b_i \sum_k a_k)\big)$, so $V'(B) = \sum_i a_i/(B + \sum_k b_k) = \lambda$.

Exercise 9.2.3 $\nabla f = (y, x) = \lambda(1, 1)$ gives $x = y = 1$, $\lambda = 1$. The bordered Hessian is $\begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$ with determinant $2 > 0$: a maximum. The Hessian of f, $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, has eigenvalues ± 1; on the tangent direction $(1, -1)$ the quadratic form is $-2 < 0$.

Exercise 9.2.5 With $f_i = a_i(1 - e^{-x/b_i})$, $f_i' = (a_i/b_i)e^{-x/b_i}$ and $x_i = \max\{0, b_i \log(a_i/(b_i\lambda))\}$. For $B = 6$: $\mathbf{x} \approx (1.905, 2.999, 1.096)$, $\lambda \approx 0.4465$, and the finite-difference $V'(6)$ agrees to five decimals.

Exercise 9.2.7 With two binding constraints the multipliers are the marginal outcome values of a cedi of wages and a cedi of drugs. A ratio of three suggests moving money from the wage bill to drugs if budget lines can be reallocated; before acting one needs evidence that the estimated marginal returns are reliable, that the reallocation is legally possible, and that complementarity between staff and drugs (a non-separable objective) is not being ignored.

Exercise 9.3.1 Write $g_1 = 4 - x_1 - 2x_2 \le 0$, $g_2 = 1 - x_1 \le 0$. With only g_1 active the minimiser is $(0.8, 1.6)$, which violates g_2; with only g_2 active, $(1, 0)$ violates g_1; with neither active, $\mathbf{0}$ is infeasible. With both active, $\mathbf{x}^* = (1, 1.5)$ and $2\mathbf{x} = \mu_1(1, 2) + \mu_2(1, 0)$ gives $\mu_1 = 1.5$, $\mu_2 = 0.5$, both non-negative; $f^* = 3.25$. Raising the right-hand side 4 by δ raises the minimum by about 1.5δ.

Exercise 9.3.3 KKT: $a_i/(x_i + \gamma_i) \le \lambda p_i$ with equality if $x_i > 0$. Hence $x_i = a_i/(\lambda p_i) - \gamma_i$ if that is positive and $x_i = 0$ otherwise; λ is set by the budget. Good i is bought only if $a_i/(\gamma_i p_i) > \lambda$.

Exercise 9.3.6 Take $\min x_1$ s.t. $-x_1 \le 0$ twice (duplicate constraints): LICQ fails, MFCQ holds (direction $(1, 0)$), and any $\mu_1 + \mu_2 = 1$, $\mu_k \ge 0$ works. With non-unique multipliers the value function has a kink in some directions, and a single reported multiplier does not give the effect of relaxing an individual constraint.

Exercise 9.4.2 The Lagrangian $\mathbf{c}^\top\mathbf{x} + \mathbf{y}^\top(\mathbf{b} - \mathbf{A}\mathbf{x}) - \mathbf{s}^\top\mathbf{x}$ has finite infimum only if $\mathbf{A}^\top\mathbf{y} + \mathbf{s} = \mathbf{c}$, giving the dual $\max \mathbf{b}^\top\mathbf{y}$ s.t. $\mathbf{A}^\top\mathbf{y} \le \mathbf{c}$ with $\mathbf{y}$ free. Writing that dual in minimisation form and dualising again returns the primal.

Exercise 9.4.3 Minimising $\frac{1}{2}\mathbf{x}^\top\mathbf{Q}\mathbf{x} + \mathbf{c}^\top\mathbf{x} + \boldsymbol{\mu}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$ gives $\mathbf{x} = -\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^\top\boldsymbol{\mu})$ and $q(\boldsymbol{\mu}) = -\frac{1}{2}(\mathbf{c} + \mathbf{A}^\top\boldsymbol{\mu})^\top\mathbf{Q}^{-1}(\mathbf{c} + \mathbf{A}^\top\boldsymbol{\mu}) - \mathbf{b}^\top\boldsymbol{\mu}$, a concave quadratic to be maximised over $\boldsymbol{\mu} \ge \mathbf{0}$.

Exercise 9.4.6 The dual function $q(\lambda) = \lambda B + \sum_i (v_i - \lambda c_i)_+$ is also the dual of the LP relaxation $0 \le x_i \le 1$, which has no duality gap, so the bound equals the LP relaxation value. In 20 random knapsacks with $n = 10$ the mean relative gap is about 5%; it falls roughly like $1/n$ as n grows, because only one item is fractional in the LP solution.

Exercise 9.5.2 The KKT conditions of $\min \frac{1}{2}\|\mathbf{x} - \mathbf{z}\|^2$ s.t. $\mathbf{x} \ge \mathbf{0}$, $\mathbf{1}^\top\mathbf{x} \le B$ give $x_i = (z_i - \tau)_+$ with $\tau \ge 0$ the multiplier of the budget; $\tau = 0$ if $\sum_i (z_i)_+ \le B$, otherwise τ solves $\sum_i (z_i - \tau)_+ = B$ (found by sorting).

Exercise 9.5.3 With $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, ridge gives $t(\alpha) = \|\boldsymbol{\beta}_\alpha\|^2 = \sum_j d_j^2(\mathbf{u}_j^\top \mathbf{y})^2/(d_j^2+\alpha)^2$ (with the $1/n$ scaling absorbed into d_j), which is strictly decreasing in α; its inverse $\alpha(t)$ is therefore decreasing in t.

Exercise 9.5.6 At $\mathbf{x}^*(t)$, $t\nabla f + \sum_k \nabla g_k/(-g_k) = \mathbf{0}$, so $\mathbf{x}^*(t)$ minimises $\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}(t))$ with $\mu_k(t) = -1/(t g_k) > 0$. Hence $q(\boldsymbol{\mu}(t)) = f(\mathbf{x}^*(t)) + \sum_k \mu_k g_k = f(\mathbf{x}^*(t)) - p/t$, and weak duality gives $f(\mathbf{x}^*(t)) - p^* \le p/t$.

Exercise 9.6.2 With $\eta = 1$, $\omega_i/(y_i + T_i) = \lambda$ for recipients, so $c_i = \omega_i/\lambda$ is proportional to the weight. As $\eta \to \infty$, $c_i = (\omega_i/\lambda)^{1/\eta}$ becomes equal across recipients: the weights cease to matter and the planner is Rawlsian (maximin).

Exercise 9.6.3 From $x_i = \mu(x_i + t_i x_i')$ and $x_i' = -e_i x_i/q_i$, $1 = \mu(1 - e_i t_i/q_i)$, so $t_i/q_i = (1-1/\mu)/e_i = k/e_i$ and $\mu = 1/(1-k)$. A larger revenue requirement requires a larger k, hence a larger μ.

Exercise 9.6.4 With a floor of GHS 20 in every region (budget GHS 60 per household), the optimal transfers become $(152.5, 105.4, 44.7, 20, 20, 20)$, the poverty gap index rises from 0.0524 to 0.0581, and the shadow price of the budget rises from 0.33 to 0.40. Minimising the squared poverty gap instead gives transfers $(149.3, 113.3, 79.0, 20.7, 0, 0)$ and $P_2 = 0.0151$ (from 0.0807): the solution is less extreme for the poorest region because P_2 rewards reducing deep gaps.

Exercise 9.6.5 With floors $x_i \ge 0.2$ and $B = 1.8$ the allocation becomes $(0.615, 0.2, 0.577, 0.2, 0.208)$, $\lambda = 1.857$, with floor multipliers 0.039 (health) and 0.387 (water); the outcome index falls from 4.163 to 4.113, a cost of about 0.050 outcome units.

Exercise 9.6.6 Since $\mathbb{E}[\sum_i a_i \log(1 + x_i/b_i)] = \sum_i \mathbb{E}[a_i] \log(1 + x_i/b_i)$, the stochastic programme is the nominal problem with a_i replaced by its mean. The average of the Monte Carlo solutions is a different object (the mean of optimal decisions, which is feasible because the feasible set is convex, but is generally not optimal for the expected objective, since the optimal allocation is a non-linear function of $\mathbf{a}$); the decision to implement is the solution of the expected-value problem, possibly with a risk adjustment.

Chapter 10: Numerical Integration and Monte Carlo Simulation

Exercise 10.1.1 $\int_{p_1}^{p_0} (a - bp)\mathrm{d}p = a(p_0 - p_1) - \frac{b}{2}(p_0 - p_1)(p_0 + p_1) = (p_0 - p_1)\big[a - \frac{b}{2}(p_0 + p_1)\big] = (p_0 - p_1)\frac{D(p_0)+D(p_1)}{2}$. With a subsidy $s = p_0 - p_1$ the fiscal cost is $sD(p_1)$, so DWL $= sD(p_1) - \Delta CS = s\big[D(p_1) - \frac{D(p_0)+D(p_1)}{2}\big] = \frac{s}{2}\big(D(p_1) - D(p_0)\big) = \frac{1}{2}bs^2$. In the text, $b = 4$, $s = 2$ gives 8.

Exercise 10.1.2 $\mathbb{P}(y = 1 \mid z) = \mathbb{P}(\varepsilon \le a + bz) = \mathbb{P}(\varepsilon - bz \le a)$, and $\varepsilon - bz \sim \mathcal{N}(-mz, 1 + s_b^2 z^2)$, so the probability is $\Phi\big((a + mz)/\sqrt{1 + s_b^2 z^2}\big)$. As $s_b \to \infty$ the argument tends to 0 and the probability to $\frac{1}{2}$, monotonically in s_b for fixed $a + mz > 0$.

Exercise 10.1.3 $P_1 = \int_0^z (1 - y/z) f(y)\mathrm{d}y = F(z) - z^{-1}\int_0^z y f(y)\mathrm{d}y$. For the lognormal, $F(z) = \Phi(d)$ and the partial mean is $\int_0^z y f(y)\mathrm{d}y = e^{\mu + s^2/2}\Phi(d - s)$ (complete the square in the exponent), which gives the result.

Exercise 10.1.4 With $\mu = 7$, $s = 0.8$, $z = 900$: $P_0 = 0.40245$, $P_1 = 0.15486$, $P_2 = 0.08015$; quad and the closed forms agree to about 10^{-13}.

Exercise 10.1.6 Normalise $p_0 = Q_0 = 1$. Linear demand with elasticity -0.4 has slope $b = 0.4$; a 20% subsidy gives $Q_1 = 1.08$. Fiscal cost $0.2 \times 1.08 = 0.216$; $\Delta CS = 0.2 \times (1 + 1.08)/2 = 0.208$; DWL $= 0.008$. Each unit of fiscal cost yields 0.963 units of consumer surplus (a 3.7% efficiency loss), but the calculation ignores who consumes the staple (the rich may capture most of the subsidy), leakage and smuggling, administrative costs and general-equilibrium price effects, all of which typically favour the cash transfer.

Exercise 10.2.1 On $[-1, 1]$ with $h = 1$, $S(f) = \frac{1}{3}\big(f(-1) + 4f(0) + f(1)\big)$. Exactness for $1, x, x^2, x^3$ is direct $(2, 0, \frac{2}{3}, 0)$. For x^4, $\int = \frac{2}{5}$ and $S = \frac{2}{3}$, error $-\frac{4}{15}$. The theorem gives $-(b-a)h^4 f^{(4)}/180 = -2 \cdot 24/180 = -\frac{4}{15}$, identifying the constant.

Exercise 10.2.3 Symmetry gives $x_{1,2} = \pm c$, $w_1 = w_2 = w$. Exactness for 1 gives $w = 1$; for x^2, $2c^2 = \frac{2}{3}$, so $c = 1/\sqrt{3}$, the zeros of P_2. Odd monomials are integrated exactly by symmetry.

Exercise 10.2.4 The certainty equivalent is $e^{\mu + (1-\gamma)s^2/2}$; ten Gauss–Hermite nodes reproduce it to about 10^{-12} relative error (slightly worse for $\gamma = 4$). Values: $\gamma = 0.5$: 1286.9; $\gamma = 1$: 1096.6; $\gamma = 2$: 796.3; $\gamma = 4$: 419.9, against a mean income of 1510.2. The risk premium (mean minus CE) rises from 223 to 1090: a highly risk-averse household would give up most of its expected income to remove the risk, which is why consumption smoothing and insurance matter so much for welfare.

Exercise 10.2.5 Errors are 1.3×10^{-8} for $h = 1$ and at machine precision for $h = 0.5$ and 0.25. By the Euler–Maclaurin formula all the h^{2k} correction terms involve derivatives at $\pm\infty$, which vanish; the error is in fact $O(e^{-2\pi^2/h^2})$ (Poisson summation).

Exercise 10.2.7 With 40 years and three shocks per year the dimension is 120; even two nodes per dimension would require $2^{120} \approx 10^{36}$ evaluations. Use Monte Carlo or randomised QMC (with antithetic or control variates, e.g. the closed-form value of a simplified lognormal model), or exploit structure (recursion year by year, as in dynamic programming, so that only low-dimensional integrals are needed at each step).

Exercise 10.3.1 $h = 4\mathbf{1}\{\{\}\cdot\}$ is 4 times a Bernoulli$(\pi/4)$, so $\sigma^2 = 16\frac{\pi}{4}(1 - \frac{\pi}{4}) = \pi(4 - \pi) \approx 2.697$. A half-width of 10^{-4} needs $n = 1.96^2\pi(4 - \pi)/10^{-8} \approx 1.04 \times 10^9$ draws.

Exercise 10.3.2 $\mathbb{E}[g(U)^2] = \frac{\pi^2}{4}\int_0^1 \sin^2(\pi u)\mathrm{d}u = \frac{\pi^2}{8}$, and by independence $\mathbb{E}f_2^2 = (\pi^2/8)^d$, $\mathbb{E}f_2 = 1$. The relative s.e. is $\sqrt{((\pi^2/8)^d - 1)/n}$; with $n = 10^6$ it exceeds 0.1 when $(\pi^2/8)^d > 10^4 + 1$, i.e. $d \geq 44$.

Exercise 10.3.5 With seed 803 and $n = 10^4$: $\hat{P}_1 = 0.1555$, MCSE 0.0024, 95% interval $[0.1508, 0.1601]$, which contains the closed-form value 0.15486.

Exercise 10.3.7 Write $\hat{p}_R = p + \delta$ with $\mathbb{E}\delta = 0$, $\mathrm{Var}\,\delta = p(1-p)/R$. Then $\log\hat{p}_R \approx \log p + \delta/p - \delta^2/(2p^2)$, so $\mathbb{E}\log\hat{p}_R - \log p \approx -(1-p)/(2Rp)$. Summed over N observations, the simulated log-likelihood has a bias of order N/R in the score, which after scaling by $1/\sqrt{N}$ is $O(\sqrt{N}/R)$; it vanishes only if $\sqrt{N}/R \to 0$.

Exercise 10.4.1 OLS of h_i on $(1, g_i - \mu_g)$ gives $\hat{\beta} = \widehat{\mathrm{Cov}}(h, g)/\widehat{\mathrm{Var}}(g)$ and $\hat{\alpha} = \bar{h} - \hat{\beta}(\bar{g} - \mu_g)$. With $\hat{c} = -\hat{\beta}$ this is exactly $\hat{I}^{\mathrm{CV}}(\hat{c})$.

Exercise 10.4.2 $\mathbb{E}_q[(hw)^2] = \int_a^\infty \phi(x)^2/\phi(x-\mu)\mathrm{d}x = e^{\mu^2}\Phi(-a-\mu)$ (complete the square). Minimising numerically for $a = 4$ gives $\mu^\star \approx 4.12$, slightly beyond a, with per-draw variance 4.48×10^{-9} versus 4.52×10^{-9} at $\mu = 4$: the choice $\mu = a$ is almost optimal. The per-draw relative s.d. is about 2.1, compared with $\sqrt{1/p} \approx 178$ for plain Monte Carlo.

Exercise 10.4.3 $w^2 q = p^2/q \propto \exp\{-x^2 + (x-\mu)^2/(2s^2)\}$, which is integrable iff the coefficient of x^2, $-1 + 1/(2s^2)$, is negative, i.e. $s^2 > \frac{1}{2}$. In Listing 10.10 the proposal $\mathcal{N}(6, 0.5^2)$ has $s^2 = 0.25$ and infinite weight variance.

Exercise 10.4.7 In stratum $[(k-1)/n, k/n)$, $\mathrm{Var}\, f(U_k) \le \mathbb{E}\big(f(U_k) - f(m_k)\big)^2 \le \|f'\|_\infty^2/(12n^2)$ (with m_k the midpoint). The estimator $\frac{1}{n}\sum_k f(U_k)$ therefore has variance at most $\frac{1}{n^2} \cdot n \cdot \|f'\|_\infty^2/(12n^2) = O(n^{-3})$.

Exercise 10.5.1 Logistic: $X = \log\big(U/(1-U)\big)$; Weibull$(k, \lambda)$: $X = \lambda(-\log(1-U))^{1/k}$; Gumbel: $X = -\log(-\log U)$. The difference of two independent standard Gumbels is logistic, so a random-utility model with iid Gumbel errors yields logit choice probabilities (McFadden).

Exercise 10.5.2 For the uniform envelope the smallest valid M is $\sup f = f(\text{mode})$. For the truncated normal with proposal $c + \mathrm{Exp}(\lambda)$, the ratio f/g is maximised at $x = \lambda$ (if $\lambda \ge c$) and the acceptance rate is maximised by $\lambda^\star = \big(c + \sqrt{c^2+4}\big)/2$; for $c = 3$, $\lambda^\star = 3.30$ and the acceptance rate is about 0.96, against 0.0013 for naive rejection.

Exercise 10.5.4 Substitute $X_2^{(t)} = \rho X_1^{(t)} + \sqrt{1-\rho^2}\,\xi_t$ into $X_1^{(t+1)} = \rho X_2^{(t)} + \sqrt{1-\rho^2}\,\zeta_t$: $X_1^{(t+1)} = \rho^2 X_1^{(t)} + \eta_t$ with $\mathrm{Var}\,\eta_t = \rho^2(1-\rho^2) + (1-\rho^2) = 1-\rho^4$. For AR(1) with coefficient $\phi = \rho^2$, $\rho_k = \phi^k$ and $\tau = 1 + 2\phi/(1-\phi) = (1+\rho^2)/(1-\rho^2)$.

Exercise 10.5.5 One chain of 6000 iterations (1000 burn-in) from the mode: scale 0.1× optimal gives acceptance 0.91 but ESS about 23 for the treatment coefficient; the optimal scale gives acceptance 0.31 and ESS about 450; 10× gives acceptance 0.001 and ESS about 10. Both extremes are some 20–45 times less efficient, in line with the theory behind the 0.234 rule.

Exercise 10.5.7 Net benefit per household is $40 \times \text{AME} - 5$. From the posterior draws of Listing 10.14, its mean is about 0.48 with 95% credible interval $[-3.8, 4.7]$, and $\mathbb{P}(\text{net benefit} > 0) \approx 0.59$. A ministry should see the full distribution (and the probability of a loss of 41%), with the assumptions on cost and value stated; a larger pilot would sharpen the decision (value of information).

Exercise 10.6.1 s^2 has $\mathrm{Var}(s^2) = 2\sigma^4/(n_{\text{sim}} - 1)$ for normal data. The delta method with $g(v) = \sqrt{v}$, $g'(\sigma^2) = 1/(2\sigma)$, gives $\mathrm{Var}(s) \approx 2\sigma^4/(n_{\text{sim}}-1) \cdot 1/(4\sigma^2) = \sigma^2/(2(n_{\text{sim}}-1))$.

Exercise 10.6.2 $\mathrm{Var}(\widehat{\mathrm{MSE}}_A - \widehat{\mathrm{MSE}}_B) = \frac{1}{n_{\text{sim}}}\big(v_A + v_B - 2\,\mathrm{Cov}(e_A^2, e_B^2)\big)$ with common datasets, versus $\frac{1}{n_{\text{sim}}}(v_A + v_B)$ with independent ones; positive correlation reduces it. For coverage, the paired comparison of indicators (McNemar-type) has variance $\frac{1}{n_{\text{sim}}}\big(c_A(1-c_A) + c_B(1-c_B) - 2\,\mathrm{Cov}\big)$.

Exercise 10.6.7 By the law of total variance, $\mathrm{Var}(Y) = \mathrm{Var}(\mathbb{E}[Y \mid \theta]) + \mathbb{E}[\mathrm{Var}(Y \mid \theta)]$; with S parameter draws and R inner replications, the between-draw mean square estimates $\mathrm{Var}(\mathbb{E}[Y \mid \theta]) + \mathbb{E}\,\mathrm{Var}(Y \mid \theta)/R$ and the within mean square estimates $\mathbb{E}\,\mathrm{Var}(Y \mid \theta)$. A quick check with fixed parameters gives an inner s.d. of the headcount reduction of about 0.14 pp, against a total

s.d. of about 0.30 pp in Listing 10.18, so roughly three-quarters of the variance is due to the parameters. For estimating the outcome distribution with a fixed budget $S \times R$, take R small (even $R = 1$) and S large; larger R is needed only to estimate the decomposition itself.

Chapter 11: Resampling: the Bootstrap, Permutation Tests and Uncertainty Intervals

Exercise 11.1.1 $\mathbf{1}\{X_i \le x\}$ are iid Bernoulli$(F(x))$, so $\mathbb{E}\widehat{F}_n(x) = F(x)$ and $\operatorname{Var}\widehat{F}_n(x) = F(x)(1-F(x))/n$. For $x \le y$, $\operatorname{Cov}(\mathbf{1}\{X \le x\}, \mathbf{1}\{X \le y\}) = \mathbb{P}(X \le x) - F(x)F(y) = F(x) - F(x)F(y)$; averaging over n independent terms divides by n.

Exercise 11.1.2 Under $\widehat{F}_n$, $\iint |x-y| \, d\widehat{F}_n \, d\widehat{F}_n = n^{-2} \sum_{i,j} |X_i - X_j| = 2n^{-2} \sum_{i>j} (X_{(i)} - X_{(j)})$. In the last sum $X_{(i)}$ appears with sign + for the $i-1$ smaller indices and − for the $n-i$ larger ones, i.e. with coefficient $2i - n - 1$. Dividing by $2\bar{X}$ gives (11.2). Since the weights are at most $n-1$ and sum to zero, the numerator is maximised (for fixed total) when all income is held by one person, giving $\widehat{G} = (n-1)/n$.

Exercise 11.1.3 With $F_\varepsilon = (1-\varepsilon)F + \varepsilon\delta_{(y,x)}$, $R(F_\varepsilon) = \{(1-\varepsilon)\mu_Y + \varepsilon y\}/\{(1-\varepsilon)\mu_X + \varepsilon x\}$. Differentiating at $\varepsilon = 0$ gives $\mathrm{IF} = (y-\mu_Y)/\mu_X - \mu_Y(x-\mu_X)/\mu_X^2 = (y - Rx)/\mu_X$, so $\operatorname{se}(\widehat{R}) \approx \{\operatorname{Var}(Y - RX)/n\}^{1/2}/\mu_X$.

Exercise 11.1.5 $n = \log(2/0.05)/(2 \cdot 0.01^2) \approx 18\,445$. The DKW band holds *simultaneously* for all x; a single headcount at a fixed line has pointwise standard error $\sqrt{F(z)(1-F(z))/n} \le 0.0037$ at this n, so its 95% interval is about ±0.007.

Exercise 11.1.6 Differentiate $F_\varepsilon(q_\varepsilon) = p$ at $\varepsilon = 0$. For the lognormal with $\mu = 7$, $\sigma = 0.9$, $m = e^7 \approx 1096.6$, $f(m) \approx 4.04\times10^{-4}$ and with $n = 150$ the asymptotic standard error is $1/(2f(m)\sqrt{n}) \approx 101.0$, in agreement with the Monte Carlo "true se" of 101.4 in `ch11_bootstrap_basic.py` (the bootstrap gave 58.8 in that particular sample).

Exercise 11.1.7 Under simple random sampling the standard error of the difference is $\sqrt{(0.251 \cdot 0.749 + 0.239 \cdot 0.761)/8000} \approx 0.0068$, so $z \approx 1.76$ and the two-sided $p \approx 0.078$: not significant at 5%. With a design effect of 2.5 the standard error is multiplied by $\sqrt{2.5}$, giving 0.0108, $z \approx 1.12$, $p \approx 0.26$. The design effect is the factor by which clustering (and other design features) inflates the variance relative to a simple random sample of the same size.

Exercise 11.2.1 A resample is a multiset of size n from n types, counted by stars and bars as $\binom{2n-1}{n}$: 92 378 for $n = 10$ and about 6.9×10^{10} for $n = 20$.

Exercise 11.2.2 X_i^* are iid from $\widehat{F}_n$ with mean $\bar{X}$ and variance $\hat{\sigma}^2$, so $\mathbb{E}^*\bar{X}^* = \bar{X}$ and $\operatorname{Var}^*\bar{X}^* = \hat{\sigma}^2/n = \frac{n-1}{n}\, s^2/n$.

Exercise 11.2.3 Each draw misses $X_{(n)}$ with probability $1 - 1/n$, independently. By linearity, the expected number of distinct observations is $\sum_i \mathbb{P}(X_i \text{ drawn}) = n\{1 - (1-1/n)^n\} \to (1 - e^{-1})n$.

Exercise 11.2.4 For the sample in `ch11_ecdf_plugin.py`, $P_1 = 0.094$ with bootstrap se ≈ 0.0141 for the fixed line; with the line set at 60% of the median ($\hat{z} \approx 590$) the estimate is 0.092 with se ≈ 0.0128. The relative line moves with the income distribution, which partly offsets sampling variation in incomes, so the se need not increase; it must however be re-estimated in every resample to be correct.

Exercise 11.2.7 $\operatorname{Var}(\hat{\tau}_B^2) = (\mu_4 - \tau^4)/B + O(B^{-2}) = \tau^4(\kappa+2)/B$, with μ_4 the fourth central moment. The delta method with $g(v) = \sqrt{v}$, $g'(\tau^2) = 1/(2\tau)$ gives $\operatorname{Var}(\hat{\tau}_B) \approx \tau^2(\kappa+2)/(4B)$.

Exercise 11.3.2 With $z_0 = a = 0$, $\beta_j = \Phi(z_{\alpha_j}) = \alpha_j$. With $\hat{z}_0 = 0.15$, $\hat{a} = 0.06$: $\beta_1 = \Phi(0.15 + (-1.81)/1.1086) = \Phi(-1.483) \approx 0.069$ and $\beta_2 = \Phi(0.15 + 2.11/0.8734) \approx 0.995$, matching the output (0.0686, 0.9948).

Exercise 11.3.3 $\widehat{\theta}_{(i)} = (n\bar{X} - X_i)/(n-1)$, so $\bar{\theta}_{(\cdot)} = \bar{X}$ and $\bar{\theta}_{(\cdot)} - \widehat{\theta}_{(i)} = (X_i - \bar{X})/(n-1)$. The factor $(n-1)^{-1}$ cancels in (11.6), leaving $\hat{a} = \sum (X_i - \bar{X})^3 / \{6(\sum (X_i - \bar{X})^2)^{3/2}\} = \hat{\gamma}/(6\sqrt{n})$.

Exercise 11.3.7 Two intervals can overlap while the difference is significant, because $\operatorname{se}(\hat{\theta}_1 - \hat{\theta}_2) < \operatorname{se}(\hat{\theta}_1) + \operatorname{se}(\hat{\theta}_2)$. Moreover, the regional sample is part of the national one, so the estimates are positively correlated. Bootstrap the *difference*: resample clusters (within strata) from the national sample, recompute both Ginis on each resample from the same resampled data, and form an interval for the difference.

Exercise 11.4.1 Without centring, $\mathbb{E}^* u_i^* = \bar{\hat{u}} \neq 0$, so $\mathbb{E}^* \widehat{\boldsymbol{\beta}}^* = \widehat{\boldsymbol{\beta}} + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{1}\, \bar{\hat{u}} \neq \widehat{\boldsymbol{\beta}}$.

Exercise 11.4.2 Each arm has $G/2$ clusters; the arm mean is the average of $G/2$ cluster means, each with variance $\sigma^2\{1 + (m-1)\rho\}/m$. Hence $\operatorname{Var}(\hat{\beta}) = \frac{4\sigma^2}{Gm}\{1 + (m-1)\rho\}$, which is the iid variance $4\sigma^2/(Gm)$ times the Moulton factor.

Exercise 11.4.3 Each cluster's weight is ± 1, giving 2^G weight vectors. In the restricted bootstrap $y^* = \tilde{y} + w \circ \tilde{u}$; replacing w by $-w$ reflects $\hat{\beta}^*$ about the restricted value 0 and leaves the CR1 standard error unchanged, so $|t^*|$ is identical. With $G = 6$ there are 64 vectors but only 32 distinct values of $|t^*|$, so the smallest attainable p-value with full enumeration is $1/32 \approx 0.031$.

Exercise 11.4.7 Ignoring end effects, each block contributes ℓ consecutive centred observations and blocks are drawn independently, so $T \operatorname{Var}^*(\bar{X}^*) \approx \ell^{-1} \widehat{\operatorname{Var}}\big(\sum_{t \in \text{block}} X_t\big) = \sum_{|h|<\ell} (1 - |h|/\ell)\hat{\gamma}(h)$, the Bartlett-weighted long-run variance estimator with bandwidth ℓ.

Exercise 11.4.8 With 14 clusters (7 treated), CR1 standard errors are biased downward and the t-statistic has fatter tails than $N(0,1)$ or even $t(13)$; the simulation in `ch11_wild_cluster.py` shows rejection rates of roughly 10–15% for nominal 5% tests at this G. Request wild cluster restricted bootstrap and district-level randomisation-inference p-values, and a power and design-effect calculation for any scale-up trial.

Exercise 11.5.1 If $N_1 = N_0$, the complement of an assignment, $\mathbf{1} - \boldsymbol{\omega}$, is also in Ω and gives $T(\mathbf{1} - \boldsymbol{\omega}) = -T(\boldsymbol{\omega})$; this bijection makes the distribution symmetric. The observed assignment and its complement both satisfy $|T| \geq |T_{\text{obs}}|$, so $p \geq 2/\binom{N}{N_1}$ (= 0.0022 for 924 assignments).

Exercise 11.5.2 Under H_0 the $M+1$ statistics are exchangeable, so the rank r of the observed one (counting from the top, ties broken in its disfavour) satisfies $\mathbb{P}(r \leq k) \leq k/(M+1)$. Since $\hat{p} = r/(M+1)$, $\mathbb{P}(\hat{p} \leq \alpha) = \mathbb{P}(r \leq \lfloor \alpha(M+1) \rfloor) \leq \alpha$.

Exercise 11.5.3 Holm's first threshold is α/m (Bonferroni's) and the later ones are larger. Example: $p = (0.01, 0.02, 0.03)$, $\alpha = 0.05$: Bonferroni (0.0167) rejects only the first; Holm compares with 0.0167, 0.025, 0.05 and rejects all three.

Exercise 11.5.4 Enumerate the $6^3 = 216$ within-district assignments and recompute the difference in means for each. For one stratified assignment of the simulated pilot (observed difference 1.40), the stratified exact p-value is 0.53 versus 0.48 from the unstratified 924 assignments. In general the unstratified reference set includes assignments the design could never produce; the two p-values differ most when outcomes differ strongly between districts.

Exercise 11.5.8 Under the global null, $24 \times 0.05 = 1.2$ false positives are expected, so four uncorrected rejections are only weak evidence. Report Holm-adjusted p-values for a small pre-specified set of primary outcomes, Benjamini–Hochberg q-values for secondary outcomes, and/or standardised family indices. A pre-analysis plan fixes outcomes, families and corrections in advance and so removes the scope for selecting the significant ones afterwards.

Exercise 11.6.1 $\widehat{\theta}_{(i)} = \frac{n}{n-1}\widehat{\theta} - \frac{n}{(n-1)^2}(X_i - \bar{X})^2$. Averaging, $\bar{\theta}_{(\cdot)} = \frac{n}{n-1}\widehat{\theta} - \frac{n}{(n-1)^2}\widehat{\theta} = \frac{n(n-2)}{(n-1)^2}\widehat{\theta}$, and $n\widehat{\theta} - (n-1)\bar{\theta}_{(\cdot)} = \frac{n}{n-1}\widehat{\theta} = s^2$.

Exercise 11.6.2 For the mean, $\widehat{\theta}_{(i)} - \bar{\theta}_{(\cdot)} = -(X_i - \bar{X})/(n-1)$, so $\widehat{\mathrm{Var}}_{\mathrm{J}} = \frac{n-1}{n} \cdot \frac{\sum (X_i - \bar{X})^2}{(n-1)^2} = s^2/n$. For the ratio, a first-order expansion gives $\widehat{\theta}_{(i)} - \widehat{R} \approx -(Y_i - \widehat{R}X_i)/\{(n-1)\bar{X}\}$, so $\widehat{\mathrm{Var}}_{\mathrm{J}} \approx \sum (Y_i - \widehat{R}X_i)^2/\{n(n-1)\bar{X}^2\}$, the linearisation variance.

Exercise 11.6.3 Sherman–Morrison: $(\mathbf{A} - \mathbf{u}\mathbf{u}^\top)^{-1} = \mathbf{A}^{-1} + \mathbf{A}^{-1}\mathbf{u}\mathbf{u}^\top\mathbf{A}^{-1}/(1 - \mathbf{u}^\top\mathbf{A}^{-1}\mathbf{u})$, verified by multiplying by $\mathbf{A} - \mathbf{u}\mathbf{u}^\top$. GCV replaces each h_{ii} by its average $\mathrm{tr}(\mathbf{H})/n$, giving $\mathrm{GCV} = \frac{1}{n}\sum e_i^2/(1 - \mathrm{tr}\,\mathbf{H}/n)^2$.

Exercise 11.6.8 If the 80% bands were calibrated, the number of hits in 40 quarters would be roughly Binomial$(40, 0.8)$ with mean 32 and sd 2.5; 22 hits is about 4 sd below (less if forecast errors are serially correlated, which reduces the effective number of quarters). The fan was too narrow, perhaps because of larger shocks (e.g. food and energy prices, exchange-rate movements) than in the estimation period. It can be recalibrated by scaling the bands so that they match the empirical quantiles of past pseudo-out-of-sample errors at each horizon, as split conformal prediction does.

Chapter 12: Matrix Calculus for Machine Learning, Regularisation and the Bias-Variance Trade-off

Exercise 12.1.1 $\|\mathbf{x}\|_2 = (\mathbf{x}^\top\mathbf{x})^{1/2}$, so by the chain rule and Theorem 12.2(c) with $\mathbf{A} = \mathbf{I}$, $\nabla\|\mathbf{x}\| = \frac{1}{2}(\mathbf{x}^\top\mathbf{x})^{-1/2} \cdot 2\mathbf{x} = \mathbf{x}/\|\mathbf{x}\|$. For the second, $\mathrm{d}\,\|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2 = 2(\mathbf{A}\mathbf{x} - \mathbf{b})^\top\mathbf{A}\,\mathrm{d}\mathbf{x}$, so the gradient is $2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$.

Exercise 12.1.2 $\mathrm{d}\,\mathrm{tr}(\mathbf{A}\mathbf{X}\mathbf{B}\mathbf{X}^\top) = \mathrm{tr}(\mathbf{A}\,\mathrm{d}\mathbf{X}\,\mathbf{B}\mathbf{X}^\top) + \mathrm{tr}(\mathbf{A}\mathbf{X}\mathbf{B}\,\mathrm{d}\mathbf{X}^\top) = \mathrm{tr}(\mathbf{B}\mathbf{X}^\top\mathbf{A}\,\mathrm{d}\mathbf{X}) + \mathrm{tr}(\mathbf{B}^\top\mathbf{X}^\top\mathbf{A}^\top\mathrm{d}\mathbf{X})$, using cyclicity and $\mathrm{tr}(\boldsymbol{M}^\top) = \mathrm{tr}(\boldsymbol{M})$. Hence $\boldsymbol{G}^\top = \mathbf{B}\mathbf{X}^\top\mathbf{A} + \mathbf{B}^\top\mathbf{X}^\top\mathbf{A}^\top$ and $\boldsymbol{G} = \mathbf{A}^\top\mathbf{X}\mathbf{B}^\top + \mathbf{A}\mathbf{X}\mathbf{B}$. For the determinant, $\mathrm{d}\det\mathbf{X} = \det(\mathbf{X})\,\mathrm{tr}(\mathbf{X}^{-1}\mathrm{d}\mathbf{X})$ (proof of Proposition 12.5(e)), so $\partial\det\mathbf{X}/\partial\mathbf{X} = \det(\mathbf{X})\mathbf{X}^{-\top}$.

Exercise 12.1.3 $\nabla \operatorname{lse} = \mathbf{s}(\mathbf{x})$, so $\nabla^2 \operatorname{lse} = D\mathbf{s} = \operatorname{diag}(\mathbf{s}) - \mathbf{s}\mathbf{s}^\top$ by (12.3). For any $\boldsymbol{v}$, $\boldsymbol{v}^\top(\operatorname{diag}\mathbf{s} - \mathbf{s}\mathbf{s}^\top)\boldsymbol{v} = \sum_j s_j v_j^2 - (\sum_j s_j v_j)^2 = \operatorname{Var}_{\mathbf{s}}(v) \geq 0$, the variance of v_J when J has distribution $\mathbf{s}$. It is zero iff $\boldsymbol{v}$ is constant, i.e. $\boldsymbol{v} \propto \mathbf{1}$. Hence lse is convex, but not strictly (it is linear along $\mathbf{1}$: $\operatorname{lse}(\mathbf{x} + t\mathbf{1}) = \operatorname{lse}(\mathbf{x}) + t$).

Exercise 12.1.4 Vectorise $\mathrm{d}(\mathbf{X}^{-1}) = -\mathbf{X}^{-1}\mathrm{d}\mathbf{X}\,\mathbf{X}^{-1}$ using $\operatorname{vec}(\mathbf{A}\boldsymbol{M}\mathbf{B}) = (\mathbf{B}^\top \otimes \mathbf{A})\operatorname{vec}\boldsymbol{M}$ with $\mathbf{A} = \mathbf{X}^{-1}$, $\mathbf{B} = \mathbf{X}^{-1}$: $\mathrm{d}\operatorname{vec}\mathbf{X}^{-1} = -(\mathbf{X}^{-\top} \otimes \mathbf{X}^{-1})\mathrm{d}\operatorname{vec}\mathbf{X}$. The Jacobian is $-(\mathbf{X}^{-\top} \otimes \mathbf{X}^{-1})$; a central-difference check on a random 3×3 matrix gives errors around 10^{-9}.

Exercise 12.1.5 (a) $\mathrm{d}\log\det(\mathbf{X}^\top\mathbf{X}) = \operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}(\mathrm{d}\mathbf{X}^\top\mathbf{X} + \mathbf{X}^\top\mathrm{d}\mathbf{X})\big) = 2\operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathrm{d}\mathbf{X}\big)$, so the gradient is $2\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}$. (b) $\mathrm{d}\operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}\big) = -2\operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-2}\mathbf{X}^\top\mathrm{d}\mathbf{X}\big)$, gradient $-2\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-2}$. With a random 6×3 matrix the central-difference errors are of order 10^{-10}; replacing $\mathbf{B}^\top$ by $\mathbf{B}$ in the gradient of $\operatorname{tr}(\mathbf{B}\mathbf{X})$ gives a maximal error of 0.23 in our run, eight orders of magnitude larger.

Exercise 12.1.6 $f(\mathbf{x} + ih\mathbf{e}_j) = f(\mathbf{x}) + ih\,\partial_j f - \frac{h^2}{2}\partial_j^2 f - \dots$, so $\operatorname{Im} f(\mathbf{x} + ih\mathbf{e}_j)/h = \partial_j f + O(h^2)$ with no subtraction of nearly equal numbers; for log-sum-exp the error is at machine precision ($\approx 10^{-16}$) for all h from about 10^{-5} down to 10^{-300}. It requires f to be analytic (holomorphic extension); $|x|$ and $\max(x, 0)$ are not, and `np.abs` of a complex number returns the modulus, destroying the imaginary part.

Exercise 12.1.7 By Proposition 12.5(d),(e) with $\mathrm{d}\boldsymbol{\Sigma} = \dot{\boldsymbol{\Sigma}}\,\mathrm{d}\theta$: $\frac{\mathrm{d}}{\mathrm{d}\theta}\log\det\boldsymbol{\Sigma} = \operatorname{tr}(\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}})$ and $\frac{\mathrm{d}}{\mathrm{d}\theta}\boldsymbol{\Sigma}^{-1} = -\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}}\boldsymbol{\Sigma}^{-1}$. For $\ell(\theta) = -\frac{1}{2}\log\det\boldsymbol{\Sigma} - \frac{1}{2}\mathbf{y}^\top\boldsymbol{\Sigma}^{-1}\mathbf{y}$ the score is $\ell'(\theta) = -\frac{1}{2}\operatorname{tr}(\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}}) + \frac{1}{2}\mathbf{y}^\top\boldsymbol{\Sigma}^{-1}\dot{\boldsymbol{\Sigma}}\boldsymbol{\Sigma}^{-1}\mathbf{y} = \frac{1}{2}\operatorname{tr}\big[(\boldsymbol{\alpha}\boldsymbol{\alpha}^\top - \boldsymbol{\Sigma}^{-1})\dot{\boldsymbol{\Sigma}}\big]$ with $\boldsymbol{\alpha} = \boldsymbol{\Sigma}^{-1}\mathbf{y}$, the standard expression used for Gaussian-process hyperparameters.

Exercise 12.2.1 $\ell(\boldsymbol{K}) = \frac{m}{2}\log\det\boldsymbol{K} - \frac{1}{2}\operatorname{tr}(\boldsymbol{K}\mathbf{W})$. The trace is linear. For log-det, $\mathrm{d}^2\log\det\boldsymbol{K} = -\operatorname{tr}(\boldsymbol{K}^{-1}\mathrm{d}\boldsymbol{K}\,\boldsymbol{K}^{-1}\mathrm{d}\boldsymbol{K}) = -\big\|\boldsymbol{K}^{-1/2}\mathrm{d}\boldsymbol{K}\,\boldsymbol{K}^{-1/2}\big\|_F^2 < 0$ for every symmetric $\mathrm{d}\boldsymbol{K} \neq \mathbf{0}$. Hence ℓ is strictly concave on the cone of positive definite matrices, and the stationary point $\boldsymbol{K} = m\mathbf{W}^{-1}$ is the unique maximiser.

Exercise 12.2.2 The Newton step solves $\mathbf{X}^\top\mathbf{W}\mathbf{X}\,\boldsymbol{\beta}^+ = \mathbf{X}^\top\mathbf{W}\mathbf{X}\boldsymbol{\beta} - \mathbf{X}^\top(\boldsymbol{p} - \mathbf{y}) = \mathbf{X}^\top\mathbf{W}\big(\mathbf{X}\boldsymbol{\beta} + \mathbf{W}^{-1}(\mathbf{y} - \boldsymbol{p})\big) = \mathbf{X}^\top\mathbf{W}\boldsymbol{z}$, which are the normal equations of weighted least squares of $\boldsymbol{z}$ on $\mathbf{X}$ with weights $\mathbf{W}$.

Exercise 12.2.3 $\nabla\ell = \mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu})$ with $\mu_i = e^{\mathbf{x}_i^\top\boldsymbol{\beta}}$, and $\nabla^2\ell = -\mathbf{X}^\top\operatorname{diag}(\boldsymbol{\mu})\mathbf{X} \preceq 0$, so ℓ is concave (strictly if $\mathbf{X}$ has full column rank).

Exercise 12.2.4 With $\boldsymbol{\Omega} = \boldsymbol{L}\boldsymbol{L}^\top$, the model $\boldsymbol{L}^{-1}\mathbf{y} = \boldsymbol{L}^{-1}\mathbf{X}\boldsymbol{\beta} + \boldsymbol{L}^{-1}\boldsymbol{\varepsilon}$ has spherical errors, and OLS on it gives (12.12). GLS is BLUE in the whitened model by Gauss–Markov; since OLS is linear and unbiased, $\operatorname{Var}(\hat{\boldsymbol{\beta}}_{\mathrm{OLS}}) - \operatorname{Var}(\hat{\boldsymbol{\beta}}_{\mathrm{GLS}}) \succeq 0$. Explicitly, with $\boldsymbol{M} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top - (\mathbf{X}^\top\boldsymbol{\Omega}^{-1}\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{\Omega}^{-1}$ we have $\boldsymbol{M}\mathbf{X} = \mathbf{0}$, and the difference equals $\sigma^2\boldsymbol{M}\boldsymbol{\Omega}\boldsymbol{M}^\top \succeq 0$.

Exercise 12.2.5 Under perfect separation the Newton iterates diverge: $\|\boldsymbol{\beta}\|$ grows roughly linearly with the iteration count, the loss decreases towards zero, and the weights $p_i(1 - p_i) \to 0$ make the condition number of the Hessian explode (numpy eventually reports a singular matrix or returns huge steps). With a ridge penalty, the gradient becomes $\mathbf{X}^\top(\boldsymbol{p} - \mathbf{y}) + \lambda\boldsymbol{\beta}$ and the Hessian $\mathbf{X}^\top\mathbf{W}\mathbf{X} + \lambda\mathbf{I}$; Newton converges in a handful of iterations to a finite solution whose norm decreases as λ increases.

Exercise 12.2.7 With 40 covariates and 600 households (and few dropouts in some cells), some covariate combinations (e.g. rare dummies) may (quasi-)separate the outcome. Then the log-likelihood is flat in some directions: the Hessian $\mathbf{X}^\top \hat{\mathbf{W}} \mathbf{X}$ has near-zero eigenvalues, so the estimated coefficients along those directions are huge and the standard errors (square roots of the diagonal of its inverse) are enormous. Remedies: penalisation (ridge/lasso or Firth's bias-reduced logit), merging sparse categories, or reducing the number of covariates using prior knowledge.

Exercise 12.3.1 Forward sweep: $\mathbf{r} = \mathbf{A}\mathbf{x} - \mathbf{b}$, $f = \mathbf{r}^\top \mathbf{r}$. Reverse sweep: $\bar{f} = 1$, $\bar{\mathbf{r}} = 2\mathbf{r}$, $\bar{\mathbf{x}} = \mathbf{A}^\top \bar{\mathbf{r}} = 2\mathbf{A}^\top(\mathbf{A}\mathbf{x} - \mathbf{b})$, as in Exercise 12.1.1.

Exercise 12.3.2 Right-to-left (forward mode) multiplies $d \times d$ matrices: $(L-2)$ products of cost d^3 plus a final $1 \times d$ by $d \times d$ product, i.e. $O(Ld^3)$. Left-to-right (reverse mode) starts from the $1 \times d$ row vector $D f_L$ and performs only vector–matrix products of cost d^2: $O(Ld^2)$. Reverse mode is cheaper by a factor d, which is the content of Theorem 12.9 for scalar outputs.

Exercise 12.3.3 For $\boldsymbol{Z} = \boldsymbol{A}\mathbf{W} + \mathbf{1}\mathbf{b}^\top$: $\mathrm{d}L = \operatorname{tr}(\bar{\boldsymbol{Z}}^\top \mathrm{d}\boldsymbol{Z}) = \operatorname{tr}(\bar{\boldsymbol{Z}}^\top \boldsymbol{A}\, \mathrm{d}\mathbf{W}) + \operatorname{tr}(\bar{\boldsymbol{Z}}^\top \mathbf{1}\, \mathrm{d}\mathbf{b}^\top) + \operatorname{tr}(\bar{\boldsymbol{Z}}^\top \mathrm{d}\boldsymbol{A}\, \mathbf{W})$, giving $\bar{\mathbf{W}} = \boldsymbol{A}^\top \bar{\boldsymbol{Z}}$, $\bar{\mathbf{b}} = \bar{\boldsymbol{Z}}^\top \mathbf{1}$ and $\bar{\boldsymbol{A}} = \bar{\boldsymbol{Z}} \mathbf{W}^\top$. Elementwise activations give $\bar{\boldsymbol{Z}}_1 = \bar{\boldsymbol{A}}_1 \odot \sigma'(\boldsymbol{Z}_1)$; for ReLU, $\sigma'(z) = \mathbb{1}\{z > 0\}$ (any value in $[0,1]$ at $z = 0$, conventionally 0).

Exercise 12.3.4 Dual numbers: $\exp(a, \dot{a}) = (e^a, e^a \dot{a})$, $(a, \dot{a})/(b, \dot{b}) = (a/b, (\dot{a}b - a\dot{b})/b^2)$. Tape: local partials e^a for exp; $1/b$ and $-a/b^2$ for division. The derivative of $\log(1 + e^{-yz})$ with respect to z is $-y/(1 + e^{yz})$, which both implementations reproduce to machine precision.

Exercise 12.3.5 Without weight decay the training loss keeps decreasing towards zero while the test loss reaches a minimum after a few tens of epochs and then drifts upwards; early stopping at the minimum of the validation loss acts as an implicit regulariser (for linear models, it is approximately equivalent to ridge with λ decreasing in the number of iterations).

Exercise 12.3.6 $\nabla^2 L(\boldsymbol{\theta})\boldsymbol{v} = \lim_{t \to 0} [\nabla L(\boldsymbol{\theta} + t\boldsymbol{v}) - \nabla L(\boldsymbol{\theta})]/t$, a directional derivative of the gradient map. Applying forward mode (one tangent direction $\boldsymbol{v}$) to the reverse-mode program that computes ∇L yields $\nabla^2 L\, \boldsymbol{v}$ at a constant multiple of the cost of one gradient, independent of the number of parameters. Conjugate-gradient (Hessian-free) Newton methods need only such products, never the full Hessian.

Exercise 12.3.7 For 24 outputs and 300 inputs, reverse mode needs 24 sweeps and forward mode 300, so reverse mode (or, equivalently, computing the 24×300 Jacobian row by row) is cheaper. For a single scalar loss, one reverse sweep suffices.

Exercise 12.4.1 Apply Theorem 12.10 conditionally on $\mathbf{x}_0$ and take the expectation over $\mathbf{x}_0 \sim P_X$; since $\mathbf{x}_0$ is independent of $\mathcal{D}$ and ε_0, the cross terms still vanish conditionally, and the law of iterated expectations gives the stated decomposition.

Exercise 12.4.2 $\operatorname{tr}(\mathbf{H}) = \operatorname{tr}\big(\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\big) = \operatorname{tr}\big((\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{X}\big) = \operatorname{tr}(\mathbf{I}_p) = p$. For k-NN, $S_{ii} = 1/k$ (each point is one of its own k neighbours), so $\operatorname{tr}(\mathbf{S}_k) = n/k$.

Exercise 12.4.3 $\mathbb{E}(Y_i^{\text{new}} - \hat{y}_i)^2 = \sigma^2 + \operatorname{Var}(\hat{y}_i) + (f_i - \mathbb{E}\hat{y}_i)^2$ and $\mathbb{E}(y_i - \hat{y}_i)^2 = \sigma^2 + \operatorname{Var}(\hat{y}_i) - 2\operatorname{Cov}(y_i, \hat{y}_i) + (f_i - \mathbb{E}\hat{y}_i)^2$. Subtract and sum: the optimism is $\frac{2}{n}\sum_i \operatorname{Cov}(y_i, \hat{y}_i) = \frac{2\sigma^2}{n} \operatorname{tr}\mathbf{S}$. For OLS, $\mathbb{E}\,\|\mathbf{y} - \mathbf{H}\mathbf{y}\|^2 = \sigma^2 \operatorname{tr}(\mathbf{I} - \mathbf{H}) = \sigma^2(n - p)$ because $(\mathbf{I} - \mathbf{H})\mathbf{X}\boldsymbol{\beta} = \mathbf{0}$; hence $\hat{\sigma}^2$ is unbiased.

Exercise 12.4.4 Let $Y \in \{0,1\}$ with $\mathbb{P}(Y=1 \mid \mathbf{x}_0) = 0.9$, and suppose a classifier predicts 1 with probability q over training sets. Its expected 0–1 loss is $0.1q + 0.9(1-q)$, which *decreases* as the variance $q(1-q)$ increases from $q = 0$; increasing variance can reduce error when the average prediction is on the wrong side of $1/2$. No additive decomposition with non-negative variance term can hold.

Exercise 12.4.5 With the script's settings the optimal degree is 5 for $n = 25$ (expected test MSE 0.236) and 7 for $n = 200$ (MSE 0.127, close to $\sigma^2 = 0.1225$): larger samples support more complex models, because variance falls like (degree + 1)$/n$. This is the "complexity must grow slowly with n" message of Geman et al. (1992).

Exercise 12.4.6 With $\lambda = 10^{-2}$ the peak at $p = n$ largely disappears: the ridge solution divides by $d_j^2 + \lambda$ instead of d_j^2, so the tiny singular values of the feature matrix near the interpolation threshold no longer blow up the coefficient norm.

Exercise 12.4.7 With $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ (n non-zero singular values), $\hat{\boldsymbol{\beta}} = \mathbf{X}^+\mathbf{X}\boldsymbol{\beta} + \mathbf{X}^+\boldsymbol{\varepsilon} = \boldsymbol{P}\boldsymbol{\beta} + \mathbf{X}^+\boldsymbol{\varepsilon}$, where $\boldsymbol{P} = \mathbf{V}\mathbf{V}^\top$ projects onto the row space. Then $\mathbb{E}[\left\|\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}\right\|^2 \mid \mathbf{X}] = \|(\mathbf{I} - \boldsymbol{P})\boldsymbol{\beta}\|^2 + \sigma^2 \sum_{j=1}^{n} d_j^{-2}$. As $p/n \to 1$ the smallest singular value of a Gaussian matrix tends to zero (Marchenko–Pastur edge $\sqrt{p} - \sqrt{n}$), so the variance term diverges; as p/n grows, all $d_j^2 \approx p$ and the variance shrinks while the bias term $\|(\mathbf{I} - \boldsymbol{P})\boldsymbol{\beta}\|^2$ grows.

Exercise 12.4.8 Key points for the memo: with 60 indicators and 70 districts, R^2 is close to one almost by construction (OLS with $p = n$ interpolates); by (12.18) the in-sample error understates out-of-sample error by about $2\sigma^2 \cdot 60/70$; demand cross-validated or held-out (e.g. later survey round) prediction error, comparison with a simple benchmark, and a regularised specification.

Exercise 12.5.1 $\left\|\hat{\boldsymbol{\beta}}_\lambda\right\|^2 = \sum_j d_j^2 (\mathbf{u}_j^\top \mathbf{y})^2/(d_j^2 + \lambda)^2$, and each term is non-increasing in λ. The push-through identity $(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}_p)\mathbf{X}^\top = \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I}_n)$ gives $\hat{\boldsymbol{\beta}}_\lambda = \mathbf{X}^\top(\mathbf{X}\mathbf{X}^\top + \lambda\mathbf{I}_n)^{-1}\mathbf{y}$; it inverts an $n \times n$ instead of a $p \times p$ matrix, $O(n^2p + n^3)$ instead of $O(p^2n + p^3)$ operations, much cheaper when $p \gg n$ (this is the "kernel" form).

Exercise 12.5.2 With $\mathbf{X}^\top\mathbf{X} = n\mathbf{I}$, the objective separates as $\sum_j \left[\tfrac{1}{2}(\beta_j - \hat{\beta}_j^{\mathrm{OLS}})^2 + \lambda\left|\beta_j\right|\right]$ plus a constant, so Lemma 12.17 gives $S(\hat{\beta}_j^{\mathrm{OLS}}, \lambda)$. Ridge gives $\hat{\beta}_j^{\mathrm{OLS}}/(1+\lambda)$ (proportional shrinkage). Best subset of size k keeps the k largest $\left|\hat{\beta}_j^{\mathrm{OLS}}\right|$ unchanged and zeroes the rest (hard thresholding). Soft thresholding is continuous with slope one beyond the threshold; hard thresholding is discontinuous.

Exercise 12.5.3 If $\mathbf{x}_j = \mathbf{x}_k$ and $\hat{\beta}_j \neq \hat{\beta}_k$, replacing both by their average leaves the fit unchanged, does not increase the ℓ_1 term (triangle inequality), and strictly decreases the ℓ_2 term (strict convexity of $t \mapsto t^2$) — a contradiction. For the lasso with $\alpha = 1$ any split $(\hat{\beta}_j, \hat{\beta}_k) = (\theta c, (1-\theta)c)$, $\theta \in [0,1]$, gives the same objective, so the solution is not unique and coordinate descent typically puts everything on one of them.

Exercise 12.5.4 On an interval where the active set $\mathcal{A}$ and sign vector $\boldsymbol{s}$ are constant, (12.24) reads $\frac{1}{n}\mathbf{X}_{\mathcal{A}}^\top(\mathbf{y} - \mathbf{X}_{\mathcal{A}}\hat{\boldsymbol{\beta}}_{\mathcal{A}}) = \lambda\boldsymbol{s}$, so $\hat{\boldsymbol{\beta}}_{\mathcal{A}}(\lambda) = (\mathbf{X}_{\mathcal{A}}^\top\mathbf{X}_{\mathcal{A}})^{-1}(\mathbf{X}_{\mathcal{A}}^\top\mathbf{y} - n\lambda\boldsymbol{s})$, an affine function of λ. Knots occur where an inactive $\left|c_j\right|$ reaches λ or an active coefficient hits zero.

Exercise 12.5.5 For the elastic net the update is $\beta_j \leftarrow S\big(\frac{1}{n}\mathbf{x}_j^\top \mathbf{r}^{(j)}, \lambda\alpha\big)/\big(\frac{1}{n}\mathbf{x}_j^\top \mathbf{x}_j + \lambda(1-\alpha)\big)$. Along a decreasing grid starting at $\lambda_{\max}$, warm starts typically reduce the number of sweeps per penalty to a few; cold starts need many more, especially at small λ.

Exercise 12.5.6 If `bmi` is multiplied by 100 (without standardising), its coefficient shrinks by a factor of 100, so its penalty becomes negligible: it enters first, at a penalty about 100 times larger than before, while the default grid (which starts at $\lambda_{\max}$, now dominated by `bmi`) no longer reaches small enough penalties for some weak predictors to enter at all. The selection order reflects units of measurement, not importance — the reason for standardising.

Exercise 12.5.7 The log posterior is $-\frac{1}{2\sigma^2}\big[\boldsymbol{\beta}^\top(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})\boldsymbol{\beta} - 2\boldsymbol{\beta}^\top\mathbf{X}^\top\mathbf{y}\big]$+const, so $\boldsymbol{\beta} \mid \mathbf{y} \sim \mathcal{N}\big(\hat{\boldsymbol{\beta}}_\lambda, \sigma^2(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})^{-1}\big)$. The posterior covariance is larger than the frequentist sampling variance $\sigma^2\boldsymbol{A}_\lambda\mathbf{X}^\top\mathbf{X}\boldsymbol{A}_\lambda$ (the difference is $\sigma^2\lambda\boldsymbol{A}_\lambda^2 \succeq 0$): the posterior also reflects uncertainty about $\boldsymbol{\beta}$ that the frequentist variance attributes to bias.

Exercise 12.6.1 Deleting observation i changes $\mathbf{X}^\top\mathbf{X}$ by the rank-one term $-\mathbf{x}_i\mathbf{x}_i^\top$. Sherman–Morrison gives $\hat{\boldsymbol{\beta}}^{(-i)} = \hat{\boldsymbol{\beta}} - (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{x}_i(y_i - \hat{y}_i)/(1 - H_{ii})$, hence $y_i - \mathbf{x}_i^\top\hat{\boldsymbol{\beta}}^{(-i)} = (y_i - \hat{y}_i)/(1 - H_{ii})$. The same algebra with $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$ gives the ridge version with $(\mathbf{S}_\lambda)_{ii}$.

Exercise 12.6.2 OLS fitted values are $\sum_j \mathbf{u}_j\mathbf{u}_j^\top\mathbf{y}$; PCR with k components uses weights 1 for $j \le k$ and 0 otherwise; ridge uses the smooth weights $d_j^2/(d_j^2 + \lambda)$, which are close to 1 for large d_j and close to 0 for small d_j. PCR is a hard, ridge a soft truncation of the principal directions.

Exercise 12.6.3 Across 20 random fold assignments, $\hat{\lambda}_{\min}$ ranged from 0.059 to 0.100 and $\hat{\lambda}_{1\text{se}}$ from 0.119 to 0.170. The set of selected variables changes accordingly. One should report the variability (or use repeated CV), and avoid over-interpreting a single selected model.

Exercise 12.6.5 A nonlinear target (e.g. through $\max(F_{1t}, 0)$) favours tree ensembles or neural networks, which can capture the threshold, while all the linear methods of Listing 12.14 miss it; the gain appears mostly when the nonlinearity is active. This mirrors the finding of Goulet Coulombe et al. (2022) that nonlinearity is the main source of machine-learning gains in macroeconomic forecasting.

Exercise 12.6.7 A good design note should include: a benchmark (AR or random walk, and a simple factor model); a short list of methods (dynamic factor/PCR, ridge, lasso/elastic net, a nonlinear method such as random forests); handling of mixed release dates (ragged edge) and real-time data vintages; an expanding-window pseudo-out-of-sample evaluation over at least the last 5–7 years including a downturn; loss functions (RMSE, and possibly directional accuracy); tests of equal accuracy; and communication through forecast decompositions by indicator block, fan charts and a record of past performance.

Chapter 13: Problem Set Clinic and Applied Foundations Review

Exercise 13.1.1 $\mathbf{M}^\top = \mathbf{I} - \mathbf{H}^\top = \mathbf{M}$; $\mathbf{M}^2 = \mathbf{I} - 2\mathbf{H} + \mathbf{H}^2 = \mathbf{M}$; $\operatorname{tr}\mathbf{M} = n - \operatorname{tr}\mathbf{H} = n - k$; $\mathbf{M}\mathbf{X} = \mathbf{X} - \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{X} = \mathbf{0}$. Being symmetric idempotent, $\mathbf{M}$ has eigenvalues in $\{0, 1\}$: eigenvalue 0 with multiplicity k (eigenspace $\operatorname{col}(\mathbf{X})$) and 1 with multiplicity $n - k$ (eigenspace $\operatorname{col}(\mathbf{X})^\perp$).

Exercise 13.1.2 Base case $n = 1$ is equality. If $(1+x)^n \geq 1 + nx$, multiply both sides by $1 + x \geq 0$ (this is where $x \geq -1$ is needed: multiplying an inequality by a negative number would reverse it): $(1+x)^{n+1} \geq (1+nx)(1+x) = 1 + (n+1)x + nx^2 \geq 1 + (n+1)x$.

Exercise 13.1.5 For $\nu = 5$, (13.1) with $n = 20$ gives $25/20 - (25/9)(17)/380 = 1.126$. A run with 2×10^5 replications gave 0.96, noticeably low, and different seeds give scattered answers. The reason is that the Monte Carlo estimate of $\mathrm{Var}(s^2)$ has finite variance only if $\mathbb{E}X^8 < \infty$, i.e. $\nu > 8$: for $\nu = 5$ the formula is correct but the simulation check converges very slowly and is dominated by rare huge draws (typically underestimating). For $\nu = 4.5$, $\sigma^2 = 1.8$ and $\mu_4 = 3\nu^2/\{(\nu-2)(\nu-4)\} = 48.6$ is finite, so the formula still holds ($\mathrm{Var}(s^2) \approx 2.29$), but the simulated value (about 10 in our run) is meaningless: as $\nu \downarrow 4$, $\mu_4 \to \infty$ and the Monte Carlo estimate is driven by a handful of extreme samples. Lesson: a simulation check needs its own moment conditions.

Exercise 13.2.1 $\mathbf{X}^\top \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{Q}\mathbf{R}\mathbf{R}^{-1}\mathbf{Q}^\top \mathbf{y} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{y} = \mathbf{X}^\top \mathbf{y}$. With $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}^\top$, so its singular values are d_j^2 and $\mathrm{cond}(\mathbf{X}^\top\mathbf{X}) = (d_1/d_k)^2$.

Exercise 13.2.4 Leverages are $h_{ii} = \sum_j Q_{ij}^2$ (row sums of squares of the thin $\mathbf{Q}$). For the simulated survey with $n = 4000$ the HC0, HC1 and HC3 standard errors of $\hat{\rho}$ are 0.002642, 0.002647 and 0.002653: indistinguishable, because all h_{ii} are tiny ($\bar{h} = k/n = 0.0035$). With $n = 200$ the three differ by several percent, HC3 being largest; HC3 is the safer choice in small samples.

Exercise 13.2.5 $\hat{E}^* = -\hat{\beta}_1/(2\hat{\beta}_2) = 29.3$ years. The gradient of $g(\beta_1, \beta_2) = -\beta_1/(2\beta_2)$ is $(-1/(2\beta_2),\ \beta_1/(2\beta_2^2))$; with the HC1 covariance the delta-method standard error is 0.64, giving $[28.0, 30.5]$. A pairs bootstrap ($B = 999$) gives the percentile interval $[28.1, 30.7]$, slightly shifted to the right because the ratio estimator is skewed.

Exercise 13.2.6 With a first-stage effect of 0.2 years the median first-stage F falls to about 1.8. The IV estimates become wildly dispersed: median 0.083 but interquartile range $[0.013, 0.159]$ and a Monte Carlo mean of about -12, reflecting the fact that the just-identified IV estimator has no finite moments and is a ratio with a denominator near zero. Normal-based intervals are badly unreliable; weak-instrument-robust inference (Anderson–Rubin tests) is needed whenever the first-stage F is small (a common rule of thumb flags $F < 10$).

Exercise 13.3.1 $\partial \ell/\partial \boldsymbol{\beta} = \sum_i (y_i - \sigma(\eta_i))\mathbf{x}_i$ using $\sigma' = \sigma(1-\sigma)$; $\partial^2 \ell/\partial\boldsymbol{\beta}\partial\boldsymbol{\beta}^\top = -\sum_i p_i(1-p_i)\mathbf{x}_i\mathbf{x}_i^\top$. For $\mathbf{v} \neq 0$, $\mathbf{v}^\top \nabla^2 \ell\, \mathbf{v} = -\sum_i p_i(1-p_i)(\mathbf{x}_i^\top \mathbf{v})^2 < 0$ because $p_i \in (0,1)$ and full column rank implies $\mathbf{X}\mathbf{v} \neq 0$.

Exercise 13.3.2 Under separation each term $y_i\eta_i - \log(1+e^{\eta_i})$ with $\eta_i = c\mathbf{x}_i^\top \boldsymbol{\beta}_0$ equals $-\log(1 + e^{-c|\mathbf{x}_i^\top \boldsymbol{\beta}_0|}) \to 0$ as $c \to \infty$; since $\ell < 0$ everywhere, the supremum 0 is not attained. With an L2 penalty the objective $\ell(\boldsymbol{\beta}) - \frac{\lambda}{2}\|\boldsymbol{\beta}\|^2$ tends to $-\infty$ as $\|\boldsymbol{\beta}\| \to \infty$ and is strictly concave, so a unique finite maximiser exists.

Exercise 13.3.5 If the selected set has size qn, the number of poor selected is $qn(1 - \mathrm{incl}) = pn(1 - \mathrm{excl})$, where p is the poverty rate. Hence inclusion and exclusion errors are equal exactly when $q = p$, whatever the score: at a budget share of about 30% in the simulation. For $q < p$ exclusion error exceeds inclusion error, and conversely.

Exercise 13.4.1 $\hat{\mathbf{y}}_\lambda = \mathbf{X}(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y} = \mathbf{U}\mathbf{D}(\mathbf{D}^2 + \lambda\mathbf{I})^{-1}\mathbf{D}\mathbf{U}^\top\mathbf{y} = \sum_j \mathbf{u}_j \frac{d_j^2}{d_j^2+\lambda}\mathbf{u}_j^\top\mathbf{y}$. PCR on the first k components projects $\mathbf{y}$ onto $\operatorname{span}(\mathbf{u}_1, \dots, \mathbf{u}_k)$, i.e. replaces the factor by $\mathbf{1}\{j \le k\}$.

Exercise 13.4.4 Computing the standardisation and PCs on the full sample uses future observations of the indicators. In this stationary simulation the effect is negligible: the DI RMSE moves from 0.766 to 0.765. Choosing the lasso penalty once on the evaluation period changes the selected α (from 0.032 to 0.056 at the last origin) but barely changes the RMSE (0.770). The lesson is not that leakage is harmless: with $N = 100$ stationary series and 150 training months the in-window estimates are already accurate. With structural breaks, trending or revised data, short samples, or a large tuning grid, leakage can make a useless model look good. Either way the evaluation is no longer a valid pseudo-out-of-sample exercise.

Exercise 13.5.1 For known parameters, the KKT conditions are $a_j/(1 + x_j/c_j) = \lambda - \mu_j$, $\mu_j x_j = 0$, $\mu_j \ge 0$. If $x_j > 0$ then $x_j = c_j(a_j/\lambda - 1)$, which requires $a_j > \lambda$; if $a_j \le \lambda$ then $x_j = 0$. Hence $x_j = c_j \max(0, a_j/\lambda - 1)$. The function $G(\lambda) = \sum_j c_j \max(0, a_j/\lambda - 1)$ is continuous and strictly decreasing where positive, so $G(\lambda) = B$ has a unique root, decreasing in B. If all are funded, $\lambda^* = \sum_j a_j c_j/(B + \sum_j c_j)$.

Exercise 13.5.2 If a_j is independent of c_j, $\mathbb{E}[a_j c_j \log(1 + x_j/c_j)] = \bar{a}_j \mathbb{E}[c_j \log(1 + x_j/c_j)]$; if c_j is known this is $\bar{a}_j c_j \log(1 + x_j/c_j)$, the plug-in objective. The expected marginal benefit is $\bar{a}_j \mathbb{E}[(1 + x_j/c_j)^{-1}]$; since $c \mapsto (1 + x/c)^{-1} = c/(c + x)$ is concave in c, Jensen's inequality gives $\mathbb{E}[c_j/(c_j + x_j)] \le \bar{c}_j/(\bar{c}_j + x_j)$ when $\mathbb{E}c_j = \bar{c}_j$: uncertainty about saturation lowers expected marginal benefits, explaining why the SAA multiplier (1.159) is below the plug-in value (1.183).

Exercise 13.6.1 $\mathbf{X} = \mathbf{Q}\mathbf{R}$ with $\mathbf{R}$ upper triangular; if $R_{jj} = 0$ then column j of $\mathbf{X}$ would lie in the span of the first $j-1$ columns, contradicting full rank. The fitted values are $\mathbf{Q}\mathbf{Q}^\top\mathbf{y}$ (projection onto $\operatorname{col}(\mathbf{X}) = \operatorname{col}(\mathbf{Q})$), so $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{Q}\mathbf{Q}^\top)\mathbf{y}$ and by Pythagoras $\|\hat{\mathbf{e}}\|^2 = \|\mathbf{y}\|^2 - \|\mathbf{Q}\mathbf{Q}^\top\mathbf{y}\|^2 = \|\mathbf{y}\|^2 - \|\mathbf{Q}^\top\mathbf{y}\|^2$.

Exercise 13.6.2 Write $\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$ and $\mathbf{v} = \mathbf{V}\mathbf{w}$; then $\mathbf{v}^\top\mathbf{S}\mathbf{v}/\mathbf{v}^\top\mathbf{v} = \sum_j \lambda_j w_j^2/\sum_j w_j^2 \le \lambda_1$, with equality for $\mathbf{w} = \mathbf{e}_1$. The variance of the j-th component is λ_j and total variance is $\operatorname{tr}\mathbf{S} = \sum_j \lambda_j$, so k components explain $\sum_{j\le k}\lambda_j/\operatorname{tr}\mathbf{S}$.

Exercise 13.6.3 $f_Y(y) = \frac{1}{y\sigma\sqrt{2\pi}} \exp\{-(\log y - \mu)^2/(2\sigma^2)\}$ for $y > 0$; $\mathbb{E}Y = e^{\mu+\sigma^2/2}$ (normal m.g.f. at 1), median $Y = e^\mu$ (monotone transformation of the normal median), $\operatorname{Var} Y = (e^{\sigma^2} - 1)e^{2\mu+\sigma^2}$. Mean/median $= e^{\sigma^2/2}$, which is $e^{0.32} = 1.38$ for $\sigma = 0.8$: mean income is 38% above the median, by Jensen's inequality for the convex exponential.

Exercise 13.6.4 $\ell(\boldsymbol{\beta}) = \sum_i \{y_i \mathbf{x}_i^\top\boldsymbol{\beta} - e^{\mathbf{x}_i^\top\boldsymbol{\beta}} - \log y_i!\}$, score $\mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu})$ with $\mu_i = e^{\mathbf{x}_i^\top\boldsymbol{\beta}}$, Hessian $-\mathbf{X}^\top \operatorname{diag}(\mu_i)\mathbf{X} \le 0$, so ℓ is concave (strictly if $\mathbf{X}$ has full rank). Newton: $\boldsymbol{\beta}^+ = (\mathbf{X}^\top\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{W}\mathbf{z}$ with $\mathbf{W} = \operatorname{diag}(\mu_i)$ and working response $z_i = \mathbf{x}_i^\top\boldsymbol{\beta} + (y_i - \mu_i)/\mu_i$.

Exercise 13.6.5 $\mathbb{P}(|\bar{X}_n - \mu| \ge \varepsilon) \le \operatorname{Var}(\bar{X}_n)/\varepsilon^2 = \sigma^2/(n\varepsilon^2) \to 0$. For iid standard Cauchy variables the characteristic function of $\bar{X}_n$ is $(e^{-|t|/n})^n = e^{-|t|}$, so $\bar{X}_n$ is again standard Cauchy for every n and cannot converge to a constant; running means in a simulation keep jumping when extreme draws occur.

Exercise 13.6.6 With $g(\rho) = 100(e^\rho - 1)$, $g'(\rho) = 100e^\rho$, so se $= 100e^{0.10107} \times 0.00265 = 0.293$ percentage points. The 95% interval is $10.64 \pm 1.96 \times 0.293 = [10.07, 11.22]$%, essentially identical to the pairs-bootstrap interval [10.04, 11.22]%; with $n = 4000$ the delta method is accurate.

Exercise 13.6.7 $\mathbf{x}_{k+1} - \mathbf{x}^* = (\mathbf{I} - \mathbf{A}/L)(\mathbf{x}_k - \mathbf{x}^*)$ because $\nabla f(\mathbf{x}^*) = 0$. The eigenvalues of $\mathbf{I} - \mathbf{A}/L$ lie in $[0, 1 - \mu/L]$, so its spectral norm is $1 - \mu/L$ and the bound follows. The rate $1 - 1/\kappa$ with $\kappa = L/\mu = \text{cond}(\mathbf{A})$ deteriorates for ill-conditioned problems: about $\kappa \log(1/\epsilon)$ iterations are needed. For least squares $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$, so $\kappa = \text{cond}(\mathbf{X})^2$ — one more reason to standardise predictors.

Exercise 13.6.8 Water-filling (Proposition 13.17): try the programmes in order of a_j. With health and training only, $\lambda = 110/(20 + 40) = 1.833 < 2.2$, so school feeding must also enter. With three programmes, $\lambda^* = (45 + 65 + 66)/(20 + 70) = 176/90 = 1.956$; cash ($a = 1.6 < 1.956$) stays at zero. Allocation: health $15(3/1.956 - 1) = 8.01$, training $25(2.6/1.956 - 1) = 8.24$, school $30(2.2/1.956 - 1) = 3.75$, cash 0. An extra unit of budget would raise benefits by about 1.96 units.

Exercise 13.6.9 Exact: $\mathbb{P}(Z > 4) = 3.167 \times 10^{-5}$. Plain Monte Carlo with $R = 10^5$ observes only about 3 exceedances (our run: 3.0×10^{-5}, s.e. 1.7×10^{-5}, relative error about 55%). Importance sampling from $\mathcal{N}(4, 1)$ with weights $w(y) = \phi(y)/\phi(y - 4) = e^{-4y+8}$ on $\{y > 4\}$ gave 3.161×10^{-5} with s.e. 2.1×10^{-7}, a relative error below 1%: a variance reduction of several thousand.

Exercise 13.6.10 Key steps: simulate log consumption normal with a treatment shift; the permutation test re-randomises treatment labels (e.g. 9999 times) and compares the observed mean difference with its permutation distribution; it is exact under the sharp null of no effect for anyone. The Welch test gives a similar p-value with $n = 600$. The bootstrap resamples treated and control groups separately; intervals for the median effect are wider than for the mean in relative terms with light tails, but more robust when consumption has extreme values.

Exercise 13.6.11 With the SVD $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$ of each training fold (centred), all $\hat{\boldsymbol{\beta}}_\lambda = \mathbf{V}\,\text{diag}\{d_j/(d_j^2 + \lambda)\}\mathbf{U}^\top\mathbf{y}$ follow from one factorisation. On `load_diabetes` (whose features are already scaled to unit norm), 10-fold CV over $\lambda \in [10^{-4}, 10^2]$ selects a small penalty of order 0.05–0.1 (`RidgeCV(cv =10)` chose 0.063 on a 61-point grid); the CV curve is flat near the minimum, and the effective degrees of freedom decrease from 10 towards 0 as λ grows.

Exercise 13.6.12 In SVD coordinates $\hat{\boldsymbol{\beta}}_\lambda = \sum_{j:d_j>0} \frac{d_j}{d_j^2+\lambda}\mathbf{v}_j\mathbf{u}_j^\top\mathbf{y} \to \sum_{j:d_j>0} d_j^{-1}\mathbf{v}_j\mathbf{u}_j^\top\mathbf{y} = \mathbf{X}^+\mathbf{y}$, the least-squares solution of minimum Euclidean norm (it has no component in null($\mathbf{X}$)). When $k > n$ the limit interpolates the training data exactly; its prediction performance depends on the implicit minimum-norm regularisation (the "benign overfitting" literature).

Exercise 13.6.13 With instruments $\mathbf{Z}$ and first-stage fitted values $\hat{\mathbf{X}} = \mathbf{P}_Z\mathbf{X}$ (compute $\mathbf{P}_Z\mathbf{X} = \mathbf{Q}_Z\mathbf{Q}_Z^\top\mathbf{X}$ from a QR of $\mathbf{Z}$), the 2SLS estimator is $\hat{\boldsymbol{\beta}} = (\hat{\mathbf{X}}^\top\mathbf{X})^{-1}\hat{\mathbf{X}}^\top\mathbf{y}$ and its HC1 covariance is $\frac{n}{n-k}(\hat{\mathbf{X}}^\top\hat{\mathbf{X}})^{-1}\big(\sum_i \hat{u}_i^2\hat{\mathbf{x}}_i\hat{\mathbf{x}}_i^\top\big)(\hat{\mathbf{X}}^\top\hat{\mathbf{X}})^{-1}$ with residuals $\hat{u}_i = y_i - \mathbf{x}_i^\top\hat{\boldsymbol{\beta}}$ computed with the *original* regressors (using $\hat{\mathbf{X}}$ here is a classic error). With strong instruments coverage should be close to 95%.

Exercise 13.6.14 A good plan includes: a validation survey measuring consumption for a random sample of applicants and non-applicants (sampling, Chapter 6); estimates of inclusion and exclusion errors at the 20% operating point with bootstrap intervals (Chapter 11); group-wise error rates and a fairness audit; a randomised or phased roll-out to estimate the transfer's effect on consumption with robust standard errors; pre-registration and a reproducible code base; and a board-level summary in natural frequencies ("of every 1000 poor households, about x are expected to be missed, with plausible range $[a, b]$").

References

E. Aiken, S. Bellue, D. Karlan, C. Udry and J. E. Blumenstock (2022). Machine learning and phone data can improve targeting of humanitarian aid. *Nature*, 603, 864–870.

J. Aitchison and J. A. C. Brown (1957). *The Lognormal Distribution, with Special Reference to its Uses in Economics*. Cambridge University Press.

H. Akaike (1973). Information theory and an extension of the maximum likelihood principle. In B. N. Petrov and F. Csáki (eds.), *Second International Symposium on Information Theory*, pp. 267–281. Akadémiai Kiadó, Budapest.

H. Akaike (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723.

A. Albert and J. A. Anderson (1984). On the existence of maximum likelihood estimates in logistic regression models. *Biometrika*, 71(1), 1–10.

J. H. Albert and S. Chib (1993). Bayesian analysis of binary and polychotomous response data. *Journal of the American Statistical Association*, 88(422), 669–679.

S. Amari and H. Nagaoka (2000). *Methods of Information Geometry*. Translations of Mathematical Monographs 191. American Mathematical Society and Oxford University Press.

S. Amari (1985). *Differential-Geometrical Methods in Statistics*. Lecture Notes in Statistics 28. Springer, New York.

S. Amari (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.

M. L. Anderson (2008). Multiple inference and gender differences in the effects of early intervention: a reevaluation of the Abecedarian, Perry Preschool, and Early Training projects. *Journal of the American Statistical Association*, 103(484), 1481–1495.

D. W. K. Andrews (1999). Estimation when a parameter is on a boundary. *Econometrica*, 67(6), 1341–1383.

A. N. Angelopoulos and S. Bates (2023). Conformal prediction: a gentle introduction. *Foundations and Trends in Machine Learning*, 16(4), 494–591.

J. D. Angrist and A. B. Krueger (1991). Does compulsory school attendance affect schooling and earnings? *Quarterly Journal of Economics*, 106(4), 979–1014.

J. D. Angrist and J.-S. Pischke (2009). *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

L. Armijo (1966). Minimization of functions having Lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1), 1–3.

K. B. Athreya (1987). Bootstrap of the mean in the infinite variance case. *Annals of Statistics*, 15(2), 724–731.

A. B. Atkinson and J. E. Stiglitz (1980). *Lectures on Public Economics*. McGraw-Hill, London.

A. B. Atkinson, T. Piketty and E. Saez (2011). Top incomes in the long run of history. *Journal of Economic Literature*, 49(1), 3–71.

J. Bai and S. Ng (2002). Determining the number of factors in approximate factor models. *Econometrica*, 70(1), 191–221.

J. Baik, G. Ben Arous and S. Péché (2005). Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices. *Annals of Probability*, 33(5), 1643–1697.

A. V. Banerjee and E. Duflo (2011). *Poor Economics: A Radical Rethinking of the Way to Fight Global Poverty*. PublicAffairs, New York.

A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153), 1–43.

T. Bayes (1763). An essay towards solving a problem in the doctrine of chances. Communicated by R. Price. *Philosophical Transactions of the Royal Society of London*, 53, 370–418.

M. Belkin, D. Hsu, S. Ma and S. Mandal (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32), 15849–15854.

A. Belloni, V. Chernozhukov and C. Hansen (2014). Inference on treatment effects after selection among high-dimensional controls. *Review of Economic Studies*, 81(2), 608–650.

D. A. Belsley, E. Kuh and R. E. Welsch (1980). *Regression Diagnostics: Identifying Influential Data and Sources of Collinearity*. Wiley, New York.

A. Ben-Israel and T. N. E. Greville (2003). *Generalized Inverses: Theory and Applications*, 2nd ed. Springer, New York.

Y. Benjamini and Y. Hochberg (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.

Commandant Benoît (1924). Note sur une méthode de résolution des équations normales provenant de l'application de la méthode des moindres carrés à un système d'équations linéaires en nombre inférieur à celui des inconnues (Procédé du Commandant Cholesky). *Bulletin Géodésique*, 2, 67–77.

C. Bergmeir, R. J. Hyndman and B. Koo (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120, 70–83.

S. Berry, J. Levinsohn and A. Pakes (1995). Automobile prices in market equilibrium. *Econometrica*, 63(4), 841–890.

D. P. Bertsekas (1999). *Nonlinear Programming*, 2nd ed. Athena Scientific, Belmont, MA.

P. J. Bickel and D. A. Freedman (1981). Some asymptotic theory for the bootstrap. *Annals of Statistics*, 9(6), 1196–1217.

P. Billingsley (1995). *Probability and Measure*, 3rd ed. Wiley, New York.

A. Birnbaum (1962). On the foundations of statistical inference. *Journal of the American Statistical Association*, 57(298), 269–306.

Å. Björck (1967). Solving linear least squares problems by Gram–Schmidt orthogonalization. *BIT Numerical Mathematics*, 7(1), 1–21.

Å. Björck (1996). *Numerical Methods for Least Squares Problems*. SIAM, Philadelphia.

P. Blanchard, D. J. Higham and N. J. Higham (2021). Accurately computing the log-sum-exp and softmax functions. *IMA Journal of Numerical Analysis*, 41(4), 2311–2330.

C. E. Bonferroni (1936). Teoria statistica delle classi e calcolo delle probabilità. *Pubblicazioni del R. Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, 8, 3–62.

L. von Bortkiewicz (1898). *Das Gesetz der kleinen Zahlen*. B. G. Teubner, Leipzig.

L. Bottou, F. E. Curtis and J. Nocedal (2018). Optimization methods for large-scale machine learning. *SIAM Review*, 60(2), 223–311.

L. Bottou (2010). Large-scale machine learning with stochastic gradient descent. In Y. Lechevallier and G. Saporta (eds.), *Proceedings of COMPSTAT'2010*, 177–186. Physica-Verlag, Heidelberg.

S. Boucheron, G. Lugosi and P. Massart (2013). *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press.

F. Bourguignon and A. Spadaro (2006). Microsimulation as a tool for evaluating redistribution policies. *Journal of Economic Inequality*, 4(1), 77–106.

G. E. P. Box and M. E. Muller (1958). A note on the generation of random normal deviates. *Annals of Mathematical Statistics*, 29(2), 610–611.

S. Boyd and L. Vandenberghe (2004). *Convex Optimization*. Cambridge University Press.

S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein (2011). Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1), 1–122.

C. Brezinski (2006). The life and work of André Cholesky. *Numerical Algorithms*, 43(3), 279–288.

S. Brin and L. Page (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117.

E. Britton, P. Fisher and J. Whitley (1998). The *Inflation Report* projections: understanding the fan chart. *Bank of England Quarterly Bulletin*, 38(1), 30–37.

C. Brown, M. Ravallion and D. van de Walle (2018). A poor means test? Econometric targeting in Africa. *Journal of Development Economics*, 134, 109–124.

C. G. Broyden (1970). The convergence of a class of double-rank minimization algorithms. *Journal of the Institute of Mathematics and its Applications*, 6(1), 76–90.

C. F. Camerer et al. (2016). Evaluating replicability of laboratory experiments in economics. *Science*, 351(6280), 1433–1436.

A. C. Cameron and P. K. Trivedi (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press.

A. C. Cameron and P. K. Trivedi (2013). *Regression Analysis of Count Data*, 2nd ed. Cambridge University Press.

A. C. Cameron, J. B. Gelbach and D. L. Miller (2008). Bootstrap-based improvements for inference with clustered errors. *Review of Economics and Statistics*, 90(3), 414–427.

D. Card (1995). Using geographic variation in college proximity to estimate the return to schooling. In L. N. Christofides, E. K. Grant and R. Swidinsky (eds), *Aspects of Labour Market Behaviour: Essays in Honour of John Vanderkamp*, 201–222. University of Toronto Press, Toronto.

D. Card (2001). Estimating the return to schooling: progress on some persistent econometric problems. *Econometrica*, 69(5), 1127–1160.

B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li and A. Riddell (2017). Stan: a probabilistic programming language. *Journal of Statistical Software*, 76(1), 1–32.

G. Casella and R. L. Berger (2002). *Statistical Inference*, 2nd ed. Duxbury, Pacific Grove, CA.

A.-L. Cauchy (1847). Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes Rendus de l'Académie des Sciences*, 25, 536–538.

A. Cayley (1858). A memoir on the theory of matrices. *Philosophical Transactions of the Royal Society of London*, 148, 17–37.

T. F. Chan, G. H. Golub and R. J. LeVeque (1983). Algorithms for computing the sample variance: analysis and recommendations. *The American Statistician*, 37(3), 242–247.

H. Chernoff (1952). A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4), 493–507.

V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey and J. Robins (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1), C1–C68.

G. Christensen and E. Miguel (2018). Transparency, reproducibility, and the credibility of economics research. *Journal of Economic Literature*, 56(3), 920–980.

T. E. Clark and K. D. West (2007). Approximately normal tests for equal predictive accuracy in nested models. *Journal of Econometrics*, 138(1), 291–311.

W. G. Cochran (1934). The distribution of quadratic forms in a normal system, with applications to the analysis of covariance. *Mathematical Proceedings of the Cambridge Philosophical Society*, 30(2), 178–191.

C. Cortes and V. Vapnik (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

H. Cramér (1946). *Mathematical Methods of Statistics*. Princeton University Press, Princeton, NJ.

G. B. Dantzig (1963). *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ.

G. Darmois (1935). Sur les lois de probabilité à estimation exhaustive. *Comptes Rendus de l'Académie des Sciences, Paris*, 200, 1265–1266.

W. C. Davidon (1959). Variable metric method for minimization. Research and Development Report ANL-5990, Argonne National Laboratory. Reprinted in *SIAM Journal on Optimization*, 1(1), 1–17 (1991).

R. Davidson and J. G. MacKinnon (2004). *Econometric Theory and Methods*. Oxford University Press, New York.

P. J. Davis and P. Rabinowitz (1984). *Methods of Numerical Integration*, 2nd ed. Academic Press, Orlando, FL.

T. A. Davis (2006). *Direct Methods for Sparse Linear Systems*. SIAM, Philadelphia.

A. C. Davison and D. V. Hinkley (1997). *Bootstrap Methods and their Application*. Cambridge University Press.

A. Deaton (1997). *The Analysis of Household Surveys: A Microeconometric Approach to Development Policy*. Johns Hopkins University Press for the World Bank, Baltimore.

M. P. Deisenroth, A. A. Faisal and C. S. Ong (2020). *Mathematics for Machine Learning*. Cambridge University Press.

C. De Mol, D. Giannone and L. Reichlin (2008). Forecasting using a large number of predictors: is Bayesian shrinkage a valid alternative to principal components? *Journal of Econometrics*, 146(2), 318–328.

W. G. Dewald, J. G. Thursby and R. G. Anderson (1986). Replication in empirical economics: the Journal of Money, Credit and Banking project. *American Economic Review*, 76(4), 587–603.

P. A. Diamond and J. A. Mirrlees (1971). Optimal taxation and public production I: Production efficiency. *American Economic Review*, 61(1), 8–27.

F. X. Diebold and R. S. Mariano (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3), 253–263.

F. X. Diebold, T. A. Gunther and A. S. Tay (1998). Evaluating density forecasts with applications to financial risk management. *International Economic Review*, 39(4), 863–883.

A. K. Dixit (1990). *Optimization in Economic Theory*, 2nd ed. Oxford University Press.

N. Duan (1983). Smearing estimate: a nonparametric retransformation method. *Journal of the American Statistical Association*, 78(383), 605–610.

S. Duane, A. D. Kennedy, B. J. Pendleton and D. Roweth (1987). Hybrid Monte Carlo. *Physics Letters B*, 195(2), 216–222.

J. Duchi, S. Shalev-Shwartz, Y. Singer and T. Chandra (2008). Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 272–279.

J. Duchi, E. Hazan and Y. Singer (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12, 2121–2159.

E. Duflo (2001). Schooling and labor market consequences of school construction in Indonesia: evidence from an unusual policy experiment. *American Economic Review*, 91(4), 795–813.

C. Eckart and G. Young (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

R. Eckhardt (1987). Stan Ulam, John von Neumann, and the Monte Carlo method. *Los Alamos Science*, 15 (Special Issue), 131–137.

B. Efron and T. Hastie (2016). *Computer Age Statistical Inference: Algorithms, Evidence, and Data Science*. Cambridge University Press.

B. Efron and D. V. Hinkley (1978). Assessing the accuracy of the maximum likelihood estimator: observed versus expected Fisher information. *Biometrika*, 65(3), 457–483.

B. Efron and C. Stein (1981). The jackknife estimate of variance. *Annals of Statistics*, 9(3), 586–596.

B. Efron and R. J. Tibshirani (1993). *An Introduction to the Bootstrap*. Chapman & Hall, New York.

B. Efron, T. Hastie, I. Johnstone and R. Tibshirani (2004). Least angle regression. *Annals of Statistics*, 32(2), 407–499.

B. Efron (1979). Bootstrap methods: another look at the jackknife. *Annals of Statistics*, 7(1), 1–26.

B. Efron (1983). Estimating the error rate of a prediction rule: improvement on cross-validation. *Journal of the American Statistical Association*, 78(382), 316–331.

B. Efron (1987). Better bootstrap confidence intervals. *Journal of the American Statistical Association*, 82(397), 171–185.

FAO, IFAD, UNICEF, WFP and WHO (2020). *The State of Food Security and Nutrition in the World 2020: Transforming Food Systems for Affordable Healthy Diets*. FAO, Rome.

D. Filmer and L. H. Pritchett (2001). Estimating wealth effects without expenditure data – or tears: an application to educational enrollments in states of India. *Demography*, 38(1), 115–132.

D. Firth (1993). Bias reduction of maximum likelihood estimates. *Biometrika*, 80(1), 27–38.

H. Fischer (2011). *A History of the Central Limit Theorem: From Classical to Modern Probability Theory*. Springer, New York.

R. A. Fisher (1912). On an absolute criterion for fitting frequency curves. *Messenger of Mathematics*, 41, 155–160.

R. A. Fisher (1921). On the "probable error" of a coefficient of correlation deduced from a small sample. *Metron*, 1, 3–32.

R. A. Fisher (1922). On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society of London, Series A*, 222, 309–368.

R. A. Fisher (1925). Theory of statistical estimation. *Proceedings of the Cambridge Philosophical Society*, 22(5), 700–725.

R. A. Fisher (1935). *The Design of Experiments*. Oliver and Boyd, Edinburgh.

R. Fletcher and M. J. D. Powell (1963). A rapidly convergent descent method for minimization. *The Computer Journal*, 6(2), 163–168.

R. Fletcher (1970). A new approach to variable metric algorithms. *The Computer Journal*, 13(3), 317–322.

J. Foster, J. Greer and E. Thorbecke (1984). A class of decomposable poverty measures. *Econometrica*, 52(3), 761–766.

J. Friedman, T. Hastie and R. Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.

R. Frisch and F. V. Waugh (1933). Partial time regressions as compared with individual trends. *Econometrica*, 1(4), 387–401.

C. F. Gauss (1809). *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Perthes et Besser, Hamburg.

S. Geisser (1975). The predictive sample reuse method with applications. *Journal of the American Statistical Association*, 70(350), 320–328.

A. E. Gelfand and A. F. M. Smith (1990). Sampling-based approaches to calculating marginal densities. *Journal of the American Statistical Association*, 85(410), 398–409.

A. Gelman and D. B. Rubin (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4), 457–472.

A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari and D. B. Rubin (2013). *Bayesian Data Analysis*, 3rd ed. CRC Press, Boca Raton, FL.

S. Geman and D. Geman (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6), 721–741.

S. Geman, E. Bienenstock and R. Doursat (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1), 1–58.

A. Géron (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, Sebastopol, CA.

C. J. Geyer (1992). Practical Markov chain Monte Carlo. *Statistical Science*, 7(4), 473–483.

D. Giannone, M. Lenza and G. E. Primiceri (2021). Economic predictions with big data: the illusion of sparsity. *Econometrica*, 89(5), 2409–2437.

C. Gini (1912). *Variabilità e Mutabilità*. Tipografia di Paolo Cuppini, Bologna.

P. Glasserman (2003). *Monte Carlo Methods in Financial Engineering*. Springer, New York.

D. Goldberg (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48.

D. Goldfarb (1970). A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109), 23–26.

G. Golub and W. Kahan (1965). Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2), 205–224.

G. H. Golub and C. F. Van Loan (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press, Baltimore.

I. Goodfellow, Y. Bengio and A. Courville (2016). *Deep Learning*. MIT Press, Cambridge, MA.

P. Goulet Coulombe, M. Leroux, D. Stevanovic and S. Surprenant (2022). How is machine learning useful for macroeconomic forecasting? *Journal of Applied Econometrics*, 37(5), 920–964.

C. Gouriéroux, A. Monfort and A. Trognon (1984). Pseudo maximum likelihood methods: theory. *Econometrica*, 52(3), 681–700.

J. P. Gram (1883). Ueber die Entwickelung reeller Functionen in Reihen mittelst der Methode der kleinsten Quadrate. *Journal für die reine und angewandte Mathematik*, 94, 41–73.

H. Grassmann (1844). *Die lineale Ausdehnungslehre, ein neuer Zweig der Mathematik*. Otto Wigand, Leipzig.

M. Greenwood and G. U. Yule (1920). An inquiry into the nature of frequency distributions representative of multiple happenings with particular reference to the occurrence of multiple attacks of disease or of repeated accidents. *Journal of the Royal Statistical Society*, 83(2), 255–279.

A. Griewank and A. Walther (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, Philadelphia.

G. Grimmett and D. Stirzaker (2020). *Probability and Random Processes*, 4th ed. Oxford University Press.

N. Halko, P.-G. Martinsson and J. A. Tropp (2011). Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2), 217–288.

P. Hall (1992). *The Bootstrap and Edgeworth Expansion*. Springer, New York.

J. H. Halton (1960). On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2, 84–90.

L. P. Hansen (1982). Large sample properties of generalized method of moments estimators. *Econometrica*, 50(4), 1029–1054.

B. E. Hansen (2022). *Econometrics*. Princeton University Press.

C. R. Harris et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

D. Harvey, S. Leybourne and P. Newbold (1997). Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2), 281–291.

T. Hastie, R. Tibshirani and J. Friedman (2009). *The Elements of Statistical Learning*, 2nd ed. Springer, New York.

T. Hastie, R. Tibshirani and M. Wainwright (2015). *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, Boca Raton, FL.

T. Hastie, A. Montanari, S. Rosset and R. J. Tibshirani (2022). Surprises in high-dimensional ridgeless least squares interpolation. *The Annals of Statistics*, 50(2), 949–986.

W. K. Hastings (1970). Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1), 97–109.

F. Hayashi (2000). *Econometrics*. Princeton University Press.

J. J. Heckman, L. J. Lochner and P. E. Todd (2006). Earnings functions, rates of return and treatment effects: the Mincer equation and beyond. In E. A. Hanushek and F. Welch (eds), *Handbook of the Economics of Education*, Vol. 1, 307–458. Elsevier, Amsterdam.

F. Heiss and V. Winschel (2008). Likelihood approximation by numerical integration on sparse grids. *Journal of Econometrics*, 144(1), 62–80.

H. V. Henderson, F. Pukelsheim and S. R. Searle (1983). On the history of the Kronecker product. *Linear and Multilinear Algebra*, 14(2), 113–120.

T. Herndon, M. Ash and R. Pollin (2014). Does high public debt consistently stifle economic growth? A critique of Reinhart and Rogoff. *Cambridge Journal of Economics*, 38(2), 257–279.

T. C. Hesterberg (2015). What teachers should know about the bootstrap: resampling in the undergraduate statistics curriculum. *The American Statistician*, 69(4), 371–386.

N. J. Higham (1993). The accuracy of floating point summation. *SIAM Journal on Scientific Computing*, 14(4), 783–799.

N. J. Higham (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, Philadelphia.

B. M. Hill (1975). A simple general approach to inference about the tail of a distribution. *The Annals of Statistics*, 3(5), 1163–1174.

D. C. Hoaglin and R. E. Welsch (1978). The hat matrix in regression and ANOVA. *The American Statistician*, 32(1), 17–22.

W. Hoeffding (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301), 13–30.

A. E. Hoerl and R. W. Kennard (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

M. D. Hoffman and A. Gelman (2014). The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15, 1593–1623.

S. Holm (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.

R. A. Horn and C. R. Johnson (2013). *Matrix Analysis*, 2nd ed. Cambridge University Press.

H. Hotelling (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441, and 24(7), 498–520.

A. S. Householder (1958). Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4), 339–342.

P. J. Huber (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1), 73–101.

P. J. Huber (1967). The behavior of maximum likelihood estimates under nonstandard conditions. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, 221–233. University of California Press, Berkeley.

IEEE (1985). *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985. Institute of Electrical and Electronics Engineers, New York.

ILO, IMF, OECD, UNECE, Eurostat and The World Bank (2004). *Consumer Price Index Manual: Theory and Practice*. International Labour Office, Geneva.

G. W. Imbens and D. B. Rubin (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press.

G. James, D. Witten, T. Hastie and R. Tibshirani (2021). *An Introduction to Statistical Learning*, 2nd ed. Springer, New York.

H. Jeffreys (1946). An invariant form for the prior probability in estimation problems. *Proceedings of the Royal Society of London, Series A*, 186(1007), 453–461.

J. L. W. V. Jensen (1906). Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta Mathematica*, 30, 175–193.

R. Johnson and T. Zhang (2013). Accelerating stochastic gradient descent using predictive variance reduction. *Advances in Neural Information Processing Systems*, 26, 315–323.

I. M. Johnstone (2001). On the distribution of the largest eigenvalue in principal components analysis. *Annals of Statistics*, 29(2), 295–327.

I. T. Jolliffe and J. Cadima (2016). Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065), 20150202.

I. T. Jolliffe (2002). *Principal Component Analysis*, 2nd ed. Springer, New York.

K. L. Judd (1998). *Numerical Methods in Economics*. MIT Press, Cambridge, MA.

W. Kahan (1965). Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1), 40.

H. Kahn and T. E. Harris (1951). Estimation of particle transmission by random sampling. In *Monte Carlo Method*, National Bureau of Standards Applied Mathematics Series 12, 27–30.

S. M. R. Kanbur (1987). Measurement and alleviation of poverty: With an application to the effects of macroeconomic adjustment. *IMF Staff Papers*, 34(1), 60–85.

J. Kiefer and J. Wolfowitz (1952). Stochastic estimation of the maximum of a regression function. *Annals of Mathematical Statistics*, 23(3), 462–466.

D. P. Kingma and J. Ba (2015). Adam: a method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

T. H. Kjeldsen (2000). A contextualized historical analysis of the Kuhn–Tucker theorem in nonlinear programming: The impact of World War II. *Historia Mathematica*, 27(4), 331–361.

J. Kleinberg, J. Ludwig, S. Mullainathan and Z. Obermeyer (2015). Prediction policy problems. *American Economic Review: Papers and Proceedings*, 105(5), 491–495.

D. E. Knuth (1976). Big Omicron and big Omega and big Theta. *ACM SIGACT News*, 8(2), 18–24.

R. Koenker and G. Bassett (1978). Regression quantiles. *Econometrica*, 46(1), 33–50.

A. N. Kolmogorov (1933). *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer, Berlin. English translation: *Foundations of the Theory of Probability*, Chelsea, New York, 1950.

B. O. Koopman (1936). On distributions admitting a sufficient statistic. *Transactions of the American Mathematical Society*, 39(3), 399–409.

T. C. Koopmans (1949). Identification problems in economic model construction. *Econometrica*, 17(2), 125–144.

H. W. Kuhn and A. W. Tucker (1951). Nonlinear programming. In J. Neyman (ed.), *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, 481–492. University of California Press, Berkeley.

S. Kullback and R. A. Leibler (1951). On information and sufficiency. *Annals of Mathematical Statistics*, 22(1), 79–86.

H. R. Künsch (1989). The jackknife and the bootstrap for general stationary observations. *Annals of Statistics*, 17(3), 1217–1241.

T. Lancaster (1990). *The Econometric Analysis of Transition Data*. Cambridge University Press.

A. N. Langville and C. D. Meyer (2006). *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press.

L. Le Cam (1960). An approximation theorem for the Poisson binomial distribution. *Pacific Journal of Mathematics*, 10(4), 1181–1197.

P. L'Ecuyer and R. Simard (2007). TestU01: a C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33(4), Article 22.

O. Ledoit and M. Wolf (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411.

J. D. Lee, M. Simchowitz, M. I. Jordan and B. Recht (2016). Gradient descent only converges to minimizers. *Proceedings of the 29th Conference on Learning Theory (COLT)*, PMLR 49, 1246–1257.

J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani and L. Wasserman (2018). Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523), 1094–1111.

W. W. Leontief (1936). Quantitative input and output relations in the economic system of the United States. *The Review of Economics and Statistics*, 18(3), 105–125.

L. Lessard, B. Recht and A. Packard (2016). Analysis and design of optimization algorithms via integral quadratic constraints. *SIAM Journal on Optimization*, 26(1), 57–95.

K. Levenberg (1944). A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2), 164–168.

D. A. Levin and Y. Peres (2017). *Markov Chains and Mixing Times*, 2nd ed. (with contributions by E. L. Wilmer). American Mathematical Society, Providence, RI.

S. Linnainmaa (1976). Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2), 146–160.

J.-L. Lions et al. (1996). *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*. European Space Agency and CNES, Paris.

D. C. Liu and J. Nocedal (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45, 503–528.

R. Y. Liu (1988). Bootstrap procedures under some non-i.i.d. models. *Annals of Statistics*, 16(4), 1696–1708.

J. S. Liu (2001). *Monte Carlo Strategies in Scientific Computing*. Springer, New York.

M. O. Lorenz (1905). Methods of measuring the concentration of wealth. *Publications of the American Statistical Association*, 9(70), 209–219.

M. C. Lovell (1963). Seasonal adjustment of economic time series and multiple regression analysis. *Journal of the American Statistical Association*, 58(304), 993–1010.

M. C. Lovell (2008). A simple proof of the FWL theorem. *The Journal of Economic Education*, 39(1), 88–91.

D. MacKenzie and T. Spears (2014). 'The formula that killed Wall Street': the Gaussian copula and modelling practices in investment banking. *Social Studies of Science*, 44(3), 393–417.

J. G. MacKinnon and M. D. Webb (2017). Wild bootstrap inference for wildly different cluster sizes. *Journal of Applied Econometrics*, 32(2), 233–254.

J. G. MacKinnon and H. White (1985). Some heteroskedasticity-consistent covariance matrix estimators with improved finite sample properties. *Journal of Econometrics*, 29(3), 305–325.

J. R. Magnus and H. Neudecker (2019). *Matrix Differential Calculus with Applications in Statistics and Econometrics*, 3rd ed. Wiley, Chichester.

P. C. Mahalanobis (1936). On the generalised distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1), 49–55.

E. Mammen (1993). Bootstrap and wild bootstrap for high dimensional linear models. *Annals of Statistics*, 21(1), 255–285.

C. F. Manski (2013). *Public Policy in an Uncertain World: Analysis and Decisions*. Harvard University Press, Cambridge, MA.

C. F. Manski (2019). Communicating uncertainty in policy analysis. *Proceedings of the National Academy of Sciences*, 116(16), 7634–7641.

H. Markowitz (1952). Portfolio selection. *Journal of Finance*, 7(1), 77–91.

D. W. Marquardt (1963). An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2), 431–441.

G. Marsaglia (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences of the USA*, 61(1), 25–28.

A. Mas-Colell, M. D. Whinston and J. R. Green (1995). *Microeconomic Theory*. Oxford University Press, New York.

P. Massart (1990). The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *Annals of Probability*, 18(3), 1269–1283.

M. Matsumoto and T. Nishimura (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3–30.

K. Matsuura (2022). *Bayesian Statistical Modeling with Stan, R, and Python*. Springer, Singapore.

M. W. McCracken and S. Ng (2016). FRED-MD: a monthly database for macroeconomic research. *Journal of Business & Economic Statistics*, 34(4), 574–589.

P. McCullagh and J. A. Nelder (1989). *Generalized Linear Models*, 2nd ed. Chapman & Hall, London.

B. D. McCullough and H. D. Vinod (1999). The numerical reliability of econometric software. *Journal of Economic Literature*, 37(2), 633–665.

D. McFadden (1989). A method of simulated moments for estimation of discrete response models without numerical integration. *Econometrica*, 57(5), 995–1026.

D. J. McKenzie (2005). Measuring inequality with asset indicators. *Journal of Population Economics*, 18(2), 229–260.

N. Metropolis and S. Ulam (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341.

N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller and E. Teller (1953). Equation of state calculations by fast computing machines. *Journal of Chemical Physics*, 21(6), 1087–1092.

N. Metropolis (1987). The beginning of the Monte Carlo method. *Los Alamos Science*, 15 (Special Issue), 125–130.

R. E. Miller and P. D. Blair (2009). *Input-Output Analysis: Foundations and Extensions*, 2nd ed. Cambridge University Press.

J. Mincer (1974). *Schooling, Experience, and Earnings*. National Bureau of Economic Research and Columbia University Press, New York.

L. Mirsky (1960). Symmetric gauge functions and unitarily invariant norms. *Quarterly Journal of Mathematics*, 11(1), 50–59.

E. H. Moore (1920). On the reciprocal of the general algebraic matrix (abstract). *Bulletin of the American Mathematical Society*, 26, 394–395.

T. P. Morris, I. R. White and M. J. Crowther (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11), 2074–2102.

B. R. Moulton (1990). An illustration of a pitfall in estimating the effects of aggregate variables on micro units. *Review of Economics and Statistics*, 72(2), 334–338.

S. Mullainathan and J. Spiess (2017). Machine learning: an applied econometric approach. *Journal of Economic Perspectives*, 31(2), 87–106.

K. P. Murphy (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press, Cambridge, MA.

R. M. Neal (2011). MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones and X.-L. Meng (eds.), *Handbook of Markov Chain Monte Carlo*, 113–162. Chapman & Hall/CRC, Boca Raton, FL.

J. A. Nelder and R. W. M. Wedderburn (1972). Generalized linear models. *Journal of the Royal Statistical Society, Series A*, 135(3), 370–384.

R. B. Nelsen (2006). *An Introduction to Copulas*, 2nd ed. Springer, New York.

Y. Nesterov (1983). A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27, 372–376.

Y. Nesterov (2018). *Lectures on Convex Optimization*, 2nd ed. Springer, Cham.

W. K. Newey and D. McFadden (1994). Large sample estimation and hypothesis testing. In R. F. Engle and D. L. McFadden (eds), *Handbook of Econometrics*, Vol. 4, 2111–2245. Elsevier, Amsterdam.

J. Neyman (1934). On the two different aspects of the representative method: the method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4), 558–625.

A. Y. Ng and M. I. Jordan (2002). On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems 14*, 841–848. MIT Press, Cambridge, MA.

H. Niederreiter (1992). *Random Number Generation and Quasi-Monte Carlo Methods*. SIAM, Philadelphia.

F. Nielsen (2020). An elementary introduction to information geometry. *Entropy*, 22(10), 1100.

J. Nocedal and S. J. Wright (2006). *Numerical Optimization*, 2nd ed. Springer, New York.

Z. Obermeyer, B. Powers, C. Vogeli and S. Mullainathan (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464), 447–453.

M. E. O'Neill (2014). PCG: a family of simple fast space-efficient statistically good algorithms for random number generation. Technical Report HMC-CS-2014-0905, Harvey Mudd College, Claremont, CA.

M. L. Overton (2001). *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, Philadelphia.

A. B. Owen (1997). Scrambled net variance for integrals of smooth functions. *Annals of Statistics*, 25(4), 1541–1562.

L. Page, S. Brin, R. Motwani and T. Winograd (1999). The PageRank citation ranking: bringing order to the Web. Technical Report 1999-66, Stanford InfoLab, Stanford University.

C. C. Paige and M. A. Saunders (1982). LSQR: an algorithm for sparse linear equations and sparse least squares. *ACM Transactions on Mathematical Software*, 8(1), 43–71.

V. Pareto (1896–1897). *Cours d'Économie Politique*, 2 vols. F. Rouge, Lausanne.

Y. Pawitan (2001). *In All Likelihood: Statistical Modelling and Inference Using Likelihood*. Oxford University Press.

J. Pearl (2009). *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press.

K. Pearson (1894). Contributions to the mathematical theory of evolution. *Philosophical Transactions of the Royal Society of London, Series A*, 185, 71–110.

K. Pearson (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, Series 6, 2(11), 559–572.

F. Pedregosa et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

R. D. Peng (2011). Reproducible research in computational science. *Science*, 334(6060), 1226–1227.

R. Penrose (1955). A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 51(3), 406–413.

K. B. Petersen and M. S. Pedersen (2012). *The Matrix Cookbook*. Technical University of Denmark.

E. J. G. Pitman (1936). Sufficient statistics and intrinsic accuracy. *Mathematical Proceedings of the Cambridge Philosophical Society*, 32(4), 567–579.

E. J. G. Pitman (1937). Significance tests which may be applied to samples from any populations. *Supplement to the Journal of the Royal Statistical Society*, 4(1), 119–130.

S.-D. Poisson (1837). *Recherches sur la Probabilité des Jugements en Matière Criminelle et en Matière Civile*. Bachelier, Paris.

D. N. Politis and J. P. Romano (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313.

G. Pólya (1945). *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ.

B. T. Polyak and A. B. Juditsky (1992). Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4), 838–855.

B. T. Polyak (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1–17.

G. Psacharopoulos and H. A. Patrinos (2018). Returns to investment in education: a decennial review of the global literature. *Education Economics*, 26(5), 445–458.

M. H. Quenouille (1949). Approximate tests of correlation in time-series. *Journal of the Royal Statistical Society, Series B*, 11(1), 68–84.

M. H. Quenouille (1956). Notes on bias in estimation. *Biometrika*, 43(3/4), 353–360.

F. P. Ramsey (1927). A contribution to the theory of taxation. *Economic Journal*, 37(145), 47–61.

C. R. Rao (1945). Information and the accuracy attainable in the estimation of statistical parameters. *Bulletin of the Calcutta Mathematical Society*, 37, 81–91.

C. R. Rao (1948). Large sample tests of statistical hypotheses concerning several parameters with applications to problems of estimation. *Mathematical Proceedings of the Cambridge Philosophical Society*, 44(1), 50–57.

M. Ravallion and K. Chao (1989). Targeted policies for poverty alleviation under imperfect information: Algorithms and applications. *Journal of Policy Modeling*, 11(2), 213–224.

S. J. Reddi, S. Kale and S. Kumar (2018). On the convergence of Adam and beyond. *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

H. Robbins and S. Monro (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407.

C. P. Robert and G. Casella (2004). *Monte Carlo Statistical Methods*, 2nd ed. Springer, New York.

G. O. Roberts, A. Gelman and W. R. Gilks (1997). Weak convergence and optimal scaling of random walk Metropolis algorithms. *Annals of Applied Probability*, 7(1), 110–120.

P. M. Robinson (1988). Root-N-consistent semiparametric regression. *Econometrica*, 56(4), 931–954.

D. E. Rumelhart, G. E. Hinton and R. J. Williams (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

P. A. Samuelson (1947). *Foundations of Economic Analysis*. Harvard University Press, Cambridge, MA.

J. M. C. Santos Silva and S. Tenreyro (2006). The log of gravity. *Review of Economics and Statistics*, 88(4), 641–658.

E. Schmidt (1907). Zur Theorie der linearen und nichtlinearen Integralgleichungen. I. Teil: Entwicklung willkürlicher Funktionen nach Systemen vorgeschriebener. *Mathematische Annalen*, 63, 433–476.

G. Schwarz (1978). Estimating the dimension of a model. *Annals of Statistics*, 6(2), 461–464.

D. F. Shanno (1970). Conditioning of quasi-Newton methods for function minimization. *Mathematics of Computation*, 24(111), 647–656.

A. Shapiro, D. Dentcheva and A. Ruszczyński (2009). *Lectures on Stochastic Programming: Modeling and Theory*. SIAM, Philadelphia.

I. Shevtsova (2011). On the absolute constants in the Berry–Esseen type inequalities for identically distributed summands. arXiv preprint arXiv:1111.6554.

C. P. Simon and L. Blume (1994). *Mathematics for Economists*. W. W. Norton, New York.

E. H. Simpson (1951). The interpretation of interaction in contingency tables. *Journal of the Royal Statistical Society, Series B*, 13(2), 238–241.

A. Sklar (1959). Fonctions de répartition à n dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8, 229–231.

I. M. Sobol' (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4), 86–112.

W. Squire and G. Trapp (1998). Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40(1), 110–112.

D. Staiger and J. H. Stock (1997). Instrumental variables regression with weak instruments. *Econometrica*, 65(3), 557–586.

G. W. Stewart (1993). On the early history of the singular value decomposition. *SIAM Review*, 35(4), 551–566.

G. J. Stigler (1945). The cost of subsistence. *Journal of Farm Economics*, 27(2), 303–314.

S. M. Stigler (1981). Gauss and the invention of least squares. *The Annals of Statistics*, 9(3), 465–474.

S. M. Stigler (1986). *The History of Statistics: The Measurement of Uncertainty before 1900*. Harvard University Press, Cambridge, MA.

S. M. Stigler (2007). The epic story of maximum likelihood. *Statistical Science*, 22(4), 598–620.

J. H. Stock and M. W. Watson (2002a). Forecasting using principal components from a large number of predictors. *Journal of the American Statistical Association*, 97(460), 1167–1179.

J. H. Stock and M. W. Watson (2002b). Macroeconomic forecasting using diffusion indexes. *Journal of Business & Economic Statistics*, 20(2), 147–162.

R. Stone, D. G. Champernowne and J. E. Meade (1942). The precision of national income estimates. *The Review of Economic Studies*, 9(2), 111–125.

M. Stone (1974). Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society, Series B*, 36(2), 111–147.

G. Strang (1993). The fundamental theorem of linear algebra. *The American Mathematical Monthly*, 100(9), 848–855.

G. Strang (2019). *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press, Wellesley, MA.

V. Strassen (1969). Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4), 354–356.

Student (1908). The probable error of a mean. *Biometrika*, 6(1), 1–25.

R. K. Sundaram (1996). *A First Course in Optimization Theory*. Cambridge University Press.

J. J. Sylvester (1850). Additions to the articles "On a new class of theorems" and "On Pascal's theorem". *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 37(251), 363–370.

R. Tibshirani (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.

A. N. Tikhonov (1943). On the stability of inverse problems. *Doklady Akademii Nauk SSSR*, 39(5), 195–198 (in Russian).

A. N. Tikhonov (1963). Solution of incorrectly formulated problems and the regularization method. *Soviet Mathematics Doklady*, 4, 1035–1038.

K. E. Train (2009). *Discrete Choice Methods with Simulation*, 2nd ed. Cambridge University Press.

L. N. Trefethen and D. Bau (1997). *Numerical Linear Algebra*. SIAM, Philadelphia.

J. W. Tukey (1958). Bias and confidence in not-quite large samples (abstract). *Annals of Mathematical Statistics*, 29(2), 614.

A. M. Turing (1948). Rounding-off errors in matrix processes. *Quarterly Journal of Mechanics and Applied Mathematics*, 1(1), 287–308.

A. W. van der Vaart (1998). *Asymptotic Statistics*. Cambridge University Press.

A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner (2021). Rank-normalization, folding, and localization: an improved $\widehat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2), 667–718.

D. J. Velleman (2019). *How to Prove It: A Structured Approach*, 3rd ed. Cambridge University Press.

R. Vershynin (2018). *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge University Press.

P. Virtanen et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

J. von Neumann and H. H. Goldstine (1947). Numerical inverting of matrices of high order. *Bulletin of the American Mathematical Society*, 53(11), 1021–1099.

J. von Neumann (1951). Various techniques used in connection with random digits. In *Monte Carlo Method*, National Bureau of Standards Applied Mathematics Series 12, 36–38. U.S. Government Printing Office, Washington, DC.

V. Vovk, A. Gammerman and G. Shafer (2005). *Algorithmic Learning in a Random World*. Springer, New York.

M. J. Wainwright (2019). *High-Dimensional Statistics: A Non-Asymptotic Viewpoint*. Cambridge University Press.

A. Wald (1943). Tests of statistical hypotheses concerning several parameters when the number of observations is large. *Transactions of the American Mathematical Society*, 54(3), 426–482.

L. Wasserman (2004). *All of Statistics: A Concise Course in Statistical Inference*. Springer, New York.

W. Weibull (1951). A statistical distribution function of wide applicability. *Journal of Applied Mechanics*, 18(3), 293–297.

B. P. Welford (1962). Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), 419–420.

R. E. Wengert (1964). A simple automatic derivative evaluation program. *Communications of the ACM*, 7(8), 463–464.

H. White (1980). A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity. *Econometrica*, 48(4), 817–838.

H. White (1982). Maximum likelihood estimation of misspecified models. *Econometrica*, 50(1), 1–25.

J. H. Wilkinson (1963). *Rounding Errors in Algebraic Processes*. Prentice-Hall, Englewood Cliffs, NJ.

S. S. Wilks (1938). The large-sample distribution of the likelihood ratio for testing composite hypotheses. *Annals of Mathematical Statistics*, 9(1), 60–62.

G. Wilson et al. (2014). Best practices for scientific computing. *PLoS Biology*, 12(1), e1001745.

J. M. Wooldridge (2010). *Econometric Analysis of Cross Section and Panel Data*, 2nd ed. MIT Press, Cambridge, MA.

C. F. J. Wu (1986). Jackknife, bootstrap and other resampling methods in regression analysis. *Annals of Statistics*, 14(4), 1261–1295.

A. Young (2019). Channeling Fisher: randomization tests and the statistical insignificance of seemingly significant experimental results. *Quarterly Journal of Economics*, 134(2), 557–598.

T. J. Ypma (1995). Historical development of the Newton–Raphson method. *SIAM Review*, 37(4), 531–551.

H. Zou and T. Hastie (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society, Series B*, 67(2), 301–320.

H. Zou, T. Hastie and R. Tibshirani (2007). On the "degrees of freedom" of the lasso. *Annals of Statistics*, 35(5), 2173–2192.